Maths

K19G

2026-10-28

Table of contents

Maths library **76**
Structure . . . 76
How chapters are written . . . 76
Suggested learning paths . . . 77
How to use this book . . . 77
Prerequisites . . . 77
Related books . . . 77
Acknowledgments . . . 78

Intro **79**

The Evolution of Mathematics: From Ancient Counting to Modern Abstractions **80**
Introduction . . . 80
Chapter 1: The Dawn of Mathematical Thinking (Prehistoric Era - 3000 BCE) . . . 80
The Birth of Counting . . . 80
The Ishango Bone: First Mathematical Tool . . . 81
Development of Number Concepts . . . 81
Chapter 2: Ancient Civilizations and Number Systems (3000 BCE - 500 BCE) . . . 82
Mesopotamian Mathematics: The Foundation . . . 82
Egyptian Mathematics: Practical Geometry . . . 83
Chinese Mathematics: Systematic Thinking . . . 84
Indian Mathematics: The Birth of Zero . . . 85
Chapter 3: Classical Period - Greek Mathematical Revolution (600 BCE - 500 CE) . . . 85
The Birth of Mathematical Proof . . . 85
Archimedes: Mathematical Physics . . . 88
Chapter 4: Medieval Mathematics - Preservation and Innovation (500 CE - 1400 CE) . . . 89
Islamic Golden Age: Algebra is Born . . . 89
Fibonacci: Bringing Eastern Mathematics West . . . 90
Chapter 5: Renaissance Mathematics - Symbolic Revolution (1400 CE - 1600 CE) . . . 92
The Development of Symbolic Notation . . . 92
The Birth of Complex Numbers . . . 93
Chapter 6: The Scientific Revolution - Calculus and Analysis (1600 CE - 1800 CE) . . . 93
Newton and Leibniz: The Calculus Wars . . . 93
Euler: The Master of All Mathematics . . . 95
Chapter 7: Modern Mathematics - Abstraction and Rigor (1800 CE - 1900 CE) . . . 96
Non-Euclidean Geometry . . . 96
Group Theory: Galois and Abstract Algebra . . . 97
Set Theory: Cantor's Infinite . . . 97

Chapter 8: Contemporary Mathematics - The Digital Age (1900 CE - Present) 98
Computer Science and Mathematics 98
Chaos Theory and Fractals 99
Machine Learning and AI Mathematics 100
Chapter 9: The Future of Mathematics 101
Quantum Computing and Mathematics 101
Artificial Intelligence and Mathematical Discovery 102
Conclusion: Mathematics as Human Heritage 102
The Interconnected Web of Mathematical Knowledge 102
The Continuing Journey 103

Arithmetic **105**

Introduction to Arithmetic: The Foundation of All Mathematics **106**
What is Arithmetic? 106
The Historical Journey of Arithmetic 106
From Fingers to Symbols 106
The Revolutionary Concept of Zero 107
Understanding Numbers: The Building Blocks 108
Natural Numbers: Where It All Begins 108
Whole Numbers: Adding the Concept of Nothing 108
Integers: Embracing the Negative 109
The Four Fundamental Operations 109
Addition: Combining Quantities 109
Subtraction: Finding the Difference 110
Multiplication: Repeated Addition 111
Division: Sharing Equally 113
Number Systems and Place Value 115
Decimal System: Base 10 115
Other Number Systems 115
Mental Math Strategies 116
Addition Strategies 116
Multiplication Shortcuts 116
Fractions: Parts of a Whole 117
Understanding Fractions 117
Fraction Operations 118
Decimals: Another Way to Express Parts 118
Decimal Place Value 118
Converting Between Fractions and Decimals 119
Percentages: Parts per Hundred 120
Understanding Percentages 120
Percentage Calculations 120
Problem-Solving Strategies 121
The Four-Step Problem-Solving Process 121
Word Problem Examples 121
Real-World Applications 122
Money and Finance 122

Measurement and Cooking . . . 123
Time and Distance . . . 123
Common Arithmetic Mistakes and How to Avoid Them . . . 124
Addition and Subtraction Errors . . . 124
Multiplication and Division Errors . . . 125
Building Number Sense . . . 126
Estimation Skills . . . 126
Pattern Recognition . . . 126
Conclusion: The Beauty of Arithmetic . . . 127

Numbers and Counting: The Foundation of Mathematics 129
Introduction . . . 129
The Evolution of Counting . . . 129
Prehistoric Counting Methods . . . 129
The Ishango Bone: Ancient Mathematical Tool . . . 130
Understanding Natural Numbers . . . 130
What Are Natural Numbers? . . . 130
Properties of Natural Numbers . . . 131
Cardinality vs. Ordinality . . . 131
Whole Numbers: Adding Zero . . . 132
The Revolutionary Concept of Zero . . . 132
Zero's Three Roles . . . 132
Integers: Embracing Negative Numbers . . . 133
The Need for Negative Numbers . . . 133
The Integer Number Line . . . 133
Absolute Value . . . 134
Number Systems and Bases . . . 134
Understanding Base 10 (Decimal System) . . . 134
Other Number Systems . . . 135
Converting Between Bases . . . 135
Counting Strategies and Patterns . . . 136
Skip Counting . . . 136
Grouping and Bundling . . . 136
Number Patterns . . . 137
Comparing and Ordering Numbers . . . 138
Comparison Symbols . . . 138
Ordering Numbers . . . 138
Comparing Multi-Digit Numbers . . . 139
Rounding and Estimation . . . 139
Rounding Rules . . . 139
Estimation Strategies . . . 140
Applications in Daily Life . . . 141
Counting Money . . . 141
Time and Counting . . . 141
Inventory and Grouping . . . 142
Building Number Sense . . . 143
Benchmarks and Reference Points . . . 143
Subitizing: Instant Recognition . . . 143

Conclusion . . . 144

Addition: Combining Quantities **146**
Introduction . . . 146
Understanding Addition Conceptually . . . 146
What Does Addition Mean? . . . 146
The Counting On Strategy . . . 147
Addition Facts and Strategies . . . 147
Basic Addition Facts (0-10) . . . 147
Mental Math Strategies . . . 148
Properties of Addition . . . 149
Commutative Property . . . 149
Associative Property . . . 150
Identity Property . . . 150
Multi-Digit Addition . . . 151
Place Value in Addition . . . 151
The Standard Algorithm . . . 152
Addition with Multiple Carries . . . 153
Adding Decimals . . . 154
Decimal Place Value . . . 154
Money Addition . . . 155
Adding Fractions . . . 155
Same Denominators . . . 155
Different Denominators . . . 156
Word Problems and Applications . . . 157
Problem-Solving Strategy . . . 157
Sample Word Problems . . . 157
Mental Math and Estimation . . . 159
Quick Addition Strategies . . . 159
Estimation Techniques . . . 160
Common Mistakes and How to Avoid Them . . . 161
Typical Addition Errors . . . 161
Self-Checking Methods . . . 162
Real-World Applications . . . 163
Financial Applications . . . 163
Measurement Applications . . . 163
Time Calculations . . . 164
Building Addition Fluency . . . 165
Practice Strategies . . . 165
Games and Activities . . . 165
Conclusion . . . 166

Subtraction: Finding Differences and Taking Away **168**
Introduction . . . 168
Understanding Subtraction Conceptually . . . 168
Models of Subtraction . . . 168
The Relationship Between Addition and Subtraction . . . 169

Basic Subtraction Facts . . . 169
Mental Math Strategies for Subtraction . . . 169
Multi-Digit Subtraction . . . 170
Subtraction With Regrouping (Borrowing) . . . 170
Word Problems and Applications . . . 171
Problem-Solving Strategy . . . 171
Conclusion . . . 171

Multiplication: Repeated Addition and Scaling **173**
Introduction . . . 173
Understanding Multiplication Conceptually . . . 173
Models of Multiplication . . . 173
The Multiplication Table . . . 174
Properties of Multiplication . . . 175
Commutative Property . . . 175
Associative Property . . . 175
Distributive Property . . . 176
Multi-Digit Multiplication . . . 176
Standard Algorithm . . . 176
Area Model for Multi-Digit Multiplication . . . 177
Multiplication with Decimals . . . 178
Decimal Multiplication Rules . . . 178
Money Multiplication . . . 179
Multiplication with Fractions . . . 179
Multiplying Fractions . . . 179
Mixed Numbers . . . 180
Mental Math Strategies . . . 180
Quick Multiplication Tricks . . . 180
Estimation in Multiplication . . . 181
Word Problems and Applications . . . 182
Types of Multiplication Problems . . . 182
Problem-Solving Framework . . . 182
Real-World Applications . . . 183
Area and Perimeter . . . 183
Business and Finance . . . 183
Cooking and Recipes . . . 184
Common Mistakes and Prevention . . . 185
Typical Multiplication Errors . . . 185
Building Multiplication Fluency . . . 186
Practice Progression . . . 186
Conclusion . . . 186

Division: Sharing Equally and Finding How Many Groups **188**
Introduction . . . 188
Understanding Division Conceptually . . . 188
Models of Division . . . 188
The Relationship Between Multiplication and Division . . . 189

Basic Division Facts . . . 190
Division Facts Table . . . 190
Mental Division Strategies . . . 190
Long Division Algorithm . . . 191
Step-by-Step Long Division . . . 191
Division with Remainders . . . 192
Division by Two-Digit Numbers . . . 193
Division with Decimals . . . 193
Dividing Decimals by Whole Numbers . . . 193
Dividing by Decimals . . . 194
Division with Fractions . . . 195
Dividing by Fractions . . . 195
Mixed Numbers in Division . . . 195
Word Problems and Applications . . . 196
Types of Division Problems . . . 196
Problem-Solving Framework . . . 196
Sample Problems . . . 197
Mental Math and Estimation . . . 199
Mental Division Strategies . . . 199
Division Estimation . . . 199
Real-World Applications . . . 200
Business and Finance . . . 200
Cooking and Measurements . . . 201
Travel and Transportation . . . 201
Common Mistakes and Prevention . . . 202
Typical Division Errors . . . 202
Building Division Fluency . . . 203
Practice Progression . . . 203
Games and Activities . . . 204
Conclusion . . . 205

Fractions: Parts of a Whole **207**
Introduction . . . 207
Understanding Fractions Conceptually . . . 207
Visual Models of Fractions . . . 207
Types of Fractions . . . 208
Equivalent Fractions . . . 208
Finding Equivalent Fractions . . . 208
Simplifying Fractions . . . 209
Comparing and Ordering Fractions . . . 210
Comparing Fractions . . . 210
Ordering Fractions . . . 211
Adding and Subtracting Fractions . . . 212
Same Denominators . . . 212
Different Denominators . . . 212
Mixed Numbers . . . 213
Multiplying Fractions . . . 214
Basic Fraction Multiplication . . . 214

Multiplying Mixed Numbers . . . 215
Simplifying Before Multiplying . . . 215
Dividing Fractions . . . 216
Division by Fractions . . . 216
Complex Division Problems . . . 217
Real-World Applications . . . 217
Cooking and Recipes . . . 217
Construction and Measurement . . . 218
Time and Scheduling . . . 218
Common Mistakes and Prevention . . . 219
Typical Fraction Errors . . . 219
Building Fraction Fluency . . . 220
Conceptual Understanding First . . . 220
Conclusion . . . 221

Decimals: Another Way to Express Parts **222**
Introduction . . . 222
Understanding Decimal Place Value . . . 222
The Decimal Place Value System . . . 222
Reading and Writing Decimals . . . 223
Comparing and Ordering Decimals . . . 223
Comparing Decimals . . . 223
Ordering Decimals . . . 224
Converting Between Fractions and Decimals . . . 225
Fraction to Decimal Conversion . . . 225
Decimal to Fraction Conversion . . . 226
Adding and Subtracting Decimals . . . 227
Decimal Addition . . . 227
Decimal Subtraction . . . 228
Multiplying Decimals . . . 229
Basic Decimal Multiplication . . . 229
Money and Practical Applications . . . 230
Dividing Decimals . . . 231
Dividing Decimals by Whole Numbers . . . 231
Dividing by Decimals . . . 232
Rounding Decimals . . . 232
Rounding Rules and Strategies . . . 232
Estimating with Decimals . . . 233
Estimation Strategies . . . 233
Real-World Applications . . . 234
Money and Finance . . . 234
Measurement and Science . . . 235
Sports and Statistics . . . 235
Common Mistakes and Prevention . . . 236
Typical Decimal Errors . . . 236
Building Decimal Fluency . . . 237
Learning Progression . . . 237
Conclusion . . . 238

Percentages: Parts per Hundred **240**
Introduction . . . 240
Understanding Percentages Conceptually . . . 240
Visual Models of Percentages . . . 240
The Percentage-Fraction-Decimal Connection . . . 241
Converting Between Forms . . . 242
Percentage to Decimal . . . 242
Decimal to Percentage . . . 242
Fraction to Percentage . . . 243
Three Types of Percentage Problems . . . 244
Type 1: Finding the Percentage . . . 244
Type 2: Finding the Part . . . 245
Type 3: Finding the Whole . . . 246
Percentage Increase and Decrease . . . 246
Calculating Percentage Change . . . 246
Successive Percentage Changes . . . 247
Mental Math with Percentages . . . 248
Quick Percentage Calculations . . . 248
Percentage Shortcuts . . . 249
Real-World Applications . . . 250
Shopping and Finance . . . 250
Statistics and Data Analysis . . . 251
Health and Science . . . 252
Common Mistakes and Prevention . . . 253
Typical Percentage Errors . . . 253
Building Percentage Fluency . . . 254
Learning Progression . . . 254
Conclusion . . . 255

Geometry **256**

Introduction to Geometry: The Study of Shape and Space **257**
What is Geometry? . . . 257
The Historical Journey of Geometry . . . 257
Ancient Origins: Practical Beginnings . . . 257
Greek Mathematical Revolution . . . 258
Modern Geometry: Beyond Euclid . . . 259
Fundamental Geometric Concepts . . . 259
The Building Blocks: Point, Line, and Plane . . . 259
Angles: Measuring Direction and Turn . . . 260
Geometric Relationships and Properties . . . 261
Parallel and Perpendicular Lines . . . 261
Symmetry: Balance and Pattern . . . 262
Two-Dimensional Shapes . . . 263
Polygons: Many-Sided Figures . . . 263
Circles: Perfect Curves . . . 264

Three-Dimensional Shapes . . . 265
Polyhedra: Many-Faced Solids . . . 265
Curved Solids . . . 266
Measurement in Geometry . . . 267
Perimeter and Area . . . 267
Volume and Surface Area . . . 268
Coordinate Geometry . . . 269
The Cartesian Plane . . . 269
Transformations . . . 270
Moving and Changing Shapes . . . 270
Applications of Geometry . . . 271
Architecture and Construction . . . 271
Art and Design . . . 271
Science and Nature . . . 272
Modern Geometry . . . 273
Non-Euclidean Geometries . . . 273
Topology: Rubber Sheet Geometry . . . 274
Building Geometric Intuition . . . 275
Visualization Skills . . . 275
Problem-Solving Strategies . . . 275
Conclusion . . . 276

Points, Lines, and Planes: The Building Blocks of Geometry 278
Introduction . . . 278
Points: The Foundation of All Geometry . . . 278
Understanding Points . . . 278
Points in Space . . . 279
Lines: One-Dimensional Infinity . . . 280
Understanding Lines . . . 280
Types of Lines and Line Segments . . . 280
Line Relationships . . . 281
Planes: Two-Dimensional Surfaces . . . 282
Understanding Planes . . . 282
Plane Relationships . . . 283
Coordinate Systems . . . 284
The Cartesian Plane . . . 284
3D Coordinate System . . . 285
Geometric Constructions . . . 286
Basic Constructions with Compass and Straightedge . . . 286
Perpendicular and Parallel Constructions . . . 287
Angle Relationships . . . 288
Types of Angles . . . 288
Angle Relationships . . . 289
Distance and Midpoint . . . 290
Distance Formulas . . . 290
Midpoint Formulas . . . 291
Applications and Problem Solving . . . 292
Real-World Applications . . . 292

Problem-Solving Strategies . . . 293
Common Mistakes and Misconceptions . . . 294
Typical Errors . . . 294
Building Geometric Intuition . . . 295
Visualization Exercises . . . 295
Conclusion . . . 296

Angles: Measuring Direction and Turn **298**
Introduction . . . 298
Understanding Angles . . . 298
Basic Angle Concepts . . . 298
Measuring Angles . . . 299
Types of Angles . . . 300
Classification by Measure . . . 300
Special Angle Relationships . . . 301
Angles and Parallel Lines . . . 303
Transversals and Parallel Lines . . . 303
Using Angle Relationships . . . 304
Angle Constructions . . . 304
Basic Angle Constructions . . . 304
Special Angle Constructions . . . 306
Angle Measurement Tools . . . 307
Using a Protractor . . . 307
Digital Angle Measurement . . . 308
Angles in Polygons . . . 309
Interior Angles of Polygons . . . 309
Exterior Angles of Polygons . . . 310
Trigonometric Angles . . . 310
Angles in Standard Position . . . 310
Unit Circle and Angles . . . 311
Real-World Applications . . . 312
Navigation and Direction . . . 312
Architecture and Construction . . . 313
Art and Design . . . 314
Problem-Solving with Angles . . . 315
Angle Calculation Strategies . . . 315
Complex Angle Problems . . . 316
Common Mistakes and Misconceptions . . . 318
Typical Angle Errors . . . 318
Building Angle Intuition . . . 318
Angle Estimation Skills . . . 318
Conclusion . . . 320

Triangles: The Strongest Shape **321**
Introduction . . . 321
Basic Triangle Properties . . . 321
Triangle Inequality . . . 321
Angle Sum Property . . . 322

Classification of Triangles . . . 323
By Side Lengths . . . 323
By Angle Measures . . . 324
Special Lines in Triangles . . . 325
Altitudes . . . 325
Medians . . . 327
Angle Bisectors . . . 328
Perpendicular Bisectors . . . 329
Triangle Congruence . . . 330
Congruence Postulates . . . 330
Non-Congruence Cases . . . 331
Triangle Similarity . . . 332
Similarity Tests . . . 332
Applications of Similarity . . . 333
Right Triangles . . . 334
Pythagorean Theorem . . . 334
Special Right Triangles . . . 336
Triangle Area Formulas . . . 337
Basic Area Formulas . . . 337
Area Relationships . . . 338
Applications and Problem Solving . . . 339
Real-World Applications . . . 339
Problem-Solving Strategies . . . 341
Common Mistakes and Misconceptions . . . 342
Typical Triangle Errors . . . 342
Building Triangle Intuition . . . 342
Visualization Exercises . . . 342
Conclusion . . . 343

Quadrilaterals: Four-Sided Figures **345**
Introduction . . . 345
Basic Properties of Quadrilaterals . . . 345
Angle Sum Property . . . 345
Diagonals in Quadrilaterals . . . 346
Classification of Quadrilaterals . . . 347
The Quadrilateral Family Tree . . . 347
Trapezoids . . . 347
Parallelograms . . . 349
Rectangles . . . 350
Rhombus . . . 351
Squares . . . 352
Kites . . . 353
Special Quadrilateral Theorems . . . 354
Midpoint Theorems . . . 354
Diagonal Relationships . . . 355
Area and Perimeter Formulas . . . 356
Area Formulas Summary . . . 356
Perimeter Formulas . . . 358

Coordinate Geometry of Quadrilaterals . . . 359
Using Coordinates . . . 359
Transformations of Quadrilaterals . . . 360
Applications and Problem Solving . . . 361
Real-World Applications . . . 361
Problem-Solving Strategies . . . 362
Common Mistakes and Misconceptions . . . 363
Typical Quadrilateral Errors . . . 363
Building Quadrilateral Intuition . . . 364
Recognition Exercises . . . 364
Conclusion . . . 365

Circles: Perfect Curves and Endless Possibilities **367**
Introduction . . . 367
Basic Circle Elements . . . 367
Fundamental Components . . . 367
Circle Measurements . . . 369
Angles in Circles . . . 370
Central Angles . . . 370
Inscribed Angles . . . 371
Tangent-Chord Angles . . . 372
Circle Theorems . . . 373
Chord Properties . . . 373
Cyclic Quadrilaterals . . . 374
Circle Constructions . . . 375
Basic Constructions . . . 375
Advanced Constructions . . . 377
Coordinate Geometry of Circles . . . 378
Circle Equations . . . 378
Circle Intersections . . . 379
Transformations of Circles . . . 380
Applications and Problem Solving . . . 381
Real-World Applications . . . 381
Problem-Solving Strategies . . . 382
Common Mistakes and Misconceptions . . . 383
Typical Circle Errors . . . 383
Building Circle Intuition . . . 384
Visualization Exercises . . . 384
Conclusion . . . 385

Area and Perimeter: Measuring Space and Boundary **387**
Introduction . . . 387
Understanding Perimeter . . . 387
Basic Perimeter Concepts . . . 387
Perimeter of Curved Shapes . . . 388
Understanding Area . . . 390
Basic Area Concepts . . . 390
Area of Circles and Sectors . . . 391

Advanced Area Formulas . . . 392
Triangles - Multiple Methods . . . 392
Regular Polygons . . . 394
Composite Shapes . . . 395
Breaking Down Complex Shapes . . . 395
Common Composite Shapes . . . 396
Units and Conversions . . . 398
Area Unit Conversions . . . 398
Perimeter Unit Conversions . . . 399
Problem-Solving Applications . . . 399
Real-World Area Problems . . . 399
Real-World Perimeter Problems . . . 400
Optimization Problems . . . 401
Maximum Area for Given Perimeter . . . 401
Minimum Perimeter for Given Area . . . 402
Common Mistakes and Problem-Solving Tips . . . 403
Typical Errors . . . 403
Problem-Solving Strategies . . . 404
Conclusion . . . 405

Pre-Algebra **407**

Introduction to Pre-Algebra: The Bridge to Abstract Mathematics **408**
What is Pre-Algebra? . . . 408
The Evolution from Numbers to Variables . . . 408
From Concrete to Abstract . . . 408
The Power of Variables . . . 409
Key Concepts in Pre-Algebra . . . 410
Expressions vs. Equations . . . 410
The Order of Operations in Algebra . . . 411
Fundamental Operations with Variables . . . 412
Combining Like Terms . . . 412
The Distributive Property . . . 413
Introduction to Equations . . . 414
What Makes an Equation True? . . . 414
Basic Equation Solving . . . 415
Patterns and Sequences . . . 416
Recognizing Mathematical Patterns . . . 416
Using Variables to Describe Patterns . . . 417
Real-World Applications . . . 418
Modeling with Variables . . . 418
Problem-Solving with Pre-Algebra . . . 419
Building Algebraic Thinking . . . 421
From Arithmetic to Algebraic Reasoning . . . 421
Preparing for Formal Algebra . . . 422
Common Challenges and Solutions . . . 423
Typical Pre-Algebra Difficulties . . . 423

Conclusion 425

Variables and Expressions: The Language of Algebra **427**
Introduction 427
Understanding Variables 427
What is a Variable? 427
Variables vs. Constants 428
Introduction to Expressions 428
What is an Expression? 428
Parts of an Expression 429
Evaluating Expressions 429
Substitution and Evaluation 429
Combining Like Terms 430
Identifying Like Terms 430
Combining Like Terms 431
The Distributive Property 431
Understanding Distribution 431
Advanced Distribution 432
Simplifying Expressions 432
Step-by-Step Simplification 432
Real-World Applications 433
Writing Expressions for Real Situations 433
Practical Applications 434
Conclusion 435

Solving Equations: Finding the Unknown **436**
Introduction 436
Understanding Equations 436
What Makes an Equation True? 436
The Balance Model 437
Basic Equation Solving 437
One-Step Equations 437
Two-Step Equations 438
Multi-Step Equations 439
Equations with Variables on Both Sides 439
Equations with Parentheses 440
Equations with Fractions 441
Clearing Fractions 441
Special Cases 442
No Solution and Infinite Solutions 442
Problem-Solving with Equations 443
Translating Word Problems 443
Real-World Applications 444
Checking Solutions 445
Verification Methods 445
Common Mistakes and How to Avoid Them 446
Typical Equation-Solving Errors 446

Building Equation-Solving Skills . . . 447
Practice Progression . . . 447
Conclusion . . . 448

Inequalities: When Things Are Not Equal **449**
Introduction . . . 449
Understanding Inequality Symbols . . . 449
Basic Inequality Symbols . . . 449
Graphing Inequalities on Number Lines . . . 450
Solving Linear Inequalities . . . 451
Basic Inequality Solving . . . 451
One-Step Inequalities . . . 451
Two-Step Inequalities . . . 452
Multi-Step Inequalities . . . 453
Inequalities with Variables on Both Sides . . . 453
Inequalities with Parentheses . . . 454
Compound Inequalities . . . 455
"And" Compound Inequalities . . . 455
"Or" Compound Inequalities . . . 456
Absolute Value Inequalities . . . 457
Understanding Absolute Value Inequalities . . . 457
Word Problems with Inequalities . . . 458
Translating Word Problems . . . 458
Business and Economics Applications . . . 459
Graphing Inequalities . . . 460
Coordinate Plane Inequalities . . . 460
Common Mistakes and Solutions . . . 461
Typical Inequality Errors . . . 461
Conclusion . . . 462

Graphing Linear Equations: Visualizing Relationships **464**
Introduction . . . 464
The Coordinate Plane . . . 465
Understanding Coordinates . . . 465
Plotting Points . . . 465
Linear Equations and Their Graphs . . . 466
What Makes an Equation Linear? . . . 466
Graphing by Making a Table . . . 467
Slope: The Rate of Change . . . 468
Understanding Slope . . . 468
Calculating Slope from Graphs . . . 469
Slope-Intercept Form . . . 471
Understanding $y = mx + b$ . . . 471
Graphing Using Slope-Intercept Form . . . 471
Intercepts . . . 473
Finding x and y Intercepts . . . 473
Graphing Using Intercepts . . . 474

Writing Linear Equations . . . 475
Writing Equations from Graphs . . . 475
Writing Equations from Word Problems . . . 476
Parallel and Perpendicular Lines . . . 477
Understanding Line Relationships . . . 477
Applications and Problem Solving . . . 478
Real-World Linear Relationships . . . 478
Common Mistakes and Solutions . . . 479
Typical Graphing Errors . . . 479
Conclusion . . . 480

Algebra **482**

Introduction to Algebra: The Language of Mathematical Relationships **483**
What is Algebra? . . . 483
The Historical Development of Algebra . . . 483
Ancient Origins . . . 483
The Word "Algebra" . . . 484
Core Concepts of Algebra . . . 484
Variables and Constants . . . 484
Expressions, Equations, and Functions . . . 485
The Fundamental Operations . . . 486
Types of Algebraic Equations . . . 487
Linear Equations . . . 487
Quadratic Equations . . . 487
Polynomial Equations . . . 488
Algebraic Thinking and Problem Solving . . . 489
From Arithmetic to Algebraic Reasoning . . . 489
Problem-Solving Strategies . . . 489
The Power and Beauty of Algebra . . . 490
Unifying Mathematical Concepts . . . 490
Patterns and Generalizations . . . 491
Modern Applications of Algebra . . . 492
Technology and Computing . . . 492
Science and Engineering . . . 493
Building Algebraic Fluency . . . 493
Essential Skills for Success . . . 493
Conclusion . . . 494

Linear Equations: The Foundation of Algebraic Problem Solving **496**
What are Linear Equations? . . . 496
Solving Linear Equations in One Variable . . . 496
Basic Solution Process . . . 496
Types of Linear Equation Solutions . . . 497
Applications of Linear Equations . . . 497
Word Problems and Real-World Applications . . . 497
Percent Problems . . . 498

Linear Equations in Two Variables . . . 499
Graphing Linear Equations . . . 499
Slope and Rate of Change . . . 499
Systems of Linear Equations . . . 500
Solving Systems by Substitution . . . 500
Solving Systems by Elimination . . . 500
Types of Systems . . . 501
Linear Inequalities . . . 502
Solving Linear Inequalities . . . 502
Graphing Linear Inequalities . . . 502
Problem-Solving Strategies . . . 503
Setting Up Linear Equations . . . 503
Common Problem Types . . . 504
Linear Functions and Modeling . . . 504
Function Notation and Linear Functions . . . 504
Linear Modeling . . . 505
Summary and Key Concepts . . . 506

Quadratic Equations: Exploring Parabolic Relationships 507
Introduction to Quadratic Equations . . . 507
The Parabola: Graph of Quadratic Functions . . . 507
Understanding Parabolic Shape . . . 507
Graphing Quadratic Functions . . . 508
Solving Quadratic Equations . . . 509
Method 1: Factoring . . . 509
Method 2: Quadratic Formula . . . 509
Method 3: Completing the Square . . . 510
Method 4: Graphing . . . 511
Applications of Quadratic Equations . . . 511
Projectile Motion . . . 511
Area and Optimization Problems . . . 512
Geometric Applications . . . 513
Complex Solutions and the Discriminant . . . 513
Understanding the Discriminant . . . 513
Introduction to Complex Numbers . . . 514
Systems of Quadratic Equations . . . 515
Linear-Quadratic Systems . . . 515
Quadratic Inequalities . . . 515
Solving Quadratic Inequalities . . . 515
Advanced Topics and Extensions . . . 516
Quadratic Functions in Vertex Form . . . 516
Quadratic Modeling . . . 517
Summary and Key Concepts . . . 517

Polynomials: Building Blocks of Advanced Algebra 519
Introduction to Polynomials . . . 519
Polynomial Classification . . . 519
Degree and Leading Terms . . . 519

Standard Form and Terminology . . . 520
Polynomial Operations . . . 521
Addition and Subtraction . . . 521
Multiplication . . . 521
Division . . . 522
Synthetic Division . . . 523
Factoring Polynomials . . . 524
Common Factoring Techniques . . . 524
Factoring Quadratic Trinomials . . . 524
Special Factoring Patterns . . . 525
Polynomial Functions and Graphs . . . 526
Behavior of Polynomial Functions . . . 526
Finding Zeros and Factors . . . 526
Multiplicity and Graph Behavior . . . 527
Applications of Polynomials . . . 528
Modeling Real-World Phenomena . . . 528
Polynomial Regression . . . 528
Advanced Polynomial Topics . . . 529
Polynomial Inequalities . . . 529
Complex Zeros and Conjugate Pairs . . . 529
Summary and Key Concepts . . . 530

Rational Expressions: Working with Algebraic Fractions 532
Introduction to Rational Expressions . . . 532
Simplifying Rational Expressions . . . 532
Finding Common Factors . . . 532
Complex Rational Expressions . . . 533
Operations with Rational Expressions . . . 534
Addition and Subtraction . . . 534
Multiplication and Division . . . 535
Solving Rational Equations . . . 535
Basic Solution Techniques . . . 535
Extraneous Solutions . . . 536
Applications of Rational Expressions . . . 537
Work Rate Problems . . . 537
Distance-Rate-Time Problems . . . 538
Mixture and Concentration Problems . . . 538
Rational Functions and Their Graphs . . . 539
Domain and Range . . . 539
Vertical and Horizontal Asymptotes . . . 539
Graphing Rational Functions . . . 540
Advanced Topics . . . 541
Partial Fraction Decomposition . . . 541
Rational Inequalities . . . 541
Summary and Key Concepts . . . 542

Linear Algebra **544**

Introduction to Linear Algebra: The Mathematics of Vector Spaces **545**
What is Linear Algebra? . . . 545
Historical Development . . . 546
Ancient Origins and Early Developments . . . 546
The Geometric Revolution . . . 546
Fundamental Concepts . . . 547
Vectors: The Building Blocks . . . 547
Vector Operations . . . 548
Matrices: Organizing Linear Information . . . 549
Systems of Linear Equations . . . 550
The Central Problem . . . 550
Solution Methods . . . 551
Vector Spaces: The Abstract Framework . . . 552
Axioms and Structure . . . 552
Subspaces and Span . . . 552
Linear Transformations . . . 553
Functions Between Vector Spaces . . . 553
Applications and Connections . . . 554
Computer Graphics and 3D Modeling . . . 554
Machine Learning and Data Science . . . 555
Physics and Engineering . . . 556
The Beauty and Power of Linear Algebra . . . 557
Unifying Mathematical Concepts . . . 557
Computational Power . . . 558
Building Linear Algebra Intuition . . . 558
Developing Geometric Insight . . . 558
Problem-Solving Strategies . . . 559
Conclusion . . . 560

Vectors: The Foundation of Linear Algebra **562**
Introduction to Vectors . . . 562
Vector Notation and Representation . . . 562
Coordinate Representation . . . 562
Geometric Interpretation . . . 563
Vector Operations . . . 564
Vector Addition . . . 564
Scalar Multiplication . . . 565
Linear Combinations . . . 565
Dot Product (Inner Product) . . . 566
Definition and Computation . . . 566
Geometric Applications . . . 567
Vector Projections . . . 568
Cross Product (3D Only) . . . 569
Definition and Properties . . . 569
Applications of Cross Product . . . 569

Vector Spaces and Subspaces . . . 570
Vector Space Axioms . . . 570
Subspaces . . . 571
Linear Independence and Basis . . . 572
Linear Independence . . . 572
Basis and Dimension . . . 573
Applications and Examples . . . 573
Computer Graphics . . . 573
Physics Applications . . . 574
Engineering Applications . . . 575
Summary and Key Concepts . . . 575

Matrices: Organizing and Transforming Linear Information 577
Introduction to Matrices . . . 577
Types of Matrices . . . 578
Special Matrix Categories . . . 578
Matrix Dimensions and Indexing . . . 579
Matrix Operations . . . 579
Matrix Addition and Subtraction . . . 579
Matrix Multiplication . . . 580
Properties of Matrix Multiplication . . . 581
Matrix Transpose . . . 582
Definition and Properties . . . 582
Determinants . . . 583
Definition and Calculation . . . 583
Geometric Interpretation . . . 584
Matrix Inverse . . . 585
Definition and Properties . . . 585
Computing Matrix Inverse . . . 585
Systems of Linear Equations . . . 586
Matrix Representation . . . 586
Solution Methods . . . 587
Solution Types and Consistency . . . 588
Elementary Matrices and Row Operations . . . 589
Elementary Row Operations . . . 589
Row Echelon Forms . . . 590
Matrix Rank . . . 591
Definition and Properties . . . 591
Applications of Matrices . . . 592
Computer Graphics and Transformations . . . 592
Data Analysis and Statistics . . . 593
Engineering Applications . . . 594
Advanced Matrix Concepts . . . 595
Matrix Norms . . . 595
Matrix Decompositions Preview . . . 596
Summary and Key Concepts . . . 596
Determinant as Area/Volume Scaling . . . 598

Systems of Linear Equations: Solving Multiple Relationships Simultaneously **599**
Introduction to Linear Systems . . . 599
Solution Types and Geometric Interpretation . . . 600
Understanding Solution Behavior . . . 600
Consistency and Rank . . . 601
Gaussian Elimination . . . 602
The Elimination Process . . . 602
Back Substitution . . . 603
Gauss-Jordan Elimination . . . 603
Reduced Row Echelon Form . . . 603
Parametric Solutions . . . 605
Special Cases and Applications . . . 606
Homogeneous Systems . . . 606
Underdetermined Systems . . . 607
Overdetermined Systems . . . 608
Matrix Methods for Linear Systems . . . 609
Using Matrix Inverse . . . 609
Cramer's Rule . . . 610
Applications and Real-World Examples . . . 611
Economic Models . . . 611
Engineering Applications . . . 612
Data Fitting and Regression . . . 613
Computational Considerations . . . 614
Numerical Stability . . . 614
Computational Complexity . . . 616
Summary and Key Concepts . . . 617

Vector Spaces, Spans, and Bases **619**
Diagram: span and basis . . . 619
1. Vector space axioms (intuition) . . . 619
Standard examples . . . 620
Non-examples . . . 620
Worked example 1 — function space . . . 620
2. Subspaces . . . 620
Fundamental subspaces of a matrix $A \in \mathbb{R}^{m\times n}$ . . . 620
Worked example 2 — plane through origin . . . 621
3. Linear combinations and span . . . 621
Worked example 3 . . . 621
4. Linear independence . . . 621
Tests . . . 621
Worked example 4 . . . 621
Worked example 5 . . . 622
5. Basis and dimension . . . 622
Coordinates . . . 622
Worked example 6 — basis of a plane . . . 622
6. Rank–nullity theorem . . . 623
Worked example 7 . . . 623
Worked example 8 — underdetermined systems . . . 623

7. Four fundamental subspaces (preview of structure) . . . 623
8. Linear maps and matrices . . . 624
9. CS / ML map . . . 624
Worked example 9 — collinear features . . . 624
10. Infinite dimensions (awareness) . . . 624
11. Pitfalls . . . 624
12. Checkpoint . . . 625
Exercises . . . 625
Easy . . . 625
Medium . . . 625
Challenge . . . 625
Checks . . . 626
Summary . . . 626

Determinants, Inverses, and Volume **627**
Diagram: area scaling . . . 627
1. Invertibility criteria . . . 627
Worked example 1 — 2×2 inverse . . . 628
Worked example 2 — singular . . . 628
2. Computing determinants . . . 628
2×2 . . . 628
3×3 (cofactor expansion along row 1) . . . 628
Via row reduction . . . 628
Multiplicativity and friends . . . 629
Worked example 3 — triangular . . . 629
Worked example 4 — product . . . 629
3. Geometric meaning . . . 629
Worked example 5 — shear preserves area . . . 629
Worked example 6 — scaling . . . 629
4. Determinant and eigenvalues . . . 630
5. Cramer's rule (theory / tiny n) . . . 630
6. Explicit inverse formulas . . . 630
2×2 . . . 630
Adjugate (general n) . . . 630
7. Jacobians and change of variables . . . 630
Worked example 7 — polar (idea) . . . 631
8. Multivariate Gaussians . . . 631
9. Numerical wisdom . . . 631
10. CS / graphics / systems connections . . . 631
11. Pitfalls . . . 631
12. Checkpoint . . . 632
Exercises . . . 632
Easy . . . 632
Medium . . . 632
Challenge . . . 633
Checks . . . 633
Summary . . . 633

Montgomery Modular Multiplication **634**

Montgomery Modular Multiplication **635**
Thesis . . . 635
Mental model . . . 635
Worked examples . . . 635
Trap . . . 635
Boring rule . . . 635
Try this . . . 635
Sources . . . 636

Discrete Mathematics 637

Introduction to Discrete Mathematics: The Mathematics of Distinct Objects **638**
Chapters in this part . . . 638
What is Discrete Mathematics? . . . 638
Historical Development . . . 639
Ancient Origins and Modern Evolution . . . 639
The Computer Science Revolution . . . 640
Core Areas of Discrete Mathematics . . . 641
Logic and Proof Techniques . . . 641
Set Theory and Relations . . . 642
Combinatorics and Counting . . . 643
Graph Theory . . . 644
Number Theory . . . 645
Applications in Computer Science . . . 646
Algorithms and Data Structures . . . 646
Cryptography and Security . . . 647
Database Theory . . . 648
Problem-Solving Techniques . . . 649
Proof Strategies . . . 649
Problem-Solving Strategies . . . 651
Modern Applications and Connections . . . 652
Machine Learning and Data Science . . . 652
Quantum Computing . . . 653
Bioinformatics and Computational Biology . . . 654
The Beauty and Unity of Discrete Mathematics . . . 655
Connections Across Mathematics . . . 655
Aesthetic and Philosophical Aspects . . . 656
Building Discrete Mathematical Thinking . . . 657
Developing Problem-Solving Skills . . . 657
Conclusion . . . 659

Logic and Proofs: The Foundation of Mathematical Reasoning **661**
Introduction to Mathematical Logic . . . 661
Propositional Logic . . . 662
Basic Concepts and Connectives . . . 662

Truth Tables . . . 662
Proof Techniques . . . 664
Direct Proof . . . 664
Proof by Contradiction . . . 664
Mathematical Induction . . . 665
Applications in Computer Science . . . 666
Programming Logic . . . 666
Database Queries . . . 666
Summary and Key Concepts . . . 667

Sets and Relations: The Language of Mathematical Structure 668
Introduction to Set Theory . . . 668
Basic Set Concepts . . . 669
Set Notation and Membership . . . 669
Set Equality and Subsets . . . 670
Set Operations . . . 671
Basic Set Operations . . . 671
Properties of Set Operations . . . 672
Venn Diagrams . . . 673
Cartesian Products and Relations . . . 674
Cartesian Products . . . 674
Binary Relations . . . 675
Properties of Relations . . . 676
Equivalence Relations and Partitions . . . 677
Equivalence Relations . . . 677
Partitions . . . 678
Functions as Relations . . . 679
Functions Defined as Relations . . . 679
Applications in Computer Science . . . 681
Database Relations . . . 681
Data Structures and Algorithms . . . 682

Combinatorics: The Art and Science of Counting 684
Introduction to Combinatorics . . . 684
Fundamental Counting Principles . . . 685
The Addition Principle . . . 685
The Multiplication Principle . . . 686
Inclusion-Exclusion Principle . . . 687
Permutations . . . 688
Basic Permutations . . . 688
Permutations with Restrictions . . . 690
Combinations . . . 691
Basic Combinations . . . 691
Combinations with Repetition . . . 692
Advanced Combination Problems . . . 694
The Binomial Theorem . . . 695
Binomial Expansions . . . 695

Applications in Computer Science . . . 696
Algorithm Analysis . . . 696
Cryptography and Security . . . 697
Network Analysis and Graph Theory . . . 698
Summary and Key Concepts . . . 699

Recurrence Relations **701**
Diagram: expand a recurrence . . . 701
1. What is a recurrence? . . . 701
Why CS needs closed forms . . . 701
2. Linear homogeneous recurrences (constant coefficients) . . . 702
Characteristic equation method . . . 702
Worked example 1 — second order . . . 702
Repeated roots . . . 703
Complex roots . . . 703
3. Fibonacci and Binet's formula . . . 703
Worked example 2 — identity via recurrence . . . 703
CS appearances . . . 703
4. Non-homogeneous linear recurrences . . . 704
Worked example 3 . . . 704
5. Master theorem (algorithmic divide-and-conquer) . . . 704
Worked example 4 — merge sort . . . 704
Worked example 5 — binary search . . . 705
Worked example 6 — case 1 . . . 705
Worked example 7 — case 3 . . . 705
When Master does not apply cleanly . . . 705
6. Recursion trees and substitution . . . 705
Tree method . . . 705
Substitution (guess + induction) . . . 705
Worked example 8 — linear scan recurrence . . . 705
7. Generating functions (preview) . . . 706
8. Recurrences in dynamic programming . . . 706
Worked example 9 — no consecutive 1s . . . 706
9. Solving checklist . . . 707
10. Pitfalls . . . 707
11. Checkpoint . . . 707
Exercises . . . 707
Easy . . . 707
Medium . . . 708
Challenge . . . 708
Checks . . . 708
Summary . . . 709

Induction and Invariants **710**
Diagram: induction staircase . . . 710
1. Standard (weak) induction . . . 710
Worked example 1 — sum formula . . . 710
Worked example 2 — inequality . . . 711

2. Strong induction . . . 711
Worked example 3 — every $n \geq 2$ has a prime factor . . . 711
Worked example 4 — Fibonacci closed bound . . . 711
3. Structural induction . . . 711
4. Classic proof patterns . . . 711
Worked example 5 — geometric series . . . 712
Worked example 6 — $n! < n^n$ for $n \geq 2$ . . . 712
5. Loop invariants . . . 712
Worked example 7 — linear search . . . 712
Worked example 8 — iterative factorial . . . 713
Worked example 9 — insertion into sorted prefix . . . 713
6. Invariants in discrete structures . . . 713
Parity and modular invariants . . . 713
Coloring arguments . . . 713
Potential functions . . . 713
7. Invariants in concurrent / networked systems . . . 714
8. Common failures (and the horses paradox) . . . 714
Classic bug: "all horses are the same color" . . . 714
9. Induction vs invariants — same idea . . . 714
10. Writing good inductive proofs (checklist) . . . 715
11. Pitfalls . . . 715
12. Checkpoint . . . 715
Exercises . . . 716
Easy . . . 716
Medium . . . 716
Challenge . . . 716
Checks . . . 717
Summary . . . 717

Category Theory of Terms: Argument Lists as Morphisms **718**

Category Theory of Terms: Argument Lists as Morphisms **719**
Thesis . . . 719
Mental model . . . 719
Worked examples . . . 720
Case 1: From trees to forests . . . 720
Case 2: `TermSeq` as a morphism . . . 720
Case 3: Why the context size must be explicit . . . 722
Case 4: Two products . . . 722
Case 5: Identities, projections, building terms by composition . . . 722
The trap . . . 723
The boring rule . . . 723
Try this . . . 723
Sources . . . 724

Lean 4.35: vcgen and lake check **725**

Lean 4.35: `vcgen` and `lake check` **726**
Thesis . . . 726
Mental model . . . 726
Worked examples . . . 727
Case 1: Swap `mvcgen` for `vcgen` . . . 727
Case 2: Intrinsic contracts on `def` (loop with invariant) . . . 727
Case 3: `lake check` and `--paranoid` . . . 728
Case 4: `lake comparator` for challenge / solution pairs . . . 728
Case 5: Cache / build packaging tweak . . . 729
The trap . . . 729
The boring rule . . . 729
Try this . . . 729
Sources . . . 730

Penrose Tilings **731**

Penrose Tilings **732**
Thesis . . . 732
Mental model . . . 732
Worked examples . . . 732
Trap . . . 732
Boring rule . . . 732
Try this . . . 733
Sources . . . 733

The Banach–Tarski Paradox **734**

The Banach–Tarski Paradox **735**
Thesis . . . 735
Mental model . . . 735
Worked examples . . . 735
What fails without Choice / in 2D . . . 735
Where Choice shows up . . . 735
Trap . . . 735
Boring rule . . . 735
Try this . . . 736
Sources . . . 736

Statistics **737**

Introduction to Statistics: Making Sense of Data in an Uncertain World **738**
What is Statistics? . . . 738
Historical Development . . . 739
From Ancient Records to Modern Data Science . . . 739
The Statistical Revolution . . . 740
Fundamental Concepts . . . 741
Data and Variables . . . 741
Population vs. Sample . . . 742

Descriptive vs. Inferential Statistics . . . 743
The Role of Probability . . . 744
Probability as Foundation . . . 744
Statistical Models . . . 745
Statistical Thinking . . . 747
The Scientific Method and Statistics . . . 747
Critical Thinking with Data . . . 748
Modern Applications . . . 749
Big Data and Data Science . . . 749
Machine Learning and AI . . . 750
Business Analytics and Decision Science . . . 751
Building Statistical Intuition . . . 753
Developing Statistical Thinking . . . 753
Conclusion . . . 754

Descriptive Statistics: Summarizing and Visualizing Data 756
Introduction to Descriptive Statistics . . . 756
Organizing Data . . . 757
Frequency Distributions . . . 757
Stem-and-Leaf Plots . . . 758
Measures of Central Tendency . . . 759
The Mean . . . 759
The Median . . . 760
The Mode . . . 761
Measures of Variability . . . 762
Range and Interquartile Range . . . 762
Variance and Standard Deviation . . . 764
Coefficient of Variation . . . 765
Measures of Position . . . 766
Percentiles and Quartiles . . . 766
Z-Scores (Standard Scores) . . . 767
Data Visualization . . . 768
Histograms . . . 768
Box Plots . . . 770
Scatter Plots . . . 771
Summary Statistics and Reports . . . 772
Creating Effective Summaries . . . 772
Summary and Key Concepts . . . 774

Probability: The Mathematics of Uncertainty 776
Introduction to Probability . . . 776
Basic Probability Concepts . . . 777
Sample Spaces and Events . . . 777
Probability Definitions and Axioms . . . 778
Counting Techniques in Probability . . . 780
Conditional Probability and Independence . . . 781
Conditional Probability . . . 781
Independence . . . 782

Bayes' Theorem . . . 783
Discrete Probability Distributions . . . 785
Random Variables . . . 785
Common Discrete Distributions . . . 786
Continuous Probability Distributions . . . 789
Continuous Random Variables . . . 789
Normal Distribution . . . 789
Other Continuous Distributions . . . 791
Summary and Key Concepts . . . 793
Monte Carlo Estimation Sketch . . . 794

Random Variables and Common Distributions **796**
Diagram: the pipeline . . . 796
1. Discrete vs continuous . . . 796
2. Expectation and variance . . . 797
3. Catalog of workhorse distributions . . . 797
Bernoulli(p) . . . 797
Binomial(n, p) . . . 797
Geometric(p) (trials until first success) . . . 797
Poisson(λ) . . . 798
Exponential(λ) . . . 798
Normal(μ, σ^2) . . . 798
Uniform(a, b) . . . 798
4. Worked examples . . . 798
5. Computer science map . . . 799
6. How to choose a distribution (checklist) . . . 799
Exercises . . . 799
Quick answers (check after you try) . . . 799

Statistical Inference: Confidence and Hypothesis Tests **801**
Diagram: inference loop . . . 801
1. Point estimates vs interval estimates . . . 801
2. Confidence intervals (means) . . . 801
3. Hypothesis testing workflow . . . 802
4. Type I, Type II, power . . . 802
5. Two-proportion A/B sketch . . . 802
6. Multiple testing . . . 803
7. Sample size (ballpark) . . . 803
8. Pitfalls . . . 803
9. Worked mini example . . . 804
Exercises . . . 804
Quick checks . . . 804

Regression and Experimental Design **805**
1. Linear regression model . . . 805
Worked example 1 — simple regression . . . 805
Worked example 2 — multiple regression . . . 805

2. Classical OLS assumptions . . . 805
Worked example 3 — omitted variable bias . . . 806
3. Fitted values, residuals, R^2 . . . 806
Worked example 4 . . . 806
4. Inference for coefficients . . . 806
Worked example 5 — confidence interval . . . 806
5. Diagnostics . . . 806
Worked example 6 . . . 807
Worked example 7 — log transform . . . 807
6. Regularized regression (link) . . . 807
Worked example 8 . . . 807
7. Correlation is not causation . . . 807
Worked example 9 — ice cream and drowning . . . 807
Worked example 10 — product metrics . . . 808
8. Experimental design and A/B tests . . . 808
Core principles . . . 808
Worked example 11 — sample size sketch . . . 808
Worked example 12 — SRM . . . 808
9. Threats to validity . . . 809
Worked example 13 — sequential testing . . . 809
Worked example 14 — CUPED / variance reduction . . . 809
10. From experiment to regression . . . 809
Worked example 15 — heterogeneous effects . . . 809
11. Pitfalls . . . 809
12. Checkpoint . . . 810
Exercises . . . 810
Easy . . . 810
Medium . . . 810
Challenge . . . 810
Summary . . . 811

Calculus **812**

Calculus for Computer Science and AI **813**
Why calculus matters in CS/AI . . . 813
What you will learn . . . 813
The derivative in one sentence . . . 814
Worked example 1 — sensitivity . . . 814
Worked example 2 — non-differentiable ML . . . 814
The integral in one sentence . . . 814
Worked example 3 . . . 814
Multivariable leap for ML . . . 814
Worked example 4 . . . 814
Prerequisites . . . 815
Study sequence (CS-oriented) . . . 815
Pitfalls . . . 815
Checkpoint . . . 815

Exercises . . . 816
Summary . . . 816

Limits and Continuity **817**
1. Informal idea . . . 817
Worked example 1 . . . 817
2. Epsilon–delta definition . . . 817
Worked example 2 — linear function . . . 817
Worked example 3 — quadratic sketch . . . 817
3. One-sided limits . . . 817
Worked example 4 — jump . . . 818
Worked example 5 — absolute value . . . 818
4. Infinite limits and asymptotes . . . 818
Worked example 6 . . . 818
5. Continuity . . . 818
Worked example 7 . . . 818
Worked example 8 — removable discontinuity . . . 818
6. Types of discontinuities . . . 819
Worked example 9 . . . 819
7. Algebra of limits . . . 819
Worked example 10 . . . 819
Worked example 11 . . . 819
8. Intermediate Value Theorem (IVT) . . . 819
Worked example 12 . . . 819
9. Extreme Value Theorem . . . 820
10. CS connections . . . 820
Worked example 13 — softplus . . . 820
11. Pitfalls . . . 820
12. Checkpoint . . . 820
Exercises . . . 821
Easy . . . 821
Medium . . . 821
Challenge . . . 821
Summary . . . 821

Derivatives and Applications in Computer Science **822**
What is a Derivative? . . . 822
Basic Derivative Rules . . . 822
Power Rule . . . 822
Chain Rule . . . 823
Applications in Machine Learning . . . 823
1. Gradient Descent . . . 823
2. Neural Network Backpropagation . . . 824
Applications in Computer Graphics . . . 824
1. Parametric Curves . . . 824
2. Surface Normals . . . 825
Applications in Algorithm Analysis . . . 827
Growth Rate Analysis . . . 827

Optimization Problems . . . 828
Finding Critical Points . . . 828
Practical Exercises . . . 829
Exercise 1: Implement Gradient Descent for Linear Regression . . . 829
Exercise 2: Numerical Differentiation . . . 831
Summary . . . 832

Integrals and Accumulation **833**
1. Definite integral as accumulation . . . 833
Worked example 1 . . . 833
2. Riemann sums . . . 833
Worked example 2 . . . 833
3. Fundamental Theorem of Calculus . . . 833
Worked example 3 . . . 834
Worked example 4 — net change . . . 834
4. Antiderivatives and techniques (brief) . . . 834
Worked example 5 . . . 834
5. Improper integrals . . . 834
Worked example 6 . . . 834
Worked example 7 — probability . . . 834
6. Numerical integration . . . 834
Worked example 8 . . . 835
Worked example 9 — high dimension . . . 835
Worked example 10 — importance sampling . . . 835
7. Probability and expectation . . . 835
Worked example 11 . . . 835
8. Multiple integrals (preview) . . . 835
Worked example 12 . . . 835
9. CS applications . . . 836
Worked example 13 — average case analysis . . . 836
10. Pitfalls . . . 836
11. Checkpoint . . . 836
Exercises . . . 837
Easy . . . 837
Medium . . . 837
Challenge . . . 837
Summary . . . 837

Multivariable Calculus for ML **838**
1. Functions of several variables . . . 838
Worked example 1 . . . 838
2. Partial derivatives . . . 838
Worked example 2 . . . 838
3. Gradient . . . 838
Worked example 3 . . . 839
Worked example 4 — GD step . . . 839
4. Directional derivatives . . . 839
Worked example 5 . . . 839

5. Chain rule and backpropagation . . . 839
Worked example 6 — logistic . . . 839
Worked example 7 — Matmul layer . . . 839
6. Jacobian matrix . . . 839
Worked example 8 . . . 840
7. Hessian and curvature . . . 840
Worked example 9 . . . 840
Worked example 10 — quadratic bowl . . . 840
8. Multivariate Taylor . . . 840
Worked example 11 . . . 840
9. Constrained gradients (preview) . . . 840
Worked example 12 . . . 841
10. Automatic vs symbolic vs numeric derivatives . . . 841
Worked example 13 . . . 841
11. Pitfalls . . . 841
12. Checkpoint . . . 841
Exercises . . . 842
Easy . . . 842
Medium . . . 842
Challenge . . . 842
Summary . . . 842

Taylor Series and Approximations **843**
1. Taylor polynomials . . . 843
Worked example 1 . . . 843
Worked example 2 . . . 843
2. Classic Maclaurin series . . . 844
Worked example 3 . . . 844
Worked example 4 . . . 844
3. Convergence radius . . . 844
Worked example 5 . . . 844
4. Big-O and little-o expansions . . . 844
Worked example 6 . . . 844
Worked example 7 — softmax stability . . . 845
5. Taylor in optimization . . . 845
Worked example 8 . . . 845
Worked example 9 — GD step sizes . . . 845
6. Series in algorithms and discrete math . . . 845
Worked example 10 . . . 845
7. Numerical evaluation caveats . . . 845
Worked example 11 . . . 845
Worked example 12 — cancellation . . . 846
8. Fourier teaser (optional horizon) . . . 846
9. Pitfalls . . . 846
10. Checkpoint . . . 846
Exercises . . . 846
Easy . . . 846
Medium . . . 847

Challenge . . . 847
Summary . . . 847

Number Theory **848**

Number Theory for Computer Science **849**
Why number theory is core CS math . . . 849
Concept map . . . 849
Divisibility and gcd (preview) . . . 849
Worked example 1 . . . 850
Worked example 2 — Bézout . . . 850
Modular arithmetic (preview) . . . 850
Worked example 3 . . . 850
Worked example 4 — fast pow . . . 850
Fermat and Euler (preview) . . . 850
Worked example 5 . . . 850
Chinese Remainder Theorem (preview) . . . 850
Worked example 6 . . . 850
RSA sketch (motivation) . . . 851
Worked example 7 — tiny RSA . . . 851
Primes and primality . . . 851
Worked example 8 . . . 851
Hashing connection . . . 851
Worked example 9 . . . 851
Proof styles to practice . . . 851
Worked example 10 . . . 852
Pitfalls . . . 852
Roadmap of this part . . . 852
Checkpoint . . . 852
Exercises . . . 852
Summary . . . 853

Divisibility, GCD, and the Euclidean Algorithm **854**
1. Division Algorithm . . . 854
Worked Example . . . 854
2. Divisibility and Basic Laws . . . 854
3. Greatest Common Divisor . . . 854
Theorem (Euclidean Step) . . . 854
Proof Sketch . . . 854
4. Euclidean Algorithm . . . 855
Example . . . 855
Complexity . . . 855
5. Bézout Identity and Extended Euclid . . . 855
Theorem (Bézout) . . . 855
Why It Matters . . . 855
Extended Euclid Example . . . 855

6. Linear Diophantine Equation . . . 855
Theorem . . . 856
Example . . . 856
7. Python Reference Implementations . . . 856
Exercises . . . 856
Summary . . . 856

Modular Arithmetic and Congruences **857**
1. Congruence Relation . . . 857
2. Arithmetic Laws . . . 857
3. Modular Inverse . . . 857
Example . . . 857
4. Fast Exponentiation . . . 857
5. Fermat and Euler Theorems . . . 858
Fermat's Little Theorem . . . 858
Euler's Theorem . . . 858
6. Chinese Remainder Theorem (CRT) . . . 858
Theorem . . . 858
Worked Example . . . 858
7. Application Patterns in CS . . . 858
Exercises . . . 859
Summary . . . 859

Prime Numbers and Cryptography Basics **860**
1. Prime Fundamentals . . . 860
Theorem (Fundamental Theorem of Arithmetic) . . . 860
Proof Idea . . . 860
2. Prime Generation . . . 860
Sieve of Eratosthenes . . . 860
3. Primality Testing . . . 861
4. Euler Totient Function . . . 861
5. RSA Cryptosystem (Sketch) . . . 861
Key Generation . . . 861
Encryption/Decryption . . . 861
Why It Works . . . 861
6. Security Intuition . . . 861
7. Tiny RSA Example (Pedagogical) . . . 862
Exercises . . . 862
Extended practice: Chinese Remainder Theorem (CRT) . . . 862
Discrete log (why Diffie–Hellman is hard) . . . 862
Map to CS . . . 862
Summary . . . 863

Lattices and Learning With Errors: the Mathematics of ML-KEM **864**
1. Lattices in one paragraph . . . 864
2. The problem: linear algebra plus a little noise . . . 864
3. Regev encryption of one bit . . . 865
4. The noise budget . . . 867

5. From vectors to polynomials: the ring $\mathbb{Z}_q[x]/(x^n+1)$ . . . 868
6. What the hardness claim is, and is not . . . 869
Exercises . . . 869
Sources . . . 870

AlphaProof Nexus: Lean-Verified Erdős Problems **871**

AlphaProof Nexus: Lean-Verified Erdős Problems **872**
Learning goals . . . 872
Thesis . . . 872
Mental model — formal vs informal . . . 872
Agent shapes (from the paper) . . . 873
Worked examples . . . 873
Case 1: Headline evaluation . . . 873
Case 2: What an Erdős solve looks like (problem #125, density) . . . 874
Case 3: Construction flavour (problem #12(i), 1970) . . . 874
Case 4: Opening a Lean artifact . . . 874
The trap . . . 875
The boring rule . . . 875
Try this . . . 875
Sources . . . 875

Reed–Solomon Erasure Coding over GF(2^8): Why Object Stores Beat Replication **876**
1. The problem as linear algebra . . . 877
2. Bytes need a field, and $\mathbb{Z}/256$ is not one . . . 877
3. Which moduli work? . . . 879
4. Encoding is a matrix product . . . 880
5. Check every survivor set, not a few . . . 883
6. The trap: why identity-plus-Vandermonde breaks . . . 885
7. Replication versus Reed–Solomon, in numbers . . . 886
Exercises . . . 887
Sources . . . 887

The Chinese Remainder Theorem **889**

The Chinese Remainder Theorem **890**
Thesis . . . 890
Mental model . . . 890
Worked example . . . 890
Trap . . . 891
Boring rule . . . 891
Try this . . . 891
Sources . . . 891

Information Theory 892

Information Theory for Computer Science **893**
Chapters in this part . . . 893

What "information" means here . . . 893
The three core problems . . . 893
1. Source coding (compression) . . . 894
2. Channel coding (reliable communication) . . . 894
3. Statistical dependence . . . 894
Why CS and ML practitioners need this . . . 894
Worked intuition: cross-entropy as a loss . . . 895
Mathematical prerequisites . . . 895
Roadmap of this part . . . 895
A first calculation you should master . . . 895
Units: bits, nats, and dits . . . 896
Pitfalls (read before the next chapter) . . . 896
How to study these chapters . . . 896
Exercises (warm-up) . . . 896
Checkpoint . . . 897

Entropy and Information Content **898**
Information Content . . . 898
Self-Information . . . 898
Shannon Entropy . . . 899
Definition . . . 899
Properties of Entropy . . . 899
Entropy in Data Analysis . . . 900
Text Analysis . . . 900
Image Entropy . . . 901
Cross-Entropy and KL Divergence . . . 902
Cross-Entropy . . . 902
Kullback-Leibler (KL) Divergence . . . 902
Applications in Machine Learning . . . 903
Decision Trees and Information Gain . . . 903
Feature Selection using Mutual Information . . . 904
Entropy in Compression . . . 905
Huffman Coding Efficiency . . . 905
Practical Applications . . . 907
Password Strength Analysis . . . 907
Network Protocol Efficiency . . . 908
Summary . . . 909

Mutual Information and Channel Capacity **911**
1. Conditional entropy . . . 911
Worked example 1 . . . 911
Worked example 2 . . . 911
2. Mutual information: definitions . . . 912
Units . . . 912
3. Fundamental properties . . . 912
Worked example 3 — 2×2 joint . . . 913
Worked example 4 — independence implies zero MI . . . 913
Worked example 5 — deterministic function . . . 913

4. Conditional mutual information and chain rules . . . 913
5. Data processing inequality (DPI) . . . 914
Worked example 6 — ML pipeline . . . 914
6. Channels and capacity . . . 914
7. Binary symmetric channel (BSC) . . . 915
Worked example 7 . . . 915
Worked example 8 — Z-channel intuition (contrast) . . . 915
8. Fano's inequality (why MI controls prediction) . . . 915
9. CS and ML applications . . . 916
Feature selection . . . 916
Representation learning . . . 916
Privacy leakage . . . 916
Compression with side information . . . 916
Protocol and systems design . . . 916
Worked example 9 — information gain in trees . . . 916
Worked example 10 — capacity vs SNR mindset . . . 916
10. Continuous caveat (differential MI) . . . 917
11. Pitfalls . . . 917
12. Checkpoint . . . 917
Exercises . . . 917
Easy . . . 917
Medium . . . 918
Challenge . . . 918
Summary . . . 918

KL Divergence, Cross-Entropy, and Coding **919**
Diagram: two distributions . . . 919
1. Definitions (discrete) . . . 919
Fundamental properties . . . 920
Worked example 1 — binary . . . 920
Worked example 2 — reverse KL differs . . . 920
2. Coding story (why the formulas look like that) . . . 920
Kraft and prefix codes (connection) . . . 921
3. ML link: cross-entropy loss . . . 921
Multi-class classification . . . 921
Binary logistic regression . . . 921
Softmax + CE gradient fact . . . 921
4. KL as expected log-likelihood gap . . . 922
5. Forward vs reverse KL (mode behavior) . . . 922
6. Continuous case (caution) . . . 922
7. Related divergences (map) . . . 923
8. Worked example 3 — sure event . . . 923
Worked example 4 — CE with uniform model . . . 923
9. Numerical and practical pitfalls in ML . . . 923
10. CS / systems connections . . . 924
11. Checkpoint . . . 924
Exercises . . . 924
Easy . . . 924

Medium . . . 924
Challenge . . . 925
Checks . . . 925
Summary . . . 925

Information Theory Applications **926**
Diagram: map of applications . . . 926
1. Compression and source coding . . . 926
Practical codecs (conceptual) . . . 926
Worked example 1 . . . 927
2. Information gain and decision trees . . . 927
Worked example 2 — pure split . . . 927
Practical caveats . . . 927
3. Mutual information for features and dependence . . . 928
Worked example 3 — independence . . . 928
Correlation vs MI . . . 928
4. Channels and capacity (systems view) . . . 928
Worked example 4 — BSC intuition . . . 928
5. Perplexity and language models . . . 929
Worked example 5 . . . 929
6. ML training objectives (recap with applications) . . . 929
7. Decision systems and expected information . . . 929
8. Privacy and leakage (conceptual) . . . 930
9. Hashing, randomness, and min-entropy . . . 930
10. End-to-end worked case: email spam filter features . . . 930
11. Pitfalls . . . 930
12. Checkpoint . . . 931
Exercises . . . 931
Easy . . . 931
Medium . . . 931
Challenge . . . 932
Checks . . . 932
Summary . . . 932

Bloom Filters — False Positives by Design **933**

Bloom Filters — False Positives by Design **934**
Thesis . . . 934
Mental model . . . 934
Worked example . . . 934
Trap . . . 935
Boring rule . . . 935
Try this . . . 935
Sources . . . 935

Optimization **936**

Optimization Theory for Computer Science **937**
Chapters in this part . . . 937
The canonical problem . . . 937
Maximization . . . 937
A taxonomy that actually helps . . . 938
By decision variables . . . 938
By structure of f and constraints . . . 938
By information used . . . 938
Why convexity is the "dividing line" . . . 938
Optimization appears everywhere in CS . . . 939
Machine learning . . . 939
Algorithms and theory . . . 939
Systems . . . 939
Worked example 1 — ridge regression as optimization . . . 939
Worked example 2 — resource allocation . . . 939
Worked example 3 — nonconvex neural loss . . . 939
Roadmap of this part . . . 940
Prerequisites . . . 940
Modeling skill (underrated) . . . 940
Worked example 4 — modeling a deadline . . . 940
Complexity reality check . . . 941
Pitfalls . . . 941
Checkpoint . . . 941
Exercises . . . 941
Summary . . . 942

Convexity Foundations **943**
1. Convex sets . . . 943
Examples of convex sets . . . 943
Non-examples . . . 943
Worked example 1 . . . 943
Operations preserving convexity of sets . . . 944
2. Convex functions . . . 944
Worked example 2 . . . 944
Worked example 3 . . . 944
Worked example 4 . . . 944
Worked example 5 — logistic loss . . . 945
3. First-order characterization . . . 945
Subgradients (nondifferentiable case) . . . 945
4. Second-order characterization . . . 945
Worked example 6 . . . 945
Worked example 7 — least squares . . . 946
5. Global optimality theorems . . . 946
6. Strong convexity . . . 946
Consequences . . . 946
Worked example 8 . . . 946

7. Calculus of convex functions (closure rules) . . . 947
Worked example 9 — ReLU networks . . . 947
8. Smoothness (companion property) . . . 947
9. CS/ML connections . . . 947
Worked example 10 — hinge loss SVM . . . 948
10. Pitfalls . . . 948
11. Checkpoint . . . 948
Exercises . . . 948
Easy . . . 948
Medium . . . 949
Challenge . . . 949
Summary . . . 949

Constrained Optimization and KKT Conditions **950**
1. Problem form . . . 950
Worked example 1 — projection onto a ball . . . 950
2. Equality constraints and Lagrange multipliers . . . 950
Unconstrained case recap . . . 950
One equality . . . 951
Worked example 2 — classic textbook problem . . . 951
Worked example 3 — entropy maximization . . . 951
3. Inequality constraints and KKT . . . 951
Geometric reading . . . 952
Worked example 4 — inequality active vs inactive . . . 952
4. Constraint qualifications (why they matter) . . . 952
5. Duality . . . 952
Worked example 5 — SVM dual intuition . . . 953
Worked example 6 — equality-constrained QP . . . 953
6. Methods that enforce constraints . . . 953
Worked example 7 — projection onto simplex . . . 953
Worked example 8 — projected gradient on a box . . . 954
7. Convex constrained problems . . . 954
Worked example 9 — water-filling sketch . . . 954
8. CS/ML applications . . . 954
9. Pitfalls . . . 954
10. Checkpoint . . . 955
Exercises . . . 955
Easy . . . 955
Medium . . . 955
Challenge . . . 955
Summary . . . 956

Regularization and Generalization **957**
1. The generalization problem . . . 957
Worked example 1 — polynomial overfitting . . . 957
2. Regularized empirical risk . . . 958
Common regularizers . . . 958

3. Ridge regression in closed form . . . 958
Worked example 2 — collinearity . . . 958
Spectral view . . . 959
4. Lasso and sparsity geometry . . . 959
Worked example 3 . . . 959
Worked example 4 — when lasso fails uniqueness . . . 959
5. Bias–variance tradeoff . . . 959
Worked example 5 — ridge path . . . 959
6. Early stopping as implicit regularization . . . 960
Worked example 6 . . . 960
7. Implicit bias of optimizers . . . 960
8. Structural risk and capacity control . . . 960
Worked example 7 — weight decay in deep nets . . . 960
9. Double descent (modern nuance) . . . 961
Worked example 8 — conceptual sketch . . . 961
10. Practical workflow . . . 961
Worked example 9 — validation curve reading . . . 961
Worked example 10 — calibration note . . . 961
11. CS/systems connections . . . 961
12. Pitfalls . . . 962
13. Checkpoint . . . 962
Exercises . . . 962
Easy . . . 962
Medium . . . 962
Challenge . . . 963
Summary . . . 963

First-Order Methods Playbook **964**
Diagram: algorithm family . . . 964
1. Problem template . . . 964
2. Gradient descent (GD) . . . 964
Step size rules . . . 964
Worked example 1 — 1D quadratic . . . 965
Complexity intuition (smooth convex) . . . 965
3. Stochastic gradient descent (SGD) . . . 965
Worked example 2 — empirical risk . . . 965
Schedules . . . 966
4. Momentum and acceleration . . . 966
Heavy ball / classical momentum . . . 966
Nesterov acceleration (convex smooth) . . . 966
Worked example 3 — valley . . . 966
5. Adaptive methods (Adam sketch) . . . 966
6. Constraints and nonsmooth regularizers . . . 967
Projected gradient . . . 967
Proximal gradient (composite objectives) . . . 967
Mirror descent / natural gradients (pointer) . . . 967
7. Gradient clipping and stability (ML) . . . 967

8. Diagnostics checklist . . . 968
Worked example 4 — learning rate range test . . . 968
9. Complexity snapshot (order-of-magnitude) . . . 968
10. When to go second-order . . . 968
11. Practical training recipe (supervised ML) . . . 969
Worked example 5 — mini-batch scaling rule of thumb . . . 969
12. Pitfalls . . . 969
13. Checkpoint . . . 969
Exercises . . . 970
Easy . . . 970
Medium . . . 970
Challenge . . . 970
Checks . . . 971
Summary . . . 971

Gradient-Based Optimization Methods **972**
Gradient Descent Fundamentals . . . 972
Basic Gradient Descent . . . 972
Learning Rate Analysis . . . 974
Advanced Gradient Methods . . . 975
Momentum . . . 975
Adaptive Learning Rates . . . 977
Stochastic Gradient Descent . . . 980
Mini-batch SGD . . . 980
Second-Order Methods . . . 983
Newton's Method . . . 983
Quasi-Newton Methods (BFGS) . . . 985
Applications in Machine Learning . . . 987
Logistic Regression . . . 987
Summary . . . 990

Machine Learning Math **991**

Mathematics for Machine Learning **992**
Chapters in this part . . . 992
What an ML problem looks like mathematically . . . 992
Worked example 1 — linear regression in one line . . . 993
The four pillars . . . 993
1. Linear algebra . . . 993
2. Calculus and optimization . . . 993
3. Probability and statistics . . . 993
4. Information theory (cross-cutting) . . . 993
Supervised learning: decision theory sketch . . . 994
Worked example 2 — logistic regression as MLE . . . 994
Worked example 3 — softmax multiclass . . . 994
Unsupervised and self-supervised (mathematical themes) . . . 994
Worked example 4 — PCA objective . . . 994

Generalization: the mathematical tension . . . 994
Worked example 5 — train/val split as Monte Carlo . . . 995
Roadmap in this book . . . 995
Notation hygiene . . . 995
Worked example 6 — shape checking . . . 995
CS systems angle . . . 995
Worked example 7 — complexity . . . 996
Pitfalls . . . 996
Checkpoint . . . 996
Exercises . . . 996
Summary . . . 997

Linear Algebra for Machine Learning **998**
Vectors in Machine Learning . . . 998
Vector Representation of Data . . . 998
Vector Operations in ML . . . 998
Matrices in Machine Learning . . . 1000
Data Representation . . . 1000
Matrix Operations in Linear Regression . . . 1000
Matrix Factorization . . . 1001
Eigenvalues and Eigenvectors . . . 1003
Understanding Eigendecomposition . . . 1003
Spectral Clustering . . . 1004
Matrix Calculus for Neural Networks . . . 1005
Gradient Computation . . . 1005
Singular Value Decomposition (SVD) . . . 1007
Recommender Systems . . . 1007
Practical Applications . . . 1009
Image Compression using SVD . . . 1009
Summary . . . 1010

Gradients, Loss Functions, and Backprop Sketch **1011**
Diagram: risk minimization . . . 1011
1. Loss menu . . . 1011
Worked example 1 — CE numerical . . . 1011
2. Empirical risk and regularization . . . 1012
Population vs empirical . . . 1012
3. Gradients and directional derivatives . . . 1012
Gradient descent . . . 1012
Worked example 2 — pure GD arithmetic . . . 1013
4. Chain rule and backpropagation . . . 1013
Worked example 3 — scalar chain . . . 1013
5. Worked example — logistic regression end-to-end . . . 1013
6. Softmax + cross-entropy . . . 1014
7. Convex vs non-convex landscapes . . . 1014
8. Subgradients (nonsmooth losses) . . . 1014
9. Second-order glance . . . 1014
10. Numerical gradient checks . . . 1015

11. Mini-batch tradeoffs . . . 1015
12. Pitfalls . . . 1015
13. Checkpoint . . . 1015
Exercises . . . 1016
Easy . . . 1016
Medium . . . 1016
Challenge . . . 1016
Checks . . . 1016
Summary . . . 1017

Probability for Machine Learning **1018**
Diagram: generative vs discriminative . . . 1018
1. Likelihood and maximum likelihood . . . 1018
Worked example 1 — Bernoulli / logistic . . . 1018
Worked example 2 — Gaussian regression . . . 1019
2. MAP estimation and regularization . . . 1019
Worked example 3 — Gaussian prior strength . . . 1019
3. Bayes rule in prediction . . . 1019
Worked example 4 — equal priors . . . 1019
4. Softmax as multi-class Bernoulli generalization . . . 1020
Worked example 5 . . . 1020
5. Calibration . . . 1020
Worked example 6 — overconfidence . . . 1020
6. Bias–variance (probabilistic view) . . . 1021
7. Decision rules and utility . . . 1021
8. Common predictive distributions in ML . . . 1021
9. Latent variables (preview) . . . 1022
10. Train / validation / test as risk estimation . . . 1022
11. Pitfalls . . . 1022
12. Checkpoint . . . 1022
Exercises . . . 1023
Easy . . . 1023
Medium . . . 1023
Challenge . . . 1023
Checks . . . 1023
Summary . . . 1024

Neural Networks: A Mathematical Sketch **1025**
Diagram: one hidden layer . . . 1025
1. Building blocks . . . 1025
Affine layer . . . 1025
Activations σ (elementwise unless noted) . . . 1025
Residual connections . . . 1026
Universal approximation (intuition) . . . 1026
2. Parameter count and FLOPs . . . 1026
Worked example 1 — MLP $10 \to 32 \to 32 \to 1$ . . . 1026
3. Composition and feature hierarchy . . . 1026

4. Jacobian and local linearization . . . 1027
Layer Jacobian sketch (elementwise σ) . . . 1027
5. Vanishing and exploding gradients . . . 1027
6. Normalization layers (math role) . . . 1027
7. Loss heads . . . 1027
8. Overfitting and capacity . . . 1028
9. Convolution and parameter sharing (sketch) . . . 1028
10. Attention as similarity (sketch) . . . 1028
11. Initialization (order-of-magnitude) . . . 1029
12. What math to study next . . . 1029
13. Pitfalls . . . 1029
14. Checkpoint . . . 1029
Exercises . . . 1030
Easy . . . 1030
Medium . . . 1030
Challenge . . . 1030
Checks . . . 1031
Summary . . . 1031

Matrix Calculus for ML (Practical Identities) 1032
Diagram: shapes . . . 1032
1. Layout discipline . . . 1032
2. Differentials (often easier than coordinates) . . . 1032
Worked example 1 . . . 1032
3. Core identities (column vectors) . . . 1033
Worked example 2 — ridge . . . 1033
Worked example 3 — shapes . . . 1033
4. Least squares and normal equations . . . 1033
5. Linear layer + scalar loss . . . 1033
Worked example 4 — bias . . . 1034
6. Softmax + cross-entropy (fact) . . . 1034
Worked example 5 . . . 1034
7. Logistic regression (vector form) . . . 1034
8. Chain rule for products and Frobenius . . . 1034
9. Elementwise nonlinearities . . . 1035
10. Finite-difference checks . . . 1035
Worked example 6 — protocol . . . 1035
11. Hessian-vector products (awareness) . . . 1035
Worked example 7 . . . 1035
12. Common ML gradients cheat-sheet . . . 1036
13. Pitfalls . . . 1036
14. Checkpoint . . . 1036
Exercises . . . 1037
Easy . . . 1037
Medium . . . 1037
Challenge . . . 1037
Checks . . . 1037
Summary . . . 1038

Automatic Differentiation from First Principles: Dual Numbers, the Tape, and Adjoint Greeks **1039**

Automatic Differentiation from First Principles **1040**
1. Mental model: one graph, two directions . . . 1040
2. Forward mode is arithmetic in $\mathbb{R}[\varepsilon]/(\varepsilon^2)$ . . . 1041
3. Reverse mode: record a tape, sweep it backwards . . . 1044
4. The cost asymmetry, measured . . . 1047
5. Adjoint Greeks: backprop for a price . . . 1049
6. The trap: AD differentiates the program, not the mathematics . . . 1051
7. Try this . . . 1053
Checkpoint . . . 1054
Summary . . . 1054
Sources . . . 1054

Algorithms and Complexity **1056**

Algorithms and Computational Complexity **1057**
Why complexity is practical . . . 1057
Two complementary activities . . . 1057
1. Algorithm analysis . . . 1057
2. Problem complexity . . . 1058
Worked example 1 . . . 1058
Worked example 2 . . . 1058
Models and cost . . . 1058
Worked example 3 . . . 1058
Roadmap of this part . . . 1059
Correctness before speed . . . 1059
Worked example 4 — invariant . . . 1059
Average, worst, high probability . . . 1059
Worked example 5 . . . 1059
Reductions as a reusable idea . . . 1059
Worked example 6 . . . 1060
Approximation and heuristics . . . 1060
Worked example 7 . . . 1060
Pitfalls . . . 1060
Checkpoint . . . 1060
Exercises . . . 1061
Summary . . . 1061

Asymptotic Analysis and Big O Notation **1062**
Big O Notation . . . 1062
Definition . . . 1062
Common Complexity Classes . . . 1063
Other Asymptotic Notations . . . 1063
Big Omega (Ω) - Lower Bound . . . 1063
Big Theta (Θ) - Tight Bound . . . 1063

Analyzing Algorithm Complexity . . . 1064
Example 1: Linear Search . . . 1064
Example 2: Nested Loops . . . 1065
Recurrence Relations . . . 1066
Master Theorem . . . 1066
Solving Recurrences by Substitution . . . 1068
Space Complexity Analysis . . . 1069
Memory Usage Patterns . . . 1069
Amortized Analysis . . . 1070
Dynamic Array (Vector) Analysis . . . 1070
Practical Algorithm Analysis . . . 1072
Sorting Algorithm Comparison . . . 1072
Cache Performance Analysis . . . 1074
Summary . . . 1075
Proof Checklist for Asymptotic Claims . . . 1075

Complexity Classes and Reductions **1077**
1. Decision problems and languages . . . 1077
Worked example 1 . . . 1077
2. The class **P** . . . 1077
Examples in **P** . . . 1078
Worked example 2 . . . 1078
3. The class **NP** . . . 1078
Worked example 3 — SAT certificates . . . 1078
Worked example 4 — what is **not** obviously a short certificate? . . . 1078
Worked example 5 — composite numbers . . . 1078
4. **coNP** and containments . . . 1079
Worked example 6 . . . 1079
5. Polynomial-time reductions . . . 1079
Properties . . . 1079
Worked example 7 — direction discipline . . . 1079
6. NP-hardness and NP-completeness . . . 1080
Worked example 8 — independent set $\leftrightarrow$ clique . . . 1080
Worked example 9 — vertex cover . . . 1080
Worked example 10 — 3-SAT sketch . . . 1080
7. Proof hygiene checklist . . . 1080
8. Beyond NP (map) . . . 1081
9. CS practice connections . . . 1081
Worked example 11 — product decision . . . 1081
10. Pitfalls . . . 1081
11. Checkpoint . . . 1081
Exercises . . . 1082
Easy . . . 1082
Medium . . . 1082
Challenge . . . 1082
Summary . . . 1082

Randomized and Approximation Algorithms **1083**
Part A — Randomized algorithms . . . 1083
1. What randomness buys . . . 1083
Worked example 1 — randomized quicksort . . . 1083
2. Error modes and amplification . . . 1083
Worked example 2 . . . 1084
Chernoff bound (useful form) . . . 1084
3. Fingerprinting and polynomial identity . . . 1084
Worked example 3 — communication-free checking . . . 1084
4. Hashing and dictionaries . . . 1084
Worked example 4 . . . 1084
5. Markov, Chebyshev, and median-of-means . . . 1085
Worked example 5 — Las Vegas to high probability . . . 1085
Part B — Approximation algorithms . . . 1085
6. Optimization and approximation ratio . . . 1085
Worked example 6 — load balancing . . . 1085
7. Vertex cover 2-approximation . . . 1085
Worked example 7 . . . 1086
8. Set cover greedy . . . 1086
Worked example 8 . . . 1086
9. Greedy knapsack and FPTAS (sketch) . . . 1086
Worked example 9 . . . 1086
10. Hardness of approximation (awareness) . . . 1086
11. Engineering tradeoff table . . . 1087
Worked example 10 — production choice . . . 1087
12. Pitfalls . . . 1087
13. Checkpoint . . . 1087
Exercises . . . 1087
Easy . . . 1087
Medium . . . 1088
Challenge . . . 1088
Summary . . . 1088

How Fast Can We Multiply Integers? Karatsuba, Toom–Cook, FFTs, n log n, and the 2026 Claim Below It **1089**
1. Mental model: digits are polynomial coefficients . . . 1089
2. Karatsuba: three half-size products instead of four . . . 1090
3. Transforms: evaluate everywhere at once . . . 1094
4. From $\log \log n$ to $n \log n$ . . . 1096
5. What your machine actually runs . . . 1097
6. The 2026 claim: below $n \log n$, but by how much? . . . 1099
7. The trap: floating-point FFTs return wrong products silently . . . 1100
8. Try this . . . 1102
Checkpoint . . . 1102
Summary . . . 1102
Sources . . . 1103

Karatsuba Multiplication **1104**

Karatsuba Multiplication **1105**
Thesis . . . 1105
Mental model . . . 1105
Worked example . . . 1105
Trap . . . 1106
Boring rule . . . 1106
Try this . . . 1106
Sources . . . 1106

Graph Theory **1107**

Graph Theory for Computer Science **1108**
What is a graph? . . . 1108
Worked example 1 — many skins, one math . . . 1108
Core computational problems . . . 1108
Proof techniques you will reuse . . . 1109
Worked example 2 — handshaking . . . 1109
Complexity baseline . . . 1109
Worked example 3 — sparse vs dense . . . 1109
Correctness mindset . . . 1110
Worked example 4 — BFS invariant . . . 1110
Roadmap . . . 1110
Pitfalls . . . 1110
Checkpoint . . . 1110
Exercises . . . 1111
Summary . . . 1111

Graph Basics and Representations **1112**
1. Formal definitions . . . 1112
Neighborhoods and degrees . . . 1112
Paths and cycles . . . 1112
Worked example 1 . . . 1112
2. Handshaking lemma . . . 1113
Worked example 2 . . . 1113
Worked example 3 — directed version . . . 1113
3. Special families . . . 1113
Worked example 4 . . . 1113
4. Adjacency matrix . . . 1114
Worked example 5 . . . 1114
5. Adjacency lists . . . 1114
Worked example 6 — build from edge list . . . 1114
6. Edge lists and CSR . . . 1115
Worked example 7 . . . 1115
7. Graph isomorphism (awareness) . . . 1115
8. Traversal foundations: BFS and DFS . . . 1115
BFS (queue) . . . 1115
DFS (stack / recursion) . . . 1115

Worked example 8 — BFS layers . . . 1115
Worked example 9 — DFS edge types (undirected) . . . 1115
9. Connectivity algorithms (basics) . . . 1116
Worked example 10 — offline Union-Find . . . 1116
10. CS representation choices . . . 1116
11. Pitfalls . . . 1116
12. Checkpoint . . . 1116
Exercises . . . 1117
Easy . . . 1117
Medium . . . 1117
Challenge . . . 1117
Summary . . . 1117

Trees and Spanning Trees **1118**
1. Characterizations of trees . . . 1118
Worked example 1 . . . 1118
Worked example 2 . . . 1118
2. Leaves and counting . . . 1118
Worked example 3 . . . 1119
3. Rooted trees in CS . . . 1119
Worked example 4 — heap property vs tree shape . . . 1119
Worked example 5 — LCA . . . 1119
4. Spanning trees . . . 1119
Worked example 6 . . . 1119
5. Minimum spanning trees . . . 1119
Cut property . . . 1120
Cycle property . . . 1120
Worked example 7 — uniqueness . . . 1120
6. Kruskal's algorithm . . . 1120
Worked example 8 . . . 1120
7. Prim's algorithm . . . 1120
Worked example 9 — Prim on same graph . . . 1121
8. Kruskal vs Prim . . . 1121
9. Applications . . . 1121
Worked example 10 — single-linkage . . . 1121
10. Directed and other variants . . . 1121
11. Pitfalls . . . 1122
Worked example 11 — SPT $\neq$ MST . . . 1122
12. Checkpoint . . . 1122
Exercises . . . 1122
Easy . . . 1122
Medium . . . 1122
Challenge . . . 1123
Summary . . . 1123

Shortest Paths **1124**
1. Problem variants . . . 1124
Worked example 1 — hop vs weight . . . 1124

2. Unweighted graphs: BFS . . . 1124
Worked example 2 . . . 1124
3. Nonnegative weights: Dijkstra . . . 1125
Worked example 3 . . . 1125
Worked example 4 — failure with negatives . . . 1125
4. Negative weights: Bellman–Ford . . . 1125
Worked example 5 — negative cycle detect . . . 1125
Worked example 6 — currency arbitrage . . . 1126
5. All-pairs: Floyd–Warshall . . . 1126
Worked example 7 . . . 1126
6. Johnson's algorithm (sketch) . . . 1126
7. Algorithm selection . . . 1126
Worked example 8 — maps . . . 1126
8. Path reconstruction . . . 1127
Worked example 9 . . . 1127
9. A* search (informed SSSP) . . . 1127
Worked example 10 . . . 1127
10. Pitfalls . . . 1127
11. Checkpoint . . . 1127
Exercises . . . 1128
Easy . . . 1128
Medium . . . 1128
Challenge . . . 1128
Summary . . . 1128

Connectivity, Components, and Network Flow **1129**
1. Undirected connectivity . . . 1129
Worked example 1 . . . 1129
Worked example 2 — reliability . . . 1129
2. Edge- and vertex-connectivity . . . 1129
3. Directed strong connectivity . . . 1130
Kosaraju's algorithm . . . 1130
Tarjan's algorithm . . . 1130
Worked example 3 . . . 1130
Worked example 4 — 2-SAT . . . 1130
4. Topological sort on DAGs . . . 1130
Kahn's algorithm . . . 1130
DFS finishing times . . . 1130
Worked example 5 — build systems . . . 1131
Worked example 6 — course prerequisites . . . 1131
5. Flow networks . . . 1131
6. Residual graphs and augmenting paths . . . 1131
Edmonds–Karp . . . 1131
Worked example 7 — tiny max flow . . . 1131
7. Max-flow min-cut theorem . . . 1132
Worked example 8 . . . 1132
8. Bipartite matching via flow . . . 1132
Worked example 9 — assignment . . . 1132

Worked example 10 — König's theorem . . . 1132
9. Other flow applications . . . 1132
10. Dinic and beyond (awareness) . . . 1132
11. Pitfalls . . . 1133
12. Checkpoint . . . 1133
Exercises . . . 1133
Easy . . . 1133
Medium . . . 1133
Challenge . . . 1134
Summary . . . 1134

Graph Algorithms in Real Systems **1135**
1. Ranking and the web graph: PageRank . . . 1135
Worked example 1 . . . 1135
Worked example 2 — systems issues . . . 1135
2. Build systems and package managers . . . 1136
Worked example 3 — Bazel / make . . . 1136
Worked example 4 — npm/cargo . . . 1136
3. Compiler graphs . . . 1136
Worked example 5 — dominance . . . 1136
Worked example 6 — dead code . . . 1137
4. Databases and query plans . . . 1137
Worked example 7 . . . 1137
5. Networking and routing . . . 1137
Worked example 8 . . . 1137
Worked example 9 — SDN . . . 1137
6. Distributed systems and consensus adjacency . . . 1138
Worked example 10 — blast radius . . . 1138
7. ML and data . . . 1138
Worked example 11 — label propagation . . . 1138
8. Production concerns checklist . . . 1138
Worked example 12 — partition quality . . . 1139
9. Choosing algorithms under constraints . . . 1139
Worked example 13 — bipartite matching at scale . . . 1139
10. Capstone synthesis patterns . . . 1139
11. Pitfalls . . . 1139
12. Checkpoint . . . 1140
Exercises . . . 1140
Easy . . . 1140
Medium . . . 1140
Challenge . . . 1140
Summary . . . 1141

Graph Theory Workshop: Exercises and Diagrams **1142**
Diagram bank . . . 1142
Undirected simple graph . . . 1142
Directed . . . 1142
Tree vs cyclic . . . 1142

1. Degree and handshaking . . . 1142
2. Paths and connectivity . . . 1143
3. BFS layers . . . 1143
4. Spanning trees . . . 1143
5. Shortest paths . . . 1143
6. Topological order . . . 1143
7. Flow cut (conceptual) . . . 1143
8. CS mapping . . . 1144
Solutions sketch (spoiler zone) . . . 1144
More practice . . . 1144

Numerical Methods **1145**

Numerical Methods for Computer Science **1146**
The central tension . . . 1146
Core themes . . . 1146
Conditioning vs stability (preview) . . . 1147
Worked example 1 . . . 1147
Worked example 2 . . . 1147
Accuracy metrics . . . 1147
Worked example 3 . . . 1147
Complexity and structure . . . 1147
Worked example 4 . . . 1148
Roadmap . . . 1148
Tools (conceptual) . . . 1148
Worked example 5 . . . 1148
A first error budget . . . 1148
Worked example 6 — cancellation vs algebra . . . 1148
Worked example 7 — residual vs truth . . . 1149
What "convergent algorithm" means . . . 1149
Worked example 8 — Newton for roots . . . 1149
Roadmap recap with deliverables . . . 1149
Pitfalls . . . 1149
Checkpoint . . . 1150
Exercises . . . 1150
Summary . . . 1150

Floating-Point Arithmetic, Error, and Stability **1151**
1. IEEE-754 style machine numbers . . . 1151
Worked example 1 . . . 1151
Worked example 2 — spacing . . . 1151
2. Rounding model . . . 1151
3. Error types . . . 1152
Worked example 3 . . . 1152
4. Catastrophic cancellation . . . 1152
Worked example 4 — classic unstable formula . . . 1152
Worked example 5 — variance . . . 1152

Worked example 6 — `log1p` / `expm1` . . . 1152
5. Conditioning of problems . . . 1153
Worked example 7 . . . 1153
Worked example 8 — linear systems . . . 1153
6. Stability of algorithms . . . 1153
Worked example 9 — solving triangular systems . . . 1153
Worked example 10 — Horner vs naive polynomial . . . 1153
7. Accumulation and summation . . . 1153
Worked example 11 . . . 1153
8. Exceptions and special values . . . 1154
9. Practical guidelines . . . 1154
Worked example 12 . . . 1154
10. Pitfalls . . . 1154
11. Checkpoint . . . 1155
Exercises . . . 1155
Easy . . . 1155
Medium . . . 1155
Challenge . . . 1155
Summary . . . 1156

Root Finding: Bisection, Newton, and Secant **1157**
1. Problem statement . . . 1157
Worked example 1 . . . 1157
2. Bisection method . . . 1157
Worked example 2 . . . 1157
3. Newton's method . . . 1158
Local quadratic convergence . . . 1158
Worked example 3 . . . 1158
Worked example 4 — failure modes . . . 1158
Worked example 5 — square root . . . 1158
4. Secant method . . . 1159
Worked example 6 . . . 1159
5. Hybrid strategies (practical) . . . 1159
Worked example 7 — finance implied vol . . . 1159
6. Systems of nonlinear equations . . . 1159
Worked example 8 . . . 1160
7. Connections to optimization . . . 1160
Worked example 9 . . . 1160
8. Stopping criteria . . . 1160
Worked example 10 . . . 1160
9. Pitfalls . . . 1160
10. Checkpoint . . . 1161
Exercises . . . 1161
Easy . . . 1161
Medium . . . 1161
Challenge . . . 1161
Summary . . . 1162

Numerical Linear Systems **1163**
1. The problem . . . 1163
Worked example 1 . . . 1163
2. Direct methods: Gaussian elimination and LU . . . 1163
Worked example 2 — why pivot . . . 1163
Cholesky . . . 1163
Worked example 3 . . . 1164
3. Complexity and structure . . . 1164
Worked example 4 . . . 1164
4. Residual, forward error, condition number . . . 1164
Worked example 5 — Hilbert . . . 1164
Worked example 6 — scaling . . . 1164
5. Never form the inverse . . . 1165
Worked example 7 . . . 1165
6. Iterative methods overview . . . 1165
Jacobi . . . 1165
Gauss–Seidel . . . 1165
Conjugate gradient (CG) . . . 1165
Worked example 8 . . . 1165
Worked example 9 — GMRES . . . 1166
7. Stopping criteria for iterative solvers . . . 1166
Worked example 10 . . . 1166
8. Least squares connection . . . 1166
Worked example 11 . . . 1166
9. CS/ML applications . . . 1166
10. Pitfalls . . . 1167
11. Checkpoint . . . 1167
Exercises . . . 1167
Easy . . . 1167
Medium . . . 1167
Challenge . . . 1168
Summary . . . 1168

Numerical Optimization in Practice **1169**
1. Problem setup . . . 1169
2. Gradient descent and step sizes . . . 1169
Worked example 1 . . . 1169
Worked example 2 — ML . . . 1169
3. Momentum and adaptive methods (numeric view) . . . 1170
Worked example 3 . . . 1170
4. Newton and quasi-Newton . . . 1170
Worked example 4 . . . 1170
Worked example 5 — Hessian indefinite . . . 1170
5. Trust regions vs line search . . . 1170
Worked example 6 — Levenberg–Marquardt . . . 1171
6. Stochastic objectives . . . 1171
Worked example 7 . . . 1171

7. Constraints in numerical practice . . . 1171
Worked example 8 . . . 1171
Worked example 9 — SVM dual . . . 1171
8. Diagnostics and failure modes . . . 1172
Worked example 10 . . . 1172
9. Worked template: ℓ_2-regularized logistic . . . 1172
Worked example 11 . . . 1172
10. Automatic differentiation note . . . 1173
Worked example 12 . . . 1173
11. Pitfalls . . . 1173
12. Checkpoint . . . 1173
Exercises . . . 1173
Easy . . . 1173
Medium . . . 1174
Challenge . . . 1174
Summary . . . 1174

Discrete Fourier Transforms and the FFT **1175**
1. The DFT . . . 1175
Worked example 1 — DC and Nyquist . . . 1175
Worked example 2 — pure tone . . . 1175
2. Matrix view and complexity . . . 1176
Worked example 3 — when the constant matters . . . 1176
3. Cooley–Tukey radix-2 intuition . . . 1176
Worked example 4 — $n = 8$ . . . 1177
4. Convolution theorem . . . 1177
Worked example 5 — polynomial multiply . . . 1177
5. Real vs complex; normalization . . . 1178
Worked example 6 — Hermitian check . . . 1178
6. Windowing, leakage, and sampling traps . . . 1178
Worked example 7 — leakage demo (conceptual) . . . 1179
7. Frequencies and units . . . 1179
Worked example 8 — reading a spectrum . . . 1179
8. Runnable mini-FFT (radix-2, educational) . . . 1179
9. Stability and floating-point notes . . . 1180
Worked example 9 — residual check . . . 1181
10. Algorithms beyond radix-2 (map) . . . 1181
11. CS applications (numerical methods view) . . . 1181
12. Pitfalls . . . 1181
13. Checkpoint . . . 1182
Exercises . . . 1182
Easy . . . 1182
Medium . . . 1182
Challenge . . . 1183
Summary . . . 1183
Sources (for further reading) . . . 1183

Numerical Methods for Ordinary Differential Equations **1184**
1. The initial value problem . . . 1184
Higher order becomes first order . . . 1184
Worked example 1 — a pendulum as a system . . . 1185
2. Forward Euler and the meaning of "order" . . . 1185
Worked example 2 — Euler by hand . . . 1185
3. Runge–Kutta methods . . . 1186
Butcher tableaux . . . 1186
Worked example 3 — one Heun step . . . 1186
4. Measure the order — do not trust the label . . . 1186
Worked example 4 — cost per digit . . . 1188
5. Absolute stability and stiffness . . . 1188
The test equation . . . 1188
Stiffness . . . 1189
Worked example 5 — Euler explodes, backward Euler shrugs . . . 1189
6. Implicit methods in practice . . . 1190
7. Adaptive step-size control . . . 1191
Embedded pairs . . . 1191
Worked example 6 — choosing tolerances . . . 1193
8. Production solvers: `solve_ivp` . . . 1193
Worked example 7 — the Van der Pol stiffness benchmark . . . 1194
9. Long-time integration and geometric methods . . . 1195
Worked example 8 — pick by the question . . . 1197
10. Multistep methods (map) . . . 1197
11. Pitfalls . . . 1197
12. Checkpoint . . . 1198
Exercises . . . 1198
Easy . . . 1198
Medium . . . 1198
Challenge . . . 1199
Summary . . . 1199
Sources (for further reading) . . . 1199

Numerical Integration (Quadrature) **1200**
1. Rules, exactness, and order . . . 1200
Worked example 1 — measure the order . . . 1201
2. The trapezoid rule on periodic functions . . . 1202
Worked example 2 — the same rule, two integrands . . . 1202
3. Richardson extrapolation and Romberg integration . . . 1203
Worked example 3 — Romberg for π . . . 1203
4. Gaussian quadrature . . . 1204
Worked example 4 — build the rule, check exactness, race Simpson . . . 1204
5. Adaptive quadrature . . . 1206
Worked example 5 — one sharp peak . . . 1206
6. Singularities and infinite ranges . . . 1207
Worked example 6 — order collapse and the cure . . . 1207
7. Production quadrature: `scipy.integrate.quad` . . . 1208
Worked example 7 — four calls, one trap . . . 1209

8. Many dimensions: tensor grids, Monte Carlo, quasi-Monte Carlo . . . 1210
Worked example 8 — $d = 6$ . . . 1210
9. Choosing a method . . . 1211
10. Pitfalls . . . 1212
11. Checkpoint . . . 1212
Exercises . . . 1213
Easy . . . 1213
Medium . . . 1213
Challenge . . . 1213
Summary . . . 1213
Sources (for further reading) . . . 1214

Interval Arithmetic and Certified Computation 1215
1. The containment principle . . . 1215
2. Enclosures you can trust, and the dependency problem . . . 1217
3. Subdivision: buying tightness with work . . . 1218
4. Interval Newton: proving that a root exists . . . 1219
5. Rump's example: two confident wrong answers . . . 1220
6. Where certified computation is used . . . 1221
7. Pitfalls . . . 1221
8. Checkpoint . . . 1222
Exercises . . . 1222
Summary . . . 1222
Sources (for further reading) . . . 1222

The Ozaki Scheme: Emulating FP64 GEMM with Integer Slices 1224
1. Why emulate binary64 matrix products . . . 1224
2. Mental model: error-free splitting, then exact products . . . 1225
3. Binary64 layout, and what "exact integer GEMM" means . . . 1225
4. Shared-place splitting . . . 1227
5. Worked example: a 2×2 product, by hand and by slices . . . 1230
6. Worked example: 8×8 random GEMM against mpmath . . . 1232
7. The trap: a wide exponent range inside one row . . . 1234
8. The boring rule . . . 1236
9. Ozaki scheme II, in miniature: residues and the CRT . . . 1239
10. What this does not emulate . . . 1242
11. Pitfalls . . . 1242
12. Checkpoint . . . 1242
Exercises . . . 1242
Summary . . . 1243
Sources (for further reading) . . . 1243

Why Gaussian Elimination with Partial Pivoting Works: The Growth Factor, Wilkinson's 2^(n−1), and the 2026 Proof 1245

Why Gaussian Elimination with Partial Pivoting Works 1246
1. Mental model: errors are bounded by how big the numbers get . . . 1246
2. The tiny pivot: why pivot at all . . . 1247

3. Wilkinson's matrix: the worst case is real . . . 1249
4. Random matrices: about $0.5\sqrt{n}$, not 2^{n-1} . . . 1250
5. The 2026 theorem . . . 1253
6. The trap: "it is proved, so GEPP is always stable" . . . 1254
7. Try this . . . 1254
Checkpoint . . . 1255
Summary . . . 1255
Sources . . . 1255

Data Science Math **1256**

Mathematics for Data Science **1257**
Chapters in this part . . . 1257
The data science loop (mathematical view) . . . 1257
Core mathematical areas . . . 1257
Statistics and probability . . . 1257
Linear algebra for tabular / embedding data . . . 1258
Optimization and calibration . . . 1258
Causality and design (often neglected) . . . 1258
Worked example 1 — metric vs estimator . . . 1258
Worked example 2 — classification pipeline . . . 1258
Descriptive → inferential bridge . . . 1258
Worked example 3 . . . 1258
Roadmap . . . 1259
Notation . . . 1259
Worked example 4 — empirical risk . . . 1259
Systems constraints . . . 1259
Worked example 5 — peeking in A/B tests . . . 1259
Estimands, estimators, and error . . . 1259
Worked example 6 — biased but useful . . . 1260
Worked example 7 — standard error . . . 1260
Supervised learning as statistics . . . 1260
Worked example 8 — class imbalance . . . 1260
Causal vs predictive summary table . . . 1260
Worked example 9 — logging policy . . . 1260
Pitfalls . . . 1261
Checkpoint . . . 1261
Exercises . . . 1261
Summary . . . 1262

Statistical Foundations for Data Science **1263**
Descriptive Statistics . . . 1263
Measures of Central Tendency . . . 1263
Measures of Variability . . . 1264
Distribution Shape . . . 1265
Probability Fundamentals . . . 1266
Basic Probability Rules . . . 1266

Conditional Probability and Bayes' Theorem . . . 1267
Sampling and Sampling Distributions . . . 1268
Central Limit Theorem . . . 1268
Confidence Intervals . . . 1270
Statistical Inference . . . 1272
Hypothesis Testing Framework . . . 1272
Type I and Type II Errors . . . 1273
Correlation and Causation . . . 1275
Correlation Analysis . . . 1275
Spurious Correlations and Confounding . . . 1276
Summary . . . 1278

Sampling, Bias, and Representativeness **1280**
Diagram: target vs sample . . . 1280
1. Populations, frames, and units . . . 1280
2. Bias vs variance of estimators . . . 1280
3. Common sampling schemes . . . 1281
Worked example 1 — stratification . . . 1281
Worked example 2 — cluster design effect . . . 1281
4. Selection bias catalogue . . . 1281
Worked example 3 — help-click logs . . . 1282
Worked example 4 — survivorship in startups . . . 1282
5. Measurement bias vs sampling bias . . . 1282
6. Standard errors (quick formulas) . . . 1282
Scaling law . . . 1283
Worked example 5 — poll . . . 1283
Worked example 6 . . . 1283
7. Weighting and post-stratification . . . 1283
8. Train / validation / test . . . 1283
9. Online A/B sampling notes . . . 1284
10. Power and planning (tie-in) . . . 1284
11. Pitfalls . . . 1284
12. Checkpoint . . . 1284
Exercises . . . 1285
Easy . . . 1285
Medium . . . 1285
Challenge . . . 1285
Checks . . . 1285
Summary . . . 1286

Hypothesis Testing for Data Work **1287**
Diagram: decision pipeline . . . 1287
1. Pick the question first . . . 1287
2. Hypotheses and errors . . . 1287
Worked example 1 — ranking CTR . . . 1288
3. Common tests (when) . . . 1288
Welch vs Student . . . 1288

4. Confidence intervals first-class . . . 1288
Worked example 2 . . . 1289
5. Effect size always . . . 1289
Worked example 3 . . . 1289
6. Bootstrap intuition . . . 1289
Bootstrap vs normal CI . . . 1289
7. Multiple metrics and peeking . . . 1290
Multiple testing . . . 1290
Optional stopping . . . 1290
Worked example 4 . . . 1290
8. Power and sample size (order of magnitude) . . . 1290
9. Practical report template . . . 1290
10. Decision beyond p . . . 1291
11. Bayesian glance (optional) . . . 1291
12. Pitfalls . . . 1291
13. Checkpoint . . . 1291
Exercises . . . 1292
Easy . . . 1292
Medium . . . 1292
Challenge . . . 1292
Checks . . . 1293
Summary . . . 1293

Dimensionality, Features, and the Curse **1294**
Diagram: curse sketch . . . 1294
1. The curse of dimensionality (geometry) . . . 1294
Worked example 1 — intuition . . . 1294
When the curse is milder . . . 1295
2. Feature types and encodings . . . 1295
Worked example 2 — one-hot countries . . . 1295
3. Scaling and leakage . . . 1295
Worked example 3 — PCA scaling . . . 1296
4. Dimensionality reduction map . . . 1296
5. Feature crosses and interactions . . . 1296
Worked example 4 . . . 1296
6. Sparsity . . . 1296
7. Metrics: when Euclidean fails . . . 1297
Worked example 5 — cosine for documents . . . 1297
8. The “average distance to k-NN” trap . . . 1297
9. Embeddings vs one-hot . . . 1297
Worked example 6 . . . 1297
10. Pipeline checklist . . . 1298
11. Pitfalls . . . 1298
12. Checkpoint . . . 1298
Exercises . . . 1299
Easy . . . 1299
Medium . . . 1299
Challenge . . . 1299

Checks . . . 1299
Summary . . . 1300

Probability (Advanced) 1301

Advanced Probability for CS and AI 1302
Why go beyond "basic stats" . . . 1302
Learning path . . . 1302
Random experiments and models . . . 1302
Worked example 1 . . . 1302
Worked example 2 — heavy tails . . . 1303
Roadmap of chapters . . . 1303
Prerequisites . . . 1303
Three calculation templates . . . 1303
Expectation via indicators . . . 1303
Concentration sketch . . . 1303
Bayes update . . . 1303
Worked example 3 — A/B test math stack . . . 1304
Worked example 4 — PageRank as probability . . . 1304
Measure-theoretic honesty (light) . . . 1304
Pitfalls . . . 1304
Checkpoint . . . 1304
Exercises . . . 1305
Summary . . . 1305

Random Variables and Distributions 1306
1. Random variables . . . 1306
Worked example 1 . . . 1306
Worked example 2 . . . 1306
2. Transformations . . . 1306
Worked example 3 . . . 1306
Worked example 4 — inverse transform sampling . . . 1307
3. Discrete families . . . 1307
Worked example 5 — Poisson limit . . . 1307
4. Continuous families . . . 1307
Worked example 6 — Gaussian tails . . . 1307
5. Memoryless property . . . 1308
Worked example 7 . . . 1308
6. Joint distributions and independence . . . 1308
Worked example 8 . . . 1308
Worked example 9 — Bayes in classification . . . 1308
7. Quantiles and heavy tails . . . 1308
Worked example 10 — Pareto . . . 1308
8. Moment-generating / characteristic functions (awareness) . . . 1309
9. Pitfalls . . . 1309
10. Checkpoint . . . 1309

Exercises . . . 1309
Easy . . . 1309
Medium . . . 1310
Challenge . . . 1310
Summary . . . 1310

Expectation, Variance, and Covariance **1311**
1. Expectation . . . 1311
Linearity (always true) . . . 1311
Worked example 1 — indicator trick . . . 1311
Worked example 2 — hashing collisions . . . 1311
Worked example 3 — geometric mean trials . . . 1311
2. Variance and standard deviation . . . 1312
Worked example 4 . . . 1312
Worked example 5 — sum of independents . . . 1312
3. Covariance and correlation . . . 1312
Worked example 6 . . . 1312
Worked example 7 — uncorrelated but dependent . . . 1312
4. Conditional expectation . . . 1312
Worked example 8 — prediction . . . 1313
Worked example 9 — bias-variance (regression) . . . 1313
5. Moment inequalities (starters) . . . 1313
Worked example 10 — Chebyshev CI sketch . . . 1313
Worked example 11 — Jensen . . . 1313
6. Covariance matrices . . . 1313
Worked example 12 — PCA link . . . 1313
7. CS applications . . . 1314
8. Pitfalls . . . 1314
9. Checkpoint . . . 1314
Exercises . . . 1314
Easy . . . 1314
Medium . . . 1315
Challenge . . . 1315
Summary . . . 1315

Law of Large Numbers and Central Limit Theorem **1316**
1. Modes of convergence (brief) . . . 1316
2. Law of large numbers . . . 1316
Interpretation . . . 1316
Worked example 1 . . . 1316
Worked example 2 — Monte Carlo integration . . . 1316
3. Proof sketch of weak LLN (finite variance) . . . 1317
4. Central limit theorem . . . 1317
Worked example 3 — SE of the mean . . . 1317
Worked example 4 — latency . . . 1317
Worked example 5 — binomial / polls . . . 1317
5. Lindeberg / Lyapunov (awareness) . . . 1317

6. Berry–Esseen (rate) . . . 1317
Worked example 6 — skewed Bernoulli . . . 1318
7. Delta method . . . 1318
Worked example 7 . . . 1318
8. Multivariate CLT . . . 1318
9. Consequences in CS/ML . . . 1318
Worked example 8 — Monte Carlo error . . . 1318
Worked example 9 — why not $1/n$ error? . . . 1318
10. When CLT fails . . . 1319
Worked example 10 — Cauchy . . . 1319
11. Pitfalls . . . 1319
12. Checkpoint . . . 1319
Exercises . . . 1319
Easy . . . 1319
Medium . . . 1320
Challenge . . . 1320
Summary . . . 1320

Bayesian Inference **1321**
1. Bayes' rule for parameters . . . 1321
Worked example 1 — discrete hypothesis . . . 1321
2. Conjugate priors . . . 1321
Worked example 2 — Beta–Binomial . . . 1322
Worked example 3 — uniform prior . . . 1322
Worked example 4 — strong prior . . . 1322
3. MAP vs MLE vs posterior mean . . . 1322
Worked example 5 . . . 1322
Worked example 6 . . . 1322
4. Posterior predictive . . . 1322
Worked example 7 — Beta–Binomial predictive . . . 1323
5. Hierarchical models (sketch) . . . 1323
Worked example 8 . . . 1323
6. Computation when conjugates fail . . . 1323
Worked example 9 — MCMC intuition . . . 1323
Worked example 10 — VI . . . 1323
7. Decision theory link . . . 1323
8. CS/ML applications . . . 1324
Worked example 11 — online Bayes . . . 1324
Worked example 12 — regularization interpretation . . . 1324
9. Pitfalls . . . 1324
10. Checkpoint . . . 1324
Exercises . . . 1325
Easy . . . 1325
Medium . . . 1325
Challenge . . . 1325
Summary . . . 1325

Markov Chains and Stochastic Processes **1326**
1. Markov property . . . 1326
Worked example 1 — weather toy . . . 1326
Worked example 2 — gambler's ruin . . . 1326
2. Transition matrix . . . 1326
Worked example 3 . . . 1326
3. Classification of states . . . 1327
Worked example 4 . . . 1327
4. Stationary distribution . . . 1327
Existence/uniqueness (finite case) . . . 1327
Worked example 5 . . . 1327
Worked example 6 — PageRank . . . 1327
5. Detailed balance and reversibility . . . 1328
Worked example 7 — MCMC design . . . 1328
6. Hitting times and absorption . . . 1328
Worked example 8 — gambler's ruin . . . 1328
7. Mixing time (awareness) . . . 1328
Worked example 9 . . . 1328
8. Continuous time (sketch) . . . 1328
Worked example 10 — M/M/1 queue . . . 1328
9. CS applications map . . . 1328
Worked example 11 — RL . . . 1329
Worked example 12 — Gibbs sampling . . . 1329
10. Pitfalls . . . 1329
11. Checkpoint . . . 1329
Exercises . . . 1330
Easy . . . 1330
Medium . . . 1330
Challenge . . . 1330
Summary . . . 1330

Thresholds in Random Structures: Expectation Thresholds and the Park–Pham Theorem **1331**
1. Increasing properties and the threshold . . . 1331
Worked example 1 — two thresholds in $G(n,p)$, by simulation . . . 1332
2. Mental model: covers, cost, and the expectation threshold . . . 1333
3. Where the logarithm comes from . . . 1334
Worked example 2 — naive expectation threshold vs threshold, exactly . . . 1335
4. The theorem . . . 1336
5. Proof idea: minimum fragments . . . 1337
Worked example 3 — checking the counting step by brute force . . . 1338
6. The trap . . . 1339
7. The boring rule . . . 1340
8. Pitfalls . . . 1340
9. Checkpoint . . . 1340
Exercises (try this) . . . 1341
Summary . . . 1341
Sources (for further reading) . . . 1341

Linear Algebra (Advanced) **1343**

Advanced Linear Algebra for CS/ML **1344**
Why go beyond basic LA . . . 1344
Learning path . . . 1344
Matrix as linear map . . . 1344
Worked example 1 . . . 1344
Worked example 2 — graphs . . . 1344
Roadmap . . . 1345
CS/ML touchpoints preview . . . 1345
Four identities worth memorizing . . . 1345
Worked example 3 — PCA in one line . . . 1345
Worked example 4 — ridge filter . . . 1345
Worked example 5 — graph Laplacian . . . 1346
Orthogonality as numerical gold . . . 1346
Pitfalls . . . 1346
Checkpoint . . . 1346
Exercises . . . 1346
Summary . . . 1347

Eigenvalues and Eigenvectors **1348**
1. Definition . . . 1348
Worked example 1 . . . 1348
Worked example 2 . . . 1348
2. Characteristic polynomial . . . 1348
Worked example 3 — defective matrix . . . 1348
3. Diagonalization . . . 1349
Worked example 4 — dynamics . . . 1349
Worked example 5 — continuous . . . 1349
4. Spectral theorem (real symmetric) . . . 1349
Worked example 6 . . . 1349
Hermitian extension . . . 1349
5. Invariant subspaces and Schur form (awareness) . . . 1349
6. Rayleigh quotient . . . 1350
Worked example 7 . . . 1350
7. Graph spectra (CS) . . . 1350
Worked example 8 . . . 1350
Worked example 9 — spectral clustering . . . 1350
8. Markov chains link . . . 1350
Worked example 10 . . . 1350
9. Non-symmetric caveats . . . 1351
Worked example 11 . . . 1351
10. Trace and determinant . . . 1351
Worked example 12 . . . 1351
11. Pitfalls . . . 1351
12. Checkpoint . . . 1351
Exercises . . . 1352
Easy . . . 1352

Medium . . . 1352
Challenge . . . 1352
Summary . . . 1352

Orthogonality and Projections **1353**
1. Inner products and norms . . . 1353
Worked example 1 . . . 1353
2. Orthogonal sets and bases . . . 1353
Worked example 2 . . . 1353
3. Orthogonal projection onto a line . . . 1354
Worked example 3 . . . 1354
4. Projection onto a subspace . . . 1354
Worked example 4 . . . 1354
Worked example 5 — residual orthogonality . . . 1354
5. Gram–Schmidt and QR . . . 1354
Worked example 6 . . . 1354
Worked example 7 — Householder idea . . . 1355
6. Orthogonal matrices . . . 1355
Worked example 8 . . . 1355
7. Orthogonal complements . . . 1355
Worked example 9 — fundamental theorem of linear algebra . . . 1355
8. Projections in ML/CS . . . 1355
Worked example 10 — centering . . . 1355
Worked example 11 — dual view . . . 1356
9. Non-orthogonal projections (warning) . . . 1356
10. Pitfalls . . . 1356
11. Checkpoint . . . 1356
Exercises . . . 1356
Easy . . . 1356
Medium . . . 1357
Challenge . . . 1357
Summary . . . 1357

SVD and PCA **1358**
1. SVD statement . . . 1358
Geometric reading . . . 1358
Worked example 1 . . . 1358
2. Links to eigenvalues . . . 1358
Worked example 2 . . . 1359
3. Eckart–Young–Mirsky theorem . . . 1359
Worked example 3 — compression . . . 1359
Worked example 4 — denoising . . . 1359
4. Moore–Penrose pseudoinverse . . . 1359
Worked example 5 . . . 1359
5. PCA via SVD . . . 1359
Worked example 6 . . . 1360
Worked example 7 — explained variance . . . 1360
Worked example 8 — whitening . . . 1360

6. Recommendations and factor models . . . 1360
Worked example 9 . . . 1360
7. Conditioning and numerical rank . . . 1360
Worked example 10 . . . 1360
8. Randomized SVD (awareness) . . . 1360
Worked example 11 . . . 1360
9. Pitfalls . . . 1361
Worked example 12 — scale . . . 1361
10. Checkpoint . . . 1361
Exercises . . . 1361
Easy . . . 1361
Medium . . . 1362
Challenge . . . 1362
Summary . . . 1362

Least Squares and Conditioning **1363**
1. Linear least squares problem . . . 1363
Worked example 1 — line fit . . . 1363
Worked example 2 — overdetermined system . . . 1363
2. Normal equations . . . 1363
Worked example 3 . . . 1363
3. QR method . . . 1364
Worked example 4 . . . 1364
4. SVD method . . . 1364
Worked example 5 — rank-deficient . . . 1364
5. Weighted and generalized LS . . . 1364
Worked example 6 . . . 1364
6. Ridge regression as conditioning fix . . . 1364
Worked example 7 . . . 1364
Worked example 8 — leverage . . . 1365
7. Condition number and error bounds . . . 1365
Worked example 9 — Hilbert design . . . 1365
Worked example 10 — collinearity . . . 1365
8. Iterative methods for large LS . . . 1365
Worked example 11 . . . 1365
9. Gauss–Markov theorem (stat view) . . . 1365
Worked example 12 — misspecification . . . 1365
10. Pitfalls . . . 1366
11. Checkpoint . . . 1366
Exercises . . . 1366
Easy . . . 1366
Medium . . . 1366
Challenge . . . 1367
Summary . . . 1367

Spectral Thinking: Eigenvalues in Practice **1368**
Diagram: eigen action . . . 1368

1. Core facts . . . 1368
Worked example 1 . . . 1368
Worked example 2 — diagonal action . . . 1368
2. Powers, iterations, and stability . . . 1369
Worked example 3 — discrete dynamics . . . 1369
Worked example 4 — Markov / PageRank flavor . . . 1369
3. Symmetric matrices and Rayleigh quotients . . . 1369
4. PCA link . . . 1369
5. Graph Laplacian . . . 1370
Worked example 5 — path of 3 nodes . . . 1370
6. Conditioning . . . 1370
Worked example 6 — non-normal caution . . . 1370
7. Power method and cousins . . . 1371
Worked example 7 . . . 1371
8. Differential equations (glimpse) . . . 1371
9. Positive matrices (Perron–Frobenius flavor) . . . 1371
10. CS applications map . . . 1371
11. Pitfalls . . . 1372
Worked example 8 — rotation . . . 1372
12. Checkpoint . . . 1372
Exercises . . . 1372
Easy . . . 1372
Medium . . . 1373
Challenge . . . 1373
Checks . . . 1373
Summary . . . 1373

Random Rotations, Johnson–Lindenstrauss, and Hadamard Transforms 1374
1. Concentration: random directions are nearly orthogonal . . . 1374
Worked example 1 — angles between 1000 random vectors . . . 1375
2. The Johnson–Lindenstrauss lemma . . . 1375
Worked example 2 — what k does a real point set need? . . . 1376
3. Hadamard matrices and the fast transform . . . 1378
Worked example 3 — build it, check it, transform fast . . . 1378
4. Cheaper JL maps: sparse signs and the subsampled randomized Hadamard transform . 1379
Worked example 4 — four projections to k = 512 . . . 1380
5. Why random signs: the spreading lemma . . . 1381
Worked example 5 — peaks before and after . . . 1382
6. Rotate, then quantize . . . 1383
Worked example 6 — 4-bit quantization with outliers . . . 1383
7. The trap and the boring rule . . . 1385
8. Pitfalls . . . 1385
9. Checkpoint . . . 1385
Exercises . . . 1386
Easy . . . 1386
Medium . . . 1386
Challenge . . . 1386
Summary . . . 1387

Sources (for further reading) . . . 1387

The Linear Algebra Inside a Large Language Model **1388**
1. Where the parameters are . . . 1388
Worked example 1 — count Llama 3 exactly . . . 1389
2. Attention as a matrix . . . 1390
Worked example 2 — the rank of S and of P . . . 1391
3. Re-association and linear attention . . . 1392
Worked example 3 — three forms, one operator . . . 1392
4. Low-rank updates: LoRA and Eckart–Young . . . 1394
Worked example 4 — counts, the Eckart–Young limit, and reaching it . . . 1394
5. Preconditioning when the gradient is a matrix . . . 1396
Worked example 5 — Shampoo, SVD, and two Newton–Schulz iterations . . . 1397
6. Pitfalls . . . 1399
7. Checkpoint . . . 1399
Exercises . . . 1399
Easy . . . 1399
Medium . . . 1400
Challenge . . . 1400
Summary . . . 1400
Sources (for further reading) . . . 1401

Complexity Theory **1402**

Complexity Theory for Computer Science **1403**
Why it matters in industry . . . 1403
Relation to Algorithms & Complexity (part 13) . . . 1403
Core map . . . 1403
Decision problems first . . . 1404
Worked example 1 . . . 1404
Roadmap . . . 1404
Proof culture . . . 1404
Worked example 2 . . . 1404
Pitfalls . . . 1405
Worked mental models . . . 1405
Verifier view of NP . . . 1405
Reduction as subroutine . . . 1405
Space vs time . . . 1405
Worked example 3 — product call . . . 1405
Worked example 4 — approx choice . . . 1405
Checkpoint . . . 1406
Exercises . . . 1406
Summary . . . 1406

Reductions and NP-Completeness **1407**
1. Languages and decision problems . . . 1407
Worked example 1 . . . 1407

2. Classes **P** and **NP** . . . 1407
Worked example 2 — certificates . . . 1407
Worked example 3 — coNP . . . 1407
3. Karp reductions . . . 1408
Worked example 4 — direction . . . 1408
4. NP-hard and NP-complete . . . 1408
5. Cook–Levin theorem (statement) . . . 1408
6. NP-completeness proof template . . . 1408
7. Classic reductions (sketches) . . . 1409
3-SAT $\leq_p$ CLIQUE . . . 1409
Independent set / vertex cover . . . 1409
Worked example 5 . . . 1409
3-SAT $\leq_p$ SUBSET-SUM (idea) . . . 1409
Worked example 6 — PARTITION / KNAPSACK decision . . . 1409
Hamiltonian cycle $\leq_p$ TSP decision . . . 1409
Worked example 7 . . . 1409
8. Optimization vs decision . . . 1409
Worked example 8 . . . 1410
9. Proof hygiene examples of bugs . . . 1410
Worked example 9 — wrong direction essay . . . 1410
10. Beyond: strong NP-completeness, APX, etc. . . . 1410
Worked example 10 . . . 1410
11. Pitfalls . . . 1410
12. Checkpoint . . . 1411
Exercises . . . 1411
Easy . . . 1411
Medium . . . 1411
Challenge . . . 1411
Summary . . . 1412

Randomized and Approximation Complexity **1413**
1. Randomized complexity classes (informal) . . . 1413
Worked example 1 — polynomial identity testing . . . 1413
Worked example 2 — primality . . . 1413
2. Error reduction . . . 1413
Worked example 3 . . . 1414
3. Approximate counting and sampling (glimpse) . . . 1414
Worked example 4 . . . 1414
4. Approximation algorithms complexity . . . 1414
Worked example 5 . . . 1414
5. Positive classics . . . 1414
Worked example 6 . . . 1415
Worked example 7 — gap from integrality . . . 1415
6. Hardness of approximation . . . 1415
Worked example 8 — set cover . . . 1415
Worked example 9 — clique . . . 1415
Worked example 10 — metric vs non-metric TSP . . . 1415
7. PCP theorem (statement level) . . . 1415

8. Promise problems and gaps . . . 1415
Worked example 11 . . . 1416
9. Engineering reading of the map . . . 1416
Worked example 12 — product . . . 1416
10. Pitfalls . . . 1416
11. Checkpoint . . . 1416
Exercises . . . 1417
Easy . . . 1417
Medium . . . 1417
Challenge . . . 1417
Summary . . . 1417

Space Complexity **1418**
1. Model of space . . . 1418
Worked example 1 . . . 1418
2. Space classes . . . 1418
Worked example 2 . . . 1419
3. Configuration graphs . . . 1419
Worked example 3 . . . 1419
4. NL-complete problem: graph reachability . . . 1419
Worked example 4 — undirected reachability . . . 1419
5. Savitch's theorem . . . 1419
Worked example 5 . . . 1419
6. PSPACE-complete problems . . . 1420
Worked example 6 — games . . . 1420
Worked example 7 — model checking fragments . . . 1420
7. Log-space reductions . . . 1420
Worked example 8 . . . 1420
8. Relationships and hierarchy . . . 1420
Worked example 9 . . . 1420
9. CS practice connections . . . 1420
Worked example 10 — streaming distinct count . . . 1421
Worked example 11 — reachability in huge implicit graphs . . . 1421
10. Complementary classes . . . 1421
Worked example 12 . . . 1421
11. Pitfalls . . . 1421
12. Checkpoint . . . 1422
Exercises . . . 1422
Easy . . . 1422
Medium . . . 1422
Challenge . . . 1422
Summary . . . 1423

Fine-Grained Complexity Breaks: Truly Subquadratic 3SUM and Subcubic APSP **1424**
1. Mental model: hypotheses as walls, reductions as algorithms . . . 1424
2. The classical 3SUM wall . . . 1425
3. Lopsided All-Edges Sparse Triangle . . . 1426
4. What the new algorithm actually does (sketch) . . . 1427

5. Consequences worth remembering 1427
6. The trap 1428
7. Try this 1428
Checkpoint 1429
Summary 1429
Sources 1429

Maths library

A **broad mathematics library** for computer science and technical practice: arithmetic through discrete math, linear algebra, probability, graphs, optimization, and ML-facing math.

Structure

Content lives under `content/` by topic area (numeric parts):

Area	Topics
Foundations	Arithmetic, geometry, pre-algebra, algebra
Core CS math	Discrete math, number theory, graph theory, algorithms & complexity
Continuous / classical	Calculus, linear algebra (+ advanced), statistics, probability (advanced)
Applied / modern	Optimization, ML math, numerical methods, information theory, data-science math

Use the sidebar (HTML) or the generated table of contents for navigation.

How chapters are written

Most chapters include:

- **Definitions** in plain language + notation
- **ASCII diagrams** (portable in HTML / PDF / EPUB)
- **Worked examples**
- **Exercises** (often with short check answers)
- **CS / ML maps** where relevant

Optional calculator or computer algebra only to **check** work after a hand solution.

Suggested learning paths

```
Path A - CS undergrad spine
  Discrete → Number theory → Graphs → Algorithms/complexity
  + Linear algebra → Probability/stats

Path B - ML practitioner
  Linear algebra → Calc (derivatives) → Probability
  → Optimization → ML math → Data-science math

Path C - refresh from zero
  Arithmetic → Pre-algebra → Algebra → Geometry
  then join Path A or B
```

How to use this book

Format	Role
HTML	Primary — search, dark/light theme
PDF / EPUB	Offline reading when built by the monorepo pipeline

Prerequisites

- Willingness to write definitions and work examples by hand
- High-school algebra is enough to *start*; later chapters build up
- Programming snippets (when present) are optional checks, not the core

Related books

Book	Overlap
Go / C	Algorithms, complexity, and numeric code practice
Networking	Graph ideas and addressing as applied math
NixOS	Optional: evaluation/store models (not formal math)

Acknowledgments

Companion textbooks remain useful for extra drill (Rosen, MIT MCS, Concrete Mathematics, standard calc/LA texts). This monorepo is a personal study library, not a substitute for a full university course when you need one.

Intro

The Evolution of Mathematics: From Ancient Counting to Modern Abstractions

Introduction

Mathematics is often called the "universal language" - a system of logical reasoning that transcends cultural boundaries and connects human understanding across time and space. From the earliest human need to count objects to today's complex algorithms powering artificial intelligence, mathematics has evolved as humanity's most powerful tool for understanding and describing the world around us.

This journey through mathematical evolution reveals not just the development of numbers and formulas, but the story of human civilization itself - how we learned to think abstractly, solve problems systematically, and build upon the discoveries of previous generations.

```
Timeline of Mathematical Evolution


Prehistoric → Ancient → Classical → Medieval → Renaissance → Modern → Contemporary
(40,000 BCE)  (3000 BCE) (600 BCE)   (500 CE)   (1400 CE)    (1600 CE)  (1900 CE - Present)


 Counting    Number      Geometry   Algebra    Symbolic     Calculus   Abstract
 Systems     Systems     & Logic    & Trig     Notation     & Analysis  Structures
```

Chapter 1: The Dawn of Mathematical Thinking (Prehistoric Era - 3000 BCE)

The Birth of Counting

Before written language, before cities, before agriculture, humans developed the fundamental concept that would become mathematics: counting. Archaeological evidence suggests that our ancestors were keeping track of quantities as early as 40,000 years ago.

```
Early Counting Methods


Tally Marks on Bone:
||||  ||||  ||||  |||  = 18 objects
```

```
Body Parts Counting:
  Thumb = 1      Hand = 5      Person = 20

Stone Arrangements:

     = 2 groups of 5 = 10
```

The Ishango Bone: First Mathematical Tool

Discovered in the Democratic Republic of Congo, the Ishango bone (circa 20,000 BCE) shows sophisticated mathematical thinking:

```
Ishango Bone Pattern Analysis


Column 1: 11, 13, 17, 19  (Prime numbers!)
Column 2: 11, 21, 19, 9   (10+1, 20+1, 20-1, 10-1)
Column 3: 7, 5, 5, 10, 8, 4, 6, 3  (Doubling: 3×2=6, 4×2=8, 5×2=10)

    ||||||||||||  (11)
    ||||||||||||  (13)
    ||||||||||||||||  (17)
    ||||||||||||||||||  (19)
```

Development of Number Concepts

The evolution from concrete to abstract thinking:

```
Conceptual Development


Stage 1: Concrete Counting
   = "three sheep"

Stage 2: Abstract Quantity
     = "three things"

Stage 3: Symbolic Number
3 = "threeness" (the concept itself)

Stage 4: Number Operations
3 + 2 = 5 (manipulation of pure concepts)
```

Chapter 2: Ancient Civilizations and Number Systems (3000 BCE - 500 BCE)

Mesopotamian Mathematics: The Foundation

The Sumerians and Babylonians created the first sophisticated mathematical systems around 3000 BCE, developing:

Base-60 Number System

```
Babylonian Cuneiform Numbers


 = 1      = 10        = 5

Examples:
23 =     (10 + 10 + 1 + 1 + 1)

For larger numbers (base 60):
 in tens place = 60
 in tens place = 600

3661 =      (1×3600 + 1×60 + 1×1)
       ↑   ↑   ↑
    3600s 60s  1s
```

Babylonian Mathematical Achievements

```
Babylonian Mathematical Tablet Layout



  Problem: Find side of square
  with area 2

  Solution Method:

    ?    Area = 2



  Answer: 1;24,51,10 (base 60)
  = 1.41421... (√2 accurate to
    6 decimal places!)
```

Egyptian Mathematics: Practical Geometry

The Egyptians developed mathematics for practical purposes - building pyramids, managing the Nile floods, and trade.

Egyptian Number System (Hieroglyphic)

```
Egyptian Hieroglyphic Numbers


  = 1        = 10       = 100      = 1,000
  = 10,000    = 100,000    = 1,000,000

Example: 2,347
```

The Rhind Papyrus: Mathematical Problem Solving

```
Egyptian Fraction System


Problem: Divide 2 loaves among 3 people

Modern: 2/3
Egyptian: 1/2 + 1/6

Visual representation:

    1         1         1   1   1   1   1   1

    2         2         6   6   6   6   6   6

     Person 1              Person 2 & 3
```

Pyramid Construction Mathematics

```
Pyramid Slope Calculation (Seked)


The Great Pyramid:
Height = 146.5 meters
Base = 230.4 meters

Slope calculation:
    /|\
   / | \
```

```
  /  |h \
 /   |  \
/____b___\

Seked = horizontal distance per 1 cubit rise
      = (b/2) ÷ h = 115.2 ÷ 146.5  5.5 palms per cubit

Egyptian geometric precision:
- Base square accurate to 2 cm
- Angles accurate to 3 arcminutes
```

Chinese Mathematics: Systematic Thinking

The Nine Chapters on Mathematical Art

```
Chinese Rod Numerals


Positive numbers (red rods):
1: |    2: ||   3: |||  4: ||||  5: |||||
6: T    7: TT   8: TTT  9: TTTT

Negative numbers (black rods):
-1: /   -2: //  -3: ///  -4: ////  -5: /////

Place value system:
2,345 = || ||| |||| |||||
        ↑   ↑   ↑    ↑
     1000s 100s 10s  1s
```

Chinese Mathematical Innovations

```
Magic Square (Lo Shu)



 4   9   2    Each row, column, diagonal = 15

 3   5   7    Discovery: ~2800 BCE

 8   1   6


Pattern recognition:
- Center: 5 (middle of 1-9)
- Opposite corners sum to 10
```

- All lines sum to 15 (3 × 5)

Indian Mathematics: The Birth of Zero

The Revolutionary Concept of Zero

Evolution of Zero Concept

Stage 1: Empty Space
Babylonian: 2 _ 3 (meaning 203)

Stage 2: Placeholder
Indian: 2 3 (sunya = empty)

Stage 3: Number
Indian: (zero as a number itself)

Stage 4: Operations
0 + 5 = 5
0 × 5 = 0
5 - 5 = 0

Indian Numeral System

Brahmi Numerals Evolution

Brahmi (300 BCE):
Devanagari (400 CE):
Arabic (800 CE):
European (1200 CE): 1 2 3 4 5 6 7 8 9

The journey of our modern numerals!

Chapter 3: Classical Period - Greek Mathematical Revolution (600 BCE - 500 CE)

The Birth of Mathematical Proof

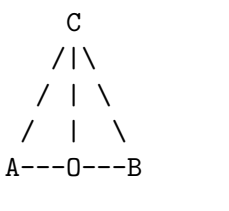
The Greeks transformed mathematics from practical calculation to logical reasoning.

Thales: First Mathematical Proofs

Thales' Theorem Proof Structure

Theorem: Angle in semicircle is always 90°

Given: Circle with diameter AB, point C on circle
Prove: $\angle ACB = 90°$

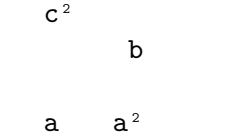


Proof outline:
1. O is center, so OA = OB = OC (radii)
2. Triangle OAC is isosceles → $\angle OAC = \angle OCA$
3. Triangle OBC is isosceles → $\angle OBC = \angle OCB$
4. $\angle AOC + \angle BOC = 180°$ (straight line)
5. Therefore $\angle ACB = 90°$ (angle sum in triangle)

Pythagoras: Numbers as Reality

Pythagorean Theorem Visualizations

Geometric Proof:



Area of large square = $(a + b)^2$
Area of inner square = c^2
Area of 4 triangles = $4 \times (1/2)ab = 2ab$

Therefore: $(a + b)^2 = c^2 + 2ab$
$a^2 + 2ab + b^2 = c^2 + 2ab$
$a^2 + b^2 = c^2$

Numerical Example:
$3^2 + 4^2 = 5^2$

```
9 + 16 = 25

Right triangle:
    |\
  5 | \
    |  \
    |___\
      3   4
```

Euclid: The Systematic Approach

```
Euclidean Geometry Foundation


Postulates (Axioms):
1. A straight line can be drawn between any two points
2. Any finite straight line can be extended
3. A circle can be drawn with any center and radius
4. All right angles are equal
5. Parallel postulate (if a line intersects two lines...)

Example Construction - Equilateral Triangle:

Step 1: Draw line AB
A      B

Step 2: Circle centered at A, radius AB



A       B

   -------

Step 3: Circle centered at B, radius AB

      C

A       B

    ------

Step 4: Connect AC and BC
      C
```

```
A    B

Result: Triangle ABC with AB = BC = CA
```

Archimedes: Mathematical Physics

Calculating (Pi)

```
Archimedes' Method for


Using inscribed and circumscribed polygons:

Circle with radius 1:


                ← Circumscribed hexagon

                 ← Inscribed hexagon

 -----------------

Hexagon perimeter <   < Hexagon perimeter
(inscribed)              (circumscribed)

Starting with hexagons (6 sides):
- Inscribed: 3.000000
- Circumscribed: 3.464102

Doubling to 12 sides:
- Inscribed: 3.105829
- Circumscribed: 3.215390

Continuing to 96 sides:
3.141031 <   < 3.142714

Modern value:   = 3.141592653589793...
```

The Method of Exhaustion

```
Area Under Parabola


Problem: Find area under y = x² from 0 to 1
```

Archimedes' approach:

Using rectangles:
Width = 1/n, Heights = $(1/n)^2$, $(2/n)^2$, ..., $(n/n)^2$

Sum = $(1/n) \times [(1/n)^2 + (2/n)^2 + ... + (n/n)^2]$
$= (1/n^3) \times [1^2 + 2^2 + ... + n^2]$
$= (1/n^3) \times [n(n+1)(2n+1)/6]$
$= (n+1)(2n+1)/(6n^2)$

As $n \to \infty$: Area = 1/3

Chapter 4: Medieval Mathematics - Preservation and Innovation (500 CE - 1400 CE)

Islamic Golden Age: Algebra is Born

Al-Khwarizmi: The Father of Algebra

Al-Khwarizmi's Algebraic Method

Problem: $x^2 + 10x = 39$

Geometric Solution:

x^2	10x	Total area = $x^2 + 10x = 39$

x	10

Complete the square:

x^2	5x	5	
	Add 25 to both sides		
5x	25	x	$(x + 5)^2 = 39 + 25 = 64$

x	5	5

Solution: $x + 5 = 8$, so $x = 3$

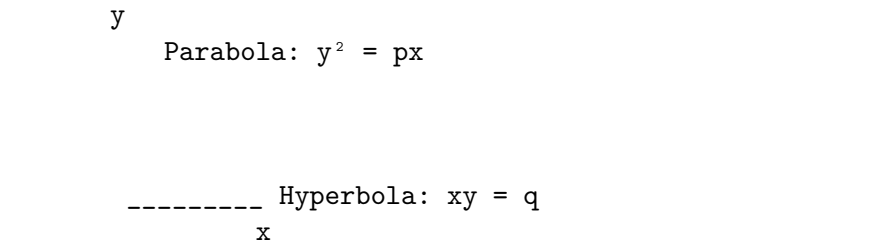
Verification: $3^2 + 10(3) = 9 + 30 = 39$

Omar Khayyam: Cubic Equations

Geometric Solution of Cubic Equations

Problem: $x^3 + px^2 + qx + r = 0$

Khayyam's method using conic sections:



Intersection points give solutions to cubic equation

Example: $x^3 = 2x + 1$
- Parabola: $y^2 = 2x$
- Hyperbola: $xy = 1$

Solutions found geometrically where curves intersect

Fibonacci: Bringing Eastern Mathematics West

The Fibonacci Sequence

Rabbit Population Problem

Month 1: 1 pair (newborn)
Month 2: 1 pair (still immature)
Month 3: 2 pairs (original pair reproduces)
Month 4: 3 pairs (first offspring mature)
Month 5: 5 pairs
...

Sequence: 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89...

```
Pattern: F(n) = F(n-1) + F(n-2)

Visual representation:
Month: 1  2  3  4  5  6  7   8   9
Pairs: 1  1  2  3  5  8  13  21  34

Ratio convergence:
21/13 = 1.615...
34/21 = 1.619...
55/34 = 1.617...
89/55 = 1.618... → Golden Ratio   = (1+√5)/2
```

Hindu-Arabic Numerals in Europe

```
Comparison of Number Systems (circa 1200 CE)


Roman Numerals vs Hindu-Arabic:

Addition:
Roman:    MCCCXLVII + DCLXXXIV = ?
         (1347)      (684)

Step by step:
M + D = M + D
CCC + C = CCCC = CD
XL + LXXX = CXX
VII + IV = XI

Result: MMXXXI (2031)

Hindu-Arabic:  1347 + 684 = 2031

Multiplication:
Roman: XXIII × XVII = ?
Hindu-Arabic: 23 × 17 = 391

The efficiency difference is obvious!
```

Chapter 5: Renaissance Mathematics - Symbolic Revolution (1400 CE - 1600 CE)

The Development of Symbolic Notation

François Viète: Father of Modern Algebra

Evolution of Algebraic Notation

Ancient (Rhetorical):
"The square of the unknown plus twice the unknown equals 15"

Medieval (Syncopated):
"1 quadratum + 2 res aequatur 15"

Viète (Symbolic):
"1A quad + 2A aequatur 15"

Modern:
"$x^2 + 2x = 15$"

Viète's Innovation - Using letters for:
- Known quantities: A, B, C (consonants)
- Unknown quantities: X, Y, Z (vowels)

Solving Cubic and Quartic Equations

Cardano's Formula for Cubic Equations

General form: $x^3 + px + q = 0$

Solution: $x = \sqrt[3]{(-q/2 + \sqrt{(q^2/4 + p^3/27)})} + \sqrt[3]{(-q/2 - \sqrt{(q^2/4 + p^3/27)})}$

Example: $x^3 - 15x - 4 = 0$
Here $p = -15$, $q = -4$

Discriminant = $q^2/4 + p^3/27 = 16/4 + (-15)^3/27 = 4 - 125 = -121$

This leads to complex numbers!
$x = \sqrt[3]{(2 + 11i)} + \sqrt[3]{(2 - 11i)} = 4$

Verification: $4^3 - 15(4) - 4 = 64 - 60 - 4 = 0$

The Birth of Complex Numbers

```
Complex Number Visualization


Imaginary Axis

   3i     • (2 + 3i)

   2i

   i
        Real Axis
  -2   1  2
    -i

   -2i
```

Complex number $z = a + bi$
- Real part: a
- Imaginary part: b
- Magnitude: $|z| = \sqrt{(a^2 + b^2)}$
- Argument: $\theta = \arctan(b/a)$

Operations:
$(2 + 3i) + (1 - 2i) = 3 + i$
$(2 + 3i) \times (1 - 2i) = 2 - 4i + 3i - 6i^2 = 2 - i + 6 = 8 - i$

Chapter 6: The Scientific Revolution - Calculus and Analysis (1600 CE - 1800 CE)

Newton and Leibniz: The Calculus Wars

The Fundamental Theorem of Calculus

```
Connecting Derivatives and Integrals

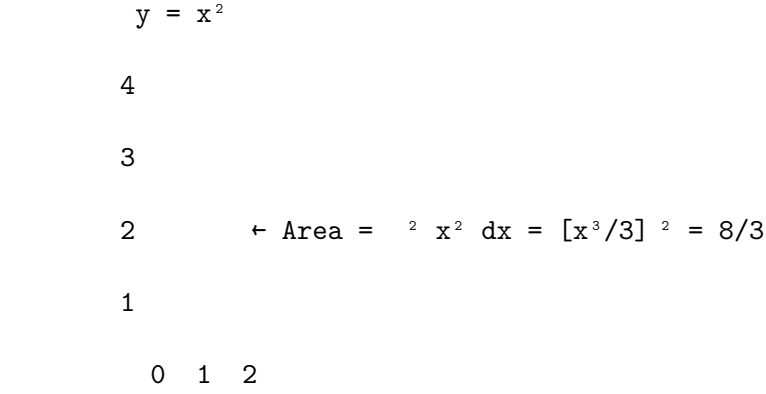

Problem: Find area under curve y = x²

Newton's Method (Fluxions):
Let area function be A(x) =    t² dt

Rate of change of area = derivative of A(x)
dA/dx = x² (the original function!)
```

Leibniz's Notation:
$f'(x)\,dx = f(x) + C$
$\frac{d}{dx} \int f(x)\,dx = f(x)$

Visual representation:



The area function $A(x) = x^3/3$ has derivative $A'(x) = x^2$

Applications of Calculus

Planetary Motion (Newton's Laws)

Kepler's Laws + Newton's Calculus = Universal Gravitation

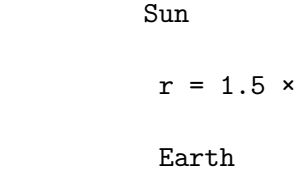
$F = GMm/r^2$

For circular orbit:
Centripetal force = Gravitational force
$mv^2/r = GMm/r^2$
$v^2 = GM/r$
$v = \sqrt{(GM/r)}$

Orbital period: $T = 2\,r/v = 2\,r/\sqrt{(GM/r)} = 2\sqrt{(r^3/GM)}$

Therefore: $T^2 \propto r^3$ (Kepler's Third Law proven!)

Earth-Sun system:



T = 1 year = 365.25 days

Euler: The Master of All Mathematics

Euler's Identity

The Most Beautiful Equation in Mathematics

e^(i) + 1 = 0

Connecting five fundamental constants:
- e (natural logarithm base) 2.71828...
- i (imaginary unit) = √(-1)
- (pi) 3.14159...
- 1 (multiplicative identity)
- 0 (additive identity)

Derivation using Taylor series:
e^x = 1 + x + x²/2! + x³/3! + x /4! + ...

For x = i :
e^(i) = 1 + i + (i)²/2! + (i)³/3! + (i) /4! + ...
       = 1 + i - ²/2! - i ³/3! + /4! + ...
       = (1 - ²/2! + /4! - ...) + i(- ³/3! + /5! - ...)
       = cos( ) + i sin( )
       = -1 + i(0)
       = -1

Therefore: e^(i) = -1, so e^(i) + 1 = 0

Graph Theory Birth

The Seven Bridges of Königsberg

Problem: Can you walk through the city crossing each bridge exactly once?

Map representation:
    A (North bank)

B   C (Island)

    D (South bank)

```
Graph abstraction:
    A


 B   C


    D

Euler's insight: Each vertex has degree (number of edges):
A: degree 3, B: degree 3, C: degree 3, D: degree 3

For Eulerian path to exist, at most 2 vertices can have odd degree.
Since all 4 vertices have odd degree, no solution exists!

This founded graph theory and topology.
```

Chapter 7: Modern Mathematics - Abstraction and Rigor (1800 CE - 1900 CE)

Non-Euclidean Geometry

Gauss, Bolyai, and Lobachevsky

```
Parallel Postulate Alternatives


Euclidean Geometry:
Through point P not on line l, exactly one parallel line exists.

    P •

         (exactly one parallel)

           l

Hyperbolic Geometry (Lobachevsky):
Through point P, infinitely many parallels exist.

    P •
             (infinitely many parallels)

           l

Spherical Geometry (Riemann):
No parallel lines exist (all great circles intersect).
```

```
       P
       •      (no parallels possible)


 ------------
      "line" l
```

Group Theory: Galois and Abstract Algebra

Symmetries and Groups

```
Symmetry Group of Square


Square with vertices labeled:
1  2


4  3

Symmetries (8 total):
- Identity: I (no change)
- Rotations: R °, R  °, R  °
- Reflections: H (horizontal), V (vertical), D , D  (diagonals)

Group table (partial):
      I  R  R   R   H  V  D  D

 I    I  R  R   R   H  V  D  D
R   R  R   R    I  D  D   V  H
R  R   R    I  R  V  H  D  D
...

Properties:
- Closure: combining any two symmetries gives another symmetry
- Associativity: (AB)C = A(BC)
- Identity: I leaves everything unchanged
- Inverse: every symmetry has an "undo" operation
```

Set Theory: Cantor's Infinite

Different Sizes of Infinity

```
Cantor's Diagonal Argument
```

```
Proving uncountably many real numbers exist:

Assume all real numbers in [0,1] can be listed:
r  = 0.a a a a ...
r  = 0.a a a a ...
r  = 0.a a a a ...
r  = 0.a a a a ...
...

Construct new number d:
d = 0.d d d d ...

Where d   a  (diagonal elements)

Example:
r  = 0.14159...  → d  = 2 (  1)
r  = 0.27182...  → d  = 8 (  7)
r  = 0.33333...  → d  = 4 (  3)
r  = 0.14159...  → d  = 6 (  5)

So d = 0.2846... differs from every r  in the list!
Contradiction → uncountably infinite real numbers.
```

Chapter 8: Contemporary Mathematics - The Digital Age (1900 CE - Present)

Computer Science and Mathematics

Boolean Algebra and Logic Gates

```
Boolean Logic Foundation


Basic Operations:
AND ( ): 1   1 = 1, otherwise 0
OR ( ):  0   0 = 0, otherwise 1
NOT (¬): ¬1 = 0, ¬0 = 1

Truth Table for (A   B)   (¬A   C):
A   B   C   A B   ¬A   ¬A C   Result

0   0   0    0    1     0       0
0   0   1    0    1     1       1
0   1   0    0    1     0       0
0   1   1    0    1     1       1
```

```
1   0   0    0    0     0       0
1   0   1    0    0     0       0
1   1   0    1    0     0       1
1   1   1    1    0     0       1

Circuit representation:
A      AND      OR    Output

B

A   NOT      AND

C
```

Chaos Theory and Fractals

The Mandelbrot Set

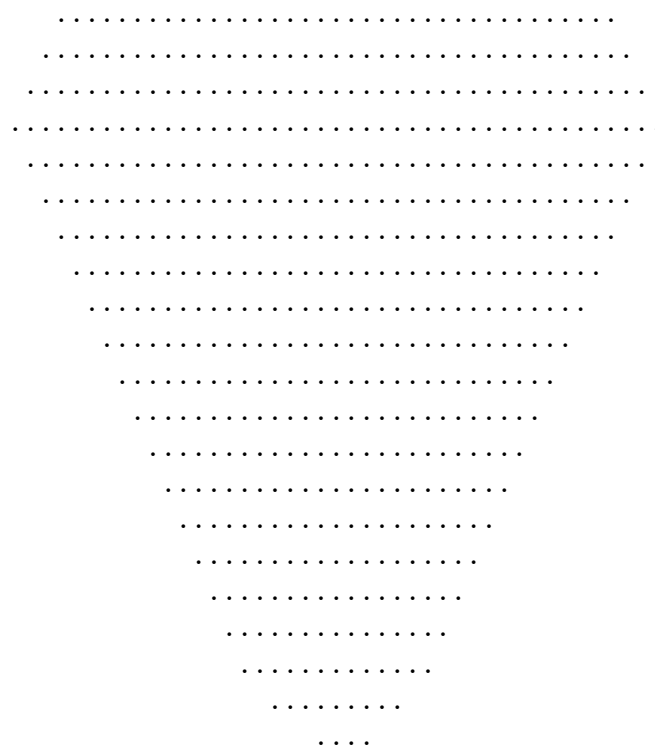
```
Mandelbrot Set Definition


For complex number c, iterate: z_{n+1} = z_n² + c
Starting with z  = 0

c is in Mandelbrot set if sequence remains bounded.

Examples:
c = 0: 0 → 0 → 0 → ... (bounded  )
c = 1: 0 → 1 → 2 → 5 → 26 → ... (unbounded  )
c = -1: 0 → -1 → 0 → -1 → ... (bounded  )
c = i: 0 → i → -1+i → -i → -1+i → ... (bounded  )

ASCII approximation of Mandelbrot set:
                    ....
                 .........
               ............
              ..............
             ................
            ..................
           ....................
          ......................
         ........................
        ..........................
       ............................
      ..............................
     ................................
    ..................................
```



Machine Learning and AI Mathematics

Neural Network Mathematics

Simple Neural Network

Input Layer → Hidden Layer → Output Layer

x
h
x y
h
x

Mathematical representation:
h = (w x + w x + w x + b)
h = (w x + w x + w x + b)
y = (v h + v h + b)

Where is activation function (e.g., sigmoid):
(x) = 1/(1 + e^(-x))

Sigmoid function graph:
1.0

```
0.5

0.0

    -4    -2     0     2     4

Learning via backpropagation:
 Error/ w =  Error/ y ×  y/ h ×  h/ w
```

Chapter 9: The Future of Mathematics

Quantum Computing and Mathematics

```
Quantum Bit (Qubit) Representation


Classical bit: 0 or 1
Quantum bit:  |0  +  |1  where | |² + | |² = 1

Bloch Sphere representation:
        |0


           ← Qubit state









        |1

Quantum gates (unitary matrices):
Pauli-X (NOT): [0 1]
               [1 0]

Hadamard: 1/√2 [1  1]  (creates superposition)
               [1 -1]
```

```
CNOT: [1 0 0 0]  (entanglement)
      [0 1 0 0]
      [0 0 0 1]
      [0 0 1 0]
```

Artificial Intelligence and Mathematical Discovery

AI-Assisted Theorem Proving

```
Automated Proof Systems


Traditional proof:
Theorem: √2 is irrational
Proof: Assume √2 = p/q (lowest terms)
       Then 2 = p²/q²
       So 2q² = p²
       Therefore p² is even, so p is even
       Let p = 2k, then 2q² = 4k²
       So q² = 2k², meaning q is even
       Contradiction: p and q both even
       Therefore √2 is irrational

AI-assisted approach:
1. Pattern recognition in existing proofs
2. Automated lemma generation
3. Proof verification and optimization
4. Discovery of new proof techniques

Recent AI achievements:
- AlphaGeometry solving IMO problems
- Lean theorem prover verification
- Automated conjecture generation
```

Conclusion: Mathematics as Human Heritage

The Interconnected Web of Mathematical Knowledge

```
Mathematical Knowledge Network


Number Theory ↔ Cryptography ↔ Computer Science

Algebra ↔ Geometry ↔ Topology ↔ Physics

Analysis ↔ Calculus ↔ Differential ↔ Engineering
```

```
                        Equations
Statistics ↔ Probability ↔ Machine Learning

Economics ↔ Game Theory ↔ Artificial Intelligence

Each connection represents centuries of human insight!
```

The Continuing Journey

Mathematics continues to evolve, driven by:

1. **Technological Advancement**: Quantum computing, AI, and big data create new mathematical challenges
2. **Scientific Discovery**: Physics, biology, and other sciences pose new mathematical questions
3. **Human Curiosity**: Pure mathematical research continues to reveal beautiful patterns and structures
4. **Global Collaboration**: Mathematicians worldwide build upon each other's work

```
The Mathematical Timeline Continues...


Past            Present          Future

Ancient                 Digital              Quantum
Counting               Computing             Computing

Geometry               Machine                AI-Human
& Logic                Learning              Collaboration

Algebra                Cryptography          Post-Quantum
& Calculus             & Security            Mathematics

Abstract               Big Data              Unknown
Structures             Analytics             Frontiers
```

From the first tally marks on ancient bones to the complex algorithms powering modern AI, mathematics represents humanity's greatest intellectual achievement - our ability to find order in chaos, patterns in complexity, and universal truths that transcend time and culture.

The story of mathematics is far from over. Each generation builds upon the discoveries of the past, pushing the boundaries of human understanding ever further into the infinite realm of mathematical possibility.

As we stand at the threshold of quantum computing, artificial intelligence, and technologies we can barely imagine, mathematics remains our most powerful tool for understanding and shaping the future. The next chapter in this magnificent story is being written right now, by mathematicians, scientists, and curious minds around the world.

The evolution of mathematics is the evolution of human thought itself - and that evolution continues with each new discovery, each solved problem, and each question that opens up entirely new worlds of mathematical wonder.

Arithmetic

Introduction to Arithmetic: The Foundation of All Mathematics

What is Arithmetic?

Arithmetic is the oldest and most fundamental branch of mathematics, dealing with the basic operations of numbers: addition, subtraction, multiplication, and division. The word "arithmetic" comes from the Greek word "arithmos," meaning number. It's the mathematical foundation that every human learns first, yet its elegant simplicity masks profound depth and beauty.

From counting sheep in ancient pastures to calculating trajectories for space missions, arithmetic forms the bedrock upon which all mathematical thinking is built. Understanding arithmetic deeply means understanding how numbers work, how they relate to each other, and how they can be manipulated to solve problems.

```
The Four Pillars of Arithmetic


    Addition         Subtraction      Multiplication    Division
       +                 -                ×               ÷

    Combining        Taking           Repeated      Sharing
    quantities       away             addition      equally




                    All connected through
                    inverse relationships
```

The Historical Journey of Arithmetic

From Fingers to Symbols

Before written numbers existed, humans used their most accessible counting tool - their fingers. This natural base-10 system still influences how we think about numbers today.

```
Evolution of Counting Systems


Finger Counting (Prehistoric):
  = 5      = 10      = 20 (person)

Tally Marks (30,000 BCE):
||||  ||||  ||||  |||  = 18

Egyptian Hieroglyphs (3000 BCE):
  = 1      = 10      = 100

Roman Numerals (500 BCE):
I = 1    V = 5    X = 10    L = 50    C = 100

Hindu-Arabic (500 CE):
1  2  3  4  5  6  7  8  9  0

Modern Digital (1940s):
Binary: 1010  = 10
```

The Revolutionary Concept of Zero

The introduction of zero wasn't just adding another digit - it was a conceptual revolution that transformed arithmetic forever.

```
The Power of Zero


Without Zero:
Roman: MCMXC + X = MM (1990 + 10 = 2000)
- Cumbersome calculations
- No placeholder concept
- Limited mathematical operations

With Zero:
Arabic: 1990 + 10 = 2000
- Positional notation
- Placeholder function
- Enables advanced calculations

Zero's Three Roles:

 Placeholder     Number       Operation
     205            0          5 - 5
   (empty)      (nothing)     (result)
```

Understanding Numbers: The Building Blocks

Natural Numbers: Where It All Begins

Natural numbers are the counting numbers: 1, 2, 3, 4, 5, … They represent the most intuitive concept of quantity.

```
Natural Numbers Visualization


Concrete Representation:
1:
2:
3:
4:
5:

Number Line:
1  2  3  4  5  6  7  8  9  10 →

Properties:
- Always positive
- No fractions or decimals
- Infinite set
- Used for counting discrete objects
```

Whole Numbers: Adding the Concept of Nothing

Whole numbers include all natural numbers plus zero: 0, 1, 2, 3, 4, 5, …

```
Whole Numbers vs Natural Numbers


Natural Numbers:    1, 2, 3, 4, 5, ...
Whole Numbers:   0, 1, 2, 3, 4, 5, ...
                 ↑
            The crucial addition

Number Line with Zero:
0  1  2  3  4  5  6  7  8  9  10 →
↑
Starting point - the concept of "nothing"
```

Integers: Embracing the Negative

Integers extend whole numbers to include negative numbers: ..., -3, -2, -1, 0, 1, 2, 3, ...

```
Integer Number Line


← -5 -4 -3 -2 -1  0  1  2  3  4  5 →
   ↑                   ↑                   ↑
Negative            Zero Point           Positive
(less than zero)   (neither +/-)      (greater than zero)

Real-world examples:
Temperature: -10°C (below freezing)
Elevation: -50m (below sea level)
Finance: -$100 (debt)
Time: -2 hours (2 hours ago)
```

The Four Fundamental Operations

Addition: Combining Quantities

Addition is the process of combining two or more quantities to find their total.

```
Addition Concepts


Visual Addition (3 + 2 = 5):
     +      =

Number Line Addition:
Start at 3, move 2 steps right:
0  1  2  3  4  5  6  7  8  9  10
            ↑   →→  ↑
          start  +2  end

Column Addition:
  247
+ 156

  403

Step by step:
  247      247      247
+ 156    + 156    + 156
```

```
   3       03      403
   ↑        ↑       ↑
 7+6=13   4+5+1=10  2+1+1=4
 write 3  write 0   write 4
 carry 1  carry 1   carry 1
```

Properties of Addition

```
Addition Properties


Commutative Property: a + b = b + a
Example: 5 + 3 = 3 + 5 = 8
Visual:          +       =       +

Associative Property: (a + b) + c = a + (b + c)
Example: (2 + 3) + 4 = 2 + (3 + 4) = 9
         ↓           ↓
         5 + 4 = 9   2 + 7 = 9

Identity Property: a + 0 = a
Example: 7 + 0 = 7
Visual:             +  (nothing)  =
```

Subtraction: Finding the Difference

Subtraction is the process of taking away one quantity from another, or finding the difference between quantities.

```
Subtraction Concepts


Visual Subtraction (8 - 3 = 5):
             →
Remove 3 dots         5 remain

Number Line Subtraction:
Start at 8, move 3 steps left:
0  1  2  3  4  5  6  7  8  9  10
                    ↑   ←←←  ↑
                   end  -3  start

Column Subtraction with Borrowing:
  523
- 187
```

```
  336

Step by step:
  523      523      523
- 187    - 187    - 187

    6       36      336
    ↑        ↑        ↑
  3-7: need  13-7=6   2-8: need
  to borrow  write 6  to borrow

  413      413
- 187    - 187

   36      336
   ↑        ↑
 11-8=3   4-1=3
 write 3  write 3
```

Types of Subtraction Problems

```
Three Types of Subtraction


1. Take Away: "I had 10 apples, ate 3, how many left?"
   10 - 3 = 7
         →

2. Comparison: "John has 12 marbles, Sue has 8, what's the difference?"
   12 - 8 = 4
   John:
   Sue:
   Diff:

3. Missing Addend: "3 + ? = 8"
   8 - 3 = 5
      +     =
```

Multiplication: Repeated Addition

Multiplication is repeated addition of the same number, or finding the total when you have equal groups.

```
Multiplication Concepts
```

```
Repeated Addition (4 × 3 = 12):
4 + 4 + 4 = 12
   +    +    =

Array Model:
4 × 3 = 12 (4 rows of 3)




Area Model:

            6


            4

4 × 6 = 24 square units

Skip Counting:
3 × 7: 7, 14, 21
0  7  14  21  28  35  42
    ↑    ↑    ↑
   +7   +7   +7
```

The Multiplication Table

```
Multiplication Table (1-10)


×    1  2  3  4  5  6  7  8  9 10

 1   1  2  3  4  5  6  7  8  9 10
 2   2  4  6  8 10 12 14 16 18 20
 3   3  6  9 12 15 18 21 24 27 30
 4   4  8 12 16 20 24 28 32 36 40
 5   5 10 15 20 25 30 35 40 45 50
 6   6 12 18 24 30 36 42 48 54 60
 7   7 14 21 28 35 42 49 56 63 70
 8   8 16 24 32 40 48 56 64 72 80
 9   9 18 27 36 45 54 63 72 81 90
10  10 20 30 40 50 60 70 80 90100

Patterns to notice:
- Diagonal (squares): 1, 4, 9, 16, 25, 36, 49, 64, 81, 100
- 5s column: always ends in 0 or 5
```

- 9s column: digits sum to 9 (18→1+8=9, 27→2+7=9)

Properties of Multiplication

Multiplication Properties

Commutative Property: a × b = b × a
Example: 3 × 4 = 4 × 3 = 12
Visual: 3 rows of 4 = 4 rows of 3

Associative Property: (a × b) × c = a × (b × c)
Example: (2 × 3) × 4 = 2 × (3 × 4) = 24
↓ ↓
6 × 4 = 24 2 × 12 = 24

Identity Property: a × 1 = a
Example: 8 × 1 = 8

Zero Property: a × 0 = 0
Example: 5 × 0 = 0

Distributive Property: a × (b + c) = (a × b) + (a × c)
Example: 3 × (4 + 2) = (3 × 4) + (3 × 2) = 12 + 6 = 18

Division: Sharing Equally

Division is the process of splitting a quantity into equal parts or finding how many times one number goes into another.

Division Concepts

Sharing Model (12 ÷ 3 = 4):
12 objects shared among 3 groups:
Group 1:
Group 2:
Group 3:
Each group gets 4 objects

Grouping Model (12 ÷ 3 = 4):
How many groups of 3 in 12?

```
   |    |    |
 1     2     3     4
Answer: 4 groups

Number Line Division:
12 ÷ 3: How many jumps of 3 to reach 12?
0  3  6  9  12
    ↑   ↑   ↑   ↑
    1   2   3   4 jumps
```

Long Division Algorithm

```
Long Division: 847 ÷ 7


Step-by-step process:
      121

  7   847
      7↓     8 ÷ 7 = 1 remainder 1

      14     Bring down 4: 14 ÷ 7 = 2
      14

       07    Bring down 7: 7 ÷ 7 = 1
        7

        0

Verification: 121 × 7 = 847

Division with Remainder:
      123 R 4

  7   865
      7↓

      16
      14

       25
       21

        4  ← Remainder

Check: (123 × 7) + 4 = 861 + 4 = 865
```

Number Systems and Place Value

Decimal System: Base 10

Our everyday number system uses base 10, likely because we have 10 fingers.

```
Place Value Chart


Number: 3,456.789

Thousands Hundreds Tens Ones Tenths Hundredths Thousandths
   10^3      10^2    10^1  10   10^-1    10^-2       10^-3
  1000       100      10   1     0.1      0.01       0.001
     3         4       5   6       7         8           9

Value breakdown:
3,456.789 = (3×1000) + (4×100) + (5×10) + (6×1) + (7×0.1) + (8×0.01) + (9×0.001)
          = 3000 + 400 + 50 + 6 + 0.7 + 0.08 + 0.009
```

Other Number Systems

```
Comparison of Number Systems


Decimal (Base 10): 0,1,2,3,4,5,6,7,8,9
Binary (Base 2):   0,1
Octal (Base 8):    0,1,2,3,4,5,6,7
Hexadecimal (16):  0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F

Converting 25 (decimal) to other bases:

Binary (Base 2):
25 ÷ 2 = 12 remainder 1
12 ÷ 2 = 6  remainder 0
6  ÷ 2 = 3  remainder 0
3  ÷ 2 = 1  remainder 1
1  ÷ 2 = 0  remainder 1
Reading upward: 25  = 11001

Verification: 1×16 + 1×8 + 0×4 + 0×2 + 1×1 = 16+8+1 = 25

Octal (Base 8):
25 ÷ 8 = 3 remainder 1
3  ÷ 8 = 0 remainder 3
Reading upward: 25  = 31
```

```
Verification: 3×8 + 1×1 = 24+1 = 25
```

Mental Math Strategies

Addition Strategies

```
Mental Addition Techniques


1. Make 10 Strategy:
   7 + 5 = ?
   7 + 3 + 2 = 10 + 2 = 12

   Visual:
        +     =      +    +   =        +

2. Compensation:
   29 + 17 = ?
   30 + 17 - 1 = 47 - 1 = 46

3. Break Apart:
   47 + 26 = ?
   (40 + 20) + (7 + 6) = 60 + 13 = 73

4. Number Line Jumps:
   38 + 25 = ?
   38 → 40 → 50 → 63
       +2   +10  +13
```

Multiplication Shortcuts

```
Multiplication Tricks


1. Multiplying by 9:
   9 × 7 = ?
   Hold up 10 fingers, fold down 7th finger
   Left side: 6 fingers = 60
   Right side: 3 fingers = 3
   Answer: 63

2. Multiplying by 11:
   23 × 11 = ?
   2_3 → 2(2+3)3 = 253
```

```
   For larger sums:
   67 × 11 = ?
   6_7 → 6(6+7)7 = 6(13)7 = 737

3. Squares ending in 5:
   25² = ?
   2 × (2+1) = 2 × 3 = 6
   Append 25: 625

   35² = ?
   3 × (3+1) = 3 × 4 = 12
   Append 25: 1225

4. Doubling and Halving:
   16 × 25 = ?
   32 × 12.5 = 8 × 50 = 4 × 100 = 400
```

Fractions: Parts of a Whole

Understanding Fractions

```
Fraction Visualization


Fraction: 3/4 (three-fourths)

Pizza Model:

          3 pieces eaten
         out of 4 total



Number Line:
0  ¼  ½  ¾  1
        ↑
       3/4

Set Model:
    (3 out of 4 objects)

Area Model:

          3/4 shaded
```

Fraction Operations

```
Adding Fractions


Same Denominator:
1/4 + 2/4 = 3/4

Visual:

       +         =


Different Denominators:
1/3 + 1/4 = ?

Find common denominator (LCD = 12):
1/3 = 4/12
1/4 = 3/12
4/12 + 3/12 = 7/12

Visual:

            +

    1/3               1/4

Convert to twelfths:



        7/12
```

Decimals: Another Way to Express Parts

Decimal Place Value

```
Decimal Number: 45.678


Whole Part   Decimal Part
    45          .678

Place Value Breakdown:
Tens Ones Decimal Tenths Hundredths Thousandths
 4    5      .      6        7          8
```

```
Value: 40 + 5 + 0.6 + 0.07 + 0.008 = 45.678

Visual representation:

                            45 whole units
                                 + 0.678

     0.6       0.07      0.008
```

Converting Between Fractions and Decimals

```
Fraction to Decimal Conversion


1/4 = ?
  0.25
4)1.00
   8
   --
   20
   20
   --
    0

3/8 = ?
  0.375
8)3.000
   24
   --
   60
   56
   --
   40
   40
   --
    0

Common Fraction-Decimal Equivalents:
1/2 = 0.5      1/4 = 0.25     3/4 = 0.75
1/3 = 0.333... 2/3 = 0.666... 1/5 = 0.2
1/8 = 0.125    3/8 = 0.375    5/8 = 0.625
1/10 = 0.1     1/100 = 0.01   1/1000 = 0.001
```

Percentages: Parts per Hundred

Understanding Percentages

Percentage Visualization

25% = 25 per 100 = 25/100 = 0.25 = 1/4

Grid Model (100 squares):

25% shaded
(25 out of 100)

Conversion Triangle:

Percentage
÷100 ×100

Decimal ↔ Fraction
×100 ÷100

Percentage Calculations

Three Types of Percentage Problems

1. Find the percentage:
   "What percent of 80 is 20?"
   20/80 = 0.25 = 25%

2. Find the part:
   "What is 30% of 150?"
   0.30 × 150 = 45

3. Find the whole:
   "25 is 20% of what number?"
   25 ÷ 0.20 = 125

Visual for 30% of 150:

Total: 150 items

45 items (30%) 105 items (70%)

Problem-Solving Strategies

The Four-Step Problem-Solving Process

Problem-Solving Framework

1. UNDERSTAND the problem

   - Read carefully
   - Identify key info
   - What are we finding?

↓

2. PLAN your approach

   - Choose operation(s)
   - Estimate answer
   - Draw/visualize

↓

3. SOLVE the problem

   - Execute your plan
   - Show your work
   - Check calculations

↓

4. CHECK your answer

   - Does it make sense?
   - Try different method
   - Verify with estimate

Word Problem Examples

Sample Problem Analysis

```
Problem: "Sarah has 24 stickers. She gives away 1/3 of them to her friends and uses 25% of the

UNDERSTAND:
- Started with: 24 stickers
- Gave away: 1/3 of 24
- Used for project: 25% of what remained
- Find: How many left?

PLAN:
Step 1: Find 1/3 of 24 (division/multiplication)
Step 2: Subtract from 24 (subtraction)
Step 3: Find 25% of remainder (multiplication)
Step 4: Subtract from step 2 result (subtraction)

SOLVE:
Step 1: 1/3 × 24 = 8 stickers given away
Step 2: 24 - 8 = 16 stickers remaining
Step 3: 25% × 16 = 0.25 × 16 = 4 stickers used
Step 4: 16 - 4 = 12 stickers left

Visual:
Original:               (24)
Gave away:      (8)
Remaining:           (16)
Used:    (4)
Final:         (12)

CHECK:
8 + 4 + 12 = 24  (accounts for all original stickers)
Estimate: About 1/3 gone (8), then 1/4 of remainder (4)
So about 24 - 8 - 4 = 12
```

Real-World Applications

Money and Finance

```
Money Calculations


Making Change:
Purchase: $7.23
Payment: $10.00
Change: $10.00 - $7.23 = $2.77

Count up method:
$7.23 → $7.25 → $7.50 → $8.00 → $10.00
```

```
       +$0.02  +$0.25  +$0.50  +$2.00
Total change: $2.77

Bill breakdown:
$2.77 = 2 × $1.00 + 3 × $0.25 + 0 × $0.10 + 0 × $0.05 + 2 × $0.01
      = 2 dollars + 3 quarters + 2 pennies

Simple Interest:
Principal: $1000
Rate: 5% per year
Time: 3 years
Interest = P × R × T = $1000 × 0.05 × 3 = $150
Total = $1000 + $150 = $1150
```

Measurement and Cooking

```
Recipe Scaling


Original Recipe (serves 4):
- 2 cups flour
- 1/2 cup sugar
- 3/4 cup milk
- 2 eggs

Scale to serve 6 people:
Scaling factor: 6 ÷ 4 = 1.5

New amounts:
- Flour: 2 × 1.5 = 3 cups
- Sugar: 1/2 × 1.5 = 3/4 cup
- Milk: 3/4 × 1.5 = 9/8 = 1 1/8 cups
- Eggs: 2 × 1.5 = 3 eggs

Unit Conversions:
1 cup = 16 tablespoons = 48 teaspoons
1 pound = 16 ounces
1 gallon = 4 quarts = 8 pints = 16 cups

Converting 1 1/8 cups to tablespoons:
1 1/8 = 9/8 cups
9/8 × 16 = 18 tablespoons
```

Time and Distance

```
Speed, Distance, Time Problems
```

```
Formula: Distance = Speed × Time
         Speed = Distance ÷ Time
         Time = Distance ÷ Speed

Problem: "A car travels 240 miles in 4 hours. What is its average speed?"
Speed = 240 miles ÷ 4 hours = 60 mph

Problem: "How long does it take to travel 180 miles at 45 mph?"
Time = 180 miles ÷ 45 mph = 4 hours

Problem: "At 55 mph, how far can you travel in 2.5 hours?"
Distance = 55 mph × 2.5 hours = 137.5 miles

Time Calculations:
Meeting starts: 2:45 PM
Duration: 1 hour 35 minutes
End time: 2:45 + 1:35 = 4:20 PM

Elapsed time from 9:30 AM to 2:15 PM:
9:30 AM → 12:00 PM = 2 hours 30 minutes
12:00 PM → 2:15 PM = 2 hours 15 minutes
Total: 4 hours 45 minutes
```

Common Arithmetic Mistakes and How to Avoid Them

Addition and Subtraction Errors

```
Common Mistakes in Addition


Mistake 1: Forgetting to carry
  247
+ 156

  393  ← Wrong! (forgot to carry from 7+6=13)

Correct:
  ¹247  ← carry the 1
+ 156

  403

Mistake 2: Misaligning place values
  247
```

```
+  56

  293  ← Wrong! (56 should align right)

Correct:
  247
+  56  ← align ones place

  303

Mistake 3: Borrowing errors in subtraction
  502
- 147

  445  ← Wrong!

Correct borrowing:
  4912  ← borrow from hundreds and tens
- 147

  355
```

Multiplication and Division Errors

```
Common Multiplication Mistakes


Mistake 1: Forgetting zeros in partial products
   23
×  45

  115  ← 23 × 5
   92  ← Wrong! Should be 920 (23 × 40)

  207  ← Wrong answer

Correct:
   23
×  45

  115  ← 23 × 5
  920  ← 23 × 40 (note the zero!)

 1035

Mistake 2: Division remainder errors
17 ÷ 3 = 5 remainder 3  ← Wrong! (5×3=15, 17-15=2)
```

```
Correct: 17 ÷ 3 = 5 remainder 2

Check: (quotient × divisor) + remainder = dividend
       (5 × 3) + 2 = 15 + 2 = 17
```

Building Number Sense

Estimation Skills

```
Estimation Strategies


Rounding for Estimation:
347 + 289  ?
Round: 350 + 290 = 640
Actual: 347 + 289 = 636 (close!)

Front-End Estimation:
4.7 × 8.2  ?
Use: 4 × 8 = 32
Actual: 4.7 × 8.2 = 38.54 (reasonable!)

Benchmark Numbers:
Is 7/8 closer to 1/2 or 1?
7/8 = 0.875, which is closer to 1 than to 0.5

Compatible Numbers:
198 ÷ 21  ?
Use: 200 ÷ 20 = 10
Actual: 198 ÷ 21  9.43 (good estimate!)

Order of Magnitude:
2,847 × 5,923  ?
Think: 3,000 × 6,000 = 18,000,000
Actual: 16,863,681 (right magnitude!)
```

Pattern Recognition

```
Number Patterns


Arithmetic Sequences:
2, 5, 8, 11, 14, ...
Pattern: +3 each time
Next terms: 17, 20, 23
```

```
Geometric Sequences:
3, 6, 12, 24, 48, ...
Pattern: ×2 each time
Next terms: 96, 192, 384

Square Numbers:
1, 4, 9, 16, 25, 36, ...
Pattern: 1², 2², 3², 4², 5², 6²
Visual:
```

```
Triangular Numbers:
1, 3, 6, 10, 15, 21, ...
Pattern: 1, 1+2, 1+2+3, 1+2+3+4, ...
Visual:
```

Conclusion: The Beauty of Arithmetic

Arithmetic is far more than just computation - it's the foundation of logical thinking, problem-solving, and understanding the quantitative world around us. From the ancient Babylonians developing place value systems to modern computers processing billions of calculations per second, arithmetic remains the cornerstone of mathematical thinking.

```
The Arithmetic Journey


Basic Counting → Number Systems → Operations → Problem Solving
      ↓              ↓              ↓              ↓
   Quantity      Place Value    Relationships   Real World
  Recognition    Understanding   Between Ops    Applications
      ↓              ↓              ↓              ↓
   Foundation    Efficient       Pattern        Critical
   for Higher    Calculation     Recognition    Thinking
   Mathematics      Skills         Skills         Skills
```

As you continue your mathematical journey, remember that every complex mathematical concept - from algebra to calculus to advanced statistics - builds upon these fundamental arithmetic principles. Master these basics with understanding, not just memorization, and you'll have a solid foundation for all future mathematical learning.

The beauty of arithmetic lies not just in getting the right answer, but in understanding why the methods work, recognizing patterns and relationships, and applying these concepts to solve real-world problems. Whether you're calculating a tip, determining how much paint you need for a room, or analyzing data trends, arithmetic provides the tools for quantitative reasoning that will serve you throughout your life.

```
Arithmetic: The Universal Language


Numbers transcend cultures, languages, and time periods.
The logic of arithmetic is the same whether you're:

Ancient Egyptian → Building pyramids
Medieval Merchant → Calculating profits
Modern Student → Learning mathematics
Computer Programmer → Writing algorithms
Scientist → Analyzing data
Parent → Helping with homework

All using the same fundamental principles
discovered and refined over thousands of years.
```

Numbers and Counting: The Foundation of Mathematics

Introduction

Before we can add, subtract, multiply, or divide, we must first understand what numbers are and how counting works. This fundamental concept is so basic that we often take it for granted, yet it represents one of humanity's greatest intellectual achievements.

Counting is the process of determining the quantity of objects in a collection. Numbers are the symbols and concepts we use to represent these quantities. Together, they form the foundation upon which all of mathematics is built.

The Evolution of Counting

Prehistoric Counting Methods

Long before written language existed, humans needed to keep track of quantities. Archaeological evidence shows various counting methods:

```
Early Counting Methods


Body Parts Counting:
  Thumb = 1
  Hand = 5
  Person = 20 (fingers + toes)

Some cultures counted:
- Up to 5 (one hand)
- Up to 10 (both hands)
- Up to 20 (hands + feet)
- Up to 27 (hands + feet + head parts)

Tally Systems:
||||  ||||  ||||  |||  = 18 objects
 5     5     5    3

Grouped tallies:
```

```
||||/ ||||/ ||||/ |||  = 18 objects
  5     5     5    3
(crossing line represents 5)
```

The Ishango Bone: Ancient Mathematical Tool

The Ishango bone, discovered in the Democratic Republic of Congo and dating to about 20,000 years ago, shows sophisticated mathematical thinking:

```
Ishango Bone Analysis


Column A: 11, 13, 17, 19 (all prime numbers!)
Column B: 11, 21, 19, 9  (10+1, 20+1, 20-1, 10-1)
Column C: 7, 5, 5, 10, 8, 4, 6, 3

Patterns discovered:
- Prime number recognition
- Base-10 awareness (±1 from multiples of 10)
- Doubling relationships (3→6, 4→8, 5→10)

Visual representation:
||||||||||  (11 notches)
|||||||||||  (13 notches)
|||||||||||||||  (17 notches)
|||||||||||||||||  (19 notches)
```

Understanding Natural Numbers

What Are Natural Numbers?

Natural numbers are the counting numbers: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, …

They represent the most basic concept of quantity - how many objects are in a collection.

```
Natural Numbers Visualization


Concrete Objects:
1:
2:
3:
4:
5:

Abstract Dots:
```

```
1:
2:
3:
4:
5:

Number Line:
1  2  3  4  5  6  7  8  9  10 →
```

Properties of Natural Numbers

```
Natural Number Properties


1. Discrete: Each number is separate and distinct
   1, 2, 3 (not 1.5 or 2.7)

2. Ordered: Each number has a definite position
   3 comes after 2 and before 4

3. Infinite: The sequence never ends
   ...998, 999, 1000, 1001, 1002...

4. Successor Property: Every natural number has a next number
   5 → 6 → 7 → 8 → ...

5. Well-Ordered: Every non-empty set has a smallest element
   In {5, 2, 8, 1, 9}, the smallest is 1
```

Cardinality vs. Ordinality

```
Two Aspects of Numbers


Cardinal Numbers (How many?):
"There are 5 books on the table"

Count: 1, 2, 3, 4, 5 books

Ordinal Numbers (What position?):
"This is the 3rd book from the left"

1st 2nd 3rd 4th 5th

Same numbers, different meanings!
```

Whole Numbers: Adding Zero

The Revolutionary Concept of Zero

Whole numbers include all natural numbers plus zero: 0, 1, 2, 3, 4, 5, …

The addition of zero was a revolutionary mathematical concept that took centuries to develop.

```
The Evolution of Zero


Stage 1: No Concept (Ancient Times)
"Empty" was just... empty. No symbol needed.

Stage 2: Placeholder (Babylonian ~400 BCE)
2 _ 3 meant 203 (empty space in middle)

Stage 3: Symbol (Indian ~500 CE)
2  3 using "sunya" (empty) symbol

Stage 4: Number (Indian ~700 CE)
 became a number itself, not just empty space

Stage 5: Operations (Medieval)
0 + 5 = 5
0 × 5 = 0
5 - 5 = 0
```

Zero's Three Roles

```
Zero's Multiple Personalities


1. As a Placeholder:
   205 ← zero holds the tens place
   Without zero: 25 (completely different!)

2. As a Number:
   "I have 0 apples" (a quantity of nothing)

3. As an Operation Result:
   5 - 5 = 0 (the result of subtraction)

Visual representation:
Placeholder: 2 0 5
             ↑ ↑ ↑
          200+0+5
```

```
Number:   (empty set)

Result:     -     =
```

Integers: Embracing Negative Numbers

The Need for Negative Numbers

Integers extend whole numbers to include negative numbers: ..., -3, -2, -1, 0, 1, 2, 3, ...

Negative numbers arose from practical needs:

```
Real-World Negative Numbers


Temperature:
-10°C ←   0°C   → +20°C
Freezing    Freezing    Room
  point      point    temperature

Elevation:
-50m ←   0m   → +100m
Below     Sea level   Above
sea level             sea level

Finance:
-$500 ←   $0   → +$1000
 Debt    Break-even   Profit

Time:
-2 hours ←   Now   → +3 hours
2 hours ago           3 hours from now
```

The Integer Number Line

```
Complete Integer Number Line


← -5 -4 -3 -2 -1  0  1  2  3  4  5 →
   ↑                    ↑                    ↑
Negative            Zero Point           Positive
(less than 0)    (neither +/-)      (greater than 0)

Properties:
- Extends infinitely in both directions
```

- Zero is neither positive nor negative
- Each positive number has a negative counterpart
- Distance from zero determines absolute value

Absolute Value

```
Absolute Value Concept


Definition: Distance from zero (always positive)

|5| = 5    (5 is 5 units from zero)
|-5| = 5   (-5 is also 5 units from zero)
|0| = 0    (0 is 0 units from zero)

Number line visualization:
← -5 -4 -3 -2 -1  0  1  2  3  4  5 →
   ↑←    5 units    →↑←  5 units  →↑
  -5                      0              5

Both -5 and 5 are exactly 5 units from zero!

Examples:
|7| = 7
|-12| = 12
|0| = 0
|-3.5| = 3.5
```

Number Systems and Bases

Understanding Base 10 (Decimal System)

Our everyday number system uses base 10, likely because humans have 10 fingers.

```
Base 10 Place Value System


Number: 3,456

Thousands Hundreds Tens Ones
  10^3      10^2    10^1  10
 1000       100      10   1
   3         4       5    6

Value calculation:
```

```
3,456 = (3×1000) + (4×100) + (5×10) + (6×1)
      = 3000 + 400 + 50 + 6

Visual representation:
Thousands:    (3 blocks of 1000)
Hundreds:     (4 blocks of 100)
Tens:          (5 blocks of 10)
Ones:          (6 individual units)
```

Other Number Systems

```
Comparison of Number Systems


Base 2 (Binary) - Used by computers:
Digits: 0, 1
Example: 1011  = (1×8) + (0×4) + (1×2) + (1×1) = 11

Base 8 (Octal):
Digits: 0, 1, 2, 3, 4, 5, 6, 7
Example: 23  = (2×8) + (3×1) = 19

Base 16 (Hexadecimal):
Digits: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F
Example: 1F  = (1×16) + (15×1) = 31

Base 60 (Babylonian):
Used for time: 1 hour = 60 minutes = 3600 seconds
```

Converting Between Bases

```
Converting Decimal to Binary


Convert 25  to binary:

Method: Repeatedly divide by 2, track remainders

25 ÷ 2 = 12 remainder 1  ↑
12 ÷ 2 = 6  remainder 0
6  ÷ 2 = 3  remainder 0   Read
3  ÷ 2 = 1  remainder 1   upward
1  ÷ 2 = 0  remainder 1  ↑

Result: 25  = 11001
```

```
Verification:
11001  = (1×16) + (1×8) + (0×4) + (0×2) + (1×1)
       = 16 + 8 + 0 + 0 + 1 = 25

Visual check:
Position: 4 3 2 1 0
Power:   16 8 4 2 1
Binary:   1 1 0 0 1
Value:   16+8+0+0+1 = 25
```

Counting Strategies and Patterns

Skip Counting

```
Skip Counting Patterns


Count by 2s (Even numbers):
2, 4, 6, 8, 10, 12, 14, 16, 18, 20...


Count by 5s:
5, 10, 15, 20, 25, 30, 35, 40, 45, 50...


Count by 10s:
10, 20, 30, 40, 50, 60, 70, 80, 90, 100...

Number line visualization:
0  5  10  15  20  25  30  35  40  45  50
    ↑   ↑   ↑   ↑   ↑   ↑   ↑   ↑   ↑   ↑
   +5  +5  +5  +5  +5  +5  +5  +5  +5  +5
```

Grouping and Bundling

```
Grouping for Easier Counting


Counting 23 objects:

Method 1: One by one
             (tedious!)

Method 2: Groups of 5

  5     5     5     5    3
```

= 4 groups of 5 + 3 extra = 20 + 3 = 23

Method 3: Groups of 10

    10   10   3
= 2 groups of 10 + 3 extra = 20 + 3 = 23

Base-10 blocks visualization:

   (10)   (10)   (3)

Number Patterns

Recognizing Number Patterns

Arithmetic Sequences (constant difference):
2, 5, 8, 11, 14, 17, 20...
 +3 +3 +3 +3 +3 +3
Pattern: Start at 2, add 3 each time

Even Numbers:
2, 4, 6, 8, 10, 12, 14...
Pattern: Multiples of 2

Odd Numbers:
1, 3, 5, 7, 9, 11, 13...
Pattern: One more than even numbers

Square Numbers:
1, 4, 9, 16, 25, 36, 49...
1^2 2^2 3^2 4^2 5^2 6^2 7^2

Visual squares:

1   4   9   16

Triangular Numbers:
1, 3, 6, 10, 15, 21, 28...

Comparing and Ordering Numbers

Comparison Symbols

Mathematical Comparison Symbols

Symbol	Meaning	Example
=	Equal to	5 = 5
	Not equal to	5 ≠ 7
<	Less than	3 < 8
>	Greater than	9 > 4
	Less than/equal	5 ≤ 5
	Greater/equal	7 ≥ 7

Memory tricks:
< looks like "L" for "Less"
> opens toward the larger number
The "mouth" always "eats" the bigger number

Examples:
3 < 7 (3 is less than 7)
7 > 3 (7 is greater than 3)

Ordering Numbers

Ordering Numbers on Number Line

Numbers: 7, 2, 9, 4, 1, 6

Step 1: Place on number line
1 2 3 4 5 6 7 8 9 10

Step 2: Read from left to right
Ascending order: 1, 2, 4, 6, 7, 9
Descending order: 9, 7, 6, 4, 2, 1

For negative numbers:
Numbers: -3, 5, -1, 0, 2, -4

-4 -3 -2 -1 0 1 2 3 4 5

Ascending: -4, -3, -1, 0, 2, 5

```
Descending: 5, 2, 0, -1, -3, -4

Key insight: Moving left = smaller, Moving right = larger
```

Comparing Multi-Digit Numbers

```
Comparing Large Numbers


Compare: 2,847 and 2,853

Method: Compare digit by digit from left to right

2,847
2,853
↑
Same thousands digit (2)

2,847
2,853
 ↑
Same hundreds digit (8)

2,847
2,853
  ↑
Same tens digit (4 vs 5)
4 < 5, so 2,847 < 2,853

Visual representation:
2,847:
2,853:
       ↑     ↑        ↑      ↑
    Same   Same   Different Same
   (2000) (800)   (40<50)  (7>3)

The first different digit determines the comparison!
```

Rounding and Estimation

Rounding Rules

```
Rounding to Nearest 10


Rule: Look at the ones digit
```

```
- If 0, 1, 2, 3, 4: Round down
- If 5, 6, 7, 8, 9: Round up

Examples:
23 → 20 (3 < 5, round down)
27 → 30 (7  5, round up)
35 → 40 (5  5, round up)
41 → 40 (1 < 5, round down)

Number line visualization:
20  25  30
  ↑   ↑   ↑
 23  25  27
  ↓   ↓   ↓
 20  30  30

Rounding to Nearest 100:
247 → 200 (47 < 50, round down)
263 → 300 (63  50, round up)
150 → 200 (50  50, round up)

Visual:
200  250  300
   ↑    ↑    ↑
  247  250  263
   ↓    ↓    ↓
  200  300  300
```

Estimation Strategies

```
Front-End Estimation


Problem: Estimate 347 + 289

Method 1: Round to nearest 100
347 → 300
289 → 300
Estimate: 300 + 300 = 600
Actual: 347 + 289 = 636 (close!)

Method 2: Front-end estimation
347 → 300 (keep hundreds digit)
289 → 200 (keep hundreds digit)
Estimate: 300 + 200 = 500
Adjust: 47 + 89  50 + 90 = 140
Better estimate: 500 + 140 = 640
```

```
Actual: 636 (very close!)

Method 3: Compatible numbers
347 + 289
Think: 350 + 290 = 640
Actual: 636 (excellent!)
```

Applications in Daily Life

Counting Money

```
Counting Coins and Bills


U.S. Currency values:
Penny: $0.01    Nickel: $0.05    Dime: $0.10    Quarter: $0.25
Dollar: $1.00   Five: $5.00      Ten: $10.00    Twenty: $20.00

Counting strategy: Start with largest denomination

Example: Count this money
$20 + $10 + $5 + $1 + $1 + $0.25 + $0.25 + $0.10 + $0.05 + $0.01

Step by step:
Bills: $20 + $10 + $5 + $1 + $1 = $37
Quarters: $0.25 + $0.25 = $0.50
Dimes: $0.10
Nickels: $0.05
Pennies: $0.01
Total: $37 + $0.50 + $0.10 + $0.05 + $0.01 = $37.66

Making change from $40.00:
$40.00 - $37.66 = $2.34
```

Time and Counting

```
Time-Based Counting


Counting seconds in a minute:
1, 2, 3, 4, 5, ..., 58, 59, 60
(60 seconds = 1 minute)

Counting minutes in an hour:
1, 2, 3, 4, 5, ..., 58, 59, 60
(60 minutes = 1 hour)
```

```
Counting hours in a day:
1, 2, 3, 4, 5, ..., 22, 23, 24
(24 hours = 1 day)

Elapsed time counting:
Start: 2:15 PM
End: 4:30 PM

Method 1: Count by hours and minutes
2:15 → 3:15 → 4:15 → 4:30
       1 hr    1 hr   15 min
Total: 2 hours 15 minutes

Method 2: Convert to minutes
2:15 PM = 14:15 = 14×60 + 15 = 855 minutes
4:30 PM = 16:30 = 16×60 + 30 = 990 minutes
Difference: 990 - 855 = 135 minutes = 2 hours 15 minutes
```

Inventory and Grouping

```
Real-World Counting Applications


Classroom supplies:
24 students, need 3 pencils each
Total needed: 24 × 3 = 72 pencils

Pencils come in boxes of 12
Boxes needed: 72 ÷ 12 = 6 boxes

Verification:
6 boxes × 12 pencils/box = 72 pencils

Seating arrangement:
48 people, tables seat 6 each
Tables needed: 48 ÷ 6 = 8 tables

Visual arrangement:
Table 1:      Table 2:      Table 3:
Table 4:      Table 5:      Table 6:
Table 7:      Table 8:

Total: 8 × 6 = 48 people
```

Building Number Sense

Benchmarks and Reference Points

```
Using Benchmark Numbers


Common benchmarks:
5, 10, 25, 50, 100, 500, 1000

Estimating with benchmarks:
"About how many?"

47 → Close to 50
23 → Close to 25
89 → Close to 100
347 → Between 300 and 400, closer to 300

Distance estimation:

0       25       50       75      100

Where does 67 belong?

0       25       50  67    75      100
67 is between 50 and 75, closer to 75

Fraction benchmarks:
0     1/4     1/2     3/4     1


Where does 3/8 belong?
3/8 = 0.375, which is between 1/4 (0.25) and 1/2 (0.5)
Closer to 1/2 than to 1/4
```

Subitizing: Instant Recognition

```
Subitizing: Recognizing Small Quantities Instantly


Most people can instantly recognize quantities up to 4 or 5:


1      2      3       4        5

Dice patterns (help with subitizing):
```

```
1 2 3 4 5 6

Domino patterns:
[]  [ ]  [ ]  [ ]  [ ]  [ ]
[]  []   []    [ ]  [ ]   [ ]
 2    3     4      4     5      6

Playing card patterns:


1    2     3     4     5

This instant recognition helps with:
- Quick addition
- Pattern recognition
- Mental math
- Understanding groups
```

Conclusion

Numbers and counting form the absolute foundation of mathematical thinking. From the earliest human civilizations using tally marks to modern computer systems using binary code, the concept of quantity and the ability to count have been central to human progress.

Understanding numbers deeply means more than just memorizing counting sequences. It involves:

```
Complete Number Understanding


Conceptual Understanding:
- What numbers represent (quantity, position, measurement)
- How numbers relate to each other
- Why number systems work the way they do

Procedural Fluency:
- Counting accurately and efficiently
- Comparing and ordering numbers
- Converting between different representations

Strategic Competence:
- Choosing appropriate counting strategies
- Estimating and checking reasonableness
- Solving problems involving quantities

Adaptive Reasoning:
- Understanding why counting methods work
- Justifying mathematical thinking
```

- Making connections between concepts

As you continue your mathematical journey, remember that every advanced concept builds upon these fundamental ideas about numbers and counting. Whether you're solving algebraic equations, analyzing statistical data, or programming computers, you're using the same basic principles that humans discovered thousands of years ago when they first began to count.

The beauty of numbers lies not just in their practical utility, but in their elegant patterns, their infinite nature, and their ability to describe and quantify the world around us. Master these fundamentals, and you'll have a solid foundation for all future mathematical learning.

Addition: Combining Quantities

Introduction

Addition is the most fundamental arithmetic operation - the process of combining two or more quantities to find their total sum. It's the first operation most people learn, yet it contains profound mathematical concepts that extend far beyond simple counting.

From a child combining toy blocks to a scientist calculating molecular interactions, addition represents the mathematical concept of "putting together" or "increasing by." Understanding addition deeply means grasping not just the procedures, but the underlying concepts, patterns, and relationships that make this operation so powerful.

```
Addition: The Foundation Operation


Concrete Level:       +   =
Abstract Level:    3 + 2 = 5
Symbolic Level:    a + b = c
Algebraic Level:   x + y = z

All represent the same fundamental concept:
COMBINING QUANTITIES TO FIND A TOTAL
```

Understanding Addition Conceptually

What Does Addition Mean?

Addition can be understood through several different models, each highlighting different aspects of the operation:

```
Models of Addition


1. Combining/Union Model:
   "I have 3 red marbles and 2 blue marbles. How many marbles total?"
   Red:    Blue:   Total:
   3 + 2 = 5

2. Adding On Model:
```

```
   "I have 3 marbles. My friend gives me 2 more. How many now?"
   Start:    Add:   Result:
   3 + 2 = 5

3. Number Line Model:
   "Start at 3, move 2 steps forward"
   0  1  2  3  4  5  6  7  8
               ↑   →→  ↑
             start +2 end
   3 + 2 = 5

4. Part-Part-Whole Model:
   "The whole is 5, one part is 3, what's the other part?"
   Whole: 5 = Part: 3 + Part: ?
   5 = 3 + 2, so ? = 2
```

The Counting On Strategy

Before learning formal addition algorithms, children naturally use counting on:

```
Counting On Strategy


Problem: 6 + 4 = ?

Method 1: Count all
    +    =
1,2,3,4,5,6,7,8,9,10

Method 2: Count on (more efficient)
Start with larger number: 6
Count on: 6... 7, 8, 9, 10
         ↑   ↑  ↑  ↑  ↑
         6  +1 +1 +1 +1
Answer: 10

Visual on number line:
0  1  2  3  4  5  6  7  8  9  10
                        ↑   →  →  →  →  ↑
                      start      +4    end
```

Addition Facts and Strategies

Basic Addition Facts (0-10)

```
Addition Facts Table (0-10)
```

```
+    0  1  2  3  4  5  6  7  8  9 10

 0   0  1  2  3  4  5  6  7  8  9 10
 1   1  2  3  4  5  6  7  8  9 10 11
 2   2  3  4  5  6  7  8  9 10 11 12
 3   3  4  5  6  7  8  9 10 11 12 13
 4   4  5  6  7  8  9 10 11 12 13 14
 5   5  6  7  8  9 10 11 12 13 14 15
 6   6  7  8  9 10 11 12 13 14 15 16
 7   7  8  9 10 11 12 13 14 15 16 17
 8   8  9 10 11 12 13 14 15 16 17 18
 9   9 10 11 12 13 14 15 16 17 18 19
10  10 11 12 13 14 15 16 17 18 19 20
```

Patterns to notice:
- Diagonal symmetry (3+5 = 5+3)
- Adding 0 doesn't change the number
- Adding 1 gives the next counting number
- Doubles: 1+1, 2+2, 3+3, etc. (diagonal)

Mental Math Strategies

Make 10 Strategy

Problem: 7 + 5 = ?

Step 1: Break apart to make 10
7 + 5 = 7 + 3 + 2 = 10 + 2 = 12

Visual:

```
     +     =      +    +   =       +
   7        5        7      3    2        10       2
```

Number line:

```
0  1  2  3  4  5  6  7  8  9  10 11 12
                           ↑   →→→  ↑   →→  ↑
                         start +3  10  +2  end
```

Doubles Plus One Strategy

Problem: 6 + 7 = ?

Think: 6 + 6 = 12, so 6 + 7 = 12 + 1 = 13

```
Visual:
6 + 7 = 6 + 6 + 1 = 12 + 1 = 13
    +     +   =         +

Near Doubles:
5 + 6 = 5 + 5 + 1 = 10 + 1 = 11
7 + 8 = 7 + 7 + 1 = 14 + 1 = 15
4 + 5 = 4 + 4 + 1 = 8 + 1 = 9

Compensation Strategy


Problem: 29 + 17 = ?

Step 1: Adjust to make easier numbers
29 + 17 = 30 + 16 = 46
(Add 1 to 29, subtract 1 from 17)

Or: 29 + 17 = 29 + 20 - 3 = 49 - 3 = 46
(Add 3 to 17, then subtract 3 from result)

Visual:
29 + 17 → 30 + 16
                +
        ↑+1                          ↑-1
                 +
```

Properties of Addition

Commutative Property

The commutative property states that changing the order of addends doesn't change the sum: a + b = b + a

```
Commutative Property Visualization


3 + 5 = 5 + 3 = 8

Visual proof:
   +     =
    +    =

Array model:
3 + 5:          5 + 3:
```

```
Both arrangements have 8 dots total!

Real-world example:
"3 boys and 5 girls" = "5 girls and 3 boys" = 8 children

This property allows flexibility in mental math:
47 + 8 = 8 + 47 (easier to count on from 47)
```

Associative Property

The associative property states that grouping doesn't affect the sum: (a + b) + c = a + (b + c)

```
Associative Property Visualization


(2 + 3) + 4 = 2 + (3 + 4) = 9

Method 1: (2 + 3) + 4
Step 1: 2 + 3 = 5
  +   =
Step 2: 5 + 4 = 9
    +    =

Method 2: 2 + (3 + 4)
Step 1: 3 + 4 = 7
   +    =
Step 2: 2 + 7 = 9
  +     =

Both methods give the same result!

Practical application:
25 + 37 + 75 = 25 + 75 + 37 = 100 + 37 = 137
(Regroup to make easier calculations)
```

Identity Property

The identity property states that adding zero to any number doesn't change it: a + 0 = a

```
Identity Property (Zero Property)


7 + 0 = 7

Visual:
     + (nothing) =

Number line:
0  1  2  3  4  5  6  7  8
                            ↑   ↑
                          start end
                        (no movement)

Real-world examples:
- "I have 5 apples, I get 0 more apples, I still have 5 apples"
- "The temperature is 20°C, it changes by 0°, it's still 20°C"

Zero is called the "additive identity" because it preserves the identity of any number when ad
```

Multi-Digit Addition

Place Value in Addition

Understanding place value is crucial for multi-digit addition:

```
Place Value Addition


Problem: 247 + 156

Expanded form:
247 = 200 + 40 + 7
156 = 100 + 50 + 6

Add by place value:
Hundreds: 200 + 100 = 300
Tens:      40 +  50 =  90
Ones:       7 +   6 =  13

Combine: 300 + 90 + 13 = 403

Visual with base-10 blocks:
247:
     200  40    7
```

```
156:
     100  50    6

Sum:
     300    90         13
   =
     300    90     3  +   (carry 10 to tens)
   =
     300   100      3
   =
     400   3 = 403
```

The Standard Algorithm

```
Standard Addition Algorithm


Problem: 247 + 156

Step-by-step:
    247
  + 156


Step 1: Add ones column
    247
  + 156

      3  (7 + 6 = 13, write 3, carry 1)
      ↑
   carry 1

Step 2: Add tens column (including carry)
    ¹247  ← carry notation
  + 156

     03  (1 + 4 + 5 = 10, write 0, carry 1)
     ↑
  carry 1

Step 3: Add hundreds column (including carry)
    ¹¹247  ← carry notation
  +  156

    403  (1 + 2 + 1 = 4)

Final answer: 403
```

```
Verification: 247 + 156 = 403
```

Addition with Multiple Carries

```
Complex Carrying Example


Problem: 789 + 456

    789
  + 456


Step 1: Ones column
9 + 6 = 15 (write 5, carry 1)
    789
  + 456

      5
      ↑
   carry 1

Step 2: Tens column
1 + 8 + 5 = 14 (write 4, carry 1)
   ¹789
  + 456

     45
     ↑
  carry 1

Step 3: Hundreds column
1 + 7 + 4 = 12 (write 2, carry 1)
  ¹¹789
  + 456

   245
   ↑
carry 1

Step 4: Thousands column
1 (from carry) = 1
  ¹¹789
  + 456

  1245
```

```
Answer: 1245

Visual verification with estimation:
789   800, 456   500
800 + 500 = 1300
1245 is close to 1300
```

Adding Decimals

Decimal Place Value

```
Decimal Addition Rules


Key rule: Align decimal points!

Problem: 12.47 + 3.8

Incorrect alignment:    Correct alignment:
  12.47                   12.47
+  3.8                  +  3.80  ← Add zero for clarity

 15.27                  16.27

Step-by-step:
  12.47
+  3.80


Hundredths: 7 + 0 = 7
Tenths: 4 + 8 = 12 (write 2, carry 1)
Ones: 1 + 2 + 3 = 6
Tens: 1 + 0 = 1

Result: 16.27

Place value visualization:
Tens Ones Decimal Tenths Hundredths
 1    2      .      4        7
 0    3      .      8        0

 1    6      .      2        7
```

Money Addition

```
Adding Money (Practical Decimals)


Problem: $12.47 + $8.95 + $3.28

Align decimal points:
  $12.47
   $8.95
+  $3.28


Step by step:
Pennies: 7 + 5 + 8 = 20 (write 0, carry 2)
Dimes: 2 + 4 + 9 + 2 = 17 (write 7, carry 1)
Dollars: 1 + 12 + 8 + 3 = 24

Result: $24.70

Verification with estimation:
$12.47  $12.50
$8.95   $9.00
$3.28   $3.25
Total   $24.75
Actual: $24.70   (very close)

Real-world context:
Receipt items:
Lunch: $12.47
Book: $8.95
Coffee: $3.28
Total: $24.70
```

Adding Fractions

Same Denominators

```
Adding Fractions with Same Denominators


Rule: Add numerators, keep denominator the same

Problem: 2/5 + 1/5

Visual representation:
```

2/5:

1/5:

Sum:
= 3/5

Calculation: 2/5 + 1/5 = (2+1)/5 = 3/5

More examples:
1/8 + 3/8 = 4/8 = 1/2
3/7 + 2/7 = 5/7
5/12 + 7/12 = 12/12 = 1

Different Denominators

Adding Fractions with Different Denominators

Problem: 1/3 + 1/4

Step 1: Find common denominator (LCD = 12)

1/3 = 4/12 (multiply by 4/4)
1/4 = 3/12 (multiply by 3/3)

Step 2: Add with same denominator
4/12 + 3/12 = 7/12

Visual proof:
1/3: 1/4:

Convert to twelfths:
4/12:

3/12:

Sum:
                        = 7/12

Word Problems and Applications

Problem-Solving Strategy

Addition Word Problem Framework

Step 1: UNDERSTAND
- What information is given?
- What are we trying to find?
- What operation is needed?

Step 2: PLAN
- Identify the numbers to add
- Estimate the answer
- Choose a solution method

Step 3: SOLVE
- Perform the calculation
- Show your work clearly
- Check your arithmetic

Step 4: CHECK
- Does the answer make sense?
- Is it close to your estimate?
- Can you verify another way?

Sample Word Problems

Problem 1: Shopping Total

"Sarah bought a book for $12.95, a pen for $3.47, and a notebook for $5.89. How much did she sp

UNDERSTAND:
- Given: $12.95, $3.47, $5.89
- Find: Total amount spent
- Operation: Addition

PLAN:
- Add all three amounts

- Estimate: $13 + $3 + $6 = $22

SOLVE:
  $12.95
   $3.47
+ $5.89

  $22.31

CHECK:
- Close to estimate of $22
- Makes sense for three items

Problem 2: Distance Traveled

"A family drove 247 miles on Monday, 189 miles on Tuesday, and 156 miles on Wednesday. What was

UNDERSTAND:
- Given: 247, 189, 156 miles
- Find: Total distance
- Operation: Addition

PLAN:
- Add all three distances
- Estimate: 250 + 190 + 160 = 600 miles

SOLVE:
   247
   189
+ 156

   592

Step-by-step:
7 + 9 + 6 = 22 (write 2, carry 2)
2 + 4 + 8 + 5 = 19 (write 9, carry 1)
1 + 2 + 1 + 1 = 5

CHECK:
- Close to estimate of 600
- Reasonable for 3-day trip

Problem 3: Time Addition

"A movie is 1 hour 45 minutes long. The previews are 15 minutes. How long will Sarah be at the

```
UNDERSTAND:
- Movie: 1 hour 45 minutes
- Previews: 15 minutes
- Find: Total time

PLAN:
- Add times together
- Convert if necessary

SOLVE:
Movie:    1 hour 45 minutes
Previews: 0 hour 15 minutes
Total:    1 hour 60 minutes = 2 hours 0 minutes

CHECK:
- 45 + 15 = 60 minutes = 1 hour
- 1 hour + 1 hour = 2 hours
```

Mental Math and Estimation

Quick Addition Strategies

```
Lightning-Fast Mental Addition


Strategy 1: Break Apart and Recombine
Problem: 47 + 28

Method:
47 + 28 = (40 + 7) + (20 + 8)
        = (40 + 20) + (7 + 8)
        = 60 + 15
        = 75

Strategy 2: Add in Steps
Problem: 56 + 39

Method:
56 + 39 = 56 + 40 - 1
        = 96 - 1
        = 95

Strategy 3: Use Friendly Numbers
Problem: 97 + 48

Method:
```

```
97 + 48 = 100 + 45
        = 145

Strategy 4: Double and Adjust
Problem: 49 + 51

Method:
49 + 51 = 50 + 50
        = 100

Visual number line for 47 + 28:
40  47  50  60  70  75
      ↑     →→→   ↑     →→→→→  ↑
    start   +3   50    +25   end
47 + 28 = 47 + 3 + 25 = 50 + 25 = 75
```

Estimation Techniques

```
Addition Estimation Methods


Method 1: Rounding
Problem: 347 + 289 + 156

Round to nearest 100:
347 → 300
289 → 300
156 → 200
Estimate: 300 + 300 + 200 = 800
Actual: 792 (very close!)

Method 2: Front-End Estimation
Problem: 4.7 + 8.2 + 3.9

Use whole number parts:
4.7 → 4
8.2 → 8
3.9 → 4
Estimate: 4 + 8 + 4 = 16
Actual: 16.8 (close!)

Method 3: Compatible Numbers
Problem: 23 + 47 + 77 + 53

Look for numbers that add to friendly sums:
23 + 77 = 100
47 + 53 = 100
```

```
Estimate: 100 + 100 = 200
Actual: 200 (exact!)

Method 4: Clustering
Problem: 89 + 91 + 88 + 92

All numbers cluster around 90:
4 × 90 = 360
Actual: 360 (exact!)
```

Common Mistakes and How to Avoid Them

Typical Addition Errors

```
Common Addition Mistakes


Mistake 1: Forgetting to Carry
Problem: 247 + 156

Wrong:          Correct:
  247             ¹247
+ 156           + 156

  393             403
  ↑               ↑
7+6=13,         7+6=13,
wrote 3,        wrote 3,
forgot carry    carried 1

Mistake 2: Misaligning Place Values
Problem: 247 + 56

Wrong:          Correct:
  247             247
+  56           +  56

  293             303
  ↑               ↑
56 not          56 aligned
aligned         properly

Mistake 3: Decimal Point Errors
Problem: 12.4 + 3.67

Wrong:          Correct:
```

```
  12.4            12.40
+  3.67         +  3.67

  15.31           16.07
  ↑               ↑
Points not      Points
aligned         aligned
```

```
Prevention Strategies:
1. Always align place values
2. Use graph paper for organization
3. Double-check carrying
4. Estimate first to catch big errors
5. Work slowly and carefully
```

Self-Checking Methods

```
Ways to Check Addition


Method 1: Reverse Order
Original: 247 + 156 = 403
Check: 156 + 247 = 403

Method 2: Estimation Check
247 + 156  250 + 150 = 400
Result: 403 (close to 400)

Method 3: Subtraction Check
If 247 + 156 = 403, then:
403 - 156 = 247
403 - 247 = 156

Method 4: Break Apart and Recombine
247 + 156 = (200 + 47) + (100 + 56)
          = (200 + 100) + (47 + 56)
          = 300 + 103
          = 403

Method 5: Digital Root Check
247: 2+4+7 = 13 → 1+3 = 4
156: 1+5+6 = 12 → 1+2 = 3
Sum: 4+3 = 7

403: 4+0+3 = 7  (matches!)
```

Real-World Applications

Financial Applications

```
Personal Finance Addition


Monthly Budget Calculation:
Rent:        $1,200.00
Groceries:     $450.75
Utilities:     $125.50
Transportation: $200.25
Entertainment:  $150.00
Savings:       $300.00

Total expenses:
  $1,200.00
    $450.75
    $125.50
    $200.25
    $150.00
  + $300.00

  $2,426.50

Bank Account Balance:
Starting balance: $2,847.63
Deposit 1:        $1,200.00
Deposit 2:          $450.00
Deposit 3:           $75.50

New balance:
  $2,847.63
  $1,200.00
    $450.00
  +   $75.50

  $4,573.13
```

Measurement Applications

```
Recipe Scaling and Measurement


Original Recipe (serves 4):
Flour: 2.5 cups
Sugar: 1.25 cups
```

```
Milk: 0.75 cups

Double the recipe (serves 8):
Flour: 2.5 + 2.5 = 5.0 cups
Sugar: 1.25 + 1.25 = 2.5 cups
Milk: 0.75 + 0.75 = 1.5 cups

Construction Project:
Board lengths needed:
Piece 1: 3 feet 8 inches
Piece 2: 2 feet 11 inches
Piece 3: 4 feet 5 inches

Total length:
  3 ft  8 in
  2 ft 11 in
+ 4 ft  5 in

  9 ft 24 in = 9 ft + 2 ft = 11 ft 0 in

(24 inches = 2 feet)
```

Time Calculations

```
Time Addition Applications


Work Schedule:
Monday:    7 hours 45 minutes
Tuesday:   8 hours 15 minutes
Wednesday: 6 hours 30 minutes
Thursday:  8 hours 0 minutes
Friday:    7 hours 30 minutes

Total weekly hours:
  7 hr 45 min
  8 hr 15 min
  6 hr 30 min
  8 hr  0 min
+ 7 hr 30 min

 36 hr 120 min = 36 hr + 2 hr = 38 hr 0 min

Travel Time Planning:
Flight 1: 2 hours 35 minutes
Layover:  1 hour 45 minutes
Flight 2: 3 hours 20 minutes
```

```
Total travel time:
  2 hr 35 min
  1 hr 45 min
+ 3 hr 20 min

  6 hr 100 min = 6 hr + 1 hr 40 min = 7 hr 40 min
```

Building Addition Fluency

Practice Strategies

```
Building Addition Fluency


Level 1: Facts to 10
Master these combinations:
0+0 through 10+0
Focus on doubles: 1+1, 2+2, 3+3, etc.
Use manipulatives and visual models

Level 2: Facts to 20
Extend to larger sums
Use make-10 strategy
Practice near doubles

Level 3: Two-digit addition
Start with no regrouping
Progress to regrouping
Use place value understanding

Level 4: Multi-digit and decimals
Apply algorithms systematically
Focus on alignment and carrying
Connect to real-world contexts

Daily Practice Routine:
1. Warm-up: 5 minutes of basic facts
2. Strategy focus: 10 minutes on one strategy
3. Problem solving: 10 minutes of word problems
4. Review: 5 minutes checking previous work
```

Games and Activities

```
Addition Games for Practice
```

```
Game 1: Addition War
- Use deck of cards (remove face cards)
- Each player draws 2 cards
- Add the numbers
- Highest sum wins all cards
- Builds fact fluency

Game 2: Target Number
- Choose target (like 15)
- Roll 3 dice
- Add any combination to get closest to target
- Develops strategic thinking

Game 3: Shopping Spree
- Use play money and price tags
- "Buy" items and calculate totals
- Practice decimal addition
- Real-world application

Game 4: Number Line Race
- Draw large number line
- Roll dice and add to current position
- First to reach end wins
- Visualizes addition concept

Activity: Addition Patterns
Look for patterns in addition:
1+9=10, 2+8=10, 3+7=10, 4+6=10, 5+5=10
11+9=20, 12+8=20, 13+7=20, 14+6=20, 15+5=20

These patterns help with mental math!
```

Conclusion

Addition is far more than a simple computational skill - it's a fundamental way of thinking about combining quantities, understanding relationships between numbers, and solving real-world problems. From the concrete act of counting objects to the abstract manipulation of algebraic expressions, addition provides the foundation for mathematical reasoning.

```
Addition: A Complete Understanding


Conceptual Understanding:
  What addition means (combining, increasing)
  Multiple models and representations
```

```
  Connection to counting and number sense

Procedural Fluency:
  Basic facts (automatic recall)
  Multi-digit algorithms
  Decimal and fraction addition

Strategic Competence:
  Mental math strategies
  Estimation techniques
  Problem-solving approaches

Adaptive Reasoning:
  Why algorithms work
  When to use different strategies
  How addition connects to other operations

Productive Disposition:
  Confidence with addition
  Willingness to persevere
  Appreciation for mathematical patterns
```

As you continue your mathematical journey, remember that addition is not just about getting the right answer - it's about understanding the underlying concepts, recognizing patterns and relationships, and applying these ideas to solve meaningful problems. Whether you're balancing a checkbook, calculating ingredients for a recipe, or working with complex mathematical expressions, the principles of addition will serve as your foundation.

The beauty of addition lies in its simplicity and power. From the earliest human civilizations counting their possessions to modern computers processing millions of calculations per second, addition remains one of our most essential mathematical tools. Master it well, and you'll have a solid foundation for all future mathematical learning.

Subtraction: Finding Differences and Taking Away

Introduction

Subtraction is the arithmetic operation that finds the difference between two numbers or determines what remains after taking away a quantity. While often viewed as the "opposite" of addition, subtraction has its own unique characteristics, applications, and conceptual depth that make it essential for mathematical understanding.

```
Subtraction: Multiple Meanings


Take Away:     8 - 3 = 5  "I had 8, took away 3, have 5 left"
Comparison:    8 - 3 = 5  "8 is 5 more than 3"
Missing Addend: 8 - 3 = 5  "3 + ? = 8, so ? = 5"
Distance:      8 - 3 = 5  "From 3 to 8 is 5 units"

All represent the same operation but different thinking!
```

Understanding Subtraction Conceptually

Models of Subtraction

```
Models of Subtraction


1. Take Away Model:
   "I have 8 cookies, I eat 3, how many are left?"
   Start:
   Remove:    (cross out)
   Left:     = 5
   8 - 3 = 5

2. Comparison Model:
   "John has 8 stickers, Mary has 3, how many more does John have?"
   John:
   Mary:
   Difference:     = 5
```

```
   8 - 3 = 5

3. Number Line Model:
   "Start at 8, move 3 steps backward"
   0  1  2  3  4  5  6  7  8  9
                      ↑   ←←←  ↑
                     end  -3  start
   8 - 3 = 5
```

The Relationship Between Addition and Subtraction

```
Addition and Subtraction: Inverse Operations


If 5 + 3 = 8, then:
8 - 3 = 5  and  8 - 5 = 3

Visual proof:
Addition:     +    =
          5     3     8

Subtraction:      -    =
             8        3     5

This inverse relationship is crucial for:
- Checking subtraction answers
- Solving equations
- Understanding algebraic concepts
- Mental math strategies
```

Basic Subtraction Facts

Mental Math Strategies for Subtraction

```
Counting Back Strategy


Problem: 9 - 4 = ?

Method: Start at 9, count back 4
9 → 8 → 7 → 6 → 5
    1   2   3   4 steps back

Answer: 5

Counting Up Strategy (More Efficient)
```

```
Problem: 9 - 4 = ?

Method: Start at 4, count up to 9
4 → 5 → 6 → 7 → 8 → 9
    1   2   3   4   5 steps up

So 9 - 4 = 5

Think Addition Strategy


Problem: 12 - 7 = ?

Think: "7 + ? = 12"
7 + 5 = 12, so 12 - 7 = 5
```

Multi-Digit Subtraction

Subtraction With Regrouping (Borrowing)

```
Regrouping in Subtraction


Problem: 523 - 187

Step 1: Ones column (3 - 7)
Can't subtract 7 from 3, need to regroup!

Borrow 1 ten from tens place:
523 becomes 5(2-1)(3+10) = 5,1,13

  513  ← regrouping notation
- 187

    6  (13 - 7 = 6)

Step 2: Tens column (1 - 8)
Can't subtract 8 from 1, need to regroup again!

  4113  ← regrouping notation
- 187

   36  (11 - 8 = 3)
```

```
Step 3: Hundreds column (4 - 1)
  4113
- 187

  336  (4 - 1 = 3)

Final answer: 336
Verification: 336 + 187 = 523
```

Word Problems and Applications

Problem-Solving Strategy

```
Subtraction Word Problem Framework


Step 1: UNDERSTAND
- Read the problem carefully
- Identify what you know (given information)
- Identify what you need to find
- Determine if subtraction is needed

Step 2: PLAN
- Identify the numbers to subtract
- Decide on minuend and subtrahend
- Estimate the answer
- Choose a solution method

Step 3: SOLVE
- Set up the subtraction problem
- Perform the calculation
- Show your work clearly
- Include units in your answer

Step 4: CHECK
- Does the answer make sense?
- Is it close to your estimate?
- Can you verify with addition?
- Does it answer the question asked?
```

Conclusion

Subtraction is a fundamental arithmetic operation that extends far beyond simple "take away" problems. Understanding subtraction deeply means grasping its multiple meanings, mastering efficient algorithms, and applying these concepts to solve real-world problems.

Master these concepts well, and you'll have a solid foundation for all future mathematical learning.

Multiplication: Repeated Addition and Scaling

Introduction

Multiplication is the arithmetic operation that represents repeated addition of the same number or scaling one quantity by another. It's one of the most powerful mathematical operations, forming the foundation for advanced concepts like area, volume, exponentials, and algebraic thinking.

From calculating the total cost of multiple items to determining the area of a rectangle, multiplication helps us solve problems involving equal groups, arrays, and proportional relationships.

```
Multiplication: Multiple Interpretations


Repeated Addition: 4 × 3 = 4 + 4 + 4 = 12
Equal Groups:      4 groups of 3 = 12
Array Model:       4 rows of 3 = 12
Area Model:        4 × 3 rectangle = 12 square units
Scaling:           4 times as much as 3 = 12

All represent the same fundamental concept!
```

Understanding Multiplication Conceptually

Models of Multiplication

```
Models of Multiplication


1. Repeated Addition Model:
   4 × 3 = "4 added 3 times" = 4 + 4 + 4 = 12
      +    +    =

2. Equal Groups Model:
   4 × 3 = "4 groups of 3" = 12
   Group 1:
   Group 2:
   Group 3:
   Group 4:
   Total: 12
```

```
3. Array Model:
   4 × 3 = "4 rows of 3" = 12
```

```
4. Area Model:
   4 × 3 = rectangle with width 4, height 3

               3


         4
   Area = 12 square units

5. Skip Counting Model:
   3 × 4 = count by 3s four times
   3, 6, 9, 12
```

The Multiplication Table

```
Multiplication Table (1-12)


×    1  2  3  4  5  6  7  8  9 10 11 12

 1   1  2  3  4  5  6  7  8  9 10 11 12
 2   2  4  6  8 10 12 14 16 18 20 22 24
 3   3  6  9 12 15 18 21 24 27 30 33 36
 4   4  8 12 16 20 24 28 32 36 40 44 48
 5   5 10 15 20 25 30 35 40 45 50 55 60
 6   6 12 18 24 30 36 42 48 54 60 66 72
 7   7 14 21 28 35 42 49 56 63 70 77 84
 8   8 16 24 32 40 48 56 64 72 80 88 96
 9   9 18 27 36 45 54 63 72 81 90 99108
10  10 20 30 40 50 60 70 80 90100110120
11  11 22 33 44 55 66 77 88 99110121132
12  12 24 36 48 60 72 84 96108120132144

Patterns to notice:
- Diagonal symmetry (commutative property)
- Squares on main diagonal: 1, 4, 9, 16, 25, 36...
- 5s column: alternates 0 and 5 endings
```

- 9s column: digits sum to 9 or multiples of 9
- 10s column: just add zero to the multiplier

Properties of Multiplication

Commutative Property

Commutative Property: a × b = b × a

3 × 4 = 4 × 3 = 12

Array visualization:
3 × 4: 4 × 3:

Both arrays contain 12 dots!

This property allows flexibility:
- 25 × 4 = 4 × 25 = 100 (easier to calculate)
- 8 × 125 = 125 × 8 = 1000

Real-world example:
"3 boxes with 4 items each" = "4 items in each of 3 boxes"
Both equal 12 items total.

Associative Property

Associative Property: (a × b) × c = a × (b × c)

(2 × 3) × 4 = 2 × (3 × 4) = 24

Method 1: (2 × 3) × 4
Step 1: 2 × 3 = 6
Step 2: 6 × 4 = 24

Method 2: 2 × (3 × 4)
Step 1: 3 × 4 = 12
Step 2: 2 × 12 = 24

Both methods give the same result!

```
Practical application:
Calculate 5 × 7 × 2:
Method 1: (5 × 7) × 2 = 35 × 2 = 70
Method 2: 5 × (7 × 2) = 5 × 14 = 70
Method 3: (5 × 2) × 7 = 10 × 7 = 70 (easiest!)
```

Distributive Property

```
Distributive Property: a × (b + c) = (a × b) + (a × c)


6 × (4 + 3) = (6 × 4) + (6 × 3) = 24 + 18 = 42

Visual proof with area model:
6 × (4 + 3) = 6 × 7 = 42


              6









    4       3

Left rectangle: 6 × 4 = 24
Right rectangle: 6 × 3 = 18
Total: 24 + 18 = 42

This property is essential for:
- Mental math: 7 × 19 = 7 × (20 - 1) = 140 - 7 = 133
- Algebra: expanding expressions
- Multi-digit multiplication algorithms
```

Multi-Digit Multiplication

Standard Algorithm

```
Multi-Digit Multiplication Algorithm


Problem: 247 × 36

Step 1: Multiply by ones digit (6)
   247
```

```
×   36

  1482  ← 247 × 6

Breakdown:
7 × 6 = 42 (write 2, carry 4)
4 × 6 = 24, plus carry 4 = 28 (write 8, carry 2)
2 × 6 = 12, plus carry 2 = 14 (write 14)

Step 2: Multiply by tens digit (30)
   247
×   36

  1482  ← 247 × 6
  7410  ← 247 × 30 (note the zero placeholder)

Step 3: Add partial products
   247
×   36

  1482
+ 7410

  8892

Verification using estimation:
247  250, 36  40
250 × 40 = 10,000
8,892 is close to 10,000
```

Area Model for Multi-Digit Multiplication

```
Area Model: 23 × 47


Break into place values:
23 = 20 + 3
47 = 40 + 7

Create rectangle divided into four parts:



 40     20 × 40     3×40
         = 800     =120

  7     20 × 7      3×7
```

```
         = 140     =21

          20          3

Total area = 800 + 120 + 140 + 21 = 1,081

So 23 × 47 = 1,081

This method shows why the standard algorithm works!
```

Multiplication with Decimals

Decimal Multiplication Rules

```
Multiplying Decimals


Rule: Multiply as if whole numbers, then place decimal point

Problem: 2.4 × 1.3

Step 1: Ignore decimal points, multiply whole numbers
24 × 13 = 312

Step 2: Count decimal places in factors
2.4 has 1 decimal place
1.3 has 1 decimal place
Total: 2 decimal places

Step 3: Place decimal point in product
312 → 3.12 (2 places from right)

Therefore: 2.4 × 1.3 = 3.12

Visual verification with area model:


    2 × 1      2×0.3  1
    = 2        =0.6

   0.4 × 1     0.4×   0.3
    = 0.4      0.3
               =0.12

       2         0.4
```

```
Total: 2 + 0.6 + 0.4 + 0.12 = 3.12
```

Money Multiplication

```
Multiplying Money


Problem: 5 items cost $3.47 each. What's the total?

5 × $3.47 = ?

Method 1: Standard algorithm
  $3.47
×     5

 $17.35

Method 2: Break apart
5 × $3.47 = 5 × ($3.00 + $0.47)
          = (5 × $3.00) + (5 × $0.47)
          = $15.00 + $2.35
          = $17.35

Method 3: Mental math
5 × $3.47 = 5 × $3.50 - 5 × $0.03
          = $17.50 - $0.15
          = $17.35

All methods give the same answer: $17.35
```

Multiplication with Fractions

Multiplying Fractions

```
Fraction Multiplication Rule


Rule: Multiply numerators, multiply denominators
a/b × c/d = (a×c)/(b×d)

Problem: 2/3 × 3/4

Solution: (2×3)/(3×4) = 6/12 = 1/2

Visual representation:
2/3 of a whole:
```

```
3/4 of that 2/3: Take 3/4 of the shaded part

     →
             (divide each shaded part into 4)


Result: 6 out of 12 parts = 6/12 = 1/2

Real-world example:
"2/3 of the students are girls, and 3/4 of the girls play sports"
2/3 × 3/4 = 1/2 of all students are girls who play sports
```

Mixed Numbers

```
Multiplying Mixed Numbers


Problem: 2 1/3 × 1 1/2

Method 1: Convert to improper fractions
2 1/3 = 7/3
1 1/2 = 3/2

7/3 × 3/2 = 21/6 = 3 1/2

Method 2: Distributive property
2 1/3 × 1 1/2 = (2 + 1/3) × (1 + 1/2)
              = 2×1 + 2×1/2 + 1/3×1 + 1/3×1/2
              = 2 + 1 + 1/3 + 1/6
              = 3 + 2/6 + 1/6
              = 3 + 3/6
              = 3 1/2

Both methods give 3 1/2
```

Mental Math Strategies

Quick Multiplication Tricks

```
Mental Multiplication Strategies


Multiplying by 10, 100, 1000:
```

```
Just add zeros!
47 × 10 = 470
47 × 100 = 4,700
47 × 1000 = 47,000

Multiplying by 5:
Multiply by 10, then divide by 2
28 × 5 = (28 × 10) ÷ 2 = 280 ÷ 2 = 140

Multiplying by 9:
Multiply by 10, then subtract original number
37 × 9 = (37 × 10) - 37 = 370 - 37 = 333

Multiplying by 11 (two-digit numbers):
Add the digits and put the sum in the middle
23 × 11: 2_(2+3)_3 = 253
47 × 11: 4_(4+7)_7 = 4_11_7 = 517 (carry the 1)

Squares ending in 5:
25² = (2 × 3) followed by 25 = 625
35² = (3 × 4) followed by 25 = 1225
45² = (4 × 5) followed by 25 = 2025

Doubling and Halving:
16 × 25 = 32 × 12.5 = 8 × 50 = 4 × 100 = 400
```

Estimation in Multiplication

```
Multiplication Estimation


Method 1: Rounding
347 × 28  ≈ 350 × 30 = 10,500
Actual: 9,716 (reasonably close)

Method 2: Front-end estimation
4.7 × 8.2  ≈ 4 × 8 = 32
Actual: 38.54 (in the right ballpark)

Method 3: Compatible numbers
19 × 52  ≈ 20 × 50 = 1,000
Actual: 988 (very close)

Method 4: One exact, one rounded
25 × 47 = 25 × 50 - 25 × 3 = 1,250 - 75 = 1,175
Actual: 1,175 (exact!)
```

```
When to estimate:
- Quick mental calculations
- Checking reasonableness of answers
- Planning and budgeting
- When exact precision isn't needed
```

Word Problems and Applications

Types of Multiplication Problems

```
Multiplication Problem Types


Type 1: Equal Groups
"There are 6 boxes with 8 pencils in each box. How many pencils total?"
6 × 8 = 48 pencils

Type 2: Array/Area
"A garden is 12 feet long and 8 feet wide. What's the area?"
12 × 8 = 96 square feet

Type 3: Scaling/Rate
"A car travels 65 miles per hour for 4 hours. How far does it go?"
65 × 4 = 260 miles

Type 4: Combinations
"There are 4 shirts and 3 pairs of pants. How many different outfits?"
4 × 3 = 12 different outfits

Type 5: Multiplicative Comparison
"Sarah has 3 times as many stickers as Tom. Tom has 15 stickers. How many does Sarah have?"
3 × 15 = 45 stickers
```

Problem-Solving Framework

```
Multiplication Word Problem Strategy


Step 1: UNDERSTAND
- What information is given?
- What are we trying to find?
- Is this a multiplication situation?
- What are the units?

Step 2: PLAN
- Identify the factors to multiply
```

```
- Estimate the answer
- Choose a calculation method
- Consider if the answer should be larger or smaller

Step 3: SOLVE
- Set up the multiplication
- Perform the calculation
- Include appropriate units
- Check your arithmetic

Step 4: CHECK
- Is the answer reasonable?
- Does it match your estimate?
- Can you verify with division?
- Does it make sense in context?
```

Real-World Applications

Area and Perimeter

```
Geometric Applications


Room Carpeting:
Room dimensions: 12 feet × 15 feet
Carpet needed: 12 × 15 = 180 square feet

Cost calculation:
Carpet costs $8.50 per square foot
Total cost: 180 × $8.50 = $1,530

Fencing a Yard:
Rectangular yard: 25 feet × 40 feet
Perimeter = 2 × (25 + 40) = 2 × 65 = 130 feet
Fence cost: 130 × $12 per foot = $1,560

Garden Planning:
Square garden plots: 8 feet × 8 feet each
Number of plots: 6
Total garden area: 6 × (8 × 8) = 6 × 64 = 384 square feet
```

Business and Finance

```
Business Applications
```

```
Payroll Calculation:
Employee works 40 hours per week
Hourly wage: $18.50
Weekly pay: 40 × $18.50 = $740

Monthly pay: $740 × 4 = $2,960
Annual pay: $2,960 × 12 = $35,520

Inventory Management:
Cases of products: 24
Items per case: 36
Total items: 24 × 36 = 864

Selling price per item: $4.75
Total revenue: 864 × $4.75 = $4,104

Bulk Purchasing:
Regular price: $3.25 per item
Bulk discount: Buy 50, get 15% off
Bulk price: $3.25 × 0.85 = $2.76 per item
Total cost: 50 × $2.76 = $138
```

Cooking and Recipes

```
Recipe Scaling


Original Recipe (serves 4):
- 2 cups flour
- 1.5 cups sugar
- 0.75 cups milk
- 3 eggs

Scale for 12 people:
Scaling factor: 12 ÷ 4 = 3

New amounts:
- Flour: 2 × 3 = 6 cups
- Sugar: 1.5 × 3 = 4.5 cups
- Milk: 0.75 × 3 = 2.25 cups
- Eggs: 3 × 3 = 9 eggs

Cost Calculation:
Flour: 6 cups × $0.25 per cup = $1.50
Sugar: 4.5 cups × $0.40 per cup = $1.80
Milk: 2.25 cups × $0.30 per cup = $0.68
Eggs: 9 eggs × $0.20 per egg = $1.80
```

```
Total ingredient cost: $5.78
```

Common Mistakes and Prevention

Typical Multiplication Errors

```
Common Multiplication Mistakes


Mistake 1: Forgetting zeros in partial products
   23
×  45

  115  ← 23 × 5
   92  ← Wrong! Should be 920 (23 × 40)

  207  ← Wrong total

Correct:
   23
×  45

  115  ← 23 × 5
  920  ← 23 × 40 (note the zero!)

 1035

Mistake 2: Decimal point placement
2.3 × 4.5 = 1035 ← Wrong! (treated as whole numbers)
Correct: 2.3 × 4.5 = 10.35 (2 decimal places total)

Mistake 3: Sign errors with negative numbers
(-3) × (-4) = -12 ← Wrong!
Correct: (-3) × (-4) = +12 (negative × negative = positive)

Prevention strategies:
- Always estimate first
- Check with division (if a × b = c, then c ÷ b = a)
- Use different methods to verify
- Pay attention to decimal places
- Remember sign rules for negative numbers
```

Building Multiplication Fluency

Practice Progression

```
Multiplication Fluency Development


Stage 1: Conceptual Understanding (Grades 2-3)
- Equal groups and arrays
- Skip counting
- Repeated addition
- Visual models

Stage 2: Basic Facts (Grades 3-4)
- Facts 0-5: Focus on patterns
- Facts 6-10: Use strategies
- Automatic recall goal
- Daily practice

Stage 3: Multi-digit (Grades 4-5)
- Two-digit × one-digit
- Two-digit × two-digit
- Decimal multiplication
- Real-world applications

Stage 4: Advanced Applications (Grades 5+)
- Fraction multiplication
- Percent problems
- Area and volume
- Algebraic thinking

Practice Schedule:
Week 1-2: 0s, 1s, 2s, 5s, 10s (easy facts)
Week 3-4: 3s, 4s, 6s (building up)
Week 5-6: 7s, 8s, 9s (challenging facts)
Week 7-8: Mixed practice and speed
Week 9+: Multi-digit and applications
```

Conclusion

Multiplication is a powerful arithmetic operation that extends far beyond repeated addition. It represents scaling, area calculation, rate problems, and forms the foundation for advanced mathematical concepts including algebra, geometry, and calculus.

```
Multiplication: Complete Understanding
```

```
Conceptual Understanding:
  Multiple models (groups, arrays, area, scaling)
  Connection to addition and division
  Properties and their applications

Procedural Fluency:
  Basic facts (automatic recall)
  Multi-digit algorithms
  Decimal and fraction multiplication

Strategic Competence:
  Mental math strategies
  Estimation techniques
  Problem-solving approaches

Adaptive Reasoning:
  Why algorithms work
  When to use different methods
  Connections to other operations

Productive Disposition:
  Confidence with multiplication
  Appreciation for patterns
  Persistence in problem-solving
```

Master multiplication well, and you'll have a powerful tool for mathematical thinking that will serve you throughout your educational journey and beyond. Whether calculating areas, solving proportions, or working with algebraic expressions, multiplication provides essential computational power for mathematical reasoning.

Division: Sharing Equally and Finding How Many Groups

Introduction

Division is the arithmetic operation that determines how many times one number is contained in another, or how to distribute a quantity into equal parts. As the inverse of multiplication, division is essential for solving problems involving equal sharing, rate calculations, and proportional reasoning.

From splitting a pizza among friends to calculating average speeds, division helps us understand relationships between quantities and solve problems involving equal distribution and grouping.

```
Division: Multiple Interpretations


Sharing: 12 ÷ 3 = 4  "Share 12 items among 3 people, each gets 4"
Grouping: 12 ÷ 3 = 4  "How many groups of 3 in 12? Answer: 4 groups"
Rate: 12 ÷ 3 = 4  "12 items in 3 hours = 4 items per hour"
Inverse: 12 ÷ 3 = 4  "3 × ? = 12, so ? = 4"

All represent the same operation but different thinking!
```

Understanding Division Conceptually

Models of Division

```
Models of Division


1. Sharing (Partitive) Model:
   "Share 12 cookies equally among 3 children"
          →
   Each child gets 4 cookies
   12 ÷ 3 = 4

2. Grouping (Quotitive) Model:
   "How many groups of 3 can you make from 12 items?"
          →   |   |   |
```

```
   You can make 4 groups
   12 ÷ 3 = 4

3. Array Model:
   "12 items arranged in 3 equal rows"

          ← 3 rows of 4

   12 ÷ 3 = 4 items per row

4. Area Model:
   "Rectangle with area 12, width 3, find length"

               3


         4
   12 ÷ 3 = 4

5. Number Line Model:
   "How many jumps of 3 to reach 12?"
   0  3  6  9  12
       ↑   ↑   ↑   ↑
       1   2   3   4 jumps
   12 ÷ 3 = 4
```

The Relationship Between Multiplication and Division

```
Multiplication and Division: Inverse Operations


If 4 × 3 = 12, then:
12 ÷ 3 = 4  and  12 ÷ 4 = 3

Fact Family:
4 × 3 = 12    3 × 4 = 12
12 ÷ 3 = 4    12 ÷ 4 = 3

Visual proof:
Multiplication: 4 groups of 3 =           = 12
Division: 12 shared into 4 groups =           (3 each)

This inverse relationship is crucial for:
- Checking division answers
- Solving equations
- Understanding fractions
- Mental math strategies
```

Basic Division Facts

Division Facts Table

```
Division Facts (Related to Multiplication Table)


If you know: 6 × 7 = 42
Then you know: 42 ÷ 6 = 7 and 42 ÷ 7 = 6

Key division facts to memorize:
÷1: Any number ÷ 1 = that number (8 ÷ 1 = 8)
÷2: Half the number (16 ÷ 2 = 8)
÷5: Think "how many 5s?" (35 ÷ 5 = 7)
÷10: Remove the last zero (80 ÷ 10 = 8)

Special cases:
0 ÷ any number = 0 (0 ÷ 5 = 0)
Any number ÷ itself = 1 (7 ÷ 7 = 1)
Division by 0 is undefined (5 ÷ 0 = undefined)

Common division facts:
12 ÷ 3 = 4    15 ÷ 3 = 5    18 ÷ 3 = 6
16 ÷ 4 = 4    20 ÷ 4 = 5    24 ÷ 4 = 6
25 ÷ 5 = 5    30 ÷ 5 = 6    35 ÷ 5 = 7
36 ÷ 6 = 6    42 ÷ 6 = 7    48 ÷ 6 = 8
```

Mental Division Strategies

```
Mental Division Strategies


Strategy 1: Think Multiplication
Problem: 56 ÷ 8 = ?
Think: "8 × ? = 56"
8 × 7 = 56, so 56 ÷ 8 = 7

Strategy 2: Use Known Facts
Problem: 72 ÷ 9 = ?
Know: 9 × 8 = 72
So: 72 ÷ 9 = 8

Strategy 3: Break Apart (Distributive Property)
Problem: 84 ÷ 4 = ?
84 = 80 + 4
84 ÷ 4 = (80 ÷ 4) + (4 ÷ 4) = 20 + 1 = 21
```

```
Strategy 4: Use Doubling/Halving
Problem: 48 ÷ 6 = ?
48 ÷ 6 = (48 ÷ 2) ÷ 3 = 24 ÷ 3 = 8

Strategy 5: Estimate and Adjust
Problem: 91 ÷ 7 = ?
Estimate: 90 ÷ 7  13 (since 7 × 13 = 91)
Check: 7 × 13 = 91
```

Long Division Algorithm

Step-by-Step Long Division

```
Long Division: 847 ÷ 7


Step 1: Set up the problem

    7  847

Step 2: Divide hundreds
8 ÷ 7 = 1 remainder 1
      1

  7  847
     7↓    ← 7 × 1 = 7

     14    ← bring down the 4

Step 3: Divide tens
14 ÷ 7 = 2 remainder 0
      12

  7  847
     7↓

     14
     14    ← 7 × 2 = 14

      07   ← bring down the 7

Step 4: Divide ones
7 ÷ 7 = 1 remainder 0
      121

  7  847
```

```
      7↓

      14
      14

       07
        7   ← 7 × 1 = 7

        0

Answer: 847 ÷ 7 = 121

Verification: 121 × 7 = 847
```

Division with Remainders

```
Division with Remainders


Problem: 865 ÷ 7

      123 R 4

  7   865
      7↓

      16
      14

       25
       21

        4  ← Remainder

Answer: 865 ÷ 7 = 123 R 4

Check: (123 × 7) + 4 = 861 + 4 = 865

Interpreting remainders:
- In sharing: "123 items each, with 4 left over"
- As fraction: 865 ÷ 7 = 123 4/7
- As decimal: 865 ÷ 7 = 123.571428...
- In context: "123 full groups, 4 items remaining"

Real-world example:
"25 students, 4 per table. How many tables needed?"
25 ÷ 4 = 6 R 1
```

```
Need 7 tables (6 full tables + 1 more for the remaining student)
```

Division by Two-Digit Numbers

```
Two-Digit Division: 1,248 ÷ 24


Step 1: Estimate first digit
124 ÷ 24   5 (since 24 × 5 = 120)

       5

 24   1248
     120↓   ← 24 × 5 = 120

       048 ← bring down 8

Step 2: Continue division
48 ÷ 24 = 2

       52

 24   1248
     120↓

       048
        48  ← 24 × 2 = 48

         0

Answer: 1,248 ÷ 24 = 52

Estimation check:
1,248   1,200, 24   25
1,200 ÷ 25 = 48 (close to 52)
```

Division with Decimals

Dividing Decimals by Whole Numbers

```
Decimal Division: 12.6 ÷ 3


Method: Divide as usual, keep decimal point aligned

      4.2
```

```
  3   12.6
      12↓

       06
        6

        0
```

Answer: 12.6 ÷ 3 = 4.2

Visual check with money:
$12.60 ÷ 3 people = $4.20 each

Step-by-step:
1. 12 ÷ 3 = 4 (place 4 in ones place)
2. 6 tenths ÷ 3 = 2 tenths (place 2 in tenths place)
3. Result: 4.2

Dividing by Decimals

Division by Decimals: 8.4 ÷ 2.1

Rule: Move decimal points to make divisor a whole number

Step 1: Move decimal point in divisor to make it whole
2.1 → 21 (moved 1 place right)

Step 2: Move decimal point same number of places in dividend
8.4 → 84 (moved 1 place right)

Step 3: Divide whole numbers
84 ÷ 21 = 4

Therefore: 8.4 ÷ 2.1 = 4

Another example: 15.6 ÷ 0.12
Step 1: 0.12 → 12 (moved 2 places right)
Step 2: 15.6 → 1560 (moved 2 places right)
Step 3: 1560 ÷ 12 = 130

Visual representation:
8.4 ÷ 2.1 = "How many 2.1s in 8.4?"
2.1 + 2.1 + 2.1 + 2.1 = 8.4
So there are 4 groups of 2.1 in 8.4

Division with Fractions

Dividing by Fractions

Fraction Division Rule: "Multiply by the Reciprocal"

Rule: a/b ÷ c/d = a/b × d/c

Problem: 3/4 ÷ 1/2

Solution: 3/4 ÷ 1/2 = 3/4 × 2/1 = 6/4 = 3/2 = 1 1/2

Why does this work?
Visual explanation with pizza:

3/4 of a pizza:

Divide into pieces of size 1/2:
Each 1/2 piece:

How many 1/2 pieces fit in 3/4?
→ +
(1/2 piece left)

Answer: 1 1/2 pieces of size 1/2

Alternative thinking:
"How many halves in three-fourths?"
3/4 ÷ 1/2 = 3/4 × 2/1 = 6/4 = 1 1/2

Mixed Numbers in Division

Dividing Mixed Numbers

Problem: 2 1/3 ÷ 1 1/6

Step 1: Convert to improper fractions
2 1/3 = 7/3
1 1/6 = 7/6

```
Step 2: Multiply by reciprocal
7/3 ÷ 7/6 = 7/3 × 6/7 = 42/21 = 2

Answer: 2 1/3 ÷ 1 1/6 = 2

Real-world interpretation:
"If each serving is 1 1/6 cups, how many servings in 2 1/3 cups?"
Answer: 2 servings

Verification:
2 × 1 1/6 = 2 × 7/6 = 14/6 = 2 2/6 = 2 1/3
```

Word Problems and Applications

Types of Division Problems

```
Division Problem Types


Type 1: Equal Sharing
"24 stickers shared equally among 6 children. How many each?"
24 ÷ 6 = 4 stickers per child

Type 2: Equal Grouping
"24 stickers, 4 per pack. How many packs?"
24 ÷ 4 = 6 packs

Type 3: Rate Problems
"360 miles in 6 hours. What's the average speed?"
360 ÷ 6 = 60 miles per hour

Type 4: Comparison
"Sarah has 48 stickers, 3 times as many as Tom. How many does Tom have?"
48 ÷ 3 = 16 stickers (Tom has 16)

Type 5: Area Problems
"Garden area is 72 square feet, width is 8 feet. What's the length?"
72 ÷ 8 = 9 feet long

Type 6: Unit Rate
"$15 for 3 pounds of apples. What's the price per pound?"
$15 ÷ 3 = $5 per pound
```

Problem-Solving Framework

```
Division Word Problem Strategy
```

```
Step 1: UNDERSTAND
- What information is given?
- What are we trying to find?
- Is this a sharing or grouping situation?
- What are the units?

Step 2: PLAN
- Identify dividend and divisor
- Estimate the answer
- Decide how to handle remainders
- Choose calculation method

Step 3: SOLVE
- Set up the division problem
- Perform the calculation
- Interpret the remainder appropriately
- Include correct units

Step 4: CHECK
- Is the answer reasonable?
- Does it match your estimate?
- Can you verify with multiplication?
- Does it make sense in context?
```

Sample Problems

```
Problem 1: Party Planning


"72 people are coming to a party. Each table seats 8 people. How many tables are needed?"

UNDERSTAND:
- Total people: 72
- People per table: 8
- Find: Number of tables needed

PLAN:
- Divide total by capacity per table
- Estimate: 70 ÷ 8  9 tables

SOLVE:
72 ÷ 8 = 9 tables exactly

CHECK:
- 9 × 8 = 72
```

- Makes sense for party size

Problem 2: Baking Cookies

"A recipe makes 36 cookies. How many batches needed for 150 cookies?"

UNDERSTAND:
- Cookies per batch: 36
- Total needed: 150
- Find: Number of batches

PLAN:
- Divide total needed by batch size
- May need to round up

SOLVE:
150 ÷ 36 = 4.17... = 4 R 6

Since we need whole batches: 5 batches
(4 full batches + 1 partial batch)

CHECK:
- 4 × 36 = 144 (6 cookies short)
- 5 × 36 = 180 (30 extra cookies)
- Need 5 batches to have enough

Problem 3: Speed Calculation

"A train travels 420 miles in 3.5 hours. What's the average speed?"

UNDERSTAND:
- Distance: 420 miles
- Time: 3.5 hours
- Find: Speed (miles per hour)

PLAN:
- Speed = Distance ÷ Time
- Estimate: 420 ÷ 4 105 mph

SOLVE:
420 ÷ 3.5 = 420 ÷ (7/2) = 420 × (2/7) = 840/7 = 120 mph

CHECK:
- 120 × 3.5 = 420
- Close to estimate
- Reasonable train speed

Mental Math and Estimation

Mental Division Strategies

```
Mental Division Techniques


Strategy 1: Compatible Numbers
Problem: 240 ÷ 8
Think: 240 = 24 × 10
24 ÷ 8 = 3, so 240 ÷ 8 = 30

Strategy 2: Break Apart
Problem: 156 ÷ 12
156 = 120 + 36
156 ÷ 12 = (120 ÷ 12) + (36 ÷ 12) = 10 + 3 = 13

Strategy 3: Use Multiplication Facts
Problem: 144 ÷ 16
Think: "16 × ? = 144"
16 × 9 = 144, so 144 ÷ 16 = 9

Strategy 4: Halving
Problem: 84 ÷ 4
84 ÷ 4 = (84 ÷ 2) ÷ 2 = 42 ÷ 2 = 21

Strategy 5: Adjust and Compensate
Problem: 195 ÷ 15
195 ÷ 15 = (195 ÷ 15) = (200 - 5) ÷ 15
  200 ÷ 15 - 5 ÷ 15   13.33 - 0.33 = 13
```

Division Estimation

```
Division Estimation Methods


Method 1: Round Both Numbers
Problem: 847 ÷ 23
Round: 850 ÷ 25 = 34
Actual: 36.8... (reasonably close)

Method 2: Compatible Numbers
Problem: 376 ÷ 19
Think: 380 ÷ 20 = 19
Actual: 19.8... (very close)

Method 3: Use Benchmark Divisors
```

```
Problem: 432 ÷ 18
Think: 432 ÷ 20 = 21.6
Actual: 24 (in the right range)

Method 4: Front-End Estimation
Problem: 5,847 ÷ 67
Think: 5,800 ÷ 70  58 ÷ 7  8 × 10 = 80
Actual: 87.3... (reasonable estimate)

When to estimate:
- Quick mental calculations
- Checking reasonableness
- Planning and budgeting
- When exact precision isn't critical
```

Real-World Applications

Business and Finance

```
Business Applications


Unit Cost Calculation:
Total cost: $240 for 15 items
Cost per item: $240 ÷ 15 = $16 per item

Profit Margin:
Revenue: $50,000
Expenses: $35,000
Profit: $15,000
Items sold: 500
Profit per item: $15,000 ÷ 500 = $30 per item

Employee Productivity:
Total output: 1,440 units
Number of workers: 12
Hours worked: 8
Units per worker per hour: 1,440 ÷ (12 × 8) = 1,440 ÷ 96 = 15 units

Budget Allocation:
Monthly budget: $3,600
Number of weeks: 4
Weekly budget: $3,600 ÷ 4 = $900 per week
Daily budget: $900 ÷ 7 = $128.57 per day
```

Cooking and Measurements

```
Cooking Applications


Recipe Scaling Down:
Original recipe serves 12, need for 4 people
Scaling factor: 4 ÷ 12 = 1/3

Original ingredients:
- 6 cups flour → 6 ÷ 3 = 2 cups
- 4.5 cups sugar → 4.5 ÷ 3 = 1.5 cups
- 9 eggs → 9 ÷ 3 = 3 eggs

Portion Control:
Whole cake serves 16 people
Each person gets: 1 ÷ 16 = 1/16 of the cake

If cake weighs 4 pounds:
Each portion: 4 ÷ 16 = 0.25 pounds = 4 ounces

Unit Conversion:
Recipe calls for 2.5 cups, only have tablespoons
1 cup = 16 tablespoons
2.5 cups = 2.5 × 16 = 40 tablespoons

Cost per Serving:
Total ingredient cost: $12.50
Recipe serves 8 people
Cost per serving: $12.50 ÷ 8 = $1.56 per person
```

Travel and Transportation

```
Travel Applications


Fuel Efficiency:
Distance traveled: 420 miles
Gas used: 15 gallons
Miles per gallon: 420 ÷ 15 = 28 mpg

Trip Planning:
Total distance: 1,200 miles
Driving speed: 60 mph
Driving time: 1,200 ÷ 60 = 20 hours

With breaks every 2 hours:
```

```
Number of breaks: 20 ÷ 2 = 10 breaks
Break time: 10 × 15 minutes = 150 minutes = 2.5 hours
Total trip time: 20 + 2.5 = 22.5 hours

Cost Sharing:
Total trip cost: $480
Number of people: 6
Cost per person: $480 ÷ 6 = $80 each

Hotel cost: $120 per night × 3 nights = $360
Hotel cost per person: $360 ÷ 6 = $60 each
```

Common Mistakes and Prevention

Typical Division Errors

```
Common Division Mistakes


Mistake 1: Incorrect placement in long division
Problem: 156 ÷ 12

Wrong:
      103  ← Wrong! 0 shouldn't be in tens place

12   156
    12

     36
     36

      0

Correct:
      13   ← Correct placement

12   156
    12

     36
     36

      0

Mistake 2: Forgetting to bring down digits
Problem: 248 ÷ 4
```

```
Wrong: Only dividing 24, forgetting the 8

Correct: Systematically bring down each digit

Mistake 3: Mishandling remainders
Problem: "How many 4-person tables for 23 people?"
23 ÷ 4 = 5 R 3

Wrong interpretation: "5 tables" (3 people left standing)
Correct interpretation: "6 tables needed" (to seat everyone)

Mistake 4: Decimal point errors
Problem: 12.6 ÷ 3

Wrong: 126 ÷ 3 = 42 (forgot decimal point)
Correct: 12.6 ÷ 3 = 4.2

Prevention Strategies:
- Always check with multiplication
- Estimate before calculating
- Pay attention to decimal points
- Consider context for remainders
- Work systematically through long division
```

Building Division Fluency

Practice Progression

```
Division Fluency Development


Stage 1: Conceptual Foundation
- Sharing and grouping models
- Connection to multiplication
- Visual representations
- Simple division facts

Stage 2: Basic Facts Mastery
- Division facts related to multiplication tables
- Mental math strategies
- Fact families
- Automatic recall

Stage 3: Algorithm Development
- Single-digit divisors
```

- Long division process
- Handling remainders
- Checking answers

Stage 4: Advanced Applications
- Multi-digit divisors
- Decimal division
- Fraction division
- Real-world problem solving

Practice Sequence:
Week 1-2: Division facts 0-5 (easy divisors)
Week 3-4: Division facts 6-10 (harder divisors)
Week 5-6: Two-digit dividends, one-digit divisors
Week 7-8: Three-digit dividends, one-digit divisors
Week 9-10: Two-digit divisors
Week 11+: Decimals, fractions, applications

Games and Activities

Division Games and Practice

Game 1: Division Bingo
- Create bingo cards with quotients
- Call out division problems
- Students solve and mark answers
- Builds fact fluency

Game 2: Remainder Race
- Roll dice to create division problems
- Calculate quotient and remainder
- Points for correct answers
- Makes remainders fun

Game 3: Division War
- Use cards to create division problems
- Higher quotient wins the round
- Develops quick mental division
- Competitive practice

Game 4: Real-World Division
- Use grocery store flyers
- Calculate unit prices
- Compare deals
- Practical application

```
Activity: Division Patterns
Explore patterns in division:
100 ÷ 10 = 10, 100 ÷ 5 = 20, 100 ÷ 4 = 25...
Notice: As divisor decreases, quotient increases

These patterns help with estimation and mental math!
```

Conclusion

Division is a fundamental arithmetic operation that extends far beyond simple sharing problems. It encompasses rate calculations, unit conversions, proportional reasoning, and forms the foundation for advanced mathematical concepts including fractions, ratios, and algebraic thinking.

```
Division: Complete Understanding


Conceptual Understanding:
  Multiple models (sharing, grouping, rate)
  Relationship to multiplication (inverse operations)
  Connection to fractions and ratios

Procedural Fluency:
  Basic division facts (automatic recall)
  Long division algorithm
  Decimal and fraction division

Strategic Competence:
  Mental math strategies
  Estimation techniques
  Problem-solving approaches
  Remainder interpretation

Adaptive Reasoning:
  Why algorithms work
  When to use different methods
  Connections to other operations

Productive Disposition:
  Confidence with division
  Persistence through complex problems
  Appreciation for mathematical relationships
```

Master division well, and you'll have a powerful tool for mathematical reasoning that will serve you throughout your educational journey and beyond. Whether calculating rates, solving proportions, or working with algebraic expressions, division provides essential computational power for mathematical thinking.

The beauty of division lies in its versatility - it helps us understand how quantities relate to each other, solve problems involving equal distribution, and make sense of rates and ratios in the world around us. From calculating tips at restaurants to determining fuel efficiency, division is an indispensable life skill wrapped in mathematical elegance.

Fractions: Parts of a Whole

Introduction

Fractions represent parts of a whole or parts of a group. They are essential for understanding division, ratios, proportions, and many real-world situations where we need to work with quantities that aren't whole numbers.

From sharing a pizza among friends to measuring ingredients for cooking, fractions help us express and work with partial quantities in precise and meaningful ways.

```
Fraction Fundamentals


A fraction has two parts:
    3  ← Numerator (how many parts we have)

    4  ← Denominator (how many equal parts in the whole)

This represents "3 out of 4 equal parts"
```

Understanding Fractions Conceptually

Visual Models of Fractions

```
Fraction Models


1. Area Model (Pizza/Pie):
   3/4 = three-fourths

              3 pieces shaded
            out of 4 total



2. Linear Model (Number Line):
   0  ¼  ½  ¾  1
            ↑
           3/4
```

```
3. Set Model (Groups):
   3/4 of 8 objects = 6 objects
         (6 out of 8 are shaded)

4. Discrete Model:
   3/4 means 3 out of every 4 items
             (pattern repeats)
```

Types of Fractions

```
Fraction Classifications


Proper Fractions (numerator < denominator):
3/4, 2/5, 7/8
- Less than 1 whole
- Represent part of a whole

Improper Fractions (numerator  denominator):
5/4, 7/3, 9/9
- Equal to or greater than 1 whole
- Can be converted to mixed numbers

Mixed Numbers:
1¾, 2 , 5½
- Whole number + proper fraction
- Greater than 1

Unit Fractions:
1/2, 1/3, 1/4, 1/5...
- Numerator is 1
- Building blocks for other fractions

Equivalent Fractions:
1/2 = 2/4 = 3/6 = 4/8...
- Same value, different representation
- Multiply/divide numerator and denominator by same number
```

Equivalent Fractions

Finding Equivalent Fractions

```
Creating Equivalent Fractions
```

Method 1: Multiply by 1 in fraction form
1/2 = 1/2 × 2/2 = 2/4
1/2 = 1/2 × 3/3 = 3/6
1/2 = 1/2 × 4/4 = 4/8

Visual proof:
1/2: 2/4:

Both show the same amount shaded!

Method 2: Divide by common factors
12/16 = (12÷4)/(16÷4) = 3/4
18/24 = (18÷6)/(24÷6) = 3/4

Finding the pattern:
2/3 = 4/6 = 6/9 = 8/12 = 10/15...
Pattern: multiply both parts by 2, 3, 4, 5...

Simplifying Fractions

Reducing Fractions to Lowest Terms

Method 1: Find Greatest Common Factor (GCF)
Simplify 18/24:

Factors of 18: 1, 2, 3, 6, 9, 18
Factors of 24: 1, 2, 3, 4, 6, 8, 12, 24
GCF = 6

18/24 = (18÷6)/(24÷6) = 3/4

Method 2: Divide by common factors step by step
20/30 → 10/15 → 2/3
  ÷2 ÷5

Method 3: Prime factorization
24/36:
24 = 2^3 × 3
36 = 2^2 × 3^2
GCF = 2^2 × 3 = 12
24/36 = (24÷12)/(36÷12) = 2/3

Visual check:
24/36:

2/3:

Same proportion shaded!

Comparing and Ordering Fractions

Comparing Fractions

Fraction Comparison Strategies

Strategy 1: Same Denominators
Compare numerators directly
3/8 vs 5/8: Since 3 < 5, then 3/8 < 5/8

Visual:
3/8:

5/8:

Strategy 2: Same Numerators
Compare denominators (smaller denominator = larger fraction)
3/4 vs 3/8: Since 4 < 8, then 3/4 > 3/8

Visual:
3/4:

3/8:

Strategy 3: Convert to Common Denominators
Compare 2/3 and 3/4:
2/3 = 8/12
3/4 = 9/12
Since 8 < 9, then 2/3 < 3/4

Strategy 4: Convert to Decimals

```
2/3 = 0.667...
3/4 = 0.75
Since 0.667 < 0.75, then 2/3 < 3/4

Strategy 5: Cross Multiplication
Compare a/b and c/d:
If a×d < b×c, then a/b < c/d

2/3 vs 3/4: 2×4 = 8, 3×3 = 9
Since 8 < 9, then 2/3 < 3/4
```

Ordering Fractions

```
Ordering Multiple Fractions


Order from least to greatest: 1/2, 2/3, 3/8, 5/6

Step 1: Find common denominator (LCD = 24)
1/2 = 12/24
2/3 = 16/24
3/8 = 9/24
5/6 = 20/24

Step 2: Order by numerators
9/24 < 12/24 < 16/24 < 20/24

Step 3: Convert back to original fractions
3/8 < 1/2 < 2/3 < 5/6

Number line visualization:
0  3/8  1/2  2/3  5/6  1
    ↑    ↑    ↑    ↑
   0.375  0.5  0.667 0.833

Benchmark Strategy:
Compare to 1/2:
- 3/8 < 1/2 (since 3 < 4)
- 1/2 = 1/2
- 2/3 > 1/2 (since 4 > 3)
- 5/6 > 1/2 (since 10 > 6)

Then order within each group.
```

Adding and Subtracting Fractions

Same Denominators

Adding/Subtracting with Same Denominators

Rule: Add/subtract numerators, keep denominator

Addition: 2/7 + 3/7 = (2+3)/7 = 5/7

Visual:
2/7:

3/7:

Sum:
= 5/7

Subtraction: 6/8 - 2/8 = (6-2)/8 = 4/8 = 1/2

Visual:
6/8:

Remove 2/8:
= 4/8 = 1/2

Different Denominators

Adding/Subtracting with Different Denominators

Step 1: Find Least Common Denominator (LCD)
Step 2: Convert to equivalent fractions
Step 3: Add/subtract numerators
Step 4: Simplify if possible

Example: 1/3 + 1/4

Step 1: LCD of 3 and 4 = 12
Step 2: 1/3 = 4/12, 1/4 = 3/12
Step 3: 4/12 + 3/12 = 7/12
Step 4: 7/12 is already simplified

Visual proof:
1/3:

1/4:

Convert to twelfths:
4/12:

3/12:

Sum:
= 7/12

Subtraction Example: 3/4 - 1/6
LCD = 12
3/4 = 9/12, 1/6 = 2/12
9/12 - 2/12 = 7/12

Mixed Numbers

Adding/Subtracting Mixed Numbers

Method 1: Add/subtract parts separately
2 + 1¼ = (2 + 1) + (+ ¼) = 3 + (4/12 + 3/12) = 3 + 7/12 = 3 7/12

Method 2: Convert to improper fractions
2 = 7/3, 1¼ = 5/4
7/3 + 5/4 = 28/12 + 15/12 = 43/12 = 3 7/12

Subtraction with regrouping:
4 - 1

```
Can't subtract   from  , so regroup:
4  = 3 + 1 +   = 3 + 9/8 = 3 9/8

Now subtract:
3 9/8 - 1  = (3 - 1) + (9/8 - 5/8) = 2 + 4/8 = 2½

Visual representation:
4 :



After regrouping:
3 9/8:
                      +
```

Multiplying Fractions

Basic Fraction Multiplication

```
Multiplying Fractions Rule


Rule: Multiply numerators, multiply denominators
a/b × c/d = (a×c)/(b×d)

Example: 2/3 × 3/4 = (2×3)/(3×4) = 6/12 = 1/2

Visual interpretation:
"2/3 of 3/4"

Start with 3/4:



Take 2/3 of the shaded part:
Divide each shaded section into 3 parts, take 2:




Result: 6 out of 12 parts = 6/12 = 1/2

Real-world example:
"2/3 of the students are girls, 3/4 of the girls play sports"
```

2/3 × 3/4 = 1/2 of all students are girls who play sports

Multiplying Mixed Numbers

Mixed Number Multiplication

Method 1: Convert to improper fractions
2 × 1½ = 7/3 × 3/2 = 21/6 = 3½

Method 2: Use distributive property
2 × 1½ = (2 +) × (1 + ½)
        = 2×1 + 2×½ + ×1 + ×½
        = 2 + 1 +   +
        = 3 + 2/6 + 1/6
        = 3 + 3/6
        = 3½

Area model visualization:
2 × 1½ means rectangle with dimensions 2 by 1½

 1   2 × 1   ×1
        = 2   =

 ½   2 × ½   ×½
        = 1   =

          2

Total area: 2 +   + 1 +   = 3 + 2/6 + 1/6 = 3½

Simplifying Before Multiplying

Canceling Common Factors

Instead of: 4/9 × 3/8 = 12/72 = 1/6

Cancel first: 4/9 × 3/8

            4÷4=1 3÷3=1
            9÷3=3 8÷4=2

Result: 1/3 × 1/2 = 1/6 (much easier!)

Complex example:
15/28 × 14/45

Cancel common factors:
15/28 × 14/45

15÷15=1 14÷14=1
28÷14=2 45÷15=3

Result: 1/2 × 1/3 = 1/6

This method prevents large numbers and reduces errors!

Dividing Fractions

Division by Fractions

"Multiply by the Reciprocal" Rule

Rule: a/b ÷ c/d = a/b × d/c

Example: 3/4 ÷ 1/2 = 3/4 × 2/1 = 6/4 = 3/2 = 1½

Why does this work?
Think: "How many halves are in three-fourths?"

Visual with pizza:
3/4 pizza:

Each 1/2 piece:

How many 1/2 pieces fit in 3/4?
→ +
(half of a 1/2 piece)

Answer: 1½ pieces of size 1/2

Alternative explanation:
3/4 ÷ 1/2 = "3/4 × how many to make 1"

```
Since 1/2 × 2 = 1, we multiply by 2
3/4 × 2 = 6/4 = 1½
```

Complex Division Problems

```
Dividing Mixed Numbers


Problem: 2  ÷ 1

Step 1: Convert to improper fractions
2  = 8/3
1  = 4/3

Step 2: Multiply by reciprocal
8/3 ÷ 4/3 = 8/3 × 3/4 = 24/12 = 2

Answer: 2  ÷ 1  = 2

Real-world interpretation:
"If each serving is 1  cups, how many servings in 2  cups?"
Answer: 2 servings

Verification: 2 × 1  = 2 × 4/3 = 8/3 = 2

Word Problem Example:
"A recipe calls for 3¾ cups of flour. If you want to make ¾ of the recipe, how much flour do yo

3¾ × ¾ = 15/4 × 3/4 = 45/16 = 2 13/16 cups

But if the question was division:
"How many ¾-cup servings can you make from 3¾ cups?"
3¾ ÷ ¾ = 15/4 ÷ 3/4 = 15/4 × 4/3 = 60/12 = 5 servings
```

Real-World Applications

Cooking and Recipes

```
Recipe Applications


Recipe Scaling:
Original recipe (serves 4): 2¾ cups flour
Need to serve 6 people: 6 ÷ 4 = 1½ times the recipe
Flour needed: 2¾ × 1½ = 11/4 × 3/2 = 33/8 = 4  cups
```

Recipe Reduction:
Original recipe (serves 8): 3 cups sugar
Need to serve 3 people: 3 ÷ 8 = of the recipe
Sugar needed: 3 × = 10/3 × 3/8 = 30/24 = 5/4 = 1¼ cups

Ingredient Substitution:
Recipe calls for 2 cups milk
Only have cup measuring cup
Number of scoops: 2 ÷ = 8/3 ÷ 1/3 = 8/3 × 3/1 = 8 scoops

Cost Calculation:
Flour costs $2.40 per pound
Recipe uses ¾ pound
Cost: $2.40 × ¾ = $2.40 × 3/4 = $7.20/4 = $1.80

Construction and Measurement

Construction Applications

Board Cutting:
8-foot board, need pieces of 1 feet each
Number of pieces: 8 ÷ 1 = 8 ÷ 4/3 = 8 × 3/4 = 6 pieces

Remaining wood: 8 - (6 × 1) = 8 - 8 = 0 feet (perfect fit!)

Paint Coverage:
1 gallon covers 400 square feet
Room area: 320 square feet
Paint needed: 320 ÷ 400 = 32/40 = 4/5 = gallon

Tile Installation:
Room: 12½ feet × 10¼ feet
Area: 12½ × 10¼ = 25/2 × 41/4 = 1025/8 = 128 square feet

Tiles are 1¼ feet × 1¼ feet each
Tile area: 1¼ × 1¼ = 5/4 × 5/4 = 25/16 square feet

Number of tiles: 128 ÷ 25/16 = 1025/8 ÷ 25/16 = 1025/8 × 16/25 = 82 tiles

Time and Scheduling

Time Applications

Work Schedule:

```
Employee works 6¾ hours per day
Hourly wage: $18.50
Daily pay: 6¾ × $18.50 = 27/4 × $18.50 = $124.88

Project Planning:
Project takes 15½ hours total
Work 2¾ hours per day
Days needed: 15½ ÷ 2¾ = 31/2 ÷ 11/4 = 31/2 × 4/11 = 124/22 = 5 7/11 days
So need 6 full days to complete

Meeting Duration:
Meeting: 1  hours
Break every ½ hour
Number of breaks: 1  ÷ ½ = 4/3 ÷ 1/2 = 4/3 × 2/1 = 8/3 = 2
So 2 breaks during the meeting

Travel Time:
Distance: 45¾ miles
Speed: 55 mph
Time: 45¾ ÷ 55 = 183/4 ÷ 55 = 183/4 × 1/55 = 183/220 hours
= 183/220 × 60 minutes = 49.9 minutes   50 minutes
```

Common Mistakes and Prevention

Typical Fraction Errors

```
Common Fraction Mistakes


Mistake 1: Adding denominators
Wrong: 1/3 + 1/4 = 2/7
Correct: 1/3 + 1/4 = 4/12 + 3/12 = 7/12

Mistake 2: Cross-multiplying in addition
Wrong: 1/3 + 1/4 = (1×4 + 1×3)/(3×4) = 7/12 ← accidentally correct!
This method doesn't always work: 1/2 + 1/3   (1×3 + 1×2)/(2×3) = 5/6
Correct: 1/2 + 1/3 = 3/6 + 2/6 = 5/6 ← happens to match, but wrong method

Mistake 3: Multiplying denominators in multiplication
Wrong: 2/3 × 1/4 = 2/(3×4) = 2/12 = 1/6
Correct: 2/3 × 1/4 = (2×1)/(3×4) = 2/12 = 1/6 ← same answer, wrong process

Mistake 4: Not simplifying answers
Problem: 3/4 + 1/4 = 4/4
Wrong: Leave as 4/4
Correct: 4/4 = 1
```

```
Mistake 5: Improper conversion of mixed numbers
Wrong: 2  = (2×3)/3 = 6/3 = 2
Correct: 2  = (2×3 + 1)/3 = 7/3

Prevention Strategies:
- Always find common denominators for addition/subtraction
- Remember: multiply straight across for multiplication
- Always simplify final answers
- Use visual models to check reasonableness
- Practice conversion between mixed and improper fractions
```

Building Fraction Fluency

Conceptual Understanding First

```
Fraction Learning Progression


Stage 1: Concrete Understanding
- Use manipulatives (fraction bars, circles)
- Real-world contexts (pizza, chocolate bars)
- Visual models and drawings
- Part-whole relationships

Stage 2: Equivalent Fractions
- Pattern recognition (1/2 = 2/4 = 3/6...)
- Simplifying fractions
- Finding common denominators
- Comparing fractions

Stage 3: Operations
- Addition/subtraction with like denominators
- Addition/subtraction with unlike denominators
- Multiplication concepts and algorithms
- Division concepts and algorithms

Stage 4: Applications
- Word problems
- Mixed numbers
- Real-world contexts
- Connections to decimals and percents

Daily Practice Routine:
1. Visual warm-up (5 minutes): Identify fractions from pictures
2. Equivalent practice (10 minutes): Create equivalent fractions
```

```
3. Operation focus (15 minutes): Practice one operation type
4. Problem solving (10 minutes): Real-world applications
5. Reflection (5 minutes): What did we learn?
```

Conclusion

Fractions are fundamental to mathematical understanding, bridging the gap between whole numbers and the continuous nature of real-world quantities. They provide the foundation for understanding ratios, proportions, algebra, and advanced mathematical concepts.

```
Fractions: Complete Understanding


Conceptual Understanding:
  Multiple representations (visual, numerical, contextual)
  Part-whole relationships
  Equivalence concepts

Procedural Fluency:
  Finding equivalent fractions
  Comparing and ordering
  All four operations with fractions

Strategic Competence:
  Choosing appropriate methods
  Estimation with fractions
  Problem-solving approaches

Adaptive Reasoning:
  Why algorithms work
  When to use different representations
  Connections to other mathematical concepts

Productive Disposition:
  Confidence with fractions
  Appreciation for precision
  Persistence through complex problems
```

Master fractions well, and you'll have powerful tools for mathematical reasoning that extend far beyond arithmetic. Whether working with ratios in science, proportions in art, or rates in business, fractions provide essential mathematical language for describing and working with the continuous quantities that surround us in daily life.

Decimals: Another Way to Express Parts

Introduction

Decimals are another way to represent fractions and parts of a whole, using a place value system based on powers of 10. They provide a convenient way to work with non-whole quantities, especially in measurement, money, and scientific calculations.

From measuring lengths in centimeters to calculating prices at the store, decimals are everywhere in our daily lives, making them essential for practical mathematical literacy.

```
Decimal Fundamentals


A decimal number has two parts separated by a decimal point:
    12.47
    ↑  ↑
Whole  Decimal
part   part

The decimal point separates whole numbers from fractional parts
expressed in tenths, hundredths, thousandths, etc.
```

Understanding Decimal Place Value

The Decimal Place Value System

```
Decimal Place Value Chart


Number: 3,456.789

Thousands Hundreds Tens Ones • Tenths Hundredths Thousandths
   10³      10²    10¹  10     10⁻¹     10⁻²       10⁻³
  1000      100     10   1   .  0.1      0.01      0.001
    3        4      5    6   .   7        8          9

Value breakdown:
3,456.789 = 3000 + 400 + 50 + 6 + 0.7 + 0.08 + 0.009
```

```
Pattern: Each place is 10 times the place to its right
         Each place is 1/10 the place to its left

Visual representation:
Whole part:
Decimal part:
              0.7   0.08    0.009
```

Reading and Writing Decimals

```
How to Read Decimals


Method 1: Place Value Method
0.47 = "four tenths and seven hundredths"
0.047 = "zero tenths, four hundredths, and seven thousandths"

Method 2: Fraction Method
0.47 = "forty-seven hundredths" (47/100)
0.047 = "forty-seven thousandths" (47/1000)

Method 3: Mixed Number Method
12.47 = "twelve and forty-seven hundredths"

Common Decimal Readings:
0.1 = "one tenth" or "zero point one"
0.25 = "twenty-five hundredths" or "zero point two five"
0.125 = "one hundred twenty-five thousandths" or "zero point one two five"

Writing Decimals from Words:
"Three and forty-two hundredths" = 3.42
"Seven thousandths" = 0.007
"Fifteen and six tenths" = 15.6

Key Rules:
- The decimal point is read as "and"
- The last digit's place value names the entire decimal part
- Leading zeros after the decimal point are important for place value
```

Comparing and Ordering Decimals

Comparing Decimals

```
Decimal Comparison Strategies
```

```
Strategy 1: Line up decimal points and compare digit by digit
Compare 3.47 and 3.5:

3.47
3.5  ← Add zero: 3.50

Compare: 3.47 vs 3.50
- Ones place: 3 = 3
- Tenths place: 4 < 5
Therefore: 3.47 < 3.5

Strategy 2: Convert to fractions
3.47 = 347/100
3.5 = 350/100
Since 347 < 350, then 3.47 < 3.5

Strategy 3: Use place value understanding
3.47 has 4 tenths, 3.5 has 5 tenths
Since 4 < 5, then 3.47 < 3.5

Number line visualization:
3.4  3.45  3.47  3.5  3.55
       ↑              ↑
     3.47           3.5

Common Mistakes:
Wrong: 3.47 > 3.5 (thinking 47 > 5)
Correct: 3.47 < 3.5 (comparing place values correctly)

Wrong: 0.8 < 0.75 (thinking 8 < 75)
Correct: 0.8 > 0.75 (0.80 vs 0.75, so 80 > 75 hundredths)
```

Ordering Decimals

```
Ordering Multiple Decimals


Order from least to greatest: 0.6, 0.06, 0.66, 0.606

Step 1: Line up decimal points and add zeros for clarity
0.600
0.060
0.660
0.606

Step 2: Compare digit by digit from left to right
Tenths place: 0, 0, 6, 6
```

```
- 0.060 and 0.600 have 0 tenths (smaller)
- 0.660 and 0.606 have 6 tenths (larger)

Step 3: Within each group, continue comparing
Group 1 (0 tenths): 0.060 vs 0.600
Hundredths: 6 vs 0, so 0.060 < 0.600

Group 2 (6 tenths): 0.660 vs 0.606
Hundredths: 6 vs 0, so 0.606 < 0.660

Final order: 0.06 < 0.6 < 0.606 < 0.66

Number line visualization:
0  0.06  0.6  0.606  0.66  1
    ↑      ↑     ↑       ↑
   0.06   0.6   0.606   0.66

Real-world context:
These could represent distances in meters:
6 cm < 60 cm < 60.6 cm < 66 cm
```

Converting Between Fractions and Decimals

Fraction to Decimal Conversion

```
Converting Fractions to Decimals


Method 1: Long Division
Convert 3/8 to decimal:

  0.375
8)3.000
  24
  --
  60
  56
  --
  40
  40
  --
   0

Therefore: 3/8 = 0.375

Method 2: Equivalent Fractions (powers of 10)
```

```
Convert 3/4 to decimal:
3/4 = (3×25)/(4×25) = 75/100 = 0.75

Method 3: Calculator or known equivalents
Common fraction-decimal equivalents:
1/2 = 0.5      1/4 = 0.25     3/4 = 0.75
1/3 = 0.333... 2/3 = 0.666... 1/5 = 0.2
1/8 = 0.125    3/8 = 0.375    5/8 = 0.625
1/10 = 0.1     1/100 = 0.01   1/1000 = 0.001

Types of Decimal Results:
Terminating: 1/4 = 0.25 (ends)
Repeating: 1/3 = 0.333... (repeats forever)
Non-repeating:  = 3.14159... (never repeats, never ends)
```

Decimal to Fraction Conversion

```
Converting Decimals to Fractions


Method: Use place value to create fraction, then simplify

Example 1: 0.75
Step 1: 0.75 = 75/100 (75 hundredths)
Step 2: Simplify by dividing by GCF
75/100 = (75÷25)/(100÷25) = 3/4

Example 2: 0.125
Step 1: 0.125 = 125/1000 (125 thousandths)
Step 2: Simplify
125/1000 = (125÷125)/(1000÷125) = 1/8

Example 3: 2.6
Step 1: 2.6 = 2 + 0.6 = 2 + 6/10
Step 2: Simplify decimal part
6/10 = 3/5
Step 3: Combine
2.6 = 2

Repeating Decimals:
0.333... = 1/3
0.666... = 2/3
0.142857142857... = 1/7

Visual verification for 0.75 = 3/4:
0.75:
```

3/4:

Same amount shaded!

Adding and Subtracting Decimals

Decimal Addition

Adding Decimals

Key Rule: Line up the decimal points!

Example: 12.47 + 8.9 + 0.156

Step 1: Line up decimal points and add zeros for clarity

```
  12.470
   8.900
+  0.156
```

Step 2: Add as with whole numbers

```
  12.470
   8.900
+  0.156

  21.526
```

Common Mistakes and Prevention:

```
Wrong alignment:    Correct alignment:
  12.47               12.47
   8.9         →       8.90
+  0.156           +  0.156

  20.626              21.526
```

Visual representation with base-10 blocks:

12.47:

```
       12 ones      4 tenths  7 hundredths
```

8.9:

```
       8 ones   9 tenths
```

0.156:

```
       1 tenth  56 thousandths

Sum:
       21 ones                  13 tenths     26 hundredths
     = 21 ones + 1 one + 3 tenths + 26 hundredths
     = 21 ones + 1 one + 5 tenths + 2 hundredths + 6 thousandths
     = 21.526
```

Decimal Subtraction

```
Subtracting Decimals


Key Rule: Line up decimal points and regroup as needed

Example: 15.6 - 7.89

Step 1: Line up decimal points and add zeros
  15.60
-  7.89


Step 2: Subtract with regrouping
  15.60  →  14.150  (regroup as needed)
-  7.89   -  7.89

           7.71

Step-by-step regrouping:
- Can't subtract 9 from 0 in hundredths
- Regroup 1 tenth to 10 hundredths: 6 tenths → 5 tenths, 0 hundredths → 10 hundredths
- Can't subtract 8 from 5 in tenths
- Regroup 1 one to 10 tenths: 15 ones → 14 ones, 5 tenths → 15 tenths

Final calculation:
  14.1510
-  7.89

   7.71

Money example:
Purchase total: $23.47
Payment: $30.00
Change: $30.00 - $23.47 = $6.53

  $30.00
- $23.47
```

$6.53

Multiplying Decimals

Basic Decimal Multiplication

Multiplying Decimals

Rule: Multiply as whole numbers, then place decimal point

Example: 2.4 × 1.3

Step 1: Ignore decimal points, multiply whole numbers
24 × 13 = 312

Step 2: Count total decimal places in factors
2.4 has 1 decimal place
1.3 has 1 decimal place
Total: 2 decimal places

Step 3: Place decimal point in product
312 → 3.12 (2 places from right)

Therefore: 2.4 × 1.3 = 3.12

Area model verification:
2.4 × 1.3 = rectangle with dimensions 2.4 by 1.3

1	2 × 1 = 2	0.4×1 =0.4
0.3	2 × 0.3 = 0.6	0.4× 0.3 =0.12
	2	0.4

Total area: 2 + 0.4 + 0.6 + 0.12 = 3.12

Special Cases:
Multiplying by 10: Move decimal point 1 place right
2.47 × 10 = 24.7

```
Multiplying by 100: Move decimal point 2 places right
2.47 × 100 = 247

Multiplying by 0.1: Move decimal point 1 place left
2.47 × 0.1 = 0.247
```

Money and Practical Applications

```
Money Multiplication


Example: 7 items at $3.49 each

7 × $3.49 = ?

Method 1: Standard algorithm
  $3.49
×     7

 $24.43

Method 2: Mental math
7 × $3.49 = 7 × ($3.50 - $0.01)
          = 7 × $3.50 - 7 × $0.01
          = $24.50 - $0.07
          = $24.43

Tax Calculation:
Purchase: $45.60
Tax rate: 8.25% = 0.0825
Tax: $45.60 × 0.0825 = $3.762  $3.76 (rounded to nearest cent)
Total: $45.60 + $3.76 = $49.36

Unit Rate Problems:
Gas: $3.459 per gallon
Amount: 12.5 gallons
Cost: $3.459 × 12.5 = $43.2375  $43.24

Tip Calculation:
Bill: $67.80
Tip rate: 18% = 0.18
Tip: $67.80 × 0.18 = $12.204  $12.20
Total: $67.80 + $12.20 = $80.00
```

Dividing Decimals

Dividing Decimals by Whole Numbers

```
Decimal Division by Whole Numbers


Rule: Divide as usual, keep decimal point aligned

Example: 12.6 ÷ 3

      4.2

  3   12.6
      12↓

       06
        6

        0

Step-by-step:
1. 12 ÷ 3 = 4 (place in ones position)
2. Bring down 6 tenths
3. 6 tenths ÷ 3 = 2 tenths
4. Result: 4.2

Money example:
$15.75 ÷ 3 people = $5.25 each

      $5.25

  3   $15.75
      15↓

       07
        6

       15
       15

        0

Verification: $5.25 × 3 = $15.75
```

Dividing by Decimals

```
Division by Decimals


Rule: Move decimal points to make divisor a whole number

Example: 8.4 ÷ 2.1

Step 1: Move decimal point in divisor to make it whole
2.1 → 21 (moved 1 place right)

Step 2: Move decimal point same number of places in dividend
8.4 → 84 (moved 1 place right)

Step 3: Divide whole numbers
84 ÷ 21 = 4

Therefore: 8.4 ÷ 2.1 = 4

Complex example: 15.6 ÷ 0.12
Step 1: 0.12 → 12 (moved 2 places right)
Step 2: 15.6 → 1560 (moved 2 places right)
Step 3: 1560 ÷ 12 = 130

Unit Rate Calculation:
$4.68 for 1.2 pounds of apples
Price per pound: $4.68 ÷ 1.2

Move decimal points: $468 ÷ 12 = $39 per 10 pounds
So $3.90 per pound

Alternative: $4.68 ÷ 1.2 = $468 ÷ 120 = $3.90 per pound
```

Rounding Decimals

Rounding Rules and Strategies

```
Rounding Decimals


Basic Rule: Look at the digit to the right of the rounding place
- If 5 or greater: round up
- If less than 5: round down

Round 3.476 to nearest tenth:
```

```
3.476
  ↑ ← Look at hundredths place (7)
Since 7  5, round up: 3.5

Round 3.476 to nearest hundredth:
3.476
   ↑ ← Look at thousandths place (6)
Since 6  5, round up: 3.48

Rounding Examples:
To nearest whole: 7.8 → 8, 7.4 → 7, 7.5 → 8
To nearest tenth: 3.47 → 3.5, 3.43 → 3.4, 3.45 → 3.5
To nearest hundredth: 2.347 → 2.35, 2.343 → 2.34

Number Line Visualization:
Rounding 3.47 to nearest tenth:
3.4  3.45  3.5
       ↑      ↑
     3.47   closer to 3.5

Money Rounding:
$12.347 → $12.35 (round to nearest cent)
$12.344 → $12.34 (round to nearest cent)

Real-world Applications:
- Gas prices: $3.459 displayed as $3.46
- Test scores: 87.6% rounded to 88%
- Measurements: 5.73 cm rounded to 5.7 cm
```

Estimating with Decimals

Estimation Strategies

```
Decimal Estimation Techniques


Strategy 1: Round to whole numbers
12.7 + 8.3  13 + 8 = 21
Actual: 21.0 (exact!)

Strategy 2: Round to convenient decimals
4.8 × 6.2  5 × 6 = 30
Actual: 29.76 (very close)

Strategy 3: Use benchmark fractions
0.24  0.25 = 1/4
```

```
0.49  0.5 = 1/2
0.74  0.75 = 3/4

Strategy 4: Front-end estimation
3.47 + 2.89 + 1.23  3 + 2 + 1 = 6
Then adjust: 0.47 + 0.89 + 0.23  1.6
Total estimate: 6 + 1.6 = 7.6
Actual: 7.59 (excellent estimate!)

Shopping Estimation:
Items: $3.47, $8.99, $12.25, $5.89
Estimate: $3.50 + $9.00 + $12.25 + $6.00 = $30.75
Actual: $30.60 (great for budgeting!)

Measurement Estimation:
Room dimensions: 3.2m × 4.7m
Area estimate: 3 × 5 = 15 square meters
Actual: 15.04 square meters (very close)
```

Real-World Applications

Money and Finance

```
Financial Applications



Banking:
Account balance: $1,247.83
Deposit: $350.00
Withdrawal: $125.47
New balance: $1,247.83 + $350.00 - $125.47 = $1,472.36

Interest Calculation:
Principal: $1,000.00
Interest rate: 3.5% annually = 0.035
Time: 2.5 years
Simple interest: $1,000 × 0.035 × 2.5 = $87.50
Total: $1,000.00 + $87.50 = $1,087.50

Budget Planning:
Monthly income: $3,247.50
Expenses:
- Rent: $1,200.00
- Food: $450.75
- Transportation: $287.50
- Utilities: $156.25
```

- Entertainment: $200.00
Total expenses: $2,294.50
Remaining: $3,247.50 - $2,294.50 = $953.00

Unit Price Comparison:
Brand A: $4.68 for 1.2 pounds = $4.68 ÷ 1.2 = $3.90 per pound
Brand B: $3.75 for 0.9 pounds = $3.75 ÷ 0.9 = $4.17 per pound
Brand A is cheaper per pound

Measurement and Science

Measurement Applications

Recipe Scaling:
Original recipe (serves 4): 2.5 cups flour
Need to serve 6: 6 ÷ 4 = 1.5 times the recipe
Flour needed: 2.5 × 1.5 = 3.75 cups

Temperature Conversion:
Celsius to Fahrenheit: F = (C × 1.8) + 32
25°C = (25 × 1.8) + 32 = 45 + 32 = 77°F

Distance and Speed:
Distance: 247.5 miles
Time: 4.5 hours
Speed: 247.5 ÷ 4.5 = 55 miles per hour

Fuel Efficiency:
Distance: 387.6 miles
Gas used: 12.9 gallons
Miles per gallon: 387.6 ÷ 12.9 = 30.05 mpg

Scientific Notation Connection:
0.000045 = 4.5 × 10
45,000 = 4.5 × 10

Precision in Measurement:
Ruler measurement: 7.3 cm (precise to nearest tenth)
Micrometer: 7.347 cm (precise to nearest thousandth)
Different tools give different precision levels

Sports and Statistics

Sports Applications

```
Batting Average:
Hits: 47
At-bats: 156
Average: 47 ÷ 156 = 0.301 (rounded to 3 decimal places)

Race Times:
Runner 1: 12.47 seconds
Runner 2: 12.5 seconds
Runner 3: 12.43 seconds
Order: Runner 3 (12.43), Runner 1 (12.47), Runner 2 (12.50)

Grade Point Average:
Course 1: 3.7 (4 credits)
Course 2: 3.3 (3 credits)
Course 3: 4.0 (3 credits)
Course 4: 3.0 (2 credits)

Total points: (3.7×4) + (3.3×3) + (4.0×3) + (3.0×2) = 14.8 + 9.9 + 12.0 + 6.0 = 42.7
Total credits: 4 + 3 + 3 + 2 = 12
GPA: 42.7 ÷ 12 = 3.558...  3.56

Stock Prices:
Opening: $47.25
Closing: $48.73
Change: $48.73 - $47.25 = $1.48 increase
Percent change: ($1.48 ÷ $47.25) × 100 = 3.13% increase
```

Common Mistakes and Prevention

Typical Decimal Errors

```
Common Decimal Mistakes


Mistake 1: Misaligning decimal points in addition/subtraction
Wrong:          Correct:
  12.4            12.40
+  3.67         +  3.67

  15.31           16.07

Mistake 2: Incorrect decimal point placement in multiplication
Problem: 2.4 × 1.3
Wrong: 24 × 13 = 312 (forgot to place decimal)
Correct: 2.4 × 1.3 = 3.12 (2 decimal places total)
```

```
Mistake 3: Comparing decimals incorrectly
Wrong: 0.8 < 0.75 (thinking 8 < 75)
Correct: 0.8 > 0.75 (0.80 > 0.75)

Mistake 4: Rounding errors
Wrong: 3.45 rounded to nearest tenth = 3.4
Correct: 3.45 rounded to nearest tenth = 3.5 (5 rounds up)

Mistake 5: Division by decimals without adjusting
Wrong: 8.4 ÷ 2.1 = 84 ÷ 21 = 4 (correct answer by accident)
Better method: Move both decimal points first, then divide

Prevention Strategies:
- Always line up decimal points for addition/subtraction
- Count decimal places carefully in multiplication
- Add zeros to help with alignment and comparison
- Use estimation to check reasonableness
- Practice place value understanding regularly
- Use visual models when confused
```

Building Decimal Fluency

Learning Progression

```
Decimal Fluency Development


Stage 1: Place Value Foundation
- Understand decimal place value system
- Read and write decimals correctly
- Connect to fractions (0.5 = 1/2)
- Use visual models and manipulatives

Stage 2: Comparison and Ordering
- Compare decimals using place value
- Order sets of decimals
- Round decimals to specified places
- Estimate with decimals

Stage 3: Operations
- Add and subtract decimals
- Multiply decimals
- Divide decimals
- Check answers with estimation
```

```
Stage 4: Applications
- Money problems
- Measurement contexts
- Real-world problem solving
- Connect to percents and scientific notation

Daily Practice Routine:
1. Place value warm-up (5 minutes)
2. Comparison practice (5 minutes)
3. Operation focus (15 minutes)
4. Word problems (10 minutes)
5. Estimation check (5 minutes)

Games and Activities:
- Decimal war (comparing decimals)
- Decimal target (operations to reach target)
- Shopping simulations (money applications)
- Measurement activities (real contexts)
```

Conclusion

Decimals provide a powerful and practical way to work with non-whole quantities. They bridge the gap between fractions and whole numbers, offering a consistent place-value system that extends naturally from our base-10 number system.

```
Decimals: Complete Understanding


Conceptual Understanding:
  Place value system extending to the right of decimal point
  Connection to fractions and mixed numbers
  Relationship to money and measurement

Procedural Fluency:
  Reading, writing, and comparing decimals
  All four operations with decimals
  Rounding and estimation skills

Strategic Competence:
  Choosing appropriate methods for problems
  Using estimation to check reasonableness
  Converting between fractions and decimals

Adaptive Reasoning:
  Understanding why algorithms work
  Recognizing when to use decimals vs fractions
```

```
  Making connections to real-world contexts

Productive Disposition:
  Confidence with decimal operations
  Appreciation for precision in measurement
  Comfort with technology and calculators
```

Master decimals well, and you'll have essential tools for navigating our decimal-based world. From handling money and measurements to understanding scientific data and statistics, decimals provide the mathematical language for precision and accuracy in countless real-world applications.

Whether you're calculating tips, measuring ingredients, analyzing sports statistics, or working with scientific data, decimals offer a clear, consistent way to express and manipulate quantities with the precision that modern life demands.

Percentages: Parts per Hundred

Introduction

Percentages are a way of expressing fractions and decimals as parts per hundred. The word "percent" comes from the Latin "per centum," meaning "per hundred." Percentages provide an intuitive way to compare quantities, express rates, and communicate proportional relationships.

From calculating discounts while shopping to understanding test scores and statistical data, percentages are everywhere in modern life, making them essential for mathematical literacy and informed decision-making.

```
Percentage Fundamentals


Percent means "per hundred" or "out of 100"
25% = 25 per 100 = 25/100 = 0.25

The % symbol is shorthand for "per hundred"
```

Understanding Percentages Conceptually

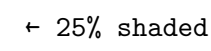
Visual Models of Percentages

```
Percentage Models


1. Grid Model (100 squares):
   25% = 25 out of 100 squares shaded


                  25% shaded
                (25 out of 100)
```

2. Circle Model (Pie Chart):
   25% = quarter of a circle



   ----

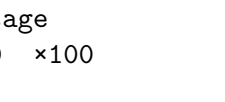
3. Bar Model:
   25% of 80 = 20



4. Number Line Model:
   0% 25% 50% 75% 100%
   0   0.25   0.5   0.75   1

The Percentage-Fraction-Decimal Connection

The Conversion Triangle



Common Equivalents:

Percentage	Decimal	Fraction
25%	0.25	1/4
50%	0.50	1/2
75%	0.75	3/4
100%	1.00	1/1
10%	0.10	1/10
20%	0.20	1/5
33 %	0.333...	1/3

```
   66 %      0.666...    2/3
   12.5%     0.125       1/8
   37.5%     0.375       3/8

Memory aids:
- 50% = half
- 25% = quarter
- 10% = tenth
- 1% = hundredth
```

Converting Between Forms

Percentage to Decimal

```
Converting Percentages to Decimals


Rule: Divide by 100 (or move decimal point 2 places left)

Examples:
25% = 25 ÷ 100 = 0.25
7% = 7 ÷ 100 = 0.07
150% = 150 ÷ 100 = 1.50
0.5% = 0.5 ÷ 100 = 0.005

Visual representation:
25% → 25.% → 0.25%
      ↑      ↑
   Move decimal point 2 places left

Step-by-step process:
1. Remove the % symbol
2. Move decimal point 2 places to the left
3. Add zeros if necessary

Practice examples:
8% → 0.08
125% → 1.25
0.75% → 0.0075
3.5% → 0.035
```

Decimal to Percentage

```
Converting Decimals to Percentages
```

```
Rule: Multiply by 100 (or move decimal point 2 places right)

Examples:
0.25 = 0.25 × 100 = 25%
0.07 = 0.07 × 100 = 7%
1.50 = 1.50 × 100 = 150%
0.005 = 0.005 × 100 = 0.5%

Visual representation:
0.25 → 0.25 → 25.%
       ↑      ↑
   Move decimal point 2 places right

Step-by-step process:
1. Move decimal point 2 places to the right
2. Add the % symbol
3. Remove unnecessary zeros

Practice examples:
0.08 → 8%
1.25 → 125%
0.0075 → 0.75%
0.035 → 3.5%
```

Fraction to Percentage

```
Converting Fractions to Percentages


Method 1: Convert to decimal first, then to percentage
3/4 → 3 ÷ 4 = 0.75 → 75%

Method 2: Create equivalent fraction with denominator 100
3/4 = (3×25)/(4×25) = 75/100 = 75%

Method 3: Cross multiply with 100
3/4 = x/100
3 × 100 = 4 × x
300 = 4x
x = 75, so 3/4 = 75%

Complex fractions:
5/8 = 5 ÷ 8 = 0.625 = 62.5%
2/3 = 2 ÷ 3 = 0.666... = 66 %
7/12 = 7 ÷ 12 = 0.583... = 58 %

Mixed numbers:
```

1¼ = 1.25 = 125%
2 = 2.6 = 260%

Visual verification for 3/4 = 75%:
3/4:

75%:
(75 out of 100)

Same proportion shaded!

Three Types of Percentage Problems

Type 1: Finding the Percentage

"What percent of A is B?"

Formula: (Part ÷ Whole) × 100 = Percentage

Example: What percent of 80 is 20?
Solution: (20 ÷ 80) × 100 = 0.25 × 100 = 25%

Visual representation:
Total: 80
Part: 20
Percentage: 20/80 = 1/4 = 25%

Real-world examples:
- "15 out of 60 students passed. What percentage passed?"
  (15 ÷ 60) × 100 = 25%

- "A team won 12 out of 20 games. What's their winning percentage?"
  (12 ÷ 20) × 100 = 60%

- "Sales increased from $50,000 to $65,000. What's the percent increase?"
  Increase: $65,000 - $50,000 = $15,000
  Percent increase: ($15,000 ÷ $50,000) × 100 = 30%

Step-by-step process:
1. Identify the part and the whole

```
2. Divide part by whole
3. Multiply by 100
4. Add % symbol
```

Type 2: Finding the Part

```
"What is X% of Y?"


Formula: Percentage × Whole = Part

Example: What is 30% of 150?
Solution: 0.30 × 150 = 45

Visual representation:
Whole: 150
30%:   45

Methods:
Method 1: Convert to decimal
30% of 150 = 0.30 × 150 = 45

Method 2: Use fraction
30% of 150 = 30/100 × 150 = 4500/100 = 45

Method 3: Mental math
30% of 150 = 3 × 10% of 150 = 3 × 15 = 45

Real-world examples:
- "A 20% tip on a $45 bill"
  20% of $45 = 0.20 × $45 = $9

- "25% discount on a $80 item"
  25% of $80 = 0.25 × $80 = $20 discount
  Sale price: $80 - $20 = $60

- "15% tax on a $200 purchase"
  15% of $200 = 0.15 × $200 = $30 tax
  Total: $200 + $30 = $230

Mental math shortcuts:
10% → Move decimal point 1 place left
1% → Move decimal point 2 places left
50% → Divide by 2
25% → Divide by 4
```

Type 3: Finding the Whole

"X is Y% of what number?"

Formula: Part ÷ Percentage = Whole

Example: 25 is 20% of what number?
Solution: 25 ÷ 0.20 = 125

Visual representation:
If 25 is 20%, then:
20%: 25
100%: ?
Answer: 125

Alternative method using proportion:
25/x = 20/100
25 × 100 = 20 × x
2500 = 20x
x = 125

Real-world examples:
- "A student got 18 problems correct, which was 75% of the test. How many problems were on the
  18 ÷ 0.75 = 24 problems

- "After a 15% discount, an item costs $68. What was the original price?"
  If $68 is 85% of original price (100% - 15% = 85%)
  $68 ÷ 0.85 = $80 original price

- "A salesperson earned $450 commission, which is 6% of sales. What were total sales?"
  $450 ÷ 0.06 = $7,500 in sales

Step-by-step process:
1. Identify what you know (part and percentage)
2. Convert percentage to decimal
3. Divide part by decimal percentage
4. Check: Does percentage of your answer equal the part?

Percentage Increase and Decrease

Calculating Percentage Change

Percentage Change Formula

```
Percentage Change = (New Value - Original Value) / Original Value × 100

Increase: Result is positive
Decrease: Result is negative

Example 1: Price increase
Original price: $50
New price: $65
Change: $65 - $50 = $15
Percentage increase: ($15 ÷ $50) × 100 = 30%

Example 2: Population decrease
Original: 8,000 people
New: 6,400 people
Change: 6,400 - 8,000 = -1,600
Percentage decrease: (-1,600 ÷ 8,000) × 100 = -20%

Visual representation of 30% increase:
Original: $50
Increase: $15
New:      $65

The increase ($15) is 30% of the original ($50)

Real-world applications:
Stock prices:
- Stock A: $40 → $46 = ($6 ÷ $40) × 100 = 15% increase
- Stock B: $75 → $60 = (-$15 ÷ $75) × 100 = -20% decrease

Test scores:
- First test: 80%, Second test: 92%
- Improvement: (92 - 80) ÷ 80 × 100 = 15% increase

Sales performance:
- Last month: $12,000, This month: $15,600
- Change: ($3,600 ÷ $12,000) × 100 = 30% increase
```

Successive Percentage Changes

```
Multiple Percentage Changes


Important: Successive percentages are NOT additive!

Example: Price increases 20%, then decreases 20%
Original price: $100
```

```
After 20% increase: $100 × 1.20 = $120
After 20% decrease: $120 × 0.80 = $96

Final result: $96 (not back to $100!)

Why? The 20% decrease is calculated on the new higher price ($120), not the original price ($10

General formula for successive changes:
Final = Original × (1 + r ) × (1 + r ) × ... × (1 + r )
Where r is the decimal form of percentage change (positive for increase, negative for decrease)

Complex example:
Investment: $1,000
Year 1: +15% → $1,000 × 1.15 = $1,150
Year 2: −10% → $1,150 × 0.90 = $1,035
Year 3: +25% → $1,035 × 1.25 = $1,293.75

Overall change: ($1,293.75 - $1,000) ÷ $1,000 × 100 = 29.375%

Alternative calculation:
$1,000 × 1.15 × 0.90 × 1.25 = $1,293.75

Compound interest connection:
This is why compound interest is so powerful - each year's interest earns interest in subsequen
```

Mental Math with Percentages

Quick Percentage Calculations

```
Mental Math Strategies


10% Strategy (Foundation):
10% of any number = move decimal point 1 place left
10% of 340 = 34
10% of 47.5 = 4.75

Building from 10%:
1% = 10% ÷ 10
5% = 10% ÷ 2
20% = 10% × 2
30% = 10% × 3

Example: 15% of 80
10% of 80 = 8
5% of 80 = 4
```

```
15% of 80 = 8 + 4 = 12

50% Strategy:
50% = half
50% of 86 = 43

25% Strategy:
25% = quarter = half of half
25% of 80 = 80 ÷ 4 = 20
Or: 25% of 80 = 50% of 50% of 80 = 50% of 40 = 20

Complex percentages:
75% = 50% + 25%
75% of 80 = 40 + 20 = 60

12.5% = half of 25%
12.5% of 80 = 25% of 80 ÷ 2 = 20 ÷ 2 = 10

Estimation techniques:
33%  33 % = 1/3
67%  66 % = 2/3

33% of 90   90 ÷ 3 = 30
67% of 90   2 × 30 = 60
```

Percentage Shortcuts

```
Useful Percentage Shortcuts


Doubling and Halving:
If you know 25% of a number, then:
50% = 2 × 25%
12.5% = 25% ÷ 2

Complementary percentages:
If you know 30% of a number, then:
70% = 100% - 30% = whole number - 30%

Example: 30% of 150 = 45
So: 70% of 150 = 150 - 45 = 105

Fraction shortcuts:
20% = 1/5 → divide by 5
16 % = 1/6 → divide by 6
12.5% = 1/8 → divide by 8
```

```
Quick tip calculations:
15% tip = 10% + 5%
18% tip = 10% + 8% (or 20% - 2%)
20% tip = 10% × 2

Example: 18% tip on $45
10% of $45 = $4.50
8% of $45 = 0.8 × $4.50 = $3.60
18% tip = $4.50 + $3.60 = $8.10

Discount calculations:
30% off means pay 70%
25% off means pay 75%

Example: 30% off $80 item
Pay: 70% of $80 = 0.7 × $80 = $56
Or: Discount = 30% of $80 = $24, Pay = $80 - $24 = $56
```

Real-World Applications

Shopping and Finance

```
Shopping Applications


Sales and Discounts:
Original price: $120
Discount: 25% off
Discount amount: 25% of $120 = 0.25 × $120 = $30
Sale price: $120 - $30 = $90

Or directly: Sale price = 75% of $120 = 0.75 × $120 = $90

Tax Calculations:
Purchase: $85.00
Sales tax: 8.5%
Tax amount: 8.5% of $85.00 = 0.085 × $85.00 = $7.23
Total: $85.00 + $7.23 = $92.23

Tip Calculations:
Bill: $67.50
Tip: 18%
Tip amount: 18% of $67.50 = 0.18 × $67.50 = $12.15
Total: $67.50 + $12.15 = $79.65

Credit Card Interest:
```

```
Balance: $2,500
Annual interest rate: 18.9%
Monthly rate: 18.9% ÷ 12 = 1.575%
Monthly interest: 1.575% of $2,500 = 0.01575 × $2,500 = $39.38

Investment Returns:
Investment: $5,000
Return: 7.5% annually
Gain: 7.5% of $5,000 = 0.075 × $5,000 = $375
New value: $5,000 + $375 = $5,375

Commission Calculations:
Sales: $45,000
Commission rate: 3.5%
Commission: 3.5% of $45,000 = 0.035 × $45,000 = $1,575
```

Statistics and Data Analysis

```
Statistical Applications


Survey Results:
Total respondents: 500
Favorable responses: 325
Approval rating: (325 ÷ 500) × 100 = 65%

Test Scores:
Class of 25 students
Scores of 90% or above: 8 students
Percentage with A grades: (8 ÷ 25) × 100 = 32%

Population Demographics:
City population: 150,000
Age 65 and older: 22,500
Senior citizen percentage: (22,500 ÷ 150,000) × 100 = 15%

Business Metrics:
Total revenue: $2,000,000
Profit: $300,000
Profit margin: ($300,000 ÷ $2,000,000) × 100 = 15%

Quality Control:
Items produced: 10,000
Defective items: 45
Defect rate: (45 ÷ 10,000) × 100 = 0.45%

Sports Statistics:
```

```
Free throws attempted: 250
Free throws made: 200
Free throw percentage: (200 ÷ 250) × 100 = 80%

Market Share:
Company sales: $12 million
Total market: $80 million
Market share: ($12M ÷ $80M) × 100 = 15%
```

Health and Science

```
Health and Science Applications


Medical Dosages:
Patient weight: 70 kg
Medication: 5 mg per kg of body weight
Dose: 70 × 5 = 350 mg

If patient can only take 80% of full dose:
Reduced dose: 80% of 350 mg = 0.80 × 350 = 280 mg

Nutrition Labels:
Daily Value for sodium: 2,300 mg
Amount in food item: 460 mg
Percentage of Daily Value: (460 ÷ 2,300) × 100 = 20%

Solution Concentrations:
Salt solution: 250 mL total volume
Salt: 15 mL
Concentration: (15 ÷ 250) × 100 = 6%

Body Composition:
Total body weight: 70 kg
Muscle mass: 28 kg
Muscle percentage: (28 ÷ 70) × 100 = 40%

Scientific Experiments:
Trials conducted: 200
Successful outcomes: 174
Success rate: (174 ÷ 200) × 100 = 87%

Environmental Data:
Forest area 10 years ago: 50,000 hectares
Forest area today: 42,000 hectares
Deforestation: (8,000 ÷ 50,000) × 100 = 16% decrease
```

```
Chemical Purity:
Pure substance: 95.5 g
Total sample: 100 g
Purity: (95.5 ÷ 100) × 100 = 95.5%
```

Common Mistakes and Prevention

Typical Percentage Errors

```
Common Percentage Mistakes


Mistake 1: Confusing percentage OF vs percentage INCREASE
Wrong: "Sales increased 20% from $100 to $120"
The increase is $20, which is 20% OF the original $100
Correct: "Sales increased BY 20%" or "Sales increased TO 120% of original"

Mistake 2: Adding percentages incorrectly
Wrong: 20% increase followed by 30% increase = 50% total increase
Correct: $100 → $120 (20% increase) → $156 (30% of $120 increase) = 56% total increase

Mistake 3: Using wrong base for percentage calculation
Problem: "Price increased from $80 to $100. What's the percentage increase?"
Wrong: ($20 ÷ $100) × 100 = 20%
Correct: ($20 ÷ $80) × 100 = 25%
(Always use original value as base for percentage change)

Mistake 4: Percentage vs percentage points
Wrong: "Interest rate increased from 3% to 5%, a 67% increase"
Correct: "Interest rate increased by 2 percentage points" or "increased by 67%"
Both are correct but mean different things!

Mistake 5: Forgetting to convert percentage to decimal
Wrong: 25% of 80 = 25 × 80 = 2000
Correct: 25% of 80 = 0.25 × 80 = 20

Prevention Strategies:
- Always identify what the percentage is OF
- Convert percentages to decimals before calculating
- Use the correct base for percentage change calculations
- Draw pictures or use visual models when confused
- Check answers for reasonableness
- Practice distinguishing between percentage points and percentages
```

Building Percentage Fluency

Learning Progression

```
Percentage Fluency Development


Stage 1: Conceptual Foundation
- Understand "per hundred" meaning
- Connect to fractions and decimals
- Use visual models (grids, circles, bars)
- Learn common equivalents (50% = 1/2, etc.)

Stage 2: Conversions
- Percentage  decimal  fraction
- Mental math with common percentages
- Estimation skills
- Benchmark percentages

Stage 3: Three Types of Problems
- Finding percentage: "What percent of A is B?"
- Finding part: "What is X% of Y?"
- Finding whole: "A is X% of what?"
- Problem identification skills

Stage 4: Advanced Applications
- Percentage increase/decrease
- Successive percentage changes
- Real-world problem solving
- Data interpretation

Daily Practice Routine:
1. Conversion warm-up (5 minutes)
2. Mental math practice (10 minutes)
3. Problem type focus (15 minutes)
4. Real-world applications (10 minutes)
5. Error analysis and reflection (5 minutes)

Games and Activities:
- Percentage war (comparing percentages)
- Shopping simulations (discounts, tax, tips)
- Data analysis projects (surveys, statistics)
- Percentage estimation games
```

Conclusion

Percentages provide a universal language for expressing proportions, rates, and comparisons. They bridge the gap between abstract mathematical concepts and practical applications, making them essential for informed citizenship and decision-making in our data-driven world.

```
Percentages: Complete Understanding


Conceptual Understanding:
  "Per hundred" meaning and visual models
  Connection to fractions and decimals
  Proportional reasoning

Procedural Fluency:
  Conversions between forms
  Three types of percentage problems
  Mental math strategies

Strategic Competence:
  Problem identification and setup
  Choosing appropriate methods
  Estimation and checking

Adaptive Reasoning:
  Understanding when to use percentages
  Recognizing common error patterns
  Making real-world connections

Productive Disposition:
  Confidence with percentage calculations
  Critical thinking about data and claims
  Appreciation for mathematical precision
```

Master percentages well, and you'll have powerful tools for navigating our modern world. Whether analyzing financial investments, interpreting scientific data, understanding political polls, or simply calculating tips and discounts, percentages provide essential mathematical literacy for informed decision-making.

From the classroom to the boardroom, from the laboratory to the voting booth, percentages help us quantify change, compare alternatives, and communicate proportional relationships with clarity and precision. They are truly one of mathematics' most practical and widely-used concepts.

Geometry

Introduction to Geometry: The Study of Shape and Space

What is Geometry?

Geometry is the branch of mathematics that deals with shapes, sizes, positions, angles, and dimensions of objects. The word "geometry" comes from the Greek words "geo" (earth) and "metron" (measure), literally meaning "earth measurement." This ancient origin reflects geometry's practical beginnings in surveying land, constructing buildings, and creating art.

From the pyramids of Egypt to modern skyscrapers, from the hexagonal patterns in honeycombs to the spiral galaxies in space, geometry helps us understand and describe the world around us. It bridges the gap between abstract mathematical thinking and tangible, visual reality.

```
Geometry: The Visual Mathematics


Points → Lines → Planes → Solids
  •       --
Basic   1D      2D      3D
building shapes  shapes  shapes
blocks

All geometric concepts build from these fundamentals
```

The Historical Journey of Geometry

Ancient Origins: Practical Beginnings

Geometry began as a practical necessity in ancient civilizations:

```
Ancient Geometric Applications


Egyptian Pyramids (2600 BCE):
    /\
   /  \
  /____\
Perfect geometric precision:
```

```
- Base square: 230.4m × 230.4m
- Height: 146.5m
- Angle accuracy: within 3 arcminutes

Babylonian Mathematics (2000 BCE):
Right triangle relationships:
   |\
 5 | \ 13
   |  \
   |___\
     12
5² + 12² = 13² (Pythagorean theorem)

Greek Temples (500 BCE):
Golden ratio proportions:

               = (1+√5)/2   1.618
```

Greek Mathematical Revolution

The Greeks transformed geometry from practical measurement to logical reasoning:

```
Greek Geometric Achievements


Thales (624-546 BCE):
- First geometric proofs
- Angle in semicircle = 90°
    C
   /|\
  / | \
 /  |  \
A---O---B

Pythagoras (570-495 BCE):
- Pythagorean theorem
- Mathematical proof culture
a² + b² = c²

Euclid (300 BCE):
- "Elements" - systematic geometry
- Axiomatic method
- 13 books of geometric knowledge
```

```
Archimedes (287-212 BCE):
- Area and volume formulas
-   approximation using polygons
- Method of exhaustion
```

Modern Geometry: Beyond Euclid

```
Evolution of Geometric Thinking


Classical Period (300 BCE - 1600 CE):
- Euclidean geometry dominates
- Compass and straightedge constructions
- Geometric algebra

Renaissance (1400-1600):
- Perspective in art
- Coordinate geometry (Descartes)
- Analytic methods

Modern Era (1800-present):
- Non-Euclidean geometries
- Topology
- Fractal geometry
- Computer graphics

Timeline:
300 BCE     1637     1826     1975
Euclid's    Descartes'   Non-Euclidean  Fractal
Elements    Coordinates   Geometry      Geometry
```

Fundamental Geometric Concepts

The Building Blocks: Point, Line, and Plane

```
Geometric Primitives


Point:
• A
- No dimension (0D)
- Exact location in space
- Named with capital letters

Line:
```

```
A ←      → B
- One dimension (1D)
- Extends infinitely in both directions
- Contains infinitely many points
- Named with two points or lowercase letter

Line Segment:
A          B
- Part of a line
- Has two endpoints
- Finite length

Ray:
A        →
- Part of a line
- Has one endpoint
- Extends infinitely in one direction

Plane:
```



```
- Two dimensions (2D)
- Extends infinitely in all directions
- Contains infinitely many lines and points
- Named with three non-collinear points or Greek letter
```

Angles: Measuring Direction and Turn

```
Types of Angles


Acute Angle (0° <  < 90°):
    \
     \
      \
       \

Right Angle (  = 90°):
    |
```

```
    |
    |____

Obtuse Angle (90° <   < 180°):
  \
   \
    \
     \
      \______

Straight Angle (  = 180°):
←       →

Reflex Angle (180° <   < 360°):









←

Full Rotation (  = 360°):
    ↑


 ←     →


    ↓
```

Angle Measurement:
- Degrees (°): 1/360 of full rotation
- Radians: Arc length / radius
- 180° = radians
- 90° = /2 radians

Geometric Relationships and Properties

Parallel and Perpendicular Lines

```
Line Relationships


Parallel Lines (||):
A ←       →
B ←       →
```

- Never intersect
- Same direction
- Equal distance apart

Perpendicular Lines ( ):

- Intersect at 90°
- Form right angles

Intersecting Lines:

- Meet at one point
- Form four angles
- Vertical angles are equal

Skew Lines (in 3D):

- Don't intersect
- Not parallel
- In different planes

Symmetry: Balance and Pattern

Types of Symmetry

Line Symmetry (Reflection):

↑ Line of symmetry

Mirror image across a line

Rotational Symmetry:

```
   ----
Triangle: 120° rotational symmetry
Square: 90° rotational symmetry

Point Symmetry:


      ← Center of symmetry


180° rotation looks the same

Examples in Nature:
- Butterfly wings (line symmetry)
- Flowers (rotational symmetry)
- Snowflakes (multiple symmetries)
- Human face (approximate line symmetry)
```

Two-Dimensional Shapes

Polygons: Many-Sided Figures

```
Polygon Classification


By Number of Sides:
Triangle (3):    Pentagon (5):    Octagon (8):
    /\              /  \           /   \
   /  \            /    \         /     \
  /____\          /      \       /       \
                 /        \     /         \
                /          \   /           \
                \__________/   \___________/

Quadrilateral (4): Hexagon (6):    Decagon (10):
                  /   \         /    \
                 /     \       /      \
                /       \     /        \
               /         \   /          \
                /         \ /            \
                \_________/ \____________/

Regular vs Irregular:
Regular: All sides equal, all angles equal
Irregular: Sides or angles not all equal
```

```
Convex vs Concave:
Convex: All interior angles < 180°
Concave: At least one interior angle > 180°

Convex:        Concave:
  /  \          /  \
 /     \        /     \
/       \      /       \
\       /      \      /
 \     /        \    /
  \___/          \__/
                   ↑
                Interior angle > 180°
```

Circles: Perfect Curves

```
Circle Components


    A

          ← Radius (r)
           ← Center (O)
       O


         ← Diameter (d = 2r)
B             C









       D
```

```
Key Elements:
- Center: Fixed point equidistant from all points on circle
- Radius: Distance from center to any point on circle
- Diameter: Distance across circle through center
- Circumference: Distance around circle
- Chord: Line segment connecting two points on circle
- Arc: Part of the circumference
- Sector: "Pie slice" region
- Tangent: Line touching circle at exactly one point
```

Formulas:
- Circumference: $C = 2\pi r = \pi d$
- Area: $A = \pi r^2$
- $\pi \approx 3.14159...$

Three-Dimensional Shapes

Polyhedra: Many-Faced Solids

Common Polyhedra

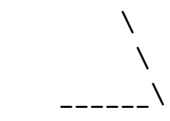
Cube:

6 faces, 8 vertices, 12 edges

Rectangular Prism:

Triangular Prism:

Pyramid (Square base):



Tetrahedron:

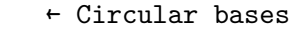


4 faces, 4 vertices, 6 edges

Curved Solids

Solids with Curved Surfaces

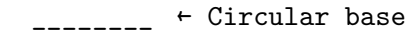
Cylinder:



Volume: $V = \pi r^2 h$
Surface Area: $SA = 2\pi r^2 + 2\pi rh$

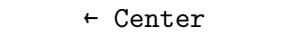
Cone:

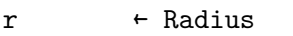




Volume: $V = (1/3)\pi r^2 h$
Surface Area: $SA = \pi r^2 + \pi rl$ (l = slant height)

Sphere:





```
    -------
Volume: V = (4/3) r³
Surface Area: SA = 4 r²
```

Measurement in Geometry

Perimeter and Area

```
Perimeter and Area Concepts


Perimeter: Distance around a shape
Area: Space inside a shape

Rectangle:

      b       h


Perimeter: P = 2l + 2w = 2(l + w)
Area: A = l × w

Square:

  s   s


Perimeter: P = 4s
Area: A = s²

Triangle:
     \
 b  h \
      \
  _____\
    a
Perimeter: P = a + b + c
Area: A = (1/2) × base × height

Circle:

     r
```

```
   _______
Circumference: C = 2 r
Area: A =  r²

Parallelogram:

             h
  ___________
      b
Perimeter: P = 2(a + b)
Area: A = base × height

Trapezoid:
         a
              h
  ___________
       b
Area: A = (1/2)(a + b) × h
```

Volume and Surface Area

```
3D Measurement Concepts


Volume: Space inside a 3D shape
Surface Area: Total area of all faces

Rectangular Prism:

        l
               h

      w


Volume: V = l × w × h
Surface Area: SA = 2(lw + lh + wh)

Cube:

      s
           s
```

```
   s

Volume: V = s³
Surface Area: SA = 6s²

Cylinder:

     r
          h


Volume: V =  r²h
Surface Area: SA = 2 r² + 2 rh

Sphere:

      r
```

$$\text{Volume: } V = \frac{4}{3}\pi r^3$$

```
Surface Area: SA = 4 r²
```

Coordinate Geometry

The Cartesian Plane

```
Coordinate System


    y

  4          Point (3, 4)

  3

  2

  1

                x
 -2   -1     1     2     3
```

```
 -1

 -2

Quadrants:
I:   (+, +)  II:  (-, +)
III: (-, -)  IV:  (+, -)

Distance Formula:
d = √[(x -x )² + (y -y )²]

Midpoint Formula:
M = ((x +x )/2, (y +y )/2)

Slope Formula:
m = (y -y )/(x -x )
```

Transformations

Moving and Changing Shapes

```
Geometric Transformations


Translation (Slide):
Original:
Translated:
- Same size and shape
- Different position

Reflection (Flip):
Original:     Mirror: |    Reflected:
- Same size and shape
- Opposite orientation

Rotation (Turn):
Original: ↑    90° CW: →    180°: ↓    270° CW: ←
- Same size and shape
- Different orientation

Dilation (Scale):
Original:     Scale 2:      Scale 0.5:

- Different size
- Same shape
- Scale factor determines size change
```

```
Composition of Transformations:
Multiple transformations applied in sequence
Example: Translate, then rotate, then reflect
```

Applications of Geometry

Architecture and Construction

```
Geometric Applications in Building


Structural Stability:
Triangles are strongest shape
     \
      \  ← Triangular trusses
   ____\     in roof construction

Right Angles:
Essential for:
- Square foundations
- Vertical walls
- Level floors

Golden Ratio in Design:
 = (1+√5)/2   1.618

             ← Pleasing proportions
                   in architecture



Arches and Domes:

                ← Distribute weight
                  efficiently
 ---------------

Perspective in Design:

                 ← Vanishing point
 ---------------    creates depth
```

Art and Design

```
Geometry in Visual Arts
```

```
Symmetry in Art:
- Bilateral symmetry in portraits
- Radial symmetry in mandalas
- Translational symmetry in patterns

Perspective Drawing:
One-point perspective:



          ← Vanishing point

       -----


Two-point perspective:



    ---------------

Tessellations:
Regular patterns that fill plane:




Fractals in Art:
Self-similar patterns at all scales
- Sierpinski triangle
- Mandelbrot set
- Natural forms (ferns, coastlines)
```

Science and Nature

```
Geometry in Natural World


Crystal Structures:
Salt (cubic):     Quartz (hexagonal):
```

```
                  ---
```

Honeycomb Pattern:


```
    ---  ---  ---
```

Hexagons use least material for maximum storage

Spiral Patterns:
- Nautilus shells (logarithmic spiral)
- Galaxy arms
- Sunflower seed arrangements
- DNA double helix

Sphere Packing:
Most efficient arrangement:

Used in atomic structures

Modern Geometry

Non-Euclidean Geometries

Beyond Euclid's Geometry

Euclidean Geometry (Flat):
Parallel lines never meet
Sum of triangle angles = 180°

```
     \
      \
   ____\
   +   +   = 180°
```

Spherical Geometry (Curved):
"Parallel" lines meet at poles
Sum of triangle angles > 180°

```
     \
      \  ← On sphere surface
   ____\
   +   +   > 180°
```

```
Hyperbolic Geometry (Saddle):
Many "parallels" through a point
Sum of triangle angles < 180°
     \
      \  ← On saddle surface
   ____\
   +   +   < 180°
```

```
Applications:
- GPS systems (spherical geometry)
- General relativity (curved spacetime)
- Computer graphics (hyperbolic geometry)
```

Topology: Rubber Sheet Geometry

```
Topology Concepts


Properties preserved under continuous deformation:
- Connectedness
- Inside vs outside
- Number of holes

Topologically Equivalent:
Coffee cup   Donut (both have 1 hole)




   ---       -----

Möbius Strip:
One-sided surface with one edge




 ---------------
                 ← Twist and connect
 ---------------

Klein Bottle:
Bottle that passes through itself
No inside or outside!

Euler's Formula for Polyhedra:
V - E + F = 2
(Vertices - Edges + Faces = 2)
```

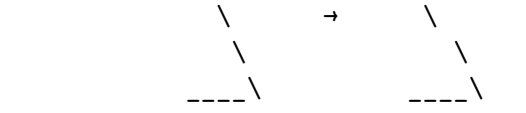
Cube: 8 - 12 + 6 = 2

Building Geometric Intuition

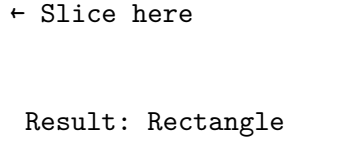
Visualization Skills

Developing Spatial Reasoning

Mental Rotation:
Can you rotate this shape mentally?



Cross-Sections:
What shape do you get when you slice:
Cube with plane:



Net Folding:
Which net folds into a cube?

Perspective Drawing:
Draw 3D objects on 2D paper:
- Use vanishing points
- Show hidden lines as dashed
- Maintain proportions

Problem-Solving Strategies

Geometric Problem-Solving

```
1. Draw a Diagram:
Always start with a clear, labeled diagram

2. Look for Patterns:
- Symmetries
- Similar shapes
- Parallel/perpendicular lines

3. Use Known Formulas:
- Area and perimeter
- Pythagorean theorem
- Angle relationships

4. Break Complex Shapes:
Divide into simpler parts:

              = Triangle + Rectangle
   -------




5. Check Your Answer:
- Does it make sense?
- Are units correct?
- Is the scale reasonable?

Example Problem:
Find the area of a regular hexagon with side length 6.

Solution approach:
1. Draw the hexagon
2. Divide into 6 equilateral triangles
3. Find area of one triangle
4. Multiply by 6

Area of equilateral triangle = (√3/4)s²
Area of hexagon = 6 × (√3/4) × 6² = 54√3
```

Conclusion

Geometry is the mathematics of shape, space, and visual reasoning. It connects abstract mathematical concepts with the tangible world around us, providing tools for understanding everything from the microscopic structure of crystals to the vast architecture of the universe.

```
Geometry: A Complete Understanding


Conceptual Understanding:
  Points, lines, planes, and their relationships
  Properties of 2D and 3D shapes
  Symmetry, transformations, and patterns

Procedural Fluency:
  Measuring angles, perimeter, area, volume
  Coordinate geometry calculations
  Construction and drawing techniques

Strategic Competence:
  Problem-solving with geometric reasoning
  Choosing appropriate formulas and methods
  Breaking complex problems into simpler parts

Adaptive Reasoning:
  Understanding why geometric relationships work
  Making connections between different concepts
  Applying geometry to real-world situations

Productive Disposition:
  Appreciation for geometric beauty and patterns
  Confidence in spatial reasoning
  Curiosity about geometric relationships
```

From ancient surveyors measuring fields to modern computer graphics designers creating virtual worlds, geometry provides the mathematical language for describing and manipulating space. Whether you're an artist exploring perspective, an architect designing buildings, or a scientist studying molecular structures, geometric thinking offers powerful tools for understanding and creating in our three-dimensional world.

The journey through geometry reveals not just mathematical relationships, but fundamental patterns that govern the structure of reality itself. As you continue exploring geometric concepts, you'll discover that this ancient branch of mathematics remains as relevant and beautiful today as it was to the Greek mathematicians who first systematized its study over two millennia ago.

Points, Lines, and Planes: The Building Blocks of Geometry

Introduction

All of geometry begins with three fundamental concepts: points, lines, and planes. These are the basic building blocks from which all geometric shapes and relationships are constructed. Understanding these primitives deeply is essential for mastering geometry, as every theorem, proof, and application builds upon these foundational ideas.

Like atoms in chemistry or notes in music, points, lines, and planes are the elementary components that combine to create the rich and beautiful world of geometric forms.

```
The Geometric Hierarchy


Point (0D) → Line (1D) → Plane (2D) → Space (3D)
    •           --

Each dimension builds upon the previous:
- Points define lines
- Lines define planes
- Planes define space
```

Points: The Foundation of All Geometry

Understanding Points

A point represents an exact location in space. It has no size, no width, no length, no height - it is purely positional. While we draw points as small dots, the actual geometric point is dimensionless.

```
Point Representation


Visual representation: • A
Mathematical concept: Exact location with no dimension

Properties of Points:
- Zero-dimensional (0D)
```

- No length, width, or height
- Infinite number can fit in any space
- Named with capital letters: A, B, C, P, Q, etc.

Point Notation:
• A ← Point A
• B ← Point B
• P ← Point P

Real-world approximations:
- Tip of a sharp pencil
- Intersection of two lines
- Corner of a room
- Star in the night sky (from our perspective)

Points in Space

Point Relationships

Collinear Points:
Points that lie on the same line
A • • B • C
Points A, B, and C are collinear

Non-collinear Points:
Points that do NOT lie on the same line
    • B

A • • C
Points A, B, and C are non-collinear

Coplanar Points:
Points that lie in the same plane
    • B

A • • C

    • D
Points A, B, C, and D are coplanar

Distance Between Points:
The shortest path between two points is a straight line
A •   • B
  ← d(A,B) →

```
In coordinate plane:
A(x , y ) and B(x , y )
Distance = √[(x -x )² + (y -y )²]
```

Lines: One-Dimensional Infinity

Understanding Lines

A line is a straight path that extends infinitely in both directions. It has length but no width or height, making it one-dimensional. A line contains infinitely many points.

```
Line Representation


Visual representation:
A ←       → B
   Line AB or line l

Mathematical properties:
- One-dimensional (1D)
- Infinite length
- No width or height
- Contains infinitely many points
- Perfectly straight

Line Notation:
←       →  Line AB (written as AB or line AB)
        l         Line l (named with lowercase letter)

Postulates about Lines:
1. Through any two points, exactly one line exists
2. A line contains infinitely many points
3. A line extends infinitely in both directions
```

Types of Lines and Line Segments

```
Line Variations


Line:
A ←       → B
- Extends infinitely in both directions
- Named by any two points on it
```

Ray:
A • →
- Has one endpoint (A)
- Extends infinitely in one direction
- Named by endpoint and another point: Ray AB

Line Segment:
A • • B
- Has two endpoints (A and B)
- Finite length
- Named by its endpoints: Segment AB or AB

Midpoint:
A • • • B
  M
- Point M is equidistant from A and B
- AM = MB
- M = ((x +x)/2, (y +y)/2) in coordinates

Length of Segment:
A • • B
 ← |AB| →
- Distance between endpoints
- Always positive
- Measured in units (cm, inches, etc.)

Line Relationships

How Lines Interact

Parallel Lines (||):
l ← →
l ← →
- Never intersect
- Same direction
- Always same distance apart
- Symbol: l || l

Intersecting Lines:

  P

- Meet at exactly one point P
- Form four angles at intersection

- Most common relationship

Perpendicular Lines ( ):

- Intersect at 90° (right angle)
- Form four right angles
- Symbol: l l

Concurrent Lines:

- Three or more lines meeting at one point
- Common in geometric constructions

Skew Lines (3D only):

- Do not intersect
- Not parallel
- Exist in different planes

Planes: Two-Dimensional Surfaces

Understanding Planes

A plane is a flat surface that extends infinitely in all directions. It has length and width but no thickness, making it two-dimensional. A plane contains infinitely many points and lines.

Plane Representation

Visual representation:

← Plane (pi)

```
      A            ← Points A, B, C in plane
   B    C
```

Mathematical properties:
- Two-dimensional (2D)
- Infinite length and width
- No thickness
- Contains infinitely many points and lines
- Perfectly flat

Plane Notation:
- Named by three non-collinear points: Plane ABC
- Named by a single letter: Plane , Plane M
- Named by a parallelogram figure: ABCD

Postulates about Planes:
1. Through any three non-collinear points, exactly one plane exists
2. A plane contains infinitely many points and lines
3. If two points lie in a plane, the entire line through them lies in the plane

Plane Relationships

How Planes Interact

Parallel Planes:

```
   Plane

       (constant distance)

   Plane
```

- Never intersect
- Always same distance apart
- Symbol: ||

Intersecting Planes:

```
      Plane
```

```
      ← Line of intersection
    Plane
```

- Intersect in exactly one line
- Most common relationship
- Line of intersection contains all common points

```
Perpendicular Planes:
      Plane


      Plane
```

- Intersect at 90°
- Form four right dihedral angles
- Symbol:

```
Point-Plane Relationships:
Point in plane: A    (A is in plane )
Point not in plane: B    (B is not in plane )

Line-Plane Relationships:
Line in plane: All points of line are in plane
Line intersects plane: Line and plane meet at one point
Line parallel to plane: Line and plane never meet
```

Coordinate Systems

The Cartesian Plane

```
2D Coordinate System


    y-axis

    4        Point P(3, 4)

    3

    2

    1
```

```
                x-axis
   -2   -1     1     2     3

   -1

   -2
```

```
Origin: O(0, 0) - intersection of axes

Quadrants:
I:   x > 0, y > 0  (upper right)
II:  x < 0, y > 0  (upper left)
III: x < 0, y < 0  (lower left)
IV:  x > 0, y < 0  (lower right)

Coordinate Notation:
Point P has coordinates (x, y)
- x-coordinate: horizontal position
- y-coordinate: vertical position
- Ordered pair: order matters!
```

3D Coordinate System

```
3D Coordinate System


         z-axis



                 y-axis



   x-axis

Point P(x, y, z):
- x-coordinate: left/right position
- y-coordinate: forward/back position
- z-coordinate: up/down position

Example: P(3, 2, 4)
         z

       4      P(3, 2, 4)
```

```
              2


           y
         3


      x

Distance in 3D:
d = √[(x -x )² + (y -y )² + (z -z )²]

Midpoint in 3D:
M = ((x +x )/2, (y +y )/2, (z +z )/2)
```

Geometric Constructions

Basic Constructions with Compass and Straightedge

```
Classical Construction Tools


Straightedge:

- Draws straight lines
- No measurement marks
- Infinite length (theoretically)

Compass:




  ------
- Draws circles and arcs
- Maintains fixed distance
- Can transfer lengths

Construction 1: Copy a Line Segment
Given: Segment AB
Construct: Segment CD with CD = AB

Step 1: Draw ray from C
C         →
```

```
Step 2: Set compass to length AB
A           B
   ← compass →

Step 3: Mark point D on ray
C           D
   ←   AB   →
```

```
Construction 2: Bisect a Line Segment
Given: Segment AB
Construct: Midpoint M

Step 1: Draw arcs from A and B


A      B



Step 2: Connect intersection points


A     B
     M


Step 3: M is the midpoint
AM = MB
```

Perpendicular and Parallel Constructions

```
Advanced Constructions


Construction 3: Perpendicular at a Point
Given: Line l and point P on l
Construct: Line perpendicular to l through P

Step 1: Draw arcs on both sides of P


          l
  A     P     B

Step 2: Draw arcs from A and B above and below
```

```
        l
       P


Step 3: Connect intersection points through P


        l
       P


Construction 4: Parallel Line
Given: Line l and point P not on l
Construct: Line through P parallel to l

Method: Copy corresponding angles
1. Draw transversal through P and l
2. Copy the angle at intersection
3. Draw line through P with copied angle

    P

       ← Parallel to l

        l
```

Angle Relationships

Types of Angles

```
Angle Classification


Acute Angle (0° <  < 90°):




Right Angle (  = 90°):


     ----
      ← Right angle symbol

Obtuse Angle (90° <  < 180°):
```

```
      ------

Straight Angle (  = 180°):
←       →

Reflex Angle (180° <   < 360°):




←

Angle Measurement:
- Degrees (°): 1/360 of full rotation
- Radians: Arc length / radius
- 180° =  radians
- 90° =  /2 radians
- 60° =  /3 radians
- 45° =  /4 radians
- 30° =  /6 radians
```

Angle Relationships

```
Special Angle Pairs


Adjacent Angles:
Share a common vertex and side



 AOB and  BOC are adjacent

Vertical Angles:
Opposite angles formed by intersecting lines

   1    2

   4    3

 1 =  3 and  2 =  4 (vertical angles are equal)
```

```
Linear Pair:
Adjacent angles that form a straight line



 1 +  2 = 180°

Complementary Angles:
Two angles that sum to 90°

      30°
      ----
       60°
30° + 60° = 90°

Supplementary Angles:
Two angles that sum to 180°

      120°
      ------
       60°
120° + 60° = 180°

Angles and Parallel Lines:
When parallel lines are cut by a transversal:

l  ← 1 2 →


l  ← 3 4  →

Corresponding angles:  1 =  3,  2 =  4
Alternate interior:  2 =  3
Alternate exterior:  1 =  4
Co-interior (same side):  2 +  4 = 180°
```

Distance and Midpoint

Distance Formulas

```
Measuring Distance


1D Distance (Number Line):
A         B
  -3          5
```

```
Distance = |5 - (-3)| = |8| = 8

2D Distance (Coordinate Plane):
A(x , y ) and B(x , y )

    B(5, 4)

              4-1=3


A(2, 1)
       5-2=3

Distance = √[(x -x )² + (y -y )²]
         = √[(5-2)² + (4-1)²]
         = √[3² + 3²]
         = √[9 + 9]
         = √18 = 3√2

3D Distance:
A(x , y , z ) and B(x , y , z )
Distance = √[(x -x )² + (y -y )² + (z -z )²]

Example: A(1, 2, 3) and B(4, 6, 7)
Distance = √[(4-1)² + (6-2)² + (7-3)²]
         = √[9 + 16 + 16]
         = √41
```

Midpoint Formulas

```
Finding Midpoints


1D Midpoint (Number Line):
A        B
  2    M    8
Midpoint M = (2 + 8)/2 = 5

2D Midpoint (Coordinate Plane):
A(x , y ) and B(x , y )
Midpoint M = ((x +x )/2, (y +y )/2)

Example: A(2, 1) and B(8, 5)
    B(8, 5)
```

```
    M(5,3)


A(2, 1)

M = ((2+8)/2, (1+5)/2) = (5, 3)

3D Midpoint:
A(x , y , z ) and B(x , y , z )
Midpoint M = ((x +x )/2, (y +y )/2, (z +z )/2)

Properties of Midpoints:
- Equidistant from both endpoints
- Divides segment into two equal parts
- Unique for each line segment
```

Applications and Problem Solving

Real-World Applications

```
Practical Applications


Architecture and Construction:
- Points: Corner locations, intersections
- Lines: Edges of buildings, property boundaries
- Planes: Walls, floors, ceilings

Example: Foundation Layout
    D       C

                 Rectangle ABCD
                 Right angles at corners
    A       B

Check: All angles should be 90°
Diagonals AC and BD should be equal

Navigation and GPS:
- Points: Locations (latitude, longitude)
- Lines: Routes, paths
- Planes: Maps, coordinate systems

Example: Distance between cities
City A: (40.7°N, 74.0°W)
City B: (34.1°N, 118.2°W)
```

```
Use spherical distance formula for Earth's surface

Computer Graphics:
- Points: Pixels, vertices
- Lines: Edges, wireframes
- Planes: Surfaces, screens

Example: 3D Model
Vertices define shape:
V (0, 0, 0), V (1, 0, 0), V (0, 1, 0), V (0, 0, 1)
Lines connect vertices
Planes form surfaces
```

Problem-Solving Strategies

```
Geometric Problem-Solving


Strategy 1: Draw and Label
Always start with a clear diagram
- Mark given information
- Label points, lines, angles
- Use proper notation

Strategy 2: Identify Relationships
Look for:
- Parallel/perpendicular lines
- Equal segments/angles
- Special triangles
- Symmetries

Strategy 3: Use Coordinate Geometry
Place figure in coordinate system:
- Origin at convenient point
- Axes along important lines
- Use distance/midpoint formulas

Example Problem:
"Prove that the diagonals of a rectangle bisect each other"

Solution:
1. Place rectangle in coordinate system
   A(0, 0), B(a, 0), C(a, b), D(0, b)

2. Find diagonal midpoints
   Diagonal AC: midpoint = (a/2, b/2)
   Diagonal BD: midpoint = (a/2, b/2)
```

3. Same midpoint proves bisection

Strategy 4: Use Properties and Theorems
Apply known results:
- Angle relationships
- Parallel line properties
- Distance formulas
- Midpoint theorems

Strategy 5: Work Backwards
Start with what you want to prove:
- What would make this true?
- What conditions are needed?
- How can I create those conditions?

Common Mistakes and Misconceptions

Typical Errors

Common Geometric Mistakes

Mistake 1: Confusing Lines and Segments
Wrong: "Line AB has length 5"
Correct: "Segment AB has length 5"
(Lines are infinite, segments have finite length)

Mistake 2: Assuming from Appearance
Wrong: "These lines look parallel"
Correct: Check slopes or use parallel line tests
Visual appearance can be deceiving

Mistake 3: Incorrect Notation
Wrong: AB = 5 (this means point A equals point B equals 5)
Correct: |AB| = 5 or AB = 5 units (length of segment)

Mistake 4: Midpoint Confusion
Wrong: Midpoint of A(2, 4) and B(6, 8) is (8, 12)
Correct: Midpoint is ((2+6)/2, (4+8)/2) = (4, 6)
(Add coordinates, then divide by 2)

Mistake 5: Distance Formula Errors
Wrong: $d = (x - x)^2 + (y - y)^2$
Correct: $d = \sqrt{[(x - x)^2 + (y - y)^2]}$
(Don't forget the square root!)

```
Prevention Strategies:
- Use precise mathematical language
- Check calculations with different methods
- Verify answers make geometric sense
- Draw accurate diagrams
- Practice with coordinates regularly
```

Building Geometric Intuition

Visualization Exercises

```
Developing Spatial Sense


Exercise 1: Point Plotting
Plot these points and describe the pattern:
A(1, 1), B(2, 4), C(3, 9), D(4, 16)
Pattern: Points lie on curve y = x²

Exercise 2: Line Relationships
Given points A(0, 0), B(3, 4), C(6, 8):
- Are A, B, C collinear?
- Check: Do they lie on same line?
- Slope AB = 4/3, Slope BC = 4/3
- Yes, they're collinear!

Exercise 3: Geometric Constructions
Practice with compass and straightedge:
1. Construct equilateral triangle
2. Bisect an angle
3. Construct perpendicular bisector
4. Construct parallel lines

Exercise 4: Coordinate Transformations
Start with triangle A(0, 0), B(3, 0), C(0, 4)
- Translate by (2, 1)
- Reflect over x-axis
- Rotate 90° counterclockwise
- What's the final position?

Exercise 5: 3D Visualization
Imagine a cube with vertices at:
(0,0,0), (1,0,0), (0,1,0), (0,0,1),
(1,1,0), (1,0,1), (0,1,1), (1,1,1)
- Which vertices are connected by edges?
```

```
- What's the length of each edge?
- What's the length of each diagonal?
```

Conclusion

Points, lines, and planes form the foundation of all geometric thinking. These seemingly simple concepts contain profound mathematical depth and provide the building blocks for understanding space, shape, and spatial relationships.

```
Points, Lines, and Planes: Complete Understanding


Conceptual Understanding:
  Dimensionality: 0D points, 1D lines, 2D planes
  Infinite nature of lines and planes
  Relationships between geometric objects

Procedural Fluency:
  Distance and midpoint calculations
  Coordinate geometry applications
  Geometric constructions

Strategic Competence:
  Problem-solving with coordinate methods
  Using properties and relationships
  Choosing appropriate representations

Adaptive Reasoning:
  Understanding why formulas work
  Making connections between concepts
  Applying to real-world situations

Productive Disposition:
  Appreciation for geometric precision
  Confidence with spatial reasoning
  Curiosity about geometric relationships
```

From the ancient Greek geometers who first formalized these concepts to modern computer scientists working with virtual reality, the fundamental ideas of points, lines, and planes continue to provide the mathematical language for describing and manipulating space.

Whether you're an architect designing buildings, a programmer creating graphics, or simply trying to understand the geometric world around you, mastering these basic concepts provides the solid foundation needed for all further geometric learning. Every theorem in geometry, every construction, every proof ultimately traces back to these simple yet profound ideas about the nature of space and position.

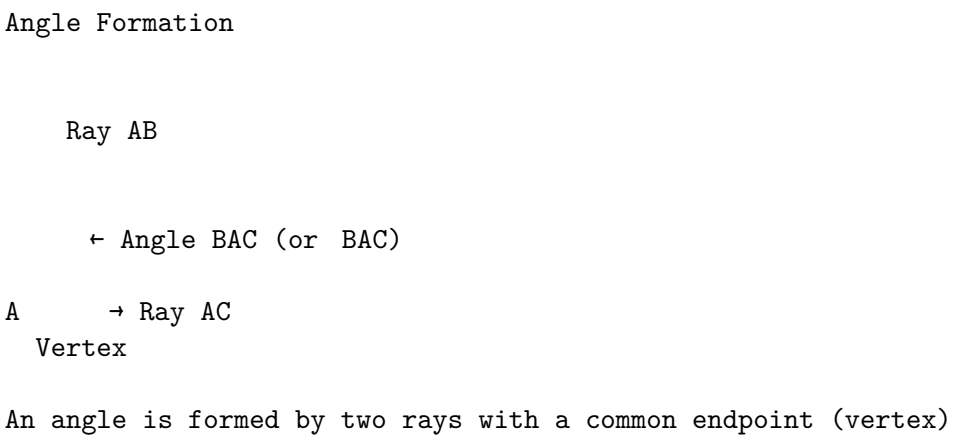
As you continue your geometric journey, remember that these building blocks are not just abstract mathematical concepts - they are the tools that help us understand and describe the spatial relationships that surround us every day, from the layout of our cities to the structure of the molecules that make up our world.

Angles: Measuring Direction and Turn

Introduction

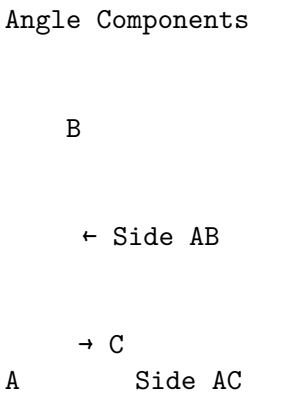
An angle is formed when two rays share a common endpoint, creating a measure of rotation or turn between them. Angles are fundamental to geometry, appearing in everything from the corners of buildings to the navigation of ships, from the design of gears to the analysis of light rays.

Understanding angles is crucial for geometric reasoning, as they help us describe relationships between lines, classify shapes, and solve problems involving rotation, direction, and spatial orientation.



Understanding Angles

Basic Angle Concepts



Vertex A

Components:
- Vertex: Common endpoint of the two rays (point A)
- Sides: The two rays that form the angle (AB and AC)
- Interior: The region "inside" the angle
- Exterior: The region "outside" the angle

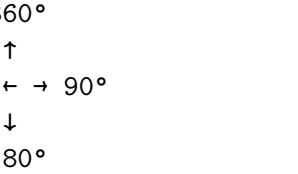
Angle Notation:
 BAC or CAB (vertex in middle)
 A (when context is clear)
 1, 2, 3 (numbered angles)

Reading Angles:
 BAC is read as "angle BAC"
The vertex (A) is always in the middle
Order of other letters doesn't matter: BAC = CAB

Measuring Angles

Angle Measurement Systems

Degrees (°):
- Full rotation = 360°
- Based on ancient Babylonian system
- Most common in elementary geometry



Common degree measures:
- Right angle: 90°
- Straight angle: 180°
- Full rotation: 360°

Radians (rad):
- Based on circle's radius
- Full rotation = 2 radians
- Used in advanced mathematics

Relationship: 180° = radians

```
Common radian measures:
-  /6 rad = 30°
-  /4 rad = 45°
-  /3 rad = 60°
-  /2 rad = 90°
-   rad = 180°
- 2  rad = 360°

Gradians (gon):
- Full rotation = 400 gradians
- Used in surveying
- 100 gradians = 90°

Minutes and Seconds:
- 1° = 60 minutes (60')
- 1' = 60 seconds (60")
- Used for precise measurements
- Example: 45°30'15" = 45.504167°
```

Types of Angles

Classification by Measure

```
Angle Types by Size


Acute Angle (0° <   < 90°):

       45°



Examples: 30°, 45°, 60°, 89°
- Less than a right angle
- "Sharp" angle

Right Angle (  = 90°):

      90°
     ----
      ← Right angle symbol

- Exactly 90°
- Forms square corner
- Perpendicular lines form right angles
```

```
Obtuse Angle (90° <   < 180°):

     120°


      ------

Examples: 91°, 120°, 150°, 179°
- Greater than right angle
- Less than straight angle

Straight Angle (  = 180°):
←        →
      180°

- Forms a straight line
- Two opposite rays

Reflex Angle (180° <   < 360°):

       270°




←

Examples: 181°, 270°, 300°, 359°
- Greater than straight angle
- Less than full rotation

Full Angle (  = 360°):
    ↑

       360°
 ←     →

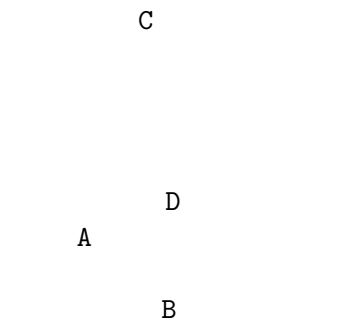

    ↓

- Complete rotation
- Back to starting position
```

Special Angle Relationships

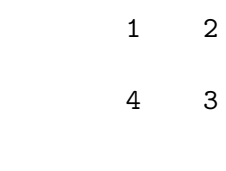
```
Angle Pair Relationships
```

Adjacent Angles:



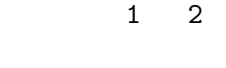
CAB and BAD are adjacent
- Share common vertex (A)
- Share common side (AB)
- No interior points in common

Vertical Angles:



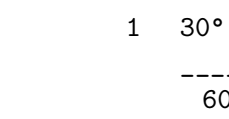
1 and 3 are vertical angles
2 and 4 are vertical angles
- Formed by intersecting lines
- Always equal: 1 = 3, 2 = 4

Linear Pair:



1 and 2 form a linear pair
- Adjacent angles
- Form straight line
- Sum to 180°: 1 + 2 = 180°

Complementary Angles:
Two angles that sum to 90°



1 + 2 = 30° + 60° = 90°
- Can be adjacent or non-adjacent
- Each angle is the complement of the other

Supplementary Angles:
Two angles that sum to 180°



1 + 2 = 120° + 60° = 180°
- Can be adjacent or non-adjacent
- Each angle is the supplement of the other

Angles and Parallel Lines

Transversals and Parallel Lines

When a transversal (a line that intersects two or more lines) cuts through parallel lines, it creates eight angles with special relationships.

Parallel Lines Cut by Transversal

l ← 1 2 →

l ← 3 4 →

l ←5 6 →

l ←7 8 →

If l || l , then:

Corresponding Angles (same position):
1 = 3, 2 = 4, 5 = 7, 6 = 8

Alternate Interior Angles (inside, opposite sides):
3 = 6, 4 = 5

Alternate Exterior Angles (outside, opposite sides):
1 = 8, 2 = 7

Co-interior Angles (same side interior):

 3 + 5 = 180°, 4 + 6 = 180°

Co-exterior Angles (same side exterior):
 1 + 7 = 180°, 2 + 8 = 180°

Using Angle Relationships

Problem-Solving with Parallel Lines

Example 1: Finding Unknown Angles
Given: l || l , 1 = 65°
Find: All other angles

l ← 1 2 →

l ←3 4 →

Solution:
 2 = 180° - 65° = 115° (linear pair with 1)
 3 = 65° (corresponding to 1)
 4 = 115° (corresponding to 2)

Example 2: Proving Lines are Parallel
Given: 1 = 3 (corresponding angles)
Prove: l || l

If corresponding angles are equal,
then the lines are parallel.

Converse Relationships:
- If corresponding angles are equal → lines are parallel
- If alternate interior angles are equal → lines are parallel
- If co-interior angles are supplementary → lines are parallel

Angle Constructions

Basic Angle Constructions

Constructing Angles with Compass and Straightedge

Construction 1: Copy an Angle
Given: ABC
Construct: DEF = ABC

```
Step 1: Draw ray EF
D       → F

Step 2: Draw arc from B intersecting both sides of  ABC
    A


       ← Arc intersects at P and Q

      C
B   Q   P

Step 3: Draw same arc from E
D       → F

    -------
    E

Step 4: Measure PQ with compass
Step 5: Mark same distance on arc from E
Step 6: Draw ray from E through mark

Result:  DEF =  ABC

Construction 2: Bisect an Angle
Given:  ABC
Construct: Ray BD that bisects  ABC

Step 1: Draw arc from B intersecting both sides
    A


       ← Arc intersects at P and Q

      C
B   P   Q

Step 2: Draw arcs from P and Q with same radius
    A



        ← Arcs intersect at R
      C
B   P   Q

Step 3: Draw ray BR
    A
```

```
      C
B
    D
```

Result: ABD = DBC

Special Angle Constructions

Advanced Constructions

Construction 3: 90° Angle (Right Angle)
Method 1: Perpendicular at point on line
Given: Line l and point P on l
Construct: Line perpendicular to l at P

Step 1: Draw arcs on both sides of P

```
          l
  A     P     B
```

Step 2: Draw arcs from A and B above line

```
       C
       l
      P
```

Step 3: Draw line PC

```
       C
       l
      P
```

Result: PC ⊥ l

Construction 4: 60° and 30° Angles
Step 1: Construct equilateral triangle
- Draw line segment AB
- Draw arcs from A and B with radius AB
- Connect intersection point C to A and B

```
    C
```

```
      ← All angles are 60°

A    B

Step 2: Bisect 60° angle to get 30°

Construction 5: 45° Angle
Step 1: Construct 90° angle
Step 2: Bisect the right angle


     45°
```

Angle Measurement Tools

Using a Protractor

```
Protractor Usage


Standard Protractor:
    180°      0°




         90°



   ---------------

Steps to Measure an Angle:
1. Place center point on vertex
2. Align one side with 0° line
3. Read where other side crosses scale
4. Choose correct scale (inner or outer)

Example: Measuring  ABC
    B
```

```
    35°

     → C
A

1. Center on A
2. Align AC with 0°
3. Read where AB crosses: 35°

Common Protractor Errors:
- Wrong scale (inner vs outer)
- Misaligned center point
- Reading wrong direction
- Not accounting for reflex angles
```

Digital Angle Measurement

```
Modern Angle Tools


Digital Protractor:
- LCD display
- More precise readings
- Can measure in degrees or radians

Angle Finder Apps:
- Use phone's accelerometer
- Measure angles in real world
- Useful for construction/carpentry

CAD Software:
- Computer-aided design
- Precise angle specification
- Automatic angle calculation

Theodolite (Surveying):
- Professional surveying instrument
- Measures horizontal and vertical angles
- High precision (seconds of arc)

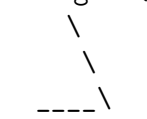
Clinometer:
- Measures angles of elevation/depression
- Used in forestry, geology
- Handheld or digital versions
```

Angles in Polygons

Interior Angles of Polygons

Polygon Interior Angles

Triangle (3 sides):



Sum = 180°

Quadrilateral (4 sides):

Sum = 360°

Pentagon (5 sides):



Sum = 540°

General Formula:
Sum of interior angles = (n - 2) × 180°
where n = number of sides

Examples:
Triangle: (3 - 2) × 180° = 180°
Quadrilateral: (4 - 2) × 180° = 360°
Pentagon: (5 - 2) × 180° = 540°
Hexagon: (6 - 2) × 180° = 720°
Octagon: (8 - 2) × 180° = 1080°

Regular Polygon Interior Angle:
Each angle = (n - 2) × 180° / n

Examples:
Equilateral triangle: 180° / 3 = 60°
Square: 360° / 4 = 90°
Regular pentagon: 540° / 5 = 108°

```
Regular hexagon: 720° / 6 = 120°
```

Exterior Angles of Polygons

```
Polygon Exterior Angles


Exterior Angle Definition:
Formed by extending one side of polygon

     \
      \
   ____\______
        ↑
    Exterior angle

Key Property:
Sum of exterior angles = 360° (for any polygon)

Triangle:
     \
      \
   ____\______
Each exterior + adjacent interior = 180°
Sum of all exterior angles = 360°

Regular Polygon Exterior Angle:
Each exterior angle = 360° / n

Examples:
Equilateral triangle: 360° / 3 = 120°
Square: 360° / 4 = 90°
Regular pentagon: 360° / 5 = 72°
Regular hexagon: 360° / 6 = 60°
Regular octagon: 360° / 8 = 45°

Relationship:
Interior angle + Exterior angle = 180°
```

Trigonometric Angles

Angles in Standard Position

```
Standard Position Angles
```

```
    y

         Terminal side


       x
      Initial side (positive x-axis)
```

Standard Position:
- Vertex at origin
- Initial side on positive x-axis
- Measured counterclockwise (positive)
- Measured clockwise (negative)

Quadrant Angles:
I: 0° < < 90°
II: 90° < < 180°
III: 180° < < 270°
IV: 270° < < 360°

Reference Angles:
Acute angle between terminal side and x-axis

Quadrant I: Reference angle =
Quadrant II: Reference angle = 180° -
Quadrant III: Reference angle = - 180°
Quadrant IV: Reference angle = 360° -

Special Angles:
0°, 30°, 45°, 60°, 90°, 120°, 135°, 150°, 180°,
210°, 225°, 240°, 270°, 300°, 315°, 330°, 360°

Unit Circle and Angles

Unit Circle Angles

```
    y

  1        (1, 0) = 0°, 360°




         x
 -1        1
```

```
 -1

Key Points on Unit Circle:
0° = (1, 0)
30° = (√3/2, 1/2)
45° = (√2/2, √2/2)
60° = (1/2, √3/2)
90° = (0, 1)
120° = (-1/2, √3/2)
135° = (-√2/2, √2/2)
150° = (-√3/2, 1/2)
180° = (-1, 0)
210° = (-√3/2, -1/2)
225° = (-√2/2, -√2/2)
240° = (-1/2, -√3/2)
270° = (0, -1)
300° = (1/2, -√3/2)
315° = (√2/2, -√2/2)
330° = (√3/2, -1/2)
360° = (1, 0)

Coordinates give trigonometric ratios:
Point (x, y) on unit circle at angle  :
cos   = x-coordinate
sin   = y-coordinate
tan   = y/x (when x   0)
```

Real-World Applications

Navigation and Direction

```
Angles in Navigation


Compass Bearings:
    N (0°)


W     E


    S (180°)

True Bearing: Angle measured clockwise from North
```

```
Example: 045° = Northeast direction

Magnetic Declination:
Difference between magnetic north and true north
Must be accounted for in navigation

Aviation:
- Heading: Direction aircraft is pointing
- Track: Actual path over ground
- Wind correction angle

Example: Aircraft heading 090° (due east)
Wind from 180° at 20 knots
Results in track of 085° (slightly north of east)

GPS Coordinates:
Latitude: Angle north/south of equator (0° to 90°)
Longitude: Angle east/west of Prime Meridian (0° to 180°)

Example: New York City
Latitude: 40.7° N
Longitude: 74.0° W
```

Architecture and Construction

```
Angles in Building


Roof Pitch:
Rise over run, often expressed as angle

      Rise

      30° pitch
  ---
  Run

Common roof pitches:
- 30° (moderate slope)
- 45° (steep slope)
- 15° (low slope)

Stair Angles:
Optimal angle: 30° to 35°
Too steep: > 40° (dangerous)
Too shallow: < 25° (inefficient)
```

```
      Rise

  ---
  Run

Angle = arctan(Rise/Run)

Solar Panel Angles:
Optimal angle  Latitude of location
Adjustable for seasonal optimization

Example: Location at 40° N latitude
Summer: 40° - 15° = 25°
Winter: 40° + 15° = 55°

Structural Bracing:
45° braces provide maximum strength
Triangular trusses use 60° angles

     \
      \  ← 60° angles in
   ____\    equilateral triangle
```

Art and Design

```
Angles in Visual Arts


Perspective Drawing:
Vanishing points create depth illusion

One-point perspective:



          ← Vanishing point

        -----


Two-point perspective:



    ---------------
```

Photography:
- Angle of view (lens focal length)
- Camera angle (high, low, eye level)
- Lighting angles

Wide angle: > 60° field of view
Normal: 40° to 60°
Telephoto: < 40°

Golden Angle:
137.5° 360°/ ² (where is golden ratio)
Found in plant growth patterns
- Sunflower seed spirals
- Pine cone arrangements
- Leaf positioning

Logo Design:
- 60° angles suggest stability
- 45° angles suggest movement
- 90° angles suggest strength
- Curved angles suggest friendliness

Problem-Solving with Angles

Angle Calculation Strategies

Problem-Solving Techniques

Strategy 1: Use Angle Relationships
Given: Vertical angles, linear pairs, etc.
Apply: Known angle relationships

Example: Two intersecting lines form angles
If one angle is 65°, find all others.

1 = 65° (given)
3 = 65° (vertical angles)
2 = 180° - 65° = 115° (linear pair)
4 = 115° (vertical angles)

Strategy 2: Use Parallel Line Properties
Given: Parallel lines cut by transversal
Apply: Corresponding, alternate, co-interior angles

Example: l || l , transversal creates 50° angle
Find corresponding angle.

l ← 50° →

l ← ? →

Corresponding angle = 50°

Strategy 3: Use Polygon Angle Sums
Given: Polygon with known angles
Apply: Interior angle sum formula

Example: Pentagon with four angles: 100°, 110°, 120°, 95°
Find fifth angle.

Sum = (5-2) × 180° = 540°
Fifth angle = 540° - (100° + 110° + 120° + 95°) = 115°

Strategy 4: Set Up Equations
Given: Algebraic expressions for angles
Apply: Angle relationships to create equations

Example: Adjacent angles (3x + 10)° and (2x - 5)°
form linear pair. Find x.

(3x + 10) + (2x - 5) = 180
5x + 5 = 180
5x = 175
x = 35°

Complex Angle Problems

Advanced Problem Types

Problem 1: Multiple Parallel Lines
Three parallel lines cut by two transversals
Given some angles, find others

l ← →

l ← →
l ← →

Use properties systematically:
- Corresponding angles
- Alternate angles
- Linear pairs
- Vertical angles

Problem 2: Polygon with Exterior Angles
Regular polygon where each exterior angle is 40°
How many sides?

Each exterior angle = 360°/n = 40°
n = 360°/40° = 9 sides (nonagon)

Problem 3: Angle Bisectors
Triangle with angle bisectors
Given some angles, find others

    A

B   C
     D

If AD bisects BAC and BAC = 60°
Then BAD = CAD = 30°

Problem 4: Inscribed Angles
Circle with inscribed angles
Use circle theorems

    A

B   C

Inscribed angle = (1/2) × central angle

Common Mistakes and Misconceptions

Typical Angle Errors

```
Common Angle Mistakes


Mistake 1: Confusing Angle Types
Wrong: "This 120° angle is acute"
Correct: "This 120° angle is obtuse"
(Acute < 90°, Obtuse > 90°)

Mistake 2: Incorrect Protractor Reading
Wrong: Reading inner scale when should read outer
Check: Which scale starts at 0° for your angle?

Mistake 3: Adding Angles Incorrectly
Wrong: 45° + 50° = 95° when angles overlap
Correct: Consider whether angles are adjacent or overlapping

Mistake 4: Parallel Line Confusion
Wrong: "Corresponding angles are supplementary"
Correct: "Corresponding angles are equal" (when lines are parallel)

Mistake 5: Polygon Angle Formula Errors
Wrong: Sum of interior angles = n × 180°
Correct: Sum of interior angles = (n - 2) × 180°

Mistake 6: Degree/Radian Confusion
Wrong: sin(30) = 0.5 (using degrees in radian mode)
Correct: sin(30°) = 0.5 or sin( /6) = 0.5

Prevention Strategies:
- Draw clear, labeled diagrams
- Double-check protractor alignment
- Verify answers make geometric sense
- Practice angle relationships regularly
- Use multiple methods to check answers
- Be careful with calculator mode (degrees vs radians)
```

Building Angle Intuition

Angle Estimation Skills

```
Developing Angle Sense
```

Benchmark Angles:
Learn to recognize common angles by sight

30°:

45°:

60°:

90°:

```
        ----
```

120°:

```
         ------
```

Estimation Practice:
1. Look at angle
2. Compare to benchmarks
3. Estimate measure
4. Check with protractor

Real-World Angle Recognition:
- Clock hands: 3:00 = 90°, 6:00 = 180°
- Stairs: typically 30-35°
- Roof pitch: 15-45°
- Road grades: 3-8° (steep hills)

Body Angle References:
- Straight arm: 180°
- Right angle: 90° (arm to body)
- Comfortable sitting: 110-120°
- Walking stride: 30-40°

Mental Rotation:
Practice visualizing angle rotations
- Start with 0°
- Rotate mentally to target angle
- Check with physical rotation

Conclusion

Angles are fundamental to understanding geometric relationships, spatial reasoning, and mathematical problem-solving. They provide the language for describing rotation, direction, and the relationships between lines and shapes.

```
Angles: Complete Understanding


Conceptual Understanding:
  Angle formation and components
  Angle types and classifications
  Relationships between angles

Procedural Fluency:
  Measuring and constructing angles
  Using angle relationships to solve problems
  Working with parallel lines and transversals

Strategic Competence:
  Choosing appropriate angle relationships
  Setting up equations with angles
  Using angles in polygon problems

Adaptive Reasoning:
  Understanding why angle relationships work
  Making connections between different concepts
  Applying angles to real-world situations

Productive Disposition:
  Confidence with angle measurements
  Appreciation for geometric precision
  Curiosity about angular relationships
```

From ancient astronomers tracking celestial movements to modern engineers designing precision machinery, angles provide essential tools for describing and manipulating the spatial relationships that surround us. Whether you're navigating by compass, designing a building, creating art with perspective, or simply trying to understand the geometry of everyday objects, a solid understanding of angles opens doors to deeper geometric insight.

The study of angles reveals the elegant mathematical relationships that govern rotation, direction, and spatial orientation. As you continue exploring geometry, you'll find that angles appear everywhere - in the symmetries of crystals, the mechanics of gears, the optics of lenses, and the architecture of both natural and human-made structures. Mastering angles provides a crucial foundation for all advanced geometric thinking.

Triangles: The Strongest Shape

Introduction

The triangle is the simplest polygon and arguably the most important shape in geometry. With just three sides and three angles, triangles form the foundation for understanding more complex geometric figures and provide the structural basis for countless applications in engineering, architecture, art, and nature.

Triangles are unique among polygons - they are the only polygon that is inherently rigid. This property makes them the strongest shape for construction and the building block for analyzing all other polygons.

```
Triangle Fundamentals


    A
     \
      \
       \
        \
B    C

Three vertices: A, B, C
Three sides: AB, BC, CA
Three angles:  A,  B,  C

The sum of interior angles is always 180°
 A +  B +  C = 180°
```

Basic Triangle Properties

Triangle Inequality

The triangle inequality states that the sum of the lengths of any two sides of a triangle must be greater than the length of the third side.

```
Triangle Inequality Theorem
```

For triangle with sides a, b, c:
a + b > c
a + c > b
b + c > a

Example 1: Can sides 3, 4, 5 form a triangle?
Check: 3 + 4 = 7 > 5
       3 + 5 = 8 > 4
       4 + 5 = 9 > 3
Yes, they can form a triangle.

Example 2: Can sides 2, 3, 8 form a triangle?
Check: 2 + 3 = 5 < 8
No, they cannot form a triangle.

Visual Understanding:
    A

       5

   4
B   C
    3

To reach from B to C directly (distance 3),
the path B→A→C (distance 4 + 5 = 9) must be longer.
This is why 4 + 5 > 3.

Geometric Interpretation:
The shortest distance between two points is a straight line.
Any detour must be longer than the direct path.

Angle Sum Property

Triangle Angle Sum Theorem

The sum of interior angles in any triangle is 180°.

Proof by Parallel Lines:
    A

B   C

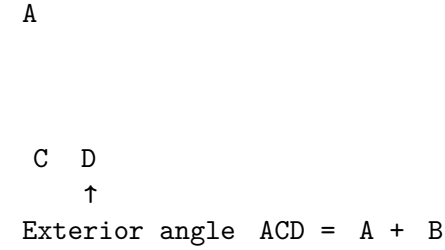
Draw line through A parallel to BC

Angles on straight line: 1 + A + 2 = 180°
But 1 = B (alternate interior angles)
And 2 = C (alternate interior angles)
Therefore: B + A + C = 180°

Applications:
If two angles are known, the third can be found:
C = 180° - A - B

Example: A = 60°, B = 70°
C = 180° - 60° - 70° = 50°

Exterior Angle Theorem:
An exterior angle equals the sum of the two non-adjacent interior angles.



Classification of Triangles

By Side Lengths

Triangle Types by Sides

Scalene Triangle:
All sides different lengths



Properties:
- No equal sides
- No equal angles
- Most general triangle type

```
Isosceles Triangle:
Two sides equal length
    A


       5
  5
B    C
    4

Properties:
- Two equal sides (legs): AB = AC
- Two equal angles (base angles):  B =  C
- Line of symmetry through vertex A
- Base angles theorem: If two sides are equal,
  then angles opposite those sides are equal

Equilateral Triangle:
All sides equal length
    A

       6

   6
B    C
    6

Properties:
- All sides equal: AB = BC = CA
- All angles equal:  A =  B =  C = 60°
- Three lines of symmetry
- Regular polygon (both equilateral and equiangular)
- Height = (√3/2) × side length
```

By Angle Measures

```
Triangle Types by Angles


Acute Triangle:
All angles less than 90°
    A

       70°

  60°
```

```
B    C
   50°

Properties:
- All angles acute (< 90°)
- All altitudes lie inside triangle
- Circumcenter inside triangle

Right Triangle:
One angle equals 90°
    A


     90°

B  C

Properties:
- One right angle (90°)
- Two acute angles
- Hypotenuse: longest side (opposite right angle)
- Legs: two shorter sides
- Pythagorean theorem applies: a² + b² = c²
- Altitude to hypotenuse creates similar triangles

Obtuse Triangle:
One angle greater than 90°
    A


         110°

B    C

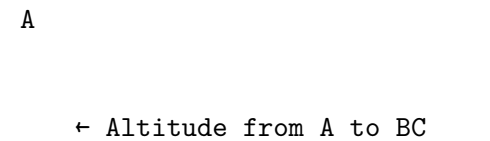
Properties:
- One obtuse angle (> 90°)
- Two acute angles
- Obtuse angle opposite longest side
- Circumcenter outside triangle
- Some altitudes lie outside triangle
```

Special Lines in Triangles

Altitudes

```
Triangle Altitudes
```

Altitude: Perpendicular from vertex to opposite side



B C
D

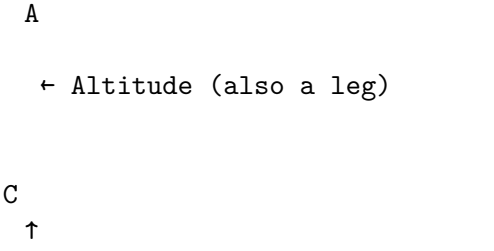
Properties:
- Every triangle has three altitudes
- Altitudes may lie inside, outside, or on the triangle
- All three altitudes meet at one point (orthocenter)

Acute Triangle Altitudes:



All altitudes inside triangle
Orthocenter inside triangle

Right Triangle Altitudes:



Two altitudes are the legs themselves
Orthocenter at the right angle vertex

Obtuse Triangle Altitudes:



```
B      C

  ← Altitude extended outside


Some altitudes lie outside triangle
Orthocenter outside triangle

Area Formula using Altitude:
Area = (1/2) × base × height
Area = (1/2) × BC × AD
```

Medians

```
Triangle Medians


Median: Line from vertex to midpoint of opposite side

    A


        ← Median from A to midpoint M

B    C
    M

Properties:
- Every triangle has three medians
- All medians lie inside the triangle
- Medians divide triangle into 6 smaller triangles of equal area
- All three medians meet at centroid

Centroid Properties:
- Point where all three medians intersect
- Center of mass (balance point) of triangle
- Divides each median in ratio 2:1
- Distance from vertex = (2/3) × median length
- Distance from side = (1/3) × median length

    A


   G    ← G is centroid
           AG:GM = 2:1
B    C
    M
```

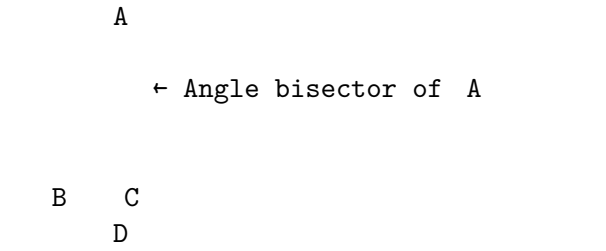
Median Length Formula:
For triangle with sides a, b, c:
Median to side a: $m_a = (1/2)\sqrt{(2b^2 + 2c^2 - a^2)}$

Example: Triangle with sides 3, 4, 5
Median to side 5: $m = (1/2)\sqrt{(2(3^2) + 2(4^2) - 5^2)}$
$= (1/2)\sqrt{(18 + 32 - 25)}$
$= (1/2)\sqrt{25} = 2.5$

Angle Bisectors

Triangle Angle Bisectors

Angle Bisector: Ray that divides angle into two equal parts



Properties:
- Every triangle has three angle bisectors
- All angle bisectors lie inside triangle
- All three angle bisectors meet at incenter
- Incenter is equidistant from all three sides
- Incenter is center of inscribed circle (incircle)

Angle Bisector Theorem:
The angle bisector divides the opposite side in the ratio of the adjacent sides.

$BD/DC = AB/AC$

Example: AB = 6, AC = 9, BC = 12
If AD bisects $\angle A$, then:
$BD/DC = 6/9 = 2/3$
Since BD + DC = 12:
$BD = 12 \times (2/5) = 4.8$
$DC = 12 \times (3/5) = 7.2$

Incircle Properties:
- Radius = Area/semiperimeter
- Touches all three sides

```
- Center at incenter
- Largest circle that fits inside triangle

    A


        ← Incenter I

B    C

Inscribed circle radius: r = Area/s
where s = (a + b + c)/2 (semiperimeter)
```

Perpendicular Bisectors

```
Triangle Perpendicular Bisectors


Perpendicular Bisector: Line perpendicular to side at its midpoint

    A




B    C
      ← Perpendicular bisector of BC


Properties:
- Every triangle has three perpendicular bisectors
- All points on perpendicular bisector are equidistant from endpoints
- All three perpendicular bisectors meet at circumcenter
- Circumcenter is equidistant from all three vertices
- Circumcenter is center of circumscribed circle (circumcircle)

Circumcenter Location:
Acute Triangle: Inside triangle
Right Triangle: On hypotenuse (midpoint)
Obtuse Triangle: Outside triangle

    A


        ← Circumcenter O

B    C
```

```
Circumcircle Properties:
- Passes through all three vertices
- Center at circumcenter
- Radius = distance from circumcenter to any vertex
- Smallest circle containing the triangle

Circumradius Formula:
R = (abc)/(4 × Area)

For right triangle: R = hypotenuse/2
```

Triangle Congruence

Congruence Postulates

```
Triangle Congruence Tests


Two triangles are congruent if they have the same size and shape.

SSS (Side-Side-Side):
If three sides of one triangle equal three sides of another triangle,
then the triangles are congruent.

Triangle 1:     Triangle 2:
    A               D

       5              5

   4              4
B    C       E    F
    3              3

If AB = DE, BC = EF, CA = FD, then  ABC   DEF

SAS (Side-Angle-Side):
If two sides and the included angle of one triangle equal
two sides and the included angle of another triangle,
then the triangles are congruent.

    A               D

       5              5
  60°            60°
```

```
B    C       E    F
    4              4

If AB = DE,  A =  D, AC = DF, then  ABC   DEF
```

ASA (Angle-Side-Angle):
If two angles and the included side of one triangle equal
two angles and the included side of another triangle,
then the triangles are congruent.

```
    A               D


   60°             60°

B    C       E    F
 70°  4   50°    70°  4   50°

If  A =  D, AC = DF,  C =  F, then  ABC   DEF
```

AAS (Angle-Angle-Side):
If two angles and a non-included side of one triangle equal
two angles and the corresponding non-included side of another triangle,
then the triangles are congruent.

RHS (Right angle-Hypotenuse-Side):
For right triangles: If the hypotenuse and one leg of one right triangle
equal the hypotenuse and corresponding leg of another right triangle,
then the triangles are congruent.

```
    A               D

     5              5


B  C           E  F
  3               3

If AC = DF (hypotenuse), BC = EF (leg),  B =  E = 90°,
then  ABC   DEF
```

Non-Congruence Cases

Insufficient Conditions for Congruence

SSA (Side-Side-Angle) - Not sufficient:

```
Two sides and a non-included angle may create:
- No triangle
- One triangle
- Two different triangles (ambiguous case)

Example: a = 5, b = 8,  A = 30°
    A               A


   30°             30°

B    C       B     C
    8                8

Two different triangles possible with same SSA conditions!

AAA (Angle-Angle-Angle) - Not sufficient:
Three angles determine shape but not size.
Triangles are similar but not necessarily congruent.

    A                D


   60°             60°

B    C        E     F
 70°     50°      70°      50°

Same angles, different sizes - similar but not congruent.
```

Triangle Similarity

Similarity Tests

```
Triangle Similarity Criteria


Two triangles are similar if they have the same shape (but not necessarily size).

AA (Angle-Angle):
If two angles of one triangle equal two angles of another triangle,
then the triangles are similar.

    A                D
```

```
  60°            60°

B   C       E   F
 70°     50°     70°     50°
```

A = D = 60°, B = E = 70° → ABC ~ DEF

SSS (Side-Side-Side):
If the ratios of corresponding sides are equal,
then the triangles are similar.

Triangle 1: sides 3, 4, 5
Triangle 2: sides 6, 8, 10
Ratios: 6/3 = 8/4 = 10/5 = 2
Therefore triangles are similar with scale factor 2.

SAS (Side-Angle-Side):
If two sides are proportional and the included angles are equal,
then the triangles are similar.

```
    A               D

       4               8
  60°            60°

B   C       E   F
    3               6
```

AB/DE = 3/6 = 1/2, AC/DF = 4/8 = 1/2, A = D = 60°
Therefore ABC ~ DEF

Properties of Similar Triangles:
- Corresponding angles are equal
- Corresponding sides are proportional
- Ratio of areas = (scale factor)^2
- Ratio of perimeters = scale factor

Applications of Similarity

Using Similar Triangles

Shadow Problems:
A person 6 feet tall casts a 4-foot shadow.
A tree casts a 20-foot shadow.
How tall is the tree?

```
Person: height/shadow = 6/4 = 3/2
Tree: height/shadow = h/20
Since ratios are equal: h/20 = 3/2
h = 20 × 3/2 = 30 feet

    Person          Tree

    6                 h


      4              20

Indirect Measurement:
To find width of river:
Set up similar triangles using accessible measurements

    A




B    C

      River

    D

Measure AB, BC, BD on accessible side
Calculate AC using similar triangles
AC represents river width

Scale Models:
Model airplane scale 1:48
If model wingspan is 10 inches,
actual wingspan = 10 × 48 = 480 inches = 40 feet

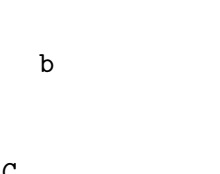
Map Scales:
Map scale 1:50,000
1 cm on map = 50,000 cm = 500 m in reality
Distance of 5 cm on map = 5 × 500 = 2,500 m = 2.5 km
```

Right Triangles

Pythagorean Theorem

```
The Pythagorean Theorem
```

In a right triangle, the square of the hypotenuse equals
the sum of squares of the other two sides.

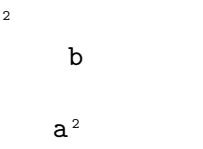


$a^2 + b^2 = c^2$

where c is the hypotenuse (longest side, opposite right angle)
and a, b are the legs

Proof by Area:
Large square area = $(a + b)^2$
Large square area = $c^2 + 4 \times (1/2)ab$
$(a + b)^2 = c^2 + 2ab$
$a^2 + 2ab + b^2 = c^2 + 2ab$
$a^2 + b^2 = c^2$

Visual Proof:



Pythagorean Triples:
Sets of three positive integers that satisfy $a^2 + b^2 = c^2$

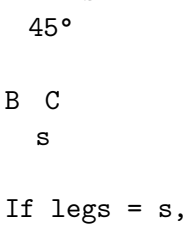
Common triples:
(3, 4, 5): $3^2 + 4^2 = 9 + 16 = 25 = 5^2$
(5, 12, 13): $5^2 + 12^2 = 25 + 144 = 169 = 13^2$
(8, 15, 17): $8^2 + 15^2 = 64 + 225 = 289 = 17^2$
(7, 24, 25): $7^2 + 24^2 = 49 + 576 = 625 = 25^2$

Multiples of basic triples:
If (a, b, c) is a triple, then (ka, kb, kc) is also a triple
(3, 4, 5) → (6, 8, 10), (9, 12, 15), (12, 16, 20), etc.

Special Right Triangles

45-45-90 Triangle

Isosceles right triangle with angles 45°, 45°, 90°



If legs = s, then hypotenuse = s√2

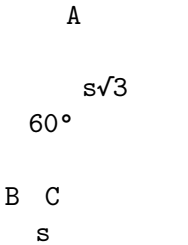
Ratio of sides: s : s : s√2 = 1 : 1 : √2

Example: If legs = 5, then hypotenuse = 5√2 ≈ 7.07

Applications:
- Square diagonal
- Isosceles right triangle problems

30-60-90 Triangle

Right triangle with angles 30°, 60°, 90°



If short leg (opposite 30°) = s,
then long leg (opposite 60°) = s√3,
and hypotenuse (opposite 90°) = 2s

Ratio of sides: s : s√3 : 2s = 1 : √3 : 2

Example: If short leg = 4,
then long leg = 4√3 ≈ 6.93,
and hypotenuse = 8

```
Applications:
- Equilateral triangle height
- Regular hexagon problems
- Trigonometry problems

Derivation from Equilateral Triangle:
    A

       2s

 60° 60°
B    C
  s   s

Height of equilateral triangle = s√3
This creates two 30-60-90 triangles
```

Triangle Area Formulas

Basic Area Formulas

```
Triangle Area Calculations


Formula 1: Base × Height
Area = (1/2) × base × height

    A


    h

B    C
    b

Area = (1/2) × b × h

Any side can be the base; height is perpendicular distance
to that base from opposite vertex.

Formula 2: Two Sides and Included Angle
Area = (1/2) × a × b × sin(C)

    A

       b
```

```
   C

B    C
    a

Area = (1/2) × a × b × sin(C)

Example: a = 5, b = 7, C = 60°
Area = (1/2) × 5 × 7 × sin(60°)
     = (1/2) × 5 × 7 × (√3/2)
     = 35√3/4   15.16

Formula 3: Heron's Formula
For triangle with sides a, b, c:
s = (a + b + c)/2 (semiperimeter)
Area = √[s(s-a)(s-b)(s-c)]

Example: Triangle with sides 3, 4, 5
s = (3 + 4 + 5)/2 = 6
Area = √[6(6-3)(6-4)(6-5)]
     = √[6 × 3 × 2 × 1]
     = √36 = 6

Formula 4: Coordinate Formula
For triangle with vertices (x ,y ), (x ,y ), (x ,y ):
Area = (1/2)|x (y -y ) + x (y -y ) + x (y -y )|

Example: Vertices (0,0), (4,0), (2,3)
Area = (1/2)|0(0-3) + 4(3-0) + 2(0-0)|
     = (1/2)|0 + 12 + 0|
     = 6
```

Area Relationships

```
Area Properties and Relationships


Median and Area:
Each median divides triangle into two triangles of equal area.

    A


   G    ← G is centroid

B    C
    M
```

Area(ABM) = Area(ACM) = (1/2) × Area(ABC)

The three medians divide triangle into 6 smaller triangles,
all with equal area = (1/6) × Area(ABC)

Altitude and Area:
Different altitudes give same area:
Area = (1/2) × a × h_a = (1/2) × b × h_b = (1/2) × c × h_c

Therefore: a × h_a = b × h_b = c × h_c

Similar Triangles and Area:
If triangles are similar with scale factor k,
then ratio of areas = k^2

Triangle 1: sides 3, 4, 5 → Area = 6
Triangle 2: sides 6, 8, 10 → Area = 24
Scale factor = 2, Area ratio = 2^2 = 4
Indeed: 24/6 = 4

Inscribed and Circumscribed Circles:
Area = r × s (where r = inradius, s = semiperimeter)
Area = (abc)/(4R) (where R = circumradius)

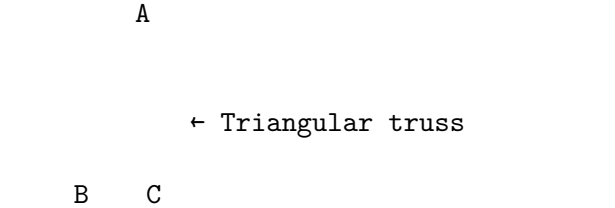
For right triangle with legs a, b and hypotenuse c:
Inradius: r = (a + b - c)/2
Circumradius: R = c/2

Applications and Problem Solving

Real-World Applications

Triangles in Engineering and Architecture

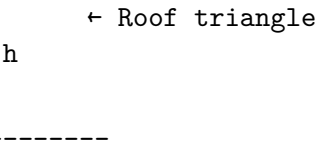
Structural Trusses:
Triangles provide maximum strength with minimum material



Support beam

Forces are distributed along triangle sides
Cannot be deformed without changing side lengths

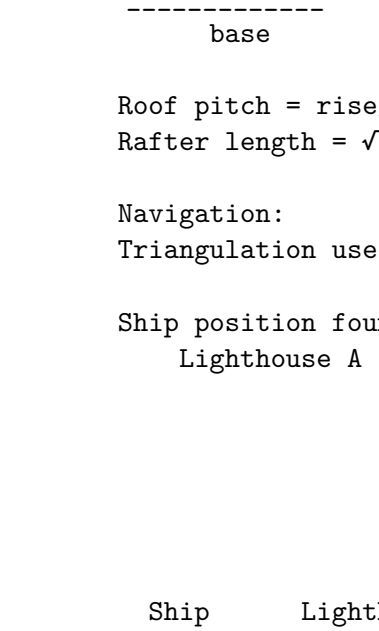
Roof Construction:



Roof pitch = rise/run = h/(base/2)
Rafter length = $\sqrt{[(\text{base}/2)^2 + h^2]}$

Navigation:
Triangulation uses triangles to determine position

Ship position found using angles to two known landmarks:



Measure angles at ship to both lighthouses
Use triangle properties to calculate distances

Surveying:
Land area calculated using triangulation
Divide irregular plot into triangles
Calculate area of each triangle
Sum for total area

Art and Design:
Golden triangle: isosceles triangle with ratio of leg to base = (golden ratio)
Used in classical architecture and art

Photography:
Rule of thirds creates triangular compositions
Leading lines form triangular patterns

Problem-Solving Strategies

Triangle Problem-Solving Techniques

Strategy 1: Identify Triangle Type
- Right triangle → Use Pythagorean theorem
- Isosceles → Use equal sides/angles
- Equilateral → All sides and angles equal
- Special right triangles → Use ratios

Strategy 2: Use Appropriate Formulas
- Area problems → Choose best area formula
- Side length problems → Law of cosines/sines
- Angle problems → Angle sum property

Strategy 3: Draw and Label Diagrams
- Mark given information
- Label unknowns clearly
- Add auxiliary lines if needed

Strategy 4: Look for Similar Triangles
- Same angles → Similar triangles
- Proportional sides → Set up ratios
- Use similarity to find unknowns

Example Problem:
"A ladder 10 feet long leans against a wall. The bottom of the ladder is 6 feet from the wall.

Solution:
1. Identify: Right triangle problem
2. Given: Hypotenuse = 10 ft, base = 6 ft
3. Find: Height (other leg)
4. Use Pythagorean theorem:
   $6^2 + h^2 = 10^2$
   $36 + h^2 = 100$
   $h^2 = 64$
   $h = 8$ feet

Strategy 5: Check Answers
- Do angles sum to 180°?
- Does triangle inequality hold?
- Are units correct?
- Is answer reasonable?

Common Mistakes and Misconceptions

Typical Triangle Errors

```
Common Triangle Mistakes


Mistake 1: Confusing Hypotenuse and Legs
Wrong: In right triangle with legs 3 and 4, hypotenuse = 3² + 4² = 25
Correct: Hypotenuse = √(3² + 4²) = √25 = 5

Mistake 2: Misapplying Pythagorean Theorem
Wrong: Using a² + b² = c² for non-right triangles
Correct: Pythagorean theorem only applies to right triangles

Mistake 3: Angle Sum Errors
Wrong: Triangle with angles 70°, 80°, 40° (sum = 190°)
Correct: Angles must sum to exactly 180°

Mistake 4: Triangle Inequality Violations
Wrong: Triangle with sides 2, 3, 8 (2 + 3 = 5 < 8)
Correct: Sum of any two sides must exceed third side

Mistake 5: Similarity vs Congruence
Wrong: "Triangles with same angles are congruent"
Correct: Same angles → similar; need equal sides for congruent

Mistake 6: Area Formula Confusion
Wrong: Area = base × height
Correct: Area = (1/2) × base × height

Prevention Strategies:
- Always check triangle inequality
- Verify angle sum equals 180°
- Draw accurate diagrams
- Double-check which formula applies
- Use multiple methods to verify answers
- Practice identifying triangle types
```

Building Triangle Intuition

Visualization Exercises

```
Developing Triangle Sense
```

```
Exercise 1: Triangle Construction
Given three side lengths, can they form a triangle?
Practice with: (3,4,5), (1,2,4), (5,5,8), (2,3,6)

Exercise 2: Angle Estimation
Look at triangles and estimate angles
Check: Do they sum to 180°?
Identify: Acute, right, or obtuse?

Exercise 3: Special Triangle Recognition
Identify 45-45-90 and 30-60-90 triangles
Practice using their special ratios

Exercise 4: Area Comparison
Which triangle has larger area?
- Same base, different heights
- Same area, different shapes
- Similar triangles with different scales

Exercise 5: Real-World Triangles
Find triangles in:
- Architecture (roof trusses, bridges)
- Nature (mountain peaks, tree shapes)
- Art (compositions, patterns)
- Sports (playing fields, equipment)

Exercise 6: Triangle Transformations
Start with triangle ABC
- Reflect over a line
- Rotate around a point
- Scale by factor k
- What properties are preserved?
```

Conclusion

Triangles are the fundamental building blocks of geometry, combining simplicity with remarkable mathematical richness. Their unique properties - rigidity, angle sum of 180°, and diverse classification systems - make them essential for understanding more complex geometric concepts.

```
Triangles: Complete Understanding


Conceptual Understanding:
  Triangle inequality and angle sum properties
  Classification by sides and angles
  Special lines and points in triangles
```

```
Procedural Fluency:
  Congruence and similarity tests
  Area calculations using multiple methods
  Pythagorean theorem applications

Strategic Competence:
  Choosing appropriate triangle relationships
  Problem-solving with similar triangles
  Using special right triangle ratios

Adaptive Reasoning:
  Understanding why triangle properties work
  Making connections between different concepts
  Applying triangles to real-world situations

Productive Disposition:
  Confidence with triangle calculations
  Appreciation for geometric relationships
  Recognition of triangles in the world around us
```

From ancient Egyptian pyramid builders to modern structural engineers, from artists using triangular compositions to GPS systems using triangulation, triangles provide essential tools for understanding and manipulating the geometric world around us.

The study of triangles reveals fundamental principles that extend far beyond geometry - concepts of stability, optimization, and mathematical proof that appear throughout mathematics and science. Whether you're calculating the height of a building using shadows, designing a bridge truss, or simply trying to understand the geometric relationships in a work of art, triangles provide the mathematical foundation for spatial reasoning and problem-solving.

As you continue your geometric journey, remember that triangles are not just abstract mathematical objects - they are the structural elements that give strength to buildings, the navigational tools that guide ships and planes, and the artistic elements that create visual harmony. Mastering triangles opens the door to understanding the geometric principles that shape our physical world.

Quadrilaterals: Four-Sided Figures

Introduction

A quadrilateral is a polygon with four sides, four vertices, and four interior angles. Quadrilaterals are among the most common and practical shapes in geometry, appearing everywhere from the pages of books to the walls of buildings, from computer screens to playing fields.

Understanding quadrilaterals is essential for geometry because they bridge the gap between the fundamental triangle and more complex polygons, while also providing the basis for understanding area, perimeter, and spatial relationships in two dimensions.

```
Quadrilateral Fundamentals


    A       B



    D       C

Four vertices: A, B, C, D
Four sides: AB, BC, CD, DA
Four angles:  A,  B,  C,  D

Sum of interior angles = 360°
 A +  B +  C +  D = 360°
```

Basic Properties of Quadrilaterals

Angle Sum Property

```
Quadrilateral Angle Sum Theorem


The sum of interior angles in any quadrilateral is 360°.

Proof by Triangulation:
    A       B
```

```
    D       C

Draw diagonal AC, dividing quadrilateral into two triangles.
Triangle ABC:  BAC +  ABC +  BCA = 180°
Triangle ACD:  CAD +  ACD +  CDA = 180°

Adding: ( BAC +  CAD) +  ABC + ( BCA +  ACD) +  CDA = 360°
This gives:  A +  B +  C +  D = 360°

Applications:
If three angles are known, the fourth can be found:
 D = 360° -  A -  B -  C

Example:  A = 90°,  B = 110°,  C = 80°
 D = 360° - 90° - 110° - 80° = 80°

Exterior Angle Sum:
Like all polygons, the sum of exterior angles = 360°
```

Diagonals in Quadrilaterals

```
Quadrilateral Diagonals


Every quadrilateral has exactly two diagonals.

    A       B









    D       C

Diagonals: AC and BD

Diagonal Properties (vary by quadrilateral type):
- Length
- Intersection point
- Angle of intersection
- Whether they bisect each other
```

General Properties:
- Diagonals divide quadrilateral into four triangles
- Sum of areas of opposite triangles may be equal
- Diagonals may or may not be equal in length
- Diagonals may or may not bisect each other
- Diagonals may or may not be perpendicular

Area using Diagonals:
For quadrilateral with diagonals d and d intersecting at angle :
Area = (1/2) × d × d × sin( )

Special case: If diagonals are perpendicular (= 90°):
Area = (1/2) × d × d

Classification of Quadrilaterals

The Quadrilateral Family Tree

Quadrilateral Hierarchy

Quadrilateral

Trapezoid Parallelogram Kite

Isosceles Rectangle Rhombus
Trapezoid

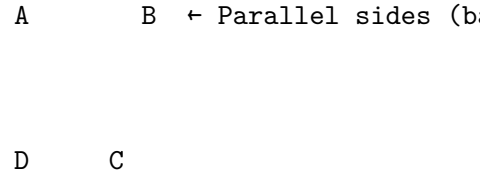
Square

Each level inherits properties from levels above.
Square has properties of rectangle, rhombus, parallelogram, and quadrilateral.

Trapezoids

Trapezoid Properties

Definition: Quadrilateral with exactly one pair of parallel sides.



Properties:
- One pair of parallel sides (bases): AB || DC
- Non-parallel sides are called legs: AD and BC
- Base angles: angles adjacent to same base
- Median (midsegment): line connecting midpoints of legs

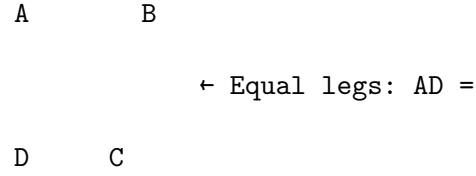
Median Properties:
- Parallel to both bases
- Length = average of base lengths
- Median length = (AB + DC)/2

Area Formula:
Area = (1/2) × (sum of parallel sides) × height
Area = (1/2) × (b_1 + b_2) × h



Example: b_1 = 8, b_2 = 12, h = 5
Area = (1/2) × (8 + 12) × 5 = 50 square units

Isosceles Trapezoid:
Special trapezoid where legs are equal length.

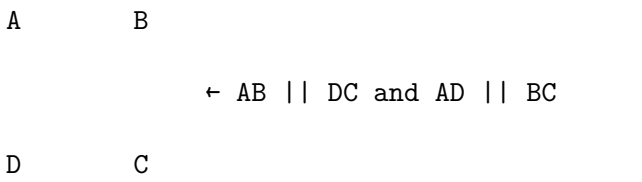


Additional properties:
- Base angles are equal: $\angle A = \angle D$, $\angle B = \angle C$
- Diagonals are equal: AC = BD
- Line of symmetry perpendicular to bases

Parallelograms

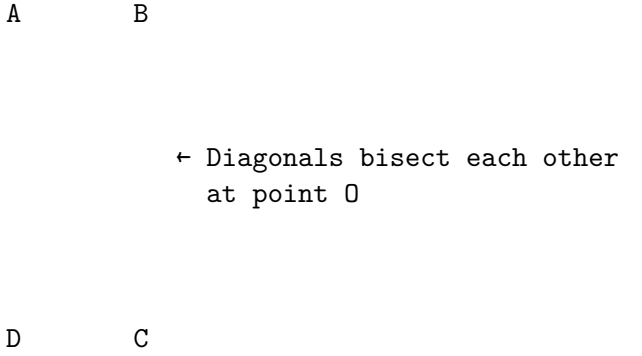
Parallelogram Properties

Definition: Quadrilateral with both pairs of opposite sides parallel.



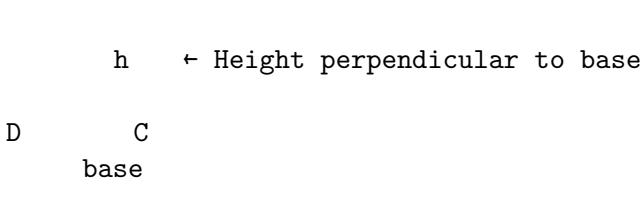
Properties:
1. Opposite sides are parallel: AB || DC, AD || BC
2. Opposite sides are equal: AB = DC, AD = BC
3. Opposite angles are equal: A = C, B = D
4. Consecutive angles are supplementary: A + B = 180°
5. Diagonals bisect each other

Diagonal Properties:



AO = OC and BO = OD

Area Formulas:
1. Base × Height: Area = base × height
2. Two sides and included angle: Area = ab sin( )
3. Diagonals: Area = (1/2) × d × d × sin( )



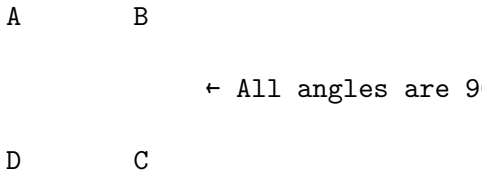
Area = base × h

Proving a Quadrilateral is a Parallelogram:
1. Both pairs of opposite sides are parallel
2. Both pairs of opposite sides are equal
3. One pair of opposite sides is both parallel and equal
4. Both pairs of opposite angles are equal
5. Diagonals bisect each other

Rectangles

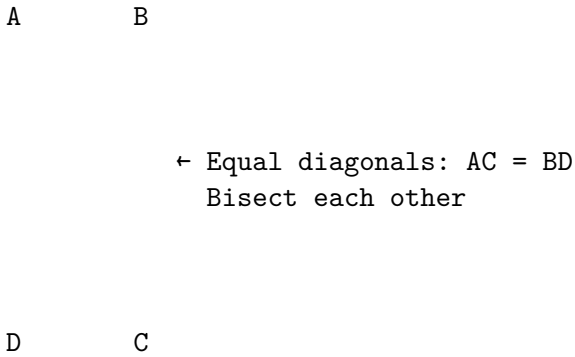
Rectangle Properties

Definition: Parallelogram with four right angles.

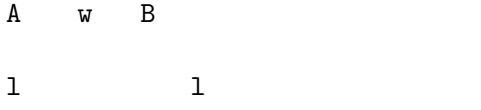


Properties (inherits all parallelogram properties plus):
1. All angles are right angles: A = B = C = D = 90°
2. Diagonals are equal in length: AC = BD
3. Diagonals bisect each other
4. Opposite sides are equal and parallel
5. Has two lines of symmetry

Diagonal Properties:



Area and Perimeter:
Area = length × width = lw
Perimeter = 2(length + width) = 2(l + w)



```
    D    w    C

Diagonal Length:
Using Pythagorean theorem: d = √(l² + w²)

Golden Rectangle:
Rectangle where length/width =   (golden ratio   1.618)
- Appears in art, architecture, nature
- Has pleasing proportions to human eye
- Can be subdivided into square and smaller golden rectangle
```

Rhombus

```
Rhombus Properties


Definition: Parallelogram with four equal sides.

    A       B

                ← All sides equal: AB = BC = CD = DA

    D       C

Properties (inherits all parallelogram properties plus):
1. All sides are equal: AB = BC = CD = DA
2. Diagonals are perpendicular: AC   BD
3. Diagonals bisect the vertex angles
4. Diagonals bisect each other
5. Has two lines of symmetry (along diagonals)

Diagonal Properties:
    A       B



                ← Diagonals perpendicular
                  and bisect each other



    D       C

 AOB =  BOC =  COD =  DOA = 90°

Area Formulas:
1. Base × Height: Area = base × height
```

2. Diagonals: Area = (1/2) × d × d
3. Side and angle: Area = s^2 × sin( )

Using diagonals (most common):
Area = (1/2) × AC × BD

Example: Diagonals 6 and 8
Area = (1/2) × 6 × 8 = 24 square units

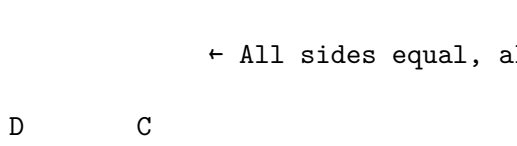
Perimeter:
Perimeter = 4s (where s is side length)

Relationship to Square:
A rhombus with right angles is a square.
A square is a special case of rhombus.

Squares

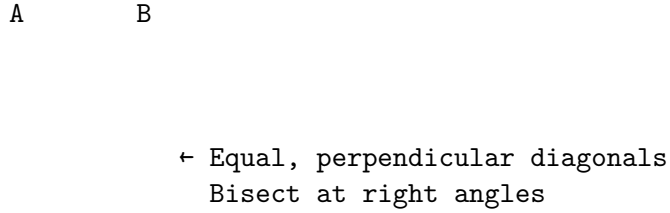
Square Properties

Definition: Rectangle with four equal sides (or rhombus with right angles).



Properties (inherits all rectangle and rhombus properties):
1. All sides equal: AB = BC = CD = DA
2. All angles are right angles: A = B = C = D = 90°
3. Diagonals are equal: AC = BD
4. Diagonals are perpendicular: AC BD
5. Diagonals bisect each other at right angles
6. Diagonals bisect vertex angles (each 45°)
7. Has four lines of symmetry

Diagonal Properties:



```
    D       C

Diagonal length: d = s√2 (where s is side length)

Area and Perimeter:
Area = s² (side squared)
Perimeter = 4s

Example: Side length = 5
Area = 5² = 25 square units
Perimeter = 4 × 5 = 20 units
Diagonal = 5√2  7.07 units

Symmetries:
- 4 lines of reflection symmetry
- Rotational symmetry: 90°, 180°, 270°
- Point symmetry about center
```

Kites

```
Kite Properties


Definition: Quadrilateral with two pairs of adjacent sides equal.

    A


          ← AB = AD (one pair)

            B


         ← CB = CD (other pair)

    D     C

Properties:
1. Two pairs of adjacent sides are equal: AB = AD, CB = CD
2. One diagonal bisects the other at right angles
3. One diagonal bisects the vertex angles
4. One line of symmetry (along one diagonal)
5. One pair of opposite angles are equal

Diagonal Properties:
    A
```

```
            B


            ← AC bisects BD at right angles
              AC bisects  A and  C
    D     C

AC   BD, and AC bisects BD
 BAC =  DAC,  BCA =  DCA
```

Area Formula:
Area = (1/2) × d × d
where d and d are the diagonal lengths

Example: Diagonals 8 and 6
Area = (1/2) × 8 × 6 = 24 square units

Special Cases:
- Rhombus: kite with all sides equal
- Square: kite with all sides equal and all angles right angles

Concave Kite (Dart):

```
    A




        ← Reflex angle here
D           B

        C
```

Still has kite properties but one angle > 180°

Special Quadrilateral Theorems

Midpoint Theorems

Varignon's Theorem

```
The quadrilateral formed by connecting the midpoints of any quadrilateral
is always a parallelogram.

Original quadrilateral ABCD:
    A       B



    D       C

Midpoints P, Q, R, S:
    A   P   B


    S        Q



    D   R   C

Quadrilateral PQRS is always a parallelogram.

Properties of Varignon Parallelogram:
- Perimeter = sum of diagonals of original quadrilateral
- Area = half the area of original quadrilateral
- Sides are parallel to diagonals of original quadrilateral

Proof outline:
P and Q are midpoints, so PQ || AC and PQ = (1/2)AC
R and S are midpoints, so RS || AC and RS = (1/2)AC
Therefore PQ || RS and PQ = RS → PQRS is parallelogram

Special Cases:
- If original is rectangle → Varignon parallelogram is rhombus
- If original is rhombus → Varignon parallelogram is rectangle
- If original is square → Varignon parallelogram is square
```

Diagonal Relationships

```
Quadrilateral Classification by Diagonals



Diagonal properties can determine quadrilateral type:

Equal Diagonals:
- Rectangle: diagonals equal and bisect each other
- Isosceles trapezoid: diagonals equal
```

- Square: diagonals equal, perpendicular, bisect each other

Perpendicular Diagonals:
- Rhombus: diagonals perpendicular and bisect each other
- Kite: diagonals perpendicular, one bisects the other
- Square: diagonals perpendicular, equal, bisect each other

Bisecting Diagonals:
- Parallelogram: diagonals bisect each other
- Rectangle: diagonals equal and bisect each other
- Rhombus: diagonals perpendicular and bisect each other
- Square: all diagonal properties

Summary Table:

Quadrilateral	Equal	Perpendicular	Bisect Each Other
General	No	No	No
Trapezoid	No	No	No
Isosceles Trap.	Yes	No	No
Parallelogram	No	No	Yes
Rectangle	Yes	No	Yes
Rhombus	No	Yes	Yes
Square	Yes	Yes	Yes
Kite	No	Yes	One bisects

Area and Perimeter Formulas

Area Formulas Summary

Quadrilateral Area Formulas

General Quadrilateral:
1. Divide into triangles: Area = sum of triangle areas
2. Shoelace formula (coordinates):
   Area = (1/2)| (x y - x y)|
3. Diagonals: Area = (1/2) × d × d × sin( )

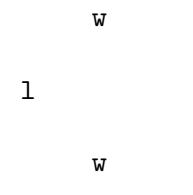
Trapezoid:
Area = (1/2) × (b + b) × h
where b , b are parallel sides, h is height



Parallelogram:
Area = base × height = bh
Area = ab sin( ) (two sides and included angle)



Rectangle:
Area = length × width = lw



Rhombus:
Area = base × height = bh
Area = (1/2) × d × d (diagonals)
Area = s^2 sin( ) (side and angle)

Square:
Area = $side^2$ = s^2
Area = (1/2) × d^2 (diagonal)

Kite:
Area = (1/2) × d × d (diagonals)



Perimeter Formulas

Quadrilateral Perimeter Formulas

General Quadrilateral:
Perimeter = a + b + c + d (sum of all sides)

Trapezoid:
Perimeter = a + b + c + b
where b , b are parallel sides, a, c are legs

Parallelogram:
Perimeter = 2(a + b) = 2a + 2b
where a, b are adjacent sides

Rectangle:
Perimeter = 2(length + width) = 2(l + w)

Rhombus:
Perimeter = 4s (where s is side length)

Square:
Perimeter = 4s (where s is side length)

Kite:
Perimeter = 2(a + b)
where a, b are the lengths of the two different sides

Example Calculations:
Rectangle: l = 8, w = 5
Perimeter = 2(8 + 5) = 26 units
Area = 8 × 5 = 40 square units

Rhombus: s = 6, diagonals d = 8, d = 10
Perimeter = 4 × 6 = 24 units
Area = (1/2) × 8 × 10 = 40 square units

Trapezoid: b = 6, b = 10, h = 4, legs = 5 each
Perimeter = 6 + 10 + 5 + 5 = 26 units
Area = (1/2) × (6 + 10) × 4 = 32 square units

Coordinate Geometry of Quadrilaterals

Using Coordinates

```
Quadrilateral Analysis with Coordinates


Given vertices A(x ,y ), B(x ,y ), C(x ,y ), D(x ,y )

Distance Formula:
Side length AB = √[(x -x )² + (y -y )²]

Midpoint Formula:
Midpoint of AB = ((x +x )/2, (y +y )/2)

Slope Formula:
Slope of AB = (y -y )/(x -x )

Parallel Lines: Equal slopes
Perpendicular Lines: Slopes are negative reciprocals

Example: Prove ABCD is a rectangle
A(0,0), B(4,0), C(4,3), D(0,3)

Step 1: Find side lengths
AB = √[(4-0)² + (0-0)²] = 4
BC = √[(4-4)² + (3-0)²] = 3
CD = √[(0-4)² + (3-3)²] = 4
DA = √[(0-0)² + (0-3)²] = 3

Step 2: Check opposite sides equal
AB = CD = 4
BC = DA = 3

Step 3: Check angles (using slopes)
Slope AB = 0, Slope BC = undefined (vertical)
AB  BC (horizontal  vertical)
All angles are 90°

Therefore ABCD is a rectangle.

Shoelace Formula for Area:
Area = (1/2)|x (y -y ) + x (y -y ) + x (y -y ) + x (y -y )|

For rectangle above:
Area = (1/2)|0(0-3) + 4(3-0) + 4(3-0) + 0(0-3)|
     = (1/2)|0 + 12 + 12 + 0| = 12 square units
```

Check: Area = length × width = 4 × 3 = 12

Transformations of Quadrilaterals

Quadrilateral Transformations

Translation (Slide):
Add same values to all coordinates
A(x,y) → A'(x+h, y+k)

Original square: A(0,0), B(2,0), C(2,2), D(0,2)
Translate by (3,1): A'(3,1), B'(5,1), C'(5,3), D'(3,3)

Properties preserved:
- Shape and size
- Parallel relationships
- Angle measures
- Area and perimeter

Reflection:
Over x-axis: (x,y) → (x,-y)
Over y-axis: (x,y) → (-x,y)
Over line y=x: (x,y) → (y,x)

Rectangle A(1,1), B(4,1), C(4,3), D(1,3)
Reflect over x-axis: A'(1,-1), B'(4,-1), C'(4,-3), D'(1,-3)

Rotation:
90° counterclockwise about origin: (x,y) → (-y,x)
180° about origin: (x,y) → (-x,-y)
270° counterclockwise about origin: (x,y) → (y,-x)

Square A(0,0), B(2,0), C(2,2), D(0,2)
Rotate 90° CCW: A'(0,0), B'(0,2), C'(-2,2), D'(-2,0)

Dilation (Scale):
Scale factor k: (x,y) → (kx,ky)

Rectangle A(1,1), B(3,1), C(3,2), D(1,2)
Scale by factor 2: A'(2,2), B'(6,2), C'(6,4), D'(2,4)

Properties:
- Shape preserved
- Size changes by factor k
- Area changes by factor k^2

- Perimeter changes by factor k

Applications and Problem Solving

Real-World Applications

Quadrilaterals in Architecture and Design

Building Design:
- Rectangular rooms for efficiency
- Square courtyards for symmetry
- Trapezoidal roofs for drainage
- Rhombus patterns in tilework

Floor Planning:
Room area = length × width
Carpet needed = floor area
Baseboard needed = perimeter − doorway widths

Example: Room 12 ft × 15 ft with 3 ft doorway
Area = 12 × 15 = 180 sq ft
Perimeter = 2(12 + 15) = 54 ft
Baseboard = 54 − 3 = 51 ft

Sports Fields:
- Soccer field: rectangle ~100m × 60m
- Baseball diamond: square with 90 ft sides
- Tennis court: rectangle 78 ft × 36 ft

Land Surveying:
Property boundaries often form quadrilaterals
Area calculation for:
- Property taxes
- Development planning
- Agricultural use

Irregular quadrilateral property:
Divide into triangles or use coordinate methods
Sum triangle areas for total property area

Art and Design:
- Golden rectangle in classical art
- Square formats in photography
- Rhombus patterns in Islamic art
- Parallelogram perspective in drawing

Engineering:
- Truss design using triangulated quadrilaterals
- Bridge deck sections (rectangular)
- Gear teeth (trapezoidal profiles)
- Solar panel arrays (rectangular grids)

Problem-Solving Strategies

Quadrilateral Problem-Solving Techniques

Strategy 1: Identify the Type
- Look for parallel sides
- Check for equal sides
- Measure angles
- Examine diagonal properties

Strategy 2: Use Appropriate Formulas
- Area: choose formula based on given information
- Perimeter: sum of sides or use shortcuts
- Diagonal lengths: use coordinate geometry or Pythagorean theorem

Strategy 3: Apply Properties
- Opposite sides equal in parallelograms
- Diagonals bisect each other in parallelograms
- All angles 90° in rectangles
- All sides equal in rhombus

Strategy 4: Use Coordinate Methods
- Place quadrilateral in coordinate system
- Use distance, midpoint, slope formulas
- Apply transformations if needed

Example Problem:
"A parallelogram has sides of 8 and 12 units, with an included angle of 60°. Find the area and

Solution:
Area = ab sin( ) = 8 × 12 × sin(60°) = 96 × (√3/2) = 48√3 83.14 sq units

For diagonals, use law of cosines:
$d^2 = a^2 + b^2 - 2ab\cos(\ ) = 8^2 + 12^2 - 2(8)(12)\cos(60°)$
$= 64 + 144 - 192(0.5) = 208 - 96 = 112$
$d = \sqrt{112} = 4\sqrt{7}$ 10.58 units

$d^2 = a^2 + b^2 - 2ab\cos(180° - \ ) = 8^2 + 12^2 - 2(8)(12)\cos(120°)$
$= 64 + 144 - 192(-0.5) = 208 + 96 = 304$

d = √304 = 4√19 17.44 units

Strategy 5: Check Your Work
- Do angles sum to 360°?
- Are parallel sides actually parallel?
- Does area make sense?
- Are units correct?

Common Mistakes and Misconceptions

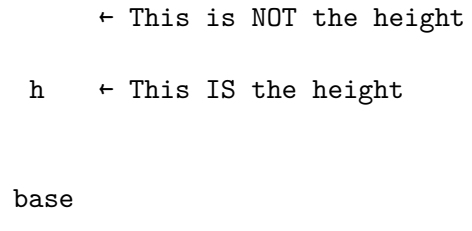
Typical Quadrilateral Errors

Common Quadrilateral Mistakes

Mistake 1: Confusing Quadrilateral Types
Wrong: "All rectangles are squares"
Correct: "All squares are rectangles, but not all rectangles are squares"

Hierarchy confusion:
- Square Rectangle Parallelogram Quadrilateral
- Square Rhombus Parallelogram Quadrilateral

Mistake 2: Area Formula Confusion
Wrong: Parallelogram area = length × width
Correct: Parallelogram area = base × height (perpendicular height)



Mistake 3: Diagonal Properties
Wrong: "All parallelograms have equal diagonals"
Correct: "Only rectangles (and squares) have equal diagonals"

Wrong: "All quadrilaterals with perpendicular diagonals are squares"
Correct: "Rhombi and kites also have perpendicular diagonals"

Mistake 4: Angle Sum Errors
Wrong: Quadrilateral with angles 80°, 90°, 100°, 80° (sum = 350°)
Correct: Angles must sum to exactly 360°

```
Mistake 5: Perimeter vs Area Confusion
Wrong: "Doubling the sides doubles the area"
Correct: "Doubling the sides quadruples the area"

Example: Square with side 3
Original: Perimeter = 12, Area = 9
Doubled sides: Perimeter = 24, Area = 36 (4 times larger)

Prevention Strategies:
- Draw clear, labeled diagrams
- Learn the quadrilateral hierarchy
- Practice identifying types by properties
- Double-check angle sums
- Use multiple methods to verify answers
- Understand the difference between linear and area scaling
```

Building Quadrilateral Intuition

Recognition Exercises

```
Developing Quadrilateral Sense


Exercise 1: Property Identification
Given a quadrilateral, identify:
- Which sides are parallel?
- Which sides are equal?
- Which angles are equal?
- What do the diagonals do?

Exercise 2: Classification Practice
Look at quadrilaterals and classify as:
- General quadrilateral
- Trapezoid (isosceles or not)
- Parallelogram
- Rectangle
- Rhombus
- Square
- Kite

Exercise 3: Real-World Recognition
Find quadrilaterals in:
- Architecture (windows, doors, rooms)
- Art (paintings, patterns, designs)
- Nature (crystal structures, leaf shapes)
- Technology (screens, keyboards, panels)
```

```
Exercise 4: Construction Challenges
Using compass and straightedge:
- Construct a square given one side
- Construct a rectangle with given dimensions
- Construct a rhombus with given side and angle
- Construct a parallelogram with given sides and angle

Exercise 5: Transformation Visualization
Start with a square:
- What happens when you stretch it horizontally?
- What if you shear it (keep one side fixed, slide opposite side)?
- How do the properties change?

Exercise 6: Area and Perimeter Relationships
- Which quadrilaterals can have the same area but different perimeters?
- Which have the same perimeter but different areas?
- What's the quadrilateral with maximum area for given perimeter?
```

Conclusion

Quadrilaterals represent a rich family of geometric shapes that bridge the fundamental simplicity of triangles with the complexity of higher-order polygons. Their diverse properties and relationships make them essential for understanding geometric principles and solving real-world problems.

```
Quadrilaterals: Complete Understanding


Conceptual Understanding:
  Quadrilateral hierarchy and relationships
  Properties of each quadrilateral type
  Diagonal characteristics and their significance

Procedural Fluency:
  Area and perimeter calculations
  Classification by properties
  Coordinate geometry applications

Strategic Competence:
  Choosing appropriate formulas and methods
  Using properties to solve problems
  Applying transformations and symmetries

Adaptive Reasoning:
  Understanding why properties hold
  Making connections between different types
```

```
  Recognizing quadrilaterals in various contexts

Productive Disposition:
  Confidence with quadrilateral problems
  Appreciation for geometric relationships
  Recognition of quadrilaterals in the world around us
```

From the rectangular pages of this book to the square tiles on floors, from the rhombus patterns in art to the trapezoidal cross-sections of bridges, quadrilaterals surround us in countless forms. Understanding their properties, relationships, and applications provides essential tools for geometric reasoning, architectural design, engineering analysis, and artistic creation.

The study of quadrilaterals reveals how mathematical classification systems help us organize and understand geometric relationships. Whether you're calculating the area of a room, designing a building facade, analyzing the efficiency of a solar panel array, or simply appreciating the geometric patterns in Islamic art, quadrilaterals provide the mathematical framework for understanding and working with four-sided shapes.

As you continue exploring geometry, remember that quadrilaterals demonstrate how mathematical concepts build upon each other - each type inheriting properties from more general categories while adding its own special characteristics. This hierarchical structure reflects the elegant organization underlying all of mathematics, where specific cases illuminate general principles and general principles help us understand specific applications.

Circles: Perfect Curves and Endless Possibilities

Introduction

A circle is the set of all points in a plane that are equidistant from a fixed point called the center. This simple definition gives rise to one of the most perfect and important shapes in mathematics, appearing everywhere from the wheels that move our vehicles to the orbits of planets around the sun.

Circles represent mathematical perfection - they have no corners, no beginning, no end, and infinite symmetry. They bridge geometry and algebra, connect to trigonometry and calculus, and provide the foundation for understanding curves, rotation, and periodic phenomena.

```
Circle Fundamentals


    A

          ← Radius (r)
           ← Center (O)
       O


         ← Diameter (d = 2r)
B             C







       D

All points on the circle are exactly distance r from center O
```

Basic Circle Elements

Fundamental Components

```
Circle Vocabulary
```

Center (O): Fixed point equidistant from all points on circle

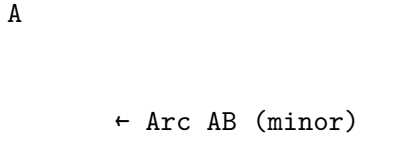
Radius (r): Distance from center to any point on circle
- All radii are equal in length
- Infinite number of radii possible

Diameter (d): Distance across circle through center
- Longest chord possible
- d = 2r
- All diameters are equal in length

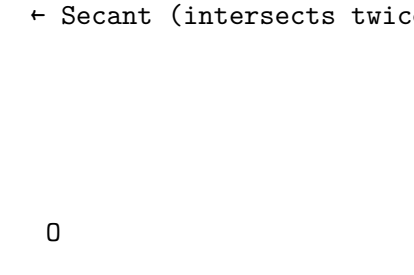
Chord: Line segment connecting any two points on circle



Arc: Part of the circumference between two points
- Minor arc: less than semicircle
- Major arc: greater than semicircle
- Semicircle: exactly half the circle



Secant: Line that intersects circle at two points
Tangent: Line that touches circle at exactly one point



```
      ↑
   Tangent (touches once)
```

Sector: "Pie slice" region bounded by two radii and an arc
Segment: Region between chord and arc

Circle Measurements

Circumference and Area

Circumference (C): Distance around the circle
C = 2 r = d

where (pi) 3.14159...

Historical note: is the ratio of circumference to diameter for ANY circle, discovered by ancient mathematicians.

Area (A): Space inside the circle
A = r^2

Derivation of area formula:
Imagine circle divided into many thin triangles:

```
   O
 -------- -- --
```

Each triangle has base small arc length, height = r
Total area (1/2) × (sum of arc lengths) × r
        = (1/2) × circumference × r
        = (1/2) × 2 r × r = r^2

Examples:
Circle with radius 5:
Circumference = 2 (5) = 10 31.42 units
Area = $(5)^2$ = 25 78.54 square units

Circle with diameter 12:
Radius = 6
Circumference = (12) = 12 37.70 units
Area = $(6)^2$ = 36 113.10 square units

Arc Length:
For arc with central angle (in radians):

Arc length = r

For arc with central angle (in degrees):
Arc length = (/360°) × 2 r

Sector Area:
For sector with central angle (in radians):
Sector area = (1/2)r²

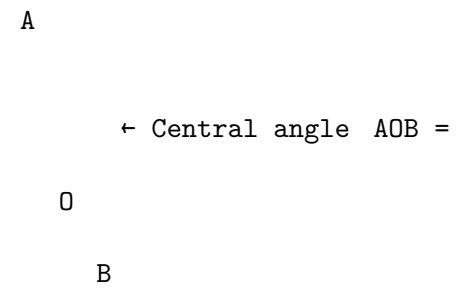
For sector with central angle (in degrees):
Sector area = (/360°) × r²

Angles in Circles

Central Angles

Central Angles

Central Angle: Angle with vertex at center of circle



Properties:
- Vertex at center O
- Sides are radii
- Intercepts arc AB
- Measure equals intercepted arc measure

Arc Measure:
Arc measure = central angle measure
If AOB = 60°, then arc AB = 60°

Full circle = 360°
Semicircle = 180°
Quarter circle = 90°

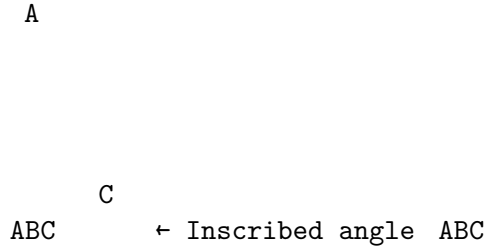
Example: Circle divided into 8 equal sectors
Each central angle = 360° ÷ 8 = 45°

Each arc = 45°

Inscribed Angles

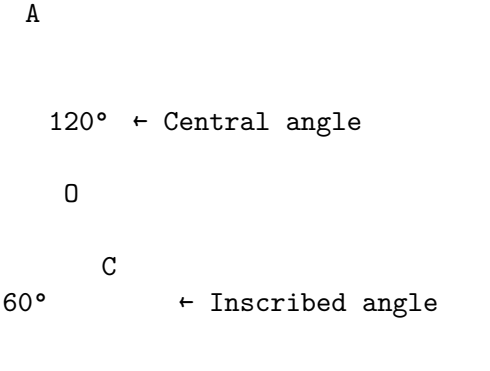
Inscribed Angles

Inscribed Angle: Angle with vertex on circle, sides are chords



Inscribed Angle Theorem:
An inscribed angle is half the central angle that subtends the same arc.

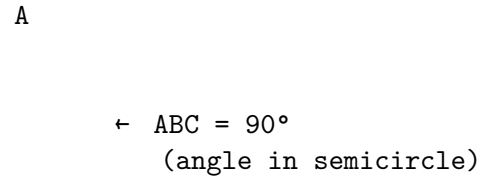
ABC = (1/2) × AOC



ABC = 60° = (1/2) × 120°

Corollaries:
1. All inscribed angles subtending the same arc are equal
2. An inscribed angle subtending a semicircle is 90°
3. Opposite angles in a cyclic quadrilateral sum to 180°

Angle in Semicircle:

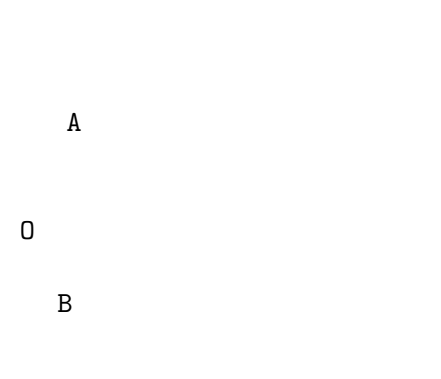


Any angle inscribed in a semicircle is a right angle.
This is because it subtends a 180° arc, so angle = 180°/2 = 90°.

Tangent-Chord Angles

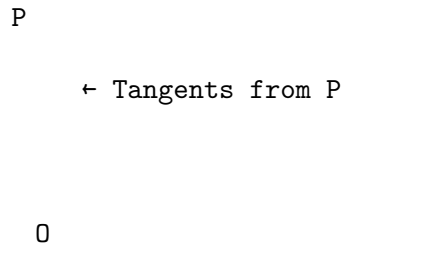
Angles Involving Tangents

Tangent-Chord Angle:
Angle between tangent and chord at point of tangency



ATB = (1/2) × arc AB

Tangent-Tangent Angle:
Angle between two tangents from external point



A B

APB = (1/2) × |arc AB - arc AB'|
where AB and AB' are the two arcs between tangent points

Properties of Tangents:
1. Tangent is perpendicular to radius at point of tangency
2. Two tangents from external point are equal in length
3. Tangent segments from external point to circle are equal

Power of a Point:
For point P outside circle with tangent PT:
$PT^2 = PA \times PB$ (where PAB is any secant through P)

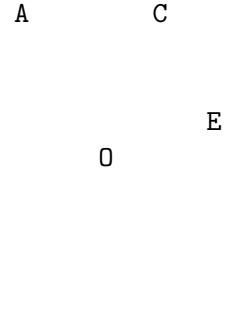
This relationship is constant regardless of which secant is chosen.

Circle Theorems

Chord Properties

Chord Theorems

Equal Chords Theorem:
In the same circle, equal chords subtend equal arcs and equal central angles.



If chord AB = chord CD, then:
- Arc AB = Arc CD
- AOB = COD

Perpendicular from Center to Chord:
The perpendicular from the center of a circle to a chord bisects the chord.

A

```
            ← OM ⊥ AB, so AM = MB
       O

         B
A      M
```

This gives us a way to find the distance from center to chord:
If chord length = 2c and radius = r, then:
Distance from center = $\sqrt{(r^2 - c^2)}$

Intersecting Chords Theorem:
When two chords intersect inside a circle:
PA × PB = PC × PD

```
    A


           C

       P      ← Intersection point

         B
D
```

PA × PB = PC × PD

This is another form of the "power of a point" theorem.

Cyclic Quadrilaterals

Cyclic Quadrilaterals

Cyclic Quadrilateral: Quadrilateral with all vertices on a circle

```
    A


           B

       O


D             C
```

Properties:

```
1. Opposite angles sum to 180°
   A + C = 180°, B + D = 180°

2. An exterior angle equals the opposite interior angle

3. The product of diagonals equals the sum of products of opposite sides:
   AC × BD = AB × CD + AD × BC (Ptolemy's Theorem)

Proof of opposite angles:
 A is inscribed angle subtending arc BCD
 C is inscribed angle subtending arc DAB
Arc BCD + Arc DAB = 360° (full circle)
So  A + C = (1/2)(Arc BCD) + (1/2)(Arc DAB) = 180°

Tests for Cyclic Quadrilateral:
1. Opposite angles sum to 180°
2. An exterior angle equals opposite interior angle
3. All vertices are equidistant from some point (circumcenter)

Ptolemy's Theorem:
For cyclic quadrilateral ABCD:
AC × BD = AB × CD + AD × BC

This gives a relationship between the sides and diagonals
that only holds for cyclic quadrilaterals.
```

Circle Constructions

Basic Constructions

```
Circle Constructions with Compass and Straightedge


Construction 1: Circle through Three Points
Given: Three non-collinear points A, B, C
Construct: Circle passing through all three points

Step 1: Find perpendicular bisector of AB
Step 2: Find perpendicular bisector of BC
Step 3: Intersection point O is circumcenter
Step 4: Draw circle with center O and radius OA

    A


          C
```

```
    B
```

The circumcenter is equidistant from all three points.

Construction 2: Tangent to Circle from External Point
Given: Circle with center O, external point P
Construct: Tangent lines from P to circle

Step 1: Draw line OP
Step 2: Find midpoint M of OP
Step 3: Draw circle with center M and radius MO
Step 4: This circle intersects original circle at tangent points
Step 5: Draw lines from P through tangent points

```
      P

           ← Tangent lines



        O
```

Construction 3: Inscribed Regular Hexagon
Given: Circle
Construct: Regular hexagon inscribed in circle

Step 1: Mark any point A on circle
Step 2: With compass set to radius, mark point B on circle
Step 3: Continue marking points C, D, E, F
Step 4: Connect consecutive points

```
    A

         B

      O
 F        C


         D
```

```
    E

The radius equals the side length of inscribed regular hexagon.

Construction 4: Circle Tangent to Three Lines
Given: Three lines forming a triangle
Construct: Inscribed circle (incircle)

Step 1: Find angle bisectors of two angles
Step 2: Intersection point I is incenter
Step 3: Drop perpendicular from I to any side
Step 4: Draw circle with center I and radius = perpendicular distance

The incenter is equidistant from all three sides.
```

Advanced Constructions

```
Complex Circle Constructions


Construction 5: Circle through Two Points with Given Radius
Given: Points A and B, radius r
Construct: Circle of radius r passing through A and B

Condition: Distance AB  2r (otherwise impossible)

Step 1: Draw circles of radius r centered at A and B
Step 2: Intersection points are possible centers
Step 3: Choose one intersection point as center
Step 4: Draw circle with chosen center and radius r

If AB = 2r, there's exactly one solution (A and B are diametrically opposite)
If AB < 2r, there are two solutions
If AB > 2r, there's no solution

Construction 6: Common Tangents to Two Circles
Given: Two circles with centers O  and O
Construct: Common tangent lines

External Tangents (don't cross between circles):
Step 1: Draw line O O
Step 2: Construct circle with center O  and radius |r  - r |
Step 3: From O , draw tangents to this circle
Step 4: These directions give external tangent directions

Internal Tangents (cross between circles):
```

Similar process using radius r + r

Number of common tangents:
- Separate circles: 4 tangents (2 external, 2 internal)
- Externally tangent: 3 tangents (2 external, 1 internal)
- Intersecting: 2 tangents (2 external, 0 internal)
- Internally tangent: 1 tangent
- One inside other: 0 tangents

Construction 7: Circle Tangent to Two Circles and a Line
This is one of the classic "Apollonius problems"
Requires advanced techniques involving inversion or analytic geometry

Coordinate Geometry of Circles

Circle Equations

Circle Equations in Coordinate Plane

Standard Form:
$(x - h)^2 + (y - k)^2 = r^2$

where (h, k) is center and r is radius

Example: Circle with center (3, -2) and radius 5
$(x - 3)^2 + (y + 2)^2 = 25$

General Form:
$x^2 + y^2 + Dx + Ey + F = 0$

Converting to standard form by completing the square:
$x^2 + Dx + y^2 + Ey = -F$
$(x + D/2)^2 - D^2/4 + (y + E/2)^2 - E^2/4 = -F$
$(x + D/2)^2 + (y + E/2)^2 = D^2/4 + E^2/4 - F$

Center: (-D/2, -E/2)
Radius: $\sqrt{(D^2/4 + E^2/4 - F)}$

Example: $x^2 + y^2 - 6x + 4y - 3 = 0$
Complete the square:
$(x^2 - 6x + 9) + (y^2 + 4y + 4) = 3 + 9 + 4$
$(x - 3)^2 + (y + 2)^2 = 16$

Center: (3, -2), Radius: 4

Parametric Form:
$x = h + r\cos(t)$
$y = k + r\sin(t)$

where t is parameter (angle from positive x-axis)
As t varies from 0 to 2 , point traces complete circle

Example: Circle center (0,0), radius 3
$x = 3\cos(t)$
$y = 3\sin(t)$

Points: t = 0 → (3,0), t = /2 → (0,3), t = → (-3,0), etc.

Circle Intersections

Intersections with Lines and Circles

Line-Circle Intersection:
Substitute line equation into circle equation

Example: Circle $x^2 + y^2 = 25$, Line $y = x + 1$
Substitute: $x^2 + (x + 1)^2 = 25$
$x^2 + x^2 + 2x + 1 = 25$
$2x^2 + 2x - 24 = 0$
$x^2 + x - 12 = 0$
$(x + 4)(x - 3) = 0$
$x = -4$ or $x = 3$

Points: (-4, -3) and (3, 4)

Number of intersections:
- 2 intersections: line is secant
- 1 intersection: line is tangent
- 0 intersections: line misses circle

Circle-Circle Intersection:
Solve system of two circle equations

Example:
Circle 1: $x^2 + y^2 = 25$
Circle 2: $(x - 3)^2 + (y - 4)^2 = 16$

Expand circle 2: $x^2 - 6x + 9 + y^2 - 8y + 16 = 16$
Simplify: $x^2 + y^2 - 6x - 8y + 9 = 0$

Subtract circle 1: $-6x - 8y + 9 = -25$

Solve for y: y = (3x - 17)/4

Substitute back into circle 1:
$x^2 + ((3x - 17)/4)^2 = 25$

This gives quadratic in x, solve for intersection points.

Number of intersections:
- 2 intersections: circles intersect at two points
- 1 intersection: circles are tangent
- 0 intersections: circles are separate or one inside other

Distance between centers determines relationship:
If d = distance between centers, r and r are radii:
- d > r + r : separate circles
- d = r + r : externally tangent
- |r - r | < d < r + r : intersecting
- d = |r - r |: internally tangent
- d < |r - r |: one inside other

Transformations of Circles

Circle Transformations

Translation:
Circle $(x - h)^2 + (y - k)^2 = r^2$
Translate by (a, b): $(x - h - a)^2 + (y - k - b)^2 = r^2$
New center: (h + a, k + b), same radius

Reflection:
Over x-axis: $(x - h)^2 + (y + k)^2 = r^2$
Over y-axis: $(x + h)^2 + (y - k)^2 = r^2$
Over line y = x: $(y - h)^2 + (x - k)^2 = r^2$

Dilation (Scaling):
Scale by factor k: $(x - h)^2 + (y - k)^2 = r^2$
becomes: $((x - h)/k)^2 + ((y - k)/k)^2 = r^2$
or: $(x - h)^2 + (y - k)^2 = (kr)^2$

New radius: kr, same center

Rotation:
Rotation about origin by angle :
x' = x cos( ) - y sin( )
y' = x sin( ) + y cos( )

Circle $x^2 + y^2 = r^2$ remains $x^2 + y^2 = r^2$ (unchanged)
Circle $(x - h)^2 + (y - k)^2 = r^2$ becomes more complex

General principle: Circles remain circles under:
- Translation
- Rotation
- Reflection
- Uniform scaling

But not under non-uniform scaling (becomes ellipse)

Applications and Problem Solving

Real-World Applications

Circles in Engineering and Design

Mechanical Engineering:
- Gears: circular motion transmission
- Wheels: circular for smooth rolling
- Bearings: circular for reduced friction
- Pulleys: circular for rope/belt systems

Gear ratios: / = r /r
where is angular velocity, r is radius

Architecture:
- Arches: circular arcs for strength
- Domes: circular cross-sections
- Windows: circular for aesthetics
- Columns: circular cross-sections

Circular arch strength:
Load distributed along curve
Compression forces, no tension
Self-supporting structure

Navigation and GPS:
- Satellite orbits: approximately circular
- Radio range: circular coverage areas
- Position finding: intersection of circles

GPS triangulation:
Distance to satellite 1: circle 1
Distance to satellite 2: circle 2

```
Distance to satellite 3: circle 3
Position = intersection of three circles

Optics:
- Lenses: circular cross-sections
- Mirrors: circular or spherical
- Apertures: circular openings

Lens formula: 1/f = 1/u + 1/v
where f = focal length, u = object distance, v = image distance

Sports and Recreation:
- Athletic tracks: circular curves
- Playing fields: circular center circles
- Wheels: bicycles, cars, etc.

Track design:
Straight sections connected by semicircular curves
Banking angle for circular sections: tan( ) = v²/(rg)
where v = speed, r = radius, g = gravity
```

Problem-Solving Strategies

```
Circle Problem-Solving Techniques


Strategy 1: Identify Circle Elements
- Center and radius
- Chords, tangents, secants
- Inscribed or central angles
- Arcs and sectors

Strategy 2: Use Appropriate Theorems
- Inscribed angle = (1/2) central angle
- Angle in semicircle = 90°
- Tangent perpendicular to radius
- Power of a point

Strategy 3: Apply Coordinate Methods
- Use circle equation
- Find intersections algebraically
- Use distance formula
- Apply transformations

Strategy 4: Use Symmetry
- Circles have infinite rotational symmetry
- Any diameter is line of symmetry
```

- Use symmetry to simplify problems

Example Problem:
"A circular garden has radius 10 meters. A straight path of width 2 meters crosses the garden t

Solution:
Garden area = $(10)^2$ = 100 square meters
Path area = length × width = 20 × 2 = 40 square meters
Uncovered area = 100 - 40 314.16 - 40 = 274.16 square meters

Strategy 5: Break Complex Problems into Parts
- Divide circle into sectors or segments
- Use multiple circle theorems
- Combine geometric and algebraic methods

Strategy 6: Check Reasonableness
- Are angles between 0° and 360°?
- Is radius positive?
- Do areas make sense?
- Are units consistent?

Common Mistakes and Misconceptions

Typical Circle Errors

Common Circle Mistakes

Mistake 1: Confusing Radius and Diameter
Wrong: Circle with diameter 10 has area $(10)^2$ = 100
Correct: Circle with diameter 10 has radius 5, area $(5)^2$ = 25

Mistake 2: Inscribed Angle Confusion
Wrong: Inscribed angle equals central angle
Correct: Inscribed angle = (1/2) × central angle

Mistake 3: Arc Length vs Arc Measure
Wrong: Arc length = central angle in degrees
Correct: Arc length = (/360°) × 2 r or r (if in radians)

Mistake 4: Tangent Properties
Wrong: Tangent passes through center
Correct: Tangent is perpendicular to radius at point of tangency

Mistake 5: Circle Equation Errors
Wrong: $(x - 3)^2 + (y + 2)^2 = 25$ has center (3, 2)

Correct: Center is (3, -2) - watch the signs!

Mistake 6: Area vs Circumference Formulas
Wrong: Area = 2 r
Correct: Area = r^2, Circumference = 2 r

Mistake 7: Degree vs Radian Confusion
Wrong: Arc length = r with   in degrees
Correct: Arc length = r only when   is in radians
For degrees: Arc length = (/360°) × 2 r

Prevention Strategies:
- Draw clear diagrams with labels
- Double-check radius vs diameter
- Remember inscribed angle theorem
- Practice converting between degrees and radians
- Verify formulas before using
- Check units in final answers
- Use estimation to verify reasonableness

Building Circle Intuition

Visualization Exercises

Developing Circle Sense

Exercise 1: Circle Recognition
Identify circles in:
- Architecture (arches, domes, windows)
- Nature (tree rings, ripples, celestial objects)
- Technology (wheels, gears, lenses)
- Art (mandalas, rose windows, pottery)

Exercise 2: Angle Relationships
Given a circle with various chords and tangents:
- Identify central angles
- Find inscribed angles
- Locate tangent-chord angles
- Verify angle relationships

Exercise 3: Construction Practice
Using compass and straightedge:
- Construct circle through three points
- Find center of given circle
- Construct tangent from external point

```
- Inscribe regular polygons

Exercise 4: Coordinate Circles
Plot circles with different centers and radii
- Find intersections with lines
- Determine tangent lines
- Apply transformations
- Solve systems of circle equations

Exercise 5: Real-World Measurements
Measure circular objects:
- Calculate  using circumference and diameter
- Find areas of circular regions
- Determine arc lengths and sector areas
- Estimate angles and distances

Exercise 6: Circle Theorems
Verify theorems experimentally:
- Inscribed angle theorem
- Tangent-radius perpendicularity
- Power of a point
- Properties of cyclic quadrilaterals
```

Conclusion

Circles represent mathematical perfection and infinite possibility. Their elegant simplicity - all points equidistant from a center - gives rise to rich geometric relationships, practical applications, and connections to advanced mathematics.

```
Circles: Complete Understanding


Conceptual Understanding:
  Circle elements and their relationships
  Angle theorems and their applications
  Properties of tangents, chords, and arcs

Procedural Fluency:
  Circumference and area calculations
  Arc length and sector area formulas
  Circle constructions and coordinate methods

Strategic Competence:
  Applying appropriate circle theorems
  Solving problems involving multiple circles
  Using coordinate geometry with circles
```

```
Adaptive Reasoning:
  Understanding why circle theorems work
  Making connections between different concepts
  Recognizing circles in various contexts

Productive Disposition:
  Confidence with circle calculations
  Appreciation for circular symmetry and beauty
  Recognition of circles in the world around us
```

From ancient astronomers studying planetary orbits to modern engineers designing precision machinery, from artists creating mandala patterns to architects designing domed structures, circles provide essential tools for understanding and creating in our curved world.

The study of circles reveals fundamental principles that extend throughout mathematics - the relationship between linear and angular measure, the power of symmetry in problem-solving, and the elegant connections between geometry and algebra. Whether you're calculating the area of a pizza, designing a circular garden, analyzing the motion of a Ferris wheel, or simply appreciating the perfect symmetry of a soap bubble, circles provide the mathematical framework for understanding curved relationships and rotational phenomena.

As you continue exploring geometry, remember that circles bridge many areas of mathematics. They connect to trigonometry through the unit circle, to calculus through rates of change in circular motion, to physics through orbital mechanics, and to art through principles of proportion and harmony. Mastering circles opens doors to understanding the mathematical beauty and practical power of curved geometry.

Area and Perimeter: Measuring Space and Boundary

Introduction

Area and perimeter are fundamental measurements in geometry that help us quantify the size and boundary of two-dimensional shapes. Perimeter measures the distance around a shape's boundary, while area measures the space contained within that boundary.

These concepts are essential for practical applications ranging from calculating how much paint is needed for a wall to determining the amount of fencing required for a garden. Understanding area and perimeter provides the foundation for more advanced concepts in geometry, calculus, and real-world problem-solving.

```
Area vs Perimeter Concepts


Perimeter: Distance around the outside

                    ← Perimeter = sum of all sides




Area: Space inside the boundary

             ← Area = space covered



Units:
Perimeter: linear units (cm, m, ft, in)
Area: square units (cm², m², ft², in²)
```

Understanding Perimeter

Basic Perimeter Concepts

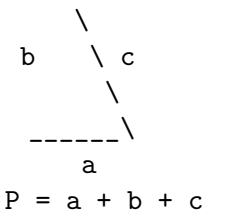
```
Perimeter Fundamentals
```

Definition: The total distance around the boundary of a shape

For any polygon: Perimeter = sum of all side lengths
P = a + b + c + d + ... (for all sides)

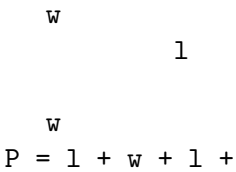
Examples:

Triangle:



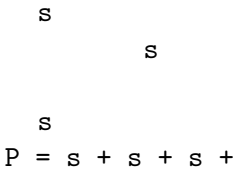
P = a + b + c

Rectangle:



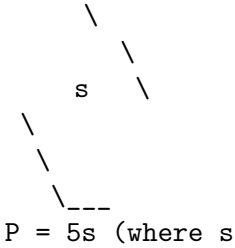
P = l + w + l + w = 2l + 2w = 2(l + w)

Square:



P = s + s + s + s = 4s

Regular Pentagon:



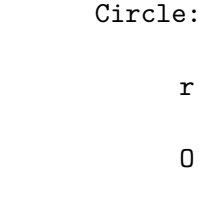
P = 5s (where s is side length)

Regular n-gon:
P = n × s (where n is number of sides, s is side length)

Perimeter of Curved Shapes

Curved Shape Perimeters

Circle:



Circumference = 2 r = d
where r = radius, d = diameter

Example: Circle with radius 5
C = 2 (5) = 10 31.42 units

Semicircle:

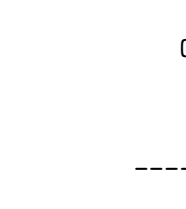
Perimeter = r + d = r + 2r = r(+ 2)

Quarter Circle:



Perimeter = (r/2) + 2r = r(/2 + 2)

Ellipse (approximate):



Perimeter ≈ √[2(a² + b²)] (Ramanujan's approximation)
where a and b are semi-major and semi-minor axes

Exact formula involves elliptic integrals (advanced calculus)

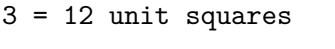
Understanding Area

Basic Area Concepts

Area Fundamentals

Definition: The amount of space inside a two-dimensional shape

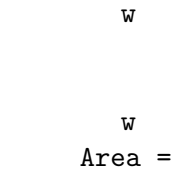
Unit Squares:
Area is measured by counting unit squares that fit inside the shape



Area = 12 square units

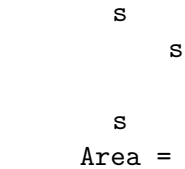
Basic Shapes:

Rectangle:



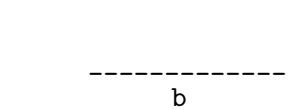
Area = length × width = lw

Square:



Area = side² = s²

Parallelogram:



```
Area = base × height = bh
(height is perpendicular distance between parallel sides)

Triangle:
     \
     \
     \ h
     \
 ________\
    b
Area = (1/2) × base × height = (1/2)bh

Trapezoid:
         a
         h


 -------------
       b
Area = (1/2) × (sum of parallel sides) × height
     = (1/2)(a + b)h
```

Area of Circles and Sectors

```
Circular Areas


Circle:


        r
   O




  -------

Area = r²

Example: Circle with radius 4
Area = (4)² = 16   50.27 square units

Sector (pie slice):



     O
```

```
   -------

Area = ( /360°) × r² (if   in degrees)
Area = (1/2)r²  (if   in radians)

Example: Sector with radius 6, central angle 60°
Area = (60°/360°) × (6)² = (1/6) × 36 = 6 square units

Segment (between chord and arc):




        ← Chord


   -------

Area = Sector area - Triangle area
     = ( /360°) r² - (1/2)r²sin( )

Annulus (ring):


             R

            r




   -----------

Area = (R² - r²) = (R - r)(R + r)
where R = outer radius, r = inner radius
```

Advanced Area Formulas

Triangles - Multiple Methods

```
Triangle Area Formulas


Method 1: Base and Height
```

```
Area = (1/2) × base × height

    A
     \
      \
   h   \
       \
B    C
    b

Area = (1/2) × b × h

Method 2: Two Sides and Included Angle
Area = (1/2) × a × b × sin(C)

    A
     \
      \ b
   C   \
       \
B    C
    a

Area = (1/2) × a × b × sin(C)

Example: a = 5, b = 7, C = 60°
Area = (1/2) × 5 × 7 × sin(60°) = (1/2) × 35 × (√3/2) = 35√3/4

Method 3: Heron's Formula
For triangle with sides a, b, c:
s = (a + b + c)/2 (semiperimeter)
Area = √[s(s-a)(s-b)(s-c)]

Example: Triangle with sides 3, 4, 5
s = (3 + 4 + 5)/2 = 6
Area = √[6(6-3)(6-4)(6-5)] = √[6 × 3 × 2 × 1] = √36 = 6

Method 4: Coordinate Formula
For triangle with vertices (x ,y ), (x ,y ), (x ,y ):
Area = (1/2)|x (y -y ) + x (y -y ) + x (y -y )|

Example: Vertices (0,0), (4,0), (2,3)
Area = (1/2)|0(0-3) + 4(3-0) + 2(0-0)| = (1/2)|0 + 12 + 0| = 6

Method 5: Using Circumradius
Area = (abc)/(4R)
where a, b, c are sides and R is circumradius
```

Method 6: Using Inradius
Area = r × s
where r is inradius and s is semiperimeter

Regular Polygons

Regular Polygon Areas

Regular polygon: All sides equal, all angles equal

General Formula:
Area = (1/2) × perimeter × apothem
     = (1/2) × ns × a

where n = number of sides, s = side length, a = apothem

Apothem: Distance from center to middle of any side
       = s/(2 tan(/n)) = s/(2 tan(180°/n))

Alternative Formula:
Area = (1/4) × n × s^2 × cot(/n)
     = (1/4) × n × s^2 × cot(180°/n)

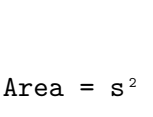
Specific Cases:

Equilateral Triangle (n = 3):



Area = (√3/4) × s^2

Square (n = 4):

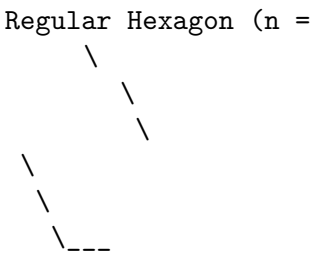


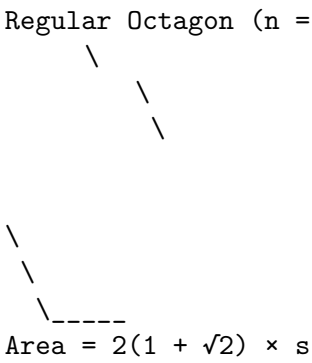
Area = s^2

Regular Pentagon (n = 5):



Area = (1/4)√(25 + 10√5) × s^2 1.720 × s^2

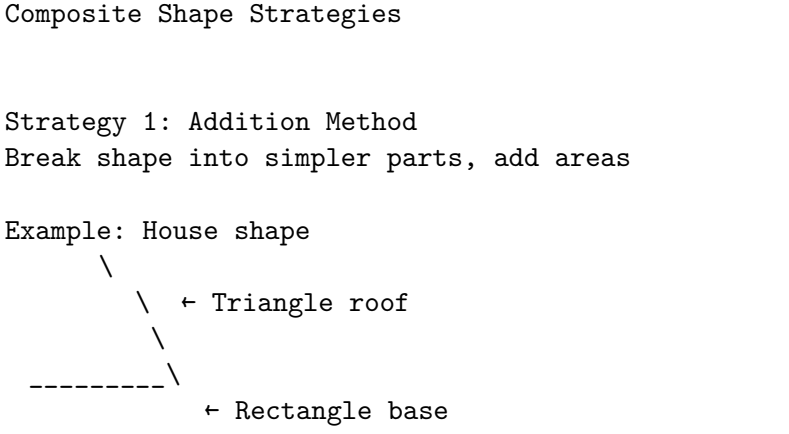




As n increases, regular polygon approaches circle:
lim(n→∞) Area = r² where r = circumradius

Composite Shapes

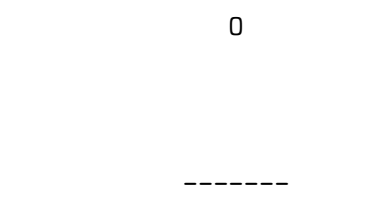
Breaking Down Complex Shapes



Total Area = Triangle area + Rectangle area
= (1/2) × base × height + length × width

Strategy 2: Subtraction Method
Start with larger shape, subtract removed parts

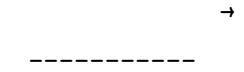
Example: Rectangle with circular hole



Area = Rectangle area - Circle area
= lw - r^2

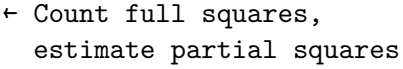
Strategy 3: Rearrangement Method
Move parts to form simpler shapes

Example: Parallelogram to rectangle



Same area, easier to calculate

Strategy 4: Grid Method
Overlay grid, count squares



Useful for irregular shapes

Common Composite Shapes

Typical Composite Shape Examples

L-Shape:

Method 1: Two rectangles
Area = Area + Area

Method 2: Large rectangle minus cut-out
Area = Large rectangle - Cut-out rectangle

T-Shape:

Area = Top rectangle + Bottom rectangle

U-Shape:

Area = Large rectangle - Middle rectangle

Semicircular Arch:

Area = Rectangle + Semicircle
     = lw + (1/2) r^2

Quarter Circle Corner:

```
Area = Rectangle - Quarter circle
     = lw - (1/4) r²
```

Units and Conversions

Area Unit Conversions

```
Area Unit Relationships


Metric System:
1 m² = 10,000 cm² = 1,000,000 mm²
1 km² = 1,000,000 m² = 100 hectares
1 hectare = 10,000 m²

Imperial System:
1 ft² = 144 in²
1 yd² = 9 ft² = 1,296 in²
1 acre = 43,560 ft² = 4,840 yd²
1 mi² = 640 acres

Conversion Examples:
Convert 5 m² to cm²:
5 m² × 10,000 cm²/m² = 50,000 cm²

Convert 2.5 acres to ft²:
2.5 acres × 43,560 ft²/acre = 108,900 ft²

Convert 1,500 cm² to m²:
1,500 cm² × (1 m²/10,000 cm²) = 0.15 m²

Common Area Benchmarks:
- Sheet of paper (8.5" × 11"): 93.5 in²  0.65 ft²
- Standard door: ~21 ft²
- Parking space: ~150 ft²
- Basketball court: ~4,700 ft²
- Football field: ~57,600 ft²  1.3 acres
- City block: ~2-5 acres
```

Perimeter Unit Conversions

```
Perimeter Unit Relationships


Linear units (same as length):

Metric:
1 m = 100 cm = 1,000 mm
1 km = 1,000 m

Imperial:
1 ft = 12 in
1 yd = 3 ft = 36 in
1 mi = 5,280 ft = 1,760 yd

Conversion Examples:
Convert 2.5 km to meters:
2.5 km × 1,000 m/km = 2,500 m

Convert 15 ft to inches:
15 ft × 12 in/ft = 180 in

Convert 500 cm to meters:
500 cm × (1 m/100 cm) = 5 m

Scale Relationships:
If linear dimensions scale by factor k:
- Perimeter scales by factor k
- Area scales by factor k²

Example: Double all dimensions
Original: 3 × 4 rectangle
Perimeter = 14, Area = 12

Doubled: 6 × 8 rectangle
Perimeter = 28 = 2 × 14
Area = 48 = 4 × 12
```

Problem-Solving Applications

Real-World Area Problems

```
Practical Area Applications
```

Home Improvement:
"A room is 12 ft × 15 ft. How much carpet is needed?"
Area = 12 × 15 = 180 ft^2

"How much paint for walls 8 ft high, room perimeter 54 ft,
with 80 ft^2 of doors and windows?"
Wall area = perimeter × height − openings
= 54 × 8 − 80 = 432 − 80 = 352 ft^2

Landscaping:
"Circular garden with radius 8 ft needs mulch.
One bag covers 12 ft^2. How many bags needed?"
Garden area = $(8)^2$ = 64 ≈ 201 ft^2
Bags needed = 201 ÷ 12 ≈ 17 bags

"Rectangular lawn 40 ft × 60 ft has circular flower bed
with radius 6 ft. How much grass seed needed?"
Lawn area = 40 × 60 = 2,400 ft^2
Flower bed area = $(6)^2$ = 36 ≈ 113 ft^2
Grass area = 2,400 − 113 = 2,287 ft^2

Agriculture:
"Rectangular field 200 m × 150 m. What's the area in hectares?"
Area = 200 × 150 = 30,000 m^2
In hectares: 30,000 ÷ 10,000 = 3 hectares

Construction:
"Concrete slab 20 ft × 30 ft, 4 inches thick.
How many cubic yards of concrete?"
Area = 20 × 30 = 600 ft^2
Thickness = 4 in = 1/3 ft
Volume = 600 × (1/3) = 200 ft^3
In cubic yards: 200 ÷ 27 ≈ 7.4 yd^3

Real-World Perimeter Problems

Practical Perimeter Applications

Fencing:
"Rectangular garden 25 ft × 40 ft needs fence.
Gate is 4 ft wide. How much fencing needed?"
Perimeter = 2(25 + 40) = 130 ft
Fencing = 130 − 4 = 126 ft

"Circular dog run with radius 15 ft needs fence."
Circumference = 2 (15) = 30 ≈ 94.2 ft

Trim and Molding:
"Room 12 ft × 16 ft needs baseboard.
Doorways total 8 ft. How much baseboard?"
Perimeter = 2(12 + 16) = 56 ft
Baseboard = 56 - 8 = 48 ft

Track and Field:
"Standard track has two semicircles (radius 36.5 m)
connected by 100 m straights. What's the total distance?"
Semicircle perimeter = × 36.5 = 36.5 m
Total = 2 × 100 + 36.5 200 + 114.6 = 314.6 m

Border Design:
"Rectangular poster 18 in × 24 in needs decorative border
2 inches wide. What's the border area?"
Outer dimensions: 22 in × 28 in
Outer area = 22 × 28 = 616 in^2
Inner area = 18 × 24 = 432 in^2
Border area = 616 - 432 = 184 in^2

Swimming Pool:
"Rectangular pool 20 ft × 40 ft has 5 ft walkway around it.
What's the walkway area?"
Pool area = 20 × 40 = 800 ft^2
Total area = 30 × 50 = 1,500 ft^2
Walkway area = 1,500 - 800 = 700 ft^2

Optimization Problems

Maximum Area for Given Perimeter

Optimization: Fixed Perimeter, Maximum Area

Problem: "What rectangle with perimeter 100 ft has maximum area?"

Let length = l, width = w
Constraint: 2l + 2w = 100, so l + w = 50, so w = 50 - l
Area = lw = l(50 - l) = 50l - l^2

To maximize: Take derivative and set to zero
dA/dl = 50 - 2l = 0
l = 25, so w = 25

Maximum area rectangle is a square: 25 × 25 = 625 ft^2

General Result: For fixed perimeter, square has maximum area

Problem: "What shape has maximum area for given perimeter?"

Answer: Circle

For perimeter P:
Circle: radius = P/(2), area = [P/(2)]^2 = P^2/(4)
Square: side = P/4, area = (P/4)^2 = P^2/16

Ratio: Circle area/Square area = [P^2/(4)]/[P^2/16] = 16/(4) = 4/ 1.27

Circle has ~27% more area than square with same perimeter

Isoperimetric Inequality:
For any closed curve with perimeter P and area A:
A P^2/(4)

Equality holds only for circles.

Minimum Perimeter for Given Area

Optimization: Fixed Area, Minimum Perimeter

Problem: "What rectangle with area 400 ft^2 has minimum perimeter?"

Let length = l, width = w
Constraint: lw = 400, so w = 400/l
Perimeter = 2l + 2w = 2l + 2(400/l) = 2l + 800/l

To minimize: Take derivative and set to zero
dP/dl = 2 - 800/l^2 = 0
2 = 800/l^2
l^2 = 400
l = 20, so w = 20

Minimum perimeter rectangle is a square: 20 × 20
Perimeter = 4 × 20 = 80 ft

General Result: For fixed area, square has minimum perimeter

Problem: "What shape has minimum perimeter for given area?"

Answer: Circle

For area A:
Circle: radius = √(A/), perimeter = 2 √(A/) = 2√(A)
Square: side = √A, perimeter = 4√A

Ratio: Circle perimeter/Square perimeter = 2√(A)/(4√A) = √ /2 0.886

Circle has ~11% less perimeter than square with same area

Applications:
- Soap bubbles form spheres (minimum surface area for volume)
- Cells tend toward circular cross-sections
- Efficient packing problems
- Optimal design in engineering

Common Mistakes and Problem-Solving Tips

Typical Errors

Common Area and Perimeter Mistakes

Mistake 1: Confusing Area and Perimeter Formulas
Wrong: Rectangle area = 2l + 2w
Correct: Rectangle area = lw, perimeter = 2l + 2w

Mistake 2: Unit Confusion
Wrong: Room 12 ft × 15 ft has area 180 ft
Correct: Area = 180 ft^2 (square feet, not linear feet)

Mistake 3: Triangle Area Errors
Wrong: Triangle area = base × height
Correct: Triangle area = (1/2) × base × height

Mistake 4: Circle Formula Mix-up
Wrong: Circle area = 2 r
Correct: Circle area = r^2, circumference = 2 r

Mistake 5: Composite Shape Errors
Wrong: Adding areas when should subtract
Example: Rectangle with hole - forgot to subtract hole area

Mistake 6: Scale Factor Confusion
Wrong: "Double dimensions doubles area"
Correct: "Double dimensions quadruples area"

Mistake 7: Irregular Shape Approximation

```
Wrong: Treating curved boundary as straight
Better: Use appropriate formulas or approximation methods

Prevention Strategies:
- Draw clear, labeled diagrams
- Write down known formulas before starting
- Check units in final answer
- Verify answers make sense (area positive, reasonable size)
- Use estimation to check calculations
- Practice identifying shape types
- Double-check composite shape breakdowns
```

Problem-Solving Strategies

```
Effective Problem-Solving Approach


Step 1: Understand the Problem
- What shape(s) are involved?
- What measurements are given?
- What are you asked to find?
- What units should the answer have?

Step 2: Draw and Label
- Sketch the shape(s)
- Label all given measurements
- Mark what you need to find

Step 3: Identify the Method
- Simple shape: use direct formula
- Composite shape: break down or subtract
- Optimization: set up equation and optimize
- Unit conversion: identify conversion factors

Step 4: Apply Formulas
- Write the appropriate formula
- Substitute known values
- Solve for unknown

Step 5: Check Your Work
- Are units correct?
- Is the answer reasonable?
- Does it make sense in context?
- Can you verify with different method?

Example Problem:
"A semicircular window has diameter 4 feet.
```

```
What's the perimeter of the window frame?"

Step 1: Semicircle, diameter = 4 ft, find perimeter
Step 2: [Draw semicircle with diameter labeled]
Step 3: Perimeter = curved part + straight part
Step 4: Curved =  r =  (2) = 2  ft
        Straight = diameter = 4 ft
        Total = 2  + 4   6.28 + 4 = 10.28 ft
Step 5: Units correct (ft), reasonable size
```

Conclusion

Area and perimeter are fundamental measurements that connect abstract geometric concepts to practical, real-world applications. Understanding these concepts deeply provides the foundation for advanced mathematics and enables us to solve countless practical problems.

```
Area and Perimeter: Complete Understanding


Conceptual Understanding:
  Distinction between boundary (perimeter) and interior (area)
  Relationship between linear and square units
  How shapes affect area-to-perimeter ratios

Procedural Fluency:
  Formulas for basic and composite shapes
  Unit conversions and scaling relationships
  Optimization techniques

Strategic Competence:
  Breaking complex shapes into simpler parts
  Choosing appropriate formulas and methods
  Solving real-world application problems

Adaptive Reasoning:
  Understanding why formulas work
  Recognizing when to use different approaches
  Making connections between geometry and algebra

Productive Disposition:
  Confidence with measurement calculations
  Appreciation for geometric relationships
  Recognition of optimization principles in nature and design
```

From calculating the amount of material needed for construction projects to understanding why soap bubbles are spherical, from designing efficient layouts to analyzing natural patterns, area and

perimeter provide essential tools for quantifying and optimizing the two-dimensional world around us.

The study of area and perimeter reveals deep mathematical principles - the isoperimetric inequality, the relationship between linear and quadratic scaling, and the optimization principles that govern efficient design in both human engineering and natural systems. Whether you're planning a garden, designing a building, analyzing biological structures, or simply trying to understand the mathematical relationships that govern shape and space, mastering area and perimeter provides the quantitative foundation for geometric reasoning and practical problem-solving.

As you continue exploring mathematics, remember that these measurement concepts connect to many advanced topics - calculus uses area and perimeter in integration and optimization, physics applies them in analyzing motion and forces, and engineering relies on them for design and analysis. The principles you learn here will serve as building blocks for understanding the mathematical description of our physical world.

Pre-Algebra

Introduction to Pre-Algebra: The Bridge to Abstract Mathematics

What is Pre-Algebra?

Pre-algebra is the mathematical bridge between arithmetic and algebra, introducing students to abstract thinking, symbolic representation, and algebraic reasoning. It takes the concrete numerical skills developed in arithmetic and begins to generalize them using variables, expressions, and equations.

Pre-algebra represents a crucial transition in mathematical thinking - from working with specific numbers to working with general patterns and relationships. It's where mathematics begins to reveal its true power as a language for describing and analyzing the world around us.

```
The Mathematical Journey


Arithmetic → Pre-Algebra → Algebra → Advanced Mathematics
    ↓            ↓           ↓              ↓
Specific     General     Abstract      Complex
Numbers     Patterns    Structures    Systems

3 + 5 = 8  →  n + 5  →  ax + b = c  →  f(x) = ax² + bx + c
```

The Evolution from Numbers to Variables

From Concrete to Abstract

Pre-algebra marks the beginning of abstract mathematical thinking, where we move from specific calculations to general patterns and relationships.

```
Progression of Mathematical Abstraction


Level 1: Concrete Arithmetic
"5 apples + 3 apples = 8 apples"
Working with specific quantities

Level 2: Numerical Patterns
```

```
"5 + 3 = 8, 15 + 3 = 18, 25 + 3 = 28, ..."
Recognizing patterns in numbers

Level 3: Variable Introduction
"n + 3 represents any number plus 3"
Using symbols to represent unknowns

Level 4: Algebraic Relationships
"If x + 3 = 10, then x = 7"
Solving for unknown values

Level 5: General Principles
"For any numbers a and b: a + b = b + a"
Understanding universal mathematical laws

This progression shows how pre-algebra builds the foundation
for all higher mathematics.
```

The Power of Variables

Variables are letters or symbols that represent unknown or changing quantities. They are the fundamental tool that allows mathematics to become truly powerful and general.

```
Understanding Variables


What is a Variable?
A variable is a symbol (usually a letter) that represents:
- An unknown number: "Find x if x + 5 = 12"
- A changing quantity: "Let t = time in hours"
- Any number in a set: "For any number n, n + 0 = n"

Common Variable Names:
x, y, z - most common for unknowns
a, b, c - often used for constants or coefficients
n, m - frequently used for counting numbers
t - commonly used for time
d - often used for distance
r - frequently used for rate

Variable vs. Constant:
Variable: Can change or is unknown (x, y, t)
Constant: Has a fixed value (5,  , -3)

Examples:
Expression: 3x + 7
- x is the variable (can change)
```

```
- 3 and 7 are constants (fixed values)

Real-World Variables:
- Speed limit: s   65 mph
- Temperature: T = 32°F + (9/5)C
- Cost: C = 5n (where n = number of items)
- Area: A =  r² (where r = radius)
```

Key Concepts in Pre-Algebra

Expressions vs. Equations

Understanding the difference between expressions and equations is fundamental to algebraic thinking.

```
Expressions vs. Equations


Expression: A mathematical phrase that can contain:
- Numbers: 5, -3, 1/2,
- Variables: x, y, n
- Operations: +, -, ×, ÷, ^

Examples of Expressions:
3x + 7
2y - 5
x² + 4x - 1
(a + b)/2

Key Point: Expressions can be simplified but not "solved"
They represent a value but don't make a statement

Equation: A mathematical statement that two expressions are equal
Contains an equals sign (=)

Examples of Equations:
3x + 7 = 19
2y - 5 = 11
x² + 4x - 1 = 0
(a + b)/2 = 10

Key Point: Equations can be solved to find variable values
They make a statement that can be true or false

Visual Comparison:
Expression: 3x + 7     (What is this worth?)
```

```
Equation:   3x + 7 = 19 (This equals that!)

Think of it this way:
Expression = Recipe ingredient list
Equation = Recipe instruction ("Mix A with B to get C")
```

The Order of Operations in Algebra

The order of operations (PEMDAS/BODMAS) becomes even more important in pre-algebra as expressions become more complex.

```
Order of Operations with Variables


PEMDAS/BODMAS Rules:
P/B - Parentheses/Brackets first
E/O - Exponents/Orders (powers, roots)
MD - Multiplication and Division (left to right)
AS - Addition and Subtraction (left to right)

Examples with Variables:

Expression: 2x + 3(x - 4)
Step 1: Parentheses first: 2x + 3x - 12
Step 2: Combine like terms: 5x - 12

Expression: x² + 2x(3 - x)
Step 1: Parentheses: x² + 2x(3) - 2x(x)
Step 2: Multiplication: x² + 6x - 2x²
Step 3: Combine like terms: -x² + 6x

Expression: (2x + 1)²
Step 1: Exponent applies to entire parentheses
Step 2: (2x + 1)(2x + 1)
Step 3: Expand: 4x² + 4x + 1

Common Mistakes:
Wrong: 2x² = (2x)² = 4x²
Right: 2x² = 2 × x²

Wrong: (x + 3)² = x² + 9
Right: (x + 3)² = x² + 6x + 9

Memory Device: "Please Excuse My Dear Aunt Sally"
Or: "Brackets, Orders, Division/Multiplication, Addition/Subtraction"
```

Fundamental Operations with Variables

Combining Like Terms

Like terms are terms that have the same variable raised to the same power. They can be combined by adding or subtracting their coefficients.

```
Like Terms and Combining


Like Terms: Same variable, same power
3x and 7x are like terms (both have x¹)
2y² and -5y² are like terms (both have y²)
4 and -9 are like terms (both are constants)

Unlike Terms: Different variables or different powers
3x and 7y are unlike (different variables)
2x and 5x² are unlike (different powers)
3x and 7 are unlike (variable vs. constant)

Combining Like Terms:
Add/subtract the coefficients, keep the variable part

Examples:
3x + 7x = (3 + 7)x = 10x
5y² - 2y² = (5 - 2)y² = 3y²
4a + 3b - 2a + b = (4a - 2a) + (3b + b) = 2a + 4b

Complex Example:
3x² + 5x - 2x² + 7x - 4
= (3x² - 2x²) + (5x + 7x) - 4
= x² + 12x - 4

Visual Representation:
3x + 7x = xxx + xxxxxxx = xxxxxxxxxx = 10x

Think of it like counting:
3 apples + 7 apples = 10 apples
3x + 7x = 10x

But you can't combine:
3 apples + 7 oranges = 3 apples + 7 oranges
3x + 7y = 3x + 7y (cannot simplify further)
```

The Distributive Property

The distributive property is one of the most important tools in algebra, allowing us to multiply expressions and factor them.

```
Distributive Property


Basic Form: a(b + c) = ab + ac
"Distribute" the multiplication over addition

Examples:
3(x + 4) = 3x + 12
-2(y - 5) = -2y + 10
x(x + 3) = x² + 3x

Reverse Distribution (Factoring):
6x + 9 = 3(2x + 3)
x² + 5x = x(x + 5)

Multiple Terms:
2(3x + 4y - 1) = 6x + 8y - 2
-3(2a - b + 4) = -6a + 3b - 12

With Variables as Distributors:
x(y + z) = xy + xz
(a + b)(c + d) = ac + ad + bc + bd

Visual Representation:
3(x + 4) = 3 × (x + 4)

Think of it as:

  x    4    4    4

   3x     +     12     = 3x + 12

Area Model:
    x    4

3   3x   12

Total area = 3x + 12

Common Mistakes:
Wrong: 3(x + 4) = 3x + 4
Right: 3(x + 4) = 3x + 12
```

```
Wrong: 2(3x - 1) = 6x - 1
Right: 2(3x - 1) = 6x - 2

Remember: Distribute to ALL terms inside parentheses!
```

Introduction to Equations

What Makes an Equation True?

An equation is a statement that two expressions are equal. Understanding when equations are true or false is fundamental to solving them.

```
Equation Truth Values


An equation can be:
1. Always true (identity)
2. Sometimes true (conditional)
3. Never true (contradiction)

Always True (Identity):
x + 3 = x + 3   (true for any value of x)
2(x + 1) = 2x + 2   (true for any value of x)

Sometimes True (Conditional):
x + 5 = 12   when x = 7,   when x   7
2x = 10   when x = 5,   when x   5

Never True (Contradiction):
x + 1 = x + 2   (impossible for any value of x)
0 = 5   (always false)

Testing Equation Truth:
For equation: 3x - 1 = 8

Test x = 3:
3(3) - 1 = 9 - 1 = 8   True!

Test x = 2:
3(2) - 1 = 6 - 1 = 5   8   False!

Solution: The value(s) that make the equation true
For 3x - 1 = 8, the solution is x = 3

Checking Solutions:
Always substitute back into original equation
```

```
If both sides equal the same value, solution is correct
```

Basic Equation Solving

Solving equations involves finding the value(s) of the variable that make the equation true. This requires understanding balance and inverse operations.

```
Equation Solving Principles


Golden Rule: Whatever you do to one side, do to the other
Think of equation as a balanced scale

    3x + 1 = 10
           =
      3x + 1          10


Goal: Isolate the variable (get x by itself)

Inverse Operations:
Addition  Subtraction
Multiplication  Division
Squaring  Square root

Basic Solving Steps:
1. Simplify both sides if needed
2. Use inverse operations to isolate variable
3. Check your answer

Example 1: x + 7 = 15
Subtract 7 from both sides:
x + 7 - 7 = 15 - 7
x = 8

Check: 8 + 7 = 15

Example 2: 3x = 21
Divide both sides by 3:
3x ÷ 3 = 21 ÷ 3
x = 7

Check: 3(7) = 21

Example 3: 2x + 5 = 17
Subtract 5 from both sides:
2x + 5 - 5 = 17 - 5
```

```
2x = 12

Divide both sides by 2:
2x ÷ 2 = 12 ÷ 2
x = 6

Check: 2(6) + 5 = 12 + 5 = 17

Visual Balance Model:
Original: [2x + 5] = [17]
Step 1:   [2x] = [12] (removed 5 from both sides)
Step 2:   [x] = [6] (divided both sides by 2)
```

Patterns and Sequences

Recognizing Mathematical Patterns

Pattern recognition is a crucial skill in pre-algebra that helps students understand relationships and make predictions.

```
Types of Mathematical Patterns


Arithmetic Sequences:
Add the same number each time (common difference)

Example: 3, 7, 11, 15, 19, ...
Pattern: +4 each time
Next terms: 23, 27, 31, ...
General term: 4n - 1 (where n = position)

Position: 1  2  3  4  5
Term:     3  7  11 15 19
          +4 +4 +4 +4

Geometric Sequences:
Multiply by the same number each time (common ratio)

Example: 2, 6, 18, 54, 162, ...
Pattern: ×3 each time
Next terms: 486, 1458, ...
General term: 2 × 3^(n-1)

Position: 1  2  3  4   5
Term:     2  6  18 54  162
          ×3 ×3 ×3  ×3
```

```
Square Number Patterns:
1, 4, 9, 16, 25, 36, ...
Pattern: n² (perfect squares)

Visual:



1²    2²    3²     4²

Triangular Number Patterns:
1, 3, 6, 10, 15, 21, ...
Pattern: n(n+1)/2

Visual:



1     3     6      10

Fibonacci Pattern:
1, 1, 2, 3, 5, 8, 13, 21, ...
Pattern: Each term = sum of previous two terms
```

Using Variables to Describe Patterns

Variables allow us to write general formulas for patterns, making them more powerful and useful.

```
Pattern Formulas with Variables


Arithmetic Sequence Formula:
If first term = a, common difference = d
nth term = a + (n-1)d

Example: 5, 9, 13, 17, ...
a = 5, d = 4
nth term = 5 + (n-1)4 = 5 + 4n - 4 = 4n + 1

Check: n = 1: 4(1) + 1 = 5
       n = 2: 4(2) + 1 = 9
       n = 3: 4(3) + 1 = 13

Geometric Sequence Formula:
```

```
If first term = a, common ratio = r
nth term = a × r^(n-1)

Example: 3, 12, 48, 192, ...
a = 3, r = 4
nth term = 3 × 4^(n-1)

Check: n = 1: 3 × 4^0 = 3 × 1 = 3
       n = 2: 3 × 4^1 = 3 × 4 = 12
       n = 3: 3 × 4^2 = 3 × 16 = 48

Pattern Tables:
Input (n) | Output | Pattern
    1     |   4    |
    2     |   7    | +3 each time
    3     |   10   |
    4     |   13   | Formula: 3n + 1

Input (n) | Output | Pattern
    1     |   2    |
    2     |   8    | ×4 each time
    3     |   32   |
    4     |   128  | Formula: 2 × 4^(n-1)

Real-World Pattern Example:
"A cell phone plan costs $30 plus $0.10 per text"
Cost = 30 + 0.10t (where t = number of texts)

For 100 texts: Cost = 30 + 0.10(100) = $40
For 250 texts: Cost = 30 + 0.10(250) = $55
```

Real-World Applications

Modeling with Variables

Pre-algebra allows us to create mathematical models of real-world situations, making abstract mathematics practical and relevant.

```
Real-World Variable Applications


Distance, Rate, Time:
Formula: d = rt (distance = rate × time)

Example: "A car travels at 60 mph for t hours"
Distance = 60t miles
```

```
If t = 2 hours: d = 60(2) = 120 miles
If t = 3.5 hours: d = 60(3.5) = 210 miles

Cost Calculations:
"Movie tickets cost $12 each plus $3 parking"
Total cost = 12n + 3 (where n = number of tickets)

For 4 tickets: Cost = 12(4) + 3 = $51
For 7 tickets: Cost = 12(7) + 3 = $87

Temperature Conversion:
Celsius to Fahrenheit: F = (9/5)C + 32
Fahrenheit to Celsius: C = (5/9)(F - 32)

Example: Convert 25°C to Fahrenheit
F = (9/5)(25) + 32 = 45 + 32 = 77°F

Geometry Applications:
Rectangle perimeter: P = 2l + 2w
Rectangle area: A = lw
Circle area: A =  r²
Circle circumference: C = 2 r

Business Applications:
Profit = Revenue - Costs
P = R - C

If R = 50x (revenue from x items)
And C = 200 + 30x (fixed costs + variable costs)
Then P = 50x - (200 + 30x) = 20x - 200

Break-even point: When P = 0
0 = 20x - 200
20x = 200
x = 10 items

Simple Interest:
I = Prt (Interest = Principal × rate × time)

Example: $1000 at 5% for 3 years
I = 1000 × 0.05 × 3 = $150
```

Problem-Solving with Pre-Algebra

Pre-algebra provides systematic methods for solving word problems by translating English into mathematical expressions and equations.

Word Problem Translation

Key Phrases and Their Mathematical Meanings:

Addition (+):
- "sum of", "total", "plus", "increased by"
- "more than", "added to", "combined"

Subtraction (-):
- "difference", "minus", "decreased by"
- "less than", "reduced by", "take away"

Multiplication (×):
- "product", "times", "of" (as in "half of")
- "twice", "double", "triple"

Division (÷):
- "quotient", "divided by", "per"
- "ratio", "rate", "average"

Equals (=):
- "is", "equals", "is the same as"
- "results in", "gives", "yields"

Translation Examples:

"Five more than a number" → x + 5
"Three less than twice a number" → 2x - 3
"The product of a number and 7" → 7x
"A number divided by 4" → x/4
"Half of a number plus 10" → (1/2)x + 10

Word Problem Strategy:
1. Read the problem carefully
2. Identify what you're looking for (define variable)
3. Translate words to mathematical expressions
4. Set up equation
5. Solve equation
6. Check answer in original problem
7. Answer the question asked

Example Problem:
"The sum of three consecutive integers is 48. Find the integers."

Step 1: Let x = first integer
Step 2: Then x+1 = second integer, x+2 = third integer
Step 3: Sum equation: x + (x+1) + (x+2) = 48

```
Step 4: Simplify: 3x + 3 = 48
Step 5: Solve: 3x = 45, so x = 15
Step 6: The integers are 15, 16, 17
Step 7: Check: 15 + 16 + 17 = 48

Answer: The three consecutive integers are 15, 16, and 17.
```

Building Algebraic Thinking

From Arithmetic to Algebraic Reasoning

The transition from arithmetic to algebraic thinking involves learning to see patterns, generalize relationships, and work with abstract symbols.

```
Developing Algebraic Thinking


Arithmetic Thinking:
"What is 3 + 5?"
Answer: 8
Focus: Getting the answer

Algebraic Thinking:
"What patterns do you see in addition?"
3 + 5 = 5 + 3 (commutative property)
(3 + 5) + 2 = 3 + (5 + 2) (associative property)
3 + 0 = 3 (identity property)
Focus: Understanding relationships

Progression Examples:

Level 1: Specific calculations
2 + 3 = 5
4 + 6 = 10
7 + 8 = 15

Level 2: Pattern recognition
"When I add two numbers, I get a sum"
"The order doesn't matter: a + b = b + a"

Level 3: General relationships
"For any numbers a and b: a + b = b + a"
"This is called the commutative property"

Level 4: Abstract manipulation
If a + b = c, then a = c - b
```

```
If 2x + 3 = 11, then 2x = 8, so x = 4

Algebraic Habits of Mind:
1. Look for patterns and relationships
2. Generalize from specific examples
3. Use symbols to represent unknowns
4. Think about operations as processes
5. Justify reasoning with properties
6. Check answers for reasonableness

Example of Algebraic Thinking:
Instead of: "5 × 7 = 35"
Think: "5 × 7 = 5 × (10 - 3) = 5 × 10 - 5 × 3 = 50 - 15 = 35"
This shows understanding of distributive property
```

Preparing for Formal Algebra

Pre-algebra serves as the foundation for formal algebra by introducing key concepts and thinking patterns that will be essential for success in higher mathematics.

```
Pre-Algebra to Algebra Bridge


Pre-Algebra Skills → Algebra Applications

1. Combining like terms → Simplifying complex expressions
   3x + 5x = 8x → 3x² + 5x - 2x² + 7x = x² + 12x

2. Distributive property → Factoring and expanding
   3(x + 4) = 3x + 12 → x² + 5x + 6 = (x + 2)(x + 3)

3. Basic equations → Systems of equations
   2x + 3 = 11 → {2x + y = 5
                 {x - y = 1

4. Pattern recognition → Function notation
   y = 2x + 1 → f(x) = 2x + 1

5. Word problems → Mathematical modeling
   "Cost = $5 per item" → C(x) = 5x + fixed costs

Key Concepts for Algebra Success:

Variables and Expressions:
- Comfort with using letters for numbers
- Understanding that variables can represent any number
- Ability to evaluate expressions for given values
```

```
Equation Solving:
- Understanding that equations state relationships
- Systematic approach to isolating variables
- Checking solutions for accuracy

Properties of Operations:
- Commutative: a + b = b + a, ab = ba
- Associative: (a + b) + c = a + (b + c)
- Distributive: a(b + c) = ab + ac
- Identity: a + 0 = a, a × 1 = a

Graphical Thinking:
- Understanding coordinate planes
- Plotting points and recognizing patterns
- Connecting tables, graphs, and equations

Problem-Solving Strategies:
- Translating words to symbols
- Breaking complex problems into steps
- Using multiple representations (tables, graphs, equations)
- Checking answers for reasonableness

Study Tips for Algebra Preparation:
1. Practice with variables daily
2. Master basic equation solving
3. Memorize key properties and formulas
4. Work on word problem translation
5. Connect mathematics to real-world situations
6. Develop number sense and estimation skills
```

Common Challenges and Solutions

Typical Pre-Algebra Difficulties

Understanding common challenges helps students overcome obstacles and build confidence in algebraic thinking.

```
Common Pre-Algebra Challenges


Challenge 1: Fear of Variables
Problem: "I don't understand what x means"
Solution: Start with concrete examples
- "Let x = your age. If x = 15, then x + 2 = 17"
- Use familiar contexts: "Let h = hours worked"
```

```
- Practice substituting numbers for variables

Challenge 2: Combining Unlike Terms
Mistake: 3x + 5y = 8xy
Correct: 3x + 5y cannot be simplified further
Solution: Use visual models
- 3x = xxx, 5y = yyyyy
- You can't combine different variables
- Like terms must have same variable and power

Challenge 3: Distributive Property Errors
Mistake: 3(x + 4) = 3x + 4
Correct: 3(x + 4) = 3x + 12
Solution: Use area models or "distribute to all"
- Think: 3 groups of (x + 4)
- Each group gets 3x and each group gets 3(4) = 12

Challenge 4: Equation Solving Confusion
Mistake: x + 5 = 12, so x = 12 + 5 = 17
Correct: x + 5 = 12, so x = 12 - 5 = 7
Solution: Use balance model
- Whatever you do to one side, do to the other
- Use inverse operations: +5 requires -5

Challenge 5: Word Problem Translation
Problem: "I don't know how to start word problems"
Solution: Use systematic approach
1. Define variables clearly
2. Identify key phrases and translate
3. Look for relationships between quantities
4. Set up equation step by step

Challenge 6: Negative Number Operations
Mistake: -3x + 5x = -8x
Correct: -3x + 5x = 2x
Solution: Use number line or chip models
- Think: -3 + 5 = 2, so -3x + 5x = 2x

Overcoming Challenges:
1. Practice regularly with small steps
2. Use multiple representations (visual, numerical, symbolic)
3. Connect to real-world contexts
4. Check answers for reasonableness
5. Ask "Does this make sense?"
6. Build on arithmetic foundations
```

Conclusion

Pre-algebra represents a crucial transition in mathematical education, bridging the concrete world of arithmetic with the abstract realm of algebra. It introduces students to the power of mathematical generalization, symbolic representation, and algebraic reasoning.

```
Pre-Algebra: Complete Foundation


Conceptual Understanding:
  Variables as representations of unknown or changing quantities
  Expressions vs. equations and their different purposes
  Patterns and relationships in mathematical contexts

Procedural Fluency:
  Combining like terms and using distributive property
  Solving basic linear equations systematically
  Translating word problems into mathematical expressions

Strategic Competence:
  Recognizing patterns and writing general formulas
  Choosing appropriate problem-solving strategies
  Using multiple representations to understand concepts

Adaptive Reasoning:
  Understanding why algebraic procedures work
  Making connections between arithmetic and algebra
  Justifying mathematical reasoning with properties

Productive Disposition:
  Confidence with abstract mathematical thinking
  Appreciation for the power of algebraic generalization
  Persistence in solving complex problems
```

From ancient Babylonian algebraists solving quadratic equations to modern scientists modeling climate change, the algebraic thinking introduced in pre-algebra provides essential tools for understanding and describing patterns, relationships, and change in our world.

Pre-algebra reveals mathematics as more than just computation - it's a language for expressing ideas, a tool for solving problems, and a way of thinking that applies to countless situations. Whether you're calculating the best cell phone plan, determining how long it takes to save for a purchase, analyzing population growth, or simply trying to understand the mathematical relationships that govern everyday life, pre-algebra provides the foundation for mathematical literacy and algebraic reasoning.

As you continue your mathematical journey, remember that pre-algebra is not just preparation for algebra - it's an introduction to mathematical thinking that will serve you throughout your education and career. The skills you develop here - pattern recognition, symbolic manipulation,

problem-solving strategies, and abstract reasoning - are fundamental tools for success in all areas of mathematics and science.

The transition from arithmetic to algebra represents one of the most important intellectual developments in human history, and mastering pre-algebra means participating in this grand tradition of mathematical thinking that continues to shape our understanding of the world around us.

Variables and Expressions: The Language of Algebra

Introduction

Variables and expressions form the fundamental vocabulary of algebra. A variable is a symbol (usually a letter) that represents an unknown or changing quantity, while an expression is a mathematical phrase that combines numbers, variables, and operations.

Understanding variables and expressions is like learning a new language - the language of mathematics. Once mastered, this language provides powerful tools for describing patterns, relationships, and solving real-world problems.

```
From Numbers to Variables


Arithmetic: 5 + 3 = 8 (specific numbers)
Algebra: x + 3 (general expression with variable)

The variable x can represent any number:
If x = 5, then x + 3 = 8
If x = 10, then x + 3 = 13
If x = -2, then x + 3 = 1
```

Understanding Variables

What is a Variable?

A variable is a symbol that represents: 1. An unknown number we want to find 2. A quantity that can change or vary 3. Any number from a given set

```
Common Variable Symbols


x, y, z - most common for unknowns
a, b, c - often used for known constants
n, m - frequently used for counting numbers
t - commonly used for time
d - often used for distance
```

```
r - frequently used for rate

Examples in Context:
"Find x if x + 7 = 15" (unknown number)
"Let h = height after t days" (changing quantity)
"For any number n, n + 0 = n" (general number)
```

Variables vs. Constants

```
Variables vs. Constants


Variable: A symbol whose value can change
Examples: x, y, t, n

Constant: A symbol or number with a fixed value
Examples: 5, -3,  , 1/2

In the expression 3x + 7:
- x is a variable (can be any number)
- 3 and 7 are constants (fixed values)

Coefficient: A constant that multiplies a variable
In 5x: 5 is the coefficient of x
In -3y²: -3 is the coefficient of y²
In x: the coefficient is 1 (usually not written)
```

Introduction to Expressions

What is an Expression?

An expression is a mathematical phrase that can contain: - Numbers (constants) - Variables - Operation symbols (+, -, ×, ÷, ^) - Grouping symbols (parentheses, brackets)

```
Examples of Expressions


5x + 3
2y - 7
x² + 4x - 1
3(a + b)
(x + y)/2

Key Point: Expressions represent values but don't make statements
They can be evaluated or simplified, but not "solved"
```

```
Expression vs. Equation:
Expression: 3x + 5 (represents a value)
Equation: 3x + 5 = 17 (makes a statement)
```

Parts of an Expression

```
Expression Components


Term: A single number, variable, or product of numbers and variables
Examples: 5, x, 3y, -2x², 7xy

In the expression 4x² - 3x + 7:
- 4x² is a term
- -3x is a term
- 7 is a term

Coefficient: The numerical factor of a term
In 4x²: coefficient is 4
In -3x: coefficient is -3
In 7: coefficient is 7 (constant term)

Like Terms: Terms with the same variable part
3x and 7x are like terms (both have x)
2y² and -5y² are like terms (both have y²)
4 and -9 are like terms (both constants)

Unlike Terms: Terms with different variable parts
3x and 7y (different variables)
2x and 5x² (different powers)
```

Evaluating Expressions

Substitution and Evaluation

To evaluate an expression: 1. Substitute given values for variables 2. Follow order of operations (PEMDAS) 3. Simplify to get a numerical result

```
Evaluation Examples


Example 1: Evaluate 3x + 7 when x = 4
Step 1: Substitute x = 4
3(4) + 7
```

```
Step 2: Follow order of operations
12 + 7
Step 3: Simplify
19

Example 2: Evaluate 2x² - 5x + 1 when x = 3
Step 1: Substitute x = 3
2(3)² - 5(3) + 1
Step 2: Follow order of operations
2(9) - 15 + 1
18 - 15 + 1
Step 3: Simplify
4

Example 3: Evaluate (x + y)² when x = 5, y = -2
Step 1: Substitute values
(5 + (-2))²
Step 2: Simplify inside parentheses
(3)²
Step 3: Apply exponent
9
```

Combining Like Terms

Identifying Like Terms

Like terms have: - Same variable(s) - Same power(s) on each variable

```
Like Terms Examples


Like Terms:
3x and 7x (both have x¹)
-2y² and 5y² (both have y²)
4ab and -ab (both have ab)
8 and -3 (both are constants)

Unlike Terms:
3x and 7y (different variables)
2x and 5x² (different powers)
3xy and 4x²y (different powers on x)
5x and 8 (variable vs. constant)

Visual Representation:
3x means xxx
7x means xxxxxxx
```

```
3x + 7x means xxx + xxxxxxx = xxxxxxxxxx = 10x
```

Combining Like Terms

Rule: Add or subtract the coefficients, keep the variable part

```
Combining Examples


Simple Combining:
5x + 3x = (5 + 3)x = 8x
7y - 2y = (7 - 2)y = 5y
-4a + 9a = (-4 + 9)a = 5a

Multiple Like Terms:
3x + 7x - 2x = (3 + 7 - 2)x = 8x
5y² - 3y² + y² = (5 - 3 + 1)y² = 3y²

Mixed Terms:
4x + 3y + 2x - y
= (4x + 2x) + (3y - y)
= 6x + 2y

Complex Example:
3x² + 5x - 2x² + 7x - 4
= (3x² - 2x²) + (5x + 7x) - 4
= x² + 12x - 4
```

The Distributive Property

Understanding Distribution

```
Distributive Property


Basic Form: a(b + c) = ab + ac
"Distribute" the multiplication over addition/subtraction

Examples:
3(x + 4) = 3x + 12
5(2y - 1) = 10y - 5
-2(3a + 7) = -6a - 14
x(x + 5) = x² + 5x

Visual Model:
```

```
a(b + c) = a × (b + c)

Think of it as area:
    b    c

a   ab   ac

Total area = ab + ac

Reverse Distribution (Factoring):
6x + 9 = 3(2x + 3)
x² + 4x = x(x + 4)
```

Advanced Distribution

```
Complex Distribution Examples


Multiple Terms:
4(2x + 3y - 1) = 8x + 12y - 4
-3(x - 2y + 5) = -3x + 6y - 15

Variable Distributors:
x(y + z) = xy + xz
2a(3b - c) = 6ab - 2ac

Distributing Negative Signs:
-(x + 3) = -x - 3
-(2y - 5) = -2y + 5
-(-3x + 1) = 3x - 1

Common Mistakes:
Wrong: 3(x + 4) = 3x + 4
Right: 3(x + 4) = 3x + 12

Wrong: -(x - 3) = -x - 3
Right: -(x - 3) = -x + 3

Remember: Distribute to ALL terms inside parentheses!
```

Simplifying Expressions

Step-by-Step Simplification

Steps to Simplify: 1. Remove parentheses using distributive property 2. Combine like terms 3. Arrange in standard form (highest to lowest powers)

Simplification Examples

Example 1: 3(x + 2) + 5x - 1
Step 1: Distribute
3x + 6 + 5x - 1
Step 2: Combine like terms
(3x + 5x) + (6 - 1)
8x + 5

Example 2: 2(3y - 1) - (y + 4)
Step 1: Distribute (remember -(y + 4) = -y - 4)
6y - 2 - y - 4
Step 2: Combine like terms
(6y - y) + (-2 - 4)
5y - 6

Example 3: x^2 + 3x - 2(x^2 - x + 1)
Step 1: Distribute
x^2 + 3x - 2x^2 + 2x - 2
Step 2: Combine like terms
(x^2 - 2x^2) + (3x + 2x) - 2
-x^2 + 5x - 2

Real-World Applications

Writing Expressions for Real Situations

Translating Words to Expressions

Common Phrases and Their Translations:

Addition:
"5 more than a number" → x + 5
"the sum of x and 7" → x + 7
"increased by 3" → n + 3

Subtraction:
"4 less than a number" → x - 4
"decreased by 6" → n - 6
"the difference of a and b" → a - b

Multiplication:
"3 times a number" → 3x
"the product of 5 and y" → 5y

"twice a number" → 2n
"half of a number" → (1/2)x or x/2

Division:
"a number divided by 4" → x/4
"the quotient of x and 5" → x/5

Complex Expressions:
"5 more than twice a number" → 2x + 5
"3 less than half a number" → (1/2)x - 3
"the sum of a number and its square" → $x + x^2$

Practical Applications

Real-World Expression Examples

Cost Calculations:
"Concert tickets cost $25 each plus $5 service fee"
Total cost = 25n + 5 (where n = number of tickets)

Distance Problems:
"A car travels at 60 mph for h hours"
Distance = 60h miles

Geometry:
"Rectangle with length 3 more than width"
If width = w, then length = w + 3
Perimeter = 2w + 2(w + 3) = 4w + 6
Area = $w(w + 3) = w^2 + 3w$

Temperature:
"Celsius to Fahrenheit conversion"
F = (9/5)C + 32

Business:
"Profit = Revenue - Costs"
If revenue = 50x and costs = 200 + 30x
Then profit = 50x - (200 + 30x) = 20x - 200

Simple Interest:
"Interest = Principal × Rate × Time"
I = Prt
For $1000 at 5% for t years: I = 50t

Conclusion

Variables and expressions form the foundation of algebraic thinking, providing the tools needed to represent unknown quantities, describe patterns, and solve real-world problems. Mastering these concepts opens the door to all higher mathematics.

```
Variables and Expressions: Complete Understanding


Conceptual Understanding:
  Variables as symbols representing quantities
  Expressions as mathematical phrases
  Relationship between coefficients and variables

Procedural Fluency:
  Evaluating expressions by substitution
  Combining like terms systematically
  Using distributive property correctly

Strategic Competence:
  Translating word problems into expressions
  Choosing appropriate variables
  Simplifying complex expressions

Adaptive Reasoning:
  Understanding why like terms combine
  Recognizing equivalent expressions
  Connecting arithmetic and algebra

Productive Disposition:
  Confidence with abstract symbols
  Appreciation for algebraic generalization
  Persistence in complex problems
```

Whether you're calculating costs, analyzing patterns, or modeling real-world situations, variables and expressions provide the essential language for mathematical communication and problem-solving. These skills serve as the foundation for success in all areas of advanced mathematics and science.

Solving Equations: Finding the Unknown

Introduction

An equation is a mathematical statement that two expressions are equal. Solving an equation means finding the value(s) of the variable that make the equation true. This fundamental skill is essential for all of algebra and provides powerful tools for solving real-world problems.

Equation solving is like being a mathematical detective - you use logical reasoning and systematic methods to uncover the mystery value that makes the equation balance perfectly.

```
Equation vs. Expression


Expression: 3x + 7 (represents a value)
Equation: 3x + 7 = 19 (makes a statement)

The equation states that 3x + 7 equals 19
Our job: Find the value of x that makes this true
```

Understanding Equations

What Makes an Equation True?

An equation can be: 1. Always true (identity) 2. Sometimes true (conditional) 3. Never true (contradiction)

```
Types of Equations


Always True (Identity):
x + 3 = x + 3   (true for any value of x)
2(x + 1) = 2x + 2   (true for any value of x)

Sometimes True (Conditional):
x + 5 = 12   when x = 7,   when x   7
2x = 10   when x = 5,   when x   5

Never True (Contradiction):
x + 1 = x + 2   (impossible for any value of x)
```

```
0 = 5   (always false)

Solution: The value(s) that make the equation true
For 3x - 1 = 8, the solution is x = 3
```

The Balance Model

Think of an equation as a balanced scale. Whatever you do to one side, you must do to the other to maintain balance.

```
Balance Model Visualization


    3x + 1 = 10
           =
      3x + 1          10


To solve: Isolate x by using inverse operations
Goal: Get x by itself on one side

Subtract 1 from both sides:
           =
        3x             9


Divide both sides by 3:
           =
         x             3


Solution: x = 3
```

Basic Equation Solving

One-Step Equations

These equations require only one operation to solve.

```
One-Step Equation Types


Addition Equations:
x + 7 = 15
Subtract 7 from both sides:
```

```
x + 7 - 7 = 15 - 7
x = 8

Check: 8 + 7 = 15

Subtraction Equations:
x - 5 = 12
Add 5 to both sides:
x - 5 + 5 = 12 + 5
x = 17

Check: 17 - 5 = 12

Multiplication Equations:
3x = 21
Divide both sides by 3:
3x ÷ 3 = 21 ÷ 3
x = 7

Check: 3(7) = 21

Division Equations:
x/4 = 9
Multiply both sides by 4:
(x/4) × 4 = 9 × 4
x = 36

Check: 36/4 = 9
```

Two-Step Equations

These equations require two operations to solve. Always undo addition/subtraction first, then multiplication/division.

```
Two-Step Equation Process


General Form: ax + b = c
Step 1: Subtract b from both sides → ax = c - b
Step 2: Divide both sides by a → x = (c - b)/a

Example 1: 2x + 5 = 17
Step 1: Subtract 5 from both sides
2x + 5 - 5 = 17 - 5
2x = 12

Step 2: Divide both sides by 2
```

```
2x ÷ 2 = 12 ÷ 2
x = 6

Check: 2(6) + 5 = 12 + 5 = 17

Example 2: 3x - 7 = 14
Step 1: Add 7 to both sides
3x - 7 + 7 = 14 + 7
3x = 21

Step 2: Divide both sides by 3
3x ÷ 3 = 21 ÷ 3
x = 7

Check: 3(7) - 7 = 21 - 7 = 14

Example 3: -4x + 1 = 13
Step 1: Subtract 1 from both sides
-4x + 1 - 1 = 13 - 1
-4x = 12

Step 2: Divide both sides by -4
-4x ÷ (-4) = 12 ÷ (-4)
x = -3

Check: -4(-3) + 1 = 12 + 1 = 13
```

Multi-Step Equations

Equations with Variables on Both Sides

When variables appear on both sides, collect all variable terms on one side and all constants on the other.

```
Variables on Both Sides


Example 1: 3x + 7 = x + 15
Step 1: Subtract x from both sides
3x - x + 7 = x - x + 15
2x + 7 = 15

Step 2: Subtract 7 from both sides
2x + 7 - 7 = 15 - 7
2x = 8
```

```
Step 3: Divide both sides by 2
2x ÷ 2 = 8 ÷ 2
x = 4

Check: 3(4) + 7 = 12 + 7 = 19
       4 + 15 = 19

Example 2: 5x - 3 = 2x + 12
Step 1: Subtract 2x from both sides
5x - 2x - 3 = 2x - 2x + 12
3x - 3 = 12

Step 2: Add 3 to both sides
3x - 3 + 3 = 12 + 3
3x = 15

Step 3: Divide both sides by 3
x = 5

Check: 5(5) - 3 = 25 - 3 = 22
       2(5) + 12 = 10 + 12 = 22

Strategy: Move variables to the side with more variable terms
This often results in positive coefficients
```

Equations with Parentheses

Use the distributive property first, then solve as usual.

```
Equations with Parentheses


Example 1: 3(x + 4) = 21
Step 1: Distribute
3x + 12 = 21

Step 2: Subtract 12 from both sides
3x = 9

Step 3: Divide by 3
x = 3

Check: 3(3 + 4) = 3(7) = 21

Example 2: 2(x - 3) + 5 = 15
Step 1: Distribute
2x - 6 + 5 = 15
```

```
Step 2: Combine like terms
2x - 1 = 15

Step 3: Add 1 to both sides
2x = 16

Step 4: Divide by 2
x = 8

Check: 2(8 - 3) + 5 = 2(5) + 5 = 10 + 5 = 15

Example 3: 4(2x + 1) = 3(x - 2)
Step 1: Distribute both sides
8x + 4 = 3x - 6

Step 2: Subtract 3x from both sides
5x + 4 = -6

Step 3: Subtract 4 from both sides
5x = -10

Step 4: Divide by 5
x = -2

Check: 4(2(-2) + 1) = 4(-4 + 1) = 4(-3) = -12
       3(-2 - 2) = 3(-4) = -12
```

Equations with Fractions

Clearing Fractions

Multiply both sides by the least common denominator (LCD) to eliminate fractions.

```
Fraction Equations


Example 1: x/3 + 2 = 7
Method 1: Work with fractions
Subtract 2: x/3 = 5
Multiply by 3: x = 15

Method 2: Clear fractions first
Multiply everything by 3:
3(x/3) + 3(2) = 3(7)
x + 6 = 21
```

```
x = 15

Example 2: (x + 1)/4 = 3
Multiply both sides by 4:
4 · (x + 1)/4 = 4 · 3
x + 1 = 12
x = 11

Check: (11 + 1)/4 = 12/4 = 3

Example 3: x/2 + x/3 = 10
LCD = 6, multiply everything by 6:
6(x/2) + 6(x/3) = 6(10)
3x + 2x = 60
5x = 60
x = 12

Check: 12/2 + 12/3 = 6 + 4 = 10

Example 4: (2x - 1)/3 = (x + 4)/2
Cross multiply or find LCD = 6:
2(2x - 1) = 3(x + 4)
4x - 2 = 3x + 12
x = 14

Check: (2(14) - 1)/3 = 27/3 = 9
       (14 + 4)/2 = 18/2 = 9
```

Special Cases

No Solution and Infinite Solutions

```
Special Solution Cases


No Solution (Contradiction):
Example: 2x + 3 = 2x + 7
Subtract 2x from both sides:
3 = 7 (False!)

This means there is no value of x that makes the equation true.
Answer: No solution or  (empty set)

Infinite Solutions (Identity):
Example: 3x + 6 = 3(x + 2)
Distribute right side:
```

```
3x + 6 = 3x + 6 (Always true!)

This means any value of x makes the equation true.
Answer: All real numbers or infinite solutions

How to Recognize:
- No solution: Variables cancel, leaving false statement (3 = 7)
- Infinite solutions: Variables cancel, leaving true statement (6 = 6)
- One solution: Variables don't cancel, can solve for x

Example Analysis:
4x + 8 = 4(x + 3)
4x + 8 = 4x + 12
8 = 12 (False)
Answer: No solution

5x - 10 = 5(x - 2)
5x - 10 = 5x - 10
-10 = -10 (True)
Answer: Infinite solutions
```

Problem-Solving with Equations

Translating Word Problems

```
Word Problem Strategy


Steps:
1. Read the problem carefully
2. Define the variable (let x = ...)
3. Translate words to equation
4. Solve the equation
5. Check answer in original problem
6. Answer the question asked

Key Phrases:
"is", "equals", "gives" → =
"more than", "sum", "plus" → +
"less than", "difference", "minus" → -
"times", "product", "of" → ×
"divided by", "quotient" → ÷

Example 1: Number Problems
"Five more than twice a number is 17. Find the number."
```

```
Step 1: Let x = the number
Step 2: "Five more than twice a number" = 2x + 5
Step 3: 2x + 5 = 17
Step 4: Solve
2x = 12
x = 6
Step 5: Check: 2(6) + 5 = 17
Step 6: The number is 6.

Example 2: Age Problems
"Maria is 3 years older than John. The sum of their ages is 27. How old is each person?"

Step 1: Let x = John's age, then x + 3 = Maria's age
Step 2: Sum of ages = 27
Step 3: x + (x + 3) = 27
Step 4: Solve
2x + 3 = 27
2x = 24
x = 12
Step 5: John is 12, Maria is 15
Check: 12 + 15 = 27
Step 6: John is 12 years old, Maria is 15 years old.
```

Real-World Applications

```
Practical Equation Applications


Cost Problems:
"A cell phone plan costs $30 per month plus $0.10 per text.
If the monthly bill is $45, how many texts were sent?"

Let x = number of texts
30 + 0.10x = 45
0.10x = 15
x = 150 texts

Geometry Problems:
"The perimeter of a rectangle is 36 cm. The length is 4 cm
more than the width. Find the dimensions."

Let w = width, then w + 4 = length
Perimeter = 2w + 2(w + 4) = 36
2w + 2w + 8 = 36
4w = 28
w = 7 cm
Length = 11 cm
```

```
Distance Problems:
"Two cars start from the same point and travel in opposite
directions. One travels at 60 mph, the other at 70 mph.
After how many hours will they be 390 miles apart?"

Let t = time in hours
Distance apart = 60t + 70t = 130t
130t = 390
t = 3 hours

Business Problems:
"A company's profit is $50 per item sold minus $200 in fixed costs.
How many items must be sold to make a profit of $800?"

Let x = number of items
Profit = 50x - 200 = 800
50x = 1000
x = 20 items
```

Checking Solutions

Verification Methods

Always check your solutions by substituting back into the original equation.

```
Solution Checking


Method 1: Direct Substitution
Original equation: 3x - 7 = 14
Solution: x = 7
Check: 3(7) - 7 = 21 - 7 = 14

Method 2: Estimation Check
For 2x + 5 = 23, solution x = 9
Estimate: If x   10, then 2(10) + 5 = 25   23
This confirms x = 9 is reasonable

Method 3: Graphical Check
Plot y = left side and y = right side
Solution is where graphs intersect

Why Check Solutions?
- Catch arithmetic errors
- Verify answer makes sense
```

- Confirm you solved correctly
- Build confidence in your work

Common Checking Mistakes:
- Substituting into simplified equation instead of original
- Making arithmetic errors during checking
- Not checking units in word problems
- Forgetting to check if answer makes sense in context

Common Mistakes and How to Avoid Them

Typical Equation-Solving Errors

Common Mistakes and Solutions

Mistake 1: Sign Errors
Wrong: x - 5 = 12, so x = 12 - 5 = 7
Right: x - 5 = 12, so x = 12 + 5 = 17
Solution: Use inverse operations carefully

Mistake 2: Distribution Errors
Wrong: 3(x + 4) = 3x + 4
Right: 3(x + 4) = 3x + 12
Solution: Distribute to ALL terms

Mistake 3: Combining Unlike Terms
Wrong: 3x + 5y = 8xy
Right: 3x + 5y cannot be simplified
Solution: Only combine like terms

Mistake 4: Moving Terms Incorrectly
Wrong: 2x + 3 = 7, so 2x = 7 + 3 = 10
Right: 2x + 3 = 7, so 2x = 7 - 3 = 4
Solution: Use inverse operations on both sides

Mistake 5: Fraction Operations
Wrong: x/3 = 5, so x = 5/3
Right: x/3 = 5, so x = 15
Solution: Multiply both sides by denominator

Prevention Strategies:
- Work step by step
- Show all work clearly
- Check each step
- Verify final answer

- Practice regularly

Building Equation-Solving Skills

Practice Progression

```
Skill Development Sequence


Week 1: One-Step Equations
- Addition/subtraction equations
- Multiplication/division equations
- Checking solutions

Week 2: Two-Step Equations
- ax + b = c format
- Negative coefficients
- Fraction coefficients

Week 3: Multi-Step Equations
- Variables on both sides
- Combining like terms
- Parentheses and distribution

Week 4: Special Cases and Applications
- No solution/infinite solutions
- Word problem translation
- Real-world applications

Daily Practice Routine:
1. Warm-up: 3 one-step equations (5 minutes)
2. Main focus: Current skill level (15 minutes)
3. Word problem: 1 application (10 minutes)
4. Review: Check previous work (5 minutes)

Study Tips:
- Keep a solution journal
- Practice different equation types daily
- Work with a study partner
- Use online equation solvers to check work
- Connect equations to real situations
```

Conclusion

Solving equations is a fundamental skill that opens doors to advanced mathematics and real-world problem-solving. The systematic approach of maintaining balance while isolating variables provides a powerful method for finding unknown quantities.

```
Equation Solving: Complete Understanding


Conceptual Understanding:
  Equations as balanced statements
  Solutions as values that make equations true
  Inverse operations for maintaining balance

Procedural Fluency:
  Systematic solving of linear equations
  Handling special cases (no solution, infinite solutions)
  Working with fractions and parentheses

Strategic Competence:
  Translating word problems into equations
  Choosing appropriate solving strategies
  Checking solutions for accuracy and reasonableness

Adaptive Reasoning:
  Understanding why equation-solving procedures work
  Recognizing when equations have special solutions
  Making connections between equations and real situations

Productive Disposition:
  Confidence in systematic problem-solving
  Persistence through multi-step procedures
  Appreciation for the power of algebraic methods
```

From ancient Babylonian mathematicians solving quadratic equations to modern engineers designing bridges, the ability to solve equations has been central to mathematical and scientific progress. Whether you're calculating loan payments, determining optimal business strategies, or analyzing scientific data, equation-solving skills provide essential tools for quantitative reasoning and problem-solving in countless real-world situations.

Inequalities: When Things Are Not Equal

Introduction

An inequality is a mathematical statement that compares two expressions using symbols like $<$, $>$, , or . Unlike equations that have specific solutions, inequalities often have ranges of solutions, making them powerful tools for describing real-world constraints and limitations.

Inequalities help us answer questions like "What scores do I need to get an A?" or "How many items must we sell to make a profit?" They describe relationships where one quantity is greater than, less than, or within a certain range of another.

```
Inequality vs. Equation


Equation: x + 3 = 7 (x must equal 4)
Inequality: x + 3 > 7 (x can be any number greater than 4)

The inequality has infinitely many solutions:
x = 5, x = 10, x = 100, etc.
```

Understanding Inequality Symbols

Basic Inequality Symbols

```
Inequality Symbol Meanings


< : "less than"
> : "greater than"
  : "less than or equal to"
  : "greater than or equal to"
  : "not equal to"

Examples:
5 < 8 (5 is less than 8)
10 > 3 (10 is greater than 3)
x   7 (x is less than or equal to 7)
y   -2 (y is greater than or equal to -2)
a   0 (a is not equal to 0)
```

```
Memory Tricks:
- The "mouth" opens toward the larger number
- Think of < as "L" for "Less than"
-  means "less than OR equal to"
-  means "greater than OR equal to"

Reading Inequalities:
3 < x < 7 reads "3 is less than x, and x is less than 7"
or "x is between 3 and 7"
```

Graphing Inequalities on Number Lines

```
Number Line Representations


x > 3 (x is greater than 3):
←        →
      3
Open circle at 3 (3 not included)
Arrow points right (greater values)

x   -1 (x is less than or equal to -1):
←        →
     -1
Closed circle at -1 (-1 is included)
Arrow points left (lesser values)

-2 < x   4 (x is between -2 and 4, including 4):
←         →
  -2         4
Open circle at -2, closed circle at 4
Line segment between them

x   0 (x is greater than or equal to 0):
←        →
      0
Closed circle at 0 (0 is included)
Arrow points right

Circle Types:
  Open circle: value NOT included (<, >)
  Closed circle: value IS included ( ,  )
```

Solving Linear Inequalities

Basic Inequality Solving

Solving inequalities is similar to solving equations, with one crucial difference: when you multiply or divide by a negative number, you must flip the inequality sign.

```
Inequality Solving Rules


Same as equations:
- Add/subtract same number to both sides
- Multiply/divide by positive number

SPECIAL RULE:
When multiplying or dividing by a negative number,
FLIP the inequality sign!

Examples:

Addition/Subtraction:
x + 5 > 12
x + 5 - 5 > 12 - 5
x > 7

Multiplication/Division by Positive:
3x   15
3x ÷ 3   15 ÷ 3
x   5

Multiplication/Division by Negative:
-2x < 10
-2x ÷ (-2) > 10 ÷ (-2)  ← Sign flips!
x > -5

Why flip the sign?
If -2 < -1, then multiplying by -1 gives:
2 > 1 (inequality flips)
```

One-Step Inequalities

```
One-Step Inequality Examples


Type 1: Addition
x + 7   12
```

```
x   5

Graph: ←        →
              5

Type 2: Subtraction
x - 4 < 9
x < 13

Graph: ←        →
             13

Type 3: Multiplication (positive)
3x > 18
x > 6

Graph: ←        →
              6

Type 4: Multiplication (negative)
-5x   20
x   -4  ← Sign flipped!

Graph: ←        →
             -4

Type 5: Division (positive)
x/2   8
x   16

Type 6: Division (negative)
x/(-3) < 6
x > -18  ← Sign flipped!

Checking Solutions:
For x > 6, test x = 7:
3(7) = 21 > 18

For x   -4, test x = 0:
-5(0) = 0   20
```

Two-Step Inequalities

```
Two-Step Inequality Process


General form: ax + b < c
```

```
Step 1: Subtract b from both sides
Step 2: Divide by a (flip sign if a is negative)

Example 1: 2x + 3  11
Step 1: 2x  8
Step 2: x  4

Graph: ←       →
             4

Example 2: -3x + 7 > 1
Step 1: -3x > -6
Step 2: x < 2  ← Sign flipped!

Graph: ←       →
             2

Example 3: 5 - 2x  13
Step 1: -2x  8
Step 2: x  -4  ← Sign flipped!

Graph: ←       →
            -4

Example 4: (x + 1)/3 < 4
Step 1: x + 1 < 12
Step 2: x < 11

Checking: For x  4, test x = 0:
2(0) + 3 = 3  11
```

Multi-Step Inequalities

Inequalities with Variables on Both Sides

```
Variables on Both Sides


Example 1: 3x + 5 > x + 13
Step 1: Subtract x from both sides
2x + 5 > 13
Step 2: Subtract 5 from both sides
2x > 8
Step 3: Divide by 2
x > 4
```

```
Example 2: 7x - 2  4x + 10
Step 1: Subtract 4x from both sides
3x - 2  10
Step 2: Add 2 to both sides
3x  12
Step 3: Divide by 3
x  4

Example 3: 2x - 7 < 5x + 8
Step 1: Subtract 2x from both sides
-7 < 3x + 8
Step 2: Subtract 8 from both sides
-15 < 3x
Step 3: Divide by 3
-5 < x  or  x > -5

Strategy: Move variables to side with larger coefficient
This often avoids negative coefficients
```

Inequalities with Parentheses

```
Parentheses in Inequalities


Example 1: 3(x - 2)  15
Step 1: Distribute
3x - 6  15
Step 2: Add 6
3x  21
Step 3: Divide by 3
x  7

Example 2: 2(x + 1) < 4(x - 3)
Step 1: Distribute both sides
2x + 2 < 4x - 12
Step 2: Subtract 2x
2 < 2x - 12
Step 3: Add 12
14 < 2x
Step 4: Divide by 2
7 < x  or  x > 7

Example 3: -2(3x - 1) > 10
Step 1: Distribute
-6x + 2 > 10
Step 2: Subtract 2
-6x > 8
```

```
Step 3: Divide by -6 (flip sign!)
x < -4/3

Example 4: 5 - 3(x + 2)   8
Step 1: Distribute
5 - 3x - 6   8
Step 2: Combine like terms
-1 - 3x   8
Step 3: Add 1
-3x   9
Step 4: Divide by -3 (flip sign!)
x   -3
```

Compound Inequalities

"And" Compound Inequalities

These represent values that satisfy both conditions simultaneously.

```
"And" Compound Inequalities


Form: a < x < b (x is between a and b)
This means: x > a AND x < b

Example 1: -3 < x < 5
Solution: All numbers between -3 and 5
Graph: ←       →
         -3        5

Example 2: Solve 1 < 2x + 3 < 9
Method: Solve both parts simultaneously
1 < 2x + 3 < 9
Subtract 3 from all parts:
1 - 3 < 2x < 9 - 3
-2 < 2x < 6
Divide all parts by 2:
-1 < x < 3

Graph: ←       →
         -1        3

Example 3: Solve -4   3x - 1   8
-4   3x - 1   8
Add 1 to all parts:
-3   3x   9
```

```
Divide by 3:
-1   x   3

Graph: ←        →
         -1         3

Checking: Test x = 0 (should work)
-4   3(0) - 1   8
-4   -1   8
```

"Or" Compound Inequalities

These represent values that satisfy at least one condition.

```
"Or" Compound Inequalities


Form: x < a OR x > b
Solution: Two separate regions

Example 1: x < -2 OR x > 4
Graph: ←        →
         -2         4
       ←             →

Example 2: Solve 2x + 1 < -3 OR 2x + 1 > 7
Solve each inequality separately:

Left side: 2x + 1 < -3
2x < -4
x < -2

Right side: 2x + 1 > 7
2x > 6
x > 3

Solution: x < -2 OR x > 3

Graph: ←        →
         -2         3
       ←             →

Example 3: |x| > 5
This means: x < -5 OR x > 5
(Distance from 0 is greater than 5)

Graph: ←        →
```

```
         -5        5
       ←            →
```

```
No Solution Case:
x > 5 AND x < 2
This is impossible (no overlap)
Answer: No solution or
```

```
All Real Numbers Case:
x > 2 OR x < 5
This covers all real numbers
Answer: All real numbers or (-ϖ, ϖ)
```

Absolute Value Inequalities

Understanding Absolute Value Inequalities

Absolute value represents distance from zero, so absolute value inequalities describe ranges of distances.

```
Absolute Value Inequality Types


Type 1: |x| < a (distance less than a)
Solution: -a < x < a
Graph: ←       →
         -a        a

Type 2: |x| > a (distance greater than a)
Solution: x < -a OR x > a
Graph: ←       →
         -a        a
       ←            →

Examples:

|x| < 3
Solution: -3 < x < 3
Graph: ←       →
         -3        3

|x|  2
Solution: x  -2 OR x  2
Graph: ←       →
         -2        2
       ←            →
```

```
|x + 1| < 4
Think: Distance from -1 is less than 4
-4 < x + 1 < 4
-5 < x < 3

|2x - 3| > 5
This means: 2x - 3 < -5 OR 2x - 3 > 5

Left: 2x - 3 < -5    Right: 2x - 3 > 5
      2x < -2              2x > 8
      x < -1               x > 4

Solution: x < -1 OR x > 4
```

Word Problems with Inequalities

Translating Word Problems

```
Inequality Word Problem Strategy


Key Phrases:
"at least" →
"at most" →
"more than" → >
"less than" → <
"between" → compound inequality
"maximum" →
"minimum" →

Example 1: Grade Requirements
"To get an A, you need at least 90% average on 4 tests.
Your first three scores are 85, 92, and 88. What score
do you need on the fourth test?"

Let x = fourth test score
Average  90
(85 + 92 + 88 + x)/4  90
(265 + x)/4  90
265 + x  360
x  95

You need at least 95% on the fourth test.

Example 2: Budget Constraints
```

"You have $50 to spend on books. Each book costs $12.
How many books can you buy?"

Let x = number of books
Cost Budget
12x 50
x 4.17...

Since you can't buy part of a book: x 4
You can buy at most 4 books.

Example 3: Temperature Range
"The temperature must be between 68°F and 72°F.
Write an inequality for Celsius temperature."

F = (9/5)C + 32
68 (9/5)C + 32 72
36 (9/5)C 40
20 C 22.22...

The Celsius temperature must be between 20°C and 22.2°C.

Business and Economics Applications

Real-World Inequality Applications

Profit Analysis:
"A company makes $15 profit per item but has $300 in
fixed costs. How many items must they sell to make
at least $500 profit?"

Let x = number of items
Profit = Revenue - Costs
15x - 300 500
15x 800
x 53.33...

They must sell at least 54 items.

Break-Even Analysis:
"Revenue is $25 per item, costs are $18 per item plus
$140 fixed costs. How many items for break-even?"

Revenue Costs
25x 18x + 140
7x 140

```
x   20

Need to sell at least 20 items to break even.

Shipping Constraints:
"A box can hold at most 50 pounds. Small items weigh
2 pounds each, large items weigh 5 pounds each. If you
have 8 small items, how many large items can you add?"

Let x = number of large items
Total weight   50
2(8) + 5x   50
16 + 5x   50
5x   34
x   6.8

You can add at most 6 large items.

Investment Planning:
"You want to invest in stocks (risky) and bonds (safe).
You have $10,000 and want at most 30% in stocks.
How much can you invest in stocks?"

Let x = amount in stocks
x   0.30(10,000)
x   3,000

You can invest at most $3,000 in stocks.
```

Graphing Inequalities

Coordinate Plane Inequalities

```
Graphing Linear Inequalities


Steps:
1. Graph the boundary line (y = mx + b)
2. Use solid line for   or
3. Use dashed line for < or >
4. Shade appropriate region
5. Test a point to verify

Example 1: y > 2x + 1
Step 1: Graph y = 2x + 1 (dashed line)
Step 2: Shade above the line (y > means above)
```

```
Test point (0, 0):
0 > 2(0) + 1
0 > 1 (False)
So (0,0) is NOT in solution region
Shade the region that doesn't contain (0,0)

Example 2: y   -x + 3
Step 1: Graph y = -x + 3 (solid line)
Step 2: Shade below the line (y   means below)

Test point (0, 0):
0   -(0) + 3
0   3 (True)
So (0,0) IS in solution region
Shade the region containing (0,0)

Example 3: x   -2
This is a vertical line at x = -2
Shade to the right (x   means right of line)

Memory Aid:
y > or y   → shade ABOVE
y < or y   → shade BELOW
x > or x   → shade RIGHT
x < or x   → shade LEFT
```

Common Mistakes and Solutions

Typical Inequality Errors

```
Common Inequality Mistakes


Mistake 1: Not Flipping Sign with Negatives
Wrong: -2x < 6, so x < -3
Right: -2x < 6, so x > -3

Solution: Always flip when multiplying/dividing by negative

Mistake 2: Wrong Circle Type on Graph
Wrong: x > 3 with closed circle
Right: x > 3 with open circle

Solution:
  for < and > (not included)
```

```
 for   and   (included)

Mistake 3: Compound Inequality Errors
Wrong: x < 2 OR x > 5 written as 2 > x > 5
Right: x < 2 OR x > 5 (two separate regions)

Solution: "AND" gives overlap, "OR" gives union

Mistake 4: Absolute Value Confusion
Wrong: |x| > 3 means -3 > x > 3
Right: |x| > 3 means x < -3 OR x > 3

Solution:
|x| < a → -a < x < a (between)
|x| > a → x < -a OR x > a (outside)

Mistake 5: Word Problem Translation
Wrong: "at most 5" means x > 5
Right: "at most 5" means x   5

Solution: Learn key phrase meanings
```

Conclusion

Inequalities extend our problem-solving toolkit beyond exact solutions to ranges and constraints. They model real-world situations where we need to find acceptable ranges rather than precise values.

```
Inequalities: Complete Understanding


Conceptual Understanding:
 Inequalities as comparisons between expressions
 Solution sets as ranges rather than specific values
 Absolute value as distance from zero

Procedural Fluency:
 Solving linear inequalities systematically
 Handling compound and absolute value inequalities
 Graphing solutions on number lines and coordinate planes

Strategic Competence:
 Translating word problems involving constraints
 Choosing appropriate inequality symbols
 Interpreting solutions in context
```

```
Adaptive Reasoning:
  Understanding why signs flip with negative multiplication
  Recognizing when to use "and" vs "or" logic
  Making connections between algebraic and graphical representations

Productive Disposition:
  Confidence working with ranges and constraints
  Appreciation for inequality applications in real life
  Persistence in multi-step inequality problems
```

From quality control in manufacturing to financial planning, from sports statistics to scientific research, inequalities provide essential tools for describing and working with constraints, limitations, and acceptable ranges in countless real-world applications.

Graphing Linear Equations: Visualizing Relationships

Introduction

Graphing linear equations transforms abstract algebraic relationships into visual representations that reveal patterns, trends, and connections. A linear equation creates a straight line when graphed, making it one of the most fundamental and useful tools in mathematics.

Linear graphs help us understand relationships between variables, make predictions, and solve real-world problems involving rates, trends, and proportional relationships.

```
From Equation to Graph


Equation: y = 2x + 1
Table:    x | y
         -1 | -1
          0 |  1
          1 |  3
          2 |  5

Graph:    y
          |
        5 |     •
          |   /
        3 |  •
          | /
        1 |•
          |
       -1 |•
          +       x
         -1  0  1  2

The points form a straight line!
```

The Coordinate Plane

Understanding Coordinates

The coordinate plane (also called the Cartesian plane) uses two perpendicular number lines to locate points in two-dimensional space.

```
Coordinate Plane Structure


    y-axis (vertical)
      |
    4         Point (3, 4)
      |     |
    3
      |     |
    2
      |     |
    1
      |     |
                  x-axis (horizontal)
   -2  -1     1     2     3
      |     |     |     |
   -1
      |     |     |     |
   -2
```

```
Origin: (0, 0) - where axes intersect

Ordered Pair: (x, y)
- x-coordinate: horizontal position (left/right)
- y-coordinate: vertical position (up/down)
- Order matters! (3, 4)   (4, 3)

Quadrants:
I:   x > 0, y > 0  (upper right)
II:  x < 0, y > 0  (upper left)
III: x < 0, y < 0  (lower left)
IV:  x > 0, y < 0  (lower right)
```

Plotting Points

```
Point Plotting Process


To plot point (a, b):
```

```
1. Start at origin (0, 0)
2. Move 'a' units horizontally (right if positive, left if negative)
3. Move 'b' units vertically (up if positive, down if negative)
4. Mark the point

Examples:

Plot (2, 3):
- Start at (0, 0)
- Move 2 units right
- Move 3 units up
- Mark point

Plot (-1, 4):
- Start at (0, 0)
- Move 1 unit left
- Move 4 units up
- Mark point

Plot (3, -2):
- Start at (0, 0)
- Move 3 units right
- Move 2 units down
- Mark point

Plot (-2, -1):
- Start at (0, 0)
- Move 2 units left
- Move 1 unit down
- Mark point

Special Points:
(0, b): on y-axis
(a, 0): on x-axis
(0, 0): origin
```

Linear Equations and Their Graphs

What Makes an Equation Linear?

A linear equation in two variables has the form Ax + By = C, where A, B, and C are constants and A and B are not both zero.

```
Linear Equation Forms
```

```
Standard Form: Ax + By = C
Examples: 2x + 3y = 6, x - y = 4, 3x + y = -2

Slope-Intercept Form: y = mx + b
Examples: y = 2x + 1, y = -3x + 5, y = (1/2)x - 3

Point-Slope Form: y - y  = m(x - x )
Examples: y - 2 = 3(x - 1), y + 1 = -2(x + 3)

Linear Characteristics:
- Variables have power of 1 only
- No products of variables (no xy terms)
- No variables in denominators
- No variables under radicals
- Graph is always a straight line

Non-Linear Examples:
y = x² (parabola)
y = 1/x (hyperbola)
x² + y² = 4 (circle)
y = |x| (absolute value)
```

Graphing by Making a Table

The most basic method for graphing linear equations is to create a table of values.

```
Table Method Process


Steps:
1. Choose several x-values
2. Calculate corresponding y-values
3. Plot the points
4. Connect with a straight line

Example: Graph y = 2x - 1

Step 1: Choose x-values
x = -2, -1, 0, 1, 2

Step 2: Calculate y-values
x = -2: y = 2(-2) - 1 = -5
x = -1: y = 2(-1) - 1 = -3
x = 0:  y = 2(0) - 1 = -1
x = 1:  y = 2(1) - 1 = 1
x = 2:  y = 2(2) - 1 = 3
```

```
Step 3: Create table
x | y
-2| -5
-1| -3
 0| -1
 1|  1
 2|  3

Step 4: Plot points and connect
    y
    |
  3 |     •
    |   /
  1 |  •
    | /
 -1 |•
    |/
 -3 |•
    |
 -5 |•
    +       x
   -2 -1 0 1 2

Tips:
- Use at least 3 points (more for accuracy)
- Include x = 0 if possible (y-intercept)
- Choose easy numbers when possible
- Check that points form a straight line
```

Slope: The Rate of Change

Understanding Slope

Slope measures the steepness and direction of a line. It represents the rate of change between variables.

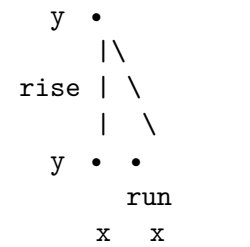
```
Slope Definition and Formula


Slope = rise/run = change in y/change in x

Formula: m = (y  - y )/(x  - x )

where (x , y ) and (x , y ) are any two points on the line

Visual Representation:
```



Example: Find slope of line through (1, 2) and (4, 8)
m = (8 - 2)/(4 - 1) = 6/3 = 2

This means: for every 1 unit right, go up 2 units

Types of Slope:
Positive slope: line rises left to right (/)
Negative slope: line falls left to right (\)
Zero slope: horizontal line (-)
Undefined slope: vertical line (|)

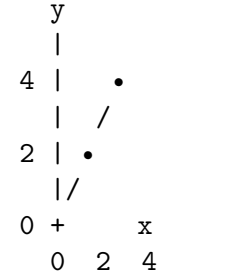
Slope Interpretations:
m = 2: steep upward
m = 1/2: gentle upward
m = -3: steep downward
m = 0: horizontal
m = undefined: vertical

Calculating Slope from Graphs

Finding Slope from Graphs

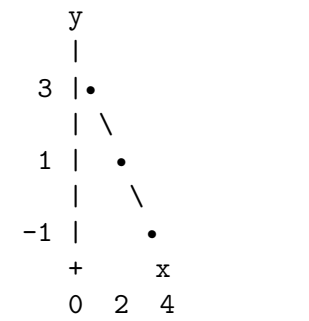
Method: Pick two clear points and count rise over run

Example 1:



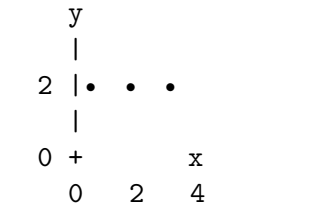
Points: (0, 0) and (4, 4)
Rise = 4, Run = 4
Slope = 4/4 = 1

Example 2:



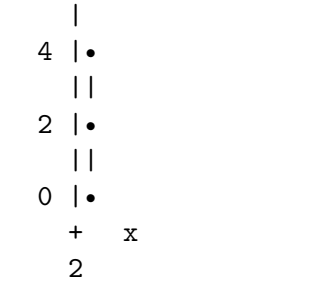
Points: (0, 3) and (4, -1)
Rise = -1 - 3 = -4
Run = 4 - 0 = 4
Slope = -4/4 = -1

Example 3: Horizontal line



All points have same y-coordinate
Rise = 0, Run = any value
Slope = 0/run = 0

Example 4: Vertical line



All points have same x-coordinate
Rise = any value, Run = 0
Slope = rise/0 = undefined

Slope-Intercept Form

Understanding y = mx + b

The slope-intercept form y = mx + b is the most useful form for graphing and understanding linear equations.

```
Slope-Intercept Form Components


y = mx + b

m = slope (rate of change)
b = y-intercept (where line crosses y-axis)

Example: y = 3x - 2
- Slope (m) = 3
- Y-intercept (b) = -2
- Line crosses y-axis at (0, -2)
- For every 1 unit right, go up 3 units

Example: y = -1/2 x + 4
- Slope (m) = -1/2
- Y-intercept (b) = 4
- Line crosses y-axis at (0, 4)
- For every 2 units right, go down 1 unit

Special Cases:
y = 5 (same as y = 0x + 5)
- Slope = 0 (horizontal line)
- Y-intercept = 5

x = 3 (vertical line)
- Cannot be written in y = mx + b form
- Slope is undefined
- No y-intercept (unless line is y-axis)

Converting to Slope-Intercept Form:
2x + 3y = 12
3y = -2x + 12
y = -2/3 x + 4

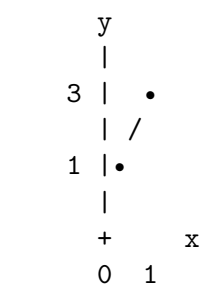
Slope = -2/3, Y-intercept = 4
```

Graphing Using Slope-Intercept Form

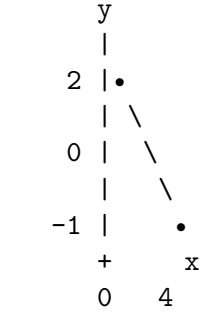
```
Slope-Intercept Graphing Method
```

```
Steps:
1. Identify y-intercept (b) and plot point (0, b)
2. Use slope (m = rise/run) to find next point
3. Connect points with straight line

Example 1: Graph y = 2x + 1
Step 1: Y-intercept = 1, plot (0, 1)
Step 2: Slope = 2 = 2/1 (up 2, right 1)
        From (0, 1): go right 1, up 2 → (1, 3)
Step 3: Connect points
```




```
Example 2: Graph y = -3/4 x + 2
Step 1: Y-intercept = 2, plot (0, 2)
Step 2: Slope = -3/4 (down 3, right 4)
        From (0, 2): go right 4, down 3 → (4, -1)
Step 3: Connect points
```




```
Alternative slope interpretation:
-3/4 = 3/(-4) (up 3, left 4)
From (0, 2): go left 4, up 3 → (-4, 5)

Example 3: Graph y = -1/3 x
Step 1: Y-intercept = 0, plot (0, 0)
Step 2: Slope = -1/3 (down 1, right 3)
        From (0, 0): go right 3, down 1 → (3, -1)
Step 3: Connect points
```

```
This line passes through the origin.
```

Intercepts

Finding x and y Intercepts

Intercepts are points where the line crosses the axes. They provide key information about the graph and are useful for graphing.

```
Intercept Definitions


Y-intercept: Point where line crosses y-axis
- x-coordinate is always 0
- Form: (0, b)
- To find: substitute x = 0 into equation

X-intercept: Point where line crosses x-axis
- y-coordinate is always 0
- Form: (a, 0)
- To find: substitute y = 0 into equation

Example 1: Find intercepts of 2x + 3y = 12

Y-intercept (let x = 0):
2(0) + 3y = 12
3y = 12
y = 4
Y-intercept: (0, 4)

X-intercept (let y = 0):
2x + 3(0) = 12
2x = 12
x = 6
X-intercept: (6, 0)

Example 2: Find intercepts of y = -2x + 8

Y-intercept (let x = 0):
y = -2(0) + 8 = 8
Y-intercept: (0, 8)

X-intercept (let y = 0):
0 = -2x + 8
2x = 8
x = 4
```

```
X-intercept: (4, 0)

Example 3: Find intercepts of y = 3x

Y-intercept (let x = 0):
y = 3(0) = 0
Y-intercept: (0, 0)

X-intercept (let y = 0):
0 = 3x
x = 0
X-intercept: (0, 0)

This line passes through the origin - both intercepts are (0, 0)
```

Graphing Using Intercepts

```
Intercept Method for Graphing


Steps:
1. Find x-intercept (set y = 0)
2. Find y-intercept (set x = 0)
3. Plot both intercepts
4. Connect with straight line
5. Check with a third point

Example: Graph 3x - 2y = 6

Step 1: X-intercept (y = 0)
3x - 2(0) = 6
3x = 6
x = 2
X-intercept: (2, 0)

Step 2: Y-intercept (x = 0)
3(0) - 2y = 6
-2y = 6
y = -3
Y-intercept: (0, -3)

Step 3: Plot points (2, 0) and (0, -3)

Step 4: Connect with line

    y
    |
```

```
  0 |•   •
    |
 -3 |•
    |
    +     x
    0   2
```

```
Step 5: Check with third point (x = 4)
3(4) - 2y = 6
12 - 2y = 6
-2y = -6
y = 3
Point (4, 3) should be on the line

Advantages of Intercept Method:
- Only need to find two points
- Intercepts are often easy to calculate
- Gives good overview of graph
- Works well for standard form equations
```

Writing Linear Equations

Writing Equations from Graphs

```
Equation from Graph Process


Method 1: Using Slope-Intercept Form
Steps:
1. Identify y-intercept from graph
2. Calculate slope using two points
3. Write equation y = mx + b

Example: Graph shows line through (0, 3) and (2, 7)
Step 1: Y-intercept = 3 (b = 3)
Step 2: Slope = (7-3)/(2-0) = 4/2 = 2 (m = 2)
Step 3: Equation: y = 2x + 3

Method 2: Using Two Points
If y-intercept isn't clear, use any two points:

Example: Line through (1, 5) and (3, 11)
Step 1: Find slope
m = (11-5)/(3-1) = 6/2 = 3

Step 2: Use point-slope form with either point
```

```
Using (1, 5): y - 5 = 3(x - 1)
y - 5 = 3x - 3
y = 3x + 2

Step 3: Check with other point
Using (3, 11): y = 3(3) + 2 = 9 + 2 = 11

Method 3: Using Intercepts
If both intercepts are visible:

Example: X-intercept (4, 0), Y-intercept (0, -2)
Step 1: Slope = (-2-0)/(0-4) = -2/(-4) = 1/2
Step 2: Y-intercept gives b = -2
Step 3: Equation: y = (1/2)x - 2
```

Writing Equations from Word Problems

```
Real-World Linear Equations


Strategy:
1. Identify variables
2. Find initial value (y-intercept)
3. Find rate of change (slope)
4. Write equation in appropriate form

Example 1: Cell Phone Plan
"A cell phone plan costs $30 per month plus $0.10 per text message."

Variables: x = number of texts, y = total cost
Initial value: $30 (fixed monthly cost)
Rate of change: $0.10 per text
Equation: y = 0.10x + 30

Example 2: Water Tank
"A 500-gallon tank is being drained at 25 gallons per hour."

Variables: x = hours, y = gallons remaining
Initial value: 500 gallons
Rate of change: -25 gallons/hour (negative because decreasing)
Equation: y = -25x + 500

Example 3: Temperature Conversion
"Water freezes at 32°F, and temperature increases 1.8°F for each 1°C increase."

Variables: x = Celsius, y = Fahrenheit
Initial value: 32°F (when C = 0)
```

```
Rate of change: 1.8°F per °C
Equation: y = 1.8x + 32

Example 4: Rental Car
"Car rental costs $25 per day plus $0.15 per mile."

Variables: x = miles driven, y = total cost
But this depends on number of days too!
For 3 days: y = 0.15x + 25(3) = 0.15x + 75

Example 5: Savings Account
"You start with $200 and save $15 per week."

Variables: x = weeks, y = total savings
Initial value: $200
Rate of change: +$15 per week
Equation: y = 15x + 200
```

Parallel and Perpendicular Lines

Understanding Line Relationships

```
Parallel and Perpendicular Lines


Parallel Lines:
- Same slope, different y-intercepts
- Never intersect
- Always same distance apart

Example: y = 2x + 1 and y = 2x - 3
Both have slope = 2, so they're parallel

Perpendicular Lines:
- Slopes are negative reciprocals
- Intersect at right angles (90°)
- If one slope is m, the other is -1/m

Example: y = 3x + 1 and y = -1/3 x + 4
Slopes: 3 and -1/3
3 × (-1/3) = -1   (negative reciprocals)

Special Cases:
- Horizontal line (slope = 0)   Vertical line (undefined slope)
- y = 5   x = 2
```

```
Finding Parallel Line:
Given: y = 4x - 1, find parallel line through (2, 3)
Parallel slope = 4 (same slope)
Using point-slope form:
y - 3 = 4(x - 2)
y - 3 = 4x - 8
y = 4x - 5

Finding Perpendicular Line:
Given: y = -2x + 7, find perpendicular line through (4, 1)
Perpendicular slope = -1/(-2) = 1/2
Using point-slope form:
y - 1 = 1/2(x - 4)
y - 1 = 1/2 x - 2
y = 1/2 x - 1

Checking Perpendicularity:
(-2) × (1/2) = -1
```

Applications and Problem Solving

Real-World Linear Relationships

```
Linear Applications


Distance-Time Graphs:
"A car travels at constant 60 mph starting 100 miles from home."
Distance from home = 100 + 60t
where t = time in hours

At t = 0: distance = 100 miles
At t = 2: distance = 100 + 60(2) = 220 miles

Cost Analysis:
"Manufacturing costs $500 setup plus $12 per item."
Total cost = 500 + 12x
where x = number of items

Break-even analysis:
If selling price is $20 per item:
Revenue = 20x
Break-even when Revenue = Cost:
20x = 500 + 12x
8x = 500
x = 62.5 items
```

```
Temperature Relationships:
Celsius to Fahrenheit: F = 1.8C + 32
Fahrenheit to Celsius: C = (F - 32)/1.8 = 5/9(F - 32)

Depreciation:
"A car worth $25,000 depreciates $3,000 per year."
Value = 25,000 - 3,000t
where t = years

After 5 years: Value = 25,000 - 3,000(5) = $10,000

Supply and Demand:
Supply: p = 2q + 10 (price increases with quantity)
Demand: p = -q + 40 (price decreases with quantity)
Equilibrium when supply = demand:
2q + 10 = -q + 40
3q = 30
q = 10 units, p = $30
```

Common Mistakes and Solutions

Typical Graphing Errors

```
Common Graphing Mistakes


Mistake 1: Confusing x and y coordinates
Wrong: Plot (3, 2) as 3 up, 2 right
Right: Plot (3, 2) as 3 right, 2 up
Solution: Remember (x, y) = (horizontal, vertical)

Mistake 2: Incorrect slope calculation
Wrong: Slope from (1, 2) to (3, 6) is (3-1)/(6-2) = 2/4 = 1/2
Right: Slope = (6-2)/(3-1) = 4/2 = 2
Solution: Always use (y -y )/(x -x )

Mistake 3: Wrong y-intercept identification
Wrong: In y = 3x - 4, y-intercept is 3
Right: In y = 3x - 4, y-intercept is -4
Solution: Y-intercept is the constant term (b in y = mx + b)

Mistake 4: Parallel/perpendicular confusion
Wrong: Lines y = 2x + 1 and y = -2x + 3 are perpendicular
Right: These lines have slopes 2 and -2, not negative reciprocals
Solution: Perpendicular slopes multiply to -1
```

```
Mistake 5: Scale errors on graphs
Wrong: Using different scales on x and y axes without noting
Right: Keep consistent scales or clearly mark different scales
Solution: Always label axes and note scale

Prevention Strategies:
- Double-check coordinate order
- Verify slope calculations with two different point pairs
- Always identify slope and y-intercept separately
- Test perpendicular slopes by multiplying
- Use graph paper for accuracy
```

Conclusion

Graphing linear equations provides a powerful visual tool for understanding relationships between variables. The ability to move between algebraic and graphical representations is fundamental to mathematical literacy and problem-solving.

```
Linear Graphing: Complete Understanding


Conceptual Understanding:
  Coordinate plane as a system for locating points
  Linear equations as straight-line relationships
  Slope as rate of change between variables

Procedural Fluency:
  Plotting points and graphing lines accurately
  Finding slope, intercepts, and equations
  Converting between different equation forms

Strategic Competence:
  Choosing appropriate graphing methods
  Interpreting graphs in real-world contexts
  Writing equations from given information

Adaptive Reasoning:
  Understanding connections between algebraic and graphical representations
  Recognizing parallel and perpendicular relationships
  Making predictions using linear models

Productive Disposition:
  Confidence with coordinate graphing
  Appreciation for visual representation of data
  Persistence in multi-step graphing problems
```

From tracking business profits to analyzing scientific data, from GPS navigation to economic forecasting, linear relationships and their graphs provide essential tools for understanding and predicting patterns in our quantitative world. The skills developed in graphing linear equations form the foundation for all advanced mathematics and data analysis.

Algebra

Introduction to Algebra: The Language of Mathematical Relationships

What is Algebra?

Algebra is the branch of mathematics that uses symbols, variables, and equations to represent and solve problems involving unknown quantities and general mathematical relationships. It extends arithmetic by introducing variables that can represent any number, allowing us to work with general patterns and solve complex problems systematically.

Algebra is often called the "gateway to higher mathematics" because it provides the foundation for calculus, statistics, physics, engineering, and countless other fields. It transforms mathematics from a collection of computational techniques into a powerful language for describing and analyzing the world around us.

```
The Evolution of Mathematical Thinking


Arithmetic: 3 + 5 = 8 (specific calculation)
Pre-Algebra: x + 5 (general expression)
Algebra: ax² + bx + c = 0 (general equation with structure)

From specific → general → abstract → universal
```

The Historical Development of Algebra

Ancient Origins

Algebra has ancient roots, developing independently in several civilizations as mathematicians sought to solve increasingly complex problems.

```
Timeline of Algebraic Development


2000 BCE: Babylonians solve quadratic equations
          using geometric methods

300 CE:  Diophantus introduces algebraic symbolism
          in ancient Greece
```

```
820 CE:  Al-Khwarizmi writes "Al-Jabr"
         (The word "algebra" comes from "al-jabr")

1200 CE: Fibonacci brings Arabic numerals and
         algebraic methods to Europe

1500s:   Italian mathematicians solve cubic and
         quartic equations

1600s:   Descartes develops coordinate geometry,
         connecting algebra and geometry

1800s:   Abstract algebra emerges with group theory
         and field theory

1900s:   Modern algebra becomes foundation for
         computer science and advanced mathematics
```

The Word "Algebra"

The word "algebra" comes from the Arabic word "al-jabr," which appeared in the title of a book by the Persian mathematician Al-Khwarizmi around 820 CE. "Al-jabr" means "reunion of broken parts" or "restoration," referring to the process of moving terms from one side of an equation to the other.

```
Al-Khwarizmi's Contribution


Original Arabic: "Hisab al-jabr w'al-muqābala"
Translation: "The Calculation of Restoration and Completion"

Al-jabr (restoration): Moving negative terms to positive
Example: x² - 5x = 6 becomes x² = 5x + 6

Al-muqābala (completion): Combining like terms
Example: x² + 3x + 2x = 15 becomes x² + 5x = 15

These operations are still fundamental to algebra today!
```

Core Concepts of Algebra

Variables and Constants

In algebra, we distinguish between quantities that can change (variables) and those that remain fixed (constants).

```
Variables vs. Constants in Algebra


Variables:
- Represent unknown or changing quantities
- Usually letters: x, y, z, a, b, c, t, n
- Can take on different values
- The "unknowns" we solve for

Constants:
- Have fixed, unchanging values
- Numbers: 5, -3,  , √2
- Known quantities in problems
- Coefficients of variables

Examples:
In 3x + 7 = 19:
- x is the variable (unknown)
- 3, 7, and 19 are constants

In ax² + bx + c = 0:
- x is the variable
- a, b, c are constants (parameters)
- This represents a whole family of equations

In d = rt (distance = rate × time):
- d, r, t are all variables
- The relationship itself is constant
```

Expressions, Equations, and Functions

Algebra works with three main types of mathematical objects, each serving different purposes.

```
Mathematical Objects in Algebra


Expression: A mathematical phrase
- Contains variables, constants, operations
- Represents a value
- Can be simplified but not "solved"
- Examples: 3x + 7, x² - 4x + 1, (a + b)²

Equation: A mathematical statement
- Says two expressions are equal
- Contains an equals sign (=)
- Can be solved for variable values
- Examples: 3x + 7 = 19, x² - 4x + 1 = 0
```

```
Function: A rule that assigns outputs to inputs
- Describes relationships between variables
- Often written as f(x) = expression
- Examples: f(x) = 3x + 7, g(x) = x² - 4x + 1

Progression:
Expression → Equation → Function
   ↓           ↓          ↓
 Value    Solution   Relationship
```

The Fundamental Operations

Algebra extends the basic arithmetic operations to work with variables and more complex expressions.

```
Algebraic Operations


Addition and Subtraction:
- Combine like terms: 3x + 5x = 8x
- Cannot combine unlike terms: 3x + 5y stays as 3x + 5y
- Distribute over parentheses: a + (b + c) = a + b + c

Multiplication:
- Distribute over addition: a(b + c) = ab + ac
- Multiply variables: x · x = x²
- Multiply coefficients: 3x · 4y = 12xy

Division:
- Divide coefficients: 12x ÷ 4 = 3x
- Divide variables: x³ ÷ x = x²
- Cannot divide by zero: x ÷ 0 is undefined

Exponentiation:
- Repeated multiplication: x³ = x · x · x
- Rules: x^a · x^b = x^(a+b), (x^a)^b = x^(ab)
- Special cases: x  = 1, x¹ = x

Order of Operations (PEMDAS):
Still applies with variables:
2x + 3(x - 1)² = 2x + 3(x² - 2x + 1) = 2x + 3x² - 6x + 3 = 3x² - 4x + 3
```

Types of Algebraic Equations

Linear Equations

Linear equations involve variables raised only to the first power and graph as straight lines.

```
Linear Equations


One Variable:
ax + b = c
Example: 3x + 7 = 19
Solution: x = 4

Two Variables:
ax + by = c
Example: 2x + 3y = 12
Solution: Infinitely many (x,y) pairs
Graph: Straight line

Systems of Linear Equations:
{ax + by = c
{dx + ey = f
Example: {2x + y = 7
         {x - y = 2
Solution: x = 3, y = 1

Applications:
- Cost analysis
- Distance-rate-time problems
- Mixture problems
- Break-even analysis
```

Quadratic Equations

Quadratic equations involve variables raised to the second power and graph as parabolas.

```
Quadratic Equations


Standard Form: ax² + bx + c = 0 (a  0)

Examples:
x² - 5x + 6 = 0
2x² + 3x - 1 = 0
x² - 9 = 0
```

```
Solution Methods:
1. Factoring: x² - 5x + 6 = (x - 2)(x - 3) = 0
2. Quadratic Formula: x = (-b ± √(b² - 4ac))/(2a)
3. Completing the Square
4. Graphing

Number of Solutions:
- Two real solutions: b² - 4ac > 0
- One real solution: b² - 4ac = 0
- No real solutions: b² - 4ac < 0

Applications:
- Projectile motion
- Area optimization
- Profit maximization
- Physics problems
```

Polynomial Equations

Polynomial equations involve variables raised to various positive integer powers.

```
Polynomial Equations


General Form: a x  + a  x ¹ + ... + a x + a  = 0

Degree: Highest power of the variable
- Linear: degree 1
- Quadratic: degree 2
- Cubic: degree 3
- Quartic: degree 4
- Quintic: degree 5

Examples:
Cubic: x³ - 2x² + x - 3 = 0
Quartic: x  - 5x² + 4 = 0

Fundamental Theorem of Algebra:
A polynomial of degree n has exactly n solutions
(counting complex solutions and multiplicities)

Solution Strategies:
- Factor when possible
- Use rational root theorem
- Synthetic division
- Numerical methods for higher degrees
```

Algebraic Thinking and Problem Solving

From Arithmetic to Algebraic Reasoning

Algebra requires a shift from computational thinking to relational and structural thinking.

```
Arithmetic vs. Algebraic Thinking


Arithmetic Thinking:
"What is 3 + 5?"
Focus: Getting the answer (8)
Process: Direct computation

Algebraic Thinking:
"What patterns exist in addition?"
Focus: Understanding relationships
Process: Generalization and abstraction

Example Progression:
Level 1: 3 + 5 = 8, 4 + 6 = 10, 7 + 3 = 10
Level 2: "I notice the sum increases when both numbers increase"
Level 3: "For any numbers a and b, a + b = b + a"
Level 4: "This is the commutative property of addition"

Algebraic Habits of Mind:
1. Look for patterns and structure
2. Generalize from specific cases
3. Use symbols to represent relationships
4. Reason about operations and their properties
5. Make and test conjectures
6. Prove or disprove mathematical statements
```

Problem-Solving Strategies

Algebra provides systematic approaches to solving complex problems.

```
Algebraic Problem-Solving Process


1. UNDERSTAND the Problem
   - What is given?
   - What are we looking for?
   - What are the constraints?

2. REPRESENT Algebraically
```

```
   - Define variables
   - Write expressions and equations
   - Identify relationships

3. SOLVE the Mathematics
   - Use appropriate algebraic techniques
   - Check solutions in original equations
   - Verify reasonableness

4. INTERPRET and COMMUNICATE
   - Translate back to original context
   - Check if solution makes sense
   - Communicate clearly

Example: Age Problem
"Maria is 3 times as old as her daughter. In 12 years, Maria will be twice as old as her daught

UNDERSTAND:
- Maria's current age related to daughter's age
- Future relationship given
- Find both current ages

REPRESENT:
Let d = daughter's current age
Then 3d = Maria's current age
In 12 years: daughter will be d + 12, Maria will be 3d + 12
Equation: 3d + 12 = 2(d + 12)

SOLVE:
3d + 12 = 2d + 24
d = 12
Maria's age = 3(12) = 36

INTERPRET:
Daughter is 12, Maria is 36
Check: In 12 years, daughter will be 24, Maria will be 48
48 = 2(24)
```

The Power and Beauty of Algebra

Unifying Mathematical Concepts

Algebra serves as a unifying language that connects different areas of mathematics.

```
Algebra as Mathematical Bridge
```

```
Connects to Geometry:
- Coordinate geometry: y = mx + b
- Area formulas: A =  r²
- Distance formula: d = √[(x -x )² + (y -y )²]

Connects to Statistics:
- Linear regression: y = ax + b
- Standard deviation:   = √[Σ(x- )²/n]
- Probability distributions

Connects to Physics:
- Motion equations: s = ut + ½at²
- Energy relationships: E = mc²
- Wave equations: y = A sin( t +  )

Connects to Economics:
- Supply and demand curves
- Cost functions: C(x) = mx + b
- Optimization problems

Connects to Computer Science:
- Algorithms and complexity
- Cryptography and security
- Data structures and databases
```

Patterns and Generalizations

Algebra reveals the underlying patterns that govern mathematical relationships.

```
Algebraic Patterns


Number Patterns:
Arithmetic sequence: a, a+d, a+2d, a+3d, ...
General term: a  = a + (n-1)d

Geometric sequence: a, ar, ar², ar³, ...
General term: a  = ar^(n-1)

Algebraic Identities:
(a + b)² = a² + 2ab + b²
(a - b)² = a² - 2ab + b²
(a + b)(a - b) = a² - b²

These patterns appear everywhere:
- Binomial expansion
```

```
- Factoring techniques
- Trigonometric identities
- Calculus formulas

Fibonacci Pattern:
F  = 1, F  = 1, F  = F   + F
Algebraic formula: F  = (  -  )/√5
where   = (1+√5)/2,   = (1-√5)/2

Pascal's Triangle:
Each entry is sum of two entries above
Connects to binomial coefficients: (n choose k)
Algebraic significance in probability and combinatorics
```

Modern Applications of Algebra

Technology and Computing

Algebra forms the mathematical foundation of modern technology.

```
Algebra in Technology


Computer Graphics:
- 3D transformations: matrix algebra
- Animation curves: polynomial functions
- Color mixing: linear combinations

Cryptography:
- RSA encryption: modular arithmetic
- Error correction: polynomial codes
- Digital signatures: algebraic structures

Machine Learning:
- Linear regression: y = X  +
- Neural networks: matrix operations
- Optimization: gradient descent algorithms

Internet Search:
- PageRank algorithm: eigenvalue problems
- Data compression: algebraic coding
- Network analysis: graph theory

GPS Navigation:
- Satellite positioning: system of equations
- Route optimization: linear programming
```

```
- Signal processing: Fourier analysis
```

Science and Engineering

Algebra provides the language for describing natural phenomena and engineering solutions.

```
Algebra in Science


Physics:
- Newton's laws: F = ma
- Electromagnetic waves: E = hf
- Quantum mechanics: Schrödinger equation

Chemistry:
- Chemical equations: balanced reactions
- Gas laws: PV = nRT
- Reaction rates: differential equations

Biology:
- Population growth: exponential models
- Genetics: Hardy-Weinberg equilibrium
- Enzyme kinetics: Michaelis-Menten equation

Engineering:
- Structural analysis: systems of equations
- Control systems: transfer functions
- Signal processing: Fourier transforms

Environmental Science:
- Climate models: differential equations
- Pollution dispersion: algebraic models
- Resource management: optimization
```

Building Algebraic Fluency

Essential Skills for Success

Success in algebra requires developing both procedural fluency and conceptual understanding.

```
Algebraic Skill Development


Foundational Skills:
  Variable manipulation and substitution
```

```
  Combining like terms and distribution
  Solving linear equations systematically
  Graphing linear relationships
  Working with inequalities

Intermediate Skills:
  Factoring polynomials
  Solving quadratic equations
  Working with rational expressions
  Understanding function notation
  Solving systems of equations

Advanced Skills:
  Polynomial operations and division
  Exponential and logarithmic functions
  Radical expressions and equations
  Sequences and series
  Mathematical modeling

Study Strategies:
1. Practice regularly with varied problems
2. Connect algebraic and graphical representations
3. Focus on understanding, not just memorization
4. Use real-world applications to build meaning
5. Work collaboratively to discuss concepts
6. Seek help when concepts are unclear
```

Conclusion

Algebra represents one of humanity's greatest intellectual achievements - the development of a symbolic language that can describe, analyze, and predict patterns in the world around us. From its ancient origins in Babylonian problem-solving to its modern applications in artificial intelligence and quantum computing, algebra continues to be an essential tool for understanding and shaping our world.

```
Algebra: The Gateway to Mathematical Power


Historical Significance:
  4000 years of mathematical development
  Foundation for scientific revolution
  Bridge between arithmetic and higher mathematics

Conceptual Power:
  Generalizes arithmetic to abstract relationships
  Provides systematic problem-solving methods
```

```
  Reveals underlying mathematical structures

Practical Applications:
  Science and engineering foundations
  Technology and computing algorithms
  Business and economic modeling
  Everyday problem-solving tools

Educational Importance:
  Develops logical reasoning skills
  Builds abstract thinking abilities
  Prepares for advanced mathematics
  Enhances quantitative literacy
```

As you embark on your algebraic journey, remember that you're not just learning computational techniques - you're developing a new way of thinking about relationships, patterns, and problem-solving that will serve you throughout your academic and professional life. Algebra is truly the language of mathematical relationships, and mastering this language opens doors to understanding the mathematical structure underlying our universe.

Whether you're calculating the trajectory of a spacecraft, analyzing market trends, designing computer algorithms, or simply trying to understand the patterns in everyday life, algebra provides the essential tools for mathematical reasoning and quantitative analysis. The investment you make in understanding algebra will pay dividends throughout your mathematical education and beyond.

Linear Equations: The Foundation of Algebraic Problem Solving

What are Linear Equations?

A linear equation is an algebraic equation where each term is either a constant or the product of a constant and a single variable raised to the first power. Linear equations graph as straight lines, hence the name "linear."

```
Linear Equation Characteristics


Standard Forms:
One variable: ax + b = c
Two variables: ax + by = c
Slope-intercept: y = mx + b

Key Properties:
  Variables have degree 1 (first power only)
  No variables in denominators
  No variables under radicals
  No variables as exponents
  Graph as straight lines

Examples:
Linear: 3x + 7 = 19, 2x - 5y = 10, y = 4x - 3
Not Linear: x² + 3 = 7, 1/x = 5, √x = 4
```

Solving Linear Equations in One Variable

Basic Solution Process

The goal is to isolate the variable on one side of the equation using inverse operations.

```
Solution Strategy


1. Simplify both sides (distribute, combine like terms)
```

```
2. Move variable terms to one side
3. Move constant terms to the other side
4. Divide by the coefficient of the variable
5. Check your solution

Example: 3(x + 2) - 5 = 2x + 7

Step 1: Distribute and simplify
3x + 6 - 5 = 2x + 7
3x + 1 = 2x + 7

Step 2: Move variable terms
3x - 2x = 7 - 1
x = 6

Step 3: Check
3(6 + 2) - 5 = 3(8) - 5 = 24 - 5 = 19
2(6) + 7 = 12 + 7 = 19
```

Types of Linear Equation Solutions

```
Solution Types


One Solution (Most Common):
2x + 3 = 7
x = 2
The equation has exactly one value that makes it true

No Solution (Contradiction):
2x + 3 = 2x + 5
3 = 5 (impossible)
The equation is never true

Infinite Solutions (Identity):
2x + 3 = 2x + 3
0 = 0 (always true)
Every value of x makes the equation true
```

Applications of Linear Equations

Word Problems and Real-World Applications

Linear equations model many real-world situations involving constant rates of change.

Common Application Types

Age Problems:
"John is 5 years older than Mary. In 3 years, their combined age will be 35."
Let x = Mary's current age
Then x + 5 = John's current age
Equation: (x + 3) + (x + 5 + 3) = 35

Distance-Rate-Time Problems:
"A car travels 240 miles in 4 hours. What is its speed?"
d = rt → 240 = r(4) → r = 60 mph

Mixture Problems:
"How many pounds of $8/lb coffee should be mixed with 5 pounds of $12/lb coffee to get a mixtur
Let x = pounds of $8 coffee
8x + 12(5) = 10(x + 5)

Money Problems:
"Sarah has $3.50 in quarters and dimes. She has 5 more quarters than dimes. How many of each co
Let d = number of dimes
Then d + 5 = number of quarters
0.10d + 0.25(d + 5) = 3.50

Percent Problems

Percent Applications

Basic Percent Equation: Part = Percent × Whole
Or: P = r × W

Percent Increase/Decrease:
New Amount = Original ± (Percent × Original)
A = P ± rP = P(1 ± r)

Examples:
1. "What is 15% of 80?"
   P = 0.15 × 80 = 12

2. "25 is what percent of 200?"
   25 = r × 200 → r = 0.125 = 12.5%

3. "30% of what number is 18?"
   18 = 0.30 × W → W = 60

4. "A $200 item is marked up 25%. What's the new price?"

```
   New price = 200(1 + 0.25) = 200(1.25) = $250
```

Linear Equations in Two Variables

Graphing Linear Equations

Linear equations in two variables represent relationships between two quantities and graph as straight lines.

```
Graphing Methods


Method 1: Table of Values
For y = 2x + 1:
x | y = 2x + 1 | (x,y)
-2| 2(-2)+1=-3| (-2,-3)
-1| 2(-1)+1=-1| (-1,-1)
 0| 2(0)+1=1  | (0,1)
 1| 2(1)+1=3  | (1,3)
 2| 2(2)+1=5  | (2,5)

Method 2: Intercepts
x-intercept: Set y = 0, solve for x
y-intercept: Set x = 0, solve for y

For 2x + 3y = 12:
x-intercept: 2x + 3(0) = 12 → x = 6 → (6,0)
y-intercept: 2(0) + 3y = 12 → y = 4 → (0,4)

Method 3: Slope-Intercept Form
y = mx + b
m = slope, b = y-intercept
Start at (0,b), use slope to find next points
```

Slope and Rate of Change

```
Understanding Slope


Definition: Slope = rise/run = (y  - y )/(x  - x )

Slope Interpretation:
m > 0: Line rises from left to right
m < 0: Line falls from left to right
m = 0: Horizontal line
```

```
m undefined: Vertical line

Rate of Change:
Slope represents the rate at which y changes with respect to x

Examples:
1. Points (1,3) and (4,9):
   m = (9-3)/(4-1) = 6/3 = 2

2. y = -3x + 7:
   Slope = -3 (for every 1 unit right, go 3 units down)

3. Real-world: "Water drains from a tank at 5 gallons per minute"
   Slope = -5 (negative because water is decreasing)
```

Systems of Linear Equations

Solving Systems by Substitution

```
Substitution Method


Steps:
1. Solve one equation for one variable
2. Substitute into the other equation
3. Solve for the remaining variable
4. Back-substitute to find the first variable
5. Check the solution

Example:
{2x + y = 7
{x - y = 2

Step 1: From equation 2: x = y + 2
Step 2: Substitute into equation 1:
        2(y + 2) + y = 7
        2y + 4 + y = 7
        3y = 3
        y = 1
Step 3: Back-substitute: x = 1 + 2 = 3
Step 4: Check: 2(3) + 1 = 7  , 3 - 1 = 2
Solution: (3, 1)
```

Solving Systems by Elimination

```
Elimination Method
```

```
Steps:
1. Align equations vertically
2. Multiply equations to create opposite coefficients
3. Add equations to eliminate one variable
4. Solve for remaining variable
5. Back-substitute
6. Check solution

Example:
{3x + 2y = 16
{5x - 2y = 8

Step 1: Coefficients of y are already opposites
Step 2: Add equations:
        3x + 2y = 16
        5x - 2y = 8
        -----------
        8x = 24
        x = 3

Step 3: Substitute x = 3 into first equation:
        3(3) + 2y = 16
        9 + 2y = 16
        2y = 7
        y = 3.5

Solution: (3, 3.5)
```

Types of Systems

```
System Solution Types


One Solution (Consistent and Independent):
Lines intersect at exactly one point
Different slopes: m   m

No Solution (Inconsistent):
Lines are parallel (never intersect)
Same slope, different y-intercepts: m  = m , b   b

Infinite Solutions (Consistent and Dependent):
Lines are identical (same line)
Same slope and y-intercept: m  = m , b  = b
```

```
Graphical Interpretation:
One solution: Lines cross
No solution: Parallel lines
Infinite solutions: Same line
```

Linear Inequalities

Solving Linear Inequalities

Linear inequalities are solved similarly to equations, with one important difference: when multiplying or dividing by a negative number, the inequality sign flips.

```
Inequality Solution Rules


Same as equations:
  Add/subtract same number to both sides
  Multiply/divide by positive number

Different from equations:
  When multiplying/dividing by negative number,
   flip the inequality sign

Examples:
1. 3x + 5 > 14
   3x > 9
   x > 3

2. -2x + 7   15
   -2x   8
   x   -4  (sign flipped!)

3. -3(x - 2) < 9
   -3x + 6 < 9
   -3x < 3
   x > -1  (sign flipped!)
```

Graphing Linear Inequalities

```
Graphing on Number Line


x > 3: Open circle at 3, arrow right
x   3: Closed circle at 3, arrow right
x < 3: Open circle at 3, arrow left
```

```
x   3: Closed circle at 3, arrow left

Compound Inequalities:
-2 < x   5: Open at -2, closed at 5, between them
x < -1 or x > 3: Two separate regions

Graphing in Coordinate Plane:
y > 2x + 1: Dashed line, shade above
y   -x + 3: Solid line, shade below

Test Point Method:
1. Graph the boundary line
2. Choose test point not on line
3. Substitute into inequality
4. If true, shade that side; if false, shade other side
```

Problem-Solving Strategies

Setting Up Linear Equations

```
Problem-Solving Framework


1. READ and UNDERSTAND
   - What information is given?
   - What are we asked to find?
   - What are the relationships?

2. DEFINE VARIABLES
   - Choose meaningful variable names
   - State what each variable represents
   - Include units if applicable

3. WRITE EQUATIONS
   - Translate words to mathematical expressions
   - Use given relationships
   - Check that units are consistent

4. SOLVE
   - Use appropriate algebraic techniques
   - Show all steps clearly
   - Check solution in original equation

5. INTERPRET and VERIFY
   - Does the answer make sense?
   - Check against original problem
```

- Include appropriate units

Common Problem Types

Consecutive Integer Problems

Consecutive integers: n, n+1, n+2, ...
Consecutive even: n, n+2, n+4, ... (n even)
Consecutive odd: n, n+2, n+4, ... (n odd)

Example: "Find three consecutive integers whose sum is 48."
Let n = first integer
Then n+1 = second, n+2 = third
Equation: n + (n+1) + (n+2) = 48
Solution: 3n + 3 = 48 → n = 15
Answer: 15, 16, 17

Geometry Problems

Perimeter: P = sum of all sides
Area formulas: Rectangle A = lw, Triangle A = ½bh

Example: "A rectangle's length is 3 more than twice its width. If the perimeter is 36, find the
Let w = width
Then 2w + 3 = length
Equation: 2w + 2(2w + 3) = 36
Solution: 2w + 4w + 6 = 36 → w = 5
Answer: width = 5, length = 13

Linear Functions and Modeling

Function Notation and Linear Functions

Linear Function Properties

Function Notation: f(x) = mx + b
- f(x) represents the output (y-value)
- x represents the input
- m is the slope (rate of change)
- b is the y-intercept (initial value)

Domain and Range:
- Domain: all real numbers (unless restricted)

- Range: all real numbers (unless restricted)

Key Features:
  Constant rate of change (slope)
  Straight line graph
  One-to-one correspondence (passes vertical line test)

Examples:
f(x) = 2x + 3: slope = 2, y-intercept = 3
g(x) = -½x + 1: slope = -½, y-intercept = 1
h(x) = 5: slope = 0, horizontal line at y = 5

Linear Modeling

Real-World Linear Models

Cost Functions:
C(x) = mx + b
m = variable cost per unit
b = fixed cost

Example: "A company has fixed costs of $500 and variable costs of $25 per item."
C(x) = 25x + 500

Revenue Functions:
R(x) = px (where p = price per unit)

Profit Functions:
P(x) = R(x) - C(x)

Break-even Point:
Set R(x) = C(x) and solve for x

Temperature Conversion:
F = (9/5)C + 32
C = (5/9)(F - 32)

Population Growth (linear):
P(t) = P + rt
P = initial population
r = rate of change per time unit

Summary and Key Concepts

Linear equations form the foundation of algebraic problem-solving and provide essential tools for modeling real-world relationships. Mastering linear equations prepares you for more advanced algebraic concepts and applications.

Chapter Summary

Essential Skills Mastered:
Solving linear equations in one variable
Graphing linear equations and inequalities
Solving systems of linear equations
Applying linear equations to word problems
Understanding slope and rate of change
Working with linear functions and modeling

Key Formulas:
- Standard form: ax + by = c
- Slope-intercept form: y = mx + b
- Point-slope form: $y - y = m(x - x)$
- Slope formula: $m = (y - y)/(x - x)$
- Distance formula: $d = \sqrt{[(x - x)^2 + (y - y)^2]}$

Problem-Solving Tools:
- Substitution and elimination methods
- Graphical interpretation
- Function notation and modeling
- Real-world applications

Next Steps:
These linear equation skills provide the foundation for:
- Quadratic equations and functions
- Polynomial operations
- Exponential and logarithmic functions
- Advanced algebraic concepts

Linear equations represent the gateway to algebraic thinking, providing both computational tools and conceptual frameworks that extend throughout mathematics. The skills developed in this chapter - systematic problem-solving, graphical interpretation, and mathematical modeling - will serve as essential building blocks for all future algebraic learning.

Quadratic Equations: Exploring Parabolic Relationships

Introduction to Quadratic Equations

A quadratic equation is a polynomial equation of degree 2, meaning the highest power of the variable is 2. These equations model many real-world phenomena involving acceleration, optimization, and curved relationships.

```
Quadratic Equation Forms


Standard Form: ax² + bx + c = 0 (a   0)
- a, b, c are constants (coefficients)
- a   0 (otherwise it's linear, not quadratic)
- x is the variable

Examples:
x² - 5x + 6 = 0     (a=1, b=-5, c=6)
2x² + 3x - 1 = 0    (a=2, b=3, c=-1)
x² - 9 = 0          (a=1, b=0, c=-9)
3x² + 12x = 0       (a=3, b=12, c=0)

Other Forms:
Vertex Form: y = a(x - h)² + k
Factored Form: y = a(x - r )(x - r )
```

The Parabola: Graph of Quadratic Functions

Understanding Parabolic Shape

Quadratic functions graph as parabolas - U-shaped curves that open upward or downward.

```
Parabola Characteristics


Direction of Opening:
a > 0: Opens upward ( )
```

a < 0: Opens downward ( )

Key Features:
• Vertex: Highest or lowest point
• Axis of symmetry: Vertical line through vertex
• y-intercept: Point where graph crosses y-axis
• x-intercepts: Points where graph crosses x-axis (roots)

Vertex Location:
For $y = ax^2 + bx + c$:
x-coordinate of vertex: $x = -b/(2a)$
y-coordinate: substitute x-value back into equation

Example: $y = x^2 - 4x + 3$
Vertex x-coordinate: $x = -(-4)/(2 \cdot 1) = 2$
Vertex y-coordinate: $y = (2)^2 - 4(2) + 3 = -1$
Vertex: (2, -1)

Graphing Quadratic Functions

Graphing Process

Method 1: Using Vertex and Additional Points
1. Find vertex using $x = -b/(2a)$
2. Find y-intercept (set x = 0)
3. Find x-intercepts if they exist (set y = 0)
4. Plot additional points for accuracy
5. Draw smooth parabola

Method 2: Using Transformations
For $y = a(x - h)^2 + k$:
- Start with parent function $y = x^2$
- Horizontal shift: h units (right if +, left if -)
- Vertical shift: k units (up if +, down if -)
- Vertical stretch/compression: factor of |a|
- Reflection: if a < 0, flip over x-axis

Example: $y = -2(x + 1)^2 + 3$
- Start with $y = x^2$
- Shift left 1 unit: $y = (x + 1)^2$
- Stretch by factor 2: $y = 2(x + 1)^2$
- Reflect over x-axis: $y = -2(x + 1)^2$
- Shift up 3 units: $y = -2(x + 1)^2 + 3$

Solving Quadratic Equations

Method 1: Factoring

When a quadratic can be written as a product of linear factors, we can use the zero product property.

```
Factoring Strategies


Zero Product Property:
If ab = 0, then a = 0 or b = 0

Common Factoring Patterns:
1. Greatest Common Factor (GCF)
   3x² + 6x = 3x(x + 2) = 0
   Solutions: x = 0 or x = -2

2. Difference of Squares
   x² - 9 = (x + 3)(x - 3) = 0
   Solutions: x = -3 or x = 3

3. Perfect Square Trinomials
   x² + 6x + 9 = (x + 3)² = 0
   Solution: x = -3 (double root)

4. General Trinomials (ax² + bx + c)
   x² - 5x + 6 = (x - 2)(x - 3) = 0
   Solutions: x = 2 or x = 3

Factoring Process for x² + bx + c:
Find two numbers that:
- Multiply to give c
- Add to give b

Example: x² - 7x + 12
Need two numbers that multiply to 12 and add to -7
-3 and -4: (-3)(-4) = 12, (-3) + (-4) = -7
So: x² - 7x + 12 = (x - 3)(x - 4) = 0
Solutions: x = 3 or x = 4
```

Method 2: Quadratic Formula

The quadratic formula provides a systematic way to solve any quadratic equation.

```
The Quadratic Formula
```

```
For ax² + bx + c = 0:

x = (-b ± √(b² - 4ac))/(2a)

Components:
- Discriminant: Δ = b² - 4ac
- ± means two solutions (usually)

Discriminant Analysis:
Δ > 0: Two distinct real solutions
Δ = 0: One repeated real solution
Δ < 0: No real solutions (two complex solutions)

Example: 2x² + 3x - 1 = 0
a = 2, b = 3, c = -1
Δ = 3² - 4(2)(-1) = 9 + 8 = 17 > 0 (two real solutions)

x = (-3 ± √17)/(2·2) = (-3 ± √17)/4

Solutions: x = (-3 + √17)/4  0.28 or x = (-3 - √17)/4  -1.28
```

Method 3: Completing the Square

This method transforms a quadratic into perfect square form, useful for finding vertex form and solving equations.

```
Completing the Square Process


For ax² + bx + c = 0:

1. If a  1, divide entire equation by a
2. Move constant to right side
3. Add (b/2)² to both sides
4. Factor left side as perfect square
5. Solve by taking square root

Example: x² + 6x + 5 = 0

Step 1: x² + 6x = -5
Step 2: Add (6/2)² = 9 to both sides
        x² + 6x + 9 = -5 + 9
        x² + 6x + 9 = 4
Step 3: Factor: (x + 3)² = 4
Step 4: Take square root: x + 3 = ±2
```

```
Step 5: Solve: x = -3 ± 2
        x = -1 or x = -5

Converting to Vertex Form:
x² + 6x + 5 = (x + 3)² - 9 + 5 = (x + 3)² - 4
Vertex: (-3, -4)
```

Method 4: Graphing

Graphical solutions involve finding where the parabola crosses the x-axis.

```
Graphical Solution Method


Steps:
1. Graph the quadratic function y = ax² + bx + c
2. Find x-intercepts (where y = 0)
3. These x-values are the solutions

Advantages:
  Visual understanding of solutions
  Shows relationship between equation and graph
  Useful for approximate solutions

Limitations:
  May not give exact values
  Requires accurate graphing
  Some solutions may not be visible on standard viewing window

Technology Tools:
- Graphing calculators
- Computer algebra systems
- Online graphing tools
- Spreadsheet programs
```

Applications of Quadratic Equations

Projectile Motion

Quadratic equations naturally model objects moving under the influence of gravity.

```
Projectile Motion Models


Height Equation: h(t) = -16t² + v t + h
```

- h(t) = height at time t
- -16 = acceleration due to gravity (ft/s²)
- v = initial velocity
- h = initial height

Key Questions:
1. When does object hit ground? (h = 0)
2. What is maximum height? (vertex)
3. When is object at specific height?

Example: Ball thrown upward
"A ball is thrown upward from a 6-foot platform with initial velocity 32 ft/s."

Equation: $h(t) = -16t^2 + 32t + 6$

Maximum height:
$t = -32/(2(-16)) = 1$ second
$h(1) = -16(1)^2 + 32(1) + 6 = 22$ feet

Time to hit ground:
$-16t^2 + 32t + 6 = 0$
Using quadratic formula: t 2.18 seconds

Area and Optimization Problems

Optimization Applications

Maximum/Minimum Problems:
Since parabolas have vertex as extreme point, quadratics are perfect for optimization.

Example: Rectangular Enclosure
"A farmer has 100 feet of fencing to enclose a rectangular area against a barn. What dimensions

Setup:
Let x = width of rectangle
Then 100 - 2x = length (since one side is the barn)
Area: $A(x) = x(100 - 2x) = 100x - 2x^2$

This is quadratic with a = -2 < 0 (maximum exists)
Maximum at $x = -100/(2(-2)) = 25$ feet
Maximum area: $A(25) = 25(50) = 1250$ square feet

Business Applications:
Revenue: R(x) = px (price × quantity)
If price affects demand: p = a - bx
Then: $R(x) = x(a - bx) = ax - bx^2$

```
Maximum revenue occurs at vertex
```

Geometric Applications

```
Geometric Problem Types


Pythagorean Theorem Applications:
a² + b² = c² often leads to quadratic equations

Example: "A ladder leans against a wall. The ladder is 13 feet long, and its base is 5 feet fro

Setup: 5² + h² = 13²
       25 + h² = 169
       h² = 144
       h = 12 feet

Area Problems:
Often involve quadratic relationships

Example: "A square has the same area as a rectangle with length 12 and width 3. What is the sid

Setup: s² = 12 × 3 = 36
       s = 6

Number Problems:
"Find two consecutive integers whose product is 132."
Let n = first integer, n+1 = second
n(n+1) = 132
n² + n - 132 = 0
(n + 12)(n - 11) = 0
n = 11 or n = -12
Positive solution: 11 and 12
```

Complex Solutions and the Discriminant

Understanding the Discriminant

The discriminant $\Delta = b^2 - 4ac$ provides crucial information about quadratic solutions.

```
Discriminant Analysis


Δ > 0: Two distinct real solutions
- Parabola crosses x-axis at two points
```

```
- Factoring possible (usually)
- Example: x² - 5x + 6 = 0, Δ = 25 - 24 = 1

Δ = 0: One repeated real solution
- Parabola touches x-axis at vertex
- Perfect square trinomial
- Example: x² - 6x + 9 = 0, Δ = 36 - 36 = 0

Δ < 0: No real solutions (two complex solutions)
- Parabola doesn't cross x-axis
- Solutions involve imaginary numbers
- Example: x² + x + 1 = 0, Δ = 1 - 4 = -3

Applications:
- Determine solution type before solving
- Analyze when problems have real solutions
- Understand graphical behavior
```

Introduction to Complex Numbers

When the discriminant is negative, solutions involve the imaginary unit $i = \sqrt{(-1)}$.

```
Complex Number Basics


Imaginary Unit: i = √(-1)
Properties: i² = -1, i³ = -i, i⁴ = 1

Complex Number Form: a + bi
- a = real part
- b = imaginary part

Example: x² + 2x + 5 = 0
Δ = 4 - 20 = -16 < 0

x = (-2 ± √(-16))/2 = (-2 ± 4i)/2 = -1 ± 2i

Solutions: x = -1 + 2i and x = -1 - 2i
(These are complex conjugates)

Verification:
(-1 + 2i)² + 2(-1 + 2i) + 5
= 1 - 4i - 4 - 2 + 4i + 5 = 0
```

Systems of Quadratic Equations

Linear-Quadratic Systems

Systems involving one linear and one quadratic equation.

```
Solution Methods


Substitution Method:
1. Solve linear equation for one variable
2. Substitute into quadratic equation
3. Solve resulting quadratic
4. Back-substitute to find other variable

Example:
{y = x + 1
{x² + y² = 25

Substitute: x² + (x + 1)² = 25
           x² + x² + 2x + 1 = 25
           2x² + 2x - 24 = 0
           x² + x - 12 = 0
           (x + 4)(x - 3) = 0
           x = -4 or x = 3

When x = -4: y = -4 + 1 = -3
When x = 3: y = 3 + 1 = 4

Solutions: (-4, -3) and (3, 4)

Graphical Interpretation:
Solutions are intersection points of line and parabola
Can have 0, 1, or 2 solutions
```

Quadratic Inequalities

Solving Quadratic Inequalities

Quadratic inequalities involve finding where a quadratic expression is positive or negative.

```
Solution Process


1. Write in standard form: ax² + bx + c > 0 (or <,  ,  )
```

```
2. Find zeros of corresponding equation ax² + bx + c = 0
3. Plot zeros on number line
4. Test sign in each interval
5. Choose intervals that satisfy inequality

Example: x² - 5x + 6 > 0

Step 1: Factor: (x - 2)(x - 3) > 0
Step 2: Zeros: x = 2 and x = 3
Step 3: Number line intervals: (-ϖ, 2), (2, 3), (3, ϖ)
Step 4: Test points:
        x = 0: (0-2)(0-3) = 6 > 0
        x = 2.5: (2.5-2)(2.5-3) = -0.25 < 0
        x = 4: (4-2)(4-3) = 2 > 0
Step 5: Solution: x < 2 or x > 3
        Interval notation: (-ϖ, 2)  (3, ϖ)

Sign Chart Method:
     - - - - - + + + + + + + + + +
     -------|-------|-------
           2       3
```

Advanced Topics and Extensions

Quadratic Functions in Vertex Form

```
Vertex Form Applications


Form: f(x) = a(x - h)² + k
- (h, k) is the vertex
- a determines opening direction and width
- Useful for transformations and optimization

Converting Standard to Vertex Form:
Complete the square process

Example: f(x) = 2x² - 8x + 3
Factor out coefficient of x²: f(x) = 2(x² - 4x) + 3
Complete square inside: x² - 4x + 4 = (x - 2)²
Adjust: f(x) = 2(x² - 4x + 4 - 4) + 3
       = 2((x - 2)² - 4) + 3
       = 2(x - 2)² - 8 + 3
       = 2(x - 2)² - 5

Vertex: (2, -5)
```

Quadratic Modeling

Real-World Modeling

Revenue and Profit Models:
Often quadratic due to price-demand relationships

Population Models:
$P(t) = at^2 + bt + c$ (when growth rate changes)

Physics Applications:
- Projectile motion: $h(t) = -\frac{1}{2}gt^2 + v\,t + h$
- Kinetic energy: $KE = \frac{1}{2}mv^2$
- Electrical power: $P = I^2R$

Engineering Applications:
- Beam deflection
- Signal processing
- Optimization problems

Data Analysis:
Quadratic regression for curved data patterns

Summary and Key Concepts

Quadratic equations extend algebraic problem-solving to curved relationships and optimization problems, providing essential tools for modeling real-world phenomena.

Chapter Summary

Essential Skills Mastered:
- Solving quadratic equations by multiple methods
- Graphing parabolas and identifying key features
- Applying quadratics to real-world problems
- Understanding the discriminant and solution types
- Working with quadratic inequalities
- Converting between different forms

Key Formulas:
- Standard form: $ax^2 + bx + c = 0$
- Quadratic formula: $x = (-b \pm \sqrt{(b^2 - 4ac)})/(2a)$
- Vertex formula: $x = -b/(2a)$
- Discriminant: $\Delta = b^2 - 4ac$
- Vertex form: $y = a(x - h)^2 + k$

Solution Methods:
• Factoring (when possible)
• Quadratic formula (always works)
• Completing the square (useful for vertex form)
• Graphing (visual understanding)

Applications Covered:
• Projectile motion and physics
• Area and optimization problems
• Business and economics models
• Geometric relationships

Next Steps:
Quadratic concepts prepare you for:
- Higher-degree polynomials
- Exponential and logarithmic functions
- Conic sections
- Calculus applications

Quadratic equations represent a significant step forward in algebraic sophistication, introducing curved relationships, optimization concepts, and complex number systems. The problem-solving strategies and mathematical reasoning developed through quadratic equations form essential foundations for advanced mathematics and real-world applications across science, engineering, and business.

Polynomials: Building Blocks of Advanced Algebra

Introduction to Polynomials

A polynomial is an expression consisting of variables and coefficients, involving operations of addition, subtraction, multiplication, and non-negative integer exponents. Polynomials form the foundation for much of advanced algebra and calculus.

Polynomial Structure

General Form: $a_n x^n + a_{n-1} x^{n-1} + ... + a_1 x + a_0$

Components:
- Terms: Individual parts separated by + or -
- Coefficients: Numbers multiplying the variables (a_n, a_{n-1}, ...)
- Variables: Letters representing unknown values (usually x)
- Exponents: Powers to which variables are raised
- Constant term: Term without a variable (a_0)

Examples:
$3x^4 - 2x^3 + 5x - 7$ (4th degree polynomial)
$x^2 + 1$ (2nd degree polynomial)
$5x - 3$ (1st degree polynomial)
42 (0th degree polynomial - constant)

Non-polynomials:
$\sqrt{x} + 3$ (fractional exponent)
$1/x + 2$ (negative exponent)
x^x (variable exponent)

Polynomial Classification

Degree and Leading Terms

Polynomial Classification

By Degree (highest exponent):
Degree 0: Constant f(x) = 5
Degree 1: Linear f(x) = 3x + 2
Degree 2: Quadratic f(x) = x^2 - 4x + 1
Degree 3: Cubic f(x) = $2x^3$ + x^2 - 5
Degree 4: Quartic f(x) = x - $3x^2$ + 2
Degree 5: Quintic f(x) = x + $2x^3$ - x
Degree n: nth degree f(x) = a x + ... + a

By Number of Terms:
Monomial: 1 term $5x^3$
Binomial: 2 terms $3x^2$ - 7
Trinomial: 3 terms x^2 + 2x - 1
Polynomial: 4+ terms x + $3x^3$ - 2x + 5

Leading Term: Term with highest degree
Leading Coefficient: Coefficient of leading term

Example: $4x^3$ - $2x^2$ + 7x - 1
- Degree: 3 (cubic)
- Leading term: $4x^3$
- Leading coefficient: 4
- Constant term: -1

Standard Form and Terminology

Standard Form Requirements

Arranged in descending order of exponents:
a x + a x ^{1} + ... + a x + a

Example Conversions:
Not standard: 3 + 2x - x^2 + $5x^3$
Standard: $5x^3$ - x^2 + 2x + 3

Key Terms:
- Degree: Highest exponent when in standard form
- Leading coefficient: Coefficient of highest degree term
- End behavior: How function behaves as x → ±∞
- Zeros/roots: Values where polynomial equals zero
- Turning points: Local maxima and minima

Polynomial Equality:
Two polynomials are equal if and only if all corresponding coefficients are equal.

If ax^2 + bx + c = dx^2 + ex + f, then:

```
a = d, b = e, c = f
```

Polynomial Operations

Addition and Subtraction

Polynomial addition and subtraction involve combining like terms.

```
Addition and Subtraction Rules


Like Terms: Terms with same variable and same exponent
- 3x² and -5x² are like terms
- 2x³ and 4x² are not like terms

Addition Process:
1. Arrange polynomials in standard form
2. Group like terms
3. Add coefficients of like terms
4. Write result in standard form

Example:
(3x³ - 2x² + 5x - 1) + (x³ + 4x² - 3x + 7)

Group like terms:
= (3x³ + x³) + (-2x² + 4x²) + (5x - 3x) + (-1 + 7)
= 4x³ + 2x² + 2x + 6

Subtraction Process:
Distribute negative sign and add

Example:
(2x³ + 3x - 4) - (x³ - 2x² + x - 1)
= 2x³ + 3x - 4 - x³ + 2x² - x + 1
= (2x³ - x³) + 2x² + (3x - x) + (-4 + 1)
= x³ + 2x² + 2x - 3
```

Multiplication

Polynomial multiplication uses the distributive property extensively.

```
Multiplication Strategies


Monomial × Polynomial:
```

Distribute the monomial to each term

Example: $3x^2(2x^3 - 4x + 1)$
$= 3x^2 \cdot 2x^3 - 3x^2 \cdot 4x + 3x^2 \cdot 1$
$= 6x - 12x^3 + 3x^2$

Binomial × Binomial (FOIL):
$(a + b)(c + d) = ac + ad + bc + bd$

Example: $(2x + 3)(x - 4)$
First: $2x \cdot x = 2x^2$
Outer: $2x \cdot (-4) = -8x$
Inner: $3 \cdot x = 3x$
Last: $3 \cdot (-4) = -12$
Result: $2x^2 - 8x + 3x - 12 = 2x^2 - 5x - 12$

General Polynomial Multiplication:
Use distributive property repeatedly

Example: $(x^2 + 2x - 1)(x + 3)$
$= x^2(x + 3) + 2x(x + 3) - 1(x + 3)$
$= x^3 + 3x^2 + 2x^2 + 6x - x - 3$
$= x^3 + 5x^2 + 5x - 3$

Special Products:
$(a + b)^2 = a^2 + 2ab + b^2$
$(a - b)^2 = a^2 - 2ab + b^2$
$(a + b)(a - b) = a^2 - b^2$

Division

Polynomial division can be performed using long division or synthetic division.

Polynomial Long Division

Similar to numerical long division

Example: $(2x^3 + 3x^2 - 5x + 1) \div (x + 2)$

Step-by-step process:
1. Divide leading terms: $2x^3 \div x = 2x^2$
2. Multiply: $2x^2(x + 2) = 2x^3 + 4x^2$
3. Subtract: $(2x^3 + 3x^2 - 5x + 1) - (2x^3 + 4x^2) = -x^2 - 5x + 1$
4. Repeat with new dividend

Complete division:

```
        2x² - x - 3
       ----------------
x + 2 | 2x³ + 3x² - 5x + 1
        2x³ + 4x²
        -----------
             -x² - 5x
             -x² - 2x
             --------
                  -3x + 1
                  -3x - 6
                  -------
                       7
```

Result: $2x^2 - x - 3 + 7/(x + 2)$

Verification:
$(x + 2)(2x^2 - x - 3) + 7 = 2x^3 + 3x^2 - 5x + 1$

Synthetic Division

A shortcut method for dividing by linear factors of the form (x - c).

Synthetic Division Process

For dividing by (x - c):
1. Write coefficients of dividend in order
2. Use c (not -c) as divisor
3. Bring down first coefficient
4. Multiply and add repeatedly

Example: $(2x^3 - 5x^2 + 3x - 1) \div (x - 2)$

Setup: c = 2, coefficients: [2, -5, 3, -1]

```
    2 |  2  -5   3  -1
      |     4  -2   2
      |________________
        2  -1   1   1
```

Process:
- Bring down 2
- 2 × 2 = 4, add to -5: -5 + 4 = -1
- 2 × (-1) = -2, add to 3: 3 + (-2) = 1
- 2 × 1 = 2, add to -1: -1 + 2 = 1

Result: $2x^2 - x + 1 + 1/(x - 2)$

Remainder Theorem:
When P(x) is divided by (x - c), remainder = P(c)
Check: P(2) = 2(8) - 5(4) + 3(2) - 1 = 16 - 20 + 6 - 1 = 1

Factoring Polynomials

Common Factoring Techniques

Factoring Strategy Hierarchy

1. Greatest Common Factor (GCF)
Always check first!

Example: $6x^3 + 9x^2 - 12x = 3x(2x^2 + 3x - 4)$

2. Grouping
For 4-term polynomials

Example: $x^3 + 2x^2 + 3x + 6$
$= x^2(x + 2) + 3(x + 2)$
$= (x^2 + 3)(x + 2)$

3. Special Patterns
- Difference of squares: $a^2 - b^2 = (a + b)(a - b)$
- Perfect square trinomials: $a^2 \pm 2ab + b^2 = (a \pm b)^2$
- Sum/difference of cubes: $a^3 \pm b^3 = (a \pm b)(a^2 \mp ab + b^2)$

4. Trinomial Factoring
For $ax^2 + bx + c$

5. Advanced Techniques
- Substitution
- Rational root theorem
- Factor theorem

Factoring Quadratic Trinomials

Trinomial Factoring Methods

Case 1: $x^2 + bx + c$
Find two numbers that multiply to c and add to b

Example: $x^2 + 7x + 12$

Need: product = 12, sum = 7
Numbers: 3 and 4 (3 × 4 = 12, 3 + 4 = 7)
Factor: (x + 3)(x + 4)

Case 2: $ax^2 + bx + c$ (a 1)
Method 1 - Trial and Error:
Try different factor combinations

Method 2 - AC Method:
1. Multiply a and c
2. Find factors of ac that add to b
3. Rewrite middle term
4. Factor by grouping

Example: $6x^2 + 7x - 3$
ac = 6(-3) = -18
Need factors of -18 that add to 7: 9 and -2
$6x^2 + 9x - 2x - 3$
$= 3x(2x + 3) - 1(2x + 3)$
$= (3x - 1)(2x + 3)$

Special Factoring Patterns

Important Factoring Formulas

Difference of Squares:
$a^2 - b^2 = (a + b)(a - b)$

Examples:
$x^2 - 9 = (x + 3)(x - 3)$
$4x^2 - 25 = (2x + 5)(2x - 5)$
$x - 16 = (x^2)^2 - 4^2 = (x^2 + 4)(x^2 - 4) = (x^2 + 4)(x + 2)(x - 2)$

Perfect Square Trinomials:
$a^2 + 2ab + b^2 = (a + b)^2$
$a^2 - 2ab + b^2 = (a - b)^2$

Examples:
$x^2 + 6x + 9 = (x + 3)^2$
$4x^2 - 12x + 9 = (2x - 3)^2$

Sum and Difference of Cubes:
$a^3 + b^3 = (a + b)(a^2 - ab + b^2)$
$a^3 - b^3 = (a - b)(a^2 + ab + b^2)$

Examples:

$x^3 + 8 = x^3 + 2^3 = (x + 2)(x^2 - 2x + 4)$
$27x^3 - 1 = (3x)^3 - 1^3 = (3x - 1)(9x^2 + 3x + 1)$

Memory Aid: "SOAP"
Same sign, Opposite sign, Always Positive

Polynomial Functions and Graphs

Behavior of Polynomial Functions

End Behavior Analysis

End behavior depends on:
1. Degree (even or odd)
2. Leading coefficient (positive or negative)

Rules:
Even degree, positive leading coefficient:
Even degree, negative leading coefficient:
Odd degree, positive leading coefficient:
Odd degree, negative leading coefficient:

Examples:
$f(x) = x^4 - 2x^2 + 1$ (even degree, positive leading coefficient)
As $x \to -\infty$, $f(x) \to +\infty$
As $x \to +\infty$, $f(x) \to +\infty$

$g(x) = -2x^3 + 3x - 1$ (odd degree, negative leading coefficient)
As $x \to -\infty$, $g(x) \to +\infty$
As $x \to +\infty$, $g(x) \to -\infty$

Intermediate Value Theorem:
If f is continuous on [a,b] and k is between f(a) and f(b),
then there exists c in [a,b] such that f(c) = k.

Application: Guarantees existence of zeros between sign changes.

Finding Zeros and Factors

Relationship Between Zeros and Factors

Factor Theorem:
(x - c) is a factor of P(x) if and only if P(c) = 0

Fundamental Theorem of Algebra:
A polynomial of degree n has exactly n zeros
(counting multiplicities and complex zeros)

Rational Root Theorem:
If p/q is a rational zero of polynomial with integer coefficients,
then p divides the constant term and q divides the leading coefficient.

Example: $P(x) = 2x^3 - 3x^2 - 11x + 6$
Possible rational zeros: ±1, ±2, ±3, ±6, ±1/2, ±3/2

Testing: $P(1) = 2 - 3 - 11 + 6 = -6 \neq 0$
$P(2) = 16 - 12 - 22 + 6 = -12 \neq 0$
$P(3) = 54 - 27 - 33 + 6 = 0$

So $(x - 3)$ is a factor.
Using synthetic division: $P(x) = (x - 3)(2x^2 + 3x - 2)$
Factor further: $2x^2 + 3x - 2 = (2x - 1)(x + 2)$
Complete factorization: $P(x) = (x - 3)(2x - 1)(x + 2)$
Zeros: $x = 3$, $x = 1/2$, $x = -2$

Multiplicity and Graph Behavior

Zero Multiplicity Effects

Multiplicity: Number of times a factor appears

Behavior at zeros:
- Odd multiplicity: Graph crosses x-axis
- Even multiplicity: Graph touches x-axis (turns around)

Examples:
$f(x) = (x - 2)^2(x + 1)^3(x - 4)$

Zero $x = 2$: multiplicity 2 (even) → touches x-axis
Zero $x = -1$: multiplicity 3 (odd) → crosses x-axis
Zero $x = 4$: multiplicity 1 (odd) → crosses x-axis

Higher multiplicities create "flatter" behavior near the zero.

Turning Points:
A polynomial of degree n has at most n-1 turning points.
Actual number depends on the specific polynomial.

Example: $f(x) = x^4 - 4x^2 + 3$
Degree 4 → at most 3 turning points

This function actually has 3 turning points.

Applications of Polynomials

Modeling Real-World Phenomena

Polynomial Models

Volume and Area Problems:
Often lead to cubic polynomials

Example: Box Construction
"A box is made by cutting squares of side x from corners of a 20×30 sheet and folding up the si

Volume: $V(x) = x(20-2x)(30-2x)$
$= x(600 - 40x - 60x + 4x^2)$
$= x(600 - 100x + 4x^2)$
$= 4x^3 - 100x^2 + 600x$

Domain: $0 < x < 10$ (physical constraints)
Maximum volume found using calculus or graphing.

Population Models:
$P(t) = at^3 + bt^2 + ct + d$

Economic Models:
Cost, revenue, and profit functions often polynomial

Physics Applications:
- Position functions: $s(t) = at^3 + bt^2 + ct + d$
- Potential energy functions
- Wave equations

Polynomial Regression

Fitting Polynomial Models to Data

When linear models don't fit data well, try higher-degree polynomials.

Process:
1. Plot data points
2. Determine appropriate degree
3. Use technology to find coefficients
4. Evaluate model quality (R^2 value)

5. Make predictions within reasonable range

Example Data:
Year: 2000, 2005, 2010, 2015, 2020
Population: 100, 150, 180, 190, 185

Linear model might not capture the leveling off.
Quadratic model: $P(t) = at^2 + bt + c$ might fit better.

Caution: Higher-degree polynomials can overfit data.
Balance between fit quality and model simplicity.

Extrapolation Warning:
Polynomial models can behave wildly outside data range.
Use caution when predicting beyond known data.

Advanced Polynomial Topics

Polynomial Inequalities

Solving Polynomial Inequalities

Process:
1. Write in standard form: $P(x) > 0$ (or $<$, $\geq$, $\leq$)
2. Find zeros of $P(x)$
3. Create sign chart using zeros
4. Test signs in each interval
5. Select intervals satisfying inequality

Example: $x^3 - 4x > 0$
Factor: $x(x^2 - 4) = x(x + 2)(x - 2) > 0$
Zeros: $x = -2, 0, 2$

Sign chart:

Interval:	$(-\infty,-2)$	$(-2,0)$	$(0,2)$	$(2,\infty)$
x:	−	−	+	+
(x+2):	−	+	+	+
(x-2):	−	−	−	+
Product:	−	+	−	+

Solution: $x \in (-2, 0) \cup (2, \infty)$

Complex Zeros and Conjugate Pairs

Complex Zeros Theorem

If a polynomial with real coefficients has a complex zero a + bi,
then its complex conjugate a - bi is also a zero.

Example: $P(x) = x^3 - 2x^2 + 4x - 8$
If 2i is a zero, then -2i is also a zero.

Finding the third zero:
Since degree is 3, there are 3 zeros total.
If two are 2i and -2i, the third must be real.

Using the fact that $(x - 2i)(x + 2i) = x^2 + 4$:
$P(x) = (x^2 + 4)(x - c)$ for some real c

Expanding: $P(x) = x^3 - cx^2 + 4x - 4c$
Comparing with $P(x) = x^3 - 2x^2 + 4x - 8$:
-c = -2, so c = 2
-4c = -8, so c = 2

Therefore: $P(x) = (x^2 + 4)(x - 2)$
Zeros: 2i, -2i, 2

Summary and Key Concepts

Polynomials provide the algebraic foundation for modeling complex relationships and form the basis for calculus and advanced mathematics.

Chapter Summary

Essential Skills Mastered:
- Classifying polynomials by degree and terms
- Performing polynomial operations (add, subtract, multiply, divide)
- Factoring polynomials using various techniques
- Finding zeros and understanding their multiplicities
- Analyzing polynomial function behavior and graphs
- Solving polynomial equations and inequalities
- Applying polynomials to real-world problems

Key Concepts:
- Standard form and polynomial terminology
- End behavior analysis using degree and leading coefficient
- Relationship between zeros, factors, and graphs
- Rational Root Theorem and Factor Theorem
- Multiplicity effects on graph behavior

• Complex zeros and conjugate pairs

Factoring Techniques:
• Greatest Common Factor (GCF)
• Grouping method
• Special patterns (difference of squares, perfect squares, sum/difference of cubes)
• Trinomial factoring methods
• Advanced techniques for higher degrees

Applications:
• Volume and area optimization
• Population and economic modeling
• Physics and engineering problems
• Data analysis and polynomial regression

Next Steps:
Polynomial concepts prepare you for:
- Rational functions and their properties
- Exponential and logarithmic functions
- Calculus (limits, derivatives, integrals)
- Advanced algebra and mathematical analysis

Polynomials represent a crucial bridge between elementary algebra and advanced mathematics. The skills developed in working with polynomials - factoring, graphing, solving equations, and modeling real-world phenomena - form essential foundations for success in calculus, statistics, and applied mathematics. Understanding polynomial behavior provides insight into the nature of mathematical functions and their applications across science, engineering, and economics.

Rational Expressions: Working with Algebraic Fractions

Introduction to Rational Expressions

A rational expression is a fraction where both the numerator and denominator are polynomials. Just as rational numbers are ratios of integers, rational expressions are ratios of polynomials.

```
Rational Expression Structure


General Form: P(x)/Q(x) where P(x) and Q(x) are polynomials and Q(x)  0

Examples:
Simple: 3/x, (x+1)/(x-2), 5/(x²+1)

Complex: (x²-4)/(x²+3x+2), (2x³-x+1)/(x -16)

Mixed: 3 + 2/x = (3x+2)/x

Domain Restrictions:
The denominator cannot equal zero
Find values that make Q(x) = 0 and exclude them

Example: f(x) = (x+3)/(x²-9)
Domain restriction: x² - 9  0
x²  9
x  ±3
Domain: all real numbers except x = 3 and x = -3
```

Simplifying Rational Expressions

Finding Common Factors

The key to simplifying rational expressions is factoring both numerator and denominator, then canceling common factors.

Simplification Process

Steps:
1. Factor numerator completely
2. Factor denominator completely
3. Cancel common factors
4. State domain restrictions

Example 1: $(x^2-9)/(x^2+3x)$

Step 1: Factor numerator: $x^2 - 9 = (x+3)(x-3)$
Step 2: Factor denominator: $x^2 + 3x = x(x+3)$
Step 3: Cancel common factor (x+3):
$(x+3)(x-3)/(x(x+3)) = (x-3)/x$
Step 4: Domain: x 0, x -3

Example 2: $(x^2+5x+6)/(x^2+2x-3)$

Step 1: Factor numerator: $x^2 + 5x + 6 = (x+2)(x+3)$
Step 2: Factor denominator: $x^2 + 2x - 3 = (x+3)(x-1)$
Step 3: Cancel (x+3): $(x+2)(x+3)/((x+3)(x-1)) = (x+2)/(x-1)$
Step 4: Domain: x -3, x 1

Important: Even after canceling, original domain restrictions remain!

Complex Rational Expressions

Simplifying Complex Fractions

Complex fraction: Fraction containing fractions in numerator or denominator

Method 1: Multiply by LCD of all fractions

Example: $(1/x + 1/y)/(1/x - 1/y)$

LCD of all fractions = xy

Multiply numerator and denominator by xy:
$= (xy(1/x + 1/y))/(xy(1/x - 1/y))$
$= (y + x)/(y - x)$

Method 2: Simplify numerator and denominator separately, then divide

Example: $(2 + 3/x)/(1 - 1/x^2)$

```
Numerator: 2 + 3/x = (2x + 3)/x
Denominator: 1 - 1/x² = (x² - 1)/x²

Result: (2x + 3)/x ÷ (x² - 1)/x² = (2x + 3)/x · x²/(x² - 1) = x(2x + 3)/(x² - 1)

Factor further: = x(2x + 3)/((x + 1)(x - 1))
```

Operations with Rational Expressions

Addition and Subtraction

Like numerical fractions, rational expressions need common denominators for addition and subtraction.

```
Addition and Subtraction Process


Steps:
1. Find LCD (Least Common Denominator)
2. Rewrite each fraction with LCD
3. Add/subtract numerators
4. Simplify if possible

Example 1: 3/x + 2/(x+1)

LCD = x(x+1)

3/x = 3(x+1)/(x(x+1)) = (3x+3)/(x(x+1))
2/(x+1) = 2x/(x(x+1))

Sum: (3x+3)/(x(x+1)) + 2x/(x(x+1)) = (3x+3+2x)/(x(x+1)) = (5x+3)/(x(x+1))

Example 2: (x+2)/(x²-4) - 1/(x+2)

Factor: x² - 4 = (x+2)(x-2)
LCD = (x+2)(x-2)

(x+2)/(x²-4) = (x+2)/((x+2)(x-2))
1/(x+2) = (x-2)/((x+2)(x-2))

Difference: (x+2)/((x+2)(x-2)) - (x-2)/((x+2)(x-2)) = ((x+2)-(x-2))/((x+2)(x-2)) = 4/((x+2)(x-2

Simplified: 4/(x²-4)
```

Multiplication and Division

```
Multiplication and Division Rules


Multiplication: (A/B) · (C/D) = (AC)/(BD)
Division: (A/B) ÷ (C/D) = (A/B) · (D/C) = (AD)/(BC)

Process:
1. Factor all polynomials
2. Cancel common factors before multiplying
3. Multiply remaining factors
4. State domain restrictions

Example 1: (x²-1)/(x+2) · (x+2)/(x²+x)

Factor: x² - 1 = (x+1)(x-1), x² + x = x(x+1)

= ((x+1)(x-1))/(x+2) · (x+2)/(x(x+1))
= ((x+1)(x-1)(x+2))/((x+2)x(x+1))

Cancel (x+1) and (x+2):
= (x-1)/x

Domain: x   0, x   -1, x   -2

Example 2: (x²+3x+2)/(x²-4) ÷ (x+1)/(x-2)

= (x²+3x+2)/(x²-4) · (x-2)/(x+1)

Factor: x² + 3x + 2 = (x+1)(x+2), x² - 4 = (x+2)(x-2)

= ((x+1)(x+2))/((x+2)(x-2)) · (x-2)/(x+1)
= ((x+1)(x+2)(x-2))/((x+2)(x-2)(x+1))
= 1

Domain: x   ±2, x   -1
```

Solving Rational Equations

Basic Solution Techniques

A rational equation contains one or more rational expressions. To solve, clear the denominators by multiplying by the LCD.

```
Solution Process
```

Steps:
1. Find LCD of all denominators
2. Multiply entire equation by LCD
3. Solve resulting polynomial equation
4. Check solutions in original equation
5. Reject any extraneous solutions

Example 1: $3/x + 2 = 7/x$

LCD = x
Multiply by x: $x(3/x + 2) = x(7/x)$
$3 + 2x = 7$
$2x = 4$
$x = 2$

Check: $3/2 + 2 = 1.5 + 2 = 3.5$, $7/2 = 3.5$

Example 2: $1/(x-1) + 2/(x+1) = 4/(x^2-1)$

Factor: $x^2 - 1 = (x-1)(x+1)$
LCD = $(x-1)(x+1)$

Multiply by LCD:
$(x+1) + 2(x-1) = 4$
$x + 1 + 2x - 2 = 4$
$3x - 1 = 4$
$3x = 5$
$x = 5/3$

Check: Substitute $x = 5/3$ into original equation
Domain check: $x \neq \pm 1$, and $5/3 \neq \pm 1$

Extraneous Solutions

Why Extraneous Solutions Occur

When we multiply by LCD, we may introduce solutions that make the original denominators zero.

Example: $1/(x-2) + 1/(x+2) = 4/(x^2-4)$

LCD = $x^2 - 4 = (x-2)(x+2)$
Multiply by LCD:
$(x+2) + (x-2) = 4$
$2x = 4$

```
x = 2

Check: x = 2 makes denominators zero in original equation!
This is an extraneous solution.

The equation has no solution.

Always check solutions:
1. Substitute back into original equation
2. Verify denominators are not zero
3. Reject any extraneous solutions

Example with valid solution:
2/(x-1) = 3/(x+2)

Cross multiply: 2(x+2) = 3(x-1)
2x + 4 = 3x - 3
7 = x

Check: x = 7 doesn't make any denominator zero
2/(7-1) = 2/6 = 1/3
3/(7+2) = 3/9 = 1/3
```

Applications of Rational Expressions

Work Rate Problems

```
Work Rate Formula


If person A can complete a job in a hours and person B can complete it in b hours, then:
- A's rate: 1/a jobs per hour
- B's rate: 1/b jobs per hour
- Combined rate: 1/a + 1/b jobs per hour

Example: "John can paint a fence in 6 hours. Mary can paint the same fence in 4 hours. How long

John's rate: 1/6 fence per hour
Mary's rate: 1/4 fence per hour
Combined rate: 1/6 + 1/4 = 2/12 + 3/12 = 5/12 fence per hour

Time together: 1 ÷ (5/12) = 12/5 = 2.4 hours

Verification: In 2.4 hours:
John completes: 2.4 × (1/6) = 0.4 of fence
Mary completes: 2.4 × (1/4) = 0.6 of fence
```

```
Total: 0.4 + 0.6 = 1.0 fence
```

Distance-Rate-Time Problems

```
Motion Problems with Rational Expressions


Basic formula: Distance = Rate × Time, so Time = Distance/Rate

Example: "A boat travels 60 miles upstream in the same time it takes to travel 80 miles downstr

Let x = boat's speed in still water
Upstream speed: x - 2 mph
Downstream speed: x + 2 mph

Time upstream = 60/(x-2)
Time downstream = 80/(x+2)

Since times are equal:
60/(x-2) = 80/(x+2)

Cross multiply: 60(x+2) = 80(x-2)
60x + 120 = 80x - 160
280 = 20x
x = 14 mph

Check: Upstream time = 60/12 = 5 hours
       Downstream time = 80/16 = 5 hours
```

Mixture and Concentration Problems

```
Concentration Problems


Amount of pure substance = Concentration × Total volume

Example: "How many gallons of a 20% salt solution must be mixed with 5 gallons of a 60% salt so

Let x = gallons of 20% solution needed

Salt from 20% solution: 0.20x
Salt from 60% solution: 0.60(5) = 3
Total salt: 0.20x + 3

Total volume: x + 5
Final concentration: 40% = 0.40
```

```
Equation: (0.20x + 3)/(x + 5) = 0.40

Solve: 0.20x + 3 = 0.40(x + 5)
       0.20x + 3 = 0.40x + 2
       1 = 0.20x
       x = 5 gallons

Check: Salt: 0.20(5) + 3 = 4 gallons
       Total: 5 + 5 = 10 gallons
       Concentration: 4/10 = 0.40 = 40%
```

Rational Functions and Their Graphs

Domain and Range

```
Analyzing Rational Functions


For f(x) = P(x)/Q(x):

Domain: All real numbers except where Q(x) = 0
Range: Depends on function behavior and asymptotes

Example: f(x) = (x+1)/(x-2)

Domain: x   2 (since x - 2 = 0 when x = 2)

To find range, solve for x in terms of y:
y = (x+1)/(x-2)
y(x-2) = x+1
yx - 2y = x + 1
yx - x = 2y + 1
x(y-1) = 2y + 1
x = (2y+1)/(y-1)

For x to be real, y - 1   0, so y   1
Range: all real numbers except y = 1
```

Vertical and Horizontal Asymptotes

```
Asymptote Analysis


Vertical Asymptotes:
Occur where denominator = 0 but numerator   0
```

Lines: x = a where Q(a) = 0 and P(a) 0

Horizontal Asymptotes:
Depend on degrees of numerator and denominator

Let n = degree of numerator, d = degree of denominator:
- If n < d: horizontal asymptote at y = 0
- If n = d: horizontal asymptote at y = (leading coefficient of P)/(leading coefficient of Q)
- If n > d: no horizontal asymptote (oblique asymptote may exist)

Example: f(x) = $(2x^2+1)/(x^2-4)$

Vertical asymptotes: $x^2 - 4 = 0 \to x = \pm 2$
Horizontal asymptote: degrees equal, so y = 2/1 = 2

Behavior near asymptotes:
As $x \to 2$: $f(x) \to +\infty$ or $-\infty$ (check sign)
As $x \to 2$: $f(x) \to +\infty$ or $-\infty$ (check sign)
As $x \to \pm\infty$: $f(x) \to 2$

Graphing Rational Functions

Graphing Process

Steps:
1. Find domain (vertical asymptotes)
2. Find horizontal/oblique asymptotes
3. Find x-intercepts (numerator = 0)
4. Find y-intercept (f(0))
5. Analyze behavior near asymptotes
6. Plot additional points as needed
7. Sketch graph

Example: f(x) = (x-1)/(x+2)

1. Domain: x -2 (vertical asymptote at x = -2)
2. Horizontal asymptote: y = 1 (degrees equal, coefficients both 1)
3. x-intercept: x - 1 = 0 → x = 1, point (1, 0)
4. y-intercept: f(0) = -1/2, point (0, -1/2)
5. Behavior near x = -2:
   As $x \to -2$: $f(x) \to +\infty$
   As $x \to -2$: $f(x) \to -\infty$
6. Additional points: f(-1) = -2, f(2) = 1/4
7. Sketch showing asymptotes and key points

Graph characteristics:

- Vertical asymptote at x = -2
- Horizontal asymptote at y = 1
- Passes through (1, 0) and (0, -1/2)
- Two separate branches

Advanced Topics

Partial Fraction Decomposition

Decomposing Rational Expressions

Used to break complex rational expressions into simpler parts.

For proper fractions (degree of numerator < degree of denominator):

Case 1: Distinct Linear Factors
$(3x+1)/((x-1)(x+2)) = A/(x-1) + B/(x+2)$

Solve for A and B:
$3x + 1 = A(x+2) + B(x-1)$

Method 1 - Substitution:
$x = 1$: $3(1) + 1 = A(3) + B(0) \rightarrow 4 = 3A \rightarrow A = 4/3$
$x = -2$: $3(-2) + 1 = A(0) + B(-3) \rightarrow -5 = -3B \rightarrow B = 5/3$

Result: $(3x+1)/((x-1)(x+2)) = (4/3)/(x-1) + (5/3)/(x+2)$

Case 2: Repeated Linear Factors
$(2x+3)/((x-1)^2(x+2)) = A/(x-1) + B/(x-1)^2 + C/(x+2)$

Case 3: Irreducible Quadratic Factors
$(x+1)/((x^2+1)(x-2)) = (Ax+B)/(x^2+1) + C/(x-2)$

Rational Inequalities

Solving Rational Inequalities

Process:
1. Move all terms to one side
2. Find common denominator
3. Factor numerator and denominator
4. Find critical points (zeros and undefined points)
5. Create sign chart
6. Test intervals

```
7. Select appropriate intervals

Example: (x+1)/(x-2) > 0

Critical points: x = -1 (zero), x = 2 (undefined)

Sign chart:
Interval: (-∞,-1) | (-1,2) | (2,∞)
(x+1):       -    |   +    |   +
(x-2):       -    |   -    |   +
Quotient:    +    |   -    |   +

Solution: x   (-∞,-1)   (2,∞)

Note: x = 2 is excluded from domain
      x = -1 makes expression equal 0, not > 0
```

Summary and Key Concepts

Rational expressions extend fraction arithmetic to algebraic expressions, providing tools for modeling complex relationships and solving advanced problems.

```
Chapter Summary


Essential Skills Mastered:
 Simplifying rational expressions by factoring and canceling
 Performing operations (add, subtract, multiply, divide)
 Solving rational equations and checking for extraneous solutions
 Analyzing rational functions and their graphs
 Finding asymptotes and key features
 Applying rational expressions to real-world problems
 Working with complex rational expressions

Key Concepts:
• Domain restrictions and their importance
• Relationship between zeros, factors, and asymptotes
• Proper vs. improper rational expressions
• Vertical, horizontal, and oblique asymptotes
• Extraneous solutions in rational equations

Problem-Solving Applications:
• Work rate problems
• Distance-rate-time problems
• Mixture and concentration problems
• Optimization problems involving ratios
```

Advanced Topics:
• Partial fraction decomposition
• Rational inequalities
• Complex rational expressions
• Graphing techniques for rational functions

Next Steps:
Rational expression skills prepare you for:
- Limits and continuity in calculus
- Integration techniques (partial fractions)
- Differential equations
- Advanced function analysis
- Engineering and physics applications

Rational expressions represent a sophisticated extension of algebraic thinking, combining polynomial operations with fraction arithmetic. The skills developed in this chapter - domain analysis, asymptote identification, equation solving, and function graphing - form essential foundations for calculus and advanced mathematical applications. Understanding rational expressions provides insight into the behavior of complex functions and their real-world applications in science, engineering, and economics.

Linear Algebra

Introduction to Linear Algebra: The Mathematics of Vector Spaces

What is Linear Algebra?

Linear algebra is the branch of mathematics that studies vectors, vector spaces, linear transformations, and systems of linear equations. It provides a powerful framework for understanding and solving problems involving multiple variables and their linear relationships.

Unlike elementary algebra, which focuses on solving equations with single unknowns, linear algebra deals with systems of equations involving multiple unknowns simultaneously. It extends the concept of numbers to vectors and matrices, creating a rich mathematical structure that underlies much of modern mathematics, science, and technology.

```
The Scope of Linear Algebra


Core Objects:
• Vectors: Quantities with magnitude and direction
• Matrices: Rectangular arrays of numbers
• Vector Spaces: Collections of vectors with defined operations
• Linear Transformations: Functions that preserve vector operations

Key Operations:
• Vector addition and scalar multiplication
• Matrix multiplication and inversion
• Solving systems of linear equations
• Finding eigenvalues and eigenvectors

Applications:
• Computer graphics and 3D modeling
• Machine learning and data analysis
• Quantum mechanics and physics
• Economics and optimization
• Engineering and signal processing
```

Historical Development

Ancient Origins and Early Developments

Linear algebra concepts have ancient roots, though the modern framework developed relatively recently in mathematical history.

```
Timeline of Linear Algebra Development


2000 BCE: Babylonians solve systems of linear equations
          using elimination methods

300 BCE:  Euclid's Elements includes geometric vectors
          (though not called vectors)

1683:     Leibniz uses determinants to solve systems
          of linear equations

1750:     Cramer publishes Cramer's Rule for solving
          linear systems using determinants

1844:     Grassmann develops theory of vector spaces
          in "Die Lineale Ausdehnungslehre"

1858:     Cayley introduces matrix algebra and
          matrix multiplication

1888:     Peano axiomatizes vector spaces with
          his famous axioms

1918:     Weyl formalizes linear algebra in
          "Space, Time, Matter"

1940s:    Linear algebra becomes essential for
          computer science and numerical analysis

1960s:    Modern abstract approach emphasizes
          vector spaces and linear transformations

Today:    Linear algebra is fundamental to AI,
          machine learning, and data science
```

The Geometric Revolution

Linear algebra bridges the gap between algebraic computation and geometric intuition, providing both computational tools and visual understanding.

```
Geometric Foundations


Ancient Geometry → Modern Linear Algebra:

Euclidean Geometry:
- Points, lines, and planes
- Distances and angles
- Parallel and perpendicular relationships

Vector Geometry:
- Vectors as directed line segments
- Vector addition as parallelogram law
- Dot product for angles and projections

Coordinate Geometry:
- Cartesian coordinate system
- Algebraic representation of geometric objects
- Transformation of geometric problems to algebraic ones

Linear Transformations:
- Rotations, reflections, and scaling
- Matrix representation of transformations
- Composition of transformations

Modern Applications:
- Computer graphics and animation
- Robotics and navigation
- Image processing and computer vision
- 3D modeling and virtual reality
```

Fundamental Concepts

Vectors: The Building Blocks

Vectors are the fundamental objects of linear algebra, representing quantities that have both magnitude and direction.

```
Understanding Vectors


Geometric Interpretation:
- Directed line segments in space
- Have magnitude (length) and direction
- Can be translated without changing identity
- Independent of starting point (free vectors)
```

```
Algebraic Representation:
2D vector: v = [3, 4] or v = (3, 4)
3D vector: w = [1, -2, 5] or w = (1, -2, 5)
n-D vector: u = [u , u , ..., u ]

Physical Examples:
- Velocity: speed and direction of motion
- Force: magnitude and direction of push/pull
- Displacement: distance and direction of movement
- Electric field: strength and direction at each point

Abstract Examples:
- Color: RGB values [red, green, blue]
- Sound: frequency components
- Data point: features in machine learning
- Portfolio: weights of different investments

Vector Notation:
- Bold lowercase: v, w, u
- Arrow notation: v, w, u
- Component form: v =  v , v , v
- Column vector: v = [v ]
                     [v ]
                     [v ]
```

Vector Operations

The power of linear algebra comes from well-defined operations on vectors that preserve important properties.

```
Essential Vector Operations


Vector Addition:
Geometric: Parallelogram law or tip-to-tail method
Algebraic: Add corresponding components

Example: [2, 3] + [1, -1] = [2+1, 3+(-1)] = [3, 2]

Properties:
- Commutative: u + v = v + u
- Associative: (u + v) + w = u + (v + w)
- Zero vector: v + 0 = v
- Additive inverse: v + (-v) = 0

Scalar Multiplication:
```

```
Geometric: Scales magnitude, preserves/reverses direction
Algebraic: Multiply each component by scalar

Example: 3[2, -1] = [3·2, 3·(-1)] = [6, -3]

Properties:
- Distributive: a(u + v) = au + av
- Distributive: (a + b)u = au + bu
- Associative: a(bu) = (ab)u
- Identity: 1u = u

Linear Combinations:
au + bv + cw (where a, b, c are scalars)

This is the fundamental operation that gives linear algebra its name!

Dot Product (Inner Product):
u · v = u v  + u v  + ... + u v

Geometric meaning: u · v = |u||v|cos( )
where   is the angle between vectors

Applications:
- Finding angles between vectors
- Determining orthogonality (u · v = 0)
- Computing projections
- Measuring similarity
```

Matrices: Organizing Linear Information

Matrices provide a compact way to represent and manipulate linear relationships and transformations.

```
Matrix Fundamentals


Definition: Rectangular array of numbers arranged in rows and columns

Notation: A = [a   a   a ]
              [a   a   a ]
              [a   a   a ]

Dimensions: m × n (m rows, n columns)

Special Matrices:
Square matrix: m = n (same number of rows and columns)
Identity matrix: I = [1  0  0]  (1's on diagonal, 0's elsewhere)
```

```
                    [0  1  0]
                    [0  0  1]

Zero matrix: O = [0  0  0]  (all entries are 0)
                 [0  0  0]

Diagonal matrix: Non-zero entries only on main diagonal

Matrix as Linear Transformation:
A matrix A transforms vector x to vector Ax
This represents a linear transformation from one vector space to another

Matrix as System of Equations:
Ax = b represents the system:
a x  + a x  + ... + a x  = b
a x  + a x  + ... + a x  = b
...
a x  + a x  + ... + a x  = b

Matrix as Data Storage:
- Rows: observations/data points
- Columns: features/variables
- Entry a : value of feature j for observation i
```

Systems of Linear Equations

The Central Problem

Systems of linear equations form the historical and practical foundation of linear algebra.

```
System Structure and Representation


General System:
a x  + a x  + ... + a x  = b
a x  + a x  + ... + a x  = b
...
a x  + a x  + ... + a x  = b

Matrix Form: Ax = b
where A is coefficient matrix, x is variable vector, b is constant vector

Augmented Matrix: [A|b] = [a   a   ...  a  | b ]
                          [a   a   ...  a  | b ]
                          [...  ...  ...  ... | ...]
                          [a   a   ...  a  | b ]
```

```
Solution Types:
1. Unique solution: Exactly one solution vector x
2. No solution: System is inconsistent
3. Infinite solutions: System is underdetermined

Geometric Interpretation:
2D: Lines intersecting at point, parallel, or coincident
3D: Planes intersecting at point, no common intersection, or infinite intersection
nD: Hyperplanes in n-dimensional space
```

Solution Methods

```
Classical Solution Techniques


Gaussian Elimination:
Transform augmented matrix to row echelon form using elementary row operations

Elementary Row Operations:
1. Swap two rows: R   R
2. Multiply row by nonzero constant: kR  → R
3. Add multiple of one row to another: R  + kR  → R

Example: Solve system
x + 2y - z = 3
2x - y + z = 1
x + y + z = 6

Augmented matrix: [1   2  -1 |  3]
                  [2  -1   1 |  1]
                  [1   1   1 |  6]

Step 1: Eliminate below first pivot
R  - 2R  → R : [1   2  -1 |  3]
               [0  -5   3 | -5]
               [1   1   1 |  6]

R  - R  → R :  [1   2  -1 |  3]
               [0  -5   3 | -5]
               [0  -1   2 |  3]

Step 2: Continue elimination process...

Gauss-Jordan Elimination:
Continue to reduced row echelon form (RREF)
Results in identity matrix on left side (when possible)
```

```
Back Substitution:
Work backwards from row echelon form to find solution
```

Vector Spaces: The Abstract Framework

Axioms and Structure

Vector spaces provide the abstract mathematical framework that unifies all of linear algebra.

```
Vector Space Axioms


A vector space V over field F is a set with two operations:
- Vector addition: u + v   V for all u, v   V
- Scalar multiplication: au   V for all a   F, u   V

Axioms (Peano's axioms for vector spaces):

Addition Axioms:
A1. Closure: u + v   V
A2. Commutativity: u + v = v + u
A3. Associativity: (u + v) + w = u + (v + w)
A4. Zero vector:   0   V such that v + 0 = v
A5. Additive inverse:  v   V,  (-v) such that v + (-v) = 0

Scalar Multiplication Axioms:
S1. Closure: au   V
S2. Distributivity: a(u + v) = au + av
S3. Distributivity: (a + b)u = au + bu
S4. Associativity: a(bu) = (ab)u
S5. Identity: 1u = u

Examples of Vector Spaces:
-   : n-tuples of real numbers
- Polynomials of degree   n
- Continuous functions on [a,b]
- Matrices of size m × n
- Solutions to homogeneous differential equations
```

Subspaces and Span

```
Subspaces: Vector Spaces Within Vector Spaces
```

Definition: A subset W of vector space V is a subspace if:
1. 0 W (contains zero vector)
2. Closed under addition: u, v W u + v W
3. Closed under scalar multiplication: u W, a F au W

Examples in ^{3}:
- {0}: trivial subspace (just zero vector)
- Lines through origin: span{v} for some v 0
- Planes through origin: span{u, v} for linearly independent u, v
- ^{3} itself: improper subspace

Span of Vectors:
span{v , v , ..., v } = {a v + a v + ... + a v : a F}

The span is always a subspace (smallest subspace containing the vectors)

Linear Independence:
Vectors v , v , ..., v are linearly independent if:
a v + a v + ... + a v = 0 a = a = ... = a = 0

Otherwise, they are linearly dependent.

Basis and Dimension:
- Basis: Linearly independent set that spans the space
- Dimension: Number of vectors in any basis
- Every vector space has a basis
- All bases of a space have the same number of elements

Linear Transformations

Functions Between Vector Spaces

Linear transformations are functions between vector spaces that preserve the linear structure.

Linear Transformation Definition

A function T: V → W is a linear transformation if:
1. T(u + v) = T(u) + T(v) for all u, v V
2. T(au) = aT(u) for all a F, u V

Equivalently: T(au + bv) = aT(u) + bT(v)

Matrix Representation:
Every linear transformation T: → can be represented by an m × n matrix A such that T(x) = Ax

```
The columns of A are T(e ), T(e ), ..., T(e )
where {e , e , ..., e } is the standard basis for

Examples of Linear Transformations:
- Rotation by angle  : [cos    -sin  ]
                       [sin     cos  ]

- Reflection across x-axis: [1   0]
                            [0  -1]

- Scaling by factor k: [k  0]
                       [0  k]

- Projection onto x-axis: [1  0]
                          [0  0]

- Shear transformation: [1  k]
                        [0  1]

Kernel and Image:
- Kernel (null space): ker(T) = {v   V : T(v) = 0}
- Image (range): im(T) = {T(v) : v   V}
- Both are subspaces
- dim(V) = dim(ker(T)) + dim(im(T)) (Rank-Nullity Theorem)
```

Applications and Connections

Computer Graphics and 3D Modeling

Linear algebra is the mathematical foundation of computer graphics and 3D rendering.

```
Graphics Applications


3D Transformations:
- Translation: Moving objects in space
- Rotation: Rotating around axes
- Scaling: Changing size uniformly or non-uniformly
- Projection: Converting 3D to 2D for display

Homogeneous Coordinates:
Represent 3D points as 4D vectors: [x, y, z, 1]
Allows translation to be represented as matrix multiplication

Transformation Pipeline:
```

```
Model → World → View → Projection → Screen

Each step involves matrix multiplication:
Final position = P × V × W × M × vertex

Where:
- M: Model transformation matrix
- W: World transformation matrix
- V: View transformation matrix
- P: Projection transformation matrix

Lighting and Shading:
- Normal vectors for surface orientation
- Dot products for lighting calculations
- Reflection vectors using linear algebra

Animation:
- Interpolation between keyframes
- Skeletal animation using transformation hierarchies
- Physics simulation using linear systems
```

Machine Learning and Data Science

Modern machine learning relies heavily on linear algebra for data representation and algorithm implementation.

```
Machine Learning Applications


Data Representation:
- Data matrix: rows = samples, columns = features
- Feature vectors in high-dimensional space
- Similarity measures using dot products

Principal Component Analysis (PCA):
- Dimensionality reduction technique
- Find directions of maximum variance
- Uses eigenvalue decomposition
- Projects data onto lower-dimensional subspace

Linear Regression:
- Find best-fit line/plane/hyperplane
- Minimize sum of squared errors
- Solution: x = (A A) ¹A b (normal equation)
- Uses matrix operations for efficient computation

Neural Networks:
```

```
- Each layer is a linear transformation followed by nonlinearity
- Forward pass: series of matrix multiplications
- Backpropagation: computing gradients using chain rule
- Weight updates using gradient descent

Support Vector Machines:
- Find optimal separating hyperplane
- Maximize margin between classes
- Involves solving quadratic optimization problem
- Uses kernel methods (implicit high-dimensional spaces)

Recommendation Systems:
- Matrix factorization techniques
- Collaborative filtering using linear algebra
- Singular Value Decomposition (SVD)
- Low-rank approximations
```

Physics and Engineering

Linear algebra provides the mathematical language for describing physical phenomena and engineering systems.

```
Physics Applications


Quantum Mechanics:
- State vectors in Hilbert space
- Observables as linear operators (matrices)
- Eigenvalues = possible measurement outcomes
- Eigenvectors = corresponding quantum states
- Schrödinger equation: H  = E  (eigenvalue problem)

Classical Mechanics:
- State vectors: position and momentum
- Linear transformations for coordinate changes
- Rotation matrices for reference frame transformations
- Moment of inertia tensor

Electromagnetics:
- Electric and magnetic field vectors
- Maxwell's equations in vector form
- Electromagnetic wave propagation
- Antenna array processing

Engineering Applications:
- Structural analysis: solving for forces and displacements
- Control systems: state-space representation
```

```
- Signal processing: Fourier transforms and filtering
- Circuit analysis: nodal and mesh analysis
- Optimization: linear programming

Vibrations and Oscillations:
- Normal modes as eigenvectors
- Natural frequencies as eigenvalues
- Modal analysis of structures
- Coupled oscillator systems
```

The Beauty and Power of Linear Algebra

Unifying Mathematical Concepts

Linear algebra serves as a unifying framework that connects diverse areas of mathematics and applications.

```
Mathematical Connections


Geometry  Algebra:
- Geometric problems → algebraic computations
- Algebraic solutions → geometric interpretations
- Coordinate geometry as bridge

Analysis  Linear Algebra:
- Function spaces as infinite-dimensional vector spaces
- Differential equations as linear transformations
- Fourier analysis using orthogonal bases
- Approximation theory using projections

Abstract Algebra  Linear Algebra:
- Vector spaces as algebraic structures
- Linear transformations as homomorphisms
- Matrix groups and their properties
- Representation theory

Topology  Linear Algebra:
- Continuous linear transformations
- Normed vector spaces
- Banach and Hilbert spaces
- Functional analysis

Number Theory  Linear Algebra:
- Lattices as discrete subgroups
- Diophantine equations as integer linear systems
```

```
- Cryptography using linear algebra over finite fields
- Error-correcting codes
```

Computational Power

Linear algebra provides efficient computational methods for solving large-scale problems.

```
Computational Advantages


Parallel Processing:
- Matrix operations naturally parallelizable
- Vector operations can be distributed
- GPU acceleration for linear algebra
- Distributed computing for large matrices

Numerical Stability:
- Well-developed algorithms for matrix computations
- Error analysis and conditioning
- Iterative methods for large systems
- Sparse matrix techniques

Scalability:
- Algorithms that scale to millions of variables
- Efficient storage formats for special matrices
- Approximation methods for very large problems
- Streaming algorithms for data that doesn't fit in memory

Software Ecosystem:
- BLAS (Basic Linear Algebra Subprograms)
- LAPACK (Linear Algebra Package)
- High-level languages: MATLAB, Python (NumPy), R
- Specialized libraries for different applications
```

Building Linear Algebra Intuition

Developing Geometric Insight

Success in linear algebra requires developing both computational skills and geometric intuition.

```
Visualization Strategies


2D and 3D Visualization:
- Plot vectors as arrows
```

```
- Visualize linear transformations as geometric operations
- See matrix multiplication as transformation composition
- Understand eigenvalues/eigenvectors geometrically

Higher Dimensions:
- Use analogies from 2D/3D
- Focus on algebraic properties
- Understand through projections to lower dimensions
- Develop abstract reasoning skills

Interactive Tools:
- Graphing software for visualization
- Computer algebra systems for computation
- Online interactive demonstrations
- Programming environments for experimentation

Geometric Interpretations:
- Systems of equations as intersecting hyperplanes
- Linear transformations as geometric operations
- Eigenspaces as invariant directions
- Orthogonality as perpendicularity generalized
```

Problem-Solving Strategies

```
Effective Learning Approaches


Conceptual Understanding:
1. Start with geometric intuition
2. Connect to algebraic computation
3. Practice with concrete examples
4. Generalize to abstract settings

Computational Fluency:
1. Master basic operations (addition, multiplication)
2. Learn systematic algorithms (Gaussian elimination)
3. Understand when to use different methods
4. Develop checking and verification skills

Application Awareness:
1. See connections to other subjects
2. Work with real-world problems
3. Understand modeling assumptions
4. Appreciate computational considerations

Progressive Complexity:
1. Start with small examples (2×2, 3×3)
```

```
2. Understand patterns and generalizations
3. Work with larger systems
4. Tackle abstract vector spaces

Study Techniques:
- Work many problems with different contexts
- Verify answers using multiple methods
- Explain concepts to others
- Connect new ideas to previous knowledge
- Use technology appropriately
```

Conclusion

Linear algebra represents one of the most powerful and widely applicable areas of mathematics. From its origins in solving systems of equations to its modern applications in artificial intelligence, computer graphics, and quantum mechanics, linear algebra provides both computational tools and conceptual frameworks that are essential for understanding our technological world.

```
Linear Algebra: Gateway to Modern Mathematics


Historical Significance:
  Evolved from practical problem-solving needs
  Unified geometric and algebraic thinking
  Provided foundation for modern mathematics
  Enabled computational revolution

Conceptual Power:
  Abstracts common patterns across mathematics
  Provides language for multidimensional thinking
  Connects discrete and continuous mathematics
  Bridges pure and applied mathematics

Practical Applications:
  Essential for computer science and engineering
  Foundation for data science and machine learning
  Critical for physics and natural sciences
  Enables modern technology and innovation

Educational Value:
  Develops abstract reasoning skills
  Teaches systematic problem-solving methods
  Builds computational thinking abilities
  Prepares for advanced mathematics and applications
```

As you begin your journey through linear algebra, remember that you're learning not just computational techniques, but a new way of thinking about mathematical relationships. The concepts of vectors, matrices, and linear transformations will become powerful tools for understanding and solving complex problems across many fields.

Linear algebra is truly the mathematics of the modern world - from the graphics on your computer screen to the algorithms that power search engines and artificial intelligence. The investment you make in understanding these concepts will pay dividends throughout your academic and professional career, opening doors to advanced mathematics, cutting-edge technology, and innovative problem-solving approaches.

Whether you're interested in pure mathematics, applied sciences, engineering, computer science, or data analysis, linear algebra provides essential foundations that will serve you well. The beauty of linear algebra lies not just in its practical utility, but in its elegant mathematical structure and its power to reveal the underlying patterns that govern our universe.

Vectors: The Foundation of Linear Algebra

Introduction to Vectors

Vectors are mathematical objects that represent quantities having both magnitude and direction. They form the fundamental building blocks of linear algebra and provide a powerful way to describe and analyze multidimensional relationships.

```
Vector Concepts


Physical Interpretation:
- Displacement: How far and in what direction
- Velocity: Speed and direction of motion
- Force: Magnitude and direction of push/pull
- Acceleration: Rate and direction of velocity change

Mathematical Representation:
- Geometric: Directed line segments (arrows)
- Algebraic: Ordered lists of numbers
- Abstract: Elements of vector spaces

Key Properties:
  Independent of starting position (free vectors)
  Defined by magnitude and direction only
  Can be added and scaled
  Form the basis for linear combinations
```

Vector Notation and Representation

Coordinate Representation

Vectors can be represented using coordinates in various dimensional spaces.

```
Vector Notation Systems


2D Vectors:
Component form: v =  3, 4  or v = (3, 4)
```

```
Column vector: v = [3]
                   [4]
Row vector: v = [3  4]

3D Vectors:
Component form: w =  1, -2, 5
Column vector: w = [1]
                   [-2]
                   [5]

n-Dimensional Vectors:
General form: u =  u , u , u , ..., u
Column form: u = [u ]
                 [u ]
                 [u ]
                 [ ]
                 [u ]

Standard Notation:
- Bold lowercase letters: v, w, u
- Arrow notation: v, w, u
- Component notation: v  (i-th component of v)
- Magnitude notation: |v| or ||v||
```

Geometric Interpretation

```
Geometric Vector Properties


Position vs. Direction:
- Position vector: From origin to point
- Direction vector: Represents direction and magnitude only
- Free vector: Can be placed anywhere in space

Magnitude (Length):
2D: |v| = √(v ² + v ²)
3D: |w| = √(w ² + w ² + w ²)
nD: |u| = √(u ² + u ² + ... + u ²)

Unit Vectors:
- Magnitude = 1
- Represent pure direction
- Standard unit vectors:
  2D: î =  1, 0 , ĵ =  0, 1
  3D: î =  1, 0, 0 , ĵ =  0, 1, 0 , k̂ =  0, 0, 1

Direction Angles:
```

```
Angles that vector makes with coordinate axes
cos   = v /|v|, cos   = v /|v|, cos   = v /|v|
where  ,  ,   are angles with x, y, z axes respectively
```

Vector Operations

Vector Addition

Vector addition combines two vectors to produce a resultant vector.

```
Vector Addition Methods


Algebraic Method:
Add corresponding components
u + v =  u  + v , u  + v , u  + v

Example:
u =  2, 3, -1
v =  -1, 4, 2
u + v =  2 + (-1), 3 + 4, -1 + 2  =  1, 7, 1

Geometric Methods:

1. Parallelogram Law:
   - Place vectors tail-to-tail
   - Complete parallelogram
   - Diagonal from common tail is sum

2. Triangle Law (Tip-to-Tail):
   - Place second vector's tail at first vector's tip
   - Sum vector goes from first tail to second tip

Properties of Vector Addition:
- Commutative: u + v = v + u
- Associative: (u + v) + w = u + (v + w)
- Identity: v + 0 = v (zero vector is additive identity)
- Inverse: v + (-v) = 0 (every vector has additive inverse)

Applications:
- Displacement: Total displacement = sum of individual displacements
- Forces: Resultant force = vector sum of individual forces
- Velocities: Relative velocity calculations
```

Scalar Multiplication

Scalar multiplication scales a vector by a real number, changing its magnitude and possibly direction.

```
Scalar Multiplication Properties


Algebraic Definition:
cv =  cv , cv , cv

Examples:
v =  2, -3, 1
2v =  4, -6, 2
-0.5v =  -1, 1.5, -0.5

Geometric Effects:
- c > 1: Stretches vector (increases magnitude)
- 0 < c < 1: Shrinks vector (decreases magnitude)
- c = 1: No change
- c = 0: Results in zero vector
- c < 0: Reverses direction and scales magnitude

Properties:
- Distributive over vector addition: c(u + v) = cu + cv
- Distributive over scalar addition: (a + b)v = av + bv
- Associative: a(bv) = (ab)v
- Identity: 1v = v

Unit Vector Formula:
For any non-zero vector v, the unit vector in same direction:
û = v/|v| = (1/|v|)v

Example:
v =  3, 4
|v| = √(3² + 4²) = 5
û = (1/5) 3, 4  =  3/5, 4/5
```

Linear Combinations

Linear combinations form the foundation of vector spaces and span concepts.

```
Linear Combination Definition


A linear combination of vectors v , v , ..., v  is:
```

c v + c v + ... + c v

where c , c , ..., c are scalars (coefficients)

Examples:
Given u = 1, 2 and v = 3, -1

3u + 2v = 3 1, 2 + 2 3, -1
        = 3, 6 + 6, -2
        = 9, 4

-u + 4v = - 1, 2 + 4 3, -1
        = -1, -2 + 12, -4
        = 11, -6

Geometric Interpretation:
- Linear combinations create new vectors
- All possible linear combinations form a subspace
- Two non-parallel vectors span a plane
- Three non-coplanar vectors span 3D space

Standard Basis Representation:
Any vector in   can be written as linear combination of standard basis vectors

2D: v = v , v = v î + v ĵ
3D: w = w , w , w = w î + w ĵ + w k̂

Applications:
- Computer graphics: Interpolation between points
- Physics: Superposition of forces, fields
- Economics: Portfolio combinations
- Engineering: Signal processing, control systems

Dot Product (Inner Product)

Definition and Computation

The dot product is a fundamental operation that produces a scalar from two vectors.

Dot Product Definitions

Algebraic Definition:
u · v = u v + u v + u v + ... + u v

Examples:

u = 2, 3, -1 , v = 1, -2, 4
u · v = (2)(1) + (3)(-2) + (-1)(4) = 2 - 6 - 4 = -8

u = 5, 0 , v = 3, 4
u · v = (5)(3) + (0)(4) = 15

Geometric Definition:
u · v = |u||v|cos

where is the angle between vectors u and v

Properties:
- Commutative: u · v = v · u
- Distributive: u · (v + w) = u · v + u · w
- Scalar associative: (cu) · v = c(u · v) = u · (cv)
- Positive definite: v · v 0, with equality iff v = 0

Self Dot Product:
v · v = |v|² = v ² + v ² + ... + v ²
This gives the squared magnitude of the vector

Geometric Applications

Angle Between Vectors

Formula: cos = (u · v)/(|u||v|)

Example:
u = 1, 2 , v = 3, 1
u · v = (1)(3) + (2)(1) = 5
|u| = √(1² + 2²) = √5
|v| = √(3² + 1²) = √10

cos = 5/(√5 · √10) = 5/√50 = 1/√2
 = 45°

Special Cases:
- = 0°: Vectors point in same direction (cos = 1)
- = 90°: Vectors are perpendicular (cos = 0)
- = 180°: Vectors point in opposite directions (cos = -1)

Orthogonality:
Two vectors are orthogonal (perpendicular) if and only if u · v = 0

Examples of Orthogonal Vectors:
1, 0 and 0, 1 (standard basis vectors)

3, 4 and 4, -3
1, 1, 1 and 1, -1, 0

Orthogonal Complement:
For vector v = a, b , orthogonal vectors have form -b, a or b, -a

Vector Projections

Projection Formulas

Scalar Projection (Component):
comp_u v = (v · u)/|u| = |v|cos

This gives the signed length of v in direction of u

Vector Projection:
proj_u v = ((v · u)/(u · u))u = ((v · u)/|u|²)u

This gives the vector component of v in direction of u

Example:
Project v = 3, 4 onto u = 1, 0

v · u = (3)(1) + (4)(0) = 3
u · u = 1² + 0² = 1

proj_u v = (3/1) 1, 0 = 3, 0

Geometric Interpretation:
- Projection is the "shadow" of v onto line containing u
- Always lies along the direction of u
- Length equals |v|cos where is angle between vectors

Orthogonal Decomposition:
Any vector v can be decomposed as:
v = proj_u v + (v - proj_u v)

where proj_u v is parallel to u and (v - proj_u v) is orthogonal to u

Applications:
- Physics: Component of force in given direction
- Computer graphics: Lighting calculations
- Engineering: Stress analysis
- Statistics: Regression analysis

Cross Product (3D Only)

Definition and Properties

The cross product is defined only in 3D space and produces a vector perpendicular to both input vectors.

```
Cross Product Definition


Algebraic Formula:
u × v =  u v  - u v , u v  - u v , u v  - u v

Determinant Form:
u × v = |î  ĵ  k̂|
        |u  u  u |
        |v  v  v |

Example:
u =  2, 1, -1 , v =  1, 3, 2

u × v =  (1)(2) - (-1)(3), (-1)(1) - (2)(2), (2)(3) - (1)(1)
      =  2 + 3, -1 - 4, 6 - 1
      =  5, -5, 5

Properties:
- Anti-commutative: u × v = -(v × u)
- Distributive: u × (v + w) = u × v + u × w
- Scalar associative: (cu) × v = c(u × v) = u × (cv)
- u × u = 0 (any vector crossed with itself is zero)
- u × v = 0 if and only if u and v are parallel

Geometric Properties:
- Result is perpendicular to both u and v
- Direction follows right-hand rule
- Magnitude: |u × v| = |u||v|sin
- Magnitude equals area of parallelogram formed by u and v
```

Applications of Cross Product

```
Cross Product Applications


Area Calculations:
Area of parallelogram = |u × v|
Area of triangle = (1/2)|u × v|
```

Example:
Find area of triangle with vertices A(1,0,0), B(0,1,0), C(0,0,1)

AB = -1, 1, 0
AC = -1, 0, 1

AB × AC = 1, 1, 1
|AB × AC| = √(1² + 1² + 1²) = √3

Area = (1/2)√3

Normal Vectors:
u × v gives normal vector to plane containing u and v
Used in computer graphics for surface normals

Torque in Physics:
= r × F
where r is position vector and F is force vector
Magnitude: | | = |r||F|sin

Angular Velocity:
v = × r
where is angular velocity vector and r is position vector

Right-Hand Rule:
Point fingers in direction of first vector
Curl toward second vector
Thumb points in direction of cross product

Vector Spaces and Subspaces

Vector Space Axioms

Formal Vector Space Definition

A vector space V over field F satisfies:

Closure Properties:
1. u + v V for all u, v V
2. cu V for all c F, u V

Addition Properties:
3. u + v = v + u (commutative)
4. (u + v) + w = u + (v + w) (associative)

5. 0 V such that v + 0 = v for all v V
6. For each v V, (-v) V such that v + (-v) = 0

Scalar Multiplication Properties:
7. c(u + v) = cu + cv
8. (c + d)u = cu + du
9. c(du) = (cd)u
10. 1u = u

Examples of Vector Spaces:
- : n-tuples of real numbers
- : n-tuples of complex numbers
- P : polynomials of degree n
- C[a,b]: continuous functions on interval [a,b]
- M : m×n matrices with real entries

Non-Examples:
- Positive real numbers (no additive identity)
- Integers (not closed under scalar multiplication)
- Vectors in ^{2} with first component = 1 (no zero vector)

Subspaces

Subspace Definition and Tests

A subset W of vector space V is a subspace if:
1. 0 W (contains zero vector)
2. Closed under addition: u, v W u + v W
3. Closed under scalar multiplication: u W, c F cu W

Equivalent Test:
W is a subspace if and only if:
au + bv W for all u, v W and scalars a, b

Examples in ^{3}:
- {0}: trivial subspace
- Lines through origin: {t a, b, c : t }
- Planes through origin: {s a , b , c + t a , b , c : s, t }
- ^{3} itself: improper subspace

Non-Examples:
- Line not through origin (no zero vector)
- First quadrant in ^{2} (not closed under scalar multiplication)
- Unit sphere (not closed under addition)

Important Subspaces:

- Span of vectors: span{v , v , ..., v }
- Null space of matrix: {x : Ax = 0}
- Column space of matrix: span of column vectors
- Row space of matrix: span of row vectors

Linear Independence and Basis

Linear Independence

Linear Independence Definition

Vectors v , v , ..., v are linearly independent if:
c v + c v + ... + c v = 0 c = c = ... = c = 0

Otherwise, they are linearly dependent.

Testing Linear Independence:
Set up equation c v + c v + ... + c v = 0
Solve for coefficients c , c , ..., c
If only solution is all c = 0, vectors are independent

Example:
Test independence of v = 1, 2, 0 , v = 0, 1, 1 , v = 1, 0, -1

c 1, 2, 0 + c 0, 1, 1 + c 1, 0, -1 = 0, 0, 0

This gives system:
c + c = 0
2c + c = 0
c - c = 0

Solving: c = c = c = 0 (only solution)
Therefore, vectors are linearly independent.

Geometric Interpretation:
- 2 vectors: independent if not parallel
- 3 vectors in ^{3}: independent if not coplanar
- n vectors in : independent if they span n-dimensional space

Key Facts:
- Any set containing zero vector is dependent
- More than n vectors in must be dependent
- Subset of independent set is independent
- Superset of dependent set is dependent

Basis and Dimension

```
Basis Definition


A basis for vector space V is a set of vectors that:
1. Spans V (every vector in V is a linear combination)
2. Is linearly independent

Properties of Bases:
- Every vector space has a basis
- All bases of a space have the same number of elements
- Basis provides unique representation for each vector

Standard Bases:
 ²: { 1, 0 ,  0, 1 }
 ³: { 1, 0, 0 ,  0, 1, 0 ,  0, 0, 1 }
  : {e , e , ..., e } where e  has 1 in position i, 0 elsewhere

Alternative Bases:
 ²: { 1, 1 ,  1, -1 }
 ³: { 1, 1, 0 ,  0, 1, 1 ,  1, 0, 1 }

Dimension:
The dimension of vector space V is the number of vectors in any basis for V

dim( ) = n
dim({0}) = 0
dim(line through origin) = 1
dim(plane through origin) = 2

Coordinate Representation:
If B = {v , v , ..., v } is basis for V and v = c v  + c v  + ... + c v ,
then [v]  =  c , c , ..., c  is the coordinate vector of v relative to B

Change of Basis:
Converting between different coordinate systems
Involves matrix transformations between bases
```

Applications and Examples

Computer Graphics

```
Graphics Applications
```

```
3D Transformations:
- Translation: v' = v + d (where d is displacement vector)
- Scaling: v' = sv (where s is scale factor)
- Rotation: v' = Rv (where R is rotation matrix)

Lighting Calculations:
- Surface normals using cross products
- Diffuse lighting: intensity  n · l (normal dot light direction)
- Specular reflection using vector reflections

Camera Systems:
- View vectors: direction camera is pointing
- Up vectors: orientation of camera
- Right vectors: perpendicular to view and up

Animation:
- Interpolation between keyframes using linear combinations
- Velocity vectors for motion
- Acceleration vectors for realistic physics

Example: Rotating point (3, 4) by 90° counterclockwise
Rotation matrix: R = [0  -1]
                     [1   0]

v' = Rv = [0  -1][3] = [-4]
          [1   0][4]   [3]

Result: (3, 4) → (-4, 3)
```

Physics Applications

```
Physics Vector Applications


Force Analysis:
- Resultant force: F_net = F  + F  + ... + F
- Equilibrium: ΣF = 0
- Components: F_x = |F|cos  , F_y = |F|sin

Motion in 2D and 3D:
- Position: r(t) =  x(t), y(t), z(t)
- Velocity: v(t) = dr/dt
- Acceleration: a(t) = dv/dt

Electromagnetic Fields:
- Electric field: E = F/q (force per unit charge)
- Magnetic field: F = q(v × B) (Lorentz force)
```

- Electromagnetic waves: E ⊥ B ⊥ direction of propagation

Example: Projectile Motion
Initial velocity: v = v cos , v sin
Acceleration: a = 0, -g
Position: r(t) = r + v t + (1/2)at²
= x + v cos · t, y + v sin · t - (1/2)gt²

Engineering Applications

Engineering Vector Uses

Structural Analysis:
- Force vectors in trusses and beams
- Moment vectors for rotational effects
- Stress and strain tensors (advanced)

Signal Processing:
- Signals as vectors in function space
- Fourier analysis using orthogonal basis functions
- Filtering as projection operations

Control Systems:
- State vectors representing system conditions
- Input and output vectors
- Feedback control using vector operations

Robotics:
- Position and orientation vectors
- Joint angles as configuration vectors
- Path planning in configuration space

Example: Truss Analysis
Forces at joint must sum to zero:
F + F + F = 0

If F = 100, 0 N and F = -50, 86.6 N,
then F = -F - F = -50, -86.6 N

Summary and Key Concepts

Vectors provide the fundamental language for describing multidimensional quantities and relationships, forming the cornerstone of linear algebra and its applications.

Chapter Summary

Essential Skills Mastered:
- Vector representation in multiple dimensions
- Vector operations (addition, scalar multiplication, dot product, cross product)
- Geometric interpretation of vector operations
- Linear combinations and their significance
- Understanding of vector spaces and subspaces
- Linear independence and basis concepts
- Applications in graphics, physics, and engineering

Key Concepts:
- Vectors as quantities with magnitude and direction
- Algebraic and geometric approaches to vector operations
- Dot product for angles, projections, and orthogonality
- Cross product for areas, normals, and rotations (3D)
- Vector spaces as abstract mathematical structures
- Basis as minimal spanning sets
- Dimension as measure of space "size"

Fundamental Operations:
- Addition: u + v (parallelogram law)
- Scalar multiplication: cv (scaling and direction)
- Dot product: u · v = |u||v|cos
- Cross product: u × v (perpendicular vector, 3D only)
- Linear combination: c v + c v + ... + c v

Applications Covered:
- Computer graphics and 3D transformations
- Physics: forces, motion, electromagnetic fields
- Engineering: structural analysis, signal processing
- Geometric calculations: areas, angles, projections

Next Steps:
Vector concepts prepare you for:
- Matrix operations and linear transformations
- Systems of linear equations
- Eigenvalues and eigenvectors
- Advanced applications in data science and machine learning

Vectors represent one of the most intuitive yet powerful concepts in mathematics. By mastering vector operations and their geometric interpretations, you've built essential foundations for understanding linear transformations, solving systems of equations, and applying linear algebra to real-world problems. The skills developed in this chapter will serve as building blocks for all subsequent topics in linear algebra and its applications across science, engineering, and technology.

Matrices: Organizing and Transforming Linear Information

Introduction to Matrices

A matrix is a rectangular array of numbers, symbols, or expressions arranged in rows and columns. Matrices provide a compact and powerful way to represent linear transformations, solve systems of equations, and organize multidimensional data.

```
Matrix Fundamentals


Definition: An m × n matrix has m rows and n columns

General Form:
A = [a   a   a   ...  a ]
    [a   a   a   ...  a ]
    [a   a   a   ...  a ]
    [                   ]
    [a   a   a   ...  a ]

Notation:
- Capital letters: A, B, C
- Entry notation: a  (row i, column j)
- Size notation: A

Examples:
2×3 matrix: A = [1   2  -1]
                [3  -1   4]

3×3 matrix: B = [2   0   1]
                [1  -3   2]
                [0   1  -1]

Column vector: v = [3]  (3×1 matrix)
                   [2]
                   [-1]

Row vector: w = [1  4  -2]  (1×3 matrix)
```

Types of Matrices

Special Matrix Categories

```
Important Matrix Types


Square Matrix: m = n (same number of rows and columns)
Example: [1  2]
         [3  4]

Identity Matrix: Square matrix with 1's on diagonal, 0's elsewhere
I  = [1  0]    I  = [1  0  0]
     [0  1]         [0  1  0]
                    [0  0  1]

Zero Matrix: All entries are 0
0 = [0  0  0]
    [0  0  0]

Diagonal Matrix: Non-zero entries only on main diagonal
D = [3  0  0]
    [0 -2  0]
    [0  0  5]

Upper Triangular: All entries below diagonal are 0
U = [2  3  1]
    [0 -1  4]
    [0  0  3]

Lower Triangular: All entries above diagonal are 0
L = [2  0  0]
    [1 -3  0]
    [4  2  1]

Symmetric Matrix: A = A  (equals its transpose)
S = [1  2  3]
    [2  4  5]
    [3  5  6]

Skew-Symmetric: A = -A  (diagonal entries are 0)
K = [ 0  2 -1]
    [-2  0  3]
    [ 1 -3  0]
```

Matrix Dimensions and Indexing

```
Matrix Structure and Access


Dimension Rules:
- Matrix A has dimensions m × n
- Entry a  is in row i, column j
- Row index: 1   i   m
- Column index: 1   j   n

Row and Column Vectors:
Row i of matrix A: [a   a   ...  a ]
Column j of matrix A: [a ]
                      [a ]
                      [  ]
                      [a ]

Example: A = [1  2  3]
             [4  5  6]

- A is 2×3 matrix
- a  = 2 (row 1, column 2)
- a  = 6 (row 2, column 3)
- Row 1: [1  2  3]
- Column 2: [2]
            [5]

Submatrices:
Extract rectangular portions of larger matrices
Useful for block operations and partitioning

Principal Submatrix:
Square submatrix formed by deleting same rows and columns
Important for eigenvalue analysis
```

Matrix Operations

Matrix Addition and Subtraction

Matrix addition and subtraction are performed element-wise and require matrices of the same dimensions.

```
Addition and Subtraction Rules
```

```
Requirement: Matrices must have same dimensions

Definition: (A + B)  = a  + b
            (A - B)  = a  - b

Example:
A = [1  2]    B = [3  1]
    [3 -1]        [2  4]

A + B = [1+3  2+1] = [4  3]
        [3+2 -1+4]   [5  3]

A - B = [1-3  2-1] = [-2  1]
        [3-2 -1-4]   [ 1 -5]

Properties:
- Commutative: A + B = B + A
- Associative: (A + B) + C = A + (B + C)
- Zero matrix: A + 0 = A
- Additive inverse: A + (-A) = 0

Scalar Multiplication:
(cA)  = c·a

Example:
3A = 3[1  2] = [3   6]
     [3 -1]   [9  -3]

Properties:
- Distributive: c(A + B) = cA + cB
- Distributive: (c + d)A = cA + dA
- Associative: c(dA) = (cd)A
- Identity: 1A = A
```

Matrix Multiplication

Matrix multiplication is more complex than addition and follows specific rules about dimensions and computation.

```
Matrix Multiplication Rules


Dimension Requirement:
A(m×n) × B(n×p) = C(m×p)
Number of columns in A must equal number of rows in B

Definition:
```

```
c  = Σ   a b  = a b  + a b  + ... + a b

Example:
A = [1  2]    B = [3  1]
    [3 -1]        [2  4]

AB = [1·3 + 2·2   1·1 + 2·4] = [7   9]
     [3·3 + (-1)·2  3·1 + (-1)·4]   [7  -1]

Step-by-step for c :
c  = a b  + a b  = (1)(3) + (2)(2) = 3 + 4 = 7

Matrix-Vector Multiplication:
Ax = b where A is m×n, x is n×1, b is m×1

Example:
[1  2  3][2]   [1·2 + 2·1 + 3·0]   [4]
[4  5  6][1] = [4·2 + 5·1 + 6·0] = [13]
         [0]

Geometric Interpretation:
Matrix multiplication represents composition of linear transformations
Ax transforms vector x according to transformation represented by A
```

Properties of Matrix Multiplication

```
Matrix Multiplication Properties


Non-Commutative: Generally AB  BA
Example:
A = [1  2]    B = [0  1]
    [0  1]        [1  0]

AB = [2  1]    BA = [0  1]
     [1  0]         [1  2]

Clearly AB  BA

Associative: (AB)C = A(BC)
When dimensions allow, grouping doesn't matter

Distributive:
- A(B + C) = AB + AC (right distributive)
- (A + B)C = AC + BC (left distributive)

Identity Property:
```

```
AI = IA = A (when dimensions allow)

Zero Property:
AO = OA = O (when dimensions allow)

Transpose Property:
(AB)  = B A  (order reverses!)

Block Multiplication:
Matrices can be partitioned into blocks and multiplied block-wise
Useful for large matrices and parallel computation

Example:
[A   A ][B   B ] = [A B  + A B   A B  + A B ]
[A   A ][B   B ]   [A B  + A B   A B  + A B ]
```

Matrix Transpose

Definition and Properties

The transpose of a matrix is formed by interchanging rows and columns.

```
Transpose Operation


Definition: (A )  = a

Example:
A = [1  2  3]    A  = [1  4]
    [4  5  6]          [2  5]
                       [3  6]

For square matrix:
B = [1  2  3]    B  = [1  4  7]
    [4  5  6]          [2  5  8]
    [7  8  9]          [3  6  9]

Properties:
- (A )  = A (transpose of transpose is original)
- (A + B)  = A  + B
- (cA)  = cA
- (AB)  = B A  (order reverses!)

Special Cases:
- Row vector transpose = column vector
- Column vector transpose = row vector
```

```
- Symmetric matrix: A = A
- Skew-symmetric matrix: A = -A

Applications:
- Converting between row and column vectors
- Defining inner products: x y
- Least squares problems: (A A)x = A b
- Orthogonal matrices: Q Q = I
```

Determinants

Definition and Calculation

The determinant is a scalar value that provides important information about a square matrix.

```
Determinant Basics


2×2 Determinant:
det(A) = |a  b| = ad - bc
         |c  d|

Example:
A = [3  2]
    [1  4]
det(A) = (3)(4) - (2)(1) = 12 - 2 = 10

3×3 Determinant (Cofactor Expansion):
|a   a   a |
|a   a   a | = a |a   a | - a |a   a | + a |a   a |
|a   a   a |     |a   a |     |a   a |     |a   a |

Example:
A = [2  1  3]
    [1  4  2]
    [3  2  1]

det(A) = 2|4  2| - 1|1  2| + 3|1  4|
          |2  1|    |3  1|    |3  2|
       = 2(4-4) - 1(1-6) + 3(2-12)
       = 2(0) - 1(-5) + 3(-10)
       = 0 + 5 - 30 = -25

Properties:
- det(I) = 1 (identity matrix)
- det(AB) = det(A)det(B)
```

- det(A) = det(A)
- det(cA) = c det(A) for n×n matrix
- If A has zero row/column, det(A) = 0
- Swapping rows changes sign of determinant

Geometric Interpretation

Determinant Geometric Meaning

2D Interpretation:
det(A) = signed area of parallelogram formed by column vectors
- Positive: counterclockwise orientation
- Negative: clockwise orientation
- Zero: vectors are parallel (degenerate parallelogram)

3D Interpretation:
det(A) = signed volume of parallelepiped formed by column vectors
- Positive: right-handed orientation
- Negative: left-handed orientation
- Zero: vectors are coplanar (degenerate parallelepiped)

Linear Transformation:
If T is linear transformation with matrix A, then:
|det(A)| = factor by which T scales areas/volumes

Examples:
- det(A) = 2: transformation doubles all areas
- det(A) = 0.5: transformation halves all areas
- det(A) = -1: transformation preserves area but reverses orientation

Invertibility:
Matrix A is invertible if and only if det(A) 0
- det(A) = 0: matrix is singular (not invertible)
- det(A) 0: matrix is non-singular (invertible)

Applications:
- Testing linear independence of vectors
- Solving systems using Cramer's rule
- Computing cross products in higher dimensions
- Change of variables in integration

Matrix Inverse

Definition and Properties

The matrix inverse generalizes the concept of reciprocal to matrices.

```
Matrix Inverse Definition


For square matrix A, inverse A ¹ satisfies:
AA ¹ = A ¹A = I

Existence Condition:
A ¹ exists if and only if det(A)   0

2×2 Inverse Formula:
A = [a  b]    A ¹ = 1/(ad-bc) [ d  -b]
    [c  d]                    [-c   a]

Example:
A = [3  2]
    [1  4]
det(A) = 12 - 2 = 10

A ¹ = 1/10 [ 4  -2] = [ 0.4  -0.2]
           [-1   3]   [-0.1   0.3]

Verification:
AA ¹ = [3  2][ 0.4  -0.2] = [1  0]
       [1  4][-0.1   0.3]   [0  1] = I

Properties:
- (A ¹) ¹ = A
- (AB) ¹ = B ¹A ¹ (order reverses!)
- (A ) ¹ = (A ¹)
- det(A ¹) = 1/det(A)
- If A is invertible, then Ax = b has unique solution x = A ¹b
```

Computing Matrix Inverse

```
Methods for Finding Inverse


Method 1: Gauss-Jordan Elimination
Set up augmented matrix [A|I] and reduce to [I|A ¹]
```

```
Example: Find inverse of A = [2  1]
                             [1  1]

[A|I] = [2  1 | 1  0]
        [1  1 | 0  1]

R   R : [1  1 | 0  1]
         [2  1 | 1  0]

R  - 2R : [1  1 | 0  1]
           [0 -1 | 1 -2]

-R :      [1  1 | 0  1]
           [0  1 |-1  2]

R  - R :  [1  0 | 1 -1]
           [0  1 |-1  2]

Therefore: A ¹ = [ 1 -1]
                 [-1  2]

Method 2: Adjugate Formula (for small matrices)
A ¹ = (1/det(A)) × adj(A)
where adj(A) is adjugate (transpose of cofactor matrix)

Method 3: LU Decomposition
For larger matrices, factor A = LU and solve:
- Ly = b (forward substitution)
- Ux = y (backward substitution)

Computational Considerations:
- Direct inversion is expensive: O(n³) operations
- Often better to solve Ax = b directly
- Numerical stability issues for ill-conditioned matrices
- Use specialized algorithms for structured matrices
```

Systems of Linear Equations

Matrix Representation

Systems of linear equations can be compactly represented using matrices.

```
System Representation


General System:
```

```
a x  + a x  + ... + a x  = b
a x  + a x  + ... + a x  = b

a x  + a x  + ... + a x  = b
```

Matrix Form: Ax = b

```
where A = [a   a   ...  a ]  (coefficient matrix)
          [a   a   ...  a ]
          [               ]
          [a   a   ...  a ]

      x = [x ]  (variable vector)
          [x ]
          [  ]
          [x ]

      b = [b ]  (constant vector)
          [b ]
          [  ]
          [b ]
```

Augmented Matrix: [A|b]

Example:
2x + 3y = 7
x - y = 1

```
Matrix form: [2   3][x] = [7]
             [1  -1][y]   [1]

Augmented: [2   3 | 7]
           [1  -1 | 1]
```

Solution Methods

Matrix Solution Techniques

Method 1: Matrix Inverse (when A is square and invertible)
If Ax = b and A^{-1} exists, then $x = A^{-1}b$

Example:
[2 3][x] = [7]
[1 -1][y] [1]

A^{-1} = 1/(-5) [-1 -3] = [0.2 0.6]

```
              [-1   2]   [ 0.2  -0.4]

x = A⁻¹b = [ 0.2   0.6][7] = [2]
           [ 0.2  -0.4][1]   [1]

Solution: x = 2, y = 1

Method 2: Gaussian Elimination
Transform augmented matrix to row echelon form

[2   3 | 7]  R   R   [1  -1 | 1]
[1  -1 | 1]      → [2   3 | 7]

R  - 2R   [1  -1 | 1]
    → [0   5 | 5]

(1/5)R    [1  -1 | 1]
    → [0   1 | 1]

Back substitution:
From row 2: y = 1
From row 1: x - y = 1 → x = 2

Method 3: Cramer's Rule (for square systems with det(A)   0)
x  = det(A )/det(A)
where A  is A with column i replaced by b

Example:
A = [2   3]    A  = [7   3]    A  = [2   7]
    [1  -1]         [1  -1]         [1   1]

det(A) = -2 - 3 = -5
det(A ) = -7 - 3 = -10
det(A ) = 2 - 7 = -5

x = det(A )/det(A) = -10/(-5) = 2
y = det(A )/det(A) = -5/(-5) = 1
```

Solution Types and Consistency

```
System Classification


Consistent System: Has at least one solution
Inconsistent System: Has no solution

For m×n system Ax = b:
```

```
Case 1: m = n (square system)
- If det(A)   0: unique solution x = A ¹b
- If det(A) = 0: either no solution or infinitely many

Case 2: m < n (underdetermined system)
- More variables than equations
- If consistent: infinitely many solutions
- Solution space has dimension n - rank(A)

Case 3: m > n (overdetermined system)
- More equations than variables
- Usually inconsistent (no exact solution)
- Can find least squares solution

Rank and Consistency:
System Ax = b is consistent if and only if:
rank(A) = rank([A|b])

Homogeneous Systems: Ax = 0
- Always consistent (x = 0 is always a solution)
- If det(A)   0: only trivial solution x = 0
- If det(A) = 0: infinitely many solutions

Example of Inconsistent System:
x + y = 1
x + y = 2

Augmented matrix: [1  1 | 1]
                  [1  1 | 2]

R  - R : [1  1 | 1]
         [0  0 | 1]

Last row represents 0 = 1, which is impossible.
System is inconsistent.
```

Elementary Matrices and Row Operations

Elementary Row Operations

```
Three Types of Row Operations


Type 1: Row Interchange
R   R  (swap rows i and j)
```

```
Type 2: Row Scaling
kR → R  (multiply row i by nonzero constant k)

Type 3: Row Addition
R + kR → R  (add k times row j to row i)

Elementary Matrices:
Each row operation corresponds to multiplying by an elementary matrix

Type 1 Elementary Matrix (swap rows 1 and 2 in 3×3):
E = [0  1  0]
    [1  0  0]
    [0  0  1]

Type 2 Elementary Matrix (multiply row 2 by k):
E = [1  0  0]
    [0  k  0]
    [0  0  1]

Type 3 Elementary Matrix (add k times row 2 to row 1):
E = [1  k  0]
    [0  1  0]
    [0  0  1]

Properties:
- All elementary matrices are invertible
- Elementary matrices are their own inverses (Type 1)
- Or have simple inverses (Types 2 and 3)
- Any invertible matrix is product of elementary matrices
```

Row Echelon Forms

```
Row Echelon Form (REF)


Properties:
1. All zero rows are at bottom
2. Leading entry (pivot) of each row is to right of pivot above
3. All entries below pivots are zero

Example:
[1  2  3  4]
[0  1  2  1]
[0  0  0  1]
[0  0  0  0]
```

```
Reduced Row Echelon Form (RREF):
Additional properties:
4. All pivots equal 1
5. All entries above and below pivots are zero

Example:
[1  0  1  0]
[0  1  2  0]
[0  0  0  1]
[0  0  0  0]

Uniqueness:
Every matrix has unique RREF
REF is not unique (depends on elimination choices)

Gauss-Jordan Elimination:
Process to transform matrix to RREF
1. Forward elimination (to REF)
2. Backward elimination (to RREF)

Applications:
- Solving systems of equations
- Finding matrix rank
- Determining linear independence
- Computing matrix inverse
```

Matrix Rank

Definition and Properties

```
Matrix Rank Definition


Rank of matrix A = maximum number of linearly independent:
- Row vectors (row rank)
- Column vectors (column rank)

Fundamental Theorem: Row rank = Column rank

Alternative Definitions:
- Number of pivots in REF
- Dimension of column space
- Dimension of row space

Examples:
A = [1  2  3]  →  REF: [1  2  3]
```

```
    [2  4  6]           [0  0  0]
    [1  2  3]           [0  0  0]

rank(A) = 1 (one pivot)

B = [1  0  2]  →  REF: [1  0  2]
    [0  1  3]           [0  1  3]
    [2  1  7]           [0  0  0]

rank(B) = 2 (two pivots)

Properties:
- 0  rank(A)  min(m,n) for m×n matrix
- rank(A) = rank(A )
- rank(AB)  min(rank(A), rank(B))
- rank(A + B)  rank(A) + rank(B)
- If A is invertible, rank(A) = n

Full Rank:
- m×n matrix has full rank if rank(A) = min(m,n)
- Square matrix: full rank  invertible  det(A)  0
```

Applications of Matrices

Computer Graphics and Transformations

```
2D Transformations


Rotation by angle  :
R( ) = [cos   -sin  ]
       [sin    cos  ]

Example: Rotate point (1,0) by 90°
R(90°) = [0  -1][1] = [0]
         [1   0][0]   [1]

Scaling by factors sx, sy:
S = [sx  0 ]
    [0   sy]

Reflection across x-axis:
Rx = [1   0]
     [0  -1]

Shear transformation:
```

```
H = [1  k]  (horizontal shear by factor k)
    [0  1]

Composition of Transformations:
Combined transformation = product of matrices
Order matters: T T  means apply T  first, then T

3D Transformations:
Rotation about z-axis:
Rz( ) = [cos    -sin    0]
        [sin     cos    0]
        [0       0      1]

Homogeneous Coordinates:
Represent 2D point (x,y) as 3D vector [x,y,1]
Allows translation as matrix multiplication:

Translation by (tx,ty):
T = [1  0  tx]
    [0  1  ty]
    [0  0  1 ]

Combined transformation pipeline:
Final = Projection × View × Model × vertex
```

Data Analysis and Statistics

```
Matrix Applications in Data Science


Data Matrix:
Rows = observations/samples
Columns = features/variables

Example: Student grades
        [Math  Science  English]
Alice   [85    92      78     ]
Bob     [76    88      85     ]
Carol   [92    85      90     ]

Covariance Matrix:
C = (1/(n-1))X X (for centered data)
Measures relationships between variables

Correlation Matrix:
Normalized covariance matrix
Entries between -1 and 1
```

```
Principal Component Analysis (PCA):
1. Center data (subtract means)
2. Compute covariance matrix C
3. Find eigenvalues and eigenvectors of C
4. Principal components = eigenvectors
5. Explained variance = eigenvalues

Linear Regression:
Model: y = X  +
Normal equation:   = (X X) ¹X y

Example: Predict house prices
X = [1  1200]  (1 for intercept, 1200 sq ft)
    [1  1500]
    [1  1800]

y = [200000]  (prices)
    [250000]
    [300000]

Solve for   = [intercept, price per sq ft]
```

Engineering Applications

```
Engineering Matrix Uses


Structural Analysis:
Stiffness matrix K relates forces F to displacements u:
Ku = F

Example: Spring system
K = [k +k   -k   ]  u = [u ]  F = [F ]
    [-k     k +k ]      [u ]      [F ]

Circuit Analysis:
Nodal analysis: YV = I
where Y is admittance matrix, V is voltage vector, I is current vector

Control Systems:
State-space representation:
ẋ = Ax + Bu  (state equation)
y = Cx + Du  (output equation)

A = system matrix
B = input matrix
```

```
C = output matrix
D = feedthrough matrix

Signal Processing:
Discrete Fourier Transform (DFT) as matrix multiplication:
X = Wx where W is DFT matrix

Finite Element Method:
Discretize continuous problems into matrix equations
[K]{u} = {F} where K is global stiffness matrix

Network Analysis:
Adjacency matrix represents graph connections
A[i,j] = 1 if edge from node i to node j, 0 otherwise

Markov Chains:
Transition matrix P where P[i,j] = probability of moving from state i to state j
Steady state:  P =
```

Advanced Matrix Concepts

Matrix Norms

```
Matrix Norms


Vector Norms Extended to Matrices:

Frobenius Norm:
||A||F = √(Σ  a ²) = √(trace(A A))

Example:
A = [1  2]
    [3  4]
||A||F = √(1² + 2² + 3² + 4²) = √30

Induced Norms:
||A||p = max{||Ax||p : ||x||p = 1}

Common Induced Norms:
- ||A||  = max column sum
- ||A||ϖ = max row sum
- ||A||  = largest singular value

Properties:
- ||A||  0, with equality iff A = 0
```

- ||cA|| = |c|||A||
- ||A + B|| ||A|| + ||B||
- ||AB|| ||A|||B||

Applications:
- Measuring matrix "size"
- Error analysis in numerical computations
- Convergence analysis of iterative methods
- Condition number: (A) = ||A|||A ¹||

Matrix Decompositions Preview

Important Decompositions

LU Decomposition:
A = LU (lower triangular × upper triangular)
Efficient for solving multiple systems with same A

QR Decomposition:
A = QR (orthogonal × upper triangular)
Used in least squares and eigenvalue algorithms

Singular Value Decomposition (SVD):
A = UΣV (orthogonal × diagonal × orthogonal)
Most general decomposition, works for any matrix

Eigenvalue Decomposition:
A = PDP ¹ (for diagonalizable matrices)
P contains eigenvectors, D contains eigenvalues

Cholesky Decomposition:
A = LL (for positive definite matrices)
Efficient for symmetric positive definite systems

Applications:
- Efficient equation solving
- Data compression and dimensionality reduction
- Principal component analysis
- Image processing and computer vision
- Numerical stability improvements

Summary and Key Concepts

Matrices provide the computational framework for linear algebra, enabling efficient representation and manipulation of linear transformations and systems of equations.

Chapter Summary

Essential Skills Mastered:
Matrix notation, types, and basic operations
Matrix multiplication and its properties
Transpose operations and symmetric matrices
Determinant calculation and geometric interpretation
Matrix inverse computation and applications
Systems of linear equations using matrices
Row operations and echelon forms
Matrix rank and its significance

Key Concepts:
• Matrices as rectangular arrays of numbers
• Matrix multiplication as composition of transformations
• Determinant as measure of scaling and invertibility
• Matrix inverse as generalization of reciprocal
• Row operations as elementary matrix multiplications
• Rank as measure of linear independence
• Applications in graphics, data analysis, and engineering

Fundamental Operations:
• Addition/subtraction: element-wise for same dimensions
• Multiplication: row-by-column with dimension matching
• Transpose: interchange rows and columns
• Determinant: scalar measure of matrix properties
• Inverse: multiplicative "reciprocal" when it exists

Problem-Solving Tools:
• Gaussian elimination for systems
• Matrix inverse for unique solutions
• Determinants for testing invertibility
• Rank for analyzing solution spaces
• Elementary matrices for understanding row operations

Applications Covered:
• Computer graphics transformations
• Data analysis and statistics
• Engineering systems and networks
• Circuit analysis and control systems
• Structural analysis and finite elements

Next Steps:
Matrix concepts prepare you for:
- Eigenvalues and eigenvectors
- Vector spaces and linear transformations
- Matrix decompositions and factorizations

- Numerical linear algebra methods
- Advanced applications in machine learning and data science

Matrices represent the computational heart of linear algebra, providing both theoretical foundations and practical tools for solving real-world problems. The skills developed in this chapter - matrix operations, system solving, and geometric interpretation - form essential building blocks for advanced topics in linear algebra and its applications across science, engineering, and data analysis. Understanding matrices opens the door to powerful computational methods and elegant mathematical insights that drive modern technology and scientific discovery. ## Matrix as Linear Transformation

A matrix is not just a table of numbers; it is a transformation from one vector space to another.

```
flow:
  A -> Y
```

Example transformation: `A = [[2,0],[0,1]]` scales x-axis by 2 and leaves y-axis unchanged.

Determinant as Area/Volume Scaling

For 2x2 matrix `A`, `|det(A)|` is area scale factor.

- `det(A)=0`: collapses area to line/point (non-invertible)
- `det(A)<0`: includes orientation flip

Quick example: `A = [[3,0],[0,2]]` -> unit square area `1` becomes rectangle area `6`.

Systems of Linear Equations: Solving Multiple Relationships Simultaneously

Introduction to Linear Systems

A system of linear equations consists of multiple linear equations involving the same set of variables. These systems arise naturally when modeling real-world situations with multiple constraints and relationships.

System Structure

General Form:
a x + a x + ... + a x = b
a x + a x + ... + a x = b

a x + a x + ... + a x = b

Matrix Representation: Ax = b
where:
A = coefficient matrix (m×n)
x = variable vector (n×1)
b = constant vector (m×1)

System Classifications:
• m = n: Square system (same number of equations and variables)
• m < n: Underdetermined (fewer equations than variables)
• m > n: Overdetermined (more equations than variables)

Examples:

2×2 System:	3×2 System:	2×3 System:
2x + 3y = 7	x + y = 3	x + 2y + z = 4
x - y = 1	2x - y = 1	2x - y + 3z = 1
	x + 2y = 5	

Homogeneous vs. Nonhomogeneous:
• Homogeneous: Ax = 0 (all constants are zero)
• Nonhomogeneous: Ax = b (at least one constant is nonzero)

Solution Types and Geometric Interpretation

Understanding Solution Behavior

```
Solution Classifications


Three Possibilities:
1. Unique Solution: Exactly one solution vector
2. No Solution: System is inconsistent
3. Infinite Solutions: System has infinitely many solutions

2D Geometric Interpretation:
Each equation represents a line in the plane

Unique Solution:
Two lines intersect at exactly one point
Example: x + y = 3, x - y = 1
Solution: (2, 1)

No Solution:
Lines are parallel (never intersect)
Example: x + y = 3, x + y = 5
Contradiction: same slope, different intercepts

Infinite Solutions:
Lines are identical (same line)
Example: x + y = 3, 2x + 2y = 6
Second equation is multiple of first

3D Geometric Interpretation:
Each equation represents a plane in 3D space

Unique Solution:
Three planes intersect at exactly one point
Common intersection point

No Solution:
Planes have no common intersection
Example: parallel planes or triangular prism configuration

Infinite Solutions:
Planes intersect along a line or are identical
Line of intersection or coincident planes

Higher Dimensions:
Each equation represents a hyperplane in n-dimensional space
```

```
Solution is intersection of all hyperplanes
```

Consistency and Rank

```
Consistency Conditions


Fundamental Theorem:
System Ax = b is consistent if and only if:
rank(A) = rank([A|b])

where [A|b] is the augmented matrix

Rank Analysis:
Let r = rank(A), n = number of variables

Case 1: rank(A) = rank([A|b]) = r
System is consistent

- If r = n: Unique solution
- If r < n: Infinite solutions (n - r free variables)

Case 2: rank(A) < rank([A|b])
System is inconsistent (no solution)

Examples:
Consistent with unique solution:
[1  2 | 3]  rank(A) = rank([A|b]) = 2 = n
[3  1 | 4]

Consistent with infinite solutions:
[1  2  1 | 3]  rank(A) = rank([A|b]) = 2 < n = 3
[2  4  3 | 7]

Inconsistent:
[1  2 | 3]  rank(A) = 1, rank([A|b]) = 2
[2  4 | 7]

Rouché-Capelli Theorem:
Provides complete characterization of solution existence
Essential for theoretical analysis
```

Gaussian Elimination

The Elimination Process

Gaussian elimination systematically transforms a system into an equivalent system that's easier to solve.

```
Gaussian Elimination Steps


Goal: Transform augmented matrix to row echelon form

Elementary Row Operations:
1. R   R : Swap rows i and j
2. kR  → R : Multiply row i by nonzero constant k
3. R  + kR  → R : Add k times row j to row i

Forward Elimination Process:
1. Choose pivot (leftmost nonzero entry in current row)
2. Use row swaps to move pivot to diagonal position
3. Use row operations to make all entries below pivot zero
4. Move to next row and repeat

Example: Solve system
x + 2y + 3z = 14
2x + 3y + z = 11
3x + y + 2z = 13

Augmented matrix:
[1  2  3 | 14]
[2  3  1 | 11]
[3  1  2 | 13]

Step 1: Eliminate below first pivot
R  - 2R : [1  2  3 | 14]
          [0 -1 -5 |-17]
          [3  1  2 | 13]

R  - 3R : [1  2  3 | 14]
          [0 -1 -5 |-17]
          [0 -5 -7 |-29]

Step 2: Eliminate below second pivot
-R :      [1  2  3 | 14]
          [0  1  5 | 17]
          [0 -5 -7 |-29]
```

```
R  + 5R : [1  2  3 | 14]
          [0  1  5 | 17]
          [0  0 18 | 56]

Row Echelon Form achieved!
```

Back Substitution

```
Back Substitution Process


After forward elimination, solve from bottom up:

From our example:
[1  2  3 | 14]
[0  1  5 | 17]
[0  0 18 | 56]

Step 1: Solve for z from last equation
18z = 56
z = 56/18 = 28/9

Step 2: Substitute into second equation
y + 5z = 17
y + 5(28/9) = 17
y = 17 - 140/9 = 153/9 - 140/9 = 13/9

Step 3: Substitute into first equation
x + 2y + 3z = 14
x + 2(13/9) + 3(28/9) = 14
x + 26/9 + 84/9 = 14
x = 14 - 110/9 = 126/9 - 110/9 = 16/9

Solution: x = 16/9, y = 13/9, z = 28/9

Verification:
Substitute back into original equations to check
```

Gauss-Jordan Elimination

Reduced Row Echelon Form

Gauss-Jordan elimination extends Gaussian elimination to produce the reduced row echelon form (RREF).

RREF Properties

Reduced Row Echelon Form has:
1. All properties of row echelon form
2. All pivot entries equal 1 (leading 1's)
3. All entries above and below pivots are 0

RREF Process:
1. Perform forward elimination (Gaussian)
2. Make all pivots equal to 1
3. Eliminate above pivots (backward elimination)

Example: Continue from previous REF
[1 2 3 | 14]
[0 1 5 | 17]
[0 0 18 | 56]

Step 1: Make pivots equal to 1
(1/18)R : [1 2 3 | 14]
          [0 1 5 | 17]
          [0 0 1 | 28/9]

Step 2: Eliminate above third pivot
R - 5R : [1 2 3 | 14]
          [0 1 0 | 13/9]
          [0 0 1 | 28/9]

R - 3R : [1 2 0 | 16/9]
          [0 1 0 | 13/9]
          [0 0 1 | 28/9]

Step 3: Eliminate above second pivot
R - 2R : [1 0 0 | 16/9]
          [0 1 0 | 13/9]
          [0 0 1 | 28/9]

Solution directly readable: x = 16/9, y = 13/9, z = 28/9

Advantages of RREF:
- Solution immediately visible
- Easier to identify free variables
- Systematic approach for all cases
- Useful for finding matrix inverse

Parametric Solutions

```
Systems with Infinite Solutions


When system has infinite solutions, RREF reveals the structure:

Example: Solve system
x + 2y + z = 3
2x + 4y + 3z = 7
x + 2y + 2z = 4

Augmented matrix:
[1  2  1 | 3]
[2  4  3 | 7]
[1  2  2 | 4]

Forward elimination:
R  - 2R : [1  2  1 | 3]
          [0  0  1 | 1]
          [1  2  2 | 4]

R  - R :  [1  2  1 | 3]
          [0  0  1 | 1]
          [0  0  1 | 1]

R  - R :  [1  2  1 | 3]
          [0  0  1 | 1]
          [0  0  0 | 0]

RREF:
R  - R :  [1  2  0 | 2]
          [0  0  1 | 1]
          [0  0  0 | 0]

Interpretation:
x + 2y = 2  →  x = 2 - 2y
z = 1

y is a free variable (can be any real number)

Parametric Solution:
Let y = t (parameter)
x = 2 - 2t
y = t
z = 1

Vector Form:
```

```
[x]   [2]     [-2]
[y] = [0] + t [1]
[z]   [1]     [0]

General solution = particular solution + homogeneous solution
```

Special Cases and Applications

Homogeneous Systems

```
Homogeneous Systems: Ax = 0


Properties:
- Always consistent (x = 0 is always a solution)
- Either unique solution (x = 0) or infinite solutions
- Solution set forms a subspace (null space of A)

Trivial vs. Nontrivial Solutions:
- Trivial solution: x = 0
- Nontrivial solutions: x   0

Existence of Nontrivial Solutions:
For square matrix A (n×n):
- det(A)   0: Only trivial solution
- det(A) = 0: Infinite nontrivial solutions

For general matrix A (m×n):
- rank(A) = n: Only trivial solution
- rank(A) < n: Infinite nontrivial solutions

Example: Find nontrivial solutions
x + 2y - z = 0
2x + 3y + z = 0
x + y + 2z = 0

Augmented matrix (b = 0, so we work with A only):
[1  2 -1]
[2  3  1]
[1  1  2]

Row reduction:
[1  2 -1]
[0 -1  3]
[0 -1  3]
```

```
[1  2 -1]
[0  1 -3]
[0  0  0]

[1  0  5]
[0  1 -3]
[0  0  0]

Solution:
x + 5z = 0  →  x = -5z
y - 3z = 0  →  y = 3z
z is free

Parametric solution: x = -5t, y = 3t, z = t
Vector form: t[-5, 3, 1]

Null space is line through origin in direction [-5, 3, 1]
```

Underdetermined Systems

```
More Variables Than Equations


Characteristics:
- m < n (fewer equations than variables)
- If consistent, always has infinite solutions
- At least n - m free variables

Example: 2 equations, 3 variables
x + 2y + z = 5
2x - y + 3z = 1

Augmented matrix:
[1  2  1 | 5]
[2 -1  3 | 1]

Row reduction:
[1  2  1 | 5]
[0 -5  1 |-9]

[1  2  1 | 5]
[0  1 -1/5| 9/5]

[1  0  7/5 | 7/5]
[0  1 -1/5 | 9/5]

Solution:
```

```
x + (7/5)z = 7/5  →  x = 7/5 - (7/5)z
y - (1/5)z = 9/5  →  y = 9/5 + (1/5)z
z is free

Parametric solution:
x = 7/5 - (7/5)t
y = 9/5 + (1/5)t
z = t

Applications:
- Optimization problems with constraints
- Curve fitting with fewer data points than parameters
- Control systems with multiple actuators
```

Overdetermined Systems

```
More Equations Than Variables


Characteristics:
- m > n (more equations than variables)
- Usually inconsistent (no exact solution)
- When consistent, typically unique solution

Example: 3 equations, 2 variables
x + y = 3
2x - y = 0
x + 2y = 5

Augmented matrix:
[1  1 | 3]
[2 -1 | 0]
[1  2 | 5]

Row reduction:
[1  1 | 3]
[0 -3 |-6]
[0  1 | 2]

[1  1 | 3]
[0  1 | 2]
[0  1 | 2]

[1  0 | 1]
[0  1 | 2]
[0  0 | 0]
```

```
This system is consistent!
Solution: x = 1, y = 2

Verification:
1 + 2 = 3
2(1) - 2 = 0
1 + 2(2) = 5

Inconsistent Example:
x + y = 3
2x - y = 0
x + 2y = 6

Row reduction leads to:
[1  0 | 1]
[0  1 | 2]
[0  0 | 1]  ← Inconsistent row!

Last row represents 0 = 1, which is impossible.

Least Squares Solution:
When overdetermined system is inconsistent, find x that minimizes ||Ax - b||²
Solution: x = (A A) ¹A b (normal equation)
```

Matrix Methods for Linear Systems

Using Matrix Inverse

```
Inverse Method for Square Systems


For square system Ax = b where A is invertible:
Solution: x = A ¹b

Example:
[2  1][x] = [7]
[1  3][y]   [8]

Find A ¹:
det(A) = 2(3) - 1(1) = 5

A ¹ = (1/5)[3  -1] = [3/5  -1/5]
           [-1   2]   [-1/5  2/5]

Solution:
x = A ¹b = [3/5  -1/5][7] = [21/5 - 8/5] = [13/5]
```

```
           [-1/5  2/5][8]   [-7/5 + 16/5]   [9/5]

Therefore: x = 13/5, y = 9/5

Computational Considerations:
- Computing A⁻¹ is expensive: O(n³) operations
- Numerically unstable for ill-conditioned matrices
- Usually better to solve Ax = b directly
- Useful when solving multiple systems with same A

When to Use Inverse Method:
  Multiple right-hand sides with same coefficient matrix
  Theoretical analysis and derivations
  Small systems where inverse has simple form
  Large systems (use elimination instead)
  Ill-conditioned systems (numerical instability)
```

Cramer's Rule

```
Cramer's Rule for Square Systems


For n×n system Ax = b with det(A)   0:

x  = det(A )/det(A)

where A  is matrix A with column i replaced by vector b

Example:
2x + 3y = 7
x - y = 1

A = [2   3]    b = [7]
    [1  -1]        [1]

det(A) = 2(-1) - 3(1) = -5

For x (replace column 1):
A  = [7   3]    det(A ) = 7(-1) - 3(1) = -10
     [1  -1]

x = det(A )/det(A) = -10/(-5) = 2

For y (replace column 2):
A  = [2  7]     det(A ) = 2(1) - 7(1) = -5
     [1  1]
```

```
y = det(A )/det(A) = -5/(-5) = 1

Solution: x = 2, y = 1

3×3 Example:
x + 2y + z = 6
2x + y + 2z = 8
x + y + 3z = 9

A = [1  2  1]    det(A) = 1(3-2) - 2(6-2) + 1(2-1) = -6
    [2  1  2]
    [1  1  3]

A  = [6  2  1]   det(A ) = 6(3-2) - 2(24-18) + 1(8-9) = -11
     [8  1  2]
     [9  1  3]

x = -11/(-6) = 11/6

Limitations:
- Only works for square systems with det(A)   0
- Computationally expensive for large systems
- Requires n+1 determinant calculations
- Less efficient than elimination for n > 3
```

Applications and Real-World Examples

Economic Models

```
Economic System Example


Supply and Demand Model:
Market 1: S  = 2P  - P  + 10, D  = -P  + 20
Market 2: S  = -P  + 3P  + 5,  D  = -2P  + 25

Equilibrium conditions: S  = D , S  = D

System:
2P  - P  + 10 = -P  + 20  →  3P  - P  = 10
-P  + 3P  + 5 = -2P  + 25  →  -P  + 5P  = 20

Matrix form:
[3  -1][P ] = [10]
[-1  5][P ]   [20]
```

```
Solution using elimination:
[3  -1 | 10]
[-1  5 | 20]

R  + (1/3)R : [3  -1 | 10]
              [0  14/3 | 70/3]

P  = (70/3)/(14/3) = 5
P  = (10 + 5)/3 = 5

Equilibrium prices: P  = $5, P  = $5

Input-Output Model (Leontief):
x = Ax + d
where x = output vector, A = technology matrix, d = final demand

Rearranging: (I - A)x = d
Solution: x = (I - A) ¹d

Example: Two-sector economy
Sector 1 uses 0.2 of its output and 0.3 from sector 2
Sector 2 uses 0.4 from sector 1 and 0.1 of its output
Final demands: d  = 100, d  = 200

A = [0.2  0.3]    I - A = [0.8  -0.3]
    [0.4  0.1]            [-0.4  0.9]

Solve: (I - A)x = d
```

Engineering Applications

```
Circuit Analysis Example


Kirchhoff's Laws lead to linear systems:

Circuit with 3 loops:
Loop 1: 10I  - 5I  = 12
Loop 2: -5I  + 15I  - 8I  = 0
Loop 3: -8I  + 20I  = -8

Matrix form:
[10  -5   0][I ]   [12]
[-5  15  -8][I ] = [0]
[0   -8  20][I ]   [-8]

Structural Analysis:
```

```
Force equilibrium at joints creates linear systems

Truss with 3 joints:
Joint 1: F cos(30°) + F  = P
         F sin(30°) = P
Joint 2: -F cos(30°) + F cos(45°) = P
         -F sin(30°) - F sin(45°) = P

Chemical Engineering:
Material balance equations

Reactor system:
Input  + Input  = Output  + Output  + Accumulation
Component A: 0.3F  + 0.1F  = 0.2F  + 0.4F
Component B: 0.7F  + 0.9F  = 0.8F  + 0.6F
Total: F  + F  = F  + F

Traffic Flow:
Conservation of vehicles at intersections

Intersection analysis:
Inflow = Outflow at each node
x  + x  = x  + x  (node 1)
x  + x  = x  + x  (node 2)
...
```

Data Fitting and Regression

```
Linear Regression as System Solution


Problem: Fit line y = mx + b to data points
(x ,y ), (x ,y ), ..., (x ,y )

Overdetermined system:
mx  + b = y
mx  + b = y

mx  + b = y

Matrix form: A  = y
where A = [x   1]      = [m]    y = [y ]
          [x   1]        [b]        [y ]
          [    ]                   [ ]
          [x   1]                  [y ]

Normal equation solution:
```

$= (A^T A)^{-1} A^T y$

Example: Fit line to points (1,2), (2,3), (3,5), (4,4)

```
A = [1  1]    y = [2]
    [2  1]        [3]
    [3  1]        [5]
    [4  1]        [4]

A^T A = [1 2 3 4][1  1] = [30  10]
        [1 1 1 1][2  1]   [10   4]
                 [3  1]
                 [4  1]

A^T y = [1 2 3 4][2] = [40]
        [1 1 1 1][3]   [14]
                 [5]
                 [4]

Solve: [30  10][m] = [40]
       [10   4][b]   [14]

Solution: m = 0.7, b = 1.25
Best-fit line: y = 0.7x + 1.25
```

Polynomial Fitting:
For polynomial $y = a_0 + a_1 x + a_2 x^2 + \ldots + a_n x^n$

Vandermonde matrix:

$$A = \begin{bmatrix} 1 & x_1 & x_1^2 & \ldots & x_1^n \\ 1 & x_2 & x_2^2 & \ldots & x_2^n \\ \vdots & & & & \vdots \\ 1 & x_m & x_m^2 & \ldots & x_m^n \end{bmatrix}$$

Same normal equation approach applies

Computational Considerations

Numerical Stability

Numerical Issues in Linear Systems

Ill-Conditioned Systems:
Small changes in coefficients cause large changes in solution

```
Example: Nearly singular system
[1.0000  1.0000][x] = [2.0000]
[1.0000  1.0001][y]   [2.0001]

Exact solution: x = 1, y = 1

Perturbed system:
[1.0000  1.0000][x] = [2.0000]
[1.0000  1.0001][y]   [2.0002]

Solution: x = 0, y = 2 (completely different!)

Condition Number:
 (A) = ||A|||A 1||
-  (A)   1: well-conditioned
-  (A) >> 1: ill-conditioned

Pivoting Strategies:
Partial pivoting: Choose largest element in column as pivot
Complete pivoting: Choose largest element in remaining submatrix

Benefits:
- Reduces round-off error accumulation
- Improves numerical stability
- Essential for practical implementations

Example with pivoting:
Original:     [0.001  1][x] = [1]
              [1      1][y]   [2]

Without pivoting: x   1000, y   -999 (inaccurate)

With row swap:  [1      1][x] = [2]
                [0.001  1][y]   [1]

Accurate solution: x = 1, y = 1

Iterative Methods:
For large sparse systems, iterative methods may be preferred:
- Jacobi method
- Gauss-Seidel method
- Conjugate gradient method
- GMRES method

Advantages:
- Lower memory requirements
- Better for sparse matrices
- Can be stopped when desired accuracy reached
```

Computational Complexity

```
Algorithm Efficiency Analysis


Gaussian Elimination:
Time complexity: O(n³) for n×n system
Space complexity: O(n²)

Operations count:
- Forward elimination: ~n³/3 multiplications
- Back substitution: ~n²/2 multiplications
- Total: ~n³/3 operations

Matrix Inverse:
Time complexity: O(n³)
Generally more expensive than direct solving

Cramer's Rule:
Time complexity: O(n! × n) (factorial growth!)
Impractical for n > 4

LU Decomposition:
One-time cost: O(n³)
Each solve: O(n²)
Efficient for multiple right-hand sides

Sparse Matrices:
Special techniques for matrices with many zeros:
- Store only nonzero elements
- Specialized algorithms (sparse Gaussian elimination)
- Iterative methods often preferred
- Complexity depends on sparsity pattern

Parallel Computing:
Matrix operations can be parallelized:
- Block algorithms
- Distributed memory systems
- GPU acceleration
- Significant speedup possible for large systems

Memory Considerations:
- In-place algorithms to reduce memory usage
- Block algorithms for cache efficiency
- Out-of-core methods for very large systems
- Numerical libraries (LAPACK, BLAS) for optimized implementations
```

Summary and Key Concepts

Systems of linear equations provide the computational foundation for solving real-world problems involving multiple constraints and relationships simultaneously.

Chapter Summary

Essential Skills Mastered:
Classifying systems by solution type and geometry
Gaussian elimination and back substitution
Gauss-Jordan elimination and RREF
Handling homogeneous and parametric solutions
Matrix methods (inverse, Cramer's rule)
Applications in economics, engineering, and data analysis
Understanding numerical stability and computational complexity

Key Concepts:
- Systems as intersections of hyperplanes
- Consistency determined by rank conditions
- Three solution types: unique, none, infinite
- Elementary row operations preserve solutions
- RREF provides systematic solution method
- Parametric solutions for underdetermined systems
- Least squares for overdetermined systems

Solution Methods:
- Gaussian elimination: systematic and reliable
- Gauss-Jordan: produces RREF directly
- Matrix inverse: efficient for multiple right-hand sides
- Cramer's rule: theoretical importance, limited practical use
- Iterative methods: for large sparse systems

Problem-Solving Framework:
- Set up coefficient matrix and constant vector
- Analyze system type and expected solution behavior
- Choose appropriate solution method
- Interpret results in original problem context
- Verify solutions and check reasonableness

Applications Covered:
- Economic equilibrium and input-output models
- Circuit analysis and structural engineering
- Traffic flow and network problems
- Data fitting and regression analysis
- Chemical process balancing

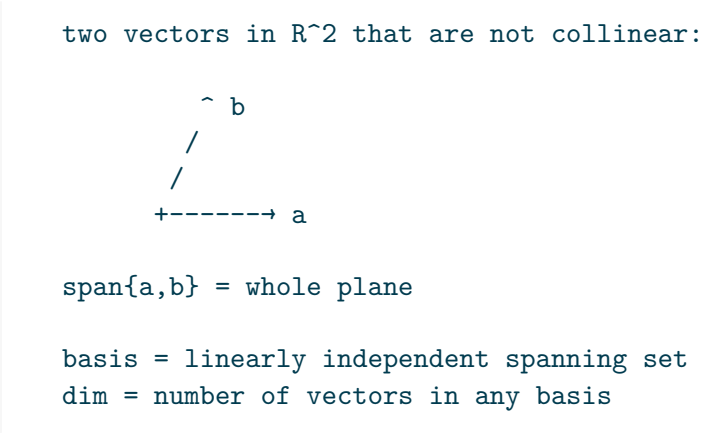
```
Next Steps:
Linear systems concepts prepare you for:
- Vector spaces and linear transformations
- Eigenvalue problems and matrix diagonalization
- Numerical linear algebra methods
- Optimization and linear programming
- Advanced applications in machine learning and data science
```

Systems of linear equations represent the practical heart of linear algebra, providing essential tools for modeling and solving real-world problems across science, engineering, and economics. The systematic methods developed in this chapter - elimination techniques, matrix approaches, and solution analysis - form the computational foundation for advanced topics in linear algebra and its applications. Understanding how to set up, solve, and interpret linear systems opens doors to powerful problem-solving capabilities that drive modern technology and scientific discovery.

Vector Spaces, Spans, and Bases

Beyond arrows in the plane: a **vector space** is any set where addition and scalar multiplication behave like vectors. **Bases** give coordinates; **dimension** is the size of a basis. This is the language of solutions to linear systems, signal spaces, and feature spaces in ML.

Diagram: span and basis



1. Vector space axioms (intuition)

A set V with addition $+$ and scalar multiplication by $\mathbb{R}$ (or $\mathbb{C}$) is a **vector space** if:

1. **Closure:** $u, v \in V$, $\alpha \in \mathbb{R}$ $\quad u + v \in V$ and $\alpha u \in V$
2. **Associativity / commutativity** of $+$
3. **Zero vector** $0 \in V$ with $u + 0 = u$
4. **Additive inverses** $-u$ with $u + (-u) = 0$
5. **Scalar rules:** $1 \cdot u = u$, $(\alpha\beta)u = \alpha(\beta u)$
6. **Distributivity:** $\alpha(u + v) = \alpha u + \alpha v$, $(\alpha + \beta)u = \alpha u + \beta u$

You do not need to memorize a theatrical list every time — check closure, zero, and inverses first; the rest usually follow for standard constructions.

Standard examples

Space	Vectors	Operations
$\mathbb{R}^n$	n-tuples	componentwise
$M_{m\times n}$	matrices	entrywise + / scalar
P_d	polynomials degree $\le d$	usual + / scalar
$C[0,1]$	continuous functions	$(f+g)(x) = f(x)+g(x)$
$\ker A$	solutions of $Ax = 0$	same as ambient

Non-examples

- Positive reals $\mathbb{R}_{>0}$ with ordinary + (no zero, no inverses in the set)
- Unit circle with ordinary vector addition (not closed)
- $\{(x, y) : xy = 1\}$ (hyperbola; not closed under +)

Worked example 1 — function space

$f(x) = x^2$ and $g(x) = \sin x$ live in a huge vector space of functions; $3f - 2g$ is another function. Coordinates need a basis (e.g. monomials, Fourier) — infinite-dimensional territory.

2. Subspaces

$W \subseteq V$ is a **subspace** if W itself is a vector space under the same operations. Practical test:

1. $0 \in W$
2. Closed under + and scalar multiplication

Equivalent: closed under all linear combinations $\alpha u + \beta v$.

Fundamental subspaces of a matrix $A \in \mathbb{R}^{m\times n}$

Name	Definition	Ambient
Column space $\mathrm{col}(A)$	span of columns	$\mathbb{R}^m$
Row space $\mathrm{row}(A)$	span of rows	$\mathbb{R}^n$
Nullspace / kernel $\ker(A)$	$\{x : Ax = 0\}$	$\mathbb{R}^n$
Left nullspace $\ker(A^\top)$	$\{y : A^\top y = 0\}$	$\mathbb{R}^m$

Worked example 2 — plane through origin

$W = \{(x, y, z) : x + y + z = 0\}$ is a subspace of $\mathbb{R}^3$.
$W' = \{(x, y, z) : x + y + z = 1\}$ is **not** (misses 0; affine plane).

3. Linear combinations and span

$$\text{span}\{v_1, \dots, v_k\} = \{c_1 v_1 + \cdots + c_k v_k : c_i \in \mathbb{R}\}.$$

Span is always a subspace (the smallest subspace containing the set).

```
span of one nonzero vector: a line through 0
span of two independent vectors in R^3: a plane through 0
span of three independent vectors in R^3: all of R^3
```

Worked example 3

$\text{span}\{(1,0),(0,1)\} = \mathbb{R}^2$.
$\text{span}\{(1,1),(2,2)\} =$ the line $y = x$.

4. Linear independence

$\{v_1, \dots, v_k\}$ is **linearly independent** if

$$c_1 v_1 + \cdots + c_k v_k = 0 \implies c_1 = \cdots = c_k = 0.$$

Otherwise the set is **dependent**: some vector is a linear combination of the others (redundant for spanning).

Tests

- Form matrix V with those columns; independent iff $Vx = 0$ has only $x = 0$ iff columns have pivot in every column (full column rank).
- In $\mathbb{R}^n$, more than n vectors always dependent.
- Orthogonal nonzero vectors are independent.

Worked example 4

$v_1 = (1,0,1)$, $v_2 = (0,1,1)$, $v_3 = (1,1,2)$ in $\mathbb{R}^3$.
$v_3 = v_1 + v_2$ dependent. $\{v_1, v_2\}$ independent (not scalar multiples).

Worked example 5

$\{(1,1),(2,2)\}$ in $\mathbb{R}^2$: $2\cdot(1,1)+(-1)\cdot(2,2)=0$ dependent. Span is still the line $y=x$.

5. Basis and dimension

A **basis** of V is a linearly independent spanning set.

Theorem (finite dimension). If V has a finite spanning set, then:

- Every basis has the same number of vectors, called $\dim V$
- Any independent set of size $\dim V$ is a basis
- Any spanning set of size $\dim V$ is a basis
- Independent sets extend to bases; spanning sets can be thinned to bases

Space	Typical basis	dim
$\mathbb{R}^n$	standard e_i	n
P_2	$\{1, x, x^2\}$	3
$\{x+y+z=0\}$ in $\mathbb{R}^3$	e.g. $(1,-1,0),(1,0,-1)$	2
$\operatorname{col}\begin{bmatrix}1&2\\2&4\end{bmatrix}$	e.g. $(1,2)$	1

Coordinates

If $\mathcal{B}=\{b_1,\dots,b_n\}$ is a basis, every v has **unique** coordinates c with $v=\sum c_i b_i$. Changing basis multiplies coordinates by a transition matrix — core of "change of coordinates" in graphics and spectral methods.

Worked example 6 — basis of a plane

For $x+y+z=0$: free variables $y=s$, $z=t$, then $x=-s-t$:

$$\begin{bmatrix}x\\y\\z\end{bmatrix}=s\begin{bmatrix}-1\\1\\0\end{bmatrix}+t\begin{bmatrix}-1\\0\\1\end{bmatrix}.$$

Those two vectors form a basis; dim = 2.

6. Rank–nullity theorem

For $A \in \mathbb{R}^{m \times n}$ (linear map $x \mapsto Ax$):

$$\operatorname{rank}(A) + \operatorname{nullity}(A) = n,$$

where $\operatorname{rank}(A) = \dim \operatorname{col}(A)$ and $\operatorname{nullity}(A) = \dim \ker(A)$.

```
domain R^n

      A
   v
R^m
ker A  dim   nullity
col A  dim   rank
rank + nullity = n
```

Also: $\operatorname{rank}(A) = \dim \operatorname{row}(A) = \dim \operatorname{col}(A)$.

Worked example 7

$A = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$: columns multiples rank 1.
$\ker A$: $x + 2y = 0$, basis $(2, -1)$, nullity 1.
$1 + 1 = 2 = n$.

Worked example 8 — underdetermined systems

If A is 3×5 with rank 3, nullity 2: solution space to $Ax = b$ (if nonempty) is an affine translate of a 2D nullspace — two free parameters.

7. Four fundamental subspaces (preview of structure)

For $A \in \mathbb{R}^{m \times n}$:

$$\mathbb{R}^n = \operatorname{row}(A) \oplus \ker(A),$$
$$\mathbb{R}^m = \operatorname{col}(A) \oplus \ker(A^\top),$$

with orthogonal complements under the standard dot product. This is Strang's diagram and underlies least squares: b splits into $\operatorname{col}(A)$ part (fit) and left-null part (residual).

8. Linear maps and matrices

A map $T : V \to W$ is **linear** if $T(\alpha u + \beta v) = \alpha T(u) + \beta T(v)$.

Choosing bases of V and W represents T by a matrix A. Changing bases conjugates A (similarity when $V = W$). Dimension theorems above are coordinate-free facts about T.

9. CS / ML map

Idea	Linear algebra view
Feature space	points as vectors in $\mathbb{R}^d$
One-hot categories	standard basis vectors
Embeddings	coordinates in a learned frame
Null space	non-identifiability / gauge freedom
Column space	reachable predictions of a linear model Xw
Rank of data matrix	effective dimensionality before noise
Word co-occurrence	rows as vectors; rank reduction PCA/SVD

Worked example 9 — collinear features

If two feature columns of X are multiples, $\operatorname{rank}(X) < \#\text{features}$, $\ker(X)$ nontrivial: infinitely many w give the same predictions Xw — regularization picks one.

10. Infinite dimensions (awareness)

Function spaces, ℓ^2 sequences, RKHS in kernel methods: bases can be infinite (Hilbert bases). Finite-dimensional intuition (rank–nullity as stated) needs care; still, "span," "independence," and "coordinates in a dictionary" remain the right vocabulary.

11. Pitfalls

1. Calling any spanning set a basis (must also be independent)
2. Forgetting subspaces must contain 0
3. Confusing affine spaces (solutions to $Ax = b$) with subspaces ($Ax = 0$)
4. Thinking dimension is the ambient n rather than $\dim W$
5. More vectors "more independent" — false beyond ambient dimension

12. Checkpoint

- Test subspace membership
- Decide independence via $Vx = 0$
- Find a basis for a simple nullspace
- Apply rank–nullity
- Connect column space to what a linear model can represent

Exercises

Easy

1. Prove $\{0\}$ is a subspace of any V.
2. Are $\{(1,1),(2,2)\}$ independent in $\mathbb{R}^2$? What is their span?
3. Find a basis for $\{(x,y,z) : x+y+z=0\}$.
4. Show that if more than n vectors sit in $\mathbb{R}^n$, they are dependent.
5. For $A = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$, find rank and a basis for $\ker A$.

Medium

6. Prove that the intersection of two subspaces is a subspace. Is the union?
7. Show $\operatorname{col}(A) = \operatorname{col}(AE)$ if E is invertible (same column space after right-multiplication by invertible).
8. Find bases for all four fundamental subspaces of $A = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$.
9. Prove that any independent set in a finite-dimensional space can be extended to a basis (outline).
10. If $T : \mathbb{R}^n \to \mathbb{R}^m$ is linear and injective, what is $\dim \ker T$?

Challenge

11. Prove $\dim(U+W) = \dim U + \dim W - \dim(U \cap W)$ for subspaces of a finite-dimensional space.

12. Show row rank equals column rank without quoting the theorem name (use RREF structure).

13. Polynomials: show $\{1, x, x^2, ...\}$ is independent in the space of all polynomials.

14. Connect rank–nullity to "number of free variables after elimination."

15. Data matrix $X \in \mathbb{R}^{n \times d}$ with $n < d$ and full row rank: describe $\ker(X)$ dimension and implication for overparameterized least squares.

Checks

2. Dependent; span is the line $y = x$.

3. Rank 1; ker spanned by $(2, -1)$.

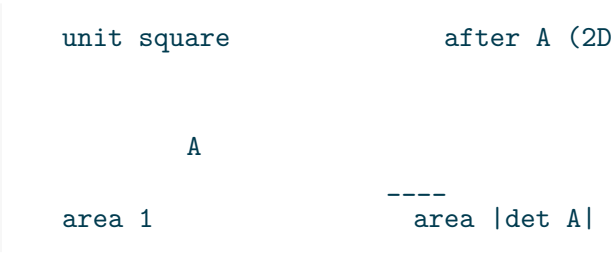
Summary

Vector spaces abstract the algebra of linear combinations. Spans describe reachable sets; independence removes redundancy; bases provide coordinates; dimension is basis size. Rank–nullity balances degrees of freedom in domain against constraints. For CS readers: features, kernels, and linear models are subspace stories — identify the ambient space, the subspace of interest, and a basis when you need coordinates.

Determinants, Inverses, and Volume

The **determinant** $\det A$ is a scalar measuring oriented volume scaling under the linear map A. It decides invertibility and appears in change-of-variables formulas, multivariate Gaussians, and geometric algorithms.

Diagram: area scaling



1. Invertibility criteria

For square $A \in \mathbb{R}^{n \times n}$, the following are equivalent (**TFAE**):

1. A is invertible (exists B with $AB = BA = I$)
2. $\det A \neq 0$
3. Columns form a basis of $\mathbb{R}^n$ (linearly independent)
4. Rows form a basis of $\mathbb{R}^n$
5. $\operatorname{rank}(A) = n$
6. $\ker(A) = \{0\}$
7. 0 is not an eigenvalue of A
8. $Ax = b$ has a unique solution for every b

Inverse: $A^{-1}A = I = AA^{-1}$ when invertible.

Worked example 1 — 2×2 inverse

$A = \begin{bmatrix} 2 & 1 \\ 0 & 3 \end{bmatrix}$, $\det A = 6 \neq 0$.

$$A^{-1} = \frac{1}{6}\begin{bmatrix} 3 & -1 \\ 0 & 2 \end{bmatrix} = \begin{bmatrix} 1/2 & -1/6 \\ 0 & 1/3 \end{bmatrix}.$$

Verify $AA^{-1} = I$.

Worked example 2 — singular

$A = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$, $\det = 0$, columns collinear, ker spanned by $(2, -1)$, not invertible.

2. Computing determinants

2×2

$$\det \begin{bmatrix} a & b \\ c & d \end{bmatrix} = ad - bc.$$

3×3 **(cofactor expansion along row 1)**

$$\det \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix} = a(ei - fh) - b(di - fg) + c(dh - eg).$$

Via row reduction

Row-reduce A to upper triangular U with pivots $u_{11}, \dots, u_{nn}$:

$$\det A = (-1)^s \Big(\prod_i u_{ii}\Big) \Big/ (\text{scale factors}),$$

more carefully tracking:

Row operation	Effect on det
Swap two rows	multiply by -1
Scale a row by k	multiply by k
Add multiple of one row to another	**unchanged**

Product of pivots (with sign from swaps) gives det if you only used swaps and shears (no row scaling), or adjust for scalings.

Multiplicativity and friends

$$\det(AB) = \det(A)\det(B), \qquad \det(A^\top) = \det(A), \qquad \det(A^{-1}) = \frac{1}{\det A} \ (A \text{ invertible}),$$

$$\det(cA) = c^n \det A, \qquad \det(A^{-1}BA) = \det B$$

(similarity preserves det — eigenvalues product invariant).

Worked example 3 — triangular

$$\det \begin{bmatrix} 1 & 2 & 0 \\ 0 & 3 & 4 \\ 0 & 0 & 5 \end{bmatrix} = 1 \cdot 3 \cdot 5 = 15.$$

Worked example 4 — product

If $\det A = 2$, $\det B = -3$, then

$$\det(A^2 B^{-1}) = \det(A)^2 \det(B)^{-1} = 4 \cdot \left(-\frac{1}{3}\right) = -\frac{4}{3}.$$

3. Geometric meaning

- $|\det A|$ = volume of the parallelepiped spanned by the columns of A
- $\det A > 0$: orientation-preserving; $\det A < 0$: orientation-reversing (includes a reflection component)
- $\det A = 0$: columns collapse volume to a lower-dimensional flat
- Orthogonal matrices: $Q^\top Q = I \quad \det Q = \pm 1$ (rotations vs improper rotations)

Worked example 5 — shear preserves area

$S = \begin{bmatrix} 1 & k \\ 0 & 1 \end{bmatrix}$, $\det S = 1$. Shears preserve area; shape changes.

Worked example 6 — scaling

$D = \operatorname{diag}(\lambda_1, \dots, \lambda_n)$, $\det D = \prod \lambda_i$ — product of axis scale factors.

4. Determinant and eigenvalues

Characteristic polynomial $p_A(\lambda) = \det(A - \lambda I)$ (sign convention variants exist).

$$\det A = \prod_{i=1}^{n} \lambda_i$$

(over $\mathbb{C}$, counting algebraic multiplicity). Trace $\operatorname{tr}(A) = \sum \lambda_i$.

So $\det A \neq 0$ iff no zero eigenvalue — matches invertibility.

5. Cramer's rule (theory / tiny n)

For invertible $Ax = b$,

$$x_i = \frac{\det(A_i)}{\det A},$$

where A_i is A with column i replaced by b.

Practice: use elimination / library solvers for $n \gtrsim 3$. Cramer is exponential cost if determinants are expanded naively and numerically unstable if implemented poorly.

6. Explicit inverse formulas

2×2

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix}^{-1} = \frac{1}{ad - bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}.$$

Adjugate (general n)

$A^{-1} = \frac{1}{\det A}\operatorname{adj}(A)$ where adj is the cofactor matrix transpose. Useful in proofs; rarely the right computational tool.

7. Jacobians and change of variables

If $T : \mathbb{R}^n \to \mathbb{R}^n$ is differentiable and $y = T(x)$, volumes transform by $|\det DT(x)|$:

$$\int_{T(U)} f(y)\, dy = \int_U f(T(x))\, |\det DT(x)|\, dx.$$

In probability: if $Y = g(X)$ invertible, densities pick up $|\det Dg^{-1}|$.

Worked example 7 — polar (idea)

$x = r\cos\theta$, $y = r\sin\theta$, Jacobian determinant magnitude is r — area element $r\,dr\,d\theta$.

8. Multivariate Gaussians

For $X \sim \mathcal{N}(\mu, \Sigma)$ with $\Sigma \succ 0$,

$$p(x) = (2\pi)^{-n/2}(\det\Sigma)^{-1/2}\exp\Big(-\tfrac{1}{2}(x-\mu)^\top\Sigma^{-1}(x-\mu)\Big).$$

$\det\Sigma$ scales the normalizing constant; ill-conditioned Σ (tiny eigenvalues) makes density peaks sharp and numerics fragile.

9. Numerical wisdom

1. **Do not** compute $x = A^{-1}b$ by forming A^{-1} explicitly when you only need x — solve $Ax = b$
2. Determinants of large matrices can overflow/underflow; work in log-domain ($\sum \log|\text{pivots}|$)
3. Near-singular matrices: $\det \approx 0$ is a symptom; prefer condition number $\kappa(A)$ and residual checks
4. Integer matrices can have huge dets (Hadamard bound); exact arithmetic may need big integers

10. CS / graphics / systems connections

Domain	Role of det / inverse
Computer graphics	orientation tests, back-face, volume of tetrahedra
Robotics / vision	homogeneous transforms; watch det of rotation blocks = 1
Optimization	Newton steps solve linear systems, not inverses as matrices
Random matrices	volume of zonotopes; covariance geometry
Finite elements	Jacobian of reference maps

11. Pitfalls

1. $\det(A+B) \neq \det A + \det B$
2. $\det(cA) = c^n \det A$, not $c\det A$

3. Using Cramer for large systems
4. Interpreting det ≈ 0 without scaling context (scale rows by 10^{10} inflates det)
5. Forming inverses "because the formula has A^{-1}"

12. Checkpoint

- Compute 2×2 and triangular determinants
- Use row-operation rules
- Apply $\det(AB) = \det A \det B$
- Interpret $|\det|$ as volume scaling
- Explain why solvers beat explicit inverses

Exercises

Easy

1. Compute $\det \begin{bmatrix} 1 & 2 & 0 \\ 0 & 3 & 4 \\ 0 & 0 & 5 \end{bmatrix}$.
2. Show that if two columns are equal, $\det = 0$.
3. If $\det A = 2$ and $\det B = -3$, find $\det(A^2 B^{-1})$ (B invertible).
4. For shear $\begin{bmatrix} 1 & k \\ 0 & 1 \end{bmatrix}$, compute det and interpret area.
5. Invert $\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ if possible.

Medium

6. Explain why solving $Ax = b$ via $x = A^{-1}b$ is usually a bad numerical plan.
7. Prove $\det(A^\top) = \det A$ for 2×2 by expansion.
8. Using only row ops, compute $\det \begin{bmatrix} 0 & 1 & 2 \\ 3 & 4 & 5 \\ 6 & 7 & 8 \end{bmatrix}$.

9. If Q orthogonal, show $|\det Q| = 1$.

10. Relate $\det A$ to the product of eigenvalues for a diagonalizable A.

Challenge

11. Prove $\det(AB) = \det A \det B$ for 2×2 by direct expansion.

12. Show $\det(e^B) = e^{\operatorname{tr}(B)}$ for square B (use eigenvalues or series intuition).

13. Hadamard inequality: $|\det A| \le \prod_j \|a_j\|_2$ for columns a_j — interpret geometrically.

14. Cramer's rule derivation from $A \operatorname{adj}(A) = (\det A) I$.

15. Floating point: construct a matrix with huge condition number but moderate entries; compare det vs reliable rank-revealing QR.

Checks

1. 15.

2. $4 \cdot (-1/3) = -4/3$.

3. $\det = 1$, area preserved.

4. $\det = -2$, inverse $-\frac{1}{2}\begin{bmatrix} 4 & -2 \\ -3 & 1 \end{bmatrix}$.

Summary

Determinants package volume, orientation, and invertibility into one scalar. Compute them via small expansions, pivots, or library routines; reason with multiplicativity and eigenvalue products. Inverses exist exactly when $\det \neq 0$, but the mature computational habit is: **solve systems, factor matrices, monitor conditioning** — not chase giant explicit inverses. Geometry (shears, orthogonal maps) and applications (Jacobians, Gaussians) keep the scalar meaningful beyond algebra drills.

Montgomery Modular Multiplication

Montgomery Modular Multiplication

Thesis

Montgomery multiplication replaces costly `mod n` divisions with cheaper reductions in a residue representation. This chapter implements a tiny Python version for correctness. Pure-Python teaching code is **slower** than naïve `%` on small moduli — the win appears in big-int/constant-time code.

Mental model

```
a → aR mod n; mont_mul; leave with ·R⁻¹
```

Worked examples

```
n 1000000007 R 4294967296
classic (a*b)%n = 259106859
montgomery      = 259106859
match True
classic 200000 muls: 12.61 ms  result=641624714
mont    200000 muls: 58.49 ms  result=641624714 match=True
```

Trap

Copying Montgomery into app Python for 64-bit primes. Slower and bug-prone.

Boring rule

Learn it to read crypto code; call audited libraries for production.

Try this

1. Reimplement in C with `__uint128_t`.
2. Read HAC §14.3.
3. Inspect Go `math/big` Montgomery paths.

Sources

- Montgomery 1985; HAC §14.3

Discrete Mathematics

Introduction to Discrete Mathematics: The Mathematics of Distinct Objects

Chapters in this part

Chapter	Focus
Logic and proofs	Propositional/predicate reasoning
Sets and relations	Relations, functions, orderings
Combinatorics	Counting principles
Recurrence relations	Linear recurrences, Master theorem
Induction and invariants	Induction, loop invariants

What is Discrete Mathematics?

Discrete mathematics is the study of mathematical structures that are fundamentally discrete rather than continuous. Unlike calculus, which deals with smooth, continuous functions, discrete mathematics focuses on countable, distinct objects and their relationships.

Discrete mathematics provides the mathematical foundation for computer science, cryptography, combinatorics, and many areas of modern technology. It deals with structures like integers, graphs, statements in logic, and finite sets, rather than real numbers and continuous functions.

```
Discrete vs. Continuous Mathematics


Discrete Mathematics:
• Deals with countable, separate objects
• Examples: integers, graphs, logical statements
• Uses: computer science, cryptography, combinatorics
• Tools: counting, logic, graph theory, number theory

Continuous Mathematics:
• Deals with smooth, unbroken structures
• Examples: real numbers, continuous functions
• Uses: physics, engineering, calculus
• Tools: limits, derivatives, integrals

Key Distinction:
```

```
Discrete: Can you count the objects? (1, 2, 3, ...)
Continuous: Does it flow smoothly without breaks?

Examples:
Discrete: Number of students in a class, network connections, DNA sequences
Continuous: Temperature, velocity, area under a curve
```

Historical Development

Ancient Origins and Modern Evolution

Discrete mathematics has ancient roots but has experienced explosive growth with the rise of computer science.

```
Timeline of Discrete Mathematics


Ancient Period:
3000 BCE: Counting systems and basic arithmetic
500 BCE:  Greek mathematics - Euclidean algorithm
300 CE:   Diophantine equations (number theory)

Medieval Period:
800 CE:   Islamic mathematics - algebra and algorithms
1200 CE:  Fibonacci sequence and combinatorics
1400 CE:  Development of symbolic logic

Renaissance and Enlightenment:
1654:     Pascal and Fermat - probability theory
1736:     Euler - graph theory (Seven Bridges of Königsberg)
1847:     Boole - Boolean algebra and logic

Modern Era:
1930s:    Gödel - incompleteness theorems
1936:     Turing - computability theory
1940s:    Shannon - information theory
1950s:    Computer science emergence

Digital Age:
1960s:    Formal verification and program correctness
1970s:    Complexity theory (P vs NP)
1980s:    Cryptography and public key systems
1990s:    Internet algorithms and data structures
2000s:    Machine learning and discrete optimization
Today:    Quantum computing and algorithmic game theory
```

The Computer Science Revolution

The development of computers transformed discrete mathematics from a collection of specialized topics into a unified field essential for technology.

```
Discrete Mathematics in Computing


Fundamental Connections:

Logic and Computation:
- Boolean algebra → Digital circuits
- Propositional logic → Programming logic
- Predicate logic → Database queries
- Proof theory → Program verification

Combinatorics and Algorithms:
- Counting principles → Algorithm analysis
- Permutations → Sorting algorithms
- Graph theory → Network algorithms
- Optimization → Efficient computation

Number Theory and Security:
- Modular arithmetic → Hash functions
- Prime numbers → Cryptography
- Discrete logarithms → Public key systems
- Error-correcting codes → Data transmission

Set Theory and Data Structures:
- Sets and relations → Database design
- Functions → Programming concepts
- Trees and graphs → Data organization
- Recursion → Algorithm design

Modern Applications:
- Search engines (graph algorithms)
- Social networks (graph theory)
- Cryptography (number theory)
- Machine learning (optimization)
- Bioinformatics (combinatorics)
- Quantum computing (linear algebra over finite fields)
```

Core Areas of Discrete Mathematics

Logic and Proof Techniques

Logic provides the foundation for mathematical reasoning and computer science.

```
Logical Foundations


Propositional Logic:
- Statements that are true or false
- Logical connectives:  (and),  (or), ¬ (not), → (implies)
- Truth tables and logical equivalences
- Applications: Digital circuits, programming logic

Example:
P: "It is raining"
Q: "I carry an umbrella"
P → Q: "If it is raining, then I carry an umbrella"

Predicate Logic:
- Statements with variables and quantifiers
- Universal quantifier:  (for all)
- Existential quantifier:  (there exists)
- Applications: Database queries, mathematical proofs

Example:
 x   , y    such that y > x
"For every natural number x, there exists a natural number y such that y > x"

Proof Techniques:
- Direct proof: P → Q by assuming P and deriving Q
- Proof by contradiction: Assume ¬Q and derive contradiction
- Proof by induction: Base case + inductive step
- Proof by contrapositive: Prove ¬Q → ¬P instead of P → Q

Mathematical Induction:
Base case: Prove P(1) is true
Inductive step: Prove P(k) → P(k+1)
Conclusion: P(n) is true for all n  1

Example: Prove 1 + 2 + ... + n = n(n+1)/2
Base: 1 = 1(2)/2 = 1
Inductive: Assume true for k, prove for k+1
1 + 2 + ... + k + (k+1) = k(k+1)/2 + (k+1) = (k+1)(k+2)/2
```

Set Theory and Relations

Sets provide the fundamental language for describing collections of objects.

```
Set Theory Fundamentals


Basic Concepts:
- Set: Collection of distinct objects
- Element: Object in a set (a   A)
- Empty set:   (contains no elements)
- Universal set: U (contains all objects under consideration)

Set Operations:
- Union: A   B = {x : x   A or x   B}
- Intersection: A   B = {x : x   A and x   B}
- Complement: A' = {x   U : x   A}
- Difference: A - B = {x : x   A and x   B}

Example:
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}
A   B = {1, 2, 3, 4, 5, 6}
A   B = {3, 4}
A - B = {1, 2}

Relations:
- Binary relation: R   A × B
- Properties: reflexive, symmetric, transitive
- Equivalence relation: reflexive, symmetric, transitive
- Partial order: reflexive, antisymmetric, transitive

Functions:
- Special type of relation
- Each input has exactly one output
- Types: injective (one-to-one), surjective (onto), bijective

Cardinality:
- Size of finite sets: |A| = number of elements
- Infinite sets: countable vs uncountable
- Cantor's theorem: |P(A)| > |A| (power set is larger)

Applications:
- Database design (relations)
- Programming (data structures)
- Probability (sample spaces)
- Computer graphics (transformations)
```

Combinatorics and Counting

Combinatorics studies methods of counting and arranging objects.

```
Fundamental Counting Principles


Basic Principles:

Addition Principle:
If task can be done in m ways OR n ways (mutually exclusive),
then total ways = m + n

Example: Choose a letter from {A,B,C} or a digit from {1,2}
Total choices = 3 + 2 = 5

Multiplication Principle:
If task requires m ways AND then n ways (sequential),
then total ways = m × n

Example: Choose a letter from {A,B,C} and then a digit from {1,2}
Total combinations = 3 × 2 = 6

Permutations:
Arrangements where order matters
P(n,r) = n!/(n-r)! = number of ways to arrange r objects from n

Example: Arrange 3 books from 5 books
P(5,3) = 5!/(5-3)! = 5!/2! = 60

Combinations:
Selections where order doesn't matter
C(n,r) = n!/(r!(n-r)!) = number of ways to choose r objects from n

Example: Choose 3 books from 5 books
C(5,3) = 5!/(3!2!) = 10

Binomial Theorem:
(x + y)  = Σ(k=0 to n) C(n,k) x   y

Pascal's Triangle:
Row n contains binomial coefficients C(n,k)
     1
   1   1
  1  2  1
 1  3  3  1
1  4  6  4  1
```

```
Applications:
- Probability calculations
- Algorithm analysis
- Coding theory
- Cryptography
- Bioinformatics
```

Graph Theory

Graph theory studies networks of connected objects.

```
Graph Theory Basics


Definitions:
- Graph G = (V, E): V = vertices (nodes), E = edges (connections)
- Directed vs undirected graphs
- Weighted vs unweighted graphs
- Simple graph: no loops or multiple edges

Examples:
Social network: People = vertices, friendships = edges
Internet: Computers = vertices, connections = edges
Transportation: Cities = vertices, roads = edges

Basic Properties:
- Degree of vertex: number of edges connected to it
- Path: sequence of vertices connected by edges
- Cycle: path that starts and ends at same vertex
- Connected graph: path exists between any two vertices

Special Graphs:
- Complete graph K : every pair of vertices connected
- Bipartite graph: vertices can be divided into two sets
- Tree: connected graph with no cycles
- Planar graph: can be drawn without edge crossings

Graph Algorithms:
- Depth-First Search (DFS): explore as far as possible
- Breadth-First Search (BFS): explore level by level
- Shortest path: Dijkstra's algorithm
- Minimum spanning tree: Kruskal's or Prim's algorithm

Famous Problems:
- Seven Bridges of Königsberg (Euler paths)
- Traveling Salesman Problem (Hamiltonian cycles)
- Four Color Theorem (graph coloring)
```

```
- Network flow problems

Applications:
- Social network analysis
- Internet routing protocols
- GPS navigation systems
- Circuit design
- Molecular structure analysis
- Project scheduling (PERT/CPM)
```

Number Theory

Number theory studies properties of integers and their relationships.

```
Number Theory Foundations


Divisibility:
- a divides b (a|b) if b = ak for some integer k
- Properties: transitivity, linear combinations
- Greatest Common Divisor: gcd(a,b)
- Least Common Multiple: lcm(a,b)

Euclidean Algorithm:
Efficient method to find gcd(a,b)
gcd(48, 18):
48 = 2 × 18 + 12
18 = 1 × 12 + 6
12 = 2 × 6 + 0
Therefore gcd(48, 18) = 6

Prime Numbers:
- Prime: integer > 1 with exactly two divisors (1 and itself)
- Fundamental Theorem of Arithmetic: unique prime factorization
- Infinitely many primes (Euclid's proof)
- Prime distribution: Prime Number Theorem

Modular Arithmetic:
- a   b (mod n) if n divides (a - b)
- Arithmetic operations preserve congruence
- Applications: cryptography, hash functions, error detection

Example:
17   2 (mod 5) because 17 - 2 = 15 = 3 × 5
Clock arithmetic: 15:00 + 10 hours = 1:00 (mod 12)

Fermat's Little Theorem:
```

```
If p is prime and gcd(a,p) = 1, then a ¹  1 (mod p)

Applications:
- RSA cryptography (public key systems)
- Hash functions and checksums
- Pseudorandom number generation
- Error-correcting codes
- Digital signatures
- Blockchain and cryptocurrency
```

Applications in Computer Science

Algorithms and Data Structures

Discrete mathematics provides the theoretical foundation for algorithm design and analysis.

```
Algorithmic Applications


Algorithm Analysis:
- Time complexity: Big O notation
- Space complexity: memory usage
- Best, average, worst case analysis
- Recurrence relations for recursive algorithms

Example: Binary Search
T(n) = T(n/2) + O(1)
Solution: T(n) = O(log n)

Data Structures:
- Arrays and linked lists (sequences)
- Stacks and queues (linear structures)
- Trees (hierarchical structures)
- Hash tables (associative structures)
- Graphs (network structures)

Sorting Algorithms:
- Comparison-based: O(n log n) lower bound
- Counting sort: O(n + k) for integers in range [0,k]
- Radix sort: O(d(n + k)) for d-digit numbers

Graph Algorithms:
- Shortest path: Dijkstra's O((V + E) log V)
- Minimum spanning tree: Kruskal's O(E log V)
- Maximum flow: Ford-Fulkerson O(E × max_flow)
- Topological sort: O(V + E)
```

```
Dynamic Programming:
- Optimal substructure property
- Overlapping subproblems
- Memoization vs tabulation
- Examples: Fibonacci, knapsack, longest common subsequence

Greedy Algorithms:
- Make locally optimal choices
- Examples: Huffman coding, activity selection
- Correctness requires proof of greedy choice property

Divide and Conquer:
- Break problem into smaller subproblems
- Solve recursively and combine solutions
- Examples: merge sort, quick sort, fast multiplication
```

Cryptography and Security

Number theory and discrete mathematics form the backbone of modern cryptography.

```
Cryptographic Applications


Classical Cryptography:
- Caesar cipher: shift each letter by fixed amount
- Substitution ciphers: replace each letter with another
- Vigenère cipher: polyalphabetic substitution
- Frequency analysis: breaking substitution ciphers

Modern Symmetric Cryptography:
- Block ciphers: encrypt fixed-size blocks
- Stream ciphers: encrypt bit by bit
- Advanced Encryption Standard (AES)
- Key distribution problem

Public Key Cryptography:
Based on mathematical problems that are easy one way, hard the other

RSA Algorithm:
1. Choose large primes p, q
2. Compute n = pq,  (n) = (p-1)(q-1)
3. Choose e with gcd(e,  (n)) = 1
4. Compute d with ed   1 (mod  (n))
5. Public key: (n, e), Private key: (n, d)
6. Encrypt: c   m  (mod n)
7. Decrypt: m   c  (mod n)
```

```
Security based on difficulty of factoring large integers

Elliptic Curve Cryptography:
- Based on elliptic curves over finite fields
- Smaller key sizes than RSA for same security
- Discrete logarithm problem in elliptic curve groups

Hash Functions:
- One-way functions: easy to compute, hard to invert
- Properties: deterministic, fixed output size, avalanche effect
- Applications: digital signatures, password storage, blockchain

Digital Signatures:
- Authenticity: verify sender identity
- Non-repudiation: sender cannot deny sending
- Integrity: detect message tampering
- Based on public key cryptography

Applications:
- Secure communication (HTTPS, VPN)
- Digital currencies (Bitcoin, Ethereum)
- Authentication systems
- Secure voting systems
- Digital rights management
```

Database Theory

Set theory and logic provide the mathematical foundation for database systems.

```
Database Mathematical Foundations


Relational Model:
- Relation: set of tuples (rows)
- Attribute: column in relation
- Domain: set of possible values for attribute
- Key: minimal set of attributes that uniquely identify tuple

Example:
Students relation:
{(123, "Alice", "CS", 3.8), (456, "Bob", "Math", 3.5), ...}

Relational Algebra:
Mathematical operations on relations

Selection ( ): Choose rows satisfying condition
```

```
 _{GPA > 3.5}(Students) = students with GPA > 3.5

Projection ( ): Choose specific columns
 _{Name, Major}(Students) = names and majors only

Union ( ): Combine relations with same schema
Intersection ( ): Common tuples
Difference (-): Tuples in first but not second

Join ( ): Combine relations based on common attributes
Students  Enrollments = student info with course enrollments

SQL and Logic:
- SELECT corresponds to projection and selection
- WHERE clause uses propositional logic
- EXISTS corresponds to existential quantification
- Subqueries use nested logical structures

Query Optimization:
- Use relational algebra to transform queries
- Cost-based optimization using combinatorics
- Index structures based on tree data structures

Normalization:
- Functional dependencies: X → Y
- Normal forms: 1NF, 2NF, 3NF, BCNF
- Decomposition algorithms
- Trade-offs between normalization and performance

Transaction Theory:
- ACID properties: Atomicity, Consistency, Isolation, Durability
- Concurrency control using graph theory
- Deadlock detection in wait-for graphs
- Distributed consensus algorithms
```

Problem-Solving Techniques

Proof Strategies

Discrete mathematics emphasizes rigorous proof techniques essential for computer science and mathematics.

```
Proof Methodologies


Direct Proof:
```

To prove P → Q:
1. Assume P is true
2. Use logical reasoning to show Q must be true

Example: Prove "If n is even, then n^2 is even"
Proof: Assume n is even, so n = 2k for some integer k
Then $n^2 = (2k)^2 = 4k^2 = 2(2k^2)$
Since $2k^2$ is an integer, n^2 is even.

Proof by Contradiction:
To prove P:
1. Assume ¬P (not P)
2. Derive a logical contradiction
3. Conclude P must be true

Example: Prove √2 is irrational
Proof: Assume √2 = p/q where p, q are integers with gcd(p,q) = 1
Then $2 = p^2/q^2$, so $2q^2 = p^2$
This means p^2 is even, so p is even: p = 2r
Then $2q^2 = (2r)^2 = 4r^2$, so $q^2 = 2r^2$
This means q^2 is even, so q is even
But then gcd(p,q) 2, contradicting gcd(p,q) = 1
Therefore √2 is irrational.

Proof by Induction:
To prove P(n) for all n n :
1. Base case: Prove P(n)
2. Inductive step: Prove P(k) → P(k+1)
3. Conclude P(n) is true for all n n

Example: Prove $2^{\ } > n$ for all n 1
Base case: $2^1 = 2 > 1$
Inductive step: Assume $2^{\ } > k$, prove $2^{\ 1} > k+1$
$2^{\ 1} = 2 \cdot 2^{\ } > 2k$ (by inductive hypothesis)
For k 1, we have 2k k+1, so $2^{\ 1} > k+1$

Strong Induction:
Assume P(1), P(2), ..., P(k) all true to prove P(k+1)
Useful when P(k+1) depends on multiple previous cases

Proof by Contrapositive:
To prove P → Q, instead prove ¬Q → ¬P

Example: Prove "If n^2 is odd, then n is odd"
Contrapositive: "If n is even, then n^2 is even"
(Already proved above)

Proof by Cases:

```
Divide problem into exhaustive, mutually exclusive cases
Prove statement for each case separately

Existence Proofs:
Constructive: Explicitly construct object with desired property
Non-constructive: Prove object exists without constructing it
```

Problem-Solving Strategies

```
Systematic Problem-Solving Approach


Understanding the Problem:
1. Read carefully and identify what's given
2. Determine what needs to be proved or found
3. Look for patterns or similar problems
4. Consider special cases or examples

Planning the Solution:
1. Choose appropriate proof technique
2. Identify relevant definitions and theorems
3. Work backwards from conclusion
4. Consider multiple approaches

Executing the Plan:
1. Write clear, logical steps
2. Justify each step with reasons
3. Use proper mathematical notation
4. Check intermediate results

Verification:
1. Review logic for gaps or errors
2. Check special cases
3. Verify the conclusion answers the original question
4. Consider alternative approaches

Common Strategies:

Pattern Recognition:
- Look for familiar structures
- Use analogies with known problems
- Identify recursive patterns

Reduction:
- Break complex problems into simpler parts
- Use previously solved subproblems
- Apply divide-and-conquer approach
```

```
Generalization:
- Solve simpler or more general version first
- Look for underlying principles
- Extend solutions to broader contexts

Contradiction and Extremal Arguments:
- Assume opposite and derive contradiction
- Consider minimal or maximal elements
- Use pigeonhole principle

Counting Arguments:
- Count objects in two different ways
- Use inclusion-exclusion principle
- Apply probabilistic methods

Invariant Arguments:
- Find quantities that don't change
- Use conservation principles
- Track what remains constant through transformations
```

Modern Applications and Connections

Machine Learning and Data Science

Discrete mathematics provides essential tools for understanding algorithms and data structures in AI.

```
ML and Discrete Mathematics Connections


Graph-Based Learning:
- Social network analysis: centrality measures
- Recommendation systems: bipartite graphs
- Knowledge graphs: semantic relationships
- Neural networks: computational graphs

Combinatorial Optimization:
- Feature selection: subset selection problems
- Hyperparameter tuning: grid search and random search
- Model selection: combinatorial search spaces
- Ensemble methods: combining multiple models

Information Theory:
- Entropy: measure of information content
- Mutual information: feature relevance
```

```
- Decision trees: information gain splitting
- Compression: Huffman coding, arithmetic coding

Probability and Statistics:
- Discrete probability distributions
- Bayesian networks: probabilistic graphical models
- Markov chains: sequential data modeling
- Monte Carlo methods: sampling techniques

Algorithm Analysis:
- Time complexity of learning algorithms
- Space complexity of data structures
- Approximation algorithms for NP-hard problems
- Online algorithms for streaming data

Example: PageRank Algorithm
Models web as directed graph
Computes stationary distribution of random walk
Uses linear algebra over discrete structures
Fundamental to Google's search ranking

Cryptographic Applications in ML:
- Differential privacy: protecting individual data
- Homomorphic encryption: computing on encrypted data
- Secure multi-party computation: collaborative learning
- Federated learning: distributed privacy-preserving training
```

Quantum Computing

Quantum computing relies heavily on discrete mathematics and linear algebra over finite fields.

```
Quantum Computing and Discrete Math


Quantum States:
- Qubits: quantum bits (superposition of 0 and 1)
- State space: complex vector space
- Measurement: probabilistic outcomes
- Entanglement: non-classical correlations

Quantum Gates:
- Unitary matrices acting on qubit states
- Universal gate sets: can approximate any unitary
- Quantum circuits: sequences of gates
- Reversible computation: all quantum operations are reversible

Quantum Algorithms:
```

```
- Shor's algorithm: factoring integers exponentially faster
- Grover's algorithm: searching unsorted database quadratically faster
- Quantum Fourier transform: basis for many quantum algorithms
- Variational quantum algorithms: hybrid classical-quantum optimization

Quantum Error Correction:
- Quantum error-correcting codes
- Stabilizer codes: group theory applications
- Fault-tolerant quantum computation
- Threshold theorem: error correction is possible

Discrete Structures in Quantum Computing:
- Finite groups: symmetries in quantum systems
- Boolean functions: classical-quantum interfaces
- Graph states: multiparty entanglement
- Lattice problems: post-quantum cryptography

Applications:
- Cryptography: breaking RSA, new quantum-safe methods
- Optimization: quantum annealing, QAOA
- Simulation: quantum chemistry, materials science
- Machine learning: quantum neural networks, quantum advantage
```

Bioinformatics and Computational Biology

Discrete mathematics provides tools for analyzing biological sequences and structures.

```
Bioinformatics Applications


Sequence Analysis:
- DNA/RNA/protein sequences as strings over finite alphabets
- String matching algorithms: finding patterns in sequences
- Sequence alignment: dynamic programming algorithms
- Phylogenetic trees: evolutionary relationships

Example: DNA Sequence Alignment
ATCGATCG
ATCG-TCG
Use dynamic programming to find optimal alignment
Score matches, mismatches, and gaps

Combinatorial Problems:
- Genome assembly: reconstructing sequences from fragments
- Shortest superstring problem: overlapping fragments
- Traveling salesman: optimizing lab workflows
- Graph coloring: scheduling experiments
```

```
Graph Theory Applications:
- Protein interaction networks: vertices = proteins, edges = interactions
- Metabolic pathways: biochemical reaction networks
- Gene regulatory networks: transcriptional control
- Phylogenetic networks: evolutionary relationships with horizontal transfer

Probability and Statistics:
- Hidden Markov models: gene finding, protein structure prediction
- Bayesian networks: modeling biological systems
- Statistical significance: multiple testing corrections
- Machine learning: classification and prediction

Structural Biology:
- Protein folding: discrete conformational states
- RNA secondary structure: dynamic programming prediction
- Molecular docking: combinatorial search problems
- Drug design: chemical space exploration

Population Genetics:
- Hardy-Weinberg equilibrium: allele frequency calculations
- Coalescent theory: genealogical relationships
- Selection models: discrete-time dynamical systems
- Phylogeography: spatial population structure

Applications:
- Personalized medicine: genetic variant analysis
- Drug discovery: target identification and validation
- Agricultural genomics: crop improvement
- Conservation biology: genetic diversity assessment
- Forensics: DNA fingerprinting and paternity testing
```

The Beauty and Unity of Discrete Mathematics

Connections Across Mathematics

Discrete mathematics reveals deep connections between seemingly different areas of mathematics.

```
Mathematical Connections


Number Theory  Cryptography:
- Prime numbers → RSA encryption
- Modular arithmetic → Hash functions
- Elliptic curves → Advanced cryptosystems
- Lattice problems → Post-quantum cryptography
```

Combinatorics Probability:
- Counting outcomes → Probability calculations
- Generating functions → Moment generating functions
- Inclusion-exclusion → Bonferroni inequalities
- Ramsey theory → Probabilistic method

Graph Theory Linear Algebra:
- Adjacency matrices → Spectral graph theory
- Eigenvalues → Graph properties
- Random walks → Markov chains
- Network analysis → Matrix computations

Logic Computer Science:
- Boolean algebra → Digital circuits
- Predicate logic → Database queries
- Proof theory → Program verification
- Complexity theory → Computational limits

Set Theory Topology:
- Open and closed sets → Topological spaces
- Continuity → Limit points
- Compactness → Finite subcovers
- Connectedness → Path components

Algebra Discrete Structures:
- Groups → Symmetries and transformations
- Rings → Polynomial arithmetic
- Fields → Error-correcting codes
- Lattices → Cryptographic applications

Universal Principles:
- Duality: optimization problems have dual formulations
- Recursion: self-similar structures appear everywhere
- Symmetry: group theory unifies diverse phenomena
- Optimization: extremal principles guide solutions
- Information: entropy measures appear in many contexts

Aesthetic and Philosophical Aspects

The Beauty of Discrete Mathematics

Elegance in Simplicity:
- Simple rules generate complex behavior
- Recursive definitions create infinite structures
- Basic counting principles solve sophisticated problems

```
- Elementary logic captures deep reasoning

Examples of Mathematical Beauty:
- Fibonacci sequence: appears in nature, art, and mathematics
- Pascal's triangle: connects combinatorics, algebra, and number theory
- Euler's formula for graphs: V - E + F = 2 (planar graphs)
- Ramsey theory: "complete disorder is impossible"

Surprising Connections:
- Birthday paradox: probability and combinatorics
- Four color theorem: topology and graph theory
- Gödel's incompleteness: logic and computability
- P vs NP: complexity and practical computation

Philosophical Questions:
- What is computation? (Church-Turing thesis)
- What can be proved? (Gödel's theorems)
- What can be computed efficiently? (P vs NP)
- How much information is needed? (Kolmogorov complexity)

Mathematical Creativity:
- Problem-solving requires insight and intuition
- Multiple approaches often lead to same result
- Abstraction reveals underlying patterns
- Generalization extends applicability

Practical Beauty:
- Elegant algorithms are often efficient
- Simple data structures are robust
- Clear proofs are convincing
- General theories have broad applications

Cultural Impact:
- Discrete mathematics shapes digital culture
- Algorithms influence social interactions
- Cryptography enables privacy and security
- Network theory explains social phenomena
- Information theory quantifies communication
```

Building Discrete Mathematical Thinking

Developing Problem-Solving Skills

Success in discrete mathematics requires developing specific thinking patterns and problem-solving approaches.

Discrete Mathematical Thinking

Key Mental Models:

Structural Thinking:
- Recognize patterns and relationships
- Identify underlying mathematical structures
- Use abstraction to simplify problems
- Apply known results to new situations

Algorithmic Thinking:
- Break problems into step-by-step procedures
- Analyze efficiency and correctness
- Consider edge cases and boundary conditions
- Design and implement solutions systematically

Logical Reasoning:
- Construct valid arguments
- Identify assumptions and conclusions
- Use formal proof techniques
- Distinguish between necessary and sufficient conditions

Combinatorial Intuition:
- Develop counting skills
- Recognize when to use different counting principles
- Understand symmetry and equivalence
- Apply inclusion-exclusion and other techniques

Study Strategies:

Active Problem Solving:
1. Work through many examples
2. Try different approaches to same problem
3. Explain solutions to others
4. Connect new problems to familiar ones

Pattern Recognition:
1. Look for recurring themes
2. Study classic problems and their solutions
3. Build a toolkit of standard techniques
4. Practice applying techniques in new contexts

Proof Writing:
1. Start with simple, direct proofs
2. Practice different proof techniques
3. Focus on clarity and logical flow
4. Learn from well-written proofs

```
Computational Practice:
1. Implement algorithms to understand them
2. Experiment with small examples
3. Use technology to explore patterns
4. Verify theoretical results computationally

Building Intuition:
1. Visualize problems when possible
2. Use concrete examples before generalizing
3. Develop geometric and algebraic perspectives
4. Connect abstract concepts to real applications
```

Conclusion

Discrete mathematics represents the mathematical foundation of the digital age, providing essential tools for computer science, cryptography, data analysis, and modern technology. From its ancient roots in counting and logic to its modern applications in artificial intelligence and quantum computing, discrete mathematics continues to evolve and expand its influence.

```
Discrete Mathematics: Foundation of the Digital World


Historical Significance:
  Ancient counting systems to modern algorithms
  Logic and proof techniques spanning millennia
  Graph theory emerging from practical problems
  Number theory enabling secure communication

Conceptual Power:
  Unifies diverse areas of mathematics and computer science
  Provides rigorous foundation for computational thinking
  Enables analysis of complex discrete systems
  Bridges pure mathematics and practical applications

Modern Applications:
  Computer science algorithms and data structures
  Cryptography and information security
  Network analysis and social media
  Machine learning and artificial intelligence
  Bioinformatics and computational biology
  Quantum computing and advanced technologies

Educational Value:
  Develops logical reasoning and proof skills
  Builds problem-solving and analytical thinking
```

```
Prepares for advanced computer science topics
Provides foundation for mathematical research
Enhances understanding of digital technologies
```

As you embark on your journey through discrete mathematics, remember that you're learning the language of computation and digital reasoning. The concepts of logic, sets, counting, graphs, and number theory form the intellectual toolkit that powers everything from search engines and social networks to cryptographic systems and artificial intelligence.

Discrete mathematics is not just about solving problems—it's about understanding the fundamental structures that govern information, computation, and digital communication. The skills you develop in logical reasoning, proof techniques, and algorithmic thinking will serve you throughout your career in mathematics, computer science, engineering, or any field that involves systematic problem-solving.

Whether you're interested in theoretical computer science, practical software development, cybersecurity, data science, or emerging technologies like quantum computing, discrete mathematics provides the essential mathematical foundation. The investment you make in understanding these concepts will pay dividends as technology continues to evolve and create new opportunities for those who understand its mathematical foundations.

The beauty of discrete mathematics lies not only in its practical utility but also in its elegant theoretical structure and surprising connections across different areas of knowledge. As you explore this fascinating field, you'll discover that discrete mathematics is truly the mathematics of the modern world—precise, powerful, and endlessly applicable to the challenges and opportunities of our digital age.

Logic and Proofs: The Foundation of Mathematical Reasoning

Introduction to Mathematical Logic

Mathematical logic provides the rigorous foundation for all mathematical reasoning and proof. It gives us precise tools for expressing mathematical statements, analyzing their truth values, and constructing valid arguments.

Logic is essential not only for pure mathematics but also for computer science, where it forms the basis for programming languages, database queries, artificial intelligence, and digital circuit design.

```
Logic in Mathematics and Computing


Mathematical Applications:
• Expressing theorems and definitions precisely
• Constructing rigorous proofs
• Analyzing mathematical structures
• Developing axiomatic systems

Computer Science Applications:
• Programming language semantics
• Database query languages (SQL)
• Artificial intelligence and expert systems
• Digital circuit design and verification
• Software specification and verification

Everyday Reasoning:
• Analyzing arguments and detecting fallacies
• Making logical decisions
• Understanding conditional statements
• Reasoning about cause and effect

Key Benefits:
  Precision in mathematical communication
  Systematic approach to problem-solving
  Foundation for automated reasoning
  Bridge between mathematics and computation
```

Propositional Logic

Basic Concepts and Connectives

Propositional logic deals with statements that are either true or false, and ways to combine them using logical connectives.

```
Propositional Logic Fundamentals


Proposition: A declarative statement that is either true or false

Examples of Propositions:
  "2 + 3 = 5" (True)
  "Paris is the capital of France" (True)
  "7 is an even number" (False)
  "It is raining" (True or False, depending on circumstances)

Not Propositions:
  "What time is it?" (Question)
  "Close the door" (Command)
  "x + 1 = 5" (Contains variable, truth depends on x)
  "This statement is false" (Paradox)

Propositional Variables:
Use letters p, q, r, s, ... to represent propositions
p: "It is sunny"
q: "I will go to the beach"

Logical Connectives:
Symbol | Name        | Meaning
-------|-------------|----------
¬      | Negation    | not
       | Conjunction | and
       | Disjunction | or
→      | Implication | if...then
       | Biconditional| if and only if
```

Truth Tables

Truth tables systematically show the truth values of compound propositions for all possible combinations of their components.

```
Truth Tables for Basic Connectives
```

Negation (¬p):

p	¬p
T	F
F	T

Conjunction (p ∧ q):

p	q	p ∧ q
T	T	T
T	F	F
F	T	F
F	F	F

Disjunction (p ∨ q):

p	q	p ∨ q
T	T	T
T	F	T
F	T	T
F	F	F

Implication (p → q):

p	q	p → q
T	T	T
T	F	F
F	T	T
F	F	T

Biconditional (p ↔ q):

p	q	p ↔ q
T	T	T
T	F	F
F	T	F
F	F	T

Key Insights:

- Conjunction is true only when both parts are true
- Disjunction is false only when both parts are false
- Implication is false only when antecedent is true and consequent is false
- Biconditional is true when both parts have same truth value

Proof Techniques

Direct Proof

Direct proof is the most straightforward method: assume the hypothesis and logically derive the conclusion.

```
Direct Proof Method


Structure:
To prove P → Q:
1. Assume P is true
2. Use logical reasoning, definitions, and known facts
3. Derive Q

Template:
Proof: Assume P. [reasoning steps] Therefore Q.

Example 1: Prove "If n is even, then n² is even"
Proof:
Assume n is even. Then n = 2k for some integer k.
Therefore n² = (2k)² = 4k² = 2(2k²).
Since 2k² is an integer, n² is even.

Example 2: Prove "If x and y are rational, then x + y is rational"
Proof:
Assume x and y are rational.
Then x = a/b and y = c/d where a, b, c, d are integers and b, d  0.
Therefore x + y = a/b + c/d = (ad + bc)/(bd).
Since ad + bc and bd are integers and bd  0, x + y is rational.

Key Strategies:
• Start with definitions of terms in hypothesis
• Use algebraic manipulation when appropriate
• Apply known theorems and properties
• Work step-by-step toward conclusion
• Ensure each step follows logically from previous steps
```

Proof by Contradiction

Proof by contradiction assumes the negation of what we want to prove and derives a logical contradiction.

```
Proof by Contradiction Method
```

```
Structure:
To prove P:
1. Assume ¬P (not P)
2. Use logical reasoning
3. Derive a contradiction (Q  ¬Q)
4. Conclude P must be true

Example: Prove "√2 is irrational"
Proof:
Assume for the sake of contradiction that √2 is rational.
Then √2 = p/q where p, q are integers with gcd(p,q) = 1.
Squaring both sides: 2 = p²/q², so 2q² = p².
This means p² is even, so p is even. Let p = 2r.
Then 2q² = (2r)² = 4r², so q² = 2r².
This means q² is even, so q is even.
But if both p and q are even, then gcd(p,q)  2.
This contradicts gcd(p,q) = 1.
Therefore √2 is irrational.
```

Mathematical Induction

Mathematical induction is used to prove statements about all positive integers.

```
Mathematical Induction Method


Principle:
To prove  n  n , P(n):
1. Base case: Prove P(n )
2. Inductive step: Prove  k  n , P(k) → P(k+1)
3. Conclude  n  n , P(n)

Example: Prove 1 + 2 + ... + n = n(n+1)/2 for all n  1
Proof:
Base case (n = 1): 1 = 1(2)/2 = 1

Inductive step: Assume 1 + 2 + ... + k = k(k+1)/2.
We need to prove 1 + 2 + ... + k + (k+1) = (k+1)(k+2)/2.

1 + 2 + ... + k + (k+1) = k(k+1)/2 + (k+1)    [by inductive hypothesis]
                        = (k+1)(k/2 + 1)
                        = (k+1)(k+2)/2

By mathematical induction, the formula holds for all n  1.
```

Applications in Computer Science

Programming Logic

```
Logic in Programming


Boolean Expressions:
if (age >= 18 && hasLicense) {
    // Can drive
}

Loop Invariants:
Property that remains true before and after each loop iteration

Example: Array sum algorithm
sum = 0
for i = 0 to n-1:
    sum = sum + array[i]

Invariant: sum = array[0] + array[1] + ... + array[i-1]

Software Verification:
Specify program behavior using logical formulas
Verify correctness using logical reasoning
```

Database Queries

```
SQL and Logic


SQL WHERE clauses use propositional logic:

SELECT * FROM Students
WHERE (major = 'CS' OR major = 'Math')
  AND gpa > 3.5;

Quantifiers in SQL:
EXISTS (existential quantifier)
ALL (universal quantifier)

Query optimization uses logical equivalences
```

Summary and Key Concepts

Logic and proofs form the rigorous foundation for all mathematical reasoning and provide essential tools for computer science applications.

```
Chapter Summary


Essential Skills Mastered:
  Propositional logic: connectives, truth tables, equivalences
  Predicate logic: quantifiers and mathematical statements
  Direct proof techniques and logical reasoning
  Proof by contradiction and mathematical induction
  Applications in computer science and mathematics

Key Concepts:
• Logic as foundation for mathematical reasoning
• Truth tables and logical equivalences
• Systematic proof techniques for different problem types
• Rigorous argumentation and logical validity
• Applications in programming, databases, and AI

Proof Techniques:
• Direct proof: assume hypothesis, derive conclusion
• Contradiction: assume negation, derive contradiction
• Induction: base case + inductive step

Next Steps:
Logic and proof skills prepare you for:
- Advanced mathematics and theorem proving
- Computer science theory and algorithms
- Software engineering and formal methods
- Artificial intelligence and machine learning
```

Logic and proofs represent the intellectual foundation of mathematics and computer science, providing the tools for rigorous reasoning and systematic problem-solving. The skills developed in this chapter are essential for advanced mathematics, programming, software engineering, and any field requiring precise reasoning.

Sets and Relations: The Language of Mathematical Structure

Introduction to Set Theory

Set theory provides the fundamental language for describing collections of objects and their relationships. It forms the foundation for virtually all areas of mathematics and computer science, from number systems and functions to databases and algorithms.

A set is simply a collection of distinct objects, called elements or members. Set theory gives us precise tools for working with these collections and understanding their properties and relationships.

```
Set Theory Foundations


What is a Set?
• Collection of distinct objects
• Objects called elements or members
• Order doesn't matter: {1, 2, 3} = {3, 1, 2}
• No repetition: {1, 1, 2} = {1, 2}
• Can contain any type of objects

Examples:
• {1, 2, 3, 4, 5} - set of integers
• {red, blue, green} - set of colors
• { } - set containing the empty set
•   = {1, 2, 3, ...} - set of natural numbers
•   - set of real numbers

Applications:
• Database design (relations as sets of tuples)
• Programming (data structures, collections)
• Probability (sample spaces and events)
• Logic (domains of discourse)
• Computer graphics (sets of pixels, vertices)
```

Basic Set Concepts

Set Notation and Membership

Set Notation

Element Membership:
- a A: "a is an element of A" or "a belongs to A"
- a A: "a is not an element of A"

Examples:
- 3 {1, 2, 3, 4}
- 5 {1, 2, 3, 4}
- apple {apple, banana, orange}

Set Specification Methods:

1. Roster Method (List Elements):
A = {1, 2, 3, 4, 5}
B = {red, blue, green}
C = {2, 4, 6, 8, 10, ...}

2. Set-Builder Notation:
A = {x | P(x)} - "set of all x such that P(x) is true"
A = {x S | P(x)} - "set of all x in S such that P(x) is true"

Examples:
- {x | x is an even integer} = {... -4, -2, 0, 2, 4, ...}
- {x | x < 10} = {1, 2, 3, 4, 5, 6, 7, 8, 9}
- {x | x^2 = 4} = {-2, 2}
- {x | x is prime} = {2, 3, 5, 7, 11, 13, ...}

3. Interval Notation (for real numbers):
- [a, b] = {x | a x b} (closed interval)
- (a, b) = {x | a < x < b} (open interval)
- [a, b) = {x | a x < b} (half-open interval)

Special Sets:
- or {} - empty set (contains no elements)
- = {1, 2, 3, ...} - natural numbers
- = {..., -2, -1, 0, 1, 2, ...} - integers
- - rational numbers
- - real numbers
- - complex numbers

Set Equality and Subsets

Set Relationships

Set Equality:
A = B if and only if A and B have exactly the same elements
Formally: A = B x (x A x B)

Examples:
• {1, 2, 3} = {3, 1, 2} (order doesn't matter)
• {1, 2, 2, 3} = {1, 2, 3} (repetition doesn't matter)
• {a, b} {a, b, c}

Subset:
A B if every element of A is also an element of B
Formally: A B x (x A → x B)

Examples:
• {1, 2} {1, 2, 3, 4}
•
• A for any set A
• A A for any set A

Proper Subset:
A B if A B and A B
A is a proper subset of B

Examples:
• {1, 2} {1, 2, 3}
•
• {1, 2, 3} (unless A =)

Superset:
A B if B A
A B if B A

Properties:
• Reflexive: A A
• Antisymmetric: (A B B A) → A = B
• Transitive: (A B B C) → A C

Set Operations

Basic Set Operations

Fundamental Set Operations

Union (A B):
Set of elements that are in A or B (or both)
A B = {x | x A x B}

Examples:
• {1, 2, 3} {3, 4, 5} = {1, 2, 3, 4, 5}
• {a, b} {c, d} = {a, b, c, d}
• A = A

Intersection (A B):
Set of elements that are in both A and B
A B = {x | x A x B}

Examples:
• {1, 2, 3} {3, 4, 5} = {3}
• {a, b, c} {b, c, d} = {b, c}
• A =

Difference (A - B or A \ B):
Set of elements that are in A but not in B
A - B = {x | x A x B}

Examples:
• {1, 2, 3, 4} - {3, 4, 5} = {1, 2}
• {a, b, c} - {b, d} = {a, c}
• A - = A
• A - A =

Complement (A' or $\bar{A}$):
Set of elements in universal set U that are not in A
A' = U - A = {x U | x A}

Examples (with U = {1, 2, 3, 4, 5}):
• If A = {1, 3, 5}, then A' = {2, 4}
• If A = {2, 4}, then A' = {1, 3, 5}
• ' = U
• U' =

Symmetric Difference (A B):
Set of elements that are in A or B but not in both

A B = (A - B) (B - A) = (A B) - (A B)

Examples:
• {1, 2, 3} {3, 4, 5} = {1, 2, 4, 5}
• {a, b} {b, c} = {a, c}

Properties of Set Operations

Set Operation Properties

Identity Laws:
• A = A
• A U = A

Domination Laws:
• A U = U
• A =

Idempotent Laws:
• A A = A
• A A = A

Complement Laws:
• A A' = U
• A A' =
• (A')' = A

Commutative Laws:
• A B = B A
• A B = B A

Associative Laws:
• (A B) C = A (B C)
• (A B) C = A (B C)

Distributive Laws:
• A (B C) = (A B) (A C)
• A (B C) = (A B) (A C)

De Morgan's Laws:
• (A B)' = A' B'
• (A B)' = A' B'

Absorption Laws:
• A (A B) = A
• A (A B) = A

Example Application:
Simplify: (A ∪ B) ∩ (A ∪ B')

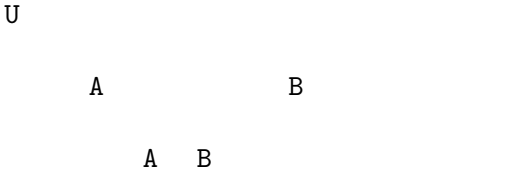
Using distributive law:
(A ∪ B) ∩ (A ∪ B') = A ∪ (B ∩ B')
= A (complement law)
= A (identity law)

Venn Diagrams

Visual Representation of Sets

Venn Diagrams:
Visual tool for representing sets and their relationships
Rectangles represent universal set
Circles represent individual sets
Overlapping regions show intersections

Two-Set Venn Diagram:



Regions:
- A only: A - B
- B only: B - A
- Both A and B: A ∩ B
- Neither A nor B: (A ∪ B)'

Three-Set Venn Diagram:
Shows all possible intersections of three sets A, B, C
8 distinct regions total

Applications:
- Visualizing set operations
- Solving word problems
- Understanding logical relationships

- Database query design
- Probability problems

Example Problem:
In a class of 30 students:
- 18 study math
- 15 study physics
- 10 study both math and physics
- How many study neither?

Solution using Venn diagram:
- Math only: 18 - 10 = 8
- Physics only: 15 - 10 = 5
- Both: 10
- Total studying at least one: 8 + 5 + 10 = 23
- Neither: 30 - 23 = 7

Cartesian Products and Relations

Cartesian Products

Cartesian Product Definition

Ordered Pair:
(a, b) where order matters: (a, b) (b, a) unless a = b

Cartesian Product:
A × B = {(a, b) | a A b B}
Set of all ordered pairs where first element from A, second from B

Examples:
- {1, 2} × {a, b} = {(1, a), (1, b), (2, a), (2, b)}
- {x, y} × {1, 2, 3} = {(x, 1), (x, 2), (x, 3), (y, 1), (y, 2), (y, 3)}
- × A =
- A × =

Properties:
- Generally not commutative: A × B B × A
- |A × B| = |A| × |B| (cardinality)
- A × (B C) = (A × B) (A × C)
- A × (B C) = (A × B) (A × C)

Higher-Order Products:
A × B × C = {(a, b, c) | a A, b B, c C}
A = A × A × ... × A (n times)

Examples:
- ^{2} = × (coordinate plane)
- ^{3} = × × (3D space)
- {0, 1}^{3} = {(0,0,0), (0,0,1), (0,1,0), (0,1,1), (1,0,0), (1,0,1), (1,1,0), (1,1,1)}

Applications:
- Coordinate systems
- Database tables (tuples)
- Function domains and codomains
- State spaces in computer science
- Probability sample spaces

Binary Relations

Relations Between Sets

Binary Relation:
R A × B is a relation from A to B
If (a, b) R, we write aRb or R(a, b)

Examples:
- "Less than" relation on integers: R = {(a, b) | a < b}
- "Divides" relation: R = {(a, b) | a divides b}
- "Is parent of" relation on people
- "Is enrolled in" relation from students to courses

Relation on a Set:
R A × A is a relation on set A

Examples:
- Equality: R = {(a, a) | a A}
- "Less than or equal": R = {(a, b) | a b}
- "Is sibling of" on set of people

Domain and Range:
- Domain: dom(R) = {a A | b B, (a, b) R}
- Range: range(R) = {b B | a A, (a, b) R}

Example:
R = {(1, a), (2, b), (2, c), (3, a)}
- dom(R) = {1, 2, 3}
- range(R) = {a, b, c}

Inverse Relation:
R^1 = {(b, a) | (a, b) R}

Example:
If R = {(1, 2), (3, 4), (5, 6)}
Then R^{-1} = {(2, 1), (4, 3), (6, 5)}

Composition of Relations:
If R A × B and S B × C, then
S R = {(a, c) | b B, (a, b) R (b, c) S}

Example:
R = {(1, 2), (3, 4)} (from A to B)
S = {(2, x), (4, y)} (from B to C)
S R = {(1, x), (3, y)}

Properties of Relations

Relation Properties

Let R be a relation on set A:

Reflexive:
 a A, (a, a) R
Every element is related to itself

Examples:
• on real numbers (reflexive: a a)
• = on any set
• "Is subset of" on sets

Irreflexive:
 a A, (a, a) R
No element is related to itself

Examples:
• < on real numbers
• "Is proper subset of" on sets
• "Is parent of" on people

Symmetric:
 a, b A, (a, b) R → (b, a) R
If a is related to b, then b is related to a

Examples:
• = on any set
• "Is sibling of" on people
• "Is married to" on people

Antisymmetric:
 a, b A, ((a, b) R (b, a) R) → a = b
If a is related to b and b is related to a, then a = b

Examples:
• on real numbers
• on sets
• "Divides" on positive integers

Asymmetric:
 a, b A, (a, b) R → (b, a) R
If a is related to b, then b is not related to a

Examples:
• < on real numbers
• "Is parent of" on people

Transitive:
 a, b, c A, ((a, b) R (b, c) R) → (a, c) R
If a is related to b and b is related to c, then a is related to c

Examples:
• < and on real numbers
• on sets
• "Is ancestor of" on people
• "Divides" on integers

Property Combinations:
• Equivalence relation: reflexive, symmetric, transitive
• Partial order: reflexive, antisymmetric, transitive
• Strict partial order: irreflexive, asymmetric, transitive

Equivalence Relations and Partitions

Equivalence Relations

Equivalence Relations

Definition:
A relation R on set A is an equivalence relation if it is:
1. Reflexive: a A, aRa
2. Symmetric: a, b A, aRb → bRa
3. Transitive: a, b, c A, (aRb bRc) → aRc

Examples:
• Equality (=) on any set
• Congruence modulo n: a b (mod n) iff n|(a-b)
• "Has same birthday as" on people
• "Is similar to" on triangles
• "Has same cardinality as" on sets

Equivalence Class:
For equivalence relation R on A and element a A:
[a] = {x A | xRa}
Set of all elements equivalent to a

Examples:
Congruence modulo 3 on integers:
• [0] = {..., -6, -3, 0, 3, 6, 9, ...}
• [1] = {..., -5, -2, 1, 4, 7, 10, ...}
• [2] = {..., -4, -1, 2, 5, 8, 11, ...}

Properties of Equivalence Classes:
• Every element belongs to exactly one equivalence class
• [a] = [b] if and only if aRb
• [a] [b] = or [a] = [b]
• {[a] | a A} = A

Quotient Set:
A/R = {[a] | a A}
Set of all equivalence classes

Example:
/(mod 3) = {[0], [1], [2]}

Partitions

Partitions of Sets

Definition:
A partition of set A is a collection of non-empty, pairwise disjoint subsets whose union is A

Formally: P = {A , A , ..., A } is a partition of A if:
1. A for all i
2. A A = for i j
3. A = A

Examples:
• {{1, 3}, {2, 4}, {5}} is a partition of {1, 2, 3, 4, 5}
• {Even integers, Odd integers} partitions

• {Freshmen, Sophomores, Juniors, Seniors} partitions students

Fundamental Theorem:
There is a one-to-one correspondence between:
• Equivalence relations on A
• Partitions of A

Given equivalence relation R:
→ Partition: {[a] | a A}

Given partition P = {A , A , ..., A }:
→ Equivalence relation: aRb iff a and b are in same A

Applications:
• Classification systems
• Database normalization
• Clustering algorithms
• Modular arithmetic
• Abstract algebra (quotient structures)

Example: Student Classification
Students = {Alice, Bob, Carol, Dave, Eve}
Partition by class year:
• Freshmen: {Alice, Carol}
• Sophomores: {Bob, Eve}
• Juniors: {Dave}

Corresponding equivalence relation:
"Has same class year as"
Alice ~ Carol, Bob ~ Eve, etc.

Functions as Relations

Functions Defined as Relations

Functions as Special Relations

Function Definition:
A function f: A → B is a relation f A × B such that:
 a A, !b B, (a, b) f

In other words: each element in A is related to exactly one element in B

Notation:
• f(a) = b means (a, b) f

- A is the domain
- B is the codomain
- range(f) = {f(a) | a A} B

Examples:
- f: → , f(x) = x^2
- g: {1, 2, 3} → {a, b}, g = {(1, a), (2, b), (3, a)}
- h: → , h(n) = 2n

Not Functions:
- R = {(1, a), (1, b), (2, c)} (1 maps to two values)
- S = {(1, a), (3, b)} on domain {1, 2, 3} (2 has no image)

Types of Functions:

Injective (One-to-One):
 a , a A, f(a) = f(a) → a = a
Different inputs give different outputs

Examples:
- f(x) = 2x on
- f(x) = x^3 on

Surjective (Onto):
 b B, a A, f(a) = b
Every element in codomain is an output

Examples:
- f: → , f(x) = x^3
- g: → [0, ϖ), g(x) = x^2

Bijective (One-to-One and Onto):
Function that is both injective and surjective
Establishes one-to-one correspondence between A and B

Examples:
- f: → , f(x) = 2x + 1
- g: (0, 1) → , g(x) = tan((x - 1/2))

Function Composition:
If f: A → B and g: B → C, then g f: A → C
(g f)(a) = g(f(a))

Inverse Function:
If f: A → B is bijective, then f^{-1}: B → A exists
f^{-1}(b) = a iff f(a) = b

Applications in Computer Science

Database Relations

Relational Database Model

Tables as Relations:
Database table is a relation (subset of Cartesian product)
Rows are tuples, columns are attributes

Example: Students table
Students ID × Name × Major × GPA
{(123, "Alice", "CS", 3.8), (456, "Bob", "Math", 3.5), ...}

Relational Operations:

Selection ( ):
Choose rows satisfying condition
_{GPA > 3.5}(Students) = students with GPA > 3.5

Projection ( ):
Choose specific columns
_{Name, Major}(Students) = names and majors only

Join ( ):
Combine tables based on common attributes
Students Enrollments = student info with course data

Union, Intersection, Difference:
Standard set operations on compatible relations

Keys and Constraints:
• Primary key: uniquely identifies each tuple
• Foreign key: references primary key in another relation
• Functional dependencies: determine relationships between attributes

Normalization:
Use set theory and functional dependencies to:
• Eliminate redundancy
• Prevent update anomalies
• Ensure data integrity

Example:
Instead of: Students(ID, Name, CourseID, CourseName, Grade)
Use: Students(ID, Name), Courses(CourseID, CourseName), Enrollments(ID, CourseID, Grade)

Data Structures and Algorithms

Sets in Programming

Set Data Structures:
• Hash sets: O(1) average insertion, deletion, lookup
• Tree sets: O(log n) operations, maintain order
• Bit sets: efficient for small universes

Set Operations in Code:
```python
# Python set operations
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}

union = A | B          # {1, 2, 3, 4, 5, 6}
intersection = A & B   # {3, 4}
difference = A - B     # {1, 2}
symmetric_diff = A ^ B # {1, 2, 5, 6}
```

Applications: • Removing duplicates from data • Membership testing • Set-based algorithms (graph algorithms) • Database query optimization • Caching and memoization

Graph Theory: Graphs as relations on vertex sets • Adjacency relation: R V × V • Path relation: transitive closure of adjacency • Equivalence classes: connected components

Algorithm Design: • Union-Find data structure (disjoint sets) • Set cover and hitting set problems • Bloom filters (probabilistic set membership) • Set intersection algorithms

Summary and Key Concepts

Sets and relations provide the fundamental language for describing mathematical structures and

Chapter Summary

Essential Skills Mastered: Set notation, membership, and basic operations Set properties and algebraic laws Cartesian products and ordered pairs Binary relations and their properties Equivalence relations and partitions Functions as special relations Applications in databases and programming

Key Concepts: • Sets as collections of distinct objects • Set operations: union, intersection, complement, difference • Relations as subsets of Cartesian products • Equivalence relations and their connection to partitions • Functions as single-valued relations • Database tables as relations

Fundamental Operations: • Union ( ): elements in either set • Intersection ( ): elements in both sets • Complement (’): elements not in set • Cartesian product (×): ordered pairs • Relation composition: chaining relationships

Problem-Solving Tools: • Venn diagrams for visualization • Set algebra for simplification • Equivalence classes for classification • Function properties for analysis • Relational operations for data manipulation

Applications Covered: • Database design and relational algebra • Programming data structures • Mathematical foundations • Classification and equivalence • Graph theory and networks

Next Steps: Set and relation concepts prepare you for: - Functions and their properties - Graph theory and network analysis - Database design and query optimization - Abstract algebra and mathematical structures - Algorithm design and data structures

```
Sets and relations represent the fundamental building blocks of mathematical reasoning and comp

Understanding sets and relations enables you to think precisely about collections of objects ar

`<!-- quarto-file-metadata: eyJyZXNvdXJjZURpciI6ImNvbnRlbnQvMDYtZGlzY3JldGUtbWF0aGVtYXRpY3MifQ=

```{=html}
<!-- quarto-file-metadata: eyJyZXNvdXJjZURpciI6ImNvbnRlbnQvMDYtZGlzY3JldGUtbWF0aGVtYXRpY3MiLCJi
```
```

Combinatorics: The Art and Science of Counting

Introduction to Combinatorics

Combinatorics is the branch of mathematics concerned with counting, arranging, and selecting objects. It provides systematic methods for solving problems involving discrete structures and finite sets, making it essential for probability theory, computer science, and many areas of applied mathematics.

From determining the number of ways to arrange books on a shelf to analyzing the complexity of algorithms, combinatorics gives us powerful tools for quantifying possibilities and making informed decisions in uncertain situations.

```
Combinatorics Applications


Pure Mathematics:
• Probability theory foundations
• Graph theory enumeration
• Number theory problems
• Algebraic combinatorics

Computer Science:
• Algorithm analysis and complexity
• Data structure design
• Cryptography and coding theory
• Network analysis and optimization

Real-World Applications:
• Quality control and testing
• Resource allocation and scheduling
• Game theory and strategy
• Bioinformatics and genetics
• Market research and polling

Key Questions:
• How many ways can we arrange objects?
• How many ways can we select objects?
• What is the probability of a specific outcome?
• How can we count efficiently without listing everything?
```

Fundamental Counting Principles

The Addition Principle

The addition principle (also called the sum rule) is used when we have mutually exclusive choices.

```
Addition Principle


Statement:
If a task can be performed in m ways OR in n ways, where these ways are mutually exclusive, the

General Form:
If sets A , A , ..., A  are pairwise disjoint, then:
|A   A   ...  A | = |A | + |A | + ... + |A |

Examples:

Example 1: Menu Selection
A restaurant offers:
• 3 appetizers
• 4 main courses
• 2 desserts

If you can choose exactly one item, how many choices do you have?
Answer: 3 + 4 + 2 = 9 choices

Example 2: Transportation
To get to work, you can:
• Take one of 3 bus routes
• Take one of 2 train routes
• Drive (1 way)

Total ways to get to work: 3 + 2 + 1 = 6 ways

Example 3: Password Characters
A password character can be:
• One of 26 lowercase letters
• One of 26 uppercase letters
• One of 10 digits
• One of 8 special symbols

Total possible characters: 26 + 26 + 10 + 8 = 70

Key Requirements:
• Choices must be mutually exclusive (can't do both)
• Must account for all possibilities
```

• No overlap between categories

Common Mistakes:
• Forgetting that choices are exclusive
• Double-counting overlapping cases
• Missing some categories

The Multiplication Principle

The multiplication principle (also called the product rule) is used when we have sequential choices or independent events.

Multiplication Principle

Statement:
If a task consists of k steps, where step 1 can be performed in n ways, step 2 can be performe

General Form:
|A × A × ... × A| = |A| × |A| × ... × |A|

Examples:

Example 1: Complete Meal
A restaurant offers:
• 3 appetizers
• 4 main courses
• 2 desserts

If you must choose one from each category, how many complete meals are possible?
Answer: 3 × 4 × 2 = 24 complete meals

Example 2: License Plates
A license plate has:
• 3 letters (26 choices each)
• 3 digits (10 choices each)

Total possible license plates: $26^3 \times 10^3 = 17{,}576 \times 1{,}000 = 17{,}576{,}000$

Example 3: Password Creation
A 4-character password where:
• Each character can be any of 70 possible symbols
• Repetition is allowed

Total possible passwords: 70 = 24,010,000

Example 4: Committee Formation

Form a committee with:
• 1 president (chosen from 10 people)
• 1 vice president (chosen from remaining 9 people)
• 1 secretary (chosen from remaining 8 people)

Total ways: 10 × 9 × 8 = 720

Key Requirements:
• Steps must be performed in sequence
• Number of choices at each step is independent of previous choices (or depends in a known way)
• All combinations are valid

Dependent Choices:
When later choices depend on earlier ones:

Example: Seating Arrangement
Arrange 5 people in a row:
• First position: 5 choices
• Second position: 4 remaining choices
• Third position: 3 remaining choices
• Fourth position: 2 remaining choices
• Fifth position: 1 remaining choice

Total arrangements: 5 × 4 × 3 × 2 × 1 = 5! = 120

Inclusion-Exclusion Principle

The inclusion-exclusion principle handles counting when sets overlap.

Inclusion-Exclusion Principle

Two Sets:
|A B| = |A| + |B| - |A B|

Three Sets:
|A B C| = |A| + |B| + |C| - |A B| - |A C| - |B C| + |A B C|

General Form (n sets):
|A A ... A | = Σ|A | - Σ|A A | + Σ|A A A | - ... + (-1) ¹|A A ... A |

Examples:

Example 1: Student Enrollment
In a class of 100 students:
• 60 study mathematics
• 50 study physics

```
• 30 study both mathematics and physics

How many study at least one subject?
Solution:
|M ∪ P| = |M| + |P| - |M ∩ P| = 60 + 50 - 30 = 80 students

Example 2: Programming Languages
Among 200 programmers:
• 120 know Java
• 80 know Python
• 90 know C++
• 50 know both Java and Python
• 60 know both Java and C++
• 40 know both Python and C++
• 30 know all three languages

How many know at least one language?
Solution:
|J ∪ P ∪ C| = 120 + 80 + 90 - 50 - 60 - 40 + 30 = 170 programmers

Example 3: Divisibility
How many integers from 1 to 1000 are divisible by 2, 3, or 5?

Let A = multiples of 2, B = multiples of 3, C = multiples of 5
|A| = 1000/2 = 500
|B| = 1000/3 = 333
|C| = 1000/5 = 200
|A ∩ B| = 1000/6 = 166 (multiples of lcm(2,3) = 6)
|A ∩ C| = 1000/10 = 100 (multiples of lcm(2,5) = 10)
|B ∩ C| = 1000/15 = 66 (multiples of lcm(3,5) = 15)
|A ∩ B ∩ C| = 1000/30 = 33 (multiples of lcm(2,3,5) = 30)

Answer: 500 + 333 + 200 - 166 - 100 - 66 + 33 = 734

Applications:
• Probability calculations
• Set theory problems
• Number theory (divisibility)
• Database queries with multiple conditions
• Network reliability analysis
```

Permutations

Basic Permutations

Permutations count the number of ways to arrange objects where order matters.

Permutation Fundamentals

Definition:
A permutation is an arrangement of objects where order matters.

n-Factorial:
n! = n × (n-1) × (n-2) × ... × 2 × 1
By convention: 0! = 1

Examples:
• 3! = 3 × 2 × 1 = 6
• 5! = 5 × 4 × 3 × 2 × 1 = 120
• 10! = 3,628,800

Permutations of n Objects:
Number of ways to arrange n distinct objects = n!

Example: Arranging Books
How many ways can you arrange 5 different books on a shelf?
Answer: 5! = 120 ways

Reasoning:
• First position: 5 choices
• Second position: 4 remaining choices
• Third position: 3 remaining choices
• Fourth position: 2 remaining choices
• Fifth position: 1 remaining choice
Total: 5 × 4 × 3 × 2 × 1 = 5! = 120

r-Permutations of n Objects:
P(n,r) = n!/(n-r)! = number of ways to arrange r objects from n distinct objects

Formula Derivation:
• First position: n choices
• Second position: n-1 choices
• ...
• r-th position: n-r+1 choices
Total: n × (n-1) × ... × (n-r+1) = n!/(n-r)!

Examples:

Example 1: Race Positions
In a race with 10 runners, how many ways can the top 3 positions be filled?
P(10,3) = 10!/(10-3)! = 10!/7! = 10 × 9 × 8 = 720

Example 2: Committee Officers
From 15 club members, how many ways can you choose a president, vice president, and secretary?

P(15,3) = 15!/12! = 15 × 14 × 13 = 2,730

Example 3: Password Creation
How many 4-letter passwords can be formed using letters A-Z with no repetition?
P(26,4) = 26!/22! = 26 × 25 × 24 × 23 = 358,800

Special Cases:
• P(n,n) = n!/(n-n)! = n!/0! = n!/1 = n!
• P(n,1) = n!/(n-1)! = n
• P(n,0) = n!/n! = 1 (empty arrangement)

Permutations with Restrictions

Restricted Permutations

Circular Permutations:
Arrangements around a circle where rotations are considered identical
Number of circular permutations of n objects = (n-1)!

Example: Round Table Seating
How many ways can 6 people sit around a circular table?
Answer: (6-1)! = 5! = 120 ways

Reasoning: Fix one person's position to eliminate rotational symmetry, then arrange the remaini

Permutations with Identical Objects:
When some objects are identical, we must account for overcounting

Formula: n!/(n ! × n ! × ... × n !)
where n , n , ..., n are the frequencies of each type of identical object

Examples:

Example 1: Letter Arrangements
How many distinct arrangements of the letters in "MISSISSIPPI"?
M: 1, I: 4, S: 4, P: 2 (total: 11 letters)
Answer: 11!/(1! × 4! × 4! × 2!) = 39,916,800/(1 × 24 × 24 × 2) = 34,650

Example 2: Binary Strings
How many 8-bit binary strings contain exactly three 1's?
Answer: 8!/(3! × 5!) = 40,320/(6 × 120) = 56

Restricted Positions:
Objects that must be in specific positions or cannot be in certain positions

Example 1: Seating with Constraints

```
Arrange 5 people in a row where Alice and Bob must sit together.
Solution: Treat Alice and Bob as a single unit
• Arrange 4 units: 4! = 24 ways
• Arrange Alice and Bob within their unit: 2! = 2 ways
Total: 4! × 2! = 48 ways

Example 2: Vowels and Consonants
Arrange the letters of "COMPUTER" so that vowels and consonants alternate.
COMPUTER has 3 vowels (O, U, E) and 5 consonants (C, M, P, T, R)
Pattern must be: C-V-C-V-C-V-C-C
• Arrange 5 consonants in 5 positions: 5! = 120
• Arrange 3 vowels in 3 positions: 3! = 6
Total: 5! × 3! = 720 ways

Derangements:
Permutations where no object is in its original position
Number of derangements of n objects: D  = n! × Σ   (-1) /k!

Example: Hat Check Problem
n people check their hats. In how many ways can the hats be returned so that no one gets their
For n = 4: D  = 4! × (1 - 1 + 1/2 - 1/6 + 1/24) = 24 × 9/24 = 9
```

Combinations

Basic Combinations

Combinations count the number of ways to select objects where order doesn't matter.

```
Combination Fundamentals



Definition:
A combination is a selection of objects where order doesn't matter.

r-Combinations of n Objects:
C(n,r) = (n choose r) = n!/(r!(n-r)!)

Alternative Notations:
• C(n,r)
•  C
• (n r) (binomial coefficient)

Formula Derivation:
• Number of r-permutations: P(n,r) = n!/(n-r)!
• Each combination corresponds to r! permutations
• Number of combinations: P(n,r)/r! = n!/(r!(n-r)!)
```

Examples:

Example 1: Committee Selection
From 10 people, how many ways can you choose a 3-person committee?
C(10,3) = 10!/(3!7!) = (10 × 9 × 8)/(3 × 2 × 1) = 720/6 = 120

Example 2: Pizza Toppings
A pizza shop offers 12 toppings. How many ways can you choose 4 toppings?
C(12,4) = 12!/(4!8!) = (12 × 11 × 10 × 9)/(4 × 3 × 2 × 1) = 11,880/24 = 495

Example 3: Card Hands
How many 5-card poker hands can be dealt from a standard 52-card deck?
C(52,5) = 52!/(5!47!) = 2,598,960

Properties of Binomial Coefficients:

Symmetry:
C(n,r) = C(n,n-r)

Pascal's Identity:
C(n,r) = C(n-1,r-1) + C(n-1,r)

Special Values:
• C(n,0) = 1 (one way to choose nothing)
• C(n,1) = n (n ways to choose one object)
• C(n,n) = 1 (one way to choose everything)
• C(n,r) = 0 if r > n

Sum Property:
Σ C(n,r) = 2 (total number of subsets)

Pascal's Triangle:
Row n contains the values C(n,0), C(n,1), ..., C(n,n)

```
Row 0:           1
Row 1:         1   1
Row 2:       1   2   1
Row 3:     1   3   3   1
Row 4:   1   4   6   4   1
Row 5: 1   5  10  10   5   1
```

Each entry is the sum of the two entries above it.

Combinations with Repetition

Combinations with Repetition

Problem Type:
Choose r objects from n types where repetition is allowed and order doesn't matter.

Formula:
Number of r-combinations with repetition from n types = C(n+r-1, r) = C(n+r-1, n-1)

Alternative Approach (Stars and Bars):
Distribute r identical objects into n distinct bins
Equivalent to arranging r stars and n-1 bars
Total positions: r + n - 1
Choose r positions for stars: C(r+n-1, r)

Examples:

Example 1: Ice Cream Scoops
An ice cream shop has 5 flavors. How many ways can you choose 3 scoops (repetition allowed)?
Answer: C(5+3-1, 3) = C(7,3) = 35

Visualization with stars and bars:
3 scoops, 5 flavors → 3 stars, 4 bars
Example: **|*|| represents 2 vanilla, 1 chocolate, 0 strawberry, 0 mint, 0 pistachio

Example 2: Coin Distribution
How many ways can you distribute 10 identical coins among 4 people?
Answer: C(10+4-1, 10) = C(13,10) = C(13,3) = 286

Example 3: Equation Solutions
How many non-negative integer solutions are there to x + x + x + x = 15?
Answer: C(15+4-1, 15) = C(18,15) = C(18,3) = 816

Constraints on Solutions:
For positive integer solutions (each x 1):
Transform to y = x - 1 0
Solve y + y + ... + y = k - n

Example: Positive integer solutions to x + x + x = 10
Transform to y + y + y = 7 where y 0
Answer: C(7+3-1, 7) = C(9,7) = C(9,2) = 36

Multiset Coefficients:
The number C(n+r-1, r) is also called a multiset coefficient
Denoted as ((n r)) or MC(n,r)
Counts multisets of size r from n types of elements

Advanced Combination Problems

Complex Combination Problems

Combinations with Multiple Constraints:

Example 1: Committee with Requirements
From 8 men and 6 women, form a 5-person committee with at least 2 women.

Method 1 (Direct counting):
• Exactly 2 women: C(6,2) × C(8,3) = 15 × 56 = 840
• Exactly 3 women: C(6,3) × C(8,2) = 20 × 28 = 560
• Exactly 4 women: C(6,4) × C(8,1) = 15 × 8 = 120
• Exactly 5 women: C(6,5) × C(8,0) = 6 × 1 = 6
Total: 840 + 560 + 120 + 6 = 1,526

Method 2 (Complementary counting):
Total committees: C(14,5) = 2,002
Committees with 0 women: C(6,0) × C(8,5) = 1 × 56 = 56
Committees with 1 woman: C(6,1) × C(8,4) = 6 × 70 = 420
Answer: 2,002 - 56 - 420 = 1,526

Example 2: Distributing Objects
Distribute 20 identical balls into 5 distinct boxes such that each box gets at least 2 balls.

Solution:
First place 2 balls in each box (uses 10 balls)
Distribute remaining 10 balls freely among 5 boxes
Answer: C(10+5-1, 10) = C(14,10) = C(14,4) = 1,001

Inclusion-Exclusion with Combinations:

Example: Arrangements with Forbidden Positions
How many ways can 5 people sit in 5 chairs if persons A and B cannot sit in chairs 1 and 2 resp

Let S = all arrangements = 5! = 120
Let A = arrangements where person A sits in chair 1
Let A = arrangements where person B sits in chair 2

|A | = 4! = 24 (fix A in chair 1, arrange others)
|A | = 4! = 24 (fix B in chair 2, arrange others)
|A   A | = 3! = 6 (fix A in chair 1 and B in chair 2)

Answer: |S| - |A   A | = 120 - (24 + 24 - 6) = 78

Generating Functions Approach:
Use algebraic methods to solve complex counting problems

Coefficient of x in $(1+x)$ is $C(n,r)$
Coefficient of x in $(1+x+x^2+\ldots)$ is $C(n+r-1,r)$

Example: Number of ways to make change for \$1.00 using quarters, dimes, nickels, and pennies
Generating function: $(1+x^2+x+x+x^1)(1+x^1+x^2+\ldots)(1+x+x^1+\ldots)(1+x+x^2+\ldots)$
Find coefficient of x^1

The Binomial Theorem

Binomial Expansions

Binomial Theorem

Statement:
$(x + y) = \Sigma \; C(n,k)\, x \; y$

Expanded Form:
$(x + y) = C(n,0)x\,y + C(n,1)x^1y^1 + C(n,2)x^2y^2 + \ldots + C(n,n)x\,y$

Examples:

$(x + y)^2 = C(2,0)x^2 + C(2,1)xy + C(2,2)y^2 = x^2 + 2xy + y^2$

$(x + y)^3 = C(3,0)x^3 + C(3,1)x^2y + C(3,2)xy^2 + C(3,3)y^3 = x^3 + 3x^2y + 3xy^2 + y^3$

$(x + y) = x + 4x^3y + 6x^2y^2 + 4xy^3 + y$

General Term:
The $(k+1)$th term in the expansion of $(x + y)$ is:
$T = C(n,k)\, x \; y$

Applications:

Example 1: Specific Term
Find the coefficient of $x\,y^3$ in $(x + y)$
Solution: $C(8,3) = 56$ (since we need $x^3y^3 = x\,y^3$)

Example 2: Numerical Calculation
Calculate 1.01^1 using binomial theorem
$(1 + 0.01)^1 = \Sigma \; ^1 \; C(10,k)(0.01)$
$1 + 10(0.01) + 45(0.01)^2 + 120(0.01)^3 + \ldots$
$1 + 0.1 + 0.0045 + 0.00012 + \ldots \quad 1.10462$

Example 3: Alternating Series
$(1 - x) = \Sigma \; C(n,k)(-1)\, x = \Sigma \; (-1)\, C(n,k)x$

Special Cases:
• (1 + x) = Σ C(n,k)x
• (1 + 1) = 2 = Σ C(n,k)
• (1 - 1) = 0 = Σ (-1) C(n,k) (for n > 0)

Multinomial Theorem:
(x + x + ... + x) = Σ (n!)/(k !k !...k !) x ^k x ^k ... x ^k
where the sum is over all non-negative integers k , k , ..., k such that k + k + ... + k = n

Example: $(x + y + z)^3 = x^3 + y^3 + z^3 + 3x^2y + 3x^2z + 3y^2x + 3y^2z + 3z^2x + 3z^2y + 6xyz$

Applications in Computer Science

Algorithm Analysis

Combinatorics in Algorithm Complexity

Sorting Algorithms:
Comparison-based sorting has lower bound Ω(n log n)
Proof: n! possible permutations, each comparison gives at most 2 outcomes
Need at least log (n!) n log n comparisons

Search Problems:
Binary search: log n comparisons for n sorted elements
Linear search: average n/2 comparisons, worst case n

Graph Algorithms:
• Complete graph K has C(n,2) = n(n-1)/2 edges
• Number of spanning trees in K : n^(n-2) (Cayley's formula)
• Number of Hamiltonian cycles in K : (n-1)!/2

Subset Generation:
Generate all 2 subsets of n-element set
Gray code: generate subsets so consecutive ones differ by one element

Example: Subsets of {1,2,3}
, {1}, {1,2}, {2}, {2,3}, {1,2,3}, {1,3}, {3}

Permutation Generation:
Generate all n! permutations
Lexicographic order, Heap's algorithm, Johnson-Trotter algorithm

Dynamic Programming:
Many DP problems involve combinatorial counting

- Fibonacci numbers: F(n) = F(n-1) + F(n-2)
- Catalan numbers: C(n) = C(0)C(n-1) + C(1)C(n-2) + ... + C(n-1)C(0)
- Binomial coefficients: C(n,k) = C(n-1,k-1) + C(n-1,k)

Randomized Algorithms:
Probability analysis uses combinatorics
- QuickSort expected time: O(n log n)
- Hash table collision probability
- Monte Carlo methods

Cryptography and Security

Combinatorics in Cryptography

Key Space Analysis:
Security depends on size of key space

Examples:
- DES: 2 possible keys
- AES-128: 2^{12} possible keys
- RSA-2048: approximately $2^{}$ possible keys

Password Strength:
n-character password with alphabet size k: k possibilities

Example: 8-character password
- Lowercase only: 26 2.1×10^{11}
- Lowercase + uppercase: 52 5.3×10^{13}
- Alphanumeric: 62 2.2×10^{1}
- Full ASCII: 95 6.6×10^{1}

Brute Force Attack Time:
Average time to break: (key space size)/2 × (time per attempt)

Birthday Paradox:
Probability of collision in hash function with n-bit output
Approximately 50% chance after 2^(n/2) attempts
Important for hash function security

Combinatorial Cryptanalysis:
- Frequency analysis of substitution ciphers
- Pattern analysis in block ciphers
- Linear and differential cryptanalysis

Error-Correcting Codes:
Use combinatorics to design codes that detect/correct errors

- Hamming codes: detect and correct single-bit errors
- Reed-Solomon codes: used in CDs, DVDs, QR codes
- BCH codes: multiple error correction

Example: Hamming(7,4) code
4 data bits, 3 parity bits
Can correct any single-bit error
$2^4 = 16$ valid codewords out of $2^7 = 128$ possible 7-bit strings

Network Analysis and Graph Theory

Combinatorics in Networks

Network Topology:
- Complete network: C(n,2) connections for n nodes
- Star network: n-1 connections
- Ring network: n connections
- Tree network: n-1 connections (minimum for connectivity)

Routing Problems:
Number of paths between nodes in different topologies

Example: Grid Network
Paths from (0,0) to (m,n) in rectangular grid
Must make m right moves and n up moves
Total moves: m + n
Number of paths: C(m+n, m) = C(m+n, n)

Network Reliability:
Probability that network remains connected when edges fail
Use inclusion-exclusion principle

Social Network Analysis:
- Clustering coefficient: ratio of actual triangles to possible triangles
- Degree distribution: how many nodes have each degree
- Small world phenomenon: most pairs connected by short paths

Internet Structure:
- AS (Autonomous System) graph: power-law degree distribution
- Router-level topology: hierarchical structure
- Web graph: scale-free network properties

Communication Complexity:
Minimum number of bits needed for distributed computation
Related to combinatorial properties of problem structure

```
Example: Set Disjointness
Alice has set A, Bob has set B
Determine if A   B =
Communication complexity: Θ(n) bits required

Load Balancing:
Distribute n jobs among m servers
Balls-and-bins problem: expected maximum load
Related to occupancy problems in probability
```

Summary and Key Concepts

Combinatorics provides essential tools for counting and analyzing discrete structures, with applications spanning mathematics, computer science, and real-world problem-solving.

```
Chapter Summary


Essential Skills Mastered:
  Fundamental counting principles (addition and multiplication)
  Inclusion-exclusion principle for overlapping sets
  Permutations: arrangements where order matters
  Combinations: selections where order doesn't matter
  Binomial theorem and its applications
  Advanced counting techniques and problem-solving strategies

Key Concepts:
• Addition principle: mutually exclusive choices
• Multiplication principle: sequential or independent choices
• Permutations: P(n,r) = n!/(n-r)!
• Combinations: C(n,r) = n!/(r!(n-r)!)
• Binomial coefficients and Pascal's triangle
• Stars and bars method for distributions

Problem-Solving Strategies:
• Identify whether order matters (permutation vs combination)
• Use complementary counting for complex constraints
• Apply inclusion-exclusion for overlapping conditions
• Transform problems using bijections
• Use generating functions for advanced problems

Fundamental Formulas:
• Permutations: P(n,r) = n!/(n-r)!
• Combinations: C(n,r) = n!/(r!(n-r)!)
• Combinations with repetition: C(n+r-1,r)
• Binomial theorem: (x+y)  = Σ C(n,k)x^(n-k)y^k
```

```
• Inclusion-exclusion: |A B| = |A| + |B| - |A B|

Applications Covered:
• Algorithm analysis and complexity theory
• Cryptography and security analysis
• Network topology and routing problems
• Probability theory foundations
• Database query optimization
• Resource allocation and scheduling

Next Steps:
Combinatorial skills prepare you for:
- Probability theory and statistics
- Graph theory and network analysis
- Algorithm design and analysis
- Cryptography and coding theory
- Discrete optimization problems
- Advanced mathematics and research
```

Combinatorics represents the systematic study of counting and arrangement, providing powerful tools for analyzing discrete structures and solving complex problems. The techniques developed in this chapter - from basic counting principles to advanced methods like inclusion-exclusion and generating functions - form essential foundations for probability theory, algorithm analysis, cryptography, and many other areas of mathematics and computer science.

Understanding combinatorics enables you to quantify possibilities, analyze algorithms, design secure systems, and solve optimization problems across diverse fields. Whether you're calculating the complexity of an algorithm, analyzing the security of a cryptographic system, or modeling network behavior, combinatorial thinking provides the mathematical framework for precise analysis and informed decision-making in our increasingly complex technological world.

Recurrence Relations

Recurrences describe sequences by relating a_n to earlier terms. They appear in algorithm runtime ($T(n) = 2T(n/2)+\Theta(n)$), combinatorics, dynamic programming, and amortized analysis. Mastering closed forms and the Master theorem is core discrete math for CS.

Diagram: expand a recurrence

```
T(n)
    T(n/2)
          T(n/4)
          T(n/4)
    T(n/2)
           ...
           ...

levels   log_b n, work per level depends on f(n)
```

1. What is a recurrence?

A **recurrence relation** defines a sequence (a_n) by one or more base cases plus a rule that expresses a_n in terms of previous values.

Example (factorial via recurrence).

$$a_0 = 1, \qquad a_n = n\, a_{n-1} \ (n \geq 1) \quad \Rightarrow \quad a_n = n!.$$

Example (divide-and-conquer cost). Mergesort:

$$T(1) = \Theta(1), \qquad T(n) = 2T(n/2) + \Theta(n).$$

Why CS needs closed forms

- Compare algorithms asymptotically without expanding trees by hand every time
- Prove DP solutions match combinatorial counts
- Predict growth: exponential vs polynomial vs $n \log n$

2. Linear homogeneous recurrences (constant coefficients)

Definition. Order k linear homogeneous:

$$a_n = c_1 a_{n-1} + c_2 a_{n-2} + \cdots + c_k a_{n-k},$$

with constant c_i and k initial conditions $a_0, \ldots, a_{k-1}$.

Characteristic equation method

Guess $a_n = r^n$ ($r \neq 0$). Substitute:

$$r^n = c_1 r^{n-1} + \cdots + c_k r^{n-k}.$$

Divide by r^{n-k}:

$$r^k - c_1 r^{k-1} - \cdots - c_k = 0.$$

Distinct real roots $r_1, \ldots, r_k$: general solution

$$a_n = A_1 r_1^n + A_2 r_2^n + \cdots + A_k r_k^n.$$

Fit A_i from initial conditions.

Worked example 1 — second order

$$a_n = 5a_{n-1} - 6a_{n-2}, \qquad a_0 = 2,\ a_1 = 5.$$

Characteristic: $r^2 - 5r + 6 = 0 = (r-2)(r-3)$.

$$a_n = A \cdot 2^n + B \cdot 3^n.$$

$n = 0$: $A + B = 2$.
$n = 1$: $2A + 3B = 5$.
Subtract: $A + 2B = 3$ wait carefully: from $2A + 3B = 5$ and $2A + 2B = 4$ $\quad B = 1$, $A = 1$.

$$a_n = 2^n + 3^n.$$

Check $n = 2$: recurrence gives $5 \cdot 5 - 6 \cdot 2 = 13$; formula $4 + 9 = 13$.

Repeated roots

If root r has multiplicity m, include factors $n^j r^n$ for $j = 0, \dots, m-1$.

Example: $a_n = 4a_{n-1} - 4a_{n-2}$ has double root $r = 2$:

$$a_n = (A + Bn)\, 2^n.$$

Complex roots

Conjugate pairs $r = \rho e^{\pm i\theta}$ yield real form

$$a_n = \rho^n (A\cos(n\theta) + B\sin(n\theta)).$$

Useful for oscillatory sequences (e.g. certain signal models).

3. Fibonacci and Binet's formula

Define $F_0 = 0$, $F_1 = 1$, $F_n = F_{n-1} + F_{n-2}$ for $n \geq 2$.

Characteristic $r^2 - r - 1 = 0$:

$$\varphi = \frac{1+\sqrt{5}}{2}, \qquad \hat{\varphi} = \frac{1-\sqrt{5}}{2}.$$

Binet:

$$F_n = \frac{\varphi^n - \hat{\varphi}^n}{\sqrt{5}}.$$

Since $|\hat{\varphi}| < 1$, F_n is the integer closest to $\varphi^n/\sqrt{5}$.

Worked example 2 — identity via recurrence

Prove $F_0 + \cdots + F_n = F_{n+2} - 1$ by induction, or derive from telescoping using $F_{k+2} - F_{k+1} = F_k$.

CS appearances

- Naive recursive Fibonacci is exponential time; memoized / bottom-up is $O(n)$
- Fibonacci heaps, Zeckendorf representation, analysis of Euclidean algorithm (worst case related to Fibonacci ratios)

4. Non-homogeneous linear recurrences

$$a_n = c_1 a_{n-1} + \cdots + c_k a_{n-k} + g(n).$$

General solution = homogeneous general solution + one particular solution.

$g(n)$ form	Particular ansatz (if not a root case)
constant C	constant K
polynomial degree d	polynomial degree d
q^n	Kq^n
q^n and q is a root of mult. m	$Kn^m q^n$

Worked example 3

$a_n = 2a_{n-1} + 3$, $a_0 = 1$.

Homogeneous: $a_n^{(h)} = A \cdot 2^n$.
Particular: constant $K = 2K + 3 \quad K = -3$.
General: $a_n = A \cdot 2^n - 3$.
$a_0 = 1$: $A - 3 = 1 \quad A = 4$. So $a_n = 4 \cdot 2^n - 3 = 2^{n+2} - 3$.

5. Master theorem (algorithmic divide-and-conquer)

For

$$T(n) = a\,T(n/b) + f(n), \qquad a \geq 1,\ b > 1,$$

compare $f(n)$ to $n^{\log_b a}$ (the work of the leaves if $f \equiv 0$ up to constants).

Case	Condition (rough)	Solution
1	$f(n) = O(n^{\log_b a - \varepsilon})$ for some $\varepsilon > 0$	$T(n) = \Theta(n^{\log_b a})$
2	$f(n) = \Theta(n^{\log_b a} \log^k n)$, $k \geq 0$	$T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$ (classic $k = 0$: $\Theta(n^{\log_b a} \log n)$)
3	$f(n) = \Omega(n^{\log_b a + \varepsilon})$ + regularity	$T(n) = \Theta(f(n))$

Regularity (case 3): $af(n/b) \leq cf(n)$ for some $c < 1$ and large n.

Worked example 4 — merge sort

$T(n) = 2T(n/2) + \Theta(n)$: $a = 2$, $b = 2$, $\log_b a = 1$, $f = \Theta(n) = \Theta(n^{\log_b a})$. Case 2 $\quad \Theta(n \log n)$.

Worked example 5 — binary search

$T(n) = T(n/2) + \Theta(1)$: $a = 1$, $b = 2$, $\log_b a = 0$, $n^0 = 1$, $f = \Theta(1)$. Case 2 $\Theta(\log n)$.

Worked example 6 — case 1

$T(n) = 4T(n/2) + n$: $\log_2 4 = 2$, $f = n = O(n^{2-\varepsilon})$. Case 1 $\Theta(n^2)$.

Worked example 7 — case 3

$T(n) = 2T(n/2) + n^2$: $\log_2 2 = 1$, $f = n^2 = \Omega(n^{1+\varepsilon})$. Regularity: $2(n/2)^2 = n^2/2 \le cn^2$ with $c = 1/2$. Case 3 $\Theta(n^2)$.

When Master does not apply cleanly

- Uneven splits: $T(n) = T(\lfloor n/3 \rfloor) + T(\lceil 2n/3 \rceil) + \Theta(n)$ — use recursion tree / Akra–Bazzi
- f oscillates or is not polynomial-comparable to $n^{\log_b a}$
- Floors/ceilings usually do not change asymptotics for standard cases

6. Recursion trees and substitution

Tree method

Draw levels; sum work per level; sum over $\approx \log_b n$ levels (plus base).

Merge sort tree: each level $\Theta(n)$ work, $\Theta(\log n)$ levels $\Theta(n \log n)$.

Substitution (guess + induction)

1. Guess form, e.g. $T(n) \le cn \log n$
2. Prove by strong induction, carefully handling base and floors
3. Adjust constants; sometimes subtract lower-order terms ($cn \log n - dn$) to absorb edges

Worked example 8 — linear scan recurrence

$T(n) = T(n-1) + \Theta(n)$, $T(1) = \Theta(1)$.
Unroll: $T(n) = \Theta(1 + 2 + \cdots + n) = \Theta(n^2)$.

7. Generating functions (preview)

Encode sequence as

$$A(x) = \sum_{n=0}^{\infty} a_n x^n.$$

A linear recurrence becomes an algebraic equation for $A(x)$; partial fractions recover closed forms.

Fibonacci generating function:

$$\sum_{n=0}^{\infty} F_n x^n = \frac{x}{1 - x - x^2}.$$

Useful for hard counting recurrences and asymptotic extraction (singularity analysis — advanced).

8. Recurrences in dynamic programming

DP often writes:

$$\mathrm{OPT}(n) = \min_i \big(\mathrm{cost}(i) + \mathrm{OPT}(n - i)\big)$$

or similar. The recurrence is a **specification**; an algorithm evaluates it with memoization/tabulation in an order that respects dependencies.

Classic: 0/1 knapsack, edit distance, matrix chain ordering — all recurrence-first designs.

Worked example 9 — no consecutive 1s

Let a_n = number of binary strings of length n with no two consecutive 1s.

- Ends in 0: a_{n-1} prefixes
- Ends in 01? Ends in 1 previous must end in 0: a_{n-2}

$$a_n = a_{n-1} + a_{n-2}, \quad a_1 = 2\ (0, 1),\ a_2 = 3\ (00, 01, 10).$$

So $a_n = F_{n+2}$ (shifted Fibonacci).

9. Solving checklist

1. Identify order and whether linear / constant coeff / homogeneous
2. For D&C runtimes: try Master theorem first
3. Else: unroll, tree, or characteristic equation
4. Fit constants; verify small n
5. State asymptotics with Θ carefully (constants matter for engineering, not only big-O class)

10. Pitfalls

1. Forgetting base cases when implementing recursive algorithms
2. Applying Master case 2 when f is not $\Theta(n^{\log_b a})$ (off by polylog needs the extended form)
3. Assuming $T(n) = 2T(n/2) + n$ without Θ hides constants that matter in practice
4. Integer floors: $n/2$ vs $\lfloor n/2 \rfloor$ — usually OK asymptotically, not always for exact closed forms
5. Naive recursion exponential time even when closed form is simple (Fibonacci)

11. Checkpoint

- Solve second-order linear homogeneous recurrences via characteristic roots
- State and apply all three Master cases with an example each
- Unroll $T(n) = T(n-1) + f(n)$
- Translate a counting problem into a recurrence
- Explain when to prefer tree vs substitution vs Master

Exercises

Easy

1. Solve $a_n = 3a_{n-1}$, $a_0 = 2$.

2. Solve $a_n = a_{n-1} + 2a_{n-2}$, $a_0 = 1$, $a_1 = 1$.

3. Apply Master theorem to $T(n) = 4T(n/2) + n$.

4. $T(n) = T(n-1) + \Theta(n)$ — what is $T(n)$?

5. Write a recurrence for the number of binary strings of length n with no two consecutive 1s.

6. Compute F_6 from the recurrence and compare to Binet rounding.

Medium

7. Solve $a_n = 6a_{n-1} - 9a_{n-2}$, $a_0 = 1$, $a_1 = 6$ (repeated root).

8. Solve $a_n = 2a_{n-1} + n$, $a_0 = 0$ (non-homogeneous).

9. Apply Master: $T(n) = 9T(n/3) + n$, $T(n) = 3T(n/2) + n^2$, $T(n) = 2T(n/2) + n\log n$ (state case).

10. Prove by induction that $F_n \le \varphi^{n-1}$ for $n \ge 1$ (adjust base carefully).

11. Recursion tree for $T(n) = 3T(n/2) + \Theta(n)$: sum geometric levels.

Challenge

12. Prove by induction your closed form in exercise 1.

13. Derive Binet's formula from the characteristic equation and initial conditions.

14. Show that the Euclidean algorithm on (F_{n+1}, F_n) takes $n-1$ division steps (Lamé-type fact).

15. Akra–Bazzi idea: for $T(n) = \sum a_i T(b_i n) + g(n)$, the exponent p solves $\sum a_i b_i^p = 1$. Compute p for $T = T(n/3) + T(2n/3) + \Theta(n)$.

Checks

1. $a_n = 2 \cdot 3^n$.

2. $n^{\log_2 4} = n^2$, $f = n = O(n^{2-\varepsilon}) \quad \Theta(n^2)$.

3. $\Theta(n^2)$.

4. $a_n = a_{n-1} + a_{n-2}$ with suitable base ($a_1 = 2$, $a_2 = 3$).

Summary

Recurrences turn recursive structure into equations. Linear constant-coefficient recurrences yield exponential closed forms via characteristic roots; algorithm costs often fall under the Master theorem or recursion trees. The same language specifies DP and counting problems. Practice moving fluidly among: write recurrence $\to$ solve or asymptote $\to$ verify on small n $\to$ implement without exponential blow-up.

Induction and Invariants

Mathematical induction is the main proof tool for statements indexed by natural numbers. **Invariants** prove that algorithms and discrete processes never break a property — the heart of correctness arguments, from loop proofs to distributed protocols.

Diagram: induction staircase

```
prove P(n0)           base

        if P(k) then P(k+1)
     v
P(n) for all n   n0
```

1. Standard (weak) induction

To prove $\forall n \geq n_0 : P(n)$:

1. **Base case:** prove $P(n_0)$ directly.
2. **Inductive step:** assume $P(k)$ for some $k \geq n_0$ (**inductive hypothesis**, IH), then prove $P(k+1)$.

Logical shape: $P(n_0)$ and $\forall k \geq n_0\,(P(k) \Rightarrow P(k+1))$ together imply $\forall n \geq n_0\, P(n)$.

Worked example 1 — sum formula

Claim: $P(n) : \sum_{i=1}^{n} i = n(n+1)/2$ for $n \geq 1$.

Base $n = 1$: $1 = 1 \cdot 2/2$.

Step: Assume true for k. Then

$$\sum_{i=1}^{k+1} i = \frac{k(k+1)}{2} + (k+1) = (k+1)\Big(\frac{k}{2} + 1\Big) = \frac{(k+1)(k+2)}{2}.$$

So $P(k+1)$ holds.

Worked example 2 — inequality

Claim: $2^n \geq n + 1$ for all integers $n \geq 0$.

Base $n = 0$: $1 \geq 1$. IH: $2^k \geq k + 1$. Then $2^{k+1} = 2 \cdot 2^k \geq 2(k + 1) = 2k + 2 \geq k + 2$ for $k \geq 0$.

2. Strong induction

Strong form: in the step, assume $P(n_0), \ldots, P(k)$ all true, prove $P(k + 1)$.

Use when $P(k + 1)$ depends on several earlier values, not only $P(k)$.

Worked example 3 — every $n \geq 2$ has a prime factor

Base $n = 2$: 2 is prime.
Assume every m with $2 \leq m \leq k$ has a prime factor. For $k+1$: if $k+1$ is prime, done; if composite, $k + 1 = ab$ with $1 < a, b \leq k$, so a has a prime factor, which divides $k + 1$.

Worked example 4 — Fibonacci closed bound

Many Fibonacci identities need strong induction because $F_{k+1} = F_k + F_{k-1}$.

3. Structural induction

For recursively defined objects (lists, trees, formulas, well-formed expressions):

1. Prove property for base constructors

2. Assume it for substructures; prove for the combined structure

Example: every full binary tree with n internal nodes has $n + 1$ leaves — induct on tree structure, not only on n.

This is the same logic as induction on $\mathbb{N}$, but the "order" is construction depth.

4. Classic proof patterns

Pattern	Typical claim
Summation formulas	closed form for $\sum f(i)$
Divisibility	$n^3 - n$ divisible by 3
Inequalities	growth rates, binomial bounds
Algorithm correctness	recursive procedures match a recurrence
Combinatorial identities	Pascal: $\binom{n}{k} = \binom{n-1}{k} + \binom{n-1}{k-1}$

Worked example 5 — geometric series

For $r \neq 1$,

$$\sum_{i=0}^{n} r^i = \frac{r^{n+1} - 1}{r - 1}.$$

Base $n = 0$: $1 = (r-1)/(r-1)$.
Step: sum to k plus r^{k+1} yields $(r^{k+2} - 1)/(r - 1)$.

Worked example 6 — $n! < n^n$ for $n \geq 2$

Base $n = 2$: $2 < 4$.
IH: $k! < k^k$. Then $(k+1)! = (k+1)k! < (k+1)k^k < (k+1)(k+1)^k = (k+1)^{k+1}$.

5. Loop invariants

For iterative algorithms, a **loop invariant** I is a predicate such that:

1. **Initialization:** I is true before the first iteration
2. **Maintenance:** if I is true at the start of an iteration and the loop body runs, I is true at the end
3. **Termination:** when the loop ends, I and ¬Cond imply the postcondition

```
// I holds here
while Cond do
   // I true at start of body
   Body
   // I true at end of body
// I   ¬Cond    postcondition
```

Worked example 7 — linear search

Search for target t in array $A[0..n-1]$.

Invariant after k probes of indices $0, \ldots, k-1$: either we already returned success, or $t \notin \{A[0], \ldots, A[k-1]\}$.

At $k = n$, if not found, t is absent. Correctness of "not found" follows.

Worked example 8 — iterative factorial

```
r ← 1
i ← 1
while i   n do
   r ← r * i
   i ← i + 1
// r = n!
```

Invariant: at the start of the loop (and after each iteration), $r = (i-1)!$ and $1 \le i \le n+1$. When $i = n+1$, $r = n!$.

Worked example 9 — insertion into sorted prefix

Insertion sort invariant: after i iterations, $A[0..i-1]$ is sorted and is a permutation of the original first i elements.

6. Invariants in discrete structures

Parity and modular invariants

- Sum of degrees in a graph is even (**handshaking lemma**) number of odd-degree vertices is even
- Invariant of 15-puzzle: permutation parity + blank row distance
- Digital root / sum of digits mod 9 equals n mod 9

Coloring arguments

Color a chessboard; show a mutilated board cannot be tiled by dominoes (each domino covers one black and one white; two same-color corners removed).

Potential functions

Amortized analysis: a potential Φ stays nonnegative and accounts for "saved" work — an invariant-style accounting.

7. Invariants in concurrent / networked systems

Though this book is math-first:

- **Mutual exclusion:** at most one process in critical section (invariant on lock state)
- **Conservation:** token count in a ring algorithm remains 1
- **CRDTs / merge:** certain merge operations preserve commutativity (algebraic invariant)

Thinking "what quantity never changes / always stays legal" is the same skill as loop invariants.

8. Common failures (and the horses paradox)

Mistake	Fix
Missing or wrong base	check n_0 carefully; sometimes need two bases
Using $P(k+1)$ inside its own proof	only use IH on smaller instances
Vague $P(n)$	write the predicate in full, with quantifiers
Off-by-one in loops	test $n = 0, 1$ and last iteration
Strong induction needed but weak used	if recurrence looks back > 1, switch

Classic bug: "all horses are the same color"

Fake proof: any set of 1 horse is monochromatic. Assume any k horses same color; for $k+1$, drop one horse — first k same color, last k same color, share overlap all $k+1$ same.

Flaw: for $k = 1 \to k+1 = 2$, the two groups of size 1 have **empty overlap**, so colors need not match. Step fails exactly where the overlap assumption breaks.

9. Induction vs invariants — same idea

Setting	"Base"	"Step"
$\mathbb{N}$	$P(n_0)$	$P(k) \Rightarrow P(k+1)$
Loop	init before loop	body preserves I
Recursive data	constructors	combination preserves property
Algorithm	empty / $n = 0$ input	one more element processed

10. Writing good inductive proofs (checklist)

1. State $P(n)$ explicitly
2. State the range of n
3. Base case with arithmetic or direct check
4. Clear "Assume $P(k)$…"
5. Algebra that **uses** the IH
6. Conclude $P(k+1)$ and thus $\forall n$

For code: name the invariant in a comment; check init, preserve, exit.

11. Pitfalls

1. Inducting on the wrong quantity (try smaller size of structure)
2. Base too small / too large for the inequality
3. Circular reasoning in algorithm proofs (assuming correctness of recursive call without IH)
4. Forgetting that $P(n)$ must be a **proposition** (true/false), not a number
5. Using induction when a direct bijection or invariant is simpler

12. Checkpoint

- Prove summation and inequality claims by weak induction
- Use strong induction for prime factorization and multi-step recurrences
- Write init/maintain/terminate for a simple loop
- Spot the flaw in a broken inductive argument
- Apply a parity or coloring invariant to a impossibility claim

Exercises

Easy

1. Prove by induction $\sum_{i=0}^{n} r^i = (r^{n+1}-1)/(r-1)$ for $r \neq 1$.
2. Prove $n! < n^n$ for $n \geq 2$.
3. Write a loop invariant for computing factorial iteratively.
4. Prove $\sum_{i=1}^{n} i^2 = n(n+1)(2n+1)/6$.
5. Prove 3 divides $n^3 - n$ for all integers $n \geq 0$.

Medium

6. Strong induction: any integer $n \geq 2$ has a prime factor.
7. Invariant: in a finite graph, the number of odd-degree vertices is even — prove from handshaking.
8. Prove by induction that a tree with n vertices has $n-1$ edges (use leaf removal).
9. Binary search correctness: state a loop invariant on the search interval.
10. Prove $\binom{n}{k} = \binom{n-1}{k} + \binom{n-1}{k-1}$ by induction on n (for fixed ranges of k).

Challenge

11. **Bug hunt:** find the flaw in the fake proof that all horses are the same color.
12. Prove correctness of Euclid's algorithm via invariant: $\gcd(a, b) = \gcd(b, a \bmod b)$.
13. Structural induction: every propositional formula has equally many left and right parentheses (for a standard grammar).
14. Show that the 15-puzzle's half of permutations are unreachable (parity invariant sketch).
15. Amortized: define a potential for a binary counter so that flipping bits costs $O(1)$ amortized per increment.

Checks

6. Base $n = 1$ OK for horses; step fails when splitting into groups that do not share a horse for $n = 2$.

7. $\sum \deg = 2|E|$ even even number of odd summands.

Summary

Induction is the natural-number engine of discrete proofs; strong and structural variants cover recurrences and recursive data. Loop invariants and modular/parity invariants extend the same discipline to algorithms and impossibility results. Always write $P(n)$ clearly, nail the base, and ensure the step uses only legitimate earlier cases — that is most of mathematical maturity in CS foundations.

Category Theory of Terms: Argument Lists as Morphisms

Category Theory of Terms: Argument Lists as Morphisms

Thesis

The first term datatype everyone writes is a tree: an application node holding a function symbol and a **list of child terms**, or a variable. That is fine for interpreters. It is awkward for the mathematics of substitution, because the interesting object is not a single tree but an **ordered forest of terms in a fixed context** — an argument list. Once you promote that forest to a first-class type with an explicit context size, composition, pairing, and shifting become ordinary categorical operations. Philip Zucker's note *Category Theory of Terms Emphasizes the Argument List* (12 October 2026) makes that refactoring concrete in Python. This chapter teaches the same idea with worked examples you can run.

Mental model

```
Naive view:     Term  =  App(f, [Term, …])  |  Var(name)

Refactored:     TermSeq(ctx, [Term, …])

                                codomain = number of terms (a forest)
                     domain  = number of variables in scope

A TermSeq is a morphism  ctx  →  n
Composition (@) is simultaneous substitution.
Product (+) pairs two forests over the SAME context.
Monoidal (*) juxtaposes forests and SHIFTS variables apart.
```

Concept	In the datatype	Categorical reading
Context size `ctx`	How many variables exist (`Var(i)` needs `i < ctx`)	Domain of the morphism
`len(terms)`	How many results the forest produces	Codomain
Substitution	Replace each `Var(i)` by `.terms[i]`	Composition of morphisms
`id_(n)`	`[Var(0),…,Var(n-1)]`	Identity arrow $n \to n$
Ground term	Forest with `ctx = 0`	Morphism $0 \to 1$ (or $0 \to k$)

You do **not** need a course in category theory to use this. Treat "morphism" as "map that you can compose when the middle sizes match", like matrix multiplication with dimensions.

Worked examples

Python **3.13.5**. The listings follow Zucker's structure (names shortened for clarity).

Case 1: From trees to forests

```
# terms_naive.py - the shoot-from-the-hip datatype
from dataclasses import dataclass

class Term: ...

@dataclass
class App(Term):
    f: str
    args: list  # list[Term]

@dataclass
class Var(Term):
    name: str

print(App("f", [Var("X")]))
```

```
App(f='f', args=[Var(name='X')])
```

The list is already there, but it has no standalone meaning: you cannot say "this is a map from 2 variables to 3 terms" without inspecting the tree.

Case 2: `TermSeq` as a morphism

```
# termseq_morphisms.py
from dataclasses import dataclass

class Term: ...

@dataclass
class TermSeq:
    ctx: int
    terms: list

    @property
```

```
    def dom(self):
        return self.ctx

    @property
    def cod(self):
        return len(self.terms)

    def __add__(self, other):
        # categorical product of morphisms (same domain)
        if self.dom != other.dom:
            raise ValueError("domains must match")
        return TermSeq(self.ctx, self.terms + other.terms)

    def __matmul__(self, other):
        # composition = substitution; self after other
        if self.dom != other.cod:
            raise ValueError("dom(self) must equal cod(other)")
        return TermSeq(other.dom, [subst(t, other) for t in self.terms])

@dataclass
class App(Term):
    f: str
    args: "TermSeq"

@dataclass
class Var(Term):
    index: int

def subst(t: Term, s: TermSeq) -> Term:
    match t:
        case Var(i):
            return s.terms[i]
        case App(f, args):
            return App(f, args @ s)
        case _:
            raise TypeError(t)

def id_(ctx: int) -> TermSeq:
    return TermSeq(ctx, [Var(i) for i in range(ctx)])

def Function(f: str, arity: int) -> TermSeq:
    """The generic application f(V0,…,V_{arity-1}) as a morphism arity → 1."""
    return TermSeq(arity, [App(f, TermSeq(arity, [Var(i) for i in range(arity)]))])

a = Function("a", 0)   # constant a() : 0 → 1
f = Function("f", 1)   # f(V0)       : 1 → 1
print("a.dom,a.cod =", a.dom, a.cod)
```

```
print("f @ a =", f @ a)
print("id_(1)+id_(1) =", id_(1) + id_(1))
print("swap =", TermSeq(2, [Var(1), Var(0)]))
```

```
python3 termseq_morphisms.py
```

```
a.dom,a.cod = 0 1
f @ a = TermSeq(ctx=0, terms=[App(f='f', args=TermSeq(ctx=0, terms=[App(f='a', args=TermSeq(
id_(1)+id_(1) = TermSeq(ctx=1, terms=[Var(index=0), Var(index=0)])
swap = TermSeq(ctx=2, terms=[Var(index=1), Var(index=0)])
```

Reading the output:

- `f @ a` is the ground term $f(a())$ — a morphism $0 \to 1$.
- `id_(1)+id_(1)` duplicates the single variable: the diagonal / `dup`.
- `swap` is the symmetry $2 \to 2$.

Case 3: Why the context size must be explicit

With only `Var(3)` and no annotation, a term *suggests* a minimum context of size 4, but cannot express "I am in a context of size 27 and I only mention variable 3". Categorical semantics (and any careful capture-avoiding substitution story) wants the domain on the nose. Zucker's `TermSeq.ctx` is that annotation, checked so every `Var(i)` satisfies `i < ctx` and nested `App` arguments share the same context.

Case 4: Two products

Over a **shared** context, `__add__` is the categorical product of morphisms: $\langle \sigma, \tau \rangle : n \to k + \ell$ when $\sigma : n \to k$ and $\tau : n \to \ell$.

A second product (often written `*` in the blog) **juxtaposes independent contexts**: domains add, and variables in the right forest are shifted by the left domain. That is the monoidal / "separate environments" product. Use `+` when both forests talk about the *same* variables; use `*` when they talk about *disjoint* binders.

Case 5: Identities, projections, building terms by composition

```
# termseq_combinators.py  (continues the types from Case 2)
def proj(ctx: int, i: int) -> TermSeq:
    return TermSeq(ctx, [Var(i)])

g = Function("g", 2)
assert (g @ id_(2)).cod == 1
assert proj(2, 0).terms[0].index == 0
```

```
assert (id_(1) + id_(1)).cod == 2  # dup
print("g @ id_(2) =", g @ id_(2))
print("proj(2,1) =", proj(2, 1))
```

```
g @ id_(2) = TermSeq(ctx=2, terms=[App(f='g', args=TermSeq(ctx=2, terms=[Var(index=0), Var(i
proj(2,1) = TermSeq(ctx=2, terms=[Var(index=1)])
```

Building terms with `@` and `+` instead of nested list literals is the payoff: substitutions become matrix-like programs, and unification can be phrased as finding a mediating morphism (equalizer story — see Rydeheard & Burstall).

The trap

1. **Treating a lone `Term` as the morphism.** The morphism is the *forest* `TermSeq`. A single `App` is one component.

2. **Forgetting shifts when combining independent contexts.** Using `+` where you needed `*` silently reuses variable indices and captures.

3. **Chasing category-theory mystique.** The useful content here is "argument lists have domain/codomain and compose by substitution" — the same algebra as functions, with partial composition when sizes mismatch.

The boring rule

Represent open terms as morphisms out of an explicit context, and treat argument lists (forests) as the primary datatype. Composition is substitution; identities are variable tuples; ground terms are arrows from 0. When you need category theory vocabulary, borrow *product* and *composition*; leave the rest on the shelf until a proof demands it.

Try this

1. Run `termseq_morphisms.py` and print `(f @ a).dom`, `(f @ a).cod`.

2. Implement `__mul__` (shift product) following Zucker's `shift` / `extend` helpers; check `id_(1) * id_(1) * id_(1) == id_(3)`.

3. Encode $g(f(a()), a())$ purely with `Function`, `@`, and `+` / `*`.

4. Read the blog's unification sketch and relate it to "solve $\sigma = \tau$ by an equalizer".

Sources

- Philip Zucker, *Category Theory of Terms Emphasizes the Argument List* (12 October 2026): https://www.philipzucker.com/categorical_multiterms/
- Related background: Lawvere theories; Rydeheard & Burstall, *Computational Category Theory* (unification as equalizers); nLab — Lawvere theory
- Inspiration: https://x.com/SandMouth/status/2109696775938617793

Lean 4.35: vcgen and lake check

Lean 4.35: `vcgen` and `lake check`

Thesis

Lean 4.35.0 (12 October 2026) replaces the monadic verification-condition tactic `mvcgen` with `vcgen`: faster, more general, and aimed at both `do`-programs and deeply embedded languages. The same release folds external kernel checking into Lake via `lake check` and `lake comparator`, optionally running every bundled checker with `--paranoid`.

This chapter is a discrete-math / verification user's guide to those two headlines. Lean is **not** installed on this scout host, so listings are taken from the official 4.35.0 release notes and neighbouring reference pages. When you have `elan` + 4.35 locally, re-run them and paste real output.

Mental model

```
Imperative / monadic program  +  Hoare-style contract

          vcgen          ← weakest-precondition / VC generation (Std.WP)

      verification conditions

   discharge with grind / finish / manual tactics

      lake check [--paranoid]   ← rebuild + replay through kernel(s)
```

Concept	Role in 4.35
`mvcgen`	Previous tactic; **deprecated** — emits deprecation warnings
`vcgen`	Successor; enable experimental ack with `set_option experimental.vcgen true`
`Std.WP`	Weakest-precondition framework behind `vcgen` (not only monads)
`lake check`	Build default targets, export, replay through Lean's kernel; fail on non-standard axioms
`lake check --paranoid`	Also run `lean4lean`, `nanoda`, `con-leche`, `con-ron`, `leanchecker-paranoid`

Concept	Role in 4.35
`lake comparator`	Check solution theorems against a challenge module (competition / homework shape)

Worked examples

Case 1: Swap `mvcgen` for `vcgen`

Release notes: `mvcgen` / `mvcgen?` are deprecated in favour of `vcgen` (PR #14874). New code should open the WP namespace and use `vcgen`:

```
-- VcgenHello.lean - shape from the 4.35 notes / reference (not executed here)
import Std.Tactic.Do   -- legacy mvcgen path lived here; follow current vcgen imports in the

set_option experimental.vcgen true

-- Sketch: a trivial Id program with a contract, discharged by vcgen
-- (exact import paths: see "Verifying Imperative Programs Using vcgen" in the reference.)
--
-- def addOne (n : Nat) : Id Nat
--     ensures r => r = n + 1 := do
--   return n + 1
--
-- example : True := by
--   -- vcgen on goals arising from intrinsic contracts / WP goals
--   trivial
```

Honest status: **not elaborated on this host** (no `lean` / `lake` binary). Treat the sketch as a pointer to the reference tutorial; copy a full worked file from the Lean reference once the toolchain is present.

Why the rename matters:

1. **Deep embeddings.** `vcgen` applies to any program type with a WP interpretation — `do` in a monad, but also embedded ISAs (the notes mention x64 / Wasm-style languages).
2. **Richer assertions.** Assertions may live in complete lattices (separation logic, probabilistic logics), with a generalized frame rule.
3. **Performance.** On the AddSubCancel benchmark (stateful loop body × 2200), notes report `vcgen` **0.32 s** vs `mvcgen` **66 s**, and kernel checking **0.53 s** vs **246 s**, with roughly linear growth (~0.14 ms per loop iteration).

Case 2: Intrinsic contracts on `def` (loop with invariant)

Also new around WP work in 4.35: loops can carry the annotations `vcgen` needs:

```
-- from the release highlights (countDown) - docs excerpt
def countDown (n : Nat) : Id Nat
    ensures r => r = 0 := do
  let mut i := n
  while i > 0
      invariant exit => if exit then i = 0 else True
      decreasing i
    do
    i := i - 1
  return i
```

`invariant` / `decreasing` / `assert` / `requires` / `ensures` report as experimental unless you acknowledge with the documented `experimental.*` options. Prefer the reference chapter over blog paraphrases when proof mode details matter.

Case 3: `lake check` and `--paranoid`

```
# In a Lake project on Lean 4.35+ (not run on this scout box):
lake check              # official kernel, standard axioms only
lake check --paranoid   # + lean4lean, nanoda, con-leche, con-ron, leanchecker-paranoid
```

Behaviour (from the notes):

- Builds default targets inside a **bubblewrap** sandbox (Linux), exports, replays through the kernel in a separate process.
- Fails if the export uses axioms outside the permitted set.
- `--from-export` checks an NDJSON export produced elsewhere (CI VM, third party).
- Sandbox can be disabled only with an explicitly inadvisable flag — do not use that in production review.

Leonardo de Moura's 15 Oct 2026 note frames the motivation: AI agents find kernel bugs; multiple independent checkers raise confidence. Scalability of external checking for multi-million-line developments remains an open FRO problem — still run `--paranoid` on *your* project size today.

Case 4: `lake comparator` for challenge / solution pairs

```
{
  "challenge_module": "Challenge",
  "solution_module": "Solution",
  "theorem_names": ["imo2024_p1"],
  "permitted_axioms": ["propext", "Quot.sound", "Classical.choice"]
}
```

```
lake comparator   # reads comparator.json by default
```

Checks: same statement as the challenge, only permitted axioms, kernel accepts. Same `--paranoid` / export flags family as `lake check`.

Case 5: Cache / build packaging tweak

```
lake build -o --package <pkg>
lake cache put --package=<pkg>
# lake cache put-staged now requires an explicit revision:
lake cache put-staged --rev="$(git rev-parse HEAD)"
```

Unrelated to VC generation, but part of the same 4.35 Lake surface you will hit in CI.

The trap

- Leaving `mvcgen` in new proofs — noisy deprecations and missing deep-embedding features.
- Forgetting `set_option experimental.vcgen true` and treating the warning as failure.
- Running `lake check --inadvisably-no-sandbox` on untrusted formalisations.
- Assuming `--paranoid` scales unchanged to 10 -line developments — the FRO post is explicit that it does not yet.
- Inventing tactic syntax beyond the release notes / reference.

The boring rule

On Lean **4.35**: write new imperative proofs with **`vcgen`**, acknowledge the experimental flag, and gate merges with **`lake check`** (add `--paranoid` when the project fits in the checkers' comfort zone). Retire `mvcgen` on touch.

Try this

1. `elan install leanprover/lean4:v4.35.0` on a workstation and open the reference tutorial *Verifying Imperative Programs Using vcgen*.
2. Replace one `mvcgen` proof with `vcgen … with finish` and compare wall time.
3. Run `lake check` on a small library; then `lake check --paranoid` and record which checkers ran.
4. Add a `comparator.json` for a homework theorem and break it on purpose (wrong statement) to see the failure mode.

Sources

- Lean 4.35.0 release notes
- Reference: `mvcgen` superseded by `vcgen`; tutorial *Verifying Imperative Programs Using vcgen*
- Lean now ships with verified kernels (Leonardo de Moura, 2026-10-15)
- Inspiration: leanprover on X — 4.35.0 / vcgen

Penrose Tilings

Penrose Tilings

Thesis

Penrose tilings cover the plane aperiodically with finitely many prototiles. Substitution counts track the golden ratio ϕ.

Mental model

```
thick/thin substitution → ratio →
```

Worked examples

```
step thick thin total ratio_thick/thin
   0     1     0     1  inf
   1     2     1     3  2.0000
   2     5     3     8  1.6667
   3    13     8    21  1.6250
   4    34    21    55  1.6190
   5    89    55   144  1.6182
   6   233   144   377  1.6181
   7   610   377   987  1.6180
   8  1597   987  2584  1.6180
phi = 1.618033988749895 phi^2 = 2.618033988749895
```

Trap

Calling any non-repeating pattern Penrose. Matching rules matter.

Boring rule

Cite the substitution rule and ϕ, not vibes.

Try this

1. Draw three inflation steps.
2. Read about quasicrystals.
3. Compare Ammann–Beenker tilings.

Sources

- Penrose; Grünbaum & Shephard

The Banach–Tarski Paradox

The Banach–Tarski Paradox

Thesis

Banach–Tarski: a ball in 3D can be split into finitely many pieces and rigidly reassembled into two balls of the same size. Pieces are non-measurable; the proof uses the **Axiom of Choice**. Conceptual chapter — not a physical claim about oranges.

Mental model

```
ball → AC pieces → isometries → two balls
volume only for measurable sets
```

Worked examples

What fails without Choice / in 2D

Analogous finite decompositions fail in $\mathbb{R}^2$ with isometries; without Choice the pieces need not exist.

Where Choice shows up

Free groups inside $SO(3)$ and Choice-picked orbits yield non-measurable pieces (see Wagon).

Trap

Using Banach–Tarski as an engineering gotcha. It does not break energy conservation; volume is undefined for those pieces.

Boring rule

Stay in Lebesgue-measurable sets when you need volume.

Try this

1. Read the free-group paradox (Hausdorff).
2. Compare Vitali sets.
3. Skim Wagon, *The Banach–Tarski Paradox*.

Sources

- Wagon; measure-theory texts

Statistics

Introduction to Statistics: Making Sense of Data in an Uncertain World

What is Statistics?

Statistics is the science of collecting, organizing, analyzing, interpreting, and presenting data to make informed decisions in the face of uncertainty. It provides the mathematical framework for understanding patterns, relationships, and trends in data, enabling us to draw meaningful conclusions from observations and make predictions about future events.

In our data-driven world, statistics has become essential across virtually every field of human endeavor, from scientific research and business analytics to public policy and personal decision-making.

```
Statistics in the Modern World


Core Functions:
• Data collection and experimental design
• Data organization and visualization
• Pattern recognition and trend analysis
• Hypothesis testing and inference
• Prediction and forecasting
• Decision-making under uncertainty

Applications Across Fields:
• Science: Clinical trials, experimental validation
• Business: Market research, quality control, forecasting
• Government: Census data, policy evaluation, economics
• Technology: Machine learning, data mining, A/B testing
• Sports: Performance analysis, player evaluation
• Social Sciences: Survey research, behavioral studies

Key Questions Statistics Answers:
• What does the data tell us?
• How confident can we be in our conclusions?
• What patterns exist in the data?
• How can we predict future outcomes?
• What decisions should we make based on evidence?
```

Historical Development

From Ancient Records to Modern Data Science

Statistics has evolved from simple record-keeping to sophisticated mathematical theory, driven by practical needs and theoretical advances.

```
Timeline of Statistical Development


Ancient Period (3000 BCE - 500 CE):
• Census taking in ancient civilizations
• Tax records and population counts
• Early probability concepts in gambling

Medieval Period (500 - 1500 CE):
• Insurance and risk assessment
• Mortality tables for life insurance
• Trade statistics and accounting

Renaissance and Enlightenment (1500 - 1800):
1654: Pascal and Fermat develop probability theory
1662: John Graunt analyzes mortality data (first vital statistics)
1713: Jakob Bernoulli's "Ars Conjectandi" (Law of Large Numbers)
1733: De Moivre discovers normal distribution
1763: Bayes' theorem published posthumously

19th Century - Foundation Era:
1805: Legendre develops method of least squares
1809: Gauss develops normal distribution theory
1835: Quetelet applies statistics to social phenomena
1857: Mendel's genetic experiments (statistical analysis)
1886: Galton develops correlation and regression
1890: Pearson develops chi-square test

20th Century - Modern Statistics:
1908: Student's t-test (William Gosset)
1925: Fisher develops analysis of variance (ANOVA)
1928: Neyman-Pearson hypothesis testing framework
1940s: Quality control methods (Shewhart, Deming)
1950s: Computer-aided statistical analysis begins

Digital Age (1980s - Present):
• Statistical software packages (SAS, SPSS, R)
• Big data analytics and data mining
• Machine learning integration
• Real-time statistical analysis
```

- Bayesian computational methods
- Data visualization and interactive analytics

The Statistical Revolution

The 20th and 21st centuries have witnessed an explosion in statistical applications and methodologies.

```
Modern Statistical Paradigms


Classical (Frequentist) Statistics:
• Probability as long-run frequency
• Hypothesis testing framework
• Confidence intervals
• P-values and significance testing
• Developed by Fisher, Neyman, Pearson

Bayesian Statistics:
• Probability as degree of belief
• Prior and posterior distributions
• Bayes' theorem as foundation
• Credible intervals
• Decision theory integration

Computational Statistics:
• Monte Carlo methods
• Bootstrap and resampling techniques
• Markov Chain Monte Carlo (MCMC)
• Machine learning algorithms
• Big data processing techniques

Robust Statistics:
• Methods resistant to outliers
• Non-parametric approaches
• Distribution-free methods
• Exploratory data analysis
• Developed by Tukey and others

Modern Applications:
• Bioinformatics and genomics
• Financial risk modeling
• Climate change analysis
• Social media analytics
• Artificial intelligence and machine learning
• Quality improvement and Six Sigma
• Evidence-based medicine
```

- Sports analytics and sabermetrics

Fundamental Concepts

Data and Variables

Understanding the nature of data is crucial for choosing appropriate statistical methods.

```
Types of Data and Variables


Data Classification:

Quantitative (Numerical) Data:
• Discrete: Countable values (number of students, cars sold)
• Continuous: Measurable values (height, weight, temperature)

Qualitative (Categorical) Data:
• Nominal: Categories with no natural order (colors, brands, gender)
• Ordinal: Categories with natural order (grades, satisfaction levels)

Levels of Measurement:

Nominal Scale:
• Categories with no inherent order
• Examples: Eye color, marital status, blood type
• Operations: Counting, mode
• Statistics: Frequencies, proportions, chi-square tests

Ordinal Scale:
• Categories with meaningful order but no consistent intervals
• Examples: Letter grades (A, B, C, D, F), survey ratings
• Operations: Ranking, median
• Statistics: Percentiles, rank correlation

Interval Scale:
• Ordered categories with equal intervals, no true zero
• Examples: Temperature in Celsius, calendar years, IQ scores
• Operations: Addition, subtraction
• Statistics: Mean, standard deviation, correlation

Ratio Scale:
• Interval scale with meaningful zero point
• Examples: Height, weight, income, age
• Operations: All arithmetic operations
• Statistics: All measures, geometric mean, coefficient of variation
```

Data Collection Methods:

Observational Studies:
• Observe subjects without intervention
• Cannot establish causation
• Examples: Surveys, case studies, cohort studies

Experimental Studies:
• Manipulate variables to observe effects
• Can establish causation
• Examples: Clinical trials, A/B tests, laboratory experiments

Sampling Methods:
• Simple random sampling
• Stratified sampling
• Cluster sampling
• Systematic sampling
• Convenience sampling

Population vs. Sample

Population and Sample Concepts

Population:
• Complete collection of all individuals or items of interest
• Usually too large or impossible to study entirely
• Parameters: Numerical characteristics of population (, ,)
• Examples: All registered voters, all light bulbs produced

Sample:
• Subset of population selected for study
• Should be representative of population
• Statistics: Numerical characteristics of sample ($\bar{x}$, s, $\hat{p}$)
• Used to make inferences about population

Key Relationships:
Population Parameter Sample Statistic
  (population mean) $\bar{x}$ (sample mean)
  (population standard deviation) s (sample standard deviation)
  (population proportion) $\hat{p}$ (sample proportion)

Sampling Distribution:
• Distribution of sample statistics across all possible samples
• Foundation for statistical inference
• Central Limit Theorem: Sample means approach normal distribution

Example:
Population: All college students in the US (20 million)
Parameter: = average GPA of all college students
Sample: 1,000 randomly selected college students
Statistic: $\bar{x}$ = average GPA of sample = 3.2
Inference: Estimate 3.2 with some margin of error

Sampling Error:
- Difference between sample statistic and population parameter
- Inevitable in sampling (unless census is taken)
- Can be quantified and controlled through proper sampling
- Decreases as sample size increases

Non-sampling Errors:
- Measurement errors
- Response bias
- Non-response bias
- Coverage bias
- Processing errors

Descriptive vs. Inferential Statistics

Two Main Branches of Statistics

Descriptive Statistics:
- Summarize and describe data
- No generalizations beyond the data
- Tools: Tables, graphs, summary measures

Methods:
- Measures of central tendency (mean, median, mode)
- Measures of variability (range, variance, standard deviation)
- Measures of position (percentiles, quartiles)
- Data visualization (histograms, box plots, scatter plots)

Examples:
- "The average test score was 85"
- "25% of students scored below 75"
- "Sales increased by 15% last quarter"
- "The most popular color choice was blue"

Inferential Statistics:
- Make generalizations about populations based on samples
- Quantify uncertainty in conclusions
- Tools: Hypothesis tests, confidence intervals, regression

Methods:
• Estimation (point estimates, interval estimates)
• Hypothesis testing (significance tests)
• Regression analysis (relationships between variables)
• Analysis of variance (comparing multiple groups)

Examples:
• "We are 95% confident the population mean is between 82 and 88"
• "There is significant evidence that the new treatment is effective"
• "The correlation between study time and grades is statistically significant"
• "The difference between groups is not due to chance"

Relationship:
Descriptive → Inferential
First describe the sample data, then make inferences about the population

Process Flow:
1. Collect sample data
2. Describe sample using descriptive statistics
3. Use inferential methods to draw conclusions about population
4. Quantify uncertainty in conclusions
5. Make decisions based on statistical evidence

The Role of Probability

Probability as Foundation

Probability theory provides the mathematical foundation for statistical inference.

Probability in Statistics

Why Probability Matters:
• Quantifies uncertainty in data and conclusions
• Provides framework for making inferences
• Enables calculation of confidence levels
• Foundation for hypothesis testing
• Models random variation in data

Key Probability Concepts:

Random Variables:
• Variables whose values are determined by chance
• Discrete: Countable outcomes (coin flips, dice rolls)
• Continuous: Uncountable outcomes (heights, weights)

Probability Distributions:
• Mathematical functions describing likelihood of outcomes
• Discrete: Binomial, Poisson, geometric
• Continuous: Normal, exponential, uniform

Expected Value and Variance:
• E(X): Average value of random variable over many trials
• Var(X): Measure of spread around expected value
• Foundation for sample statistics

Law of Large Numbers:
• Sample statistics approach population parameters as n increases
• Theoretical justification for using samples to estimate populations
• Example: Coin flip proportion approaches 0.5 as flips increase

Central Limit Theorem:
• Sample means approach normal distribution regardless of population shape
• Enables inference about means using normal distribution
• Foundation for confidence intervals and hypothesis tests

Applications in Statistics:

Sampling Distributions:
• Distribution of sample statistics across all possible samples
• Normal distribution often applies due to Central Limit Theorem
• Used to calculate probabilities for statistical tests

Confidence Intervals:
• Range of plausible values for population parameter
• Based on probability distribution of sample statistic
• Example: "95% confident is between 45 and 55"

Hypothesis Testing:
• Calculate probability of observing sample result if null hypothesis true
• P-value: Probability of more extreme result under null hypothesis
• Decision rule based on probability threshold (= 0.05)

Regression Analysis:
• Model relationship between variables with random error
• Probability distributions for error terms
• Inference about regression coefficients

Statistical Models

Models in Statistical Analysis

What is a Statistical Model?
- Mathematical representation of data-generating process
- Combines systematic patterns with random variation
- Form: Data = Model + Error

Types of Models:

Parametric Models:
- Assume specific probability distribution
- Finite number of parameters
- Examples: Normal distribution, linear regression
- Advantages: Efficient, well-developed theory
- Disadvantages: May not fit real data well

Non-parametric Models:
- Make minimal distributional assumptions
- More flexible but less efficient
- Examples: Rank tests, kernel density estimation
- Advantages: Robust, fewer assumptions
- Disadvantages: Less powerful, harder to interpret

Linear Models:
- Response variable is linear function of predictors
- Examples: Linear regression, ANOVA, ANCOVA
- Form: Y = + X + X + ... +

Generalized Linear Models:
- Extend linear models to non-normal responses
- Examples: Logistic regression, Poisson regression
- Link function connects linear predictor to response

Time Series Models:
- Account for temporal dependence in data
- Examples: ARIMA, exponential smoothing
- Applications: Forecasting, trend analysis

Model Selection:
- Choose appropriate model for data and research question
- Balance complexity with interpretability
- Validation techniques: Cross-validation, information criteria

Model Assumptions:
- Independence of observations
- Appropriate probability distribution
- Constant variance (homoscedasticity)
- Linearity (for linear models)
- Normality (for many parametric tests)

Checking Assumptions:
• Residual analysis
• Diagnostic plots
• Statistical tests for assumptions
• Robust methods when assumptions violated

Statistical Thinking

The Scientific Method and Statistics

Statistics in Scientific Inquiry

Scientific Method Steps:
1. Observation and question formulation
2. Hypothesis development
3. Experimental design
4. Data collection
5. Statistical analysis
6. Interpretation and conclusion
7. Replication and validation

Statistical Contributions:

Experimental Design:
• Control for confounding variables
• Randomization to ensure validity
• Power analysis for sample size
• Blocking and stratification strategies

Hypothesis Testing:
• Formalize research questions
• Null and alternative hypotheses
• Type I and Type II error control
• Statistical significance vs. practical significance

Causal Inference:
• Distinguish correlation from causation
• Control for confounding variables
• Randomized controlled trials
• Observational study limitations

Reproducibility:
• Statistical methods must be replicable
• P-hacking and multiple testing problems

- Pre-registration of analyses
- Open science and data sharing

Evidence-Based Decision Making:
- Quantify uncertainty in conclusions
- Meta-analysis to combine studies
- Systematic reviews of evidence
- Clinical practice guidelines

Common Pitfalls:
- Correlation implies causation
- Cherry-picking favorable results
- Misinterpreting p-values
- Ignoring effect sizes
- Overgeneralization from samples

Critical Thinking with Data

Statistical Literacy and Reasoning

Essential Skills:

Data Interpretation:
- Read and understand statistical summaries
- Recognize misleading presentations
- Distinguish between different types of averages
- Understand variability and its importance

Graph Literacy:
- Interpret common statistical graphs
- Recognize misleading visualizations
- Understand scale and axis manipulation
- Choose appropriate graph types

Probability Understanding:
- Interpret probability statements correctly
- Understand conditional probability
- Recognize independence vs. dependence
- Avoid probability fallacies

Sampling Concepts:
- Understand representativeness
- Recognize sampling bias
- Appreciate margin of error
- Distinguish sample from population

Common Statistical Fallacies:

Correlation vs. Causation:
• Strong correlation doesn't imply causation
• Confounding variables can create spurious relationships
• Need experimental evidence for causal claims

Base Rate Neglect:
• Ignore prior probability when updating beliefs
• Important in medical testing and screening
• Bayes' theorem provides correct framework

Regression to the Mean:
• Extreme values tend to be closer to average on retest
• Often misinterpreted as real improvement
• Important in performance evaluation

Survivorship Bias:
• Focus on successful cases while ignoring failures
• Leads to overestimation of success rates
• Important in business and investment analysis

Simpson's Paradox:
• Trend appears in groups but reverses when combined
• Importance of considering confounding variables
• Example: University admission rates by gender

Media and Statistics:
• Sensationalized reporting of studies
• Misuse of statistical significance
• Cherry-picking of favorable results
• Lack of context for statistical claims

Questions to Ask:
• Who collected the data and why?
• How was the sample selected?
• What is the sample size?
• Are there potential confounding variables?
• Is the conclusion supported by the data?
• Could there be alternative explanations?

Modern Applications

Big Data and Data Science

Statistics in the Digital Age

Big Data Characteristics:
- Volume: Massive amounts of data
- Velocity: High-speed data generation
- Variety: Multiple data types and sources
- Veracity: Data quality and reliability challenges

Statistical Challenges:
- Traditional methods may not scale
- Multiple testing problems
- Spurious correlations in large datasets
- Computational limitations
- Storage and processing requirements

New Methodologies:
- Machine learning algorithms
- Distributed computing frameworks
- Streaming data analysis
- Non-parametric methods
- Robust statistical procedures

Data Science Integration:
- Statistics + Computer Science + Domain Expertise
- Emphasis on prediction over explanation
- Automated model selection
- Cross-validation and regularization
- Ensemble methods

Applications:
- Recommendation systems (Netflix, Amazon)
- Search algorithms (Google)
- Social media analysis (Facebook, Twitter)
- Financial trading algorithms
- Healthcare analytics and personalized medicine
- Smart city infrastructure
- Climate modeling and environmental monitoring

Machine Learning and AI

Statistics and Machine Learning

Relationship:
- Machine learning builds on statistical foundations
- Statistics provides theoretical framework
- ML emphasizes prediction and automation

- Statistics emphasizes inference and understanding

Shared Concepts:
- Probability distributions
- Bias-variance tradeoff
- Cross-validation
- Regularization techniques
- Model selection criteria

Statistical Learning Theory:
- Mathematical framework for learning from data
- Generalization bounds and sample complexity
- PAC (Probably Approximately Correct) learning
- VC (Vapnik-Chervonenkis) dimension

Common Algorithms with Statistical Roots:
- Linear and logistic regression
- Naive Bayes classifiers
- Decision trees and random forests
- Support vector machines
- Neural networks and deep learning
- Clustering algorithms (k-means, hierarchical)

Bayesian Methods in ML:
- Bayesian neural networks
- Gaussian processes
- Markov Chain Monte Carlo
- Variational inference
- Probabilistic programming

Statistical Validation:
- Training, validation, and test sets
- Cross-validation techniques
- Bootstrap methods
- Confidence intervals for predictions
- Statistical significance of model comparisons

Ethical Considerations:
- Algorithmic bias and fairness
- Privacy and data protection
- Interpretability vs. accuracy tradeoffs
- Responsible AI development
- Statistical disclosure control

Business Analytics and Decision Science

Statistics in Business and Industry

Quality Control:
- Statistical process control (SPC)
- Control charts for monitoring processes
- Six Sigma methodology
- Design of experiments for process improvement
- Acceptance sampling plans

Market Research:
- Survey design and sampling
- Consumer behavior analysis
- A/B testing for product features
- Market segmentation
- Brand perception studies

Financial Analytics:
- Risk modeling and assessment
- Portfolio optimization
- Credit scoring models
- Fraud detection algorithms
- Algorithmic trading strategies

Operations Research:
- Forecasting demand and sales
- Inventory optimization
- Supply chain analytics
- Resource allocation
- Scheduling and planning

Customer Analytics:
- Customer lifetime value modeling
- Churn prediction and retention
- Recommendation systems
- Personalization algorithms
- Customer satisfaction measurement

Business Intelligence:
- Dashboard design and KPI selection
- Data warehousing and ETL processes
- OLAP (Online Analytical Processing)
- Data mining and pattern recognition
- Predictive analytics for business planning

Performance Measurement:
- Balanced scorecard approaches
- Statistical significance in business metrics
- Confidence intervals for KPIs

- Trend analysis and forecasting
- Benchmarking and comparative analysis

Building Statistical Intuition

Developing Statistical Thinking

Cultivating Statistical Mindset

Key Principles:

Embrace Uncertainty:
- All data contains variability
- Perfect predictions are impossible
- Quantify and communicate uncertainty
- Make decisions despite incomplete information

Think in Distributions:
- Focus on patterns, not individual values
- Consider the full range of possibilities
- Understand central tendency and spread
- Recognize different distribution shapes

Question Everything:
- Where did the data come from?
- What might be missing or biased?
- Are there alternative explanations?
- What assumptions are being made?

Context Matters:
- Statistical significance vs. practical importance
- Domain knowledge informs interpretation
- Consider economic, social, and ethical implications
- Understand the real-world consequences of decisions

Practical Strategies:

Start with Graphs:
- Visualize data before formal analysis
- Look for patterns, outliers, and anomalies
- Choose appropriate visualization methods
- Use graphs to communicate findings

Check Assumptions:
- Understand what methods require

- Verify assumptions before applying tests
- Use robust methods when assumptions fail
- Report limitations and caveats

Validate Results:
- Use multiple approaches when possible
- Cross-validate findings
- Seek replication and confirmation
- Be skeptical of surprising results

Communicate Clearly:
- Avoid statistical jargon with non-experts
- Focus on practical implications
- Quantify uncertainty appropriately
- Use visualizations effectively

Common Mistakes to Avoid:
- Confusing statistical and practical significance
- Ignoring assumptions of statistical methods
- Over-interpreting small samples
- Failing to account for multiple comparisons
- Misunderstanding correlation and causation

Conclusion

Statistics provides the essential tools for making sense of data and making informed decisions in an uncertain world. From its historical roots in census-taking and probability theory to its modern applications in big data and artificial intelligence, statistics continues to evolve and expand its influence across all areas of human knowledge.

Statistics: The Science of Learning from Data

Historical Significance:
  Evolution from simple record-keeping to sophisticated theory
  Foundation for scientific method and evidence-based reasoning
  Integration with probability theory and mathematical modeling
  Adaptation to computational age and big data challenges

Conceptual Power:
  Framework for quantifying uncertainty and variability
  Methods for making inferences from samples to populations
  Tools for discovering patterns and relationships in data
  Bridge between theoretical models and real-world applications

Modern Applications:

```
  Scientific research and experimental design
  Business analytics and decision-making
  Machine learning and artificial intelligence
  Public policy and social science research
  Quality control and process improvement
  Medical research and healthcare analytics

Educational Value:
  Develops critical thinking and analytical skills
  Builds quantitative literacy for informed citizenship
  Provides foundation for data-driven careers
  Enhances decision-making abilities
  Promotes scientific reasoning and skepticism
```

As you begin your journey through statistics, remember that you're learning more than just mathematical techniques—you're developing a way of thinking about uncertainty, evidence, and decision-making that will serve you throughout your personal and professional life. Statistics is fundamentally about learning from data, and in our increasingly data-rich world, this skill has never been more valuable.

Whether you're evaluating medical treatments, analyzing business performance, conducting scientific research, or simply trying to make sense of information in the news, statistical thinking provides the tools for separating signal from noise, quantifying uncertainty, and making informed decisions based on evidence rather than intuition alone.

The concepts and methods you'll learn in the following chapters—from descriptive statistics and probability to hypothesis testing and regression analysis—form an integrated framework for understanding and analyzing data. Each topic builds on previous knowledge while contributing to your overall statistical literacy and analytical capabilities.

Statistics is both an art and a science: it requires technical knowledge of methods and procedures, but also judgment, creativity, and wisdom in applying these tools to real-world problems. As you progress through your statistical education, focus not just on learning formulas and procedures, but on developing the statistical intuition and critical thinking skills that will enable you to use statistics effectively and responsibly in whatever field you choose to pursue.

Descriptive Statistics: Summarizing and Visualizing Data

Introduction to Descriptive Statistics

Descriptive statistics provides the tools and techniques for summarizing, organizing, and presenting data in meaningful ways. Before we can make inferences or draw conclusions about populations, we must first understand what our data tells us through careful description and visualization.

Descriptive statistics transforms raw data into comprehensible information, revealing patterns, trends, and characteristics that might otherwise remain hidden in long lists of numbers.

Purpose of Descriptive Statistics

Primary Goals:
- Summarize large datasets with key measures
- Identify patterns and trends in data
- Detect outliers and unusual observations
- Communicate findings clearly and effectively
- Prepare data for further statistical analysis

Key Questions Answered:
- What is the typical or central value?
- How spread out or variable are the data?
- What is the shape of the data distribution?
- Are there any unusual or extreme values?
- How do different groups or variables compare?

Tools and Techniques:
- Measures of central tendency (mean, median, mode)
- Measures of variability (range, variance, standard deviation)
- Measures of position (percentiles, quartiles, z-scores)
- Data visualization (histograms, box plots, scatter plots)
- Summary tables and frequency distributions

Applications:
- Business reporting and dashboards
- Scientific data analysis and presentation
- Quality control and process monitoring
- Market research and consumer analysis

• Educational assessment and evaluation

Organizing Data

Frequency Distributions

Frequency distributions organize data by showing how often each value or range of values occurs.

Types of Frequency Distributions

For Qualitative Data:
Simple frequency table showing categories and their counts

Example: Student Majors

Major	Frequency	Relative Frequency	Percentage
Computer Sci	45	0.30	30%
Mathematics	30	0.20	20%
Engineering	38	0.25	25%
Business	22	0.15	15%
Other	15	0.10	10%
Total	150	1.00	100%

For Quantitative Data:
Grouped frequency distribution using class intervals

Example: Test Scores (0-100)

Class Interval	Frequency	Relative Frequency	Cumulative Frequency
60-69	5	0.10	5
70-79	12	0.24	17
80-89	18	0.36	35
90-99	15	0.30	50
Total	50	1.00	50

Key Concepts:
• Frequency: Number of observations in each category/class
• Relative frequency: Proportion of total (frequency ÷ total)
• Cumulative frequency: Running total of frequencies
• Class width: Size of each interval (should be equal)
• Class midpoint: Middle value of each interval

Guidelines for Creating Classes:
• Use 5-20 classes (typically 5-15)
• Make class widths equal when possible

- Avoid overlapping classes
- Include all data points
- Use convenient class boundaries

Stem-and-Leaf Plots

Stem-and-Leaf Displays

Purpose:
- Show distribution shape while preserving actual data values
- Quick way to organize and visualize small to moderate datasets
- Useful for identifying outliers and gaps

Construction:
1. Separate each number into stem (leading digits) and leaf (trailing digit)
2. List stems vertically in order
3. List leaves horizontally for each stem in order
4. Include key explaining the format

Example: Test Scores
Data: 67, 72, 73, 75, 78, 81, 83, 85, 87, 89, 91, 93, 95, 97

```
Stem | Leaf
-----|----------
6    | 7
7    | 2 3 5 8
8    | 1 3 5 7 9
9    | 1 3 5 7
```

Key: 6|7 = 67

Advantages:
- Preserves original data values
- Shows distribution shape
- Easy to construct by hand
- Identifies outliers clearly

Disadvantages:
- Limited to relatively small datasets
- Not suitable for very large or very small numbers
- Less flexible than histograms

Variations:
- Split stems: Use 6* for 60-64, 6• for 65-69
- Back-to-back: Compare two distributions
- Truncated: Drop decimal places for simplicity

Measures of Central Tendency

The Mean

The arithmetic mean is the most commonly used measure of central tendency.

Arithmetic Mean

Population Mean: = (Σx)/N = (x + x + ... + x)/N

Sample Mean: $\bar{x}$ = (Σx)/n = (x + x + ... + x)/n

Where:
• Σx = sum of all values
• N = population size
• n = sample size

Example: Test Scores
Data: 85, 92, 78, 88, 95, 82, 90
$\bar{x}$ = (85 + 92 + 78 + 88 + 95 + 82 + 90)/7 = 610/7 = 87.14

Properties of the Mean:
• Uses all data values in calculation
• Unique value for any dataset
• Affected by extreme values (outliers)
• Sum of deviations from mean equals zero: Σ(x - $\bar{x}$) = 0
• Minimizes sum of squared deviations: Σ(x - $\bar{x}$)²

When to Use:
 Data is roughly symmetric
 No extreme outliers present
 Interval or ratio level data
 Further statistical analysis planned

When Not to Use:
 Highly skewed distributions
 Presence of extreme outliers
 Ordinal data
 Open-ended classes in grouped data

Weighted Mean:
When observations have different importance or frequency

$\bar{x}_w$ = (Σw x)/(Σw)

Example: Course Grade Calculation

```
Component    | Score | Weight | Weighted Score
-------------|-------|--------|---------------
Homework     |  85   |  0.20  |     17.0
Midterm      |  78   |  0.30  |     23.4
Final Exam   |  92   |  0.50  |     46.0
Total        |       |  1.00  |     86.4

Weighted mean = 86.4
```

The Median

The median is the middle value when data is arranged in order.

```
Median Calculation


For Odd Number of Values:
Median = middle value when arranged in order

Example: 12, 15, 18, 22, 25, 28, 30
Median = 22 (4th value out of 7)

For Even Number of Values:
Median = average of two middle values

Example: 12, 15, 18, 22, 25, 28
Median = (18 + 22)/2 = 20

Position Formula:
Position of median = (n + 1)/2

For n = 7: Position = (7 + 1)/2 = 4th value
For n = 8: Position = (8 + 1)/2 = 4.5 (average of 4th and 5th values)

Properties of the Median:
• Not affected by extreme values (robust)
• Divides data into two equal halves
• Unique value for any dataset
• Can be used with ordinal data
• May not use all data values

When to Use:
  Skewed distributions
  Presence of outliers
  Ordinal level data
  Open-ended distributions
  Income and housing price data
```

```
Example: Income Data
$25,000, $28,000, $32,000, $35,000, $38,000, $42,000, $250,000

Mean = $64,286 (pulled up by high income)
Median = $35,000 (better represents typical income)

Quartiles and Percentiles:
• Q  (25th percentile): 25% of data below this value
• Q  (50th percentile): Median
• Q  (75th percentile): 75% of data below this value

Finding Quartiles:
1. Find median (Q )
2. Q  = median of lower half
3. Q  = median of upper half
```

The Mode

The mode is the most frequently occurring value in a dataset.

```
Mode Identification



Definition:
Value that appears most frequently in the dataset

Examples:

Unimodal (One Mode):
Data: 2, 3, 4, 4, 4, 5, 6, 7
Mode = 4 (appears 3 times)

Bimodal (Two Modes):
Data: 1, 2, 2, 3, 4, 5, 5, 6
Modes = 2 and 5 (each appears twice)

Multimodal (Multiple Modes):
Data: 1, 1, 2, 2, 3, 3, 4
Modes = 1, 2, and 3 (each appears twice)

No Mode:
Data: 1, 2, 3, 4, 5, 6, 7
No mode (all values appear once)

For Grouped Data:
Modal class = class interval with highest frequency
```

```
Example: Test Scores
Class Interval | Frequency
---------------|----------
60-69          |     3
70-79          |     8
80-89          |    12  ← Modal class
90-99          |     7

Properties of the Mode:
• Can be used with any level of data
• Not affected by extreme values
• May not exist or may not be unique
• Doesn't use all data values
• Easy to identify in frequency distributions

When to Use:
  Nominal (categorical) data
  Finding most popular item
  Highly skewed distributions
  Quick rough estimate needed

Applications:
• Most popular product size
• Most common defect type
• Peak hours for service
• Most frequent customer complaint

Relationship Between Mean, Median, and Mode:

Symmetric Distribution:
Mean = Median = Mode

Right-Skewed (Positively Skewed):
Mode < Median < Mean

Left-Skewed (Negatively Skewed):
Mean < Median < Mode

This relationship helps identify distribution shape.
```

Measures of Variability

Range and Interquartile Range

Measures of variability describe how spread out the data values are.

Range Measures

Range:
Difference between largest and smallest values
Range = Maximum - Minimum

Example: Test Scores
Data: 65, 72, 78, 85, 88, 92, 95
Range = 95 - 65 = 30

Properties:
• Simple to calculate and understand
• Uses only two values (extremes)
• Heavily influenced by outliers
• Doesn't describe internal variability

Interquartile Range (IQR):
Difference between third and first quartiles
IQR = Q - Q

Advantages:
• Not affected by outliers
• Describes spread of middle 50% of data
• Useful for identifying outliers

Outlier Detection Rule:
• Lower outlier: Below Q - 1.5(IQR)
• Upper outlier: Above Q + 1.5(IQR)

Example: Calculating IQR
Data: 12, 15, 18, 22, 25, 28, 30, 35, 40, 45, 50

Step 1: Find quartiles
Q = 18 (25th percentile)
Q = 28 (median)
Q = 40 (75th percentile)

Step 2: Calculate IQR
IQR = Q - Q = 40 - 18 = 22

Step 3: Check for outliers
Lower fence: 18 - 1.5(22) = 18 - 33 = -15
Upper fence: 40 + 1.5(22) = 40 + 33 = 73
No outliers in this dataset

Semi-Interquartile Range:
Also called quartile deviation

Semi-IQR = (Q - Q)/2 = IQR/2

Used when median is reported as central tendency measure

Variance and Standard Deviation

Variance and Standard Deviation

Population Variance:
 ² = Σ(x -)²/N

Population Standard Deviation:
  = √[Σ(x -)²/N]

Sample Variance:
s² = Σ(x - x̄)²/(n - 1)

Sample Standard Deviation:
s = √[Σ(x - x̄)²/(n - 1)]

Note: Sample formulas use (n-1) for unbiased estimation

Calculation Example:
Data: 2, 4, 6, 8, 10

Step 1: Calculate mean
x̄ = (2 + 4 + 6 + 8 + 10)/5 = 30/5 = 6

Step 2: Calculate deviations and squared deviations

x	(x - x̄)	(x - x̄)²
2	-4	16
4	-2	4
6	0	0
8	2	4
10	4	16
	0	40

Step 3: Calculate variance
s² = 40/(5-1) = 40/4 = 10

Step 4: Calculate standard deviation
s = √10 = 3.16

Properties:
• Uses all data values

- Measures average distance from mean
- Same units as original data (for standard deviation)
- Larger values indicate more variability
- Always non-negative

Computational Formula (easier for hand calculation):
$s^2 = [\Sigma x^2 - (\Sigma x)^2/n]/(n - 1)$

Using previous example:
$\Sigma x = 30$, $\Sigma x^2 = 220$, $n = 5$
$s^2 = [220 - (30)^2/5]/(5-1) = [220 - 180]/4 = 40/4 = 10$

Interpretation:
- About 68% of data within 1 standard deviation of mean
- About 95% of data within 2 standard deviations of mean
- About 99.7% of data within 3 standard deviations of mean

(For approximately normal distributions)

When to Use:
- Interval or ratio level data
- Roughly symmetric distributions
- Further statistical analysis planned
- Comparing variability between groups

Coefficient of Variation

Coefficient of Variation

Definition:
Relative measure of variability expressed as percentage
$CV = (s/\bar{x}) \times 100\%$

Purpose:
- Compare variability between datasets with different units
- Compare variability between datasets with different means
- Determine relative consistency

Example 1: Comparing Test Scores
Class A: $\bar{x} = 85$, $s = 10$
Class B: $\bar{x} = 75$, $s = 8$

$CV_A = (10/85) \times 100\% = 11.8\%$
$CV_B = (8/75) \times 100\% = 10.7\%$

Class B has less relative variability despite similar absolute variability.

Example 2: Comparing Different Measurements
Height: $\bar{x}$ = 68 inches, s = 3 inches
Weight: $\bar{x}$ = 150 pounds, s = 20 pounds

$CV_{height} = (3/68) \times 100\% = 4.4\%$
$CV_{weight} = (20/150) \times 100\% = 13.3\%$

Weight shows more relative variability than height.

Interpretation Guidelines:
• CV < 15%: Low variability
• 15% CV < 35%: Moderate variability
• CV 35%: High variability

When to Use:
 Comparing datasets with different units
 Comparing datasets with very different means
 Quality control applications
 Financial risk assessment

Limitations:
• Not meaningful when mean is close to zero
• Can be misleading with negative values
• Assumes ratio-level data

Measures of Position

Percentiles and Quartiles

Percentiles

Definition:
The kth percentile is the value below which k% of the data falls

Common Percentiles:
• 25th percentile (Q_1): First quartile
• 50th percentile (Q_2): Median
• 75th percentile (Q_3): Third quartile
• 90th percentile: Commonly used in testing
• 95th percentile: Often used as cutoff points

Calculating Percentiles:
1. Arrange data in ascending order
2. Find position: L = (k/100) × n
3. If L is whole number, percentile = average of values at positions L and L+1

4. If L is not whole number, round up to next integer position

Example: Finding 30th Percentile
Data: 12, 15, 18, 22, 25, 28, 30, 35, 40 (n = 9)

L = (30/100) × 9 = 2.7
Round up to position 3
30th percentile = 18

Five-Number Summary:
1. Minimum value
2. First quartile (Q)
3. Median (Q)
4. Third quartile (Q)
5. Maximum value

Example:
Data: 5, 8, 12, 15, 18, 22, 25, 28, 30, 35, 40

Five-number summary:
Min = 5
Q = 12
Q = 22
Q = 30
Max = 40

Applications:
• Standardized test scores (SAT, GRE)
• Growth charts for children
• Income distribution analysis
• Performance benchmarking
• Quality control limits

Z-Scores (Standard Scores)

Z-Scores

Definition:
Number of standard deviations a value is from the mean
z = (x -)/ (population)
z = (x - $\bar{x}$)/s (sample)

Interpretation:
• z = 0: Value equals the mean
• z > 0: Value is above the mean
• z < 0: Value is below the mean

• |z| = 1: Value is 1 standard deviation from mean

Example: Test Scores
Class average: $\bar{x}$ = 75, standard deviation: s = 10
Student score: x = 85

z = (85 - 75)/10 = 1.0

The student scored 1 standard deviation above the mean.

Properties of Z-Scores:
• Mean of z-scores = 0
• Standard deviation of z-scores = 1
• Shape of distribution unchanged
• Unitless (standardized)

Uses:
• Compare scores from different distributions
• Identify outliers (|z| > 2 or 3)
• Calculate probabilities with normal distribution
• Standardize data for analysis

Example: Comparing Performance
Math test: x = 85, $\bar{x}$ = 75, s = 10
z_math = (85 - 75)/10 = 1.0

English test: x = 92, $\bar{x}$ = 88, s = 6
z_english = (92 - 88)/6 = 0.67

Better relative performance in math despite lower absolute score.

Outlier Detection:
• Mild outliers: 2 < |z| < 3
• Extreme outliers: |z| > 3

Modified Z-Score (using median):
More robust for skewed data
Modified z = 0.6745(x - median)/MAD
where MAD = median absolute deviation

Data Visualization

Histograms

Histogram Construction

Purpose:
• Show distribution shape
• Identify patterns and outliers
• Compare distributions
• Estimate probabilities

Construction Steps:
1. Determine number of classes (5-20, typically)
2. Calculate class width = (max - min)/number of classes
3. Create class boundaries
4. Count frequencies for each class
5. Draw bars with heights equal to frequencies

Example: Test Scores
Data: 65, 68, 72, 75, 78, 80, 82, 85, 88, 90, 92, 95

Classes:
60-69: 2 students
70-79: 3 students
80-89: 4 students
90-99: 3 students

Histogram Features:
• Bars touch each other (continuous data)
• Height represents frequency
• Area represents relative frequency
• No gaps between bars (unless no data)

Distribution Shapes:
• Normal (bell-shaped): Symmetric, single peak
• Right-skewed: Tail extends to right
• Left-skewed: Tail extends to left
• Uniform: All bars approximately same height
• Bimodal: Two distinct peaks

Guidelines:
• Use equal class widths when possible
• Avoid too few or too many classes
• Label axes clearly
• Include title and sample size
• Consider relative frequency for comparisons

Variations:
• Relative frequency histogram (proportions)
• Density histogram (area = 1)
• Cumulative frequency histogram
• Back-to-back histogram (comparing groups)

Box Plots

Box Plot Construction

Components:
• Box: Extends from Q to Q (contains middle 50%)
• Median line: Line inside box at Q
• Whiskers: Lines extending to furthest non-outlier points
• Outliers: Points beyond whiskers (circles or asterisks)

Construction Steps:
1. Calculate five-number summary
2. Identify outliers using IQR rule
3. Draw box from Q to Q
4. Draw median line at Q
5. Extend whiskers to furthest non-outlier points
6. Plot outliers as individual points

Example:
Data: 12, 15, 18, 22, 25, 28, 30, 35, 40, 45, 60

Five-number summary:
Min = 12, Q = 18, Q = 28, Q = 40, Max = 60

IQR = 40 - 18 = 22
Lower fence: 18 - 1.5(22) = -15
Upper fence: 40 + 1.5(22) = 73

No outliers, so whiskers extend to 12 and 60.

Advantages:
• Shows distribution shape
• Identifies outliers clearly
• Compact display
• Good for comparing groups
• Shows median and quartiles

Disadvantages:
• Less detail than histogram
• Doesn't show sample size
• May hide multimodal distributions
• Requires understanding of quartiles

Variations:
• Notched box plot: Shows confidence interval for median
• Variable width: Box width proportional to sample size
• Violin plot: Combines box plot with density curve

Interpretation:
• Symmetric: Median near center of box, equal whiskers
• Right-skewed: Median toward left side, longer right whisker
• Left-skewed: Median toward right side, longer left whisker

Scatter Plots

Scatter Plot Analysis

Purpose:
• Show relationship between two quantitative variables
• Identify correlation patterns
• Detect outliers and unusual points
• Assess linearity of relationships

Construction:
• x-axis: Independent (explanatory) variable
• y-axis: Dependent (response) variable
• Each point represents one observation
• Plot all (x, y) pairs

Relationship Patterns:

Positive Linear:
• Points form upward-sloping pattern
• As x increases, y tends to increase
• Example: Height vs. weight

Negative Linear:
• Points form downward-sloping pattern
• As x increases, y tends to decrease
• Example: Price vs. demand

No Relationship:
• Points scattered randomly
• No clear pattern
• Example: Shoe size vs. GPA

Nonlinear:
• Points form curved pattern
• May be exponential, quadratic, etc.
• Example: Age vs. reaction time

Strength Assessment:
• Strong: Points close to pattern line

- Moderate: Points somewhat scattered around pattern
- Weak: Points widely scattered, pattern unclear

Outliers:
- Points that don't fit the general pattern
- May indicate data errors or special cases
- Can strongly influence correlation

Example Analysis:
Study time (hours) vs. Test score

Observations:
- Positive relationship: More study time → higher scores
- Moderately strong: Points fairly close to line
- One outlier: Student with high study time but low score
- Generally linear relationship

Enhancements:
- Add trend line to show relationship
- Use different colors/symbols for groups
- Add marginal histograms
- Include correlation coefficient
- Size points by third variable (bubble plot)

Summary Statistics and Reports

Creating Effective Summaries

Statistical Summary Reports

Essential Components:
- Sample size (n)
- Measures of central tendency
- Measures of variability
- Distribution shape indicators
- Outlier identification
- Confidence intervals (when appropriate)

Standard Summary Format:
Variable: Test Scores
n = 50
Mean = 82.4
Median = 84.0
Mode = 85
Standard Deviation = 8.2

Range = 35 (65 to 100)
IQR = 12 (Q_3 = 78, Q_1 = 90)
Outliers: 2 (scores of 65 and 67)

Choosing Appropriate Statistics:

For Symmetric Distributions:
• Central tendency: Mean
• Variability: Standard deviation
• Position: Z-scores

For Skewed Distributions:
• Central tendency: Median
• Variability: IQR
• Position: Percentiles

For Categorical Data:
• Central tendency: Mode
• Variability: Not applicable
• Position: Frequencies and percentages

Comparative Summaries:
When comparing groups, include:
• Side-by-side statistics
• Relative measures (CV, percentages)
• Visual comparisons (box plots)
• Effect size measures

Example: Comparing Two Classes

	Class A	Class B
n	25	30
Mean	85.2	78.6
Median	86.0	80.0
Std Dev	6.8	9.2
Range	28	35
IQR	10	14

Interpretation: Class A performed better on average with less variability.

Report Writing Guidelines:
• Start with context and data source
• Present statistics in logical order
• Use appropriate precision (2-3 significant digits)
• Include units of measurement
• Highlight key findings
• Discuss limitations and assumptions
• Use tables and graphs effectively

Summary and Key Concepts

Descriptive statistics provides the foundation for understanding and communicating what data reveals, serving as the essential first step in any statistical analysis.

Chapter Summary

Essential Skills Mastered:
Organizing data with frequency distributions and displays
Calculating measures of central tendency (mean, median, mode)
Computing measures of variability (range, variance, standard deviation)
Finding measures of position (percentiles, quartiles, z-scores)
Creating and interpreting data visualizations
Writing effective statistical summaries

Key Concepts:
- Central tendency describes typical values
- Variability measures spread of data
- Position measures locate individual values
- Distribution shape affects choice of statistics
- Outliers can significantly impact results
- Visualization reveals patterns not apparent in numbers

Fundamental Measures:
- Mean: Arithmetic average, sensitive to outliers
- Median: Middle value, robust to outliers
- Mode: Most frequent value, useful for categories
- Standard deviation: Average distance from mean
- IQR: Spread of middle 50% of data
- Percentiles: Position relative to other values

Problem-Solving Framework:
- Examine data type and distribution shape
- Choose appropriate measures of center and spread
- Identify and investigate outliers
- Create meaningful visualizations
- Summarize findings clearly and accurately

Visualization Tools:
- Histograms: Show distribution shape and frequency
- Box plots: Display five-number summary and outliers
- Scatter plots: Reveal relationships between variables
- Stem-and-leaf: Preserve actual data values
- Frequency tables: Organize categorical data

Next Steps:

```
Descriptive statistics skills prepare you for:
- Probability theory and distributions
- Inferential statistics and hypothesis testing
- Regression analysis and correlation
- Quality control and process improvement
- Data analysis in research and business
```

Descriptive statistics represents the essential foundation of statistical analysis, providing the tools to transform raw data into meaningful information. The techniques covered in this chapter—from basic measures of center and spread to sophisticated visualization methods—enable you to explore data systematically, identify important patterns, and communicate findings effectively.

Understanding descriptive statistics is crucial not only for further statistical study but also for making informed decisions in any field that involves data. Whether you're analyzing business performance, evaluating research results, or simply trying to make sense of information in daily life, these descriptive tools provide the framework for clear, accurate, and insightful data analysis.

The skills developed in this chapter form the building blocks for more advanced statistical methods. As you progress to inferential statistics, you'll use these descriptive techniques to understand sample data before making generalizations about populations. The visualization and summary skills you've learned will remain essential throughout your statistical journey, helping you communicate results and validate assumptions in more complex analyses.

Probability: The Mathematics of Uncertainty

Introduction to Probability

Probability is the mathematical framework for quantifying uncertainty and randomness. It provides the theoretical foundation for statistical inference, enabling us to make informed decisions when outcomes are uncertain and to quantify our confidence in conclusions drawn from data.

Understanding probability is essential for interpreting statistical results, designing experiments, and making predictions about future events based on available information.

```
Probability in Statistics and Life


Why Probability Matters:
• Quantifies uncertainty in a precise mathematical way
• Provides foundation for statistical inference
• Enables prediction and risk assessment
• Models random phenomena in nature and society
• Guides decision-making under uncertainty

Applications:
• Weather forecasting and climate modeling
• Medical diagnosis and treatment effectiveness
• Financial risk assessment and insurance
• Quality control and reliability engineering
• Games of chance and sports betting
• Machine learning and artificial intelligence
• Scientific hypothesis testing

Key Questions Probability Answers:
• What is the likelihood of a specific outcome?
• How confident can we be in our predictions?
• What are the chances of multiple events occurring?
• How do we update beliefs with new information?
• What decisions minimize expected losses?

Historical Development:
• 1654: Pascal and Fermat solve gambling problems
• 1713: Bernoulli's Law of Large Numbers
• 1733: De Moivre's normal approximation
• 1812: Laplace's classical probability theory
```

- 1933: Kolmogorov's axiomatic foundation
- Modern: Computational and applied probability

Basic Probability Concepts

Sample Spaces and Events

```
Fundamental Probability Concepts


Sample Space (S):
Set of all possible outcomes of an experiment
Must be mutually exclusive and collectively exhaustive

Examples:
• Coin flip: S = {H, T}
• Die roll: S = {1, 2, 3, 4, 5, 6}
• Card draw: S = {52 different cards}
• Lifetime: S = {t : t   0} (continuous)

Event (E):
Subset of the sample space
Collection of outcomes of interest

Examples:
• Getting heads: E = {H}
• Rolling even number: E = {2, 4, 6}
• Drawing a heart: E = {13 heart cards}
• Living past 80: E = {t : t > 80}

Types of Events:

Simple Event:
Contains exactly one outcome
Example: Rolling a 3 on a die

Compound Event:
Contains more than one outcome
Example: Rolling an even number

Certain Event:
Always occurs (equals sample space)
P(S) = 1

Impossible Event:
Never occurs (empty set)
```

P( ) = 0

Complement Event:
All outcomes not in the event
E' or $\bar{E}$ = S - E
P(E') = 1 - P(E)

Event Relationships:

Union (E E):
Event that occurs if E or E (or both) occurs
"At least one event occurs"

Intersection (E E):
Event that occurs if both E and E occur
"Both events occur"

Mutually Exclusive Events:
Cannot occur simultaneously
E E =
P(E E) = 0

Example: Rolling a die
E = {rolling odd number} = {1, 3, 5}
E = {rolling even number} = {2, 4, 6}
E and E are mutually exclusive

Probability Definitions and Axioms

Approaches to Probability

Classical (Theoretical) Probability:
Based on equally likely outcomes
P(E) = Number of favorable outcomes / Total number of outcomes

Example: Fair die
P(rolling a 3) = 1/6
P(rolling even) = 3/6 = 1/2

Requirements:
- Finite sample space
- All outcomes equally likely
- Known structure of experiment

Empirical (Relative Frequency) Probability:
Based on observed data

P(E) = Number of times E occurred / Total number of trials

Example: Manufacturing defects
Out of 1000 items, 25 were defective
P(defective) = 25/1000 = 0.025

Approaches true probability as trials increase (Law of Large Numbers)

Subjective Probability:
Based on personal judgment or belief
Reflects degree of confidence in outcome

Example: "I'm 70% confident it will rain tomorrow"
Used when classical or empirical approaches not feasible

Kolmogorov Axioms:
Mathematical foundation for probability theory

Axiom 1: P(E) 0 for any event E
(Probabilities are non-negative)

Axiom 2: P(S) = 1
(Probability of sample space is 1)

Axiom 3: For mutually exclusive events E , E , ...
P(E E ...) = P(E) + P(E) + ...
(Addition rule for disjoint events)

Properties Derived from Axioms:
• 0 P(E) 1 for any event E
• P( ) = 0
• P(E') = 1 - P(E)
• If E E , then P(E) P(E)

Basic Probability Rules:

Addition Rule (General):
P(E E) = P(E) + P(E) - P(E E)

Addition Rule (Mutually Exclusive):
P(E E) = P(E) + P(E)

Complement Rule:
P(E') = 1 - P(E)

Example Application:
Drawing a card from standard deck
P(King or Heart) = P(King) + P(Heart) - P(King of Hearts)

= 4/52 + 13/52 - 1/52 = 16/52 = 4/13

Counting Techniques in Probability

Combinatorics and Probability

When outcomes are equally likely, probability calculations often involve counting.

Multiplication Principle:
If task has k steps with n , n , ..., n ways respectively,
total ways = n × n × ... × n

Example: License plates (3 letters, 3 digits)
Total possibilities = $26^3 \times 10^3$ = 17,576,000

Permutations:
Arrangements where order matters
P(n,r) = n!/(n-r)!

Example: Selecting president, VP, secretary from 10 people
Number of ways = P(10,3) = 10!/7! = 720

Combinations:
Selections where order doesn't matter
C(n,r) = n!/(r!(n-r)!)

Example: Selecting 5-card poker hand from 52 cards
Number of ways = C(52,5) = 2,598,960

Probability Applications:

Example 1: Lottery
Choose 6 numbers from 1 to 49
Total combinations = C(49,6) = 13,983,816
P(winning) = 1/13,983,816 7.15 × 10

Example 2: Committee Selection
From 8 men and 6 women, select 4-person committee
P(2 men, 2 women) = [C(8,2) × C(6,2)] / C(14,4)
= [28 × 15] / 1001 = 420/1001 0.42

Example 3: Birthday Problem
What's probability that at least 2 people in group of 23 share birthday?

P(at least one match) = 1 - P(no matches)
P(no matches) = (365/365) × (364/365) × ... × (343/365)

```
               0.493
P(at least one match)  1 - 0.493 = 0.507

Surprisingly, probability exceeds 50% with just 23 people!

Hypergeometric Distribution:
Sampling without replacement from finite population

Example: Defective items
Population: 100 items (10 defective, 90 good)
Sample: 5 items without replacement
P(exactly 2 defective) = [C(10,2) × C(90,3)] / C(100,5)
                       = [45 × 117,480] / 75,287,520  0.070
```

Conditional Probability and Independence

Conditional Probability

```
Conditional Probability Concepts


Definition:
Probability of event A given that event B has occurred
P(A|B) = P(A  B) / P(B), provided P(B) > 0

Interpretation:
• Restricts sample space to outcomes where B occurs
• Updates probability based on additional information
• Foundation for Bayesian reasoning

Example: Card Drawing
Standard deck, draw one card
A = {card is King}, B = {card is face card}

P(A) = 4/52 = 1/13 (unconditional probability)
P(A|B) = P(King and face card) / P(face card)
       = (4/52) / (12/52) = 4/12 = 1/3

Given card is face card, probability of King increases.

Multiplication Rule:
P(A  B) = P(A|B) × P(B) = P(B|A) × P(A)

Example: Medical Testing
Disease prevalence: P(D) = 0.01
Test sensitivity: P(+|D) = 0.95 (detects disease when present)
```

Test specificity: P(-|D') = 0.98 (negative when no disease)

P(+ and D) = P(+|D) × P(D) = 0.95 × 0.01 = 0.0095

Law of Total Probability:
If B , B , ..., B partition the sample space, then:
P(A) = P(A|B)P(B) + P(A|B)P(B) + ... + P(A|B)P(B)

Example: Manufacturing
Two machines produce items:
Machine 1: 60% of production, 2% defective rate
Machine 2: 40% of production, 5% defective rate

P(defective) = P(defective|M1)P(M1) + P(defective|M2)P(M2)
             = 0.02 × 0.60 + 0.05 × 0.40 = 0.032

Tree Diagrams:
Visual tool for conditional probability problems
• Branches represent conditional probabilities
• Path probabilities multiply along branches
• Final probabilities sum for all paths to same outcome

Example: Two-stage experiment
Stage 1: Select box (Box 1: 0.6, Box 2: 0.4)
Stage 2: Draw ball from selected box
Box 1: 3 red, 2 blue balls
Box 2: 1 red, 4 blue balls

P(red) = P(red|Box1)P(Box1) + P(red|Box2)P(Box2)
       = (3/5)(0.6) + (1/5)(0.4) = 0.36 + 0.08 = 0.44

Independence

Statistical Independence

Definition:
Events A and B are independent if:
P(A|B) = P(A) or equivalently P(A B) = P(A) × P(B)

Interpretation:
• Occurrence of B doesn't change probability of A
• Knowledge of B provides no information about A
• Events are unrelated

Testing Independence:
Check if P(A B) = P(A) × P(B)

Example: Coin Flips
Two fair coin flips
A = {first flip heads}, B = {second flip heads}
P(A) = 1/2, P(B) = 1/2
P(A B) = 1/4 = (1/2) × (1/2) = P(A) × P(B)
Therefore, A and B are independent.

Example: Card Drawing (with replacement)
Draw two cards with replacement
A = {first card is King}, B = {second card is King}
P(A) = 4/52, P(B) = 4/52
P(A B) = (4/52) × (4/52) = P(A) × P(B)
Events are independent.

Example: Card Drawing (without replacement)
Draw two cards without replacement
A = {first card is King}, B = {second card is King}
P(A) = 4/52
P(B|A) = 3/51 P(B) = 4/52
Events are not independent.

Mutual Independence:
Events A , A , ..., A are mutually independent if:
P(A A ... A) = P(A) × P(A) × ... × P(A)
for any subset {i , i , ..., i }

Applications:
• System reliability (components fail independently)
• Quality control (items produced independently)
• Survey sampling (responses independent)
• Experimental design (treatments applied independently)

Independence vs. Mutual Exclusivity:
• Independent events can occur together
• Mutually exclusive events cannot occur together
• If P(A) > 0 and P(B) > 0, events cannot be both independent and mutually exclusive

Common Misconception:
Independence doesn't mean events are unrelated in real world
Statistical independence is mathematical property that may not reflect causal relationships

Bayes' Theorem

Bayes' Theorem

Statement:
$P(A|B) = P(B|A) \times P(A) / P(B)$

Alternative form using Law of Total Probability:
$P(A|B) = P(B|A) \times P(A) / [P(B|A) \times P(A) + P(B|A') \times P(A')]$

Components:
- $P(A)$: Prior probability (before observing B)
- $P(A|B)$: Posterior probability (after observing B)
- $P(B|A)$: Likelihood (probability of B given A)
- $P(B)$: Marginal probability of B

Example: Medical Diagnosis
Disease prevalence: $P(D) = 0.01$
Test sensitivity: $P(+|D) = 0.95$
Test specificity: $P(-|D') = 0.98$, so $P(+|D') = 0.02$

Patient tests positive. What's probability of having disease?

$P(D|+) = P(+|D) \times P(D) / P(+)$

First find $P(+)$:
$P(+) = P(+|D) \times P(D) + P(+|D') \times P(D')$
$= 0.95 \times 0.01 + 0.02 \times 0.99 = 0.0293$

Therefore:
$P(D|+) = (0.95 \times 0.01) / 0.0293 = 0.324$

Despite positive test, probability of disease is only 32.4%!

This counterintuitive result occurs because:
- Disease is rare (low prior probability)
- False positives outnumber true positives

Bayesian Reasoning Process:
1. Start with prior probability $P(A)$
2. Observe evidence B
3. Update to posterior probability $P(A|B)$
4. Use posterior as new prior for next observation

Applications:
- Medical diagnosis and screening
- Spam email filtering
- Machine learning and AI
- Legal evidence evaluation
- Quality control and testing
- Weather forecasting
- Financial risk assessment

```
Example: Spam Filtering
Prior: P(spam) = 0.4
Word "free" appears in email
P("free"|spam) = 0.8
P("free"|not spam) = 0.1

P(spam|"free") = P("free"|spam) × P(spam) / P("free")

P("free") = 0.8 × 0.4 + 0.1 × 0.6 = 0.38

P(spam|"free") = (0.8 × 0.4) / 0.38 = 0.842

Email with "free" has 84.2% probability of being spam.

Bayesian Networks:
Graphical models representing conditional dependencies
Used in expert systems and machine learning
Nodes represent variables, edges represent dependencies
```

Discrete Probability Distributions

Random Variables

```
Random Variable Concepts


Definition:
Function that assigns numerical value to each outcome in sample space
X: S →

Types:
• Discrete: Countable values (finite or countably infinite)
• Continuous: Uncountable values (intervals of real numbers)

Examples:
Discrete:
• Number of heads in 10 coin flips: X  {0, 1, 2, ..., 10}
• Number of customers per hour: X  {0, 1, 2, ...}
• Score on multiple choice test: X  {0, 1, 2, ..., 100}

Continuous:
• Height of randomly selected person: X  (0, ϖ)
• Time until next customer arrives: X  [0, ϖ)
• Temperature at noon tomorrow: X  (-ϖ, ϖ)
```

Probability Mass Function (PMF):
For discrete random variable X
P(X = x) = probability that X equals specific value x

Properties:
• P(X = x) 0 for all x
• ΣP(X = x) = 1 (sum over all possible values)

Example: Rolling two dice, X = sum
P(X = 2) = 1/36, P(X = 3) = 2/36, ..., P(X = 12) = 1/36

Cumulative Distribution Function (CDF):
F(x) = P(X x)

Properties:
• 0 F(x) 1
• F(x) is non-decreasing
• F(-ϖ) = 0, F(ϖ) = 1
• P(a < X b) = F(b) - F(a)

Expected Value (Mean):
E(X) = = Σx × P(X = x)

Interpretation: Long-run average value

Example: Die roll, X = outcome
E(X) = 1×(1/6) + 2×(1/6) + ... + 6×(1/6) = 21/6 = 3.5

Variance:
Var(X) = ² = E[(X -)²] = E(X²) - [E(X)]²

Standard Deviation:
 = √Var(X)

Properties of Expected Value:
• E(aX + b) = aE(X) + b
• E(X + Y) = E(X) + E(Y)
• If X and Y independent: E(XY) = E(X)E(Y)

Properties of Variance:
• Var(aX + b) = a²Var(X)
• If X and Y independent: Var(X + Y) = Var(X) + Var(Y)

Common Discrete Distributions

Bernoulli Distribution

Models single trial with two outcomes (success/failure)
Parameter: p = probability of success

PMF: P(X = x) = p^x(1-p)^(1-x) for x {0, 1}

Mean: E(X) = p
Variance: Var(X) = p(1-p)

Example: Single coin flip (p = 0.5)
P(X = 0) = 0.5, P(X = 1) = 0.5

Binomial Distribution

Models number of successes in n independent Bernoulli trials
Parameters: n (trials), p (success probability)

PMF: P(X = x) = C(n,x) × p^x × (1-p)^(n-x) for x {0, 1, ..., n}

Mean: E(X) = np
Variance: Var(X) = np(1-p)

Example: 10 coin flips, count heads (n = 10, p = 0.5)
P(X = 5) = C(10,5) × (0.5)^5 × (0.5)^5 = 252 × (0.5)^10 0.246

Applications:
• Quality control (defective items)
• Medical trials (treatment success)
• Survey research (yes/no responses)
• Marketing (conversion rates)

Poisson Distribution

Models number of events in fixed interval
Parameter: = average rate of occurrence

PMF: P(X = x) = (e^(-) × ^x) / x! for x {0, 1, 2, ...}

Mean: E(X) =
Variance: Var(X) =

Example: Phone calls per hour (= 3)
P(X = 2) = (e^(-3) × 3^2) / 2! = (0.0498 × 9) / 2 0.224

Applications:
• Customer arrivals

- Equipment failures
- Radioactive decay
- Network packet arrivals
- Biological mutations

Approximations:
- Binomial approximation: When n large, p small, np moderate
- Normal approximation: When large (> 10)

Geometric Distribution

Models number of trials until first success
Parameter: p = success probability

PMF: P(X = x) = (1-p)^(x-1) × p for x {1, 2, 3, ...}

Mean: E(X) = 1/p
Variance: Var(X) = (1-p)/p²

Example: Rolling die until getting 6 (p = 1/6)
P(X = 3) = (5/6)² × (1/6) = 25/216 0.116

Memoryless Property:
P(X > m + n | X > m) = P(X > n)
Past failures don't affect future probability

Applications:
- Time until equipment failure
- Number of attempts until success
- Waiting time problems
- Reliability engineering

Hypergeometric Distribution

Models sampling without replacement from finite population
Parameters: N (population), K (successes in population), n (sample size)

PMF: P(X = x) = [C(K,x) × C(N-K,n-x)] / C(N,n)

Mean: E(X) = n × (K/N)
Variance: Var(X) = n × (K/N) × (1-K/N) × (N-n)/(N-1)

Example: 52 cards, 13 hearts, draw 5 cards
P(X = 2 hearts) = [C(13,2) × C(39,3)] / C(52,5)

Applications:

- Quality control sampling
- Survey sampling
- Lottery problems
- Acceptance sampling

Continuous Probability Distributions

Continuous Random Variables

Continuous Distribution Concepts

Probability Density Function (PDF):
f(x) such that P(a X b) = [a to b] f(x)dx

Properties:
- f(x) 0 for all x
- [-ϖ to ϖ] f(x)dx = 1
- P(X = x) = 0 for any specific value x
- P(a X b) = P(a < X < b) = P(a < X b) = P(a X < b)

Cumulative Distribution Function:
F(x) = P(X x) = [-ϖ to x] f(t)dt

Relationship: f(x) = F'(x) (PDF is derivative of CDF)

Expected Value:
E(X) = [-ϖ to ϖ] x × f(x)dx

Variance:
Var(X) = [-ϖ to ϖ] (x -)² × f(x)dx = E(X²) - [E(X)]²

Percentiles:
The pth percentile x satisfies: F(x) = p/100

Median: 50th percentile where F(x .) = 0.5

Mode: Value where f(x) is maximum (if unique)

Normal Distribution

Normal Distribution

Most important continuous distribution
Parameters: (mean), ² (variance)

PDF: f(x) = (1/(√(2))) × e^(-(x-)²/(2 ²))

Properties:
• Bell-shaped, symmetric about
• Mean = Median = Mode =
• Inflection points at ±
• Total area under curve = 1

Standard Normal Distribution:
Z ~ N(0,1) with = 0, = 1

PDF: (z) = (1/√(2)) × e^(-z²/2)
CDF: Φ(z) = P(Z z)

Standardization:
If X ~ N(, ²), then Z = (X -)/ ~ N(0,1)

Empirical Rule (68-95-99.7 Rule):
• 68% of data within ±
• 95% of data within ± 2
• 99.7% of data within ± 3

Example: IQ Scores
X ~ N(100, 15²)
P(85 < X < 115) = P(-1 < Z < 1) 0.68

Finding Probabilities:
P(X x) = P(Z (x-)/) = Φ((x-)/)

Example: Heights
X ~ N(68, 3²) inches
P(X > 74) = P(Z > (74-68)/3) = P(Z > 2) = 1 - Φ(2) 0.0228

Finding Values:
If P(X x) = p, then x = + × Φ⁻¹(p)

Example: Find height exceeded by 10% of population
P(X > x) = 0.10, so P(X x) = 0.90
x = 68 + 3 × Φ⁻¹(0.90) = 68 + 3 × 1.28 = 71.84 inches

Central Limit Theorem:
If X , X , ..., X are independent with mean and variance ²,
then X̄ = (X + X + ... + X)/n approaches N(, ²/n) as n → ∞

This holds regardless of original distribution shape!

Applications:

- Measurement errors
- Test scores and grades
- Physical characteristics (height, weight)
- Financial returns
- Quality control
- Natural phenomena

Other Continuous Distributions

Uniform Distribution

All values in interval equally likely
Parameters: a (minimum), b (maximum)

PDF: f(x) = 1/(b-a) for a x b, 0 otherwise

Mean: E(X) = (a+b)/2
Variance: Var(X) = (b-a)²/12

Example: Random number generator [0,1]
f(x) = 1 for 0 x 1
P(0.3 < X < 0.7) = 0.7 - 0.3 = 0.4

Applications:
- Random number generation
- Modeling uncertainty when only range known
- Simulation studies

Exponential Distribution

Models time between events in Poisson process
Parameter: (rate parameter)

PDF: f(x) = e^(- x) for x 0

Mean: E(X) = 1/
Variance: Var(X) = 1/ ²

CDF: F(x) = 1 - e^(- x)

Memoryless Property:
P(X > s + t | X > s) = P(X > t)

Example: Time between customer arrivals (= 2 per hour)
P(X > 0.5) = e^(-2×0.5) = e^(-1) 0.368

Applications:
• Reliability engineering (time to failure)
• Queueing theory (service times)
• Radioactive decay
• Network modeling

Gamma Distribution

Generalizes exponential distribution
Parameters: (shape), (scale) or (rate = 1/)

PDF: f(x) = (^ /Γ( )) × x^(-1) × e^(- x) for x > 0

Mean: E(X) = /
Variance: Var(X) = / ²

Special Cases:
• = 1: Exponential distribution
• = n/2, = 1/2: Chi-square distribution with n degrees of freedom

Applications:
• Modeling waiting times
• Reliability analysis
• Bayesian statistics (conjugate prior)

Beta Distribution

Defined on interval [0,1]
Parameters: , (shape parameters)

PDF: f(x) = (Γ(+)/(Γ( )Γ( ))) × x^(-1) × (1-x)^(-1)

Mean: E(X) = /(+)
Variance: Var(X) = /[(+)²(+ +1)]

Special Cases:
• = = 1: Uniform[0,1]
• = : Symmetric about 0.5

Applications:
• Modeling proportions and percentages
• Bayesian statistics (conjugate prior for binomial)
• Project management (PERT distribution)
• Quality control

Summary and Key Concepts

Probability provides the mathematical foundation for understanding uncertainty and making statistical inferences from data.

Chapter Summary

Essential Skills Mastered:
Understanding sample spaces, events, and probability axioms
Calculating probabilities using counting techniques
Working with conditional probability and independence
Applying Bayes' theorem for updating probabilities
Analyzing discrete probability distributions
Working with continuous distributions, especially normal
Using probability models for real-world applications

Key Concepts:
• Probability quantifies uncertainty mathematically
• Sample spaces and events provide framework for analysis
• Conditional probability updates beliefs with new information
• Independence means events don't influence each other
• Random variables assign numbers to outcomes
• Distributions describe probability patterns

Fundamental Rules:
• Addition rule: P(A B) = P(A) + P(B) - P(A B)
• Multiplication rule: P(A B) = P(A|B) × P(B)
• Complement rule: P(A') = 1 - P(A)
• Bayes' theorem: P(A|B) = P(B|A) × P(A) / P(B)
• Law of total probability for partitioned sample spaces

Important Distributions:
• Binomial: Fixed trials, constant success probability
• Poisson: Events in fixed intervals
• Normal: Bell-shaped, symmetric, ubiquitous
• Exponential: Time between events, memoryless

Problem-Solving Framework:
• Define sample space and events clearly
• Identify appropriate probability approach
• Use counting techniques when outcomes equally likely
• Apply conditional probability for updated information
• Choose appropriate distribution for modeling
• Verify results make intuitive sense

Applications Covered:

- Medical diagnosis and screening
- Quality control and reliability
- Financial risk assessment
- Games and gambling analysis
- Scientific hypothesis testing
- Machine learning and AI

Next Steps:
Probability concepts prepare you for:
- Sampling distributions and Central Limit Theorem
- Confidence intervals and hypothesis testing
- Regression analysis and correlation
- Advanced statistical modeling
- Bayesian statistics and decision theory

Probability represents the mathematical language of uncertainty, providing the theoretical foundation that makes statistical inference possible. The concepts developed in this chapter—from basic probability rules to sophisticated distribution theory—enable you to model random phenomena, quantify uncertainty, and make informed decisions based on incomplete information.

Understanding probability is essential not only for advanced statistical methods but also for critical thinking in our uncertain world. Whether you're evaluating medical test results, assessing financial risks, or interpreting research findings, probability provides the framework for reasoning logically about uncertain outcomes and making decisions that account for the inherent variability in data and predictions.

The probability distributions and techniques you've learned form the building blocks for statistical inference, where we use sample data to draw conclusions about populations. As you progress to inferential statistics, these probability concepts will provide the theoretical justification for confidence intervals, hypothesis tests, and other methods that allow us to quantify our uncertainty and make reliable generalizations from limited data. ## Bayes with Probability Tree (Medical Test)

```
flow:
  A -> C  (99%)
  B -> D  (95%)
  B -> E  (5%)
  C -> F  (10%)
  C -> G  (90%)
```

From the tree: - `P(+)=0.01*0.95 + 0.99*0.10 = 0.1085` - `P(Disease|+) = (0.01*0.95)/0.1085`
`0.0876`

Even good tests can have low positive predictive value when prevalence is low.

Monte Carlo Estimation Sketch

Use random simulation to approximate probabilities when analytic formulas are difficult.

```
import random

def estimate_prob_two_heads(trials=100000):
    success = 0
    for _ in range(trials):
        a = random.choice([0, 1])
        b = random.choice([0, 1])
        if a == 1 and b == 1:
            success += 1
    return success / trials
```

Random Variables and Common Distributions

A **random variable** turns the outcome of a random experiment into a number (or vector of numbers). Distributions are the rulebook for those numbers. This chapter is the bridge from "probability of events" to "models we can fit and simulate."

Diagram: the pipeline

```
sample space Ω
         outcome
     v
X( ) = number          ← random variable

        discrete:  PMF  p(x) = P(X=x)
        continuous: PDF f, CDF F

             v
       E[X], Var(X), quantiles, simulations
```

1. Discrete vs continuous

	Discrete	Continuous
Values	countable	intervals of reals
Law	PMF $p(x)$	PDF $f(x)$ with $\int f = 1$
Probabilities	sum of PMF	areas under PDF
$P(X = a)$	may be positive	usually 0 for continuous

CDF (always defined): $F(x) = P(X \le x)$.

Properties of F:

- non-decreasing
- right-continuous (standard construction)
- $\lim_{x\to-\infty} F(x) = 0$, $\lim_{x\to+\infty} F(x) = 1$

For continuous X with PDF f: $F'(x) = f(x)$ where continuous, and

$$P(a < X \le b) = F(b) - F(a) = \int_a^b f(t)\,dt.$$

2. Expectation and variance

Discrete:

$$\mathbb{E}[X] = \sum_x x\,p(x), \qquad \mathrm{Var}(X) = \mathbb{E}[X^2] - (\mathbb{E}[X])^2.$$

Continuous: replace sum by integral against f.

Linearity always: $\mathbb{E}[aX + bY] = a\mathbb{E}[X] + b\mathbb{E}[Y]$ (no independence needed).

Variance of independent sum: $\mathrm{Var}(X + Y) = \mathrm{Var}(X) + \mathrm{Var}(Y)$ if independent.

3. Catalog of workhorse distributions

Bernoulli(p)

$X \in \{0, 1\}$, $P(X = 1) = p$.
$\mathbb{E}[X] = p$, $\mathrm{Var}(X) = p(1 - p)$.

Binomial(n, p)

Number of successes in n independent Bernoulli trials:

$$P(X = k) = \binom{n}{k} p^k (1 - p)^{n-k}, \quad k = 0, \dots, n.$$

$\mathbb{E}[X] = np$, $\mathrm{Var}(X) = np(1 - p)$.

```
trial:  · · · · · · · · · ·   n flips
success? Y N Y Y N ...
X = count of Y
```

Geometric(p) (trials until first success)

$\mathbb{E}[X] = 1/p$ (if X counts trials). Model for retries until success.

Poisson(λ)

Counts of rare events in a fixed window:

$$P(X = k) = e^{-\lambda}\frac{\lambda^k}{k!}.$$

$\mathbb{E}[X] = \mathrm{Var}(X) = \lambda$.

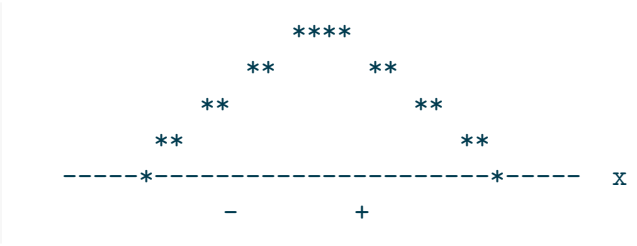
Exponential(λ)

Waiting time between Poisson events; PDF $f(x) = \lambda e^{-\lambda x}$ for $x \geq 0$.
Memoryless: $P(X > s + t \mid X > s) = P(X > t)$.

Normal(μ, σ^2)

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\Big(-\frac{(x-\mu)^2}{2\sigma^2}\Big).$$

CLT: averages of i.i.d. noise often look approximately normal.



Uniform(a, b)

Equal density on $[a, b]$; base case for inverse-transform sampling.

4. Worked examples

Example A. $X \sim \mathrm{Binomial}(10, 0.4)$.
$\mathbb{E}[X] = 4$, $\mathrm{Var}(X) = 10 \cdot 0.4 \cdot 0.6 = 2.4$.

Example B. $X \sim \mathrm{Poisson}(5)$.
$P(X = 0) = e^{-5} \approx 0.0067$.

Example C (standardize). $X \sim N(\mu, \sigma^2)$, then $Z = (X - \mu)/\sigma \sim N(0, 1)$.
$P(X \leq \mu + 1.96\sigma) \approx 0.975$.

Example D (CDF from PMF). Support $\{0, 1, 2\}$ with $p = (0.2, 0.5, 0.3)$:

x	$F(x)$
0	0.2
1	0.7
2	1.0

5. Computer science map

Model	CS use
Bernoulli / Binomial	A/B click, bit errors
Poisson	requests/sec, packet counts
Exponential	inter-arrival, simple reliability
Geometric	retry loops
Normal	measurement noise, CLT for metrics
Uniform	hash bucket idealization, RNG base

6. How to choose a distribution (checklist)

1. Discrete counts or continuous measurements?
2. Bounded range or open-ended?
3. Rare events / memoryless waiting?
4. Symmetric noise around a mean?
5. Fit with QQ-plot / histogram vs model — don't only "pick normal by default."

Exercises

1. **Derive** $\mathrm{Var}(X)$ for Bernoulli(p) from $\mathbb{E}[X^2] - \mathbb{E}[X]^2$.
2. For $X \sim \mathrm{Binomial}(20, 0.5)$, find $\mathbb{E}[X]$ and $\mathrm{Var}(X)$. Is $P(X = 10)$ larger or smaller than $P(X = 0)$? Why?
3. Build the CDF of a discrete RV with PMF $p(-1) = p(1) = 1/4$, $p(0) = 1/2$. Sketch the step function (ASCII is fine).
4. If failures arrive as Poisson(3) per hour, approximate P(at least one failure in 20 minutes).
5. Explain in one paragraph when a normal approximation to Binomial(n, p) is poor.
6. **Challenge:** Show memorylessness of Exponential: $P(X > s + t \mid X > s) = P(X > t)$.

Quick answers (check after you try)

1. $p - p^2 = p(1 - p)$.

2. Mean 10, var 5; $P(X = 10) \gg P(X = 0)$.

3. Window 1/3 hour $\to \lambda = 1$, $P(\geq 1) = 1 - e^{-1}$.

4. When np or $n(1-p)$ is small, or n tiny — mass piles near boundary.

Statistical Inference: Confidence and Hypothesis Tests

Inference turns a finite sample into statements about an unknown population — with explicit uncertainty. This chapter covers confidence intervals, hypothesis tests, errors, power, and A/B testing discipline.

Diagram: inference loop

```
population parameter    (unknown)

            probabilistic claim

    sample X1..Xn      statistic T      CI / test / p-value

            design: sample size, bias, multiple testing
```

1. Point estimates vs interval estimates

- **Point estimate:** single guess $\widehat{\theta}$ (mean, proportion, median).
- **Interval estimate:** set of plausible θ values at a chosen confidence level.

A good estimator is unbiased (or nearly), low variance, and robust to mild outliers when needed.

2. Confidence intervals (means)

For i.i.d. sample with mean $\bar{x}$, sample sd s, large n (or normal data):

$$\bar{x} \pm z_{\alpha/2} \cdot \frac{s}{\sqrt{n}}$$

(for known σ use σ; for small n use t critical values).

Correct interpretation:
If you repeat the *procedure* many times, about $(1-\alpha)$ of the intervals cover the true θ.
After you see one interval, θ is fixed — do **not** say "there is 95% probability θ is in this interval" in the frequentist reading.

```
many replications
   CI     covers
   CI     covers
   CI     misses
   CI     covers
coverage  1-
```

3. Hypothesis testing workflow

1. State H_0 (status quo) and H_1 (research claim).
2. Choose test statistic and null distribution.
3. Compute **p-value** = probability, under H_0, of data as extreme as observed (or more).
4. Compare to significance level α (e.g. 0.05).
5. Report **effect size** and a CI — not only reject/fail-to-reject.

```
H0 true world    sampling distribution of T

                  observed t_obs

                  tail probability = p

             p <  ?  yes   reject H0
                     no    do not reject H0
```

4. Type I, Type II, power

	H_0 true	H_0 false
Reject H_0	Type I error (α)	correct (power)
Keep H_0	correct	Type II error (β)

Power $= 1 - \beta$ increases with larger n, larger true effect, smaller noise, and (sometimes) better design.

5. Two-proportion A/B sketch

Conversion rates $\hat{p}_A$, $\hat{p}_B$ with sizes n_A, n_B.

Pooled proportion under $H_0 : p_A = p_B$:

$$\hat{p} = \frac{x_A + x_B}{n_A + n_B}, \quad z = \frac{\hat{p}_A - \hat{p}_B}{\sqrt{\hat{p}(1-\hat{p})(1/n_A + 1/n_B)}}.$$

Always pair the test with a **CI for** $p_A - p_B$ and a business threshold (0.1% lift may be statistically significant and operationally irrelevant — or the reverse).

6. Multiple testing

Running m independent tests at $\alpha = 0.05$ yields expected $m\alpha$ false positives under global null.

Controls:

- **Bonferroni:** use α/m (conservative).
- **FDR** (Benjamini–Hochberg): control expected fraction of false discoveries among rejects.

```
tests: T1 T2 T3 ... Tm
without correction → false alarms accumulate
with FDR/Bonferroni → threshold adapted
```

7. Sample size (ballpark)

For a z-test on means, detecting difference δ with variance σ^2, power related to

$$n \propto \frac{\sigma^2}{\delta^2}.$$

Halving the detectable effect roughly **quadruples** required n.

8. Pitfalls

Pitfall	Fix
"p=0.04 proves H1 true"	p is tail prob under H0 only
Peeking at dashboards then testing	pre-register or sequential methods
Tiny effect, huge n, "significant"	report effect size + CI
Underpowered study	compute power *before* launch
p-hacking many metrics	multiple-testing control / primary metric

9. Worked mini example

$n = 100$, $\bar{x} = 52$, $s = 10$. 95% CI for mean (large-sample):

$$52 \pm 1.96 \cdot \frac{10}{10} = 52 \pm 1.96 = [50.04, 53.96].$$

Test $H_0 : \mu = 50$ vs two-sided: $z = (52 - 50)/(10/\sqrt{100}) = 2$, p 0.045 $\rightarrow$ reject at 5%, but effect is only 2 units — interpret practically.

Exercises

1. Compute a 95% CI given $\bar{x} = 12$, $s = 4$, $n = 64$ (normal/large-sample).
2. For the CI in (1), what happens to width if n becomes 256?
3. Define Type I and Type II in an A/B test for "new checkout converts better."
4. Explain why a 95% CI and a two-sided test at $\alpha = 0.05$ are linked.
5. You run 20 independent tests of null features at $\alpha = 0.05$. Expected false positives under all-null?
6. **Design:** Want to detect lift $\delta = 0.02$ in conversion with baseline $p = 0.1$. Qualitatively, why is n large?
7. Distinguish **confidence interval** vs **prediction interval** for a new observation.

Quick checks

1. $12 \pm 1.96 \cdot 0.5 \approx [11.02, 12.98]$.

2. Width halves when $n \times 4$.

3. About 1 false positive in expectation.

4. CI for mean parameter; prediction interval for a new draw (wider).

Regression and Experimental Design

Regression models relationships between variables; experimental design produces data that can support causal claims. Together they power A/B testing, forecasting, and scientific product decisions. This chapter covers linear models, diagnostics, the correlation–causation gap, and randomized experiments.

1. Linear regression model

$$y = X\beta + \varepsilon,$$

where $y \in \mathbb{R}^n$ responses, $X \in \mathbb{R}^{n \times p}$ design matrix (often with a column of ones for intercept), $\beta \in \mathbb{R}^p$ coefficients, ε errors.

Ordinary least squares (OLS):

$$\hat{\beta} = \arg\min_{\beta} \|y - X\beta\|_2^2 = (X^\top X)^{-1} X^\top y$$

when X has full column rank.

Worked example 1 — simple regression

$y_i = \beta_0 + \beta_1 x_i + \varepsilon_i$. Slope $\hat{\beta}_1$ is $\mathrm{Cov}(x, y)/\mathrm{Var}(x)$ on the sample. Interpret: expected change in y per unit x, **holding the model true**.

Worked example 2 — multiple regression

Include multiple predictors; $\hat{\beta}_j$ is partial effect controlling for other columns of X (linearly).

2. Classical OLS assumptions

1. Linearity in parameters: $\mathbb{E}[y \mid X] = X\beta$
2. Random sampling / correct conditioning on X
3. Full rank X (no perfect multicollinearity)

4. $\mathbb{E}[\varepsilon \mid X] = 0$ (exogeneity)

5. Homoscedasticity: $\mathrm{Var}(\varepsilon \mid X) = \sigma^2 I$ (for classical SE)

6. Optional: Gaussian errors for exact finite-sample t tests

Violations change interpretation and valid inference methods (robust SE, GLS, nonlinear models).

Worked example 3 — omitted variable bias

True model includes confounder z correlated with x and y. OLS of y on x alone attributes z's effect to x—biased for causal β_x.

3. Fitted values, residuals, R^2

$\hat{y} = X\hat{\beta} = Hy$ with hat matrix $H = X(X^\top X)^{-1}X^\top$.
Residuals $e = y - \hat{y} = (I - H)y$.

$R^2 = 1 - \frac{\|e\|^2}{\|y - \bar{y}1\|^2}$ measures in-sample linear fit—not causality, not guarantee of prediction on new data.

Worked example 4

High R^2 with nonsense predictors on small n: overfitting. Prefer adjusted R^2, CV error, or information criteria for model comparison.

4. Inference for coefficients

Under classical Gaussian assumptions,

$$\hat{\beta}_j \sim N(\beta_j,\ \sigma^2 (X^\top X)^{-1}_{jj}).$$

t-statistics test $H_0 : \beta_j = 0$. Use heteroscedasticity-robust (Eicker–Huber–White) SEs when variance is non-constant.

Worked example 5 — confidence interval

Approx $\hat{\beta}_j \pm t_{1-\alpha/2} \cdot \mathrm{se}_j$. Large n: $z_{1-\alpha/2} \approx 1.96$ for 95%.

5. Diagnostics

Tool	What it catches
Residuals vs fitted	nonlinearity, heteroscedasticity
QQ plot of residuals	non-Gaussian tails
Leverage H_{ii}	high-influence design points
Cook's distance	influential observations
VIF	multicollinearity

Worked example 6

One extreme x with ordinary y can dominate slope—check leverage and leave-one-out.

Worked example 7 — log transform

Right-skewed positive y (traffic, revenue): model $\log y$ often stabilizes variance and linearizes multiplicative effects.

6. Regularized regression (link)

Ridge/lasso when p large or columns collinear—see Optimization regularization chapter. Prediction vs interpretation tradeoffs remain.

Worked example 8

With $p \approx n$, OLS variance explodes; ridge shrinks coefficients and improves MSE prediction.

7. Correlation is not causation

Observational regression estimates associations. Causal claims need:

- Randomized experiment, or
- Credible identification (instruments, RD, differences-in-differences, structural assumptions)

Worked example 9 — ice cream and drowning

Both rise in summer; temperature confounds. Regression of drowning on ice cream sales is not a causal effect of ice cream.

Worked example 10 — product metrics

Users who use feature F have higher retention. Maybe power users adopt F. Need experiment or quasi-experiment to claim F causes retention.

8. Experimental design and A/B tests

Randomized controlled experiment: assign treatment T randomly; compare outcomes.

Core principles

1. **Randomization** balances confounders in expectation
2. **Control group** provides counterfactual baseline
3. **Unit of randomization** (user, session, cluster) matches interference structure
4. **Blinding** when subjective outcomes / experimenter effects matter
5. **Pre-registration / primary metric** fights p-hacking
6. **Power and sample size** before launching

Worked example 11 — sample size sketch

To detect effect δ on a mean with sd σ, two-sided $\alpha = 0.05$, power 80%, roughly

$$n \approx 2 \cdot \frac{(z_{1-\alpha/2} + z_{1-\beta})^2 \sigma^2}{\delta^2}$$

per arm (order of magnitude; use exact calculators in practice).

Worked example 12 — SRM

Sample ratio mismatch: if assignment should be 50/50 but is 48/52 systematically, instrumentation bug—do not trust results until fixed.

Threat	Example

9. Threats to validity

Threat	Example
Interference / network effects	treated users affect controls
Novelty effects	short-term lift fades
Selection into logging	only survivors observed
Multiple testing	20 metrics, one "significant" by chance
Peeking	optional stopping inflates Type I error

Worked example 13 — sequential testing

Always-valid p-values or group sequential designs allow monitoring without naive inflation.

Worked example 14 — CUPED / variance reduction

Use pre-period covariates to reduce variance of estimators—more power, same causal justification under randomization.

10. From experiment to regression

Randomized T: OLS of Y on T estimates ATE under SUTVA/no interference. Add covariates for precision (ANCOVA), not for "fixing selection" that randomization already handled.

Worked example 15 — heterogeneous effects

Interact T with segment indicators—or better, pre-specify subgroups to avoid fishing.

11. Pitfalls

1. Causal language for observational $\hat{\beta}$
2. Stepwise regression as automatic truth
3. Extrapolating far outside observed X range
4. A/B tests without power $\rightarrow$ noisy ship decisions

5. Multiple comparisons uncorrected
6. Cluster randomization analyzed as if independent users

12. Checkpoint

- Write OLS objective and solution
- List classical assumptions and one violation fix
- Run residual diagnostics conceptually
- Explain why randomization enables causation
- Design a basic A/B test (unit, metric, power)

Exercises

Easy

1. Fit by hand: three points, line through origin $y = \beta x$.
2. Interpret $\widehat{\beta}_1 = 2.5$ for $x =$ hours studied, $y =$ score (with caveats).
3. Why include an intercept column of ones?
4. Define residual; show they sum to 0 when intercept present.
5. What does $R^2 = 1$ mean? Is it always good?

Medium

6. Derive normal equations from $\nabla_\beta \|y - X\beta\|^2 = 0$.
7. Construct a numeric omitted-variable bias example (small simulation table).
8. Compute leverage intuition: which x is high leverage on a line?
9. Design A/B: primary metric, guardrail metric, randomization unit for a social feed change.
10. Explain why peeking inflates false positives.

Challenge

11. Prove $\widehat{\beta}$ unbiased under $\mathbb{E}[\varepsilon \mid X] = 0$ and full rank.
12. Heteroscedasticity: write sandwich SE formula sketch.
13. Cluster-randomized experiment: why usual SE understate uncertainty.
14. Instrumental variables: one-paragraph identification idea.
15. Capstone: write analysis plan for an A/B test including SRM check, CUPED option, and multiple-testing policy.

Summary

Regression quantifies associations with transparent assumptions and diagnostics; experiments create the conditions for causal claims. Sound data science keeps the two distinct, designs for power and validity, and refuses to overclaim from observational fits alone.

Calculus

Calculus for Computer Science and AI

Calculus studies continuous change: limits, derivatives, integrals, and infinite series. In CS and AI it is the language of optimization, learning dynamics, continuous probability, graphics, and the analysis of algorithms that involve growth and accumulation.

Why calculus matters in CS/AI

Area	Calculus role
Machine learning	Gradients, backprop, continuous losses
Optimization	Stationarity, Taylor models, Hessians
Computer graphics	Curves, surfaces, shading rates
Physics engines	ODEs, integration of motion
Probability	Densities, expectations as integrals
Algorithms	Continuous relaxations, potential functions
Signal processing	Transforms, filtering (continuous intuition)

What you will learn

1. **Limits and continuity** — precise "approaches L" language; when models are well-behaved
2. **Derivatives** — local linear approximation; optimization and rates
3. **Integrals** — accumulation, area, FTC, numerical quadrature
4. **Multivariable calculus** — gradients, Jacobians, Hessians for ML
5. **Series and approximations** — Taylor models used everywhere in numerics

```
flow:
  B -> C
  C -> D
  D -> E
  D -> F
  B -> G
  G -> H
  D -> I
```

The derivative in one sentence

If f is differentiable at x, then near x

$$f(x+h) \approx f(x) + f'(x)h,$$

with error $o(h)$ as $h \to 0$. Gradient descent, Newton methods, and sensitivity analysis are organized around this fact.

Worked example 1 — sensitivity

If loss $L(w)$ has $L'(w) = 0.01$, a step $\Delta w = -0.1$ predicts $\Delta L \approx -0.001$ to first order—step-size reasoning.

Worked example 2 — non-differentiable ML

$|w|$ and $\mathrm{ReLU}(z) = \max(0, z)$ are not differentiable at kinks; practice uses subgradients / one-sided derivatives. Continuity still holds.

The integral in one sentence

The definite integral $\int_a^b f$ accumulates f over $[a, b]$—signed area, total mass, probability from a density, or total cost over time.

Worked example 3

If f_X is a PDF, $P(a \le X \le b) = \int_a^b f_X$. No density calculus, no continuous probability.

Multivariable leap for ML

Parameters $w \in \mathbb{R}^d$: gradient $\nabla L(w)$ is the vector of partials; Hessian $\nabla^2 L$ encodes curvature. Backpropagation is the chain rule applied to computational graphs.

Worked example 4

$L(w) = \frac{1}{2}\|Xw - y\|_2^2 \Rightarrow \nabla L = X^\top(Xw - y)$—matrix calculus as multivariable derivatives.

Prerequisites

- Algebra of functions, polynomials, exp/log
- Coordinates in $\mathbb{R}^2$ and $\mathbb{R}^3$
- Optional: basic linear algebra for multivariable chapters

Study sequence (CS-oriented)

1. Limits and continuity mechanics
2. Derivatives and 1D optimization
3. Integrals and FTC; probability link
4. Gradients/Jacobians/Hessians
5. Taylor series for local models and numerics

Pitfalls

1. Treating all ML losses as everywhere differentiable
2. Finite differences with h too small (cancellation)
3. Confusing partial vs total derivatives in time-dependent systems
4. Integrating probability densities that do not normalize
5. Trusting Taylor far from expansion point

Checkpoint

- Explain derivative as local linear map
- State FTC linking derivatives and integrals
- Write a gradient for a simple $f : \mathbb{R}^2 \to \mathbb{R}$
- Give two CS applications of series
- Know when ReLU breaks classical derivative hypotheses

Exercises

1. Why is $f(x) = |x|$ not differentiable at 0?
2. Differentiate $x^3 e^x$ (product rule).
3. Interpret $\int_0^1 v(t)\,dt$ if v is velocity.
4. Give one optimization problem in CS modeled with derivatives.
5. Compute ∇f for $f(x, y) = x^2 + y^2$.
6. What does a large Hessian condition number mean for GD?
7. Approximate $\sqrt{1 + x}$ for small x by first-order Taylor.
8. Link cross-entropy training to gradient steps on probabilities.
9. Explain continuity of ReLU vs differentiability.
10. Checkpoint project: write the gradient of logistic loss for one example (x, y).

Summary

Calculus gives CS the mathematics of local change and global accumulation. The chapters ahead make limits precise, turn derivatives into optimizers and backprop, connect integrals to probability and numerics, and extend everything to the multivariable setting of modern ML.

Limits and Continuity

Limits make precise the idea that $f(x)$ approaches a value as x approaches a point. Continuity means the function's value matches that limiting behavior. These notions underwrite derivatives, integrals, and the well-posedness of many numerical methods.

1. Informal idea

$\lim_{x \to a} f(x) = L$ means: $f(x)$ can be forced arbitrarily close to L by taking x sufficiently close to a (but not necessarily equal to a).

Worked example 1

$f(x) = (x^2 - 1)/(x - 1)$ for $x \neq 1$. As $x \to 1$, $f(x) \to 2$, even though f may be undefined at 1.

2. Epsilon–delta definition

Definition. $\lim_{x \to a} f(x) = L$ means: for every $\varepsilon > 0$ there exists $\delta > 0$ such that if $0 < |x - a| < \delta$, then $|f(x) - L| < \varepsilon$.

Worked example 2 — linear function

Prove $\lim_{x \to 3}(2x + 1) = 7$. Given $\varepsilon > 0$, need $|2x + 1 - 7| = 2|x - 3| < \varepsilon$, so pick $\delta = \varepsilon/2$.

Worked example 3 — quadratic sketch

$\lim_{x \to 2} x^2 = 4$: bound $|x^2 - 4| = |x - 2||x + 2|$; restrict $\delta \leq 1$ so $|x + 2| \leq 5$, then $\delta = \min(1, \varepsilon/5)$.

3. One-sided limits

- Right-hand: $x \to a^+$ with $x > a$
- Left-hand: $x \to a^-$ with $x < a$

Two-sided limit exists iff both one-sided limits exist and are equal.

Worked example 4 — jump

$f(x) = 0$ for $x < 0$, $f(x) = 1$ for $x \geq 0$: $\lim_{x \to 0^-} f = 0$, $\lim_{x \to 0^+} f = 1$—no two-sided limit.

Worked example 5 — absolute value

$\lim_{x \to 0} |x| = 0$ even though not differentiable there—limits weaker than derivatives.

4. Infinite limits and asymptotes

$\lim_{x \to a} f(x) = \infty$ means f grows without bound as $x \to a$ (vertical asymptote patterns). Horizontal asymptotes: $\lim_{x \to \pm\infty} f(x) = L$.

Worked example 6

$1/x \to +\infty$ as $x \to 0^+$; $\to -\infty$ as $x \to 0^-$.
$(1+1/n)^n \to e$ as $n \to \infty$—sequence limit important in CS interest rates / continuous compounding / Poisson processes.

5. Continuity

Definition. f continuous at a if:

1. $f(a)$ defined
2. $\lim_{x \to a} f(x)$ exists
3. $\lim_{x \to a} f(x) = f(a)$

Continuous on an interval if continuous at every point (one-sided at endpoints).

Worked example 7

Polynomials, exp, sin, cos are continuous everywhere on $\mathbb{R}$. Rational functions continuous where denominator $\neq 0$.

Worked example 8 — removable discontinuity

Define $f(1) = 2$ for the simplified $(x^2 - 1)/(x - 1)$ to make continuous at 1.

6. Types of discontinuities

Type	Behavior
Removable	limit exists, $f(a)$ or $f(a)$ missing
Jump	one-sided limits differ
Infinite	blow-up
Oscillatory	e.g. $\sin(1/x)$ as $x \to 0$

Worked example 9

$\sin(1/x)$ as $x \to 0$: no limit (oscillates wildly).

7. Algebra of limits

If $\lim f = L$, $\lim g = M$:

- $\lim(f \pm g) = L \pm M$
- $\lim(fg) = LM$
- $\lim(f/g) = L/M$ if $M \neq 0$
- $\lim g \circ f = g(L)$ if g continuous at L

Worked example 10

$\lim_{x\to 0} \frac{\sin x}{x} = 1$ (standard theorem)—foundation for derivative of sin.

Worked example 11

$\lim_{x\to 0} \frac{e^x - 1}{x} = 1$—derivative of exp at 0.

8. Intermediate Value Theorem (IVT)

Theorem. If f continuous on $[a, b]$ and N between $f(a)$ and $f(b)$, then $\exists c \in [a, b]$ with $f(c) = N$.

CS use: bisection root finding requires a sign change and continuity—IVT guarantees a root.

Worked example 12

$f(x) = x^3 - x - 2$ on $[1, 2]$: $f(1) < 0 < f(2) \Rightarrow$ root exists.

9. Extreme Value Theorem

Continuous f on compact $[a, b]$ attains min and max. Underpins optimization existence on closed bounded sets.

10. CS connections

- Stability of numerical methods needs continuous dependence on data
- Activation functions: sigmoid smooth; ReLU continuous, not everywhere differentiable
- Floating-point "limits" are not mathematical limits—watch cancellation near removable holes
- Algorithm analysis: $\lim_{n\to\infty} T(n)/g(n)$ for asymptotics (discrete n, same language)

Worked example 13 — softplus

softplus$(x) = \log(1 + e^x)$ continuous smooth approximation to ReLU—limits as $x \to \pm\infty$ match ReLU asymptotics.

11. Pitfalls

1. Plugging $x = a$ when f undefined (always simplify first)
2. Assuming limit equals function value without continuity
3. $\infty - \infty$ or $0/0$ indeterminate forms without algebra
4. Sequence limits vs real variable limits confusion
5. Claiming IVT for discontinuous f

12. Checkpoint

- State ε–δ definition
- Compute standard trig/exp limits
- Classify discontinuities
- Apply IVT to guarantee roots
- Explain continuity of common ML activations

Exercises

Easy

1. ε–δ proof for $\lim_{x\to 1}(5x-2)=3$.
2. One-sided limits of $\operatorname{sign}(x)$ at 0.
3. Make $(x^2-9)/(x-3)$ continuous at 3 by defining $f(3)$.
4. Does $\lim_{x\to 0} x\sin(1/x)$ exist?
5. Why bisection needs continuity + sign change?

Medium

6. Prove algebra rule for sum of limits (outline).
7. Show polynomials are continuous using algebra of limits.
8. Classify discontinuities of a floor function and $1/(x-1)$.
9. Use IVT to show any continuous $f:[0,1]\to[0,1]$ has a fixed point (hint: $f(x)-x$).
10. Compute $\lim_{h\to 0}\frac{(x+h)^n-x^n}{h}$ as preview of power rule.

Challenge

11. Full ε–δ for $\lim_{x\to a} x^2=a^2$.
12. Prove $\lim_{x\to 0}\sin x/x=1$ using geometric arguments or known inequalities.
13. Show $\sin(1/x)$ has no limit as $x\to 0$ rigorously.
14. Heine definition: sequential characterization of limits; use to disprove a limit.
15. Discuss continuity of neural network maps built from continuous activations and affine layers.

Summary

Limits formalize approachable values; continuity glues limit and function value. Algebra rules, IVT, and standard limits are the working toolkit—and they justify root-finding, asymptotic reasoning, and the smoothness assumptions behind much of continuous optimization.

Derivatives and Applications in Computer Science

Derivatives are crucial in computer science for optimization, machine learning, and algorithm analysis. This section covers derivatives with practical CS applications.

What is a Derivative?

A derivative measures the rate of change of a function. In CS contexts: - **Gradient descent**: Uses derivatives to find optimal parameters - **Backpropagation**: Computes derivatives to train neural networks - **Algorithm analysis**: Derivatives help analyze growth rates

Basic Derivative Rules

Power Rule

If f(x) = x^n, then f'(x) = nx^(n-1)

```
import numpy as np
import matplotlib.pyplot as plt

# Example: f(x) = x^2, f'(x) = 2x
x = np.linspace(-3, 3, 100)
f = x**2
f_prime = 2*x

plt.figure(figsize=(10, 6))
plt.subplot(1, 2, 1)
plt.plot(x, f, 'b-', label='f(x) = x²')
plt.title('Original Function')
plt.grid(True)
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(x, f_prime, 'r-', label="f'(x) = 2x")
plt.title('Derivative')
plt.grid(True)
plt.legend()
plt.show()
```

Chain Rule

If f(x) = g(h(x)), then f'(x) = g'(h(x)) × h'(x)

This is fundamental in neural networks for backpropagation.

Applications in Machine Learning

1. Gradient Descent

Gradient descent uses derivatives to minimize cost functions:

```
import numpy as np

def gradient_descent(f, df, x0, learning_rate=0.01, iterations=1000):
    """
    Minimize function f using gradient descent
    f: function to minimize
    df: derivative of f
    x0: starting point
    """
    x = x0
    history = [x]

    for i in range(iterations):
        gradient = df(x)
        x = x - learning_rate * gradient
        history.append(x)

    return x, history

# Example: minimize f(x) = x^2 + 2x + 1
def f(x):
    return x**2 + 2*x + 1

def df(x):
    return 2*x + 2

# Find minimum
minimum, path = gradient_descent(f, df, x0=5.0)
print(f"Minimum found at x = {minimum:.4f}")
print(f"Function value: {f(minimum):.4f}")
```

2. Neural Network Backpropagation

```python
import numpy as np

class SimpleNeuron:
    def __init__(self, weights, bias):
        self.weights = np.array(weights)
        self.bias = bias

    def sigmoid(self, x):
        return 1 / (1 + np.exp(-x))

    def sigmoid_derivative(self, x):
        s = self.sigmoid(x)
        return s * (1 - s)

    def forward(self, inputs):
        self.inputs = np.array(inputs)
        self.z = np.dot(self.weights, self.inputs) + self.bias
        self.output = self.sigmoid(self.z)
        return self.output

    def backward(self, error):
        # Compute gradients using chain rule
        d_output = error
        d_z = d_output * self.sigmoid_derivative(self.z)
        d_weights = d_z * self.inputs
        d_bias = d_z
        d_inputs = d_z * self.weights

        return d_weights, d_bias, d_inputs

# Example usage
neuron = SimpleNeuron([0.5, -0.3], 0.1)
output = neuron.forward([1.0, 2.0])
gradients = neuron.backward(0.1)  # Small error
print(f"Output: {output:.4f}")
print(f"Weight gradients: {gradients[0]}")
```

Applications in Computer Graphics

1. Parametric Curves

Derivatives help create smooth curves in graphics:

```
import numpy as np
import matplotlib.pyplot as plt

def bezier_curve(t, P0, P1, P2, P3):
    """Cubic Bezier curve"""
    return (1-t)**3 * P0 + 3*(1-t)**2*t * P1 + 3*(1-t)*t**2 * P2 + t**3 * P3

def bezier_derivative(t, P0, P1, P2, P3):
    """Derivative of cubic Bezier curve (tangent vector)"""
    return 3*(1-t)**2 * (P1-P0) + 6*(1-t)*t * (P2-P1) + 3*t**2 * (P3-P2)

# Control points
P0, P1, P2, P3 = np.array([0,0]), np.array([1,2]), np.array([3,2]), np.array([4,0])

t = np.linspace(0, 1, 100)
curve = np.array([bezier_curve(ti, P0, P1, P2, P3) for ti in t])
tangents = np.array([bezier_derivative(ti, P0, P1, P2, P3) for ti in t])

plt.figure(figsize=(10, 6))
plt.plot(curve[:, 0], curve[:, 1], 'b-', linewidth=2, label='Bezier Curve')
plt.plot([P0[0], P1[0], P2[0], P3[0]], [P0[1], P1[1], P2[1], P3[1]], 'ro--', alpha=0.5, labe

# Show tangent vectors at a few points
for i in range(0, len(t), 20):
    start = curve[i]
    direction = tangents[i] * 0.1  # Scale for visibility
    plt.arrow(start[0], start[1], direction[0], direction[1],
              head_width=0.05, head_length=0.05, fc='red', ec='red')

plt.axis('equal')
plt.grid(True)
plt.legend()
plt.title('Bezier Curve with Tangent Vectors')
plt.show()
```

2. Surface Normals

In 3D graphics, derivatives help compute surface normals:

```
import numpy as np
from mpl_toolkits.mplot3d import Axes3D
import matplotlib.pyplot as plt

def surface_function(x, y):
    """Example surface: z = x^2 + y^2"""
    return x**2 + y**2
```

```
def surface_normal(x, y):
    """Compute normal vector using partial derivatives"""
    #  z/ x = 2x,  z/ y = 2y
    dz_dx = 2*x
    dz_dy = 2*y

    # Normal vector is (- z/ x, - z/ y, 1)
    normal = np.array([-dz_dx, -dz_dy, 1])
    # Normalize
    return normal / np.linalg.norm(normal)

# Create surface
x = np.linspace(-2, 2, 20)
y = np.linspace(-2, 2, 20)
X, Y = np.meshgrid(x, y)
Z = surface_function(X, Y)

# Compute normals
normals = np.array([[surface_normal(xi, yi) for xi, yi in zip(row_x, row_y)]
                    for row_x, row_y in zip(X, Y)])

fig = plt.figure(figsize=(12, 5))

# Plot surface
ax1 = fig.add_subplot(121, projection='3d')
ax1.plot_surface(X, Y, Z, alpha=0.7, cmap='viridis')
ax1.set_title('Surface: z = x² + y²')

# Plot surface with normals
ax2 = fig.add_subplot(122, projection='3d')
ax2.plot_surface(X, Y, Z, alpha=0.3, cmap='viridis')

# Show normal vectors at selected points
step = 3
for i in range(0, X.shape[0], step):
    for j in range(0, X.shape[1], step):
        start = [X[i,j], Y[i,j], Z[i,j]]
        direction = normals[i,j] * 0.5
        ax2.quiver(start[0], start[1], start[2],
                   direction[0], direction[1], direction[2],
                   color='red', arrow_length_ratio=0.1)

ax2.set_title('Surface with Normal Vectors')
plt.show()
```

Applications in Algorithm Analysis

Growth Rate Analysis

Derivatives help analyze algorithm complexity:

```
import numpy as np
import matplotlib.pyplot as plt

def analyze_growth_rate(f, df, x_range):
    """Analyze how fast a function grows"""
    x = np.linspace(x_range[0], x_range[1], 1000)
    y = f(x)
    dy = df(x)

    plt.figure(figsize=(12, 4))

    plt.subplot(1, 3, 1)
    plt.plot(x, y, 'b-', linewidth=2)
    plt.title('Function f(x)')
    plt.grid(True)

    plt.subplot(1, 3, 2)
    plt.plot(x, dy, 'r-', linewidth=2)
    plt.title("Growth Rate f'(x)")
    plt.grid(True)

    plt.subplot(1, 3, 3)
    plt.loglog(x[x>0], y[x>0], 'b-', linewidth=2, label='f(x)')
    plt.loglog(x[x>0], dy[x>0], 'r-', linewidth=2, label="f'(x)")
    plt.title('Log-Log Plot')
    plt.legend()
    plt.grid(True)

    plt.tight_layout()
    plt.show()

# Compare different complexity functions
functions = [
    (lambda x: x, lambda x: np.ones_like(x), "O(n)"),
    (lambda x: x*np.log(x), lambda x: np.log(x) + 1, "O(n log n)"),
    (lambda x: x**2, lambda x: 2*x, "O(n²)"),
    (lambda x: x**3, lambda x: 3*x**2, "O(n³)")
]

x = np.linspace(1, 100, 1000)
plt.figure(figsize=(12, 8))
```

```
for i, (f, df, label) in enumerate(functions):
    plt.subplot(2, 2, i+1)
    y = f(x)
    dy = df(x)

    plt.plot(x, y, 'b-', linewidth=2, label=f'f(x) = {label}')
    plt.plot(x, dy, 'r--', linewidth=2, label="f'(x)")
    plt.title(f'Growth Analysis: {label}')
    plt.legend()
    plt.grid(True)

plt.tight_layout()
plt.show()
```

Optimization Problems

Finding Critical Points

```
import numpy as np
from scipy.optimize import minimize_scalar
import matplotlib.pyplot as plt

def find_critical_points(f, df, x_range):
    """Find critical points where f'(x) = 0"""
    x = np.linspace(x_range[0], x_range[1], 1000)
    y = f(x)
    dy = df(x)

    # Find where derivative is approximately zero
    critical_points = []
    for i in range(1, len(dy)-1):
        if dy[i-1] * dy[i+1] < 0:  # Sign change
            # Use more precise method
            result = minimize_scalar(lambda x: abs(df(x)),
                                     bounds=(x[i-1], x[i+1]),
                                     method='bounded')
            critical_points.append(result.x)

    plt.figure(figsize=(12, 4))

    plt.subplot(1, 2, 1)
    plt.plot(x, y, 'b-', linewidth=2, label='f(x)')
    for cp in critical_points:
        plt.plot(cp, f(cp), 'ro', markersize=8, label=f'Critical point: x={cp:.2f}')
```

```
    plt.title('Function with Critical Points')
    plt.legend()
    plt.grid(True)

    plt.subplot(1, 2, 2)
    plt.plot(x, dy, 'r-', linewidth=2, label="f'(x)")
    plt.axhline(y=0, color='k', linestyle='--', alpha=0.5)
    for cp in critical_points:
        plt.plot(cp, df(cp), 'ro', markersize=8)
    plt.title('Derivative (zeros are critical points)')
    plt.legend()
    plt.grid(True)

    plt.tight_layout()
    plt.show()

    return critical_points

# Example: f(x) = x^3 - 3x^2 + 2x + 1
def f(x):
    return x**3 - 3*x**2 + 2*x + 1

def df(x):
    return 3*x**2 - 6*x + 2

critical_points = find_critical_points(f, df, (-1, 3))
print("Critical points found at:", critical_points)
```

Practical Exercises

Exercise 1: Implement Gradient Descent for Linear Regression

```
import numpy as np
import matplotlib.pyplot as plt

def linear_regression_gradient_descent():
    # Generate sample data
    np.random.seed(42)
    X = np.random.randn(100, 1)
    y = 2 * X.flatten() + 1 + 0.1 * np.random.randn(100)

    # Initialize parameters
    w = 0.0  # weight
    b = 0.0  # bias
```

```
learning_rate = 0.01
iterations = 1000

# Cost function: MSE
def cost_function(w, b, X, y):
    predictions = w * X.flatten() + b
    return np.mean((predictions - y)**2)

# Gradients
def compute_gradients(w, b, X, y):
    m = len(y)
    predictions = w * X.flatten() + b
    dw = (2/m) * np.sum((predictions - y) * X.flatten())
    db = (2/m) * np.sum(predictions - y)
    return dw, db

# Training loop
costs = []
for i in range(iterations):
    cost = cost_function(w, b, X, y)
    costs.append(cost)

    dw, db = compute_gradients(w, b, X, y)
    w -= learning_rate * dw
    b -= learning_rate * db

# Plot results
plt.figure(figsize=(12, 4))

plt.subplot(1, 2, 1)
plt.scatter(X, y, alpha=0.5, label='Data')
plt.plot(X, w * X.flatten() + b, 'r-', linewidth=2, label=f'Fitted line: y = {w:.2f}x +
plt.xlabel('X')
plt.ylabel('y')
plt.legend()
plt.title('Linear Regression Result')

plt.subplot(1, 2, 2)
plt.plot(costs)
plt.xlabel('Iteration')
plt.ylabel('Cost')
plt.title('Cost Function During Training')
plt.grid(True)

plt.tight_layout()
plt.show()
```

```
    print(f"Final parameters: w = {w:.4f}, b = {b:.4f}")
    print(f"True parameters: w = 2.0, b = 1.0")

linear_regression_gradient_descent()
```

Exercise 2: Numerical Differentiation

```
def numerical_derivative(f, x, h=1e-7):
    """Compute numerical derivative using central difference"""
    return (f(x + h) - f(x - h)) / (2 * h)

def compare_derivatives():
    """Compare analytical and numerical derivatives"""

    # Test function: f(x) = x^3 + 2x^2 - x + 1
    def f(x):
        return x**3 + 2*x**2 - x + 1

    def analytical_derivative(x):
        return 3*x**2 + 4*x - 1

    x_values = np.linspace(-3, 3, 100)
    analytical = [analytical_derivative(x) for x in x_values]
    numerical = [numerical_derivative(f, x) for x in x_values]

    plt.figure(figsize=(10, 6))
    plt.plot(x_values, analytical, 'b-', linewidth=2, label='Analytical')
    plt.plot(x_values, numerical, 'r--', linewidth=2, label='Numerical')
    plt.xlabel('x')
    plt.ylabel("f'(x)")
    plt.title('Analytical vs Numerical Derivatives')
    plt.legend()
    plt.grid(True)
    plt.show()

    # Compute error
    error = np.abs(np.array(analytical) - np.array(numerical))
    print(f"Maximum error: {np.max(error):.2e}")
    print(f"Average error: {np.mean(error):.2e}")

compare_derivatives()
```

Summary

Derivatives are essential in computer science for:

1. **Optimization**: Finding optimal solutions using gradient-based methods
2. **Machine Learning**: Training neural networks through backpropagation
3. **Computer Graphics**: Creating smooth curves and computing surface properties
4. **Algorithm Analysis**: Understanding growth rates and complexity
5. **Numerical Methods**: Approximating solutions to complex problems

The key insight is that derivatives measure rates of change, which is fundamental to optimization and learning algorithms that form the backbone of modern AI and machine learning systems.

Integrals and Accumulation

Integration measures accumulation: area, total mass, probability, and net change. The Fundamental Theorem of Calculus (FTC) links integrals to antiderivatives, while numerical quadrature and Monte Carlo extend integration to settings without closed forms—common in CS, graphics, and ML.

1. Definite integral as accumulation

For f on $[a, b]$, the definite integral $\int_a^b f(x)\,dx$ is the signed accumulation of f. If $f \geq 0$, it is area under the graph.

Worked example 1

$\int_0^1 x^2\,dx = \frac{1}{3}$ (area under parabola).

2. Riemann sums

Partition $a = x_0 < \cdots < x_n = b$, widths $\Delta x_i = x_i - x_{i-1}$, sample $x_i^* \in [x_{i-1}, x_i]$:

$$\sum_{i=1}^{n} f(x_i^*)\Delta x_i.$$

If these sums approach I as mesh $\max \Delta x_i \to 0$, then $\int_a^b f = I$.

Worked example 2

Right Riemann sum for x^2 on $[0, 1]$ with n equal parts: $\sum_{i=1}^{n} (i/n)^2 \cdot (1/n) \to \int_0^1 x^2$.

3. Fundamental Theorem of Calculus

FTC Part 1. If f continuous and $F(x) = \int_a^x f(t)\,dt$, then $F'(x) = f(x)$.

FTC Part 2. If $F' = f$ on $[a, b]$, then $\int_a^b f = F(b) - F(a)$.

Worked example 3

$\int_0^{\pi} \sin x \, dx = [-\cos x]_0^{\pi} = 2$.

Worked example 4 — net change

If $v(t)$ is velocity, $\int_{t_1}^{t_2} v(t)\,dt$ is net displacement—not always total distance (use $\int |v|$).

4. Antiderivatives and techniques (brief)

Linearity: $\int (af + bg) = a\int f + b\int g$.
Substitution and parts reverse chain/product rules.

Worked example 5

$\int 2xe^{x^2}\,dx = e^{x^2} + C$ via $u = x^2$.

5. Improper integrals

Replace infinite bounds or singularities by limits:

$$\int_1^{\infty} x^{-p}\,dx \text{ converges iff } p > 1.$$

Worked example 6

$\int_1^{\infty} \frac{1}{x^2}\,dx = 1$; $\int_1^{\infty} \frac{1}{x}\,dx$ diverges—harmonic series continuous cousin.

Worked example 7 — probability

A PDF f needs $\int_{-\infty}^{\infty} f = 1$; tails must integrate finitely.

6. Numerical integration

Method	Idea	Error (smooth f)
Trapezoidal	piecewise linear	$O(h^2)$ global
Simpson	piecewise quadratic	$O(h^4)$ global
Monte Carlo	random samples	$O(1/\sqrt{N})$ statistical

Worked example 8

Trapezoid on $[0,1]$ for x^2 with 2 panels: approx 0.375 vs true $1/3 \approx 0.333$.

Worked example 9 — high dimension

Grid methods suffer curse of dimensionality; Monte Carlo error depends weakly on dimension—key for rendering and Bayesian computation.

Worked example 10 — importance sampling

Sample from proposal q, estimate $\int f = \mathbb{E}_q[f(X)/q(X)]$—variance reduction art.

7. Probability and expectation

Continuous RV with density f_X:

$$P(a \le X \le b) = \int_a^b f_X, \qquad \mathbb{E}[g(X)] = \int g(x) f_X(x)\,dx.$$

Worked example 11

Exponential(λ): $\int_0^\infty \lambda e^{-\lambda x} dx = 1$; mean $\int_0^\infty x\lambda e^{-\lambda x} dx = 1/\lambda$.

8. Multiple integrals (preview)

$\iint_D f$ accumulates over 2D regions—mass, probabilities for joint densities. Fubini's theorem justifies iterated integrals under integrability.

Worked example 12

Uniform density on unit square: area 1; $P(X + Y \le 1) = \frac{1}{2}$.

9. CS applications

- Softmax partition functions / free energies as integrals/sums
- Continuous-time system total cost $\int L(x(t), u(t))\, dt$
- Image irradiance integrals in graphics
- Kernel density estimation and smoothing
- Discrete sums approximated by integrals (Euler–Maclaurin)

Worked example 13 — average case analysis

Integral method for sums: $\sum_{k=1}^{n} k \log k \sim \int x \log x \, dx$.

10. Pitfalls

1. Forgetting absolute value for arc length / distance
2. Divergent improper integrals treated as finite
3. Trapezoid on non-smooth f (error rates drop)
4. Monte Carlo without variance reporting
5. Confusing antiderivative $+C$ with definite integral values

11. Checkpoint

- Write Riemann sum $\to$ integral
- Apply FTC to evaluate simple definite integrals
- Test p-integral convergence at infinity
- Compare trapezoid vs Monte Carlo regimes
- Connect PDF integrals to probabilities

Exercises

Easy

1. $\int_0^2 (3x^2 - 1)\,dx$.
2. Riemann sum definition for $\int_0^1 e^x$ with $n = 4$ right endpoints (compute number).
3. Antiderivative of $1/(1 + x^2)$.
4. Does $\int_0^1 x^{-1/2}\,dx$ converge?
5. Trapezoid rule formula for one panel.

Medium

6. Prove FTC Part 2 assuming Part 1 and mean value ideas (outline).
7. Compare trapezoid and Simpson on sin over $[0, \pi]$ with $n = 4$.
8. Monte Carlo estimate of π via unit disk indicator; SE scaling.
9. Show $\int_1^\infty x^{-p}$ convergence criterion.
10. Expectation of Uniform(a, b) via integral.

Challenge

11. Euler–Maclaurin: state how sums relate to integrals + Bernoulli corrections.
12. Importance sampling variance formula; when it fails.
13. Change of variables in 1D integrals carefully with limits.
14. Gaussian integral $\int_{-\infty}^{\infty} e^{-x^2} dx = \sqrt{\pi}$ (use polar trick outline).
15. Graphics: write the rendering integral for radiance (high-level) and say why MC is used.

Summary

Integrals accumulate continuous contributions; FTC computes them via antiderivatives when available. Numerical and Monte Carlo methods handle the rest. Probability, simulation, and analysis of algorithms all rely on this accumulation viewpoint.

Multivariable Calculus for ML

Machine learning optimizes scalar losses depending on many parameters. Multivariable calculus introduces gradients, directional derivatives, Jacobians, Hessians, and the chain rule on computational graphs—the mathematics of backpropagation and curvature-aware optimization.

1. Functions of several variables

$f : \mathbb{R}^n \to \mathbb{R}$ maps a vector $x = (x_1, \dots, x_n)$ to a real loss or objective. Level sets $\{x : f(x) = c\}$ generalize contours.

Worked example 1

$f(x, y) = x^2 + y^2$ has circular level sets; minimum at origin.

2. Partial derivatives

$\partial f/\partial x_i$ differentiates treating other variables as constant.

Worked example 2

$f(x, y) = x^2 y + \sin y$: $f_x = 2xy$, $f_y = x^2 + \cos y$.

3. Gradient

$$\nabla f(x) = \begin{bmatrix} \partial f/\partial x_1 \\ \vdots \\ \partial f/\partial x_n \end{bmatrix}.$$

Facts:

- Points in direction of steepest ascent
- $\|\nabla f\|$ is the steepest-ascent rate
- $\nabla f = 0$ at smooth critical points

Worked example 3

$f(x,y) = x^2 + 3xy + y^2$: $\nabla f = (2x + 3y,\ 3x + 2y)$. Critical point solve $\nabla f = 0 \Rightarrow (x,y) = (0,0)$.

Worked example 4 — GD step

$x \leftarrow x - \alpha \nabla f(x)$ moves opposite the gradient—steepest descent for small α.

4. Directional derivatives

For unit u, $D_u f = \nabla f \cdot u$. Maximized when $u \parallel \nabla f$, value $\|\nabla f\|$.

Worked example 5

At $(1,1)$ for $f = x^2 + y^2$, $\nabla f = (2,2)$, direction $u = (1,0)$: $D_u f = 2$.

5. Chain rule and backpropagation

If $z = f(y)$, $y = g(x)$, then Dz/Dx multiplies Jacobians.

Scalar case: $\frac{dz}{dx} = \frac{dz}{dy}\frac{dy}{dx}$.

Vector case: if $x \in \mathbb{R}^n$, $y \in \mathbb{R}^m$, $z \in \mathbb{R}$,

$$\nabla_x z = (D_x y)^\top \nabla_y z.$$

Backprop: recurse this identity on a graph of elementary ops.

Worked example 6 — logistic

$z = w^\top x$, $p = \sigma(z)$, $\ell = -y \log p - (1-y)\log(1-p)$.
$\partial \ell / \partial z = p - y$, then $\nabla_w \ell = (p-y)x$.

Worked example 7 — Matmul layer

$Y = XW$ shapes: gradients w.r.t. W are $X^\top G$ if $G = \partial L/\partial Y$—shape-check discipline.

6. Jacobian matrix

For $F : \mathbb{R}^n \to \mathbb{R}^m$, J_F has $J_{ij} = \partial F_i / \partial x_j$. Linearization $F(x+h) \approx F(x) + J_F(x)h$.

Worked example 8

Softmax $F : \mathbb{R}^k \to \mathbb{R}^k$ has Jacobian $\operatorname{diag}(p) - pp^\top$.

7. Hessian and curvature

$H_{ij} = \partial^2 f/(\partial x_i \partial x_j)$. For smooth f, H symmetric (mixed partials equal).
At critical point $\nabla f = 0$:

- $H \succ 0$: local minimum
- $H \prec 0$: local maximum
- indefinite: saddle

Worked example 9

$f = x^2 + 3xy + y^2$: $H = \begin{bmatrix} 2 & 3 \\ 3 & 2 \end{bmatrix}$, eigenvalues $5, -1$—saddle at 0 despite some positive curvature directions.

Worked example 10 — quadratic bowl

$f(x) = \frac{1}{2} x^\top Q x$ with $Q \succ 0$: $\nabla f = Qx$, $H = Q$; GD condition number $\kappa = \lambda_{\max}/\lambda_{\min}$.

8. Multivariate Taylor

$$f(x + \Delta) = f(x) + \nabla f(x)^\top \Delta + \frac{1}{2} \Delta^\top H(x) \Delta + o(\|\Delta\|^2).$$

Newton step solves $H\Delta = -\nabla f$ using the quadratic model.

Worked example 11

Newton on strongly convex smooth f converges quadratically near the min—when H is cheap and well-conditioned enough to invert/solve.

9. Constrained gradients (preview)

On constraint $g(x) = 0$, optima satisfy $\nabla f = \lambda \nabla g$ (Lagrange)—gradients align. See Optimization KKT chapter.

Worked example 12

Maximize $x^\top a$ on $\|x\|_2 = 1$: solution $x = a/\|a\|$—Cauchy–Schwarz geometry.

10. Automatic vs symbolic vs numeric derivatives

Method	Notes
Symbolic	exact; may explode expression size
Numeric finite diff	easy; cancellation & cost $O(n)$
Forward/reverse AD	exact to FP; reverse ideal when n large, scalar loss

Worked example 13

ML frameworks implement reverse-mode AD = backprop on tensors.

11. Pitfalls

1. Shape errors in Jacobian/transpose chain rule
2. Assuming critical point is minimum without Hessian test
3. Ill-conditioned $H \Rightarrow$ slow GD / unstable Newton
4. Nondifferentiable ops (max, ReLU at 0) in "gradient"
5. Vanishing/exploding gradients in deep chains (product of Jacobians)

12. Checkpoint

- Compute gradients and Hessians for simple f
- State directional derivative max property
- Apply chain rule to logistic loss
- Classify critical points via H
- Explain reverse-mode AD motivation

Exercises

Easy

1. $\nabla(x^\top A x)$ for symmetric A (result $2Ax$).
2. Directional derivative of $f = xy$ at $(1, 2)$ along unit $u \propto (1, 1)$.
3. Jacobian of $F(x, y) = (x^2, xy)$.
4. Hessian of $\|x\|_2^2$.
5. Why is ∇f orthogonal to level sets (intuition)?

Medium

6. Derive $\nabla_w \frac{1}{2}\|Xw - y\|_2^2$.
7. Softmax cross-entropy gradient w.r.t. logits.
8. Show mixed partials equal for $f = e^{xy}$ (compute both).
9. Eigenvalues of Hessian in Example 9; classify.
10. Chain rule for $f(g(h(x)))$ scalar case expanded.

Challenge

11. Prove $D_u f = \nabla f \cdot u$ from definition of derivative of $t \mapsto f(x + tu)$.
12. Gauss–Newton Hessian approximation for nonlinear least squares.
13. Vanishing gradient: product of many $\sigma'(z) < 1/4$ factors.
14. Riemannian gradient on the sphere (project Euclidean gradient).
15. Implement (pseudocode) reverse-mode for a tiny 2-node graph.

Summary

Multivariable calculus upgrades one-dimensional derivatives to gradients, Jacobians, and Hessians. The chain rule on graphs is backpropagation; curvature matrices govern optimization difficulty. Mastering shapes, critical-point tests, and AD modes is essential ML mathematics.

Taylor Series and Approximations

Taylor series represent functions as infinite polynomials built from derivatives at a point. Finite Taylor polynomials are the local models behind Newton methods, numeric `exp`/`sin` implementations, small-angle approximations, and many asymptotic arguments in algorithms.

1. Taylor polynomials

If f is n times differentiable at a,

$$T_n(x) = \sum_{k=0}^{n} \frac{f^{(k)}(a)}{k!}(x-a)^k.$$

Remainder $R_n(x) = f(x) - T_n(x)$. Lagrange form: exists ξ between x and a with

$$R_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!}(x-a)^{n+1}.$$

Worked example 1

$f(x) = e^x$, $a = 0$: $T_2(x) = 1 + x + x^2/2$. $e^{0.1} \approx 1.105$ vs true ≈ 1.1051709.

Worked example 2

$f(x) = \sin x$, $a = 0$: $T_3(x) = x - x^3/6$. $\sin(0.2) \approx 0.198666\ldots$ vs $0.198669\ldots$.

2. Classic Maclaurin series

$$
\begin{aligned}
e^x &= \sum_{k=0}^{\infty} \frac{x^k}{k!}, \\
\sin x &= \sum_{k=0}^{\infty} \frac{(-1)^k x^{2k+1}}{(2k+1)!}, \\
\cos x &= \sum_{k=0}^{\infty} \frac{(-1)^k x^{2k}}{(2k)!}, \\
\frac{1}{1-x} &= \sum_{k=0}^{\infty} x^k \quad (|x| < 1), \\
\log(1+x) &= \sum_{k=1}^{\infty} (-1)^{k+1} \frac{x^k}{k} \quad (|x| < 1).
\end{aligned}
$$

Worked example 3

Geometric series sums finite geometric progressions in algorithm analysis ($1 + r + \cdots + r^{n-1}$).

Worked example 4

$\log(1+x) \approx x - x^2/2$ for small x—used in multiplicative weights / continuous compounding approximations.

3. Convergence radius

Power series $\sum c_k (x-a)^k$ converge for $|x-a| < R$ (radius R). Inside the disk, termwise differentiation/integration often valid.

Worked example 5

$1/(1-x)$ series fails at $x = 1$ even though function blows up there—radius limited by nearest singularity in complex plane (here $x = 1$).

4. Big-O and little-o expansions

Write $f(x) = T_n(x) + O((x-a)^{n+1})$ as $x \to a$. Asymptotic notation packages remainder control for algorithm and numeric reasoning.

Worked example 6

$\sqrt{1+x} = 1 + \frac{1}{2}x - \frac{1}{8}x^2 + O(x^3)$ as $x \to 0$.

Worked example 7 — softmax stability

$\log \sum_i e^{z_i} = m + \log \sum_i e^{z_i - m}$ with $m = \max z_i$—algebraic rewrite; Taylor not required, but local linearizations of log and exp appear in analyses.

5. Taylor in optimization

Quadratic model $f(x + \Delta) \approx f(x) + \nabla f^\top \Delta + \frac{1}{2}\Delta^\top H \Delta$ is multivariate Taylor. Newton chooses Δ to minimize this model.

Worked example 8

For 1D f, Newton $x - f'/f''$ matches root finding on f' when minimizing.

Worked example 9 — GD step sizes

Descent lemma for L-smooth f: $f(y) \le f(x) + \nabla f(x)^\top (y - x) + \frac{L}{2}\|y - x\|^2$—a globalized Taylor-with-remainder inequality.

6. Series in algorithms and discrete math

- Generating functions encode sequences; singularities → asymptotics (Flajolet–Odlyzko)
- Exponential series in continuous-time Markov chains e^{tQ}
- $\sum 1/k^2$ etc. in analysis of algorithms constants

Worked example 10

Expected coupon collector $\sim n \log n$ uses harmonic numbers $H_n = \log n + \gamma + O(1/n)$—integral/series approximation of harmonics.

7. Numerical evaluation caveats

Naive power series for large $|x|$ is unstable or slow; use range reduction ($e^x = e^{n \log 2 + r} = 2^n e^r$) then series on small r.

Worked example 11

Computing e^{-100} via series of e^x at -100 is a bad idea—overflow intermediate terms; rewrite.

Worked example 12 — cancellation

$e^x - 1$ for small x: use `expm1` or series $x + x^2/2 + \cdots$—direct subtraction cancels.

8. Fourier teaser (optional horizon)

Taylor is local polynomial; Fourier series are global trigonometric expansions on intervals—signal processing backbone. Different approximation theory (Gibbs, rates for smooth periodic f).

9. Pitfalls

1. Using Taylor far outside radius of usefulness
2. Truncating alternating series without error bound
3. Confusing formal power series with convergent functions
4. Applying geometric series at $|x| \geq 1$
5. Multivariate Taylor without controlling remainder in high d

10. Checkpoint

- Write T_n and Lagrange remainder
- Expand $e^x, \sin x, \log(1+x)$
- Use $O(\cdot)$ expansions for small x
- Connect quadratic Taylor to Newton
- Practice stable evaluation patterns for exp

Exercises

Easy

1. T_3 for cos at 0; approximate $\cos(0.1)$.
2. Geometric sum formula for $|r| < 1$ infinite case.
3. Derive $\lim_{x\to 0}(e^x - 1)/x = 1$ from series.
4. Linear approximation of $\sqrt{1+x}$ near 0.
5. Why is T_1 the tangent line?

Medium

6. Bound $|R_4|$ for e^x on $|x| \le 1$ using Lagrange remainder.
7. Expand $\log(1-x)$ for $|x| < 1$.
8. Show binomial series idea for $(1+x)^\alpha$ first three terms.
9. Relate second-order Taylor to Newton step for 1D minimization.
10. Harmonic number integral bound: $\int_1^{n+1} \frac{1}{x} \le H_n \le 1 + \int_1^n \frac{1}{x}$.

Challenge

11. Prove uniqueness of power series coefficients for analytic f.
12. Radius of convergence via ratio test for e^x coefficients.
13. Multivariate Taylor to order 2; write Hessian form.
14. Explain range reduction for sin using π periodicity and series on $[-\pi/4, \pi/4]$.
15. Generating function for Fibonacci; pole $\to$ Binet asymptotic sketch.

Summary

Taylor polynomials are controlled local models with explicit remainders; classic series for exp/trig/log fuel both analysis and numeric libraries. In CS they appear as optimization models, asymptotic expansions, and stable algorithm design for elementary functions.

Number Theory

Number Theory for Computer Science

Number theory studies the integers—and in CS it is the mathematics of modular arithmetic, primes, and discrete structure behind cryptography, hashing, coding, and competitive programming. This introduction builds the conceptual map and proves you can already use the main tools.

Why number theory is core CS math

Domain	Number-theoretic ingredient
Public-key crypto	Modular exponentiation, primes, groups $(\mathbb{Z}/n\mathbb{Z})^\times$
Hashing / PRGs	Modular reduction, linear congruential ideas
Checksums	Parity and modular checksums
Competitive programming	GCD, inverses, CRT, fast powmod
Coding theory	Finite fields (built from modular poly arithmetic)
Graphics / games	Integer lattice tricks, wraparound arithmetic

Concept map

```
flow:
  B -> C
  B -> D
  C -> E
  E -> F
  E -> G
  G -> H
  A -> I
  I -> H
```

Divisibility and gcd (preview)

$d \mid a$ means $\exists k \in \mathbb{Z},\ a = dk$.
$\gcd(a, b)$ is the largest positive common divisor.

Euclidean algorithm: $\gcd(a, b) = \gcd(b, a \bmod b)$ with $\gcd(a, 0) = |a|$. Runtime $O(\log \min(|a|, |b|))$ steps.

Worked example 1

$\gcd(1001, 143)$: $1001 = 7 \cdot 143 + 0$? $143 \cdot 7 = 1001$, so gcd $= 143$.

Worked example 2 — Bézout

$\gcd(a, b) = ax + by$ for some integers x, y (extended Euclid). Essential for modular inverses.

Modular arithmetic (preview)

$a \equiv b \pmod{n}$ means $n \mid (a - b)$. Residues $\{0, 1, \ldots, n-1\}$ form a ring.

Inverse: $a^{-1} \bmod n$ exists iff $\gcd(a, n) = 1$. Then $ax \equiv 1 \pmod{n}$ solvable.

Worked example 3

Inverse of 3 mod 10: $3 \cdot 7 = 21 \equiv 1$, so 7.
Inverse of 2 mod 10: none ($\gcd = 2 \neq 1$).

Worked example 4 — fast pow

Compute $a^e \bmod n$ by binary exponentiation in $O(\log e)$ multiplications—backbone of RSA and Diffie–Hellman operations.

Fermat and Euler (preview)

Fermat: if p prime and $p \nmid a$, then $a^{p-1} \equiv 1 \pmod{p}$.
Euler: if $\gcd(a, n) = 1$, then $a^{\varphi(n)} \equiv 1 \pmod{n}$.

Worked example 5

Inverse of a mod prime p is $a^{p-2} \bmod p$ (Fermat)—when p huge, use powmod carefully.

Chinese Remainder Theorem (preview)

If $n_1, \ldots, n_k$ pairwise coprime, system $x \equiv a_i \pmod{n_i}$ has unique solution mod $N = \prod n_i$.

Worked example 6

$x \equiv 2 \pmod{5}$, $x \equiv 3 \pmod{7}$: try $x = 17$ ($17 = 3 \cdot 5 + 2$, $17 = 2 \cdot 7 + 3$). Mod 35, unique class.

RSA sketch (motivation)

Pick primes p, q, $n = pq$, $\varphi = (p-1)(q-1)$, choose e coprime to φ, $d \equiv e^{-1} \pmod{\varphi}$.
Encrypt $c \equiv m^e \pmod{n}$; decrypt $m \equiv c^d \pmod{n}$ (for m in range, with padding in practice).

Security intuition: factoring n (or equivalent hardness) should be hard—**not** a theorem that $\mathbf{P} \neq \mathbf{NP}$, but a concrete computational assumption.

Worked example 7 — tiny RSA

$p = 3, q = 11, n = 33, \varphi = 20, e = 3, d = 7$ because $21 \equiv 1 \pmod{20}$. Message $m = 4$: $c = 4^3 = 64 \equiv 31 \pmod{33}$; 31^7 mod 33 recovers 4 (compute carefully).

Primes and primality

Infinitely many primes (Euclid). Density $\sim 1/\ln n$ (prime number theorem). Primality is in $\mathbf{P}$ (AKS); practice uses Miller–Rabin probabilistic tests.

Worked example 8

Trial division to $\sqrt{n}$ is $O(\sqrt{n})$—fine for 64-bit sometimes, hopeless for 2048-bit RSA moduli.

Hashing connection

Map keys into $\{0, \ldots, m-1\}$ by $h(k) = k \bmod m$ or multiplicative methods. Universality needs number-theoretic or combinatorial designs—mod alone is not always enough for adversarial keys.

Worked example 9

Bad m and structured keys cause collisions; cryptographic hashes use different designs but still modular arithmetic internally in many constructions.

Proof styles to practice

1. **Constructive:** exhibit Bezout coefficients
2. **Contradiction:** Euclid's infinitude of primes
3. **Induction:** modular power laws
4. **Equivalence:** $a \equiv b \pmod{m} \Leftrightarrow m \mid (a-b)$

Worked example 10

Prove: if $d \mid a$ and $d \mid b$ then $d \mid (ax + by)$ for all integers x, y—one line from definitions.

Pitfalls

1. Division mod n without invertibility
2. Reducing exponent incorrectly (use $\varphi(n)$ only when coprime)
3. Overflow in intermediate products before mod
4. Using Fermat test alone as primality proof (Carmichael numbers)
5. Toy RSA without padding as "secure encryption"

Roadmap of this part

1. Divisibility and GCD
2. Modular arithmetic
3. Primes and crypto

Checkpoint

- Run Euclid and extended Euclid mentally on small inputs
- Solve $ax \equiv 1 \pmod{n}$ when possible
- State Fermat and when it applies
- Solve a 2-modulus CRT system
- Explain RSA's (n, e, d) roles without claiming security proofs

Exercises

1. Prove: if $d \mid a$ and $d \mid b$ then $d \mid (ax + by)$ for all $x, y \in \mathbb{Z}$.
2. Compute $\gcd(1001, 143)$ and Bézout coefficients.
3. Solve $x \equiv 2 \pmod{5}$, $x \equiv 3 \pmod{7}$.
4. When is modular division by b valid mod m?

5. Compute $3^{100} \bmod 7$ using Fermat.
6. Find inverse of 5 mod 17 two ways (extended Euclid; Fermat).
7. Show there are infinitely many primes (Euclid outline).
8. Why is $a^{n-1} \equiv 1 \pmod{n}$ for all a coprime to n not true for all composite n with a cheap test alone? (Carmichael mention)
9. Implement (pseudocode) binary modular exponentiation; count multiplications for e with k bits.
10. CRT for three moduli $3, 4, 5$ with remainders $2, 3, 1$.
11. Tiny RSA: encrypt and decrypt one message with $n = 55$, choose valid e, d.
12. Hashing: give an example where $k \bmod m$ collides heavily for arithmetic progressions of keys.
13. Prove $\gcd(a, b) = \gcd(a, b - a)$ and connect to Euclid.
14. Explain why RSA decryption works using Euler's theorem (coprime m case).
15. Checkpoint essay: list three production systems using modular arithmetic and what fails if inverses are missing.

Summary

Number theory supplies the integer toolkit for secure and efficient computation: gcd, modular inverses, exponentiation, CRT, and primes. The following chapters develop each pillar with algorithms and cryptographic constructions.

Divisibility, GCD, and the Euclidean Algorithm

1. Division Algorithm

For integers `a` and positive integer `b`, there exist unique integers `q, r` such that:

`a = bq + r`, with `0 <= r < b`.

This is the foundation of modular arithmetic and Euclid's algorithm.

Worked Example

For `a = 252, b = 17`:

`252 = 17*14 + 14`.

So quotient `q=14`, remainder `r=14`.

2. Divisibility and Basic Laws

`d | n` means `n = dk` for some integer `k`.

Useful rules: - If `d|a` and `d|b`, then `d|(a+b)` and `d|(a-b)`. - If `d|a`, then `d|ax` for any integer `x`. - If `d|a` and `a|b`, then `d|b`.

3. Greatest Common Divisor

`gcd(a,b)` is the greatest positive integer dividing both.

Theorem (Euclidean Step)

`gcd(a,b) = gcd(b, a mod b)`.

Proof Sketch

Let `a = bq + r`. A number `d` divides both `a` and `b` iff it divides `r = a - bq`. So common divisors of `(a,b)` and `(b,r)` are identical. Hence gcd is identical.

4. Euclidean Algorithm

Algorithm: 1. Replace `(a,b)` with `(b, a mod b)` repeatedly. 2. Stop when second value is zero. 3. First value is gcd.

Example

`gcd(252,105)`: - `252 = 2*105 + 42` - `105 = 2*42 + 21` - `42 = 2*21 + 0` Answer: `21`.

Complexity

Euclid runs in `O(log min(a,b))` arithmetic steps.

5. Bézout Identity and Extended Euclid

Theorem (Bézout)

There exist integers `x,y` such that:

`ax + by = gcd(a,b)`.

Why It Matters

- Modular inverse computation
- Solving linear Diophantine equations
- RSA key generation step

Extended Euclid Example

Find inverse of `17 mod 43`.

Compute: - `43 = 2*17 + 9` - `17 = 1*9 + 8` - `9 = 1*8 + 1` - `8 = 8*1 + 0`

Back-substitute: - `1 = 9 - 1*8` - `8 = 17 - 1*9` - `1 = 2*9 - 17` - `9 = 43 - 2*17` - `1 = 2*43 - 5*17`

So `-5*17  1 (mod 43)`, inverse is `38`.

6. Linear Diophantine Equation

Equation: `ax + by = c`.

Theorem

A solution exists iff `gcd(a,b) | c`.

Example

Solve `14x + 21y = 35`. `gcd(14,21)=7`, and `7|35`, so solutions exist. One solution from simplification: `2x + 3y = 5` gives `x=1, y=1`. General solution obtained by parameterization.

7. Python Reference Implementations

```
def gcd(a, b):
    while b:
        a, b = b, a % b
    return abs(a)

def egcd(a, b):
    if b == 0:
        return a, 1, 0
    g, x1, y1 = egcd(b, a % b)
    return g, y1, x1 - (a // b) * y1
```

Exercises

1. Compute `gcd(987654, 123456)` via Euclid.
2. Find integers `x,y` such that `391x + 299y = gcd(391,299)`.
3. Solve `35x + 50y = 10`.
4. Prove that consecutive Fibonacci numbers are coprime.
5. Show that if `gcd(a,b)=1`, then `gcd(a, b+ka)=1`.

Summary

Euclid's algorithm plus Bézout identity gives a complete computational engine for gcd, inverses, and integer equations.

Modular Arithmetic and Congruences

1. Congruence Relation

`a  b (mod m)` means `m | (a-b)`.

It partitions integers into residue classes.

Example for `mod 5`: - Class 0: `..., -10, -5, 0, 5, 10, ...` - Class 1: `..., -9, -4, 1, 6, 11, ...`

2. Arithmetic Laws

If `a  b (mod m)` and `c  d (mod m)`: - `a+c  b+d (mod m)` - `a-c  b-d (mod m)` - `ac  bd (mod m)`

Division needs caution: valid only if divisor has inverse modulo `m`.

3. Modular Inverse

`a^{-1} mod m` exists iff `gcd(a,m)=1`.

When inverse exists: `a * a^{-1}  1 (mod m)`.

Example

Inverse of `7 mod 26`. Extended Euclid gives `15*7 - 4*26 = 1`, so inverse is `15`.

4. Fast Exponentiation

Need to compute `a^e mod m` efficiently.

Binary method complexity: `O(log e)` multiplications.

```
def mod_pow(a, e, m):
    a %= m
    ans = 1
    while e > 0:
        if e & 1:
```

```
            ans = (ans * a) % m
        a = (a * a) % m
        e >>= 1
    return ans
```

5. Fermat and Euler Theorems

Fermat's Little Theorem

If `p` is prime and `p !| a`, then:

`a^(p-1)  1 (mod p)`.

Corollary: `a^(p-2)  a^{-1} (mod p)`.

Euler's Theorem

If `gcd(a,n)=1`, then:

`a^{phi(n)}  1 (mod n)`.

6. Chinese Remainder Theorem (CRT)

Theorem

If moduli `m1,...,mk` are pairwise coprime, then system `x  ai (mod mi)` has unique solution modulo `M = product(mi)`.

Worked Example

Solve: `- x  2 (mod 3) - x  3 (mod 5) - x  2 (mod 7)`

Unique solution: `x  23 (mod 105)`.

7. Application Patterns in CS

- Ring arithmetic for rolling hash
- Circular buffers (`index mod capacity`)
- Time wheels/schedulers
- Finite field foundations in crypto and coding

Exercises

1. Compute `3^123 mod 17`.
2. Find inverse of `19 mod 41`.
3. Solve with CRT: `x  1 (mod 4), x  2 (mod 9), x  5 (mod 7)`.
4. Prove: if `a  b (mod m)` then `a^k  b^k (mod m)` for `k>=1`.
5. Show a counterexample where modular division fails.

Summary

Modular arithmetic lets you perform large integer computation safely and quickly. Together with Euclid and CRT, it forms a core algorithmic toolkit.

Prime Numbers and Cryptography Basics

1. Prime Fundamentals

Prime number: integer `p > 1` with no divisors except `1` and `p`.

Composite numbers factor into primes uniquely.

Theorem (Fundamental Theorem of Arithmetic)

Every integer `n > 1` can be expressed as a product of primes, uniquely up to ordering.

Proof Idea

- Existence by induction.
- Uniqueness via Euclid's lemma: if prime `p` divides `ab`, then `p|a` or `p|b`.

2. Prime Generation

Sieve of Eratosthenes

Compute all primes `<= n` in `O(n log log n)`.

```
def sieve(n):
    is_prime = [True] * (n + 1)
    is_prime[0] = is_prime[1] = False
    p = 2
    while p * p <= n:
        if is_prime[p]:
            for x in range(p*p, n+1, p):
                is_prime[x] = False
        p += 1
    return [i for i, v in enumerate(is_prime) if v]
```

3. Primality Testing

- Trial division: simple, slow for large `n`
- Probabilistic tests (Miller-Rabin): practical and fast
- Deterministic variants for bounded ranges

4. Euler Totient Function

`phi(n)` = count of integers in `[1..n]` coprime to `n`.

Useful formulas: - If `p` prime: `phi(p)=p-1` - If `p,q` distinct primes: `phi(pq)=(p-1)(q-1)`

5. RSA Cryptosystem (Sketch)

Key Generation

1. Pick large primes `p, q`
2. Compute `n = pq`
3. Compute `phi(n) = (p-1)(q-1)`
4. Choose `e` with `gcd(e,phi(n))=1`
5. Compute `d  e^{-1} (mod phi(n))`

Public key: `(n,e)` Private key: `(n,d)`

Encryption/Decryption

- Encrypt: `c = m^e mod n`
- Decrypt: `m = c^d mod n`

Why It Works

From Euler/Fermat structure of modular exponentiation under coprimality assumptions.

6. Security Intuition

RSA security relies on hardness of factoring large semiprimes (`n = pq`). Given only `n`, deriving `phi(n)` (and thus `d`) is hard in classical computation.

7. Tiny RSA Example (Pedagogical)

Choose `p=11, q=17.` - `n=187` - `phi(n)=160` - choose `e=7` (`gcd(7,160)=1`) - inverse `d=23` since `7*23=161  1 (mod 160)`

Message `m=88`: - Encrypt: `c = 88^7 mod 187 = 11` - Decrypt: `11^23 mod 187 = 88`

Exercises

1. Use sieve to list primes up to 200.
2. Factor `13860` into primes.
3. Compute `phi(84)`.
4. Build toy RSA using `p=13, q=19`, choose valid `e`, compute `d`.
5. Explain why tiny RSA is insecure even if mathematically correct.

Extended practice: Chinese Remainder Theorem (CRT)

If $n = n_1 n_2$ with $\gcd(n_1, n_2) = 1$, and you know $x \bmod n_1$ and $x \bmod n_2$, you can recover $x \bmod n$.

Toy: $x \equiv 2 \pmod 3$, $x \equiv 3 \pmod 5$. Then $x \equiv 8 \pmod{15}$ (check: $8 = 2 + 3 \cdot 2$, $8 = 3 + 5 \cdot 1$).

CRT speeds RSA private ops when p and q are known (compute mod p and mod q, recombine). Same math as "two congruences $\rightarrow$ one modulus."

Discrete log (why Diffie–Hellman is hard)

Given prime p, generator g, and $y = g^a \bmod p$, recovering a is the **discrete log** problem. Easy direction: exponentiate. Hard direction: invert. DH key exchange relies on that asymmetry—not on secrecy of p or g.

Map to CS

Idea	Where it shows up
GCD / Euclid	Fraction reduce, calendar math, RSA setup
Modular inverse	RSA d, affine ciphers, some hash constructions
$\varphi(n)$	Order of multiplicative group mod n
Primality	Key generation; Miller–Rabin in libraries
CRT	Fast RSA, calendar/concurrent scheduling puzzles

Summary

Primes, totients, modular inverses, and fast exponentiation combine into real cryptographic systems. This chapter links pure number theory to practical security engineering. For algorithms that *use* number theory at scale, continue to **Algorithms and Complexity** and **Complexity Theory** parts later in this book.

Lattices and Learning With Errors: the Mathematics of ML-KEM

The previous chapter built public-key cryptography on two problems: factoring and discrete logarithms. Both fall to Shor's algorithm on a large quantum computer. The replacement standardised by NIST in 2024, **ML-KEM** (FIPS 203, formerly CRYSTALS-Kyber), rests on a different problem, **Learning With Errors (LWE)**. It is easy to state: solve a system of linear equations modulo q in which every equation is off by a small random amount. Without the "off by a small amount", Gaussian elimination solves it in a fraction of a second. With it, the best known classical and quantum algorithms take time exponential in the dimension.

This chapter shows that gap with running code, builds Regev's original LWE encryption scheme for one bit, measures its **noise budget**, and then introduces the polynomial ring $\mathbb{Z}_q[x]/(x^n+1)$ that turns the toy into something with ML-KEM's shape. All listings are complete Python files using only the standard library. They were run with **Python 3.13.5**, and the outputs are pasted unedited. The parameters are toys. **Never** use these listings to protect anything.

1. Lattices in one paragraph

A **lattice** is the set of all integer combinations of n linearly independent vectors $b_1, \ldots, b_n \in \mathbb{R}^n$:

$$\mathcal{L}(B) = \{\, Bz : z \in \mathbb{Z}^n \,\}.$$

It is a grid, but usually a skewed one, and the same lattice has infinitely many bases. With a short, nearly orthogonal basis, finding the lattice point closest to a target is easy: round the coordinates (Babai). With a long, skewed basis of the same lattice, nobody knows how to do this efficiently in high dimension. That is the **Closest Vector Problem**. Its approximate versions (and the related Shortest Vector Problem) are the hard problems underneath everything below. Regev proved in 2005 that solving LWE on average is at least as hard as solving approximate worst-case lattice problems, using a quantum reduction. That theorem is the reason LWE is trusted.

2. The problem: linear algebra plus a little noise

Fix a prime modulus q, a dimension n, and a secret $s \in \mathbb{Z}_q^n$. An LWE sample is a pair

$$(a,\ b = \langle a, s\rangle + e \bmod q), \qquad a \text{ uniform in } \mathbb{Z}_q^n, \quad e \text{ small}.$$

Given many samples, written as $b = As + e$, find s. Drop e and this is a linear system:

```
# noiseless.py - without noise, "LWE" is just linear algebra mod q
import random
q, n = 97, 4
random.seed(1)
s = [random.randrange(q) for _ in range(n)]                    # the secret
A = [[random.randrange(q) for _ in range(n)] for _ in range(n)]
b = [sum(a * x for a, x in zip(row, s)) % q for row in A]      # b = A s  (no error)

def solve_mod(A, b, q):                                        # Gauss-Jordan over Z_q (q p
    M = [row[:] + [bi] for row, bi in zip(A, b)]
    for c in range(n):
        p = next(r for r in range(c, n) if M[r][c] % q)
        M[c], M[p] = M[p], M[c]
        inv = pow(M[c][c], -1, q)
        M[c] = [v * inv % q for v in M[c]]
        for r in range(n):
            if r != c and M[r][c]:
                f = M[r][c]; M[r] = [(v - f * w) % q for v, w in zip(M[r], M[c])]
    return [M[r][n] for r in range(n)]

print("secret   :", s)
print("recovered:", solve_mod(A, b, q))
e = [random.choice([-1, 0, 1]) for _ in range(n)]
b_noisy = [(bi + ei) % q for bi, ei in zip(b, e)]
print("with error e =", e, "-> 'recovered':", solve_mod(A, b_noisy, q))
```

```
$ python3 noiseless.py
secret   : [17, 72, 8, 32]
recovered: [17, 72, 8, 32]
with error e = [0, 1, -1, 1] -> 'recovered': [78, 67, 58, 23]
```

Elimination recovers the secret exactly, and then a perturbation of ± 1 in three of the four equations produces garbage. The algorithm does not degrade gracefully. Each row operation multiplies the error by field elements as large as q, so after elimination the noise is spread uniformly over $\mathbb{Z}_q$. Geometrically, b is a point *near* the lattice $\{Az \bmod q\}$ and recovering s means finding the closest lattice point, which is CVP again.

3. Regev encryption of one bit

The public key is a batch of LWE samples $(A, b = As + e)$ with $A \in \mathbb{Z}_q^{m \times n}$. To encrypt a bit, add a random subset of the samples and hide the bit in the most significant position:

$$u = r^{\mathsf{T}} A, \qquad v = r^{\mathsf{T}} b + \text{bit} \cdot \lfloor q/2 \rfloor, \qquad r \in \{0,1\}^m.$$

The secret-key holder computes

$$v - \langle u, s \rangle = r^{\mathsf{T}}(As + e) - r^{\mathsf{T}}As + \text{bit} \cdot \lfloor q/2 \rfloor = \text{bit} \cdot \lfloor q/2 \rfloor + \underbrace{r^{\mathsf{T}}e}_{\text{small}} .$$

The result is near 0 for a 0 and near $q/2$ for a 1. Decrypt by asking which of the two it is closer to.

```
# regev.py - encrypt one bit with Learning With Errors (Regev 2005), toy parameters
import random
random.seed(7)
n, m, q = 32, 64, 3329            # dimension, samples, modulus (q borrowed from ML-KEM)
def small(): return random.choice([-2, -1, 0, 1, 2])          # "small" error/secret

s = [small() for _ in range(n)]
A = [[random.randrange(q) for _ in range(n)] for _ in range(m)]
e = [small() for _ in range(m)]
b = [(sum(a * x for a, x in zip(A[i], s)) + e[i]) % q for i in range(m)]   # public key (A,

def encrypt(bit):
    r = [random.randint(0, 1) for _ in range(m)]                # random subset of samples
    u = [sum(r[i] * A[i][j] for i in range(m)) % q for j in range(n)]
    v = (sum(r[i] * b[i] for i in range(m)) + bit * (q // 2)) % q
    return u, v

def decrypt(u, v):
    d = (v - sum(x * y for x, y in zip(u, s))) % q              # = bit*q/2 + sum(r_i e_i)
    centred = d - q if d > q // 2 else d
    return int(abs(centred) > q // 4), centred

for bit in (0, 1, 1, 0):
    u, v = encrypt(bit)
    out, noise = decrypt(u, v)
    print(f"bit={bit} decrypted={out} (v - <u,s> centred = {noise:+d}, threshold ±{q//4})")
```

```
$ python3 regev.py
bit=0 decrypted=0 (v - <u,s> centred = +2, threshold ±832)
bit=1 decrypted=1 (v - <u,s> centred = -1661, threshold ±832)
bit=1 decrypted=1 (v - <u,s> centred = +1658, threshold ±832)
bit=0 decrypted=0 (v - <u,s> centred = +9, threshold ±832)
```

The values $+2$ and $+9$ are the accumulated noise $r^{\mathsf{T}}e$. The values ± 1658 are $\lfloor q/2 \rfloor = 1664$ plus noise, seen from the "centred" representative in $(-q/2, q/2]$. The decision threshold is $q/4 = 832$, so this ciphertext has hundreds of units of headroom. Anyone without s sees (u, v), which under the LWE assumption is indistinguishable from uniform random.

4. The noise budget

Decryption is correct exactly when $|r^{\mathsf{T}}e| < q/4$. That is a statement about a *sum of random variables*, so we can predict when it fails. Take e_i uniform on $\{-E, \dots, E\}$ (variance $E(E+1)/3$) and r_i uniform on $\{0, 1\}$. Then

$$\mathrm{Var}(r^{\mathsf{T}}e) = \frac{m}{2} \cdot \frac{E(E+1)}{3},$$

and by the central limit theorem $r^{\mathsf{T}}e$ is close to normal.

```
# failure.py - the noise budget: grow the error and decryption starts to fail
import random
def trial(err, n=32, m=64, q=3329, runs=2000, seed=0):
    rnd = random.Random(seed)
    s = [rnd.randint(-2, 2) for _ in range(n)]
    A = [[rnd.randrange(q) for _ in range(n)] for _ in range(m)]
    e = [rnd.randint(-err, err) for _ in range(m)]
    b = [(sum(a*x for a, x in zip(A[i], s)) + e[i]) % q for i in range(m)]
    fails = 0
    for k in range(runs):
        bit = k & 1
        r = [rnd.randint(0, 1) for _ in range(m)]
        u = [sum(r[i]*A[i][j] for i in range(m)) % q for j in range(n)]
        v = (sum(r[i]*b[i] for i in range(m)) + bit*(q//2)) % q
        d = (v - sum(x*y for x, y in zip(u, s))) % q
        fails += (int(q//4 < d < 3*q//4) != bit)
    return fails / runs
for err in (2, 20, 40, 80, 160):
    print(f"|e_i| <= {err:3d}: decryption failure rate {trial(err):.4f}")
```

```
$ python3 failure.py
|e_i| <=   2: decryption failure rate 0.0000
|e_i| <=  20: decryption failure rate 0.0000
|e_i| <=  40: decryption failure rate 0.0000
|e_i| <=  80: decryption failure rate 0.0000
|e_i| <= 160: decryption failure rate 0.1145
```

Check the last row against the formula. With $m = 64$ and $E = 160$ the standard deviation is $\sqrt{32 \cdot 160 \cdot 161/3} \approx 524$. The threshold 832 is 1.59 standard deviations, and $P(|Z| > 1.59) \approx 0.112$ for a standard normal Z. The measured rate is 0.1145.

This is the central tension in every LWE design. **More noise** makes the underlying lattice problem harder. **Less noise** keeps decryption correct. Real schemes choose q, n and the error distribution so that the failure probability is cryptographically negligible. FIPS 203 reports roughly 2^{-139} to 2^{-174} for ML-KEM's three parameter sets, while the lattice problem stays out of reach of known attacks.

5. From vectors to polynomials: the ring $\mathbb{Z}_q[x]/(x^n+1)$

Plain LWE has big keys: the matrix A has mn entries. Structured variants replace the random matrix by multiplication in a polynomial ring

$$R_q = \mathbb{Z}_q[x]/(x^n+1), \qquad n \text{ a power of two.}$$

In R_q, $x^n = -1$, so multiplying by x shifts the coefficients and *negates* whatever wraps around. Multiplication by a fixed polynomial is therefore a structured ("negacyclic") $n \times n$ matrix, described by only n numbers:

```
# negacyclic.py - the ring Z_q[x]/(x^n + 1) that ML-KEM works in: x^n wraps to -1
def mul(a, b, q):
    n = len(a); c = [0] * n
    for i in range(n):
        for j in range(n):
            k, sign = (i + j) % n, (-1 if i + j >= n else 1)
            c[k] = (c[k] + sign * a[i] * b[j]) % q
    return c
q = 3329
x = [0, 1, 0, 0]                        # the polynomial x, with n = 4
p = [1, 0, 0, 0]
for k in range(1, 9):
    p = mul(p, x, q)
    print(f"x^{k} =", [c if c <= q // 2 else c - q for c in p])
```

```
$ python3 negacyclic.py
x^1 = [0, 1, 0, 0]
x^2 = [0, 0, 1, 0]
x^3 = [0, 0, 0, 1]
x^4 = [-1, 0, 0, 0]
x^5 = [0, -1, 0, 0]
x^6 = [0, 0, -1, 0]
x^7 = [0, 0, 0, -1]
x^8 = [1, 0, 0, 0]
```

The powers of x have order $2n$: $x^4 = -1$ and $x^8 = 1$. For $n = 256$ and $q = 3329$, the factor $q-1 = 3328 = 2^8 \cdot 13$ contains a primitive 256-th root of unity ($\zeta = 17$) but no 512-th root. So $x^{256}+1$ splits modulo q into 128 quadratic factors $x^2 - \zeta^{2i+1}$, not into linear ones. ML-KEM multiplies ring elements with this "incomplete" number-theoretic transform, the modular analogue of the FFT. It costs $O(n \log n)$ operations instead of $O(n^2)$, plus a cheap product of degree-1 polynomials in each of the 128 slots. That arithmetic is the reason $q = 3329$ was chosen.

ML-KEM works with *modules* over this ring: vectors of k ring elements. Its secret is $\mathbf{s} \in R_q^k$ with small coefficients, its public key is $(\mathbf{A}, \mathbf{t} = \mathbf{A}\mathbf{s} + \mathbf{e})$ with $\mathbf{A} \in R_q^{k \times k}$ expanded from a 32-byte seed, and encryption is the module version of Section 3. FIPS 203 fixes:

Parameter set	n	q	k	η_1	η_2	(d_u, d_v)	NIST category
ML-KEM-512	256	3329	2	3	2	(10, 4)	1
ML-KEM-768	256	3329	3	2	2	(10, 4)	3
ML-KEM-1024	256	3329	4	2	2	(11, 5)	5

Here η sets the width of the centred binomial error distribution, and (d_u, d_v) are the numbers of bits kept when ciphertext coefficients are rounded. The rounding adds deterministic "noise" and shrinks ciphertexts, and the failure analysis must absorb it. The KEM itself wraps this public-key encryption in a Fujisaki–Okamoto transform to obtain security against chosen-ciphertext attacks. That transform is a protocol argument, not new mathematics, and is out of scope here.

6. What the hardness claim is, and is not

- LWE hardness is a **conjecture backed by a reduction**: average-case LWE is at least as hard as worst-case approximate lattice problems. No proof shows those lattice problems are hard, just as no proof shows factoring is hard.
- The best known attacks are lattice reduction (BKZ and its sieving subroutines). Their cost grows exponentially in the dimension, and parameter sets are chosen from concrete cost estimates for these algorithms.
- Ring and module structure is extra structure that an attacker might exploit. Decades of cryptanalysis have not found a general way to do so for these parameters, and that is a judgement call the community keeps re-examining. Public debates in 2026 about lattices versus hash-based signatures come down to how much margin that structure leaves. Each claim needs to be checked against the published analyses.

Exercises

1. In `noiseless.py`, replace the error with e such that $\sum |e_i| = 1$ (one equation off by one). How many of the recovered coordinates are still right? Explain using the row operations.
2. Derive the variance formula in Section 4 and use it to predict the smallest E with a failure rate above 1%. Check with `failure.py`.
3. Modify `regev.py` to encrypt two bits per ciphertext by using the quarter points $0, q/4, q/2, 3q/4$. What does the noise budget become?
4. Show that multiplication by a fixed $a(x) \in R_q$ is the linear map with matrix $M_{ij} = \pm a_{(i-j) \bmod n}$, and determine the sign rule.
5. Check with `pow` that $17^{128} \equiv -1$ and $17^{256} \equiv 1 \pmod{3329}$. Explain why this means 17 has order exactly 256, and why no element can have order 512.

Sources

- NIST FIPS 203, *Module-Lattice-Based Key-Encapsulation Mechanism Standard* (August 2024): https://csrc.nist.gov/pubs/fips/203/final
- O. Regev, "On Lattices, Learning with Errors, Random Linear Codes, and Cryptography", *J. ACM* 56(6), 2009 (STOC 2005): https://doi.org/10.1145/1568318.1568324
- C. Peikert, "A Decade of Lattice Cryptography", *Foundations and Trends in TCS*, 2016: https://eprint.iacr.org/2015/939
- V. Lyubashevsky, C. Peikert, O. Regev, "On Ideal Lattices and Learning with Errors over Rings", EUROCRYPT 2010: https://eprint.iacr.org/2012/230

AlphaProof Nexus: Lean-Verified Erdős Problems

AlphaProof Nexus: Lean-Verified Erdős Problems

Paul Erdős left hundreds of open questions — many elementary to *state*, stubborn to *settle*. In October 2026, Tsoukalas et al. reported in *Science* that an agent framework called **AlphaProof Nexus** autonomously resolved **9 of 353** Lean-formalized open Erdős problems (and 44 of 492 OEIS conjectures), with proofs checked by the **Lean 4** kernel rather than by informal review alone. This chapter is the mathematical reading of that result: what "formal proof search" means, why Lean's `sorry` is the right hole to fill, what kinds of Erdős statements yielded, and how to look at a public Lean artifact without treating the AI as an oracle.

Learning goals

By the end of this chapter you can:

- Contrast informal LLM proofs with Lean-checked proofs
- Outline the AlphaProof Nexus loop: sketch → LLM edit → Lean feedback → (optional) evolution / AlphaProof tool
- State the headline Erdős evaluation numbers from the *Science* / arXiv paper
- Sketch the *shape* of one solved problem (density / construction) without claiming a hand proof
- Know where the Lean sources live and what human validation still covers

Thesis

An LLM can write a wrong proof that looks right. A **proof assistant** accepts a proof only when every tactic closes goals under a fixed logical foundation. AlphaProof Nexus's contribution is not "AI vibes about number theory" — it is **search in the space of Lean scripts**, grounded by compiler errors, at research-problem scale.

Mental model — formal vs informal

```
Informal path                          Formal path (AlphaProof Nexus)
-------------                          ------------------------------
LLM writes English proof               Start from Lean theorem with `sorry`
  → human must re-check every step       → agent edits only marked regions
                                         → Lean compiles; errors steer next edit
                                         → success   sorry-free + type-correct
```

Lean 4 (Moura & Ullrich) treats definitions and proofs as code. The special tactic `sorry` closes goals unsoundly for scaffolding; a finished proof must not rely on `sorry` (or `sorryAx`). Mathlib supplies the library of already-checked lemmas the agent may import.

Agent shapes (from the paper)

Agent	Idea
Basic	Independent "Ralph loops": multi-turn LLM edits + Lean feedback each episode
+ AlphaProof	Same, but can call AlphaProof (RL olympiad prover) on stubborn subgoals
Evolutionary	Shared population of sketches; Elo-style LLM ratings; P-UCB sampling
Full-featured	Evolution + AlphaProof tool — used for the large Erdős / OEIS sweeps

Post-hoc analysis: the **basic** agent could also solve the same nine Erdős problems, often at higher cost on the hardest ones; the full agent helped more on problems such as #125 and #138.

Worked examples

Case 1: Headline evaluation

From *Advancing Mathematics Research with AI-Driven Formal Proof Search* (Science 394, 8 Oct 2026; arXiv:2605.22763):

Corpus	Attempted	Solved by full agent
Lean Erdős formalizations (Formal Conjectures snapshot)	353	**9**
Autoformalized OEIS open conjectures	492	**44**

Additional deployments claimed in the paper (outside the Erdős table): algebraic geometry (Hilbert / pure O-sequences), optimization (anchored GDA rate), graph theory collaborations, quantum optics constructions. After each Erdős solve, the authors report **expert checks that the Lean statement matched the intended conjecture** — formal success does not auto-fix misformalization.

Public artifacts: google-deepmind/alphaproof-nexus-results (Lean under `APNOutputs/ErdosProblems/`, prose under `NaturalLanguageProofs/`).

Case 2: What an Erdős solve looks like (problem #125, density)

Informal statement (shape). Let A be the integers whose base-3 digits are only $0, 1$, and B the analogous set in base 4. Does the sumset $A + B$ have positive **lower** density?

Result reported. The agent proved that the **lower density is zero**, using an inductive thinning argument that exploits Diophantine proximity $3^m \approx 4^k$ (irrational $\log 4/\log 3$). Human discussion afterward noted remaining questions about **upper** density.

You do not need the full Lean file to see the pattern: many Erdős wins are **constructions** or **density upper bounds** that decompose into lemmas Mathlib can check, not deep new axiomatic foundations.

Case 3: Construction flavour (problem #12(i), 1970)

Erdős–Sárközy-type question: infinite $A \subseteq \mathbb{Z}^+$ with a divisibility restriction (no a divides $b + c$ for distinct larger b, c) yet

$$\liminf_{N\to\infty} \frac{|A \cap [1, N]|}{\sqrt{N}} > 0.$$

The published deformalized proof builds A as a union of **blocks** separated by rapid growth, using the Chinese Remainder Theorem and a 3-AP-free digit map (base-2 $\to$ base-3) so same-block violations collapse to forbidden arithmetic progressions. Two of the nine solves had been open on the order of **56 years**.

Case 4: Opening a Lean artifact

A typical successful file (conceptually):

```
-- illustrative shape - see alphaproof-nexus-results for real files
import Mathlib

namespace Erdos125
-- definitions of A, B, lower density …
theorem lower_density_zero : … := by
  -- EVOLVE-BLOCK region: lemmas + tactics, no sorry left
  …
end Erdos125
```

Reading tips:

1. Check imports and the **theorem statement** against ErdosProblems.com / Formal Conjectures — that is where misformalization hides.
2. Search for `sorry` — should be absent in `APNOutputs` successes.
3. Prefer the paired **natural-language** write-up when learning the idea; use Lean when you need machine certainty.

The trap

- Treating a Lean proof of a **wrong formalization** as a proof of Erdős's intended question (the paper itself found density-vs-lower/upper-density ambiguities).
- Assuming every open Erdős problem is now in reach — **9/353** with a large search budget; most remain open.
- Confusing **AlphaProof** (olympiad RL prover used as a tool) with **AlphaProof Nexus** (the agent framework around Lean + LLMs).
- Citing X/news summaries instead of the Science paper, arXiv text, or the GitHub Lean sources.

The boring rule

For research claims that matter: **formalize the statement carefully**, search with a Lean-checked loop, then have a human confirm the formal statement matches the mathematics you care about.

Try this

1. Read the Science abstract / arXiv §§1–3 (https://arxiv.org/html/2605.22763 or https://www.science.org/doi/10.1126/science.aej2213).
2. Browse one Lean file under `APNOutputs/ErdosProblems/` and its prose twin if present.
3. On ErdosProblems.com, open problem #125 or #12 and compare the informal wording to the Lean theorem type.
4. If you use Lean 4 locally, `#check` a Mathlib density or sumset lemma to feel the library the agent stands on (no new installs required for reading proofs on GitHub).

Sources

- G. Tsoukalas et al., *Advancing mathematics research with AI-driven formal proof search*, Science (8 Oct 2026). DOI: 10.1126/science.aej2213
- arXiv:2605.22763 — https://arxiv.org/abs/2605.22763 / https://arxiv.org/html/2605.22763
- Results repo: https://github.com/google-deepmind/alphaproof-nexus-results
- Lean 4: Moura & Ullrich, CADE 2021; Mathlib community
- Erdős catalog: https://www.erdosproblems.com

Reed–Solomon Erasure Coding over GF(2^8): Why Object Stores Beat Replication

Keeping three copies of every object is the simplest way not to lose data, and at exabyte scale it is ruinously expensive: two of every three disks hold nothing new. The Apache Hadoop documentation puts it plainly: "the default 3x replication scheme in HDFS has 200% overhead in storage space", while "in typical Erasure Coding (EC) setups, the storage overhead is no more than 50%". Backblaze stores each file as 17 data pieces plus 3 parity pieces, and "the original file can then be reconstructed from any 17 of the 20 pieces". That costs 18% extra instead of 200%, and still survives three failures.

The trick is **Reed–Solomon coding**. It is linear algebra, with one twist: the arithmetic happens in a field with exactly 256 elements, so that every byte is a number you can multiply and, more importantly, divide. This chapter builds that field from polynomials modulo 2, finds every modulus that works, encodes and rebuilds real bytes, checks **every** possible set of surviving shards, and then shows a coding matrix that looks right and silently fails for 12 of the 1,001 ways to lose 4 shards out of 14. All listings are complete Python files using only the standard library. They were run with **Python 3.13.5**, and the outputs are pasted unedited.

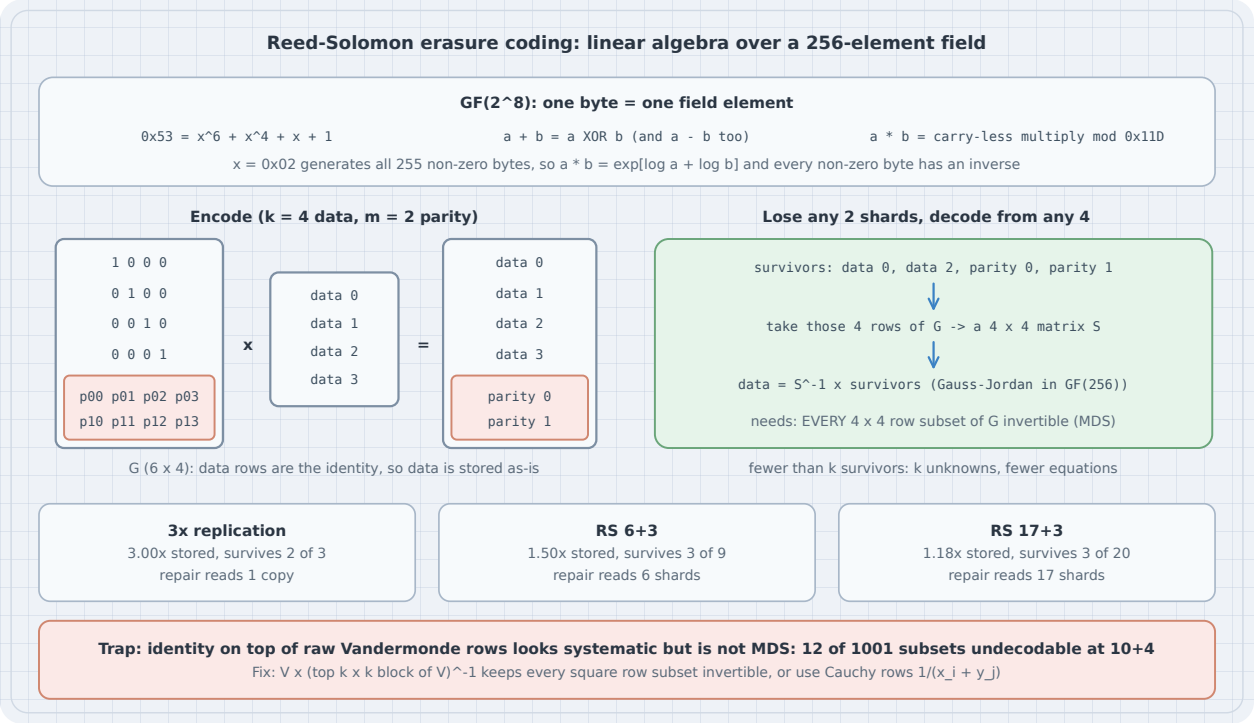


Figure 1: Reed–Solomon erasure coding over GF(2^8): byte arithmetic, encoding as a matrix product, decoding from any k survivors, the storage trade-off, and the trap

1. The problem as linear algebra

Cut a stripe of data into k equal **data shards** $d_0, \dots, d_{k-1}$ and look at one byte position across all of them: a vector $d \in F^k$. Store $n = k + m$ shards whose bytes at that position are

$$c = G\,d, \qquad G \in F^{n \times k}.$$

If the top k rows of G are the identity, the first k stored shards *are* the data (the code is **systematic**), and the other m rows compute **parity**. After failures, some set R of k shards survives. Their bytes are $c_R = G_R\,d$, where G_R is the $k \times k$ matrix made of the surviving rows, so

$$d = G_R^{-1}\,c_R.$$

Decoding works for *every* loss of up to m shards exactly when **every** $k \times k$ row subset of G is invertible. Such a code is called **maximum distance separable (MDS)**. It meets the Singleton bound: no code that stores n shards for k shards of data can survive more than $n - k$ arbitrary losses, because with fewer than k shards left you have k unknowns and fewer equations.

Two questions remain. Which number system F makes "invert" possible for bytes? And how do you build a G with the MDS property?

2. Bytes need a field, and $\mathbb{Z}/256$ is not one

Gaussian elimination divides. Ordinary arithmetic modulo 256 cannot divide by any even number: $2 \cdot 128 \equiv 0$, so 2 has no inverse. Arithmetic modulo a prime p is a field, but 257 values do not fit in a byte. The answer is **GF(2^8)**: read a byte $b_7 \dots b_0$ as the polynomial $b_7x^7 + \cdots + b_0$ with coefficients mod 2, add polynomials (XOR), and multiply them modulo an **irreducible** degree-8 polynomial. This is the same construction as $\mathbb{Z}/p$: a polynomial that cannot be factored plays the part of a prime. This chapter uses $P(x) = x^8 + x^4 + x^3 + x^2 + 1$ (`0x11D`), the modulus in Backblaze's open-source Reed–Solomon library (`GENERATING_POLYNOMIAL = 29`, the low byte of `0x11D`).

```
# gf256.py: the field GF(2^8) that Reed-Solomon storage codes compute in
# A byte b7..b0 is the polynomial b7*x^7 + ... + b0 with coefficients mod 2.
# Products are reduced modulo P(x) = x^8 + x^4 + x^3 + x^2 + 1 (0x11D).
POLY = 0x11D

def mul_slow(a, b, poly=POLY):
    """Carry-less multiply with reduction: the definition, one bit at a time."""
    r = 0
    while b:
        if b & 1:
            r ^= a          # adding polynomials over GF(2) is XOR
        b >>= 1
        a <<= 1
        if a & 0x100:       # degree reached 8: subtract (XOR) the modulus
```

```
            a ^= poly
    return r

# x (the byte 0x02) generates every non-zero element, so log/exp tables replace multiplicati
EXP = [0] * 510
LOG = [0] * 256
v = 1
for i in range(255):
    EXP[i] = v
    LOG[v] = i
    v = mul_slow(v, 2)
for i in range(255, 510):
    EXP[i] = EXP[i - 255]   # doubled table: no "mod 255" needed in mul

def add(a, b):
    return a ^ b            # also subtraction: every element is its own negative

def mul(a, b):
    if a == 0 or b == 0:
        return 0
    return EXP[LOG[a] + LOG[b]]

def inv(a):
    if a == 0:
        raise ZeroDivisionError("0 has no inverse in GF(256)")
    return EXP[255 - LOG[a]]

def div(a, b):
    return mul(a, inv(b))

def power(a, n):
    if n == 0:
        return 1
    if a == 0:
        return 0
    return EXP[(LOG[a] * n) % 255]


if __name__ == "__main__":
    print("order of x = 0x02:", next(i for i in range(1, 256) if EXP[i] == 1))
    print("tables agree with bit-by-bit multiply on all 65,536 pairs:",
          all(mul(a, b) == mul_slow(a, b) for a in range(256) for b in range(256)))
    print("every non-zero byte has an inverse:", all(mul(a, inv(a)) == 1 for a in range(1, 2
    print("0x53 * 0xCA =", hex(mul(0x53, 0xCA)), "| 0x53 + 0xCA =", hex(add(0x53, 0xCA)))
    print("inverse of 0x53 =", hex(inv(0x53)))
    print("same product with AES's modulus 0x11B:", hex(mul_slow(0x53, 0xCA, 0x11B)))
```

```
    # Why not ordinary arithmetic mod 256? Z/256 is a ring, not a field.
    from math import gcd
    no_inv = [a for a in range(1, 256) if gcd(a, 256) != 1]
    print("non-zero bytes with no inverse mod 256:", len(no_inv), "(every even byte)")
    print("in Z/256, 2 * 128 =", (2 * 128) % 256, "-> two non-zero numbers multiply to zero"
```

```
python gf256.py
```

Output:

```
order of x = 0x02: 255
tables agree with bit-by-bit multiply on all 65,536 pairs: True
every non-zero byte has an inverse: True
0x53 * 0xCA = 0x8f | 0x53 + 0xCA = 0x99
inverse of 0x53 = 0x8c
same product with AES's modulus 0x11B: 0x1
non-zero bytes with no inverse mod 256: 127 (every even byte)
in Z/256, 2 * 128 = 0 -> two non-zero numbers multiply to zero
```

The tables are not an approximation. They agree with the bit-by-bit definition on all 65,536 products. Two details matter in practice:

- **Addition is XOR, and so is subtraction.** In characteristic 2 every element is its own negative, so all the minus signs in Gaussian elimination disappear.
- **The modulus changes the arithmetic.** The same bytes `0x53 * 0xCA` give `0x8f` here and `0x01` under AES's modulus `0x11B` (FIPS 197): in AES's field they are inverses, in this one they are not. Two implementations that disagree on $P(x)$ will each "work" and produce incompatible parity. A storage format must fix its field.

3. Which moduli work?

A modulus gives a field if it is irreducible. For log/exp tables you want more: the element x (`0x02`) should **generate** all 255 non-zero elements, which makes the polynomial **primitive**. There are only 256 monic degree-8 candidates, so check them all:

```
# polys.py: which degree-8 moduli give a field, and which make x a generator?
from gf256 import mul_slow

def is_irreducible(p):
    """p has degree 8; it is irreducible iff no polynomial of degree 1..4 divides it."""
    def mod(a, b):
        db = b.bit_length()
        while a.bit_length() >= db:
            a ^= b << (a.bit_length() - db)
        return a
```

```
    return all(mod(p, d) != 0 for d in range(2, 32))   # all divisors of degree 1..4

def order_of_x(p):
    v, n = 2, 1
    while v != 1:
        v, n = mul_slow(v, 2, p), n + 1
        if n > 255:
            return None          # x is not invertible: p is reducible
    return n

irreducible = [p for p in range(0x100, 0x200) if is_irreducible(p)]
primitive = [p for p in irreducible if order_of_x(p) == 255]
print("monic degree-8 polynomials over GF(2):", 256)
print("irreducible (each gives a field with 256 elements):", len(irreducible))
print("primitive (x itself generates all 255 non-zero elements):", len(primitive))
print("primitive, low byte only:", [p & 0xFF for p in primitive])
print("AES modulus 0x11B irreducible:", is_irreducible(0x11B), "| order of x:", order_of_x(0
print("0x11D: irreducible:", is_irreducible(0x11D), "| order of x:", order_of_x(0x11D))
```

```
python polys.py
```

Output:

```
monic degree-8 polynomials over GF(2): 256
irreducible (each gives a field with 256 elements): 30
primitive (x itself generates all 255 non-zero elements): 16
primitive, low byte only: [29, 43, 45, 77, 95, 99, 101, 105, 113, 135, 141, 169, 195, 207, 2
AES modulus 0x11B irreducible: True | order of x: 51
0x11D: irreducible: True | order of x: 255
```

Thirty moduli give a field. Sixteen of them make x a generator, which matches the count $\varphi(255)/8 = 128/8 = 16$ of primitive polynomials of degree 8. The list of sixteen low bytes is exactly the list in the comment above `GENERATING_POLYNOMIAL` in Backblaze's `Galois.java` ("29, 43, 45, 77, …, 245"). AES's `0x11B` is irreducible, so it gives a perfectly good field, but it is *not* primitive: x has order 51 there. Log/exp tables for AES's field therefore use another generator, such as `0x03` (order 255 under `0x11B`, checked on this box). Every one of the thirty choices gives the same field up to relabelling; what differs is only which byte stands for which element.

4. Encoding is a matrix product

Matrices over GF(256) need only the four operations above. `invert` is ordinary Gauss–Jordan elimination with the field's `add`, `mul` and `inv`. The module also contains three ways to build a systematic $n \times k$ coding matrix, compared in the next section.

```
# gfmatrix.py: matrices over GF(256), and three ways to build a k-of-n coding matrix
from gf256 import add, mul, inv, power

def matmul(A, B):
    out = []
    for row in A:
        r = []
        for j in range(len(B[0])):
            s = 0
            for t, a in enumerate(row):
                s = add(s, mul(a, B[t][j]))
            r.append(s)
        out.append(r)
    return out

def invert(M):
    """Gauss-Jordan over GF(256). Returns None if M is singular."""
    n = len(M)
    A = [list(row) + [int(i == j) for j in range(n)] for i, row in enumerate(M)]
    for col in range(n):
        piv = next((r for r in range(col, n) if A[r][col]), None)
        if piv is None:
            return None
        A[col], A[piv] = A[piv], A[col]
        f = inv(A[col][col])
        A[col] = [mul(f, x) for x in A[col]]
        for r in range(n):
            if r != col and A[r][col]:
                g = A[r][col]
                A[r] = [add(x, mul(g, y)) for x, y in zip(A[r], A[col])]
    return [row[n:] for row in A]

def vandermonde(n, k, points=None):
    points = list(range(n)) if points is None else points
    return [[power(x, j) for j in range(k)] for x in points]

def systematic_vandermonde(k, m):
    """Backblaze's construction: V (n x k, distinct points 0..n-1) times inverse of its top
    V = vandermonde(k + m, k)
    return matmul(V, invert(V[:k]))

def naive_identity_plus_vandermonde(k, m):
    """Identity on top, raw Vandermonde rows j^i (j = 1..k) underneath: looks right, is not
    I = [[int(i == j) for j in range(k)] for i in range(k)]
    return I + [[power(j, i) for j in range(1, k + 1)] for i in range(m)]
```

```
def cauchy_systematic(k, m):
    """Identity on top, Cauchy rows 1/(x_i + y_j) underneath, with x's and y's all distinct.
    xs, ys = list(range(k, k + m)), list(range(k))
    I = [[int(i == j) for j in range(k)] for i in range(k)]
    return I + [[inv(add(x, y)) for y in ys] for x in xs]
```

The construction used for real data is Backblaze's. Start from a **Vandermonde** matrix V with rows $(1, x_r, x_r^2, \ldots, x_r^{k-1})$ at n distinct points $x_r = 0, 1, \ldots, n-1$. Any k of its rows form a square Vandermonde matrix with distinct points, whose determinant $\prod_{i<j}(x_j - x_i)$ is non-zero in any field. Then multiply on the right by the inverse of the top $k \times k$ block. That turns the top into the identity, and because right-multiplying by an invertible matrix scales every $k \times k$ minor by the same non-zero determinant, it keeps every square row subset invertible. In Backblaze's words from the source: "This will make the top square be the identity matrix, but preserve the property that any square subset of rows is invertible."

```
# rs.py: encode a file into k data + m parity shards, lose any m, rebuild the original bytes
import random
from gf256 import add, mul
from gfmatrix import invert, systematic_vandermonde

def encode(data: bytes, k: int, m: int):
    size = -(-len(data) // k)                       # shard length, rounded up
    padded = data + bytes(size * k - len(data))
    shards = [list(padded[i * size:(i + 1) * size]) for i in range(k)]
    G = systematic_vandermonde(k, m)
    parity = []
    for row in G[k:]:                               # data rows are the identity: copy, don'
        p = [0] * size
        for coef, shard in zip(row, shards):
            if coef:
                p = [add(x, mul(coef, b)) for x, b in zip(p, shard)]
        parity.append(p)
    return [bytes(s) for s in shards + parity], len(data)

def decode(surviving: dict, k: int, m: int, length: int) -> bytes:
    if len(surviving) < k:
        raise ValueError(f"need {k} shards, have {len(surviving)}: {k} unknowns per byte col
                         f"only {len(surviving)} equations")
    G = systematic_vandermonde(k, m)
    rows = sorted(surviving)[:k]                    # any k survivors will do
    D = invert([G[r] for r in rows])                # k x k decoding matrix
    size = len(surviving[rows[0]])
    data = []
    for drow in D:
        out = [0] * size
        for coef, r in zip(drow, rows):
            if coef:
```

```
                out = [add(x, mul(coef, b)) for x, b in zip(out, surviving[r])]
        data.append(bytes(out))
    return b"".join(data)[:length]

if __name__ == "__main__":
    k, m = 6, 3
    msg = b"Erasure coding stores a stripe once, plus parity. Any 6 of these 9 shards rebuil
    shards, n = encode(msg, k, m)
    stored = sum(map(len, shards))
    print(f"{len(msg)} bytes (padded to {k * len(shards[0])}) -> {k}+{m} shards of {len(shar
          f"stored {stored} bytes = {stored / (k * len(shards[0])):.2f}x the padded data")
    print("data shard 0 is the plain bytes:", shards[0])
    print("parity shard 6:", shards[6].hex())
    random.seed(7)
    trials = 0
    for lost in [(6, 7, 8), (0, 1, 2), (0, 4, 8)] + [tuple(random.sample(range(9), 3)) for _
        surv = {i: s for i, s in enumerate(shards) if i not in lost}
        assert decode(surv, k, m, n) == msg, lost
        trials += 1
    print(f"rebuilt the exact message after {trials} different 3-shard losses")
    surv = {i: s for i, s in enumerate(shards) if i not in (0, 1, 2, 3)}
    try:
        decode(surv, k, m, n)
    except ValueError as e:
        print("losing 4 shards:", e)
```

```
python rs.py
```

Output:

```
85 bytes (padded to 90) -> 6+3 shards of 15 bytes, stored 135 bytes = 1.50x the padded data
data shard 0 is the plain bytes: b'Erasure coding '
parity shard 6: 9a0e0842c172c3a8e5bcb09ec10d5f
rebuilt the exact message after 53 different 3-shard losses
losing 4 shards: need 6 shards, have 5: 6 unknowns per byte column, only 5 equations
```

Data shard 0 is the original bytes, so reads that find all data shards healthy never touch the parity. Any three losses were repaired, including losing three data shards (`(0, 1, 2)`), where the data is recovered entirely from parity. The fourth loss is not a bug in the decoder. Six unknowns per byte column with five equations has 256 solutions per column, and no algorithm can pick the right one.

5. Check every survivor set, not a few

Random erasure tests like the 53 above are reassuring but prove nothing about the other subsets. For storage-sized parameters the number of subsets is small enough to check them all. The script below does that for three constructions, all of which have the identity on top:

```
# mds_check.py: is EVERY choice of k surviving shards decodable? Check all of them.
from itertools import combinations
from math import comb
from gfmatrix import invert, systematic_vandermonde, naive_identity_plus_vandermonde, cauchy

def singular_subsets(G, k):
    return [rows for rows in combinations(range(len(G)), k) if invert([G[r] for r in rows])

for k, m in [(4, 2), (6, 3), (10, 4), (17, 3)]:
    print(f"k={k}, m={m}: {comb(k + m, k)} ways to keep {k} of {k + m} shards")
    for name, build in [("systematic Vandermonde (V * top^-1)", systematic_vandermonde),
                        ("identity + raw Vandermonde rows   ", naive_identity_plus_vandermon
                        ("identity + Cauchy rows            ", cauchy_systematic)]:
        G = build(k, m)
        assert G[:k] == [[int(i == j) for j in range(k)] for i in range(k)], "not systematic
        bad = singular_subsets(G, k)
        print(f"   {name}  undecodable subsets: {len(bad)}" + (f"  e.g. keep {bad[0]}" if ba
```

```
python mds_check.py
```

Output:

```
k=4, m=2: 15 ways to keep 4 of 6 shards
   systematic Vandermonde (V * top^-1)  undecodable subsets: 0
   identity + raw Vandermonde rows     undecodable subsets: 0
   identity + Cauchy rows              undecodable subsets: 0
k=6, m=3: 84 ways to keep 6 of 9 shards
   systematic Vandermonde (V * top^-1)  undecodable subsets: 0
   identity + raw Vandermonde rows     undecodable subsets: 0
   identity + Cauchy rows              undecodable subsets: 0
k=10, m=4: 1001 ways to keep 10 of 14 shards
   systematic Vandermonde (V * top^-1)  undecodable subsets: 0
   identity + raw Vandermonde rows     undecodable subsets: 12  e.g. keep (0, 1, 2, 3, 6, 7,
   identity + Cauchy rows              undecodable subsets: 0
k=17, m=3: 1140 ways to keep 17 of 20 shards
   systematic Vandermonde (V * top^-1)  undecodable subsets: 0
   identity + raw Vandermonde rows     undecodable subsets: 0
   identity + Cauchy rows              undecodable subsets: 0
```

The Backblaze construction and the Cauchy construction pass every subset at every size. (A Cauchy matrix $1/(x_i + y_j)$ with all x's and y's distinct has every square submatrix invertible, so stacking it under the identity is MDS by construction.) The middle construction, which simply stacks the identity on top of raw Vandermonde rows j^i, passes at 4+2, 6+3 and 17+3, and fails at 10+4: there are **12** ways to lose four shards that it cannot recover from.

6. The trap: why identity-plus-Vandermonde breaks

The failure has a precise cause. Take the first bad subset found above: data shards 4, 5 and 9 are lost, and parity rows 0, 2 and 3 survive. The identity rows of the survivors pin down the other seven data bytes, so decoding comes down to a 3×3 system in the three lost data values. Its matrix has entries x^e for the lost points $x \in \{5, 6, 10\}$ (data shard j sits at the point $j + 1$) and the surviving powers $e \in \{0, 2, 3\}$. Because the power 1 is missing, this is a *generalized* Vandermonde matrix, and its determinant is the ordinary Vandermonde product times a Schur polynomial. For the exponents $\{0, 2, 3\}$ that polynomial is $e_2 = x_1x_2 + x_1x_3 + x_2x_3$:

```
# why_naive_fails.py: the undecodable subset from mds_check.py, reduced to a 3 x 3 determina
from gf256 import add, mul, power

# keep (0,1,2,3,6,7,8,10,12,13) of k=10, m=4: data shards 4, 5, 9 are lost, parity rows 0, 2
lost_points = [5, 6, 10]          # data shard j is evaluated at the point j + 1
powers = [0, 2, 3]                # parity row i holds the powers j^i

M = [[power(x, e) for x in lost_points] for e in powers]
def det3(A):
    a, b, c = A[0]; d, e, f = A[1]; g, h, i = A[2]
    t1 = mul(a, add(mul(e, i), mul(f, h)))
    t2 = mul(b, add(mul(d, i), mul(f, g)))
    t3 = mul(c, add(mul(d, h), mul(e, g)))
    return add(add(t1, t2), t3)   # signs vanish: -1 = +1 in characteristic 2
print("the 3 x 3 block the decoder must invert:", [[hex(v) for v in row] for row in M])
print("its determinant:", det3(M))

x1, x2, x3 = lost_points
vandermonde_part = mul(mul(add(x1, x2), add(x1, x3)), add(x2, x3))
e2 = add(add(mul(x1, x2), mul(x1, x3)), mul(x2, x3))
print("Vandermonde factor (x1+x2)(x1+x3)(x2+x3):", vandermonde_part, "(non-zero: points are
print("missing-power factor e2 = x1x2 + x1x3 + x2x3:", e2)
```

```
python why_naive_fails.py
```

Output:

```
the 3 x 3 block the decoder must invert: [['0x1', '0x1', '0x1'], ['0x11', '0x14', '0x44'], [
its determinant: 0
Vandermonde factor (x1+x2)(x1+x3)(x2+x3): 204 (non-zero: points are distinct)
missing-power factor e2 = x1x2 + x1x3 + x2x3: 0
```

The Vandermonde factor is non-zero because the points are distinct. The extra factor e_2 happens to be zero in GF(256), so the block is singular and those four losses are unrecoverable. The construction is not MDS because "identity rows plus Vandermonde rows" is not a Vandermonde matrix. Only the rows of V itself have the any-k-rows property. This exact pitfall made it into print: Plank's widely used 1997 *Software—Practice & Experience* tutorial presented an information dispersal matrix that,

as Plank and Ding's 2003 correction puts it, "does not have the properties claimed – that the deletion of any m rows results in an invertable n*n matrix" (their n is this chapter's k). The correction derives the coding matrix from a Vandermonde matrix by elementary column operations. That is the same idea as multiplying by the inverse of the top block.

Two habits prevent this class of bug. Derive G from a construction with a proof (Vandermonde times the inverse of its top block, or Cauchy). Then, for the parameters you actually ship, verify *all* $\binom{n}{k}$ subsets once, as `mds_check.py` does. At 4+2, 6+3 and 17+3 the naive matrix happens to be fine. At 10+4 a random test has to hit one of 12 bad subsets out of 1,001 (about 1.2%) to notice anything.

7. Replication versus Reed–Solomon, in numbers

Three numbers decide the trade-off: raw bytes stored per usable byte, how many simultaneous losses a stripe survives, and how much must be read to rebuild one lost shard. The listing below also computes the probability of losing a stripe when each shard independently fails with probability p before it can be repaired. That is a deliberately simple model, labelled as such: real failures are correlated, which is why shards are spread across failure domains (disks, hosts, racks, zones).

```
# durability.py: replication vs Reed-Solomon on the three numbers an operator cares about
from math import comb

def p_loss(n, k, p):
    """Probability that more than n - k of n shards fail, each independently with probabilit
    return sum(comb(n, j) * p**j * (1 - p) ** (n - j) for j in range(n - k + 1, n + 1))

schemes = [("2x replication", 2, 1), ("3x replication", 3, 1), ("RS 6+3 (HDFS RS-6-3)", 9, 6
           ("RS 10+4 (HDFS RS-10-4)", 14, 10), ("RS 17+3 (Backblaze Vault)", 20, 17)]
print(f"{'scheme':27s} {'raw/usable':>10s} {'overhead':>9s} {'survives':>9s} {'repair reads'
      f" {'P(loss) p=1e-2':>15s} {'p=1e-3':>9s}")
for name, n, k in schemes:
    print(f"{name:27s} {n / k:10.2f} {100 * (n - k) / k:8.0f}% {n - k:>3d} of {n:<3d} {k:>6d
          f" {p_loss(n, k, 1e-2):15.1e} {p_loss(n, k, 1e-3):9.1e}")
```

```
python durability.py
```

Output:

```
scheme                      raw/usable  overhead  survives  repair reads  P(loss) p=1e-2
2x replication                    2.00      100%   1 of 2        1 shards         1.0e-04
3x replication                    3.00      200%   2 of 3        1 shards         1.0e-06
RS 6+3 (HDFS RS-6-3)              1.50       50%   3 of 9        6 shards         1.2e-06
RS 10+4 (HDFS RS-10-4)            1.40       40%   4 of 14      10 shards         1.9e-07
RS 17+3 (Backblaze Vault)         1.18       18%   3 of 20      17 shards         4.3e-05
```

Read the table one column at a time:

- **Storage.** RS 10+4 survives *four* losses while storing 1.40× the data, against 3.00× for triple replication, which survives two. This is the whole economic argument.
- **Loss probability depends on p.** At $p = 10^{-2}$, RS 6+3 is slightly *worse* than 3× replication (1.2×10^{-6} vs 1.0×10^{-6}), because nine shards give more chances for four failures. At $p = 10^{-3}$ it is nearly eight times better. Fast repair (small p per repair window) is what makes wide codes safe. RS 17+3 trades durability for overhead: at 1.18× it is the cheapest by far, and in this model its loss probability is higher than RS 6+3's. Backblaze accepts that trade: a Vault "is comprised of 20 Storage Pods, with the data evenly spread across all 20 pods", and each file is stored as those 20 shards.
- **Repair is the hidden cost.** Rebuilding one lost replica reads one copy. Rebuilding one lost RS shard reads k shards (6, 10 or 17) across the network. This is why Windows Azure Storage introduced **Local Reconstruction Codes** (Huang et al., USENIX ATC 2012): their LRC (12, 2, 2) splits 12 data fragments into two local groups of six, each with its own parity, plus two global parities. Most single failures are then repaired from six fragments instead of twelve, at 1.33× overhead.

The HDFS built-in erasure-coding policies sit at the conservative end of the table: `RS-3-2-1024k`, `RS-6-3-1024k`, `RS-10-4-1024k`, the older `RS-LEGACY-6-3-1024k` and `XOR-2-1-1024k`. The names encode the codec, the data and parity counts, and the striping cell size.

Exercises

1. Change `POLY` in `gf256.py` to `0x11B` (AES). Which check in the `__main__` block fails first, and why? Fix it by building the tables with generator `0x03`, and confirm that the tables then agree with `mul_slow(a, b, 0x11B)` on all 65,536 pairs.
2. Using `mds_check.py`, find the smallest (k, m) for which `naive_identity_plus_vandermonde` is not MDS. Is there a pattern in which subsets fail?
3. Prove that if every $k \times k$ row subset of V is invertible and T is any invertible $k \times k$ matrix, then every $k \times k$ row subset of VT is invertible. (Hint: $\det\left((VT)_R\right) = \det(V_R)\det(T)$.)
4. With `p_loss`, find the per-shard failure probability p at which RS 6+3 and 3× replication have equal stripe-loss probability. What does that tell you about the repair-time budget an RS 6+3 cluster needs?
5. In GF(256), how many points are available for a Vandermonde matrix with distinct points? What is the largest $n = k + m$ this gives, and does it match Backblaze's limit of "no more than 256 shards total"?
6. A Reed–Solomon *error*-correcting code (unknown corrupted positions, as on a CD or in a QR code) can correct only $\lfloor m/2 \rfloor$ errors with m parity symbols, while erasure coding recovers m losses. Explain the factor of two using the counting argument from Section 1.

Sources

- I. S. Reed and G. Solomon, "Polynomial Codes Over Certain Finite Fields", *Journal of the Society for Industrial and Applied Mathematics* 8(2), 300–304, 1960.
- J. S. Plank, "A Tutorial on Reed–Solomon Coding for Fault-Tolerance in RAID-like Systems", *Software—Practice & Experience* 27(9), 995–1012, 1997; and J. S. Plank and Y. Ding, "Note:

Correction to the 1997 Tutorial on Reed–Solomon Coding", *Software—Practice & Experience* 35(2), 189–194, 2005 (technical report UT-CS-03-504, 2003): https://web.eecs.utk.edu/~jplank/plank/papers/CS-03-504.html.

- Backblaze, "Backblaze Open-sources Reed-Solomon Erasure Coding Source Code" (B. Beach, 2015): https://www.backblaze.com/blog/reed-solomon/, and the library source, `Galois.java` and `ReedSolomon.java` (`buildMatrix`, `vandermonde`): https://github.com/Backblaze/JavaReedSolomon.
- Backblaze, "Backblaze Vaults: Zettabyte-Scale Cloud Storage Architecture" (B. Beach, 2015): https://www.backblaze.com/blog/vault-cloud-storage-architecture/.
- Apache Hadoop, *HDFS Erasure Coding* (purpose, overhead figures, built-in policies): https://hadoop.apache.org/docs/stable/hadoop-project-dist/hadoop-hdfs/HDFSErasureCoding.html.
- C. Huang, H. Simitci, Y. Xu, A. Ogus, B. Calder, P. Gopalan, J. Li and S. Yekhanin, "Erasure Coding in Windows Azure Storage", USENIX ATC 2012.
- NIST FIPS 197, *Advanced Encryption Standard (AES)* (the field GF(2^8) with modulus `0x11B`): https://csrc.nist.gov/pubs/fips/197/final.
- R. Lidl and H. Niederreiter, *Finite Fields*, Cambridge University Press (irreducible and primitive polynomials, counting formulas).
- I. G. Macdonald, *Symmetric Functions and Hall Polynomials*, 2nd ed., Oxford University Press, 1995 (generalized Vandermonde determinants as Schur polynomials times the Vandermonde product).

The Chinese Remainder Theorem

The Chinese Remainder Theorem

Thesis

The **Chinese Remainder Theorem (CRT)** says that if moduli $m_1, \dots, m_k$ are pairwise coprime, the system $x \equiv a_i \pmod{m_i}$ has a unique solution modulo $M = m_1 \cdots m_k$. It is the algebraic engine behind secret sharing fragments, some RSA optimisations, and calendar problems.

Mental model

```
x  a1 (mod m1)
x  a2 (mod m2)
…
Mi = M / mi,  yi = Mi^{-1} mod mi
x = Σ ai Mi yi  (mod M)
```

Pairwise coprime each M_i is invertible mod m_i.

Worked example

Solve $x \equiv 2 \pmod 3$, $x \equiv 3 \pmod 5$, $x \equiv 2 \pmod 7$.

```
# crt.py
def egcd(a, b):
    if b == 0:
        return a, 1, 0
    g, x, y = egcd(b, a % b)
    return g, y, x - (a // b) * y

def inv(a, m):
    g, x, _ = egcd(a, m)
    assert g == 1
    return x % m

def crt(rems, mods):
    M = 1
    for m in mods:
        M *= m
```

```
    x = 0
    for a, m in zip(rems, mods):
        Mi = M // m
        x += a * Mi * inv(Mi, m)
    return x % M, M

x, M = crt([2, 3, 2], [3, 5, 7])
print(f"x={x} mod {M}; checks", x % 3, x % 5, x % 7)
```

```
x=23 mod 105; checks 2 3 2
```

So $x = 23$ works: $23 = 7 \cdot 3 + 2$, $23 = 4 \cdot 5 + 3$, $23 = 3 \cdot 7 + 2$, and solutions repeat every 105.

Trap

Applying the formula when moduli share a factor. If $\gcd(m_i, m_j) \neq 1$, a solution may not exist (inconsistent remainders) or uniqueness fails. Always check coprimality or use the general simultaneous-congruence algorithm.

Boring rule

For pairwise-coprime moduli, compute M, each M_i, each inverse, sum, reduce mod M. Verify by evaluating x mod m_i.

Try this

1. Change the third remainder to 3 and watch the assertions fail (or detect inconsistency).
2. Use CRT to merge two digits of a product in a modular multiplication pipeline.
3. Prove uniqueness: if x, y both work, M divides $x - y$.

Sources

- Ireland & Rosen, *A Classical Introduction to Modern Number Theory* (CRT chapter)
- Any abstract algebra text: $\mathbb{Z}/M\mathbb{Z} \cong \prod \mathbb{Z}/m_i\mathbb{Z}$

Information Theory

Information Theory for Computer Science

Information theory quantifies **uncertainty**, **information**, and **communication limits**. Claude Shannon's 1948 framework answers questions that still drive systems design: How many bits do you need to store a source? How fast can you send data reliably over a noisy channel? How much does one random variable tell you about another?

This section is mathematical, but every definition maps to engineering practice: compression codecs, error-correcting codes, ML loss functions, feature selection, and privacy leakage analysis.

Chapters in this part

Chapter	Focus
Entropy and information	Self-information, $H(X)$
Mutual information and capacity	$I(X;Y)$, channels
KL divergence and coding	Cross-entropy, coding length, ML loss
Applications	Compression, trees, features, perplexity

What "information" means here

Everyday language treats "information" as content or meaning. Shannon information is **not** about semantics. It is about **surprise under a probability model**.

- A fair coin flip carries 1 bit of self-information per outcome.
- A nearly deterministic event carries almost 0 bits: you already expected it.
- A rare event carries many bits: learning that it occurred is highly informative.

Formally, if an event A has probability $p(A) > 0$, its **self-information** (in bits) is

$$I(A) = -\log_2 p(A).$$

Entropy, mutual information, and channel capacity are built from this atomic idea by averaging, conditioning, and optimizing.

The three core problems

Shannon's theory organizes around three canonical problems.

1. Source coding (compression)

Given a random source X producing symbols with distribution p_X, how many bits per symbol are needed on average for lossless encoding?

Answer (source coding theorem, informal): You need about $H(X)$ bits per symbol, where $H(X)$ is the Shannon entropy. You cannot do better (in the large-block limit), and good codes approach $H(X)$.

2. Channel coding (reliable communication)

Given a noisy channel $p(y \mid x)$, what is the highest rate (bits per channel use) at which you can communicate with vanishing error probability?

Answer (channel coding theorem, informal): That rate is the **channel capacity**

$$C = \max_{p(x)} I(X;Y),$$

the maximum mutual information over input distributions.

3. Statistical dependence

How much does observing Y reduce uncertainty about X?

Answer: Mutual information $I(X;Y) = H(X) - H(X \mid Y)$. It is zero if and only if X and Y are independent (for discrete variables under standard technical conditions).

Why CS and ML practitioners need this

Area	Information-theoretic role
Compression	Huffman, arithmetic coding, Lempel–Ziv; rate–distortion for lossy codecs
Networking	Capacity limits, ACK/NACK efficiency, FEC vs retransmission tradeoffs
Storage	RAID/erasure codes; entropy of data for backup sizing
ML training	Cross-entropy loss = coding cost under model q for data from p
Representation learning	MI bounds, InfoNCE, variational information bottlenecks
Feature selection	Rank features by $I(X_j;Y)$ or conditional MI
Privacy	Mutual information as a leakage metric between secrets and releases
Security	Entropy of keys; min-entropy for unpredictability

Worked intuition: cross-entropy as a loss

Suppose labels follow p, and a model predicts q. The average number of bits to encode true labels using code tailored to q is the **cross-entropy** $H(p, q)$. Training often minimizes $H(p, q)$ (or an empirical version). The excess cost over the ideal $H(p)$ is the **KL divergence** $D_{\mathrm{KL}}(p\|q)$:

$$H(p, q) = H(p) + D_{\mathrm{KL}}(p\|q).$$

So "fitting the distribution" is literally "learning a good code for the data."

Mathematical prerequisites

You should be comfortable with:

- Discrete probability: joint, marginal, and conditional distributions
- Logarithms (especially base 2 for bits; natural log for nats)
- Expectation as a weighted sum
- Basic inequalities (Jensen) for convexity arguments
- Optional later: continuous densities for differential entropy caveats

Notation used throughout:

- log means $\log_2$ unless stated otherwise (bits)
- $H(X)$ entropy; $H(X \mid Y)$ conditional entropy
- $I(X; Y)$ mutual information
- $D_{\mathrm{KL}}(p\|q)$ Kullback–Leibler divergence

Roadmap of this part

1. **Entropy and information content** — self-information, Shannon entropy, joint/conditional entropy, cross-entropy, KL divergence, information gain, compression links.
2. **Mutual information and capacity** — MI definitions and properties, data processing inequality, channels, BSC capacity, applications in ML and systems.

Planned extensions (not required for the core path): rate–distortion theory, explicit code constructions (Huffman, LDPC/turbo intuition), and information bottleneck methods in deep learning.

A first calculation you should master

Example. Binary source with $P(X = 1) = p$, $P(X = 0) = 1 - p$. Entropy:

$$H_2(p) = -p \log_2 p - (1 - p) \log_2(1 - p)$$

(with the convention $0 \log 0 = 0$).

- $H_2(0.5) = 1$ bit (maximum uncertainty)
- $H_2(0.1) \approx 0.469$ bits (biased source is more compressible)
- $H_2(0) = H_2(1) = 0$ (deterministic)

Checkpoint: Why does a highly biased source compress better? Because typical sequences have fewer "surprising" patterns; good codes assign short codewords to common symbols.

Units: bits, nats, and dits

Entropy depends on the log base:

- base $2 \to$ **bits** (standard in CS)
- base $e \to$ **nats** (common in analysis and ML papers)
- base $10 \to$ **dits/hartleys** (rare)

Conversion: $H_{\text{bits}} = H_{\text{nats}} / \ln 2$. Always check units when reading formulas that omit the base.

Pitfalls (read before the next chapter)

1. **Entropy is not "amount of meaning."** High-entropy noise is not "informative" in the human sense.
2. $0 \log 0$**:** Define as 0 by continuity; never leave raw $\log 0$ in code.
3. **Differential entropy** of continuous r.v.s can be negative and is **not** a pure "information content" analogue of discrete entropy.
4. **KL is not a metric:** $D_{\text{KL}}(p\|q) \neq D_{\text{KL}}(q\|p)$ in general; triangle inequality fails.
5. **Empirical entropy** from finite samples underestimates true entropy without care (especially for large alphabets).

How to study these chapters

For each definition:

1. State it formally.
2. Compute it on a 2-outcome and a 4-outcome toy distribution.
3. Connect it to one CS artifact (codec, loss, feature score, channel).
4. Do the exercises that force a short proof or inequality, not only numerical plugging.

Exercises (warm-up)

1. Compute self-information of drawing an ace from a 52-card deck (in bits).
2. Show $H_2(p) = H_2(1-p)$ from the definition.
3. Argue without heavy machinery why entropy is maximized for a uniform distribution on a fixed finite alphabet (use symmetry or Jensen's inequality sketch).

4. A file of English text is compressed from 100 MB to 30 MB losslessly. What does this say about empirical entropy **relative to** 8 bits/byte ASCII? What does it **not** say about "true language entropy"?
5. Explain in one paragraph why cross-entropy loss for classification is an information-theoretic coding cost.
6. List three systems where mutual information is more natural than correlation as a dependence measure.
7. Convert 2 nats to bits.
8. Why is password strength sometimes discussed in **bits of entropy**? What distributional assumption is often smuggled in?
9. If X is deterministic, prove $H(X) = 0$ from the definition.
10. Give an example of two variables with zero correlation but nonzero mutual information (describe qualitatively).

Checkpoint

You should now be able to:

- Explain Shannon information without invoking "meaning"
- State the source coding and channel coding problems in one sentence each
- Compute binary entropy $H_2(p)$ and interpret bias vs compressibility
- Map entropy / KL / MI to at least one ML or systems tool each
- Avoid the main conceptual traps (semantics, continuous entropy, asymmetric KL)

Next: **Entropy and Information Content** develops the full discrete entropy toolkit with proofs, examples, and exercises.

Entropy and Information Content

Entropy is the fundamental measure of information content and uncertainty in information theory. It quantifies how much information is contained in a message or how uncertain we are about the outcome of a random variable.

Information Content

Self-Information

The information content of an event is inversely related to its probability:

$$\mathbf{I(x) = -\log(P(x))}$$

```
import numpy as np
import matplotlib.pyplot as plt

def information_content(probability):
    """Calculate information content in bits"""
    return -np.log2(probability)

# Example: Information content of different events
events = [
    ("Coin flip (heads)", 0.5),
    ("Rolling a 6", 1/6),
    ("Drawing ace of spades", 1/52),
    ("Winning lottery", 1e-8)
]

print("Information Content Examples:")
for event, prob in events:
    info = information_content(prob)
    print(f"{event}: P = {prob:.2e}, I = {info:.2f} bits")

# Visualize information content vs probability
probs = np.logspace(-6, 0, 1000)  # From 10^-6 to 1
info_content = information_content(probs)

plt.figure(figsize=(10, 6))
plt.semilogx(probs, info_content, 'b-', linewidth=2)
```

```
plt.xlabel('Probability')
plt.ylabel('Information Content (bits)')
plt.title('Information Content vs Probability')
plt.grid(True)
plt.show()
```

Shannon Entropy

Definition

For a discrete random variable X with possible values {x , x , ..., x }:

$$\mathbf{H(X) = -\Sigma\ P(x)\ log\ \ P(x)}$$

```
def shannon_entropy(probabilities):
    """Calculate Shannon entropy in bits"""
    # Remove zero probabilities to avoid log(0)
    probs = np.array(probabilities)
    probs = probs[probs > 0]
    return -np.sum(probs * np.log2(probs))

# Example 1: Fair coin
fair_coin = [0.5, 0.5]
print(f"Fair coin entropy: {shannon_entropy(fair_coin):.3f} bits")

# Example 2: Biased coin
biased_coin = [0.9, 0.1]
print(f"Biased coin entropy: {shannon_entropy(biased_coin):.3f} bits")

# Example 3: Fair die
fair_die = [1/6] * 6
print(f"Fair die entropy: {shannon_entropy(fair_die):.3f} bits")

# Example 4: Loaded die
loaded_die = [0.5, 0.1, 0.1, 0.1, 0.1, 0.1]
print(f"Loaded die entropy: {shannon_entropy(loaded_die):.3f} bits")
```

Properties of Entropy

```
def plot_binary_entropy():
    """Plot entropy of binary random variable"""
    p = np.linspace(0.001, 0.999, 1000)
    entropy = -p * np.log2(p) - (1-p) * np.log2(1-p)
```

```
    plt.figure(figsize=(10, 6))
    plt.plot(p, entropy, 'b-', linewidth=2)
    plt.axhline(y=1, color='r', linestyle='--', alpha=0.7, label='Maximum entropy')
    plt.axvline(x=0.5, color='r', linestyle='--', alpha=0.7)
    plt.xlabel('Probability p')
    plt.ylabel('Entropy H(X) (bits)')
    plt.title('Binary Entropy Function')
    plt.grid(True)
    plt.legend()
    plt.show()

    print(f"Maximum entropy occurs at p = 0.5: {shannon_entropy([0.5, 0.5]):.3f} bits")

plot_binary_entropy()
```

Entropy in Data Analysis

Text Analysis

```
from collections import Counter
import string

def text_entropy(text):
    """Calculate entropy of text based on character frequencies"""
    # Convert to lowercase and remove punctuation
    text = text.lower().translate(str.maketrans('', '', string.punctuation))
    text = text.replace(' ', '')  # Remove spaces for character-level analysis

    # Count character frequencies
    char_counts = Counter(text)
    total_chars = len(text)

    # Calculate probabilities
    probabilities = [count / total_chars for count in char_counts.values()]

    return shannon_entropy(probabilities), char_counts

# Example texts
texts = [
    ("English text", "The quick brown fox jumps over the lazy dog"),
    ("Repetitive text", "aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa"),
    ("Random text", "xqzjkwpvmcnbghytrueioadslf"),
    ("Structured text", "abcabcabcabcabcabcabcabcabc")
]
```

```
print("Text Entropy Analysis:")
for name, text in texts:
    entropy, char_counts = text_entropy(text)
    print(f"{name}: {entropy:.3f} bits per character")
    print(f"  Unique characters: {len(char_counts)}")
    print(f"  Most common: {char_counts.most_common(3)}")
    print()
```

Image Entropy

```
def image_entropy(image):
    """Calculate entropy of image pixel values"""
    # Flatten image and get pixel value counts
    pixels = image.flatten()
    pixel_counts = np.bincount(pixels)

    # Calculate probabilities (excluding zero counts)
    probabilities = pixel_counts[pixel_counts > 0] / len(pixels)

    return shannon_entropy(probabilities)

# Create sample images with different entropy levels
def create_sample_images():
    # Low entropy: mostly uniform
    low_entropy = np.full((100, 100), 128, dtype=np.uint8)
    low_entropy += np.random.randint(-10, 10, (100, 100))

    # Medium entropy: gradient
    x, y = np.meshgrid(np.linspace(0, 255, 100), np.linspace(0, 255, 100))
    medium_entropy = ((x + y) / 2).astype(np.uint8)

    # High entropy: random noise
    high_entropy = np.random.randint(0, 256, (100, 100), dtype=np.uint8)

    return low_entropy, medium_entropy, high_entropy

low_ent_img, med_ent_img, high_ent_img = create_sample_images()

# Calculate entropies
entropies = [
    ("Low entropy", low_ent_img),
    ("Medium entropy", med_ent_img),
    ("High entropy", high_ent_img)
]
```

```
fig, axes = plt.subplots(1, 3, figsize=(15, 5))

for i, (name, img) in enumerate(entropies):
    entropy = image_entropy(img)
    axes[i].imshow(img, cmap='gray')
    axes[i].set_title(f'{name}\nH = {entropy:.2f} bits')
    axes[i].axis('off')

plt.tight_layout()
plt.show()
```

Cross-Entropy and KL Divergence

Cross-Entropy

Cross-entropy measures the average number of bits needed to encode events from distribution P using a code optimized for distribution Q:

$$\mathbf{H(P,Q) = -\Sigma\ P(x)\ \log\ \ Q(x)}$$

```
def cross_entropy(p, q):
    """Calculate cross-entropy between distributions p and q"""
    p, q = np.array(p), np.array(q)
    # Avoid log(0) by adding small epsilon
    epsilon = 1e-15
    q = np.clip(q, epsilon, 1)
    return -np.sum(p * np.log2(q))

# Example: True vs predicted distributions
true_dist = [0.7, 0.2, 0.1]
pred_dist1 = [0.6, 0.3, 0.1]  # Close to true
pred_dist2 = [0.1, 0.1, 0.8]  # Far from true

print(f"True distribution entropy: {shannon_entropy(true_dist):.3f} bits")
print(f"Cross-entropy (close prediction): {cross_entropy(true_dist, pred_dist1):.3f} bits")
print(f"Cross-entropy (poor prediction): {cross_entropy(true_dist, pred_dist2):.3f} bits")
```

Kullback-Leibler (KL) Divergence

KL divergence measures how different two probability distributions are:

$$\mathbf{D_KL(P||Q) = \Sigma\ P(x)\ \log\ (P(x)/Q(x))}$$

```
def kl_divergence(p, q):
    """Calculate KL divergence from q to p"""
    p, q = np.array(p), np.array(q)
    epsilon = 1e-15
    p = np.clip(p, epsilon, 1)
    q = np.clip(q, epsilon, 1)
    return np.sum(p * np.log2(p / q))

# Properties of KL divergence
print("KL Divergence Properties:")
print(f"D_KL(P||P) = {kl_divergence(true_dist, true_dist):.6f} (always 0)")
print(f"D_KL(P||Q1) = {kl_divergence(true_dist, pred_dist1):.3f}")
print(f"D_KL(P||Q2) = {kl_divergence(true_dist, pred_dist2):.3f}")
print(f"D_KL(Q1||P) = {kl_divergence(pred_dist1, true_dist):.3f} (asymmetric)")

# Relationship: H(P,Q) = H(P) + D_KL(P||Q)
h_p = shannon_entropy(true_dist)
h_pq1 = cross_entropy(true_dist, pred_dist1)
kl_pq1 = kl_divergence(true_dist, pred_dist1)

print(f"\nRelationship verification:")
print(f"H(P) = {h_p:.3f}")
print(f"H(P,Q) = {h_pq1:.3f}")
print(f"D_KL(P||Q) = {kl_pq1:.3f}")
print(f"H(P) + D_KL(P||Q) = {h_p + kl_pq1:.3f}")
```

Applications in Machine Learning

Decision Trees and Information Gain

```
def information_gain(parent, children):
    """Calculate information gain for a split"""
    parent_entropy = shannon_entropy(parent)

    # Weighted average of children entropies
    total_samples = sum(len(child) for child in children)
    weighted_entropy = sum(
        (len(child) / total_samples) * shannon_entropy(child)
        for child in children
    )

    return parent_entropy - weighted_entropy

# Example: Binary classification dataset
```

```
# Class distribution before split
parent_classes = [0.6, 0.4]  # 60% class 0, 40% class 1

# After split on feature
left_child = [0.9, 0.1]   # 90% class 0, 10% class 1
right_child = [0.2, 0.8]  # 20% class 0, 80% class 1

# Assume equal-sized children for simplicity
children = [left_child, right_child]

gain = information_gain(parent_classes, children)
print(f"Information gain from split: {gain:.3f} bits")

# Compare with different splits
print("\nComparing different splits:")
splits = [
    ("Good split", [[0.9, 0.1], [0.2, 0.8]]),
    ("Poor split", [[0.55, 0.45], [0.65, 0.35]]),
    ("Perfect split", [[1.0, 0.0], [0.0, 1.0]])
]

for name, children in splits:
    gain = information_gain(parent_classes, children)
    print(f"{name}: {gain:.3f} bits")
```

Feature Selection using Mutual Information

```
from sklearn.feature_selection import mutual_info_classif
from sklearn.datasets import make_classification

# Generate sample dataset
X, y = make_classification(n_samples=1000, n_features=10, n_informative=5,
                           n_redundant=2, n_clusters_per_class=1, random_state=42)

# Calculate mutual information between features and target
mi_scores = mutual_info_classif(X, y, random_state=42)

# Visualize feature importance
plt.figure(figsize=(10, 6))
feature_names = [f'Feature {i}' for i in range(X.shape[1])]
plt.bar(feature_names, mi_scores)
plt.xlabel('Features')
plt.ylabel('Mutual Information')
plt.title('Feature Importance using Mutual Information')
plt.xticks(rotation=45)
```

```
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

print("Feature ranking by mutual information:")
for i, score in sorted(enumerate(mi_scores), key=lambda x: x[1], reverse=True):
    print(f"Feature {i}: {score:.3f}")
```

Entropy in Compression

Huffman Coding Efficiency

```
import heapq
from collections import defaultdict, Counter

class HuffmanNode:
    def __init__(self, char=None, freq=0, left=None, right=None):
        self.char = char
        self.freq = freq
        self.left = left
        self.right = right

    def __lt__(self, other):
        return self.freq < other.freq

def build_huffman_tree(text):
    """Build Huffman tree from text"""
    # Count character frequencies
    freq = Counter(text)

    # Create priority queue
    heap = [HuffmanNode(char, freq) for char, freq in freq.items()]
    heapq.heapify(heap)

    # Build tree
    while len(heap) > 1:
        left = heapq.heappop(heap)
        right = heapq.heappop(heap)
        merged = HuffmanNode(freq=left.freq + right.freq, left=left, right=right)
        heapq.heappush(heap, merged)

    return heap[0] if heap else None

def get_huffman_codes(root):
```

```
    """Get Huffman codes from tree"""
    if not root:
        return {}

    codes = {}

    def traverse(node, code=""):
        if node.char is not None:  # Leaf node
            codes[node.char] = code or "0"  # Handle single character case
        else:
            traverse(node.left, code + "0")
            traverse(node.right, code + "1")

    traverse(root)
    return codes

def analyze_compression_efficiency(text):
    """Analyze Huffman coding efficiency"""
    # Calculate entropy
    char_counts = Counter(text)
    total_chars = len(text)
    probabilities = [count / total_chars for count in char_counts.values()]
    entropy = shannon_entropy(probabilities)

    # Build Huffman tree and get codes
    root = build_huffman_tree(text)
    codes = get_huffman_codes(root)

    # Calculate average code length
    avg_length = sum(char_counts[char] * len(code) for char, code in codes.items()) / total_

    # Calculate compression ratio
    original_bits = len(text) * 8  # ASCII encoding
    compressed_bits = sum(char_counts[char] * len(codes[char]) for char in char_counts)
    compression_ratio = compressed_bits / original_bits

    print(f"Text: '{text[:50]}{'...' if len(text) > 50 else ''}'")
    print(f"Entropy: {entropy:.3f} bits per symbol")
    print(f"Average Huffman code length: {avg_length:.3f} bits per symbol")
    print(f"Efficiency: {entropy / avg_length:.3f} ({entropy / avg_length * 100:.1f}%)")
    print(f"Compression ratio: {compression_ratio:.3f} ({compression_ratio * 100:.1f}%)")
    print(f"Space saved: {(1 - compression_ratio) * 100:.1f}%")
    print()

    return entropy, avg_length, codes

# Test with different types of text
```

```
test_texts = [
    "AAAAAAAAAA",  # Low entropy
    "ABABABAB",    # Medium entropy
    "ABCDEFGHIJ",  # High entropy
    "The quick brown fox jumps over the lazy dog"  # Natural text
]

for text in test_texts:
    analyze_compression_efficiency(text)
```

Practical Applications

Password Strength Analysis

```
import string
import math

def password_entropy(password):
    """Estimate password entropy"""
    # Determine character set size
    charset_size = 0
    if any(c.islower() for c in password):
        charset_size += 26  # lowercase
    if any(c.isupper() for c in password):
        charset_size += 26  # uppercase
    if any(c.isdigit() for c in password):
        charset_size += 10  # digits
    if any(c in string.punctuation for c in password):
        charset_size += len(string.punctuation)  # special characters

    # Calculate entropy assuming uniform distribution
    entropy = len(password) * math.log2(charset_size)

    return entropy, charset_size

def analyze_password_strength(passwords):
    """Analyze strength of different passwords"""
    print("Password Strength Analysis:")
    print("-" * 60)

    for pwd in passwords:
        entropy, charset = password_entropy(pwd)

        # Strength categories
```

```
        if entropy < 30:
            strength = "Very Weak"
        elif entropy < 50:
            strength = "Weak"
        elif entropy < 70:
            strength = "Moderate"
        elif entropy < 90:
            strength = "Strong"
        else:
            strength = "Very Strong"

        print(f"Password: {'*' * len(pwd)} (length: {len(pwd)})")
        print(f"Charset size: {charset}")
        print(f"Entropy: {entropy:.1f} bits")
        print(f"Strength: {strength}")
        print(f"Time to crack (1B guesses/sec): {2**(entropy-1) / 1e9:.2e} seconds")
        print()

# Test passwords
test_passwords = [
    "123456",
    "password",
    "Password1",
    "P@ssw0rd!",
    "MyVeryLongAndComplexPassword123!@#"
]

analyze_password_strength(test_passwords)
```

Network Protocol Efficiency

```
def protocol_efficiency_analysis():
    """Analyze efficiency of different encoding schemes"""

    # Simulate message types and their frequencies
    message_types = {
        'ACK': 0.4,      # Acknowledgment
        'DATA': 0.3,     # Data packet
        'NACK': 0.1,     # Negative acknowledgment
        'PING': 0.1,     # Ping
        'CLOSE': 0.05,   # Close connection
        'ERROR': 0.05    # Error message
    }

    # Fixed-length encoding (3 bits per message type)
```

```
    fixed_length = 3

    # Calculate optimal variable-length encoding
    probabilities = list(message_types.values())
    entropy = shannon_entropy(probabilities)

    # Huffman-like optimal encoding lengths
    # Sort by probability (descending)
    sorted_types = sorted(message_types.items(), key=lambda x: x[1], reverse=True)

    # Assign code lengths (simplified Huffman)
    optimal_codes = {
        sorted_types[0][0]: 1,  # Most frequent: 1 bit
        sorted_types[1][0]: 2,  # Second: 2 bits
        sorted_types[2][0]: 3,  # Third: 3 bits
        sorted_types[3][0]: 3,  # Fourth: 3 bits
        sorted_types[4][0]: 4,  # Fifth: 4 bits
        sorted_types[5][0]: 4   # Sixth: 4 bits
    }

    # Calculate average lengths
    avg_fixed = fixed_length
    avg_optimal = sum(message_types[msg] * length for msg, length in optimal_codes.items())

    print("Network Protocol Encoding Analysis:")
    print("-" * 50)
    print(f"Message entropy: {entropy:.3f} bits")
    print(f"Fixed-length encoding: {avg_fixed} bits per message")
    print(f"Optimal variable-length: {avg_optimal:.3f} bits per message")
    print(f"Efficiency gain: {(avg_fixed - avg_optimal) / avg_fixed * 100:.1f}%")
    print()

    print("Message type frequencies and optimal codes:")
    for msg, prob in sorted_types:
        print(f"{msg}: {prob:.2%} -> {optimal_codes[msg]} bits")

protocol_efficiency_analysis()
```

Summary

Entropy and information content are fundamental concepts that:

1. **Quantify Information**: Measure how much information is contained in data
2. **Guide Compression**: Determine theoretical limits of data compression
3. **Optimize Communication**: Design efficient encoding schemes
4. **Analyze Uncertainty**: Measure randomness and predictability

5. **Feature Selection**: Identify informative features in machine learning
6. **Security Analysis**: Evaluate password strength and cryptographic systems

Understanding these concepts enables you to: - Design efficient data compression algorithms - Analyze the information content of datasets - Optimize communication protocols - Build better machine learning models - Evaluate security systems ## Source Coding Sketch (Huffman Intuition)

Given symbol frequencies, assign shorter codes to frequent symbols and longer codes to rare symbols.

Example frequencies: - A: 0.4, B: 0.3, C: 0.2, D: 0.1

One valid prefix code: - A -> 0 - B -> 10 - C -> 110 - D -> 111

Average length: `L = 0.4*1 + 0.3*2 + 0.2*3 + 0.1*3 = 1.9 bits/symbol`

This is close to entropy lower bound for efficient lossless coding.

Mutual Information and Channel Capacity

Mutual information measures **shared information** between random variables: how much knowing one reduces uncertainty about the other. Channel capacity is mutual information optimized over input distributions—the fundamental rate limit for reliable communication. Together they connect statistics, coding, machine learning, and systems design.

1. Conditional entropy

Let X, Y be discrete random variables with joint pmf $p(x, y)$.

Definition (conditional entropy).

$$H(X \mid Y) = \sum_{y} p(y)\, H(X \mid Y = y) = -\sum_{x,y} p(x, y) \log p(x \mid y).$$

Interpretation: average remaining uncertainty about X after observing Y.

Chain rule for entropy.

$$H(X, Y) = H(X) + H(Y \mid X) = H(Y) + H(X \mid Y).$$

Worked example 1

X fair bit, $Y = X$ with probability 1 (noiseless copy). Then $H(X \mid Y) = 0$ and $H(X, Y) = H(X) = 1$.

Worked example 2

X fair bit, Y independent fair bit. Then $H(X \mid Y) = H(X) = 1$ and $H(X, Y) = 2$.

2. Mutual information: definitions

Definition (mutual information).

$$I(X;Y) = H(X) - H(X \mid Y) = H(Y) - H(Y \mid X).$$

Equivalent forms:

$$I(X;Y) = H(X) + H(Y) - H(X,Y),$$

$$I(X;Y) = \sum_{x,y} p(x,y) \log \frac{p(x,y)}{p(x)p(y)} = D_{\mathrm{KL}}(p(x,y) \,\|\, p(x)p(y)).$$

So MI is the KL divergence between the joint and the product of marginals—a pure measure of **dependence**.

Units

With $\log_2$, $I(X;Y)$ is in **bits**. With ln, **nats**.

3. Fundamental properties

Theorem (nonnegativity). $I(X;Y) \geq 0$, with equality iff $X \perp Y$ (independence).

Proof sketch. KL divergence is nonnegative (Gibbs / log-sum inequality / Jensen on $-\log$). Equality in KL holds iff the two distributions are identical, i.e. $p(x,y) = p(x)p(y)$ for all x, y.

Symmetry. $I(X;Y) = I(Y;X)$.

Bounds.

$$0 \leq I(X;Y) \leq \min\{H(X), H(Y)\}.$$

Indeed $I(X;Y) = H(X) - H(X \mid Y) \leq H(X)$ because conditional entropy is nonnegative for discrete variables.

Self-information identity. $I(X;X) = H(X)$.

	$Y=0$	$Y=1$

Worked example 3 — 2×2 joint

Joint table:

	$Y=0$	$Y=1$
$X=0$	$1/2$	0
$X=1$	$1/4$	$1/4$

Marginals: $p_X(0) = 1/2$, $p_X(1) = 1/2$; $p_Y(0) = 3/4$, $p_Y(1) = 1/4$.

Compute $H(X) = 1$, $H(Y) = H_2(3/4) \approx 0.811$, and

$$H(X,Y) = -\Big(\tfrac{1}{2}\log\tfrac{1}{2} + \tfrac{1}{4}\log\tfrac{1}{4} + \tfrac{1}{4}\log\tfrac{1}{4}\Big) = 1.5.$$

Then $I(X;Y) = H(X) + H(Y) - H(X,Y) \approx 1 + 0.811 - 1.5 = 0.311$ bits.

Worked example 4 — independence implies zero MI

If $p(x,y) = p(x)p(y)$, each term $\log \frac{p(x,y)}{p(x)p(y)} = \log 1 = 0$, so $I(X;Y) = 0$.

Worked example 5 — deterministic function

If $Y = g(X)$ deterministic, then $H(Y \mid X) = 0$, so $I(X;Y) = H(Y)$. Observing X reveals Y completely; MI equals the entropy of the function's output distribution.

4. Conditional mutual information and chain rules

Definition.

$$I(X;Y \mid Z) = H(X \mid Z) - H(X \mid Y, Z).$$

Chain rule for mutual information.

$$I(X_1, \dots, X_n; Y) = \sum_{i=1}^{n} I(X_i; Y \mid X_1, \dots, X_{i-1}).$$

This is the information-theoretic backbone of sequential feature scoring and of many proofs about multipartite systems.

5. Data processing inequality (DPI)

Definition (Markov chain). $X \to Y \to Z$ means $p(x, y, z) = p(x)p(y \mid x)p(z \mid y)$, i.e. $X \perp Z \mid Y$.

Theorem (data processing inequality). If $X \to Y \to Z$, then

$$I(X; Z) \leq I(X; Y).$$

Proof sketch. Expand $I(X; Y, Z) = I(X; Z) + I(X; Y \mid Z) = I(X; Y) + I(X; Z \mid Y)$. Markovity implies $I(X; Z \mid Y) = 0$, and $I(X; Y \mid Z) \geq 0$, so $I(X; Z) \leq I(X; Y)$.

Interpretation: no post-processing of Y can create new information about X. Filtering, quantization, and learned encoders can only **destroy or preserve** (not invent) mutual information with the original source.

Worked example 6 — ML pipeline

Let X be raw features, $Y = \mathrm{PCA}(X)$ a linear projection, $Z = \mathrm{ReLU}(WY)$ a neural transform of Y. Then $X \to Y \to Z$, so $I(X; Z) \leq I(X; Y) \leq H(X)$. Feature engineering cannot magically increase total information about X; it can only reallocate what is useful for a **task label** T (where $I(T; Z)$ may rise even as $I(X; Z)$ falls—task-relevant compression).

6. Channels and capacity

Definition (discrete memoryless channel, DMC). A channel is a conditional pmf $p(y \mid x)$ used independently each time: for input sequence x^n,

$$p(y^n \mid x^n) = \prod_{i=1}^{n} p(y_i \mid x_i).$$

Definition (channel capacity).

$$C = \max_{p(x)} I(X; Y),$$

where the joint is $p(x)p(y \mid x)$ and the max is over input distributions on the finite alphabet $\mathcal{X}$.

Operational meaning (Shannon's channel coding theorem, informal).

- Rates $R < C$: there exist codes with block length $n \to \infty$ and error probability $\to 0$.
- Rates $R > C$: error probability is bounded away from 0 for any code.

Capacity is not "what your current protocol achieves"; it is an **ultimate limit**.

7. Binary symmetric channel (BSC)

Model. Input bit $X \in \{0, 1\}$. Output $Y = X$ with probability $1 - p$, flipped with probability p (crossover probability).

Binary entropy. $H_2(p) = -p \log_2 p - (1 - p) \log_2(1 - p)$.

Theorem (BSC capacity).

$$C_{\text{BSC}}(p) = 1 - H_2(p) \quad \text{(bits per channel use)}.$$

Derivation sketch. For any input with $P(X = 1) = \pi$,

$$H(Y \mid X) = H_2(p)$$

(independent of π: noise is symmetric). Then $I(X;Y) = H(Y) - H_2(p)$. $H(Y)$ is maximized at 1 bit when Y is fair, achieved at $\pi = 1/2$. Hence $C = 1 - H_2(p)$.

Worked example 7

p	$H_2(p)$	$C = 1 - H_2(p)$
0	0	1
0.11	≈ 0.5	≈ 0.5
0.5	1	0

At $p = 1/2$, the channel output is independent of the input: capacity is zero—you cannot communicate reliably at any positive rate.

Worked example 8 — Z-channel intuition (contrast)

A Z-channel flips $0 \to 1$ never, but $1 \to 0$ with probability p. Capacity is **not** $1 - H_2(p)$; asymmetry changes the optimizing input distribution. (Computing it is a good exercise in maximizing $I(X;Y)$ numerically over π.)

8. Fano's inequality (why MI controls prediction)

Theorem (Fano, informal). If $\hat{X} = g(Y)$ estimates discrete X with error probability $P_e = P(\hat{X} \neq X)$, then

$$H(X \mid Y) \leq H_2(P_e) + P_e \log(|\mathcal{X}| - 1).$$

Thus small $H(X \mid Y)$ (large $I(X;Y)$ relative to $H(X)$) is **necessary** for accurate recovery. This underpins converse theorems in communication and lower bounds in statistical learning.

9. CS and ML applications

Feature selection

Score feature X_j by $I(X_j;T)$ with target T. Prefer conditional MI $I(X_j;T \mid S)$ when set S of features is already chosen (reduces redundancy).

Representation learning

Many objectives maximize a lower bound on $I(Z;T)$ or $I(Z;X^+)$ (positive pairs) while compressing $I(Z;X)$ (information bottleneck). Contrastive losses (InfoNCE) are MI lower bounds under specific critic models.

Privacy leakage

If secret S and released data R satisfy $I(S;R) \approx 0$, release reveals little about S in the Shannon sense. (Note: cryptographic / differential privacy notions are stronger or different; MI is one leakage metric, not the only one.)

Compression with side information

Slepian–Wolf coding: separate encoders for X and Y can still achieve joint rates near $H(X,Y)$ when decoding is joint—powered by conditional entropy and MI structure.

Protocol and systems design

- FEC rate selection relative to estimated channel statistics
- Quantizer design: trade $I(X;\hat{X})$ vs bitrate
- Caching and prefetch: predictive value as reduction in entropy of next request

Worked example 9 — information gain in trees

Split feature reduces class entropy: information gain IG $= H(T) - H(T \mid X_j)$, which equals $I(T;X_j)$ for discrete features/labels. Decision trees greedily maximize MI with the label at each step (locally, not globally optimally).

Worked example 10 — capacity vs SNR mindset

On a BSC, improving hardware to reduce p from 0.1 to 0.01 raises capacity from $1 - H_2(0.1) \approx 0.531$ to $1 - H_2(0.01) \approx 0.919$ bits/use. That is a systems ROI argument phrased in nats/bits, not only "lower BER."

10. Continuous caveat (differential MI)

For continuous variables, **differential entropy** $h(X) = -\int f \log f$ can be negative, but mutual information

$$I(X;Y) = \int \int f(x,y) \log \frac{f(x,y)}{f(x)f(y)}\, dx\, dy$$

remains nonnegative when defined. Still: estimation of continuous MI from samples is statistically delicate in high dimension.

11. Pitfalls

1. **Correlation vs MI.** Uncorrelated does not imply $I = 0$ (nonlinear dependence).
2. **DPI direction.** Processing Y cannot increase $I(X;\cdot)$; but processing can increase $I(T;\cdot)$ for a **different** target T if you discard nuisance parts of X.
3. **Capacity is asymptotic.** Finite-blocklength codes need backoff from C; random coding bounds quantify the gap.
4. **Plug-in MI estimates** on continuous or high-cardinality discrete data are badly biased without binning / kNN / variational methods.
5. **Confusing $I(X;Y)$ with causation.** MI is symmetric and associational.
6. **Using $H(Y \mid X) = 0$ for continuous equality.** Continuous "$Y = X$" needs care with densities.

12. Checkpoint

Before moving on, verify you can:

- Write three equivalent definitions of $I(X;Y)$
- Prove nonnegativity via KL (sketch)
- Apply the chain rule and DPI on a Markov chain
- Derive BSC capacity $1 - H_2(p)$
- Connect MI to feature selection, tree splits, and channel limits

Exercises

Easy

1. Prove $I(X;Y) = H(X) + H(Y) - H(X,Y)$ from the definition $I = H(X) - H(X \mid Y)$.
2. Compute $I(X;Y)$ for the joint $p(0,0) = p(1,1) = 1/2$ (perfect coupling of fair bits).
3. Show that if $X \perp Y$ then $I(X;Y) = 0$ by direct substitution.
4. For BSC(p), verify $H(Y \mid X) = H_2(p)$ for any input distribution.
5. Plot or tabulate $C_{\text{BSC}}(p)$ for $p \in \{0, 0.05, 0.1, 0.2, 0.5\}$.

Medium

6. For a fair bit X and $Y = X \oplus Z$ with $Z \sim \text{Bern}(p)$ independent, show $I(X;Y) = 1 - H_2(p)$.
7. Prove $I(X;Y) \le H(X)$ and interpret equality cases.
8. Let $X \to Y \to Z$. Prove $I(X;Y,Z) = I(X;Y)$.
9. Construct a pair (X,Y) with $\text{Corr}(X,Y) = 0$ but $I(X;Y) > 0$ (e.g. $Y = X^2$ with symmetric discrete X).
10. Explain why greedy information-gain tree learning can miss globally optimal feature sets (give a conceptual counterexample).

Challenge

11. Numerically maximize $I(X;Y)$ over $\pi = P(X = 1)$ for a Z-channel with flip probability $p = 0.2$ on the $1 \to 0$ transition; compare to BSC(0.2).
12. Using Fano's inequality, argue that if $I(X;Y)$ is much smaller than $H(X)$, no decoder $\hat{X}(Y)$ can have tiny error probability when $|\mathcal{X}|$ is large.
13. Prove the chain rule $I(X_1, X_2; Y) = I(X_1;Y) + I(X_2; Y \mid X_1)$.
14. **Privacy sketch:** A mechanism releases $R = X + N$ with noise N. Discuss qualitatively how increasing noise variance affects $I(X;R)$ and utility $I(X;\hat{t}(R))$ for a target statistic t.
15. Read off DPI for the pipeline $X \to \text{encode} \to \text{channel} \to \text{decode} \to \hat{X}$ and state the inequality chain for $I(X;\hat{X})$.

Summary

Mutual information is entropy reduction and KL dependence in one package. Capacity is MI optimized over channel inputs—the sharp dividing line between reliable and impossible communication rates. The same mathematics scores features, justifies compression with side information, and constrains what any neural encoder can extract under Markov processing.

KL Divergence, Cross-Entropy, and Coding

Cross-entropy and **KL divergence** connect coding length to how wrong a model distribution is. They are the mathematical reason machine learning uses "cross-entropy loss," and they quantify wasted bits when the code (or model) assumes the wrong distribution.

Diagram: two distributions

```
true P          model Q



             x

H(P)      = ideal code length under P
H(P,Q)    = code length if you use Q while truth is P
KL(P Q)   = H(P,Q) - H(P)    0   extra bits wasted
```

1. Definitions (discrete)

Let P and Q be probability mass functions on a finite alphabet $\mathcal{X}$. Use log for $\log_2$ (bits) unless noted; natural log yields nats.

Entropy

$$H(P) = -\sum_{x \in \mathcal{X}} P(x) \log P(x),$$

with the convention $0 \log 0 = 0$.

Cross-entropy

$$H(P, Q) = -\sum_{x} P(x) \log Q(x).$$

Requires $Q(x) > 0$ whenever $P(x) > 0$ (absolute continuity on the support of P); otherwise $H(P, Q) = \infty$.

Kullback–Leibler divergence

$$D_{\mathrm{KL}}(P\|Q) = \sum_x P(x) \log \frac{P(x)}{Q(x)} = H(P,Q) - H(P).$$

Fundamental properties

Property	Statement
Nonnegativity	$D_{\mathrm{KL}}(P\|Q) \geq 0$ (Gibbs / information inequality)
Identity of indiscernibles	$= 0$ iff $P = Q$ (on $\operatorname{supp} P$)
Asymmetry	$D_{\mathrm{KL}}(P\|Q) \neq D_{\mathrm{KL}}(Q\|P)$ in general
Not a metric	triangle inequality fails
Units	bits or nats with the log base

Worked example 1 — binary

$P = (1/2, 1/2)$, $Q = (3/4, 1/4)$ on $\{0, 1\}$.

$$H(P) = 1 \text{ bit.}$$

$$H(P,Q) = -\tfrac{1}{2}\log_2 \tfrac{3}{4} - \tfrac{1}{2}\log_2 \tfrac{1}{4} \approx 0.2075 + 1 = 1.2075.$$

$$D_{\mathrm{KL}}(P\|Q) \approx 0.2075 \text{ bits.}$$

Worked example 2 — reverse KL differs

Same P, Q:

$$D_{\mathrm{KL}}(Q\|P) = \tfrac{3}{4}\log_2 \frac{3/4}{1/2} + \tfrac{1}{4}\log_2 \frac{1/4}{1/2} = \tfrac{3}{4}\log_2 \tfrac{3}{2} + \tfrac{1}{4}\log_2 \tfrac{1}{2} \approx 0.1887 \neq D_{\mathrm{KL}}(P\|Q).$$

2. Coding story (why the formulas look like that)

Idealized Shannon code: symbol x gets length about $-\log Q(x)$ if you design for Q.

- If nature draws from P, average length $\approx \sum_x P(x)(-\log Q(x)) = H(P,Q)$.
- Best possible average length (large-block limit) is $H(P)$.
- **Extra** average length is $H(P,Q) - H(P) = D_{\mathrm{KL}}(P\|Q)$.

So KL is literally **expected excess code length** from using the wrong distribution.

Kraft and prefix codes (connection)

For prefix-free codes with lengths $\ell(x)$, Kraft: $\sum_x 2^{-\ell(x)} \le 1$. Setting $Q(x) \propto 2^{-\ell(x)}$ makes coding length and probability dual views of the same object.

3. ML link: cross-entropy loss

Multi-class classification

One training example with true class y (one-hot vector e_y) and model probabilities $Q_\theta(x) = p_\theta(\cdot \mid x)$:

$$\ell(\theta; x, y) = -\log Q_\theta(y) = H(e_y, Q_\theta).$$

Average over the empirical data distribution $\hat{P}$:

$$\mathbb{E}_{(x,y)\sim\hat{P}}[-\log p_\theta(y \mid x)]$$

is empirical cross-entropy. Minimizing it pushes Q_θ toward the conditional label distribution on the data.

Binary logistic regression

$y \in \{0, 1\}$, $\hat{y} = \sigma(w^\top x)$:

$$\ell = -y \log \hat{y} - (1 - y) \log(1 - \hat{y}),$$

which is $H((y, 1-y), (\hat{y}, 1-\hat{y}))$.

Softmax + CE gradient fact

With logits z and $p = \text{softmax}(z)$, for one-hot y:

$$\nabla_z \ell = p - y.$$

This algebraic simplification is a major reason the CE+softmax pair dominates multi-class training.

4. KL as expected log-likelihood gap

If data $\sim P$ and model Q_θ,

$$D_{\mathrm{KL}}(P\|Q_\theta) = \underbrace{-\,\mathbb{E}_P \log Q_\theta}_{\text{cross-entropy}} - H(P).$$

$H(P)$ does not depend on θ, so

$$\arg\min_\theta D_{\mathrm{KL}}(P\|Q_\theta) = \arg\min_\theta H(P, Q_\theta) = \arg\max_\theta \mathbb{E}_P \log Q_\theta.$$

MLE on infinite data = minimize KL to the true distribution (within the model family). Finite-sample MLE is the same program with P replaced by the empirical distribution.

5. Forward vs reverse KL (mode behavior)

Divergence	Heavy penalty when…	Typical behavior of $\arg\min_Q D$
$D_{\mathrm{KL}}(P\|Q)$ "forward"	$P > 0$ but $Q \approx 0$	**Mode-covering**: Q must put mass on all modes of P
$D_{\mathrm{KL}}(Q\|P)$ "reverse"	$Q > 0$ but $P \approx 0$	**Mode-seeking**: Q can collapse to one mode of P

Variational inference often minimizes reverse KL (or a bound); mass-covering objectives appear in other approximations. Choice of KL direction changes algorithm behavior.

```
multimodal P:   ^^^   ^^^
forward KL Q:   ^^^^^^^^^^   (covers both, may smear)
reverse KL Q:      ^^^       (picks one mode)
```

6. Continuous case (caution)

For densities p, q on $\mathbb{R}^d$:

$$D_{\mathrm{KL}}(p\|q) = \int p(x) \log \frac{p(x)}{q(x)}\, dx.$$

Differential entropy $h(p) = -\int p \log p$ can be **negative** and is not a pure "bits of content" analogue. Cross-entropy and KL remain meaningful as relative quantities; absolute continuous entropy needs care with units and reparameterization.

7. Related divergences (map)

Name	Idea
Jensen–Shannon	Symmetrized, bounded KL mixture
Total variation	L_1 distance of measures
Wasserstein	Cost of transporting mass (geometry)
f-divergences	Family including KL, χ^2, Hellinger

Pinsker's inequality links KL and total variation: $\mathrm{TV}(P,Q)^2 \le \frac{1}{2} D_{\mathrm{KL}}(P\|Q)$ (in nats conventions vary — check constants).

8. Worked example 3 — sure event

If $P = (1, 0, \ldots, 0)$, then $H(P) = 0$ and $H(P,Q) = -\log Q(x^\star)$ for the sure symbol $x^\star$. KL is $-\log Q(x^\star)$: certainty under P, model still pays if it is not certain.

Worked example 4 — CE with uniform model

$P = (0.9, 0.1)$, $Q = (0.5, 0.5)$:

$$H(P,Q) = -0.9 \log_2 0.5 - 0.1 \log_2 0.5 = 1 \text{ bit}.$$

$H(P) = H_2(0.9) \approx 0.469$, so $D_{\mathrm{KL}} \approx 0.531$ bits.

9. Numerical and practical pitfalls in ML

1. $\log 0$**:** clamp probabilities or use `log_softmax` in the logit domain
2. **Label smoothing:** replaces one-hot P by a slightly mixed distribution — changes the CE target
3. **Class imbalance:** average CE weights frequent classes more unless reweighted
4. **Comparing models:** lower CE on the same data = better coding of labels; still need calibration / decision metrics
5. **Asymmetry:** never treat KL as a distance without checking direction

10. CS / systems connections

- Compression: wrong source model larger files (excess KL per symbol)
- Language models: perplexity $2^{H(P,Q)}$ on tokens (cross-entropy exponential)
- Anomaly detection: high $-\log Q(x)$ under a background model
- Privacy / leakage: KL between output laws under neighboring datasets (related to DP analysis variants)

11. Checkpoint

- Write $H(P)$, $H(P,Q)$, $D_{\mathrm{KL}}(P\|Q)$ and the identity relating them
- Explain excess code length
- Derive why MLE minimizes KL to the data-generating distribution
- Contrast forward vs reverse KL mode behavior
- Compute a 2-point KL by hand

Exercises

Easy

1. Prove $D_{\mathrm{KL}}(P\|P) = 0$.
2. Compute $H(P)$ for a deterministic $P = (1, 0, ...)$ (limit / discrete sure event).
3. For $P = (0.9, 0.1)$, $Q = (0.5, 0.5)$, compute $H(P,Q)$ in bits.
4. Why is $\log Q$ in the classification loss and not $\log P$?
5. Show $H(P,Q) \geq H(P)$ from nonnegativity of KL.

Medium

6. **Challenge starter:** KL is not a metric — give a numerical counterexample to symmetry with 2-point distributions.
7. Show that for fixed P, $H(P,Q)$ is minimized at $Q = P$ with value $H(P)$.

8. Binary CE: expand $-y \log \hat{y} - (1-y) \log(1-\hat{y})$ and interpret each term.

9. If $Q_\varepsilon = (1-\varepsilon)P + \varepsilon U$ mixes with uniform U, what happens to $H(P, Q_\varepsilon)$ as $\varepsilon \to 0$?

10. Convert a KL of 0.5 nats to bits.

Challenge

11. Prove Gibbs inequality $D_{\mathrm{KL}} \geq 0$ via Jensen on $-\log$ (or log-sum inequality).

12. Softmax+CE: derive $\partial \ell / \partial z_k = p_k - y_k$.

13. Construct P, Q, R violating the triangle inequality for KL (or argue using asymmetry).

14. Show that if $\operatorname{supp} P \not\subseteq \operatorname{supp} Q$ then $D_{\mathrm{KL}}(P \| Q) = \infty$.

15. Relate label smoothing CE to CE against a mixture target; discuss gradient effect on overconfident models.

Checks

3. $-0.9 \log_2 0.5 - 0.1 \log_2 0.5 = 1$ bit.

4. From $D_{\mathrm{KL}}(P \| Q) \geq 0$.

5. $0.5 / \ln 2 \approx 0.721$ bits.

Summary

Cross-entropy is the average code length when nature uses P and you code with Q; KL is the excess over the ideal $H(P)$. Machine learning's cross-entropy loss is this coding cost on labels. KL is nonnegative, asymmetric, and direction-sensitive: forward vs reverse KL encode different approximation geometries. Keep the coding story in mind whenever you see $-\log p_\theta$ in a training objective.

Information Theory Applications

Entropy is not only abstract: it shows up in compression, feature selection, decision trees, language-model evaluation, privacy leakage bounds, and ML objectives. This chapter maps the definitions from earlier chapters onto engineering practice.

Diagram: map of applications

```
entropy H(X)

        compression / coding length
        decision tree split (info gain)
        feature selection (mutual information)
        channel capacity (noisy pipes)
        LM evaluation (perplexity)
        ML losses (cross-entropy / KL)
```

1. Compression and source coding

Shannon source coding theorem (informal): for an i.i.d. source with entropy $H(X)$, lossless compression needs about $H(X)$ bits per symbol in the large-block limit; you cannot do better on average, and good codes approach $H(X)$.

Source property	Compressibility
Low $H(X)$	highly predictable $\rightarrow$ shorter codes
Max entropy (uniform)	incompressible at symbol level
Dependence over time	use conditional / joint coding; rate $\approx H(X_t \mid \text{past})$

Practical codecs (conceptual)

- **Huffman:** optimal prefix codes for known symbol frequencies (per-symbol)
- **Arithmetic / ANS:** approach H more closely for streams
- **LZ family:** universal; learn repeats without an explicit P

Worked example 1

Fair six-sided die: $H = \log_2 6 \approx 2.585$ bits/roll. No lossless scheme averages fewer bits per independent roll in the Shannon limit.

Biased coin $P(X = 1) = 0.1$: $H_2(0.1) \approx 0.469$ bits — much more compressible than a fair coin.

2. Information gain and decision trees

Split feature A to reduce uncertainty about label Y:

$$\mathrm{IG}(Y, A) = H(Y) - H(Y \mid A) = I(Y; A).$$

```
before split:  H(Y) large (mixed labels)

        A=yes  → purer → small H
        A=no   → purer → small H
weighted average H(Y|A) < H(Y)   positive gain
```

Conditional entropy after split:

$$H(Y \mid A) = \sum_a P(A = a)\, H(Y \mid A = a).$$

Worked example 2 — pure split

Binary label 50/50, $H(Y) = 1$ bit. Split yields two pure leaves: $H(Y \mid A) = 0$, so IG $= 1$ bit.

Practical caveats

- **Greedy** IG can miss globally optimal feature sets
- High-arity features can look good by fragmentation (many pure tiny leaves) — use gain ratio / regularization / min leaf size
- Continuous features need thresholds; search is part of the algorithm

3. Mutual information for features and dependence

$$I(X;Y) = H(X) - H(X \mid Y) = H(Y) - H(Y \mid X) = D_{\mathrm{KL}}(P_{X,Y} \| P_X P_Y).$$

- $I(X;Y) = 0$ iff $X \perp Y$ (discrete case under standard conditions)
- Feature ranking: score features by $\hat{I}(X_j; Y)$
- **Caution:** estimating MI in high dimension is hard (bias, curse of dimensionality); use careful estimators or proxies

Worked example 3 — independence

If $X \perp Y$, then $P_{X,Y} = P_X P_Y$, so $D_{\mathrm{KL}} = 0$ and $I(X;Y) = 0$.

Correlation vs MI

Uncorrelated does **not** imply independent. Example: discrete $X \in \{-1, 0, 1\}$ symmetric, $Y = X^2$. Correlation can be zero while $I(X;Y) > 0$.

4. Channels and capacity (systems view)

Noisy channel: input X, output Y, law $p(y \mid x)$.

Capacity

$$C = \max_{p(x)} I(X;Y)$$

is the supremum of rates (bits per channel use) for reliable communication (channel coding theorem, informal).

Setting	Role of capacity
Wireless / modem design	ultimate rate ceiling
FEC vs retransmission	operate below C with codes
Disk / flash	constrained coding + ECC

Worked example 4 — BSC intuition

Binary symmetric channel flip probability p: $C = 1 - H_2(p)$ bits (for fair-ish optimal input). At $p = 0$, $C = 1$; at $p = 1/2$, $C = 0$.

5. Perplexity and language models

For a model q evaluated on tokens from p (or on a test corpus treated as empirical p):

Cross-entropy rate (nats or bits per token) and **perplexity**

$$\mathrm{PPL} = 2^{H(p,q)} \quad \text{(bits convention)}$$

or e^H in nats. Interpretation: effective branching factor — how many sides a fair die would need to match the model's average surprise.

Lower perplexity model less surprised by the test data (better coding of the corpus), not automatically better task utility.

Worked example 5

If average CE is 3 bits/token, $\mathrm{PPL} = 2^3 = 8$.

6. ML training objectives (recap with applications)

Objective	Information view
Cross-entropy classification	code labels with model q_θ
KL to teacher (distillation)	match soft distributions
InfoNCE / contrastive	lower bound / proxy related to MI
Variational bounds	ELBO involves KL to posterior

Training with CE is not "just a loss hack": it is maximum likelihood under a categorical model and coding-optimal under that model class.

7. Decision systems and expected information

Before an experiment or A/B test, **expected information gain** about a parameter θ from observing Y:

$$\mathrm{EIG} = H(\theta) - H(\theta \mid Y) = I(\theta; Y).$$

Bayesian optimal design picks interventions maximizing EIG (or related utilities). Same math as feature selection with a different "feature."

8. Privacy and leakage (conceptual)

If S is a secret and R is a released message (query answer, model prediction, noisy aggregate):

$$I(S;R)$$

measures residual dependence — a leakage metric. Perfect privacy in the MI sense would require $I(S;R) = 0$ (often too strong); differential privacy uses a different, worst-case neighboring-dataset guarantee, but MI still guides intuition about how much a release can say about individuals.

9. Hashing, randomness, and min-entropy

Shannon entropy averages surprise; **min-entropy**

$$H_\infty(X) = -\log \max_x P(x)$$

captures worst-case guessability (passwords, cryptographic keys). High Shannon entropy with a heavy atom can still be weak for security. CS security prefers min-entropy and extractors.

10. End-to-end worked case: email spam filter features

1. Labels $Y \in \{\text{spam}, \text{ham}\}$ with empirical $H(Y)$.
2. For each binary feature A_j ("word w present"), estimate $\text{IG}(Y, A_j)$.
3. Keep top-k features or use MI filter then train logistic regression.
4. Train with CE loss; report CE and calibration on a holdout.
5. Compression analogy: a good model assigns short codes (high probability) to true labels.

11. Pitfalls

1. Treating high entropy as "meaningful content"
2. Greedy IG trees without depth/leaf constraints
3. MI feature selection without correcting for finite-sample bias
4. Using perplexity alone to claim product quality

5. Ignoring that capacity is an asymptotic, ideal-code limit
6. Confusing Shannon entropy with cryptographic strength

12. Checkpoint

- State source coding lower bound $H(X)$
- Compute information gain for a pure binary split
- Write $I(X;Y)$ three ways
- Define channel capacity as max MI
- Convert CE to perplexity
- Distinguish Shannon vs min-entropy for security

Exercises

Easy

1. Binary label 50/50, split yields two pure leaves. What is IG in bits?
2. If $X \perp Y$, what is $I(X;Y)$?
3. Compute H of a fair six-sided die in bits ($\log_2 6$).
4. Connect cross-entropy training to "coding with model Q."
5. Perplexity if CE = 2 bits/token?

Medium

6. Why is estimating $I(X;Y)$ hard for high-dimensional continuous X?
7. Name one failure mode of greedy info-gain tree splits.
8. BSC: explain why $C \to 0$ as $p \to 1/2$.
9. Feature A independent of Y: what is $\mathrm{IG}(Y, A)$?
10. Password distribution with $H(X) = 20$ bits but one password has probability 2^{-5}: bound H_∞.

Challenge

11. Construct (small joint table) X, Y with Corr $= 0$ but $I(X;Y) > 0$.

12. Gain ratio: define $\mathrm{IG}(Y, A)/H(A)$ and explain the high-arity bias fix.

13. Show that $I(X;Y) \leq \min\{H(X), H(Y)\}$.

14. Sketch how arithmetic coding achieves average length near H for a known P.

15. Privacy sketch: adding noise to R tends to decrease $I(S;R)$; discuss utility tradeoff $I(S;\hat{t}(R))$ for a target statistic t.

Checks

1. $H(Y) = 1$, $H(Y \mid A) = 0$, IG $= 1$.

2. 0.

3. ≈ 2.585 bits.

4. PPL $= 4$.

Summary

Information measures are engineering tools: H bounds compression, I scores dependence and channels, CE/KL train and evaluate probabilistic models, perplexity rephrases CE for language, and min-entropy speaks to worst-case unpredictability. Use the right functional for the job, and never forget estimation and asymptotics when moving from chalkboard to production data.

Bloom Filters — False Positives by Design

Bloom Filters — False Positives by Design

Thesis

A **Bloom filter** is a bit array plus k hash functions that answers "might be in the set" with a tunable **false-positive** rate and **no false negatives**. Space is $O(n)$ bits for n insertions — far below storing the elements.

Mental model

```
  m bits, k hashes
  add(x): set bits h1(x)…hk(x)
  query(x): yes iff all those bits are set
  false positive   (1 - e^{-kn/m})^k
  optimal k   (m/n) ln 2
```

Worked example

```
# bloom.py
import math, hashlib

class Bloom:
    def __init__(self, n, p):
        self.m = math.ceil(-(n * math.log(p)) / (math.log(2) ** 2))
        self.k = max(1, round((self.m / n) * math.log(2)))
        self.bits = bytearray((self.m + 7) // 8)
    def _hashes(self, s):
        h = hashlib.sha256(s.encode()).digest()
        for i in range(self.k):
            yield int.from_bytes(h[i*4:(i+1)*4], "big") % self.m
    def add(self, s):
        for x in self._hashes(s):
            self.bits[x // 8] |= 1 << (x % 8)
    def __contains__(self, s):
        return all(self.bits[x // 8] & (1 << (x % 8)) for x in self._hashes(s))

b = Bloom(1000, 0.01)
```

```
for i in range(1000):
    b.add(f"user-{i}")
fp = sum(1 for i in range(1000, 2000) if f"user-{i}" in b)
print(f"m={b.m} k={b.k} false_positives_in_1000={fp} rate={fp/1000:.3f}")
print("member", "user-42" in b, "nonmember approx", "user-1999" in b)
```

```
m=9586 k=7 false_positives_in_1000=15 rate=0.015
member True nonmember approx False
```

Target $p = 0.01$; observed rate about 0.015 on this draw — Monte Carlo noise around the design point.

Trap

Deleting elements with a plain Bloom filter. You cannot turn bits off safely (aliases). Use a counting Bloom filter or recreate. Also: hashing poorly correlated bits destroys the analysis.

Boring rule

Size from (n, p) with the standard formulae, store m and k with the filter, and treat "yes" as "maybe — confirm in the source of truth."

Try this

1. Sweep p from 0.1 to 0.001 and plot $m(n, p)$.
2. Feed non-member queries only and estimate p empirically.
3. Compare to a hash set's RSS for the same n strings.

Sources

- Bloom, "Space/Time Trade-offs in Hash Coding with Allowable Errors" (1970)
- Broder & Mitzenmacher survey on network applications of Bloom filters

Optimization

Optimization Theory for Computer Science

Optimization is the mathematical study of **choosing the best feasible point**. Almost every modern CS stack hides an optimizer: training a model, allocating resources, compiling code, routing traffic, or packing VMs. This part develops the language (objectives, constraints, convexity), the guarantees (global vs local minima, duality), and the algorithms (gradients, projections, regularized objectives) that make those systems work.

Chapters in this part

Chapter	Focus
Convexity	Convex sets/functions, why local = global
Constrained optimization	Feasibility, KKT intuition
Regularization	Ridge/Lasso and generalization
First-order methods playbook	GD, SGD, momentum, Adam diagnostics
Gradient methods	Deeper gradient algorithms

The canonical problem

$$\begin{aligned} \text{minimize}_{x \in \mathcal{X}} \quad & f(x) \\ \text{subject to} \quad & g_i(x) \le 0, \quad i = 1, \dots, m, \\ & h_j(x) = 0, \quad j = 1, \dots, p. \end{aligned}$$

- f: **objective** (loss, cost, negative reward)
- g_i: **inequality constraints** (capacity, risk, nonnegativity)
- h_j: **equality constraints** (conservation, normalization)
- $\mathcal{X}$: ambient space, often $\mathbb{R}^n$ or a discrete set

A point is **feasible** if it satisfies all constraints. A feasible $x^\star$ is a **global minimizer** if $f(x^\star) \le f(x)$ for every feasible x. It is a **local minimizer** if this holds in a neighborhood of $x^\star$.

Maximization

Maximizing f is minimizing $-f$. All theory is stated for minimization without loss of generality.

A taxonomy that actually helps

By decision variables

Type	Variables	Typical tools
Continuous	$x \in \mathbb{R}^n$	Gradient methods, Newton, interior-point
Discrete / integer	$x \in \mathbb{Z}^n$ or combinatorial	Branch-and-bound, dynamic programming, approximation
Mixed-integer	continuous + integer	MILP/MIQP solvers
Functional / infinite-dim	functions, measures	calculus of variations, optimal control

By structure of f and constraints

- **Linear program (LP):** linear objective and constraints — polynomial-time solvable in theory; extremely scalable in practice.
- **Quadratic program (QP):** quadratic objective, linear constraints — core of SVM duals, portfolio optimization.
- **Convex program:** convex f, convex inequality constraints, affine equalities — **local = global**.
- **Nonconvex:** neural net training, many combinatorial relaxations — local minima, saddles, initialization matter.

By information used

- Zeroth-order: only f values (black-box, bandits)
- First-order: gradients ∇f (SGD, Adam)
- Second-order: Hessians $\nabla^2 f$ (Newton, quasi-Newton)

Why convexity is the "dividing line"

For a **convex** problem, any local minimum is global, first-order stationarity $\nabla f(x^\star) = 0$ (unconstrained, differentiable) characterizes optima, and duality theory is strong under mild conditions. For nonconvex problems, you usually settle for **stationarity**, **approximate optimality**, or **problem-specific structure** (e.g. matrix completion under incoherence).

This is why ML theory obsesses over convex surrogates even when the "true" goal is combinatorial (0-1 loss).

Optimization appears everywhere in CS

Machine learning

- Empirical risk minimization: $\min_w \frac{1}{n}\sum_i \ell(y_i, f_w(x_i)) + \lambda\Omega(w)$
- Constrained training: fairness, safety, simplex constraints on probabilities
- Hyperparameter search: bilevel / black-box optimization

Algorithms and theory

- Shortest paths and flows as combinatorial optimization
- Approximation algorithms: provable ratios for NP-hard objectives
- Online optimization: regret minimization

Systems

- Compiler register allocation and instruction scheduling (ILP-ish)
- Database query plan selection (search over trees with cost models)
- Autoscaling and bin packing in cloud orchestration
- Congestion control as feedback optimization

Worked example 1 — ridge regression as optimization

$$\min_w \|Xw - y\|_2^2 + \lambda\|w\|_2^2.$$

This is **strictly convex** for $\lambda > 0$; the unique minimizer solves the linear system $(X^\top X + \lambda I)w = X^\top y$. Optimization theory tells you existence, uniqueness, and a stable algorithm (linear solve), not just "run GD."

Worked example 2 — resource allocation

Maximize $\sum_i \log(r_i)$ subject to $\sum_i r_i \le C$, $r_i \ge 0$ (proportional fairness). KKT conditions yield water-filling-like structure; used in networking and scheduling.

Worked example 3 — nonconvex neural loss

$f(w) = \frac{1}{n}\sum_i \ell(y_i, f_w(x_i))$ for a deep net is nonconvex. Theory gives: smooth optimization finds **stationary points**; generalization needs statistics; practice uses SGD + overparameterization + regularization. Do not import convex guarantees blindly.

Roadmap of this part

1. **Convexity** — sets, functions, first- and second-order characterizations, strong convexity, preservation rules.
2. **Constrained optimization** — Lagrangians, KKT conditions, dual problems, geometric intuition.
3. **Regularization and generalization** — ridge/lasso geometry, bias–variance, early stopping as implicit regularization.
4. **Gradient methods** — GD, SGD, momentum, adaptive methods, rates under smoothness/convexity.

Prerequisites

- Multivariable calculus: gradients, Hessians, chain rule
- Linear algebra: norms, eigenvalues, PSD matrices
- Probability: expectations for stochastic optimization
- Comfort with "for all / exists" statements in definitions

Modeling skill (underrated)

Bad models make perfect solvers useless. Good practice:

1. State decision variables **explicitly**.
2. Write the objective as a real-valued function (not vibes).
3. Encode hard requirements as constraints; soft preferences as penalties.
4. Check units and scaling (features, stepsizes).
5. Ask: convex? smooth? sparse? stochastic?

Worked example 4 — modeling a deadline

"Finish jobs quickly" might be:

- minimize $\max_i C_i$ (makespan), or
- minimize $\sum_i C_i$ (total completion), or
- minimize $\sum_i w_i C_i$ (weighted).

These are **different mathematical problems** with different algorithms and hardness.

Problem class	Typical status

Complexity reality check

Problem class	Typical status
LP, convex QP (reasonable size)	Tractable
General integer linear programs	NP-hard; solvable at medium scale with solvers
Nonconvex continuous	Hard in worst case; heuristics + structure
Hyperparameter search	Expensive black box; Bayesian opt / bandits

Pitfalls

1. **Local vs global.** Reporting a training loss minimum is not global optimality unless convex (or proven otherwise).
2. **Constraints forgotten.** Unconstrained GD on probabilities can leave the simplex.
3. **Scaling.** Features with different magnitudes warp gradients and condition numbers.
4. **Discrete treated as continuous.** Rounding LP solutions needs approximation theory.
5. **Confusing modeling error with solver error.** A wrong objective is not fixed by Adam.

Checkpoint

You should be able to:

- Write a general constrained optimization problem and define feasibility / global min
- Classify a problem as LP / convex / nonconvex at a glance
- Give three CS domains where optimization is the core mathematical problem
- Explain why convexity changes the meaning of "solving"
- Convert a vague product goal into variables + objective + constraints (at least roughly)

Exercises

1. Write ERM for logistic regression with L2 penalty in the canonical min f form. Identify f and $\mathcal{X}$.
2. Is $f(x) = x^4$ convex on $\mathbb{R}$? Is it strictly convex? Strongly convex?
3. Maximize $x(1-x)$ on $[0,1]$ by converting to a minimization problem.
4. Give a feasible set that is nonconvex; explain why projected gradient may fail if used naively.
5. Model: assign n jobs to m machines to minimize makespan. What are variables and constraints? (ILP sketch)
6. Why is $\min \|w\|_0$ s.t. $Xw = y$ harder than ridge regression?
7. Explain the difference between a **stationary point** and a **global minimizer**.

8. For $f(x, y) = x^2 + y^2$ s.t. $x + y = 1$, guess the optimum by geometry (closest point to origin on the line).
9. List two reasons SGD is preferred over full-batch GD for large n in ML.
10. A team says "we optimized accuracy." Rewrite as a precise mathematical objective (at least two legitimate versions).
11. Give an example where a local minimum is terrible compared to the global one (nonconvex 1D sketch).
12. Research prompt: what does "polynomial-time solvable" mean for LP in theory vs practical simplex?

Summary

Optimization is the shared mathematical backbone of learning, resource control, and algorithmic decision-making. The rest of this part builds the convex foundation, the KKT/duality toolkit, regularization geometry, and gradient algorithms you will use constantly in ML and systems.

Convexity Foundations

Convexity is the structural property that turns optimization from a landscape of traps into a tractable geometry: **every local minimum is global**, first-order conditions are sufficient, and averaging feasible points stays feasible. This chapter develops convex sets and functions with characterizations, examples, and CS/ML payoffs.

1. Convex sets

Definition. A set $C \subseteq \mathbb{R}^n$ is **convex** if for all $x, y \in C$ and all $\theta \in [0,1]$,

$$\theta x + (1-\theta)y \in C.$$

The point $\theta x + (1-\theta)y$ is a **convex combination** of x and y. Geometrically: the line segment between any two points of C lies entirely in C.

Examples of convex sets

- Empty set, singletons, $\mathbb{R}^n$
- Open/closed balls in any norm; ellipsoids $\{x : (x-c)^\top A(x-c) \le 1\}$ for $A \succ 0$
- Affine subspaces $\{x : Ax = b\}$
- Halfspaces $\{x : a^\top x \le b\}$ and polyhedra $\{x : Ax \le b\}$
- Probability simplex $\Delta^{n-1} = \{x : x \ge 0,\ \mathbf{1}^\top x = 1\}$
- PSD cone $\mathbf{S}^n_+ = \{M = M^\top : M \succeq 0\}$

Non-examples

- Two disjoint balls (segment between centers leaves the set)
- Integer lattice $\mathbb{Z}^n$
- Unit circle $\{x : \|x\|_2 = 1\}$ (sphere surface)
- Rank-at-most-k matrices (nonconvex for $0 < k <$ min dimensions)

Worked example 1

Prove the simplex is convex: if $x, y \ge 0$, $\mathbf{1}^\top x = \mathbf{1}^\top y = 1$, then $z = \theta x + (1-\theta)y \ge 0$ and $\mathbf{1}^\top z = 1$.

Operations preserving convexity of sets

- Intersection: arbitrary intersections of convex sets are convex
- Affine images: C convex $\Rightarrow AC + b = \{Ac + b : c \in C\}$ convex
- Minkowski sums: C, D convex $\Rightarrow C + D$ convex
- **Not** generally closed under unions

Theorem. A polyhedron $\{x : Ax \le b\}$ is convex.

Proof. Intersection of halfspaces; each halfspace is convex; intersection preserves convexity.

2. Convex functions

Definition. Let $C \subseteq \mathbb{R}^n$ be convex. $f : C \to \mathbb{R}$ is **convex** if for all $x, y \in C$ and $\theta \in [0, 1]$,

$$f(\theta x + (1-\theta)y) \le \theta f(x) + (1-\theta) f(y).$$

f is **strictly convex** if the inequality is strict for $x \ne y$ and $\theta \in (0, 1)$.

f is **concave** if $-f$ is convex.

Epigraph characterization. f is convex iff its **epigraph**

$$\operatorname{epi} f = \{(x, t) : x \in C,\ f(x) \le t\}$$

is a convex set in $\mathbb{R}^{n+1}$.

Worked example 2

$f(x) = x^2$ on $\mathbb{R}$: expand $\theta x^2 + (1-\theta)y^2 - (\theta x + (1-\theta)y)^2 = \theta(1-\theta)(x-y)^2 \ge 0$. Convex; strictly convex.

Worked example 3

$f(x) = x^3$ on $\mathbb{R}$ is **not** convex: midpoint of -1 and 1 gives $f(0) = 0 > \frac{1}{2}(f(-1) + f(1)) = -1$? Wait: $\frac{1}{2}((-1) + 1) = 0 = f(0)$, need a better test. Use second derivative $f''(x) = 6x$, negative for $x < 0$, so not convex on $\mathbb{R}$. On $[0, \infty)$ it is convex.

Worked example 4

$f(x) = \|x\|$ any norm: triangle inequality + absolute homogeneity $\Rightarrow$ convex. Not strictly convex for ℓ_1 or ℓ_∞ (flat faces).

Worked example 5 — logistic loss

$\ell(z) = \log(1 + e^{-z})$ is convex in z (composition rules / second derivative $\sigma(z)(1-\sigma(z)) \geq 0$ after careful differentiation of related forms). Thus logistic regression ERM in w is convex when ℓ is applied to linear predictors $y_i w^\top x_i$ with appropriate labeling conventions.

3. First-order characterization

Theorem. Let f be differentiable on an open convex set C. Then f is convex iff for all $x, y \in C$,

$$f(y) \geq f(x) + \nabla f(x)^\top (y - x).$$

Interpretation: the graph lies above all tangent hyperplanes. The linear approximation is a **global underestimator**.

Proof sketch ($\Rightarrow$). Write $y = x + t(d)$ path, use definition of directional derivative / difference quotient and take limits; or apply definition to $x_\theta = \theta y + (1-\theta)x$ and rearrange:

$$\frac{f(x_\theta) - f(x)}{\theta} \leq f(y) - f(x),$$

let $\theta \to 0$.

($\Leftarrow$): Apply the inequality at x_θ toward x and y, then convex-combine.

Subgradients (nondifferentiable case)

Definition. g is a **subgradient** of f at x if for all y,

$$f(y) \geq f(x) + g^\top (y - x).$$

The **subdifferential** $\partial f(x)$ is the set of all subgradients. For convex f, $\partial f(x)$ is nonempty on the relative interior of the domain (under mild conditions). Example: $\partial |x|$ at 0 is $[-1, 1]$.

4. Second-order characterization

Theorem. Let f be twice continuously differentiable on open convex C. Then f is convex iff $\nabla^2 f(x) \succeq 0$ (positive semidefinite) for all $x \in C$. Strict convexity is implied by $\nabla^2 f(x) \succ 0$ everywhere, but the converse needs care (e.g. x^4 is strictly convex with $f''(0) = 0$).

Worked example 6

$f(x) = \frac{1}{2} x^\top Q x + c^\top x + d$ with $Q = Q^\top$. Hessian is Q. Convex iff $Q \succeq 0$; strictly convex if $Q \succ 0$.

Worked example 7 — least squares

$f(w) = \|Xw - y\|_2^2$ has Hessian $2X^\top X \succeq 0$, always convex; strictly convex iff X has full column rank.

5. Global optimality theorems

Theorem (local ⇒ global). Let f be convex on convex C. If $x^\star$ is a local minimizer, then $x^\star$ is a global minimizer.

Proof. Suppose $f(y) < f(x^\star)$ for some $y \in C$. For $\theta \in (0,1)$ small, $x_\theta = \theta y + (1-\theta)x^\star$ is arbitrarily close to $x^\star$ and

$$f(x_\theta) \le \theta f(y) + (1-\theta) f(x^\star) < f(x^\star),$$

contradicting local minimality.

Theorem (first-order sufficiency, unconstrained). If f is convex and differentiable on $\mathbb{R}^n$, then $x^\star$ minimizes f **iff** $\nabla f(x^\star) = 0$.

Proof. Necessity: standard calculus. Sufficiency: $f(y) \ge f(x^\star) + \nabla f(x^\star)^\top (y - x^\star) = f(x^\star)$.

Theorem (subgradient optimality). For convex f, $x^\star$ is a minimizer iff $0 \in \partial f(x^\star)$.

6. Strong convexity

Definition. f is **μ-strongly convex** ($\mu > 0$) if for all x, y and $\theta \in [0,1]$,

$$f(\theta x + (1-\theta)y) \le \theta f(x) + (1-\theta) f(y) - \frac{\mu}{2}\theta(1-\theta)\|x - y\|_2^2.$$

Equivalent (differentiable): $f(y) \ge f(x) + \nabla f(x)^\top (y - x) + \frac{\mu}{2}\|y - x\|_2^2$.

Equivalent (twice diff.): $\nabla^2 f(x) \succeq \mu I$.

Consequences

- Unique minimizer
- Linear convergence of gradient descent under L-smoothness with step $1/L$ (rate depends on condition number L/μ)
- Stability of solutions under data perturbation (related to regularization)

Worked example 8

Ridge objective $\|Xw - y\|_2^2 + \lambda\|w\|_2^2$ is μ-strongly convex with $\mu \ge 2\lambda$ (actually Hessian $2X^\top X + 2\lambda I \succeq 2\lambda I$).

7. Calculus of convex functions (closure rules)

If f, g convex and $\alpha, \beta \geq 0$, then $\alpha f + \beta g$ convex.

- Pointwise maximum: $x \mapsto \max_i f_i(x)$ convex if each f_i is
- Affine composition: $f(Ax + b)$ convex if f convex
- Nonnegative weighted sums and expectations: $x \mapsto \mathbb{E}_\xi[f(x, \xi)]$ convex if each $f(\cdot, \xi)$ is
- **Perspective**, **infimal convolution**, and many more advanced rules (see Boyd–Vandenberghe)

Dangerous compositions: $h(f(x))$ with h increasing convex and f convex is convex; but $h \circ f$ can destroy convexity if h is not increasing (e.g. $h(t) = -t$, f convex $\Rightarrow$ concave).

Worked example 9 — ReLU networks

A single ReLU unit $w \mapsto \max(0, w^\top x)$ is convex in w for fixed x. Composition of many layers with products of weights is **not** jointly convex in all parameters—the deep net loss is typically nonconvex.

8. Smoothness (companion property)

Definition. f is L**-smooth** if ∇f is L-Lipschitz:

$$\|\nabla f(x) - \nabla f(y)\|_2 \leq L\|x - y\|_2.$$

For twice differentiable f, equivalent to $-LI \preceq \nabla^2 f(x) \preceq LI$ in operator sense for convex smooth functions often stated as $0 \preceq \nabla^2 f \preceq LI$.

Descent lemma: $f(y) \leq f(x) + \nabla f(x)^\top (y - x) + \frac{L}{2}\|y - x\|_2^2$.

Smoothness + convexity $\Rightarrow$ standard GD rates $O(1/k)$ on function values; strong convexity upgrades this.

9. CS/ML connections

Concept	Role
Convex losses	Logistic, hinge, squared — tractable ERM
Simplex / PSD cone	Probabilities, covariance, SDP relaxations
Strong convexity via $\lambda\|w\|_2^2$	Unique solution, faster opt, stability
Nonconvex deep nets	Rely on overparameterization + SGD dynamics

Concept	Role
Convex relaxations	Max-cut SDP, matrix completion nuclear norm

Worked example 10 — hinge loss SVM

$\min_w \frac{\lambda}{2}\|w\|_2^2 + \sum_i \max(0, 1 - y_i w^\top x_i)$ is convex (sum of convex pieces + strongly convex regularizer). Subgradient methods apply; dual QP is the classic SVM dual.

10. Pitfalls

1. **Strict vs strong convexity.** x^4 is strictly but not strongly convex on $\mathbb{R}$.
2. **Convex domain required** in the definition; saying "f is convex" includes the domain geometry.
3. **Level sets of convex f are convex; converse false** (f could be quasiconvex only).
4. **Product of convex nonnegative functions need not be convex.**
5. **Discrete sets** break segment arguments—integer programs need different theory.
6. **Numerical Hessians** with tiny negative eigenvalues due to roundoff: use tolerances carefully when testing PSD.

11. Checkpoint

- Define convex sets/functions and test with segments
- Use first- and second-order characterizations
- Prove local minima are global for convex f
- Explain strong convexity and one algorithmic consequence
- Know which ML losses are convex in parameters for linear models

Exercises

Easy

1. Prove that the intersection of any collection of convex sets is convex.
2. Show $f(x) = e^{a^\top x}$ is convex on $\mathbb{R}^n$.
3. Is $f(x, y) = xy$ convex on $\mathbb{R}^2$? On $\mathbb{R}^2_+$?
4. Prove any norm is convex using the triangle inequality.
5. Show that if f is convex, then sublevel sets $\{x : f(x) \le \alpha\}$ are convex.

Medium

6. Prove the first-order characterization of convexity for differentiable f (both directions).
7. Determine convexity of $f(t) = t \log t$ on $(0, \infty)$ (extended by continuity at 0).
8. Show $f(X) = -\log\det(X)$ is convex on $X \succ 0$ (you may use known spectral facts or restrict to lines).
9. Prove that a strictly convex function has at most one minimizer.
10. For $f(w) = \|Xw - y\|_2^2 + \lambda\|w\|_1$, is f convex? Strongly convex? Differentiable?

Challenge

11. Prove that f convex and L-smooth implies $\|\nabla f(x) - \nabla f(y)\|_2^2 \leq 2L(f(x) - f(y) - \nabla f(y)^\top(x - y))$ (cocoercivity / standard inequality).
12. Give a quasiconvex function that is not convex; explain optimization consequences.
13. Show the nuclear norm $\|M\|_* = \sum_i \sigma_i(M)$ is convex (hint: dual characterization $\sup_{\|U\|_{2\to 2}\leq 1}\langle U, M\rangle$).
14. Prove strong convexity of ridge regression with explicit μ in terms of λ and optionally $\sigma_{\min}(X)$.
15. Construct a nonconvex f with a unique global minimizer where gradient descent from some start converges to a suboptimal stationary point (1D sketch OK).

Summary

Convexity is the geometry of segments and epigraphs. First- and second-order tests make it checkable; strong convexity upgrades uniqueness and rates; closure rules let you build large convex models from simple blocks. When convexity fails—as in deep learning—you must replace global guarantees with stationarity, structure, or empiricism.

Constrained Optimization and KKT Conditions

Most real optimization problems are constrained: probabilities lie on a simplex, weights satisfy budgets, flows conserve mass, embeddings have unit norm. This chapter develops Lagrangians, KKT conditions, dual problems, and the geometry of active constraints—the toolkit behind SVM duals, resource allocation, and projected training.

1. Problem form

$$\begin{aligned} \min_{x \in \mathbb{R}^n} \quad & f(x) \\ \text{s.t.} \quad & g_i(x) \le 0, \quad i = 1, \dots, m, \\ & h_j(x) = 0, \quad j = 1, \dots, p. \end{aligned}$$

Assume f, g_i, h_j continuously differentiable unless stated. Feasible set:

$$\mathcal{F} = \{x : g_i(x) \le 0 \ \forall i, \ h_j(x) = 0 \ \forall j\}.$$

Active set at x: $\mathcal{A}(x) = \{i : g_i(x) = 0\}$. Equality constraints are always "active."

Worked example 1 — projection onto a ball

$\min_x \|x - y\|_2^2$ s.t. $\|x\|_2 \le 1$. If $\|y\|_2 \le 1$, solution is y. Else $x^\star = y/\|y\|_2$. Constraint active on the boundary case.

2. Equality constraints and Lagrange multipliers

Unconstrained case recap

For smooth unconstrained f, $\nabla f(x^\star) = 0$ is necessary for a local min (and sufficient under convexity).

One equality

$\min f(x)$ s.t. $h(x) = 0$.

Lagrangian:

$$\mathcal{L}(x, \nu) = f(x) + \nu\, h(x).$$

Necessary condition (regular points): there exists $\nu^\star$ such that

$$\nabla_x \mathcal{L}(x^\star, \nu^\star) = \nabla f(x^\star) + \nu^\star \nabla h(x^\star) = 0, \quad h(x^\star) = 0.$$

Geometry: ∇f is normal to the constraint surface; level sets of f are tangent to $\{h = 0\}$.

Worked example 2 — classic textbook problem

Minimize $f(x, y) = x^2 + y^2$ subject to $h(x, y) = x + y - 1 = 0$.

$\mathcal{L} = x^2 + y^2 + \nu(x + y - 1)$.

Stationarity: $2x + \nu = 0$, $2y + \nu = 0$, and $x + y = 1$.

Thus $x = y$ and $2x = 1 \Rightarrow x = y = \frac{1}{2}$, $\nu = -1$. Minimum value $\frac{1}{2}$.

Check: on the line, $f(x, 1 - x) = x^2 + (1 - x)^2 = 2x^2 - 2x + 1$, vertex at $x = \frac{1}{2}$.

Worked example 3 — entropy maximization

$\max -\sum_i p_i \log p_i$ s.t. $\sum_i p_i = 1$, $p_i \geq 0$ (ignore inequalities first via substitution). Lagrange for equality yields the **uniform** distribution on a finite alphabet—maximum entropy.

3. Inequality constraints and KKT

For inequalities $g_i(x) \leq 0$, multipliers λ_i enter with sign conventions matching the inequality direction.

Lagrangian:

$$\mathcal{L}(x, \lambda, \nu) = f(x) + \sum_{i=1}^{m} \lambda_i g_i(x) + \sum_{j=1}^{p} \nu_j h_j(x).$$

Karush–Kuhn–Tucker (KKT) conditions (necessary at a local min under a constraint qualification, e.g. LICQ: active constraint gradients linearly independent):

1. **Stationarity:** $\nabla_x \mathcal{L}(x^\star, \lambda^\star, \nu^\star) = 0$
2. **Primal feasibility:** $g_i(x^\star) \leq 0$, $h_j(x^\star) = 0$
3. **Dual feasibility:** $\lambda_i^\star \geq 0$ for all i

4. **Complementary slackness:** $\lambda_i^\star g_i(x^\star) = 0$ for all i

Complementary slackness: either the constraint is active ($g_i = 0$) or its multiplier is zero (inactive constraints do not affect stationarity).

Geometric reading

At the optimum, $-\nabla f(x^\star)$ lies in the cone generated by active inequality gradients and the span of equality gradients (outward/inward depending on sign conventions). You cannot decrease f without leaving the feasible region.

Worked example 4 — inequality active vs inactive

$\min (x-3)^2$ s.t. $x \le 1$. Unconstrained min at $x = 3$ is infeasible. Optimum $x^\star = 1$, $g = x - 1$, $\lambda > 0$, stationarity $2(x-3) + \lambda = 0 \Rightarrow \lambda = 4$ at $x = 1$.

If constraint were $x \le 10$, unconstrained $x = 3$ is feasible, $\lambda = 0$, constraint inactive.

4. Constraint qualifications (why they matter)

Without regularity, KKT may fail to hold at a true optimum.

Example pathology. Minimize x s.t. $x^3 \le 0$… actually a standard example is $f(x,y) = x$ with $g_1 = -y \le 0$? Classic: minimize x subject to $x^2 + y^2 \le 0$ forces $(0,0)$ but gradients can be degenerate depending on formulation.

LICQ: $\{\nabla g_i(x^\star) : i \in \mathcal{A}(x^\star)\} \cup \{\nabla h_j(x^\star)\}$ linearly independent.

Slater's condition (for convex problems): exists x with $g_i(x) < 0$ all i and $h_j(x) = 0$. Ensures strong duality.

5. Duality

Lagrange dual function:

$$q(\lambda, \nu) = \inf_x \mathcal{L}(x, \lambda, \nu).$$

q may be $-\infty$ for some multipliers. For $\lambda \ge 0$, weak duality says $q(\lambda, \nu) \le f(x)$ for every feasible x.

Dual problem:

$$\max_{\lambda \ge 0,\ \nu} q(\lambda, \nu).$$

Weak duality: dual optimal $\le$ primal optimal always.

Strong duality: equality of optimal values. Holds for convex problems under Slater-type conditions.

Duality gap: $f(x^\star) - q(\lambda^\star, \nu^\star)$. Zero under strong duality.

Worked example 5 — SVM dual intuition

Hard-margin SVM primal is a QP with inequalities $y_i(w^\top x_i + b) \geq 1$. Dual involves multipliers $\alpha_i \geq 0$ with $\sum_i \alpha_i y_i = 0$ and objective depending on Gram matrix $y_i y_j x_i^\top x_j$. Support vectors are points with $\alpha_i > 0$ (active margins)—complementary slackness in action.

Worked example 6 — equality-constrained QP

$\min \frac{1}{2} x^\top P x + c^\top x$ s.t. $Ax = b$ with $P \succ 0$. KKT system is a linear saddle-point system:

$$\begin{bmatrix} P & A^\top \\ A & 0 \end{bmatrix} \begin{bmatrix} x \\ \nu \end{bmatrix} = \begin{bmatrix} -c \\ b \end{bmatrix}.$$

Solved by factorization / Schur complements in numerical optimization and graphics (constrained least squares).

6. Methods that enforce constraints

Method	Idea	Notes
Elimination	Solve equalities, reduce variables	Best when h linear simple
Projection	GD step then project onto $\mathcal{F}$	Needs cheap projection
Barrier / interior-point	Add $-\mu \sum \log(-g_i)$	Central path; LP/QP/SOCP/SDP
Penalty	Add $\rho \sum \max(g_i, 0)^2$	Simple; need $\rho \to \infty$ carefully
Augmented Lagrangian	Penalty + multiplier updates	Robust practical method
Active-set	Guess active inequalities, solve EQ	Classic QP solvers

Worked example 7 — projection onto simplex

Euclidean projection onto $\{x : x \geq 0, \mathbf{1}^\top x = 1\}$ can be done by sorting / thresholding (standard algorithm). Used in probability simplex constrained models and some attention normalizations (conceptually).

Worked example 8 — projected gradient on a box

$\mathcal{F} = [L, U]^n$. Projection is coordinatewise clipping. Extremely common in bound-constrained least squares.

7. Convex constrained problems

If f convex, g_i convex, h_j affine, the problem is a **convex optimization problem**. Then:

- Any local min is global
- KKT conditions are also **sufficient** for optimality (under standard setup)
- Strong duality often holds (Slater)

This is the theoretical home of logistic regression with linear constraints, ridge with linear equalities, SDP relaxations, etc.

Worked example 9 — water-filling sketch

Maximize $\sum_i \log(1 + p_i/\sigma_i^2)$ s.t. $\sum_i p_i = P$, $p_i \geq 0$ (power allocation). KKT yields $p_i = \max(0,\ \mu - \sigma_i^2)$—allocate power to better channels first.

8. CS/ML applications

- **SVM / kernel machines:** constrained QP + dual kernels
- **Optimal transport (basic):** coupling constraints; dual potentials
- **Portfolio / risk:** budget and inequality risk constraints
- **Fairness constraints:** demographic parity as linear constraints on predictors
- **MPC / control:** constrained quadratic programs in real time
- **Phase retrieval / nonconvex QCQP:** hard; relaxations or projected methods

9. Pitfalls

1. **Wrong inequality direction** flips multiplier signs and dual feasibility.
2. **Forgetting complementary slackness** when interpreting support vectors / active resources.
3. **Applying unconstrained Newton** inside a constrained region without projection/barriers.
4. **Nonconvex constraints** ($\|x\|_2 = 1$ equality is nonconvex as a set? The sphere is nonconvex; the constraint function $h = x^\top x - 1$ is nonconvex as equality manifold optimization needs care). Unit-norm sphere requires Riemannian or penalty methods.
5. **Strong duality assumed** outside convexity—gap may be nonzero.
6. **Numerical KKT residuals:** check stationarity, feasibility, and complementarity separately when debugging solvers.

10. Checkpoint

- Form Lagrangians for equality and inequality problems
- State all four KKT blocks and interpret complementary slackness
- Solve simple 2-variable equality problems by hand
- Explain weak vs strong duality in one paragraph
- Name three algorithmic strategies for enforcing constraints

Exercises

Easy

1. Minimize $x^2 + y^2$ s.t. $x + 2y = 1$. Find (x, y, ν).
2. Minimize $(x - 2)^2$ s.t. $x \geq 0$. Write KKT and solve.
3. Explain why $\lambda_i \geq 0$ is required for inequalities $g_i \leq 0$ in standard form.
4. For $\min \|x\|_2^2$ s.t. $a^\top x = 1$, find closed form $x^\star$ via Lagrange.
5. Give an example where the unconstrained minimizer is feasible, so all inequality multipliers are zero.

Medium

6. Derive the KKT system for $\min \frac{1}{2}\|x - y\|_2^2$ s.t. $x \geq 0$ (nonnegative projection). Show $x^\star = \max(y, 0)$ coordinatewise.
7. Prove weak duality: for $\lambda \geq 0$ and feasible x, $q(\lambda, \nu) \leq f(x)$.
8. Soft-margin SVM primal: identify which constraints become active for support vectors vs margin violators (qualitative is OK with definitions).
9. Convert $\max f$ with $g_i \geq 0$ into standard $\min$ / $g \leq 0$ form carefully.
10. Explain LICQ for $g_1(x, y) = x$, $g_2 = -x$, at $(0, 0)$—are gradients independent?

Challenge

11. Solve $\min c^\top x$ s.t. $Ax = b$, $x \geq 0$ (LP) dual derivation sketch; interpret complementary slackness economically.
12. Show that for convex problem with Slater condition, KKT $\Rightarrow$ global optimality (sufficiency sketch).
13. Implement (pseudocode) projected gradient for $\min f(x)$ on the probability simplex; state a step-size rule under L-smoothness.
14. For $\min_w \frac{1}{2}\|w\|_2^2$ s.t. $y_i w^\top x_i \geq 1$ $\forall i$ (hard-margin, separable), write dual and identify the role of α_i.
15. Discuss why $\|X\|_* = 1$ nuclear-norm ball constraints are convex and how they differ from $\text{rank}(X) \leq k$.

Summary

Constraints turn stationarity into a balance between objective gradients and constraint gradients. KKT packages feasibility, multiplier signs, and complementary slackness into necessary (and, for convex programs, sufficient) optimality conditions. Duality supplies certificates and often simpler dual algorithms—central in ML and resource allocation.

Regularization and Generalization

A model that fits training data perfectly can still fail on new data. **Regularization** is the mathematical practice of preferring simpler or more stable solutions when minimizing empirical risk. This chapter connects penalized objectives, geometry of ℓ_1/ℓ_2 balls, bias–variance tradeoffs, and implicit regularization from early stopping and gradient methods.

1. The generalization problem

Data $(x_i, y_i)_{i=1}^n$ drawn i.i.d. from unknown P. Hypothesis f_w with parameters $w \in \mathcal{W}$.

True risk:

$$R(w) = \mathbb{E}_{(x,y)\sim P}\big[\ell(y, f_w(x))\big].$$

Empirical risk:

$$\hat{R}_n(w) = \frac{1}{n}\sum_{i=1}^{n} \ell(y, f_w(x_i)).$$

ERM: $\hat{w} \in \arg\min_w \hat{R}_n(w)$. Without restriction, if $\mathcal{W}$ is huge, $\hat{R}_n(\hat{w})$ can be near zero while $R(\hat{w})$ is large (**overfitting**).

Generalization gap: $R(w) - \hat{R}_n(w)$ (or absolute value). Learning theory bounds this gap via complexity of $\mathcal{W}$ (VC dimension, Rademacher complexity, covering numbers, stability, PAC-Bayes, etc.).

Worked example 1 — polynomial overfitting

Fitting degree-$n-1$ polynomial through n noisy points: training error ≈ 0, test error huge. High-capacity interpolators need regularization or implicit bias.

2. Regularized empirical risk

$$\min_w \hat{R}_n(w) + \lambda \Omega(w).$$

- Ω: regularizer (complexity penalty)
- $\lambda \geq 0$: tradeoff hyperparameter

Equivalent constrained form (under mild conditions, Lagrange duality):

$$\min_w \hat{R}_n(w) \quad \text{s.t.} \quad \Omega(w) \leq \tau.$$

Common regularizers

Name	$\Omega(w)$	Effect
Ridge (Tikhonov)	$\|w\|_2^2$	Shrinkage, unique min if $\lambda > 0$
Lasso	$\|w\|_1$	Sparsity, feature selection
Elastic net	$\alpha\|w\|_1 + (1-\alpha)\|w\|_2^2$	Sparse + grouped stability
Group lasso	$\sum_g \|w_g\|_2$	Group sparsity
Nuclear norm	$\|W\|_*$	Low-rank matrices
Total variation	$\|\nabla w\|_1$	Piecewise smooth signals

3. Ridge regression in closed form

$$\min_w \|Xw - y\|_2^2 + \lambda \|w\|_2^2, \quad \lambda > 0.$$

Normal equations:

$$(X^\top X + \lambda I) w = X^\top y.$$

Solution:

$$w_\lambda = (X^\top X + \lambda I)^{-1} X^\top y = X^\top (X X^\top + \lambda I)^{-1} y$$

(the second form helps when $n \ll d$).

Worked example 2 — collinearity

If two features are nearly identical, $X^\top X$ is ill-conditioned; OLS coefficients explode with opposite signs. Ridge eigenvalues of $X^\top X + \lambda I$ are at least λ, stabilizing the solve.

Spectral view

If $X = U\Sigma V^\top$ (economy SVD), ridge shrinks singular components: directions with small singular values get damped more—**spectral filtering**.

4. Lasso and sparsity geometry

$$\min_w \frac{1}{2n}\|Xw - y\|_2^2 + \lambda\|w\|_1.$$

Geometry: ℓ_1 balls are diamonds (in 2D). Contours of quadratic loss meet the diamond preferably at axes $\Rightarrow$ coordinates set to zero.

Soft-thresholding (orthogonal design $X^\top X = I$ after scaling):

$$\widehat{w}_j = S_\lambda(z_j), \quad S_\lambda(z) = \operatorname{sign}(z)\max(|z| - \lambda, 0).$$

Worked example 3

$z = (3, 0.2, -1.5)$, $\lambda = 0.5$: soft-threshold $\to (2.5, 0, -1.0)$. Small coefficient wiped out.

Worked example 4 — when lasso fails uniqueness

Highly correlated features: lasso may pick one arbitrarily. Elastic net mixes ℓ_2 to stabilize groups.

5. Bias–variance tradeoff

For squared error and many estimators,

$$\mathbb{E}\big[(f_{\widehat{w}}(x) - y)^2\big] = \text{Bias}^2 + \text{Variance} + \sigma^2.$$

Increasing λ:

- **Bias up:** solution pulled toward 0 (or other prior)
- **Variance down:** less sensitive to training noise

Optimal λ balances the two on **validation** data, not training loss.

Worked example 5 — ridge path

As $\lambda : 0 \to \infty$, w_λ moves from OLS (or min-norm interpolator) to 0. Cross-validation picks intermediate λ minimizing estimated risk.

6. Early stopping as implicit regularization

Iterate $w_{k+1} = w_k - \alpha \nabla \hat{R}_n(w_k)$ from $w_0 = 0$. Stopping at iteration k often behaves like spectral filtering similar to ridge: early iterates align with high-curvature (important) directions first.

Practical rule: monitor validation loss; stop when it rises (**early stopping**).

Worked example 6

Unregularized linear regression with GD on an overparameterized least-squares problem: without stopping, you may drift toward large-norm interpolators; early stop keeps smaller effective norm.

7. Implicit bias of optimizers

Even without explicit Ω, algorithm + initialization select a particular minimizer among many.

- **Gradient descent on underdetermined least squares** from 0 converges to the **minimum ℓ_2-norm** interpolator.
- **Coordinate descent / certain sign patterns** relate to ℓ_1 biases in some settings.
- **SGD noise** can prefer flatter regions (heuristic / active research).

This is why "we didn't use regularization" is often false in deep learning—the optimizer is the regularizer.

8. Structural risk and capacity control

Penalized ERM is one implementation of **structural risk minimization**: choose a nested family $\mathcal{W}_1 \subset \mathcal{W}_2 \subset \cdots$ with increasing capacity; pick level via validation or complexity penalties.

Related classical tools:

- AIC / BIC (parametric model selection)
- MDL (minimum description length)—links to coding/information theory
- Dropout, data augmentation, stochastic depth: **implicit / algorithmic** regularizers

Worked example 7 — weight decay in deep nets

PyTorch `weight_decay` in AdamW decouples ℓ_2 penalty from adaptive scaling—practitioners treat it as ridge-like shrinkage on weights, crucial for generalization in large models.

9. Double descent (modern nuance)

Classical U-shaped risk vs capacity curves are incomplete for interpolating regimes. **Double descent:** risk can decrease again past the interpolation threshold as models grow, with min-norm / benign overfitting phenomena in some linear and random-feature models.

Regularization still matters, but "always increase λ until underfit" is too crude for modern overparameterized ML. Use validation, not folklore alone.

Worked example 8 — conceptual sketch

Risk vs model size: descent, peak near interpolation, second descent for very wide models with appropriate training. Explicit ridge can shift the peak and improve finite-sample risk.

10. Practical workflow

1. Split **train / validation / test** (or cross-validation); freeze test until the end.
2. Choose Ω matching inductive bias (sparse? low-rank? smooth?).
3. Sweep λ on a log grid; plot train vs val curves.
4. Inspect coefficients / sparsity / norms—not only scalar loss.
5. For iterative methods, jointly consider early stopping and explicit penalties.
6. Report test metrics **once** with uncertainty if possible.

Worked example 9 — validation curve reading

If train loss $\ll$ val loss and gap grows with capacity, overfit: increase λ, reduce features, or add data. If both losses high, underfit: decrease λ or enrich model class.

Worked example 10 — calibration note

Regularization that improves 0-1 accuracy may still hurt probability calibration; for decision systems, track proper scoring rules (log loss, Brier) too.

11. CS/systems connections

- **Ill-conditioned least squares** in graphics/scientific computing: Tikhonov regularization
- **Compressed sensing:** ℓ_1 recovery of sparse signals under RIP
- **Recommender systems:** regularized matrix factorization
- **Control:** process noise / state penalties analogous to priors
- **Robust statistics:** penalties related to influence function control

12. Pitfalls

1. **Tuning λ on the test set** — leaks and invalidates claims.
2. **Comparing unnormalized features** — ℓ_2 penalty assumes comparable scales; standardize.
3. **Thinking lasso always selects the "true" features** — inconsistent under strong correlations without conditions (irrepresentable condition, etc.).
4. **Ignoring implicit bias** — different optimizers $\neq$ same solution in overparameterized models.
5. **Only watching training loss** — will systematically overfit.
6. **Huge λ** — model collapses to prior; "stable" but useless.

13. Checkpoint

- Write regularized ERM and its constrained twin
- Derive ridge closed form and explain conditioning
- Explain ℓ_1 vs ℓ_2 geometry and sparsity
- Describe bias–variance vs λ and how to choose λ
- Name two implicit regularizers (early stopping, min-norm bias)

Exercises

Easy

1. Show that $\lambda\|w\|_2^2$ is equivalent (via Lagrange) to constraining $\|w\|_2 \le \tau$ for appropriate paired (λ, τ) at optimality (sketch).
2. Compute ridge solution for 1D: $x_i \in \mathbb{R}$, closed form $w = (\sum x_i^2 + \lambda)^{-1} \sum x_i y_i$.
3. Apply soft-thresholding $S_{0.3}$ to $(1.0, -0.2, 0.5)$.
4. Why standardize features before ridge/lasso?
5. Sketch train and val error vs λ for a typical well-specified problem.

Medium

6. Derive the normal equations for ridge from $\nabla_w = 0$.
7. Prove that for $\lambda > 0$, ridge objective is strongly convex even if $X^\top X$ is singular.
8. Explain why elastic net can select groups of correlated features more stably than lasso (geometry/heuristic OK).
9. Show that GD on $\frac{1}{2}\|Xw - y\|_2^2$ from $w_0 = 0$ stays in the row space of X (span of feature vectors)—link to min-norm solution.
10. Design a 5-fold CV protocol for choosing λ; list leakage mistakes to avoid.

Challenge

11. For orthogonal design $X^\top X = nI$, derive lasso soft-thresholding explicitly.
12. Read/prove a simple stability bound: regularization $\Rightarrow$ algorithmic stability $\Rightarrow$ generalization (outline).
13. Compare explicit ridge vs early-stopped GD on a diagonal least-squares problem in the eigenbasis (spectral filters $1/(\sigma^2 + \lambda)$ vs $(1 - (1 - \alpha\sigma^2)^k)$).
14. Nuclear norm regularized matrix completion: write the optimization problem and one proximal-gradient step sketch.
15. Discuss double descent: when might increasing model size **and** decreasing λ both improve test error?

Summary

Regularization encodes inductive bias—shrinkage, sparsity, low rank—into the objective or algorithm. Ridge stabilizes and shrinks; lasso selects; early stopping and optimizer bias regularize even without penalties. Generalization is about controlling the gap to true risk: use validation, respect feature scaling, and remember that modern overparameterized regimes still use regularizers, just sometimes implicit ones.

First-Order Methods Playbook

Between the pure math of convexity and production training loops: a **playbook** of first-order algorithms, step sizes, projections, and diagnostics. Complements gradient-methods material with recipes you can apply to convex problems and deep learning alike.

Diagram: algorithm family

```
gradient available?

    yes

     v
full-batch GD    SGD / mini-batch    momentum / Adam

        projection / proximal for constraints & L1
```

1. Problem template

Minimize $f : \mathbb{R}^d \to \mathbb{R}$ (possibly $f = \hat{R}_n$ empirical risk). **First-order** methods use ∇f (or unbiased stochastic estimates), not Hessians.

Assumptions you will see in rates:

- **L-smooth:** $\|\nabla f(x) - \nabla f(y)\| \le L\|x - y\|$
- **μ-strongly convex:** $f(y) \ge f(x) + \langle \nabla f(x), y - x\rangle + \frac{\mu}{2}\|y - x\|^2$
- **Convex** but not strongly: rates slow from linear to sublinear

2. Gradient descent (GD)

$$x_{k+1} = x_k - \eta \nabla f(x_k).$$

Step size rules

Rule	When
Constant $\eta \in (0, 2/L)$	L-smooth; $\eta = 1/L$ classic
Exact / backtracking line search	unknown L; Armijo condition
Diminishing η_k	stochastic or non-smooth setups
Learning-rate schedules	ML: cosine, step decay, warmup

Worked example 1 — 1D quadratic

$f(x) = \frac{1}{2}Lx^2$, $\nabla f = Lx$.
$x_{k+1} = (1 - \eta L)x_k$.

- $0 < \eta < 2/L$: contracts to 0
- $\eta = 1/L$: one step to optimum from any start
- $\eta > 2/L$: diverges / oscillates with growing amplitude

Complexity intuition (smooth convex)

To reach $f(x_k) - f^\star \le \varepsilon$, GD needs $\mathcal{O}(L\|x_0 - x^\star\|^2/\varepsilon)$ iterations (order of magnitude). **Strongly convex + smooth:** linear rate $\mathcal{O}((1 - \mu/L)^k)$ — condition number $\kappa = L/\mu$ controls speed.

3. Stochastic gradient descent (SGD)

$$x_{k+1} = x_k - \eta_k g_k, \qquad \mathbb{E}[g_k \mid x_k] = \nabla f(x_k)$$

(unbiased noisy gradient). Mini-batch: average g over B samples — variance scales $\sim 1/B$.

Batch size	Pros	Cons
$B = 1$	cheap updates, noise exploration	high variance, hardware underuse
Medium B	parallelism, stable	tune η with B
Full batch	true gradient	costly; can stick in sharp regions

Worked example 2 — empirical risk

$f(w) = \frac{1}{n}\sum_i \ell_i(w)$, $g_k = \nabla \ell_{i_k}(w_k)$ for random i_k. Unbiased if uniform i_k.

Schedules

- Constant η small: approaches a noise ball
- $\eta_k \sim 1/\sqrt{k}$ or $1/k$: classic convergent schedules
- **Warmup then decay:** stabilizes early deep-net training

4. Momentum and acceleration

Heavy ball / classical momentum

$$v_{k+1} = \beta v_k + \nabla f(x_k), \qquad x_{k+1} = x_k - \eta v_{k+1}$$

(with $\beta \in [0, 1)$, e.g. 0.9). Velocity accumulates consistent gradient directions and damps zigzagging in narrow valleys.

```
zigzag GD:    /\/\/\/
momentum:       →  smoother
```

Nesterov acceleration (convex smooth)

Evaluates gradient at a look-ahead point; achieves optimal $\mathcal{O}(1/k^2)$ rate among first-order methods for smooth convex problems (in the standard oracle model).

Worked example 3 — valley

$f(x, y) = \frac{1}{2}(x^2 + 100y^2)$: GD zigzags in y; momentum carries progress along x.

5. Adaptive methods (Adam sketch)

Maintain exponential moving averages of gradient m_k and squared gradient v_k; step

$$x_{k+1} = x_k - \eta \frac{\hat{m}_k}{\sqrt{\hat{v}_k} + \varepsilon}$$

(with bias correction). Per-coordinate scaling helps sparse/noisy gradients.

Practice notes:

- Default for many deep nets

- On simple convex problems, carefully tuned SGD/momentum often matches or beats final training loss
- **AdamW:** decouple weight decay from adaptive scaling
- Watch for large η + sharp minima late in training — decay η

6. Constraints and nonsmooth regularizers

Projected gradient

$$x_{k+1} = \Pi_C(x_k - \eta \nabla f(x_k)),$$

Π_C Euclidean projection onto closed convex C (simplex, box, PSD cone with care).

Proximal gradient (composite objectives)

For f smooth + h nonsmooth convex (e.g. $h = \lambda \|x\|_1$):

$$x_{k+1} = \operatorname{prox}_{\eta h}(x_k - \eta \nabla f(x_k)).$$

Soft-thresholding for $h = \lambda \| \cdot \|_1$:

$$\left(\operatorname{prox}_{\eta\lambda\|\cdot\|_1}(z)\right)_j = \operatorname{sign}(z_j)\max(|z_j| - \eta\lambda, 0).$$

Shrinks coordinates; exact zeros when small — the algorithmic heart of lasso.

Mirror descent / natural gradients (pointer)

Geometry-aware first-order steps (Bregman divergences, Fisher metric) — use when parameters live on probability simplices or when preconditioning is structural.

7. Gradient clipping and stability (ML)

When $\|\nabla f\|$ is huge (RNNs, transformers early training):

$$g \leftarrow g \cdot \min\Big(1, \frac{\tau}{\|g\|}\Big).$$

Not a textbook convex algorithm, but a practical stabilizer. Combine with smaller η, better init, normalization layers.

8. Diagnostics checklist

Symptom	Likely causes	Try
Loss $\to$ NaN/Inf	η too large, log(0), bad data	lower η, grad clip, sanitize inputs
No progress	η too small, bug, zero grads	raise η, check shapes / `detach`
Train$\downarrow$ val$\uparrow$	overfit	regularize, early stop, more data
Oscillation	η large, bad conditioning	lower η, momentum, precondition
Dead ReLU	bad init, high LR	Leaky ReLU, lower η, init
Exploding activations	depth, residual missing	norm layers, residuals, clip

Worked example 4 — learning rate range test

Sweep η on a log grid for a few hundred steps; plot loss vs η; pick η before the explosion region (Smith LR range test idea).

9. Complexity snapshot (order-of-magnitude)

Setting	Iterations (oracle complexity flavor)
Smooth convex GD	$\mathcal{O}(1/\varepsilon)$
Smooth strongly convex GD	$\mathcal{O}(\kappa \log(1/\varepsilon))$
Nesterov smooth convex	$\mathcal{O}(1/\sqrt{\varepsilon})$
SGD convex (diminishing)	$\mathcal{O}(1/\varepsilon^2)$ typical
Finite-sum variance reduction	improved dependence on n

Constants and assumptions matter; use this as orientation, not a warranty.

10. When to go second-order

Newton / quasi-Newton (L-BFGS) worth it when:

- d moderate

- f smooth, expensive to evaluate but d^3 or d^2 acceptable
- High-accuracy solutions required (scientific computing)

Avoid pure Newton on deep nets (d millions+); use first-order + adaptive + normalization.

11. Practical training recipe (supervised ML)

1. Standardize / normalize inputs; sanity-check labels
2. Start with AdamW or SGD+momentum; log-spaced η
3. Weight decay + data augmentation as needed
4. Monitor train **and** validation; early stop on val
5. Decay η when val plateaus
6. Final test once

Worked example 5 — mini-batch scaling rule of thumb

If you multiply batch size by k, try scaling η by k (linear scaling rule) with warmup — not universal, but a common starting point.

12. Pitfalls

1. Tuning η on the test set
2. Comparing optimizers without equal tuning budget
3. Forgetting that full-batch GD and SGD minimize the **same** f in expectation but different paths
4. Using proximal operators with wrong step scaling
5. Interpreting training loss only — generalization is separate

13. Checkpoint

- Write GD and SGD updates

- State a safe constant step size for L-smooth f
- Explain momentum's effect on valleys
- Soft-threshold for L_1
- Run a diagnostic table mentally when loss misbehaves
- Know when adaptive methods help

Exercises

Easy

1. For $f(x) = \frac{1}{2}Lx^2$, what happens if $\eta > 2/L$ in GD?
2. One advantage and one disadvantage of mini-batch size 1 vs full batch.
3. Soft-thresholding $\text{prox}_{\lambda\|\cdot\|_1}$ — effect on coordinates?
4. Why might gradient *direction* be more trustworthy than step length early in training?
5. Name a problem class where Newton/L-BFGS is worth the cost.

Medium

6. Design a log-spaced η grid protocol on a validation split.
7. Show that for $f(x) = \frac{1}{2}x^\top Ax$ with $A \succ 0$, GD contracts in the eigenbasis when $0 < \eta < 2/\lambda_{\max}$.
8. Derive soft-thresholding as the proximal map of $\lambda\|x\|_1$ (coordinatewise).
9. Explain bias correction in Adam (sketch).
10. Projected GD onto the probability simplex: describe the projection idea (sorting / threshold).

Challenge

11. Prove that for L-smooth f, $\eta = 1/L$ yields $f(x_{k+1}) \le f(x_k) - \frac{1}{2L}\|\nabla f(x_k)\|^2$.
12. Compare heavy-ball vs Nesterov on a 2D quadratic (simulate or analyze eigenvalues of the iteration matrix).
13. Variance of mini-batch gradients: show $1/B$ scaling under i.i.d. sampling with replacement.

14. Composite $f + h$: write the fixed-point equation of proximal gradient and connect to optimality $0 \in \nabla f(x) + \partial h(x)$.

15. Learning-rate warmup: argue why large η at step 0 with random init can be harmful.

Checks

1. Diverges / oscillates with increasing amplitude.

2. Shrinks coefficients; exact zero if below threshold.

Summary

First-order methods move opposite the gradient with a carefully chosen step. GD is the clean baseline; SGD scales to data; momentum and adaptive methods stabilize and accelerate practice; projections and prox handle constraints and L_1. Most training failures are step size, conditioning, or data bugs — use the diagnostics table, validate honestly, and match the algorithm to smoothness, stochasticity, and scale.

Gradient-Based Optimization Methods

Gradient-based methods are the backbone of modern machine learning and optimization. They use derivative information to iteratively find optimal solutions, making them essential for training neural networks and solving large-scale optimization problems.

Gradient Descent Fundamentals

Basic Gradient Descent

The gradient descent algorithm updates parameters in the direction of steepest descent:

$$x_{k+1} = x_k - \quad f(x_k)$$

Where: - is the learning rate (step size) - f(x_k) is the gradient at point x_k

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

def gradient_descent_basic(f, grad_f, x0, learning_rate=0.01, max_iterations=1000, tolerance
    """Basic gradient descent implementation"""
    x = x0.copy()
    history = [x.copy()]

    for i in range(max_iterations):
        gradient = grad_f(x)
        x_new = x - learning_rate * gradient

        # Check convergence
        if np.linalg.norm(x_new - x) < tolerance:
            print(f"Converged after {i+1} iterations")
            break

        x = x_new
        history.append(x.copy())

    return x, np.array(history)

# Example: Minimize f(x,y) = x² + 2y²
def quadratic_function(x):
```

```
    return x[0]**2 + 2*x[1]**2

def quadratic_gradient(x):
    return np.array([2*x[0], 4*x[1]])

# Optimize
x0 = np.array([3.0, 2.0])
optimal_x, history = gradient_descent_basic(quadratic_function, quadratic_gradient, x0, lear

print(f"Starting point: {x0}")
print(f"Optimal point: {optimal_x}")
print(f"Function value: {quadratic_function(optimal_x):.6f}")

# Visualize optimization path
x = np.linspace(-4, 4, 100)
y = np.linspace(-3, 3, 100)
X, Y = np.meshgrid(x, y)
Z = X**2 + 2*Y**2

plt.figure(figsize=(12, 5))

# 2D contour plot
plt.subplot(1, 2, 1)
contours = plt.contour(X, Y, Z, levels=20)
plt.plot(history[:, 0], history[:, 1], 'ro-', markersize=4, linewidth=2, label='Optimization
plt.plot(optimal_x[0], optimal_x[1], 'g*', markersize=15, label='Optimum')
plt.xlabel('x')
plt.ylabel('y')
plt.title('Gradient Descent Path')
plt.legend()
plt.grid(True, alpha=0.3)

# 3D surface plot
ax = plt.subplot(1, 2, 2, projection='3d')
ax.plot_surface(X, Y, Z, alpha=0.6, cmap='viridis')
ax.plot(history[:, 0], history[:, 1], [quadratic_function(h) for h in history],
        'ro-', markersize=4, linewidth=2, label='Optimization path')
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('f(x,y)')
ax.set_title('3D Optimization Path')

plt.tight_layout()
plt.show()
```

Learning Rate Analysis

```
def analyze_learning_rates():
    """Analyze the effect of different learning rates"""

    learning_rates = [0.01, 0.1, 0.5, 0.9]
    x0 = np.array([3.0, 2.0])

    plt.figure(figsize=(15, 10))

    for i, lr in enumerate(learning_rates):
        optimal_x, history = gradient_descent_basic(
            quadratic_function, quadratic_gradient, x0,
            learning_rate=lr, max_iterations=50
        )

        plt.subplot(2, 2, i+1)

        # Plot contours
        x = np.linspace(-4, 4, 100)
        y = np.linspace(-3, 3, 100)
        X, Y = np.meshgrid(x, y)
        Z = X**2 + 2*Y**2
        plt.contour(X, Y, Z, levels=15, alpha=0.6)

        # Plot optimization path
        plt.plot(history[:, 0], history[:, 1], 'ro-', markersize=3, linewidth=1.5)
        plt.plot(0, 0, 'g*', markersize=15, label='True optimum')
        plt.xlabel('x')
        plt.ylabel('y')
        plt.title(f'Learning Rate = {lr}')
        plt.grid(True, alpha=0.3)
        plt.legend()

        # Print convergence info
        final_error = np.linalg.norm(optimal_x)
        print(f"LR = {lr}: Final error = {final_error:.6f}, Iterations = {len(history)}")

    plt.tight_layout()
    plt.show()

analyze_learning_rates()
```

Advanced Gradient Methods

Momentum

Momentum helps accelerate convergence and reduces oscillations:

$$v_{k+1} = \quad v_k + \quad f(x_k)\ x_{k+1} = x_k - v_{k+1}$$

```
def gradient_descent_momentum(f, grad_f, x0, learning_rate=0.01, momentum=0.9,
                              max_iterations=1000, tolerance=1e-6):
    """Gradient descent with momentum"""
    x = x0.copy()
    velocity = np.zeros_like(x)
    history = [x.copy()]

    for i in range(max_iterations):
        gradient = grad_f(x)
        velocity = momentum * velocity + learning_rate * gradient
        x_new = x - velocity

        if np.linalg.norm(x_new - x) < tolerance:
            print(f"Momentum GD converged after {i+1} iterations")
            break

        x = x_new
        history.append(x.copy())

    return x, np.array(history)

# Compare standard GD vs momentum
def compare_momentum():
    """Compare gradient descent with and without momentum"""

    # Ill-conditioned quadratic: f(x,y) = 10x² + y²
    def ill_conditioned_f(x):
        return 10*x[0]**2 + x[1]**2

    def ill_conditioned_grad(x):
        return np.array([20*x[0], 2*x[1]])

    x0 = np.array([2.0, 2.0])

    # Standard gradient descent
    opt_std, hist_std = gradient_descent_basic(
        ill_conditioned_f, ill_conditioned_grad, x0,
        learning_rate=0.05, max_iterations=100
    )
```

```
# Gradient descent with momentum
opt_mom, hist_mom = gradient_descent_momentum(
    ill_conditioned_f, ill_conditioned_grad, x0,
    learning_rate=0.05, momentum=0.9, max_iterations=100
)

# Visualize comparison
plt.figure(figsize=(15, 5))

# Create contour plot
x = np.linspace(-3, 3, 100)
y = np.linspace(-3, 3, 100)
X, Y = np.meshgrid(x, y)
Z = 10*X**2 + Y**2

# Standard GD
plt.subplot(1, 3, 1)
plt.contour(X, Y, Z, levels=20)
plt.plot(hist_std[:, 0], hist_std[:, 1], 'ro-', markersize=3, label='Standard GD')
plt.plot(0, 0, 'g*', markersize=15, label='Optimum')
plt.title('Standard Gradient Descent')
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.grid(True, alpha=0.3)

# Momentum GD
plt.subplot(1, 3, 2)
plt.contour(X, Y, Z, levels=20)
plt.plot(hist_mom[:, 0], hist_mom[:, 1], 'bo-', markersize=3, label='Momentum GD')
plt.plot(0, 0, 'g*', markersize=15, label='Optimum')
plt.title('Gradient Descent with Momentum')
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.grid(True, alpha=0.3)

# Convergence comparison
plt.subplot(1, 3, 3)
std_errors = [np.linalg.norm(h) for h in hist_std]
mom_errors = [np.linalg.norm(h) for h in hist_mom]

plt.semilogy(std_errors, 'r-', label='Standard GD', linewidth=2)
plt.semilogy(mom_errors, 'b-', label='Momentum GD', linewidth=2)
plt.xlabel('Iteration')
plt.ylabel('Distance to Optimum (log scale)')
```

```
    plt.title('Convergence Comparison')
    plt.legend()
    plt.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.show()

    print(f"Standard GD: {len(hist_std)} iterations, final error: {np.linalg.norm(opt_std):.
    print(f"Momentum GD: {len(hist_mom)} iterations, final error: {np.linalg.norm(opt_mom):.

compare_momentum()
```

Adaptive Learning Rates

AdaGrad

AdaGrad adapts the learning rate based on historical gradients:

```
def adagrad(f, grad_f, x0, learning_rate=0.1, epsilon=1e-8, max_iterations=1000, tolerance=1
    """AdaGrad optimizer"""
    x = x0.copy()
    G = np.zeros_like(x)  # Accumulated squared gradients
    history = [x.copy()]

    for i in range(max_iterations):
        gradient = grad_f(x)
        G += gradient**2

        # Adaptive learning rate
        adapted_lr = learning_rate / (np.sqrt(G) + epsilon)
        x_new = x - adapted_lr * gradient

        if np.linalg.norm(x_new - x) < tolerance:
            print(f"AdaGrad converged after {i+1} iterations")
            break

        x = x_new
        history.append(x.copy())

    return x, np.array(history)
```

Adam (Adaptive Moment Estimation)

Adam combines momentum with adaptive learning rates:

```
def adam(f, grad_f, x0, learning_rate=0.001, beta1=0.9, beta2=0.999,
         epsilon=1e-8, max_iterations=1000, tolerance=1e-6):
    """Adam optimizer"""
    x = x0.copy()
    m = np.zeros_like(x)  # First moment estimate
    v = np.zeros_like(x)  # Second moment estimate
    history = [x.copy()]

    for i in range(max_iterations):
        gradient = grad_f(x)

        # Update biased first moment estimate
        m = beta1 * m + (1 - beta1) * gradient

        # Update biased second raw moment estimate
        v = beta2 * v + (1 - beta2) * gradient**2

        # Compute bias-corrected first moment estimate
        m_hat = m / (1 - beta1**(i + 1))

        # Compute bias-corrected second raw moment estimate
        v_hat = v / (1 - beta2**(i + 1))

        # Update parameters
        x_new = x - learning_rate * m_hat / (np.sqrt(v_hat) + epsilon)

        if np.linalg.norm(x_new - x) < tolerance:
            print(f"Adam converged after {i+1} iterations")
            break

        x = x_new
        history.append(x.copy())

    return x, np.array(history)

def compare_optimizers():
    """Compare different optimization algorithms"""

    # Rosenbrock function: f(x,y) = (a-x)² + b(y-x²)²
    def rosenbrock(x, a=1, b=100):
        return (a - x[0])**2 + b * (x[1] - x[0]**2)**2

    def rosenbrock_grad(x, a=1, b=100):
        dx = -2*(a - x[0]) - 4*b*x[0]*(x[1] - x[0]**2)
        dy = 2*b*(x[1] - x[0]**2)
        return np.array([dx, dy])
```

```
x0 = np.array([-1.0, 1.0])

# Run different optimizers
optimizers = {
    'GD': lambda: gradient_descent_basic(rosenbrock, rosenbrock_grad, x0, 0.001, 2000),
    'Momentum': lambda: gradient_descent_momentum(rosenbrock, rosenbrock_grad, x0, 0.001
    'AdaGrad': lambda: adagrad(rosenbrock, rosenbrock_grad, x0, 0.1, max_iterations=2000
    'Adam': lambda: adam(rosenbrock, rosenbrock_grad, x0, 0.01, max_iterations=2000)
}

results = {}
for name, optimizer in optimizers.items():
    opt_x, history = optimizer()
    results[name] = {
        'optimum': opt_x,
        'history': history,
        'final_value': rosenbrock(opt_x)
    }

# Visualize results
plt.figure(figsize=(15, 10))

# Create Rosenbrock function contour
x = np.linspace(-2, 2, 100)
y = np.linspace(-1, 3, 100)
X, Y = np.meshgrid(x, y)
Z = (1 - X)**2 + 100 * (Y - X**2)**2

colors = ['red', 'blue', 'green', 'orange']

for i, (name, result) in enumerate(results.items()):
    plt.subplot(2, 2, i+1)
    plt.contour(X, Y, Z, levels=np.logspace(-1, 3, 20))

    history = result['history']
    plt.plot(history[:, 0], history[:, 1], 'o-', color=colors[i],
             markersize=2, linewidth=1, alpha=0.7, label=name)
    plt.plot(1, 1, 'r*', markersize=15, label='Global minimum')

    plt.xlabel('x')
    plt.ylabel('y')
    plt.title(f'{name} Optimizer')
    plt.legend()
    plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

```
    # Print results summary
    print("Optimizer Comparison Results:")
    print("=" * 50)
    print(f"{'Optimizer':>10} {'Iterations':>12} {'Final Value':>15} {'Distance to Opt':>15}")
    print("-" * 50)

    for name, result in results.items():
        iterations = len(result['history'])
        final_value = result['final_value']
        distance = np.linalg.norm(result['optimum'] - np.array([1, 1]))
        print(f"{name:>10} {iterations:>12} {final_value:>15.6f} {distance:>15.6f}")

compare_optimizers()
```

Stochastic Gradient Descent

Mini-batch SGD

For large datasets, we use mini-batch stochastic gradient descent:

```
def stochastic_gradient_descent(X, y, loss_fn, grad_fn, theta0,
                                learning_rate=0.01, batch_size=32,
                                epochs=100, shuffle=True):
    """Mini-batch stochastic gradient descent"""
    theta = theta0.copy()
    n_samples = X.shape[0]
    history = {'loss': [], 'theta': []}

    for epoch in range(epochs):
        # Shuffle data
        if shuffle:
            indices = np.random.permutation(n_samples)
            X_shuffled = X[indices]
            y_shuffled = y[indices]
        else:
            X_shuffled, y_shuffled = X, y

        epoch_loss = 0
        n_batches = 0

        # Process mini-batches
        for i in range(0, n_samples, batch_size):
            batch_X = X_shuffled[i:i+batch_size]
            batch_y = y_shuffled[i:i+batch_size]
```

```
            # Compute gradient on mini-batch
            gradient = grad_fn(theta, batch_X, batch_y)

            # Update parameters
            theta = theta - learning_rate * gradient

            # Track loss
            batch_loss = loss_fn(theta, batch_X, batch_y)
            epoch_loss += batch_loss
            n_batches += 1

        # Record history
        avg_loss = epoch_loss / n_batches
        history['loss'].append(avg_loss)
        history['theta'].append(theta.copy())

        if epoch % 10 == 0:
            print(f"Epoch {epoch}: Loss = {avg_loss:.6f}")

    return theta, history

# Example: Linear regression with SGD
def linear_regression_sgd_example():
    """Linear regression using SGD"""

    # Generate synthetic data
    np.random.seed(42)
    n_samples, n_features = 1000, 5
    X = np.random.randn(n_samples, n_features)
    true_theta = np.random.randn(n_features)
    y = X @ true_theta + 0.1 * np.random.randn(n_samples)

    # Add bias term
    X = np.column_stack([np.ones(n_samples), X])
    true_theta = np.concatenate([[0], true_theta])

    # Define loss and gradient functions
    def mse_loss(theta, X, y):
        predictions = X @ theta
        return np.mean((predictions - y)**2)

    def mse_gradient(theta, X, y):
        predictions = X @ theta
        return 2 * X.T @ (predictions - y) / len(y)

    # Initialize parameters
```

```
    theta0 = np.random.randn(X.shape[1]) * 0.1

    # Run SGD
    theta_sgd, history = stochastic_gradient_descent(
        X, y, mse_loss, mse_gradient, theta0,
        learning_rate=0.01, batch_size=32, epochs=100
    )

    # Compare with analytical solution
    theta_analytical = np.linalg.solve(X.T @ X, X.T @ y)

    print(f"\nResults Comparison:")
    print(f"True parameters: {true_theta}")
    print(f"SGD parameters: {theta_sgd}")
    print(f"Analytical solution: {theta_analytical}")
    print(f"SGD error: {np.linalg.norm(theta_sgd - true_theta):.6f}")
    print(f"Analytical error: {np.linalg.norm(theta_analytical - true_theta):.6f}")

    # Plot convergence
    plt.figure(figsize=(12, 5))

    plt.subplot(1, 2, 1)
    plt.plot(history['loss'])
    plt.xlabel('Epoch')
    plt.ylabel('Loss')
    plt.title('SGD Convergence')
    plt.grid(True, alpha=0.3)

    plt.subplot(1, 2, 2)
    theta_history = np.array(history['theta'])
    for i in range(len(true_theta)):
        plt.plot(theta_history[:, i], label=f' _{i}')
        plt.axhline(true_theta[i], color=f'C{i}', linestyle='--', alpha=0.7)

    plt.xlabel('Epoch')
    plt.ylabel('Parameter Value')
    plt.title('Parameter Evolution')
    plt.legend()
    plt.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.show()

linear_regression_sgd_example()
```

Second-Order Methods

Newton's Method

Newton's method uses second-order information (Hessian matrix):

$$x_{k+1} = x_k - H^{-1}(x_k) \nabla f(x_k)$$

```
def newtons_method(f, grad_f, hessian_f, x0, max_iterations=100, tolerance=1e-6):
    """Newton's method for optimization"""
    x = x0.copy()
    history = [x.copy()]

    for i in range(max_iterations):
        gradient = grad_f(x)
        hessian = hessian_f(x)

        # Check if Hessian is positive definite
        eigenvals = np.linalg.eigvals(hessian)
        if np.any(eigenvals <= 0):
            print(f"Warning: Hessian not positive definite at iteration {i}")

        # Newton step
        try:
            newton_step = np.linalg.solve(hessian, gradient)
            x_new = x - newton_step
        except np.linalg.LinAlgError:
            print(f"Singular Hessian at iteration {i}")
            break

        if np.linalg.norm(x_new - x) < tolerance:
            print(f"Newton's method converged after {i+1} iterations")
            break

        x = x_new
        history.append(x.copy())

    return x, np.array(history)

# Example: Compare Newton's method with gradient descent
def compare_newton_gd():
    """Compare Newton's method with gradient descent"""

    # Quadratic function with Hessian
    def quad_hessian(x):
        return np.array([[2, 0], [0, 4]])
```

```
x0 = np.array([3.0, 2.0])

# Newton's method
opt_newton, hist_newton = newtons_method(
    quadratic_function, quadratic_gradient, quad_hessian, x0
)

# Gradient descent
opt_gd, hist_gd = gradient_descent_basic(
    quadratic_function, quadratic_gradient, x0, learning_rate=0.1
)

# Visualize comparison
plt.figure(figsize=(15, 5))

# Create contour plot
x = np.linspace(-4, 4, 100)
y = np.linspace(-3, 3, 100)
X, Y = np.meshgrid(x, y)
Z = X**2 + 2*Y**2

# Newton's method
plt.subplot(1, 3, 1)
plt.contour(X, Y, Z, levels=20)
plt.plot(hist_newton[:, 0], hist_newton[:, 1], 'ro-', markersize=6, linewidth=2, label="
plt.plot(0, 0, 'g*', markersize=15, label='Optimum')
plt.title("Newton's Method")
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.grid(True, alpha=0.3)

# Gradient descent
plt.subplot(1, 3, 2)
plt.contour(X, Y, Z, levels=20)
plt.plot(hist_gd[:, 0], hist_gd[:, 1], 'bo-', markersize=3, linewidth=1, label='Gradient
plt.plot(0, 0, 'g*', markersize=15, label='Optimum')
plt.title('Gradient Descent')
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.grid(True, alpha=0.3)

# Convergence comparison
plt.subplot(1, 3, 3)
newton_errors = [np.linalg.norm(h) for h in hist_newton]
```

```
    gd_errors = [np.linalg.norm(h) for h in hist_gd]

    plt.semilogy(newton_errors, 'ro-', label="Newton's Method", linewidth=2)
    plt.semilogy(gd_errors, 'bo-', label='Gradient Descent', linewidth=2)
    plt.xlabel('Iteration')
    plt.ylabel('Distance to Optimum (log scale)')
    plt.title('Convergence Comparison')
    plt.legend()
    plt.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.show()

    print(f"Newton's method: {len(hist_newton)} iterations")
    print(f"Gradient descent: {len(hist_gd)} iterations")
    print(f"Newton's method final error: {np.linalg.norm(opt_newton):.10f}")
    print(f"Gradient descent final error: {np.linalg.norm(opt_gd):.10f}")

compare_newton_gd()
```

Quasi-Newton Methods (BFGS)

BFGS approximates the Hessian using gradient information:

```
def bfgs(f, grad_f, x0, max_iterations=100, tolerance=1e-6):
    """BFGS quasi-Newton method"""
    x = x0.copy()
    n = len(x)
    B = np.eye(n)  # Initial Hessian approximation
    history = [x.copy()]

    gradient = grad_f(x)

    for i in range(max_iterations):
        # Solve B * p = -gradient for search direction p
        p = -np.linalg.solve(B, gradient)

        # Line search (simplified - use fixed step size)
        alpha = 1.0
        x_new = x + alpha * p
        gradient_new = grad_f(x_new)

        # Check convergence
        if np.linalg.norm(gradient_new) < tolerance:
            print(f"BFGS converged after {i+1} iterations")
            break
```

```
        # BFGS update
        s = x_new - x
        y = gradient_new - gradient

        # Avoid division by zero
        if np.dot(s, y) > 1e-10:
            rho = 1.0 / np.dot(s, y)
            I = np.eye(n)
            B = (I - rho * np.outer(s, y)) @ B @ (I - rho * np.outer(y, s)) + rho * np.outer

        x = x_new
        gradient = gradient_new
        history.append(x.copy())

    return x, np.array(history)

# Compare BFGS with other methods
def compare_all_methods():
    """Compare all optimization methods"""

    x0 = np.array([3.0, 2.0])

    methods = {
        'Gradient Descent': lambda: gradient_descent_basic(quadratic_function, quadratic_gra
        'Newton': lambda: newtons_method(quadratic_function, quadratic_gradient,
                                         lambda x: np.array([[2, 0], [0, 4]]), x0),
        'BFGS': lambda: bfgs(quadratic_function, quadratic_gradient, x0),
        'Adam': lambda: adam(quadratic_function, quadratic_gradient, x0, 0.1)
    }

    plt.figure(figsize=(12, 8))
    colors = ['red', 'blue', 'green', 'orange']

    # Create contour plot
    x = np.linspace(-4, 4, 100)
    y = np.linspace(-3, 3, 100)
    X, Y = np.meshgrid(x, y)
    Z = X**2 + 2*Y**2
    plt.contour(X, Y, Z, levels=15, alpha=0.6)

    for i, (name, method) in enumerate(methods.items()):
        opt_x, history = method()
        plt.plot(history[:, 0], history[:, 1], 'o-', color=colors[i],
                 markersize=4, linewidth=2, label=f'{name} ({len(history)} iter)')

    plt.plot(0, 0, 'k*', markersize=15, label='Optimum')
```

```
    plt.xlabel('x')
    plt.ylabel('y')
    plt.title('Optimization Methods Comparison')
    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.show()

compare_all_methods()
```

Applications in Machine Learning

Logistic Regression

```
def logistic_regression_optimization():
    """Optimize logistic regression using different methods"""

    # Generate binary classification data
    np.random.seed(42)
    n_samples, n_features = 1000, 2
    X = np.random.randn(n_samples, n_features)
    true_w = np.array([1.5, -2.0])
    true_b = 0.5

    # Generate labels
    logits = X @ true_w + true_b
    probabilities = 1 / (1 + np.exp(-logits))
    y = np.random.binomial(1, probabilities)

    # Add bias term
    X_with_bias = np.column_stack([np.ones(n_samples), X])
    true_params = np.array([true_b, true_w[0], true_w[1]])

    # Logistic regression loss and gradient
    def logistic_loss(params, X, y):
        logits = X @ params
        return np.mean(np.log(1 + np.exp(logits)) - y * logits)

    def logistic_gradient(params, X, y):
        logits = X @ params
        probabilities = 1 / (1 + np.exp(-logits))
        return X.T @ (probabilities - y) / len(y)

    # Initialize parameters
    params0 = np.random.randn(X_with_bias.shape[1]) * 0.1
```

```
# Optimize using different methods
methods = {
    'SGD': lambda: stochastic_gradient_descent(
        X_with_bias, y, logistic_loss, logistic_gradient, params0,
        learning_rate=0.1, batch_size=64, epochs=100
    ),
    'Adam': lambda: adam(
        lambda p: logistic_loss(p, X_with_bias, y),
        lambda p: logistic_gradient(p, X_with_bias, y),
        params0, learning_rate=0.01, max_iterations=1000
    )
}

results = {}
for name, method in methods.items():
    if name == 'SGD':
        params_opt, history = method()
        loss_history = history['loss']
    else:
        params_opt, param_history = method()
        loss_history = [logistic_loss(p, X_with_bias, y) for p in param_history]

    results[name] = {
        'params': params_opt,
        'loss_history': loss_history,
        'final_loss': logistic_loss(params_opt, X_with_bias, y)
    }

# Visualize results
plt.figure(figsize=(15, 5))

# Decision boundary visualization
plt.subplot(1, 3, 1)

# Plot data points
plt.scatter(X[y==0, 0], X[y==0, 1], c='red', marker='o', alpha=0.6, label='Class 0')
plt.scatter(X[y==1, 0], X[y==1, 1], c='blue', marker='s', alpha=0.6, label='Class 1')

# Plot true decision boundary
x_boundary = np.linspace(-3, 3, 100)
y_boundary = -(true_params[0] + true_params[1] * x_boundary) / true_params[2]
plt.plot(x_boundary, y_boundary, 'g--', linewidth=2, label='True boundary')

# Plot learned decision boundary (using Adam result)
adam_params = results['Adam']['params']
y_learned = -(adam_params[0] + adam_params[1] * x_boundary) / adam_params[2]
```

```
    plt.plot(x_boundary, y_learned, 'k-', linewidth=2, label='Learned boundary')

    plt.xlabel('Feature 1')
    plt.ylabel('Feature 2')
    plt.title('Logistic Regression Decision Boundary')
    plt.legend()
    plt.grid(True, alpha=0.3)

    # Loss convergence
    plt.subplot(1, 3, 2)
    for name, result in results.items():
        plt.plot(result['loss_history'], label=f'{name}', linewidth=2)

    plt.xlabel('Iteration/Epoch')
    plt.ylabel('Logistic Loss')
    plt.title('Loss Convergence')
    plt.legend()
    plt.grid(True, alpha=0.3)

    # Parameter convergence
    plt.subplot(1, 3, 3)
    print("Parameter Comparison:")
    print(f"True parameters: {true_params}")
    for name, result in results.items():
        params = result['params']
        error = np.linalg.norm(params - true_params)
        print(f"{name} parameters: {params}")
        print(f"{name} error: {error:.6f}")

        plt.bar(range(len(params)), params, alpha=0.7, label=f'{name}')

    plt.bar(range(len(true_params)), true_params, alpha=0.7,
            color='black', label='True', width=0.3)
    plt.xlabel('Parameter Index')
    plt.ylabel('Parameter Value')
    plt.title('Parameter Comparison')
    plt.legend()
    plt.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.show()

logistic_regression_optimization()
```

Summary

Gradient-based optimization methods are essential for:

1. **Machine Learning**: Training neural networks and other models
2. **Parameter Estimation**: Finding optimal model parameters
3. **Function Minimization**: Solving continuous optimization problems
4. **Large-Scale Optimization**: Handling high-dimensional problems

Key insights: - **Learning rate** is crucial for convergence - **Momentum** helps accelerate convergence and reduce oscillations - **Adaptive methods** (Adam, AdaGrad) automatically adjust learning rates - **Second-order methods** converge faster but require more computation - **Stochastic methods** are essential for large datasets

The choice of optimization method depends on: - Problem size and computational budget - Gradient availability and computational cost - Convergence requirements - Noise in the objective function

Understanding these methods enables you to select appropriate optimizers for different machine learning and optimization tasks.

Machine Learning Math

Mathematics for Machine Learning

Machine learning systems are optimization procedures applied to statistical models expressed with linear algebra and calculus. This part focuses on the mathematical objects that appear in almost every pipeline: data matrices, embeddings, losses, gradients, and the probabilistic semantics of prediction. The goal is not to catalog algorithms, but to make the math **legible** so you can read papers, debug training, and choose models deliberately.

Chapters in this part

Chapter	Focus
Linear algebra for ML	Data matrices, similarity, least squares view
Gradients and loss	Risk, GD/SGD, chain rule / backprop sketch
Probability for ML	Likelihood, MAP, calibration, softmax
Neural nets math sketch	Affine + nonlinearity, depth, gradients
Matrix calculus for ML	Practical identities and grad checks
Automatic differentiation	Dual numbers, the reverse-mode tape, cost asymmetry, adjoint Greeks

What an ML problem looks like mathematically

Data. n examples with inputs $x_i \in \mathbb{R}^d$ and targets $y_i \in \mathcal{Y}$. Stack inputs as

$$X = \begin{bmatrix} x_1^\top \\ \vdots \\ x_n^\top \end{bmatrix} \in \mathbb{R}^{n \times d}, \qquad y = \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix}.$$

Model. A parameterized map $f_w : \mathbb{R}^d \to \hat{y}$, e.g. linear $f_w(x) = w^\top x$, or a neural net $f_w = f^{(L)} \circ \cdots \circ f^{(1)}$.

Loss. $\ell(y, \hat{y})$ measures pointwise error (squared, logistic, cross-entropy, hinge, Huber, …).

Objective. Usually regularized empirical risk:

$$J(w) = \frac{1}{n} \sum_{i=1}^{n} \ell(y_i, f_w(x_i)) + \lambda \Omega(w).$$

Training. Compute $\widehat{w} \approx \arg\min_w J(w)$ by gradient methods, convex solvers, or specialized algorithms.

Evaluation. Estimate true risk $R(w) = \mathbb{E}[\ell(y, f_w(x))]$ on held-out data; optionally calibrate probabilities, measure fairness, latency, robustness.

Worked example 1 — linear regression in one line

$J(w) = \frac{1}{n}\|Xw - y\|_2^2 + \lambda\|w\|_2^2$ with $\nabla J(w) = \frac{2}{n}X^\top(Xw - y) + 2\lambda w$. Math used: matrix calculus, PSD Hessians, conditioning of $X^\top X + \lambda n I$.

The four pillars

1. Linear algebra

- Vectors as examples/embeddings; matrices as datasets, linear maps, batches
- Norms and inner products: similarity, margins, regularization
- Eigen and singular structure: PCA, whitening, recommendation, attention scores as Gram-like structure
- Least squares and pseudoinverses: normal equations, min-norm solutions

2. Calculus and optimization

- Gradients / Jacobians / Hessians
- Chain rule $\Leftrightarrow$ backpropagation
- Convex vs nonconvex training dynamics
- Step sizes, momentum, adaptive methods (see Optimization part)

3. Probability and statistics

- Likelihood and maximum likelihood $\Leftrightarrow$ many losses
- Bias–variance, estimation error, confidence
- Bayesian posteriors and regularization-as-prior
- Calibration and proper scoring rules

4. Information theory (cross-cutting)

- Cross-entropy loss as coding cost
- KL regularization (VAEs, knowledge distillation)
- Mutual information in representation learning

Supervised learning: decision theory sketch

Given features x, predict action a (or label $\hat{y}$) to minimize expected loss. Bayes optimal predictor depends on loss:

- 0-1 loss $\Rightarrow$ predict $\arg\max_y P(y \mid x)$
- Squared loss $\Rightarrow$ predict $\mathbb{E}[y \mid x]$
- Absolute loss $\Rightarrow$ predict median of $y \mid x$

Surrogate losses (hinge, logistic) upper-bound or stand in for 0-1 loss to make optimization tractable.

Worked example 2 — logistic regression as MLE

Bernoulli likelihood with $p_w(x) = \sigma(w^\top x)$ yields negative log-likelihood equal to binary cross-entropy. Convex in w for linear models.

Worked example 3 — softmax multiclass

$p_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$, $z = Wx$. Cross-entropy with one-hot labels: $\ell = -\sum_k y_k \log p_k$. Jacobian of softmax is diagonal$(p)-$pp$^\top$—central in backprop.

Unsupervised and self-supervised (mathematical themes)

- **PCA / SVD:** maximize variance of projections; equivalent to low-rank approximation in Frobenius norm (Eckart–Young)
- **Clustering:** k-means objective $\sum_i \min_j \|x_i - \mu_j\|_2^2$ (nonconvex; alternating minimization)
- **Density models:** maximize $\sum_i \log p_\theta(x_i)$
- **Contrastive learning:** score similarity of positive pairs vs negatives—related to MI lower bounds

Worked example 4 — PCA objective

$\max_{U^\top U = I_k} \operatorname{tr}(U^\top \hat{\Sigma} U)$ with $\hat{\Sigma} = \frac{1}{n} X^\top X$ (centered X). Solution: top k eigenvectors of $\hat{\Sigma}$.

Generalization: the mathematical tension

Training drives $\hat{R}_n(w)$ down; we care about $R(w)$. Bridging tools:

- Classical capacity control (VC, Rademacher)
- Stability and PAC-Bayes
- Overparameterized phenomena (double descent, benign overfitting)—still active research

Practical math: validation estimates of risk, confidence intervals for metrics, ablation as experimental design.

Worked example 5 — train/val split as Monte Carlo

Val loss is an unbiased estimate of risk for a **fixed** w. After you tune using val, that estimate becomes optimistic—hence a final test set or nested CV.

Roadmap in this book

This part	Focus
Linear algebra for ML	Data matrices, projections, SVD/PCA in learning, gradients of matrix objectives
Related parts	Calculus (`08`), Optimization (`11`), Probability advanced (`17`), Information theory (`10`), Numerical methods (`15`)

Read **Linear Algebra for ML** next for concrete embeddings of these ideas in vectors/matrices. Use Optimization and Probability parts as references when training dynamics or statistical claims appear.

Notation hygiene

- Scalars a, vectors v (column by default), matrices A
- $\nabla_w J$ gradient; $D_w f$ Jacobian when f is vector-valued
- $\|\cdot\|_2$ Euclidean; $\|\cdot\|_F$ Frobenius; $\|\cdot\|_*$ nuclear
- $\sigma(z) = 1/(1+e^{-z})$ sigmoid; softmax as above

Worked example 6 — shape checking

Batch matrix $X \in \mathbb{R}^{B\times d}$, weights $W \in \mathbb{R}^{d\times k}$, logits $XW \in \mathbb{R}^{B\times k}$. If shapes fail, the math—not the framework—should catch you first.

CS systems angle

ML math becomes systems when:

- **Batching** is matrix multiplication throughput (BLAS/GPU)
- **Conditioning** of $X^\top X$ affects convergence and numeric stability
- **Sparsity** of features changes feasible algorithms (sparse matvec)

- **Quantization** approximates w in low precision—error analysis matters
- **Distributed training** shards X or w; gradients average (data parallel)

Worked example 7 — complexity

One full-batch gradient of linear model: $O(nd)$ time. SGD with batch B: $O(Bd)$ per step. Wall-clock optimum depends on hardware, not only big-O of full gradient.

Pitfalls

1. **Confusing training loss with risk.** Always hold out data.
2. **Nonconvex ⇒ no global guarantees**—report seeds and variance.
3. **Leaky preprocessing** (scaling using test statistics).
4. **High-dimensional geometry:** distances concentrate; nearest neighbors need care.
5. **Probabilities from softmax are not calibrated** by default.
6. **Math model vs implementation:** broadcasting bugs are dimension errors.

Checkpoint

- Write ERM + regularization as a precise optimization problem
- Map squared / logistic / cross-entropy losses to statistical principles
- Identify which pillar (LA / calculus / probability / info) a paper subsection uses
- Explain why validation is a Monte Carlo estimate of risk
- Track tensor shapes for a linear layer batch

Exercises

1. Write $J(w)$ for ridge regression and compute $\nabla J(w)$.
2. Show that for linear regression with squared loss and X full column rank, the critical point is unique.
3. Derive binary cross-entropy from Bernoulli NLL.
4. For one-hot y and softmax p, show $\nabla_z \ell = p - y$.
5. Centered data matrix X: show that $\mathbf{1}^\top X = 0$ if rows sum appropriately—why center before PCA?
6. Give Bayes optimal predictor for absolute loss (median)—one paragraph.
7. Explain the difference between $R(w)$ and $\hat{R}_n(w)$ with an overfitting sketch.
8. Complexity: cost of computing $X^\top X$ for $X \in \mathbb{R}^{n \times d}$.
9. When is hinge loss preferred over logistic? (margins vs probabilities)
10. Map dropout to an approximate ensemble / regularization story (qualitative).
11. Write k-means objective and argue it is nonconvex by giving a simple ambiguity (label switching).
12. Information theory: express multiclass cross-entropy as $H(\hat{p}, p_{\text{model}})$ on the empirical label distribution in a batch.

13. Systems: why mixed-precision training needs loss scaling (numeric, not ML folklore only).
14. Take a paper abstract you know and list the math objects (X, loss, constraint, probabilistic model).
15. Checkpoint project: pick one supervised dataset and write the full mathematical training specification (model, loss, regularizer, optimizer assumptions) without code.

Summary

ML mathematics is the interplay of **representation** (linear algebra), **fitting** (optimization/calculus), and **uncertainty** (probability/information). Mastering the shared language lets you transfer insights across models—from linear baselines to deep networks—without treating frameworks as magic.

Linear Algebra for Machine Learning

Linear algebra is the backbone of machine learning. Most ML algorithms can be expressed as operations on vectors and matrices, making linear algebra essential for understanding and implementing these algorithms.

Vectors in Machine Learning

Vector Representation of Data

In ML, data points are typically represented as vectors:

```
import numpy as np
import matplotlib.pyplot as plt

# Example: House price prediction
# Features: [square_feet, bedrooms, bathrooms, age]
house1 = np.array([2000, 3, 2, 5])    # House 1
house2 = np.array([1500, 2, 1, 10])   # House 2
house3 = np.array([2500, 4, 3, 2])    # House 3

# Dataset as matrix (each row is a data point)
X = np.array([house1, house2, house3])
print("Dataset shape:", X.shape)
print("Dataset:\n", X)
```

Vector Operations in ML

Dot Product and Similarity

```
def cosine_similarity(v1, v2):
    """Compute cosine similarity between two vectors"""
    dot_product = np.dot(v1, v2)
    norms = np.linalg.norm(v1) * np.linalg.norm(v2)
    return dot_product / norms

# Example: Document similarity
doc1 = np.array([1, 2, 0, 1])  # Word frequencies
```

```
doc2 = np.array([2, 1, 1, 0])  # Word frequencies
doc3 = np.array([0, 0, 1, 2])  # Word frequencies

similarity_12 = cosine_similarity(doc1, doc2)
similarity_13 = cosine_similarity(doc1, doc3)

print(f"Similarity between doc1 and doc2: {similarity_12:.3f}")
print(f"Similarity between doc1 and doc3: {similarity_13:.3f}")
```

Distance Metrics

```
def euclidean_distance(v1, v2):
    """Euclidean distance between vectors"""
    return np.linalg.norm(v1 - v2)

def manhattan_distance(v1, v2):
    """Manhattan distance between vectors"""
    return np.sum(np.abs(v1 - v2))

# Example: K-Nearest Neighbors
def knn_classify(X_train, y_train, x_test, k=3):
    """Simple KNN classifier"""
    distances = [euclidean_distance(x_test, x_train) for x_train in X_train]
    k_indices = np.argsort(distances)[:k]
    k_labels = y_train[k_indices]
    return np.bincount(k_labels).argmax()

# Sample data
X_train = np.array([[1, 2], [2, 3], [3, 1], [6, 5], [7, 7], [8, 6]])
y_train = np.array([0, 0, 0, 1, 1, 1])  # Class labels
x_test = np.array([4, 4])

predicted_class = knn_classify(X_train, y_train, x_test, k=3)
print(f"Predicted class for {x_test}: {predicted_class}")

# Visualize
plt.figure(figsize=(8, 6))
colors = ['red', 'blue']
for i in range(2):
    mask = y_train == i
    plt.scatter(X_train[mask, 0], X_train[mask, 1],
                c=colors[i], label=f'Class {i}', s=100)

plt.scatter(x_test[0], x_test[1], c='green', marker='x', s=200, label='Test point')
plt.xlabel('Feature 1')
```

```
plt.ylabel('Feature 2')
plt.legend()
plt.title('K-Nearest Neighbors Classification')
plt.grid(True)
plt.show()
```

Matrices in Machine Learning

Data Representation

```
# Dataset matrix: rows = samples, columns = features
n_samples, n_features = 1000, 4
X = np.random.randn(n_samples, n_features)
y = np.random.randint(0, 2, n_samples)

print(f"Data matrix shape: {X.shape}")
print(f"Labels shape: {y.shape}")

# Feature statistics
print(f"Feature means: {np.mean(X, axis=0)}")
print(f"Feature std: {np.std(X, axis=0)}")
```

Matrix Operations in Linear Regression

```
class LinearRegression:
    def __init__(self):
        self.weights = None
        self.bias = None

    def fit(self, X, y):
        """Fit linear regression using normal equation"""
        # Add bias term (intercept)
        X_with_bias = np.column_stack([np.ones(X.shape[0]), X])

        # Normal equation:   = (X^T X)^(-1) X^T y
        XtX = X_with_bias.T @ X_with_bias
        Xty = X_with_bias.T @ y
        theta = np.linalg.solve(XtX, Xty)

        self.bias = theta[0]
        self.weights = theta[1:]

    def predict(self, X):
```

```
        """Make predictions"""
        return X @ self.weights + self.bias

    def score(self, X, y):
        """R-squared score"""
        y_pred = self.predict(X)
        ss_res = np.sum((y - y_pred) ** 2)
        ss_tot = np.sum((y - np.mean(y)) ** 2)
        return 1 - (ss_res / ss_tot)

# Generate sample data
np.random.seed(42)
X = np.random.randn(100, 2)
true_weights = np.array([3, -2])
true_bias = 1
y = X @ true_weights + true_bias + 0.1 * np.random.randn(100)

# Fit model
model = LinearRegression()
model.fit(X, y)

print(f"True weights: {true_weights}")
print(f"Estimated weights: {model.weights}")
print(f"True bias: {true_bias}")
print(f"Estimated bias: {model.bias:.3f}")
print(f"R-squared: {model.score(X, y):.3f}")
```

Matrix Factorization

Principal Component Analysis (PCA)

```
class PCA:
    def __init__(self, n_components):
        self.n_components = n_components
        self.components = None
        self.mean = None
        self.explained_variance_ratio = None

    def fit(self, X):
        """Fit PCA using eigendecomposition"""
        # Center the data
        self.mean = np.mean(X, axis=0)
        X_centered = X - self.mean

        # Compute covariance matrix
```

```
        cov_matrix = np.cov(X_centered.T)

        # Eigendecomposition
        eigenvalues, eigenvectors = np.linalg.eigh(cov_matrix)

        # Sort by eigenvalues (descending)
        idx = np.argsort(eigenvalues)[::-1]
        eigenvalues = eigenvalues[idx]
        eigenvectors = eigenvectors[:, idx]

        # Select top components
        self.components = eigenvectors[:, :self.n_components].T
        self.explained_variance_ratio = eigenvalues[:self.n_components] / np.sum(eigenvalues

    def transform(self, X):
        """Transform data to lower dimension"""
        X_centered = X - self.mean
        return X_centered @ self.components.T

    def fit_transform(self, X):
        """Fit and transform in one step"""
        self.fit(X)
        return self.transform(X)

# Generate 2D data with correlation
np.random.seed(42)
mean = [0, 0]
cov = [[1, 0.8], [0.8, 1]]
X = np.random.multivariate_normal(mean, cov, 300)

# Apply PCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)

# Visualize
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))

# Original data
ax1.scatter(X[:, 0], X[:, 1], alpha=0.6)
ax1.set_title('Original Data')
ax1.set_xlabel('Feature 1')
ax1.set_ylabel('Feature 2')
ax1.grid(True)
ax1.axis('equal')

# PCA transformed data
ax2.scatter(X_pca[:, 0], X_pca[:, 1], alpha=0.6)
```

```
ax2.set_title('PCA Transformed Data')
ax2.set_xlabel('First Principal Component')
ax2.set_ylabel('Second Principal Component')
ax2.grid(True)
ax2.axis('equal')

plt.tight_layout()
plt.show()

print(f"Explained variance ratio: {pca.explained_variance_ratio}")
print(f"Total variance explained: {np.sum(pca.explained_variance_ratio):.3f}")
```

Eigenvalues and Eigenvectors

Understanding Eigendecomposition

```
def visualize_eigenvectors(A):
    """Visualize how matrix A transforms eigenvectors"""
    # Compute eigenvalues and eigenvectors
    eigenvalues, eigenvectors = np.linalg.eig(A)

    # Create unit circle
    theta = np.linspace(0, 2*np.pi, 100)
    unit_circle = np.array([np.cos(theta), np.sin(theta)])

    # Transform unit circle
    transformed_circle = A @ unit_circle

    plt.figure(figsize=(12, 5))

    # Original space
    plt.subplot(1, 2, 1)
    plt.plot(unit_circle[0], unit_circle[1], 'b-', label='Unit circle')

    # Plot eigenvectors
    for i, (val, vec) in enumerate(zip(eigenvalues, eigenvectors.T)):
        plt.arrow(0, 0, vec[0], vec[1], head_width=0.1, head_length=0.1,
                  fc=f'C{i}', ec=f'C{i}', label=f'Eigenvector {i+1} (λ={val:.2f})')

    plt.axis('equal')
    plt.grid(True)
    plt.legend()
    plt.title('Original Space')
```

```
    # Transformed space
    plt.subplot(1, 2, 2)
    plt.plot(transformed_circle[0], transformed_circle[1], 'r-', label='Transformed circle')

    # Plot transformed eigenvectors
    for i, (val, vec) in enumerate(zip(eigenvalues, eigenvectors.T)):
        transformed_vec = A @ vec
        plt.arrow(0, 0, transformed_vec[0], transformed_vec[1],
                  head_width=0.1, head_length=0.1,
                  fc=f'C{i}', ec=f'C{i}', label=f'Transformed eigenvector {i+1}')

    plt.axis('equal')
    plt.grid(True)
    plt.legend()
    plt.title('Transformed Space')

    plt.tight_layout()
    plt.show()

# Example matrix
A = np.array([[2, 1], [1, 2]])
visualize_eigenvectors(A)
```

Spectral Clustering

```
def spectral_clustering(X, n_clusters=2, sigma=1.0):
    """Simple spectral clustering implementation"""
    n_samples = X.shape[0]

    # Compute similarity matrix (RBF kernel)
    distances = np.sum(X**2, axis=1, keepdims=True) + np.sum(X**2, axis=1) - 2 * X @ X.T
    similarity = np.exp(-distances / (2 * sigma**2))

    # Compute degree matrix
    degree = np.diag(np.sum(similarity, axis=1))

    # Compute normalized Laplacian
    D_inv_sqrt = np.diag(1.0 / np.sqrt(np.sum(similarity, axis=1)))
    L_norm = np.eye(n_samples) - D_inv_sqrt @ similarity @ D_inv_sqrt

    # Eigendecomposition
    eigenvalues, eigenvectors = np.linalg.eigh(L_norm)

    # Use smallest eigenvectors for clustering
    embedding = eigenvectors[:, :n_clusters]
```

```
    # K-means on embedding
    from sklearn.cluster import KMeans
    kmeans = KMeans(n_clusters=n_clusters, random_state=42)
    labels = kmeans.fit_predict(embedding)

    return labels, embedding

# Generate two moons dataset
from sklearn.datasets import make_moons
X, y_true = make_moons(n_samples=200, noise=0.1, random_state=42)

# Apply spectral clustering
labels_spectral, embedding = spectral_clustering(X, n_clusters=2, sigma=0.3)

# Visualize results
fig, axes = plt.subplots(1, 3, figsize=(15, 5))

# Original data
axes[0].scatter(X[:, 0], X[:, 1], c=y_true, cmap='viridis')
axes[0].set_title('True Clusters')
axes[0].grid(True)

# Spectral embedding
axes[1].scatter(embedding[:, 0], embedding[:, 1], c=y_true, cmap='viridis')
axes[1].set_title('Spectral Embedding')
axes[1].grid(True)

# Spectral clustering result
axes[2].scatter(X[:, 0], X[:, 1], c=labels_spectral, cmap='viridis')
axes[2].set_title('Spectral Clustering Result')
axes[2].grid(True)

plt.tight_layout()
plt.show()
```

Matrix Calculus for Neural Networks

Gradient Computation

```
class SimpleNeuralNetwork:
    def __init__(self, input_size, hidden_size, output_size):
        # Initialize weights
        self.W1 = np.random.randn(input_size, hidden_size) * 0.1
        self.b1 = np.zeros((1, hidden_size))
```

```
        self.W2 = np.random.randn(hidden_size, output_size) * 0.1
        self.b2 = np.zeros((1, output_size))

    def sigmoid(self, x):
        return 1 / (1 + np.exp(-np.clip(x, -250, 250)))

    def sigmoid_derivative(self, x):
        s = self.sigmoid(x)
        return s * (1 - s)

    def forward(self, X):
        """Forward pass"""
        self.z1 = X @ self.W1 + self.b1
        self.a1 = self.sigmoid(self.z1)
        self.z2 = self.a1 @ self.W2 + self.b2
        self.a2 = self.sigmoid(self.z2)
        return self.a2

    def backward(self, X, y, output):
        """Backward pass using matrix calculus"""
        m = X.shape[0]

        # Output layer gradients
        dz2 = output - y
        dW2 = (1/m) * self.a1.T @ dz2
        db2 = (1/m) * np.sum(dz2, axis=0, keepdims=True)

        # Hidden layer gradients
        da1 = dz2 @ self.W2.T
        dz1 = da1 * self.sigmoid_derivative(self.z1)
        dW1 = (1/m) * X.T @ dz1
        db1 = (1/m) * np.sum(dz1, axis=0, keepdims=True)

        return dW1, db1, dW2, db2

    def train(self, X, y, epochs=1000, learning_rate=0.1):
        """Train the network"""
        losses = []

        for epoch in range(epochs):
            # Forward pass
            output = self.forward(X)

            # Compute loss
            loss = np.mean((output - y)**2)
            losses.append(loss)
```

```
            # Backward pass
            dW1, db1, dW2, db2 = self.backward(X, y, output)

            # Update weights
            self.W1 -= learning_rate * dW1
            self.b1 -= learning_rate * db1
            self.W2 -= learning_rate * dW2
            self.b2 -= learning_rate * db2

            if epoch % 100 == 0:
                print(f"Epoch {epoch}, Loss: {loss:.4f}")

        return losses

# Generate XOR dataset
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y = np.array([[0], [1], [1], [0]])

# Create and train network
nn = SimpleNeuralNetwork(2, 4, 1)
losses = nn.train(X, y, epochs=5000, learning_rate=1.0)

# Test the network
predictions = nn.forward(X)
print("\nFinal predictions:")
for i in range(len(X)):
    print(f"Input: {X[i]}, Target: {y[i][0]}, Prediction: {predictions[i][0]:.3f}")

# Plot training loss
plt.figure(figsize=(10, 6))
plt.plot(losses)
plt.title('Training Loss')
plt.xlabel('Epoch')
plt.ylabel('Mean Squared Error')
plt.grid(True)
plt.show()
```

Singular Value Decomposition (SVD)

Recommender Systems

```
def matrix_factorization_svd(R, k=10):
    """Matrix factorization using SVD for recommender systems"""
    # R is user-item rating matrix
```

```
    # Fill missing values with mean
    R_filled = R.copy()
    mask = ~np.isnan(R)
    R_filled[~mask] = np.nanmean(R)

    # Center the data
    user_means = np.nanmean(R, axis=1, keepdims=True)
    R_centered = R_filled - user_means

    # SVD decomposition
    U, s, Vt = np.linalg.svd(R_centered, full_matrices=False)

    # Keep only top k components
    U_k = U[:, :k]
    s_k = s[:k]
    Vt_k = Vt[:k, :]

    # Reconstruct matrix
    R_reconstructed = U_k @ np.diag(s_k) @ Vt_k + user_means

    return R_reconstructed, U_k, s_k, Vt_k

# Create sample user-item rating matrix
np.random.seed(42)
n_users, n_items = 100, 50
R = np.random.rand(n_users, n_items) * 5

# Introduce missing values (80% sparsity)
mask = np.random.rand(n_users, n_items) < 0.8
R[mask] = np.nan

print(f"Original matrix shape: {R.shape}")
print(f"Sparsity: {np.sum(mask) / (n_users * n_items):.2%}")

# Apply matrix factorization
R_pred, U, s, Vt = matrix_factorization_svd(R, k=10)

# Compute RMSE on observed entries
observed_mask = ~np.isnan(R)
rmse = np.sqrt(np.mean((R[observed_mask] - R_pred[observed_mask])**2))
print(f"RMSE on observed entries: {rmse:.3f}")

# Visualize singular values
plt.figure(figsize=(10, 6))
plt.plot(s, 'bo-')
plt.title('Singular Values')
plt.xlabel('Component')
```

```
plt.ylabel('Singular Value')
plt.grid(True)
plt.show()
```

Practical Applications

Image Compression using SVD

```
def compress_image_svd(image, k):
    """Compress image using SVD"""
    if len(image.shape) == 3:
        # Color image - compress each channel
        compressed = np.zeros_like(image)
        for i in range(3):
            U, s, Vt = np.linalg.svd(image[:, :, i], full_matrices=False)
            compressed[:, :, i] = U[:, :k] @ np.diag(s[:k]) @ Vt[:k, :]
    else:
        # Grayscale image
        U, s, Vt = np.linalg.svd(image, full_matrices=False)
        compressed = U[:, :k] @ np.diag(s[:k]) @ Vt[:k, :]

    return np.clip(compressed, 0, 255).astype(np.uint8)

# Create a simple synthetic image
def create_test_image():
    x = np.linspace(0, 4*np.pi, 100)
    y = np.linspace(0, 4*np.pi, 100)
    X, Y = np.meshgrid(x, y)
    image = 128 + 127 * np.sin(X) * np.cos(Y)
    return image.astype(np.uint8)

# Test image compression
original_image = create_test_image()
compression_ratios = [5, 10, 20, 50]

fig, axes = plt.subplots(1, len(compression_ratios) + 1, figsize=(15, 3))

# Original image
axes[0].imshow(original_image, cmap='gray')
axes[0].set_title('Original')
axes[0].axis('off')

# Compressed images
for i, k in enumerate(compression_ratios):
```

```
    compressed = compress_image_svd(original_image, k)
    axes[i+1].imshow(compressed, cmap='gray')

    # Calculate compression ratio
    original_size = original_image.size
    compressed_size = k * (original_image.shape[0] + original_image.shape[1] + 1)
    ratio = compressed_size / original_size

    axes[i+1].set_title(f'k={k}\n({ratio:.1%} of original)')
    axes[i+1].axis('off')

plt.tight_layout()
plt.show()
```

Summary

Linear algebra is fundamental to machine learning because:

1. **Data Representation**: Data is naturally represented as vectors and matrices
2. **Efficient Computation**: Matrix operations enable vectorized computations
3. **Dimensionality Reduction**: Techniques like PCA use eigendecomposition
4. **Neural Networks**: Forward and backward passes are matrix multiplications
5. **Optimization**: Gradient-based methods rely on matrix calculus
6. **Similarity and Distance**: Vector operations measure data relationships

Understanding these concepts enables you to: - Implement ML algorithms from scratch - Debug and optimize existing implementations - Understand the mathematical foundations of modern AI systems - Design new algorithms and approaches

Gradients, Loss Functions, and Backprop Sketch

Training is almost always: pick a **loss**, form **empirical risk**, descend on **gradients**. This chapter is the calculus you need to read training loops without treating them as magic.

Diagram: risk minimization

```
data (x,y)    model f_w(x)    loss  (y, ŷ)

                          _w J        v
                            J(w) = avg   + reg
```

1. Loss menu

Loss	Formula (scalar)	Typical use	Notes
Squared	$(y-\hat{y})^2$	regression	outlier-sensitive
Absolute	$\lvert y-\hat{y}\rvert$	robust regression	subgradient at 0
Huber	piecewise quad/lin	robust + smooth	threshold δ
Logistic / log	$-y\log\hat{y}-(1-y)\log(1-\hat{y})$	binary clf	$\hat{y}\in(0,1)$
Cross-entropy	$-\log p_c$	multi-class	with softmax p
Hinge	$\max(0,1-y\hat{y})$	SVM-style	margin; $y\in\{\pm1\}$
Cosine	$1-\cos(y,\hat{y})$	embeddings	direction focus

Multi-class CE: model outputs probabilities p_k, true class c:

$$\ell = -\log p_c.$$

Worked example 1 — CE numerical

$p=(0.1,0.7,0.2)$, true class $c=2$ (0-index class 1): $\ell=-\log 0.7\approx 0.357$ nats if ln.

2. Empirical risk and regularization

Given data $(x_i, y_i)_{i=1}^n$:

$$J(w) = \frac{1}{n}\sum_{i=1}^{n} \ell(y_i, f_w(x_i)) + \lambda\Omega(w).$$

$\Omega(w)$	Effect
$\|w\|_2^2$	ridge / weight decay
$\|w\|_1$	sparsity
elastic net	mix

λ trades fit vs complexity; choose on validation data.

Population vs empirical

True risk $R(w) = \mathbb{E}[\ell(y, f_w(x))]$. ERM minimizes J without the expectation. Generalization: $R(\widehat{w})$ vs J without Ω.

3. Gradients and directional derivatives

If J is differentiable,

$$\nabla J(w) \in \mathbb{R}^d, \qquad \lim_{t\to 0} \frac{J(w+tv) - J(w)}{t} = \langle \nabla J(w), v\rangle.$$

Steepest ascent direction is ∇J; **descent** uses $-\nabla J$.

Gradient descent

$$w \leftarrow w - \eta \nabla J(w).$$

```
J(w)
  \
   \          start
    \     /
     \
      \/      near minimum
```

- η too large: diverge / oscillate

- η too small: crawl
- **SGD / mini-batch:** replace ∇J by average over a batch — noisy but scalable

Worked example 2 — pure GD arithmetic

$J(w) = \frac{1}{2}(w-3)^2$, $w_0 = 0$, $\eta = 0.5$.
$\nabla J = w - 3$.

$w_1 = 0 - 0.5(0-3) = 1.5$,
$w_2 = 1.5 - 0.5(1.5-3) = 2.25$,
$w_3 = 2.25 - 0.5(2.25-3) = 2.625 \to$ approaches 3.

4. Chain rule and backpropagation

If $z = h(w)$, $\hat{y} = g(z)$, $\ell = \ell(y, \hat{y})$:

$$\frac{\partial \ell}{\partial w} = \frac{\partial \ell}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w}.$$

Neural nets stack many maps; **reverse-mode automatic differentiation** multiplies local Jacobians from the scalar loss backward while reusing forward activations — **backpropagation**.

```
forward:   x → layer1 → layer2 → … → ŷ →
backward:   / ŷ ← … ←  / w   (reuse forward values)
```

Complexity: for scalar loss, reverse mode costs a small constant factor over one forward pass — not "one forward per parameter."

Worked example 3 — scalar chain

$f(w) = (w^2+1)^3$.

Expand: messy. Chain: $u = w^2+1$, $f = u^3$, $f' = 3u^2 \cdot 2w = 6w(w^2+1)^2$.

5. Worked example — logistic regression end-to-end

Model: $f_w(x) = \sigma(w^\top x)$, $\sigma(t) = \frac{1}{1+e^{-t}}$, label $y \in \{0, 1\}$.

$$\ell = -(y \log \hat{y} + (1-y)\log(1-\hat{y})).$$

Using $\sigma' = \sigma(1-\sigma)$ and algebra:

$$\nabla_w \ell = (\hat{y} - y)\, x.$$

Average over a batch: same residual-times-features structure as least squares, with residual $\hat{y} - y$.

Regularized: $\nabla J = \frac{1}{n} \sum_i (\hat{y}_i - y_i) x_i + 2\lambda w$ (for $\Omega = \|w\|_2^2$ with conventional factor — match your code's λ scaling).

6. Softmax + cross-entropy

Logits $z \in \mathbb{R}^K$, $p = \text{softmax}(z)$, one-hot y:

$$\ell = -\sum_k y_k \log p_k = -\log p_c.$$

$$\frac{\partial \ell}{\partial z} = p - y.$$

Stable implementation: `log_softmax` in the log domain, never `log(softmax(z))` with naive overflow.

7. Convex vs non-convex landscapes

Setting	Landscape	Implication
Ridge least squares	strongly convex quadratic	unique global min
Logistic + L2	convex	local = global
Deep nets	non-convex	saddles, many minima; SGD often still works
ReLU nets	piecewise linear regions	gradients sparse

Convexity guarantees are about $J(w)$ as a function of parameters — not about whether the data model is "correct."

8. Subgradients (nonsmooth losses)

For $J(w) = |w|$, $\partial J(0) = [-1, 1]$. **Subgradient descent** picks any $g \in \partial J(w)$. Hinge and ReLU need this language at kinks; in practice autodiff returns one element of the subdifferential (e.g. 0 at ReLU kink).

9. Second-order glance

Hessian $H = \nabla^2 J$. Newton: $w \leftarrow w - H^{-1} \nabla J$. Expensive in high d; explains curvature: large eigenvalues need smaller η along those axes. Momentum / Adam partially compensate without forming H.

10. Numerical gradient checks

Central difference:

$$\frac{\partial J}{\partial w_i} \approx \frac{J(w + \varepsilon e_i) - J(w - \varepsilon e_i)}{2\varepsilon}.$$

Relative error $\ll 10^{-5}$ (double, smooth J) builds trust in backprop. Failures are bugs, not mysticism.

11. Mini-batch tradeoffs

Larger batch	Smaller batch
stabler gradient	more noise (sometimes useful)
better matrix hardware use	more updates per epoch
may need larger η	generalization folklore varies

12. Pitfalls

1. Squared loss on classification probabilities (wrong likelihood, poor calibration)
2. Softmax without log-sum-exp stability
3. Averaging loss wrong (sum vs mean) while reusing an η tuned for the other
4. Forgetting regularization term in the gradient
5. Leaking test data into early stopping

13. Checkpoint

- Pick a loss for regression vs classification
- Write ERM + regularizer
- Run a few GD steps by hand on a 1D quadratic
- State the logistic gradient $(\hat{y} - y)x$
- Explain backprop as reverse-mode AD on a scalar loss
- Gradient-check a suspicious implementation

Exercises

Easy

1. Compute $f'(x)$ for $f(x) = \log(1 + e^x)$ (softplus).
2. For $J(w) = \frac{1}{2}(w - 3)^2$, run 3 steps of GD from $w_0 = 0$ with $\eta = 0.5$.
3. Why is cross-entropy preferred over squared loss for classification probabilities?
4. One sentence: what does larger λ do to optimal $\|w\|$?
5. Mini-batch size: list two bullets on noise vs parallelism.

Medium

6. Sketch backprop for $f(w) = (w^2 + 1)^3$: expand vs chain.
7. Derive $\nabla_w \frac{1}{2}\|Xw - y\|_2^2 = X^\top(Xw - y)$.
8. Binary CE: show $\partial \ell / \partial \hat{y} = (\hat{y} - y)/(\hat{y}(1 - \hat{y}))$ then chain through sigmoid.
9. Hinge loss: write a subgradient w.r.t. w for linear scores.
10. Show that adding $\lambda\|w\|_2^2$ adds $2\lambda w$ to the gradient.

Challenge

11. Softmax+CE: prove $\partial \ell / \partial z = p - y$.
12. Compare full-batch GD and SGD on a finite-sum quadratic (bias/variance of updates).
13. Explain vanishing gradients for a product of many Jacobians with spectral norms < 1.
14. Design a finite-difference check protocol for a 2-layer MLP Jacobian.
15. Map MAP inference with Gaussian prior to L2-regularized ERM.

Checks

2. $w_1 = 1.5$, $w_2 = 2.25$, $w_3 = 2.625$.
3. Squared loss does not match Bernoulli log-likelihood; can be poorly calibrated.
4. Larger λ shrinks weights.

Summary

Losses encode task goals; gradients point downhill on empirical risk; backprop implements the chain rule efficiently for deep compositions. Logistic and softmax+CE give clean gradients that match classification likelihoods. Master the scalar chain rule, the residual-times-features pattern, and gradient checks — then optimizers and architectures become readable rather than magical.

Probability for Machine Learning

ML models are often **conditional probability machines**: $p(y \mid x)$ or density estimators $p(x)$. This chapter connects Bayes, likelihood, calibration, and common predictive distributions to training objectives.

Diagram: generative vs discriminative

```
discriminative:          generative:
x    p(y|x)    ŷ      assume p(x,y)=p(y)p(x|y)

                                predict via Bayes p(y|x)
```

1. Likelihood and maximum likelihood

Given i.i.d. data and model $\{p_w\}$, the **likelihood** is

$$L(w) = \prod_{i=1}^{n} p_w(y_i \mid x_i)$$

(or $p_w(x_i)$ unsupervised). Equivalently maximize the log-likelihood

$$\ell(w) = \sum_{i=1}^{n} \log p_w(y_i \mid x_i).$$

MLE: $\hat{w} = \arg\max_w \ell(w)$.
Minimizing negative log-likelihood (NLL) is the same problem; for categorical models NLL = cross-entropy on labels.

Worked example 1 — Bernoulli / logistic

$y \in \{0, 1\}$, $p_w(y = 1 \mid x) = \sigma(w^\top x) = \hat{y}$.

$$\log L = \sum_i (y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)).$$

Maximizing this yields logistic regression (no regularizer yet).

Worked example 2 — Gaussian regression

$y = w^\top x + \varepsilon$, $\varepsilon \sim \mathcal{N}(0, \sigma^2)$. MLE for w least squares. NLL squared loss up to constants and σ.

2. MAP estimation and regularization

Prior $p(w)$ encodes beliefs before data.

$$p(w \mid \text{data}) \propto L(w)\, p(w).$$

MAP: $\hat{w} = \arg\max_w L(w)p(w) = \arg\max_w (\log L(w) + \log p(w))$.

Prior	Penalty $\Omega(w)$ flavor
$w \sim \mathcal{N}(0, \tau^2 I)$	$\propto \|w\|_2^2$ (ridge)
Laplace (i.i.d.)	$\propto \|w\|_1$ (lasso-like)

```
posterior  likelihood × prior
log posterior = log lik - Ω(w) + const
```

Worked example 3 — Gaussian prior strength

If $w \sim N(0, \tau^2 I)$, MAP penalty strength scales like $1/\tau^2$: small τ strong shrink to 0.

3. Bayes rule in prediction

$$p(y \mid x) = \frac{p(x \mid y)p(y)}{p(x)}.$$

Generative classifiers model $p(x \mid y)$ and $p(y)$; **discriminative** model $p(y \mid x)$ directly.

Naive Bayes: assume features independent given class:

$$p(x \mid y) = \prod_{j=1}^{d} p(x_j \mid y).$$

Crude independence, often strong baseline for text (bag-of-words).

Worked example 4 — equal priors

Two classes, $p(y = 1) = p(y = 0) = \frac{1}{2}$. Bayes chooses class 1 when $p(x \mid y = 1) > p(x \mid y = 0)$, i.e. likelihood ratio > 1.

4. Softmax as multi-class Bernoulli generalization

Logits $z \in \mathbb{R}^K$:

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}.$$

Properties: $p_k > 0$, $\sum_k p_k = 1$. Shift-invariant: $z + c\mathbf{1}$ same softmax (identifiability: often fix one logit 0).

Worked example 5

$z = (0, 0, 1)$: $p \propto (1, 1, e)$, normalize by $2 + e$:

$$p \approx (0.212, 0.212, 0.576).$$

5. Calibration

A predicted probability $\hat{p} = 0.8$ is **calibrated** if among all cases with $\hat{p} \approx 0.8$, about 80% are positive (long-run frequency).

Tool	Role
Reliability diagram	plot empirical frequency vs predicted bin
Brier score	$\frac{1}{n}\sum(\hat{p}_i - y_i)^2$
Log loss	CE; penalizes confident wrong answers hard
ECE	expected calibration error (binned)

High accuracy calibrated. A model can get argmax labels right while distorting probabilities (common after overtraining or temperature-unaware decoding).

Worked example 6 — overconfidence

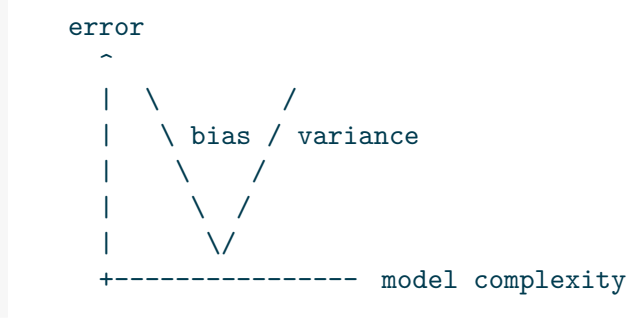
Neural net outputs 0.99 on many examples but is only 90% accurate in that bin overconfident; temperature scaling or isotonic regression can recalibrate post hoc.

6. Bias–variance (probabilistic view)

For squared error and many estimators of a regression function,

$$\mathbb{E}[(f_{\hat{w}}(x) - y)^2] = \text{Bias}^2 + \text{Variance} + \sigma^2,$$

where σ^2 is irreducible noise.



Underfitting: high bias. Overfitting: high variance. Regularization and more data move the trade-off.

7. Decision rules and utility

Given $p(y \mid x)$, optimal actions depend on **costs**. For binary classification with asymmetric false positive/negative costs, threshold $\hat{p}$ at c not necessarily 1/2:

$$\text{predict 1 if } p(y = 1 \mid x) \geq \tau(c).$$

Probability models + utility always maximize accuracy.

8. Common predictive distributions in ML

Task	Typical $p(y \mid x)$
Binary clf	Bernoulli(sigmoid)
Multi-class	Categorical(softmax)
Regression	Gaussian (or Laplace / Student-t robust)
Count data	Poisson / negative binomial
Sequences	Autoregressive categorical tokens

Choosing p **is** choosing the loss (NLL).

9. Latent variables (preview)

Mixture models, VAEs, HMMs: introduce z with $p(x) = \sum_z p(x \mid z)p(z)$ or integral. Training uses EM or variational bounds (ELBO) involving KL terms — links to the information-theory chapters.

10. Train / validation / test as risk estimation

- **Train:** fit w (possibly with early stopping signals from val)
- **Validation:** estimate risk for model selection / hyperparameters
- **Test:** final unbiased-ish estimate — touch once

All are Monte Carlo estimates of expectations under (hopefully) the deployment distribution. Distribution shift breaks the story.

11. Pitfalls

1. Interpreting softmax outputs as calibrated without checking
2. MLE without regularization in overparameterized models
3. Naive Bayes independence taken as literal truth
4. Optimizing accuracy when probabilities drive decisions
5. Using test set for model selection (optimistic bias)

12. Checkpoint

- Write MLE and MAP objectives
- Connect Gaussian prior to L2
- Softmax formula and shift invariance
- Define calibration vs accuracy
- State bias–variance components for squared error
- Choose Bernoulli vs categorical vs Gaussian likelihoods

Exercises

Easy

1. Show that maximizing Bernoulli likelihood yields logistic log-loss.
2. If prior $w \sim N(0, \tau^2 I)$, what regularizer appears in MAP?
3. Give an example where a model is accurate but poorly calibrated.
4. For two classes with equal prior, when does Bayes choose class 1?
5. Compute softmax of $z = (0, 0, 1)$ (approx values).
6. Explain train/val split as estimating generalization risk.

Medium

7. Derive NLL for $y \sim \mathcal{N}(w^\top x, \sigma^2)$ and relate to least squares.
8. Show softmax is invariant to adding a constant to all logits.
9. Brier score vs 0-1 loss: construct a tiny dataset where rankings differ.
10. Naive Bayes for binary features: write $\log p(y \mid x)$ up to constants.
11. Temperature T: $p \propto e^{z/T}$. What do $T \to 0$ and $T \to \infty$ do?

Challenge

12. MAP with Laplace prior: connect to L1 geometry (soft-thresholding intuition).
13. Proper scoring rules: why is log loss proper? (definition + one-line idea).
14. Class imbalance: how does changing prior $p(y)$ affect the Bayes threshold?
15. Sketch ELBO: $\log p(x) \geq \mathbb{E}_q \log p(x \mid z) - D_{\mathrm{KL}}(q(z) \| p(z))$ role of each term.

Checks

2. $\|w\|_2^2$ penalty strength $\propto 1/\tau^2$.
3. $p \propto (1, 1, e)$ normalized $\approx (0.21, 0.21, 0.58)$.

Summary

Probabilistic ML treats predictions as distributions: likelihoods define losses, priors define regularizers, and Bayes combines them. Softmax and Bernoulli models cover most classification heads; calibration and utility decide whether probabilities are decision-ready. Bias–variance and data splits remind you that fitting p_w on a sample is not the same as knowing the world — validate under the distribution you care about.

Neural Networks: A Mathematical Sketch

A neural net is a **composition of affine maps and nonlinearities**. Depth creates hierarchical features; training uses gradients of a scalar loss. This chapter is the math skeleton — not a framework tutorial.

Diagram: one hidden layer

```
x   R^d

     W1, b1
  v
z = W1 x + b1

       elementwise
  v
h =  (z)   R^h

     W2, b2
  v
ŷ = W2 h + b2
```

1. Building blocks

Affine layer

$$z = Wx + b, \qquad W \in \mathbb{R}^{h \times d},\ b \in \mathbb{R}^h.$$

Activations σ (elementwise unless noted)

σ	Formula	Notes
ReLU	$\max(0, z)$	sparse grads; default MLP
Leaky ReLU	$\max(\alpha z, z)$	avoids dead units somewhat
sigmoid	$(1 + e^{-z})^{-1}$	saturates; rare in deep hidden
tanh	$\tanh z$	zero-centered
GELU / SiLU	smooth ReLU-like	transformers / modern nets

σ	Formula	Notes
softmax	on vector	output distributions

Residual connections

$$y = x + F(x)$$

eases deep training: identity path keeps gradients from vanishing purely by depth; F learns residuals.

Universal approximation (intuition)

A single hidden layer with suitable σ and enough width can approximate continuous functions on compact sets arbitrarily well. Depth often needs fewer parameters for hierarchical structure (images, language) — theory and practice both matter.

2. Parameter count and FLOPs

Dense layer $d \to h$: $dh + h$ parameters (weights + biases).

Worked example 1 — MLP $10 \to 32 \to 32 \to 1$

$$\begin{aligned} &10 \cdot 32 + 32 + 32 \cdot 32 + 32 + 32 \cdot 1 + 1 \\ &= 320 + 32 + 1024 + 32 + 32 + 1 = 1441. \end{aligned}$$

(If you saw 1473 elsewhere, recount with the same bias convention — always state whether biases are included.)

Matrix multiplies dominate FLOPs: roughly $2dh$ for a dense multiply-add of one example (order-of-magnitude).

3. Composition and feature hierarchy

Write $f = f_L \circ f_{L-1} \circ \cdots \circ f_1$. Early layers often detect simple patterns; later layers mix them. Mathematically each f_ℓ is affine+σ (or attention, norm, etc.).

Critical: if every σ is identity (or affine), the whole net collapses to **one** affine map — nonlinear σ is essential for depth to add expressive power.

4. Jacobian and local linearization

Near a point,

$$f(x+\delta) \approx f(x) + J_f(x)\,\delta.$$

Backprop multiplies Jacobians efficiently for **scalar** losses without materializing full J_f when only $v^\top J$ or Ju products are needed (autodiff VJPs/JVPs).

Layer Jacobian sketch (elementwise σ)

For $h = \sigma(Wx + b)$,

$$\frac{\partial h_i}{\partial x} = \sigma'(z_i)\, W_{i,:}$$

(row-wise scaling of W by $\sigma'(z)$).

5. Vanishing and exploding gradients

Backprop to early layers multiplies many Jacobians:

$$\frac{\partial \ell}{\partial W_1} \sim \Big(\prod_{\ell=2}^{L} J_\ell\Big) \frac{\partial \ell}{\partial \text{out}}.$$

```
/ early layer    (many matrices) ×  / late

         singular values <1   vanish
         singular values >1   explode
```

Mitigations (conceptual): residual links, normalization (Batch/LayerNorm), careful initialization (He/Xavier), gated RNNs / transformers replacing naive deep stacks, gradient clipping.

6. Normalization layers (math role)

LayerNorm (idea): for activation vector h, standardize coordinates then affine rescale with learned γ, β. Stabilizes distributions of pre-activations so optimization sees better-conditioned landscapes. Not just "engineering" — it changes the geometry of $J(w)$.

7. Loss heads

Task	Head
Regression	linear output + squared / Huber
Binary	sigmoid + BCE
Multi-class	softmax + CE
Embeddings	contrastive / InfoNCE

Training minimizes empirical risk on the head's loss; backbone features are whatever gradients request.

8. Overfitting and capacity

More parameters + flexible σ can interpolate noise.

Controls: more data, weight decay, dropout (multiplicative noise / ensemble intuition), early stopping, architecture constraints (CNNs share weights; attention has structure), data augmentation.

Double descent phenomena: classical U-curve incomplete in overparameterized regimes — still use validation.

9. Convolution and parameter sharing (sketch)

A conv layer applies the same small kernel at every spatial location — **tied weights**. Fewer parameters than dense on images; equivariance to translations (approximately, with boundary effects). Math: linear map with Toeplitz/circulant structure + nonlinearity.

10. Attention as similarity (sketch)

Queries Q, keys K, values V:

$$\text{Attention}(Q, K, V) = \text{softmax}\Big(\frac{QK^\top}{\sqrt{d}}\Big)V.$$

Rows of the softmax are probability distributions over positions — a data-dependent linear combination of value vectors. Linear algebra + softmax probability; no mystic step.

11. Initialization (order-of-magnitude)

Goal: keep activation and gradient variances $\mathcal{O}(1)$ across layers at start.

- **Xavier/Glorot:** $\mathrm{Var}(W) \sim 1/d_{\text{in}}$ (tanh-like)
- **He:** $\mathrm{Var}(W) \sim 2/d_{\text{in}}$ (ReLU)

Bad init immediate vanish/explode before learning starts.

12. What math to study next

Topic	Why
Matrix calculus	gradients w.r.t. W
Probability	losses as likelihoods
Optimization	SGD, momentum, Adam
Linear algebra	attention as similarity, PCA of activations
Information theory	CE, KL, MI bounds

13. Pitfalls

1. Stacking linear layers without nonlinearities
2. Counting parameters wrong (forgetting biases or embeddings)
3. Interpreting deep success as "non-convex is easy" universally
4. Ignoring depth-related gradient issues until loss is NaN
5. Comparing architectures without matched optimization effort

14. Checkpoint

- Write a one-hidden-layer net formula
- Count parameters of a small MLP
- Explain why identity activations collapse depth
- State residual connection formula
- Describe vanishing gradients as Jacobian products

- Name two capacity controls

Exercises

Easy

1. Count parameters of MLP $10 \to 32 \to 32 \to 1$ with biases.
2. Compute ReLU derivative almost everywhere.
3. Why is identity activation stacked still just one affine map?
4. Name two reasons mini-batch SGD is used instead of full-batch GD on big data.
5. Softmax on equal logits: what distribution?

Medium

6. Sketch how residual $y = x + F(x)$ changes the "multiply many Jacobians" story.
7. For $h = \mathrm{ReLU}(Wx)$, write $\partial h_i / \partial W_{ik}$ when $z_i > 0$ vs $z_i < 0$.
8. Compare parameter count: dense $d \to d$ vs 3×3 conv with c channels (same c, spatial size $N \times N$) order-of-magnitude.
9. Attention: show each row of $\mathrm{softmax}(QK^\top/\sqrt{d})$ sums to 1.
10. He init variance: argue why factor 2 appears for ReLU (half neurons active).

Challenge

11. Prove that a composition of affine maps is affine (induct on layers).
12. Lipschitz constant of a deep net: bound via product of Lipschitz constants of layers.
13. Dead ReLU: construct a 1D example where a unit never activates for a data set.
14. Gradient check a 2-layer net Jacobian-vector product numerically.
15. Relate weight decay on W to a Gaussian prior MAP story from the probability chapter.

Checks

1. $10 \cdot 32 + 32 + 32 \cdot 32 + 32 + 32 \cdot 1 + 1 = 1441$.

2. Composition of affines is affine.

3. Uniform $1/K$.

Summary

Neural nets compose affine transforms with nonlinearities; depth only buys power when nonlinearities (or data-dependent maps like attention) prevent collapse to a single linear model. Parameter counts, Jacobians, and residual/normalization geometry explain trainability. Losses at the head define the probabilistic goal; the backbone is a flexible feature map optimized by gradient methods. Keep the linear-algebra and chain-rule picture — frameworks implement it; they do not replace it.

Matrix Calculus for ML (Practical Identities)

You rarely expand every partial $\partial J/\partial W_{ij}$ by hand. A few **layout-consistent identities** cover least squares, logistic regression, and dense layers. This chapter uses **numerator layout** habits common in ML notes: gradients of a scalar match the shape of the parameter.

Diagram: shapes

```
X   R^{n×d}     data (rows = examples)
w   R^d         weights
Xw   R^n        predictions (linear)
 _w J   R^d     same shape as w
```

1. Layout discipline

- J scalar $\quad \nabla_w J$ same shape as w
- Always check dimensions: every term in an identity must match
- For matrices, $\frac{\partial J}{\partial W}$ has the same shape as W
- When reading papers, confirm layout (numerator vs denominator) — identities can transpose

Rule of thumb: if shapes do not multiply cleanly, the formula is wrong or the layout differs.

2. Differentials (often easier than coordinates)

If J is scalar and u is a vector expression,

$$dJ = \langle \nabla_u J,\, du \rangle.$$

Push d through linear maps, then read off the gradient.

Worked example 1

$J = \frac{1}{2}\|w\|_2^2$: $dJ = w^\top dw \quad \nabla_w J = w$ (or $2w$ without $\frac{1}{2}$).

3. Core identities (column vectors)

Let a, b be vectors and A a matrix of compatible size; w the variable.

Expression	Gradient w.r.t. marked variable
$w^\top a$ (const a)	$\nabla_w = a$
$\|w\|_2^2$	$\nabla_w = 2w$
$\|Xw - y\|_2^2$	$\nabla_w = 2X^\top(Xw - y)$
$\frac{1}{2}\|Xw - y\|_2^2$	$\nabla_w = X^\top(Xw - y)$
$a^\top W b$	$\partial/\partial W = ab^\top$
$\|W\|_F^2$	$\partial/\partial W = 2W$
$\mathrm{tr}(W^\top A)$	$\partial/\partial W = A$

Worked example 2 — ridge

$$J(w) = \frac{1}{2}\|Xw - y\|_2^2 + \frac{\lambda}{2}\|w\|_2^2 \quad \Rightarrow \quad \nabla J = X^\top(Xw - y) + \lambda w.$$

Set to zero: $(X^\top X + \lambda I)w = X^\top y$.

Worked example 3 — shapes

$X \in \mathbb{R}^{100\times 5}$, $w \in \mathbb{R}^5$, $y \in \mathbb{R}^{100}$:
$Xw - y \in \mathbb{R}^{100}$, $X^\top(Xw - y) \in \mathbb{R}^5$.

4. Least squares and normal equations

$J(w) = \|Xw - y\|_2^2$, $\nabla J = 0$:

$$X^\top X w = X^\top y.$$

If $X^\top X$ ill-conditioned, ridge: $(X^\top X + \lambda I)w = X^\top y$. Prefer QR/SVD numerically when $\lambda = 0$ and κ large.

5. Linear layer + scalar loss

Single example, vector output $\hat{y} = Wx$ (absorb bias via augmented x if needed), loss $\ell(\hat{y})$:

$$\frac{\partial \ell}{\partial W} = \frac{\partial \ell}{\partial \hat{y}}\, x^\top = \delta x^\top \quad \text{(outer product)}.$$

```
error signal   =  / ŷ   (column, same dim as ŷ)
input x                  (column)
 / W =   x^T             (matrix, same shape as W)
```

Mini-batch: sum/mean outer products over examples (or one matrix multiply).

Worked example 4 — bias

$\hat{y} = Wx + b$, $\delta = \partial \ell / \partial \hat{y}$:
$\partial \ell / \partial b = \delta$, $\partial \ell / \partial W = \delta x^\top$.

6. Softmax + cross-entropy (fact)

For one-hot y and softmax probabilities p from logits z:

$$\frac{\partial \ell}{\partial z} = p - y.$$

Why popular: the Jacobian of softmax and the CE gradient cancel into a simple residual. Implement as fused `cross_entropy_with_logits` for stability.

Worked example 5

$p = (0.2, 0.5, 0.3)$, $y = (0, 1, 0)$: $\partial \ell / \partial z = (0.2, -0.5, 0.3)$.

7. Logistic regression (vector form)

$\hat{y} = \sigma(Xw)$ elementwise, binary CE averaged:

$$\nabla_w J = \frac{1}{n} X^\top (\hat{y} - y)$$

(with $y, \hat{y} \in \mathbb{R}^n$). Same residual-times-design-matrix pattern.

8. Chain rule for products and Frobenius

Inner product on matrices: $\langle A, B \rangle_F = \operatorname{tr}(A^\top B)$.

If $Z = WX$, then for scalar $J(Z)$,

$$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial Z} X^\top, \qquad \frac{\partial J}{\partial X} = W^\top \frac{\partial J}{\partial Z}.$$

These two lines power dense backprop.

9. Elementwise nonlinearities

$H = \sigma(Z)$ elementwise, $\delta_H = \partial J/\partial H$:

$$\frac{\partial J}{\partial Z} = \delta_H \odot \sigma'(Z)$$

(Hadamard product). Then backprop into W, x as linear layer with that δ_Z.

10. Finite-difference checks

$$\frac{\partial J}{\partial w_i} \approx \frac{J(w + \varepsilon e_i) - J(w - \varepsilon e_i)}{2\varepsilon}.$$

For matrices, perturb single entries. Relative error should be tiny for smooth J in double precision. Mismatch bug in analytic backprop, not "deep learning mystery."

Worked example 6 — protocol

1. Fix small random W
2. Compute analytic $\partial J/\partial W$
3. Check 5–10 random coordinates by finite differences
4. Use $\varepsilon \sim 10^{-5}$–10^{-6} as a starting point

11. Hessian-vector products (awareness)

Second derivatives: $Hv = \nabla(\langle \nabla J, v \rangle)$ can be obtained by differentiating the gradient JVPs — used in second-order optimization and some training analyses without forming full H.

Worked example 7

For $J = \frac{1}{2}(a^\top w)^2$, $\nabla J = (a^\top w)a$, Hessian $H = aa^\top$. Rank-one curvature in direction a.

Model piece	Gradient

12. Common ML gradients cheat-sheet

Model piece	Gradient
Linear reg.	$X^\top(Xw - y)$
Ridge	$+\lambda w$
Logistic	$X^\top(\sigma(Xw) - y)/n$
Dense layer	$\delta x^\top$
Softmax+CE	$p - y$ on logits
L2 weight decay	add λW to $\partial J/\partial W$

13. Pitfalls

1. Shape errors from mixing row/column conventions
2. Forgetting the $1/n$ mean vs sum
3. `log(softmax)` overflow instead of `log_softmax`
4. Double-counting $\frac{1}{2}$ factors in squared norms
5. Broadcasting bugs in batched outer products

14. Checkpoint

- State $\nabla_w \frac{1}{2}\|Xw - y\|^2$
- Write outer-product rule for W
- Softmax+CE logit gradient
- Chain through elementwise σ
- Finite-difference check a gradient

Exercises

Easy

1. Derive $\nabla_w \|w - w_0\|_2^2$.
2. For $J(w) = \frac{1}{2}\|Xw - y\|_2^2 + \frac{\lambda}{2}\|w\|_2^2$, write ∇J.
3. Shapes: $X \in \mathbb{R}^{100\times 5}$, $w \in \mathbb{R}^5$ — shape of $X^\top(Xw - y)$?
4. Explain in words why outer product $\delta x^\top$ updates W.
5. Gradient of $\frac{\lambda}{2}\|W\|_F^2$ w.r.t. W.

Medium

6. Derive ∇_w of logistic NLL in matrix form.
7. Show $d(\|Xw - y\|_2^2) = 2(Xw - y)^\top X\, dw$.
8. For $Z = WA$, prove $\partial J/\partial W = (\partial J/\partial Z)A^\top$.
9. Batch of n examples stored as rows of X: write $\partial J/\partial W$ using matrix multiplies only.
10. Softmax Jacobian: show diagonal $p_i(1-p_i)$ and off-diagonal $-p_i p_j$, then recover $p - y$ for CE.

Challenge

11. Show for scalar $a^\top w$, Hessian of $\frac{1}{2}(a^\top w)^2$ is $aa^\top$.
12. Derive gradient of multi-class CE through a linear layer $z = Wx$ (batch form).
13. Frobenius inner product: prove $\langle UV, W\rangle_F = \langle V, U^\top W\rangle_F$.
14. Implement (pseudocode) a finite-difference gradient check for a 2-layer MLP.
15. Explain why reverse-mode AD costs $O(1)$ forward-equivalents for scalar J regardless of parameter count (high-level argument).

Checks

2. $X^\top(Xw - y) + \lambda w$.
3. $\mathbb{R}^5$.

Summary

Matrix calculus for ML is shape-disciplined chain rules: linear maps push gradients through multiplies and transposes; nonlinearities contribute Hadamard factors; softmax+CE collapses to $p - y$. Learn a short identity table, verify with finite differences, and never invent a formula that fails a dimension check.

Automatic Differentiation from First Principles: Dual Numbers, the Tape, and Adjoint Greeks

Automatic Differentiation from First Principles

There are three ways to get a derivative out of a computer. **Symbolic differentiation** manipulates formulas, and the expressions can grow exponentially. **Finite differences** evaluate f twice and divide, which costs one extra evaluation per input and loses about half the digits to cancellation. **Automatic differentiation (AD)** does neither. It runs the program and applies the chain rule to each elementary operation as it executes, so the result is exact to floating-point rounding and costs a small constant factor more than the program itself.

AD comes in two modes, and choosing between them is the whole game. **Forward mode** pushes a tangent from the inputs to the outputs alongside the values, and dual numbers make it a class of about forty lines. **Reverse mode** records the computation on a *tape* and then pulls adjoints from the output back to every input in one sweep. Backpropagation in neural networks is reverse mode applied to a scalar loss. "Adjoint Greeks" in quantitative finance are reverse mode applied to a scalar price. They are the same algorithm under two names.

This chapter builds both modes from scratch, reproduces the worked example of the standard survey (Baydin et al., *JMLR* 2018), measures the n versus 1 cost asymmetry, computes four Black–Scholes sensitivities from one reverse sweep, and cross-checks everything against SymPy and PyTorch. The trap at the end is the one that catches practitioners: AD differentiates the program you wrote, not the function you meant.

All listings run on **CPython 3.13.5** in a venv with NumPy 2.5.4, SymPy 1.14.0 and PyTorch 2.14.1 (CPU), installed with `pip install numpy sympy torch`. Timings come from a shared 8-core box, so compare the ratios, not the absolute times.

1. Mental model: one graph, two directions

Any program that computes $y = f(x)$ from elementary operations can be written as a **Wengert list** (Wengert, 1964). This is a sequence of intermediate variables $v_i = \phi_i(v_j, v_k)$, each depending on earlier ones. The chain rule then gives two recurrences:

- **Forward (tangent) mode.** Carry $\dot{v}_i = \partial v_i/\partial x \cdot \dot{x}$ next to every v_i: $\dot{v}_i = \sum_j \frac{\partial \phi_i}{\partial v_j}\,\dot{v}_j$. One pass computes one Jacobian–vector product $J\dot{x}$.
- **Reverse (adjoint) mode.** After the forward pass, carry $\bar{v}_i = \partial y/\partial v_i$ backwards: $\bar{v}_j = \sum_{i:\, j\to i} \bar{v}_i \,\frac{\partial \phi_i}{\partial v_j}$, seeded with $\bar{y} = 1$. One sweep computes one vector–Jacobian product $\bar{y}^\top J$, which for scalar y is the whole gradient.

For $f : \mathbb{R}^n \to \mathbb{R}^m$ with operation count $\mathrm{ops}(f)$, the full Jacobian costs $n\,c\,\mathrm{ops}(f)$ in forward mode and $m\,c\,\mathrm{ops}(f)$ in reverse mode, "where c is a constant guaranteed to be $c < 6$ and typically $c \sim [2, 3]$" (Baydin et al., citing Griewank and Walther, 2008). Training a network has n in the billions and $m = 1$, which is why every deep-learning framework is built around reverse mode.

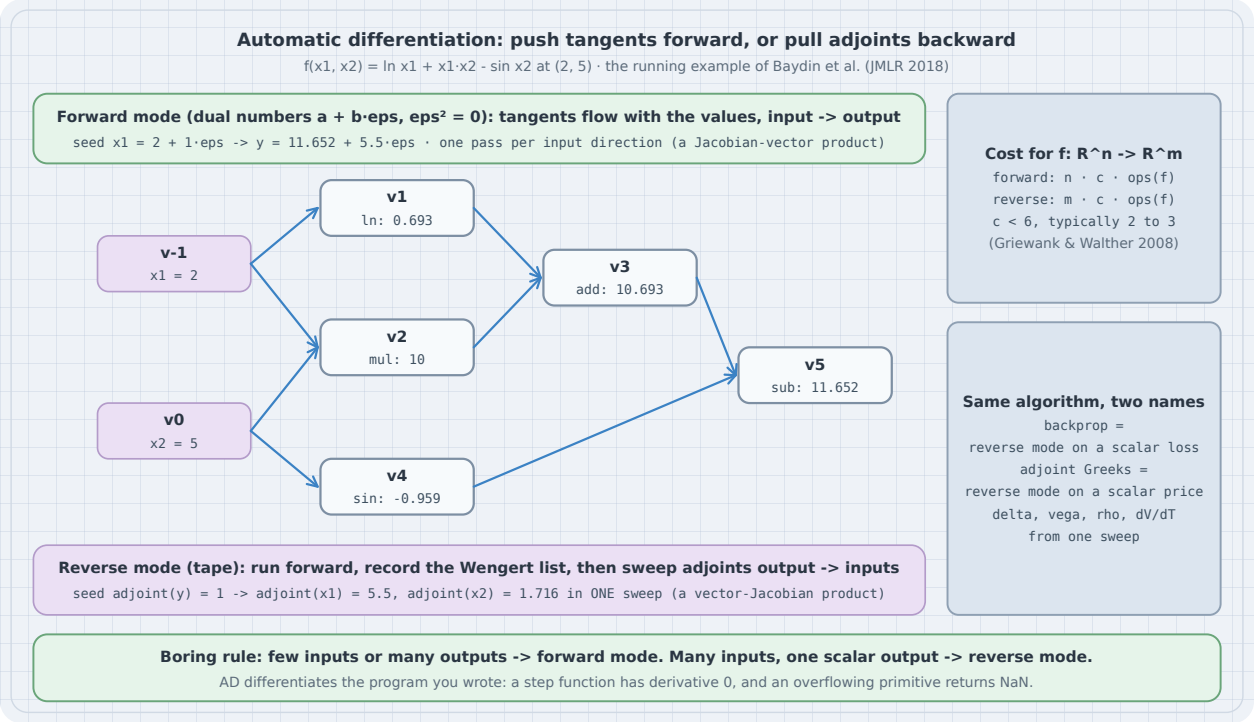


Figure 1: Forward and reverse mode on the survey's running example, the cost rule, and the backprop / adjoint-Greeks identity

2. Forward mode is arithmetic in $\mathbb{R}[\varepsilon]/(\varepsilon^2)$

A **dual number** is $a+b\varepsilon$ with $\varepsilon^2 = 0$. Multiplying two of them gives $(a+b\varepsilon)(c+d\varepsilon) = ac+(ad+bc)\varepsilon$, which is the product rule. More generally, for any smooth g, Taylor's theorem gives $g(a + b\varepsilon) = g(a) + g'(a)\, b\, \varepsilon$, because every higher power of ε vanishes. So if you evaluate f on $x + 1 \cdot \varepsilon$, the ε coefficient of the result is $f'(x)$. You only need to define each primitive's rule once.

```
# dual.py: forward-mode AD with dual numbers a + b*eps, eps**2 = 0
import math


class Dual:
    __slots__ = ("val", "eps")

    def __init__(self, val, eps=0.0):
        self.val, self.eps = val, eps

    @staticmethod
    def lift(x):
        return x if isinstance(x, Dual) else Dual(x, 0.0)

    def __add__(self, o):
```

```
        o = Dual.lift(o)
        return Dual(self.val + o.val, self.eps + o.eps)

    __radd__ = __add__

    def __sub__(self, o):
        o = Dual.lift(o)
        return Dual(self.val - o.val, self.eps - o.eps)

    def __rsub__(self, o):
        return Dual.lift(o) - self

    def __mul__(self, o):                       # (a + b eps)(c + d eps) = ac + (ad + bc) eps
        o = Dual.lift(o)
        return Dual(self.val * o.val, self.val * o.eps + self.eps * o.val)

    __rmul__ = __mul__

    def __truediv__(self, o):
        o = Dual.lift(o)
        return Dual(self.val / o.val, (self.eps * o.val - self.val * o.eps) / o.val ** 2)

    def __repr__(self):
        return f"{self.val:.6f} + {self.eps:.6f}eps"


def sin(x):
    x = Dual.lift(x)
    return Dual(math.sin(x.val), math.cos(x.val) * x.eps)


def log(x):
    x = Dual.lift(x)
    return Dual(math.log(x.val), x.eps / x.val)


def exp(x):
    x = Dual.lift(x)
    e = math.exp(x.val)
    return Dual(e, e * x.eps)


def f(x1, x2):                                  # the running example of Baydin et al. (2018)
    return log(x1) + x1 * x2 - sin(x2)


if __name__ == "__main__":
```

```
    # one forward pass per input direction
    print("f(2, 5) seeded on x1:", f(Dual(2.0, 1.0), Dual(5.0, 0.0)))
    print("f(2, 5) seeded on x2:", f(Dual(2.0, 0.0), Dual(5.0, 1.0)))
    # any direction at once: a Jacobian-vector product J @ [1, 2]
    print("directional derivative along (1, 2):", f(Dual(2.0, 1.0), Dual(5.0, 2.0)).eps)

    # it differentiates programs, loops included
    def newton_sqrt(a, steps=6):
        x = a / 2 if not isinstance(a, Dual) else Dual(a.val / 2, a.eps / 2)
        for _ in range(steps):
            x = 0.5 * (x + a / x)
        return x

    r = newton_sqrt(Dual(2.0, 1.0))
    print(f"d/da sqrt(a) at a=2 through 6 Newton steps: {r.eps:.15f}  exact 1/(2*sqrt 2) = {

    # check against SymPy
    import sympy as sp
    a, b = sp.symbols("x1 x2")
    expr = sp.log(a) + a * b - sp.sin(b)
    print("sympy:", [round(float(sp.diff(expr, v).subs({a: 2, b: 5})), 6) for v in (a, b)])
```

```
python dual.py
```

Output:

```
f(2, 5) seeded on x1: 11.652071 + 5.500000eps
f(2, 5) seeded on x2: 11.652071 + 1.716338eps
directional derivative along (1, 2): 8.932675629073547
d/da sqrt(a) at a=2 through 6 Newton steps: 0.353553390593274  exact 1/(2*sqrt 2) = 0.353553
sympy: [5.5, 1.716338]
```

The value 11.652071 and the derivatives 5.5 and 1.716338 are exactly the numbers in the survey's forward-mode table. Three points:

- **One pass per direction.** Getting both partials took two passes. Seeding $(\dot{x}_1, \dot{x}_2) = (1, 2)$ gives $5.5 + 2 \cdot 1.716 = 8.933$ in a single pass. That is a directional derivative (a Jacobian–vector product), and it is all one forward pass can give you.
- **Programs, not formulas.** `newton_sqrt` runs a loop with data-dependent arithmetic, and the dual number goes through it unchanged. Its derivative matches $1/(2\sqrt{2})$ to all 15 printed digits, because six Newton steps have converged. AD differentiates the *iteration*. Here the iteration has converged, so the result agrees with the derivative of the limit. That agreement is not automatic, as section 6 shows.
- **SymPy agrees**, but SymPy needed the closed-form expression. AD needed only the executable code.

3. Reverse mode: record a tape, sweep it backwards

Reverse mode needs the graph, so the forward pass records every operation together with its *local* partial derivatives. The backward pass walks that record in reverse order and accumulates $\bar{v}_{\text{parent}} \mathrel{+}= \bar{v}_{\text{child}} \cdot \partial(\text{child})/\partial(\text{parent})$. A node with several children, such as x_1 (used by `log` and by `mul`), receives the sum of their contributions, which is the multivariable chain rule. The implementation below accepts NumPy arrays as well as floats, so the same 117 lines handle the Monte Carlo example in section 5.

```
# tape.py: reverse-mode AD on a Wengert list (tape); values may be floats or NumPy arrays
import numpy as np

TAPE = []                                       # every Var created, in evaluation order


class Var:
    def __init__(self, value, parents=(), op="input"):
        self.value = np.asarray(value, dtype=float)
        self.parents = parents                  # tuple of (parent Var, local derivative)
        self.op = op
        self.grad = None
        TAPE.append(self)

    # elementary operations: value, plus the local partial derivative for each parent
    def __add__(self, o):
        o = lift(o)
        return Var(self.value + o.value, ((self, 1.0), (o, 1.0)), "add")

    __radd__ = __add__

    def __sub__(self, o):
        o = lift(o)
        return Var(self.value - o.value, ((self, 1.0), (o, -1.0)), "sub")

    def __rsub__(self, o):
        return lift(o) - self

    def __neg__(self):
        return Var(-self.value, ((self, -1.0),), "neg")

    def __mul__(self, o):
        o = lift(o)
        return Var(self.value * o.value, ((self, o.value), (o, self.value)), "mul")

    __rmul__ = __mul__

    def __truediv__(self, o):
```

```
        o = lift(o)
        return Var(self.value / o.value, ((self, 1 / o.value), (o, -self.value / o.value **

    def __rtruediv__(self, o):
        return lift(o) / self


def lift(x):
    return x if isinstance(x, Var) else Var(x, (), "const")


def exp(x):
    x = lift(x)
    e = np.exp(x.value)
    return Var(e, ((x, e),), "exp")


def log(x):
    x = lift(x)
    return Var(np.log(x.value), ((x, 1 / x.value),), "log")


def sin(x):
    x = lift(x)
    return Var(np.sin(x.value), ((x, np.cos(x.value)),), "sin")


def sqrt(x):
    x = lift(x)
    s = np.sqrt(x.value)
    return Var(s, ((x, 0.5 / s),), "sqrt")


def relu(x):                                        # max(x, 0): derivative 1 where x > 0, else
    x = lift(x)
    return Var(np.maximum(x.value, 0.0), ((x, (x.value > 0).astype(float)),), "relu")


def step(x):                                        # 1{x > 0}: derivative 0 almost everywhere
    x = lift(x)
    return Var((x.value > 0).astype(float), ((x, np.zeros_like(x.value)),), "step")


def sigmoid(x):                                     # 1/(1+e^-x) without overflow; derivative s(
    x = lift(x)
    v = x.value
    e = np.exp(-np.abs(v))
```

```
    s = np.where(v >= 0, 1 / (1 + e), e / (1 + e))
    return Var(s, ((x, s * (1 - s)),), "sigmoid")


def mean(x):
    x = lift(x)
    n = x.value.size
    return Var(x.value.mean(), ((x, np.full(x.value.shape, 1.0 / n)),), "mean")


def unbroadcast(g, shape):
    """Sum a gradient back down to the shape of the value it belongs to."""
    while g.ndim > len(shape):
        g = g.sum(axis=0)
    for i, s in enumerate(shape):
        if s == 1 and g.shape[i] != 1:
            g = g.sum(axis=i, keepdims=True)
    return g


def backward(out):
    """One reverse sweep: adjoint(v) = sum over children c of adjoint(c) * dc/dv."""
    for v in TAPE:
        v.grad = np.zeros_like(v.value)
    out.grad = np.ones_like(out.value)
    for v in reversed(TAPE):                    # the tape is already a topological order
        for p, local in v.parents:
            p.grad = p.grad + unbroadcast(v.grad * local, p.value.shape)


def reset():
    TAPE.clear()
```

```
# tape_demo.py: the Wengert list for f(x1, x2) = ln x1 + x1*x2 - sin x2, forward and reverse
import tape as ad

x1, x2 = ad.Var(2.0), ad.Var(5.0)
y = ad.log(x1) + x1 * x2 - ad.sin(x2)
ad.backward(y)

names = {id(v): f"v{i - 1}" for i, v in enumerate(ad.TAPE)}
print(f"{'node':>5} {'op':>6} {'inputs':>10} {'value':>10} {'adjoint':>10}")
for v in ad.TAPE:
    ins = ",".join(names[id(p)] for p, _ in v.parents)
    print(f"{names[id(v)]:>5} {v.op:>6} {ins:>10} {float(v.value):10.4f} {float(v.grad):10.4
print(f"\ny = {float(y.value):.4f}   dy/dx1 = {float(x1.grad):.4f}   dy/dx2 = {float(x2.grad
```

```
import torch
t1 = torch.tensor(2.0, dtype=torch.float64, requires_grad=True)
t2 = torch.tensor(5.0, dtype=torch.float64, requires_grad=True)
ty = torch.log(t1) + t1 * t2 - torch.sin(t2)
ty.backward()
print(f"torch {torch.__version__}: dy/dx1 = {t1.grad.item():.4f}   dy/dx2 = {t2.grad.item():
```

```
python tape_demo.py
```

Output:

```
 node     op     inputs      value    adjoint
  v-1  input                2.0000     5.5000
   v0  input                5.0000     1.7163
   v1    log        v-1     0.6931     1.0000
   v2    mul     v-1,v0    10.0000     1.0000
   v3    add      v1,v2    10.6931     1.0000
   v4    sin         v0    -0.9589    -1.0000
   v5    sub      v3,v4    11.6521     1.0000

y = 11.6521   dy/dx1 = 5.5000   dy/dx2 = 1.7163
torch 2.14.1+cu130: dy/dx1 = 5.5000   dy/dx2 = 1.7163
```

The printed tape is the survey's three-part table: the node names $v_{-1}, v_0, \dots, v_5$, the primal values, and the adjoints $\bar{v}_4 = -1$, $\bar{v}_0 = \bar{v}_2 \cdot x_1 + \bar{v}_4 \cos x_2 = 2 - \cos 5 \cdot 1 = 1.7163$. Both partials come from one sweep. PyTorch's `autograd` builds the same kind of graph and gives the same two numbers.

4. The cost asymmetry, measured

Take $f(x) = \sum_i \sin(x_i)\, x_{i+1} + \log(1 + x_i^2)$ with n inputs. A forward-mode gradient needs n passes over $O(n)$ work, which is $O(n^2)$. Reverse mode needs one forward pass and one backward pass, which is $O(n)$.

```
# cost.py: gradient of f: R^n -> R by n forward-mode passes vs one reverse sweep
import math
import random
import time

import dual
import tape as ad


def f(xs, sin, log):
    n = len(xs)
    total = 0.0
```

```
    for i in range(n):
        total = total + sin(xs[i]) * xs[(i + 1) % n] + log(1.0 + xs[i] * xs[i])
    return total


def grad_forward(x):
    g = []
    for j in range(len(x)):                          # one pass per input direction
        xs = [dual.Dual(v, 1.0 if i == j else 0.0) for i, v in enumerate(x)]
        g.append(f(xs, dual.sin, dual.log).eps)
    return g


def grad_reverse(x):
    ad.reset()
    xs = [ad.Var(v) for v in x]
    y = f(xs, ad.sin, ad.log)
    ops = len(ad.TAPE)
    ad.backward(y)
    return [float(v.grad) for v in xs], ops


def plain(x):
    return f(x, math.sin, math.log)


rng = random.Random(1)
print(f"{'n':>5} {'tape nodes':>10} {'fwd-mode s':>11} {'rev-mode s':>11} {'plain f s':>10}
for n in (10, 100, 1000):
    x = [rng.uniform(-2, 2) for _ in range(n)]
    t0 = time.perf_counter(); gf = grad_forward(x); t1 = time.perf_counter()
    gr, ops = grad_reverse(x); t2 = time.perf_counter()
    plain(x); t3 = time.perf_counter()
    diff = max(abs(a - b) for a, b in zip(gf, gr))
    print(f"{n:5d} {ops:10d} {t1 - t0:11.4f} {t2 - t1:11.4f} {t3 - t2:10.6f} {diff:11.1e}")

# where forward mode wins: g: R -> R^m (one input, many outputs) needs one dual pass
m = 1000
t0 = time.perf_counter()
out = [dual.sin(dual.Dual(0.3, 1.0) * k) for k in range(1, m + 1)]   # d/dt sin(k t) = k cos
t1 = time.perf_counter()
print(f"\nR -> R^{m}: one forward pass gives all {m} derivatives in {t1 - t0:.4f} s;"
      f" check out[9].eps = {out[9].eps:.6f} vs 10*cos(3) = {10 * math.cos(3.0):.6f}")
```

```
python cost.py
```

Output:

```
    n tape nodes  fwd-mode s  rev-mode s  plain f s  max |diff|
   10         91      0.0002      0.0004   0.000004     2.2e-16
  100        901      0.0179      0.0029   0.000016     4.4e-16
 1000       9001      1.5274      0.0260   0.000095     4.4e-16

R -> R^1000: one forward pass gives all 1000 derivatives in 0.0006 s; check out[9].eps = -9.
```

Going from $n = 100$ to $n = 1000$ multiplies forward mode's time by about $100\times$ (quadratic), while reverse mode's time grows by about $10\times$ (linear). Both gradients agree to 4×10^{-16}. The last line is the mirror case: for $g : \mathbb{R} \to \mathbb{R}^{1000}$, one forward pass gives all 1000 derivatives, where reverse mode would need 1000 sweeps.

The "plain f" column is the honest footnote. In pure Python, the tape costs hundreds of times more than evaluating f, because every operation allocates a `Var` and a NumPy scalar. The $c < 6$ bound counts *arithmetic*, not interpreter overhead. Production AD systems get close to the bound by working on whole tensors per operation (PyTorch, JAX) or by transforming source code at compile time. Section 5 already benefits from this: there, one tape node holds 400,000 values.

5. Adjoint Greeks: backprop for a price

Under Black–Scholes, $S_T = S_0 \exp\left((r - \frac{1}{2}\sigma^2)T + \sigma\sqrt{T}Z\right)$ with $Z \sim N(0,1)$. A European call is worth $V = \mathbb{E}\left[e^{-rT}\max(S_T - K, 0)\right]$. A Monte Carlo pricer averages the discounted payoff over many draws of Z. Its sensitivities (the **Greeks**: delta $\partial V/\partial S_0$, vega $\partial V/\partial\sigma$, rho $\partial V/\partial r$, and $\partial V/\partial T$) are a gradient of one scalar with respect to several inputs. That is reverse mode's best case. Giles and Glasserman's "Smoking adjoints" (2006) brought this observation to finance: the pathwise derivatives of every input come from one adjoint sweep, at a cost independent of how many inputs there are.

```
# greeks.py: adjoint (reverse-mode) Monte Carlo Greeks for a European call, vs Black-Scholes
import math

import numpy as np

import tape as ad


def Phi(x):
    return 0.5 * (1 + math.erf(x / math.sqrt(2)))


def phi(x):
    return math.exp(-x * x / 2) / math.sqrt(2 * math.pi)


def bs_call(S, K, r, sig, T):
    d1 = (math.log(S / K) + (r + sig ** 2 / 2) * T) / (sig * math.sqrt(T))
```

```
    d2 = d1 - sig * math.sqrt(T)
    price = S * Phi(d1) - K * math.exp(-r * T) * Phi(d2)
    return price, {"delta": Phi(d1), "vega": S * phi(d1) * math.sqrt(T),
                   "rho": K * T * math.exp(-r * T) * Phi(d2),
                   "dT": S * phi(d1) * sig / (2 * math.sqrt(T)) + r * K * math.exp(-r * T) *


def mc_call(params, Z, K):
    S0, r, sig, T = params
    ST = S0 * ad.exp((r - sig * sig * 0.5) * T + sig * ad.sqrt(T) * Z)
    return ad.mean(ad.exp(-r * T) * ad.relu(ST - K))


S0, K, r, sig, T = 100.0, 105.0, 0.03, 0.2, 1.0
Z = np.random.default_rng(7).standard_normal(400_000)

ad.reset()
params = [ad.Var(v) for v in (S0, r, sig, T)]
price = mc_call(params, Z, K)
ad.backward(price)                                    # ONE reverse sweep -> every sensitivit
mc = dict(zip(("delta", "rho", "vega", "dT"), (float(p.grad) for p in params)))

exact_price, exact = bs_call(S0, K, r, sig, T)
print(f"paths {Z.size}, tape nodes {len(ad.TAPE)}")
print(f"{'':>7} {'Monte Carlo + AD':>17} {'Black-Scholes':>14}")
print(f"{'price':>7} {float(price.value):17.4f} {exact_price:14.4f}")
for k in ("delta", "vega", "rho", "dT"):
    print(f"{k:>7} {mc[k]:17.4f} {exact[k]:14.4f}")
```

```
python greeks.py
```

Output:

```
paths 400000, tape nodes 24
         Monte Carlo + AD  Black-Scholes
  price            7.1205         7.1281
  delta            0.5027         0.5024
   vega           39.8508        39.8935
    rho           43.1492        43.1133
     dT            5.2796         5.2827
```

One backward sweep over a 24-node tape (each node a vector of 400,000 paths) produced all four sensitivities. They agree with the closed-form Black–Scholes values to Monte Carlo accuracy (the price itself is off by 0.008, well inside one Monte Carlo standard error of 0.020). The listing hard-codes the `max` payoff's derivative as $\mathbf{1}\{S_T > K\}$, which is the *pathwise* estimator. It is valid here because the payoff is continuous in S_0. PyTorch, given the same random numbers, gives the same Greeks to every printed digit:

```
# torch_check.py: the same adjoint Greeks from PyTorch's reverse mode, and a forward-mode JV
import numpy as np
import torch
from torch.func import jvp

Z = torch.from_numpy(np.random.default_rng(7).standard_normal(400_000))   # same paths as gr
K = 105.0


def price(S0, r, sig, T):
    ST = S0 * torch.exp((r - sig * sig / 2) * T + sig * torch.sqrt(T) * Z)
    return (torch.exp(-r * T) * torch.relu(ST - K)).mean()


args = [torch.tensor(v, dtype=torch.float64, requires_grad=True) for v in (100.0, 0.03, 0.2,
p = price(*args)
p.backward()
print("torch", torch.__version__, "reverse mode:",
      dict(zip(("delta", "rho", "vega", "dT"), (round(a.grad.item(), 4) for a in args))))

primals = tuple(a.detach() for a in args)
tangent = tuple(torch.tensor(t, dtype=torch.float64) for t in (1.0, 0.0, 0.0, 0.0))
_, d = jvp(price, primals, tangent)                 # one forward pass = one directional der
print(f"forward mode (jvp along S0): delta = {d.item():.4f}")
```

```
python -W ignore::FutureWarning torch_check.py
```

Output:

```
torch 2.14.1+cu130 reverse mode: {'delta': 0.5027, 'rho': 43.1492, 'vega': 39.8508, 'dT': 5.
forward mode (jvp along S0): delta = 0.5027
```

`torch.func.jvp` is PyTorch's forward mode, and it gives the same delta from one forward pass along S_0. To get four Greeks by forward mode you would need four such passes. A real trading book has thousands of inputs (every point on every curve), so the reverse sweep's advantage grows with the size of the book.

6. The trap: AD differentiates the program, not the mathematics

Swap the call for a **digital** option that pays 1 if $S_T > K$. The price $e^{-rT}\,\mathbb{P}(S_T > K)$ is a smooth function of S_0 with a strictly positive delta. The *program*, however, computes `step(S_T - K)`, and that step's derivative is 0 everywhere it exists.

```
# digital.py: the trap. AD differentiates the program, and a step function's derivative is 0
import math
```

```
import numpy as np

import tape as ad

S0, K, r, sig, T = 100.0, 105.0, 0.03, 0.2, 1.0
d2 = (math.log(S0 / K) + (r - sig ** 2 / 2) * T) / (sig * math.sqrt(T))
exact_delta = math.exp(-r * T) * math.exp(-d2 * d2 / 2) / math.sqrt(2 * math.pi) / (S0 * sig


def digital(S, Z, payoff):
    ST = S * ad.exp((r - sig * sig / 2) * T + sig * math.sqrt(T) * Z)
    return ad.mean(math.exp(-r * T) * payoff(ST - K))


def ad_delta(Z, payoff):
    ad.reset()
    S = ad.Var(S0)
    price = digital(S, Z, payoff)
    ad.backward(price)
    return float(price.value), float(S.grad)


def naive_smooth(w):                                # sigmoid((S_T - K) / w), written the tex
    return lambda x: 1.0 / (1.0 + ad.exp(-x / w))


def stable_smooth(w):                               # same function, overflow-free primitive
    return lambda x: ad.sigmoid(x / w)


print(f"exact Black-Scholes delta of the digital call: {exact_delta:.5f}\n")
for seed in (1, 2):
    Z = np.random.default_rng(seed).standard_normal(400_000)
    p, d = ad_delta(Z, ad.step)
    ad.reset()
    h = 0.5                                         # bump-and-revalue, same random numbers
    up = float(digital(ad.Var(S0 + h), Z, ad.step).value)
    dn = float(digital(ad.Var(S0 - h), Z, ad.step).value)
    print(f"seed {seed}: price {p:.5f}  AD delta (step) {d:.5f}  central difference h={h}: {
    for w in (5.0, 1.0, 0.2, 0.05):
        with np.errstate(over="ignore", invalid="ignore"):
            _, naive = ad_delta(Z, naive_smooth(w))
        _, stable = ad_delta(Z, stable_smooth(w))
        print(f"         sigmoid width {w:>4}: AD delta naive {naive:.5f}   stable {stable:.
```

```
python digital.py
```

Output:

```
exact Black-Scholes delta of the digital call: 0.01900

seed 1: price 0.40989  AD delta (step) 0.00000  central difference h=0.5: 0.01873
         sigmoid width  5.0: AD delta naive 0.01762   stable 0.01762
         sigmoid width  1.0: AD delta naive 0.01902   stable 0.01902
         sigmoid width  0.2: AD delta naive 0.01880   stable 0.01880
         sigmoid width 0.05: AD delta naive nan   stable 0.01888
seed 2: price 0.41168  AD delta (step) 0.00000  central difference h=0.5: 0.01906
         sigmoid width  5.0: AD delta naive 0.01759   stable 0.01759
         sigmoid width  1.0: AD delta naive 0.01904   stable 0.01904
         sigmoid width  0.2: AD delta naive 0.01910   stable 0.01910
         sigmoid width 0.05: AD delta naive nan   stable 0.01937
```

- **Exact zero, silently.** The AD delta is `0.00000` for both seeds. No warning appears and no NaN is produced: it is a confident, wrong answer. The pathwise method requires the payoff to be (Lipschitz) continuous, and a digital payoff is not. The same failure appears with `round`, `argmax`, integer casts, `if x > threshold:` branches, and sorting, in any AD system.
- **The fixes all trade something.** Bump-and-revalue with common random numbers works here, but costs two extra pricings per input. Smoothing the step with a sigmoid of width w restores a derivative, with bias at large w (0.0176 against 0.0190 at $w = 5$) and variance at small w (0.0189 and 0.0194 for the two seeds at $w = 0.05$). The likelihood-ratio method, which differentiates the density instead of the payoff, is the standard unbiased alternative.
- **The second trap is in the same table.** Written the textbook way, $1/(1 + e^{-x/w})$, the sigmoid overflows to `inf` for paths deep out of the money at $w = 0.05$. Its local derivative becomes `inf · 0 = nan`, and one NaN poisons the mean. The overflow-free primitive (`ad.sigmoid`, which uses $e^{-|x|}$) is the same function and gives the right answer. AD is exact for the operations you actually execute, so numerically unstable code produces unstable derivatives.

The boring rules:

1. **Pick the mode by shape.** Few inputs, or a directional derivative: forward mode (`jvp`). Many inputs and a scalar output: reverse mode (`vjp`, `grad`, `backward`). For Hessian–vector products, run forward over reverse.
2. **Check every new gradient once** against central differences at a random point, in float64.
3. **Look for piecewise-constant code on the path to the output**: steps, comparisons, rounding, indexing by a computed integer. AD returns zero through all of them. Smooth them, or use an estimator that does not differentiate them.
4. **Use stable primitives** (`log1p`, `logsumexp`, `sigmoid`, `hypot`). The survey makes the same point: the derivative inherits the program's numerics.

7. Try this

1. Add `cos`, `tanh` and `pow` to `dual.py` and `tape.py`, and check $\frac{d}{dx}x^x$ at $x = 1.5$ against SymPy.

2. Nest forward mode: compute $f''(x)$ for $f(x) = x \sin x$ by making a `Dual` whose parts are themselves `Dual`s. Then read about *perturbation confusion* (Siskind and Pearlmutter) and build an example where two untagged εs get mixed up.
3. Extend `greeks.py` to an arithmetic-average Asian call with 52 monitoring dates. How large is the tape, and how long does the reverse sweep take compared with the forward pricing?
4. Compute gamma ($\partial^2 V / \partial S_0^2$) for the European call by applying `tape.py` to the pathwise delta expression. Explain the result using section 6.
5. Replace `newton_sqrt`'s fixed 6 steps with "stop when `abs(x*x - a) < 1e-3`" and differentiate at $a = 2$. How far is the derivative from $1/(2\sqrt{2})$, and why?

Checkpoint

- State the dual-number rule $g(a + b\varepsilon) = g(a) + g'(a) b\varepsilon$ and use it to derive the product and quotient rules.
- Write the forward and reverse recurrences on a Wengert list, and say which one computes $J\dot{x}$ and which computes $\bar{y}^\top J$.
- Explain why the gradient of a scalar loss with n parameters costs $O(1)$ evaluations in reverse mode and $O(n)$ in forward mode.
- Explain why backprop and adjoint Greeks are the same algorithm.
- Give two programs whose AD derivative is wrong even though every primitive is implemented correctly.

Summary

AD applies the chain rule to the executed program, one elementary operation at a time. Forward mode is arithmetic on dual numbers and gives one directional derivative per pass. Reverse mode records a tape and gives a whole gradient per sweep, at a cost within a small constant factor of the program (Griewank and Walther's $c < 6$). That constant is the reason training billion-parameter networks and computing thousand-input risk are both feasible. AD is exact for the program you wrote, which also means it is exactly wrong when the program contains steps, branches or unstable primitives.

Sources

- A. G. Baydin, B. A. Pearlmutter, A. A. Radul, J. M. Siskind, "Automatic differentiation in machine learning: a survey", *Journal of Machine Learning Research* 18(153):1–43, 2018, arXiv:1502.05767. Running example $f(x_1, x_2) = \ln x_1 + x_1 x_2 - \sin x_2$ (Tables 2–3), the cost statement $c < 6$, $c \sim [2, 3]$ (§3.2), perturbation confusion and numerics (§5).
- A. Griewank, A. Walther, *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation*, 2nd ed., SIAM, 2008.
- R. E. Wengert, "A simple automatic derivative evaluation program", *Communications of the ACM* 7(8):463–464, 1964.

- M. Giles, P. Glasserman, "Smoking adjoints: fast Monte Carlo Greeks", *Risk*, January 2006. P. Glasserman, *Monte Carlo Methods in Financial Engineering*, Springer, 2003, ch. 7 (pathwise and likelihood-ratio estimators, and why the pathwise method needs continuous payoffs).
- J. M. Siskind, B. A. Pearlmutter, "Perturbation confusion and referential transparency", IFL 2005.
- PyTorch 2.14 documentation: `torch.autograd` and `torch.func.jvp` / `vjp`.

Algorithms and Complexity

Algorithms and Computational Complexity

Algorithms transform inputs into outputs with finite resources. **Complexity theory** classifies how those resources scale and which problems admit efficient solutions at all. This part builds the mathematical toolkit for analyzing algorithms (asymptotics, recurrences, amortization) and for reasoning about hardness (classes, reductions, randomization, approximation).

Why complexity is practical

Question	Complexity answer
Will this service hold at 100× traffic?	Growth rate of time/memory vs n
Is a faster algorithm possible?	Lower bounds / conditional hardness
Exact optimum too slow—what now?	Approximation, heuristics, parameterization
Can randomness help?	Monte Carlo / Las Vegas; RP, BPP intuition
Is my proof of "NP-complete" valid?	Reduction hygiene

Complexity is not only academia: billing, capacity planning, cryptography assumptions, and compiler optimization all rest on resource reasoning.

Next step: for hardness proofs and space classes, continue in **part 19 Complexity-Theory** (this part emphasizes analyzing concrete algorithms).

Two complementary activities

1. Algorithm analysis

Given a **specific procedure**, bound its worst-case (or average, amortized) time and space as functions of input size n.

Tools:

- Asymptotic notation O, Ω, Θ
- Loop counting and recurrence solving (Master theorem, substitution, trees)
- Amortized analysis (aggregate, accounting, potential)
- High-probability bounds for randomized algorithms

2. Problem complexity

Given a **problem** (a language or function), classify its intrinsic difficulty independent of any one algorithm.

Tools:

- Models of computation (Turing machines, RAM) at a conceptual level
- Classes $\mathbf{P}, \mathbf{NP}, \mathbf{PSPACE}, \ldots$
- Polynomial-time reductions
- Completeness (NP-complete, PSPACE-complete)
- Approximation classes and hardness of approximation

Worked example 1

Sorting n comparable keys: many $O(n \log n)$ algorithms; information-theoretic lower bound $\Omega(n \log n)$ for comparison sorts. Problem complexity and algorithm analysis meet.

Worked example 2

SAT: verify a satisfying assignment in poly time (in **NP**); no known poly-time algorithm; NP-complete. Algorithm analysis of DPLL/CDCL is separate from the class-theoretic status.

Models and cost

We count elementary operations in a RAM-like model unless stated: arithmetic on words, memory access, comparisons. Bit complexity matters for big integers and crypto. Parallel and distributed models add communication rounds and processors.

Input size n: bits of the encoding, or number of elements for arrays/graphs with weights assumed unit-cost when standard.

Worked example 3

Graph with v vertices, e edges: BFS is $O(v + e)$ time with adjacency lists—state both parameters, not a vague $O(n)$.

Roadmap of this part

1. **Asymptotic analysis** — $O/\Omega/\Theta$, recurrences, amortization, practical vs asymptotic
2. **Complexity classes and reductions** — **P**, **NP**, poly-time reductions, completeness method
3. **Randomized and approximation algorithms** — error amplification, approximation ratios, tradeoffs

Section `19-Complexity-Theory` deepens NP-completeness proofs, space classes, and advanced randomized/approximation themes.

Correctness before speed

An $O(1)$ wrong algorithm is worthless. Standard proof patterns:

- Loop invariants
- Induction on input size
- Exchange arguments (greedy)
- Potential functions (amortized + correctness together)

Worked example 4 — invariant

Binary search: "target is in `lo..hi` if present." Each step preserves the invariant and shrinks the interval $\Rightarrow$ correctness + $O(\log n)$ iterations.

Average, worst, high probability

- **Worst-case:** max cost over inputs of size n — standard for guarantees
- **Average-case:** expectation under a distribution — needs a realistic model
- **High-probability:** randomized algorithms with failure prob $\leq n^{-c}$
- **Amortized:** average over a **sequence** of operations (not over random inputs)

Worked example 5

Hash table insert: worst-case $O(n)$ on bad chains; expected $O(1)$ with good hashing; amortized $O(1)$ for dynamic array **regardless of randomness** when doubling.

Reductions as a reusable idea

A reduction from problem A to problem B is an efficient transform showing "B is at least as hard as A" (for decision problems, in the usual many-one sense). Used for:

- Reusing algorithms ($A \leq B$ and B solvable $\Rightarrow$ A solvable)
- Proving hardness (A hard and $A \leq B \Rightarrow$ B hard)

Worked example 6

If you poly-time reduce SAT to your new scheduling decision problem and show the latter is in **NP**, you get NP-hardness / completeness (with care).

Approximation and heuristics

For NP-hard optimization:

Approach	Guarantee
Exact exponential (ILP, DP on subsets)	Optimal; limited n
Approximation algorithm	Provable ratio
FPTAS / PTAS	Near-optimal with time tradeoffs
Heuristic / ML-based	Often good; no worst-case proof

Worked example 7

Vertex cover: greedy 2-approximation vs exact exponential ILP—pick based on n and need for optimality certificates.

Pitfalls

1. **Dropping parameters** ($O(n)$ for graphs without e)
2. **Claiming Θ without matching lower bound**
3. **Average-case without stating the distribution**
4. **Reduction in the wrong direction** for hardness proofs
5. **Confusing NP-hard decision vs optimization** versions
6. **Ignoring constants and locality** when n is moderate (cache, constants matter)

Checkpoint

- Separate algorithm analysis from problem classification
- State time bounds with correct parameters
- Know when to use worst-case vs amortized vs high-probability
- Sketch what a poly-time reduction is for
- Name strategies when exact poly-time algorithms are unlikely

Exercises

1. Give worst-case time for inserting into a balanced BST vs unsorted array (search+insert).
2. Why is "average runtime $O(1)$" incomplete without a probability model?
3. Explain amortized vs average-case using dynamic arrays vs hashing.
4. Binary search recurrence $T(n) = T(n/2) + \Theta(1)$: solve by unrolling.
5. List three problems in **P** and one believed not in **P**.
6. What does it mean that problem B is NP-hard?
7. Give a case where an $O(n^2)$ algorithm beats $O(n \log n)$ in practice (constants/input sizes).
8. For matrix multiplication, state naive complexity and note that better exponents exist (no need to prove).
9. Write a loop invariant for insertion sort's outer loop.
10. Why is verifying a graph coloring with k colors easy but finding one potentially hard?
11. Define approximation ratio for a minimization problem.
12. Sketch how randomness helps in checking polynomial identity (Schwartz–Zippel intuition).
13. Capacity planning: if latency grows as $\Theta(n^2)$, what happens when n doubles?
14. Explain why cryptographic hardness assumptions are **stronger/different** than "not known to be in P."
15. Pick a feature in a system you know (routing, build system, DB) and identify the core algorithmic problem + resource metric.

Summary

Complexity mathematics lets you predict scale, prove limits, and choose between exact, approximate, and randomized solutions. The following chapters turn these ideas into precise definitions, proof templates, and worked analyses.

Asymptotic Analysis and Big O Notation

Asymptotic analysis provides a mathematical framework for describing the growth rate of functions, particularly the time and space complexity of algorithms. It allows us to compare algorithms and predict their performance on large inputs.

Big O Notation

Definition

For functions f(n) and g(n), we say $\mathbf{f(n) = O(g(n))}$ if there exist positive constants c and n such that:

f(n) **c** · **g(n)** for all n n

This means f(n) grows no faster than g(n) asymptotically.

```
import numpy as np
import matplotlib.pyplot as plt

def plot_big_o_examples():
    """Visualize different growth rates"""
    n = np.linspace(1, 100, 1000)

    functions = {
        'O(1)': np.ones_like(n),
        'O(log n)': np.log2(n),
        'O(n)': n,
        'O(n log n)': n * np.log2(n),
        'O(n²)': n**2,
        'O(n³)': n**3,
        'O(2 )': 2**np.minimum(n/10, 20)  # Scaled to fit plot
    }

    plt.figure(figsize=(12, 8))

    for name, values in functions.items():
        plt.plot(n, values, label=name, linewidth=2)

    plt.xlabel('Input Size (n)')
    plt.ylabel('Operations')
```

```
    plt.title('Common Time Complexity Growth Rates')
    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.xlim(1, 100)
    plt.ylim(1, 1000)
    plt.yscale('log')
    plt.show()

plot_big_o_examples()
```

Common Complexity Classes

```
def analyze_complexity_examples():
    """Analyze time complexity of common algorithms"""

    examples = [
        ("Array access", "O(1)", "arr[i]"),
        ("Linear search", "O(n)", "for i in range(n): if arr[i] == target"),
        ("Binary search", "O(log n)", "Divide search space in half each step"),
        ("Merge sort", "O(n log n)", "Divide and conquer sorting"),
        ("Bubble sort", "O(n²)", "Nested loops comparing adjacent elements"),
        ("Matrix multiplication", "O(n³)", "Three nested loops"),
        ("Traveling salesman (brute force)", "O(n!)", "Try all permutations"),
        ("Subset sum (brute force)", "O(2 )", "Try all possible subsets")
    ]

    print("Common Algorithm Complexities:")
    print("-" * 60)
    for name, complexity, description in examples:
        print(f"{name:25} {complexity:10} {description}")

analyze_complexity_examples()
```

Other Asymptotic Notations

Big Omega (Ω) - Lower Bound

$f(n) = \Omega(g(n))$ means f(n) grows at least as fast as g(n).

Big Theta (Θ) - Tight Bound

$f(n) = \Theta(g(n))$ means f(n) grows exactly as fast as g(n). This is equivalent to f(n) = O(g(n)) AND f(n) = Ω(g(n)).

```
def demonstrate_asymptotic_bounds():
    """Demonstrate different asymptotic bounds"""

    def f1(n):
        return 3*n**2 + 2*n + 1

    def f2(n):
        return n**2

    def f3(n):
        return n

    n_values = np.array([10, 100, 1000, 10000])

    print("Asymptotic Bounds Example:")
    print("f(n) = 3n² + 2n + 1")
    print("-" * 40)
    print(f"{'n':>6} {'f(n)':>10} {'n²':>10} {'n':>10}")
    print("-" * 40)

    for n in n_values:
        print(f"{n:>6} {f1(n):>10.0f} {f2(n):>10.0f} {f3(n):>10.0f}")

    print("\nObservations:")
    print("- f(n) = O(n²) because f(n)  4n² for large n")
    print("- f(n) = Ω(n²) because f(n)  3n² for large n")
    print("- f(n) = Θ(n²) because it's both O(n²) and Ω(n²)")

demonstrate_asymptotic_bounds()
```

Analyzing Algorithm Complexity

Example 1: Linear Search

```
def linear_search_analysis():
    """Analyze linear search complexity"""

    def linear_search(arr, target):
        """Linear search with operation counting"""
        operations = 0
        for i in range(len(arr)):
            operations += 1  # Comparison operation
            if arr[i] == target:
                return i, operations
```

```
            return -1, operations

    # Test with different array sizes
    sizes = [10, 100, 1000, 10000]
    results = []

    for size in sizes:
        arr = list(range(size))

        # Best case: target at beginning
        _, best_ops = linear_search(arr, 0)

        # Worst case: target not found
        _, worst_ops = linear_search(arr, -1)

        # Average case: target in middle
        _, avg_ops = linear_search(arr, size // 2)

        results.append((size, best_ops, avg_ops, worst_ops))

    print("Linear Search Complexity Analysis:")
    print("-" * 50)
    print(f"{'Size':>6} {'Best':>8} {'Average':>8} {'Worst':>8}")
    print("-" * 50)

    for size, best, avg, worst in results:
        print(f"{size:>6} {best:>8} {avg:>8} {worst:>8}")

    print("\nComplexity:")
    print("Best case: O(1) - target at first position")
    print("Average case: O(n) - target in middle")
    print("Worst case: O(n) - target not found")

linear_search_analysis()
```

Example 2: Nested Loops

```
def nested_loops_analysis():
    """Analyze algorithms with nested loops"""

    def count_operations(n, algorithm):
        """Count operations for different nested loop patterns"""
        if algorithm == "double_loop":
            # for i in range(n):
            #     for j in range(n):
```

```
            #         operation()
            return n * n

        elif algorithm == "triangular":
            # for i in range(n):
            #     for j in range(i):
            #         operation()
            return n * (n - 1) // 2

        elif algorithm == "triple_loop":
            # for i in range(n):
            #     for j in range(n):
            #         for k in range(n):
            #             operation()
            return n * n * n

    sizes = [10, 20, 50, 100]
    algorithms = [
        ("Double loop", "double_loop", "O(n²)"),
        ("Triangular", "triangular", "O(n²)"),
        ("Triple loop", "triple_loop", "O(n³)")
    ]

    print("Nested Loop Complexity Analysis:")
    print("-" * 60)

    for name, algo, complexity in algorithms:
        print(f"\n{name} - {complexity}:")
        print(f"{'n':>6} {'Operations':>12} {'Ratio':>8}")
        print("-" * 30)

        prev_ops = None
        for n in sizes:
            ops = count_operations(n, algo)
            ratio = ops / prev_ops if prev_ops else 1
            print(f"{n:>6} {ops:>12} {ratio:>8.1f}")
            prev_ops = ops

nested_loops_analysis()
```

Recurrence Relations

Master Theorem

For recurrences of the form: $\mathbf{T(n) = aT(n/b) + f(n)}$

```
def master_theorem_examples():
    """Examples of Master Theorem applications"""

    examples = [
        {
            'name': 'Binary Search',
            'recurrence': 'T(n) = T(n/2) + O(1)',
            'a': 1, 'b': 2, 'f': 'O(1)',
            'result': 'O(log n)'
        },
        {
            'name': 'Merge Sort',
            'recurrence': 'T(n) = 2T(n/2) + O(n)',
            'a': 2, 'b': 2, 'f': 'O(n)',
            'result': 'O(n log n)'
        },
        {
            'name': 'Karatsuba Multiplication',
            'recurrence': 'T(n) = 3T(n/2) + O(n)',
            'a': 3, 'b': 2, 'f': 'O(n)',
            'result': 'O(n^log 3)  O(n^1.585)'
        },
        {
            'name': 'Matrix Multiplication (naive)',
            'recurrence': 'T(n) = 8T(n/2) + O(n²)',
            'a': 8, 'b': 2, 'f': 'O(n²)',
            'result': 'O(n³)'
        }
    ]

    print("Master Theorem Applications:")
    print("=" * 60)

    for ex in examples:
        print(f"\n{ex['name']}:")
        print(f"Recurrence: {ex['recurrence']}")
        print(f"a = {ex['a']}, b = {ex['b']}, f(n) = {ex['f']}")
        print(f"Solution: {ex['result']}")

master_theorem_examples()
```

Solving Recurrences by Substitution

```
def solve_recurrence_substitution():
    """Demonstrate solving recurrences by substitution method"""

    def fibonacci_recurrence(n, memo={}):
        """Fibonacci with memoization to show exponential vs polynomial"""
        if n in memo:
            return memo[n], 1  # 1 operation (lookup)

        if n <= 1:
            return n, 1

        fib_n_1, ops1 = fibonacci_recurrence(n-1, memo)
        fib_n_2, ops2 = fibonacci_recurrence(n-2, memo)

        result = fib_n_1 + fib_n_2
        total_ops = ops1 + ops2 + 1  # +1 for addition

        memo[n] = result
        return result, total_ops

    def fibonacci_naive(n):
        """Naive Fibonacci to show exponential complexity"""
        if n <= 1:
            return n, 1

        fib_n_1, ops1 = fibonacci_naive(n-1)
        fib_n_2, ops2 = fibonacci_naive(n-2)

        return fib_n_1 + fib_n_2, ops1 + ops2 + 1

    print("Fibonacci Complexity Comparison:")
    print("-" * 50)
    print(f"{'n':>3} {'Naive Ops':>12} {'Memo Ops':>10} {'Ratio':>8}")
    print("-" * 50)

    for n in range(5, 26, 5):
        _, naive_ops = fibonacci_naive(n)
        _, memo_ops = fibonacci_recurrence(n, {})
        ratio = naive_ops / memo_ops

        print(f"{n:>3} {naive_ops:>12} {memo_ops:>10} {ratio:>8.1f}")

    print("\nComplexity Analysis:")
    print("Naive: T(n) = T(n-1) + T(n-2) + O(1) → O( ) where    1.618")
```

```
    print("Memoized: T(n) = O(1) for each subproblem → O(n)")

solve_recurrence_substitution()
```

Space Complexity Analysis

Memory Usage Patterns

```
def space_complexity_examples():
    """Analyze space complexity of different algorithms"""

    def recursive_factorial_space(n, depth=0):
        """Factorial with space tracking"""
        if n <= 1:
            return 1, depth + 1  # Base case uses 1 stack frame

        result, max_depth = recursive_factorial_space(n-1, depth + 1)
        return n * result, max_depth

    def iterative_factorial_space(n):
        """Iterative factorial - constant space"""
        result = 1
        for i in range(1, n + 1):
            result *= i
        return result, 1  # Constant space

    def merge_sort_space(arr):
        """Merge sort space analysis"""
        if len(arr) <= 1:
            return arr, len(arr)

        mid = len(arr) // 2
        left, left_space = merge_sort_space(arr[:mid])
        right, right_space = merge_sort_space(arr[mid:])

        # Merge step requires O(n) additional space
        merged = []
        i = j = 0
        while i < len(left) and j < len(right):
            if left[i] <= right[j]:
                merged.append(left[i])
                i += 1
            else:
                merged.append(right[j])
```

```
                j += 1

        merged.extend(left[i:])
        merged.extend(right[j:])

        # Space complexity: max of recursive calls + current level
        total_space = max(left_space, right_space) + len(arr)
        return merged, total_space

    print("Space Complexity Analysis:")
    print("=" * 50)

    # Test factorial space complexity
    print("\nFactorial Space Complexity:")
    print(f"{'n':>3} {'Recursive':>12} {'Iterative':>12}")
    print("-" * 30)

    for n in [5, 10, 15, 20]:
        _, rec_space = recursive_factorial_space(n)
        _, iter_space = iterative_factorial_space(n)
        print(f"{n:>3} {rec_space:>12} {iter_space:>12}")

    # Test merge sort space complexity
    print("\nMerge Sort Space Complexity:")
    print(f"{'Array Size':>12} {'Space Used':>12}")
    print("-" * 25)

    for size in [8, 16, 32, 64]:
        arr = list(range(size))
        _, space_used = merge_sort_space(arr)
        print(f"{size:>12} {space_used:>12}")

space_complexity_examples()
```

Amortized Analysis

Dynamic Array (Vector) Analysis

```
class DynamicArray:
    """Dynamic array with amortized analysis"""

    def __init__(self):
        self.capacity = 1
        self.size = 0
```

```
        self.data = [None] * self.capacity
        self.total_operations = 0
        self.resize_operations = 0

    def append(self, value):
        """Append with doubling strategy"""
        if self.size == self.capacity:
            self._resize()

        self.data[self.size] = value
        self.size += 1
        self.total_operations += 1

    def _resize(self):
        """Double the capacity"""
        old_capacity = self.capacity
        self.capacity *= 2
        new_data = [None] * self.capacity

        # Copy all elements (this is the expensive operation)
        for i in range(self.size):
            new_data[i] = self.data[i]
            self.total_operations += 1

        self.data = new_data
        self.resize_operations += old_capacity
        print(f"Resized from {old_capacity} to {self.capacity}")

def analyze_dynamic_array():
    """Analyze amortized cost of dynamic array operations"""

    arr = DynamicArray()
    n_operations = 32

    print("Dynamic Array Amortized Analysis:")
    print("-" * 40)
    print(f"{'Operation':>10} {'Total Ops':>12} {'Amortized':>12}")
    print("-" * 40)

    for i in range(1, n_operations + 1):
        arr.append(i)

        if i in [1, 2, 4, 8, 16, 32] or i % 8 == 0:
            amortized_cost = arr.total_operations / i
            print(f"{i:>10} {arr.total_operations:>12} {amortized_cost:>12.2f}")

    print(f"\nTotal operations: {arr.total_operations}")
```

```
    print(f"Resize operations: {arr.resize_operations}")
    print(f"Regular operations: {arr.total_operations - arr.resize_operations}")
    print(f"Amortized cost per append: {arr.total_operations / n_operations:.2f}")
    print("Theoretical amortized cost: O(1)")

analyze_dynamic_array()
```

Practical Algorithm Analysis

Sorting Algorithm Comparison

```
import time
import random

def empirical_complexity_analysis():
    """Empirical analysis of sorting algorithms"""

    def bubble_sort(arr):
        """O(n²) sorting algorithm"""
        n = len(arr)
        operations = 0
        for i in range(n):
            for j in range(0, n - i - 1):
                operations += 1
                if arr[j] > arr[j + 1]:
                    arr[j], arr[j + 1] = arr[j + 1], arr[j]
        return operations

    def merge_sort(arr):
        """O(n log n) sorting algorithm"""
        operations = [0]  # Use list to allow modification in nested function

        def merge_sort_helper(arr):
            if len(arr) <= 1:
                return arr

            mid = len(arr) // 2
            left = merge_sort_helper(arr[:mid])
            right = merge_sort_helper(arr[mid:])

            return merge(left, right, operations)

        def merge(left, right, ops):
            result = []
```

```
            i = j = 0

            while i < len(left) and j < len(right):
                ops[0] += 1
                if left[i] <= right[j]:
                    result.append(left[i])
                    i += 1
                else:
                    result.append(right[j])
                    j += 1

            result.extend(left[i:])
            result.extend(right[j:])
            return result

        merge_sort_helper(arr)
        return operations[0]

    # Test different input sizes
    sizes = [100, 200, 400, 800]

    print("Empirical Complexity Analysis:")
    print("=" * 60)
    print(f"{'Size':>6} {'Bubble Sort':>12} {'Merge Sort':>12} {'Ratio':>8}")
    print("-" * 60)

    for size in sizes:
        # Generate random array
        arr1 = [random.randint(1, 1000) for _ in range(size)]
        arr2 = arr1.copy()

        # Measure operations
        bubble_ops = bubble_sort(arr1)
        merge_ops = merge_sort(arr2)

        ratio = bubble_ops / merge_ops if merge_ops > 0 else 0

        print(f"{size:>6} {bubble_ops:>12} {merge_ops:>12} {ratio:>8.1f}")

    print("\nTheoretical Complexity:")
    print("Bubble Sort: O(n²)")
    print("Merge Sort: O(n log n)")
    print("Expected ratio growth: O(n / log n)")

empirical_complexity_analysis()
```

Cache Performance Analysis

```
def cache_complexity_analysis():
    """Analyze cache-friendly vs cache-unfriendly algorithms"""

    def matrix_multiply_ijk(A, B):
        """Standard matrix multiplication (cache-unfriendly for large matrices)"""
        n = len(A)
        C = [[0] * n for _ in range(n)]
        cache_misses = 0

        for i in range(n):
            for j in range(n):
                for k in range(n):
                    C[i][j] += A[i][k] * B[k][j]
                    # Simulate cache miss for B[k][j] access pattern
                    if k % 4 == 0:  # Simplified cache model
                        cache_misses += 1

        return C, cache_misses

    def matrix_multiply_ikj(A, B):
        """Cache-friendly matrix multiplication"""
        n = len(A)
        C = [[0] * n for _ in range(n)]
        cache_misses = 0

        for i in range(n):
            for k in range(n):
                for j in range(n):
                    C[i][j] += A[i][k] * B[k][j]
                    # Better cache locality for C[i][j] access pattern
                    if j % 8 == 0:  # Fewer cache misses
                        cache_misses += 1

        return C, cache_misses

    print("Cache Performance Analysis:")
    print("-" * 50)
    print(f"{'Size':>6} {'IJK Misses':>12} {'IKJ Misses':>12} {'Ratio':>8}")
    print("-" * 50)

    for size in [8, 16, 32]:
        # Create test matrices
        A = [[random.randint(1, 10) for _ in range(size)] for _ in range(size)]
        B = [[random.randint(1, 10) for _ in range(size)] for _ in range(size)]
```

```
        _, ijk_misses = matrix_multiply_ijk(A, B)
        _, ikj_misses = matrix_multiply_ikj(A, B)

        ratio = ijk_misses / ikj_misses if ikj_misses > 0 else 0

        print(f"{size:>6} {ijk_misses:>12} {ikj_misses:>12} {ratio:>8.1f}")

    print("\nCache Locality Impact:")
    print("- IJK order: Poor cache locality for matrix B")
    print("- IKJ order: Better cache locality for matrix C")
    print("- Real performance difference can be 2-10x")

cache_complexity_analysis()
```

Summary

Asymptotic analysis provides essential tools for:

1. **Algorithm Comparison**: Compare algorithms independent of hardware
2. **Scalability Prediction**: Understand how algorithms behave on large inputs
3. **Resource Planning**: Estimate computational requirements
4. **Algorithm Design**: Guide design decisions for efficiency
5. **Problem Classification**: Identify inherently difficult problems

Key takeaways: - **Big O** describes upper bounds (worst-case behavior) - **Big Ω** describes lower bounds (best-case behavior) - **Big Θ** describes tight bounds (exact growth rate) - **Amortized analysis** considers average cost over sequences of operations - **Space-time tradeoffs** often exist between memory and computation - **Cache effects** can significantly impact real-world performance

Understanding these concepts enables you to write more efficient code and make informed decisions about algorithm selection and system design. ## Recurrence Tree Intuition

For recurrences like `T(n)=2T(n/2)+n`, a recurrence tree helps visualize per-level work.

```
flow:
  A -> C
  B -> D
  B -> E
  C -> F
  C -> G
```

Each level contributes about `n` total work; there are `log n` levels, giving `Theta(n log n)`.

Proof Checklist for Asymptotic Claims

1. State constants `c, n0` explicitly.

2. Show inequality holds for all `n >= n0`.
3. Separate upper and lower bounds when proving `Theta`.
4. Avoid empirical-only argument as proof.

Complexity Classes and Reductions

Complexity classes group decision problems by the resources needed to solve them. Reductions compare problems: if A efficiently reduces to B, then B is "at least as hard as" A. Together they yield the modern theory of NP-completeness and a discipline for hardness proofs in CS.

1. Decision problems and languages

A **decision problem** asks a yes/no question about an input string x. Identify the problem with a **language** $L \subseteq \{0,1\}^*$:

$$x \in L \iff \text{the answer on } x \text{ is YES.}$$

Examples:

- $\mathrm{PATH} = \{\langle G, s, t\rangle : \text{graph } G \text{ has an } s\text{–}t \text{ path}\}$
- $\mathrm{SAT} = \{\langle \phi\rangle : \text{Boolean formula } \phi \text{ is satisfiable}\}$
- $\mathrm{TSP}_{\mathrm{DEC}} = \{\langle G, k\rangle : \text{graph } G \text{ has a tour of length} \le k\}\$$

Optimization problems convert to decision versions by thresholding the objective—needed for class theory stated on languages.

Worked example 1

Shortest path length optimization $\leftrightarrow$ decision: "is $\mathrm{dist}(s,t) \le k$?" Binary search on k recovers the optimum if decisions are efficient and bounds are integral/polynomial.

2. The class $\mathbf{P}$

Definition. $L \in \mathbf{P}$ if there is a deterministic algorithm deciding L in time $O(n^c)$ for some constant c (polynomial in input length $n = |x|$).

Informally: **efficiently solvable** (coarse but powerful abstraction).

Examples in P

- Sorting-related decisions, median finding
- Shortest paths with nonnegative weights (Dijkstra)
- Linear programming (weakly poly / practical poly; theoretical status nuanced historically but LP is poly-time solvable)
- Matching in general graphs (Edmonds)—nontrivial membership in **P**

Worked example 2

Connectivity in undirected graphs: BFS/DFS in $O(v+e)$ time $\subseteq \mathbf{P}$.

3. The class NP

Definition (verifier). $L \in \mathbf{NP}$ if there exists a polynomial p and a deterministic polynomial-time **verifier** V such that:

$$x \in L \iff \exists\, u \in \{0,1\}^{p(|x|)} \text{ with } V(x,u) = 1.$$

The string u is a **certificate** / witness. So **NP** is "efficiently verifiable YES answers," not "non-polynomial" (a common misreading of the name).

Nondeterministic view (equivalent): a nondeterministic TM guesses u and checks in poly time.

Worked example 3 — SAT certificates

Certificate: assignment to variables. Verifier evaluates the formula in linear time in formula size.

Worked example 4 — what is not obviously a short certificate?

Tautology (true under **all** assignments): a single assignment is not a certificate of universality. Tautology is in **coNP** (see below).

Worked example 5 — composite numbers

Certificate: nontrivial factor. Verification: multiply (historically placed intuition for **NP**; primality is now known to be in **P** via AKS, a deep result).

4. coNP and containments

Definition. $L \in \mathbf{coNP}$ iff $\overline{L} \in \mathbf{NP}$ (complements).

- $\mathbf{P} \subseteq \mathbf{NP}$ and $\mathbf{P} \subseteq \mathbf{coNP}$
- $\mathbf{NP} \subseteq \mathbf{PSPACE}$ (try all certificates with poly space reuse—more carefully: NP $\subseteq$ PSPACE)
- Open: $\mathbf{P} \stackrel{?}{=} \mathbf{NP}$, $\mathbf{NP} \stackrel{?}{=} \mathbf{coNP}$

Worked example 6

UNSAT $= \overline{\text{SAT}}$ is **coNP**-complete (standard). Short certificates for UNSAT are not known in general (would collapse things if always short in certain ways).

5. Polynomial-time reductions

Definition (Karp / many-one reduction). $A \leq_p B$ if there is a poly-time computable f such that

$$x \in A \iff f(x) \in B.$$

Meaning: if B is decidable in poly time, so is A ($x \mapsto f(x)$ then decide B). Contrapositively: if A is "hard," so is B.

Properties

- Reflexive and transitive
- If $A \leq_p B$ and $B \in \mathbf{P}$ then $A \in \mathbf{P}$
- If $A \leq_p B$ and A is NP-hard then B is NP-hard

Worked example 7 — direction discipline

To show **new problem B is hard**, reduce **known hard A to B**: $A \leq_p B$.
Reducing $B \leq_p A$ would show B is **no harder than** A—useful for algorithms, wrong direction for hardness.

6. NP-hardness and NP-completeness

Definition. B is **NP-hard** if for every $A \in \mathbf{NP}$, $A \leq_p B$.
B is **NP-complete** if B is NP-hard and $B \in \mathbf{NP}$.

Cook–Levin theorem (statement). SAT is NP-complete.

Thereafter, new NP-completeness proofs usually:

1. Show $B \in \mathbf{NP}$ (give verifier / certificates)
2. Pick known NP-complete A
3. Construct poly-time f with $x \in A \iff f(x) \in B$
4. Prove both directions of the iff (the part people skip—and get wrong)

Worked example 8 — independent set $\leftrightarrow$ clique

Given $G = (V, E)$, complement edges $\overline{G} = (V, \binom{V}{2} \setminus E)$. Then G has an independent set of size k iff $\overline{G}$ has a clique of size k. Reduction is poly-time (build complement carefully; for dense encoding watch size).

Worked example 9 — vertex cover

S is a vertex cover of size k iff $V \setminus S$ is an independent set of size $n - k$. Yields NP-completeness transfers among these problems.

Worked example 10 — 3-SAT sketch

From general SAT: rewrite clauses to length ≤ 3 by introducing fresh variables:

$$(\ell_1 \vee \ell_2 \vee \ell_3 \vee \ell_4) \rightsquigarrow (\ell_1 \vee \ell_2 \vee y) \wedge (\neg y \vee \ell_3 \vee \ell_4).$$

Preserve satisfiability; polynomial blowup. (Full Cook–Levin is harder; this is the SAT$\rightarrow$3-SAT step.)

7. Proof hygiene checklist

When writing a reduction proof:

1. **Specify f completely** (including how numbers/graphs are encoded)
2. **Runtime:** argue $|f(x)|$ and construction time are poly in $|x|$
3. **YES $\Rightarrow$ YES:** if $x \in A$, construct witness/structure showing $f(x) \in B$
4. **NO $\Rightarrow$ NO:** if $x \notin A$, prove $f(x) \notin B$ (often the subtle direction)
5. **No exponential enumeration** inside f
6. **Decision vs optimization:** reduce decision to decision unless using Turing reductions carefully

8. Beyond NP (map)

Class	Resource intuition
PSPACE	Poly space; includes quantified Boolean formulas (QBF)
EXPTIME	Exponential time
PH	Polynomial hierarchy (alternations of quantifiers)

Containments: $\mathbf{P} \subseteq \mathbf{NP} \subseteq \mathbf{PSPACE} \subseteq \mathbf{EXPTIME}$, with some inequalities known (time hierarchy), many separations open.

9. CS practice connections

- **Compilers / verification:** many static analyses touch undecidability or hard subproblems; restrict languages to recover **P**
- **Cryptography:** needs average-case hardness, not only worst-case NP-hardness
- **Operations research:** NP-complete scheduling ⇒ ILP + heuristics
- **Parameterization:** NP-hard in n but FPT in small parameter k (treewidth, solution size)

Worked example 11 — product decision

"Can we place facilities so every user is within distance D using $\leq k$ facilities?" (dominating set / facility variants)—expect NP-complete; use ILP or approximation.

10. Pitfalls

1. **NP = "not polynomial"** — wrong etymology in practice
2. **Showing a poly algorithm for special cases** does not place the general problem in **P**
3. **Reduction direction reversed**
4. **Certificate of exponential size** — not valid for **NP**
5. **Assuming $\mathbf{P} \neq \mathbf{NP}$** without stating it when claiming "no poly algorithm"
6. **Heuristic works on my instances** $\neq$ proof of tractability

11. Checkpoint

- Define **P** and **NP** via time / verifiers
- State $\leq_p$ and use the correct direction for hardness
- Execute the 3-step NP-completeness method
- Convert an optimization problem to a decision language
- Avoid the common fallacies above

Exercises

Easy

1. Prove $\mathbf{P} \subseteq \mathbf{NP}$ from the definitions.
2. Give certificates for: Hamiltonian cycle, subset sum, graph k-colorability.
3. Why is a brute-force "try all subsets" algorithm not a proof of membership in $\mathbf{P}$?
4. Formalize CLIQUE as a language.
5. If $A \leq_p B$ and $B \leq_p C$, show $A \leq_p C$.

Medium

6. Prove that if any NP-complete problem is in $\mathbf{P}$, then $\mathbf{P} = \mathbf{NP}$.
7. Detail the independent set $\leftrightarrow$ clique reduction with both directions.
8. Show that deciding whether a DFA accepts some string is in $\mathbf{P}$ (graph reachability on the automaton).
9. Explain why tautology is in **coNP**.
10. Find the bug: "I reduce my problem B to SAT, and SAT is hard, so B is hard."

Challenge

11. Write a careful SAT$ _p$3-SAT proof for clauses of length > 3.
12. Define coNP-completeness and name one coNP-complete problem.
13. Research: what is a **Turing reduction** (Cook reduction), and why is Karp reduction preferred for NP-completeness definitions?
14. Show SUBSET-SUM is in **NP**; sketch why pseudo-polynomial DP does not place it in $\mathbf{P}$ in the strong sense (bit length of numbers).
15. Formulate a real scheduling question as a language and outline an NP-completeness proof strategy (choice of source problem + intended gadgets at high level).

Summary

$\mathbf{P}$ captures efficient solvability; **NP** captures efficient verifiability. Polynomial reductions transfer both algorithms and hardness. NP-completeness is a precise badge earned by membership plus a correct many-one reduction from a known complete problem—direction and both iff sides included.

Randomized and Approximation Algorithms

When deterministic exact algorithms are too slow or too rigid, two standard escapes appear: **use randomness** (speed or simplicity with controlled failure) and **approximate** (provably near-optimal solutions). This chapter builds the mathematical language for both, with CS applications from hashing to covering problems.

Part A — Randomized algorithms

1. What randomness buys

A randomized algorithm may flip coins during execution. Analysis is over coin tosses (and sometimes over input distributions).

Las Vegas: always correct; runtime is a random variable (e.g. randomized quicksort expected $O(n \log n)$).

Monte Carlo: may err; runtime often fixed (e.g. probabilistic primality tests historically; fingerprinting).

Worked example 1 — randomized quicksort

Pivot uniformly at random: expected comparisons $O(n \log n)$ for any input (expectation over pivots). Failure mode: none for correctness; only slow runs with small probability.

2. Error modes and amplification

For Monte Carlo decision algorithms:

- **One-sided error:** e.g. always correct on NO, may err on YES (or vice versa)—classically related to **RP**
- **Two-sided error:** may err either way—related to **BPP**

Amplification theorem (intuition). If a Monte Carlo algorithm errs with probability $\leq 1/3$, then k independent repetitions with majority vote (two-sided) or OR/AND combination (one-sided) drive error to $2^{-\Theta(k)}$.

Worked example 2

Error 1/3 per trial, majority of 100 trials: Chernoff bounds give exponentially small error. Cryptographic and fingerprinting applications rely on this.

Chernoff bound (useful form)

For independent Bernoullis X_i with $X = \sum X_i$, $\mu = \mathbb{E}[X]$, and $\delta \in (0, 1)$,

$$P(X < (1-\delta)\mu) \le e^{-\mu\delta^2/2}, \qquad P(X > (1+\delta)\mu) \le e^{-\mu\delta^2/3}.$$

(Constants vary by textbook form; the message is exponential concentration.)

3. Fingerprinting and polynomial identity

Schwartz–Zippel lemma (statement). Let $P \in \mathbb{F}[x_1, \dots, x_n]$ be nonzero of total degree d. If r_i drawn independently uniformly from finite $S \subseteq \mathbb{F}$, then

$$P(P(r_1, \dots, r_n) = 0) \le \frac{d}{|S|}.$$

Application: test $P \equiv 0$ by evaluation at a random point—core of probabilistic polynomial identity testing and some parallel algorithms.

Worked example 3 — communication-free checking

Alice holds matrix A, Bob B; check $AB = C$ by multiplying $A(Br)$ vs Cr for random r—error probability controlled, faster than full multiply verification in some models.

4. Hashing and dictionaries

Universal hash families: for $x \neq y$, $P_{h \sim \mathcal{H}}(h(x) = h(y)) \le 1/m$. Expected chain length $O(1 + \alpha)$ in chaining.

Worked example 4

With load factor $\alpha = n/m = O(1)$ and 2-universal hashing, expected search time $O(1)$. High-probability bounds need stronger tail arguments or perfect hashing variants.

5. Markov, Chebyshev, and median-of-means

- **Markov:** $P(X \geq t) \leq \mathbb{E}[X]/t$ for $X \geq 0$
- **Chebyshev:** $P(|X - \mu| \geq t) \leq \mathrm{Var}(X)/t^2$
- **Median-of-means:** reduce dependence on variance for robust mean estimation with fewer moments assumptions

Worked example 5 — Las Vegas to high probability

If Las Vegas runtime T has $\mathbb{E}[T] = O(f(n))$, Markov says $P(T > cf(n)) \leq 1/c$. Restart strategies convert expectation bounds into high-probability bounds.

Part B — Approximation algorithms

6. Optimization and approximation ratio

For a minimization problem with optimum $\mathrm{OPT}(I)$ on instance I, algorithm $\mathcal{A}$ has **approximation ratio** $\rho \geq 1$ if for all I,

$$\mathcal{A}(I) \leq \rho \cdot \mathrm{OPT}(I).$$

For maximization, $\mathcal{A}(I) \geq \mathrm{OPT}(I)/\rho$ (conventions vary; always state which).

PTAS: $(1+\varepsilon)$-approx for every $\varepsilon > 0$ with time $n^{f(1/\varepsilon)}$.
FPTAS: time $\mathrm{poly}(n, 1/\varepsilon)$.

Worked example 6 — load balancing

Assign jobs with times t_j to m machines to minimize makespan. List scheduling: $\rho = 2 - \frac{1}{m}$. Greedy LPT improves constants. Analysis uses lower bounds $\mathrm{OPT} \geq \max(\max t_j,\ \frac{1}{m}\sum t_j)$.

7. Vertex cover 2-approximation

Problem. Min vertices hitting all edges.

Algorithm: while edges remain, pick any edge, add **both** endpoints to cover, delete incident edges.

Theorem. This is a 2-approximation.

Proof. Selected edges form a matching M; cover size $2|M|$. Any cover needs $\geq |M|$ vertices (hit each matching edge). Thus $\mathcal{A} = 2|M| \leq 2\,\mathrm{OPT}$.

Worked example 7

Path of 3 edges: algorithm may take 4 vertices if unlucky in implementation order? On a simple path $v1$—$v2$—$v3$—$v4$, pick middle edge first carefully—work through a concrete graph and compare to OPT.

Better: maximal matching based 2-approx is standard; LP rounding also yields 2-approx.

8. Set cover greedy

Universe U, $|U| = n$, sets $S_1, \dots, S_m$. Greedy repeatedly picks set covering the most remaining elements.

Theorem. Approximation ratio $H_n = 1 + \frac{1}{2} + \cdots + \frac{1}{n} \leq \ln n + 1$.

Proof idea. Charge cost of each step across newly covered elements using harmonic harmonic increments; compare to OPT fractional packing of elements.

Worked example 8

This $O(\log n)$ ratio is essentially tight under complexity assumptions—cannot do much better in poly time for general set cover.

9. Greedy knapsack and FPTAS (sketch)

Fractional knapsack: greedy by value/weight optimal.
0-1 knapsack: pseudo-poly DP in $O(nW)$; FPTAS scales values to get $(1 - \varepsilon)$-approx in $\text{poly}(n, 1/\varepsilon)$.

Worked example 9

When weights are huge, bit length makes $O(nW)$ exponential in input size—pseudo-polynomial, not polynomial. FPTAS restores poly-time near-optimality.

10. Hardness of approximation (awareness)

Some ratios are impossible unless $\mathbf{P} = \mathbf{NP}$ (or under stronger conjectures like UGC):

- General TSP without triangle inequality: no constant-factor approx
- Max-clique: extremely hard to approximate
- Set cover: no $(1 - o(1)) \ln n$ under standard assumptions

So approximation algorithms are not a free lunch; they come with a theory of limits.

11. Engineering tradeoff table

Method	Pros	Cons
Exact ILP/MIP	Optimal, certificates	Scales poorly
Approximation	Provable ratio, often fast	Ratio may be weak in practice
Heuristic / local search	Excellent empirics	No worst-case guarantee
Randomized exact (Las Vegas)	Simple, good expected time	Tail latency
Monte Carlo	Very fast checks	Residual error

Worked example 10 — production choice

CDN cache placement NP-hard: use greedy set cover style with $\ln n$ guarantee, or MIP for small instances overnight, or hybrid warm-start.

12. Pitfalls

1. **Expected time vs high-probability latency** (SLOs care about tails)
2. **One-sided vs two-sided error** mishandled in amplification
3. **Approximation ratio on wrong objective** (min vs max)
4. **Claiming greedy is optimal** without proof (activity selection yes; set cover no)
5. **Pseudo-poly DP** marketed as poly-time
6. **Average-case heuristics** presented as approximation algorithms

13. Checkpoint

- Distinguish Las Vegas / Monte Carlo and amplify error
- Apply Markov/Chernoff at a basic level
- Define approximation ratio and prove the matching vertex-cover 2-approx
- State greedy set cover's harmonic ratio
- Choose among exact / approx / heuristic given constraints

Exercises

Easy

1. Is randomized quicksort Las Vegas or Monte Carlo? Why?
2. Amplify a one-sided error algorithm that errs only on YES instances with probability 1/2: how to get error 2^{-k}?
3. Apply Markov: if $\mathbb{E}[T] = 2n$, bound $P(T \geq 100n)$.
4. Define approximation ratio for MAX-CUT style maximization.
5. Run the vertex-cover 2-approx by hand on a 4-cycle; compare to OPT.

Medium

6. Prove the vertex-cover matching argument carefully.
7. Using Chebyshev, how many samples to estimate a mean with variance σ^2 within ε with probability $\geq 1 - \delta$?
8. Explain why list scheduling for makespan is at most $(2 - 1/m)\mathrm{OPT}$.
9. Give an instance where greedy set cover is asymptotically worse than OPT by a $\Theta(\log n)$ factor (classic construction sketch).
10. Schwartz–Zippel: bound error if $\deg P \leq d$ and $|S| = 2d$.

Challenge

11. Derive (or look up and rephrase) Chernoff bound application for majority vote amplification of BPP-style algorithms.
12. Design a Monte Carlo algorithm to test bipartite perfect matching existence via algebraic techniques (Edmonds matrix) at high level.
13. Give an FPTAS sketch for knapsack: how values are scaled and why error stays $\leq \varepsilon \cdot \mathrm{OPT}$.
14. Prove that if a minimization problem admits a poly-time ρ-approx for all $\rho > 1$, something collapses—or explain for TSP without triangle inequality why no constant approx exists (gap-producing reduction idea).
15. Systems design prompt: pick an NP-hard allocation problem in cloud computing; propose an approximation or greedy scheme and state what ratio you can claim (or what remains heuristic).

Summary

Randomness yields simpler algorithms, concentration-based guarantees, and powerful identity tests. Approximation algorithms trade optimality for speed with **provable** ratios—vertex cover, set cover, scheduling, and knapsack are templates. Complexity also limits how good those ratios can be, guiding honest engineering choices.

How Fast Can We Multiply Integers? Karatsuba, Toom–Cook, FFTs, n log n, and the 2026 Claim Below It

Multiplying two n-digit numbers the way school teaches takes n^2 digit products. For most of history nobody suspected that could be improved. In 1960 Kolmogorov conjectured that n^2 was optimal, and within a week of hearing that conjecture, his student Karatsuba refuted it. Sixty years of work then pushed the cost down to $O(n \log n)$ (Harvey and van der Hoeven, *Annals of Mathematics*, 2021), the bound Schönhage and Strassen had conjectured to be optimal in 1971. In October 2026 an AI-produced preprint claimed to go *below* $n \log n$ by a factor $(\log n)^{\kappa}$ with $\kappa = 2^{-182}$, and a community race pushed κ to about 0.0017 within days.

This chapter builds the whole ladder from one idea, **split, multiply smaller pieces, recombine**, and makes each rung concrete. You will count the limb products of Karatsuba and Toom-3 exactly, multiply exactly with a number-theoretic transform, measure which algorithm your Python is actually running, and compute how much the 2026 claim saves for inputs of any physically possible size. The trap is the oldest bug in FFT multiplication: floating-point rounding that silently returns a wrong product, and a correctness check that cannot see it.

All listings run on **CPython 3.15.0**. Listings that import `numpy` or `gmpy2` use a throwaway venv (`uv venv venv-mult --python 3.15 && VIRTUAL_ENV=venv-mult uv pip install gmpy2 numpy`, which installs gmpy2 2.3.2 on GMP 6.3.0 and NumPy 2.5.4). Timings come from a shared 8-core box, so read the ratios.

1. Mental model: digits are polynomial coefficients

Write an integer in base B (a *limb* is one base-B digit, for example $B = 2^{32}$):

$$x = \sum_{i=0}^{m-1} a_i B^i = A(B), \qquad y = \sum_{j=0}^{m-1} b_j B^j = C(B).$$

Then $xy = (AC)(B)$. The product polynomial's coefficients are the **convolution** $c_k = \sum_{i+j=k} a_i b_j$, and a final carry pass turns them back into digits. Every algorithm in this chapter computes AC in some cleverer way than forming all m^2 products $a_i b_j$:

- **Split and recombine.** Cut A into k pieces, evaluate the piece-polynomial at $2k-1$ points, multiply the $2k-1$ values recursively, and interpolate. Karatsuba is $k = 2$ and Toom-3 is $k = 3$. Cost: $T(n) = (2k-1)\,T(n/k) + O(n)$, so $T(n) = \Theta\big(n^{\log_k(2k-1)}\big)$.

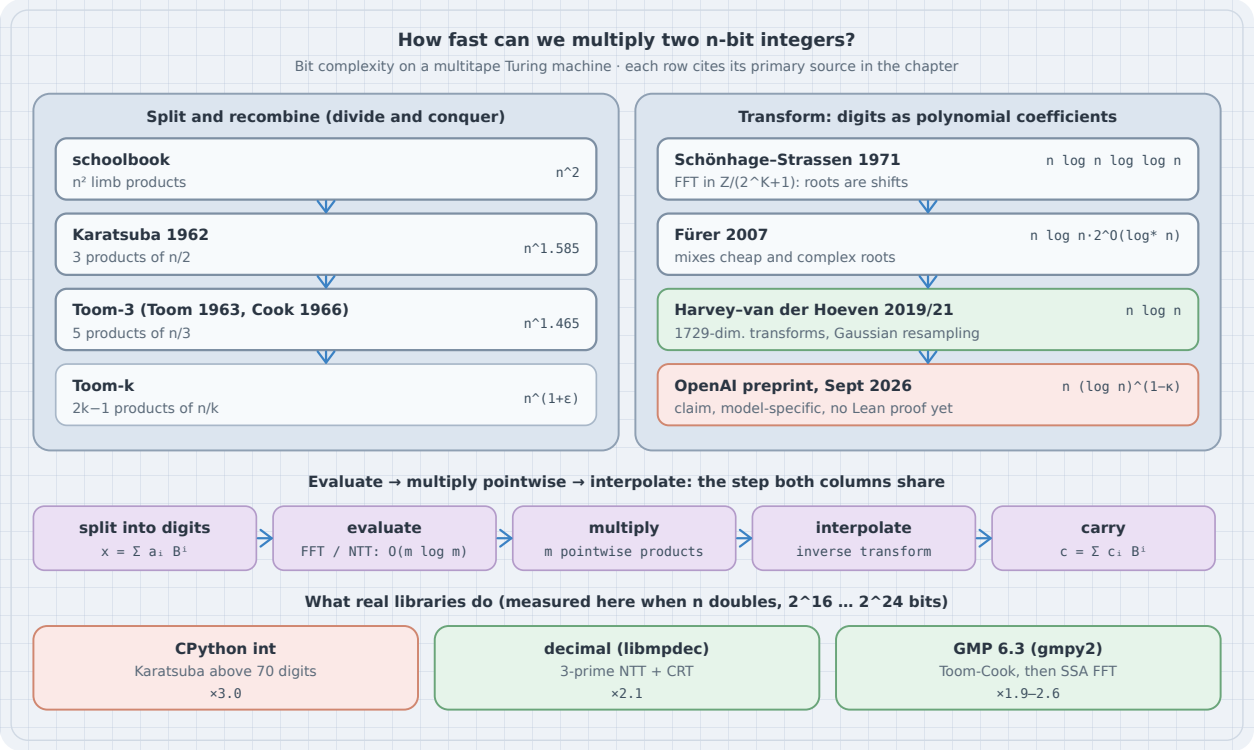


Figure 1: The integer multiplication ladder: split-and-recombine algorithms, transform algorithms, the shared evaluate/multiply/interpolate pipeline, and what real libraries run

- **Transform.** Evaluate at $N \geq 2m-1$ roots of unity all at once with a fast Fourier transform (FFT) in $O(N \log N)$ ring operations, multiply pointwise, and transform back. The difficulty is finding a ring where the roots of unity exist and the coefficients fit, and that is what separates Schönhage–Strassen, Fürer, Harvey–van der Hoeven and the 2026 claim.
- **The model matters.** These bounds count bit operations on a **multitape Turing machine**: a fixed number of one-dimensional tapes, with no random access. In a unit-cost RAM with constant-time access to arbitrary memory, multiplication is known to be easier (Schönhage's linear-time result on storage-modification machines is the classic example). So "$n \log n$ is optimal" was always a conjecture about one specific model.

2. Karatsuba: three half-size products instead of four

Split each operand in half, $x = x_1 B^h + x_0$ and $y = y_1 B^h + y_0$:

$$xy = z_2 B^{2h} + z_1 B^h + z_0, \quad z_0 = x_0 y_0,\ z_2 = x_1 y_1,\ z_1 = (x_0 + x_1)(y_0 + y_1) - z_0 - z_2.$$

That is three multiplications of size h instead of four. The extra additions and subtractions are linear in n. So $T(n) = 3T(n/2) + O(n)$, which gives $n^{\log_2 3} \approx n^{1.585}$. Toom-3 splits into thirds, evaluates the quadratic piece-polynomials at $0, 1, -1, -2, \infty$, multiplies the **five** values, and interpolates, which gives $n^{\log_3 5} \approx n^{1.465}$. Timings on a shared box are noisy, so the listing below counts **leaf products**

(one-limb by one-limb multiplications) instead of measuring time. It also checks every result against Python's own *.

```
# count_mults.py: schoolbook vs Karatsuba vs Toom-3, counted in limb products
import math
import random

W = 32                          # limb width in bits
MASK = (1 << W) - 1
calls = 0


widest = 0


def leaf(a, b):
    """One 'hardware' multiplication of two (small) limbs."""
    global calls, widest
    calls += 1
    widest = max(widest, a.bit_length(), b.bit_length())
    return a * b


def schoolbook(x, y, n):
    xs = [(x >> (W * i)) & MASK for i in range(n)]
    ys = [(y >> (W * j)) & MASK for j in range(n)]
    return sum(leaf(a, b) << (W * (i + j)) for i, a in enumerate(xs) for j, b in enumerate(y


def karatsuba(x, y, n):
    """x, y < 2**(W*n), n a power of two."""
    if n == 1:
        return leaf(x, y)
    h = n // 2
    s = W * h
    m = (1 << s) - 1
    x0, x1, y0, y1 = x & m, x >> s, y & m, y >> s
    z0 = karatsuba(x0, y0, h)
    z2 = karatsuba(x1, y1, h)
    a, b = x0 + x1, y0 + y1                          # may carry one bit past h limbs
    ca, cb, a0, b0 = a >> s, b >> s, a & m, b & m
    mid = karatsuba(a0, b0, h)
    mid += ((b0 if ca else 0) + (a0 if cb else 0)) << s   # carries: shifts and adds only
    mid += (ca & cb) << (2 * s)
    z1 = mid - z0 - z2
    return z0 + (z1 << s) + (z2 << (2 * s))
```

```
def toom3(x, y, n):
    """Toom-Cook 3-way: 5 products of a third of the size. n a power of three."""
    if n == 1:
        return leaf(x, y)
    k = n // 3
    s = W * k
    m = (1 << s) - 1

    def split(v):
        return v & m, (v >> s) & m, v >> (2 * s)

    def evaluate(p0, p1, p2):           # p(t) = p0 + p1 t + p2 t^2 at 0, 1, -1, -2, inf
        return [p0, p0 + p1 + p2, p0 - p1 + p2, p0 - 2 * p1 + 4 * p2, p2]

    def mul(a, b):                      # values may be negative and a few bits wider
        sign = -1 if (a < 0) != (b < 0) else 1
        return sign * toom3(abs(a), abs(b), k)   # extra bits ride along to the leaves

    r0, r1, rm1, rm2, rinf = (mul(a, b) for a, b in zip(evaluate(*split(x)), evaluate(*split
    # interpolation (Bodrato's sequence): only additions, shifts, exact small divisions
    c0, c4 = r0, rinf
    c3 = (rm2 - r1) // 3
    c1 = (r1 - rm1) // 2
    c2 = rm1 - r0
    c3 = (c2 - c3) // 2 + 2 * rinf
    c2 = c2 + c1 - c4
    c1 = c1 - c3
    return c0 + (c1 << s) + (c2 << (2 * s)) + (c3 << (3 * s)) + (c4 << (4 * s))


def count(fn, n, trials=20):
    global calls
    rng = random.Random(n)
    total = 0
    for _ in range(trials):
        x, y = rng.getrandbits(W * n), rng.getrandbits(W * n)
        calls = 0
        assert fn(x, y, n) == x * y, fn.__name__
        total += calls
    return total / trials


print("limbs   schoolbook   karatsuba    ratio_to_n^1.585")
for e in range(1, 9):
    n = 2 ** e
    sb, ka = count(schoolbook, n), count(karatsuba, n)
```

```
    print(f"{n:5d}  {sb:11.0f}  {ka:10.0f}    {ka / n ** math.log2(3):.3f}")
print()
print("limbs   schoolbook   toom3        ratio_to_n^1.465")
for e in range(1, 6):
    n = 3 ** e
    sb, tc = count(schoolbook, n), count(toom3, n)
    print(f"{n:5d}  {sb:11.0f}  {tc:10.1f}    {tc / n ** math.log(5, 3):.3f}")
print(f"widest leaf operand seen: {widest} bits (limb = {W} bits)")
print(f"\nexponents: schoolbook 2, Karatsuba log2(3) = {math.log2(3):.4f}, Toom-3 log3(5) =
```

```
python3.15 count_mults.py
```

Output:

```
limbs   schoolbook   karatsuba    ratio_to_n^1.585
    2            4           3    1.000
    4           16           9    1.000
    8           64          27    1.000
   16          256          81    1.000
   32         1024         243    1.000
   64         4096         729    1.000
  128        16384        2187    1.000
  256        65536        6561    1.000

limbs   schoolbook   toom3        ratio_to_n^1.465
    3            9         5.0    1.000
    9           81        25.0    1.000
   27          729       125.0    1.000
   81         6561       625.0    1.000
  243        59049      3125.0    1.000
widest leaf operand seen: 43 bits (limb = 32 bits)

exponents: schoolbook 2, Karatsuba log2(3) = 1.5850, Toom-3 log3(5) = 1.4650
```

The counts are *exactly* $3^{\log_2 n}$ and $5^{\log_3 n}$: the ratio column is 1.000 at every size, and all 20 random products per row matched `x * y`. At 256 limbs (8,192 bits) Karatsuba does 6,561 leaf products where schoolbook does 65,536. Two engineering details are visible. First, $x_0 + x_1$ can carry one bit past h limbs. The listing handles the carry with shifts and adds, so it costs no extra multiplication. Second, Toom's evaluation points (-2 in particular) make the values a few bits *wider* than a limb. Here the widest leaf operand was 43 bits for 32-bit limbs after five levels. Real libraries reserve an extra limb for this. The interpolation step uses only additions, shifts, and exact division by 2 and 3 (Bodrato's sequence).

Why not just go to Toom-k for large k? The exponent $\log_k(2k-1) \to 1$, but the interpolation constants grow with k, so each fixed k wins only over a range of sizes. That is why real libraries switch between several Toom variants by size, as Section 5 shows. To get rid of the "$+\varepsilon$" in the exponent you need transforms.

3. Transforms: evaluate everywhere at once

Evaluating a degree-$(N-1)$ polynomial at the N powers of a primitive N-th root of unity ω is a discrete Fourier transform, and the FFT does it in $O(N \log N)$ ring operations by splitting even and odd coefficients. Over the complex numbers that means floating-point arithmetic and rounding error (see Section 7). Over $\mathbb{Z}/p\mathbb{Z}$ with $p = c \cdot 2^k + 1$ prime, the field has 2^k-th roots of unity and every operation is exact. That is a **number-theoretic transform (NTT)**. The catch is a *budget*. The true convolution coefficients must be smaller than p, otherwise they wrap around mod p, and they can be as large as $m\,(B-1)^2$.

```
# ntt_mult.py: exact multiplication by a number-theoretic transform (NTT)
import random
import time

P = 998244353            # 119 * 2^23 + 1: has 2^23-th roots of unity
G = 3                    # a primitive root mod P
BASE_BITS = 8            # one coefficient = one byte


def ntt(a, invert=False):
    n = len(a)
    j = 0
    for i in range(1, n):                       # bit-reversal permutation
        bit = n >> 1
        while j & bit:
            j ^= bit
            bit >>= 1
        j |= bit
        if i < j:
            a[i], a[j] = a[j], a[i]
    length = 2
    while length <= n:                          # log2(n) butterfly layers
        w_len = pow(G, (P - 1) // length, P)
        if invert:
            w_len = pow(w_len, P - 2, P)
        half = length // 2
        ws = [1] * half
        for k in range(1, half):
            ws[k] = ws[k - 1] * w_len % P
        for start in range(0, n, length):
            for k in range(half):
                u = a[start + k]
                v = a[start + k + half] * ws[k] % P
                a[start + k] = (u + v) % P
                a[start + k + half] = (u - v) % P
        length <<= 1
    if invert:
```

```
        n_inv = pow(n, P - 2, P)
        for i in range(n):
            a[i] = a[i] * n_inv % P


def to_coeffs(x, n):
    mask = (1 << BASE_BITS) - 1
    return [(x >> (BASE_BITS * i)) & mask for i in range(n)]


def ntt_multiply(x, y):
    m = max(x.bit_length(), y.bit_length()) // BASE_BITS + 1
    n = 1
    while n < 2 * m:
        n <<= 1
    # every convolution coefficient is < m * 255^2; it must stay below P to be exact
    assert m * ((1 << BASE_BITS) - 1) ** 2 < P, "coefficients would wrap mod P"
    a, b = to_coeffs(x, n), to_coeffs(y, n)
    ntt(a)
    ntt(b)
    c = [u * v % P for u, v in zip(a, b)]     # pointwise product: n multiplications
    ntt(c, invert=True)
    return sum(ci << (BASE_BITS * i) for i, ci in enumerate(c)), n


rng = random.Random(7)
limit = (P - 1) // ((1 << BASE_BITS) - 1) ** 2
print(f"coefficient budget: m * 255^2 < P  =>  at most {limit} bytes ({limit * BASE_BITS} bi
for bits in (1_000, 10_000, 100_000, 120_000, 130_000):
    x, y = rng.getrandbits(bits), rng.getrandbits(bits)
    t0 = time.perf_counter()
    try:
        z, n = ntt_multiply(x, y)
    except AssertionError as e:
        print(f"{bits:>9} bits  AssertionError: {e}")
        continue
    dt = time.perf_counter() - t0
    print(f"{bits:>9} bits  transform length {n:>7}  exact={z == x * y}  {dt:6.2f} s")
```

```
python3.15 ntt_mult.py
```

Output:

```
coefficient budget: m * 255^2 < P  =>  at most 15351 bytes (122808 bits) per operand
     1000 bits  transform length     256  exact=True    0.00 s
    10000 bits  transform length    4096  exact=True    0.01 s
```

```
100000 bits  transform length   32768  exact=True    0.29 s
120000 bits  transform length   32768  exact=True    0.28 s
130000 bits  AssertionError: coefficients would wrap mod P
```

Up to the budget the product is exact, and the work is three transforms of length N (each $\frac{N}{2}\log_2 N$ butterflies) plus N pointwise products. Pure Python is slow, but the shape is $N \log N$. Past the budget the method breaks, and that budget is the whole story of fast multiplication. There are two ways out:

1. **Several primes and the Chinese remainder theorem.** Run the NTT modulo three primes and reconstruct each coefficient by CRT. libmpdec (the C library behind Python's `decimal`) does exactly this: `_mpd_fntmul` in `Modules/_decimal/libmpdec/mpdecimal.c` convolves modulo three primes. For the very largest sizes it wraps the transform in Karatsuba (`_mpd_kmul_fnt`). Any fixed number of primes still has a ceiling, so this is engineering, not asymptotics.
2. **Make the ring grow with n.** Schönhage and Strassen (1971) work in $\mathbb{Z}/(2^K+1)\mathbb{Z}$, where 2 is a $2K$-th root of unity. Multiplying by a root is then a *shift*, which costs $O(K)$ bit operations, not a multiplication. The pointwise products are themselves big multiplications, which they do recursively. The recursion is $\log\log n$ levels deep, and that is where $O(n \log n \log\log n)$ comes from.

4. From $\log\log n$ to $n \log n$

The rest of the ladder is about shrinking the cost of that recursion:

Year	Result	Bound (multitape TM)	Key idea
1962	Karatsuba–Ofman	$O(n^{1.585})$	3 half-size products
1963/66	Toom; Cook	$O(n^{1+\varepsilon})$, any $\varepsilon > 0$	evaluate/interpolate at $2k-1$ points
1971	Schönhage–Strassen	$O(n \log n \log\log n)$	FFT over $\mathbb{Z}/(2^K+1)$; conjectured $\Theta(n \log n)$ optimal
2007	Fürer	$n \log n \cdot 2^{O(\log^* n)}$	mixes cheap polynomial roots with complex roots
2014–19	Harvey–van der Hoeven–Lecerf; Harvey–van der Hoeven; Covanov–Thomé	$O(n \log n\, 8^{\log^* n})$, then $4^{\log^* n}$	sharper constants in Fürer's exponent
2019/2021	Harvey–van der Hoeven	$O(n \log n)$	Gaussian resampling → multidimensional power-of-two transforms

Year	Result	Bound (multitape TM)	Key idea
2026 (claim)	OpenAI preprint	$O(n(\log n)^{1-\kappa})$, $\kappa = 2^{-182}$	linear networks that cut tape movement and butterfly cost

Harvey and van der Hoeven's algorithm turns a one-dimensional transform of awkward length into a d-dimensional one whose side lengths are powers of two, by "resampling" through a Gaussian. Their proof takes $d = 1729$, and the bound only takes effect beyond an astronomically large threshold. They note that more technical refinements reduce the dimension needed to $d \geq 9$. The 1729-dimensional Fourier transform in the X post that prompted this chapter is real, and it is a proof device, not an implementation strategy. No library runs it.

5. What your machine actually runs

Asymptotic rungs are one thing. Here is what three multipliers available from Python do when the operand size doubles:

```
# time_backends.py: measured growth of three real multipliers
import decimal
import math
import random
import sys
import time

import gmpy2

ctx = decimal.Context(prec=decimal.MAX_PREC, Emax=decimal.MAX_EMAX)


def best(f, reps=3):
    t = float("inf")
    for _ in range(reps):
        t0 = time.perf_counter()
        f()
        t = min(t, time.perf_counter() - t0)
    return t


rng = random.Random(1)
rows = []
for e in range(16, 25):                          # 2^16 .. 2^24 bits (about 20k .. 5M digits)
    bits = 2 ** e
    x, y = rng.getrandbits(bits) | 1, rng.getrandbits(bits) | 1
    gx, gy = gmpy2.mpz(x), gmpy2.mpz(y)
```

```
    digits = int(bits * math.log10(2))          # same size in decimal digits
    dx = decimal.Decimal("".join(rng.choices("0123456789", k=digits)))   # random coefficien
    dy = decimal.Decimal("".join(rng.choices("0123456789", k=digits)))
    rows.append((bits, best(lambda: x * y), best(lambda: ctx.multiply(dx, dy)), best(lambda:

print(sys.version.split()[0], "| decimal", decimal.__libmpdec_version__, "| GMP", gmpy2.mp_v
print(f"{'bits':>9} {'int ms':>9} {'x':>5} {'decimal ms':>11} {'x':>5} {'gmpy2 ms':>9} {'x':
prev = None
for r in rows:
    if prev:
        g = [r[i] / prev[i] for i in (1, 2, 3)]
        print(f"{r[0]:9d} {r[1]*1e3:9.2f} {g[0]:5.2f} {r[2]*1e3:11.2f} {g[1]:5.2f} {r[3]*1e3
    else:
        print(f"{r[0]:9d} {r[1]*1e3:9.2f} {'':>5} {r[2]*1e3:11.2f} {'':>5} {r[3]*1e3:9.3f}")
    prev = r
print("doubling n: x3.00 = Karatsuba (n^1.585); x2.0-2.2 = n log n territory")
```

```
venv-mult/bin/python time_backends.py
```

Output:

```
3.15.0 | decimal 4.0.0 | GMP 6.3.0
     bits    int ms     x  decimal ms     x  gmpy2 ms     x
    65536      0.92             1.99          0.090
   131072      2.60  2.84       4.48  2.26     0.278  3.08
   262144      8.38  3.22       9.74  2.17     0.600  2.16
   524288     24.09  2.87      20.18  2.07     1.387  2.31
  1048576     71.09  2.95      42.15  2.09     2.620  1.89
  2097152    218.45  3.07      89.92  2.13     6.683  2.55
  4194304    647.81  2.97     185.88  2.07    12.736  1.91
  8388608   1906.78  2.94     389.29  2.09    31.759  2.49
 16777216   5713.79  3.00     818.72  2.10    67.639  2.13
doubling n: x3.00 = Karatsuba (n^1.585); x2.0-2.2 = n log n territory
```

The "x" columns are the time ratio when n doubles. A Karatsuba-type algorithm should give $2^{1.585} = 3.0$, and an $n \log n$-type algorithm should give just over 2.

- **CPython `int`**: ×2.84–3.22, i.e. ×3 within noise at every size. CPython switches from schoolbook to Karatsuba above `KARATSUBA_CUTOFF` = 70 internal 30-bit digits (`Objects/longobject.c`) and has no Toom or FFT path. At 16 Mbit, one product takes 5.7 s.
- **`decimal`**: ×2.07–2.26. libmpdec dispatches by size in `_mpd_qmul`: schoolbook for short operands, Karatsuba when the result is 1024 words, and the three-prime NTT (`_mpd_fntmul`) above that. These operands are already in NTT territory at the first row. At 16 Mbit it is 7× faster than `int`, even though it works in base 10^{19}.
- **GMP** (through gmpy2): ×1.89–3.08, noisier because each call is fast, and about 80× faster than `int` at 16 Mbit. GMP moves through Toom-2/3/4/6/8 variants and then a Schönhage–

Strassen FFT modulo $2^N + 1$, at thresholds tuned per CPU (GMP manual, "Multiplication Algorithms").

So the "Karatsuba row" from Section 2 is what `x * y` does in Python. If your code multiplies million-digit integers, use `gmpy2`, or at least `decimal` with an unbounded context.

6. The 2026 claim: below $n \log n$, but by how much?

On 6 October 2026 (UTC) OpenAI's `openai/math` repository published *Integer multiplication below* $n \log n$ (dated 23 September 2026). Its Theorem 1 gives a deterministic multitape Turing machine with running time $O(n(\lg n)^{1-\kappa})$, $\kappa = 2^{-182}$. The paper presents this as disproving the Schönhage–Strassen $n \log n$ optimality conjecture *in that model*. The construction keeps Harvey–van der Hoeven's resampling. It speeds up the two operations that cost $\Theta(n \log n)$ on tapes, address rearrangements and butterfly layers, using finite linear networks with a rank saving (Sections 3–5 of the paper). Community repositories quickly tightened κ. As of 17 October 2026, `CrocSwap/integer-mult-bounds` lists a reviewed value $\kappa \approx 0.00167$, explicitly *conditional* on the OpenAI framework and "not a complete Lean multiplication theorem".

What does a power saving in the logarithm mean in practice?

```
# kappa.py: how much does O(n (lg n)^(1-kappa)) save over O(n lg n)?
import math

claims = {
    "OpenAI preprint (2026-09-23)": 2.0 ** -182,
    "community, reviewed (2026-10-17)": 836330098538459 / 500000000000000000,
}
sizes = {
    "1 GB operands  (n = 2^33 bits)": 33,
    "every atom (n = 2^266 bits)": 266,
    "n = 2^(2^64) bits": 2 ** 64,
}
print("speedup factor (lg n)^kappa, ignoring constants:")
for who, kappa in claims.items():
    print(f"  {who}: kappa = {kappa:.6g}")
    for label, lg in sizes.items():
        gain = math.expm1(kappa * math.log(lg))      # (lg n)^kappa - 1, without cancellatio
        print(f"    {label:<32} {gain:.3e}  (factor 1 + that)")
# how large must n be before the saving reaches 1%?
for who, kappa in claims.items():
    lg_needed = math.log(1.01) / kappa               # ln(lg n) >= ln(1.01) / kappa
    print(f"{who}: 1% saving needs ln(lg n) >= {lg_needed:.3e}, "
          f"i.e. lg n >= e^{lg_needed:.3e}")
```

```
python3.15 kappa.py
```

Output:

```
speedup factor (lg n)^kappa, ignoring constants:
  OpenAI preprint (2026-09-23): kappa = 1.63133e-55
    1 GB operands  (n = 2^33 bits)   5.704e-55  (factor 1 + that)
    every atom (n = 2^266 bits)      9.109e-55  (factor 1 + that)
    n = 2^(2^64) bits                7.237e-54  (factor 1 + that)
  community, reviewed (2026-10-17): kappa = 0.00167266
    1 GB operands  (n = 2^33 bits)   5.866e-03  (factor 1 + that)
    every atom (n = 2^266 bits)      9.383e-03  (factor 1 + that)
    n = 2^(2^64) bits                7.702e-02  (factor 1 + that)
OpenAI preprint (2026-09-23): 1% saving needs ln(lg n) >= 6.100e+52, i.e. lg n >= e^6.100e+5
community, reviewed (2026-10-17): 1% saving needs ln(lg n) >= 5.949e+00, i.e. lg n >= e^5.94
```

With the original $\kappa = 2^{-182}$, the speedup factor is $1 + 10^{-54}$ even for inputs with $2^{2^{64}}$ bits. The result matters for *mathematics*: it refutes a 55-year-old optimality conjecture in that model, and through the Harvey–van der Hoeven reduction it gives faster matrix transposition on tapes too. It has no effect on any multiplication that will ever run. The community value gives about 0.6% at gigabyte sizes, and only before constants that the paper itself calls "extremely large".

How to read a claim like this:

1. **Check the model.** The statement is about fixed multitape Turing machines. It says nothing new about RAM-model or real-hardware costs.
2. **Check the verification status.** OpenAI's catalogue (`lean/formalization.yaml`) lists formalizations for about 42% of the collection's top-line results. As of 18 October 2026 this manuscript is not among them, and the same release withdrew three other manuscripts on 7 October after a sign error was found. Community refinements verify arithmetic certificates in Lean, not the multiplication algorithm.
3. **Separate exponent records from speed.** A larger κ is a better theorem, not a faster program.

7. The trap: floating-point FFTs return wrong products silently

The quickest way to write "fast multiplication" is a complex FFT in `float64`, and it is how many first implementations go wrong. The convolution is computed approximately and then rounded to the nearest integer. That is correct only while the accumulated rounding error stays below 0.5, and the coefficients themselves must stay below 2^{53}, where `float64` stops representing every integer.

```
# float_fft_trap.py: floating-point FFT multiplication is exact only while rounding error <
import random

import numpy as np


def limbs(x, nbytes, limb_bytes):
    raw = np.frombuffer(x.to_bytes(nbytes, "little"), dtype=np.uint8).astype(np.float64)
    weights = 256.0 ** np.arange(limb_bytes)
```

```
    return raw.reshape(-1, limb_bytes) @ weights                # little-endian limbs


def fft_multiply(x, y, limb_bytes):
    nbytes = -(-max(x.bit_length(), y.bit_length()) // (8 * limb_bytes)) * limb_bytes
    a, b = limbs(x, nbytes, limb_bytes), limbs(y, nbytes, limb_bytes)
    n = 1 << (2 * len(a) - 1).bit_length()
    c = np.fft.irfft(np.fft.rfft(a, n) * np.fft.rfft(b, n), n)
    worst = float(np.max(np.abs(c - np.rint(c))))               # distance from the nearest in
    out, carry, base = [], 0, 1 << (8 * limb_bytes)
    for v in map(int, np.rint(c)):                              # carry propagation, linear ti
        carry += v
        out.append(carry % base)
        carry //= base
    data = b"".join(v.to_bytes(limb_bytes, "little") for v in out)
    return int.from_bytes(data, "little") + (carry << (8 * limb_bytes * len(out))), worst


rng = random.Random(3)
print("limb bits  operand bits  max |c - round(c)|  exact")
for limb_bytes in (2, 3):
    for bits in (2 ** 14, 2 ** 16, 2 ** 18, 2 ** 20, 2 ** 22, 2 ** 24):
        x = rng.getrandbits(bits) | (1 << (bits - 1))
        y = rng.getrandbits(bits) | (1 << (bits - 1))
        z, worst = fft_multiply(x, y, limb_bytes)
        print(f"{8 * limb_bytes:9d}  {bits:12d}  {worst:18.6f}  {z == x * y}")
```

```
venv-mult/bin/python float_fft_trap.py
```

Output:

```
limb bits  operand bits  max |c - round(c)|  exact
       16         16384            0.000244  True
       16         65536            0.001221  True
       16        262144            0.009766  True
       16       1048576            0.031250  True
       16       4194304            0.156250  True
       16      16777216            0.500000  False
       24         16384            0.000000  False
       24         65536            0.000000  False
       24        262144            0.000000  False
       24       1048576            0.000000  False
       24       4194304            0.000000  False
       24      16777216            0.000000  False
```

With 16-bit limbs the worst rounding distance grows steadily with length: 0.0002, 0.001, 0.01, 0.03, 0.16, then **0.5 and a wrong product** at 16 Mbit. That is the expected $\sqrt{N}$-ish error growth

eating the safety margin. The 24-bit rows are worse: *every* product is wrong, yet the "max distance from an integer" check reports **0.000000**. Coefficients reach $m \cdot 2^{48} > 2^{53}$, where adjacent doubles are 32 or more apart. Every float there is already an integer, so rounding looks perfect while the low bits are gone. A test that only checks closeness to integers can pass on output that is wrong.

The boring rules:

1. **Prove the budget, don't sample it.** For an exact transform, show $m(B-1)^2 < p$ (or below the CRT modulus). For a float FFT, use a published error bound for your FFT and keep a wide margin. `ntt_mult.py` asserts its budget before it computes anything.
2. **Never accept a float-FFT product without an independent check**, such as a comparison modulo a random 61-bit prime (`(x % q) * (y % q) % q == z % q`), which costs $O(n)$.
3. **Use the libraries.** GMP, libmpdec and FLINT have done the threshold tuning and error analysis. Python's `int` is fine below roughly 10,000 digits. Above that, use `gmpy2`.

8. Try this

1. Add Toom-2.5 (split one operand in 3, the other in 2, evaluate at 4 points) to `count_mults.py` for unbalanced operands. What is the leaf count for $3^a \times 2^b$ limbs?
2. Change `BASE_BITS` in `ntt_mult.py` to 4 and to 12. How do the budget and the transform length change for a 100,000-bit product? Which is faster in pure Python, and why?
3. Add a second prime $p_2 = 7 \cdot 2^{26} + 1$ to `ntt_mult.py` and reconstruct coefficients by CRT. How far does the budget move?
4. In `float_fft_trap.py`, add the modular check from the trap section and confirm that it catches every wrong row.
5. Using `kappa.py`, find the smallest κ for which the saving reaches 1% at $n = 2^{40}$ bits.

Checkpoint

- Derive Karatsuba's identity and solve $T(n) = 3T(n/2) + O(n)$.
- Explain why Toom-k costs $n^{\log_k(2k-1)}$ and why large k is not free.
- State the evaluate → pointwise multiply → interpolate pipeline, and the coefficient budget an exact NTT must respect.
- Name the bound and key idea of Schönhage–Strassen, Fürer and Harvey–van der Hoeven, and the model in which they hold.
- Explain what $O(n(\log n)^{1-\kappa})$ with $\kappa = 2^{-182}$ does and does not change.

Summary

Every fast multiplication algorithm is polynomial multiplication in disguise. Divide-and-conquer methods (Karatsuba, Toom–Cook) evaluate at a few points and pay $n^{1+\varepsilon}$. Transform methods evaluate at N roots of unity at once and pay $n \log n$ times whatever it costs to find a ring big enough for the coefficients. The 1971 → 2021 progress removed that overhead completely, and the 2026 claim shaves a $(\log n)^{\kappa}$ factor off what was thought to be the floor. In practice, Python's `int`

is a Karatsuba machine, `decimal` is an NTT machine, and GMP is a Toom-and-FFT machine. An FFT you write yourself is correct only up to a budget you have proved.

Sources

- A. Karatsuba, Yu. Ofman, "Multiplication of many-digital numbers by automatic computers", *Doklady Akad. Nauk SSSR* 145 (1962); A. L. Toom (1963); S. A. Cook, PhD thesis, Harvard (1966).
- A. Schönhage, V. Strassen, "Schnelle Multiplikation großer Zahlen", *Computing* 7 (1971) 281–292.
- M. Fürer, "Faster integer multiplication", *STOC 2007*; *SIAM J. Comput.* 39(3) (2009).
- D. Harvey, J. van der Hoeven, "Integer multiplication in time $O(n \log n)$", *Annals of Mathematics* 193(2) (2021), doi:10.4007/annals.2021.193.2.4, open copy hal-02070778 (Section 5: $d = 1729$; Section 5.4: $d \geq 9$).
- OpenAI, *Integer multiplication below $n \log n$*, preprint dated 23 September 2026, released 6 October 2026 (Theorem 1; §1.5 historical context; §1.7 lower-bound scope). Repository README, history and `lean/formalization.yaml`.
- CrocSwap/integer-mult-bounds: community refinements of κ, conditional on the OpenAI framework (README as of 17 October 2026).
- CPython 3.15 source: `Objects/longobject.c` (`KARATSUBA_CUTOFF`), `Modules/_decimal/libmpdec/mpdecim` (`_mpd_qmul`, `_mpd_kmul`, `_mpd_fntmul`, `_mpd_kmul_fnt`).
- GNU MP manual: Multiplication Algorithms (Toom variants, FFT multiplication, thresholds).
- M. Bodrato, "Towards optimal Toom-Cook multiplication for univariate and multivariate polynomials in characteristic 2 and 0", *WAIFI 2007* (interpolation sequences).

Karatsuba Multiplication

Karatsuba Multiplication

Thesis

Karatsuba multiplies two n-digit numbers with three recursive half-sized products instead of four, cutting complexity from $O(n^2)$ toward $O(n^{\log_2 3}) \approx O(n^{1.585})$. It is the gateway to Toom–Cook and FFT multiplication.

Mental model

```
x = x1 · 10^m + x0
y = y1 · 10^m + y0
z2 = x1 y1
z0 = x0 y0
z1 = (x0+x1)(y0+y1) − z0 − z2
xy = z2 · 10^{2m} + z1 · 10^m + z0
```

Three multiplies of $\sim m$-digit numbers beat four.

Worked example

```python
# karatsuba.py
def karatsuba(x, y):
    if x < 10 or y < 10:
        return x * y
    n = max(len(str(x)), len(str(y)))
    m = n // 2
    p = 10 ** m
    x1, x0 = divmod(x, p)
    y1, y0 = divmod(y, p)
    z0 = karatsuba(x0, y0)
    z2 = karatsuba(x1, y1)
    z1 = karatsuba(x0 + x1, y0 + y1) - z0 - z2
    return z2 * p * p + z1 * p + z0

a, b = 123456789, 987654321
print("karatsuba", karatsuba(a, b))
```

```
print("builtin  ", a * b)
```

```
karatsuba 121932631112635269
builtin   121932631112635269
```

Agreement with the builtin product is the sanity check; asymptotics show up on thousands of digits.

Trap

Using Karatsuba for tiny integers. Overhead of recursion and additions dominates below a cutoff — real big-int libraries switch by size. Off-by-one in m (split point) also corrupts place values.

Boring rule

Teach the three-product identity first; implement with a clear base case; only then discuss cutovers and Toom-3 / FFT.

Try this

1. Count recursive multiplies for 64-bit inputs vs schoolbook.
2. Replace base-10 splits with base 2^{32} for machine words.
3. Read the fast-integer-multiplication chapter sibling for the broader landscape.

Sources

- Karatsuba & Ofman (1962), multiplication algorithm
- Modern big-integer library design notes (GMP threshold tables)

Graph Theory

Graph Theory for Computer Science

Graphs model pairwise relationships: machines on a network, packages in a dependency tree, people in a social graph, basic blocks in a control-flow graph. Graph theory supplies the definitions; algorithms supply the computation. This part develops both with proofs, complexity, and systems examples.

What is a graph?

An **undirected graph** is $G = (V, E)$ with finite vertex set V and edge set $E \subseteq \{\{u, v\} : u, v \in V,\ u \neq v\}$ (simple graphs: no loops/multiedges unless stated).

A **directed graph** (digraph) uses ordered pairs $(u, v) \in V \times V$.

Weighted graphs attach $w : E \to \mathbb{R}$ (length, cost, capacity, latency).

Worked example 1 — many skins, one math

Domain	Vertices	Edges
Internet routing	Routers	Links with latency/weight
Build systems	Targets	Dependencies
Social networks	Users	Follows / friendships
Compilers	Basic blocks	Control-flow transfers
Knowledge graphs	Entities	Relations
ML	Tokens / objects	Attention or GNN message edges

Core computational problems

1. **Representation:** adjacency lists vs matrices—complexity hinges on this choice
2. **Traversal:** BFS/DFS; connectivity; topological order
3. **Trees & MSTs:** minimal connected backbones
4. **Shortest paths:** BFS / Dijkstra / Bellman–Ford / Floyd–Warshall
5. **Connectivity structure:** CCs, SCCs, bridges, articulation points
6. **Flow:** max-flow / min-cut; bipartite matching
7. **Systems patterns:** PageRank, dependency scheduling, routing

```
flow:
  B -> C
```

```
A -> D
D -> E
D -> F
B -> G
G -> H
```

Proof techniques you will reuse

Technique	Typical use
Induction on $\|V\|$ or path length	Tree properties, correctness
Loop invariants	Dijkstra, BFS layers
Cut properties	MST optimality
Exchange arguments	Greedy edge swaps
Potential / residual graphs	Flow augmentation
Counting double-touch	Handshaking lemma

Worked example 2 — handshaking

$\sum_{v \in V} \deg(v) = 2|E|$ because each edge contributes two to the degree sum. Corollary: number of odd-degree vertices is even.

Complexity baseline

Algorithm	Time (typical)
BFS/DFS	$\Theta(V + E)$ adj lists
Dijkstra (binary heap)	$O(E \log V)$
Bellman–Ford	$O(VE)$
Kruskal	$O(E \log E)$
Edmonds–Karp max-flow	$O(VE^2)$
Floyd–Warshall	$\Theta(V^3)$

Always state assumptions (weights nonnegative? directed?).

Worked example 3 — sparse vs dense

Web graph: $V \sim 10^{10}$ scale conceptual, $E = O(V)$. Matrix $V \times V$ is impossible; adjacency lists (or compressed) are mandatory. Complete graph K_n: $E = \Theta(n^2)$—matrix may win for dense kernels.

Correctness mindset

Graph algorithms are famous for subtle bugs: relaxing edges in the wrong order, mishandling 0-weight cycles, confusing weak and strong connectivity, off-by-one in topo sort cycle detection.

Habit: write the invariant before the code.

Worked example 4 — BFS invariant

When node u is first reached, $d[u]$ is the minimum hop distance from s. Proof: induction on distance layers.

Roadmap

1. Graph basics and representations
2. Trees and spanning trees / MST
3. Shortest paths
4. Connectivity and flow
5. Graph algorithms in real systems

Pitfalls

1. Using adjacency matrices on huge sparse graphs
2. Dijkstra with negative edges
3. Assuming undirected algorithms work unchanged on digraphs
4. Forgetting disconnected graphs (forests, multiple sources)
5. Treating PageRank iteration as "just a heuristic" without stochastic matrix intuition

Checkpoint

- Model a system as $G = (V, E)$ with clear semantics for weights
- Choose list vs matrix representation with a complexity justification
- Name the right shortest-path algorithm from graph properties
- State one invariant-style proof idea (BFS or Dijkstra)
- Map MST and flow to at least one systems task each

Exercises

1. Model a monorepo build as a digraph; what does a cycle mean?
2. Prove that the number of odd-degree vertices is even.
3. Give V, E counts for K_n and for a path on n vertices.
4. When is an adjacency matrix preferable to lists?
5. Is BFS or DFS better to find shortest hop paths? Why?
6. Explain cut property for MSTs in one paragraph.
7. Why can't Dijkstra handle negative edges? Give a tiny counterexample idea.
8. Define strongly connected component.
9. Reduce bipartite matching to max-flow at a high level.
10. Name three compiler analyses that use graphs.
11. Space complexity of adj list vs matrix for $E = \Theta(V)$.
12. What goes wrong if weights are positive but you use unweighted BFS for "shortest"?
13. Sketch how Union-Find supports Kruskal.
14. Give a real routing protocol and the graph problem it approximates.
15. Checkpoint: pick an app you use daily and formalize its core graph.

Summary

Graphs are the default language for structure in computer science. The chapters ahead make that language computational: representations, trees, paths, connectivity, flows, and production systems that run on these ideas at scale.

Graph Basics and Representations

This chapter fixes vocabulary and data structures. Almost every later complexity claim—"$O(V + E)$," "$O(1)$ edge query"—is really a claim about **representation**.

1. Formal definitions

Undirected simple graph: $G = (V, E)$, $E \subseteq \binom{V}{2}$.

Directed graph: $E \subseteq V \times V$ (loops (v, v) optional by convention).

Multigraph: parallel edges allowed; **weighted:** $w : E \to \mathbb{R}$.

Order $n = |V|$, **size** $m = |E|$ (common CS notation: n vertices, m edges; also V, E as numbers in asymptotics).

Neighborhoods and degrees

- Undirected: $\deg(v) = |\{u : \{u, v\} \in E\}|$
- Directed: $\text{indeg}(v)$, $\text{outdeg}(v)$
- $N(v)$ open neighborhood; $N[v] = N(v) \cup \{v\}$ closed

Paths and cycles

A **walk** is a sequence $v_0, e_1, v_1, \ldots, e_k, v_k$ with incident edges.
A **path** is a walk with distinct vertices.
A **cycle** returns to start with positive length and (usually) no repeated vertices except ends.

Connected (undirected): path between every pair.
Strongly connected (directed): directed path $u \rightsquigarrow v$ and $v \rightsquigarrow u$ for all pairs.

Worked example 1

$V = \{0, 1, 2, 3\}$, undirected edges $\{0, 1\}, \{0, 2\}, \{1, 3\}, \{2, 3\}$: a 4-cycle. Connected; every degree 2.

2. Handshaking lemma

Theorem. In any undirected graph, $\sum_{v \in V} \deg(v) = 2|E|$.

Proof. Each edge contributes 1 to the degree of each endpoint, total 2 per edge. ■

Corollary. The number of odd-degree vertices is even.

Proof. Sum of degrees even ⇒ even number of odd summands. ■

Worked example 2

Is there a graph with degrees $(3, 3, 3)$? Degree sum 9 odd—impossible.

Worked example 3 — directed version

$\sum_v \text{indeg}(v) = \sum_v \text{outdeg}(v) = |E|$.

3. Special families

Family	Property
Path P_n	n verts, $n-1$ edges, connected, two degree-1 ends
Cycle C_n	n verts, n edges, 2-regular connected
Complete K_n	all $\binom{n}{2}$ edges
Bipartite	$V = L \cup R$, edges only between L and R
DAG	directed acyclic graph
Tree	connected acyclic undirected (next chapter)

Theorem. G bipartite ⇔ no odd cycle.

Proof sketch. (⇒) odd cycle not 2-colorable. (⇐) BFS 2-color each component; monochrome edge would create odd cycle. ■

Worked example 4

C_4 bipartite; C_5 not. Grid graphs bipartite; social "must have odd friendship cycle" jokes rest on this theorem.

4. Adjacency matrix

Order vertices $v_1, \dots, v_n$. Matrix $A \in \{0,1\}^{n \times n}$:

$$A_{ij} = \begin{cases} 1 & \{v_i, v_j\} \in E \text{ (or arc)} \\ 0 & \text{otherwise.} \end{cases}$$

Undirected $\Rightarrow A = A^\top$. Weighted: store weights instead of 1.

Costs:

- Space $\Theta(n^2)$
- Edge query $O(1)$
- Iterate neighbors of v: $\Theta(n)$
- A^k_{ij} = number of walks of length k from i to j (unweighted)

Worked example 5

For a path of 3 edges, $(A^2)_{1,3} = 1$ (one walk of length 2 between ends? label carefully). Counting walks is a spectral graph theory entry point.

5. Adjacency lists

Array of n lists: for each v, store neighbors (or out-neighbors).

Costs:

- Space $\Theta(n + m)$
- Edge query $O(\deg(v))$ (or $O(1)$ expected with hash sets)
- Iterate neighbors: $O(\deg(v))$
- Best default for **sparse** graphs $m \ll n^2$

Worked example 6 — build from edge list

```
function BuildAdjList(n, edges, directed):
  g ← array of n empty lists
  for (u,v) in edges:
    g[u].append(v)
    if not directed: g[v].append(u)
  return g
```

Time $\Theta(n + m)$.

6. Edge lists and CSR

- **Edge list:** array of pairs—simple I/O, slow queries
- **CSR (compressed sparse row):** offsets + neighbor array—cache-friendly static graphs in HPC/ML

Worked example 7

CSR for directed graph: `offsets[i]` start of out-neighbors of i in flat `heads[]` array. Matvec $x \mapsto Ax$ streams memory linearly.

7. Graph isomorphism (awareness)

$G \cong H$ if a bijection of vertices preserves edges. Hard in theory (GI in quasi-poly; not known NP-complete); in practice use canonical labeling tools. Do not confuse with **equality of representations**.

8. Traversal foundations: BFS and DFS

BFS (queue)

From source s, explores by **distance layers**. Yields shortest paths in **unweighted** graphs.

Invariant: vertices dequeued in nondecreasing distance from s; first time reached, distance is final.

Time: $\Theta(n + m)$ adj lists.

DFS (stack / recursion)

Explores depth-first; produces discovery/finish times; classifies edges (tree/back/forward/cross in digraphs). Foundation for topo sort, SCCs, bridge finding.

Worked example 8 — BFS layers

On the 4-cycle from Example 1, BFS from 0: layer0 $\{0\}$, layer1 $\{1, 2\}$, layer2 $\{3\}$.

Worked example 9 — DFS edge types (undirected)

In undirected DFS, non-tree edges are back edges to ancestors—detect cycles if you see a neighbor that is visited and not the parent.

9. Connectivity algorithms (basics)

Undirected connected components: DFS/BFS forest; each tree a component. Time $\Theta(n + m)$.

Directed weak connectivity: ignore directions, run undirected CC.

Strong connectivity: needs Kosaraju/Tarjan (later chapter).

Worked example 10 — offline Union-Find

For undirected edges streaming, Union-Find builds components without full adj lists if only connectivity queries matter.

10. CS representation choices

Workload	Prefer
Sparse social / web	Adj lists / CSR
Dense small n DP on subsets	Matrix
Static repeated matvec	CSR
Dynamic insert/delete edges	Hash-set adj lists
GPU kernels	CSR/COO specialized

11. Pitfalls

1. Off-by-one when $V = \{1..n\}$ vs $\{0..n-1\}$
2. Forgetting reverse edges in undirected build
3. Storing matrix for $n = 10^6$
4. Using DFS recursion depth n on huge graphs (stack overflow)—prefer explicit stack
5. Assuming simple graph when input has multis/loops
6. **Directed vs undirected** degree and path confusion

12. Checkpoint

- State handshaking and odd-degree corollary
- Compare matrix vs list on space and neighbor iteration
- Run BFS by hand and list distances
- Detect a cycle with DFS parent rule (undirected)
- Build adj lists from an edge list correctly

Exercises

Easy

1. Prove the odd-degree corollary carefully.
2. Compute $|E|$ for K_n and for the complete bipartite $K_{a,b}$.
3. Draw all nonisomorphic undirected graphs on 3 vertices.
4. Give adj matrix and adj lists for a directed path $0 \to 1 \to 2 \to 3$.
5. Count space for lists vs matrix when $n = 10^5$, $m = 10^6$ (8-byte IDs).

Medium

6. Prove that if G is simple, $m \leq \binom{n}{2}$. When is equality?
7. Show $A^2_{ii} = \deg(i)$ for simple undirected graphs.
8. Prove bipartite $\Leftrightarrow$ 2-colorable $\Leftrightarrow$ no odd cycle (fill details).
9. Implement (pseudocode) BFS returning `dist[]` and `parent[]`; reconstruct path.
10. Complexity of checking whether an edge exists in list vs matrix vs hash-set lists.

Challenge

11. Prove that in a simple graph, if $\delta = \min \deg \geq 2$ then G contains a cycle.
12. Give an $O(n + m)$ algorithm to recognize bipartite graphs or reject with an odd cycle.
13. Show how to detect a directed cycle using DFS colors (white/gray/black).
14. Spectral teaser: relate regular graph degree to largest eigenvalue of A (state Perron–Frobenius intuition).
15. Design a memory layout for huge static undirected graphs with average degree $d = 10$; justify.

Summary

Graph vocabulary (degrees, paths, connectivity) plus representation (lists, matrices, CSR) determine what is computable efficiently. Handshaking and bipartite characterizations are first theorems; BFS/DFS are the first workhorse algorithms. Everything later—trees, shortest paths, flows—builds on these foundations.

Trees and Spanning Trees

Trees are minimally connected graphs—enough edges to link vertices, none wasted on cycles. They structure filesystems, decision procedures, heaps, and the backbone of **minimum spanning tree (MST)** algorithms used in clustering, network design, and approximation schemes.

1. Characterizations of trees

Definition. An undirected graph T is a **tree** if it is connected and acyclic.

Theorem (equivalent characterizations). For a graph T with $n \geq 1$ vertices, TFAE:

1. T is a tree (connected + acyclic)
2. T is connected and has $n - 1$ edges
3. T is acyclic and has $n - 1$ edges
4. There is a unique simple path between every pair of vertices
5. T is connected, but removing any edge disconnects T
6. T is acyclic, but adding any missing edge creates exactly one cycle

Proof sketch of (1)⇒(2). Proceed by induction on n. A tree with $n \geq 2$ has a leaf (degree 1)—prove via "longest path ends at a leaf." Remove leaf: remaining graph is a tree on $n-1$ vertices with one fewer edge. ■

Worked example 1

A path on n vertices: tree with 2 leaves. A star: one center, $n - 1$ leaves, still $n - 1$ edges.

Worked example 2

Two components each trees: a **forest**. Edges $= n - c$ for c components if acyclic.

2. Leaves and counting

Lemma. Every tree with $n \geq 2$ has at least two leaves.

Proof. Longest path's endpoints have no further extension ⇒ degree 1. ■

Worked example 3

Can a tree have degrees $(4, 3, 1, 1, 1)$? Sum $= 10 = 2|E| \Rightarrow |E| = 5$, but $n = 5$ needs $|E| = 4$ for a tree—impossible.

3. Rooted trees in CS

Choose root r. Define parent, children, ancestors, depth, height, subtrees.

Binary trees, heaps, BSTs, tries, segment trees, abstract syntax trees are rooted trees with extra labels/orderings.

Worked example 4 — heap property vs tree shape

Binary heap: complete binary tree shape + heap order on keys. Graph-theoretically still a tree; algorithms exploit array indexing $i \mapsto 2i, 2i + 1$.

Worked example 5 — LCA

Lowest common ancestor queries on rooted trees underpin tarjan offline algorithms and binary lifting—used in hierarchical data and some string algorithms.

4. Spanning trees

Definition. A **spanning tree** of connected undirected G is a subgraph that is a tree on all vertices of G.

Theorem. G connected $\Leftrightarrow$ G has a spanning tree.

Proof sketch. If connected, repeatedly delete an edge on a cycle until acyclic; connectivity preserved. Converse obvious. ■

Number of spanning trees of K_n is n^{n-2} (Cayley's formula)—classic enumerative result.

Worked example 6

Cycle C_n: exactly n spanning trees (delete any one edge).

5. Minimum spanning trees

Input. Connected undirected G with weights $w(e)$.
Goal. Spanning tree T minimizing $w(T) = \sum_{e \in T} w(e)$.

Cut property

Theorem (cut property). For any cut $(S, V \setminus S)$ with S neither empty nor V, if e is a lightest edge crossing the cut, then some MST contains e.

Proof idea. Take any MST T. If $e \in T$, done. Else $T + e$ has a cycle; that cycle crosses the cut on some e'. Swap e' for e: weight does not increase; new spanning tree. ■

Cycle property

Theorem. For any cycle C, if e is a strictly heaviest edge on C, then e lies in no MST.

Worked example 7 — uniqueness

If all edge weights are distinct, the MST is unique (standard corollary of cut/cycle properties).

6. Kruskal's algorithm

1. Sort edges by increasing weight
2. Initialize Union-Find on vertices
3. For each edge uv in order: if u, v in different components, add uv and unite
4. Stop at $n - 1$ edges

Correctness: each added edge is a lightest edge across the cut between components (cut property).
Time: $O(m \log m)$ dominated by sorting (or $O(m\alpha(n))$ after sort with Union-Find).

Worked example 8

Edges: $(A, B, 1), (C, D, 1), (B, C, 2), (A, C, 4), (B, D, 5)$.
Kruskal adds AB, CD, BC; total weight 4. Rejects AC, BD as cycles.

7. Prim's algorithm

Grow a tree from start vertex s: repeatedly add the lightest edge from the tree set S to $V \setminus S$.

Implementation: binary heap of candidate edges/vertices $\Rightarrow O(m \log n)$; Fibonacci heap $O(m + n \log n)$ theoretically.

Correctness: each addition applies the cut property to $(S, V \setminus S)$.

Worked example 9 — Prim on same graph

Start at A: take AB (1), then BC (2) or compare AC (4), then to D via CD (1). Same MST weight 4.

8. Kruskal vs Prim

	Kruskal	Prim
Nature	Global sort + DSU	Grow one component
Sparse graphs	Excellent	Good with heaps
Dense graphs	Sort n^2 edges costly	Can be competitive
Parallelism	Sorting parallelizable	Less natural
Online weights	Needs all edges	Same

9. Applications

- **Network design:** cheapest cable connecting sites
- **Clustering:** single-linkage clustering related to MST
- **Approximation:** MST is a 2-approx building block for metric TSP (Christofides goes further)
- **Image segmentation / vision:** graph cuts more flow-based, but MST used in some hierarchical methods
- **Physics / percolation intuition:** lightest edges first

Worked example 10 — single-linkage

Merge clusters by min inter-cluster edge—same process as Kruskal; dendrogram is hierarchical clustering.

10. Directed and other variants

- **Arborescence / branching:** directed analogues (Edmonds' algorithm)—harder
- **Minimum bottleneck spanning tree:** minimize the max edge weight—related but different objective; still uses MST ideas
- **Steiner tree:** must span a **subset** of terminals—NP-hard

11. Pitfalls

1. Running MST algorithms on disconnected graphs (get forest; detect $c > 1$)
2. Assuming uniqueness when weights repeat
3. Using directed edges with Kruskal naively
4. Floating-point weight comparisons without stable tie breaks
5. Confusing MST with shortest-path tree (different objectives!)

Worked example 11 — SPT $\neq$ MST

A graph can have a shortest-path tree from s that is not a minimum spanning tree. Shortest paths optimize distances from s; MST optimizes total edge weight globally.

12. Checkpoint

- List equivalent tree characterizations and prove $|E| = n - 1$ by induction
- State cut property and why Kruskal/Prim are safe
- Execute Kruskal and Prim on a 5-edge graph
- Contrast MST vs shortest-path tree
- Choose Kruskal vs Prim for a sparse instance

Exercises

Easy

1. Prove that a tree with exactly two leaves is a path.
2. Show a connected graph with n vertices and n edges contains exactly one cycle (as a set of edges? carefully: at least one).
3. Run Kruskal on a weighted K_4 of your choice.
4. Prove: adding an edge to a tree creates exactly one cycle.
5. Cayley: how many spanning trees does K_3 have? Check by hand.

Medium

6. Complete the induction that trees have ≥ 2 leaves for $n \geq 2$.
7. Prove the cycle property from the cut property (or vice versa).
8. Show that if weights are distinct, MST is unique.
9. Give a counterexample graph where some shortest-path tree is not an MST.
10. Analyze Union-Find Kruskal complexity with path compression + union by rank.

Challenge

11. Prove Cayley's formula for $n \leq 5$ by enumeration; outline Prüfer code idea for general n.
12. Design an algorithm for minimum bottleneck spanning tree; prove correctness.
13. Show metric TSP has a 2-approximation via MST + Euler tour shortcut (classic).
14. Discuss implementation: when to store edges vs adj lists for Prim.
15. Research: Borůvka's algorithm and its role in parallel MST / history of the problem.

Summary

Trees are the unique minimal connectors of a vertex set. Spanning trees always exist in connected graphs; MSTs optimize total weight via the cut property. Kruskal and Prim are correct greedy algorithms with complementary implementation profiles—and they reappear across clustering, networking, and approximation algorithms.

Shortest Paths

Shortest-path problems ask for minimum-cost routes in weighted graphs. The right algorithm depends on **weight signs**, **query type** (single-source vs all-pairs), and **graph density**. This chapter develops BFS, Dijkstra, Bellman–Ford, and Floyd–Warshall with invariants and failure modes.

1. Problem variants

Let $G = (V, E)$ with weight $w : E \to \mathbb{R}$. A path P has weight $w(P) = \sum_{e \in P} w(e)$.

Variant	Output
Single-pair	$\text{dist}(s, t)$ and a path
Single-source (SSSP)	$\text{dist}(s, \cdot)$ to all
All-pairs (APSP)	$\text{dist}(u, v)$ for all pairs
k-shortest / constrained	variants with extra structure

Assumption for "shortest path exists": no **negative cycle** reachable on the relevant portion of the graph (otherwise weights unbounded below).

Worked example 1 — hop vs weight

Unweighted: minimize number of edges. Weighted: minimize sum. BFS solves the first, not the second (unless all weights equal).

2. Unweighted graphs: BFS

Algorithm. Queue from s; first time a node is discovered, set $\text{dist}[v] = \text{dist}[u] + 1$.

Theorem. BFS computes hop-distances in $O(n + m)$ time.

Invariant. Nodes are processed by nondecreasing distance; when dequeued, distance is final.

Worked example 2

Grid pathfinding without diagonal costs: BFS. With different terrain costs: Dijkstra.

3. Nonnegative weights: Dijkstra

Assumption. $w(e) \geq 0$ for all e.

Idea. Maintain tentative distances $d[v]$; repeatedly settle the unsettled vertex u with smallest $d[u]$; relax all edges out of u.

Relaxation: if $d[v] > d[u] + w(u, v)$, set $d[v] \leftarrow d[u] + w(u, v)$ and parent $v \leftarrow u$.

Theorem. When u is settled, $d[u] = \text{dist}(s, u)$.

Proof sketch. Suppose not; consider the first wrongly settled u. On a true shortest path to u, look at the first unsettled edge leaving the settled set—nonnegative weights make the cut-crossing argument work (classical Dijkstra proof). ■

Complexity. Binary heap: $O(m \log n)$. Dial's algorithm / radix structures for integer weights.

Worked example 3

Edges: $S \to A(1), S \to B(4), A \to B(2), A \to C(6), B \to C(3)$.

Order settled: $S(0), A(1), B(3)$ via A, $C(6)$ via B. Path S-A-B-C weight 6.

Worked example 4 — failure with negatives

Edge $A \to B$ weight 1, $S \to B$ weight 2, $S \to A$ weight 3, and $B \to A$ weight -2 can break the "settled forever" claim. Use Bellman–Ford when negatives exist.

4. Negative weights: Bellman–Ford

Algorithm. Initialize $d[s] = 0$, others ∞. Repeat $n - 1$ times: relax **all** edges. Optional nth pass: if any relaxation possible, a negative cycle is reachable.

Theorem. After k full relaxation rounds, $d[v]$ equals the minimum weight of an s–v path using at most k edges (or ∞).

Corollary. $n-1$ rounds suffice for simple shortest paths (at most $n-1$ edges). Extra round detects negative cycles.

Complexity. $O(nm)$.

Worked example 5 — negative cycle detect

If after $n - 1$ rounds an edge still improves a distance, there is a walk of length $\geq n$ improving cost $\Rightarrow$ cycle with negative total weight in the predecessor graph.

Worked example 6 — currency arbitrage

Log-transform exchange rates: product of rates $> 1 \Leftrightarrow$ negative cycle in $-\log$ weights. Bellman–Ford detects arbitrage.

5. All-pairs: Floyd–Warshall

DP. Let $d_{ij}^{(k)}$ be shortest $i \to j$ path using intermediate vertices only from $\{1, \dots, k\}$.

$$d_{ij}^{(k)} = \min\left(d_{ij}^{(k-1)},\ d_{ik}^{(k-1)} + d_{kj}^{(k-1)}\right).$$

In-place implementation with care about diagonal (negative cycle if $d_{ii} < 0$).

Complexity. $\Theta(n^3)$ time, $\Theta(n^2)$ space.

When to use. Dense graphs, small n ($\lesssim$ 400–1000 depending on constants), need full matrix.

Worked example 7

Compare: $n = 1000$, $m = 5000$ sparse SSSP many times with Dijkstra vs one Floyd—Dijkstra multi-source usually wins for sparse.

6. Johnson's algorithm (sketch)

APSP on sparse graphs with negatives (no neg cycles): Bellman–Ford potentials from a super-source to reweight edges to nonnegative, then Dijkstra from each vertex. Time $O(nm + n^2 \log n)$ with heaps.

7. Algorithm selection

```
flow:
  B -> C  (all equal / unweighted)
  B -> D  (nonnegative)
  B -> E  (negatives)
  A -> F
  F -> G  (yes dense)
  F -> H  (yes sparse)
```

Worked example 8 — maps

Road networks: nonnegative travel times $\Rightarrow$ Dijkstra + hierarchical speedups (contraction hierarchies, A* with landmarks)—engineering beyond textbook but based on the same invariants.

8. Path reconstruction

Store parent[v] on successful relaxations. Walk parents from t to s, reverse. For Floyd, store next[i][j] intermediate hop.

Worked example 9

From Example 3: parent[C]=B, parent[B]=A, parent[A]=S $\Rightarrow$ path S-A-B-C.

9. A* search (informed SSSP)

Priority key $d[u] + h(u)$ with heuristic h estimating dist to target. **Admissible** h (never overestimates) preserves optimality. **Consistent** heuristics keep Dijkstra-like properties.

Worked example 10

Grid with Euclidean h: A* expands fewer nodes than Dijkstra toward a goal.

10. Pitfalls

1. Dijkstra + negative edges
2. Not initializing distances to ∞ / not handling unreachable
3. Floating weights and equality tests
4. Overflow if using large "INF" in Floyd
5. Confusing undirected edges (implement as two arcs) with Dijkstra on one direction only
6. Reporting a path when a negative cycle can reach it—undefined / $-\infty$

11. Checkpoint

- Prove why BFS layers are hop-optimal
- State Dijkstra's invariant and nonnegativity need
- Detect negative cycles with Bellman–Ford
- Write Floyd recurrence and complexity

- Reconstruct paths from parent pointers

Exercises

Easy

1. Run BFS distances on a binary tree from the root (hop metric).
2. Execute Dijkstra by hand on a 6-edge digraph you draw.
3. Show a 3-node counterexample where Dijkstra fails with a negative edge.
4. Complexity of Bellman–Ford on a complete digraph.
5. When is APSP via n Dijkstras better than Floyd?

Medium

6. Prove BFS correctness by induction on distance.
7. Prove: if all weights are positive, shortest paths are simple (no cycles).
8. Implement (pseudocode) Bellman–Ford with negative-cycle vertex reporting via parent graph.
9. Show Floyd can detect negative cycles via diagonal.
10. Prove that reweighting $w'(u, v) = w(u, v) + h(u) - h(v)$ preserves shortest paths (potential method).

Challenge

11. Derive Johnson's algorithm fully and prove nonnegativity of reweighted edges when no neg cycle.
12. Analyze binary heap Dijkstra carefully (decrease-key counts).
13. A*: prove admissible heuristic $\Rightarrow$ optimal path when goal is settled.
14. Differential constraints: $x_j - x_i \leq b_k$ as edges; feasibility via Bellman–Ford.
15. Systems: sketch how OSPF relates to shortest paths and what "metrics" mean.

Summary

Shortest paths are settled by matching algorithm to weight structure: BFS, Dijkstra, Bellman–Ford, Floyd/Johnson. Invariants and negative-cycle handling separate correct systems from subtle production bugs.

Connectivity, Components, and Network Flow

Connectivity describes how a graph falls into pieces; flow describes how much commodity can move through capacitated edges. Together they power reliability analysis, bipartite matching, scheduling, and segmentation.

1. Undirected connectivity

A **connected component** is a maximal connected subgraph. Compute via DFS/BFS forest or Union-Find in $O(n + m)$ or nearly $O(m\alpha(n))$.

Bridge (cut-edge): edge whose removal increases the number of components.
Articulation point (cut-vertex): vertex whose removal increases components.

DFS low-link values find bridges/articulation points in linear time.

Worked example 1

Two cliques sharing a single vertex: that vertex is an articulation point; no bridge if cliques are large complete.

Worked example 2 — reliability

In a network, bridges are single points of link failure. Designing edge-redundant networks means ensuring edge-connectivity ≥ 2.

2. Edge- and vertex-connectivity

- $\kappa'(G)$: min edges to disconnect G (edge-connectivity)
- $\kappa(G)$: min vertices to disconnect (vertex-connectivity)

Menger's theorem (informal): max number of edge-disjoint s–t paths equals min size of an s–t edge cut. Vertex version analogous.

This is the combinatorial heart linking connectivity to flow.

3. Directed strong connectivity

Strongly connected component (SCC): maximal set where every pair is mutually reachable.

Condensing each SCC to a node yields a **DAG** of components.

Kosaraju's algorithm

1. DFS on G, push vertices in finish order
2. Transpose graph G^T
3. DFS on G^T in decreasing finish order; each tree is an SCC

Time: $\Theta(n + m)$.

Tarjan's algorithm

Single DFS with `low` and stack—same asymptotics, less graph copying.

Worked example 3

A cycle with a dangling edge: one big SCC on the cycle; the dangling vertex may be its own SCC depending on directions.

Worked example 4 — 2-SAT

Implication graph: SCCs decide satisfiability (x and $\neg x$ not in same SCC). Classic CS application of strong connectivity.

4. Topological sort on DAGs

Order vertices so every edge $u \to v$ has u before v.

Theorem. A digraph has a topological order iff it is a DAG.

Kahn's algorithm

Compute indegrees; queue zeros; pop and decrement successors. If fewer than n pops, a cycle exists.

DFS finishing times

Reverse finish order on a DAG is a topo order.

Worked example 5 — build systems

Targets with dependencies: topo order is a valid build order; cycle ⇒ circular dependency error.

Worked example 6 — course prerequisites

Same pattern; detect cycles as curriculum bugs.

5. Flow networks

Flow network: digraph with source s, sink t, capacities $c(u, v) \geq 0$.

A **flow** f satisfies:

1. **Capacity:** $0 \leq f(u, v) \leq c(u, v)$ (or antisymmetric formulation)
2. **Conservation:** for $v \notin \{s, t\}$, $\sum_u f(u, v) = \sum_w f(v, w)$

Value $|f|$ = net flow out of s.

Max-flow problem: maximize $|f|$.

6. Residual graphs and augmenting paths

Residual capacity $c_f(u, v) = c(u, v) - f(u, v)$ plus reverse edges for canceling flow. An **augmenting path** is an $s \rightsquigarrow t$ path in the residual graph. Augment by the bottleneck residual capacity.

Ford–Fulkerson method: while augmenting path exists, augment.

Termination: if capacities integral, max flow integral and algorithm terminates. With pathological real capacities, need care (Edmonds–Karp).

Edmonds–Karp

Choose **shortest** (BFS) residual augmenting path each time. Complexity $O(nm^2)$.

Worked example 7 — tiny max flow

$s \to a(2), s \to b(2), a \to t(1), b \to t(2), a \to b(1)$. Compute augmentations until residual disconnects s from t. Max flow value 3 in a standard working of this shape (verify by hand).

7. Max-flow min-cut theorem

An s–t **cut** is a partition (S, T) with $s \in S$, $t \in T$. Capacity $c(S,T) = \sum_{u \in S, v \in T} c(u,v)$.

Theorem (max-flow min-cut). Maximum flow value equals minimum cut capacity.

Proof idea. Weak duality: any flow $\leq$ any cut. When no residual s–t path, set S = residual-reachable from s; flow saturates the cut and equals its capacity. ■

Certificate: a min cut proves optimality of a flow without re-running search from scratch.

Worked example 8

After max flow, residual unreachable set defines a bottleneck cut—useful in reliability ("these links are critical").

8. Bipartite matching via flow

Bipartite graph $(L \cup R, E)$: add $s \to L$, $R \to t$, edges $L \to R$ capacity 1 (and s, t edges capacity 1). Max flow = maximum matching cardinality.

Hall's marriage theorem gives combinatorial existence conditions; flow gives an algorithm.

Worked example 9 — assignment

Workers L, jobs R, edges "qualified": max matching assigns as many as possible.

Worked example 10 — König's theorem

In bipartite graphs, max matching size = min vertex cover size—linked through flow/cut duality.

9. Other flow applications

- **Edge-disjoint paths:** unit capacities (Menger)
- **Circulation with demands:** reduce to max flow with extras
- **Project selection / closed graph cuts:** reduction to min cut
- **Image segmentation:** pixels as vertices; min cut separates foreground/background (Boykov–style energy)

10. Dinic and beyond (awareness)

Blocking flows / Dinic $O(n^2 m)$, unit network special cases faster; push-relabel practical; for huge graphs, approx or specialized matchings.

11. Pitfalls

1. Forgetting reverse residual edges (cannot "undo" flow)
2. Using DFS Ford–Fulkerson with bad paths ⇒ exponential on some graphs
3. Non-integral capacities and termination
4. Confusing weak connectivity with SCCs
5. Topo sort on graphs with cycles without detection
6. Modeling undirected edges as one-way arcs in flow

12. Checkpoint

- Compute CCs and explain bridges
- Run Kosaraju mentally on a 6-vertex digraph
- Topo-sort a DAG with Kahn and detect cycles
- State max-flow min-cut and residual augmentation
- Reduce bipartite matching to max flow

Exercises

Easy

1. Find SCCs of a digraph consisting of two cycles sharing a vertex (directions matter—draw carefully).
2. Topo-sort a small course-prerequisite DAG.
3. Define residual capacity after sending 3 units on a capacity-5 edge.
4. Why does a min cut certify max flow?
5. Build the matching flow network for $K_{2,2}$.

Medium

6. Prove that the condensation of SCCs is a DAG.
7. Prove Kahn's algorithm fails to place all vertices iff a cycle exists.
8. Execute Edmonds–Karp on a graph with at least two augmentations; list path lengths.
9. Prove weak duality: $|f| \le c(S,T)$ for every s–t cut.
10. Show that integral capacities ⇒ integral max flow exists (from FF augmentations).

Challenge

11. Detail the 2-SAT SCC algorithm and prove correctness sketch.
12. Find bridges with DFS `low` values; prove the bridge criterion.
13. Model "edge connectivity between s, t" as a flow; invoke Menger.
14. Image cut: write unary and pairwise energy as flow edges (high-level is fine).
15. Compare Dinic vs Edmonds–Karp asymptotics and when it matters.

Summary

Connectivity algorithms expose structure (components, cuts, DAGs of SCCs). Flow algorithms optimize throughput with residual augmentation and are certified by min cuts. Matching, disjoint paths, and many planning problems are flow in disguise.

Graph Algorithms in Real Systems

Textbook graph algorithms reappear—sometimes thinly disguised—in compilers, build systems, networks, databases, search engines, and ML. This chapter maps production problems to the math of previous chapters and highlights engineering constraints (scale, dynamism, approximation).

1. Ranking and the web graph: PageRank

Model. Web pages V, hyperlinks directed edges. Random surfer: with probability α follow a random out-link; with probability $1-\alpha$ jump by distribution v (typically uniform).

Iteration.

$$r^{(t+1)} = \alpha P^\top r^{(t)} + (1-\alpha)v,$$

where P is the row-stochastic matrix of the (dangling-node-fixed) graph.

Math. r converges to the stationary distribution of a related Google matrix—dominant eigenvector of a positive matrix (Perron–Frobenius).

Worked example 1

Tiny 3-page graph: write P, iterate from uniform r a few steps; observe mass concentrating on well-linked pages.

Worked example 2 — systems issues

- Power iteration on graphs with 10^{10} edges: sparse matvec, partitioning
- Dangling nodes (out-degree 0): special handling
- Spam and link farms: adversaries against the model

2. Build systems and package managers

Model. Targets/packages as vertices; dependency edges $A \to B$ meaning "A depends on B" (direction conventions vary—fix them).

Tasks.

1. Cycle detection (DFS / SCC) $\Rightarrow$ error
2. Topological scheduling of builds
3. Incremental rebuild: recompute only descendants of changed nodes
4. Parallel build: schedule ready zero-indegree tasks (Kahn online)

Worked example 3 — Bazel / make

Change a leaf library: transitive reverse-deps invalidation is graph reachability in the reverse dependency graph.

Worked example 4 — npm/cargo

Version resolution is harder than pure DAGs (constraints, diamonds); still, the backbone is dependency digraph analysis.

3. Compiler graphs

Graph	Role
Control-flow graph (CFG)	Blocks and branches; dominance, liveness
Call graph	Who calls whom; devirtualization limits
SSA def-use	Dataflow precision
Interference graph	Register allocation = graph coloring

Register allocation: color interference graph with k colors (registers). Graph coloring is NP-hard; compilers use heuristics (Chaitin-Briggs) and spilling.

Worked example 5 — dominance

Dominator tree from CFG: d dominates n if all paths from entry to n go through d. Computed by iterative dataflow or Lengauer–Tarjan—enables SSA construction.

Worked example 6 — dead code

If a block is unreachable from entry in CFG, it can be removed—graph reachability.

4. Databases and query plans

Join orders and operator pipelines form trees/DAGs. Cost-based optimizers search a space of plan graphs with dynamic programming (System R style) or heuristics for large queries.

Worked example 7

Bushy vs left-deep join trees: different graph shapes, different intermediate sizes. Math: cost models + search, not only SQL parsing.

Foreign keys / transitive closure: recursive CTEs implement graph reachability in the engine.

5. Networking and routing

- **OSPF / link-state:** routers flood topology; each runs shortest paths (Dijkstra) on weighted graphs
- **Distance-vector:** Bellman–Ford-like distributed relaxation (with historical count-to-infinity issues)
- **BGP:** path-vector with **policy**—not pure shortest path; export/import rules dominate

Worked example 8

Traffic engineering: adjust link weights so shortest-path routing balances load—inverse optimization flavor.

Worked example 9 — SDN

Central controller computes paths (often MCF multi-commodity flow approximations) and installs forwarding rules.

6. Distributed systems and consensus adjacency

- **Leader election / spanning trees** on network overlays
- **Gossip:** epidemic spreading on graphs; cover times
- **Service mesh:** dependency graphs for cascading failure analysis
- **Causal order:** DAG of vector-clock consistent events

Worked example 10 — blast radius

Call-graph of microservices: which services fail if X dies? Reverse reachability / dependents.

7. ML and data

- **Knowledge graphs:** multi-relational digraphs; path queries, embeddings
- **GNNs:** message passing $h_v^{(t+1)} = \psi(h_v^{(t)}, \bigoplus_{u \in N(v)} \phi(h_u^{(t)}, h_v^{(t)}, e_{uv}))$
- **Factor graphs / Bayesian networks:** inference structure (trees exact; loops need approx)
- **Recommendation:** bipartite user–item graphs; random walks, spectral methods

Worked example 11 — label propagation

Community detection / semi-supervised learning: iterative averaging on graph edges—related to harmonic functions and Laplacian systems.

8. Production concerns checklist

Issue	Graph response
Scale	Streaming algorithms, sketching, sampling
Dynamics	Dynamic MST/SP trees research; often recompute shards
Distribution	Graph partition (edge/vertex cut), Pregel/GraphX model
Latency	Precompute SCCs, landmarks for distances
Correctness	Property tests: no cycles in builds; flow conservation audits
Approximation	Sketch PageRank, approximate neighborhoods

Worked example 12 — partition quality

Edge-cut vs vertex-cut partitions change communication volume for GNN training and distributed PageRank.

9. Choosing algorithms under constraints

1. **Exact needed?** (compilers correctness vs ranking quality)
2. **Static or streaming?**
3. **Weighted? negative?**
4. **Memory per machine?**
5. **Is a 2-approx enough?** (matching, cover)

Worked example 13 — bipartite matching at scale

Ad assignment: Hopcroft–Karp or flow on huge graphs may be too slow; use greedy auction algorithms with approximation guarantees.

10. Capstone synthesis patterns

Map each system task:

$$\text{System problem} \to \text{Graph model} \to \text{Algorithm class} \to \text{Failure mode.}$$

Examples:

- Circular dependency → digraph cycle → DFS/SCC → hard error
- Fast web rank → stochastic matrix → power iteration → spam
- Low-latency route → weighted digraph → Dijkstra/A* → stale weights

11. Pitfalls

1. Wrong edge direction in dependency modeling
2. Treating BGP as OSPF-like shortest path

3. Full Floyd on continental road networks
4. Ignoring multi-edges / weights units (ms vs km)
5. Coloring interference graphs with exponential exact solvers in the hot path
6. GNN over-smoothing: too many message-passing layers collapse features

12. Checkpoint

- Write PageRank's update and name the stationary-distribution view
- Schedule a build with topo sort and detect cycles
- Connect register allocation to graph coloring
- Contrast link-state vs path-vector routing mathematically
- Reduce one ML graph task to message passing or spectral structure

Exercises

Easy

1. Draw a 4-service microservice dependency graph; list rebuild order after one change.
2. Explain dangling nodes in PageRank in one paragraph.
3. Why is CFG reducibility historically important for compilers?
4. Give one reason BGP is not pure Dijkstra.
5. Name a bipartite graph in recommender systems.

Medium

6. Pseudocode incremental rebuild: given changed set C, mark all reverse-reachable dependents.
7. Show a tiny interference graph needing 3 registers; attempt a greedy coloring.
8. Compare A* vs Dijkstra for point-to-point routes with a consistent heuristic.
9. Write the Google matrix $G = \alpha P + (1-\alpha)\mathbf{1}v^\top$ and argue primitivity for large enough teleport.
10. Model deadlock in lock-acquire graphs: what does a cycle mean?

Challenge

11. Implement (pseudocode) power iteration with dangling-node fix; discuss convergence criterion $\|r^{t+1} - r^t\|_1$.
12. Research Lengauer–Tarjan dominators at high level: why linear-ish time matters.
13. Design a min-cut based image segmentation energy for a 3×3 pixel toy.

14. Propose a graph partitioning objective (edge cut + balance) and why it's hard.
15. Capstone report outline: pick one production graph problem; specify model, algorithm, complexity, monitoring metrics, and degradation mode under graph growth.

Summary

Graph theory in CS is operational infrastructure. Ranking, builds, compilers, routing, databases, and ML all instantiate connectivity, orderings, shortest paths, flows, and spectral iteration—under brutal constraints of scale and change. Fluent mapping from system need to graph problem is the professional skill this part aims to build.

Graph Theory Workshop: Exercises and Diagrams

A practice chapter: definitions → diagrams → multi-step problems. Use after graph basics / trees / paths.

Diagram bank

Undirected simple graph

```
  A--B
  | /|
  |/ |
  C--D
```

Vertices $\{A, B, C, D\}$; edges as drawn.

Directed

```
A → B → C
↑
    D ←    (sketch your own arrows carefully)
```

Tree vs cyclic

```
tree:  A           not tree:
      / \             A-B
     B   C            | |
                      D-C   (cycle)
```

1. Degree and handshaking

Problem. Degrees $3, 3, 2, 2, 2$. Is there a simple graph? Draw one or prove impossible.

Hint: sum of degrees must be even; graphical sequences (Havel–Hakimi) optional.

2. Paths and connectivity

On the first diagram: list all simple paths from A to D. Is the graph 2-connected?

3. BFS layers

From A, write BFS layers (assume adjacency order alphabetical).

```
layer 0: A
layer 1: ...
layer 2: ...
```

4. Spanning trees

How many different spanning trees does a triangle K_3 have? What about K_4 (Cayley: n^{n-2} labeled trees on n vertices)?

5. Shortest paths

```
A--1--B--4--D
|     |     |
2     1     1
|     |     |
C--3--E--1--F
```

Run Dijkstra from A; report distances to all nodes.

6. Topological order

DAG:

```
1 → 3 → 5
↓   ↓
2 → 4
```

List all valid topological orders.

7. Flow cut (conceptual)

Explain max-flow min-cut in one paragraph with a 4-edge network of your drawing.

8. CS mapping

Match: (a) dependency install (b) garbage collector reachability (c) routing (d) social "friend of friend"

to: BFS / DFS / topo sort / shortest path.

Solutions sketch (spoiler zone)

1. Sum = 12 even; e.g. cycle C_5 with one chord works for multiset of degrees — verify by construction.

2. K_3: 3 spanning trees (each omit one edge). K_4: $4^2 = 16$.

3. Distances depend on exact edges; recompute carefully.

4. e.g. $1, 2, 3, 4, 5$ and $1, 3, 2, 4, 5$ variants.

5. (a) topo (b) DFS/BFS reachability (c) shortest path (d) BFS layers.

More practice

1. Prove a tree on n vertices has $n - 1$ edges.

2. Prove that if G is bipartite, every cycle is even.

3. Give an example where Dijkstra fails with a negative edge.

4. **Project:** implement BFS and check bipartite coloring; write the invariant.

Numerical Methods

Numerical Methods for Computer Science

Mathematical formulas assume real numbers and exact arithmetic. Computers use finite binary encodings and finite time. **Numerical methods** study algorithms that compute approximate answers with **controlled error**, **stability**, and **scalability**. This part is essential for scientific computing, graphics, simulation, optimization, and the numeric core of ML frameworks.

The central tension

Ideal math	Machine reality
$\mathbb{R}$ arithmetic	Floating-point $\mathbb{F}$
Exact A^{-1}	Approximate solve / iterative methods
Continuous derivatives	Finite differences or AD with roundoff
Infinite series	Truncation + rounding
Global optima	Local solvers, budgets

A method can be **mathematically convergent** yet **numerically useless** if unstable.

Core themes

1. **Floating-point representation and rounding**
2. **Forward vs backward error; conditioning vs stability**
3. **Root finding** (nonlinear equations)
4. **Linear systems** (direct and iterative)
5. **Numerical optimization** (smooth methods in finite precision)

```
flow:
  B -> C
  B -> D
  B -> E
  D -> E
```

Conditioning vs stability (preview)

- **Conditioning:** sensitivity of the **problem**. Ill-conditioned: small input changes ⇒ large output changes, even with exact arithmetic.
- **Stability:** behavior of the **algorithm**. Stable algorithms give the exact answer to a slightly perturbed problem (backward stability).

You cannot fix a wildly ill-conditioned problem by clever coding alone—but you can avoid unstable algorithms that make things worse.

Worked example 1

Solving $Ax = b$ for Hilbert matrix H_n is severely ill-conditioned as n grows. Even a stable solver returns large forward error.

Worked example 2

Computing $e^x - 1$ for tiny x via direct subtraction is unstable; `expm1` is stable. Same mathematical function, different algorithms.

Accuracy metrics

- Absolute error $|\hat{x} - x|$
- Relative error $|\hat{x} - x|/|x|$ (meaningful when $x \neq 0$)
- Residual for equations (e.g. $\|b - A\hat{x}\|$)—necessary but not sufficient for small forward error

Worked example 3

$\hat{x}$ with tiny residual for ill-conditioned A may still be far from true x. Always check condition numbers when residuals look "good enough."

Complexity and structure

Numerical linear algebra exploits structure:

- Sparse matrices (graphs, FEM, Jacobians)
- Symmetric positive definite (Cholesky, CG)
- Toeplitz / low-rank / hierarchical

Time complexities differ: dense GE $O(n^3)$, sparse methods can be near-linear in nonzeros for nice problems.

Worked example 4

Graph Laplacian systems in spectral clustering: large sparse SPD ⇒ conjugate gradient + preconditioning, not dense inverse.

Roadmap

1. Floating-point, error, stability
2. Root finding: bisection, Newton, secant
3. Linear systems: LU, iterative methods, conditioning
4. Numerical optimization practice

Related: Optimization (`11`), Linear Algebra Advanced (`18`), Calculus (`08`).

Tools (conceptual)

Libraries encode decades of numerical analysis: LAPACK, SuiteSparse, Eigen, SciPy, cuSOLVER. Prefer them over hand-rolled GE for production. Understanding when they fail (singular, ill-conditioned, wrong matrix type) is your job.

Worked example 5

`np.linalg.solve(A,b)` uses LU with pivoting—not $A^{-1}b$. Explicit inverses cost more and usually less accurate.

A first error budget

Suppose each floating operation introduces relative error $\sim u \approx 10^{-16}$. A long chain of n dependent ops can accumulate $O(nu)$ relative error in the worst naive bound—sometimes much worse with cancellation. Stable algorithms rearrange work so the **backward** error stays $O(u)$ even if the forward error is large because the problem is sensitive.

Worked example 6 — cancellation vs algebra

Evaluating $f(x) = \sqrt{x+1} - \sqrt{x}$ at $x = 10^{16}$ in double often returns 0, while $1/(\sqrt{x+1} + \sqrt{x})$ returns $\sim 5 \cdot 10^{-9}$. Same f, different algorithms.

Worked example 7 — residual vs truth

For $Ax = b$ with $\kappa(A) \sim 10^{12}$, a residual $\|b - A\hat{x}\|/\|b\| \sim 10^{-16}$ can still leave $\|\hat{x} - x\|/\|x\|$ order 10^{-4} or worse. Always pair residual checks with condition estimates.

What "convergent algorithm" means

Iterative methods produce $x^{(k)} \to x^\star$ in exact arithmetic under hypotheses (contraction, descent lemma, spectral radius < 1). In FP:

- Demand tolerances no tighter than problem conditioning allows
- Prefer residual-based stops for linear solves
- Watch stagnation from rounding

Worked example 8 — Newton for roots

Quadratic convergence of Newton is local and assumes $f'(x^\star) \neq 0$. Far from the root, or at multiple roots, behavior changes—hybridize with bisection.

Roadmap recap with deliverables

Chapter	You should be able to
FP & error	Rewrite unstable expressions; define u, κ
Roots	Choose bisection vs Newton; state rates
Linear systems	Use LU/Cholesky/CG appropriately; avoid inverses
Opt numerics	Line search/trust region; diagnose NaNs

Pitfalls

1. Testing equality of floats with `==`
2. Summing millions of values in poorly ordered reduction (use compensated summation / higher precision accumulators when needed)
3. Finite differences with h too small (cancellation) or too large (truncation)
4. Ignoring scaling/units before solving
5. Treating ML training divergence as "only LR" when numeric overflow/underflow is the cause

Checkpoint

- Explain conditioning vs stability with one example each
- Define absolute/relative error and residual
- Know why `solve` beats explicit inverse
- Name the chapter map of this part
- Spot catastrophic cancellation in a formula

Exercises

1. Convert 0.1 decimal intuition: why is it not exact in binary32/64?
2. Define machine epsilon u and relate to unit roundoff.
3. Give a well-conditioned problem solved unstably (example from next chapter OK).
4. Why is residual not enough for $Ax = b$?
5. Complexity: dense LU vs sparse for $n = 10^5$ with 10 nonzeros/row (qualitative).
6. When would you pick an iterative method over a direct method?
7. Explain overflow risk in $\prod_i e^{a_i}$ vs $\exp(\sum a_i)$.
8. What is a condition number for $f(x) = 1/x$ near 0?
9. List three production domains that depend on numerical linear algebra.
10. Checkpoint writing: rewrite an unstable expression you have seen (e.g. variance one-pass formula issues).

Summary

Numerical methods make continuous mathematics computable under finite precision and finite work. The discipline separates problem hardness (conditioning) from algorithm quality (stability) and gives reliable tools for roots, linear systems, and optimization—the computational backbone of scientific and ML systems.

Floating-Point Arithmetic, Error, and Stability

Floating-point is the lingua franca of scientific computing and ML. Understanding IEEE-754, rounding models, cancellation, and the distinction between **conditioning** and **stability** prevents an entire class of silent failures.

1. IEEE-754 style machine numbers

A binary floating-point number has the form

$$x = (-1)^s \cdot (1.m)_2 \cdot 2^e,$$

with finite mantissa bits and exponent range (subnormals and specials: $\pm 0, \pm\infty, \mathrm{NaN}$).

Double precision (binary64): 1 sign + 11 exponent + 52 explicit mantissa bits $\Rightarrow$ about 16 decimal digits, machine epsilon $\varepsilon_{\text{mach}} \approx 2^{-52} \approx 2.22 \times 10^{-16}$.

Single (binary32): $\varepsilon_{\text{mach}} \approx 1.19 \times 10^{-7}$.

Worked example 1

0.1 is not exactly representable in binary FP. Summing 0.1 ten times may not equal 1.0 exactly. Compare with integers or decimals only when you understand representation.

Worked example 2 — spacing

Between powers of two, floats are equally spaced in ulps. Near 2^{53}, double integers are not all representable—loops with large integer-valued doubles skip values.

2. Rounding model

Let $\mathrm{fl}(x)$ be the floating representation of real x (within range). Standard model for elementary ops:

$$\mathrm{fl}(x \oplus y) = (x \oplus y)(1 + \delta), \qquad |\delta| \le u,$$

where u is unit roundoff ($\approx \frac{1}{2}\varepsilon_{\text{mach}}$ depending on rounding mode; analysis often uses $u = \varepsilon_{\text{mach}}$ order-of-magnitude).

Operations $\oplus \in \{+, -, \times, /\}$. This model enables backward error analysis.

3. Error types

Error	Definition
Absolute	$\lvert\hat{x} - x\rvert$
Relative	$\lvert\hat{x} - x\rvert / \lvert x\rvert$
Forward	error in the computed output
Backward	smallest perturbation of inputs making $\hat{x}$ exact for the perturbed problem

Backward stable algorithm: computed solution is exact for a nearby problem (perturbation $O(u)$ relative, problem-dependent).

Worked example 3

LU with partial pivoting is typically backward stable for solving $Ax = b$: $\hat{x}$ solves $(A + \Delta A)\hat{x} = b$ with $\|\Delta A\|$ small relative to $\|A\|$ (with caveats).

4. Catastrophic cancellation

Subtracting nearly equal quantities cancels leading significand bits; relative error explodes even if each operand is accurate.

Worked example 4 — classic unstable formula

$f(x) = \sqrt{x+1} - \sqrt{x}$ for large x. Direct evaluation loses digits. Rationalize:

$$f(x) = \frac{1}{\sqrt{x+1} + \sqrt{x}}.$$

Worked example 5 — variance

One-pass $\sum x_i^2 - n\bar{x}^2$ can cancel. Use Welford's method or two-pass with compensated arithmetic.

Worked example 6 — `log1p` / `expm1`

$\log(1 + x)$ for small x: use specialized functions. Direct formula suffers cancellation in $1 + x$.

5. Conditioning of problems

For $f : \mathbb{R} \to \mathbb{R}$ differentiable, relative condition number approximately

$$\kappa_f(x) \approx \left| \frac{x f'(x)}{f(x)} \right|.$$

Large κ means **inherent** sensitivity.

Worked example 7

$f(x) = \sqrt{x}$ near 0^+: absolute condition moderate in some senses; $f(x) = 1/(x - a)$ near pole: extremely ill-conditioned.

Worked example 8 — linear systems

$\kappa(A) = \|A\|\|A^{-1}\|$. Relative input error may amplify by $\sim \kappa(A)$ in the solution.

6. Stability of algorithms

A stable algorithm does not introduce much more error than conditioning forces. Unstable algorithms can destroy even well-conditioned problems.

Worked example 9 — solving triangular systems

Forward/back substitution is backward stable. Explicit inverse formation is not recommended.

Worked example 10 — Horner vs naive polynomial

Horner's rule evaluates polynomials with better stability structure than expanded power sums in many cases.

7. Accumulation and summation

Summing n numbers: naive left-to-right can have error $O(nu \sum |x_i|)$. Pairwise or Kahan compensated summation reduces effective constant.

Worked example 11

Summing many tiny values into a large accumulator: large number "swallows" small addends (absorption). Sort or use higher precision accumulators.

8. Exceptions and special values

- Overflow $\to \pm\infty$
- Underflow $\to 0$ or subnormal
- $0/0$, $\infty - \infty \to$ NaN
- Comparisons with NaN are always false (including `!=` subtleties in languages)

ML training: watch mixed precision overflow; loss scaling exists for this reason.

9. Practical guidelines

1. Prefer mathematically equivalent stable rearrangements
2. Scale/normalize data and matrices
3. Use fused multiply-add (FMA) when available for $ax + b$ patterns
4. Avoid explicit inverses; use solvers
5. Monitor residuals **and** condition estimates
6. Don't use FP for currency / exact discrete counts

Worked example 12

Python sketch:

```
import math
x = 1e16
naive = math.sqrt(x + 1) - math.sqrt(x)
stable = 1.0 / (math.sqrt(x + 1) + math.sqrt(x))
# naive often 0.0; stable ~ 5e-9
```

10. Pitfalls

1. Assuming associativity: $(a + b) + c$ vs $a + (b + c)$
2. Using tiny finite-difference $h < \sqrt{\varepsilon}$ blindly
3. Interpreting all digits of a printout as correct

4. Ignoring that κ depends on norm choice (order of magnitude still matters)

5. "More iterations" of an unstable recurrence making error worse

11. Checkpoint

- Describe binary64 layout and $\varepsilon_{\text{mach}}$
- Write the standard rounding model
- Detect cancellation and rewrite one formula
- Define conditioning vs stability
- Explain backward error for $Ax = b$

Exercises

Easy

1. Show in code or by reasoning that `(1e16 + 1) - 1e16` can be 0 in double.
2. Estimate relative spacing between doubles near 1.0.
3. Define ulp.
4. Why is $x^2 - y^2$ often better as $(x-y)(x+y)$ for cancellation when $x \approx y$? (Also discuss when not.)
5. Give an example of absorption in summation.

Medium

6. Derive a stable formula for $\frac{1-\cos x}{x^2}$ near 0 (use trig identities).
7. Compute approximate κ for $f(x) = e^x$ (relative).
8. Explain why Gaussian elimination without pivoting can be unstable (growth factor).
9. Compare absolute vs relative error for approximating π by 3.14.
10. Finite difference $f'(x) \approx (f(x+h) - f(x))/h$: balance truncation $O(h)$ vs roundoff $O(u|f|/h)$; optimal h scale.

Challenge

11. Read Wilkinson's polynomial root sensitivity story; summarize conditioning of roots vs coefficient perturbation.
12. Prove error bound sketch for naive sum of n positives under the standard model.
13. Implement Kahan summation pseudocode; explain the compensation term.
14. Mixed precision: when does fp16 matmul + fp32 accumulate help throughput without destroying accuracy?

15. For $Ax = b$, relate forward error bound to $\kappa(A)$ and backward error (standard inequality).

Summary

Floating-point arithmetic is accurate **relative to a rounding unit**, not exact. Cancellation, absorption, and ill-conditioning create large forward errors; stable algorithms and careful algebra keep results honest. Mastering these ideas is prerequisite to trusting any numerical solver or ML kernel.

Root Finding: Bisection, Newton, and Secant

Root finding solves $f(x) = 0$ for scalar (or later, system) nonlinear equations. It appears in implicit time steppers, calibration, inverse kinematics, threshold equations, and as a subroutine inside optimization (e.g. line search).

1. Problem statement

Given $f : \mathbb{R} \to \mathbb{R}$, find $x^\star$ with $f(x^\star) = 0$. Multiple roots, no roots, or discontinuous f change the game—always state assumptions.

Worked example 1

$f(x) = x^3 - x - 2$: one real root near 1.52 (since $f(1) = -2$, $f(2) = 4$).

2. Bisection method

Assumptions. f continuous on $[a, b]$, $f(a)f(b) < 0$. By IVT, a root exists in (a, b).

Iteration. Set $m = (a + b)/2$. If $f(a)f(m) \le 0$, set $b \leftarrow m$; else $a \leftarrow m$.

Theorem (linear convergence). After k steps, interval width is $(b_0 - a_0)/2^k$. To achieve width $\le \varepsilon$ need

$$k \ge \log_2 \left((b_0 - a_0)/\varepsilon \right).$$

Proof. Each step halves the bracket; IVT keeps a root inside. ■

Worked example 2

$[1, 2]$, $\varepsilon = 10^{-3}$: need $k \ge \log_2(1000) \approx 10$ iterations.

Pros: robust, minimal assumptions. **Cons:** slow; needs a bracket; not easily generalized to high dimension.

3. Newton's method

Iteration.

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)},$$

assuming $f'(x_k) \neq 0$.

Derivation. Linearize $f(x_k) + f'(x_k)(x - x_k) = 0$.

Local quadratic convergence

Theorem (informal). If f is C^2, $f(x^\star) = 0$, $f'(x^\star) \neq 0$, and x_0 is close enough, then Newton converges and

$$|x_{k+1} - x^\star| \leq C|x_k - x^\star|^2$$

for some C.

Proof idea. Taylor expand $f(x^\star)$ about x_k and rearrange the Newton update; the leading error term is quadratic. ■

Worked example 3

$f(x) = x^3 - x - 2$, $x_0 = 1.5$:

$f'(x) = 3x^2 - 1$, $x_1 = 1.5 - f(1.5)/f'(1.5)$ rapidly approaches $\approx 1.521379\ldots$

Worked example 4 — failure modes

- $f'(x_k) \approx 0$: division blow-up
- Poor start on non-monotone f: divergence or cycle
- Multiple root $f(x^\star) = f'(x^\star) = 0$: convergence drops to linear (need multiplicity-aware variants)

Worked example 5 — square root

Solve $x^2 - a = 0 \Rightarrow$ Heron iteration $x \leftarrow \frac{1}{2}(x + a/x)$, a Newton method—classic stable algorithm for $\sqrt{a}$.

4. Secant method

Derivative-free:

$$x_{k+1} = x_k - f(x_k)\frac{x_k - x_{k-1}}{f(x_k) - f(x_{k-1})}.$$

Approximates f' by a finite difference using last two iterates.

Convergence: superlinear (order ≈ 1.618 golden ratio) under standard smoothness and simple root assumptions—faster than bisection, typically slower than Newton per iteration but no f'.

Worked example 6

Needs two starts x_0, x_1. If $f(x_k) \approx f(x_{k-1})$, formula unstable (cancellation / division by near-zero).

5. Hybrid strategies (practical)

Brent's method combines bisection, secant, and inverse quadratic interpolation: guaranteed progress like bisection with faster typical speed. Production scalar solvers often use Brent-like algorithms.

Strategy pattern:

1. Bracket root (sampling, sign changes)
2. Run safeguarded Newton (if f' available): if Newton step leaves bracket or f' tiny, fall back to bisection
3. Stop on $|f(x)|$, $|x_{k+1} - x_k|$, and iteration cap

Worked example 7 — finance implied vol

Solve Black–Scholes price equation for σ: monotone in σ on ranges $\Rightarrow$ robust bracket + Newton/Brent.

6. Systems of nonlinear equations

Solve $F(x) = 0$ with $F : \mathbb{R}^n \to \mathbb{R}^n$.

Newton: $x_{k+1} = x_k - J_F(x_k)^{-1}F(x_k)$, i.e. solve linear system $J_F s = -F$.

Cost: Jacobian + linear solve each step. Inexact Newton / quasi-Newton (Broyden) reduce cost. Damping / trust region globalization.

Worked example 8

Two equations equilibrium models: Newton with analytic Jacobian vs finite-difference Jacobian (noise and cost tradeoffs).

7. Connections to optimization

Critical points: solve $\nabla f(x) = 0$—Newton for optimization uses Hessian, related but minimizes f not only finds stationary points of a map. Line search and trust regions again globalize.

Worked example 9

1D: minimize g by Newton on $g' = 0$ is Newton on $f = g'$. Same quadratic convergence story when $g''(x^\star) > 0$.

8. Stopping criteria

Use combined tests:

- $|f(x_k)| < \tau_f$
- $|x_{k+1} - x_k| < \tau_x(1 + |x_k|)$ relative
- $k > k_{\max}$ failure

For noisy f (simulators), tolerances must match noise floors.

Worked example 10

Demanding $|f| < \tau$ alone may stop far from root if f is flat; demanding step size alone may stop on a plateau of nonzero f. Prefer both.

9. Pitfalls

1. No bracket $\Rightarrow$ bisection inapplicable
2. Newton without derivative checks
3. Multiple roots slowing Newton
4. Complex roots if only real arithmetic

5. Treating FP "root" as exact for ill-conditioned f (Wilkinson)

6. Finite-difference Jacobians with bad h

10. Checkpoint

- State bisection guarantees and iteration count
- Derive Newton from linearization
- Know quadratic vs linear vs superlinear rates
- Safeguard Newton with brackets
- Extend conceptually to $F(x) = 0$ with Jacobians

Exercises

Easy

1. Prove bisection error bound after k steps.
2. Perform 3 bisection steps on $f(x) = x^3 - x - 2$ in $[1, 2]$.
3. Perform 2 Newton steps from $x_0 = 1.5$ for the same f.
4. When is secant preferred over Newton?
5. Explain why $f(a)f(b) < 0$ is sufficient but not necessary for a root in $[a, b]$ if discontinuities allowed.

Medium

6. Show Newton for $x^2 - a$ yields Heron's method.
7. Analyze multiplicity: for $f(x) = x^2$, Newton iteration and rate from $x_0 \neq 0$.
8. Implement (pseudocode) safeguarded Newton with bisection fallback.
9. Give a smooth f and x_0 where Newton diverges.
10. Estimate iterations: bisection vs Newton to get 10^{-12} accuracy starting well.

Challenge

11. Prove local quadratic convergence of Newton for simple roots (Taylor expansion proof).
12. Broyden's method: state update formula and motivation (rank-one Jacobian approx).
13. Conditioning of a root: if $f(x^\star + \delta) \approx f'(x^\star)\delta$, how does residual map to root error?
14. Multivariate Newton: write algorithm; discuss when J_F is singular.
15. Application: formulate inverse kinematics for a 1-joint toy as root finding; discuss brackets.

Summary

Bisection is the reliable workhorse; Newton is the fast local engine; secant and hybrids balance derivative cost and guarantees. Understanding rates, failure modes, and stopping criteria makes scalar and small-system solvers trustworthy components in larger numerical pipelines.

Numerical Linear Systems

Solving $Ax = b$ is the kernel of simulation, least squares, Gaussian processes, spline fitting, interior-point methods, and Newton steps for nonlinear systems. This chapter covers direct factorizations, iterative methods, conditioning, and validation—without forming A^{-1}.

1. The problem

Given $A \in \mathbb{R}^{n\times n}$ nonsingular and $b \in \mathbb{R}^n$, find x with $Ax = b$.

Variants: rectangular least squares $A \in \mathbb{R}^{m\times n}$, singular/ill-posed systems (regularize), multiple right-hand sides, matrix equations.

Worked example 1

2D: $A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$, $b = \begin{bmatrix} 3 \\ 3 \end{bmatrix} \Rightarrow x = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$.

2. Direct methods: Gaussian elimination and LU

LU factorization: $A = LU$ (or $PA = LU$ with permutation P from pivoting), L unit lower triangular, U upper triangular.

Solve: $Ly = Pb$ then $Ux = y$ by substitution—$O(n^2)$ after $O(n^3)$ factorization.

Partial pivoting: swap rows to maximize pivot magnitude—controls element growth, improves stability.

Worked example 2 — why pivot

$A = \begin{bmatrix} \varepsilon & 1 \\ 1 & 1 \end{bmatrix}$ with tiny ε: without pivoting, elimination is unstable in FP; with pivoting, swap rows first.

Cholesky

If A SPD ($A = A^\top \succ 0$), $A = LL^\top$ (or $R^\top R$) with half the work and better stability constants. **Always prefer Cholesky when SPD is known.**

Worked example 3

Gram matrices $X^\top X + \lambda I$ for $\lambda > 0$ are SPD—use Cholesky for ridge solves.

3. Complexity and structure

Method	Leading cost	Notes
Dense LU/Cholesky	$\frac{1}{3}n^3$ / $\frac{1}{6}n^3$	BLAS3 heavy
Banded	depends on bandwidth	ODE/PDE discretizations
Sparse direct	fill-in dependent	Ordering (AMD, nested dissection) critical
Iterative	per-iter matvec	Need preconditioners

Worked example 4

Tridiagonal systems: Thomas algorithm $O(n)$—never use dense LU.

4. Residual, forward error, condition number

Residual $r = b - A\hat{x}$.

Rule of thumb: $\frac{\|\hat{x}-x\|}{\|x\|} \lesssim \kappa(A)\frac{\|r\|}{\|A\|\|x\|}$ (norm-dependent constants).

$$\kappa(A) = \|A\|\|A^{-1}\|.$$

For 2-norm, $\kappa_2(A) = \sigma_{\max}(A)/\sigma_{\min}(A)$.

Worked example 5 — Hilbert

Hilbert matrix $H_{ij} = 1/(i + j - 1)$ has κ growing exponentially with n. Even tiny residual can coexist with large forward error.

Worked example 6 — scaling

Row/column equilibration can change practical accuracy; ill-scaling is a common modeling bug.

5. Never form the inverse

Computing A^{-1} then $x = A^{-1}b$:

- More FLOPs and memory
- Typically worse accuracy
- Destroys sparsity

Use `solve`, factorizations, or iterative methods.

Worked example 7

```
# good
x = Solve(A, b)           # LU/Cholesky inside
# bad
x = Inverse(A) * b
```

6. Iterative methods overview

Generate $x^{(k)} \to x$ using matvecs $v \mapsto Av$.

Jacobi

$x_i^{(k+1)} = \frac{1}{a_{ii}}(b_i - \sum_{j \neq i} a_{ij} x_j^{(k)})$. Needs nonzero diagonals; converges if diagonally dominant (sufficient conditions).

Gauss–Seidel

Use newest values within the sweep—often faster than Jacobi.

Conjugate gradient (CG)

For SPD A: optimal Krylov method minimizing A-norm error over Krylov subspace. Memory: few vectors. Convergence depends on eigenvalue distribution / κ.

Preconditioning: solve $M^{-1}Ax = M^{-1}b$ with $M \approx A$ SPD easier to invert (incomplete Cholesky, multigrid, …).

Worked example 8

Poisson equation finite differences: classic CG + multigrid showcase—sparse huge n.

Worked example 9 — GMRES

For nonsymmetric A, GMRES/MINRES/BiCGSTAB families. Restarted GMRES trades optimality for memory.

7. Stopping criteria for iterative solvers

Common: $\|r^{(k)}\|/\|b\| < \tau$. Also monitor stagnation, max iters, and (when possible) cheap error estimates. Too tight τ wastes time beyond FP meaningful digits given κ.

Worked example 10

If $\kappa \sim 10^{10}$ and $u \sim 10^{-16}$, expecting 14 correct digits is fantasy—stop earlier.

8. Least squares connection

Overdetermined $Ax \approx b$: solve normal equations $A^\top A\hat{x} = A^\top b$ **or better** use QR / SVD for stability.

$\kappa(A^\top A) = \kappa(A)^2$—normal equations square the condition number. Prefer QR for moderately ill-conditioned least squares.

Worked example 11

Polynomial fitting high degree: Vandermonde ill-conditioned; use orthogonal polynomials or regularization.

9. CS/ML applications

- Newton steps: $H\Delta w = -\nabla J$
- Gaussian process / kernel methods: $(K + \sigma^2 I)v = y$
- Graph Laplacians: semi-supervised learning, embeddings
- Bundle adjustment / SLAM: sparse Schur complements
- Recommendation / ALS: alternating least squares solves

10. Pitfalls

1. Explicit inverses
2. Using CG on nonsymmetric or indefinite A without variants
3. Ignoring fill-in ordering in sparse direct
4. Trusting residual alone
5. Integer overflow in index arrays for huge sparse graphs
6. Mixed precision without iterative refinement when needed

11. Checkpoint

- Factor $PA = LU$ and solve by substitution
- Prefer Cholesky for SPD
- Define κ and interpret Hilbert pathology
- Choose iterative vs direct at a high level
- Prefer QR over naive normal equations for least squares

Exercises

Easy

1. Solve a 2×2 system by hand with GE and with Cramer; compare effort.
2. Why is pivoting unnecessary for many SPD Cholesky cases?
3. Compute κ_2 of $\mathrm{diag}(1, 10^{-8})$.
4. Cost of solving 100 RHS after one LU vs 100 independent GEs.
5. Write Jacobi update for a 3×3 diagonally dominant example.

Medium

6. Show that if A is strictly diagonally dominant, Jacobi converges (sketch/reference theorem).
7. Prove $\kappa_2(A) = \sigma_{\max}/\sigma_{\min}$.
8. Explain iterative refinement: solve, compute residual in higher precision, correct.
9. Compare flop counts: form A^{-1} vs LU solve for one b.
10. For ridge $X^\top X + \lambda I$, argue SPD and recommend a solver.

Challenge

11. Sparse Cholesky fill-in: give a matrix ordering bad vs good (path graph vs nested dissection intuition).
12. Derive CG at high level from Krylov optimality (A-norm).
13. SVD solution of least squares and relation to pseudoinverse.
14. Conditioning of Vandermonde systems—why orthogonal bases help.
15. Design a preconditioner intuition for a graph Laplacian (degree diagonal / incomplete factorization).

Summary

Linear solves are factorization plus substitution or Krylov iteration—not inversion. Stability, conditioning, and structure (SPD, sparse, banded) dictate the method. Validate with residuals **and** condition awareness, especially in ML and simulation pipelines.

Numerical Optimization in Practice

This chapter bridges smooth optimization theory and finite-precision computation: step-size policies, Newton and quasi-Newton, constraints in practice, and diagnostics when training or fitting diverges.

1. Problem setup

Minimize smooth $f : \mathbb{R}^n \to \mathbb{R}$, optionally with constraints. Assume gradients (and sometimes Hessians) are available via AD or analytic forms.

Local model: near x_k,

$$f(x_k + s) \approx f(x_k) + \nabla f(x_k)^\top s + \tfrac{1}{2} s^\top B_k s.$$

First-order methods take $B_k \sim LI$; Newton takes $B_k = \nabla^2 f(x_k)$.

2. Gradient descent and step sizes

$$x_{k+1} = x_k - \alpha_k \nabla f(x_k).$$

Fixed step: for L-smooth convex f, $\alpha \in (0, 2/L)$ ensures descent; $\alpha = 1/L$ standard.

Backtracking line search (Armijo): try $\alpha, \rho\alpha, \rho^2\alpha, \ldots$ until

$$f(x_k + \alpha d) \le f(x_k) + c\alpha \nabla f(x_k)^\top d$$

with $d = -\nabla f$, $c \in (0, 1)$, $\rho \in (0, 1)$.

Worked example 1

$f(x) = \frac{1}{2} x^\top Q x$ with known $L = \lambda_{\max}(Q)$: optimal fixed step related to $\kappa = \lambda_{\max}/\lambda_{\min}$.

Worked example 2 — ML

Learning rate schedules (cosine, step decay) are practical step-size policies for nonconvex stochastic objectives.

3. Momentum and adaptive methods (numeric view)

Momentum / Nesterov accumulate velocity—helps ill-conditioned ravines. AdaGrad/Adam rescale coordinates by gradient history—help diagonal scaling mismatches.

Numeric caution: Adam's ε prevents divide-by-zero; mixed precision needs loss scaling; exploding gradients need clipping.

Worked example 3

Feature scales 1 vs 10^6 without normalization: first-order methods crawl—conditioning of Hessian along axes differs wildly.

4. Newton and quasi-Newton

Newton:

$$x_{k+1} = x_k - \nabla^2 f(x_k)^{-1} \nabla f(x_k).$$

Quadratic convergence near nondegenerate local minima for smooth strongly convex f. Cost: Hessian form + linear solve.

BFGS / L-BFGS: build $B_k \approx \nabla^2 f$ from gradient pairs (s_k, y_k) with secant equation $B_{k+1} s_k = y_k$. L-BFGS stores last m pairs—workhorse for medium-scale deterministic problems.

Worked example 4

Logistic regression with moderate n, d: L-BFGS often beats SGD wall-clock when full batch fits in memory.

Worked example 5 — Hessian indefinite

In nonconvex regions, pure Newton may propose ascent directions. Modified Cholesky / trust-region Newton damp indefinite Hessians.

5. Trust regions vs line search

Trust region: choose s minimizing model m subject to $\|s\| \le \Delta_k$. Expand/contract Δ_k based on ratio of actual to predicted reduction.

More robust when models are poor; common in nonlinear least squares (Levenberg–Marquardt is a related damping idea).

Worked example 6 — Levenberg–Marquardt

Nonlinear least squares $f(\theta) = \frac{1}{2}\|r(\theta)\|_2^2$: solve $(J^\top J + \mu I)\delta = -J^\top r$. μ interpolates gradient descent and Gauss–Newton.

6. Stochastic objectives

$f(w) = \mathbb{E}_\xi[f(w;\xi)]$ approximated by minibatches. Noise means:

- Do not expect monotone decrease
- Use learning rate decay / averaging (Polyak–Ruppert)
- Early stopping on validation

Worked example 7

Batch size vs noise: larger batches reduce gradient variance, change optimal LR and generalization dynamics.

7. Constraints in numerical practice

Technique	Use when
Projection	Simple sets (box, simplex, spectrahedron specials)
Barrier / IPM	Convex with inequality structure, moderate n
Augmented Lagrangian	General equalities/inequalities
Penalty	Quick prototype; ill-conditioning as $\rho \uparrow$
Riemannian	Manifolds (sphere, Stiefel, SPD)

Worked example 8

Probability simplex constraints on mixture weights: projected gradient or exponentiated gradient / multiplicative updates.

Worked example 9 — SVM dual

Box constraints $0 \le \alpha_i \le C$ + linear equality: SMO coordinate methods exploit structure.

8. Diagnostics and failure modes

Track:

- $f(x_k)$, $\|\nabla f(x_k)\|$, step $\|x_{k+1} - x_k\|$
- Constraint violation
- Gradient/parameter norms (detect explosion)
- Condition estimates of Hessian or Gram mini-batches

Symptom	Likely cause
$f \to \infty$ / NaN	LR too large, overflow, bad init
Oscillation	LR large, missing momentum tuning
Stagnation high loss	LR tiny, saddle, under-model
Train val	Overfit; need reg/early stop
Residual small, params huge	Ill-conditioning / need reg

Worked example 10

Logistic regression with separable data: $\|w\| \to \infty$ without regularization—numeric overflow in exp. Add $\lambda\|w\|_2^2$.

9. Worked template: ℓ_2-regularized logistic

$$\min_w \sum_{i=1}^{n} \log\left(1 + \exp(-y_i w^\top x_i)\right) + \frac{\lambda}{2}\|w\|_2^2.$$

- Convex, smooth, strongly convex for $\lambda > 0$
- Gradient: $\sum_i \sigma(-y_i w^\top x_i)(-y_i x_i) + \lambda w$
- Methods: L-BFGS, Newton-CG, or SGD for huge n
- Monitor val log-loss and calibration

Worked example 11

Compare full-batch L-BFGS vs SGD on same objective: same f, different numeric paths and early-stopping behavior.

10. Automatic differentiation note

AD computes exact gradients of the implemented function (up to FP). Bugs in the **math of the implementation** (not AD) still yield "correct gradients of the wrong objective." Check gradients with finite differences on small examples when debugging.

Worked example 12

Softmax cross-entropy fused implementation avoids overflow via $\log \sum \exp$ trick—numeric method as modeling detail.

11. Pitfalls

1. One LR for unscaled features
2. Newton without globalization far from optimum
3. Interpreting SGD noise as bug
4. Over-tight gradient tolerances beyond FP/ limits
5. Forgetting regularization on separable classification
6. Trusting training loss alone under distribution shift

12. Checkpoint

- Implement Armijo backtracking conceptually
- Contrast GD, Newton, L-BFGS costs
- Explain trust region ratio
- Diagnose NaNs and exploding weights
- Write a robust training loop checklist

Exercises

Easy

1. For $f(x) = \frac{1}{2}Lx^2$, what fixed steps α guarantee descent?
2. Write Armijo condition with symbols defined.

3. Why does Adam include ε?
4. Name one reason to prefer L-BFGS over SGD for small data.
5. What does gradient clipping change in the algorithm mathematically (approx)?

Medium

6. Derive Newton step for $f(x) = \frac{1}{2}x^\top Qx - b^\top x$ with $Q \succ 0$—show one-step convergence from any start if exact arithmetic.
7. Explain why $J^\top J$ in Gauss–Newton is PSD.
8. Design stopping rules for noisy SGD (use val plateau, not only $\|\hat{g}\|$).
9. Projected GD onto $\|x\|_2 \leq 1$: write the projection formula.
10. Show that adding $\frac{\lambda}{2}\|w\|_2^2$ makes logistic objective strongly convex.

Challenge

11. Pseudocode BFGS update; state curvature condition $s^\top y > 0$ and damping if violated.
12. Trust-region subproblem: describe Cauchy point approximation.
13. Mixed-precision training: outline loss-scaling algorithm.
14. Compare barrier methods vs projected gradient for box constraints (pros/cons).
15. End-to-end: specify optimizer, line search/schedule, diagnostics, and success criteria for a sparse logistic problem with $n = 10^7$, $d = 10^5$.

Summary

Numerical optimization combines models (linear/quadratic), step policies (line search/trust region), and solvers (first-order, Newton, quasi-Newton) under floating-point reality. Good practice is relentless diagnostics, scaling, regularization, and matching method to problem structure and noise.

Discrete Fourier Transforms and the FFT

The discrete Fourier transform (DFT) turns a length-n sequence into n complex frequency bins. A naive implementation costs $O(n^2)$ arithmetic; the **fast Fourier transform (FFT)** family evaluates the same mathematical transform in $O(n \log n)$ time. That gap is why spectral methods, convolution via FFT, and large-scale signal processing are practical on ordinary hardware.

This chapter defines the DFT, explains the Cooley–Tukey radix-2 idea, states the convolution theorem, and catalogs the practical pitfalls that dominate real use: windowing, spectral leakage, real-vs-complex transforms, and normalization conventions.

1. The DFT

Let $a_0, \dots, a_{n-1} \in \mathbb{C}$. The **forward DFT** (NumPy / Oppenheim–Schafer "engineering" sign convention) is

$$A_k = \sum_{m=0}^{n-1} a_m \, e^{-2\pi i \, mk/n}, \qquad k = 0, \dots, n-1.$$

The **inverse DFT** recovers the samples:

$$a_m = \frac{1}{n} \sum_{k=0}^{n-1} A_k \, e^{2\pi i \, mk/n}, \qquad m = 0, \dots, n-1.$$

Mental model: each A_k is the inner product of the signal against the complex sinusoid of discrete frequency k/n cycles per sample. Index $k = 0$ is DC (sum of the signal). For real input, $A_{n-k} = \overline{A_k}$ (Hermitian symmetry).

Worked example 1 — DC and Nyquist

Take $n = 4$, $a = (1, 1, 1, 1)$. Then $A = (4, 0, 0, 0)$: all energy at DC. For $a = (1, -1, 1, -1)$, energy sits at the Nyquist bin $k = 2$ when n is even.

Worked example 2 — pure tone

Sampling $a_m = \cos(2\pi f_0 m \Delta t)$ with $f_0 = k_0/(n\Delta t)$ for integer k_0 puts energy in bins k_0 and $n - k_0$ only. If f_0 is **not** an integer number of cycles over the window, energy **leaks** into neighboring bins (Section 6).

2. Matrix view and complexity

Write $\omega = e^{-2\pi i/n}$. Then $A = Fa$ with the Fourier matrix

$$F_{km} = \omega^{km}.$$

Dense matrix–vector multiply is $\Theta(n^2)$ complex multiplies/adds. The FFT reuses shared twiddle factors and butterfly structure so the **same** Fa costs $\Theta(n \log n)$.

Method	Leading cost	Notes
Direct DFT sum	$O(n^2)$	Fine for tiny n; pedagogical
Radix-2 Cooley–Tukey	$O(n \log n)$	Best when $n = 2^p$
Mixed-radix / Bluestein	$O(n \log n)$	Arbitrary n (library default)
Real FFT (`rfft`)	$\sim$ half complex work	Exploits Hermitian symmetry

Worked example 3 — when the constant matters

For $n = 2^{20} \approx 10^6$, $n^2 \sim 10^{12}$ ops is hours of CPU; $n \log_2 n \sim 2 \cdot 10^7$ is milliseconds. Always prefer a library FFT for $n \gtrsim 64$.

3. Cooley–Tukey radix-2 intuition

Assume $n = 2p$. Split even and odd indices:

$$\begin{aligned} A_k &= \sum_{j=0}^{p-1} a_{2j}\, \omega^{2jk} + \omega^k \sum_{j=0}^{p-1} a_{2j+1}\, \omega^{2jk} \\ &= E_k + \omega^k O_k, \end{aligned}$$

where E and O are length-p DFTs of the even/odd subsequences, and ω^k is a **twiddle**. Because of periodicity, $E_{k+p} = E_k$ and $O_{k+p} = O_k$, so one pair of half-size DFTs plus n twiddle/combine steps yields the full transform.

Recurrence: $T(n) = 2T(n/2) + \Theta(n) \Rightarrow T(n) = \Theta(n \log n)$.

```
butterfly (radix-2):
  even DFT    E_k
                        A_k      = E_k + W^k O_k
  odd  DFT    O_k      A_{k+p} = E_k - W^k O_k
                  W^k = exp(-2  i k / n)
```

Worked example 4 — $n = 8$

Three stages of butterflies: bit-reversal permutation of inputs (or outputs, depending on formulation), then combine length-2, length-4, length-8 transforms. Counting: $3 \cdot 8 = 24$ butterflies, each $O(1)$ arithmetic $\Rightarrow$ matches $n \log_2 n$.

Libraries: production code (NumPy's PocketFFT, FFTW, MKL) uses mixed radices (2,3,4,5,…), not only pure radix-2. The Cooley–Tukey story still explains *why* $O(n \log n)$ is achievable.

4. Convolution theorem

For circular (periodic) convolution of length-n sequences a and b,

$$(a * b)_m = \sum_{j=0}^{n-1} a_j \, b_{(m-j) \bmod n},$$

the DFT turns convolution into pointwise products:

$$\widehat{a * b} = A \odot B \qquad \text{(elementwise)}.$$

Algorithm:

1. Zero-pad a and b to length $N \geq n_a + n_b - 1$ (linear convolution) or use $N = n$ for circular.
2. $A = \mathrm{FFT}(a)$, $B = \mathrm{FFT}(b)$.
3. $C_k = A_k B_k$.
4. $c = \mathrm{IFFT}(C)$.

Cost $O(N \log N)$ vs $O(n_a n_b)$ direct. Break-even is often modest (N hundreds), so FFT convolution dominates large filters and polynomial multiplication.

Worked example 5 — polynomial multiply

Coefficients of $p(x)q(x)$ are the linear convolution of coefficient vectors—exactly the setting for FFT multiplication of large integers / polynomials (with care for rounding if using floating FFT).

API idea	Use when	Output size

5. Real vs complex; normalization

API idea	Use when	Output size
Complex `fft` / `ifft`	General complex signals	n complex
Real `rfft` / `irfft`	Real time series	$n/2+1$ complex (positive freqs)
Hermitian `hfft`	Real spectrum / Hermitian time	complementary packing

Normalization conventions (NumPy `norm=`):

Mode	Forward	Inverse
`"backward"` (default)	unscaled	$/n$
`"ortho"`	$1/\sqrt{n}$	$1/\sqrt{n}$ (unitary)
`"forward"`	$/n$	unscaled

Parseval / energy identities need a consistent convention. Prefer `"ortho"` when comparing "energy in time = energy in frequency."

Worked example 6 — Hermitian check

```
# file: hermitian_check.py
import numpy as np

rng = np.random.default_rng(0)
x = rng.standard_normal(128)
X = np.fft.fft(x)
# X[k] should  conj(X[-k]) for real x
err = np.max(np.abs(X[1:] - np.conj(X[:0:-1])))
print(err)  # ~1e-15
```

6. Windowing, leakage, and sampling traps

The DFT implicitly assumes the length-n buffer is **one period** of a periodic signal. Abrupt truncation ⇒ discontinuities at the wrap ⇒ **spectral leakage** (sinc-like sidelobes).

Mitigations:

- Choose n so tones complete an integer number of cycles (rare in the wild).

- Apply a **window** (Hann, Hamming, Blackman, Kaiser) before the FFT; trade main-lobe width vs sidelobe height.
- Zero-pad *after* windowing for denser frequency grid (interpolation of the DTFT of the windowed segment—not new information).

Sampling: to resolve frequency f, need sample rate $f_s > 2f$ (Nyquist). Aliasing folds energy above $f_s/2$ into lower bins—anti-alias filter before downsample.

Worked example 7 — leakage demo (conceptual)

A pure cosine that is **not** periodic in the buffer shows a broad hump instead of two spikes. Hann windowing narrows the mess at the cost of a slightly wider main lobe.

7. Frequencies and units

For sample spacing Δt (seconds),

$$f_k = \frac{k}{n\Delta t}, \qquad k = 0, \ldots, n-1$$

(with negative frequencies represented in the upper half of the array). Helpers:

- `fftfreq(n, d=Δt)` — bin centers for `fft`
- `rfftfreq` — for `rfft`
- `fftshift` — rotate so DC is centered for plots

Worked example 8 — reading a spectrum

$n = 1000$, $\Delta t = 0.001$ s $\Rightarrow f_s = 1000$ Hz, frequency resolution $\Delta f = 1$ Hz. A peak at bin $k = 50$ is 50 Hz.

8. Runnable mini-FFT (radix-2, educational)

```
# file: tiny_radix2_fft.py
# Educational radix-2 FFT (not for production). Prefer numpy.fft.
import cmath
import math


def fft_radix2(a):
    """In-place-ish Cooley-Tukey; returns new list. n must be power of 2."""
```

```
    n = len(a)
    if n == 1:
        return [a[0]]
    if n & (n - 1):
        raise ValueError("length must be a power of 2")
    even = fft_radix2(a[0::2])
    odd = fft_radix2(a[1::2])
    out = [0j] * n
    for k in range(n // 2):
        w = cmath.exp(-2j * math.pi * k / n)
        t = w * odd[k]
        out[k] = even[k] + t
        out[k + n // 2] = even[k] - t
    return out


def dft_direct(a):
    n = len(a)
    return [
        sum(a[m] * cmath.exp(-2j * math.pi * m * k / n) for m in range(n))
        for k in range(n)
    ]


if __name__ == "__main__":
    x = [1, 2, 3, 4, 3, 2, 1, 0]
    F = fft_radix2(x)
    D = dft_direct(x)
    err = max(abs(F[k] - D[k]) for k in range(len(x)))
    print("max |FFT - DFT| =", err)
    # expected: ~1e-15
```

Complexity of this recursive form is still $O(n \log n)$, but constant factors and cache behavior are far worse than library FFTs.

9. Stability and floating-point notes

- Unitary (`ortho`) FFTs have excellent backward stability in practice; roundoff grows like $O(\varepsilon \sqrt{\log n})$ under mild models (see Numerical Recipes / Higham-style analyses).
- Large forward transforms of huge signals: prefer double for analysis unless memory forces `float32` (and know libraries may promote).
- FFT convolution of integers via floats needs enough mantissa bits or modular/NTT alternatives for exact arithmetic.

Worked example 9 — residual check

Round-trip `ifft(fft(x))` should match x to $\sim 10^{-15}$ relative for double and moderate n. Larger residuals often mean wrong `norm`, wrong `irfft` length, or non-Hermitian edits to a real spectrum.

10. Algorithms beyond radix-2 (map)

Family	Idea
Mixed-radix CT	Factor $n = n_1 n_2$; recurse
Stockham / self-sorting	Avoid explicit bit-reversal
Bluestein / chirp-z	Arbitrary n via convolution
Prime-factor (Good–Thomas)	Coprime factors, no twiddles between
Real-packed / split-radix	Fewer real ops for real data

You rarely implement these; you **choose** library plans (FFTW wisdom, cuFFT, PocketFFT) that pick a factorization for your n.

11. CS applications (numerical methods view)

- Fast Poisson / Helmholtz on regular grids (spectral / FFT Poisson solvers)
- Convolutional filtering and matched filters
- Polynomial and large-integer multiplication
- Time–frequency features (STFT = windowed consecutive FFTs)
- Circulant preconditioners and Toeplitz embedding

Related numerical themes: conditioning of Vandermonde-like Fourier modes is excellent for pure harmonics; leakage and window choice dominate *interpretation* error more often than roundoff.

12. Pitfalls

1. Treating DFT bins as continuous frequencies without stating $\Delta f = 1/(n\Delta t)$.
2. Circular convolution when you meant linear (forgot padding).
3. Editing only half a spectrum then `ifft` without restoring Hermitian symmetry $\Rightarrow$ complex time signal.

4. Comparing amplitudes across different `norm` modes or windows without calibration.

5. Expecting $O(n \log n)$ magic from a hand-rolled $O(n^2)$ nested loop labeled "FFT."

6. Interpreting zero-padding as higher frequency *resolution* of new information (it interpolates the spectrum of the *same* windowed segment).

13. Checkpoint

- Write the forward/inverse DFT pair and explain A_0.
- Derive the even/odd split that yields $T(n) = 2T(n/2) + O(n)$.
- State the convolution theorem and when to zero-pad.
- Explain leakage and one windowing remedy.
- Choose `fft` vs `rfft` and a `norm` mode for an energy-preserving pipeline.

Exercises

Easy

1. Compute the DFT of $(1, 0, -1, 0)$ by hand ($n = 4$).

2. Why is A_0 real when a is real?

3. Cost ratio $n^2/(n \log_2 n)$ for $n = 1024$.

4. What does `fftshift` change—values or order?

5. Give one reason to prefer `rfft` on real audio.

Medium

6. Prove Hermitian symmetry $A_{n-k} = \overline{A_k}$ for real a.

7. Show that circular convolution $\leftrightarrow$ pointwise DFT products.

8. Explain why a Hann window reduces leakage (qualitative Fourier picture).

9. Design padding length for linear convolution of lengths n and m.

10. Compare `"backward"` vs `"ortho"` for Parseval.

Challenge

11. Implement iterative (non-recursive) radix-2 FFT with explicit bit reversal.
12. Derive the Bluestein reduction of arbitrary-n DFT to convolution at high level.
13. Analyze round-trip error vs n for `float32` vs `float64`.
14. Connect Toeplitz matrix–vector products to FFT embedding.
15. STFT: trade hop size, window length, and time–frequency resolution.

Summary

The DFT is an exact $n \times n$ change of basis; the FFT is a fast algorithm for that basis change. Master the definition, the $O(n \log n)$ butterfly intuition, convolution-via-FFT, and the measurement pitfalls (windows, leakage, normalization, real packing). Use library FFTs in production; use the tiny radix-2 listing only to cement the mental model.

Sources (for further reading)

- Cooley & Tukey (1965), "An algorithm for the machine calculation of complex Fourier series."
- Oppenheim & Schafer, *Discrete-Time Signal Processing* — DFT/FFT and windowing chapters.
- Press et al., *Numerical Recipes*, FFT chapters — accessible algorithmic notes.
- NumPy `numpy.fft` documentation — conventions, `rfft`, normalization modes.

Numerical Methods for Ordinary Differential Equations

An ordinary differential equation (ODE) tells you how a state **changes**; a solver tells you where the state **goes**. Orbits, circuits, chemical kinetics, epidemics, control loops, physics engines, and the "neural ODE" family all reduce to the same computational question: given $y'(t) = f(t, y)$ and a starting value, produce a trustworthy sequence $y_0, y_1, y_2, \ldots$ that tracks the true trajectory.

This chapter builds the working toolkit: Euler and Runge–Kutta methods, order of accuracy and how to *measure* it, absolute stability and **stiffness**, implicit methods, adaptive step-size control with embedded pairs, symplectic integrators for long-time mechanics, and how to drive a production solver (`scipy.integrate.solve_ivp`) without fooling yourself.

1. The initial value problem

An **initial value problem (IVP)** is

$$y'(t) = f(t, y(t)), \qquad y(t_0) = y_0, \qquad t \in [t_0, T],$$

where $y \in \mathbb{R}^m$ may be a vector (the state of the whole system).

Theorem (Picard–Lindelöf, informal). If f is continuous in t and **Lipschitz** in y, i.e. $\|f(t, y) - f(t, z)\| \le L\|y - z\|$, then the IVP has a unique solution on some interval around t_0.

The Lipschitz constant L also bounds how fast nearby trajectories can separate: $\|y(t) - z(t)\| \le e^{L(t-t_0)}\|y_0 - z_0\|$. That factor $e^{L(T-t_0)}$ reappears in every global error bound below—it is the **conditioning** of the IVP.

Higher order becomes first order

Every explicit higher-order ODE is a first-order system. For $x'' = g(t, x, x')$ set $y = (x, v)$ with $v = x'$:

$$\begin{pmatrix} x \\ v \end{pmatrix}' = \begin{pmatrix} v \\ g(t, x, v) \end{pmatrix}.$$

Solvers only ever see the first-order form.

Worked example 1 — a pendulum as a system

$\theta'' = -\frac{g}{\ell}\sin\theta$ becomes $y = (\theta, \omega)$, $f(t, y) = (\omega, -\frac{g}{\ell}\sin\theta)$. Two scalar unknowns, one vector ODE.

2. Forward Euler and the meaning of "order"

Replace the derivative by a forward difference over a step h:

$$y_{n+1} = y_n + h\, f(t_n, y_n), \qquad t_{n+1} = t_n + h.$$

Local truncation error. Plug the exact solution into the step. Taylor expansion gives $y(t+h) = y(t) + h\,y'(t) + \frac{h^2}{2}y''(\xi)$, so one Euler step commits an error of size $\frac{h^2}{2}|y''|$: the **local** error is $O(h^2)$.

Global error. Reaching T takes $N = (T - t_0)/h$ steps; local errors accumulate (and are amplified by at most $e^{L(T-t_0)}$):

$$\max_n \|y(t_n) - y_n\| \le \frac{hM}{2L}\left(e^{L(T-t_0)} - 1\right), \qquad M = \max\|y''\|.$$

The global error is $O(h)$. Euler is a **first-order** method.

Definition. A one-step method has **order** p if its local error is $O(h^{p+1})$, hence its global error is $O(h^p)$. Halving h divides the global error by about 2^p.

Worked example 2 — Euler by hand

$y' = -2y$, $y(0) = 1$, $h = 0.1$:

n	t_n	Euler y_n	exact e^{-2t_n}	error
0	0.0	1.0000	1.0000	0
1	0.1	0.8000	0.8187	0.0187
2	0.2	0.6400	0.6703	0.0303

Each step multiplies by $(1 - 2h) = 0.8$ instead of $e^{-0.2} \approx 0.8187$; the error compounds.

3. Runge–Kutta methods

Euler samples the slope once, at the start of the step. **Runge–Kutta (RK)** methods sample it several times inside the step and combine the samples so that Taylor terms cancel to higher order.

Heun's method (explicit trapezoid, order 2):

$$k_1 = f(t_n, y_n), \quad k_2 = f(t_n + h,\, y_n + hk_1), \quad y_{n+1} = y_n + \tfrac{h}{2}(k_1 + k_2).$$

Classic RK4 (order 4):

$$\begin{aligned} k_1 &= f(t_n, y_n), & k_2 &= f\left(t_n + \tfrac{h}{2},\, y_n + \tfrac{h}{2}k_1\right), \\ k_3 &= f\left(t_n + \tfrac{h}{2},\, y_n + \tfrac{h}{2}k_2\right), & k_4 &= f(t_n + h,\, y_n + hk_3), \\ y_{n+1} &= y_n + \tfrac{h}{6}\left(k_1 + 2k_2 + 2k_3 + k_4\right). \end{aligned}$$

Butcher tableaux

An s-stage RK method is fully described by coefficients (c_i, a_{ij}, b_j): $k_i = f(t_n + c_i h,\ y_n + h\sum_j a_{ij}k_j)$, $y_{n+1} = y_n + h\sum_j b_j k_j$. Written as a table:

```
RK4:                          Heun:
  0  |                          0 |
 1/2 | 1/2                      1 | 1
 1/2 |  0   1/2                 ---+---------
  1  |  0    0    1                | 1/2  1/2
-----+--------------------
     | 1/6  1/3  1/3  1/6
```

Strictly lower-triangular A means **explicit**: each stage uses only earlier stages. A nonzero diagonal or upper part means **implicit**: stages must be solved for (Section 6).

Worked example 3 — one Heun step

$y' = -2y$, $y_0 = 1$, $h = 0.1$: $k_1 = -2$, $k_2 = -2(1 - 0.2) = -1.6$, $y_1 = 1 + 0.05(-3.6) = 0.82$. Exact 0.818731: error 1.3×10^{-3}, versus 1.9×10^{-2} for Euler with the same h.

4. Measure the order — do not trust the label

The cheapest correctness test for any integrator: solve a problem with a known solution at $h, h/2, h/4, \ldots$ and compute the **observed order** $\log_2(e_h/e_{h/2})$. A buggy RK4 coefficient usually still "works" but drops to order 1 or 2—only this test catches it.

```
# file: convergence_order.py
# Observed order of Euler, Heun (RK2) and classic RK4 on y' = -2ty, y(0)=1.
# Exact solution: y(t) = exp(-t^2).  Integrate to t = 2.
import math


def f(t, y):
    return -2.0 * t * y


def euler_step(f, t, y, h):
    return y + h * f(t, y)


def heun_step(f, t, y, h):
    k1 = f(t, y)
    k2 = f(t + h, y + h * k1)
    return y + 0.5 * h * (k1 + k2)


def rk4_step(f, t, y, h):
    k1 = f(t, y)
    k2 = f(t + 0.5 * h, y + 0.5 * h * k1)
    k3 = f(t + 0.5 * h, y + 0.5 * h * k2)
    k4 = f(t + h, y + h * k3)
    return y + (h / 6.0) * (k1 + 2 * k2 + 2 * k3 + k4)


def integrate(step, n, T=2.0):
    h, t, y = T / n, 0.0, 1.0
    for _ in range(n):
        y = step(f, t, y, h)
        t += h
    return y


exact = math.exp(-4.0)
print(f"{'n':>6} {'Euler err':>11} {'Heun err':>11} {'RK4 err':>11}")
prev = None
for n in [20, 40, 80, 160, 320]:
    errs = [abs(integrate(s, n) - exact) for s in (euler_step, heun_step, rk4_step)]
    line = f"{n:>6} " + " ".join(f"{e:11.3e}" for e in errs)
    if prev:
        orders = [math.log2(p / e) for p, e in zip(prev, errs)]
        line += "   order  " + " ".join(f"{o:4.2f}" for o in orders)
    print(line)
    prev = errs
```

Run it (standard library only):

```
python3 convergence_order.py
```

Output:

```
   n   Euler err    Heun err     RK4 err
  20   6.293e-03   1.258e-03   6.813e-06
  40   3.102e-03   2.749e-04   3.725e-07   order  1.02 2.19 4.19
  80   1.539e-03   6.464e-05   2.177e-08   order  1.01 2.09 4.10
 160   7.663e-04   1.570e-05   1.316e-09   order  1.01 2.04 4.05
 320   3.824e-04   3.869e-06   8.085e-11   order  1.00 2.02 4.02
```

The observed orders converge to 1, 2, 4. Notice the economics: RK4 at $n = 20$ (80 slope evaluations) beats Euler at $n = 320$ (320 evaluations) by a factor of about 60 in error.

Worked example 4 — cost per digit

To gain one more decimal digit, a method of order p needs $10^{1/p}$ times as many steps: $10\times$ for Euler, $1.78\times$ for RK4. High order wins whenever the solution is smooth.

5. Absolute stability and stiffness

Order describes what happens as $h \to 0$. Real computations use a **finite** h, and then a second question dominates: do errors *decay* or *grow* from step to step?

The test equation

Apply a method to $y' = \lambda y$ with $\operatorname{Re}\lambda < 0$ (true solution decays). Every RK method produces $y_{n+1} = R(z)\, y_n$ with $z = h\lambda$; R is the method's **stability function**.

Method	$R(z)$	Stable on the negative real axis for
Forward Euler	$1 + z$	$-2 \le z \le 0$
Heun / RK2	$1 + z + \frac{z^2}{2}$	$-2 \le z \le 0$
RK4	$1 + z + \frac{z^2}{2} + \frac{z^3}{6} + \frac{z^4}{24}$	$-2.785 \lesssim z \le 0$
Backward Euler	$\dfrac{1}{1 - z}$	all $z \le 0$ (whole left half-plane)
Trapezoid (implicit)	$\dfrac{1 + z/2}{1 - z/2}$	all $z \le 0$ (whole left half-plane)

The **region of absolute stability** is $\{z \in \mathbb{C} : |R(z)| \le 1\}$. A method is **A-stable** if that region contains the entire left half-plane. For systems $y' = Jy$, the condition must hold for $z = h\lambda_i$ for **every** eigenvalue λ_i of the Jacobian J.

Fact. The stability function of an explicit RK method is a polynomial, so $|R(z)| \to \infty$ as $z \to -\infty$: **no explicit RK method is A-stable**.

Stiffness

A problem is **stiff** when the Jacobian has eigenvalues with large negative real part (fast, strongly damped modes) while the solution you care about evolves slowly. An explicit method must keep $h\,|\lambda_{\max}|$ inside its small stability interval to avoid blow-up—even long after the fast transient has died and accuracy alone would allow a much bigger step. The step size is set by **stability**, not by **accuracy**: that is the working definition of stiffness.

Worked example 5 — Euler explodes, backward Euler shrugs

$y' = -a(y - \cos t)$, $y(0) = 0$, $a = 1000$. After a transient of duration $\sim 1/a$, the solution follows $\cos t$ closely. Forward Euler needs $h < 2/a = 0.002$.

Backward (implicit) Euler evaluates the slope at the *end* of the step:

$$y_{n+1} = y_n + h\, f(t_{n+1}, y_{n+1}).$$

Here f is linear in y, so each step is a scalar solve.

```
# file: stiff_euler.py
# Stiff test problem: y' = -a (y - cos t), y(0) = 0, a = 1000, on [0, 0.2].
# After a transient of length ~1/a the solution hugs cos t.
# Forward Euler is stable only if h < 2/a; backward Euler is stable for all h > 0.
import math

A = 1000.0
T = 0.2


def exact(t):
    p = A * A / (1 + A * A)
    q = A / (1 + A * A)
    return p * math.cos(t) + q * math.sin(t) - p * math.exp(-A * t)


def forward_euler(h):
    t, y = 0.0, 0.0
    for _ in range(round(T / h)):
        y = y - h * A * (y - math.cos(t))
        t += h
```

```
    return y


def backward_euler(h):
    # y_{n+1} = y_n - h*A*(y_{n+1} - cos t_{n+1}): linear in y_{n+1}, solve directly
    t, y = 0.0, 0.0
    for _ in range(round(T / h)):
        t += h
        y = (y + h * A * math.cos(t)) / (1.0 + h * A)
    return y


print(f"exact y({T}) = {exact(T):.6f}")
for h in [0.0005, 0.0016, 0.002, 0.0025, 0.01, 0.05]:
    fe, be = forward_euler(h), backward_euler(h)
    print(f"h={h:<6}  h*a={h*A:5.1f}  forward={fe:12.5g}  backward={be:.6f}")
```

```
python3 stiff_euler.py
```

Output:

```
exact y(0.2) = 0.980264
h=0.0005  h*a=  0.5  forward=     0.98026  backward=0.980264
h=0.0016  h*a=  1.6  forward=     0.98027  backward=0.980263
h=0.002   h*a=  2.0  forward=   -0.019735  backward=0.980263
h=0.0025  h*a=  2.5  forward= -1.2226e+14  backward=0.980263
h=0.01    h*a= 10.0  forward= -1.2158e+19  backward=0.980259
h=0.05    h*a= 50.0  forward= -5.7649e+06  backward=0.980240
```

Read the rows:

- $ha < 2$: forward Euler is fine.
- $ha = 2$ exactly: $R = -1$; the initial transient error (size ≈ 1) neither grows nor decays—it flips sign every step, so the answer is off by almost exactly 1.
- $ha > 2$: $|R| > 1$ and the error grows geometrically to 10^{14} and beyond.
- Backward Euler stays accurate to about 10^{-5} even with $h = 0.05$ (only four steps), because $|1/(1 + ha)| < 1$ for every $h > 0$.

6. Implicit methods in practice

For nonlinear f, each implicit step is a nonlinear system. For backward Euler:

$$G(Y) = Y - y_n - h\, f(t_{n+1}, Y) = 0, \qquad G'(Y) = I - h\, J(t_{n+1}, Y).$$

Solvers apply (simplified) **Newton** iterations: factor $I - hJ$ once, reuse it across iterations and often across several steps, refactor only when convergence slows or h changes a lot. Consequences:

Consequence	Why it matters
Each step costs a linear solve	$O(m^3)$ dense, far less if J is sparse/banded
A good Jacobian pays off	Supply `jac=` analytically or a sparsity pattern; finite-difference Jacobians cost m extra f calls
Fewer, much larger steps	The win on stiff problems is often 100–1000× in step count

Families you will meet:

- **BDF** (backward differentiation formulas): implicit multistep, orders 1–5 in common codes. BDF1 is backward Euler; BDF1–2 are A-stable; higher orders trade a sliver of the left half-plane for accuracy.
- **Radau IIA**: implicit RK, order 5 in SciPy's `Radau`, A-stable and **L-stable** ($R(z) \to 0$ as $z \to -\infty$, so very stiff modes are damped in one step rather than merely kept bounded).
- **Trapezoid / Crank–Nicolson**: A-stable but $R(z) \to -1$ as $z \to -\infty$, so stiff modes ring (sign-flipping, slowly decaying). Fine for diffusion with moderate steps; poor for very stiff transients.

Dahlquist's second barrier. An A-stable *linear multistep* method has order at most 2. That is why high-order stiff solvers either relax A-stability (BDF3–5) or use implicit Runge–Kutta (Radau).

7. Adaptive step-size control

A fixed h is either wasteful where the solution is smooth or inaccurate where it changes fast. Production solvers **choose h per step** from an error estimate.

Embedded pairs

Use two RK formulas of orders p and $p-1$ that share the same stages. Their difference estimates the local error of the lower-order one almost for free:

$$\mathrm{err} = \left\| \frac{y_{n+1}^{[p]} - y_{n+1}^{[p-1]}}{\mathrm{atol} + \mathrm{rtol} \cdot |y_{n+1}|} \right\|.$$

Accept the step if $\mathrm{err} \le 1$; either way, update the step size with

$$h_{\mathrm{new}} = h \cdot \min\Big(f_{\max},\, \max(f_{\min},\; 0.9\,\mathrm{err}^{-1/p}) \Big),$$

because the estimated error scales like h^p (the lower-order formula has local error $O(h^p)$). The safety factor 0.9 and clamps $[f_{\min}, f_{\max}]$ prevent oscillation.

Common pairs: Bogacki–Shampine 3(2) (SciPy `RK23`, MATLAB `ode23`), Dormand–Prince 5(4) (SciPy `RK45`, MATLAB `ode45`), Dormand–Prince 8(5,3) (SciPy `DOP853`). These pairs propagate

the **higher**-order solution ("local extrapolation") and are **FSAL**: the last stage of an accepted step equals the first stage of the next, saving one evaluation per step.

```
# file: adaptive_bs23.py
# Adaptive Bogacki-Shampine 3(2) embedded Runge-Kutta pair with a standard step controller.
# Problem: y' = -2ty, y(0) = 1, exact y = exp(-t^2), integrate to t = 2.
import math


def f(t, y):
    return -2.0 * t * y


def bs23(f, t0, y0, t_end, rtol, atol, h=1e-2):
    t, y = t0, y0
    k1 = f(t, y)  # FSAL: first stage of the next step equals the last stage of this one
    accepted = rejected = 0
    while t < t_end:
        h = min(h, t_end - t)
        k2 = f(t + 0.5 * h, y + 0.5 * h * k1)
        k3 = f(t + 0.75 * h, y + 0.75 * h * k2)
        y3 = y + h * (2 * k1 + 3 * k2 + 4 * k3) / 9.0          # 3rd-order solution
        k4 = f(t + h, y3)
        y2 = y + h * (7 * k1 / 24 + k2 / 4 + k3 / 3 + k4 / 8)  # embedded 2nd-order
        scale = atol + rtol * max(abs(y), abs(y3))
        err = abs(y3 - y2) / scale                              # <= 1 means "acceptable"
        if err <= 1.0:
            t, y, k1 = t + h, y3, k4                            # accept, reuse k4
            accepted += 1
        else:
            rejected += 1
        # step-size update: error ~ h^3 for the lower-order estimate -> exponent 1/3
        factor = 0.9 * (1.0 / err) ** (1.0 / 3.0) if err > 0 else 5.0
        h *= min(5.0, max(0.2, factor))
    return y, accepted, rejected


exact = math.exp(-4.0)
print(f"{'rtol':>8} {'steps':>6} {'rejects':>7} {'f-evals':>7} {'abs err':>10}")
for rtol in [1e-3, 1e-5, 1e-7, 1e-9]:
    y, acc, rej = bs23(f, 0.0, 1.0, 2.0, rtol=rtol, atol=rtol * 1e-3)
    nfev = 1 + 3 * (acc + rej)
    print(f"{rtol:8.0e} {acc:6d} {rej:7d} {nfev:7d} {abs(y - exact):10.2e}")
```

```
python3 adaptive_bs23.py
```

Output:

```
 rtol  steps rejects f-evals    abs err
1e-03     11       2      40   1.22e-03
1e-05     51       4     166   3.60e-06
1e-07    236       5     724   3.13e-08
1e-09   1100       6    3319   3.48e-10
```

Two lessons:

1. **Work scales like tol$^{-1/3}$** for a third-order method: each factor of 100 in tolerance costs $100^{1/3} \approx 4.6\times$ more steps ($51 \to 236 \to 1100$).
2. **Tolerances control local error, not global error.** Here the final error happens to sit near `rtol`, but for unstable or long-time problems the global error can be orders of magnitude larger than the tolerance. Verify by re-running at a tighter tolerance and comparing.

Worked example 6 — choosing tolerances

A state with components of very different magnitudes (e.g. concentrations near 10^{-9} alongside temperatures near 300) needs a **per-component** `atol`: a scalar `atol=1e-6` silently lets the tiny component be pure noise. SciPy accepts an array for `atol`.

8. Production solvers: `solve_ivp`

SciPy's `solve_ivp` exposes the families above behind one interface:

`method=`	Type	Order	Use when
`RK45` (default)	explicit, Dormand–Prince 5(4)	5	non-stiff, moderate accuracy
`RK23`	explicit, Bogacki–Shampine 3(2)	3	loose tolerances, cheap f
`DOP853`	explicit, Dormand–Prince 8	8	non-stiff, tight tolerances
`Radau`	implicit RK (Radau IIA)	5	stiff, high accuracy
`BDF`	implicit multistep	1–5 (variable)	stiff, large systems, sparse Jacobians
`LSODA`	Adams BDF switching (ODEPACK)	variable	unsure whether stiff

Defaults are `rtol=1e-3`, `atol=1e-6`—loose. Set them explicitly. Also useful: `dense_output=True` (continuous interpolant between steps), `t_eval=` (report at chosen times without forcing step sizes),

`events=` (root-find a condition such as "ball hits the floor"), `jac=` / `jac_sparsity=` for implicit methods.

Worked example 7 — the Van der Pol stiffness benchmark

$x'' - \mu(1 - x^2)x' + x = 0$. For $\mu = 1$ it is a gentle limit cycle; for $\mu = 1000$ it alternates slow drifts with abrupt jumps, and the Jacobian's fast eigenvalue is of order $-\mu$ on the slow branches.

```
# file: vdp_solve_ivp.py
# Van der Pol oscillator  x'' - mu (1 - x^2) x' + x = 0  as a first-order system.
# mu = 1 is non-stiff; mu = 1000 is the classic stiff benchmark.
import time

import numpy as np
from scipy.integrate import solve_ivp


def vdp(mu):
    def rhs(t, y):
        x, v = y
        return [v, mu * (1 - x * x) * v - x]

    def jac(t, y):
        x, v = y
        return [[0.0, 1.0], [-2 * mu * x * v - 1.0, mu * (1 - x * x)]]

    return rhs, jac


for mu, t_end in [(1.0, 20.0), (1000.0, 3000.0)]:
    rhs, jac = vdp(mu)
    print(f"mu = {mu:g}, t in [0, {t_end:g}]")
    for method in ["RK45", "DOP853", "Radau", "BDF", "LSODA"]:
        kw = {"jac": jac} if method in ("Radau", "BDF", "LSODA") else {}
        t0 = time.perf_counter()
        sol = solve_ivp(rhs, (0.0, t_end), [2.0, 0.0], method=method,
                        rtol=1e-6, atol=1e-9, **kw)
        dt = time.perf_counter() - t0
        print(f"  {method:7s} ok={sol.success!s:5}  steps={sol.t.size - 1:7d}  "
              f"nfev={sol.nfev:8d}  njev={sol.njev:4d}  x(end)={sol.y[0, -1]: .6f}  {dt:6.2f
```

```
python3 -m venv .venv && . .venv/bin/activate
pip install numpy scipy
python vdp_solve_ivp.py      # the two explicit stiff runs take about a minute each
```

Output (SciPy 1.18.1, NumPy 2.5.3, one Xeon core; timings vary by machine):

```
mu = 1, t in [0, 20]
  RK45    ok=True   steps=    176  nfev=    1436  njev=   0  x(end)= 2.008149    0.01s
  DOP853  ok=True   steps=     69  nfev=    1070  njev=   0  x(end)= 2.008151    0.01s
  Radau   ok=True   steps=    498  nfev=    3773  njev=  41  x(end)= 2.008150    0.09s
  BDF     ok=True   steps=    666  nfev=    1616  njev=   1  x(end)= 2.008149    0.06s
  LSODA   ok=True   steps=    478  nfev=    1046  njev=   0  x(end)= 2.008148    0.00s
mu = 1000, t in [0, 3000]
  RK45    ok=True   steps=1689345  nfev=11825108  njev=   0  x(end)=-1.510606   64.48s
  DOP853  ok=True   steps= 874676  nfev=10498658  njev=   0  x(end)=-1.510606   52.66s
  Radau   ok=True   steps=   1282  nfev=   10746  njev= 306  x(end)=-1.510607    0.23s
  BDF     ok=True   steps=   1848  nfev=    5593  njev= 102  x(end)=-1.510585    0.18s
  LSODA   ok=True   steps=   1896  nfev=    3257  njev= 232  x(end)=-1.510584    0.01s
```

What the table says:

- **Non-stiff ($\mu = 1$):** explicit methods win; `DOP853` takes the fewest steps at this tight tolerance. Implicit methods pay for Jacobians and linear solves they do not need.
- **Stiff ($\mu = 1000$):** explicit methods still *succeed*—adaptivity keeps them stable—but need about 10^6 steps and two to three orders of magnitude more wall time, because h is pinned near the stability limit for the entire run. Implicit methods take about a thousand steps.
- `LSODA` used no Jacobian at $\mu = 1$ (it stayed with non-stiff Adams formulas) and switched to BDF with Jacobians at $\mu = 1000$.
- The `x(end)` values agree to about 2×10^{-5} across methods: consistent with tolerances that bound local, not global, error on a trajectory with sharp jumps.

Diagnostic rule: if an explicit solver reports success but takes a huge number of tiny, nearly uniform steps where the solution looks smooth, the problem is stiff. Switch to `Radau`/`BDF`/`LSODA` and supply a Jacobian.

9. Long-time integration and geometric methods

For conservative mechanics (planets, molecules, ideal oscillators) small *systematic* errors matter more than small *random* ones: a method that loses 0.01% energy per orbit has lost most of it after 10^4 orbits, regardless of its order.

A Hamiltonian system $q' = \partial H/\partial p$, $p' = -\partial H/\partial q$ has a flow that preserves phase-space area (it is **symplectic**). **Symplectic integrators** preserve that structure exactly. Backward error analysis shows they exactly conserve a *modified* energy $\tilde{H} = H + O(h^p)$, so the true energy error stays **bounded** over very long times instead of drifting.

Störmer–Verlet (leapfrog) for $H = \frac{1}{2}p^2 + V(q)$ ("kick–drift–kick"):

$$p_{n+1/2} = p_n - \tfrac{h}{2} V'(q_n), \quad q_{n+1} = q_n + h\, p_{n+1/2}, \quad p_{n+1} = p_{n+1/2} - \tfrac{h}{2} V'(q_{n+1}).$$

Second order, explicit, one force evaluation per step, time-reversible.

```
# file: energy_drift.py
# Harmonic oscillator q' = p, p' = -q; energy E = (p^2 + q^2)/2 should stay 0.5.
# Compare classic RK4 with Störmer-Verlet (leapfrog), a 2nd-order symplectic method.
def rk4(q, p, h):
    def f(q, p):
        return p, -q
    k1q, k1p = f(q, p)
    k2q, k2p = f(q + h / 2 * k1q, p + h / 2 * k1p)
    k3q, k3p = f(q + h / 2 * k2q, p + h / 2 * k2p)
    k4q, k4p = f(q + h * k3q, p + h * k3p)
    return (q + h / 6 * (k1q + 2 * k2q + 2 * k3q + k4q),
            p + h / 6 * (k1p + 2 * k2p + 2 * k3p + k4p))


def verlet(q, p, h):
    p_half = p - h / 2 * q        # kick
    q_new = q + h * p_half        # drift
    p_new = p_half - h / 2 * q_new  # kick
    return q_new, p_new


h = 0.5
print(f"h = {h}; energy error E - 0.5 after N steps")
print(f"{'N':>8} {'RK4':>12} {'Verlet':>12}")
qa, pa = 1.0, 0.0
qb, pb = 1.0, 0.0
checkpoints = {10, 100, 1000, 10000, 100000}
for n in range(1, 100001):
    qa, pa = rk4(qa, pa, h)
    qb, pb = verlet(qb, pb, h)
    if n in checkpoints:
        ea = 0.5 * (qa * qa + pa * pa) - 0.5
        eb = 0.5 * (qb * qb + pb * pb) - 0.5
        print(f"{n:>8} {ea:12.3e} {eb:12.3e}")
```

```
python3 energy_drift.py
```

Output:

```
h = 0.5; energy error E - 0.5 after N steps
       N          RK4       Verlet
      10   -1.050e-03   -2.775e-02
     100   -1.040e-02   -2.232e-03
    1000   -9.481e-02   -5.571e-03
   10000   -4.389e-01   -2.751e-02
  100000   -5.000e-01   -4.552e-03
```

RK4 is *more accurate per step* (fourth order) yet bleeds energy monotonically: $|R(ih)| < 1$ for this h, so the oscillation is artificially damped until it has essentially stopped. Verlet's energy error oscillates and never exceeds $h^2/8 = 0.03125$ over all 10^5 steps (about 8,000 periods).

Worked example 8 — pick by the question

- "Where is the satellite in 3 hours, to 1 m?" → high-order adaptive RK (`DOP853`), tight tolerances.
- "Is this planetary system stable over a million orbits?" → symplectic, fixed step.
- "Concentrations in a reaction network with rates from 10^{-3} to 10^{6} s^{-1}" → stiff solver (`BDF`/`Radau`) with a sparse Jacobian.

10. Multistep methods (map)

Instead of re-sampling inside each step, **linear multistep methods** reuse slopes from previous steps:

Family	Example	Notes
Adams–Bashforth (explicit)	$y_{n+1} = y_n + \frac{h}{2}(3f_n - f_{n-1})$	One new f per step; small stability region
Adams–Moulton (implicit)	trapezoid is AM2	Used as corrector in predictor–corrector pairs
BDF (implicit)	$\frac{3}{2}y_{n+1} - 2y_n + \frac{1}{2}y_{n-1} = hf_{n+1}$ (BDF2)	Stiff workhorse

Dahlquist equivalence theorem: a linear multistep method converges if and only if it is **consistent** (order ≥ 1) and **zero-stable** (its characteristic polynomial's roots lie in the closed unit disk, with roots on the circle simple). Multistep methods need a starter (an RK step or order ramp-up) and care when h changes; mature codes handle both.

11. Pitfalls

1. **Explicit solver on a stiff problem.** It "works", slowly—or appears to hang. Watch the step count.
2. **Trusting default tolerances.** `rtol=1e-3` is three digits per step; the global error can be much worse.
3. **Scalar `atol` for multi-scale states.** Small components drown in absolute tolerance.
4. **Non-smooth right-hand sides** (`if`, `abs`, table lookups, controller saturations). Adaptive solvers shrink h to crawl across kinks, and order is lost. Split the integration at known switching times or use `events=` to stop and restart.
5. **Checking only one h or one tolerance.** Always re-run tighter; agreement is the evidence.
6. **Wrong Jacobian.** A buggy analytic `jac` makes implicit solvers take tiny steps or fail; test it against finite differences first.

7. **Energy drift misread as physics.** A slowly decaying "conservative" system usually means a non-symplectic integrator, not friction.
8. **Global error mistaken for local error.** For chaotic systems (positive Lyapunov exponent), *pointwise* accuracy is lost after a finite horizon no matter the tolerance; trust statistics, not individual long trajectories.

12. Checkpoint

- Convert a second-order ODE into a first-order system.
- Derive Euler's local error $O(h^2)$ and explain why the global error is $O(h)$.
- Write the RK4 stages and its Butcher tableau.
- Measure the observed order of a solver from errors at h and $h/2$.
- Compute the stability function for Euler and backward Euler and state their step limits on $y' = \lambda y$.
- Define stiffness operationally and name two stiff solvers.
- Explain how an embedded pair estimates error and how h is updated.
- Say when you would use a symplectic integrator instead of RK4.

Exercises

Easy

1. Write $x''' = x\,x' - t$ as a first-order system.
2. One Euler step and one Heun step for $y' = t + y$, $y(0) = 1$, $h = 0.1$. Compare to the exact $y = 2e^t - t - 1$.
3. For $y' = -50y$, what is the largest stable step for forward Euler? For RK4 (use the table)?
4. If a method's error drops from 3.2×10^{-4} to 2.0×10^{-5} when h halves, what is its observed order?
5. Why does an FSAL pair cost one fewer f evaluation per accepted step?

Medium

6. Derive $R(z)$ for Heun's method and verify that it matches the Taylor series of e^z through z^2.
7. Show that the trapezoid rule is A-stable but $R(z) \to -1$ as $z \to -\infty$. What does that imply for very stiff modes?
8. Modify `adaptive_bs23.py` to print the accepted step sizes and plot h against t. Explain the shape.
9. For $y' = Jy$ with $J = \begin{pmatrix} -1 & 0 \\ 0 & -1000 \end{pmatrix}$, which eigenvalue limits explicit Euler's step? Which one determines accuracy after $t = 0.01$?
10. Re-run `vdp_solve_ivp.py` for $\mu = 1000$ with `Radau` but *without* `jac=`. Compare `nfev` and explain the difference.

Challenge

11. Implement the implicit midpoint rule for a nonlinear f using Newton iterations. Verify order 2 and A-stability numerically.
12. Prove that Störmer–Verlet is symplectic for $H = \frac{1}{2}p^2 + V(q)$ (show that the step map has Jacobian determinant 1).
13. Derive the modified energy conserved by Verlet on the harmonic oscillator, and explain why the bound $h^2/8$ appears.
14. Add `events=` to the Van der Pol run to record every zero crossing of x; estimate the period for $\mu = 1000$ and compare it with the asymptotic $(3 - 2\ln 2)\mu$.
15. Implement BDF2 with a fixed step for the stiff problem in `stiff_euler.py`; measure its order and compare its accuracy to backward Euler at $h = 0.01$.

Summary

An ODE solver is judged on three separate axes. **Order** governs how the error shrinks as $h \to 0$, and you should measure it rather than trust it. **Stability** governs whether errors grow at the step size you can actually afford; stiffness makes it the dominant constraint and is the reason implicit methods (BDF, Radau) exist. **Structure** governs long-time fidelity: symplectic methods keep energy bounded where higher-order generic methods drift. In practice: start with an adaptive explicit pair, set tolerances deliberately, watch the step count for signs of stiffness, switch to an implicit solver with a real Jacobian when needed, and always confirm a result by tightening the tolerance.

Sources (for further reading)

- Hairer, Nørsett & Wanner, *Solving Ordinary Differential Equations I: Nonstiff Problems* (2nd ed., Springer) — order conditions, embedded pairs, step-size control.
- Hairer & Wanner, *Solving Ordinary Differential Equations II: Stiff and Differential-Algebraic Problems* (2nd ed., Springer) — stiffness, A-/L-stability, Radau IIA, the Van der Pol benchmark.
- Hairer, Lubich & Wanner, *Geometric Numerical Integration* (2nd ed., Springer) — symplectic methods, Störmer–Verlet, backward error analysis.
- Dormand & Prince (1980), "A family of embedded Runge–Kutta formulae"; Bogacki & Shampine (1989), "A 3(2) pair of Runge–Kutta formulas."
- Shampine & Reichelt (1997), "The MATLAB ODE Suite" — practical design of `ode23`/`ode45`/`ode15s`.
- SciPy documentation, `scipy.integrate.solve_ivp` — methods, tolerances, events, dense output.

Numerical Integration (Quadrature)

Most integrals that matter in computing have no closed form: the normalising constant of a posterior, the expected loss under a data distribution, the energy in a signal, the light arriving at a pixel. A **quadrature rule** replaces $\int_a^b f(x)\,dx$ by a weighted sum of function values,

$$\int_a^b f(x)\,dx \;\approx\; \sum_{i=1}^{n} w_i\, f(x_i),$$

and the whole craft is choosing the **nodes** x_i and **weights** w_i so the error is small for the functions you actually have, with as few evaluations of f as possible. Evaluations are the cost unit: f may be a simulation, a neural network, or a database query.

This chapter builds the working toolkit: composite Newton–Cotes rules and how to *measure* their order, the surprising accuracy of the trapezoid rule on periodic functions, Richardson extrapolation (Romberg), Gaussian quadrature, adaptive subdivision, singularities and infinite ranges, production use of `scipy.integrate.quad` (and where it silently fails), and what changes in many dimensions.

All listings are complete Python files. They were run on Debian 13 with **Python 3.13.5, NumPy 2.5.3, SciPy 1.18.1**; outputs are pasted unedited.

```
python3 -m venv .venv && . .venv/bin/activate
pip install numpy scipy
```

1. Rules, exactness, and order

Two numbers describe a rule:

- **Degree of exactness** p: the rule integrates every polynomial of degree $\le p$ exactly. Trapezoid has $p = 1$, Simpson $p = 3$ (one better than you would expect from quadratics, by symmetry), n-point Gauss–Legendre $p = 2n - 1$.

- **Order of convergence** of the *composite* rule: split $[a, b]$ into N panels of width $h = (b-a)/N$, apply the basic rule on each, and the error behaves like $C\,h^q$ for smooth f. Trapezoid $q = 2$, Simpson $q = 4$.

Rule	Basic form on $[x_0, x_0 + h]$ (or $[x_0, x_0 + 2h]$ for Simpson)	Error term (smooth f)
Midpoint	$h\,f(x_0 + h/2)$	$+\frac{h^3}{24} f''(\xi)$ per panel
Trapezoid	$\frac{h}{2}\,[f(x_0) + f(x_0 + h)]$	$-\frac{h^3}{12} f''(\xi)$ per panel

Rule	Basic form on $[x_0, x_0+h]$ (or $[x_0, x_0+2h]$ for Simpson)	Error term (smooth f)
Simpson	$\frac{h}{3}\left[f(x_0)+4f(x_0+h)+f(x_0+2h)\right]$	$-\frac{h^5}{90}f^{(4)}(\xi)$ per pair
Composite trapezoid		$-\frac{(b-a)h^2}{12}f''(\xi)$
Composite Simpson		$-\frac{(b-a)h^4}{180}f^{(4)}(\xi)$

Summing $N \propto 1/h$ panel errors of size h^3 gives the global h^2; that bookkeeping is where the "one order lost" comes from.

Worked example 1 — measure the order

Halve h; for an order-q method the error should drop by 2^q. Never trust a label you have not measured.

```
# file: composite_order.py
# Observed order of composite trapezoid and Simpson on I =  _0^1 e^x dx = e - 1.
import math

def trapezoid(f, a, b, n):
    h = (b - a) / n
    s = 0.5 * (f(a) + f(b)) + sum(f(a + i * h) for i in range(1, n))
    return h * s

def simpson(f, a, b, n):            # n must be even
    h = (b - a) / n
    s = f(a) + f(b)
    s += 4 * sum(f(a + i * h) for i in range(1, n, 2))
    s += 2 * sum(f(a + i * h) for i in range(2, n, 2))
    return h * s / 3

exact = math.e - 1
print(f"{'n':>5} {'trap error':>12} {'ratio':>6} {'simp error':>12} {'ratio':>6}")
prev_t = prev_s = None
for n in [2, 4, 8, 16, 32, 64, 128]:
    et = abs(trapezoid(math.exp, 0, 1, n) - exact)
    es = abs(simpson(math.exp, 0, 1, n) - exact)
    rt = f"{prev_t / et:6.2f}" if prev_t else "     -"
    rs = f"{prev_s / es:6.2f}" if prev_s else "     -"
    print(f"{n:5d} {et:12.3e} {rt} {es:12.3e} {rs}")
    prev_t, prev_s = et, es
```

```
python3 composite_order.py
```

```
   n   trap error  ratio   simp error  ratio
   2    3.565e-02      -    5.793e-04      -
   4    8.940e-03   3.99    3.701e-05  15.65
   8    2.237e-03   4.00    2.326e-06  15.91
  16    5.593e-04   4.00    1.456e-07  15.98
  32    1.398e-04   4.00    9.103e-09  15.99
  64    3.496e-05   4.00    5.690e-10  16.00
 128    8.740e-06   4.00    3.556e-11  16.00
```

Ratios settle at $4 = 2^2$ and $16 = 2^4$. If your own implementation shows a ratio of 2 where you expected 4, you have a bug (often a misplaced endpoint weight), or f is not as smooth as you think (§6).

2. The trapezoid rule on periodic functions

The **Euler–Maclaurin formula** explains the trapezoid error exactly:

$$T_h - \int_a^b f = \frac{h^2}{12}[f'(b) - f'(a)] - \frac{h^4}{720}[f'''(b) - f'''(a)] + \cdots$$

Every term is a difference of odd derivatives at the **endpoints**. If f is smooth and periodic with period $b - a$, all of them vanish, and the error decays faster than any power of h — typically geometrically.

Worked example 2 — the same rule, two integrands

```python
# file: periodic_trap.py
# Trapezoid on a smooth PERIODIC integrand over a full period:
#   I = _0^{2} e^{cos x} dx = 2 I_0(1)   (I_0 = modified Bessel function)
# Compare with the same rule on a non-periodic integrand of similar smoothness.
import math
from scipy.special import i0

def trapezoid(f, a, b, n):
    h = (b - a) / n
    return h * (0.5 * (f(a) + f(b)) + sum(f(a + i * h) for i in range(1, n)))

exact_p = 2 * math.pi * i0(1.0)
exact_np = math.e - 1                          # _0^1 e^x dx, not periodic on [0,1]
print(f"{'n':>4} {'periodic err':>14} {'non-periodic err':>17}")
for n in [2, 4, 8, 16, 32]:
    ep = abs(trapezoid(lambda x: math.exp(math.cos(x)), 0, 2 * math.pi, n) - exact_p)
    en = abs(trapezoid(math.exp, 0, 1, n) - exact_np)
    print(f"{n:4d} {ep:14.3e} {en:17.3e}")
```

```
python3 periodic_trap.py
```

```
   n   periodic err  non-periodic err
   2      1.741e+00         3.565e-02
   4      3.440e-02         8.940e-03
   8      1.252e-06         2.237e-03
  16      0.000e+00         5.593e-04
  32      1.776e-15         1.398e-04
```

Sixteen points reach machine precision on the periodic integrand; the non-periodic one is still at 5×10^{-4}. This is why the trapezoid rule, not Simpson, is the right default for integrals over a full period (Fourier coefficients, contour integrals, averages over an angle), and why the DFT is a trapezoid rule in disguise.

3. Richardson extrapolation and Romberg integration

If $T(h) = I + c_1 h^2 + c_2 h^4 + \cdots$, then $\frac{4T(h/2) - T(h)}{3}$ cancels the h^2 term. (That combination is exactly composite Simpson.) Repeating the trick column by column gives the **Romberg table**:

$$R_{k,j} = R_{k,j-1} + \frac{R_{k,j-1} - R_{k-1,j-1}}{4^j - 1}.$$

Worked example 3 — Romberg for π

```
# file: romberg.py
# Richardson extrapolation on trapezoid values = Romberg integration.
# I =  _0^1 4/(1+x^2) dx =  .
import math

def trapezoid(f, a, b, n):
    h = (b - a) / n
    return h * (0.5 * (f(a) + f(b)) + sum(f(a + i * h) for i in range(1, n)))

f = lambda x: 4 / (1 + x * x)
levels = 6
R = [[0.0] * levels for _ in range(levels)]
for k in range(levels):
    R[k][0] = trapezoid(f, 0, 1, 2 ** k)          # n = 1, 2, 4, ... panels
    for j in range(1, k + 1):                     # cancel h^2, h^4, h^6, ... terms
        R[k][j] = R[k][j - 1] + (R[k][j - 1] - R[k - 1][j - 1]) / (4 ** j - 1)

for k in range(levels):
    print(f"n={2**k:3d} " + " ".join(f"{R[k][j]:.12f}" for j in range(k + 1)))
print("errors in the last row:", " ".join(f"{abs(R[5][j] - math.pi):.1e}" for j in range(lev
```

```
print(f"function evaluations used: {2**(levels-1) + 1}")
```

```
python3 romberg.py
```

```
n=  1 3.000000000000
n=  2 3.100000000000 3.133333333333
n=  4 3.131176470588 3.141568627451 3.142117647059
n=  8 3.138988494491 3.141592502459 3.141594094126 3.141585783762
n= 16 3.140941612041 3.141592651225 3.141592661143 3.141592638397 3.141592665278
n= 32 3.141429893175 3.141592653553 3.141592653708 3.141592653590 3.141592653650 3.141592653
errors in the last row: 1.6e-04 3.7e-11 1.2e-10 2.4e-13 6.0e-11 4.8e-11
function evaluations used: 33
```

From the same 33 evaluations, the plain trapezoid value is good to 10^{-4} and the extrapolated values to 10^{-11}–10^{-13}. But the errors along the last row are **not monotone**, and that is the lesson. For $f = 4/(1+x^2)$ the coefficient c_2 is zero, because $f'''(1) - f'''(0) = 0$ (check: $f'(1) - f'(0) = -2$, $f'''(1) - f'''(0) = 0$, $f^{(5)}(1) - f^{(5)}(0) = 60$). So column 1 is already sixth order, and column 2, built to cancel an h^4 term that does not exist, removes nothing useful and adds noise. Extrapolation assumes an error expansion. When the assumption is wrong, or f is not smooth, extra columns can make things worse. Watch the differences between neighbouring entries, not just the corner.

4. Gaussian quadrature

Newton–Cotes rules fix the nodes (equally spaced) and choose weights. **Gauss** chooses both: with n free nodes and n free weights you can match $2n$ moments, so n-point Gauss–Legendre is exact for polynomials of degree $2n-1$. The nodes are the roots of the Legendre polynomial P_n. The **Golub–Welsch** algorithm gets them, stably, as eigenvalues of a small symmetric tridiagonal matrix built from the three-term recurrence; the weights come from the first components of the eigenvectors.

Worked example 4 — build the rule, check exactness, race Simpson

```
# file: gauss_legendre.py
# Gauss-Legendre nodes/weights from the Golub-Welsch eigenvalue method,
# checked against numpy, then used on I =  _{-1}^{1} e^x dx = e - 1/e.
import math
import numpy as np

def golub_welsch(n):
    k = np.arange(1, n)
    beta = k / np.sqrt(4 * k * k - 1)          # Legendre recurrence coefficients
    J = np.diag(beta, 1) + np.diag(beta, -1)   # symmetric tridiagonal Jacobi matrix
    x, V = np.linalg.eigh(J)
    w = 2 * V[0, :] ** 2                       # weight = 2 * (first eigenvector component)^
    return x, w
```

```
x, w = golub_welsch(3)
print("n=3 nodes  ", np.round(x, 12))
print("n=3 weights", np.round(w, 12))
xr, wr = np.polynomial.legendre.leggauss(3)
print("max |diff| vs numpy leggauss:", max(np.max(abs(x - xr)), np.max(abs(w - wr))))

# Degree of exactness: n points integrate polynomials up to degree 2n-1 exactly.
for deg in [4, 5, 6]:
    approx = np.sum(w * x ** deg)
    exact = 0.0 if deg % 2 else 2 / (deg + 1)
    print(f"n=3,  x^{deg}: rule {approx:.15f}  exact {exact:.15f}")

exact = math.e - 1 / math.e
print(f"{'evals':>5} {'Gauss error':>12} {'Simpson error':>14}")
for n in [3, 5, 7, 9, 11]:                      # odd n: Simpson with n-1 panels uses n point
    xg, wg = golub_welsch(n)
    eg = abs(np.sum(wg * np.exp(xg)) - exact)
    m = n - 1
    h = 2 / m
    xs = -1 + h * np.arange(m + 1)
    cs = np.ones(m + 1); cs[1:-1:2] = 4; cs[2:-1:2] = 2
    es = abs(h / 3 * np.sum(cs * np.exp(xs)) - exact)
    print(f"{n:5d} {eg:12.3e} {es:14.3e}")
```

```
python3 gauss_legendre.py
```

```
n=3 nodes   [-0.77459667  0.          0.77459667]
n=3 weights [0.55555556 0.88888889 0.55555556]
max |diff| vs numpy leggauss: 3.3306690738754696e-16
n=3,  x^4: rule 0.400000000000000  exact 0.400000000000000
n=3,  x^5: rule -0.000000000000000  exact 0.000000000000000
n=3,  x^6: rule 0.240000000000000  exact 0.285714285714286
evals  Gauss error  Simpson error
    3    6.546e-05      1.165e-02
    5    8.248e-10      7.924e-04
    7    2.665e-15      1.591e-04
    9    4.441e-16      5.063e-05
   11    0.000e+00      2.079e-05
```

Three points integrate x^4 and x^5 exactly and fail at x^6: degree of exactness $5 = 2\cdot 3 - 1$, as promised. On the smooth integrand e^x, seven Gauss points reach 3×10^{-15}, while Simpson with the same seven evaluations is at 1.6×10^{-4}. For smooth f, Gauss error decays geometrically in n; Newton–Cotes decays algebraically.

Variants worth knowing: **Gauss–Kronrod** (a $2n+1$-point rule that reuses the n Gauss nodes, so the difference gives a free error estimate; QUADPACK is built on 7–15 and 10–21 pairs), **Gauss–Hermite** for $\int e^{-x^2} f$, **Gauss–Laguerre** for $\int_0^\infty e^{-x} f$, and **Clenshaw–Curtis** (Chebyshev nodes,

nested, nearly as accurate as Gauss in practice).

5. Adaptive quadrature

Uniform panels waste evaluations where f is boring and starve the places where it changes fast. **Adaptive** rules estimate the error on each panel by comparing a coarse and a fine answer, and split only where the estimate exceeds the local tolerance.

Worked example 5 — one sharp peak

$f(x) = 1/((x-0.3)^2 + 10^{-6})$ has a spike of width $\sim 10^{-3}$ at $x = 0.3$.

```
# file: adaptive_simpson.py
# Adaptive Simpson vs uniform Simpson on a function with one sharp feature:
#   f(x) = 1 / ((x - 0.3)^2 + 1e-6) on [0, 1].  Exact via arctan.
import math

calls = 0
def f(x):
    global calls
    calls += 1
    return 1.0 / ((x - 0.3) ** 2 + 1e-6)

eps = 1e-3
exact = (math.atan(0.7 / eps) + math.atan(0.3 / eps)) / eps

def simpson_panel(a, fa, m, fm, b, fb):
    return (b - a) / 6 * (fa + 4 * fm + fb)

def adapt(a, fa, m, fm, b, fb, whole, tol, depth):
    lm, rm = (a + m) / 2, (m + b) / 2
    flm, frm = f(lm), f(rm)
    left = simpson_panel(a, fa, lm, flm, m, fm)
    right = simpson_panel(m, fm, rm, frm, b, fb)
    delta = left + right - whole
    if depth == 0 or abs(delta) <= 15 * tol:          # 15 = 2^4 - 1 (Simpson is O(h^4))
        return left + right + delta / 15              # Richardson-corrected
    return (adapt(a, fa, lm, flm, m, fm, left, tol / 2, depth - 1) +
            adapt(m, fm, rm, frm, b, fb, right, tol / 2, depth - 1))

def adaptive_simpson(a, b, tol, depth=50):
    fa, fb, m = f(a), f(b), (a + b) / 2
    fm = f(m)
    return adapt(a, fa, m, fm, b, fb, simpson_panel(a, fa, m, fm, b, fb), tol, depth)
```

```
def uniform_simpson(a, b, n):
    h = (b - a) / n
    s = f(a) + f(b) + 4 * sum(f(a + i * h) for i in range(1, n, 2)) \
        + 2 * sum(f(a + i * h) for i in range(2, n, 2))
    return h * s / 3

for tol in [1e-4, 1e-8]:
    calls = 0
    v = adaptive_simpson(0, 1, tol)
    print(f"adaptive tol={tol:.0e}: error {abs(v - exact):.2e} with {calls} evaluations")
for n in [512, 8192, 65536]:
    calls = 0
    v = uniform_simpson(0, 1, n)
    print(f"uniform  n={n:5d}:   error {abs(v - exact):.2e} with {calls} evaluations")
```

```
python3 adaptive_simpson.py
```

```
adaptive tol=1e-04: error 7.89e-07 with 1645 evaluations
adaptive tol=1e-08: error 4.55e-13 with 16249 evaluations
uniform  n=  512:   error 3.17e+02 with 513 evaluations
uniform  n= 8192:   error 4.31e-09 with 8193 evaluations
uniform  n=65536:   error 4.55e-13 with 65537 evaluations
```

At 4.6×10^{-13}, adaptive Simpson needs 16,249 evaluations where uniform Simpson needs 65,537. With 513 uniform points the spike is under-sampled and the answer is off by 317 (the integral is about 3,140). The factor 15 is $2^4 - 1$: comparing Simpson on a panel and on its two halves gives the error estimate *and* a Richardson correction for free. The tolerance is halved on each split so the panel errors add up to at most the global tolerance.

6. Singularities and infinite ranges

Every error formula above assumed bounded high derivatives. An endpoint like $\sqrt{x}$ at 0 (continuous, but f' unbounded) silently lowers the order.

Worked example 6 — order collapse and the cure

```
# file: singular_endpoint.py
#  _0^1 sqrt(x) dx = 2/3.  sqrt is continuous but not differentiable at 0:
# Simpson's h^4 rate collapses.  Substituting x = t^2 gives  _0^1 2 t^2 dt - a polynomial.
import math

def simpson(f, a, b, n):
    h = (b - a) / n
    s = f(a) + f(b) + 4 * sum(f(a + i * h) for i in range(1, n, 2)) \
```

```
        + 2 * sum(f(a + i * h) for i in range(2, n, 2))
    return h * s / 3

exact = 2 / 3
print(f"{'n':>5} {'Simpson on sqrt(x)':>19} {'ratio':>6} {'after x=t^2':>12}")
prev = ratio = None
for n in [4, 8, 16, 32, 64, 128]:
    e = abs(simpson(math.sqrt, 0, 1, n) - exact)
    e2 = abs(simpson(lambda t: 2 * t * t, 0, 1, n) - exact)
    ratio = prev / e if prev else None
    r = f"{ratio:6.2f}" if ratio else "     -"
    print(f"{n:5d} {e:19.3e} {r} {e2:12.1e}")
    prev = e
print(f"observed order = log2(last ratio) = {math.log2(ratio):.2f}   (theory: 1.5)")
```

```
python3 singular_endpoint.py
```

```
    n  Simpson on sqrt(x)  ratio  after x=t^2
    4           1.014e-02      -      0.0e+00
    8           3.587e-03   2.83      0.0e+00
   16           1.268e-03   2.83      0.0e+00
   32           4.485e-04   2.83      0.0e+00
   64           1.586e-04   2.83      0.0e+00
  128           5.606e-05   2.83      0.0e+00
observed order = log2(last ratio) = 1.50   (theory: 1.5)
```

The measured ratio $2.83 = 2^{1.5}$ says Simpson has become an $O(h^{1.5})$ method. The substitution $x = t^2$, $dx = 2t\,dt$ turns the integrand into $2t^2$, a quadratic that Simpson integrates exactly. Change variables to move singular behaviour out of the integrand before you integrate. For $\int_0^\infty$, map to a finite interval ($x = t/(1-t)$) or use a rule with the right weight (Gauss–Laguerre). The **tanh-sinh** (double-exponential) substitution handles endpoint singularities of unknown type remarkably well and is the default in arbitrary-precision libraries.

7. Production quadrature: `scipy.integrate.quad`

`quad` wraps QUADPACK (globally adaptive Gauss–Kronrod with extrapolation for singularities, plus transformations for infinite ranges). Use it, but read everything it returns.

Worked example 7 — four calls, one trap

```
# file: scipy_quad.py
# Production quadrature: scipy.integrate.quad (QUADPACK) - read its error estimate and evalu
import math
from scipy import integrate

f = lambda x: 1.0 / ((x - 0.3) ** 2 + 1e-4)
exact = (math.atan(70) + math.atan(30)) / 1e-2
val, err, info = integrate.quad(f, 0, 1, full_output=True)
print(f"peak:      value {val:.12f} est.err {err:.1e} true err {abs(val-exact):.1e} neval {i

# Integrable endpoint singularity:  _0^1 x^(-1/2) dx = 2
val, err, info = integrate.quad(lambda x: x ** -0.5, 0, 1, full_output=True)
print(f"1/sqrt(x): value {val:.12f} est.err {err:.1e} neval {info['neval']}")

# Infinite interval:  _0^ϖ e^(-x^2) dx = sqrt(pi)/2
val, err = integrate.quad(lambda x: math.exp(-x * x), 0, math.inf)
print(f"gaussian:  value {val:.15f} true err {abs(val - math.sqrt(math.pi)/2):.1e}")

# A trap: a narrow feature far from where quad looks. Exact  1 (bump area), on [0, 1000].
bump = lambda x: math.exp(-((x - 700.0) / 0.1) ** 2) / (0.1 * math.sqrt(math.pi))
val, err, info = integrate.quad(bump, 0, 1000, full_output=True)
print(f"bump, no hint:        value {val:.6e} est.err {err:.1e} neval {info['neval']}")
val, err = integrate.quad(bump, 0, 1000, points=[699.5, 700.5])
print(f"bump, points=[699.5,700.5]: value {val:.12f} est.err {err:.1e}")
```

```
python3 scipy_quad.py
```

```
peak:      value 309.398691512415 est.err 2.4e-08 true err 1.7e-13 neval 315
1/sqrt(x): value 2.000000000000 est.err 3.8e-15 neval 231
gaussian:  value 0.886226925452758 true err 0.0e+00
bump, no hint:        value 0.000000e+00 est.err 0.0e+00 neval 21
bump, points=[699.5,700.5]: value 0.999999999999 est.err 2.6e-14
```

- The peak (width 10^{-2} here) is handled in 315 evaluations, and the **reported** error (2.4×10^{-8}) is a pessimistic bound on the **true** one (1.7×10^{-13}). That is the right direction to be wrong.
- The $x^{-1/2}$ singularity at 0 is detected and extrapolated away. QUADPACK never evaluates the endpoint.
- The infinite range is mapped internally.
- **The trap:** a narrow bump at $x = 700$ in $[0, 1000]$. The first 21-point Kronrod rule never lands near it, sees a function that is zero everywhere it looked, and reports **value 0 with error estimate 0 after 21 evaluations — and no warning**. An adaptive method can only refine what it has seen. If you know where features are, say so (`points=`, or split the range yourself). If you do not, sample f on a grid first.

Related tools: `scipy.integrate.quad_vec` (vector-valued f), `fixed_quad` (plain n-point

Gauss), `simpson`/`trapezoid` for **tabulated data** you cannot re-sample, and `nquad`/`tplquad` for low-dimensional nested integrals.

8. Many dimensions: tensor grids, Monte Carlo, quasi-Monte Carlo

A tensor-product rule with n points per axis costs n^d evaluations: fine for $d \le 3$, painful by $d = 6$, hopeless at $d = 20$ ($2^{20} \approx 10^6$ points for the crudest rule). **Monte Carlo** averages f at N random points. Its error is $\sigma/\sqrt{N}$ whatever the dimension, but $\sqrt{N}$ is slow. **Quasi-Monte Carlo** uses low-discrepancy points (Sobol', Halton) and, for reasonably smooth f, gets close to $1/N$. Scrambling makes it randomised, so repetitions give an error estimate.

Worked example 8 — $d = 6$

```
# file: high_dim.py
# d = 6:  I =  _[0,1]^6  Π_i ( /2) sin(  x_i) dx = 1.
# Tensor-product Gauss-Legendre vs Monte Carlo vs scrambled Sobol' (quasi-Monte Carlo).
import numpy as np
from scipy.stats import qmc

d = 6
f = lambda X: np.prod(0.5 * np.pi * np.sin(np.pi * X), axis=1)

print("tensor Gauss-Legendre (n points per axis → n^d evaluations)")
for n in [2, 3, 4, 6]:
    x, w = np.polynomial.legendre.leggauss(n)
    x, w = (x + 1) / 2, w / 2                       # map [-1,1] → [0,1]
    grids = np.meshgrid(*([x] * d), indexing="ij")
    X = np.stack([g.ravel() for g in grids], axis=1)
    W = np.prod(np.stack(np.meshgrid(*([w] * d), indexing="ij")).reshape(d, -1), axis=0)
    print(f"  n={n}: evals {n**d:6d}  error {abs(W @ f(X) - 1):.2e}")

rng = np.random.default_rng(20261008)
print("Monte Carlo vs scrambled Sobol' (mean |error| over 20 repetitions)")
for m in [10, 12, 14, 16]:
    N = 2 ** m
    mc = [abs(f(rng.random((N, d))).mean() - 1) for _ in range(20)]
    qm = [abs(f(qmc.Sobol(d, scramble=True, seed=rng).random_base2(m)).mean() - 1) for _ in
    print(f"  N=2^{m} = {N:6d}:  MC {np.mean(mc):.2e}   Sobol {np.mean(qm):.2e}")
```

```
python3 high_dim.py
```

```
tensor Gauss-Legendre (n points per axis → n^d evaluations)
  n=2: evals     64  error 1.78e-01
  n=3: evals    729  error 4.17e-03
```

```
  n=4: evals   4096  error 4.73e-05
  n=6: evals  46656  error 1.57e-09
Monte Carlo vs scrambled Sobol' (mean |error| over 20 repetitions)
  N=2^10 =   1024:  MC 2.87e-02   Sobol 4.45e-03
  N=2^12 =   4096:  MC 1.52e-02   Sobol 2.30e-03
  N=2^14 =  16384:  MC 1.25e-02   Sobol 7.38e-04
  N=2^16 =  65536:  MC 4.26e-03   Sobol 6.87e-05
```

Read it honestly. For this **smooth, separable** integrand in six dimensions, tensor Gauss still wins (46,656 points give 10^{-9}). Monte Carlo improves by about $2\times$ per $4\times$ more points ($N^{-1/2}$), and scrambled Sobol' is 6–60$\times$ better than Monte Carlo at the same N and improves faster. The crossover to (Q)MC comes with higher d, less smoothness, or integrands you can only sample (expectations under a simulator). Sparse grids (Smolyak) sit in between.

9. Choosing a method

Situation	First choice	Why
Smooth f on $[a, b]$, cheap to call	`scipy.integrate.quad`	Adaptive Gauss–Kronrod with error estimate
Smooth periodic f over a full period	Composite trapezoid	Geometric convergence (§2)
Fixed budget, smooth f, no adaptivity	n-point Gauss–Legendre	Degree $2n-1$, geometric convergence
Tabulated samples only	`simpson` / `trapezoid` on the data	Cannot choose nodes
Known endpoint singularity	Substitution, then any rule; or `quad`	Restores smoothness (§6)
Narrow features at known locations	`quad(..., points=[...])` or split ranges	Adaptive rules cannot find what they never sample (§7)
$d \lesssim 4$, smooth	Nested `quad` / tensor Gauss	Accurate, deterministic
$d \gtrsim 6$ or rough f	Scrambled Sobol' QMC, else MC	Dimension-robust; repeat for error bars

10. Pitfalls

1. **Trusting a label instead of a measurement.** Halve h (or double n) and look at the ratio (§1).

2. **Assuming smoothness.** Kinks, $\sqrt{\cdot}$, $|x|$, and discontinuities drop the order. Split at the kink or substitute (§6).

3. **Believing "error estimate 0".** It means "no evidence of error where I looked" (§7).

4. **Extrapolating past what the error expansion supports.** Romberg columns are not guaranteed to improve (§3).

5. **Equally spaced high-order Newton–Cotes.** Beyond Simpson/Boole, weights turn negative and rounding amplifies. Go composite or Gauss.

6. **Tolerance below conditioning.** Integrals of large cancelling values ($\int$ of an oscillating function with a small net area) cannot be computed to a relative accuracy better than (cancellation $\times$ u). Ask for an absolute tolerance.

7. **Grids in high dimension.** n^d grows faster than your patience.

11. Checkpoint

- State the degree of exactness of trapezoid, Simpson, and n-point Gauss–Legendre
- Measure the order of a composite rule from an error table
- Explain, from Euler–Maclaurin, why trapezoid excels on periodic integrands
- Build one Romberg column and say when extra columns stop helping
- Describe how adaptive Simpson estimates its own error (the factor 15)
- Remove an endpoint singularity by substitution
- Read `quad`'s value, error estimate, and `neval`, and name a case where all three mislead
- Pick between tensor Gauss, MC, and QMC for a given d and smoothness

Exercises

Easy

1. Apply the composite trapezoid rule with $N = 4$ to $\int_0^1 x^3\,dx$ by hand and compare with the error formula.

2. Show that Simpson's rule integrates x^3 exactly on $[-1, 1]$ even though it is built from quadratics.

3. Verify the 3-point Gauss–Legendre nodes $0, \pm\sqrt{3/5}$ and weights $8/9, 5/9$ by integrating $1, x^2, x^4$.

Medium

4. Modify `composite_order.py` to integrate $|x - 1/3|$ on $[0, 1]$. What orders do you observe, and why does splitting at $1/3$ restore them?

5. Use `periodic_trap.py` to integrate $1/(2 + \cos x)$ over $[0, 2\pi]$ (exact: $2\pi/\sqrt{3}$). How many points reach 10^{-14}?

6. Replace Simpson inside `adaptive_simpson.py` with 3-point Gauss on each panel (compare one panel against its two halves). How many evaluations does it need for `tol=1e-8`?

7. In `high_dim.py`, set $d = 12$ and keep $n = 2$ for tensor Gauss. Compare the cost and error with Sobol' at $N = 2^{12}$.

Challenge

8. Implement Gauss–Kronrod 7–15 (look up the 15 nodes and weights in the QUADPACK documentation) and use $|G_7 - K_{15}|$ as an error estimate in a globally adaptive integrator with a priority queue of panels.

9. Implement the tanh-sinh rule $x = \tanh(\frac{\pi}{2}\sinh t)$ with trapezoid in t and test it on $\int_0^1 \ln x\,dx$ and $\int_0^1 x^{-1/2}\,dx$.

10. Prove the Euler–Maclaurin h^2 term for the composite trapezoid rule using the Peano kernel or Taylor expansion on one panel.

Summary

Quadrature is the art of choosing nodes and weights. Composite trapezoid and Simpson are $O(h^2)$ and $O(h^4)$ for smooth integrands, and you should verify that by measurement. Trapezoid is spectrally accurate on periodic integrands. Richardson/Romberg turns a cheap rule into a high-order

one when the error expansion holds. Gauss–Legendre is exact to degree $2n-1$ and converges geometrically for smooth f. Adaptive methods put evaluations where f is hard, and they cannot find what they never sample. Singularities lower the order unless you transform them away. In many dimensions, grids give way to Monte Carlo and quasi-Monte Carlo.

Sources (for further reading)

- P. J. Davis and P. Rabinowitz, *Methods of Numerical Integration*, 2nd ed., Academic Press, 1984.
- L. N. Trefethen and J. A. C. Weideman, "The Exponentially Convergent Trapezoidal Rule", *SIAM Review* 56(3), 2014: https://doi.org/10.1137/130932132
- G. H. Golub and J. H. Welsch, "Calculation of Gauss Quadrature Rules", *Mathematics of Computation* 23, 1969: https://doi.org/10.1090/S0025-5718-69-99647-1
- R. Piessens, E. de Doncker-Kapenga, C. Überhuber, D. Kahaner, *QUADPACK: A Subroutine Package for Automatic Integration*, Springer, 1983 (the library behind `scipy.integrate.quad`).
- SciPy documentation — `scipy.integrate.quad`: https://docs.scipy.org/doc/scipy/reference/generated/scipy.integrate.quad.html; `scipy.stats.qmc.Sobol`: https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.qmc.Sobol.html
- NumPy documentation — `numpy.polynomial.legendre.leggauss`: https://numpy.org/doc/stable/reference/generated/numpy.polynomial.legendre.leggauss.html
- NIST Digital Library of Mathematical Functions, §3.5 Quadrature: https://dlmf.nist.gov/3.5
- A. B. Owen, *Practical Quasi-Monte Carlo Integration* (draft book): https://artowen.su.domains/mc/practicalqmc.pdf

Interval Arithmetic and Certified Computation

Every earlier chapter in this part *estimated* error: a truncation term Ch^q, a residual, a condition number times machine epsilon. Estimates are usually right and occasionally badly wrong, and nothing in the printed number tells you which case you are in. **Interval arithmetic** changes the question. Instead of computing one floating-point number near the answer, it computes an interval $[\underline{x}, \overline{x}]$ that is **guaranteed to contain** the exact real result. Every rounding error, every inexact input, every operation is accounted for. When the interval is narrow, you have a *certified* answer. When it is wide, the computation is telling you that it does not know.

This chapter builds a small interval type on IEEE doubles. It then shows the two effects that make intervals hard to use well (**dependency** and **wrapping**), the standard cure (subdivision and rewriting), the **interval Newton** method that *proves* a root exists and is unique, and Rump's classic expression where float and multi-precision arithmetic agree on a wrong answer, but an interval does not.

All listings are complete Python files. They were run on Debian 13 with **Python 3.13.5** (standard library only) and **mpmath 1.3.0** for the last section. Outputs are pasted unedited.

```
python3 -m venv .venv && . .venv/bin/activate
pip install mpmath
```

1. The containment principle

For an operation $\circ \in \{+, -, \times, \div\}$ and intervals X, Y define

$$X \circ Y \;=\; \{\, x \circ y : x \in X,\; y \in Y \,\}.$$

For the four basic operations this set is again an interval, and its endpoints come from the endpoints of X and Y:

$$\begin{aligned}
&[a,b] + [c,d] = [a + c,\; b + d], && [a,b] - [c,d] = [a - d,\; b - c],\\
&[a,b] \times [c,d] = [\min S,\; \max S], && S = \{ac, ad, bc, bd\},\\
&[a,b] \div [c,d] = [a,b] \times [1/d,\; 1/c] && (0 \notin [c,d]).
\end{aligned}$$

The **fundamental theorem of interval arithmetic** (Moore) follows by induction over the expression: if f is built from these operations and $x \in X$, then $f(x) \in F(X)$, where F is the same expression evaluated with intervals. The containment is guaranteed, and nothing guarantees that it is tight.

On a computer the endpoints are themselves rounded. To keep containment, the lower endpoint must be rounded **down** and the upper endpoint **up**: *outward rounding*. Hardware supports this through rounding modes, which the IEEE 1788-2015 interval standard builds on. Python cannot switch rounding modes, so the type below takes a cruder, still rigorous route. It computes each endpoint with round-to-nearest and then moves it one ulp outward with `math.nextafter`. Round-to-nearest is off by at most half an ulp, so one ulp outward always covers the true value. The price is intervals that are a little wider than necessary.

```
# interval.py - a minimal outward-rounded interval type on IEEE doubles
import math
from fractions import Fraction
from dataclasses import dataclass

down = lambda x: math.nextafter(x, -math.inf)   # one ulp toward -inf
up = lambda x: math.nextafter(x, math.inf)      # one ulp toward +inf


@dataclass(frozen=True)
class I:
    lo: float
    hi: float

    def __post_init__(self):
        if not self.lo <= self.hi:
            raise ValueError(f"empty interval [{self.lo}, {self.hi}]")

    @staticmethod
    def of(x) -> "I":
        """Enclose a number given as int or decimal string ("0.1") rigorously."""
        f = float(x)
        exact = Fraction(f) == Fraction(str(x))     # is the double *equal* to the real?
        return I(f, f) if exact else I(down(f), up(f))

    def __add__(s, o): o = _i(o); return I(down(s.lo + o.lo), up(s.hi + o.hi))
    __radd__ = __add__
    def __sub__(s, o): o = _i(o); return I(down(s.lo - o.hi), up(s.hi - o.lo))
    def __rsub__(s, o): return _i(o) - s
    def __neg__(s): return I(-s.hi, -s.lo)

    def __mul__(s, o):
        o = _i(o)
        p = [s.lo * o.lo, s.lo * o.hi, s.hi * o.lo, s.hi * o.hi]
        return I(down(min(p)), up(max(p)))
    __rmul__ = __mul__

    def __truediv__(s, o):
        o = _i(o)
```

```
        if o.lo <= 0 <= o.hi:
            raise ZeroDivisionError(f"divisor {o} contains 0")
        q = [s.lo / o.lo, s.lo / o.hi, s.hi / o.lo, s.hi / o.hi]
        return I(down(min(q)), up(max(q)))

    def sq(s) -> "I":
        """x**2 as one operation: never negative, unlike x*x on [-1, 2]."""
        a, b = abs(s.lo), abs(s.hi)
        lo = 0.0 if s.lo <= 0 <= s.hi else min(a, b) ** 2
        return I(down(lo) if lo else 0.0, up(max(a, b) ** 2))

    @property
    def mid(s): return s.lo + (s.hi - s.lo) / 2
    @property
    def width(s): return s.hi - s.lo
    def __contains__(s, x):                      # x may be a Fraction: an exact real
        return Fraction(s.lo) <= Fraction(x) <= Fraction(s.hi)
    def __repr__(s): return f"[{s.lo:.17g}, {s.hi:.17g}]"


def _i(x) -> I:
    return x if isinstance(x, I) else I.of(x)
```

Two details matter. `I.of("0.1")` takes the **decimal string**: the double nearest to 0.1 is not 0.1, so the type widens it to the two neighbouring doubles. An integer like `I.of(3)` is exact and stays a point. Membership (`in`) compares with `Fraction`, so you can ask whether an *exact* real number such as 3/10 is enclosed, not only a double near it.

2. Enclosures you can trust, and the dependency problem

```
# demo_basic.py - enclosures you can trust, and the dependency problem
from fractions import Fraction
from interval import I

tenth = I.of("0.1")
print("0.1 * 3 in doubles:", 0.1 * 3)
print("enclosure of 0.1  :", tenth)
print("3 * [0.1]         :", 3 * tenth)
print("contains 3/10?    :", Fraction(3, 10) in 3 * tenth)

x = I(0.0, 1.0)
print("\nx = [0, 1]")
print("x - x             :", x - x, "   (true range: [0, 0])")
print("x * (1 - x)       :", x * (1 - x), "   (true range: [0, 0.25])")
print("0.25 - (x-0.5)^2  :", 0.25 - (x - 0.5).sq(), "  (same function, rewritten)")
```

```
0.1 * 3 in doubles: 0.30000000000000004
enclosure of 0.1  : [0.099999999999999992, 0.10000000000000002]
3 * [0.1]         : [0.29999999999999993, 0.3000000000000001]
contains 3/10?    : True

x = [0, 1]
x - x             : [-1.0000000000000002, 1.0000000000000002]    (true range: [0, 0])
x * (1 - x)       : [-9.8813129168249309e-324, 1.0000000000000004]    (true range: [0, 0.25]
0.25 - (x-0.5)^2  : [-1.6653345369377351e-16, 0.25000000000000006]   (same function, rewritt
```

The first block is the good news. The double `0.30000000000000004` is simply wrong in its last digit, and nothing flags it. The interval contains the exact $3/10$, and its width (1.7×10^{-16}) is an honest error bar.

The second block is the bad news. `x - x` should be 0, but interval arithmetic treats the two occurrences of `x` as **independent** values anywhere in $[0, 1]$. So it returns $[-1, 1]$. This is the **dependency problem**: every repeated variable can inflate the result. $x(1-x)$ is enclosed by $[0, 1]$, four times wider than its true range $[0, 0.25]$. Rewriting the same function so that x appears **once**, as $0.25 - (x - 0.5)^2$, and using a dedicated square (which knows $x^2 \geq 0$) gives the true range up to rounding.

> **Rule.** The width of an interval result depends on *how the expression is written*, not only on the function. Expressions in which each variable occurs once give exact ranges in exact arithmetic (a theorem of Moore). Rewrite towards that form when you can.

3. Subdivision: buying tightness with work

When rewriting is not possible, split the domain, enclose each piece, and take the hull. The overestimate of a *natural* interval extension shrinks linearly with the piece width h. The method is first-order:

```
# demo_subdivide.py - tighten a range enclosure by splitting the domain
from interval import I

def f(x: I) -> I:          # f(x) = x(1-x), written the "bad" way on purpose
    return x * (1 - x)

def enclose(lo: float, hi: float, pieces: int) -> I:
    h = (hi - lo) / pieces
    parts = [f(I(lo + k * h, lo + (k + 1) * h if k < pieces - 1 else hi)) for k in range(pie
    return I(min(p.lo for p in parts), max(p.hi for p in parts))

for n in (1, 2, 4, 16, 256, 4096):
    r = enclose(0.0, 1.0, n)
    print(f"{n:5d} pieces: {r}   overestimate of max = {r.hi - 0.25:.2e}")
```

```
    1 pieces: [-9.8813129168249309e-324, 1.0000000000000004]   overestimate of max = 7.50e-0
    2 pieces: [-9.8813129168249309e-324, 0.50000000000000022]   overestimate of max = 2.50e-
    4 pieces: [-9.8813129168249309e-324, 0.37500000000000017]   overestimate of max = 1.25e-
   16 pieces: [-9.8813129168249309e-324, 0.28125000000000011]   overestimate of max = 3.13e-
  256 pieces: [-9.8813129168249309e-324, 0.25195312500000011]   overestimate of max = 1.95e-
 4096 pieces: [-9.8813129168249309e-324, 0.25012207031250011]   overestimate of max = 1.22e-
```

Doubling the pieces halves the overestimate: $O(h)$. In one dimension that is affordable. In d dimensions the number of boxes grows like h^{-d}, which is why serious interval codes use better extensions. The **mean-value form** $F(X) = f(m) + F'(X)(X - m)$ has an overestimate of order $O(h^2)$. Taylor models go further. Global optimisation codes use branch-and-bound to split only the boxes that might hold the optimum.

4. Interval Newton: proving that a root exists

Floating-point Newton finds *a number* where f looks small. Interval Newton proves a statement. Let m be the midpoint of X and define

$$N(X) \;=\; m - \frac{f(m)}{F'(X)}.$$

Two facts follow from the mean value theorem:

- every root of f in X also lies in $N(X)$, so $X \leftarrow X \cap N(X)$ never loses a root, and an empty intersection **proves** that there is no root in X;
- if $N(X)$ lies **strictly inside** X, then f has **exactly one** root in X.

```
# demo_newton.py - prove that x^2 = 2 has exactly one root in [1, 2], and pin it down
from interval import I

def f(x: I) -> I:  return x.sq() - 2
def df(x: I) -> I: return 2 * x

X = I(1.0, 2.0)
for step in range(1, 7):
    m = I(X.mid, X.mid)                    # a point, but evaluated with intervals
    N = m - f(m) / df(X)                   # interval Newton operator
    if N.hi < X.lo or N.lo > X.hi:
        print("no root in", X); break
    proved = X.lo < N.lo and N.hi < X.hi    # N(X) strictly inside X  =>  unique root
    X = I(max(X.lo, N.lo), min(X.hi, N.hi))
    print(f"step {step}: {X}  width={X.width:.1e}  unique-root proof: {proved}")

from fractions import Fraction as Q
print("lo^2 < 2 < hi^2 exactly?", Q(X.lo) ** 2 < 2 < Q(X.hi) ** 2)
```

```
step 1: [1.3749999999999996, 1.4375000000000004]  width=6.3e-02  unique-root proof: True
step 2: [1.4140624999999998, 1.414417613636364]  width=3.6e-04  unique-root proof: True
step 3: [1.414213559294524, 1.4142135659471793]  width=6.7e-09  unique-root proof: True
step 4: [1.4142135623730947, 1.4142135623730956]  width=8.9e-16  unique-root proof: True
step 5: [1.4142135623730947, 1.4142135623730954]  width=6.7e-16  unique-root proof: False
step 6: [1.4142135623730947, 1.4142135623730954]  width=6.7e-16  unique-root proof: False
lo^2 < 2 < hi^2 exactly? True
```

Step 1 already proves that $[1, 2]$ holds exactly one root of $x^2 - 2$. The widths after that go $6 \times 10^{-2} \to 4 \times 10^{-4} \to 7 \times 10^{-9} \to 9 \times 10^{-16}$: the number of correct digits doubles each step, the familiar quadratic convergence of Newton's method, now with a certificate attached. At step 5 the interval is a few ulps wide, and outward rounding cannot shrink it further. "Proof: False" there only means that $N(X)$ is no longer *strictly* inside the stalled interval. The uniqueness proof from step 1 still holds. The last line checks the final enclosure with exact rational arithmetic.

5. Rump's example: two confident wrong answers

S. M. Rump (1988) built an expression that defeats ordinary floating point at almost any precision. With $a = 77617$, $b = 33096$:

$$f(a, b) = 333.75\, b^6 + a^2 \left(11 a^2 b^2 - b^6 - 121 b^4 - 2\right) + 5.5\, b^8 + \frac{a}{2b}.$$

The large terms cancel to exactly -2, leaving $f = -2 + a/(2b)$. The intermediate terms are about 10^{36}, so they must be carried to well over 30 significant digits before the cancellation leaves anything correct. The run below shows that 30 digits are not enough and 37 are.

```python
# demo_rump.py - Rump's example: confident wrong answers, and an interval that refuses to li
from fractions import Fraction
from mpmath import iv, mp, mpf

a, b = 77617, 33096

def rump(a, b, half, c1, c2):
    return c1 * b**6 + a**2 * (11 * a**2 * b**2 - b**6 - 121 * b**4 - 2) + c2 * b**8 + a / (

exact = rump(Fraction(a), Fraction(b), 2, Fraction(1335, 4), Fraction(11, 2))
print("exact (Fraction)  :", float(exact))
print("float64           :", rump(float(a), float(b), 2.0, 333.75, 5.5))

for dps in (15, 30, 37, 40):
    mp.dps = dps
    print(f"mpmath  {dps:2d} digits :", mp.nstr(rump(mpf(a), mpf(b), 2, mpf("333.75"), mpf("

for dps in (15, 30, 37, 40):
    iv.dps = dps
```

```
    r = rump(iv.mpf(a), iv.mpf(b), 2, iv.mpf("333.75"), iv.mpf("5.5"))
    print(f"iv      {dps:2d} digits : width {float(r.delta.b):9.2e}  ->", iv.nstr(r, 10))
```

```
exact (Fraction)  : -0.8273960599468214
float64           : -1.1805916207174113e+21
mpmath  15 digits : -1.18059162072e+21
mpmath  30 digits : 1.17260394005
mpmath  37 digits : -0.827396059947
mpmath  40 digits : -0.827396059947
iv      15 digits : width  1.06e+22  -> [-5.902958104e+21, 4.722366483e+21]
iv      30 digits : width  2.10e+06  -> [-1048574.827, 1048577.173]
iv      37 digits : width  2.35e-38  -> [-0.8273960599, -0.8273960599]
iv      40 digits : width  2.30e-41  -> [-0.8273960599, -0.8273960599]
```

Read the middle lines. At 30 digits, plain multi-precision arithmetic returns `1.17260394005`: wrong sign, wrong magnitude, and twelve plausible-looking digits. "Recompute at higher precision and see if the answer changes" would have caught it here, but only because we happened to try 37 next. The interval at 30 digits spans two million units around zero, which says plainly that the digits are meaningless. At 37 digits it collapses to a width of 2×10^{-38} around -0.8273960599, and that answer is now certified.

`mpmath.iv` is a ready-made, arbitrary-precision interval context. It rounds outward properly at every operation and has interval versions of the elementary functions (`iv.exp`, `iv.sin`, …). For production work in compiled languages, see the libraries listed under Sources.

6. Where certified computation is used

- **Computer-assisted proofs.** W. Tucker's 2002 proof that the Lorenz attractor exists combined normal-form theory with rigorous interval ODE integration. Hales' proof of the Kepler conjecture checked thousands of nonlinear inequalities with interval arithmetic.
- **Global optimisation and constraint solving.** Interval branch-and-bound returns an enclosure of the *global* minimum, and boxes proven to contain no minimiser are discarded.
- **Robust geometry.** Exact-predicate libraries filter with intervals first and fall back to exact arithmetic only when the interval straddles zero.
- **Sanity checking.** Running a numerical kernel once in interval mode tells you how many of its output digits survive rounding, which is cheaper than an error analysis by hand.

7. Pitfalls

1. **Treating a wide interval as a bug.** Width is information: the computation, *as written*, cannot be more precise. Rewrite, subdivide, or raise the precision. Do not trim the interval.
2. **Unrigorous inputs.** `I(0.1, 0.1)` encloses the double `0.1`, not the real number one tenth. Enclose literals from their exact decimal or rational value (`I.of("0.1")`, `iv.mpf("0.1")`).
3. **Forgetting dependency.** `x*x` on $[-1, 2]$ gives $[-2, 4]$. A real square gives $[0, 4]$. Use dedicated power functions, and write each variable once where possible.

4. **Division by an interval containing zero.** The result is unbounded or a union of two pieces. Basic interval types raise an error, and *extended* interval arithmetic (IEEE 1788) returns the pieces.
5. **Mixing in plain floats mid-expression.** One `math.sin(x.mid)` silently drops rigour for everything downstream. Every operation on the path must be an interval operation.
6. **Expecting speed.** Each multiply is four multiplies plus min/max plus outward rounding, and libraries that switch rounding modes stall the pipeline. Use intervals to *certify*, not as the default arithmetic.

8. Checkpoint

1. Why is $[a, b] - [c, d] = [a - d,\ b - c]$ and not $[a - c,\ b - d]$?
2. Give an expression for $x^2 - 2x$ on $[0, 2]$ whose natural interval extension is exact.
3. In §4, why is an *empty* intersection $X \cap N(X)$ a proof and not merely a failure?
4. Why did the 30-digit interval in §5 come out roughly symmetric around zero?

Exercises

1. **Mean-value form.** Add `mean_value(f, df, X)` that computes $f(m) + F'(X)(X - m)$ and repeat §3 for $f(x) = x(1 - x)$. Confirm that halving the piece width now divides the overestimate by about four.
2. **Root isolation.** Apply interval Newton with bisection to $f(x) = \sin x - x/3$ on $[-4, 4]$ with `mpmath.iv`. Bisect any box that is neither proven root-free nor proven to hold a unique root. How many roots are certified, and how many boxes did you need?
3. **Find the precision cliff.** In §5, find the smallest `iv.dps` for which the interval excludes zero. Compare it with the number of digits in b^8.
4. **Kahan versus intervals.** Sum `[I.of("0.1")] * 10**6` with the interval type. Compare the width of the result with the error of naive and Kahan float summation of the same series.

Summary

Interval arithmetic computes sets that are guaranteed to contain the exact result, so a narrow interval is a certificate and a wide one is an honest refusal. Containment is easy. Tightness is the craft: write each variable once, subdivide, use centred or mean-value forms, and raise the precision when cancellation needs it. Interval Newton turns root-finding into proof: it shows existence and uniqueness and converges quadratically. Rump's example shows why this matters. Floating point at 16 and 30 digits gives two different, equally confident wrong answers, while the interval says exactly how little it knows.

Sources (for further reading)

- R. E. Moore, R. B. Kearfott, M. J. Cloud, *Introduction to Interval Analysis*, SIAM, 2009: containment theorem, single-use expressions, interval Newton.

- IEEE Std 1788-2015, *Standard for Interval Arithmetic* (and the simplified IEEE 1788.1-2017).
- S. M. Rump, "Algorithms for verified inclusions: theory and practice", in *Reliability in Computing*, Academic Press, 1988. The example in §5 is the commonly used corrected form (Loh & Walster, 2002).
- W. Tucker, "A rigorous ODE solver and Smale's 14th problem", *Foundations of Computational Mathematics* 2 (2002).
- mpmath documentation, "Arbitrary-precision interval arithmetic (iv)": https://mpmath.org/doc/current/contexts.html.
- Python documentation, `math.nextafter` (3.9+) and `fractions.Fraction`: https://docs.python.org/3/library/math.html#math.nextafter.
- Libraries for compiled languages: Arb/FLINT (ball arithmetic), MPFI, Boost.Interval, and the C++ `libieeep1788` reference implementation.

The Ozaki Scheme: Emulating FP64 GEMM with Integer Slices

Matrix multiplication is the operation scientific computing and machine learning both spend their lives inside: $C \leftarrow AB$ for $A \in \mathbb{R}^{m\times q}$, $B \in \mathbb{R}^{q\times n}$. In IEEE binary64 that product is DGEMM, and for decades it was a hardware primitive. On current AI-oriented accelerators it is not. Low-precision matrix engines (INT8, FP8, FP16, BF16) are one to two orders of magnitude faster than native FP64, and some parts have started to treat FP64 as something to *synthesize*. The **Ozaki scheme** is the mathematics of that synthesis: split each floating-point matrix into a handful of short integer slices so that every slice-pair product is *exact*, accumulate those products in integer arithmetic, and recombine.

This chapter teaches the 2012 error-free transformation (now called Ozaki scheme I), the INT8 form that Ootomo, Ozaki, and Yokota put on tensor cores, and the 2025/26 CRT variant (Ozaki scheme II). It then implements scheme I in NumPy with `int8` slices and `int32`/`int64` accumulation, and compares it with `float64` GEMM, `float32` GEMM, and an `mpmath` reference on a small case.

All listings are complete Python files. They were run with **Python 3.13.5**, **NumPy 2.5.3**, and **mpmath 1.3.0**. Outputs are pasted unedited. This machine has no GPU and no vendor BLAS beyond NumPy; hardware throughputs quoted in §1 come from the cited papers and from the CUDA 13.4 release notes, not from a run here.

1. Why emulate binary64 matrix products

A binary64 number carries a 53-bit significand (52 stored bits plus the hidden 1) and an 11-bit exponent field. The unit roundoff is $u = 2^{-53} \approx 1.11 \times 10^{-16}$. Native FP64 matrix units used to be a selling point of scientific GPUs. The accelerators built for training and inference put that budget into low-precision tensor cores instead.

Ozaki, Uchino, and Imamura (Table 1 of the Ozaki scheme II paper, arXiv:2504.08009v4, citing NVIDIA's 2024 dense-data specifications) list a Blackwell B200 at 40 TFLOPS of FP64 and 4500 TOPS of INT8 tensor-core throughput: a ratio of about 110. CUDA Toolkit 13.4's cuBLAS release notes then report *emulated* DGEMM — Ozaki I or II, chosen automatically — reaching up to 175 TFLOPS on B200. Those figures were not measured on this box. The point they establish is the one this chapter needs: if you can turn one FP64 GEMM into a few dozen *exact* INT8 GEMMs, the INT8 units win on throughput, and the mathematics decides whether the answer is still a binary64 answer.

Iterative refinement recovers FP64 *linear solves* from a cheap factorization. It does not help when the application needs every matrix product itself to be accurate: electronic-structure contractions, spectral methods, some eigenvalue solvers. Those codes want DGEMM. The Ozaki scheme is DGEMM built out of integer GEMMs.

2. Mental model: error-free splitting, then exact products

Write a working-precision matrix as a sum of slices whose significands are short:

$$A = \sum_{i=1}^{k} 2^{-7i}\, 2^{e_A} A_i + R_A, \qquad B = \sum_{j=1}^{k} 2^{-7j}\, 2^{e_B} B_j + R_B,$$

where each A_i, B_j has **integer** entries that fit in signed 8-bit storage, e_A is a diagonal of per-row powers of two, e_B is per-column, and the remainders R_A, R_B are what you drop when you stop after k slices. The product expands as

$$AB \;=\; 2^{e_A} \left(\sum_{i=1}^{k} \sum_{j=1}^{k} 2^{-7(i+j)}\, A_i B_j \right) 2^{e_B} \;+\; \text{dropped terms.}$$

Two facts make this useful.

1. **Each $A_i B_j$ is an integer matrix product.** INT8 × INT8 with INT32 (or INT64) accumulation is exact as long as the inner dimension q is small enough that the accumulator cannot wrap. There is no rounding inside the GEMM.
2. **The weights are powers of two.** Scaling by $2^{\pm 7(i+j)}$ and by 2^{e_A}, 2^{e_B} is exact in binary floating-point (no overflow or underflow). The only rounding in the whole algorithm is the final FP64 sum of the weighted integer products, plus whatever you chose to drop.

The original 2012 paper (Ozaki, Ogita, Oishi, Rump) did the same split in *floating-point* GEMM: choose the slice width so that $2\beta + \lceil \log_2 q \rceil$ bits fit in the 53-bit significand, and each $A_i B_j$ is error-free in binary64. The INT8 form, due to Ootomo, Ozaki, and Yokota (IJHPCA 2024), moves that exactness into the integer tensor core and stores the slices in one byte. Later papers call this **Ozaki scheme I**.

The number of products is k^2 if you keep every pair, or

$$\frac{k(k+1)}{2}$$

if you keep only the most significant pairs $i + j \le k + 1$ (1-based). That triangle is the usual production choice: the discarded products sit below working precision when k is large enough for the data.

3. Binary64 layout, and what "exact integer GEMM" means

```python
# demo_fp64_bits.py - IEEE 754 binary64 layout of a few constants
import struct

def unpack64(x: float):
    u = struct.unpack(">Q", struct.pack(">d", x))[0]
```

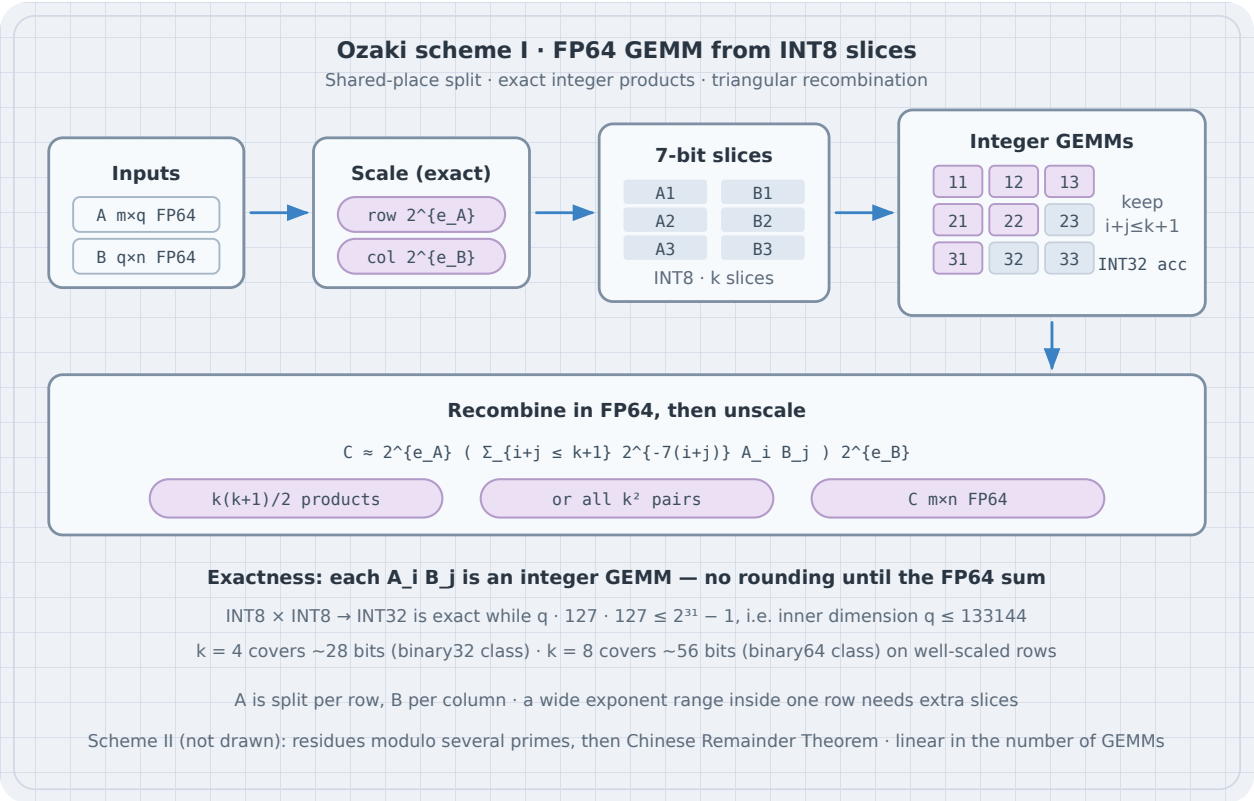


Figure 1: Ozaki scheme I: scale, peel INT8 slices, exact integer GEMMs, recombine

```
    sign = (u >> 63) & 1
    exp = (u >> 52) & 0x7FF
    man = u & ((1 << 52) - 1)
    return u, sign, exp, man

for label, x in (
    ("1.0", 1.0),
    ("-2.5", -2.5),
    ("0.1", 0.1),
    ("2**53", 2.0**53),
    ("2**53+1", 2.0**53 + 1.0),
):
    u, s, e, m = unpack64(x)
    bias = 1023
    print(f"{label:10} value={x:24.1f}  sign={s}  E={e - bias:4d}  "
          f"frac=0x{m:013x}")
print("(2**53 + 1) == 2**53 in binary64?", (2.0**53 + 1.0) == 2.0**53)
import numpy as np
print("numpy finfo(float64).eps =", np.finfo(np.float64).eps, " = 2**-52")
print("unit roundoff u = 2**-53 =", 2.0**-53)
```

```
python3 demo_fp64_bits.py
```

```
1.0        value=                     1.0  sign=0  E=   0  frac=0x0000000000000
-2.5       value=                    -2.5  sign=1  E=   1  frac=0x4000000000000
0.1        value=                     0.1  sign=0  E=  -4  frac=0x999999999999a
2**53      value=      9007199254740992.0  sign=0  E=  53  frac=0x0000000000000
2**53+1    value=      9007199254740992.0  sign=0  E=  53  frac=0x0000000000000
(2**53 + 1) == 2**53 in binary64? True
numpy finfo(float64).eps = 2.220446049250313e-16  = 2**-52
unit roundoff u = 2**-53 = 1.1102230246251565e-16
```

The last two lines of the bit dump are the same pattern. Above 2^{53}, binary64 cannot represent consecutive integers, so $2^{53}+1$ collapses onto 2^{53}. Integer GEMM does not have that problem: every integer in the accumulator's range is representable. The hardware constraint is the *width* of the accumulator.

Signed INT8 holds $\{-128, \dots, 127\}$. A product of two such entries is at most $127 \cdot 127 = 16129$. Summing q of them into a signed INT32 (max $2^{31}-1 = 2147483647$) is exact provided

$$q \cdot 127 \cdot 127 \;\le\; 2^{31}-1 \qquad \Rightarrow \qquad q \;\le\; 133144.$$

Ootomo et al. take a slightly different form, shrinking the slice width β when q is huge:

$$\beta \;=\; \min(7,\; \lfloor (31 - \log_2 q)/2 \rfloor),$$

so that $q \cdot 2^{2\beta}$ stays inside 31 bits. For every inner dimension in this chapter, $q = 8$ or $q = 3$, so $\beta = 7$ and INT32 is far from wrapping; the listings still accumulate in `int64` as well and check that the two agree.

4. Shared-place splitting

Elementwise splitting (double-double, "cast to FP16 and subtract") peels leading bits of *each entry independently*. Ozaki splitting is **shared-place**: every entry of a *row of A* is scaled by the same power of two, chosen from that row's maximum magnitude, and then sliced at the same binary place. Columns of B are scaled independently. Two consequences:

- every product $A_i[r,:]\,B_j[:,c]$ has a common scale, so the integer GEMM is a homogeneous inner product;
- a row whose entries span many orders of magnitude spends slices on the large entries, and the small ones may not appear until late. That is the trap in §7.

The scaling is $e = \lceil \log_2 M \rceil + 1$ with $M = \max|x|$ along the row (or column), so the scaled values lie in $(-1/2, 1/2]$. Then $\mathrm{round}(R \cdot 2^7)$ fits in $\{-64, \dots, 64\} \subset$ INT8. Multiplying or dividing by a power of two is exact in binary64.

```python
# ozaki.py - Ozaki scheme I: FP64 GEMM via int8 slices and integer accumulation
"""Ozaki scheme I (shared-place slicing) for binary64 matrix multiplication.

Each row of A and each column of B is scaled by a power of two (exact),
then the scaled mantissa is peeled into 7-bit integer slices. Every
slice-pair product is an exact integer GEMM; the floating-point answer
is the scaled sum of those products.
"""
from __future__ import annotations

import numpy as np

BITS = 7  # bits per slice; 2**7 = 128 fits in signed int8 after |R| <= 0.5 scaling


def _row_col_exponents(M: np.ndarray, axis: int) -> np.ndarray:
    """Power-of-two exponents so that |M| * 2^{-e} <= 1/2 along `axis`.

    axis=1: one exponent per row (for A). axis=0: one exponent per column (for B).
    """
    absmax = np.max(np.abs(M), axis=axis, keepdims=True)
    absmax = np.where(absmax == 0.0, 1.0, absmax)
    # ceil(log2(absmax)) + 1  =>  scaled values in (-1/2, 1/2]
    e = np.ceil(np.log2(absmax)).astype(np.int64) + 1
    return e


def split_slices(
    M: np.ndarray, nslices: int, axis: int, bits: int = BITS
) -> tuple[list[np.ndarray], np.ndarray]:
    """Return (list of int8 slices, per-row-or-column exponents)."""
    M = np.asarray(M, dtype=np.float64)
    e = _row_col_exponents(M, axis)
    R = np.ldexp(M, -e)  # M * 2^{-e}, exact
    out: list[np.ndarray] = []
    for i in range(1, nslices + 1):
        P = np.rint(np.ldexp(R, bits * i))  # round(R * 2^{bits*i})
        # After the |R|<=1/2 scaling, |P| <= 2^{bits-1} = 64 for bits=7.
        if np.any(np.abs(P) > 127):
            raise OverflowError("slice escaped int8; scaling is wrong")
        out.append(P.astype(np.int8))
        R = R - np.ldexp(P, -bits * i)  # exact: P is integer, scale is 2^{-k}
    return out, e


def ozaki_gemm(
    A: np.ndarray,
```

```
    B: np.ndarray,
    nslices: int,
    *,
    bits: int = BITS,
    triangular: bool = True,
    acc_dtype=np.int64,
) -> np.ndarray:
    """Approximate A @ B by k (or k(k+1)/2) exact integer GEMMs.

    triangular=True keeps pairs (i, j) with i+j <= nslices+1 (1-based),
    which is k(k+1)/2 products: the most significant slice pairs.
    triangular=False uses all k^2 products.
    """
    A = np.asarray(A, dtype=np.float64)
    B = np.asarray(B, dtype=np.float64)
    As, ea = split_slices(A, nslices, axis=1, bits=bits)
    Bs, eb = split_slices(B, nslices, axis=0, bits=bits)
    m, n = A.shape[0], B.shape[1]
    C = np.zeros((m, n), dtype=np.float64)
    nprod = 0
    for i in range(nslices):
        jmax = (nslices - i) if triangular else nslices  # 1-based: i+1+j+1 <= k+1
        for j in range(jmax):
            P = As[i].astype(acc_dtype, copy=False) @ Bs[j].astype(acc_dtype, copy=False)
            C += np.ldexp(P.astype(np.float64), -bits * ((i + 1) + (j + 1)))
            nprod += 1
    # Unscale: C[r,c] *= 2^{ea[r]} * 2^{eb[c]}
    C = np.ldexp(C, ea + eb.reshape(1, -1))
    return C


def n_products(nslices: int, triangular: bool = True) -> int:
    k = nslices
    return k * (k + 1) // 2 if triangular else k * k
```

Peel π into eight slices. Eight $\times$ 7 bits is 56 bits, which covers the 53-bit significand, so the reconstruction is exact in binary64:

```
# demo_split_scalar.py - peel one binary64 number into 7-bit integer slices
import numpy as np
from ozaki import BITS, split_slices

x = np.float64(np.pi)
print("x =", np.format_float_scientific(x, unique=True, precision=17))
print("binary64 value of pi:", x)

# Treat x as a 1x1 matrix; split along axis=1 (the dummy column axis)
```

```
slices, e = split_slices(np.array([[x]]), nslices=8, axis=1)
print("per-row exponent e =", int(e[0, 0]), " so scaled R = x * 2^{-e} =", x * 2.0 ** -int(e
print()
print(f"{'i':>3} {'P_i (int8)':>12} {'weight 2^{-7i}':>16} {'term P*2^{-7i}*2^e':>24} {'runn
acc = 0.0
ee = int(e[0, 0])
for i, P in enumerate(slices, start=1):
    p = int(P[0, 0])
    term = np.ldexp(float(p), -BITS * i + ee)
    acc += term
    print(f"{i:3d} {p:12d} {2.0**(-BITS*i):16.3e} {term:24.17g} {acc:24.17g}")
print()
print("reconstruction - x =", acc - x)
print("|rel err| =", abs(acc - x) / abs(x))
print("slices as int8:", [int(P[0, 0]) for P in slices])
```

```
python3 demo_split_scalar.py
```

```
x = 3.141592653589793e+00
binary64 value of pi: 3.141592653589793
per-row exponent e = 3  so scaled R = x * 2^{-e} = 0.39269908169872414

  i   P_i (int8)   weight 2^{-7i}       term P*2^{-7i}*2^e              running sum
  1           50        7.812e-03                    3.125                    3.125
  2           34        6.104e-05           0.0166015625             3.1416015625
  3           -2        4.768e-07       -7.62939453125e-06       3.1415939331054688
  4          -43        3.725e-09  -1.2814998626708984e-06       3.1415926516056061
  5            9        2.910e-11   2.0954757928848267e-09       3.1415926537010819
  6          -61        2.274e-13  -1.1095835361629725e-10       3.1415926535901235
  7          -23        1.776e-15  -3.2684965844964609e-13       3.1415926535897967
  8          -32        1.388e-17  -3.5527136788005009e-15       3.1415926535897931

reconstruction - x = 0.0
|rel err| = 0.0
slices as int8: [50, 34, -2, -43, 9, -61, -23, -32]
```

After four slices the running sum already agrees with π to nine digits. Slice 8 contributes a few units in the last place and lands exactly on the binary64 value of π.

5. Worked example: a 2×2 product, by hand and by slices

Take

$$A = \begin{pmatrix} 3/2 & -1/4 \\ 3/4 & 2 \end{pmatrix}, \qquad B = \begin{pmatrix} 4 & 1/2 \\ -1 & 3 \end{pmatrix}.$$

Every entry is a small dyadic rational, so it is exact in binary64, and the product is exact in $\mathbb{Q}$:

$$AB = \begin{pmatrix} 25/4 & 0 \\ 1 & 51/8 \end{pmatrix} = \begin{pmatrix} 6.25 & 0 \\ 1 & 6.375 \end{pmatrix}.$$

```
# demo_2x2.py - a 2x2 product, slice by slice, compared with exact Fraction arithmetic
from fractions import Fraction
import numpy as np
from ozaki import BITS, ozaki_gemm, split_slices, n_products

A = np.array([[1.5, -0.25],
              [0.75, 2.0]], dtype=np.float64)
B = np.array([[4.0, 0.5],
              [-1.0, 3.0]], dtype=np.float64)

def frac_gemm(A, B):
    C = np.empty((2, 2), dtype=object)
    for i in range(2):
        for j in range(2):
            C[i, j] = sum(Fraction(A[i, t]) * Fraction(B[t, j]) for t in range(2))
    return C

C_exact = frac_gemm(A, B)
print("A =\n", A)
print("B =\n", B)
print("exact AB (Fraction):", [[C_exact[i, j] for j in range(2)] for i in range(2)])

As, ea = split_slices(A, nslices=3, axis=1)
Bs, eb = split_slices(B, nslices=3, axis=0)
print("row exponents e_A =", ea.ravel().tolist())
print("col exponents e_B =", eb.ravel().tolist())
print("A slices (int8):", [S.tolist() for S in As])
print("B slices (int8):", [S.tolist() for S in Bs])

print("non-zero integer GEMMs:")
for i, Ai in enumerate(As, 1):
    for j, Bj in enumerate(Bs, 1):
        P = Ai.astype(np.int64) @ Bj.astype(np.int64)
        if np.any(P):
            print(f"  A[{i}] @ B[{j}] =\n{P}   weight 2^{-BITS*(i+j)}")

C_tri = ozaki_gemm(A, B, nslices=3, triangular=True)
C_full = ozaki_gemm(A, B, nslices=3, triangular=False)
print("ozaki triangular (6 products):\n", C_tri)
print("ozaki full (9 products):\n", C_full)
print("float64 A@B:\n", A @ B)
```

```
print("max |ozaki_tri - exact| =",
      max(abs(float(C_tri[i, j]) - float(C_exact[i, j])) for i in range(2) for j in range(2)))
print("n_products tri/full =", n_products(3, True), n_products(3, False))
```

```
python3 demo_2x2.py
```

```
A =
 [[ 1.5  -0.25]
 [ 0.75  2.  ]]
B =
 [[ 4.   0.5]
 [-1.   3. ]]
exact AB (Fraction): [[Fraction(25, 4), Fraction(0, 1)], [Fraction(1, 1), Fraction(51, 8)]]
row exponents e_A = [2, 2]
col exponents e_B = [3, 3]
A slices (int8): [[[48, -8], [24, 64]], [[0, 0], [0, 0]], [[0, 0], [0, 0]]]
B slices (int8): [[[64, 8], [-16, 48]], [[0, 0], [0, 0]], [[0, 0], [0, 0]]]
non-zero integer GEMMs:
  A[1] @ B[1] =
[[3200    0]
 [ 512 3264]]   weight 2^-14
ozaki triangular (6 products):
 [[6.25  0.   ]
 [1.    6.375]]
ozaki full (9 products):
 [[6.25  0.   ]
 [1.    6.375]]
float64 A@B:
 [[6.25  0.   ]
 [1.    6.375]]
max |ozaki_tri - exact| = 0.0
n_products tri/full = 6 9
```

One slice of each matrix captures everything, because these dyadics have only a few significant bits. The $(1,1)$ entry of the integer product is $48 \cdot 64 + (-8) \cdot (-16) = 3200$. Unscaling is exact:

$$3200 \cdot 2^{-14} \cdot 2^{2} \cdot 2^{3} \;=\; 3200 \cdot 2^{-9} \;=\; 3200/512 \;=\; 6.25.$$

Slices 2 and 3 are the zero matrix, so the triangular six-product sum and the full nine-product sum agree with each other, with `float64` GEMM, and with `Fraction`.

6. Worked example: 8×8 random GEMM against mpmath

The 2×2 case is too kind. Draw A, B with i.i.d. standard-normal binary64 entries, compute a reference product in `mpmath` at 50 decimal digits (the inputs are still binary64; the reference is the

exact real product of those dyadics, rounded once to binary64 at the end), and compare Ozaki, NumPy `float64`, and NumPy `float32`.

```
# demo_compare.py - Ozaki vs float64 vs float32 vs mpmath on a small random GEMM
import numpy as np
from mpmath import mpf, mp
from ozaki import ozaki_gemm, n_products

mp.dps = 50  # ~166 bits; enough to treat the reference as exact for binary64 inputs


def mp_gemm(A: np.ndarray, B: np.ndarray) -> np.ndarray:
    m, k = A.shape
    n = B.shape[1]
    C = np.empty((m, n), dtype=np.float64)
    for i in range(m):
        for j in range(n):
            s = mpf(0)
            for t in range(k):
                s += mpf(A[i, t]) * mpf(B[t, j])
            C[i, j] = float(s)
    return C


def rel_max(C, Cref):
    return float(np.max(np.abs(C - Cref)) / np.max(np.abs(Cref)))


rng = np.random.default_rng(0)
m = k = n = 8
A = rng.standard_normal((m, k))
B = rng.standard_normal((k, n))

print(f"shape {m}x{k}  times  {k}x{n},  seed=0,  N(0,1) binary64 entries")
print("mpmath dps =", mp.dps)
Cref = mp_gemm(A, B)
C64 = A @ B
C32 = (A.astype(np.float32) @ B.astype(np.float32)).astype(np.float64)

print(f"{'method':<28} {'max rel err vs mp':>18} {'nprod':>8}")
print(f"{'mpmath reference':<28} {0.0:18.6e} {'-':>8}")
print(f"{'numpy float64 GEMM':<28} {rel_max(C64, Cref):18.6e} {1:8d}")
print(f"{'numpy float32 GEMM':<28} {rel_max(C32, Cref):18.6e} {1:8d}")
for s in (2, 3, 4, 5, 6, 7, 8, 9):
    for tri in (True, False):
        C = ozaki_gemm(A, B, nslices=s, triangular=tri)
        tag = "tri" if tri else "full"
```

```
        print(f"{'ozaki k='+str(s)+' '+tag:<28} {rel_max(C, Cref):18.6e} {n_products(s, tri)
```

```
python3 demo_compare.py
```

```
shape 8x8  times  8x8,  seed=0,  N(0,1) binary64 entries
mpmath dps = 50
method                          max rel err vs mp    nprod
mpmath reference                     0.000000e+00        -
numpy float64 GEMM                   5.306960e-17        1
numpy float32 GEMM                   1.040517e-07        1
ozaki k=2 tri                        1.724797e-04        3
ozaki k=2 full                       1.207295e-04        4
ozaki k=3 tri                        1.552715e-06        6
ozaki k=3 full                       7.117836e-07        9
ozaki k=4 tri                        2.774233e-08       10
ozaki k=4 full                       6.648553e-09       16
ozaki k=5 tri                        1.521374e-10       15
ozaki k=5 full                       5.074156e-11       25
ozaki k=6 tri                        7.847668e-13       21
ozaki k=6 full                       5.155181e-13       36
ozaki k=7 tri                        1.042818e-14       28
ozaki k=7 full                       3.184176e-15       49
ozaki k=8 tri                        2.122784e-16       36
ozaki k=8 full                       2.122784e-16       64
ozaki k=9 tri                        2.122784e-16       45
ozaki k=9 full                       2.122784e-16       81
```

Read the table against the bit budget. Each slice is 7 bits, so $k = 4$ covers 28 bits (binary32 has a 24-bit significand) and lands at 10^{-8}, matching the `float32` GEMM's 1.04×10^{-7} to order of magnitude and then beating it. $k = 7$ covers 49 bits, short of binary64's 53, and the error is 10^{-14} — close, not there. $k = 8$ covers 56 bits and the error drops to 2.1×10^{-16}, the same ulp-scale as native `float64` GEMM (5.3×10^{-17}). Extra slices past 8 do not help: recombination is already in binary64, and the leftover products sit below a ulp. The triangular sum at $k = 8$ equals the full k^2 sum, which is the whole point of dropping $i + j > k + 1$.

On this 8×8 case, `int32` and `int64` accumulation produce bitwise-identical C (`max |C_int32 - C_int64| = 0.0`). The inner dimension is 8, and $8 \cdot 127 \cdot 127 = 129032$, well below $2^{31} - 1$.

7. The trap: a wide exponent range inside one row

Shared-place splitting scales a *whole row* by that row's largest magnitude. If one entry is 2^{40} and the others are 1, the ones live 40 bits below the spike. They do not appear until slice $\lceil 40/7 \rceil \approx 6$. Construct a product where those hidden ones are the entire answer: put the spike in $A[0, 0]$, put a *zero* in $B[0, 0]$, and fill the rest with ones. Then $C[0, 0] = 7$, and the 2^{40} never contributes.

```
# demo_trap.py - shared-place splitting: a huge entry hides small ones in the same row
import numpy as np
from ozaki import ozaki_gemm, split_slices

# Row 0 of A is [2^40, 1, 1, ..., 1].  Column 0 of B is [0, 1, 1, ..., 1]^T.
# The huge entry of A is multiplied by zero, so the true C[0,0] equals n-1.
# Shared-place slicing of row 0 is pegged to 2^40, so the ones never appear
# until slice ~ 40/7  6.
n = 8
A = np.ones((n, n), dtype=np.float64)
A[0, 0] = np.ldexp(1.0, 40)          # 2^40
B = np.ones((n, n), dtype=np.float64)
B[0, 0] = 0.0                        # kill the huge term in C[0,0]

C_true = A @ B                       # C[0,0] = 0*2^40 + 1*(n-1) = n-1 = 7
print("A[0] =", A[0].tolist())
print("B[:,0] =", B[:, 0].tolist())
print("true C[0,0] = A[0] · B[:,0] =", C_true[0, 0],
      "  (the 2^40 is multiplied by 0)")
print()

As, ea = split_slices(A, nslices=8, axis=1)
print("row exponents of A:", ea.ravel().tolist())
print("row 0 slices (the ones sit 40 bits below the spike):")
for i, S in enumerate(As, 1):
    print(f"  slice {i}: {S[0].tolist()}")
print()

print(f"{'k':>3} {'C[0,0] ozaki':>16} {'abs err':>12} {'rel err of C[0,0]':>20}")
for k in range(1, 9):
    C = ozaki_gemm(A, B, nslices=k, triangular=True)
    err = abs(C[0, 0] - C_true[0, 0])
    rel = err / abs(C_true[0, 0])
    print(f"{k:3d} {C[0,0]:16.6g} {err:12.4e} {rel:20.4e}")

print()
print("float64 GEMM C[0,0] =", (A @ B)[0, 0])

# Contrast: same magnitudes, but the spike is in a *different* row, so row 0
# is split on its own O(1) scale and one slice already gets C[0,0] right.
A2 = np.ones((n, n), dtype=np.float64)
A2[1, 0] = np.ldexp(1.0, 40)         # spike in row 1, not row 0
C2 = ozaki_gemm(A2, B, nslices=1, triangular=True)
print("spike moved to row 1; ozaki k=1 gives C[0,0] =", C2[0, 0],
      "  (row 0 is now scaled on its own)")
```

```
python3 demo_trap.py
```

```
A[0] = [1099511627776.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
B[:,0] = [0.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
true C[0,0] = A[0] · B[:,0] = 7.0   (the 2^40 is multiplied by 0)

row exponents of A: [41, 1, 1, 1, 1, 1, 1, 1]
row 0 slices (the ones sit 40 bits below the spike):
  slice 1: [64, 0, 0, 0, 0, 0, 0, 0]
  slice 2: [0, 0, 0, 0, 0, 0, 0, 0]
  slice 3: [0, 0, 0, 0, 0, 0, 0, 0]
  slice 4: [0, 0, 0, 0, 0, 0, 0, 0]
  slice 5: [0, 0, 0, 0, 0, 0, 0, 0]
  slice 6: [0, 2, 2, 2, 2, 2, 2, 2]
  slice 7: [0, 0, 0, 0, 0, 0, 0, 0]
  slice 8: [0, 0, 0, 0, 0, 0, 0, 0]

  k     C[0,0] ozaki      abs err    rel err of C[0,0]
  1                0   7.0000e+00           1.0000e+00
  2                0   7.0000e+00           1.0000e+00
  3                0   7.0000e+00           1.0000e+00
  4                0   7.0000e+00           1.0000e+00
  5                0   7.0000e+00           1.0000e+00
  6                7   0.0000e+00           0.0000e+00
  7                7   0.0000e+00           0.0000e+00
  8                7   0.0000e+00           0.0000e+00

float64 GEMM C[0,0] = 7.0
spike moved to row 1; ozaki k=1 gives C[0,0] = 7.0   (row 0 is now scaled on its own)
```

For $k \le 5$ the scheme returns $C[0,0] = 0$, a relative error of 1, with no warning. Slice 6 is the first to carry the ones (as the integer 2, because $1 \cdot 2^{-41} \cdot 2^{42} = 2$), and from that slice on the answer is exact. Native `float64` GEMM gets 7 immediately: it does not share a splitting place across the row. Moving the spike to *another* row restores $k = 1$ accuracy for $C[0,0]$, which is the other half of the lesson. Per-row scaling is the right default — a global scale would be worse — but it does not protect you from a wild dynamic range *inside* a row.

A max-norm relative error on the whole of C would have hidden this. The 2^{40} terms dominate other entries of C, so $\|C_{\text{ozaki}} - C\|_{\max} / \|C\|_{\max}$ looks small even while $C[0,0]$ is completely wrong. Check the entries that the large values do not dominate.

8. The boring rule

Scale each row of A and each column of B by a power of two so the max absolute value is $1/2$. Peel 7-bit integer slices. Accumulate every slice pair with $i + j \le k + 1$ in integer arithmetic wide enough that $q \cdot (2^\beta - 1)^2$ cannot wrap. Unscale by the same powers of two. Choose k from the **worst-case**

exponent range inside a row or column, not from the format's mantissa width: well-scaled data need $k = 8$ of 7-bit slices for binary64; a 40-bit gap inside a row needs two extra slices on top of that. If $k(k+1)/2$ integer GEMMs still beat one native FP64 GEMM on the machine you have, emulate. If the range is so wide that k explodes, fall back to native FP64 — which is what production libraries do.

Cost, as counted from the same `n_products` helper. $k = 4$ triangular is 10 GEMMs and ~28 bits; $k = 8$ triangular is 36 GEMMs and ~56 bits.

```
  k  tri GEMMs  full GEMMs   bits covered (k*7)
  1          1           1                    7
  2          3           4                   14
  3          6           9                   21
  4         10          16                   28   binary32 significand (24 bits)
  5         15          25                   35
  6         21          36                   42
  7         28          49                   49
  8         36          64                   56   binary64 significand (53 bits)
  9         45          81                   63
 10         55         100                   70
 11         66         121                   77
 12         78         144                   84
```

On a GPU whose INT8 tensor cores are R times faster than native FP64, the crude break-even against one DGEMM is $k(k+1)/2 < R$, plus splitting and FP64 reduction. With $R \sim 100$ on the B200 numbers in §1, even $k = 12$ (78 triangular products) still sits under the ratio on paper. The reduction itself becomes the new bottleneck; Uchino, Ozaki, and Imamura (IJHPCA 2025) spend their paper on that.

The INT32 bound is the other half of the rule. The $2^{2\beta}$ form of $\beta = \min(7, \lfloor (31 - \log_2 q)/2 \rfloor)$ is one bit pessimistic: INT8 values are at most 127, not 128. Direct count:

```python
# demo_overflow.py - why slice width is bounded by the inner dimension
import numpy as np

print("INT32 signed max =", 2**31 - 1)
print("max |int8| = 127")
print("n * 127 * 127  must be <= 2^31-1 for exact INT32 accumulation")
n_max = (2**31 - 1) // (127 * 127)
print("n_max for 8-bit slices =", n_max)

print("\nbeta = min(7, floor((31 - log2(n))/2))  so that  n * (2^beta)^2  fits in 31 bits:")
for n in (16, 64, 256, 1024, 4096, 16384, 65536, 131072, 262144):
    beta = min(7, int(np.floor((31 - np.log2(n)) / 2)))
    cap = n * (2**beta) * (2**beta)
    print(f"n={n:7d}  beta={beta}  n*2^{{2 beta}}={cap:12d}  fits INT32? {cap <= 2**31-1}")

print("\nThe 2^{2*beta} bound is one bit pessimistic: int8 values are at most 127, not 128."
```

```
print("n=131072, beta=7: n*128*128 =", 131072 * 128 * 128,
      "  (equals 2^31, overflows signed INT32)")
print("n=131072, beta=7: n*127*127 =", 131072 * 127 * 127,
      "  (still < 2^31-1; exact int8 GEMM fits)")

print("\nMiniature of the same overflow, using int16 accumulation and 7-bit values:")
print("int16 max =", 2**15 - 1)
k_inner = 20
a = np.full((1, k_inner), 127, dtype=np.int16)
b = np.full((k_inner, 1), 127, dtype=np.int16)
prod16 = a @ b
prod64 = a.astype(np.int64) @ b.astype(np.int64)
print(f"inner dim = {k_inner},  each factor = 127")
print("true product (int64 acc) =", int(prod64[0, 0]),
      "  (20 * 127 * 127 =", 20 * 127 * 127, ")")
print("numpy int16 @ int16     =", int(prod16[0, 0]),
      "  <-- wrapped; 322580 mod 65536 =", 322580 % 65536, "then interpreted as signed")
wrapped = np.int16(np.uint16(int(prod64[0, 0]) & 0xFFFF))
print("two's-complement wrap of true product into int16 =", int(wrapped))
```

```
python3 demo_overflow.py
```

```
INT32 signed max = 2147483647
max |int8| = 127
n * 127 * 127  must be <= 2^31-1 for exact INT32 accumulation
n_max for 8-bit slices = 133144

beta = min(7, floor((31 - log2(n))/2))  so that  n * (2^beta)^2  fits in 31 bits:
n=     16  beta=7  n*2^{2 beta}=      262144  fits INT32? True
n=     64  beta=7  n*2^{2 beta}=     1048576  fits INT32? True
n=    256  beta=7  n*2^{2 beta}=     4194304  fits INT32? True
n=   1024  beta=7  n*2^{2 beta}=    16777216  fits INT32? True
n=   4096  beta=7  n*2^{2 beta}=    67108864  fits INT32? True
n=  16384  beta=7  n*2^{2 beta}=   268435456  fits INT32? True
n=  65536  beta=7  n*2^{2 beta}=  1073741824  fits INT32? True
n= 131072  beta=7  n*2^{2 beta}=  2147483648  fits INT32? False
n= 262144  beta=6  n*2^{2 beta}=  1073741824  fits INT32? True

The 2^{2*beta} bound is one bit pessimistic: int8 values are at most 127, not 128.
n=131072, beta=7: n*128*128 = 2147483648   (equals 2^31, overflows signed INT32)
n=131072, beta=7: n*127*127 = 2114060288   (still < 2^31-1; exact int8 GEMM fits)

Miniature of the same overflow, using int16 accumulation and 7-bit values:
int16 max = 32767
inner dim = 20,  each factor = 127
true product (int64 acc) = 322580   (20 * 127 * 127 = 322580 )
```

```
numpy int16 @ int16     = -5100   <-- wrapped; 322580 mod 65536 = 60436 then interpreted as
two's-complement wrap of true product into int16 = -5100
```

NumPy integer `@` uses the input dtype as the accumulator. An `int16` GEMM wraps with no exception. Tensor cores do the same at INT32. The miniature is the INT32 failure at a size you can print: $20 \cdot 127 \cdot 127 = 322580$ does not fit in 15 bits plus a sign, and the computed inner product is the two's-complement wrap -5100. Always cast slices up to a type that can hold $q \cdot 127^2$ before multiplying.

9. Ozaki scheme II, in miniature: residues and the CRT

Scheme I pays $k(k+1)/2$ GEMMs for k slices. Ozaki, Uchino, and Imamura (IJHPCA 2026; arXiv:2504.08009, v4 16 September 2026) replace slicing with a residue number system. Scale A and B to integers, pick pairwise-coprime moduli $m_1, \ldots, m_t$ small enough that each residue GEMM is exact in INT8→INT32, compute

$$C^{(s)} \equiv (A \bmod m_s)\,(B \bmod m_s) \pmod{m_s},$$

and reconstruct $C \bmod M$ with $M = \prod m_s$ by the Chinese Remainder Theorem. If every $|C_{ij}| < M/2$, the reconstruction is the exact integer product. The number of GEMMs is t, linear in the working precision rather than quadratic.

The listing below is the CRT core on a 3×3 integer product, not the full floating-point pipeline (which also needs a careful fixed-point scale so that M covers the true range of AB). Four moduli, four integer GEMMs, exact reconstruction. Scheme I with $k = 4$ would have used ten.

```
# demo_scheme2.py - Ozaki scheme II idea: integer GEMM modulo several primes, CRT reconstruc
from math import prod
import numpy as np

# Tiny integer matrices whose product we reconstruct from residues.
A = np.array([[11, -7, 3],
              [4,  13, -2],
              [-5, 8,  9]], dtype=np.int64)
B = np.array([[6,  -3],
              [1,   8],
              [-4,  5]], dtype=np.int64)

C_true = A @ B
print("A =\n", A)
print("B =\n", B)
print("exact integer AB =\n", C_true)

# Pairwise-coprime moduli small enough that residues fit in int8.
moduli = (13, 17, 19, 23)
print("\nmoduli =", moduli, "  M =", prod(moduli))
```

```
print("true |C|_max =", int(np.max(np.abs(C_true))),
      "  unique CRT if |C_ij| < M/2 =", prod(moduli) // 2)


def symmetric_mod(X, m):
    r = np.remainder(X, m)  # in {0, ..., m-1}
    return np.where(r > m // 2, r - m, r)


residues = []
for m in moduli:
    Am = symmetric_mod(A, m).astype(np.int8)
    Bm = symmetric_mod(B, m).astype(np.int8)
    # Integer GEMM, then reduce. int64 acc is exact here (inner dim = 3).
    Cm = symmetric_mod(Am.astype(np.int64) @ Bm.astype(np.int64), m)
    residues.append(Cm)
    print(f"\n(A mod {m}) =\n{Am}")
    print(f"(B mod {m}) =\n{Bm}")
    print(f"(AB mod {m}) =\n{Cm.astype(int)}")


def crt_pair(a, m, b, n):
    """x  a (mod m), x  b (mod n), 0 <= x < m*n.  Assumes gcd(m,n)=1."""
    inv = pow(m, -1, n)
    return (a + m * ((b - a) * inv % n)) % (m * n)


# Reconstruct each entry.
M = prod(moduli)
C_hat = np.empty_like(C_true)
for i in range(C_true.shape[0]):
    for j in range(C_true.shape[1]):
        x, mm = int(residues[0][i, j]) % moduli[0], moduli[0]
        # residues are symmetric; convert to [0, m) for CRT
        x = x if x >= 0 else x + moduli[0]
        for r, m in zip(residues[1:], moduli[1:]):
            b = int(r[i, j]) % m
            x = crt_pair(x, mm, b, m)
            mm *= m
        if x > M // 2:
            x -= M
        C_hat[i, j] = x

print("\nCRT reconstruction =\n", C_hat)
print("matches exact integer AB:", np.array_equal(C_hat, C_true))
print("number of integer GEMMs = number of moduli =", len(moduli),
```

```
      "  (Ozaki I with k=4 triangular would be", 4 * 5 // 2, ")")
```

```
python3 demo_scheme2.py
```

```
A =
 [[11 -7  3]
 [ 4 13 -2]
 [-5  8  9]]
B =
 [[ 6 -3]
 [ 1  8]
 [-4  5]]
exact integer AB =
 [[ 47 -74]
 [ 45  82]
 [-58 124]]

moduli = (13, 17, 19, 23)   M = 96577
true |C|_max = 124   unique CRT if |C_ij| < M/2 = 48288

(A mod 13) =
[[-2  6  3]
 [ 4  0 -2]
 [-5 -5 -4]]
(B mod 13) =
[[ 6 -3]
 [ 1 -5]
 [-4  5]]
(AB mod 13) =
[[-5  4]
 [ 6  4]
 [-6 -6]]
```

```
...
```

```
CRT reconstruction =
 [[ 47 -74]
 [ 45  82]
 [-58 124]]
matches exact integer AB: True
number of integer GEMMs = number of moduli = 4   (Ozaki I with k=4 triangular would be 10 )
```

(The four residue blocks for moduli 17, 19, 23 are in the run; they are omitted here as a `...` line. They reconstruct uniquely because $124 < 96577/2$.)

CUDA 13.4's cuBLAS uses scheme II for emulated FP64 when it is faster than scheme I, on Ampere and newer. That is a library fact from the release notes, not a measurement here.

10. What this does not emulate

The scheme is a transformation of **matrix multiplication**. Divisions, square roots, Givens rotations, data-dependent branches, and sparse triangular solves are still native FP64 (or they are not). Emulation also does nothing for the exponent *range*: binary64's 11-bit exponent is used as-is, and underflow/overflow of the unscaled result is the same as in DGEMM. A code that is not GEMM-heavy does not become fast because tensor cores exist.

11. Pitfalls

1. **Counting slices from the format, not from the data.** $k = 8$ of 7-bit slices covers a well-scaled binary64 mantissa. A 40-bit gap inside a row needs more. Production codes estimate the range (or fall back to native FP64 when it is too wide).
2. **Reporting $\|\widehat{C} - C\|_{\max} / \|C\|_{\max}$ on a matrix that contains a spike.** The spike hides errors in the small entries. Check those entries.
3. **Integer `@` in the slice dtype.** NumPy `int8 @ int8` and `int16 @ int16` wrap. Cast to `int32` or `int64` first, and respect $q \cdot 127^2 \le 2^{31} - 1$ if you insist on INT32.
4. **Forgetting that recombination is in FP64.** Extra slices past the working-precision mantissa buy nothing; they add rounded terms below a ulp. The $k = 8$ and $k = 9$ rows of §6 are identical.
5. **Scaling the whole matrix by one global max.** That is shared-place taken too far. Per-row A / per-column B is the scheme. A global scale makes the trap of §7 the typical case.
6. **Treating emulation as a full FP64 ALU.** It is a GEMM. See §10.

12. Checkpoint

1. Why is multiplying by 2^e exact in binary64, and why does that matter for unscaling?
2. For 7-bit slices and inner dimension $q = 256$, does an INT32 accumulator wrap? What is $n_{\max}$?
3. Why does $k = 7$ of 7-bit slices fall short of binary64, and why does $k = 8$ meet it on well-scaled data?
4. In §7, why is $C[0, 0] = 0$ for $k \le 5$, and why does moving the spike to another row fix $k = 1$?
5. Scheme I with k slices, triangular: how many GEMMs? Scheme II with t moduli?

Exercises

1. **Split 0.1.** Run `split_slices` on the binary64 value of 0.1 with $k = 8$. The reconstruction will equal that binary64 value, not the real number 1/10. Enclose 1/10 with `fractions.Fraction` and report the residual.
2. **Count, then measure.** For $k = 5$ triangular, how many integer GEMMs? Confirm with `n_products`. Compare the §6 error at $k = 5$ with the `float32` GEMM error.
3. **INT16 miniature.** Repeat `demo_overflow.py` with inner dimension 8 instead of 20. At what q does `int16` first wrap for factors of 127?

4. **Full versus triangle.** On the §6 matrices, print the Frobenius norm of the *dropped* pairs at $k = 6$ (the pairs with $i + j > 7$). Is it below $u\|C\|$?
5. **The trap, quantified.** In `demo_trap.py`, replace 2^{40} by 2^e for $e = 7, 14, 21, 28, 35, 42$. Plot (or tabulate) the smallest k that recovers $C[0, 0] = 7$. Does $k \approx \lceil e/7 \rceil + 1$ hold?
6. **Scheme II on floats.** Scale the §6 matrices by 2^{20}, round to integers, run the CRT GEMM with enough 8-bit moduli that $M/2$ exceeds $\|AB\|_{\max}$, and unscale. Compare against `mp_gemm`. How many moduli did you need?
7. **Slice width versus q.** Implement $\beta = \min(7, \lfloor (31 - \log_2 q)/2 \rfloor)$ in `split_slices`. For $q = 262144$, what β do you get, and how many slices then cover 53 bits?

Summary

The Ozaki scheme rebuilds a binary64 matrix product from exact low-precision integer products. Shared-place splitting turns each row of A and each column of B into k INT8 slices; each slice-pair GEMM is exact in INT32 or INT64; recombination is a weighted sum of those products, with weights that are powers of two. Accuracy tracks k times the slice width, minus the exponent range inside a row. The usual cost is $k(k + 1)/2$ GEMMs; Ozaki scheme II replaces that quadratic with one GEMM per modulus and a CRT. On well-scaled 8×8 normals, $k = 4$ of 7-bit slices matches `float32` GEMM, and $k = 8$ matches `float64` GEMM, both measured against an `mpmath` reference. A 40-bit spike in one row delays the small entries of that row until slice 6 and returns a completely wrong entry until then. The scheme emulates GEMM, not the rest of floating-point arithmetic.

Sources (for further reading)

- K. Ozaki, T. Ogita, S. Oishi, S. M. Rump, "Error-free transformations of matrix multiplication by using fast routines of matrix multiplication and its applications", *Numerical Algorithms* 59, 95–118 (2012). https://doi.org/10.1007/s11075-011-9478-1
- H. Ootomo, K. Ozaki, R. Yokota, "DGEMM on integer matrix multiplication unit", *The International Journal of High Performance Computing Applications* 38, 297–313 (2024). arXiv: 2306.11975. https://doi.org/10.1177/10943420241239588
- K. Ozaki, Y. Uchino, T. Imamura, "Ozaki scheme II: A GEMM-oriented emulation of floating-point matrix multiplication using an integer modular technique", *The International Journal of High Performance Computing Applications* (2026). arXiv: 2504.08009v4 (revised 16 September 2026). https://doi.org/10.1177/10943420261467787
- Y. Uchino, K. Ozaki, T. Imamura, "Performance enhancement of the Ozaki scheme on integer matrix multiplication unit", *IJHPCA* 39(3), 462–476 (2025). arXiv: 2409.13313
- IEEE Std 754-2019, *IEEE Standard for Floating-Point Arithmetic*: binary64 layout, hidden bit, unit roundoff.
- NVIDIA, *CUDA Toolkit 13.4 Release Notes*, cuBLAS: emulated FP64 matrix multiplications, Ozaki-I / Ozaki-II selection, B200 DGEMM/ZGEMM figures. https://docs.nvidia.com/cuda/cuda-toolkit-release-notes/
- NVIDIA, cuBLASDx example `16_dgemm_emulation`: Ozaki-scheme DGEMM on INT8 tensor cores. https://github.com/NVIDIA/CUDALibrarySamples/tree/master/MathDx/cuBLASDx/16_dgemm_emulation

- NumPy `np.ldexp`, `np.rint`, integer `matmul` dtype rules: https://numpy.org/doc/stable/reference/generated/numpy.matmul.html
- mpmath, arbitrary-precision floats (`mp.dps`): https://mpmath.org/doc/current/

Why Gaussian Elimination with Partial Pivoting Works: The Growth Factor, Wilkinson's 2^(n−1), and the 2026 Proof

Why Gaussian Elimination with Partial Pivoting Works

Almost every dense linear solver you have used — `numpy.linalg.solve`, MATLAB's and Julia's backslash on a general matrix, LAPACK's `gesv` underneath all of them — is Gaussian elimination with **partial pivoting** (GEPP). On paper, GEPP is not stable: there are matrices for which its intermediate entries grow by a factor of 2^{n-1}, which at $n = 60$ wipes out every digit of a double-precision answer. In practice, people have run it on billions of matrices for sixty years and almost never seen that happen.

This chapter explains the gap. The quantity that controls it is the **growth factor**. You will compute it by hand on a 2×2 example, watch Wilkinson's worst-case matrix destroy a solve whose condition number is only 27, and measure growth on thousands of random matrices: it comes out near $0.5\sqrt{n}$, not 2^{n-1}. Then you will see what John Urschel proved in October 2026, settling a conjecture Trefethen had stated in 1997: for Gaussian random matrices, growth under partial pivoting is rarely much larger than $\sqrt{n}$. The trap is reading that theorem as "GEPP is always safe".

All listings run on **CPython 3.13.5** with NumPy 2.5.4 and SciPy 1.18.1 in a venv (`uv venv && uv pip install numpy scipy`). The random experiments use fixed seeds and take under a minute.

1. Mental model: errors are bounded by how big the numbers get

Elimination factors $PA = LU$, where P is a permutation, L is unit lower triangular and U is upper triangular. In floating point, each update $a_{ij} \leftarrow a_{ij} - \ell_{ik}u_{kj}$ makes a rounding error proportional to the size of the numbers involved. Higham's analysis (*Accuracy and Stability of Numerical Algorithms*, ch. 9) turns that into a backward-error statement: the computed solution exactly solves $(A+\Delta A)\hat{x} = b$ with

$$\frac{\|\Delta A\|}{\|A\|} \lesssim c(n)\, u \cdot \mathrm{growth}(A),$$

where $u = 2^{-53}$ is the unit roundoff and $c(n)$ is a modest polynomial in n. Urschel's paper uses the definition

$$\mathrm{growth}(A) = \max\left(\|L\|_{\max},\ \frac{\|U\|_{\max}}{\|A\|_{\max}} \right),$$

with $\|\cdot\|_{\max}$ the largest entry in absolute value. Everything else in the bound is small, so **the growth factor is the stability of Gaussian elimination**. Partial pivoting picks the largest $|a_{ik}|$ in the

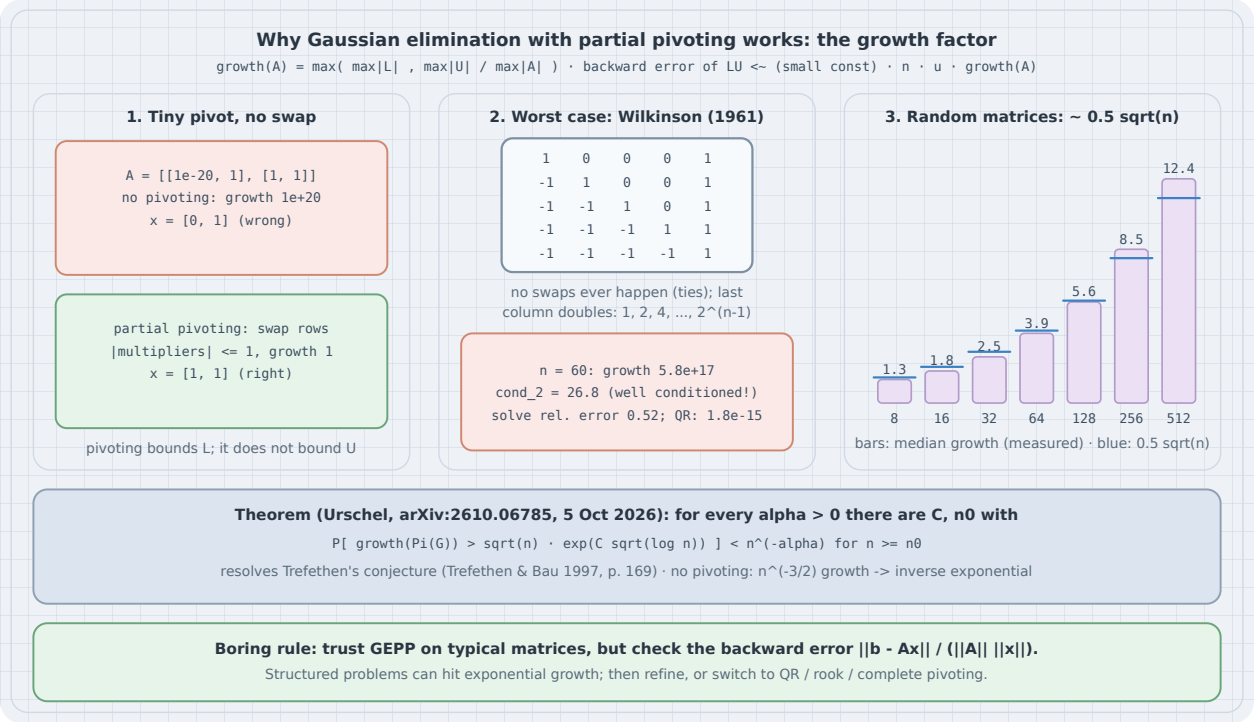


Figure 1: The growth factor: definition, the tiny-pivot failure, Wilkinson's worst case, measured random-matrix growth, and Urschel's theorem

current column as the pivot. That makes every multiplier satisfy $|\ell_{ik}| \le 1$, so $\|L\|_{\max} = 1$. It does nothing to bound U directly. A simple induction shows each step can at most double the largest entry, which gives growth $\le 2^{n-1}$.

2. The tiny pivot: why pivot at all

```
# ge.py: Gaussian elimination with and without partial pivoting, and its growth factor
import numpy as np


def lu(A, pivot=True):
    """Return perm, L, U with A[perm] = L @ U. Ties in the pivot search go to the first row.
    U = np.array(A, dtype=float)
    n = U.shape[0]
    L = np.eye(n)
    perm = np.arange(n)
    for k in range(n - 1):
        if pivot:
            p = k + int(np.argmax(np.abs(U[k:, k])))      # largest |entry| in column k
            if p != k:
                U[[k, p], k:] = U[[p, k], k:]
```

```
                L[[k, p], :k] = L[[p, k], :k]
                perm[[k, p]] = perm[[p, k]]
        L[k + 1:, k] = U[k + 1:, k] / U[k, k]
        U[k + 1:, k:] -= np.outer(L[k + 1:, k], U[k, k:])
        U[k + 1:, k] = 0.0
    return perm, L, U


def growth(A, pivot=True):
    """Urschel's definition: max(||L||_max, ||U||_max / ||A||_max)."""
    _, L, U = lu(A, pivot)
    return max(np.abs(L).max(), np.abs(U).max() / np.abs(A).max())


def solve(A, b, pivot=True):
    perm, L, U = lu(A, pivot)
    return backward(U, forward(L, b[perm]))


def forward(L, b):
    y = np.zeros_like(b)
    for i in range(len(b)):
        y[i] = b[i] - L[i, :i] @ y[:i]
    return y


def backward(U, y):
    x = np.zeros_like(y)
    for i in reversed(range(len(y))):
        x[i] = (y[i] - U[i, i + 1:] @ x[i + 1:]) / U[i, i]
    return x


if __name__ == "__main__":
    eps = 1e-20
    A = np.array([[eps, 1.0], [1.0, 1.0]])
    b = np.array([1.0, 2.0])                          # exact solution is very close to (1, 1)
    for pivot in (False, True):
        x = solve(A, b, pivot)
        print(f"pivot={pivot!s:5}  growth={growth(A, pivot):.3e}  x={x}")
```

```
python ge.py
```

Output:

```
pivot=False  growth=1.000e+20  x=[0. 1.]
pivot=True   growth=1.000e+00  x=[1. 1.]
```

Without pivoting, the multiplier is $1/10^{-20} = 10^{20}$, $U_{22} = 1 - 10^{20}$ rounds to -10^{20}, and the information in the "1" is gone. The growth factor reports exactly that: 10^{20}. Swapping the rows first gives a multiplier of 10^{-20}, growth 1, and the right answer. This is the textbook reason for pivoting. The rest of the chapter asks whether pivoting is *enough*.

3. Wilkinson's matrix: the worst case is real

Wilkinson (1961) gave a matrix that reaches the 2^{n-1} bound: 1 on the diagonal, -1 everywhere below it, and a last column of ones. Every subdiagonal entry in the pivot column has magnitude 1, equal to the pivot, so partial pivoting (ties go to the first row, as in LAPACK) never swaps. Each step adds the pivot row to every row below it, which doubles the last column: $1, 2, 4, \ldots, 2^{n-1}$.

```
# wilkinson.py: the matrix that makes partial pivoting double every entry, every step
import numpy as np
import scipy.linalg as sla

from ge import growth


def wilkinson(n):
    W = np.eye(n) - np.tril(np.ones((n, n)), -1)   # 1 on the diagonal, -1 below it
    W[:, -1] = 1.0                                 # last column all ones
    return W


print(wilkinson(5))
print()
rng = np.random.default_rng(0)
print(f"{'n':>3} {'growth':>10} {'2^(n-1)':>10} {'cond_2':>8} {'rel. error':>11} {'backward
for n in (10, 30, 50, 55, 60, 64):
    W = wilkinson(n)
    x = rng.standard_normal(n)
    b = W @ x
    xh = np.linalg.solve(W, b)                     # LAPACK gesv: LU with partial pivoting
    err = np.linalg.norm(xh - x) / np.linalg.norm(x)
    berr = np.linalg.norm(b - W @ xh) / (np.linalg.norm(W, 2) * np.linalg.norm(xh))
    print(f"{n:3d} {growth(W):10.3e} {2.0 ** (n - 1):10.3e} {np.linalg.cond(W):8.1f} {err:11

n = 60
W = wilkinson(n)
x = rng.standard_normal(n)
b = W @ x
```

```
Q, R = np.linalg.qr(W)                                  # Householder QR: no growth problem
xq = sla.solve_triangular(R, Q.T @ b)
print(f"\nn=60 via QR: rel. error {np.linalg.norm(xq - x) / np.linalg.norm(x):.2e}")
```

```
python wilkinson.py
```

Output:

```
[[ 1.  0.  0.  0.  1.]
 [-1.  1.  0.  0.  1.]
 [-1. -1.  1.  0.  1.]
 [-1. -1. -1.  1.  1.]
 [-1. -1. -1. -1.  1.]]

  n     growth    2^(n-1)   cond_2  rel. error  backward err
 10  5.120e+02  5.120e+02      4.4    1.11e-14      2.78e-15
 30  5.369e+08  5.369e+08     13.3    4.04e-09      4.83e-10
 50  5.629e+14  5.629e+14     22.3    1.89e-03      1.08e-04
 55  1.801e+16  1.801e+16     24.6    2.56e-02      1.40e-03
 60  5.765e+17  5.765e+17     26.8    5.23e-01      3.10e-02
 64  9.223e+18  9.223e+18     28.6    2.77e-01      3.46e-02

n=60 via QR: rel. error 1.84e-15
```

Read the columns together:

- **The growth is exactly** 2^{n-1} at every size. The bound is attained.
- **The matrix is well conditioned.** κ_2 is below 30 even at $n = 64$. Nothing about the *problem* is hard. The algorithm is what fails.
- **The error tracks** $u \cdot$ growth**.** At $n = 50$ the growth is 5.6×10^{14}, and $u \cdot \text{growth} \approx 0.06$. The observed relative error is 1.9×10^{-3}. By $n = 60$, about half the solution is wrong, from `numpy.linalg.solve` with no warning.
- **The backward error notices.** $\|b - A\hat{x}\|/(\|A\|\,\|\hat{x}\|)$ rises from 10^{-15} to 3×10^{-2}. A residual check catches this failure, even though nothing else would.
- **A different algorithm does not have the problem.** Householder QR is backward stable with no growth factor and solves the $n = 60$ system to 1.8×10^{-15}, at about twice the flops of LU.

4. Random matrices: about $0.5\sqrt{n}$, not 2^{n-1}

Trefethen and Schreiber (1990) ran GEPP on large numbers of random matrices and found growth near $n^{1/2}$ for Gaussian entries. In the 1997 textbook with Bau (Lecture 22), Trefethen wrote that observation down as a conjecture. To repeat the experiment at scale, the next listing eliminates whole batches of matrices at once. Each step's pivot search, row swap and rank-one update is a NumPy operation over the batch axis. It is checked against `ge.py` below.

```
# batch_growth.py: growth factors of many Gaussian matrices at once (vectorised elimination)
import numpy as np


def batch_growth(A, pivot=True):
    """A: (B, n, n). Returns max(||L||_max, ||U||_max / ||A||_max) for each matrix."""
    U = A.copy()
    B, n, _ = U.shape
    rows = np.arange(B)
    Lmax = np.ones(B)
    for k in range(n - 1):
        if pivot:
            p = k + np.argmax(np.abs(U[:, k:, k]), axis=1)
            tmp = U[rows, k, k:].copy()
            U[rows, k, k:] = U[rows, p, k:]
            U[rows, p, k:] = tmp
        l = U[:, k + 1:, k] / U[:, k, k][:, None]
        Lmax = np.maximum(Lmax, np.abs(l).max(axis=1))
        U[:, k + 1:, k:] -= l[:, :, None] * U[:, k, None, k:]
        U[:, k + 1:, k] = 0.0
    Umax = np.abs(np.triu(U)).max(axis=(1, 2))
    return np.maximum(Lmax, Umax / np.abs(A).max(axis=(1, 2)))
```

```
# random_growth.py: partial pivoting on Gaussian matrices grows like sqrt(n), not 2^(n-1)
import math
import time

import numpy as np

from batch_growth import batch_growth

rng = np.random.default_rng(2026)
t0 = time.perf_counter()
print("partial pivoting, G with iid N(0,1) entries")
print(f"{'n':>4} {'samples':>7} {'median':>8} {'median/sqrt(n)':>15} {'max seen':>9} {'max/s
for n, B in ((8, 4000), (16, 4000), (32, 2000), (64, 1000), (128, 300), (256, 80), (512, 20)
    g = batch_growth(rng.standard_normal((B, n, n)), pivot=True)
    med = float(np.median(g))
    print(f"{n:4d} {B:7d} {med:8.2f} {med / math.sqrt(n):15.3f} {g.max():9.2f} {g.max() / ma
          f" {math.log(med) / math.log(n):14.3f}")

print("\nno pivoting, n = 64: Urschel's Theorem 1.3 says n^(-3/2) growth -> X, P[X <= x] = e
n, B = 64, 20000
x = np.sort(batch_growth(rng.standard_normal((B, n, n)), pivot=False) / n ** 1.5)
c = 2 / (3 * math.sqrt(math.pi))
emp = np.arange(1, B + 1) / B
```

```
    print(f"  samples {B}, KS distance to the limit law: {np.abs(emp - np.exp(-c / x)).max():.4f
for q in (0.5, 0.9, 0.99):
    print(f"  quantile {q:4}: empirical {np.quantile(x, q):8.3f}   limit law {c / -math.log(
print(f"  largest no-pivot growth seen: {x[-1] * n ** 1.5:.3e}")
print(f"elapsed {time.perf_counter() - t0:.0f} s")
```

```
python random_growth.py
```

Output:

```
partial pivoting, G with iid N(0,1) entries
   n samples   median  median/sqrt(n)  max seen  max/sqrt(n)  log g / log n
   8    4000     1.30          0.458      4.01        1.419          0.125
  16    4000     1.77          0.442      5.34        1.334          0.206
  32    2000     2.55          0.451      6.74        1.192          0.270
  64    1000     3.85          0.481     10.95        1.369          0.324
 128     300     5.58          0.493     10.44        0.923          0.354
 256      80     8.49          0.531     13.76        0.860          0.386
 512      20    12.38          0.547     16.07        0.710          0.403

no pivoting, n = 64: Urschel's Theorem 1.3 says n^(-3/2) growth -> X, P[X <= x] = exp(-c/x),
  samples 20000, KS distance to the limit law: 0.0103
  quantile  0.5: empirical    0.536   limit law    0.543
  quantile  0.9: empirical    3.495   limit law    3.570
  quantile 0.99: empirical   35.599   limit law   37.424
  largest no-pivot growth seen: 1.795e+06
elapsed 27 s
```

Two regimes, measured:

- **With partial pivoting**, the median growth is 0.44–$0.55\sqrt{n}$ over a 64-fold range of sizes, and the largest value in thousands of samples is about $1.4\sqrt{n}$. At $n = 512$ that is 16, against a worst case of 2^{511}. The last column, $\log g / \log n$, creeps up toward $1/2$ but slowly, because $\log(0.5\sqrt{n})/\log n = 1/2 - \log 2/\log n$, which is still 0.39 at $n = 512$. Urschel's Corollary 1.5 says this ratio tends to $1/2$ in probability.
- **Without pivoting**, growth is about $n^{3/2}$ times a heavy-tailed random variable. Urschel's Theorem 1.3 identifies the limit law as an inverse exponential, $\Pr[X \le x] = e^{-c/x}$ with $c = 2/(3\sqrt{\pi})$ (his Figure 1 uses $n = 64$). Our 20,000 samples at $n = 64$ sit within KS distance 0.01 of it, and the 99% quantile is about 36, which is $36 \cdot 64^{3/2} \approx 18{,}000$. The largest sample was 1.8×10^6. Random matrices do not need pivoting for their *typical* case. They need it for the tail.

A check that the vectorised code agrees with the one-matrix-at-a-time `ge.py`, which you should always run before trusting a batched rewrite:

```
# check_batch.py: the batched elimination must agree exactly with ge.py
import numpy as np
```

```
from batch_growth import batch_growth
from ge import growth

A = np.random.default_rng(1).standard_normal((50, 20, 20))
for pivot in (True, False):
    batched = batch_growth(A, pivot)
    single = np.array([growth(a, pivot) for a in A])
    print(f"pivot={pivot!s:5}  max |batched - single| = {np.abs(batched - single).max():.1e}")
```

```
python check_batch.py
```

Output:

```
pivot=True   max |batched - single| = 0.0e+00
pivot=False  max |batched - single| = 0.0e+00
```

5. The 2026 theorem

Urschel's main result (*On the Growth Factor of Random Matrices*, arXiv:2610.06785, 5 October 2026, Theorem 1.4): for every $\alpha > 0$ there are constants C_α, n_α such that, for an $n \times n$ Gaussian matrix G and the partial-pivoting map Π,

$$\Pr\left[\text{growth}(\Pi(G)) > \sqrt{n}\, e^{C_\alpha \sqrt{\log n}}\right] < n^{-\alpha} \qquad (n \geq n_\alpha).$$

Since $e^{C\sqrt{\log n}}$ grows more slowly than any power n^c, this implies Trefethen's Conjecture 1.1: $\Pr[\text{growth} > n^{1/2+\alpha}] < n^{-M}$ for every $\alpha, M > 0$ once n is large. Trefethen had offered $1000 for a resolution. The exponent $1/2$ is sharp, because $\|U\|_{\max}$ is at least about $|\det G|^{1/n}$, which is of order $\sqrt{n}$. Some context from the paper's introduction:

- Before this, the best rigorous bound was Huang and Tikhomirov's n^c with high probability, for a large unspecified c.
- Trefethen released his own long-unpublished argument for the *last* pivot $|U_{nn}|$ in parallel.
- The same technique improves the **smoothed analysis** of GEPP (Theorem 1.6). Any matrix plus a polynomially small Gaussian perturbation has growth $n^{O(\log \log n)}$, down from Sankar's $n^{O(\log n)}$. This is the result that speaks to *non-random* inputs. Adversarial matrices like Wilkinson's are fragile: almost any perturbation destroys their growth.
- The proof works in exact arithmetic, as almost all growth-factor results do. The paper notes that for Gaussian matrices, floating point raises the growth by at most a small constant factor with high probability.

At the time of writing, the paper is a single-author preprint that has not been refereed. The numbers in section 4 agree with it, but numerical agreement is not a proof.

6. The trap: "it is proved, so GEPP is always stable"

The theorem is about *Gaussian random* matrices, and through Theorem 1.6 about matrices with random noise added. It says nothing about the structured matrices that real problems produce. There are documented natural failures. Wright (1993) showed exponential growth in GEPP for matrices from multiple-shooting methods for two-point boundary value problems. Foster (1994) did the same for a discretised Volterra integral equation. In both cases a modelling problem produced Wilkinson-like structure, and GEPP returned garbage with a small condition number, exactly as in section 3. Two mistakes are common:

1. **Checking the condition number instead of the backward error.** $\kappa(A)$ measures how hard the *problem* is. Growth measures how badly the *algorithm* did. Section 3 has $\kappa < 30$ and relative error 0.5.
2. **Trusting `solve` because it did not raise an error.** LAPACK reports only exact zero pivots (`info > 0`). Growth of 10^{17} is not an error condition.

The boring rules:

1. **Compute the normwise backward error** $\eta = \|b - A\hat{x}\| / (\|A\|\,\|\hat{x}\| + \|b\|)$ for any solve whose answer matters. It costs one matrix–vector product. If $\eta \gg nu$, do not use the answer.
2. **Fix it with something cheap first.** One or two steps of iterative refinement ($r = b - A\hat{x}$, solve $Ad = r$ with the same LU, $\hat{x} \leftarrow \hat{x} + d$) recover a backward-stable solution in most moderate-growth cases. If they do not, switch to QR, or to rook or complete pivoting.
3. **For structured problem families** (shooting, Volterra, anything with a long one-directional coupling), estimate the growth once from $\|U\|_{\max} / \|A\|_{\max}$, which is free from the factorization, and pick the solver from that.
4. **Treat the 2026 theorem as an explanation of typical behaviour**, not as a guarantee for your matrix.

7. Try this

1. Run `wilkinson.py` with one step of iterative refinement after `np.linalg.solve`. At which n does refinement stop helping, and why does it stop?
2. Add a perturbation $W + 10^{-8}G$ to Wilkinson's matrix and measure the growth for $n = 60$. Relate the result to Theorem 1.6.
3. Implement **rook pivoting** (alternately search the pivot's row and column until the entry is maximal in both) in `ge.py`, and compare its growth on Wilkinson's matrix and on random matrices.
4. Repeat section 4 with ± 1 entries instead of Gaussian ones. Does the median growth still look like $c\sqrt{n}$? What happens to the no-pivoting tail? (Ties in the pivot search now occur with positive probability.)
5. Use `batch_growth` to estimate $\Pr[\text{growth} > 3\sqrt{n}]$ for $n = 32$ with 20,000 samples. How does that compare with the theorem's $n^{-\alpha}$ shape?

Checkpoint

- Define the growth factor and explain why it, rather than $\kappa(A)$, controls the backward error of LU.
- Explain why partial pivoting gives $\|L\|_{\max} \le 1$ but only growth $\le 2^{n-1}$.
- Build Wilkinson's matrix and explain why partial pivoting never swaps on it.
- State Trefethen's conjecture and the form of Urschel's 2026 theorem.
- Give a cheap test that detects a growth-induced failure after the fact.

Summary

Gaussian elimination is exactly as stable as the numbers it creates are small. Partial pivoting bounds the multipliers but not the growth, and Wilkinson's matrix shows the 2^{n-1} worst case is real: a condition number under 30 and half the answer wrong at $n = 60$. On random matrices, the growth measured here is about $0.5\sqrt{n}$, and Urschel's October 2026 preprint proves that it is rarely much larger than $\sqrt{n}$, resolving Trefethen's conjecture. Use that as the reason GEPP works in practice, and keep checking the backward error, because your matrix is not drawn from a Gaussian distribution.

Sources

- J. Urschel, "On the Growth Factor of Random Matrices", arXiv:2610.06785, 5 October 2026 (definition of growth; Conjecture 1.1; Theorems 1.3, 1.4, 1.6; Corollary 1.5; Figure 1).
- L. N. Trefethen, R. S. Schreiber, "Average-case stability of Gaussian elimination", *SIAM J. Matrix Anal. Appl.* 11(3):335–360, 1990, doi:10.1137/0611023.
- L. N. Trefethen, D. Bau III, *Numerical Linear Algebra*, SIAM, 1997, Lectures 20–22 (Lecture 22: stability of Gaussian elimination, p. 169).
- J. H. Wilkinson, "Error analysis of direct methods of matrix inversion", *J. ACM* 8(3):281–330, 1961.
- N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2nd ed., SIAM, 2002, ch. 9 (LU backward error and the growth factor).
- S. J. Wright, "A collection of problems for which Gaussian elimination with partial pivoting is unstable", *SIAM J. Sci. Comput.* 14(1):231–238, 1993. L. V. Foster, "Gaussian elimination with partial pivoting can fail in practice", *SIAM J. Matrix Anal. Appl.* 15(4):1354–1362, 1994.
- NumPy 2.5 documentation, `numpy.linalg.solve` (computed with LAPACK `_gesv`).

Data Science Math

Mathematics for Data Science

Data science turns measurements into decisions under uncertainty. The mathematics is not a single theorem but a **stack**: descriptive summaries, probability models, estimation, hypothesis testing, prediction, and experimental design—implemented at scale with linear algebra and optimization. This part emphasizes statistical foundations with CS systems awareness (pipelines, A/B tests, metrics).

Chapters in this part

Chapter	Focus
Statistical foundations	Descriptive stats and core toolkit
Sampling and bias	Frames, SE, train/val/test discipline
Hypothesis testing for data	Practical tests, bootstrap, reporting
Dimensionality and features	Curse of dimensionality, scaling, PCA map

The data science loop (mathematical view)

1. **Collect** samples from a process (ideally i.i.d. or with known dependence)
2. **Describe** with robust summaries and visualizations
3. **Model** $P(Y \mid X)$ or $P(X)$ or causal $P(Y \mid \mathrm{do}(X))$
4. **Estimate** parameters / predict with uncertainty
5. **Decide** (ship feature? alert? allocate?)
6. **Validate** on held-out data or sequential experiments

Skipping uncertainty quantification produces confident wrong products.

Core mathematical areas

Statistics and probability

- Descriptive statistics and exploratory data analysis (EDA)
- Estimation: bias, variance, MSE, consistency
- Confidence intervals and hypothesis tests
- Bayesian updating
- Bootstrap and resampling

Linear algebra for tabular / embedding data

- Data matrix $X \in \mathbb{R}^{n \times d}$
- PCA / SVD for dimensionality reduction
- Least squares and regularized regression

Optimization and calibration

- Fitting models by minimizing empirical risk
- Proper scoring rules (log loss, Brier)
- Threshold selection under class imbalance

Causality and design (often neglected)

- Randomized experiments (A/B tests)
- Confounding in observational data
- Potential outcomes sketch

Worked example 1 — metric vs estimator

"Average revenue per user" is a functional of a distribution. The sample mean estimates it; variance and dependence (users repeat) affect standard errors. Math separates **parameter** from **estimator**.

Worked example 2 — classification pipeline

Labels $y \in \{0, 1\}$, features x. Model $\hat{p}(x) \approx P(y = 1 \mid x)$. Threshold $\hat{p} \geq t$ yields decisions. Choosing t is decision theory, not only "accuracy maximization."

Descriptive → inferential bridge

Descriptive stats summarize **the sample you have**. Inferential stats make claims about **the process that generated it**, requiring assumptions (random sampling, model class, independence).

Worked example 3

Mean latency on yesterday's logs describes yesterday. Inferring next week's P99 needs a model of traffic mix and nonstationarity—often the hard part.

Roadmap

1. **Statistical foundations** — summaries, estimation, testing, error types, power (detailed chapter)
2. Cross-links: Probability advanced (17), Linear algebra advanced (18), Optimization (11), Information theory (10)

Notation

- Sample $(x_i)_{i=1}^n$ or pairs (x_i, y_i)
- Estimator $\hat{\theta}_n$; true θ
- Empirical distribution $\hat{P}_n = \frac{1}{n}\sum \delta_{x_i}$
- Loss ℓ; risk R; empirical risk $\hat{R}_n$

Worked example 4 — empirical risk

$\hat{R}_n(f) = \frac{1}{n}\sum_i \ell(y_i, f(x_i))$ is a Monte Carlo estimate of $R(f) = \mathbb{E}[\ell(y, f(x))]$ when data are i.i.d.

Systems constraints

Constraint	Math impact
Streaming data	Online estimators, sketching
Multiple testing	Family-wise error, FDR
Logging bias	Selection bias; counterfactual issues
Delayed outcomes	Censoring; off-policy evaluation
Privacy	Noise injection; bias-variance trade

Worked example 5 — peeking in A/B tests

Repeatedly testing until $p < 0.05$ inflates Type I error. Sequential testing / always-valid p-values fix the math of early stopping.

Estimands, estimators, and error

An **estimand** is the target quantity (population mean conversion, ATE, $P(Y = 1 \mid x)$). An **estimator** is a function of the sample (sample mean, difference-in-means, logistic MLE). Quality metrics:

$$\text{Bias}(\hat{\theta}) = \mathbb{E}[\hat{\theta}] - \theta, \qquad \text{Var}(\hat{\theta}) = \mathbb{E}[(\hat{\theta} - \mathbb{E}\hat{\theta})^2], \qquad \text{MSE} = \text{Bias}^2 + \text{Var}.$$

Worked example 6 — biased but useful

Ridge coefficients are biased toward zero yet can lower MSE under collinearity. “Unbiased” is not always the right product goal.

Worked example 7 — standard error

For i.i.d. sample mean, se $= \sigma/\sqrt{n}$. Doubling precision (halving se) needs roughly 4× data—budgeting law of data science.

Supervised learning as statistics

Prediction models estimate functionals of $P(Y \mid X)$. Classification thresholds implement decision theory; calibration asks whether $\hat{p}(x) \approx P(Y = 1 \mid X = x)$. Cross-validation estimates risk of the **selection procedure**, not only a fixed model.

Worked example 8 — class imbalance

Accuracy can be 99% by always predicting “negative” when base rate is 1%. Prefer precision/recall, PR-AUC, or expected cost under a loss matrix.

Causal vs predictive summary table

Question	Tool
What will happen if we observe $X = x$?	Predictive model
What if we **set** $T = 1$?	Causal estimand + design/ID
Which features associate with Y?	Regression / MI (not automatically causal)
Did the launch move the metric?	Experiment or quasi-experiment

Worked example 9 — logging policy

Bandit logs of $\pi(a \mid x)$ enable off-policy evaluation of a new policy π' via importance weights—if support conditions hold. Pure supervised fit on logged rewards without correction is biased.

Pitfalls

1. **p-hacking and metric shopping**
2. **Train-test leakage** in feature pipelines
3. **Simpson's paradox** aggregation
4. **Confusing prediction with causation**
5. **Ignoring dependence** (time series, users, graphs)
6. **Huge** n **no bias** (systematic measurement error)

Checkpoint

- Separate description, inference, prediction, causation
- Write risk vs empirical risk
- Name bias/variance of an estimator
- Explain why A/B tests beat naive before/after
- List three pipeline leakages

Exercises

1. Define bias and variance of $\hat{\theta}$; give an example with nonzero bias.
2. Why can accuracy be a bad metric at 1% positive class?
3. Write an A/B test design: unit of randomization, primary metric, guardrails.
4. Explain bootstrap SE for the median at a high level.
5. Give a Simpson's paradox contingency table (numbers).
6. When is MAP estimation "like regularization"?
7. PCA: what objective does the first PC optimize?
8. Multiple metrics: how does multiple testing arise in dashboards?
9. Streaming mean: write Welford update (or online mean formula).
10. Causal: why does "users who clicked had higher revenue" not prove clicks cause revenue?
11. Calibration: define ECE at a high level.
12. Checkpoint project: pick a KPI at work/school; identify estimand, estimator, assumption most likely false.

Summary

Data science math is statistical decision-making with matrices and optimizers attached. Master estimands, estimators, uncertainty, and design—then scale with computation. The next chapter builds statistical foundations in depth.

Statistical Foundations for Data Science

Statistics provides the mathematical framework for understanding data, quantifying uncertainty, and making data-driven decisions. This section covers fundamental statistical concepts essential for data science.

Descriptive Statistics

Measures of Central Tendency

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats

# Generate sample data
np.random.seed(42)
data = np.random.normal(100, 15, 1000)  # Normal distribution
skewed_data = np.random.exponential(2, 1000)  # Skewed distribution

def calculate_central_tendency(data, name):
    """Calculate and display measures of central tendency"""
    mean = np.mean(data)
    median = np.median(data)
    mode_result = stats.mode(data, keepdims=True)
    mode = mode_result.mode[0] if len(mode_result.mode) > 0 else np.nan

    print(f"\n{name}:")
    print(f"Mean: {mean:.2f}")
    print(f"Median: {median:.2f}")
    print(f"Mode: {mode:.2f}")

    return mean, median, mode

# Analyze both datasets
mean1, median1, mode1 = calculate_central_tendency(data, "Normal Distribution")
mean2, median2, mode2 = calculate_central_tendency(skewed_data, "Skewed Distribution")

# Visualize the differences
```

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 6))

# Normal distribution
ax1.hist(data, bins=50, alpha=0.7, density=True, color='skyblue')
ax1.axvline(mean1, color='red', linestyle='--', label=f'Mean: {mean1:.1f}')
ax1.axvline(median1, color='green', linestyle='--', label=f'Median: {median1:.1f}')
ax1.set_title('Normal Distribution')
ax1.legend()
ax1.grid(True, alpha=0.3)

# Skewed distribution
ax2.hist(skewed_data, bins=50, alpha=0.7, density=True, color='lightcoral')
ax2.axvline(mean2, color='red', linestyle='--', label=f'Mean: {mean2:.1f}')
ax2.axvline(median2, color='green', linestyle='--', label=f'Median: {median2:.1f}')
ax2.set_title('Skewed Distribution')
ax2.legend()
ax2.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

Measures of Variability

```
def calculate_variability(data, name):
    """Calculate measures of variability"""
    variance = np.var(data, ddof=1)  # Sample variance
    std_dev = np.std(data, ddof=1)   # Sample standard deviation
    range_val = np.max(data) - np.min(data)
    iqr = np.percentile(data, 75) - np.percentile(data, 25)
    mad = np.median(np.abs(data - np.median(data)))  # Median Absolute Deviation

    print(f"\n{name} - Variability Measures:")
    print(f"Variance: {variance:.2f}")
    print(f"Standard Deviation: {std_dev:.2f}")
    print(f"Range: {range_val:.2f}")
    print(f"Interquartile Range (IQR): {iqr:.2f}")
    print(f"Median Absolute Deviation: {mad:.2f}")

    return variance, std_dev, range_val, iqr, mad

# Calculate variability for both datasets
calculate_variability(data, "Normal Distribution")
calculate_variability(skewed_data, "Skewed Distribution")

# Box plots to visualize variability
```

```
plt.figure(figsize=(12, 6))
plt.subplot(1, 2, 1)
plt.boxplot(data, labels=['Normal'])
plt.title('Normal Distribution')
plt.grid(True, alpha=0.3)

plt.subplot(1, 2, 2)
plt.boxplot(skewed_data, labels=['Skewed'])
plt.title('Skewed Distribution')
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

Distribution Shape

```
def analyze_distribution_shape(data, name):
    """Analyze skewness and kurtosis"""
    skewness = stats.skew(data)
    kurt = stats.kurtosis(data)

    print(f"\n{name} - Shape Measures:")
    print(f"Skewness: {skewness:.3f}")
    if skewness > 0.5:
        print("  → Right-skewed (positive skew)")
    elif skewness < -0.5:
        print("  → Left-skewed (negative skew)")
    else:
        print("  → Approximately symmetric")

    print(f"Kurtosis: {kurt:.3f}")
    if kurt > 0:
        print("  → Leptokurtic (heavy tails)")
    elif kurt < 0:
        print("  → Platykurtic (light tails)")
    else:
        print("  → Mesokurtic (normal tails)")

    return skewness, kurt

analyze_distribution_shape(data, "Normal Distribution")
analyze_distribution_shape(skewed_data, "Skewed Distribution")
```

Probability Fundamentals

Basic Probability Rules

```
def demonstrate_probability_rules():
    """Demonstrate basic probability rules with examples"""

    # Simulate coin flips
    n_flips = 10000
    coin_flips = np.random.choice(['H', 'T'], n_flips)

    # Calculate probabilities
    p_heads = np.sum(coin_flips == 'H') / n_flips
    p_tails = np.sum(coin_flips == 'T') / n_flips

    print("Basic Probability Rules:")
    print(f"P(Heads) = {p_heads:.3f}")
    print(f"P(Tails) = {p_tails:.3f}")
    print(f"P(Heads) + P(Tails) = {p_heads + p_tails:.3f}")
    print("Rule: Sum of all probabilities = 1")

    # Simulate dice rolls
    dice_rolls = np.random.randint(1, 7, n_flips)

    # Joint probability example
    even_rolls = dice_rolls % 2 == 0
    high_rolls = dice_rolls >= 4

    p_even = np.mean(even_rolls)
    p_high = np.mean(high_rolls)
    p_even_and_high = np.mean(even_rolls & high_rolls)
    p_even_or_high = np.mean(even_rolls | high_rolls)

    print(f"\nDice Roll Probabilities:")
    print(f"P(Even) = {p_even:.3f}")
    print(f"P(High   4) = {p_high:.3f}")
    print(f"P(Even AND High) = {p_even_and_high:.3f}")
    print(f"P(Even OR High) = {p_even_or_high:.3f}")

    # Verify addition rule: P(A   B) = P(A) + P(B) - P(A   B)
    calculated_or = p_even + p_high - p_even_and_high
    print(f"Calculated P(Even OR High) = {calculated_or:.3f}")
    print(f"Difference: {abs(p_even_or_high - calculated_or):.6f}")

demonstrate_probability_rules()
```

Conditional Probability and Bayes' Theorem

```
def bayes_theorem_example():
    """Medical diagnosis example using Bayes' theorem"""

    # Disease prevalence and test accuracy
    disease_prevalence = 0.01  # 1% of population has disease
    test_sensitivity = 0.95    # 95% true positive rate
    test_specificity = 0.90    # 90% true negative rate

    # Calculate probabilities
    p_disease = disease_prevalence
    p_no_disease = 1 - disease_prevalence
    p_positive_given_disease = test_sensitivity
    p_positive_given_no_disease = 1 - test_specificity

    # Total probability of positive test
    p_positive = (p_positive_given_disease * p_disease +
                  p_positive_given_no_disease * p_no_disease)

    # Bayes' theorem: P(Disease|Positive) = P(Positive|Disease) * P(Disease) / P(Positive)
    p_disease_given_positive = (p_positive_given_disease * p_disease) / p_positive

    print("Bayes' Theorem Example - Medical Diagnosis:")
    print(f"Disease prevalence: {disease_prevalence:.1%}")
    print(f"Test sensitivity: {test_sensitivity:.1%}")
    print(f"Test specificity: {test_specificity:.1%}")
    print(f"\nP(Positive test) = {p_positive:.3f}")
    print(f"P(Disease | Positive test) = {p_disease_given_positive:.3f} ({p_disease_given_po

    print(f"\nInterpretation:")
    print(f"Even with a positive test, there's only a {p_disease_given_positive:.1%} chance
    print(f"This is due to the low base rate (prevalence) of the disease.")

    return p_disease_given_positive

# Simulate the medical test scenario
def simulate_medical_test(n_people=100000):
    """Simulate medical testing scenario"""

    # Generate population
    has_disease = np.random.random(n_people) < 0.01  # 1% prevalence

    # Generate test results
    test_results = np.zeros(n_people, dtype=bool)
```

```
    # People with disease: 95% test positive
    disease_indices = np.where(has_disease)[0]
    test_results[disease_indices] = np.random.random(len(disease_indices)) < 0.95

    # People without disease: 10% test positive (false positives)
    no_disease_indices = np.where(~has_disease)[0]
    test_results[no_disease_indices] = np.random.random(len(no_disease_indices)) < 0.10

    # Calculate actual probabilities from simulation
    positive_tests = test_results
    true_positives = has_disease & positive_tests
    false_positives = (~has_disease) & positive_tests

    simulated_p_disease_given_positive = np.sum(true_positives) / np.sum(positive_tests)

    print(f"\nSimulation Results (n={n_people:,}):")
    print(f"People with disease: {np.sum(has_disease):,}")
    print(f"Positive tests: {np.sum(positive_tests):,}")
    print(f"True positives: {np.sum(true_positives):,}")
    print(f"False positives: {np.sum(false_positives):,}")
    print(f"Simulated P(Disease | Positive) = {simulated_p_disease_given_positive:.3f}")

theoretical_prob = bayes_theorem_example()
simulate_medical_test()
```

Sampling and Sampling Distributions

Central Limit Theorem

```
def demonstrate_central_limit_theorem():
    """Demonstrate the Central Limit Theorem"""

    # Create different population distributions
    populations = {
        'Uniform': np.random.uniform(0, 10, 10000),
        'Exponential': np.random.exponential(2, 10000),
        'Bimodal': np.concatenate([np.random.normal(2, 1, 5000),
                                   np.random.normal(8, 1, 5000)])
    }

    sample_sizes = [5, 10, 30, 100]
    n_samples = 1000

    fig, axes = plt.subplots(len(populations), len(sample_sizes) + 1,
```

```
                        figsize=(20, 12))

for i, (pop_name, population) in enumerate(populations.items()):
    # Plot original population
    axes[i, 0].hist(population, bins=50, alpha=0.7, density=True)
    axes[i, 0].set_title(f'{pop_name} Population')
    axes[i, 0].grid(True, alpha=0.3)

    # For each sample size, create sampling distribution
    for j, sample_size in enumerate(sample_sizes):
        sample_means = []

        for _ in range(n_samples):
            sample = np.random.choice(population, sample_size)
            sample_means.append(np.mean(sample))

        sample_means = np.array(sample_means)

        # Plot sampling distribution
        axes[i, j + 1].hist(sample_means, bins=30, alpha=0.7, density=True)
        axes[i, j + 1].set_title(f'Sample Means (n={sample_size})')
        axes[i, j + 1].grid(True, alpha=0.3)

        # Add normal curve overlay
        x = np.linspace(sample_means.min(), sample_means.max(), 100)
        normal_curve = stats.norm.pdf(x, np.mean(sample_means), np.std(sample_means))
        axes[i, j + 1].plot(x, normal_curve, 'r-', linewidth=2, alpha=0.8)

        # Calculate and display statistics
        mean_of_means = np.mean(sample_means)
        std_of_means = np.std(sample_means)
        theoretical_std = np.std(population) / np.sqrt(sample_size)

        axes[i, j + 1].text(0.02, 0.98,
                            f'Mean: {mean_of_means:.2f}\nStd: {std_of_means:.2f}\nTheoret
                            transform=axes[i, j + 1].transAxes,
                            verticalalignment='top',
                            bbox=dict(boxstyle='round', facecolor='white', alpha=0.8))

plt.tight_layout()
plt.show()

print("Central Limit Theorem Observations:")
print("1. As sample size increases, sampling distribution becomes more normal")
print("2. Mean of sampling distribution equals population mean")
print("3. Standard error = population std / sqrt(sample size)")
```

```
    print("4. This holds regardless of the original population distribution!")

demonstrate_central_limit_theorem()
```

Confidence Intervals

```
def confidence_intervals_analysis():
    """Analyze confidence intervals and their interpretation"""

    # True population parameters
    true_mean = 100
    true_std = 15

    # Generate samples and calculate confidence intervals
    sample_sizes = [10, 30, 100, 500]
    confidence_levels = [0.90, 0.95, 0.99]
    n_simulations = 1000

    results = {}

    for sample_size in sample_sizes:
        for conf_level in confidence_levels:
            coverage_count = 0
            intervals = []

            for _ in range(n_simulations):
                # Generate sample
                sample = np.random.normal(true_mean, true_std, sample_size)
                sample_mean = np.mean(sample)
                sample_std = np.std(sample, ddof=1)

                # Calculate confidence interval
                alpha = 1 - conf_level
                t_critical = stats.t.ppf(1 - alpha/2, df=sample_size - 1)
                margin_error = t_critical * (sample_std / np.sqrt(sample_size))

                ci_lower = sample_mean - margin_error
                ci_upper = sample_mean + margin_error

                intervals.append((ci_lower, ci_upper))

                # Check if interval contains true mean
                if ci_lower <= true_mean <= ci_upper:
                    coverage_count += 1
```

```
            coverage_rate = coverage_count / n_simulations
            results[(sample_size, conf_level)] = {
                'coverage_rate': coverage_rate,
                'intervals': intervals
            }

    # Display results
    print("Confidence Interval Coverage Analysis:")
    print("=" * 60)
    print(f"{'Sample Size':>12} {'Conf Level':>12} {'Coverage Rate':>15} {'Expected':>10}")
    print("-" * 60)

    for (sample_size, conf_level), result in results.items():
        coverage_rate = result['coverage_rate']
        print(f"{sample_size:>12} {conf_level:>12.0%} {coverage_rate:>15.1%} {conf_level:>10

    # Visualize confidence intervals for one case
    sample_size = 30
    conf_level = 0.95
    intervals = results[(sample_size, conf_level)]['intervals'][:50]  # Show first 50

    plt.figure(figsize=(12, 8))

    for i, (lower, upper) in enumerate(intervals):
        color = 'green' if lower <= true_mean <= upper else 'red'
        plt.plot([lower, upper], [i, i], color=color, linewidth=2, alpha=0.7)
        plt.plot([(lower + upper) / 2], [i], 'o', color=color, markersize=3)

    plt.axvline(true_mean, color='blue', linestyle='--', linewidth=2,
                label=f'True Mean = {true_mean}')
    plt.xlabel('Value')
    plt.ylabel('Sample Number')
    plt.title(f'{conf_level:.0%} Confidence Intervals (n={sample_size})')
    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.show()

    print(f"\nInterpretation:")
    print(f"- Green intervals contain the true mean ({true_mean})")
    print(f"- Red intervals do not contain the true mean")
    print(f"- About {conf_level:.0%} of intervals should be green")

confidence_intervals_analysis()
```

Statistical Inference

Hypothesis Testing Framework

```
def hypothesis_testing_framework():
    """Demonstrate hypothesis testing concepts"""

    print("Hypothesis Testing Framework:")
    print("=" * 50)

    # Example: Testing if a coin is fair
    print("Example: Testing if a coin is fair")
    print("H : p = 0.5 (coin is fair)")
    print("H : p  0.5 (coin is biased)")
    print()

    # Simulate coin flips
    n_flips = 100
    true_p = 0.6  # Biased coin (unknown to us)
    observed_heads = np.random.binomial(n_flips, true_p)
    observed_p = observed_heads / n_flips

    print(f"Observed: {observed_heads} heads out of {n_flips} flips")
    print(f"Sample proportion: {observed_p:.3f}")

    # Calculate test statistic (z-test for proportion)
    null_p = 0.5
    standard_error = np.sqrt(null_p * (1 - null_p) / n_flips)
    z_statistic = (observed_p - null_p) / standard_error

    # Calculate p-value (two-tailed test)
    p_value = 2 * (1 - stats.norm.cdf(abs(z_statistic)))

    print(f"\nTest Results:")
    print(f"Z-statistic: {z_statistic:.3f}")
    print(f"P-value: {p_value:.4f}")

    # Decision
    alpha = 0.05
    if p_value < alpha:
        print(f"Decision: Reject H  (p-value < {alpha})")
        print("Conclusion: Evidence suggests the coin is biased")
    else:
        print(f"Decision: Fail to reject H  (p-value  {alpha})")
        print("Conclusion: Insufficient evidence that the coin is biased")
```

```
    return z_statistic, p_value

z_stat, p_val = hypothesis_testing_framework()
```

Type I and Type II Errors

```
def error_types_analysis():
    """Analyze Type I and Type II errors"""

    # Simulation parameters
    null_mean = 100  # H :   = 100
    sample_size = 30
    alpha = 0.05
    n_simulations = 10000

    # Test different true means to see error rates
    true_means = np.arange(95, 106, 0.5)

    type_i_errors = []
    type_ii_errors = []
    power_values = []

    for true_mean in true_means:
        reject_count = 0

        for _ in range(n_simulations):
            # Generate sample from true distribution
            sample = np.random.normal(true_mean, 15, sample_size)
            sample_mean = np.mean(sample)

            # Perform t-test
            t_statistic = (sample_mean - null_mean) / (15 / np.sqrt(sample_size))
            p_value = 2 * (1 - stats.t.cdf(abs(t_statistic), df=sample_size - 1))

            if p_value < alpha:
                reject_count += 1

        rejection_rate = reject_count / n_simulations

        if true_mean == null_mean:
            # Type I error rate (rejecting true null)
            type_i_error_rate = rejection_rate
            type_i_errors.append(type_i_error_rate)
        else:
            # Power (correctly rejecting false null)
```

```
            power = rejection_rate
            power_values.append(power)

            # Type II error rate (failing to reject false null)
            type_ii_error_rate = 1 - power
            type_ii_errors.append(type_ii_error_rate)

    # Plot power curve
    plt.figure(figsize=(12, 8))

    plt.subplot(2, 1, 1)
    plt.plot(true_means[true_means != null_mean], power_values, 'b-', linewidth=2, label='Po
    plt.axhline(alpha, color='red', linestyle='--', label=f'  = {alpha}')
    plt.axvline(null_mean, color='gray', linestyle=':', alpha=0.7, label='H :   = 100')
    plt.xlabel('True Mean')
    plt.ylabel('Power (1 -  )')
    plt.title('Statistical Power Curve')
    plt.legend()
    plt.grid(True, alpha=0.3)

    plt.subplot(2, 1, 2)
    plt.plot(true_means[true_means != null_mean], type_ii_errors, 'r-', linewidth=2, label='
    plt.axhline(alpha, color='blue', linestyle='--', label=f'  = {alpha}')
    plt.axvline(null_mean, color='gray', linestyle=':', alpha=0.7, label='H :   = 100')
    plt.xlabel('True Mean')
    plt.ylabel('Type II Error Rate ( )')
    plt.title('Type II Error Rate')
    plt.legend()
    plt.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.show()

    print("Error Types Analysis:")
    print(f"Type I Error Rate ( ): {type_i_error_rate:.3f} (should be   {alpha})")
    print(f"Type II Error Rate varies with effect size")
    print(f"Power increases as true mean moves away from null hypothesis")

    # Effect size and power relationship
    effect_sizes = np.abs(true_means[true_means != null_mean] - null_mean) / 15

    plt.figure(figsize=(10, 6))
    plt.plot(effect_sizes, power_values, 'go-', linewidth=2, markersize=6)
    plt.xlabel('Effect Size (Cohen\'s d)')
    plt.ylabel('Statistical Power')
    plt.title('Power vs Effect Size')
    plt.grid(True, alpha=0.3)
```

```
    plt.axhline(0.8, color='red', linestyle='--', label='Conventional Power = 0.8')
    plt.legend()
    plt.show()

error_types_analysis()
```

Correlation and Causation

Correlation Analysis

```
def correlation_analysis():
    """Comprehensive correlation analysis"""

    # Generate different types of relationships
    n = 1000
    x = np.random.normal(0, 1, n)

    # Different correlation patterns
    relationships = {
        'Strong Positive': x + 0.2 * np.random.normal(0, 1, n),
        'Weak Positive': x + 2 * np.random.normal(0, 1, n),
        'No Correlation': np.random.normal(0, 1, n),
        'Strong Negative': -x + 0.2 * np.random.normal(0, 1, n),
        'Non-linear': x**2 + 0.5 * np.random.normal(0, 1, n)
    }

    fig, axes = plt.subplots(2, 3, figsize=(18, 12))
    axes = axes.flatten()

    correlations = {}

    for i, (name, y) in enumerate(relationships.items()):
        if i < len(axes):
            # Calculate correlations
            pearson_r, pearson_p = stats.pearsonr(x, y)
            spearman_r, spearman_p = stats.spearmanr(x, y)

            correlations[name] = {
                'pearson': pearson_r,
                'spearman': spearman_r,
                'pearson_p': pearson_p,
                'spearman_p': spearman_p
            }
```

```
            # Plot
            axes[i].scatter(x, y, alpha=0.6, s=20)
            axes[i].set_title(f'{name}\nPearson r = {pearson_r:.3f}\nSpearman   = {spearman_r
            axes[i].grid(True, alpha=0.3)

            # Add trend line for linear relationships
            if abs(pearson_r) > 0.3:
                z = np.polyfit(x, y, 1)
                p = np.poly1d(z)
                axes[i].plot(sorted(x), p(sorted(x)), "r--", alpha=0.8)

    # Remove empty subplot
    if len(relationships) < len(axes):
        fig.delaxes(axes[-1])

    plt.tight_layout()
    plt.show()

    # Display correlation summary
    print("Correlation Analysis Summary:")
    print("=" * 60)
    print(f"{'Relationship':>15} {'Pearson r':>12} {'p-value':>10} {'Spearman  ':>12} {'p-val
    print("-" * 60)

    for name, corr in correlations.items():
        print(f"{name:>15} {corr['pearson']:>12.3f} {corr['pearson_p']:>10.4f} "
              f"{corr['spearman']:>12.3f} {corr['spearman_p']:>10.4f}")

    print("\nInterpretation Guidelines:")
    print("Pearson correlation (r):")
    print("  |r| < 0.3: Weak relationship")
    print("  0.3   |r| < 0.7: Moderate relationship")
    print("  |r|   0.7: Strong relationship")
    print("\nSpearman correlation ( ): Measures monotonic relationships")
    print("Use when relationship is not linear but monotonic")

correlation_analysis()
```

Spurious Correlations and Confounding

```
def spurious_correlation_example():
    """Demonstrate spurious correlations and confounding variables"""

    n = 1000
```

```python
# Example 1: Ice cream sales and drowning deaths
# Both are caused by temperature (confounding variable)
temperature = np.random.normal(75, 10, n)  # Temperature in Fahrenheit

# Ice cream sales increase with temperature
ice_cream_sales = 50 + 2 * (temperature - 70) + np.random.normal(0, 10, n)
ice_cream_sales = np.maximum(ice_cream_sales, 0)  # Can't be negative

# Drowning deaths also increase with temperature (more swimming)
drowning_deaths = 2 + 0.1 * (temperature - 70) + np.random.poisson(1, n)
drowning_deaths = np.maximum(drowning_deaths, 0)

# Calculate correlations
corr_ice_drowning = stats.pearsonr(ice_cream_sales, drowning_deaths)[0]
corr_temp_ice = stats.pearsonr(temperature, ice_cream_sales)[0]
corr_temp_drowning = stats.pearsonr(temperature, drowning_deaths)[0]

# Partial correlation (controlling for temperature)
from scipy.stats import pearsonr

# Simple partial correlation calculation
def partial_correlation(x, y, z):
    """Calculate partial correlation of x and y controlling for z"""
    rxy = pearsonr(x, y)[0]
    rxz = pearsonr(x, z)[0]
    ryz = pearsonr(y, z)[0]

    partial_r = (rxy - rxz * ryz) / (np.sqrt(1 - rxz**2) * np.sqrt(1 - ryz**2))
    return partial_r

partial_corr = partial_correlation(ice_cream_sales, drowning_deaths, temperature)

# Visualize
fig, axes = plt.subplots(2, 2, figsize=(15, 12))

# Ice cream vs drowning (spurious correlation)
axes[0, 0].scatter(ice_cream_sales, drowning_deaths, alpha=0.6)
axes[0, 0].set_xlabel('Ice Cream Sales')
axes[0, 0].set_ylabel('Drowning Deaths')
axes[0, 0].set_title(f'Spurious Correlation\nr = {corr_ice_drowning:.3f}')
axes[0, 0].grid(True, alpha=0.3)

# Temperature vs ice cream
axes[0, 1].scatter(temperature, ice_cream_sales, alpha=0.6, color='orange')
axes[0, 1].set_xlabel('Temperature (°F)')
axes[0, 1].set_ylabel('Ice Cream Sales')
axes[0, 1].set_title(f'Temperature → Ice Cream\nr = {corr_temp_ice:.3f}')
```

```
    axes[0, 1].grid(True, alpha=0.3)

    # Temperature vs drowning
    axes[1, 0].scatter(temperature, drowning_deaths, alpha=0.6, color='red')
    axes[1, 0].set_xlabel('Temperature (°F)')
    axes[1, 0].set_ylabel('Drowning Deaths')
    axes[1, 0].set_title(f'Temperature → Drowning\nr = {corr_temp_drowning:.3f}')
    axes[1, 0].grid(True, alpha=0.3)

    # Residual plot (controlling for temperature)
    ice_cream_residuals = ice_cream_sales - (np.mean(ice_cream_sales) +
                                             corr_temp_ice * (temperature - np.mean(temperatur
    drowning_residuals = drowning_deaths - (np.mean(drowning_deaths) +
                                            corr_temp_drowning * (temperature - np.mean(temper

    axes[1, 1].scatter(ice_cream_residuals, drowning_residuals, alpha=0.6, color='green')
    axes[1, 1].set_xlabel('Ice Cream Sales (residuals)')
    axes[1, 1].set_ylabel('Drowning Deaths (residuals)')
    axes[1, 1].set_title(f'Controlling for Temperature\nPartial r = {partial_corr:.3f}')
    axes[1, 1].grid(True, alpha=0.3)

    plt.tight_layout()
    plt.show()

    print("Spurious Correlation Example:")
    print("=" * 50)
    print(f"Ice cream sales  Drowning deaths: r = {corr_ice_drowning:.3f}")
    print(f"Temperature → Ice cream sales: r = {corr_temp_ice:.3f}")
    print(f"Temperature → Drowning deaths: r = {corr_temp_drowning:.3f}")
    print(f"Partial correlation (controlling for temperature): r = {partial_corr:.3f}")
    print()
    print("Key Insight:")
    print("The correlation between ice cream sales and drowning deaths")
    print("disappears when we control for the confounding variable (temperature).")
    print("This demonstrates that correlation  causation!")

spurious_correlation_example()
```

Summary

Statistical foundations provide the mathematical framework for:

1. **Data Description**: Summarizing and characterizing datasets
2. **Uncertainty Quantification**: Understanding variability and confidence
3. **Inference**: Making conclusions about populations from samples
4. **Hypothesis Testing**: Evaluating claims using statistical evidence

5. **Relationship Analysis**: Understanding associations between variables

Key principles: - **Sampling variability** affects all statistical conclusions - **Central Limit Theorem** enables many statistical procedures - **Confidence intervals** quantify uncertainty in estimates - **Hypothesis testing** provides a framework for decision-making - **Correlation does not imply causation** - always consider confounding variables

These concepts form the foundation for more advanced data science techniques including machine learning, experimental design, and causal inference.

Sampling, Bias, and Representativeness

Data science fails when the **sample is not the population you care about**. This chapter covers sampling design, bias versus variance, standard errors for means and proportions, and the train/validation/test discipline that keeps estimates honest.

Diagram: target vs sample

```
target population  (who you want to claim about)

          sampling frame (who you can reach)
        v
sample S   frame

        v
estimate ^    ?     for target
```

If frame target (missing groups, volunteers only), even huge n does not save you. Precision is not validity.

1. Populations, frames, and units

Concept	Meaning
Target population	group your claim is about
Sampling frame	list/process that can be sampled
Sampling unit	person, session, device, day, …
Observation	measured fields on a unit

Mismatch examples: claim about all users, frame = users who opened the app last week; claim about households, unit = individuals.

2. Bias vs variance of estimators

For estimator $\hat{\theta}$ of θ:

$$\text{Bias}(\hat{\theta}) = \mathbb{E}[\hat{\theta}] - \theta, \qquad \text{Var}(\hat{\theta}) = \mathbb{E}[(\hat{\theta} - \mathbb{E}\hat{\theta})^2],$$

$$\text{MSE}(\hat{\theta}) = \text{Bias}^2 + \text{Var}.$$

```
high bias, low var     low bias, high var
```

- **Bias:** systematic error (wrong frame, bad instrument, leakage)
- **Variance:** noise from finite samples (reduce with larger n or better design)

Huge n kills variance; it does **not** kill selection bias.

3. Common sampling schemes

Scheme	Idea	Watch-out
Simple random (SRS)	equal chance	needs full list
Stratified	sample within groups	need strata sizes / weights
Cluster	sample groups then members	design effect; larger SE
Systematic	every k-th unit	periodicity risk
Convenience	easy data	often biased
Snowball	recruit via networks	hidden population bias

Worked example 1 — stratification

Estimate mean income in a city with 20% high-income, 80% low. Stratify and sample both groups; weight by population shares. Variance of the overall mean often smaller than SRS for the same n when strata means differ.

Worked example 2 — cluster design effect

Randomly sample classrooms then students: students in a class correlate. Effective sample size $< n$; SEs from i.i.d. formulas are too optimistic without design correction.

4. Selection bias catalogue

Bias	Mechanism
Survivorship	only "winners" logged
Voluntary response	strong opinions over-represented
Coverage	frame misses parts of target
Temporal / drift	train on past, deploy on shifted future
Attrition	dropouts differ from stayers
Leakage	feature contains label information
Berkson	conditioning on a collider induces fake associations
Length-biased	longer sessions more likely sampled

Worked example 3 — help-click logs

You only log users who click "Help." Population = help seekers, not all users. Metrics of "confusion" do not generalize to power users who never click Help.

Worked example 4 — survivorship in startups

Studying only surviving companies' practices ignores failed companies that did the same things — classic survivorship bias.

5. Measurement bias vs sampling bias

- **Sampling bias:** wrong units enter the sample
- **Measurement bias:** units OK, but instrument systematically wrong (bad sensor, leading survey question, timezone bugs)

Both bias $\widehat{\theta}$; fixes differ (redesign sample vs recalibrate instrument).

6. Standard errors (quick formulas)

Assume i.i.d. sample for the formulas below (cluster samples need different SE).

Sample mean, variance σ^2 estimated by s^2:

$$\mathrm{SE}(\bar{X}) \approx \frac{s}{\sqrt{n}}.$$

Sample proportion $\widehat{p}$:

$$\mathrm{SE}(\widehat{p}) \approx \sqrt{\frac{\widehat{p}(1-\widehat{p})}{n}}.$$

Margin of error for approximate 95% CI (large sample):

$$\text{ME} \approx 1.96 \cdot \text{SE}.$$

Scaling law

To **halve** SE for an i.i.d. mean, need 4× sample size ($1/\sqrt{n}$).

Worked example 5 — poll

$\hat{p} = 0.52$, $n = 1000$:

$$\text{SE} \approx \sqrt{0.52 \cdot 0.48/1000} \approx 0.0158, \quad 95\%\ \text{ME} \approx 0.031.$$

Report $52\% \pm 3.1\%$ (approx), not 52.173%.

Worked example 6

$\hat{p} = 0.1$, $n = 400$: $\text{SE} = \sqrt{0.1 \cdot 0.9/400} = \sqrt{0.09/400} = 0.015$.

7. Weighting and post-stratification

If sample strata proportions differ from population, **reweight**:

$$\hat{\theta} = \sum_s w_s \hat{\theta}_s,$$

with w_s population shares. Incorrect weights bias; correct weights can increase variance.

8. Train / validation / test

```
all labeled data

        train (fit)
        validation (tune hyperparameters)
        test (final estimate of risk)  ← touch once if possible
```

Reusing test for decisions = optimistic bias (a form of selection on the evaluation path).

Time-series: split by time, not i.i.d. shuffle — random splits leak the future.

Grouped data: split by user/entity so the same user is not in train and test.

9. Online A/B sampling notes

- Randomize **units** (users) not events when interference exists
- Spillover / network effects bias treatment effect estimates
- Novelty effects: short windows mislead
- SRM (sample ratio mismatch): check assignment proportions

10. Power and planning (tie-in)

Before collecting data, rough SE formulas plan n for a desired ME. Underpowered studies produce noisy $\hat{\theta}$ and encourage p-hacking — see hypothesis-testing chapter.

11. Pitfalls

1. "n is large so we are fine" under biased sampling
2. i.i.d. SE on clustered data
3. Shuffled CV on time-dependent logs
4. Convenience samples sold as representative panels
5. Training on label-leaking features

12. Checkpoint

- Distinguish target, frame, sample
- Write MSE = bias^2 + var
- Apply SE formulas for mean and proportion
- Halving SE needs $4 \times n$
- Name major selection biases
- Design train/val/test without leakage

Exercises

Easy

1. If SE must be half as large, how must n change (i.i.d. mean)?
2. Give a real-world convenience sample that misleads product metrics.
3. Compute SE for $\hat{p} = 0.1$, $n = 400$.
4. Distinguish **sampling bias** vs **measurement bias**.
5. **Design:** You only log users who click "Help". What population is that?

Medium

6. Why stratified sampling can beat simple random for the same n?
7. Poll $\hat{p} = 0.52$, $n = 1000$: compute approximate 95% ME.
8. Explain design effect for cluster sampling in one paragraph.
9. Why random train/test splits fail for forecasting next month's demand?
10. Survivorship bias: invent a tech industry example.

Challenge

11. Derive $\mathrm{Var}(\hat{p}) = p(1-p)/n$ for i.i.d. Bernoulli.
12. Post-stratification: write a weighted mean estimator with two strata.
13. Berkson bias: sketch a collider diagram that induces a spurious correlation.
14. Bootstrap SE vs analytic SE: when prefer bootstrap?
15. Plan n so that 95% ME for a proportion near 0.5 is at most 0.02.

Checks

1. $n \times 4$.
2. $\sqrt{0.09/400} = 0.015$.

3. ≈ 3.1 percentage points.

4. $1.96\sqrt{0.25/n} \leq 0.02 \quad n \gtrsim (1.96 \cdot 0.5/0.02)^2 \approx 2401.$

Summary

Good data science starts with **who is in the data**. Bias from frames and selection cannot be fixed by collecting the same wrong population faster. Variance shrinks with n and smart design; standard errors quantify that under stated assumptions. Keep evaluation splits as clean as your sampling story — both are about whether $\hat{\theta}$ means what you say it means.

Hypothesis Testing for Data Work

A practical companion to formal inference: how data scientists use tests, what to report, and how not to fool yourself with dashboards. Pair with the sampling chapter — a significant p-value on a biased sample is still wrong for the target population.

Diagram: decision pipeline

```
metric definition → data window → estimator

       v
pre-registered H0/H1 and

       v
test + CI + effect size + plot

       v
decision + ship/hold + monitor
```

1. Pick the question first

Bad: "run t-tests on all columns."
Good: one **primary metric**, optional secondary metrics, written **before** peeking at outcomes.

Write:

- Population / time window / exclusions
- Causal or descriptive claim?
- Practical threshold worth caring about (not only statistical)

2. Hypotheses and errors

For a parameter θ (mean lift, difference of proportions, …):

- H_0: default (often "no difference" $\theta = 0$)

- H_1: alternative (one-sided or two-sided)

Error	Meaning	Control
Type I	reject H_0 when true	significance level α
Type II	fail to reject when H_1 true	power $1 - \beta$

p-value: under a well-specified H_0 model, probability of data as extreme as observed (or more) in the direction of the test. **Not** $P(H_0 \mid \text{data})$.

Worked example 1 — ranking CTR

H_0: new ranking does not change CTR ($\theta = 0$).
H_1: $\theta > 0$ (one-sided) if only lift matters and regression is not a ship blocker — product policy chooses the side.

3. Common tests (when)

Situation	Typical tool
Mean vs constant	one-sample t
Two group means	two-sample t / **Welch**
Two proportions	z-test / χ^2
Paired before/after	paired t
Many categories	χ^2 goodness / independence
Non-normal small n	rank tests / **bootstrap**
Many predictors	regression coefficients + robust SE

Welch vs Student

Welch's t-test does not assume equal variances — usually preferred for two web variants with unequal traffic or heteroscedasticity.

4. Confidence intervals first-class

A $(1 - \alpha)$ CI is the set of parameter values not rejected at level α (for many standard tests). Report:

$$\hat{\theta} \pm z_{1-\alpha/2} \cdot \text{SE}$$

(large-sample) **and** interpret width vs practical importance.

Worked example 2

$\bar{x} = 10$, $s = 2$, $n = 100$: SE $= 0.2$, approx 95% CI

$$10 \pm 1.96 \cdot 0.2 \approx [9.61,\ 10.39].$$

5. Effect size always

Examples:

- Difference of means $\mu_B - \mu_A$
- Relative lift $(p_B - p_A)/p_A$
- Cohen's $d = (\bar{x}_1 - \bar{x}_2)/s_{\text{pooled}}$

A tiny lift can be "significant" with huge traffic and still not worth engineering cost. **Statistical significance practical significance.**

Worked example 3

Baseline conversion 10%, lift +0.05 percentage points absolute, n enormous $p < 0.001$ but maybe irrelevant if redesign costs months.

6. Bootstrap intuition

Resample the sample with replacement many times; build empirical distribution of $\hat{\theta}^\star$.

```
original sample:
bootstrap 1:              (with replacement)
bootstrap 2:
...
→ histogram of ^*  → percentile CI
```

Pros: few analytic assumptions; good for messy statistics.
Cons: still assumes sample population of interest; i.i.d. resampling wrong for dependence (time series, clusters) without block bootstrap variants.

Bootstrap vs normal CI

Normal / analytic	Bootstrap
Fast, closed form	Flexible for complex $\hat{\theta}$
Needs SE formula / asymptotics	Needs compute and exchangeability

Normal / analytic	Bootstrap

7. Multiple metrics and peeking

Multiple testing

m independent true-null tests at level α ⇒ chance of at least one false positive $\approx 1 - (1-\alpha)^m$.

Fixes: primary metric only for go/no-go; Bonferroni / FDR for exploration; hierarchical testing.

Optional stopping

Checking the A/B dashboard every hour and stopping when $p < 0.05$ **inflates Type I error** (p-hacking / peeking).

Fixes: fixed horizon; sequential tests (alpha spending); always-valid inference methods; pre-commit sample size.

Worked example 4

"We stopped when p hit 0.05 on day 3 of 14" — report is not a valid level-α fixed-horizon test.

8. Power and sample size (order of magnitude)

Roughly, for two proportions near p and lift δ, n per arm scales like

$$n \propto \frac{p(1-p)}{\delta^2}$$

for fixed power and α. Tiny δ ⇒ huge n. Underpowered tests produce noisy results and encourage chasing significance.

9. Practical report template

1. Hypothesis in plain language
2. Population / time window / exclusions
3. Sample sizes (per arm)
4. Estimator + CI

5. Test + p-value (if used)
6. Effect size vs practical threshold
7. Caveats (bias, seasonality, interference, SRM)
8. Decision + monitoring plan

10. Decision beyond p

Scenario	Approach
Revenue up, latency worse	multi-objective / constraints
Significant but tiny lift	cost–benefit
Not significant, CI includes large lifts	inconclusive — more data or redesign
Significant on secondary only	do not silently promote to primary

11. Bayesian glance (optional)

Posterior $p(\theta \mid \text{data})$ and $P(\theta > 0 \mid \text{data})$ answer different questions than p-values. Still requires priors and still fails under selection bias. Complementary, not magic.

12. Pitfalls

1. p-hacking via metric shopping
2. Optional stopping without sequential correction
3. Ignoring dependence (users in both arms, networks)
4. Interpreting p as effect size
5. Two-sided vs one-sided chosen after seeing the sign
6. χ^2 with tiny expected cell counts

13. Checkpoint

- Write H_0, H_1 for a product change
- Prefer CI + effect size alongside p

- Name Welch vs Student use case
- Explain peeking's Type I inflation
- Bootstrap idea in one paragraph
- Ship decisions with practical thresholds

Exercises

Easy

1. Write H_0, H_1 for "new ranking increases CTR."
2. Why is Welch's t-test often preferred over Student's for two web variants?
3. Bootstrap CI vs normal CI — one advantage each.
4. You check the A/B dashboard every hour and stop when $p < 0.05$. Name the problem.
5. Compute rough 95% CI for mean with $\bar{x} = 10$, $s = 2$, $n = 100$.

Medium

6. **Scenario:** significance on revenue, not on latency — how do you decide ship?
7. Ten independent metrics, all true nulls, $\alpha = 0.05$: approx P(at least one significant)?
8. Explain why p is not $P(H_0 \mid \text{data})$.
9. Paired vs unpaired test for before/after on the same users.
10. Relative lift vs absolute difference: give a case where relative misleads.

Challenge

11. Simulate (conceptually) optional stopping: why Type I exceeds α.
12. Derive two-proportion z-statistic for $\hat{p}_1 - \hat{p}_2$.
13. FDR vs FWER: when prefer Benjamini–Hochberg?
14. Power curve: sketch power vs n for fixed δ, α.
15. Cluster-randomized experiment: why user-level i.i.d. tests fail.

Checks

4. Optional stopping / p-hacking.

5. $10 \pm 1.96 \cdot 0.2 \approx [9.61, 10.39]$.

6. $1 - (0.95)^{10} \approx 0.40$.

Summary

Hypothesis tests are decision tools under noise, not oracles of truth. Pre-register the primary metric, report effect sizes and CIs, respect multiple comparisons and peeking, and never confuse a small p-value with a large or useful effect. The best analysis is the one whose assumptions match how the data were sampled and how the product will use the result.

Dimensionality, Features, and the Curse

High-dimensional data is normal in text, images, and logs. Geometry changes: distances concentrate, nearest neighbors weaken, and **feature design** often matters more than the choice among similar models.

Diagram: curse sketch

```
d=1     points spread on a line
d=2     square
d=3     cube
d large volume concentrates near the "shell"
          most mass far from center
```

1. The curse of dimensionality (geometry)

In high d, several effects appear for "typical" random data:

1. **Volume concentration:** most of a high-d ball's volume is near the surface
2. **Distance concentration:** pairwise distances between random points become relatively similar
3. **Sparsity:** fixed n samples cover $[0,1]^d$ vanishingly; local neighborhoods empty

Consequence: k-NN, distance-based density, and some clustering methods degrade unless structure (manifold, sparsity, metric learning) is exploited.

Worked example 1 — intuition

In high d, ratio of max to min distance among random pairs often approaches 1 — "nearest" and "farthest" neighbors hard to distinguish.

When the curse is milder

- Data lie near a low-dimensional **manifold**
- Features are sparse (few nonzeros per row)
- Strong signal in few coordinates
- Task uses inner products / learned metrics, not raw Euclidean distance

2. Feature types and encodings

Type	Examples	Encoding notes
Numeric	age, latency	scale / normalize; watch outliers
Categorical	country	one-hot / target / embeddings
Ordinal	rating 1–5	not always linear spacing
Text	tokens	BoW, TF-IDF, embeddings
Time	timestamps	cyclic (sin/cos), lags, calendars
IDs	user_id	embeddings; rare IDs $\rightarrow$ unk
Multi-hot	tags	sparse binary vector

Worked example 2 — one-hot countries

200 countries $\quad d \approx 200$ binary features (or 199 with a reference level in regression). High cardinality (millions of IDs) makes one-hot impractical — prefer embeddings or hashing.

3. Scaling and leakage

Standardize: $(x - \mu)/\sigma$ per feature.
Min-max: map to $[0, 1]$.
Robust: median / IQR for heavy tails.

Models sensitive to scale	Often less sensitive
Linear / logistic / SVM / k-NN / k-means / neural nets / PCA	Tree ensembles (axis-aligned splits)

Leakage rule: fit scalers, imputers, PCA, target encoders on **train only**, then apply to val/test. Using full-data μ, σ leaks future information into training transforms.

Worked example 3 — PCA scaling

PCA maximizes variance. A feature measured in milliseconds (large numbers) dominates one in kilometers if unscaled — not because it is more informative.

4. Dimensionality reduction map

```
raw features R^d

        PCA / SVD (linear subspace)
        random projections (JL lemma)
        feature selection (filter / wrapper)
        learned embeddings (nonlinear)

            v
       R^k  k   d   for viz / speed / denoise
```

PCA: directions of maximum variance (see advanced LA / SVD chapters).
Johnson–Lindenstrauss: random projections can preserve pairwise distances approximately with $k = \mathcal{O}(\varepsilon^{-2} \log n)$.
Feature selection: drop coordinates; MI / regularization / tree importances (careful with interpretation).

5. Feature crosses and interactions

A linear model on $[x_1, x_2]$ cannot represent an AND-like interaction $x_1 x_2$ without an explicit product feature or a nonlinear model (trees, nets, kernels).

Worked example 4

Spam: "free" and "money" each mild; co-occurrence strong. Cross feature $\mathbf{1}_{\text{free}} \cdot \mathbf{1}_{\text{money}}$ or use a model that builds interactions.

6. Sparsity

One-hot, n-grams, multi-hot tags sparse high-d vectors.

- Algorithms: sparse matrix formats, sparse-aware linear models
- Regularization: L_1 selects features; elastic net for correlated groups
- Hashing trick: map tokens to 2^b bins with collisions — controlled approximation

7. Metrics: when Euclidean fails

Similarity	Use when
Euclidean	dense, scaled numeric
Cosine	text BoW / TF-IDF (direction > length)
Manhattan	some sparse / robust settings
Learned (Mahalanobis, metric learning)	task-specific distances

Worked example 5 — cosine for documents

Long documents have larger BoW counts; cosine normalizes length so similarity is about relative word composition.

8. The "average distance to k-NN" trap

In high d, novelty scores based on distance to neighbors can become uninformative as distances concentrate. Prefer:

- Domain features
- Dimensionality reduction first
- One-class models with appropriate kernels
- Reconstruction error of autoencoders / PCA (with caveats)

9. Embeddings vs one-hot

One-hot	Embedding
No notion of similarity between levels	Learned similarity
$d = \#$ levels	$d =$ embedding dim (small)
Interpretable coefficients	Dense geometry
Great for low cardinality	High-cardinality IDs, words, items

Worked example 6

User IDs: millions of levels. Embedding dim 32–128 common in recommenders; cold-start needs defaults / content features.

10. Pipeline checklist

1. Define label and unit of analysis
2. Split train/val/test **before** heavy featurization that needs fit
3. Encode categoricals; handle unknowns
4. Scale if model requires
5. Optional: select / reduce dimensions on train
6. Fit model; evaluate on held-out with same transforms
7. Inspect errors; iterate features

11. Pitfalls

1. Scaling with full-data statistics
2. Target encoding without CV (leakage)
3. Huge one-hots into dense neural nets without embeddings
4. Interpreting PCA components as causal factors
5. k-NN in raw high-d Euclidean space
6. Dropping rare categories inconsistently between train and serve

12. Checkpoint

- Explain distance concentration
- Choose encodings for numeric / categorical / text
- Scale with train-only fit
- Sketch PCA vs embeddings vs selection
- Know when cosine beats Euclidean
- State leakage risks in feature pipelines

Exercises

Easy

1. Why is cosine similarity common for text bags-of-words?
2. If all features are one-hot countries (200 levels), what is d roughly?
3. Give one reason to standardize before PCA.
4. Explain train-only fitting of scalers (no leakage).
5. Sketch when embeddings beat one-hot for high-cardinality IDs.

Medium

6. **Thought:** In high d, why might "average distance to 10-NN" stop being a good novelty score?
7. Feature cross: write a linear model that includes $x_1, x_2, x_1 x_2$.
8. Hashing trick: what happens when two tokens collide?
9. Compare L_1 vs L_2 regularization for sparse one-hot features.
10. Time feature: encode hour of day with sin/cos; why not integer 0..23 alone for linear models?

Challenge

11. JL lemma idea: why random projections preserve distances with high probability (statement-level).
12. Show that for i.i.d. standard Gaussian coordinates, $\|x\|_2/\sqrt{d} \to 1$ in probability — concentration sketch.
13. Design a leakage-safe target encoding with K-fold scheme.
14. Manifold assumption: how does it reconcile high ambient d with successful k-NN on images?
15. End-to-end: propose features for churn prediction; mark which need fit-on-train only.

Checks

1. Direction matters more than length; normalizes document length.

2. ≈ 200 (or 199 with reference level).

3. PCA is not scale-invariant; large-scale features dominate variance.

Summary

High dimension changes geometry and stress-tests naive distance methods. Strong pipelines encode features carefully, scale without leakage, reduce or embed when needed, and pick metrics that match the data type. Feature design remains a primary lever in data science math — models only see the representation you build.

Probability (Advanced)

Advanced Probability for CS and AI

Probability is the mathematics of uncertainty. Advanced probability for CS/AI goes beyond coin flips: random variables and tails, concentration, limit theorems, Bayesian updating, and stochastic processes (especially Markov chains) that model systems over time.

Why go beyond "basic stats"

Task	Probability tool
Generalization bounds	Concentration inequalities
A/B testing at scale	CLT, sequential tests
Reliability / SLOs	Tail probabilities, Markov models
Bayesian ML	Priors, posteriors, predictives
MCMC / sampling	Markov chain stationary laws
Queues, caches	Birth-death / Markov processes
Randomized algorithms	Expectation + high probability

Learning path

```
flow:
  B -> C
  B -> D
  C -> E
  D -> F
  E -> F
```

Random experiments and models

A **probability space** $(\Omega, \mathcal{F}, P)$ underpins everything; CS practice often works with explicit PMFs/PDFs and independence assumptions. Always ask: what is random—data, noise, algorithm coins, or adversary?

Worked example 1

Hashing analysis: randomness over hash function choice. ML: randomness over training sample. Both use expectation, but the measure differs.

Worked example 2 — heavy tails

Latency distributions often have heavy tails; means can mislead SLOs. Medians and quantiles need different concentration tools than sub-Gaussian bounds.

Roadmap of chapters

1. Random variables and distributions
2. Expectation, variance, covariance
3. LLN, CLT, and consequences
4. Bayesian inference
5. Markov chains
6. Thresholds in random structures (expectation thresholds, Park–Pham)

Prerequisites

Discrete probability, basic calculus (for continuous densities), comfort with sums/series. Linear algebra helps for Markov chains ($\pi P = \pi$).

Three calculation templates

Expectation via indicators

To count events A_i, set $X = \sum_i \mathbf{1}_{A_i}$. Then $\mathbb{E}[X] = \sum_i P(A_i)$ even when the A_i are dependent. This single trick powers balls-and-bins, hashing, and randomized algorithm analyses.

Concentration sketch

Markov/Chebyshev give weak tails; Chernoff/Hoeffding give exponential tails for bounded or sub-Gaussian sums. High-probability bounds turn "expected $O(f(n))$" into "$O(f(n))$ except with probability n^{-c}."

Bayes update

Posterior $\propto$ likelihood $\times$ prior. For conjugate pairs (Beta–Binomial, Normal–Normal), the update is closed form; otherwise use MCMC or variational methods.

Worked example 3 — A/B test math stack

Conversion indicators Bernoulli; difference of means; CLT se; optional Beta posteriors for $P(p_A > p_B \mid \text{data})$. Same product question, multiple probabilistic languages.

Worked example 4 — PageRank as probability

Random surfer is a Markov chain; rank is stationary mass. Teleportation ensures irreducibility so the stationary distribution exists and is unique.

Measure-theoretic honesty (light)

Advanced texts use σ-algebras and almost-sure statements. For this book, discrete PMFs and continuous PDFs suffice, with the warning that continuous densities need care ($P(X = x) = 0$) and that "with probability 1" is stronger than "in expectation."

Pitfalls

1. Assuming i.i.d. when data are dependent
2. Confusing almost-sure, in-probability, and in-distribution convergence
3. Using normal approximations for tiny n or infinite variance
4. Priors that dominate small data without noticing
5. Mixing ensemble averages with time averages without ergodicity
6. Treating correlation as independence

Checkpoint

- Name five CS uses of advanced probability
- Distinguish PMF, PDF, CDF
- State LLN and CLT in one line each
- Write Bayes' rule for parameters
- Define Markov property

Exercises

1. Give examples of discrete vs continuous RVs in systems telemetry.
2. Why might $\mathbb{E}[T]$ be infinite for a positive RV used as a timer? (heavy tail sketch)
3. Explain independence vs uncorrelatedness.
4. What does "95% confidence interval" **not** mean about a fixed θ?
5. Sketch a two-state Markov chain for a machine up/down model.
6. When does Monte Carlo integration error scale as $1/\sqrt{n}$?
7. Bayesian vs frequentist one-paragraph contrast on a coin bias.
8. Checkpoint: pick a randomized algorithm and identify the probability space.

Summary

This part upgrades probability from formulas to modeling discipline: distributions, moments, limits, Bayesian updates, and Markov dynamics—the probabilistic spine of modern computing and ML.

Random Variables and Distributions

Random variables (RVs) attach numbers to uncertain outcomes so we can compute probabilities, expectations, and algorithms that depend on chance. This chapter builds discrete and continuous RVs, CDFs, standard families, and CS-facing approximations.

1. Random variables

Definition. On a probability space $(\Omega, \mathcal{F}, P)$, a random variable is a measurable function $X : \Omega \to \mathbb{R}$.

- **Discrete:** countable image; **PMF** $p_X(x) = P(X = x)$ with $\sum_x p_X(x) = 1$
- **Continuous: PDF** f_X with $P(X \in A) = \int_A f_X(x)\,dx$ and $\int f_X = 1$
- **Mixed:** atoms + density (e.g. spike-and-slab)

CDF: $F_X(t) = P(X \le t)$, always defined, nondecreasing, right-continuous, $F(-\infty) = 0$, $F(+\infty) = 1$.

Worked example 1

Die fair: $p(k) = 1/6$ for $k = 1..6$. $F(3.2) = P(X \le 3.2) = 1/2$.

Worked example 2

Uniform$(0, 1)$: $f(x) = 1$ on $(0, 1)$, $F(t) = t$ for $t \in [0, 1]$.

2. Transformations

If $Y = g(X)$ monotone smooth on support of continuous X,

$$f_Y(y) = f_X(x(y)) \left| \frac{dx}{dy} \right|.$$

Worked example 3

$X \sim N(0, 1)$, $Y = \mu + \sigma X \Rightarrow Y \sim N(\mu, \sigma^2)$.

Worked example 4 — inverse transform sampling

If $U \sim \text{Unif}(0,1)$ and F invertible, $X = F^{-1}(U)$ has CDF F. Basis of many generators.

3. Discrete families

Family	PMF / meaning	CS use
Bernoulli(p)	success bit	clicks, errors
Binomial(n, p)	# successes in n trials	A/B counts
Geometric(p)	trials until first success	retries
Poisson(λ)	rare events count	arrivals, defects
Zipf / power-law	heavy rank frequencies	words, degrees

Worked example 5 — Poisson limit

$n \to \infty$, $p \to 0$, $np \to \lambda$: $\text{Bin}(n,p) \Rightarrow \text{Poisson}(\lambda)$. Models rare hash collisions / faults.

Theorem (Poisson approximation sketch). Expand binomial coefficient and $(1-p)^{n-k} \to e^{-\lambda}$ under $np = \lambda$.

4. Continuous families

Family	Density highlight	CS use
Uniform(a, b)	flat	sampling, hashing mods (care)
Normal(μ, σ^2)	bell; CLT limit	noise, asymptotics
Exponential(λ)	$f(x) = \lambda e^{-\lambda x}$ $x > 0$	interarrival, memoryless
Gamma / Erlang	sums of exponentials	waiting times
Beta(α, β)	on $(0,1)$	Bayesian rates
Log-normal	$\log X$ normal	some size distributions

Worked example 6 — Gaussian tails

$P(|Z| > 3) \approx 0.0027$ for standard normal—rule-of-thumb anomaly thresholds (with caution on non-Gaussian data).

5. Memoryless property

Theorem. Among discrete laws on $\{1, 2, ...\}$, geometric is memoryless:

$$P(X > s + t \mid X > s) = P(X > t).$$

Among continuous positive laws, exponential is memoryless.

Worked example 7

Packet retransmission geometric trials: remaining trials given failures so far has the same law—modeling simplification (and limitation if wear-out exists).

6. Joint distributions and independence

Joint PMF $p(x, y)$; marginal $p_X(x) = \sum_y p(x, y)$.
Independence: $p(x, y) = p_X(x)p_Y(y)$ (or $f(x, y) = f_X f_Y$).

Conditional: $p(x \mid y) = p(x, y)/p_Y(y)$.

Worked example 8

Two independent Poissons: sum is Poisson. Not true for arbitrary families.

Worked example 9 — Bayes in classification

$P(Y \mid X) \propto P(X \mid Y)P(Y)$—Naive Bayes assumes feature independence given class.

7. Quantiles and heavy tails

The p-quantile $q_p = \inf\{t : F(t) \geq p\}$. Medians are $q_{1/2}$.

Heavy tails: $P(|X| > t)$ decays slower than exponential; variance may be infinite—sample means unstable.

Worked example 10 — Pareto

$P(X > x) = (x_m/x)^\alpha$ for $x \geq x_m$. If $\alpha \leq 2$, variance infinite.

8. Moment-generating / characteristic functions (awareness)

$M(t) = \mathbb{E}[e^{tX}]$ (when finite) determines distributions and eases sums of independents. Characteristic function $\mathbb{E}[e^{itX}]$ always exists—tool for CLT proofs.

9. Pitfalls

1. Continuous RVs: $P(X = x) = 0$ always for densities
2. Misusing normal for bounded data without transform
3. Assuming uniqueness of mean as "typical" under skew
4. Independence assumed for convenience, false in time series
5. Integer Poisson vs continuous approx without continuity correction when needed

10. Checkpoint

- Define PMF/PDF/CDF and compute simple probabilities
- Use Poisson approximation appropriately
- State memoryless laws
- Factor joints under independence
- Discuss quantiles vs means for SLO metrics

Exercises

Easy

1. Compute $P(X \geq 2)$ for $X \sim \text{Bin}(5, 1/2)$.
2. CDF of Exponential(λ); median in terms of λ.
3. If $X \sim \text{Unif}(0, 1)$, law of $Y = -\log X$?
4. Show Bernoulli variance $p(1 - p)$.
5. Give a joint PMF on $\{0, 1\}^2$ with dependent coordinates.

Medium

6. Prove geometric memoryless property from the PMF.
7. Derive Poisson limit from binomial (complete the algebra).
8. If X, Y i.i.d. Exp(λ), show $\min(X, Y) \sim \text{Exp}(2\lambda)$.
9. For Normal, standardize $P(X \leq a)$ via Φ.
10. Zipf: argue why mean of degrees can be dominated by hubs.

Challenge

11. Inverse transform: prove $F^{-1}(U)$ has CDF F for continuous strictly increasing F.
12. Show that memoryless continuous positive RVs must be exponential (outline).
13. Mixture: $X \mid Z = z \sim N(z, 1)$, $Z \sim N(0, \tau^2)$—marginal of X?
14. Total variation distance between Bin(n, p) and Poisson(np) bound statement (Le Cam).
15. Systems: model request arrivals as Poisson process; relate interarrivals to Exp.

Summary

RVs and standard families are the vocabulary of stochastic modeling. CDFs unify discrete and continuous; Poisson and normal approximations connect combinatorics and limits; memoryless laws and independence assumptions power algorithms—and must be checked against data.

Expectation, Variance, and Covariance

Expectation is the probability-weighted average; variance measures spread; covariance and correlation measure linear co-movement. These moments drive algorithm analysis, risk metrics, and the bias–variance decomposition in learning.

1. Expectation

Discrete: $\mathbb{E}[X] = \sum_x x\, p(x)$ (absolutely convergent when needed).
Continuous: $\mathbb{E}[X] = \int_{-\infty}^{\infty} x f(x)\, dx$.

Law of the unconscious statistician (LOTUS):

$$\mathbb{E}[g(X)] = \sum_x g(x)p(x) \quad \text{or} \quad \int g(x)f(x)\, dx,$$

without finding the law of $g(X)$ first.

Linearity (always true)

$$\mathbb{E}[aX + bY + c] = a\mathbb{E}[X] + b\mathbb{E}[Y] + c,$$

even when X, Y dependent—most powerful property in CS proofs.

Worked example 1 — indicator trick

If A is an event, $\mathbf{1}_A$ has $\mathbb{E}[\mathbf{1}_A] = P(A)$. Counting: $X = \sum_i \mathbf{1}_{A_i} \Rightarrow \mathbb{E}[X] = \sum_i P(A_i)$ (linearity of expectation)—even with dependence.

Worked example 2 — hashing collisions

Expected number of colliding pairs among n balls m bins: $\binom{n}{2} \cdot \frac{1}{m}$ order—classic.

Worked example 3 — geometric mean trials

$\mathbb{E}[\text{Geo}(p)] = 1/p$ (trials until success, $P(X = k) = (1-p)^{k-1}p$).

2. Variance and standard deviation

$$\mathrm{Var}(X) = \mathbb{E}[(X - \mathbb{E}[X])^2] = \mathbb{E}[X^2] - (\mathbb{E}[X])^2.$$

$\mathrm{sd}(X) = \sqrt{\mathrm{Var}(X)}$.

Scaling: $\mathrm{Var}(aX + b) = a^2\mathrm{Var}(X)$.

Worked example 4

Bernoulli: Var $= p(1-p) \le 1/4$.
Poisson(λ): mean=variance=λ.
Normal: Var $= \sigma^2$ by definition of $N(\mu, \sigma^2)$.

Worked example 5 — sum of independents

If $X \perp Y$, $\mathrm{Var}(X + Y) = \mathrm{Var}(X) + \mathrm{Var}(Y)$. Without independence, cross term 2Cov appears.

3. Covariance and correlation

$$\mathrm{Cov}(X, Y) = \mathbb{E}[(X - \mathbb{E}X)(Y - \mathbb{E}Y)] = \mathbb{E}[XY] - \mathbb{E}X\mathbb{E}Y.$$

$$\mathrm{Corr}(X, Y) = \frac{\mathrm{Cov}(X, Y)}{\mathrm{sd}(X)\mathrm{sd}(Y)} \in [-1, 1]$$

when sds positive.

Cauchy–Schwarz: $|\mathrm{Cov}(X, Y)| \le \mathrm{sd}(X)\mathrm{sd}(Y)$.

Worked example 6

Corr $= \pm 1$ iff $Y = aX + b$ a.s. with $a \neq 0$ (linear relationship almost surely).

Worked example 7 — uncorrelated but dependent

X symmetric ± 1, $Y = X^2$ constant? Better: $X \sim \mathrm{Unif}\{-1, 0, 1\}$ carefully, or $X \sim N(0, 1)$, $Y = X^2$: $\mathrm{Cov}(X, Y) = 0$ but dependent. Classic: X uniform on circle coordinates.

4. Conditional expectation

$\mathbb{E}[X \mid Y = y]$ is the mean of X given $Y = y$. As a RV, $\mathbb{E}[X \mid Y]$ is a function of Y.

Tower property: $\mathbb{E}[\mathbb{E}[X \mid Y]] = \mathbb{E}[X]$.
Pull-out: if Y measurable w.r.t. condition, $\mathbb{E}[YX \mid Y] = Y\mathbb{E}[X \mid Y]$.

Worked example 8 — prediction

Under squared loss, $\mathbb{E}[Y \mid X = x]$ is the optimal predictor of Y from X.

Worked example 9 — bias-variance (regression)

For fixed x, decomposing $\mathbb{E}[(Y - \hat{f}(x))^2]$ yields noise + bias^2 + variance of $\hat{f}$.

5. Moment inequalities (starters)

Markov: $X \geq 0 \Rightarrow P(X \geq t) \leq \mathbb{E}[X]/t$.
Chebyshev: $P(|X - \mu| \geq t) \leq \mathrm{Var}(X)/t^2$.
Jensen: for convex ϕ, $\phi(\mathbb{E}X) \leq \mathbb{E}\phi(X)$.

Worked example 10 — Chebyshev CI sketch

Crude: $P(|\bar{X}_n - \mu| \geq t) \leq \sigma^2/(nt^2)$. CLT gives tighter practical intervals.

Worked example 11 — Jensen

$\mathbb{E}[X^2] \geq (\mathbb{E}X)^2$ recovers nonnegative variance.

6. Covariance matrices

For random vector $X \in \mathbb{R}^d$,

$$\Sigma = \mathrm{Cov}(X) = \mathbb{E}[(X - \mu)(X - \mu)^\top] \succeq 0.$$

Affine: $\mathrm{Cov}(AX + b) = A\Sigma A^\top$.

Worked example 12 — PCA link

Principal components are eigenvectors of Σ (or sample covariance)—directions of maximal variance.

7. CS applications

- Expected runtime of randomized algorithms
- Load balancing: balls and bins expectations
- Portfolio / risk: variance and covariance
- Feature correlation screening (weak; prefer MI for nonlinear)
- Kalman filters: propagating means and covariances

8. Pitfalls

1. $\mathbb{E}[1/X] \neq 1/\mathbb{E}X$
2. Dropping absolute convergence (conditional paradoxes)
3. Variance of dependent sum without covariances
4. Correlation as causation
5. Infinite mean/variance distributions (Cauchy has no mean)

9. Checkpoint

- Use linearity with indicators
- Compute Var from $\mathbb{E}[X^2] - (\mathbb{E}X)^2$
- Relate Cov, Corr, independence
- State tower property
- Apply Markov/Chebyshev

Exercises

Easy

1. $\mathbb{E}[aX + b]$ and $\mathrm{Var}(aX + b)$.
2. Expected number of fixed points of a random permutation (indicators).
3. Cov matrix of (X, X) in terms of $\mathrm{Var}(X)$.

4. Show $\mathrm{Var}(X) = \mathbb{E}[X(X-1)] + \mathbb{E}X - (\mathbb{E}X)^2$ useful for Poisson-like.
5. If $X \perp Y$, show $\mathbb{E}[XY] = \mathbb{E}X\mathbb{E}Y$.

Medium

6. Prove Cauchy–Schwarz for covariance via $\mathrm{Var}(X - tY) \geq 0$.
7. Derive $\mathrm{Var}(X+Y) = \mathrm{Var}X + \mathrm{Var}Y + 2\mathrm{Cov}$.
8. Compute $\mathbb{E}[X \mid X > 0]$ for $X \sim N(0,1)$ (Mills ratio sketch OK).
9. Balls and bins: expected empty bins.
10. Show optimal squared-error predictor is conditional mean.

Challenge

11. Prove Jensen for discrete finite support by induction / tangent line.
12. Law of total variance: $\mathrm{Var}(X) = \mathbb{E}[\mathrm{Var}(X \mid Y)] + \mathrm{Var}(\mathbb{E}[X \mid Y])$.
13. For multivariate Normal, zero correlation $\Leftrightarrow$ independence—why special?
14. Concentration: from Chebyshev, how large n for $P(|\bar{X} - \mu| \geq 0.1\sigma) \leq 0.05$?
15. Algorithmic: expected comparisons in randomized quicksort recurrence sketch.

Summary

Moments turn distributions into scalars and matrices that algorithms can optimize and bound. Linearity of expectation is the Swiss army knife; variance and covariance quantify uncertainty and dependence—foundation for concentration, Gaussians, and PCA.

Law of Large Numbers and Central Limit Theorem

Limit theorems explain why averages stabilize and why normal distributions appear throughout science and engineering. They justify confidence intervals, Monte Carlo error rates, and many ML asymptotics.

1. Modes of convergence (brief)

For RVs X_n, X:

- **In probability:** $P(|X_n - X| > \varepsilon) \to 0$ for every $\varepsilon > 0$
- **Almost sure:** $P(\{\omega : X_n(\omega) \to X(\omega)\}) = 1$
- **In distribution:** $F_{X_n}(t) \to F_X(t)$ at continuity points of F_X
- **In L^p:** $\mathbb{E}|X_n - X|^p \to 0$

Implications (roughly): a.s. $\Rightarrow$ in prob $\Rightarrow$ in distribution; $L^2 \Rightarrow$ in prob. CLT is in distribution; strong LLN is a.s.

2. Law of large numbers

Let X_i i.i.d. with $\mathbb{E}|X_1| < \infty$, $\mu = \mathbb{E}[X_1]$, $\bar{X}_n = \frac{1}{n}\sum_{i=1}^{n} X_i$.

Weak LLN: $\bar{X}_n \to \mu$ in probability.
Strong LLN: $\bar{X}_n \to \mu$ almost surely.

Interpretation

Long-run sample averages track the true mean. Foundation of Monte Carlo, empirical risk, and frequency interpretation of probability.

Worked example 1

Coin flips: $\bar{X}_n \to p$. After 10^4 flips, typically within $O(1/\sqrt{n})$ of p (see CLT).

Worked example 2 — Monte Carlo integration

$\mathbb{E}[g(U)] \approx \frac{1}{n}\sum g(U_i)$ for $U_i \sim$ Unif. Error typically $O_p(\sigma/\sqrt{n})$.

3. Proof sketch of weak LLN (finite variance)

If $\mathrm{Var}(X_i) = \sigma^2 < \infty$, then $\mathrm{Var}(\bar{X}_n) = \sigma^2/n$. Chebyshev:

$$P(|\bar{X}_n - \mu| \geq \varepsilon) \leq \frac{\sigma^2}{n\varepsilon^2} \to 0.$$

(Full strong LLN needs more machinery.)

4. Central limit theorem

Theorem (classical CLT). X_i i.i.d. with mean μ and variance $\sigma^2 \in (0, \infty)$. Then

$$Z_n = \frac{\bar{X}_n - \mu}{\sigma/\sqrt{n}} \xrightarrow{d} N(0,1).$$

Equivalently $\sqrt{n}(\bar{X}_n - \mu) \xrightarrow{d} N(0, \sigma^2)$.

Worked example 3 — SE of the mean

Standard error $\mathrm{se} = \sigma/\sqrt{n}$. Estimated by $s/\sqrt{n}$ with sample sd s.

Worked example 4 — latency

Mean 100 ms, sd 30 ms, $n = 64$: $\mathrm{se} = 3.75$ ms. Approx 95% CI for mean: $100 \pm 1.96 \cdot 3.75$.

Worked example 5 — binomial / polls

$\hat{p} = \bar{X}$ for Bernoulli: $\mathrm{se} \approx \sqrt{\hat{p}(1-\hat{p})/n}$. Rule of thumb $n\hat{p} \geq 10$ etc. for normal approx quality.

5. Lindeberg / Lyapunov (awareness)

CLT extends beyond identical distributions under Lindeberg conditions—important for triangular arrays and some dependent settings (with extra work).

6. Berry–Esseen (rate)

Bounds $\sup_t |P(Z_n \leq t) - \Phi(t)|$ by $O(\rho/(\sigma^3\sqrt{n}))$ involving third moments. Message: convergence rate is about $1/\sqrt{n}$, worse with heavy skew/kurtosis.

Worked example 6 — skewed Bernoulli

$p = 0.01$: need larger n for $\hat{p}$ normal approx than for $p = 0.5$.

7. Delta method

If $\sqrt{n}(\hat{\theta}_n - \theta) \to N(0, \tau^2)$ and g differentiable at θ with $g'(\theta) \neq 0$, then

$$\sqrt{n}(g(\hat{\theta}_n) - g(\theta)) \to N(0, (g'(\theta))^2\tau^2).$$

Worked example 7

$g(p) = \text{logit}(p)$ or $g = \log$ for positive parameters—SEs for transformed estimates.

8. Multivariate CLT

$\sqrt{n}(\bar{X}_n - \mu) \to N(0, \Sigma)$ for i.i.d. vectors with covariance Σ. Basis of multivariate confidence ellipsoids and Wald tests.

9. Consequences in CS/ML

Application	Use of LLN/CLT
Empirical risk $\to$ risk	LLN (uniform versions need VC/Rademacher)
Bootstrap	often justified by CLT-like behavior
A/B test z-tests	CLT for means
SGD noise	asymptotic normality results (advanced)
Hashing load	concentration + CLT approximations

Worked example 8 — Monte Carlo error

To cut error in half, need $\approx 4\times$ samples ($1/\sqrt{n}$ law)—fundamental budgeting fact.

Worked example 9 — why not $1/n$ error?

Average of independents has variance $1/n$; sd $1/\sqrt{n}$. Unless using variance reduction / QMC, you rarely beat this without structure.

10. When CLT fails

- Infinite variance (stable laws with heavy tails)
- Strong dependence (long memory)
- Non-identical extremes dominated by one term
- Very small n

Worked example 10 — Cauchy

Sample mean of Cauchy does **not** concentrate on a mean (undefined)—LLN fails.

11. Pitfalls

1. "CLT says data are normal"—no, **means** are approximately normal
2. 95% CI does not mean $P(\theta \in \text{CI} \mid \text{data}) = 0.95$ in frequentist interpretation
3. Multiple looks without correction
4. Using z critical values with tiny n and unknown variance (t better)
5. Ignoring dependence in time series SE

12. Checkpoint

- State weak LLN and classical CLT
- Compute se of the mean
- Build a normal CI for a mean
- Explain $1/\sqrt{n}$ Monte Carlo rate
- Know when normal approximation is shaky

Exercises

Easy

1. If $\sigma = 2$, how large n so Chebyshev guarantees $P(|\bar{X} - \mu| \geq 0.5) \leq 0.05$?

2. SE for $\hat{p}$ when $n = 1000$, $\hat{p} = 0.4$.
3. Simulate (mentally) why averaging reduces noise.
4. Distinguish convergence in probability vs distribution with one sentence each.
5. CI for mean: write formula with known σ.

Medium

6. Prove weak LLN via Chebyshev under finite variance.
7. Apply delta method to $g(\mu) = \mu^2$.
8. Binomial: continuity correction intuition for $P(X \leq k)$.
9. Explain why $\bar{X}_n$ is consistent for μ (definition of consistency).
10. Multivariate: interpret Σ diagonal vs strong off-diagonals for joint means.

Challenge

11. Sketch proof idea of CLT via characteristic functions (expand $\log \phi$).
12. Berry–Esseen: discuss effect of third moment on needed n.
13. Counterexample: i.i.d. with infinite mean where $\bar{X}_n$ misbehaves.
14. Dependent vars: give example where $\mathrm{Var}(\bar{X}_n) \nsim \sigma^2/n$.
15. Design: sample size for A/B test detecting effect δ with power $1 - \beta$ (formula with σ).

Summary

LLN guarantees stabilization of averages; CLT quantifies Gaussian fluctuations at scale $1/\sqrt{n}$. Together they underwrite estimation, testing, and Monte Carlo—while heavy tails, dependence, and small samples define their limits.

Bayesian Inference

Bayesian inference treats unknown parameters as random variables and updates beliefs with data via Bayes' rule. It yields full posterior distributions, natural regularization through priors, and predictive distributions for new observations—central in modern ML and decision systems.

1. Bayes' rule for parameters

Parameter θ, data D:

$$p(\theta \mid D) = \frac{p(D \mid \theta)\, p(\theta)}{p(D)}, \qquad p(D) = \int p(D \mid \theta) p(\theta)\, d\theta.$$

Name	Symbol	Role
Prior	$p(\theta)$	belief before data
Likelihood	$p(D \mid \theta)$	sampling model
Evidence	$p(D)$	marginal likelihood
Posterior	$p(\theta \mid D)$	updated belief

Worked example 1 — discrete hypothesis

Two coins: fair $P(H) = 1/2$ or two-headed. Prior $1/2$ each. Observe H: posterior on two-headed is $2/3$ by Bayes.

2. Conjugate priors

A prior is **conjugate** for a likelihood if the posterior stays in the same family—closed-form updates.

Likelihood	Conjugate prior	Posterior update
Bernoulli/Binomial	Beta(α, β)	$\alpha' = \alpha + \#\text{success}$, $\beta' = \beta + \#\text{fail}$
Poisson	Gamma	add counts to shape; expose to rate
Normal mean (known var)	Normal	precision-weighted average
Multinomial	Dirichlet	add category counts

Worked example 2 — Beta–Binomial

Prior $\text{Beta}(2, 2)$, data 8 successes, 2 failures $\Rightarrow$ posterior $\text{Beta}(10, 4)$.
Mean $10/14 \approx 0.714$. Prior mean was 0.5; data pulled belief up.

Worked example 3 — uniform prior

$\text{Beta}(1, 1)$ is $\text{Uniform}(0, 1)$. After s successes, f failures: $\text{Beta}(1 + s, 1 + f)$.

Worked example 4 — strong prior

$\text{Beta}(100, 100)$ after same 8/2 data barely moves—prior as "pseudo-counts."

3. MAP vs MLE vs posterior mean

- **MLE:** $\hat{\theta}_{\text{MLE}} = \arg\max_\theta p(D \mid \theta)$
- **MAP:** $\hat{\theta}_{\text{MAP}} = \arg\max_\theta p(\theta \mid D) = \arg\max_\theta [p(D \mid \theta) p(\theta)]$
- **Posterior mean:** $\mathbb{E}[\theta \mid D]$ (optimal under squared error)

MAP with prior $p(\theta) \propto e^{-\lambda \Omega(\theta)}$ resembles regularized MLE.

Worked example 5

Gaussian prior on regression weights $\Leftrightarrow$ ridge penalty in MAP.

Worked example 6

Laplace prior $\Leftrightarrow$ lasso-like MAP (with caveats on geometric exactness).

4. Posterior predictive

For new observation x_{new}:

$$p(x_{\text{new}} \mid D) = \int p(x_{\text{new}} \mid \theta)\, p(\theta \mid D)\, d\theta.$$

Averages likelihoods under posterior uncertainty—better for forecasting than plugging a point estimate only.

Worked example 7 — Beta–Binomial predictive

Predictive success probability equals posterior mean of p for exchangeable Bernoulli—intuitive.

5. Hierarchical models (sketch)

Priors on priors: $\theta_i \sim p(\theta_i \mid \phi)$, $\phi \sim p(\phi)$. Share statistical strength across groups (empirical Bayes / full Bayes). Used in multilevel A/B tests and recommendation.

Worked example 8

Per-user conversion rates with global hyperprior—avoids wild estimates for users with 1 visit.

6. Computation when conjugates fail

Method	Idea
Grid / quadrature	low-dimensional θ
Laplace approximation	Gaussian around MAP
MCMC (MH, HMC, NUTS)	samples from posterior
Variational inference	optimize approx $q(\theta) \approx p(\theta \mid D)$

Worked example 9 — MCMC intuition

Build Markov chain with stationary law $p(\theta \mid D)$; averages of samples estimate posterior expectations (ergodic theorem).

Worked example 10 — VI

Minimize $\mathrm{KL}(q \| p(\cdot \mid D))$—connects to ELBO in VAEs.

7. Decision theory link

Loss $L(\theta, a)$; choose action a minimizing posterior expected loss $\int L(\theta, a) p(\theta \mid D)\, d\theta$. Point estimates emerge from particular losses (0-1 $\to$ MAP for discrete; squared $\to$ mean).

8. CS/ML applications

- Spam filters / Naive Bayes
- A/B tests with Beta-Binomial dashboards
- Bayesian optimization priors on functions (GPs)
- Uncertainty in deep learning (approx Bayesian)
- Kalman filters as sequential Bayes for linear-Gaussian state space

Worked example 11 — online Bayes

Process arrivals one-by-one: conjugate update is $O(1)$ per observation—streaming inference.

Worked example 12 — regularization interpretation

Without calling it Bayesian, weight decay is MAP under Gaussian prior—same math, different philosophy.

9. Pitfalls

1. Priors that put zero mass where truth lies (no recovery)
2. Improper priors without checking posterior propriety
3. Interpreting credible intervals as frequentist CIs without care
4. Label-switching in mixture posteriors
5. MCMC not converged (use diagnostics: R-hat, ESS)
6. "Uninformative" priors still informative on transformed scales

10. Checkpoint

- Write Bayes update and name all parts
- Perform Beta-Binomial update numerically
- Contrast MLE, MAP, posterior mean

- Define posterior predictive
- Name two computational approximate inference methods

Exercises

Easy

1. Update Beta$(1, 1)$ after 3 successes, 1 failure; posterior mean.
2. Fair vs double-head coin: after two heads, posterior on double-head?
3. Why is MAP with flat prior equal to MLE (when flat allowed)?
4. Interpret α, β in Beta as pseudo-counts.
5. One sentence: what is the evidence $p(D)$ used for in model comparison?

Medium

6. Derive Normal–Normal posterior for mean with known variance.
7. Show MAP for Bernoulli with Beta(α, β) prior in closed form.
8. Explain how hierarchical model reduces variance of per-group estimates (qualitative).
9. Connect ridge regression to Gaussian prior MAP (write objective).
10. Predictive: for Beta-Binomial, $P(\text{next success} \mid D)$.

Challenge

11. Prove conjugacy update for Beta-Binomial fully.
12. Laplace approximation: expand $\log p(\theta \mid D)$ to quadratic; identify covariance as inverse Hessian.
13. KL$(q\|p)$ vs KL$(p\|q)$ for VI vs expectation propagation intuition.
14. Design a Bayesian A/B test stopping rule sketch using posterior $P(p_A > p_B \mid D)$.
15. Discuss prior sensitivity: reanalyze Example 2 with Beta$(0.5, 0.5)$ vs Beta$(5, 5)$.

Summary

Bayesian inference is coherent updating of beliefs with likelihoods. Conjugacy gives exact posteriors; computation extends the framework to rich models. MAP links to regularization; predictives support decisions under parameter uncertainty—the probabilistic counterpart to point-estimate ML.

Markov Chains and Stochastic Processes

Markov chains model systems whose future depends on the past only through the present state. They power PageRank, MCMC, reinforcement learning, queues, and reliability analysis.

1. Markov property

A stochastic process $(X_t)_{t\geq 0}$ on a countable state space $\mathcal{S}$ is a **Markov chain** if

$$P(X_{t+1}=j \mid X_t=i, X_{t-1}=i_{t-1}, \ldots) = P(X_{t+1}=j \mid X_t=i).$$

Time-homogeneous: transition probabilities $P_{ij} = P(X_{t+1}=j \mid X_t=i)$ do not depend on t.

Worked example 1 — weather toy

States $\{\text{Sun}, \text{Rain}\}$ with fixed transition probabilities—classic textbook chain.

Worked example 2 — gambler's ruin

Fortune as state; absorb at 0 and N. Markov with absorbing barriers.

2. Transition matrix

$P = (P_{ij})$ is **row-stochastic**: $P_{ij} \geq 0$, $\sum_j P_{ij} = 1$.

n-step transitions: $P^{(n)} = P^n$, i.e.

$$P(X_n = j \mid X_0 = i) = (P^n)_{ij}.$$

Worked example 3

Two-state:

$$P = \begin{bmatrix} 0.8 & 0.2 \\ 0.3 & 0.7 \end{bmatrix}.$$

Compute P^2 by matrix multiply for two-step probabilities.

3. Classification of states

- **Communicate:** $i \to j$ and $j \to i$ with positive probability in some steps
- **Irreducible:** all states communicate
- **Period** of i: gcd of return times; **aperiodic** if period 1
- **Recurrent vs transient:** return with probability 1 vs < 1
- **Positive recurrent:** finite mean return time

Worked example 4

A cycle graph with deterministic rotation is periodic with period n. Adding self-loops typically breaks periodicity.

4. Stationary distribution

π is **stationary** if $\pi^\top P = \pi^\top$ (row vector convention) and $\pi_i \geq 0$, $\sum_i \pi_i = 1$. Equivalently $P^\top \pi = \pi$.

Balance: $\pi_j = \sum_i \pi_i P_{ij}$.

Existence/uniqueness (finite case)

Finite irreducible chain: unique stationary π. If also aperiodic, $P(X_t = \cdot) \to \pi$ from any start (**convergence theorem**).

Worked example 5

For the two-state P above, solve $\pi P = \pi$, $\pi_1 + \pi_2 = 1$:

$\pi = (0.6, 0.4)$ (verify: long-run fraction of time in state 2 is 0.4).

Worked example 6 — PageRank

Google matrix is a modified stochastic matrix; PageRank is its stationary distribution (with teleportation ensuring irreducibility/primitivity).

5. Detailed balance and reversibility

$\pi_i P_{ij} = \pi_j P_{ji}$ for all $i, j \Rightarrow \pi$ stationary (sum on i). Chains satisfying detailed balance for some π are **reversible**.

Worked example 7 — MCMC design

Metropolis–Hastings builds P so that a **target** π satisfies detailed balance—sample from π without knowing the normalizing constant.

6. Hitting times and absorption

$T_A = \min\{t \geq 0 : X_t \in A\}$. Expected hitting times solve linear equations from first-step analysis.

Worked example 8 — gambler's ruin

Probability of hitting N before 0 from fortune k is k/N for fair game—solve recurrence $h_k = \frac{1}{2}h_{k-1} + \frac{1}{2}h_{k+1}$.

7. Mixing time (awareness)

How large t until $\|P^t(x, \cdot) - \pi\|_{\mathrm{TV}}$ small? Determines MCMC burn-in. Can be exponential in dimension for hard targets; good chains mix rapidly.

Worked example 9

Random walk on the hypercube mixes in $O(d \log d)$ steps—classic.

8. Continuous time (sketch)

Exponential holding times + jump chain: rate matrix Q, Kolmogorov equations. Poisson process is a counting CTMC.

Worked example 10 — M/M/1 queue

States = number in system; birth rate λ, death rate μ; stationary geometric if $\lambda < \mu$.

9. CS applications map

Domain	Chain
Ranking	PageRank / random surfer
Sampling	MCMC, Gibbs
RL	MDP state process under policy
Reliability	up/down component models
Caches	stack distance / Markovian request models (approx)
NLP (classical)	n-gram as Markov of order $n-1$

Worked example 11 — RL

Policy $\pi(a \mid s)$ induces Markov chain on states with $P(s'|s) = \sum_a \pi(a \mid s)P(s'|s,a)$. Value functions solve linear systems on this chain (policy evaluation).

Worked example 12 — Gibbs sampling

Coordinate-wise conditional updates form a Markov chain on configurations with stationary equal to target joint (under mild conditions).

10. Pitfalls

1. Assuming convergence without irreducibility/aperiodicity
2. Confusing time average with ensemble average without ergodicity
3. Using reducible PageRank graph without teleports
4. MCMC without diagnostics (still correlated samples)
5. High-order memory forced into first-order Markov poorly

11. Checkpoint

- Write Markov property and row-stochastic P
- Compute π for a small chain
- Define irreducible / aperiodic / stationary
- State detailed balance use in MCMC
- Connect PageRank to $\pi P = \pi$

Exercises

Easy

1. Verify rows of a given 3×3 matrix are stochastic or fix them.
2. For two-state chain, find π in closed form from P_{01}, P_{10}.
3. Explain absorbing state: $P_{ii} = 1$.
4. Simulate three steps of weather chain by hand from Sun.
5. Why teleportation helps PageRank math?

Medium

6. Prove detailed balance $\Rightarrow$ stationarity.
7. Solve gambler's ruin fair case for absorption probabilities.
8. Show period divides return times; give period-2 example.
9. Policy evaluation: write $V = r + P^{\pi}V$ linear system.
10. Compute P^n for two-state via diagonalization sketch.

Challenge

11. Metropolis–Hastings acceptance probability derivation from detailed balance.
12. Mixing: define total variation; state coupling inequality idea.
13. Ehrenfest urn model: stationary is binomial; interpret.
14. Continuous time: write Q for two-state CTMC; relate stationary to rates.
15. Project: model a simple cache replacement as Markov on a small state space; estimate miss rate via π.

Summary

Markov chains reduce temporal dependence to a transition matrix and open the door to stationary analysis, absorption probabilities, and MCMC. From PageRank to queues to reinforcement learning, the same spectral/linear-algebraic equilibrium $\pi P = \pi$ reappears as the long-run law of a system.

Thresholds in Random Structures: Expectation Thresholds and the Park–Pham Theorem

Thesis: for almost every "does a random object contain a structure?" question, the honest first guess, *the probability at which you expect to see one copy*, is already within a logarithmic factor of the truth. Kahn and Kalai conjectured this in 2007 while writing that it would "probably be more sensible to conjecture that it is not true." Jinyoung Park and Huy Tuan Pham proved it in 2022 with a short counting argument. This chapter builds the vocabulary (increasing properties, thresholds, covers, the expectation threshold), measures two classical thresholds in random graphs, shows exactly where the logarithm comes from, and then checks the counting step at the heart of the proof by brute force.

Picture the question first. Flip a coin with bias p for every edge of the complete graph on n vertices. For which p does the result contain a triangle? A perfect matching? A Hamilton cycle? A copy of a fixed spanning tree? Each of these properties switches from "almost never" to "almost always" over a narrow range of p, and locating that range used to need a separate, often very hard, paper per property.

All listings are complete Python files. They were run with **Python 3.13.5** and **NumPy 2.5.4** on Linux; outputs are pasted unedited. Monte Carlo runs use a fixed seed, so you will get the same numbers.

1. Increasing properties and the threshold

Let X be a finite set (the edges of K_n, the k-subsets of $[n]$, the cells of a grid). For $p \in [0,1]$ let X_p be the random subset that keeps each element independently with probability p, so

$$\mu_p(A) = P(X_p = A) = p^{|A|}(1-p)^{|X \setminus A|}.$$

A family $\mathcal{F} \subseteq 2^X$ is an **increasing property** if adding elements never destroys it: $A \in \mathcal{F}$ and $B \supseteq A$ imply $B \in \mathcal{F}$. "Contains a triangle", "is connected", "has a perfect matching" are increasing; "is a forest" is not.

For a nontrivial increasing $\mathcal{F}$ (neither empty nor everything), $\mu_p(\mathcal{F}) = P(X_p \in \mathcal{F})$ is a polynomial in p that increases strictly and continuously from 0 to 1. The **threshold** $p_c(\mathcal{F})$ is the unique p with

$$\mu_{p_c}(\mathcal{F}) = \tfrac{1}{2}.$$

Bollobás and Thomason (1987) showed that this is always a threshold in the original Erdős–Rényi sense: $\mu_p(\mathcal{F}) \to 0$ when $p \ll p_c$ and $\to 1$ when $p \gg p_c$. How *wide* the transition window is relative

to p_c is a separate question (sharp versus coarse thresholds, Friedgut 1999). This chapter is about *where* p_c is.

Every increasing property is generated by its **minimal elements** $\mathcal{H}$: $A \in \mathcal{F}$ exactly when A contains some $S \in \mathcal{H}$. For "contains a triangle" the minimal elements are the $\binom{n}{3}$ triangles; for "has a perfect matching" (even n) they are the $(n-1)!!$ perfect matchings. Write

$$\ell(\mathcal{F}) = \max(2,\ \max_{S \in \mathcal{H}} |S|).$$

That is 3 for triangles and $n/2$ for perfect matchings. It will be the only structural parameter in the main theorem.

Worked example 1 — two thresholds in $G(n,p)$, by simulation

```
# er_thresholds.py - two Erdős-Rényi thresholds in G(n, p), by Monte Carlo
#   triangles appear at p = c/n           (coarse threshold, Poisson limit)
#   isolated vertices vanish at p = (ln n + c)/n   (sharp threshold, Gumbel limit)
import math
import numpy as np

rng = np.random.default_rng(20261017)
n, trials = 400, 300

def sample_adjacency(n, p):
    upper = np.triu(rng.random((n, n)) < p, k=1)
    return (upper | upper.T).astype(np.float64)

def has_triangle(A):
    return ((A @ A) * A).sum() > 0          # sum of (A^2  A) = 6 * #triangles

def no_isolated_vertex(A):
    return A.sum(axis=1).min() >= 1

print(f"n = {n}, {trials} graphs per row\n")
print("triangles, p = c/n:   c    simulated   1 - exp(-c^3/6)")
for c in (0.5, 1.0, 1.5, 2.0, 3.0):
    hits = sum(has_triangle(sample_adjacency(n, c / n)) for _ in range(trials))
    print(f"                    {c:3.1f}     {hits / trials:6.3f}        {1 - math.exp(-c**3
print("\nno isolated vertex, p = (ln n + c)/n:   c    simulated   exp(-e^-c)")
for c in (-2.0, -1.0, 0.0, 1.0, 2.0):
    p = (math.log(n) + c) / n
    hits = sum(no_isolated_vertex(sample_adjacency(n, p)) for _ in range(trials))
    print(f"                                        {c:4.1f}     {hits / trials:6.3f}      {ma
```

```
python3 er_thresholds.py
```

```
n = 400, 300 graphs per row

triangles, p = c/n:   c    simulated   1 - exp(-c^3/6)
                    0.5      0.017         0.021
                    1.0      0.117         0.154
                    1.5      0.420         0.430
                    2.0      0.737         0.736
                    3.0      0.990         0.989

no isolated vertex, p = (ln n + c)/n:   c    simulated   exp(-e^-c)
                                     -2.0      0.000       0.001
                                     -1.0      0.073       0.066
                                      0.0      0.343       0.368
                                      1.0      0.740       0.692
                                      2.0      0.863       0.873
```

Both columns on the right are classical limits (Erdős–Rényi 1959–60):

- **Triangles.** The number of triangles has mean $\binom{n}{3}p^3 \to c^3/6$ at $p = c/n$ and is asymptotically Poisson, so $P(\text{triangle}) \to 1 - e^{-c^3/6}$. The transition is **coarse**: p has to change by a constant *factor* to go from 2 % to 99 %.
- **No isolated vertex.** The number of isolated vertices has mean $n(1-p)^{n-1} \to e^{-c}$ at $p = (\ln n + c)/n$, again asymptotically Poisson, so $P(\text{none}) \to e^{-e^{-c}}$. The window is $\Theta(1/n)$ around $\ln n/n$, a vanishing *fraction* of p: a **sharp** threshold.

The simulation agrees with the limits to within two standard errors (one standard error is 0.02–0.03 for 300 trials). The two largest gaps, triangles at $c = 1$ and isolated vertices at $c = 1$, are both about 1.8 standard errors, and part of them is finite-n bias: at $n = 400$ the triangle mean is $\binom{400}{3}/400^3 = 0.9925$ times $c^3/6$, and neither count is exactly Poisson yet. "No isolated vertex" matters here because of a theorem of Erdős and Rényi (1966): for even n, $G(n,p)$ has a perfect matching with the same limiting probability $e^{-e^{-c}}$ at $p = (\ln n + c)/n$. Isolated vertices are the bottleneck.

2. Mental model: covers, cost, and the expectation threshold

The cheapest way to show that X_p is *unlikely* to have a property is the first-moment method: count the expected number of witnesses and use $P(\text{at least one}) \le E[\text{count}]$. The definition that makes this precise (Talagrand's formulation):

- A family $\mathcal{G} \subseteq 2^X$ is a **cover** of $\mathcal{F}$ if every $A \in \mathcal{F}$ contains some $S \in \mathcal{G}$.
- The **cost** of the cover at p is $\sum_{S\in\mathcal{G}} p^{|S|} = E[\#\{S \in \mathcal{G} : S \subseteq X_p\}]$.
- $\mathcal{F}$ is **p-small** if it has a cover of cost at most $\frac{1}{2}$.
- The **expectation threshold** is $q(\mathcal{F}) = \max\{p : \mathcal{F} \text{ is } p\text{-small}\}$.

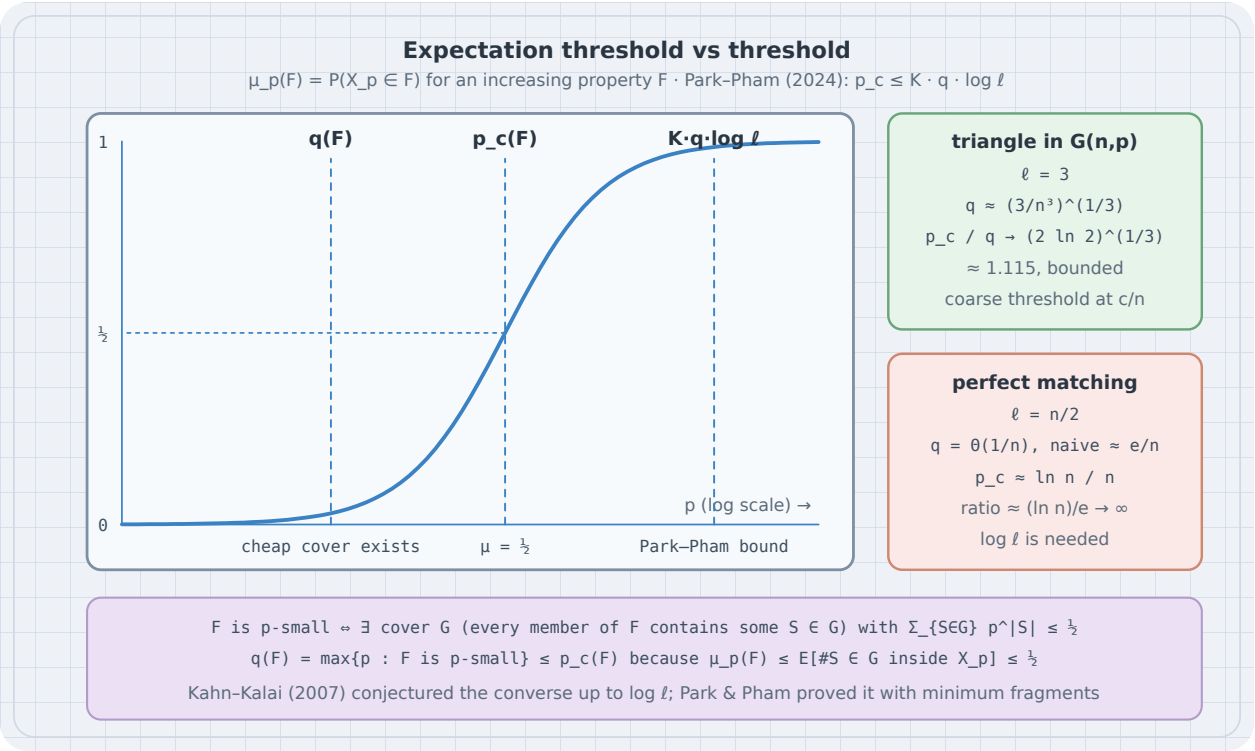


Figure 1: Expectation threshold vs threshold

The lower bound is three lines. If $\mathcal{G}$ covers $\mathcal{F}$ with cost $\leq \frac{1}{2}$ at p, then by the union bound

$$\mu_p(\mathcal{F}) \leq \mu_p(\exists S \in \mathcal{G} : S \subseteq X_p) \leq \sum_{S \in \mathcal{G}} p^{|S|} \leq \tfrac{1}{2},$$

so $p \leq p_c$. Taking the largest such p:

$$q(\mathcal{F}) \leq p_c(\mathcal{F}).$$

Two things make covers more interesting than "expected number of copies":

1. **The minimal elements are one cover, not the best one.** Using $\mathcal{G} = \mathcal{H}$ gives the naive estimate "expected number of copies $= \frac{1}{2}$". Any cleverer cover can only push q higher.
2. **Covers can be small sets.** A cover of "has a perfect matching" is $\{\{e\} : e \ni v\}$ for a fixed vertex v: every perfect matching uses an edge at v. Its cost is $(n-1)p$. That only certifies p-smallness up to $p \approx \frac{1}{2n}$, worse than the naive cover here, but it shows the idea: a cover is any family of *obstructions you would have to see*.

3. Where the logarithm comes from

For triangles the naive cover is essentially optimal and p_c sits a constant factor above it. For perfect matchings, the naive cover says the expected number of perfect matchings is already $\frac{1}{2}$ at $p \approx e/n$

(because $((n-1)!!)^{2/n} \approx n/e$), yet the graph has isolated vertices, and therefore no perfect matching, until $p \approx \ln n/n$. In fact $q = \Theta(1/n)$ for perfect matchings (this is the standard example showing the logarithm in the theorem below cannot be dropped), so the ratio p_c/q really grows like $\log n$. The obstruction is a coupon-collector effect: each of n vertices independently needs one edge, and collecting all n coupons costs a $\log n$ factor that no first-moment count over whole matchings can see.

Worked example 2 — naive expectation threshold vs threshold, exactly

```
# expectation_vs_threshold.py - the naive "expected count = 1/2" estimate vs the true thresh
#   triangle in G(n,p):        minimal elements have size 3        -> ratio stays bounded
#   perfect matching in G(n,p): minimal elements have size n/2     -> ratio grows like log n
import math

def p_expect_triangle(n):
    # cover G = all triangles: E[#triangles] = C(n,3) p^3 = 1/2
    return (0.5 / math.comb(n, 3)) ** (1 / 3)

def p_c_triangle(n):
    # #triangles is asymptotically Poisson(C(n,3) p^3): P(no triangle) = 1/2 at mean ln 2
    return (math.log(2) / math.comb(n, 3)) ** (1 / 3)

def p_expect_matching(n):
    # cover G = all perfect matchings: (n-1)!! p^(n/2) = 1/2
    log_double_fact = math.lgamma(n + 1) - (n // 2) * math.log(2) - math.lgamma(n // 2 + 1)
    return math.exp((math.log(0.5) - log_double_fact) / (n // 2))

def p_c_no_isolated(n):
    # P(no isolated vertex) ~ exp(-n (1-p)^(n-1)) = 1/2; a perfect matching needs this,
    # so it is a lower bound on p_c(perfect matching), and asymptotically equal to it
    return 1 - (math.log(2) / n) ** (1 / (n - 1))

print("     n |  triangle: p_c/q_naive |  matching: p_c/q_naive   ln(l), l=n/2   ratio/ln(l)
for n in (100, 400, 1600, 6400, 25600, 102400):
    rt = p_c_triangle(n) / p_expect_triangle(n)
    rm = p_c_no_isolated(n) / p_expect_matching(n)
    ln_l = math.log(n / 2)
    print(f"{n:6d} |        {rt:6.3f}          |        {rm:6.3f}          {ln_l:6.3f}
print(f"\n(2 ln 2)^(1/3) = {(2 * math.log(2)) ** (1 / 3):.3f};  n * q_naive(matching) at n=1
      f"{102400 * p_expect_matching(102400):.4f}  (e = {math.e:.4f})")
```

```
python3 expectation_vs_threshold.py
```

```
     n |  triangle: p_c/q_naive |  matching: p_c/q_naive   ln(l), l=n/2   ratio/ln(l)
   100 |         1.115          |         1.840           3.912         0.470
```

```
   400 |        1.115          |        2.338          5.298         0.441
  1600 |        1.115          |        2.848          6.685         0.426
  6400 |        1.115          |        3.358          8.071         0.416
 25600 |        1.115          |        3.869          9.457         0.409
102400 |        1.115          |        4.379         10.843         0.404

(2 ln 2)^(1/3) = 1.115;  n * q_naive(matching) at n=102400: 2.7182  (e = 2.7183)
```

Read the two ratio columns:

- **Triangles ($\ell = 3$).** p_c/q_{naive} is $(2\ln 2)^{1/3} \approx 1.115$ for every n: the threshold is where the Poisson mean is $\ln 2$, the naive estimate is where the mean is $\frac{1}{2}$.
- **Perfect matchings ($\ell = n/2$).** The ratio grows without bound, and dividing by $\ln \ell$ gives a column that settles slowly towards $1/e \approx 0.368$ (since $p_c \approx \ln n/n$ and $q_{\text{naive}} \approx e/n$). The last line confirms $n\, q_{\text{naive}} \to e$.

Here q_{naive} uses the cover by minimal elements, so it is a **lower** bound on q. The matching column uses p_c(no isolated vertex), which is a **lower** bound on p_c(perfect matching) for every n (a perfect matching has no isolated vertex) and is asymptotically equal to it. So the growth of the ratio is real, not an artefact of the estimates.

4. The theorem

Theorem (Kahn–Kalai conjecture; Park–Pham 2022, J. Amer. Math. Soc. 2024). There is a universal constant K such that for every finite set X and every nontrivial increasing property $\mathcal{F} \subseteq 2^X$,

$$q(\mathcal{F}) \;\le\; p_c(\mathcal{F}) \;\le\; K\, q(\mathcal{F}) \log \ell(\mathcal{F}).$$

The lower bound is the three-line union bound above. The upper bound says the union bound is never fooled by more than a logarithm of the size of the witnesses, *for every property at once*. Section 3 shows the $\log \ell$ cannot be removed.

Why this was a big deal:

- **It replaces case-by-case papers.** Thresholds that took long dedicated work become corollaries: perfect matchings in random r-uniform hypergraphs (Johansson–Kahn–Vu, 2008, "Shamir's problem"), a fixed bounded-degree spanning tree in $G(n,p)$ at $\Theta(\log n/n)$ (Montgomery, 2019, who proved a stronger universal version), powers of Hamilton cycles, and more. In each case you compute q (usually a first-moment count) and multiply by $\log \ell$.
- **It went through a fractional version first.** Talagrand proposed replacing covers by fractional covers (weights $\lambda_S \ge 0$ with $\sum_{S \subseteq F} \lambda_S \ge 1$ for all $F \in \mathcal{F}$), giving $q \le q_f \le p_c$. Frankston, Kahn, Narayanan and Park (Annals of Math., 2021) proved $p_c \le K q_f \log \ell$ by adapting the Alweiss–Lovett–Wu–Zhang sunflower argument, which relies on "spread" measures and LP duality. Park and Pham's proof of the integral version avoids spread entirely.
- **The constant is not explicit.** K is "large enough"; nobody uses this theorem to compute p_c to two digits.

5. Proof idea: minimum fragments

Work with the minimal elements $\mathcal{H}$ (an ℓ-bounded hypergraph on X, $|X| = n$) and assume $\mathcal{H}$ is **not** p-small: every cover costs more than $\frac{1}{2}$. The goal (their Theorem 1.4) is to show that a uniformly random set W of about $(Lp \log \ell)\, n$ elements contains some $S \in \mathcal{H}$ with probability $1 - o(1)$. From that, $p_c \le Kq \log \ell$ follows by a concentration step.

Pick W of size $w = Lpn$. For $S \in \mathcal{H}$, a **minimum** (S, W)**-fragment** is a smallest set of the form

$$T(S, W) = S' \setminus W, \qquad S' \in \mathcal{H},\ S' \subseteq W \cup S.$$

It is what is still missing from *some* witness once W has been placed, choosing the witness that is cheapest to finish. Note $T \subseteq S$, and $T = \emptyset$ means W already contains a witness.

One round splits $\mathcal{H}$:

- If the fragment is still **large** ($|T| \ge 0.9\ell$), W did not help S much. Put T into a cover $\mathcal{U}(W)$.
- If it is **small**, keep T as an edge of a new hypergraph $\mathcal{H}'$, which is 0.9ℓ-bounded. Covering $\mathcal{H}'$ later also covers these S.

Key lemma. Over a uniform w-set W, the expected cost of the large-fragment cover is tiny:

$$E\Big[\sum_{U \in \mathcal{U}(W)} p^{|U|}\Big] < L^{-0.8\ell}.$$

The counting. Fix $m \ge 0.9\ell$ and count pairs (W, T) with $|T| = m$:

1. Let $Z = W \cup T$, a set of size $w + m$. There are $\binom{n}{w+m} \le \binom{n}{w}(Lp)^{-m}$ choices.
2. Z contains a witness, so fix one, $\hat{S} \subseteq Z$, as a function of Z alone. **Minimality forces** $T \subseteq \hat{S}$: $\hat{S} \subseteq W \cup T \subseteq W \cup S$, so $\hat{S} \setminus W$ is itself a candidate fragment for (S, W), and $\hat{S} \setminus W \subseteq T$. If $T \not\subseteq \hat{S}$ that inclusion is strict, so $\hat{S} \setminus W$ is smaller than T, contradicting minimality.
3. So T is a subset of the fixed $\hat{S}$: at most 2^ℓ choices. (Z, T) determines $W = Z \setminus T$.

Each pair contributes p^m, so the total over all W is at most $\binom{n}{w}(Lp)^{-m} 2^\ell p^m = \binom{n}{w} L^{-m} 2^\ell$. Dividing by $\binom{n}{w}$ and summing over $m \ge 0.9\ell$ gives the lemma for large L.

Iteration. Repeat on $\mathcal{H}'$ with a fresh random W_2, then W_3, …. The edge sizes shrink by 0.9 each round, so after about $\log_{1/0.9} \ell$ rounds one of two things has happened. Either the covers $\mathcal{U}_1 \cup \cdots \cup \mathcal{U}_\gamma$ together cover $\mathcal{H}$, or the fragments have shrunk to $\emptyset$ and $W_1 \cup \cdots \cup W_\gamma$ contains a witness. Because $\mathcal{H}$ is not p-small, the first case forces the total cost above $\frac{1}{2}$; by Markov's inequality that has probability at most $2\,E[\text{cost}]$, which the key lemma makes small. So the second case is the likely one. The union of the W_i has $O(Lpn \log \ell)$ elements (the paper enlarges the last few rounds by a $\sqrt{\log \ell}$ factor to make the failure probability tend to 0). **That is where** $\log \ell$ **comes from: the number of rounds.**

Worked example 3 — checking the counting step by brute force

On a small instance we can enumerate every W, compute every minimum fragment, and test step 2 and the multiplicity bound in step 3 directly. Take $X =$ the 15 edges of K_6 and $\mathcal{H} =$ its 15 perfect matchings ($\ell = 3$).

```
# fragments.py - exhaustive check of the counting step in Park-Pham's key lemma
# X = the 15 edges of K_6, H = its 15 perfect matchings (each a 3-edge set, l = 3).
# For every w-subset W of X and every S in H, compute a minimum (S, W)-fragment
#   T(S, W) = smallest S' \ W over S' in H with S'   W   S.
# The proof encodes the pair (W, T) by Z = W   T plus T as a subset of one fixed
# edge Ŝ   Z, so #pairs with |T| = m is at most C(n, w + m) * 2^l. Check it.
import itertools, math

V = range(6)
X = list(itertools.combinations(V, 2))                    # 15 edges
idx = {e: i for i, e in enumerate(X)}

def perfect_matchings(vs):
    if not vs:
        yield frozenset()
        return
    a, rest = vs[0], vs[1:]
    for b in rest:
        for m in perfect_matchings([v for v in rest if v != b]):
            yield m | {idx[(a, b)]}

H = list(perfect_matchings(list(V)))
n, l = len(X), max(len(S) for S in H)
print(f"|X| = {n} edges, |H| = {len(H)} perfect matchings, l = {l}")

def min_fragment(S, W):
    return min((Sp - W for Sp in H if Sp <= W | S), key=len)   # ties: first found

for w in (2, 4, 6):
    pairs, violations = {}, 0
    for W in map(frozenset, itertools.combinations(range(n), w)):
        for S in H:
            T = min_fragment(S, W)
            Z = W | T
            S_hat = next(Sp for Sp in H if Sp <= Z)           # a fixed choice given Z
            violations += not T <= S_hat                      # (16): T   Ŝ must hold
            pairs.setdefault(len(T), set()).add((W, T))
    print(f"\nw = {w}:  T   Ŝ violations = {violations}")
    print("  m   pairs (W,T)   distinct Z   max T per Z (<= 2^l)   bound C(n,w+m)*2^l")
    for m in sorted(pairs):
        per_Z = {}
```

```
        for W, T in pairs[m]:
            per_Z.setdefault(W | T, set()).add(T)              # (W, T) is fixed by (Z, T)
        worst = max(len(ts) for ts in per_Z.values())
        print(f"  {m}   {len(pairs[m]):11d}   {len(per_Z):10d}   {worst:20d}   {math.comb(n,
```

```
python3 fragments.py
```

```
|X| = 15 edges, |H| = 15 perfect matchings, l = 3

w = 2:  T  Ŝ violations = 0
  m   pairs (W,T)   distinct Z   max T per Z (<= 2^l)   bound C(n,w+m)*2^l
  1            45           15                       3                 3640
  2           540          180                       3                10920
  3           900          900                       1                24024

w = 4:  T  Ŝ violations = 0
  m   pairs (W,T)   distinct Z   max T per Z (<= 2^l)   bound C(n,w+m)*2^l
  0           180          180                       1                10920
  1          2745          945                       3                24024
  2          6840         2280                       3                40040
  3          2880         2880                       1                51480

w = 6:  T  Ŝ violations = 0
  m   pairs (W,T)   distinct Z   max T per Z (<= 2^l)   bound C(n,w+m)*2^l
  0          2790         2790                       1                40040
  1         10275         4515                       3                51480
  2          4860         1620                       3                51480
  3           360          360                       1                40040
```

What it shows:

- $T \subseteq \hat{S}$ **never fails** (0 violations over every W and every S, for three sizes of W). That is the one non-obvious step of the proof.
- **"max T per Z" is at most 3**, inside the $2^\ell = 8$ the proof allows: for a fixed Z, T is an m-subset of the 3-edge set $\hat{S}$, so there are at most $\binom{3}{m} \le 3$ choices. The number of pairs is therefore at most (distinct Z) $\times$ 3, and far below $\binom{n}{w+m}2^\ell$.
- Pairs with $m = 0$ appear once $w \ge 4$: some W already contain a perfect matching.

On a 15-element ground set the lemma's conclusion is vacuous (it needs L large and $\ell \to \infty$). The listing checks the *mechanism*, not the constants.

6. The trap

Trap: "the expectation threshold is where the expected number of copies is $\frac{1}{2}$." That is only the cover by minimal elements, a lower bound on q. The definition takes the best cover. For many properties they coincide up to constants, which is why the naive estimate works so often. But

the true p_c is governed by the cheapest *obstruction*, and for perfect matchings that obstruction (an isolated vertex) is local and invisible to a count of whole matchings.

Trap: "the theorem locates thresholds." It locates them up to a factor $K \log \ell$, with K unspecified. For triangles that is a constant; for spanning structures it is a $\log n$ window. Finding the exact constant (Erdős–Rényi's $\ln n/n$ for perfect matchings) still needs property-specific work.

Trap: "threshold" means two different things. p_c is the exact point where the probability is $\frac{1}{2}$. The Erdős–Rényi notion "threshold function" is only defined up to constant factors, and says nothing about how sharp the transition is. Coarse (triangles) and sharp (isolated vertices) thresholds both satisfy the theorem.

Trap: applying it to non-monotone properties. "Has exactly one triangle" or "is a forest" are not increasing; μ_p need not even be monotone in p.

7. The boring rule

To guess where a random structure appears, compute the expected number of minimal witnesses, then ask what the cheapest local obstruction is (an isolated vertex, an uncovered element). The threshold lies between the first-moment estimate and $\log \ell$ times it. If the two estimates disagree by more than that, one of them is wrong.

8. Pitfalls

1. Confusing q (best cover) with the naive count (cover by minimal elements).
2. Forgetting that ℓ is the size of the *largest minimal* witness, not $|X|$.
3. Treating p_c as the place where the expected count equals 1. For a Poisson count it is where the mean is $\ln 2$.
4. Reading a Monte Carlo transition at small n as the asymptotic one; finite-n corrections are visible already at $n = 400$.
5. Assuming a sharp threshold because the theorem holds; sharpness is a separate theory (Friedgut, Bourgain).

9. Checkpoint

- Define an increasing property, μ_p, p_c, a cover, p-small, and $q(\mathcal{F})$.
- Prove $q \le p_c$ with the union bound.
- Compute q_{naive} for triangles and perfect matchings in $G(n,p)$.
- Explain why perfect matchings need the $\log \ell$ factor.
- State the minimum-fragment definition and the inclusion $T \subseteq \widehat{S}$.

Exercises (try this)

1. Change `er_thresholds.py` to test connectivity instead of "no isolated vertex" (BFS from vertex 0). The limiting probability is the same $e^{-e^{-c}}$. Why?
2. Compute q_{naive} and p_c (Poisson heuristic) for "contains a K_4" in $G(n,p)$. Check that the ratio is bounded.
3. For "contains a Hamilton cycle", the minimal elements number $(n-1)!/2$. Show $q_{\text{naive}} \approx e/n$ and compare with the known threshold $(\ln n + \ln\ln n)/n$.
4. In `fragments.py`, replace perfect matchings by the 20 triangles of K_6 ($\ell = 3$) and by the 60 Hamilton cycles of K_6 ($\ell = 6$). Does $T \subseteq \widehat{S}$ still always hold? (It must: the argument used nothing about matchings.)
5. Prove $q \le q_f \le p_c$ for the fractional expectation threshold, using linearity of expectation in place of the union bound.
6. Show the cover $\{\{e\} : e \ni v\}$ certifies that "has a perfect matching" is p-small for $p \le \frac{1}{2(n-1)}$, and explain why a union of such stars over many vertices does **not** give a better cover.

Summary

An increasing property on a p-random set has a threshold p_c. The union bound over any cover gives a lower bound, and the best such bound is the expectation threshold q. Park and Pham proved that p_c is never more than $K \log \ell$ times q, where ℓ is the size of the largest minimal witness. The logarithm is necessary: perfect matchings in $G(n,p)$ have $q = \Theta(1/n)$ and $p_c \sim \ln n/n$, because isolated vertices are a coupon-collector obstruction. The proof is a randomised peeling: add random elements in $\approx \log \ell$ rounds, track minimum fragments of what is still missing, and count pairs (W,T) through $Z = W \cup T$ and $T \subseteq \widehat{S}$ to show that a cheap cover would otherwise appear, which the hypothesis forbids.

Sources (for further reading)

- J. Park, H. T. Pham, "A proof of the Kahn–Kalai conjecture", *J. Amer. Math. Soc.* 37 (2024), 235–243, https://doi.org/10.1090/jams/1028; arXiv:2203.17207, https://arxiv.org/abs/2203.17207 (definitions, Theorem 1.1, Theorem 1.4, Lemma 2.1, and the iteration in §2.2 follow this paper).
- J. Kahn, G. Kalai, "Thresholds and expectation thresholds", *Combin. Probab. Comput.* 16 (2007), 495–502.
- K. Frankston, J. Kahn, B. Narayanan, J. Park, "Thresholds versus fractional expectation-thresholds", *Ann. of Math.* 194 (2021), 475–495.
- R. Alweiss, S. Lovett, K. Wu, J. Zhang, "Improved bounds for the sunflower lemma", *Ann. of Math.* 194 (2021), 795–815.
- M. Talagrand, "Are many small sets explicitly small?", *STOC 2010*, 13–35.
- B. Bollobás, A. Thomason, "Threshold functions", *Combinatorica* 7 (1987), 35–38.
- P. Erdős, A. Rényi, "On random graphs I" (1959); "On the evolution of random graphs" (1960); "On the existence of a factor of degree one of a connected random graph" (1966).
- E. Friedgut, "Sharp thresholds of graph properties, and the k-SAT problem", *J. Amer. Math. Soc.* 12 (1999), 1017–1054.

- A. Johansson, J. Kahn, V. Vu, "Factors in random graphs", *Random Structures Algorithms* 33 (2008), 1–28; R. Montgomery, "Spanning trees in random graphs", *Adv. Math.* 356 (2019).
- B. Bollobás, *Random Graphs*, 2nd ed., Cambridge University Press, 2001; S. Janson, T. Łuczak, A. Ruciński, *Random Graphs*, Wiley, 2000.

Linear Algebra (Advanced)

Advanced Linear Algebra for CS/ML

Advanced linear algebra studies structure beyond "multiply matrices": spectral decompositions, orthogonality, low-rank approximation, and the numerical meaning of conditioning. These tools run PCA, PageRank-like iterations, least squares solvers, quantum-inspired algorithms, and the theory of gradient descent on quadratics.

Why go beyond basic LA

Capability	Payoff
Eigen / spectral	Dynamics A^k, stability, graph spectra
Orthogonality	Stable bases, QR, projections
SVD	PCA, compression, pseudoinverse, recommendations
Conditioning	Predict numerical accuracy of solves

Learning path

```
flow:
  B -> C
  C -> D
  A -> C
```

Matrix as linear map

$A \in \mathbb{R}^{m \times n}$ represents $x \mapsto Ax$. Change of basis $A = PBP^{-1}$ (square case) reveals simpler action: diagonal, Jordan, or orthogonal spectral form.

Worked example 1

On an eigenbasis, A scales axes independently—powers and exponentials e^{tA} become easy.

Worked example 2 — graphs

Adjacency and Laplacian matrices encode graphs; their eigenvalues constrain cuts, connectivity, and random walk mixing.

Roadmap

1. Eigenvalues and eigenvectors
2. Orthogonality and projections
3. SVD and PCA
4. Least squares and conditioning

Prerequisites: matrix multiply, invertibility, rank, basic Gaussian elimination. Inner products strongly recommended.

CS/ML touchpoints preview

- PCA via top singular vectors
- Ridge regression spectral filtering
- Attention and Gram matrices (pairwise similarities)
- Kalman / Gaussian covariances SPD structure
- Word embeddings and matrix factorization

Four identities worth memorizing

1. **Eigen:** $Av = \lambda v \Rightarrow A^k v = \lambda^k v$ (when defined)
2. **Symmetric spectral:** $A = A^\top \Rightarrow A = Q\Lambda Q^\top$ with $Q^\top Q = I$
3. **SVD:** $A = U\Sigma V^\top$, $A_k = \sum_{i \le k} \sigma_i u_i v_i^\top$ optimal rank-k
4. **Normal equations:** $A^\top A\hat{x} = A^\top b$ for least squares (prefer QR/SVD numerically)

Worked example 3 — PCA in one line

Center X, compute thin SVD $X = U\Sigma V^\top$; top columns of V are principal axes; scores XV_k.

Worked example 4 — ridge filter

In the right singular basis, ridge multiplies component i by $\sigma_i/(\sigma_i^2 + \lambda)$—shrinks noisy small-$\sigma$ directions.

Worked example 5 — graph Laplacian

$L = D - A \succeq 0$; multiplicity of eigenvalue 0 equals the number of connected components; Fiedler vector (for λ_2) seeds spectral partitioning.

Orthogonality as numerical gold

Orthogonal/unitary transforms preserve $\|\cdot\|_2$ and have $\kappa_2 = 1$. Householder QR and SVD are built from them—why library solvers beat naive elimination on tough problems.

Pitfalls

1. Non-diagonalizable matrices (defective) break naive $A = PDP^{-1}$
2. Complex eigenvalues of real nonsymmetric A
3. Confusing left/right singular vectors
4. Interpreting every basis as orthogonal
5. Ignoring $\kappa(A)$ when solving $Ax = b$
6. PCA without centering or without scaling incomparable features

Checkpoint

- State eigen-equation $Av = \lambda v$
- Know spectral theorem for real symmetric matrices
- Name SVD factors $U\Sigma V^\top$
- Connect PCA to covariance eigenvectors
- Define condition number

Exercises

1. Compute eigenvalues of a 2×2 diagonal matrix; of a rotation by 90°.
2. Why do real symmetric matrices have real eigenvalues? (one reason)
3. Give a defective 2×2 Jordan block example.
4. What does $\sigma_{\min}(A) = 0$ mean for least squares?

5. PageRank as eigenproblem: what eigenvalue is targeted?
6. Checkpoint reading: map one ML method you know to an eigen/SVD step.

Summary

This part upgrades linear algebra from row reduction to spectral geometry and stable computation—the language of modern data and scientific computing.

Eigenvalues and Eigenvectors

Eigenvectors are directions a linear map merely stretches; eigenvalues are the stretch factors. Spectral theory unlocks matrix powers, differential systems, PCA special cases, Markov stationarity, and stability of discrete dynamics.

1. Definition

Let $A \in \mathbb{C}^{n\times n}$ (or $\mathbb{R}^{n\times n}$). A nonzero vector v is an **eigenvector** with **eigenvalue** λ if

$$Av = \lambda v \iff (A - \lambda I)v = 0.$$

So λ is eigenvalue iff $A - \lambda I$ is singular iff $\det(A - \lambda I) = 0$.

Worked example 1

$A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$. Characteristic polynomial $(2-\lambda)^2 - 1 = (\lambda - 3)(\lambda - 1)$.
$\lambda = 3$ with $v \propto (1, 1)$; $\lambda = 1$ with $v \propto (1, -1)$.

Worked example 2

Diagonal $D = \operatorname{diag}(\lambda_i)$: standard basis vectors are eigenvectors.

2. Characteristic polynomial

$p_A(\lambda) = \det(A - \lambda I)$ is degree n. Fundamental theorem of algebra: n roots in $\mathbb{C}$ counting multiplicity (**algebraic multiplicity**).

Geometric multiplicity: $\dim \ker(A - \lambda I) \geq 1$ for each eigenvalue. Always $\leq$ algebraic multiplicity.

Worked example 3 — defective matrix

$J = \begin{bmatrix} \lambda & 1 \\ 0 & \lambda \end{bmatrix}$ has one eigenvector direction (up to scale); not diagonalizable.

3. Diagonalization

Theorem. A is diagonalizable iff there is a basis of eigenvectors iff $A = PDP^{-1}$ with D diagonal.

Then $A^k = PD^kP^{-1}$ and (when defined) $f(A) = Pf(D)P^{-1}$ for analytic f.

Worked example 4 — dynamics

$x_{t+1} = Ax_t \Rightarrow x_t = A^t x_0$. In eigenbasis, each mode multiplies by λ_i^t. Stable discrete LTI if all $|\lambda_i| < 1$.

Worked example 5 — continuous

$\dot{x} = Ax \Rightarrow x(t) = e^{tA}x_0$; modes $e^{\lambda_i t}$.

4. Spectral theorem (real symmetric)

Theorem. If $A = A^\top \in \mathbb{R}^{n\times n}$, then:

1. All eigenvalues real
2. Eigenvectors for distinct eigenvalues are orthogonal
3. $A = Q\Lambda Q^\top$ with Q orthogonal ($Q^\top Q = I$), Λ diagonal

Proof sketch. Existence of at least one real eigenpair (real char poly degree odd? actually use complex eigenpair and show imaginary part vanishes via v^*Av real); induct by restricting to orthogonal complement; orthogonality from $(\lambda - \mu)u^\top v = u^\top Av - v^\top Au = 0$. ■

Worked example 6

Covariance matrices and graph Laplacians are real symmetric (PSD actually for Laplacians/cov)—spectral theorem applies fully.

Hermitian extension

$A = A^*$ over $\mathbb{C}$: unitary diagonalization with real eigenvalues.

5. Invariant subspaces and Schur form (awareness)

Every square matrix is unitarily similar to upper triangular (Schur). Eigenvalues appear on the diagonal—even when not diagonalizable.

6. Rayleigh quotient

For symmetric A,

$$R(v) = \frac{v^\top A v}{v^\top v}, \qquad \lambda_{\min} \le R(v) \le \lambda_{\max}.$$

Maximizing R recovers the top eigenpair—basis of power methods and PCA variance interpretation.

Worked example 7

Power iteration: $v \leftarrow Av/\|Av\|$ converges to dominant eigenvector if $|\lambda_1| > |\lambda_2|$ gap.

7. Graph spectra (CS)

- Adjacency A: $\lambda_{\max}$ related to max degree; regularity
- Laplacian $L = D - A$: $L \succeq 0$, 0 eigenvalue with multiplicity = # components; Fiedler value λ_2 measures connectivity

Worked example 8

Connected graph $\Leftrightarrow \ker L$ is constants only ($\lambda_2 > 0$).

Worked example 9 — spectral clustering

Embed nodes via small Laplacian eigenvectors; cluster in that space.

8. Markov chains link

Stationary π satisfies $P^\top \pi = \pi$—eigenvalue 1 of $P^\top$. Spectral gap controls mixing.

Worked example 10

PageRank: dominant eigenpair of Google matrix.

9. Non-symmetric caveats

- Complex conjugate eigenvalues
- Defective matrices / Jordan blocks: polynomial factors $t^k\lambda^t$ in powers
- Left eigenvectors $w^\top A = \lambda w^\top$ matter for biorthogonal expansions

Worked example 11

Rotation by $\theta \neq 0, \pi$ in 2D: eigenvalues $e^{\pm i\theta}$, no real eigenbasis.

10. Trace and determinant

$\mathrm{tr}(A) = \sum \lambda_i$, $\det(A) = \prod \lambda_i$ (over $\mathbb{C}$ with multiplicity). Useful checksums.

Worked example 12

$\det(A) \neq 0 \Leftrightarrow 0$ not eigenvalue $\Leftrightarrow$ invertible.

11. Pitfalls

1. Assuming all matrices diagonalizable
2. Normalizing differently and thinking eigenvectors unique
3. Sorting eigenvalues inconsistently (algebraic vs by magnitude)
4. Using Euclidean orthogonality for non-symmetric eigenbases
5. Floating-point: naive charpoly is unstable—use library eigensolvers

12. Checkpoint

- Solve 2×2 eigenproblems by hand
- State diagonalization criterion
- Apply spectral theorem for symmetric A
- Use $A^k = PD^kP^{-1}$

- Connect Laplacian λ_2 to connectivity

Exercises

Easy

1. Eigenvalues of $\begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}$; diagonalizable?
2. Show eigenvectors of distinct eigenvalues of symmetric A are orthogonal.
3. Compute $\operatorname{tr}, \det$ from eigenvalues of Example 1.
4. If $Av = \lambda v$, what is $A^{-1}v$ when A invertible?
5. Rayleigh for $A = I$: what values?

Medium

6. Diagonalize Example 1; compute A^5 via PD^5P^{-1}.
7. Prove $\ker L$ dimension equals components for graph Laplacian.
8. Show $|\lambda| \le \|A\|$ for any operator norm (spectral radius bounds).
9. Power method: why gap $|\lambda_1/\lambda_2|$ controls rate.
10. For projection matrix $P^2 = P = P^\top$, show eigenvalues in $\{0, 1\}$.

Challenge

11. Prove spectral theorem for real symmetric 2×2 fully elementary.
12. Jordan form existence statement; give dynamics when Jordan block size > 1.
13. Courant–Fischer min-max theorem statement for λ_k.
14. Google matrix: prove eigenvalue 1 exists for row-stochastic matrices.
15. Sensitivity: Bauer–Fike sketch—eigenvalue perturbation vs non-normality.

Summary

Eigenpairs reduce linear maps to scalings on special lines. Symmetric matrices are orthogonally diagonalizable—the cleanest spectral world—while defective and non-normal matrices warn that not every A behaves like a stretch on an orthonormal frame. CS applications from PageRank to spectral clustering rest on these facts.

Orthogonality and Projections

Orthogonality is the geometry of right angles in inner product spaces. It yields stable bases (QR), optimal approximation (orthogonal projections), and the Pythagorean decompositions behind least squares.

1. Inner products and norms

On $\mathbb{R}^n$, standard inner product $\langle u, v\rangle = u^\top v$, norm $\|v\|_2 = \sqrt{\langle v, v\rangle}$.

Cauchy–Schwarz: $|\langle u, v\rangle| \le \|u\|\|v\|$, equality iff linearly dependent.

Angle: $\cos\theta = \langle u, v\rangle / (\|u\|\|v\|)$ for nonzero vectors.

Worked example 1

$u = (1, 0)$, $v = (1, 1)$: $\cos\theta = 1/\sqrt{2}$, $\theta = \pi/4$.

2. Orthogonal sets and bases

Set $\{u_i\}$ **orthogonal** if $i \ne j \Rightarrow \langle u_i, u_j\rangle = 0$; **orthonormal** if also $\|u_i\| = 1$.

Theorem. Orthogonal nonzero vectors are linearly independent.

Parseval / Pythagoras: if $\{q_i\}$ orthonormal basis,

$$\|x\|^2 = \sum_i |\langle x, q_i\rangle|^2, \qquad x = \sum_i \langle x, q_i\rangle q_i.$$

Worked example 2

Fourier-like expansions in finite dimensions: coefficients are inner products.

3. Orthogonal projection onto a line

Projection of x onto $\text{span}\{u\}$ ($u \neq 0$):

$$\text{proj}_u x = \frac{\langle x, u \rangle}{\langle u, u \rangle} u.$$

Error $x - \text{proj}_u x \perp u$.

Worked example 3

Project $(2, 2)$ onto $\text{span}\{(1, 0)\}$: get $(2, 0)$; residual $(0, 2)$.

4. Projection onto a subspace

Let $U = \text{range}(A)$ with $A \in \mathbb{R}^{n \times k}$ full column rank. Orthogonal projection:

$$P = A(A^\top A)^{-1} A^\top, \qquad Px = \arg\min_{y \in U} \|x - y\|_2.$$

Properties: $P^2 = P = P^\top$; eigenvalues 0 or 1.

If columns of Q orthonormal basis of U, then $P = QQ^\top$—numerically better.

Worked example 4

Least squares $\min_b \|Ab - y\|$ is projection of y onto $\text{col}(A)$; normal equations $A^\top A\hat{b} = A^\top y$.

Worked example 5 — residual orthogonality

$A^\top(y - A\hat{b}) = 0$: residual $\perp$ column space—normal equations geometry.

5. Gram–Schmidt and QR

Classical Gram–Schmidt orthogonalizes columns of A to produce $A = QR$ with Q orthonormal columns, R upper triangular.

Modified Gram–Schmidt and **Householder QR** are more stable in FP arithmetic.

Worked example 6

QR solve for least squares: $A = QR \Rightarrow R\hat{b} = Q^\top y$ (thin QR)—avoids squaring κ via $A^\top A$.

Worked example 7 — Householder idea

Reflectors $I - 2vv^\top/\|v\|^2$ zero out entries below pivots—stable orthogonal transforms.

6. Orthogonal matrices

Q square with $Q^\top Q = I$ (so $Q^{-1} = Q^\top$). Preserves norms and angles: $\|Qx\| = \|x\|$.

Products of reflections/rotations. Condition number $\kappa_2(Q) = 1$—perfectly conditioned linear maps.

Worked example 8

Rotations in graphics: orthogonal (or unitary) transforms avoid distorting lengths.

7. Orthogonal complements

$U^\perp = \{v : \langle v, u\rangle = 0 \ \forall u \in U\}$. $\mathbb{R}^n = U \oplus U^\perp$ for subspaces.
$(U^\perp)^\perp = U$. For A, $\ker(A) = (\text{row}(A))^\perp$, $\ker(A^\top) = (\text{col}(A))^\perp$.

Worked example 9 — fundamental theorem of linear algebra

Four fundamental subspaces linked by orthogonality—Strang's diagram.

8. Projections in ML/CS

Use	Geometry
PCA reconstruction	project onto top principal subspace
Residual connections (loose analogy)	skip paths vs learned maps
OLS / GLM linear predictors	column space projection
Nearest subspace classifiers	distance to class subspaces
Kalman updates	sequential orthogonalization ideas

Worked example 10 — centering

Subtracting the mean is projection orthogonal to the all-ones vector in sample space—prep for covariance.

Worked example 11 — dual view

Ridge can be seen as shrinking coordinates in singular vector basis—orthogonal decomposition of feature space.

9. Non-orthogonal projections (warning)

Oblique projections ($P^2 = P$ but $P \neq P^\top$) appear in some domain decompositions; they do **not** solve unconstrained least squares. Stick to orthogonal projections for $\|\cdot\|_2$ optimality.

10. Pitfalls

1. Classical GS loss of orthogonality in FP for nearly dependent columns
2. Using $A(A^\top A)^{-1}A^\top$ when A ill-conditioned—prefer QR/SVD
3. Forgetting to orthonormalize before $P = QQ^\top$
4. Confusing algebraic projection matrices with statistical "projection pursuit"
5. Applying Euclidean orthogonality after arbitrary feature scaling without care

11. Checkpoint

- Project onto lines and subspaces
- State $P = QQ^\top$ for orthonormal bases
- Connect least squares to residual orthogonality
- Explain QR advantage over normal equations
- Use orthogonal complements for kernels/row spaces

Exercises

Easy

1. Prove Cauchy–Schwarz from $\| u - tv\|^2 \geq 0$ optimized in t.
2. Project $(1, 2, 3)$ onto span of $(1, 1, 1)$/normalized.
3. Show $P = qq^\top$ for unit q is a projection.
4. Verify Q rotation by 90° is orthogonal.
5. Why is $\kappa_2(Q) = 1$?

Medium

6. Derive normal equations from residual $\perp \operatorname{col}(A)$.
7. Prove orthogonal nonzero vectors are independent.
8. Gram–Schmidt two columns by hand.
9. Show $I - P$ projects onto $U^{\perp}$ when P orthoproj onto U.
10. Prove $\|Px\| \leq \|x\|$ for orthogonal projections.

Challenge

11. Modified vs classical GS: explain reorthogonalization need.
12. Householder QR: write algorithm to zero a column below diagonal.
13. Prove four fundamental subspaces orthogonality relations.
14. Conditioning: compare $\kappa(A^{\top}A)$ vs $\kappa(A)$ for thin A.
15. Implement (pseudocode) least squares via thin QR; count FLOPs vs normal eq.

Summary

Orthogonality gives independent directions, optimal subspace approximations, and norm-preserving transforms. Projections and QR are the computational geometry of least squares and of every method that says "best approximation in a subspace."

SVD and PCA

The singular value decomposition (SVD) is the Swiss army knife of applied linear algebra: it diagonalizes any matrix via two orthogonal bases, yields optimal low-rank approximations, and powers PCA, pseudoinverses, and many recommender and NLP factorizations.

1. SVD statement

Theorem. Any $A \in \mathbb{R}^{m \times n}$ admits

$$A = U \Sigma V^\top$$

where $U \in \mathbb{R}^{m \times m}$ and $V \in \mathbb{R}^{n \times n}$ are orthogonal, and $\Sigma \in \mathbb{R}^{m \times n}$ is rectangular diagonal with nonnegative **singular values**

$$\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > 0 = \cdots, \qquad r = \operatorname{rank}(A).$$

Thin/economy SVD: keep first r (or k) columns of U, V and $k \times k$ Σ_k.

Geometric reading

V orthonormal input directions; Σ axis-aligned stretches; U orthonormal output directions. A maps $v_i \mapsto \sigma_i u_i$.

Worked example 1

Rank-1: $A = uv^\top$ (up to scale) has one positive singular value $\|u\|\|v\|$ in simple cases—outer products are the atoms of SVD expansions $A = \sum_i \sigma_i u_i v_i^\top$.

2. Links to eigenvalues

$A^\top A = V \Sigma^\top \Sigma V^\top$: eigenvalues of $A^\top A$ are σ_i^2.
$AA^\top = U \Sigma \Sigma^\top U^\top$: same nonzero spectrum.

Singular vectors are eigenvectors of these Gram matrices—but computing SVD via forming $A^\top A$ can be less stable than dedicated SVD algorithms.

Worked example 2

If A is symmetric PSD, SVD coincides with eigen-decomposition ($U = V$).

3. Eckart–Young–Mirsky theorem

Theorem. Among rank-at-most-k matrices, the truncated SVD

$$A_k = \sum_{i=1}^{k} \sigma_i u_i v_i^\top$$

minimizes both $\|A - B\|_2$ and $\|A - B\|_F$. Error $\|A - A_k\|_F^2 = \sum_{i>k} \sigma_i^2$, $\|A - A_k\|_2 = \sigma_{k+1}$.

Worked example 3 — compression

Store U_k, Σ_k, V_k instead of dense A when σ_i decay rapidly—image compression and latent factor models.

Worked example 4 — denoising

If noise inflates small singular values, truncating acts as a spectral filter.

4. Moore–Penrose pseudoinverse

$$A^+ = V\Sigma^+ U^\top, \qquad \Sigma^+ \text{ reciprocates nonzero } \sigma_i.$$

Solves min-norm least squares: $\hat{x} = A^+ b$ minimizes $\|x\|$ among minimizers of $\|Ax - b\|$ (when consistent, min-norm solution of $Ax = b$).

Worked example 5

Underdetermined full-row-rank A: $A^+ = A^\top (AA^\top)^{-1}$.
Overdetermined full-column-rank: $A^+ = (A^\top A)^{-1} A^\top$.

5. PCA via SVD

Data. Centered matrix $X \in \mathbb{R}^{n \times d}$ (rows observations). Sample covariance $\propto X^\top X$.

PCA: principal directions = top eigenvectors of covariance = right singular vectors of X.
Scores = $XV_k = U_k \Sigma_k$.

Objective: maximize variance of projected data, or equivalently minimize reconstruction error $\|X - X_k\|_F$—Eckart–Young.

Worked example 6

2D correlated Gaussian cloud: first PC along major axis; second orthogonal minor axis.

Worked example 7 — explained variance

Fraction $\sigma_j^2 / \sum_i \sigma_i^2$ (for centered X) measures variance along PC j.

Worked example 8 — whitening

Transform $XV\Sigma^{-1}$ (careful with tiny σ) yields approximately identity covariance—preprocessing for some models.

6. Recommendations and factor models

Approximate ratings matrix $R \approx WH$ or via SVD of filled/centered R. Truncation = latent factors.

Worked example 9

Latent dimension k: users and items embed into $\mathbb{R}^k$; predictions inner products—classic collaborative filtering.

7. Conditioning and numerical rank

$\kappa_2(A) = \sigma_1/\sigma_{\min}$ for square invertible A.
Numerical rank: count σ_i above a threshold relative to σ_1 and machine precision.

Worked example 10

Nearly rank-deficient design matrices: tiny $\sigma_{\min} \Rightarrow$ huge coefficient sensitivity in OLS—regularize (ridge shrinks small directions).

8. Randomized SVD (awareness)

Sketch A with random projections; compute thin SVD of sketch—near-optimal low-rank approx for huge matrices (Halko–Martinsson–Tropp).

Worked example 11

Billion-scale PCA: randomized algorithms + streaming variants.

9. Pitfalls

1. Forgetting to **center** before PCA
2. Interpreting PCs as causal factors
3. Scaling features inconsistently (PCA is not scale-invariant)
4. Using too large k (fitting noise)
5. Forming $X^\top X$ explicitly for wide data when $XX^\top$ is smaller (dual PCA)
6. Confusing U and V roles (samples vs features)

Worked example 12 — scale

Feature in meters vs millimeters dominates PCs if unstandardized—standardize when variables incomparable.

10. Checkpoint

- State full and truncated SVD
- Apply Eckart–Young error formulas
- Build pseudoinverse from SVD
- Derive PCA from SVD of centered X
- Read explained variance from singular values

Exercises

Easy

1. SVD of a diagonal rectangular matrix?
2. Show $\|A\|_2 = \sigma_1$ and $\|A\|_F^2 = \sum \sigma_i^2$.
3. Rank-(1) approx of A from top singular triple.
4. Why center for PCA?
5. κ_2 from singular values for invertible square A.

Medium

6. Prove $A^\top A$ eigenvectors relate to V (compute).
7. Show A_k has rank $\leq k$ and write residual Frobenius error.
8. Dual PCA: when $n \ll d$, eigen of $XX^\top$ vs $X^\top X$.
9. Ridge: show solution filters singular components by $\sigma/(\sigma^2 + \lambda)$.
10. Pseudoinverse: verify $AA^+A = A$ for thin full-rank cases.

Challenge

11. Prove Eckart–Young for Frobenius norm using orthogonal invariance of $\|\cdot\|_F$.
12. Randomized range finder algorithm sketch + why oversampling helps.
13. PCA as maximum variance: Lagrange multiplier derivation for first PC.
14. Robust PCA awareness: sparse + low-rank decomposition goal.
15. Systems: compress a 1000×800 image matrix by keeping 5% energy; estimate storage.

Summary

SVD factors every matrix into orthogonal geometry plus nonnegative scales. Truncation is optimal low-rank approximation; PCA is SVD on centered data; pseudoinverses and ridge filters live naturally in the singular basis. For data and scientific computing, SVD is often the first decomposition to reach for.

Least Squares and Conditioning

Least squares is the default linear fitting tool; conditioning explains when solutions are trustworthy in finite precision. Together they connect geometry (projections), algebra (normal equations / QR / SVD), and numerics (error amplification).

1. Linear least squares problem

Given $A \in \mathbb{R}^{m\times n}$, $b \in \mathbb{R}^m$, $m \geq n$ typically:

$$\min_{x\in\mathbb{R}^n} \|Ax - b\|_2.$$

If A full column rank, unique solution

$$\hat{x} = (A^\top A)^{-1} A^\top b.$$

Geometry: $A\hat{x} = P_{\mathrm{col}(A)} b$.

Worked example 1 — line fit

A columns $[1, x_i]$, $b = y_i$: slope/intercept regression.

Worked example 2 — overdetermined system

No exact solution if $b \notin \mathrm{col}(A)$; LS finds best compromise in ℓ_2.

2. Normal equations

$A^\top A\hat{x} = A^\top b$. Derive by setting $\nabla_x \frac{1}{2}\|Ax - b\|^2 = A^\top(Ax - b) = 0$.

Pros: simple, $n \times n$ system.
Cons: $\kappa(A^\top A) = \kappa(A)^2$—squares condition number; forms Gram matrix with possible cancellation.

Worked example 3

If $\kappa(A) = 10^8$, $\kappa(A^\top A) = 10^{16}$—double precision collapses. Prefer QR/SVD.

3. QR method

Thin QR $A = QR$ ($Q \in \mathbb{R}^{m \times n}$ orthonormal columns, R upper triangular invertible if full rank):

$$R\hat{x} = Q^\top b.$$

Backward stable in practice with Householder QR.

Worked example 4

Same solution as normal equations in exact arithmetic; better residual/forward accuracy in FP when A moderately ill-conditioned.

4. SVD method

$A = U\Sigma V^\top$, $\hat{x} = V\Sigma^+ U^\top b$ (pseudoinverse). Handles rank deficiency via truncated or regularized inverses of σ_i.

Worked example 5 — rank-deficient

If $\sigma_n = 0$, infinite solutions to $\min \|Ax - b\|$; min-norm via Σ^+.

5. Weighted and generalized LS

$\min \|W^{1/2}(Ax - b)\|$ for heteroscedastic noise. Generalized SVD / whitening when noise covariance known.

Worked example 6

GLS: if $\mathrm{Cov}(\varepsilon) = \Sigma$, transform by $\Sigma^{-1/2}$ then OLS—BLUE under Gauss–Markov extensions.

6. Ridge regression as conditioning fix

$$\min_x \|Ax - b\|_2^2 + \lambda \|x\|_2^2, \quad \lambda > 0.$$

Solution $(A^\top A + \lambda I)^{-1} A^\top b$. In SVD basis, component along v_i multiplied by $\sigma_i/(\sigma_i^2 + \lambda)$.

Worked example 7

Small σ_i directions damped—stabilizes inversion and reduces variance.

Worked example 8 — leverage

Hat matrix $H = A(A^\top A)^{-1}A^\top$; diagonal H_{ii} leverage of point i. High leverage points dominate fit—check influence.

7. Condition number and error bounds

For square invertible systems, $\kappa(A) = \|A\|\|A^{-1}\|$. Relative residual and relative forward error linked by κ.

For LS, effective sensitivity involves $\kappa(A)$ and how close b is to $\mathrm{col}(A)$ (angle subtleties).

Worked example 9 — Hilbert design

Polynomial regression with monomial basis: Vandermonde/Hilbert-like severe ill-conditioning—use orthogonal polynomials.

Worked example 10 — collinearity

Nearly dependent columns: $\sigma_{\min}$ tiny; coefficients large and unstable. VIF diagnostics in statistics.

8. Iterative methods for large LS

- LSQR / LSMR on $Ax \approx b$
- Normal eq CG on $A^\top A$ (careful conditioning)
- Stochastic / sketching for enormous m

Worked example 11

$m = 10^7$, $n = 10^3$ sparse: iterative Krylov + good preconditioner beats dense QR.

9. Gauss–Markov theorem (stat view)

If $y = Ax+\varepsilon$, $\mathbb{E}\varepsilon = 0$, $\mathrm{Cov}(\varepsilon) = \sigma^2 I$, A full rank, then OLS is BLUE (best linear unbiased estimator). Heteroscedasticity/correlation $\Rightarrow$ GLS.

Worked example 12 — misspecification

Nonlinear truth or omitted variables: LS still projects, but coefficients may not estimate "causal" parameters.

10. Pitfalls

1. Normal equations by default for ill-conditioned A
2. Interpreting huge coefficients without checking κ
3. Ignoring intercept / centering inconsistency
4. Multiple collinear features + no regularization
5. Extrapolation far from row space of training A
6. Using R^2 alone without residual diagnostics

11. Checkpoint

- State LS geometry and normal equations
- Solve via QR and SVD conceptually
- Explain why ridge stabilizes
- Define κ and risk of collinearity
- Choose direct vs iterative at scale

Exercises

Easy

1. Derive $\nabla\|Ax-b\|^2=0 \Rightarrow$ normal equations.
2. For orthogonal columns of A, simplify $\hat{x}$.
3. Compute LS fit of points $(0,0),(1,1),(2,1)$ by a line through origin $y=\beta x$.
4. Why is $A^\top A$ SPD when A full column rank?
5. Write ridge closed form.

Medium

6. Prove residual $b-A\hat{x} \perp \mathrm{col}(A)$.
7. Show economy QR LS formula.
8. Express ridge solution in SVD basis (filter factors).
9. Relate leverage H_{ii} to sensitivity of $\hat{y}_i$.
10. Compare FLOPs: form normal eq + Cholesky vs Householder QR for $m \gg n$.

Challenge

11. Prove Gauss–Markov for simple case $n = 1$.
12. Conditioning of Vandermonde: explain growth with degree.
13. Total least squares vs OLS when A is noisy (idea).
14. Weighted LS as standard LS on transformed data.
15. Design numerical experiment: collinear features, compare OLS vs ridge coefficient variance.

Summary

Least squares is orthogonal projection onto a model subspace; algorithms (normal eq, QR, SVD) differ in stability. Conditioning and collinearity decide whether coefficients mean anything in floating point. Regularization and better bases turn fragile fits into reliable estimators—core craft for data science and scientific computing.

Spectral Thinking: Eigenvalues in Practice

Eigenpairs ($Av = \lambda v$) are not only homework: they govern **stability**, **oscillation**, **PCA**, **PageRank-like** updates, Markov mixing, graph cuts, and conditioning of linear solves.

Diagram: eigen action

```
      v

A      if Av =  v
      v
     v   (same direction, scaled by  )
```

1. Core facts

Let $A \in \mathbb{R}^{n \times n}$ (or complex).

- λ eigenvalue, $v \neq 0$ eigenvector: $Av = \lambda v$ iff $\det(A - \lambda I) = 0$
- Distinct eigenvalues linearly independent eigenvectors
- **Real symmetric** A: all λ real; orthonormal basis of eigenvectors; $A = Q\Lambda Q^\top$
- Spectral radius $\rho(A) = \max_i |\lambda_i|$ controls growth of A^k
- $\mathrm{tr}(A) = \sum \lambda_i$, $\det(A) = \prod \lambda_i$ (over $\mathbb{C}$, with multiplicity)

Worked example 1

$A = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$: $\lambda = \pm 1$, vectors $(1, 1)$ and $(1, -1)$.

Worked example 2 — diagonal action

$A = \begin{bmatrix} 2 & 0 \\ 0 & 0.5 \end{bmatrix}$: $\lambda = 2, 0.5$ with axis eigenvectors.
A^{10} stretches e_1 by 2^{10}, shrinks e_2 by 0.5^{10}.

2. Powers, iterations, and stability

If A is diagonalizable, $A = V\Lambda V^{-1}$, then

$$A^k = V\Lambda^k V^{-1}.$$

Along eigenvector v_i, components multiply by λ_i^k.

```
| |<1  →  component dies
| |>1  →  explodes
| |=1  →  persists / oscillates (sign or complex angle)
```

Linear iteration $x \leftarrow Ax$ (from a generic start) converges to 0 if $\rho(A) < 1$ (defective cases need Jordan-block care: polynomial factors $k^m\lambda^k$).

Worked example 3 — discrete dynamics

$x_{t+1} = Ax_t$. Stability of the origin requires $\rho(A) < 1$. In control and signal processing, place eigenvalues of closed-loop maps carefully.

Worked example 4 — Markov / PageRank flavor

Column-stochastic matrices have $\lambda = 1$ with stationary distribution eigenvector. Power iteration $x \leftarrow Ax/\|Ax\|$ (or normalized differently) finds dominant eigenpairs — PageRank is a damped variant guaranteeing uniqueness and convergence.

3. Symmetric matrices and Rayleigh quotients

For $A = A^\top$,

$$R(x) = \frac{x^\top Ax}{x^\top x}, \qquad \lambda_{\min} = \min_{\|x\|=1} x^\top Ax, \quad \lambda_{\max} = \max_{\|x\|=1} x^\top Ax.$$

Variational characterization drives PCA, spectral clustering objectives, and many optimization bounds.

4. PCA link

Covariance $C = \frac{1}{n}X_c^\top X_c$ (centered data) is symmetric PSD. Top eigenvectors = principal directions of variance; eigenvalues = variances along those axes. Full development in the SVD/PCA chapter — spectral thinking says: **variance spectrum of** C.

5. Graph Laplacian

For undirected graph with adjacency A and degree matrix D:

$$L = D - A.$$

Properties:

- L symmetric PSD: $x^\top L x = \sum_{(i,j)\in E}(x_i - x_j)^2$
- $L\mathbf{1} = 0$ always $\lambda = 0$ with constant eigenvector
- Multiplicity of $\lambda = 0$ = number of connected components
- **Spectral clustering:** use smallest nontrivial eigenvectors as embeddings, then k-means
- Fiedler value λ_2: algebraic connectivity — small easy cut

Worked example 5 — path of 3 nodes

Degrees $(1, 2, 1)$; L has rows summing to 0 0 is eigenvalue with $(1, 1, 1)$.

6. Conditioning

For symmetric invertible A, in the 2-norm:

$$\kappa_2(A) = \frac{|\lambda_{\max}|}{|\lambda_{\min}|}.$$

Large κ sensitive linear solves and inverted maps. For general A, $\kappa_2(A) = \sigma_{\max}/\sigma_{\min}$ (singular values) — eigenvalues alone can mislead for non-normal matrices.

Worked example 6 — non-normal caution

Some matrices have all $|\lambda|$ moderate yet $\|A^k\|$ transiently huge (non-normality). Spectrum is necessary but not always sufficient for transient behavior.

7. Power method and cousins

Power method: iterate $x \leftarrow Ax/\|Ax\|$. Converges to dominant eigenvector if $|\lambda_1| > |\lambda_2|$ strictly and start has a component along v_1.

Inverse iteration / Rayleigh quotient iteration: find interior eigenvalues.
QR algorithm / modern eigensolvers: library defaults for dense problems.
Lanczos / Arnoldi: large sparse partial spectra.

Worked example 7

Dominant eigenpair of a link matrix "importance" ranking direction when the model is linear and well-connected.

8. Differential equations (glimpse)

$\dot{x} = Ax$: solutions involve e^{At}. Modes $e^{\lambda t}v$. Stability needs $\mathrm{Re}(\lambda) < 0$ for all eigenvalues. Spectral thinking unifies discrete A^k and continuous e^{At}.

9. Positive matrices (Perron–Frobenius flavor)

For positive (or irreducible nonnegative) matrices, the spectral radius is a simple positive eigenvalue with positive eigenvector — foundation for ranking, population models, and some Markov arguments.

10. CS applications map

Area	Spectral object
PCA / whitening	eigendecomposition of covariance
Graph ML	Laplacian eigenvectors
Stability of linear systems	$\rho(A)$, Reλ
Google-style ranking	dominant eigenvector of modified transition matrix
Vibration / circuits	modal analysis
Quantum / signal	Hermitian spectra
Optimization	Hessian eigenvalues (curvature, condition)

11. Pitfalls

1. Assuming real eigenvectors for real nonsymmetric A (rotations)
2. Using eigenvalue ratios as κ for highly non-normal A
3. Power method without a spectral gap — slow/no convergence
4. Confusing algebraic vs geometric multiplicity (defective matrices)
5. Interpreting every eigenvector of a data matrix as a "concept" without validation

Worked example 8 — rotation

90° rotation $R = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$ has no **real** eigenvectors (eigenvalues $\pm i$). Over $\mathbb{C}$, spectral picture exists; over $\mathbb{R}$, think invariant planes.

12. Checkpoint

- Define eigenpairs and spectral radius
- Explain A^k via diagonalization
- Connect PCA to covariance eigenvectors
- State Laplacian kernel and connectivity fact
- Condition number for SPD matrices
- Power method's target

Exercises

Easy

1. Find eigenpairs of $\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$.
2. Why can't a real 2×2 rotation by 90° have real eigenvectors?
3. If all $|\lambda| < 1$ for diagonalizable A, show $A^k \to 0$.
4. Relate trace to sum of eigenvalues.

5. Power method idea — iterate $x \leftarrow Ax/\|Ax\|$; what does it find?

Medium

6. For Laplacian of a path of 3 nodes, argue 0 is an eigenvalue.
7. Prove $x^\top Lx = \sum_{(i,j)\in E}(x_i - x_j)^2$ for undirected graphs.
8. Rayleigh: show $\lambda_{\max} = \max_{\|x\|=1} x^\top Ax$ for symmetric A (using spectral theorem).
9. If A symmetric with eigenvalues in $[m, M]$, bound $\|Ax\|$ for $\|x\| = 1$.
10. PageRank damping: why does adding teleportation help uniqueness/convergence?

Challenge

11. Jordan block $J = \begin{bmatrix}\lambda & 1\\ 0 & \lambda\end{bmatrix}$: compute J^k and discuss $|\lambda| < 1$ vs $\|J^k\|$.
12. Show multiplicity of $\lambda = 0$ of L equals number of connected components.
13. Non-normal transient growth: construct a 2×2 example with $\rho(A) < 1$ but $\|A^k\| > 1$ for some k.
14. Connect Hessian condition number at a minimum to GD step-size limits.
15. Spectral clustering: write the RatioCut / Laplacian relaxation idea at high level.

Checks

1. $\lambda = \pm 1$ with vectors $(1, 1)$, $(1, -1)$.
2. $\mathrm{tr}(A) = \sum \lambda_i$.
3. Dominant eigenvector (largest $|\lambda|$).

Summary

Spectral thinking asks: **what does this matrix stretch, shrink, or rotate?** Eigenvalues control long-term iteration, continuous dynamics, variance directions, graph cuts, and conditioning. Symmetric problems give clean orthogonal diagonalizations; nonsymmetric and non-normal cases need extra care. From PCA to PageRank to Laplacians, reading the spectrum is a practical superpower for scientific computing and ML systems.

Random Rotations, Johnson–Lindenstrauss, and Hadamard Transforms

High-dimensional space has two properties that look like bugs and turn out to be tools. Random directions are almost always nearly perpendicular. And a random linear map to far fewer dimensions barely changes any distance in a finite point set. The **Johnson–Lindenstrauss (JL) lemma** makes the second property exact: n points need only $O(\varepsilon^{-2} \log n)$ dimensions to keep every pairwise distance within a factor $1 \pm \varepsilon$, whatever the original dimension was.

The same facts explain a technique behind current low-bit quantization of neural networks and vector databases: **rotate before you quantize**. A random orthogonal rotation spreads a vector's energy evenly over its coordinates, so a coarse quantizer with one scale per vector stops wasting its levels on a few huge entries. Dense random rotations are expensive. The **randomized Hadamard transform** gets the same effect in $O(d \log d)$ time with ± 1 arithmetic.

This chapter proves the pieces, measures them, and points out where the guarantees end. Every listing is a complete script. They were run with Python 3.13.5, NumPy 2.5.3 and SciPy 1.18.1 on Debian 13.7, and the outputs are pasted unedited. Timings depend on the machine; everything else is seeded and reproducible.

Prerequisites: orthogonal matrices and norms, expectation and variance, a tail bound such as Chernoff or Hoeffding.

---

1. Concentration: random directions are nearly orthogonal

Let u, v be independent uniform unit vectors in $\mathbb{R}^d$. A convenient way to sample one is $g/\|g\|$ with $g \sim \mathcal{N}(0, I_d)$. By symmetry $\mathbb{E}\langle u, v\rangle = 0$. Fix v by rotation invariance; then $\langle u, v\rangle$ is one coordinate of u, and since the d squared coordinates of u sum to 1,

$$\mathbb{E}\,\langle u, v\rangle^2 = \frac{1}{d}, \qquad \text{so} \qquad \langle u, v\rangle \approx \mathcal{N}\big(0, \tfrac{1}{d}\big) \text{ for large } d.$$

The angle $\theta = \arccos\langle u, v\rangle$ therefore sits at $90°$ with spread about $(180/\pi)/\sqrt{d}$ degrees. The coordinate density is exactly $\propto (1-t^2)^{(d-3)/2}$ on $[-1, 1]$, and its tails are sub-Gaussian: $\Pr(|\langle u, v\rangle| > \varepsilon) \le 2e^{-d\varepsilon^2/2}$. A union bound then shows you can pick about $e^{d\varepsilon^2/4}$ unit vectors whose pairwise $|\cos|$ are all below ε, which is exponentially many "almost orthogonal" directions in only d dimensions.

Worked example 1 — angles between 1000 random vectors

```python
# file: angles.py
# Angles between independent random directions concentrate at 90 degrees as d grows.
import numpy as np

rng = np.random.default_rng(0)
n = 1000                                    # 1000 vectors -> 499,500 distinct pairs
print(f"{'d':>6} {'mean angle':>10} {'std':>7} {'min':>7} {'max':>7} {'1/sqrt(d) in deg':>17
for d in (3, 30, 300, 3000, 12288):
    X = rng.standard_normal((n, d))
    X /= np.linalg.norm(X, axis=1, keepdims=True)
    cos = (X @ X.T)[np.triu_indices(n, k=1)]
    ang = np.degrees(np.arccos(np.clip(cos, -1, 1)))
    print(f"{d:6d} {ang.mean():10.2f} {ang.std():7.2f} {ang.min():7.2f} {ang.max():7.2f} "
          f"{np.degrees(1 / np.sqrt(d)):17.2f}")
```

```
python angles.py
```

Output:

```
     d mean angle     std     min     max  1/sqrt(d) in deg
     3      89.86   39.15    0.02  179.95             33.08
    30      90.01   10.64   40.68  134.97             10.46
   300      90.01    3.31   73.61  105.26              3.31
  3000      90.00    1.05   85.28   94.77              1.05
 12288      90.00    0.52   87.72   92.41              0.52
```

The standard deviation tracks $1/\sqrt{d}$ (in degrees) almost exactly from $d = 30$ up. At $d = 12{,}288$ (the hidden width of a GPT-3-sized model), half a million pairs all fall between 87.7° and 92.4°. In $d = 3$ the same experiment covers the whole range from 0° to 180°. Low-dimensional intuition is the wrong guide here.

---

2. The Johnson–Lindenstrauss lemma

Lemma (JL; constants of Dasgupta–Gupta). Let $0 < \varepsilon < 1$ and

$$k \;\geq\; \frac{4\ln n}{\varepsilon^2/2 - \varepsilon^3/3}.$$

For every set of n points in $\mathbb{R}^D$ there is a linear map $f : \mathbb{R}^D \to \mathbb{R}^k$ with

$$(1-\varepsilon)\|u-v\|^2 \;\leq\; \|f(u)-f(v)\|^2 \;\leq\; (1+\varepsilon)\|u-v\|^2 \quad \text{for all pairs } u, v.$$

Two things stand out. k does **not** depend on D. And the map can be random and oblivious to the data.

Proof sketch. Take $f(x) = Rx$ with $R \in \mathbb{R}^{k \times D}$ having i.i.d. $\mathcal{N}(0, 1/k)$ entries. For a fixed vector x, each coordinate of Rx is $\mathcal{N}(0, \|x\|^2/k)$, independently, so

$$\frac{k\,\|Rx\|^2}{\|x\|^2} \sim \chi^2_k, \qquad \mathbb{E}\|Rx\|^2 = \|x\|^2, \qquad \operatorname{sd}\left(\frac{\|Rx\|^2}{\|x\|^2}\right) = \sqrt{2/k}.$$

A Chernoff bound on the χ^2 moment generating function gives

$$\Pr\left(\left|\frac{\|Rx\|^2}{\|x\|^2} - 1\right| > \varepsilon\right) \le 2\exp\left(-\frac{k}{2}\left(\frac{\varepsilon^2}{2} - \frac{\varepsilon^3}{3}\right)\right).$$

Apply this to the $\binom{n}{2}$ difference vectors $x = u - v$. With k as in the lemma, each pair fails with probability at most $2/n^2$. By the union bound all pairs succeed with probability at least $1/n > 0$, so a good map exists. Replacing $4\ln n$ by $6\ln n$ pushes the success probability of a *single* random draw to at least $1 - 1/n$.

Optimality. Larsen and Nelson (2017) proved that $k = \Omega(\varepsilon^{-2}\log n)$ is necessary for some point sets, even for nonlinear maps. The lemma cannot be improved beyond constants.

The rule of thumb to remember: a Gaussian projection to k dimensions distorts each squared distance by a relative error with standard deviation $\sqrt{2/k}$. The worst of N pairs is a few standard deviations out, roughly $\sqrt{2\ln N}$ of them.

Worked example 2 — what k does a real point set need?

```
# file: jl_distortion.py
# Project n points from D=10,000 dimensions down to k with a Gaussian matrix and
# measure the worst distortion of squared pairwise distances, next to the k that the
# Johnson-Lindenstrauss bound (Dasgupta-Gupta constants) asks for.
import numpy as np

rng = np.random.default_rng(1)
n, D = 300, 10_000
# Data with structure: a 20-dim signal subspace plus small isotropic noise.
X = rng.standard_normal((n, 20)) @ rng.standard_normal((20, D)) + 0.5 * rng.standard_normal(
iu = np.triu_indices(n, k=1)


def sq_dists(Y):
    g = np.sum(Y * Y, 1)
    return (g[:, None] + g[None, :] - 2 * Y @ Y.T)[iu]


def jl_k(n, eps):
```

```
    return int(np.ceil(4 * np.log(n) / (eps**2 / 2 - eps**3 / 3)))


d0 = sq_dists(X)
print(f"n={n} points, D={D}, {len(d0):,} pairs")
print(f"{'k':>5} {'max |ratio-1|':>13} {'99.9th pct':>10} {'eps where JL k = this k':>23}")
for k in (25, 50, 100, 200, 400, 800, 1600):
    R = rng.standard_normal((D, k)) / np.sqrt(k)
    ratio = sq_dists(X @ R) / d0
    dev = np.abs(ratio - 1)
    eps = next((f"{e:.3f}" for e in np.arange(0.01, 1, 0.001) if jl_k(n, e) <= k), "none (ep
    print(f"{k:5d} {dev.max():13.3f} {np.quantile(dev, 0.999):10.3f} {eps:>23}")
print(f"JL k for eps=0.1: {jl_k(n, 0.1):,}   eps=0.2: {jl_k(n, 0.2):,}   eps=0.5: {jl_k(n, 0
```

```
python jl_distortion.py
```

Output:

```
n=300 points, D=10000, 44,850 pairs
    k max |ratio-1| 99.9th pct eps where JL k = this k
   25         1.581      1.169            none (eps<1)
   50         0.997      0.734            none (eps<1)
  100         0.590      0.423            none (eps<1)
  200         0.428      0.340                   0.626
  400         0.357      0.245                   0.394
  800         0.206      0.163                   0.263
 1600         0.132      0.110                   0.181
JL k for eps=0.1: 4,889   eps=0.2: 1,317   eps=0.5: 274
```

Reading the table:

- The worst-case distortion falls like $1/\sqrt{k}$: from 0.59 at $k = 100$ to 0.13 at $k = 1600$, a factor of 4.5 for a 16-fold increase in k.
- The rule of thumb is accurate. At $k = 400$, $\sqrt{2/k} = 0.071$ and $\sqrt{2 \ln 44850} \approx 4.6$, predicting a worst case near 0.33. The measured worst case is 0.357.
- The lemma's guarantee is not wildly pessimistic. At $k = 400$ it guarantees $\varepsilon = 0.394$; one draw achieved 0.357. Below $k = 24 \ln n \approx 137$ the bound promises nothing at all, because $\varepsilon^2/2 - \varepsilon^3/3 \leq 1/6$ on $(0, 1)$.
- Tight guarantees are expensive. $\varepsilon = 0.1$ for 300 points needs $k = 4889$, about half of D. JL pays off when D is huge or when you only need ε around 0.2 to 0.5.

---

3. Hadamard matrices and the fast transform

A **Hadamard matrix** of order n has entries ± 1 and satisfies $HH^\top = nI$, so its rows are mutually orthogonal. Sylvester's construction doubles the order:

$$H_1 = [1], \qquad H_{2n} = \begin{bmatrix} H_n & H_n \\ H_n & -H_n \end{bmatrix}.$$

Properties of the Sylvester family, for $n = 2^m$:

- $\frac{1}{\sqrt{n}} H$ is orthogonal **and** symmetric, so it is its own inverse.
- Row i is a Walsh function: entry (i, j) is $(-1)^{\langle i, j \rangle}$, where $\langle i, j \rangle$ counts the 1 bits shared by the binary expansions of i and j.
- The recursion gives a butterfly algorithm, the **fast Walsh–Hadamard transform (FWHT)**: $\log_2 n$ passes of $n/2$ add/subtract pairs, $n \log_2 n$ additions and no multiplications until the final $1/\sqrt{n}$ scale. For $n = 4096$ that is 49,152 additions, against 16.8 million multiply-adds for a dense $n \times n$ product.

Hadamard matrices exist only for $n = 1, 2$ and multiples of 4. That every multiple of 4 works is the open Hadamard conjecture; the smallest unresolved order is 668. Sylvester's construction gives only powers of two. Other sizes use Paley constructions or Kronecker products such as $H_{12} \otimes H_{2^m}$, or **block-diagonal** Hadamards on power-of-two blocks, or zero-padding.

Worked example 3 — build it, check it, transform fast

```
# file: hadamard.py
# Sylvester Hadamard matrices and the fast Walsh-Hadamard transform (FWHT).
import numpy as np


def sylvester(m):
    """H_{2^m} with entries +-1, built by H_{2n} = [[H, H], [H, -H]]."""
    H = np.array([[1.0]])
    for _ in range(m):
        H = np.block([[H, H], [H, -H]])
    return H


def fwht(x):
    """Orthonormal transform (H / sqrt(n)) x along the last axis in O(n log n)."""
    x = np.array(x, dtype=np.float64, copy=True)
    shape, n = x.shape, x.shape[-1]
    assert n & (n - 1) == 0, "length must be a power of two"
    x = x.reshape(-1, n)
    h = 1
    while h < n:
```

```
        x = x.reshape(-1, n // (2 * h), 2, h)
        a = x[:, :, 0, :].copy()
        b = x[:, :, 1, :]
        x[:, :, 0, :] = a + b
        x[:, :, 1, :] = a - b
        x = x.reshape(-1, n)
        h *= 2
    return x.reshape(shape) / np.sqrt(n)


if __name__ == "__main__":
    H4 = sylvester(2)
    print(H4.astype(int))
    H = sylvester(10) / np.sqrt(1024)
    x = np.random.default_rng(0).standard_normal((5, 1024))
    print("orthogonal:      ", np.allclose(H @ H.T, np.eye(1024)))
    print("symmetric:       ", np.allclose(H, H.T))
    print("fwht == H @ x:   ", np.allclose(fwht(x), x @ H.T))
    print("fwht(fwht(x))==x:", np.allclose(fwht(fwht(x)), x))
```

```
python hadamard.py
```

Output:

```
[[ 1  1  1  1]
 [ 1 -1  1 -1]
 [ 1  1 -1 -1]
 [ 1 -1 -1  1]]
orthogonal:       True
symmetric:        True
fwht == H @ x:    True
fwht(fwht(x))==x: True
```

---

4. Cheaper JL maps: sparse signs and the subsampled randomized Hadamard transform

A dense Gaussian R costs kD random numbers and $O(kD)$ time per vector. Two lines of work cut that down.

- **Achlioptas (2003):** entries $\sqrt{3/k} \cdot \{+1, 0, -1\}$ with probabilities $\{1/6, 2/3, 1/6\}$ satisfy the same bound. Two thirds of the matrix is zero, and the arithmetic is additions only.
- **Ailon–Chazelle (2006), the fast JL transform:** first apply HD, where D is a random ± 1 diagonal and H the normalized Hadamard matrix, and then a sparse or sampling projection.

The popular **SRHT** variant keeps k random coordinates of HDx and rescales by $\sqrt{D/k}$ (in the scale factor D is the ambient dimension, not the sign matrix). It costs $O(D \log D)$ per vector and stores only D signs plus k indices.

Why not just sample k coordinates of x? Because sampling estimates $\|x\|^2$ well only when the energy is spread out. A vector with all of its energy in a few coordinates is either missed or overcounted by a factor of D/k. The HD step exists to spread the energy first (Section 5).

Worked example 4 — four projections to k = 512

```
# file: jl_variants.py
# Three ways to project 300 points from D=8192 to k=512: dense Gaussian, sparse signs
# (Achlioptas), and the subsampled randomized Hadamard transform (SRHT).
import time

import numpy as np

from hadamard import fwht

rng = np.random.default_rng(2)
n, D, k = 300, 8192, 512
X = rng.standard_normal((n, 16)) @ rng.standard_normal((16, D))
X[:, :8] *= 40.0                                        # a few heavy coordinates
iu = np.triu_indices(n, k=1)


def sq_dists(Y):
    g = np.sum(Y * Y, 1)
    return (g[:, None] + g[None, :] - 2 * Y @ Y.T)[iu]


G = rng.standard_normal((D, k)) / np.sqrt(k)
S = rng.choice([-1.0, 0.0, 1.0], size=(D, k), p=[1 / 6, 2 / 3, 1 / 6]) * np.sqrt(3 / k)
signs = rng.choice([-1.0, 1.0], size=D)
rows = rng.choice(D, size=k, replace=False)
projections = {
    "Gaussian": lambda Z: Z @ G,
    "sparse signs": lambda Z: Z @ S,
    "SRHT": lambda Z: fwht(Z * signs)[:, rows] * np.sqrt(D / k),
    "subsample, no HD": lambda Z: Z[:, rows] * np.sqrt(D / k),
}
d0 = sq_dists(X)
print(f"{'projection':17} {'max |ratio-1|':>13} {'mean |ratio-1|':>14} {'storage':>16} {'ms'
for name, f in projections.items():
    t = time.perf_counter()
    Y = f(X)
```

```
    ms = 1e3 * (time.perf_counter() - t)
    dev = np.abs(sq_dists(Y) / d0 - 1)
    store = {"Gaussian": f"{D * k:,} floats", "sparse signs": f"{np.count_nonzero(S):,} nz",
             "SRHT": f"{D + k:,} ints", "subsample, no HD": f"{k:,} ints"}[name]
    print(f"{name:17} {dev.max():13.3f} {dev.mean():14.3f} {store:>16} {ms:6.1f}")
```

```
python jl_variants.py
```

Output:

```
projection        max |ratio-1| mean |ratio-1|          storage     ms
Gaussian                  0.238          0.050 4,194,304 floats   25.3
sparse signs              0.314          0.045     1,397,248 nz   15.0
SRHT                      0.185          0.045       8,704 ints  176.1
subsample, no HD          9.656          1.805         512 ints    2.1
```

- All three JL constructions give mean distortion about 0.045 to 0.05, which matches $\sqrt{2/512} \cdot \sqrt{2/\pi} \approx 0.050$. The worst cases differ by draw and are all in the 0.2 to 0.3 range.
- Plain coordinate sampling fails badly: worst ratio off by 9.7x, because the eight heavy coordinates are sampled or missed at random.
- The timing column is a warning about asymptotics. On this machine the dense Gaussian product (BLAS, multithreaded) beats the NumPy-level FWHT, whose butterfly passes allocate temporaries. The $O(D \log D)$ advantage appears with a compiled or GPU FWHT, or when k is close to D so that a dense map would be $D \times D$. The storage column holds regardless: 8,704 small integers against 4.2 million floats.

---

5. Why random signs: the spreading lemma

Let x be a fixed unit vector and $y = \frac{1}{\sqrt{d}} HDx$ with i.i.d. random signs D_{jj}. Each coordinate

$$y_i = \frac{1}{\sqrt{d}} \sum_j H_{ij} D_{jj} x_j$$

is a sum of independent, bounded, mean-zero terms with $\sum_j (H_{ij} x_j / \sqrt{d})^2 = 1/d$. Hoeffding's inequality gives $\Pr(|y_i| > t) \le 2e^{-t^2 d/2}$, and a union bound over the d coordinates gives

$$\max_i |y_i| \;\le\; \sqrt{\frac{2\ln(2d/\delta)}{d}} \quad \text{with probability at least } 1 - \delta.$$

A perfectly flat unit vector has every entry equal to $1/\sqrt{d}$, so the randomized Hadamard transform leaves every vector within a $\sqrt{2\ln(2d/\delta)}$ factor of flat, simultaneously and with high probability. That is what makes the SRHT work, and it is what Section 6 uses.

The signs are essential. Without them Hx is deterministic, so some inputs map to spikes. The rows of H are such inputs: H maps a Walsh function to a one-hot vector, the worst possible output.

Worked example 5 — peaks before and after

```
# file: spread.py
# How evenly does y = (1/sqrt(d)) H D x spread a vector's energy? Compare the largest
# coordinate with the Hoeffding bound sqrt(2 ln(2d/delta) / d) for a unit vector x.
import numpy as np
from scipy.linalg import hadamard

from hadamard import fwht

rng = np.random.default_rng(3)
d, delta, trials = 1024, 0.01, 2000
bound = np.sqrt(2 * np.log(2 * d / delta) / d)
H = hadamard(d).astype(float)

inputs = {
    "one-hot e_0": np.eye(d)[0],
    "4 equal spikes": np.r_[np.full(4, 0.5), np.zeros(d - 4)],
    "Gaussian": rng.standard_normal(d),
    "Walsh row 37": H[37] / np.sqrt(d),
}
print(f"d={d}: perfectly flat = {1 / np.sqrt(d):.4f}, Hoeffding bound (delta={delta}) = {bou
print(f"{'input (unit norm)':18} {'max|x|':>7} {'max|Hx|':>8} {'max|HDx| median':>15} "
      f"{'worst of 2000':>13} {'P(> bound)':>10}")
for name, x in inputs.items():
    x = x / np.linalg.norm(x)
    D = rng.choice([-1.0, 1.0], size=(trials, d))
    peaks = np.abs(fwht(D * x)).max(axis=1)
    print(f"{name:18} {np.abs(x).max():7.4f} {np.abs(fwht(x)).max():8.4f} {np.median(peaks):
          f"{peaks.max():13.4f} {np.mean(peaks > bound):10.4f}")
```

```
python spread.py
```

Output:

```
d=1024: perfectly flat = 0.0312, Hoeffding bound (delta=0.01) = 0.1546
input (unit norm)   max|x|  max|Hx| max|HDx| median worst of 2000 P(> bound)
one-hot e_0         1.0000   0.0312          0.0312        0.0312     0.0000
4 equal spikes      0.5000   0.0625          0.0625        0.0625     0.0000
Gaussian            0.1036   0.1011          0.1063        0.1545     0.0000
Walsh row 37        0.0312   1.0000          0.1055        0.1504     0.0000
```

- A one-hot or few-spike input becomes **exactly** flat (or nearly so) under H alone, and under HD too. Hadamard rows have entries of constant magnitude.
- A Walsh row is the adversarial case: Hx has a peak of 1.0, the whole vector in one coordinate. With random signs the median peak is 0.106, in line with a random input.
- The Hoeffding bound (0.155) was never exceeded in 2000 sign draws for any input. The worst draws came within 3% of it, so the bound is not slack.

---

6. Rotate, then quantize

A **uniform quantizer** with b bits and one scale per vector maps each entry to the nearest multiple of $s = \max_i |x_i| / q_{\max}$, with $q_{\max} = 2^{b-1} - 1$. When most entries span several steps, the rounding error behaves like uniform noise of variance $s^2/12$, so

$$\frac{\|x - \hat{x}\|}{\|x\|} \approx \frac{\kappa(x)}{q_{\max}\sqrt{12}}, \qquad \kappa(x) = \frac{\max_i |x_i|}{\mathrm{rms}(x)} \quad \text{(crest factor)}.$$

The error is driven entirely by the crest factor, and an orthogonal rotation changes the crest factor without changing the vector's length or its inner products. If Q is orthogonal, then $\langle Qx, Qw \rangle = \langle x, w \rangle$. A layer computing Wx can therefore store $\mathrm{quant}(Qx)$ and use the weights $WQ^\top$, folded in offline. Nothing needs to be un-rotated at run time. This **computational invariance** is how QuaRot, QuIP# and SpinQuant quantize LLM weights and activations to 4 bits. Rotation-based vector quantizers for embeddings and attention caches (DRIVE, EDEN, RaBitQ, HIGGS, TurboQuant) build on the same idea, and their codebooks rely on Section 1: after a random rotation, every coordinate of a unit vector is approximately $\mathcal{N}(0, 1/d)$.

Worked example 6 — 4-bit quantization with outliers

```
# file: rotate_quantize.py
# Uniform 4-bit quantization with one scale per vector, before and after a shared
# randomized Hadamard rotation. Inner products survive if both sides use the same rotation.
import numpy as np

from hadamard import fwht

rng = np.random.default_rng(4)
n, d, bits = 2000, 256, 4
X = rng.standard_normal((n, d))
X[:, [5, 77, 200]] *= 30.0                          # outlier coordinates, as in real acti
W = rng.standard_normal((64, d)) / np.sqrt(d)       # 64 fixed "weight" rows
signs = rng.choice([-1.0, 1.0], size=d)
rot = lambda Z: fwht(Z * signs)                     # Q z with Q = (1/sqrt d) H D, orthogo
unrot = lambda Z: fwht(Z) * signs                   # Q^T z
```

```
def quantize(Z, bits):
    qmax = 2 ** (bits - 1) - 1
    s = np.abs(Z).max(axis=1, keepdims=True) / qmax
    return np.clip(np.round(Z / s), -qmax - 1, qmax) * s


crest = lambda Z: np.mean(np.abs(Z).max(1) / np.sqrt(np.mean(Z**2, 1)))
rel = lambda A, B: np.linalg.norm(A - B) / np.linalg.norm(B)
Y = X @ W.T                                          # exact products
print(f"crest factor max|x|/rms(x): original {crest(X):.1f}, rotated {crest(rot(X)):.1f}, "
      f"i.i.d. Gaussian {crest(rng.standard_normal((n, d))):.1f}")
print(f"{'scheme':40} {'rel. error in x':>15} {'rel. error in Wx':>16}")
Xq = quantize(X, bits)
print(f"{'quantize x directly':40} {rel(Xq, X):15.4f} {rel(Xq @ W.T, Y):16.4f}")
Xr_q = quantize(rot(X), bits)                        # store this
print(f"{'rotate, quantize, unrotate':40} {rel(unrot(Xr_q), X):15.4f} {rel(unrot(Xr_q) @ W.T
Wr = rot(W)                                          # fold Q into the weights once: W Q^T
print(f"{'rotated x against rotated W (no unrotate)':40} {'':>15} {rel(Xr_q @ Wr.T, Y):16.4f
print(f"{'BUG: rotated x against original W':40} {'':>15} {rel(Xr_q @ W.T, Y):16.4f}")
```

```
python rotate_quantize.py
```

Output:

```
crest factor max|x|/rms(x): original 12.1, rotated 2.3, i.i.d. Gaussian 3.0
scheme                                   rel. error in x rel. error in Wx
quantize x directly                               0.2745           0.2605
rotate, quantize, unrotate                        0.0867           0.0824
rotated x against rotated W (no unrotate)                           0.0824
BUG: rotated x against original W                                   1.4390
```

- Three coordinates at 30 times the scale give a crest factor of 12.1. With a 4-bit absmax quantizer the step is so large that every ordinary coordinate rounds to zero, and the error (27%) is simply the energy those coordinates carried.
- After the rotation the crest factor is 2.3, and the error drops to 8.7%. The formula predicts $2.3/(7\sqrt{12}) = 0.095$. The rotated vectors are even flatter than i.i.d. Gaussian ones (crest 3.0): a few large spikes rotate into $\pm a \pm b \pm c$ patterns of nearly constant magnitude.
- Rotating the weights instead of un-rotating the activations gives the same Wx error (0.0824), confirming $\langle Qx, Qw \rangle = \langle x, w \rangle$.
- Forgetting to rotate the other side produces garbage (144% error). The mistake is silent: shapes match and nothing crashes.

---

7. The trap and the boring rule

The trap: treating a rotation as a free accuracy gain without checking its two side conditions.

1. **Consistency.** Every operand that meets the rotated vector in an inner product must see the same Q: same signs, same Hadamard ordering, same block size. A mismatched seed or a different FWHT ordering (natural versus sequency) is the last line of the table above.
2. **Randomness.** A deterministic Hadamard has adversarial inputs (Walsh-like vectors). Real data rarely looks like that, but "rarely" is an empirical claim about your data, not a theorem. The random sign vector costs d bits and turns it into a theorem.

The boring rule: rotate with $\frac{1}{\sqrt{d}}HD$ from a recorded seed, fold the rotation into whatever sits on the other side of the inner product, and measure the quantity you care about (inner products, distances, model loss), not just the quantization MSE.

---

8. Pitfalls

Pitfall	Symptom	Fix
Expecting JL to help at small D	k from the bound exceeds D	JL pays off for $D \gg \varepsilon^{-2} \log n$
Using the existence bound as a per-draw guarantee	Occasional bad projection	Use $6 \ln n$ in place of $4 \ln n$, or check distortion and redraw
Sampling coordinates without mixing first	Huge errors on spiky vectors	Apply HD (or a Gaussian map) before sampling
Plain H without random signs	Rare inputs map to spikes	Use HD with a stored seed
d not a power of two	Sylvester H undefined	Pad, block-diagonal Hadamard, or a Kronecker product with a known H_{12}, H_{20}, …
Rotating only one operand	Silent garbage in Wx	Fold Q into the other side: $WQ^\top$
Forgetting $1/\sqrt{d}$	Norms scale by $\sqrt{d}$	Use the orthonormal transform; check $\|Qx\| = \|x\|$
Trusting big-O for speed	FWHT slower than BLAS in NumPy	Benchmark the actual implementation

---

9. Checkpoint

- Show $\mathbb{E}\langle u, v\rangle^2 = 1/d$ for independent random unit vectors.
- State the JL lemma, including what k does and does not depend on.

- Explain where the union bound enters the proof and why it gives $\log n$.
- Give the cost of the FWHT and why $\frac{1}{\sqrt{n}}H$ is its own inverse.
- Derive $\max_i |y_i| \lesssim \sqrt{2\ln(2d/\delta)/d}$ for $y = \frac{1}{\sqrt{d}}HDx$.
- Explain why a rotation lowers uniform-quantization error and why it does not change Wx when folded into W.

Exercises

Easy

1. For $d = 768$, what is the standard deviation of the angle between two random directions, in degrees?
2. Compute the lemma's k for $n = 10^6$ points and $\varepsilon = 0.2$. Does it depend on whether the points live in $\mathbb{R}^{1000}$ or $\mathbb{R}^{10^9}$?
3. Write H_8 by Sylvester's construction and check that rows 3 and 5 are orthogonal.

Medium

4. Show that $\mathbb{E}\|Rx\|^2 = \|x\|^2$ and $\mathrm{Var}(\|Rx\|^2) = 2\|x\|^4/k$ for R with i.i.d. $\mathcal{N}(0, 1/k)$ entries.
5. Count the additions in an FWHT of length 2^m, and compare with a dense matrix-vector product at $m = 12$.
6. Modify `spread.py` to use a random permutation matrix P in place of D. Does HPx still flatten Walsh rows? Explain.
7. In `rotate_quantize.py`, sweep `bits` over 2 to 8 and plot both errors. At what bit-width does rotation stop mattering, and why?

Challenge

8. Prove the "exponentially many nearly orthogonal vectors" claim: using $\Pr(|\langle u, v\rangle| > \varepsilon) \le 2e^{-d\varepsilon^2/2}$, show that $m = \lfloor e^{d\varepsilon^2/4}/2 \rfloor$ random unit vectors are pairwise ε-orthogonal with positive probability.
9. Implement a block-diagonal randomized Hadamard for $d = 96$ (three blocks of 32). How does its worst-case peak compare with a full 128-point transform on the zero-padded vector?
10. Replace the uniform quantizer in `rotate_quantize.py` with a Lloyd–Max quantizer designed for $\mathcal{N}(0, 1/d)$ coordinates, applied to $Qx/\|x\|$ with $\|x\|$ stored separately. Compare the error at 2, 3 and 4 bits.

Summary

Independent random directions in $\mathbb{R}^d$ have inner products of size $1/\sqrt{d}$, so high-dimensional space is full of nearly orthogonal directions. The Johnson–Lindenstrauss lemma turns this into an algorithm: a random linear map to $k = O(\varepsilon^{-2} \log n)$ dimensions preserves all pairwise distances of n points within $1 \pm \varepsilon$, independent of the original dimension, and this is optimal. The per-pair relative error has standard deviation $\sqrt{2/k}$, which predicts measured worst cases well. Hadamard matrices give orthogonal transforms computable in $n \log n$ additions. With random signs in front, the randomized Hadamard transform spreads every vector's energy to within $\sqrt{2 \ln(2d/\delta)}$ of flat, which makes coordinate sampling (SRHT) and coarse scalar quantization work. Rotation preserves inner products, so it can be folded into the other operand for free. It is only safe when both sides use the same rotation and the rotation includes the random signs.

Sources (for further reading)

- W. B. Johnson, J. Lindenstrauss, "Extensions of Lipschitz mappings into a Hilbert space", *Contemporary Mathematics* 26 (1984).
- S. Dasgupta, A. Gupta, "An elementary proof of a theorem of Johnson and Lindenstrauss", *Random Structures & Algorithms* 22(1) (2003). Constants used in Section 2.
- D. Achlioptas, "Database-friendly random projections: Johnson–Lindenstrauss with binary coins", *JCSS* 66(4) (2003).
- N. Ailon, B. Chazelle, "Approximate nearest neighbors and the fast Johnson–Lindenstrauss transform", STOC 2006; *SIAM J. Computing* 39(1) (2009).
- J. A. Tropp, "Improved analysis of the subsampled randomized Hadamard transform", *Advances in Adaptive Data Analysis* 3 (2011).
- K. G. Larsen, J. Nelson, "Optimality of the Johnson–Lindenstrauss lemma", FOCS 2017.
- R. Vershynin, *High-Dimensional Probability* (Cambridge, 2018), chapters 2–3 (Hoeffding, concentration on the sphere).
- A. Baggag, Y. Saad, "The Numerical Linear Algebra of Large Language Models" (arXiv:2610.04631, 2026), §3.14 (near-orthogonality in 12,288 dimensions) and §4.4 (randomized methods, Linformer).
- S. Ashkboos et al., "QuaRot: Outlier-Free 4-Bit Inference in Rotated LLMs" (NeurIPS 2024). A. Tseng et al., "QuIP#" (ICML 2024). Z. Liu et al., "SpinQuant" (ICLR 2025).
- S. Vargaftik et al., DRIVE (NeurIPS 2021) and EDEN (ICML 2022). J. Gao, C. Long, "RaBitQ" (SIGMOD 2024). A. Zandieh et al., "TurboQuant" (arXiv:2504.19874, ICLR 2026).

Inspiration (not a source): a post on X by Omri Weinstein on the Hadamard transform behind low-bit quantization, https://x.com/WeinsteinOmri/status/2070482412908277797. The mathematics above follows the papers and texts listed.

The Linear Algebra Inside a Large Language Model

A large language model is mostly matrix multiplication, and the interesting questions about it are linear-algebra questions. Where do the parameters sit? What is the rank of an attention matrix, and why does it cost n^2? How can a product be re-associated to make attention linear in sequence length? Why can a rank-16 update fine-tune a model with billions of parameters? What does it mean to precondition a gradient that is itself a matrix?

This chapter answers each question with a short derivation and a script that checks it. The organizing reference is Baggag and Saad's 2026 survey *The Numerical Linear Algebra of Large Language Models*. Where the survey and a primary source disagree, the chapter goes with the primary source and says why. The listings were run with Python 3.13.5, NumPy 2.5.3 and SciPy 1.18.1 on Debian 13.7, and the outputs are pasted unedited. Timings depend on the machine; everything else is seeded.

Prerequisites: matrix products and rank, the SVD and Eckart–Young, eigendecomposition of symmetric matrices, gradient descent.

---

1. Where the parameters are

A decoder-only transformer with model width d is a stack of L identical blocks plus an embedding table and an output projection (the "LM head"). For a Llama-3-style block:

Matrix	Shape	Count
W_Q, W_O	$d \times d$	$2d^2$
W_K, W_V (grouped-query attention, h_{kv} heads of size d_h)	$d \times h_{kv} d_h$	$2d\, h_{kv} d_h$
SwiGLU MLP: gate, up, down	$d \times d_{ff}$ ($\times 3$)	$3d\, d_{ff}$
Two RMSNorm gain vectors	d ($\times 2$)	$2d$

Add $2Vd$ for an untied embedding and LM head (V = vocabulary size) and d for the final norm. There are no biases anywhere, and RMSNorm has a gain but no shift. The formula is easy to apply carelessly, so check it against the real checkpoints.

Worked example 1 — count Llama 3 exactly

```
# file: llm_params.py
# Count the parameters of a Llama-3-style decoder from its config, matrix by matrix,
# and compare with the totals Hugging Face reports for the released checkpoints.
configs = {  # values from each model's config.json
    "Llama-3-8B":     dict(d=4096,  d_ff=14336, layers=32,  heads=32,  kv_heads=8, vocab=128
    "Llama-3.1-405B": dict(d=16384, d_ff=53248, layers=126, heads=128, kv_heads=8, vocab=128
}
published = {"Llama-3-8B": 8_030_261_248, "Llama-3.1-405B": 405_853_388_800}  # safetensors

for name, c in configs.items():
    d, f, L = c["d"], c["d_ff"], c["layers"]
    dh = d // c["heads"]
    attn = d * d + 2 * d * (c["kv_heads"] * dh) + d * d      # W_Q, W_K, W_V (GQA), W_O; no
    mlp = 3 * d * f                                          # gate, up, down (SwiGLU); no b
    norms = 2 * d                                            # two RMSNorm gains, no beta
    emb = 2 * c["vocab"] * d                                 # untied input embedding and LM
    total = L * (attn + mlp + norms) + d + emb               # + final RMSNorm
    print(f"{name}: d={d}, d_ff={f}, layers={L}, head_dim={dh}")
    for part, n in (("attention", L * attn), ("MLP", L * mlp), ("embeddings", emb),
                    ("norms", L * norms + d)):
        print(f"  {part:11} {n:>16,}  {100 * n / total:5.1f}%")
    print(f"  {'total':11} {total:>16,}  published {published[name]:,}  "
          f"{'MATCH' if total == published[name] else 'DIFF ' + format(total - published[nam
    # Counted with GPT-style habits: MLP biases, LayerNorm gamma AND beta, no final norm.
    gpt_style = L * (attn + mlp + (2 * f + d) + 2 * (2 * d)) + emb
    print(f"  GPT-style count (biases, betas, no final norm): {gpt_style:,} "
          f"(+{gpt_style - total:,})")
```

```
python llm_params.py
```

Output:

```
Llama-3-8B: d=4096, d_ff=14336, layers=32, head_dim=128
  attention      1,342,177,280   16.7%
  MLP            5,637,144,576   70.2%
  embeddings     1,050,673,152   13.1%
  norms                266,240    0.0%
  total          8,030,261,248  published 8,030,261,248  MATCH
  GPT-style count (biases, betas, no final norm): 8,031,567,872 (+1,306,624)
Llama-3.1-405B: d=16384, d_ff=53248, layers=126, head_dim=128
  attention     71,873,593,344   17.7%
  MLP          329,772,957,696   81.3%
  embeddings     4,202,692,608    1.0%
  norms              4,145,152    0.0%
```

```
total        405,853,388,800  published 405,853,388,800  MATCH
GPT-style count (biases, betas, no final norm): 405,872,984,064 (+19,595,264)
```

- The formula reproduces both published totals **to the parameter**. Those totals are the `safetensors` metadata on the Hugging Face model pages: 8,030,261,248 and 405,853,388,800.
- The MLP holds 70% of the 8B model and 81% of the 405B model. Attention is under a fifth. Grouped-query attention is part of the reason: with 8 KV heads, W_K and W_V are each only $d \times 1024$.
- The embeddings matter only for small models (13% at 8B, 1% at 405B), because they scale like Vd while the blocks scale like Ld^2.

The trap: carrying over habits from a different block. GPT-2 and GPT-3 blocks have biases and LayerNorm with both γ and β; Llama blocks have neither. Counting them anyway gives the last line of each block above. For the 405B model that comes to exactly **405,872,984,064**, which is the total in Table 4 of the Baggag–Saad survey, 19.6 million (0.005%) above the real checkpoint. The error is negligible in size but not in kind. A parameter count is a statement about the architecture, and a count that does not match the checkpoint describes a different model.

---

2. Attention as a matrix

For one head with n tokens, $Q = XW_Q$, $K = XW_K$, $V = XW_V \in \mathbb{R}^{n \times d_h}$, and

$$S = \frac{QK^\top}{\sqrt{d_h}}, \qquad P = \mathrm{softmax}_{\mathrm{row}}(S), \qquad \text{output} = PV.$$

Three linear-algebra facts:

1. **The scores are low rank.** $\mathrm{rank}(S) \le \mathrm{rank}(Q) \le d_h$, typically 64 or 128 while n runs to many thousands. Bhojanapalli et al. (2020) argue that this bounds what one head can express and that splitting d across many small heads makes it worse.
2. **P is row-stochastic.** Its entries are positive and each row sums to 1. Each output row is a **convex combination** of the rows of V: attention averages, it does not extrapolate.
3. **The cost is quadratic.** Forming S takes $2n^2 d_h$ flops and n^2 memory per head. The softmax needs a whole row of S before it can normalize, so the $n \times n$ intermediate cannot simply be re-associated away.

Fact 1 is often stated about P instead of S. That is wrong.

Worked example 2 — the rank of S and of P

```
# file: attention_rank.py
# The score matrix QK^T has rank <= d_head. The attention matrix softmax(QK^T/sqrt(d))
# does not: the elementwise exponential destroys the low-rank structure.
import numpy as np

rng = np.random.default_rng(0)
n, d_model, d_head = 512, 256, 32
X = rng.standard_normal((n, d_model))
Wq, Wk, Wv = (rng.standard_normal((d_model, d_head)) / np.sqrt(d_model) for _ in range(3))
Q, K, V = X @ Wq, X @ Wk, X @ Wv

for scale in (0.1, 1.0, 4.0):                              # temperature: flat -> peaked attent
    S = scale * Q @ K.T / np.sqrt(d_head)
    P = np.exp(S - S.max(1, keepdims=True))
    P /= P.sum(1, keepdims=True)
    sv = np.linalg.svd(P, compute_uv=False)
    num_rank = int(np.sum(sv > 1e-10 * sv[0]))
    energy = np.cumsum(sv**2) / np.sum(sv**2)
    r99 = int(np.searchsorted(energy, 0.99) + 1)
    print(f"scale {scale:3}: rank(S) = {np.linalg.matrix_rank(S):3d}   rank(P) = {num_rank:3
          f"99% energy in {r99:3d} singular values   rows sum to 1: {np.allclose(P.sum(1), 1

out = P @ V                                                 # each output row: a convex mix of
print("every output row inside the box of V's rows:",
      bool(np.all(out >= V.min(0) - 1e-12) and np.all(out <= V.max(0) + 1e-12)))
```

```
python attention_rank.py
```

Output:

```
scale 0.1: rank(S) =  32   rank(P) = 512   99% energy in   2 singular values   rows sum to 1
scale 1.0: rank(S) =  32   rank(P) = 512   99% energy in 242 singular values   rows sum to 1
scale 4.0: rank(S) =  32   rank(P) = 512   99% energy in 247 singular values   rows sum to 1
every output row inside the box of V's rows: True
```

S has rank exactly $32 = d_h$. $P = \mathrm{softmax}(S)$ has full rank 512 at every temperature, because the elementwise exponential of a low-rank matrix is generically full rank. What changes with temperature is the *numerical* rank. Nearly flat attention (scale 0.1) is close to the rank-one matrix $\frac{1}{n}\mathbf{1}\mathbf{1}^\top$, and 2 singular values hold 99% of the energy. Sharper attention spreads energy over about 240 directions. The low-rank bottleneck is on the scores; the softmax is the nonlinearity that lets attention escape it. The last line confirms fact 2: every output coordinate lies between the minimum and maximum of the corresponding column of V.

---

3. Re-association and linear attention

Without the softmax, attention would be a product of three matrices, and matrix products are associative:

$$(QK^\top)V = Q(K^\top V), \qquad \underbrace{2n^2 d}_{\text{left}} \quad \text{vs} \quad \underbrace{4nd^2}_{\text{right}}.$$

The right-hand order never forms an $n \times n$ matrix, and it is cheaper whenever $n > 2d$. **Kernelized attention** keeps this structure by replacing $\exp(q \cdot k)$ with a factorizable similarity $\phi(q) \cdot \phi(k)$ for a positive feature map ϕ. Katharopoulos et al. (2020) use $\phi(x) = \mathrm{elu}(x) + 1$:

$$\mathrm{out}_i = \frac{\phi(q_i)^\top \sum_j \phi(k_j) v_j^\top}{\phi(q_i)^\top \sum_j \phi(k_j)}.$$

With a causal mask the sums run over $j \le i$, so they can be carried forward as a $d \times d_v$ state $S_i = S_{i-1} + \phi(k_i) v_i^\top$ and a d-vector $z_i = z_{i-1} + \phi(k_i)$. Causal linear attention is literally a recurrent network. Generation then costs $O(d^2)$ per token and keeps no KV cache. Shen et al.'s efficient attention takes a related route: a row softmax on Q and a column softmax on K.

Worked example 3 — three forms, one operator

```
# file: linear_attention.py
# Kernelized ("linear") attention: phi(Q) (phi(K)^T V) costs O(n d^2) instead of O(n^2 d),
# and its causal form is a recurrence with a d x d state. It is a different operator from
# softmax attention, not an approximation of it.
import time

import numpy as np

rng = np.random.default_rng(1)
phi = lambda x: np.where(x > 0, x + 1.0, np.exp(x))          # elu(x) + 1 > 0


def softmax_attn(Q, K, V, causal):
    S = Q @ K.T / np.sqrt(Q.shape[1])
    if causal:
        S = np.where(np.tril(np.ones(S.shape, bool)), S, -np.inf)
    P = np.exp(S - S.max(1, keepdims=True))
    return (P / P.sum(1, keepdims=True)) @ V


def linear_attn_quadratic(Q, K, V, causal):                  # the n x n form, for checking
    A = phi(Q) @ phi(K).T
    if causal:
```

```
        A = np.tril(A)
    return (A @ V) / A.sum(1, keepdims=True)


def linear_attn(Q, K, V):                                   # non-causal, reassociated
    fK = phi(K)
    return (phi(Q) @ (fK.T @ V)) / (phi(Q) @ fK.sum(0))[:, None]


def linear_attn_recurrent(Q, K, V):                         # causal: an RNN with state (S,
    d, dv = Q.shape[1], V.shape[1]
    S, z, out = np.zeros((d, dv)), np.zeros(d), np.empty((len(Q), dv))
    for t in range(len(Q)):
        k = phi(K[t])
        S += np.outer(k, V[t])
        z += k
        q = phi(Q[t])
        out[t] = q @ S / (q @ z)
    return out


n, d = 256, 32
Q, K, V = (rng.standard_normal((n, d)) for _ in range(3))
print("non-causal: reassociated == n x n form:",
      np.allclose(linear_attn(Q, K, V), linear_attn_quadratic(Q, K, V, False)))
print("causal:     recurrence  == masked n x n form:",
      np.allclose(linear_attn_recurrent(Q, K, V), linear_attn_quadratic(Q, K, V, True)))
ref = softmax_attn(Q, K, V, False)
lin = linear_attn(Q, K, V)
print(f"relative difference from softmax attention: {np.linalg.norm(lin - ref) / np.linalg.n

print(f"\n{'n':>6} {'softmax ms':>10} {'linear ms':>9} {'flops ratio n/d':>15}")
for n in (512, 2048, 8192):
    Q, K, V = (rng.standard_normal((n, d)) for _ in range(3))
    t0 = time.perf_counter(); softmax_attn(Q, K, V, False); t1 = time.perf_counter()
    linear_attn(Q, K, V); t2 = time.perf_counter()
    print(f"{n:6d} {1e3 * (t1 - t0):10.1f} {1e3 * (t2 - t1):9.2f} {n / d:15.0f}")
```

```
python linear_attention.py
```

Output:

```
non-causal: reassociated == n x n form: True
causal:     recurrence  == masked n x n form: True
relative difference from softmax attention: 0.84
```

```
   n softmax ms linear ms flops ratio n/d
 512        6.1      0.45              16
2048      140.4      2.84              64
8192     8723.9     14.99             256
```

- The re-associated form matches the explicit $n \times n$ form, and the recurrence matches the causal masked form. These are exact identities, not approximations.
- Linear attention differs from softmax attention on the same Q, K, V by 84%. It is a **different model**, with its own trained weights, not a fast approximation of a pretrained softmax model. Its fixed-size state also has to compress the whole past, a known source of weaker recall on retrieval-heavy tasks.
- The timing gap grows like n/d, as the flop count predicts. At $n = 8192$ the softmax form also pays for a 512 MB $n \times n$ matrix in float64.

---

4. Low-rank updates: LoRA and Eckart–Young

Fine-tuning changes a pretrained weight $W_0 \in \mathbb{R}^{d_{out} \times d_{in}}$ by some ΔW. **LoRA** (Hu et al., 2021) constrains the change to rank r:

$$W = W_0 + BA, \qquad B \in \mathbb{R}^{d_{out} \times r},\ A \in \mathbb{R}^{r \times d_{in}},\ r \ll \min(d_{in}, d_{out}).$$

W_0 stays frozen and only A and B train, $r(d_{in} + d_{out})$ numbers instead of $d_{in} d_{out}$. B starts at zero, so training starts exactly at the pretrained model. After training, BA can be added into W_0 once, at no inference cost. The survey writes the same update as $W_0 + AB$ with the roles of the letters swapped.

What can rank r capture? Eckart–Young answers exactly. The best rank-r approximation of ΔW in Frobenius norm is its truncated SVD, with relative error $\sqrt{\sum_{i>r} \sigma_i^2 / \sum_i \sigma_i^2}$. LoRA works when the useful part of a fine-tuning update has a fast-decaying spectrum. The LoRA paper measured this directly, finding that the learned updates have very low "intrinsic rank".

Worked example 4 — counts, the Eckart–Young limit, and reaching it

```
# file: lora_lowrank.py
# LoRA's update W = W0 + B A (B: d_out x r, A: r x d_in) in numbers: trainable-parameter
# counts for Llama-3-8B, the Eckart-Young limit on what rank r can represent, and a
# check that gradient descent on (A, B) from LoRA's B = 0 start reaches that limit.
import numpy as np

# 1) Trainable parameters, Llama-3-8B (d=4096, kv dim 1024, d_ff=14336, 32 layers)
d, kv, f, L, r = 4096, 1024, 14336, 32, 16
shapes = {"q": (d, d), "k": (kv, d), "v": (kv, d), "o": (d, d),
```

```
          "gate": (f, d), "up": (f, d), "down": (d, f)}
lora = lambda names: L * sum(r * (shapes[m][0] + shapes[m][1]) for m in names)
print(f"r={r}: q,v only {lora(['q', 'v']):,} ({100 * lora(['q', 'v']) / 8_030_261_248:.3f}%
      f"all 7 projections {lora(shapes):,} ({100 * lora(shapes) / 8_030_261_248:.2f}%)")

# 2) Eckart-Young: the best rank-r approximation error depends only on the spectrum
rng = np.random.default_rng(2)
n = 512
print(f"\nbest relative error ||dW - dW_r||_F / ||dW||_F for a {n}x{n} update")
print(f"{'spectrum':22} {'r=4':>6} {'r=16':>6} {'r=64':>6}")
for label, s in (("sigma_i = i^-2", np.arange(1, n + 1) ** -2.0),
                 ("sigma_i = i^-1", np.arange(1, n + 1) ** -1.0),
                 ("sigma_i = i^-0.5", np.arange(1, n + 1) ** -0.5),
                 ("flat (noise-like)", np.ones(n))):
    errs = [np.sqrt(np.sum(s[k:] ** 2) / np.sum(s**2)) for k in (4, 16, 64)]
    print(f"{label:22} " + " ".join(f"{e:6.3f}" for e in errs))

# 3) Fit W0 + B A to a target by gradient descent, starting from B = 0 as LoRA does
n, r = 128, 8
s = np.arange(1, n + 1) ** -1.0
U, _ = np.linalg.qr(rng.standard_normal((n, n)))
Vt, _ = np.linalg.qr(rng.standard_normal((n, n)))
dW = (U * s) @ Vt                                    # the update a full fine-tune would
A = rng.standard_normal((r, n)) / np.sqrt(n)         # LoRA: A random, B zero -> W = W0 a
B = np.zeros((n, r))
lr = 0.5
for step in range(4001):
    R = B @ A - dW                                   # residual of the update
    if step in (0, 100, 1000, 4000):
        print(f"step {step:4d}: relative error {np.linalg.norm(R) / np.linalg.norm(dW):.4f}"
    gB, gA = R @ A.T, B.T @ R                        # gradients of 0.5 ||B A - dW||_F^2
    B -= lr * gB
    A -= lr * gA
best = np.sqrt(np.sum(s[r:] ** 2) / np.sum(s**2))
print(f"Eckart-Young optimum for r={r}: {best:.4f}")
```

```
python lora_lowrank.py
```

Output:

```
r=16: q,v only 6,815,744 (0.085% of 8.03B); all 7 projections 41,943,040 (0.52%)

best relative error ||dW - dW_r||_F / ||dW||_F for a 512x512 update
spectrum                 r=4   r=16   r=64
sigma_i = i^-2         0.057  0.008  0.001
sigma_i = i^-1         0.365  0.189  0.091
```

```
sigma_i = i^-0.5        0.833  0.710  0.551
flat (noise-like)       0.996  0.984  0.935
step    0: relative error 1.0000
step  100: relative error 0.3480
step 1000: relative error 0.2607
step 4000: relative error 0.2589
Eckart-Young optimum for r=8: 0.2589
```

- Rank 16 on the query and value projections of Llama-3-8B trains 6.8 million parameters, 0.085% of the model. Applying it to all seven projections is still about half a percent.
- The spectrum decides everything. With $\sigma_i = i^{-2}$, rank 4 already captures all but 6% of the update. With a flat spectrum, which is what noise looks like, rank 64 out of 512 leaves 93% uncaptured.
- Plain gradient descent on the two factors, from LoRA's $B = 0$ start, converges to the Eckart–Young optimum (0.2589) even though the problem is nonconvex in (A, B). This is a known property of low-rank matrix factorization: its local minima are global. It is why the "only train a thin product" idea is benign in practice.

---

5. Preconditioning when the gradient is a matrix

For a weight matrix $W \in \mathbb{R}^{m \times n}$ the gradient G is also $m \times n$. A full second-order preconditioner would be $mn \times mn$, about 2.8×10^{14} entries for one 4096×4096 layer, so every practical method imposes structure:

Method	Preconditioner	Storage
Adagrad / Adam	diagonal, $(\sum g^2)^{-1/2}$ elementwise	mn
K-FAC (Martens–Grosse)	Fisher block $\approx \mathbb{E}[aa^\top] \otimes \mathbb{E}[gg^\top]$, inverted factor by factor	$m^2 + n^2$
Shampoo (Gupta–Koren–Singer)	$W \leftarrow W - \eta\, L^{-1/4} G R^{-1/4}$, $L = \sum GG^\top$, $R = \sum G^\top G$	$m^2 + n^2$
Muon (Jordan et al.)	$W \leftarrow W - \eta\, \mathrm{NS}_5(M)$, $M =$ momentum of G	mn

The bridge between the last two rows is the **polar factor**. With no accumulation ($L = GG^\top$, $R = G^\top G$) and $G = U\Sigma V^\top$:

$$L^{-1/4} G R^{-1/4} = U\Sigma^{-1/2} U^\top\, U\Sigma V^\top\, V\Sigma^{-1/2} V^\top = UV^\top.$$

Shampoo without memory **orthogonalizes** the gradient: it keeps the singular directions and sets every singular value to 1. Bernstein and Newhouse (2024) read this as steepest descent under the spectral norm. Muon computes the orthogonalization directly, without any eigendecomposition,

using a **Newton–Schulz** iteration. That is an odd matrix polynomial $X \leftarrow aX + b(XX^\top)X + c(XX^\top)^2X$, which acts on each singular value σ as the scalar map $p(\sigma) = a\sigma + b\sigma^3 + c\sigma^5$ while leaving U and V alone. The classical cubic $(a, b, c) = (1.5, -0.5, 0)$ converges quadratically to $UV^\top$ once every σ is near 1, but takes many steps to lift small singular values. Muon's quintic coefficients $(3.4445, -4.7750, 2.0315)$ were tuned to lift small singular values fast. The price is that they converge to a **band** around 1, not to 1.

Worked example 5 — Shampoo, SVD, and two Newton–Schulz iterations

```
# file: muon_ns.py
# Matrix preconditioning in three ways: Shampoo's L^-1/4 G R^-1/4, the SVD polar factor
# U V^T, and Newton-Schulz polynomial iterations (cubic, and Muon's tuned quintic) that
# use only matrix products.
import numpy as np

rng = np.random.default_rng(3)
m, n = 256, 128
U, _ = np.linalg.qr(rng.standard_normal((m, n)))
V, _ = np.linalg.qr(rng.standard_normal((n, n)))
sig = np.logspace(0, -3, n)                                  # singular values from 1 down to 1e
G = (U * sig) @ V.T                                          # a "gradient" with a wide spectrum
polar = U @ V.T                                              # the target: U V^T


def inv_root(M, p, eps=1e-12):
    w, Q = np.linalg.eigh(M)
    return (Q * np.maximum(w, eps) ** (-1 / p)) @ Q.T


shampoo = inv_root(G @ G.T, 4) @ G @ inv_root(G.T @ G, 4)
print(f"Shampoo one-step L^-1/4 G R^-1/4 vs U V^T: rel. error {np.linalg.norm(shampoo - pola


def newton_schulz(G, steps, coeffs):
    a, b, c = coeffs
    X = G / np.linalg.norm(G)                                # Frobenius norm puts all sigma <=
    for _ in range(steps):
        A = X @ X.T
        X = a * X + (b * A + c * A @ A) @ X                  # odd polynomial in the singular va
    return X


print(f"\n{'iteration':24} {'steps':>5} {'min sigma':>9} {'max sigma':>9} {'rel. err vs UV^T
for name, coeffs, steps_list, mm in (("cubic (1.5, -0.5, 0)", (1.5, -0.5, 0.0), (5, 10, 20,
                                      ("Muon quintic", (3.4445, -4.7750, 2.0315), (5, 10), 3)
    for steps in steps_list:
```

```
        X = newton_schulz(G, steps, coeffs)
        s = np.linalg.svd(X, compute_uv=False)
        err = np.linalg.norm(X - polar) / np.linalg.norm(polar)
        print(f"{name:24} {steps:5d} {s.min():9.3f} {s.max():9.3f} {err:16.3f} {mm * steps:7

# What the quintic does to one singular value, starting from sigma / ||G||_F
p = lambda s: 3.4445 * s - 4.7750 * s**3 + 2.0315 * s**5
s0 = np.logspace(-4, 0, 9) / np.linalg.norm(sig)
s5 = s0.copy()
for _ in range(5):
    s5 = p(s5)
print("\nquintic, 5 steps:  " + "  ".join(f"{a:.1e}->{b:.2f}" for a, b in zip(s0[::2], s5[::
```

```
python muon_ns.py
```

Output:

```
Shampoo one-step L^-1/4 G R^-1/4 vs U V^T: rel. error 1.6e-09

iteration                steps min sigma max sigma rel. err vs UV^T matmuls
cubic (1.5, -0.5, 0)         5     0.002     0.998            0.813      10
cubic (1.5, -0.5, 0)        10     0.019     1.000            0.612      20
cubic (1.5, -0.5, 0)        20     0.817     1.000            0.031      40
cubic (1.5, -0.5, 0)        30     1.000     1.000            0.000      60
Muon quintic                 5     0.155     1.202            0.356      15
Muon quintic                10     0.682     1.134            0.206      30

quintic, 5 steps:  3.2e-05->0.02  3.2e-04->0.16  3.2e-03->1.14  3.2e-02->1.13  3.2e-01->1.13
```

- One-step Shampoo reproduces $UV^\top$ to 10^{-9}, confirming the identity above.
- The cubic iteration needs about 30 steps (60 matrix products) when singular values span three decades. After 5 steps the smallest are still near 0.002.
- Muon's quintic lifts the smallest singular value from about 10^{-4} (after the Frobenius scaling) to 0.16 in 5 steps and puts everything else in $[0.68, 1.2]$. It is **not** an accurate polar decomposition (error 0.36 against $UV^\top$), and 10 steps do not fix that because the iteration oscillates inside the band. For an optimizer this is a good trade: the update direction is what matters, 5 steps cost 15 products that run well in bfloat16 on tensor cores, and no SVD is ever needed. Muon's reference implementation also transposes tall matrices so that $XX^\top$ is the smaller Gram matrix.

The survey notes the practical outcome: a distributed Shampoo won the 2024 AlgoPerf training-algorithms competition, and Muon-family optimizers have since been used to train large models (Liu et al., 2025).

---

6. Pitfalls

Pitfall	Symptom	Fix
Counting parameters with another architecture's habits	Totals that match no checkpoint	Derive from `config.json`; check against published totals
"Attention matrices are low rank"	Wrong conclusions about compressibility of P	S has rank $\leq d_h$; P is generically full rank, numerically rank-varying
Treating linear attention as a drop-in approximation	Large output differences on a pretrained model	It is a different operator; train with it
Expecting LoRA to capture any update	Poor results on tasks needing broad change	Check the spectrum of a full fine-tune delta, or raise r
Thinking Muon computes an exact polar factor	Confusion when singular values are not 1	The tuned quintic targets a band; use the cubic or an SVD for exact $UV^\top$
Forming $mn \times mn$ preconditioners	Memory blow-up	Kronecker or two-sided structure ($m^2 + n^2$) or orthogonalization (mn)
Fractional matrix powers without damping	NaNs from tiny eigenvalues	Add ϵI (Shampoo initializes $L, R = \epsilon I$)

---

7. Checkpoint

- Write the per-block parameter count of a Llama-style decoder and explain why the MLP dominates.
- Prove $\text{rank}(QK^\top) \leq d_h$, and explain why that bound does not carry over to $\text{softmax}(QK^\top)$.
- Show that each attention output row is a convex combination of value rows.
- Derive the flop counts of $(QK^\top)V$ and $Q(K^\top V)$, and write causal linear attention as a recurrence.
- State Eckart–Young, and use it to say what rank-r LoRA can and cannot represent.
- Show that $L^{-1/4}GR^{-1/4} = UV^\top$ when $L = GG^\top$ and $R = G^\top G$.
- Explain what a Newton–Schulz iteration does to singular values, and why Muon's coefficients do not converge to exactly 1.

Exercises

Easy

1. Using `llm_params.py`, count a model with $d = 8192$, $d_{ff} = 28672$, 80 layers, 64 heads, 8 KV heads and $V = 128256$ (the Llama-3-70B shape). What fraction is MLP?

2. For $d_h = 128$, above what sequence length is $Q(K^\top V)$ cheaper than $(QK^\top)V$?
3. How many trainable parameters does rank-8 LoRA add to one 4096×14336 matrix?

Medium

4. Show that softmax is invariant to adding a constant to each row of S, and use this to explain the `S - S.max(1, keepdims=True)` line.
5. Prove that for $X_0 = G/\|G\|_F$, every singular value of X_0 is at most 1. Why does the cubic Newton–Schulz iteration need that?
6. Modify `attention_rank.py` to use a causal mask. Is P still full rank? (Hint: it becomes lower triangular with a positive diagonal.)
7. In `lora_lowrank.py`, replace the target spectrum by $\sigma_i = i^{-1}$ plus Gaussian noise of relative size 5%. How does the achievable error at $r = 8$ change, and does gradient descent still reach it?

Challenge

8. Show that the K-FAC approximation $\mathbb{E}[(aa^\top) \otimes (gg^\top)] \approx \mathbb{E}[aa^\top] \otimes \mathbb{E}[gg^\top]$ is exact when a and g are independent, and give a two-variable example where it fails.
9. Find coefficients (a, b, c) that maximize $p'(0) = a$ subject to p mapping $[0, 1]$ into $[0.7, 1.3]$ after 5 iterations, and compare with Muon's.
10. Implement Shampoo with exponential moving averages of $GG^\top$ and $G^\top G$ on a least-squares problem $\min_W \|XW - Y\|_F^2$ with an ill-conditioned X, and compare its iteration count with gradient descent and with the orthogonalized update.

Summary

The parameters of a Llama-style model can be counted exactly from its configuration, and most of them sit in the MLP. Counting biases and norm shifts that do not exist is an easy error, and it appears even in careful surveys. An attention head's scores have rank at most d_h, but the softmax makes the attention matrix full rank, with an effective rank set by temperature. The softmax is also what forces the n^2 cost. Remove it with a factorizable kernel and re-associate, and attention becomes linear in n, a recurrence with a fixed-size state, at the price of being a different model. LoRA's rank-r updates are justified by Eckart–Young and by the fast-decaying spectra of fine-tuning updates, and gradient descent on the factors reaches the best rank-r approximation. Matrix-shaped gradients admit structured preconditioners: Kronecker factors (K-FAC), two-sided roots (Shampoo), or plain orthogonalization (Muon). Newton–Schulz iterations compute the last with nothing but matrix products, deliberately trading exactness for speed.

Sources (for further reading)

- A. Baggag, Y. Saad, "The Numerical Linear Algebra of Large Language Models", arXiv:2610.04631 (2026). §3.4.5 (rank bottleneck), §3.11.3 (parameter counting, Tables 3–4), §4.2 (LoRA), §4.3 (linear transformers), §5.3–5.5 (K-FAC, Shampoo, Muon).
- A. Vaswani et al., "Attention Is All You Need", NeurIPS 2017.
- Llama Team, "The Llama 3 Herd of Models", arXiv:2407.21783. Model `config.json` files and published parameter totals on Hugging Face (`meta-llama/Meta-Llama-3-8B`, `meta-llama/Llama-3.1-405B`).
- S. Bhojanapalli et al., "Low-Rank Bottleneck in Multi-head Attention Models", ICML 2020 (arXiv:2002.07028).
- A. Katharopoulos et al., "Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention", ICML 2020 (arXiv:2006.16236). Z. Shen et al., "Efficient Attention: Attention with Linear Complexities", WACV 2021 (arXiv:1812.01243).
- E. J. Hu et al., "LoRA: Low-Rank Adaptation of Large Language Models", ICLR 2022 (arXiv:2106.09685). C. Eckart, G. Young, "The approximation of one matrix by another of lower rank", *Psychometrika* 1 (1936).
- J. Martens, R. Grosse, "Optimizing Neural Networks with Kronecker-factored Approximate Curvature", ICML 2015 (arXiv:1503.05671).
- V. Gupta, T. Koren, Y. Singer, "Shampoo: Preconditioned Stochastic Tensor Optimization", ICML 2018 (arXiv:1802.09568). MLCommons AlgoPerf 2024 results.
- K. Jordan et al., "Muon: An optimizer for hidden layers in neural networks" (2024), https://kellerjordan.github.io/posts/muon/. J. Bernstein, L. Newhouse, "Old Optimizer, New Norm: An Anthology", arXiv:2409.20325. J. Liu et al., "Muon is Scalable for LLM Training", arXiv:2502.16982.
- N. J. Higham, *Functions of Matrices: Theory and Computation* (SIAM, 2008), ch. 8 (polar decomposition, Newton–Schulz).

Inspiration (not a source): the SIAM Activity Group on Dynamical Systems' post on X sharing the Baggag–Saad survey, https://x.com/DynamicsSIAM/status/2107370387306651968. The survey itself is cited above as a secondary source; the claims here are checked against the primary papers and checkpoints.

Complexity Theory

Complexity Theory for Computer Science

Complexity theory classifies problems by the resources—time, space, randomness—required to solve them. It explains why some tasks admit fast algorithms, why others need heuristics, and how reductions transfer hardness. This part deepens NP-completeness craft, randomized/approximation complexity, and space classes.

Why it matters in industry

Decision	Theory input
Exact vs approximate solver	NP-hardness, approx hardness
Crypto assumptions	average-case hardness (related landscape)
Compiler analyses	undecidable vs PTIME fragments
Verifier architectures	NP certificates, interactive proofs (advanced)
Memory-tight algorithms	L, NL, PSPACE

Relation to Algorithms & Complexity (part 13)

Part	Role
13 Algorithms-Complexity	Analyze *algorithms*: asymptotics, recurrences, class *intuition*
19 Complexity-Theory	Classify *problems*: reductions craft, NP-complete, space, approx hardness

Read 13 first for big-O fluency; use 19 when proving hardness or choosing exact vs approximate solvers.

Core map

```
decision problems

        v
  classes P / NP    reductions    NP-complete
```

```
randomized classes
space (L, NL, PSPACE)
approximation hardness
```

Decision problems first

Optimization is converted to decision ("is there a solution with cost $\leq k$?") so classes like **P** and **NP** apply cleanly. Algorithms for optimization still matter; class theory explains barriers.

Worked example 1

TSP optimization $\leftrightarrow$ decision TSP with threshold k. Binary search on k if decisions are efficient and bounds discrete.

Roadmap

1. Reductions and NP-completeness
2. Randomized and approximation complexity
3. Space complexity
4. Fine-grained complexity breaks — truly subquadratic 3SUM and subcubic APSP

Related: Algorithms part (13) for asymptotics and intro classes.

Proof culture

Complexity proofs are mostly:

- Construct poly-time reductions carefully (both directions)
- Exhibit verifiers for **NP** membership
- Use hierarchy theorems / padding for separations (advanced)

Worked example 2

A wrong-direction reduction "solves" nothing for hardness: always reduce **known hard $\rightarrow$ your problem**.

Pitfalls

1. "NP = hard" slang vs formal **NP**
2. Heuristic success as proof of **P**
3. Certificate exponential size
4. Confusing worst-case with average-case crypto
5. Ignoring promise problems / gap problems in approx hardness

Worked mental models

Verifier view of NP

A language is in **NP** if short proofs of YES can be checked quickly—not if the problem is "hard." Sorting is in $\mathbf{P} \subseteq \mathbf{NP}$; SAT is in **NP** and complete.

Reduction as subroutine

$A \leq_p B$ means: "to solve A, transform the instance and call a B-oracle." Hardness flows **from** A **to** B along this arrow.

Space vs time

Savitch's theorem collapses nondeterministic poly space to deterministic poly space—unlike the open **P** vs **NP** question for time. Memory and time have different theories.

Worked example 3 — product call

"Is this scheduling problem NP-complete?" Checklist: decision version? in NP? reduce from 3-SAT/partition/… with poly gadgets and both iff directions?

Worked example 4 — approx choice

Need a tour on metric distances: Christofides/MST 1.5–2 approx exists. Same problem without triangle inequality: no constant approx unless $\mathbf{P} = \mathbf{NP}$. Modeling assumptions change complexity.

Checkpoint

- Define **P**, **NP**, $\leq_p$
- State NP-completeness method
- Name RP/BPP intuition
- Place L, NL, PSPACE in the chain
- Convert an optimization task to decision

Exercises

1. List three NP-complete problems and one problem in P.
2. Why does $\mathbf{P} \subseteq \mathbf{NP}$?
3. Give a language in PSPACE not obviously in NP.
4. What would $\mathbf{P} = \mathbf{NP}$ imply for cryptography (high level)?
5. Checkpoint: write a 5-line NP-completeness plan for a scheduling decision problem.

Summary

Complexity theory is the map of feasible computation. Mastering reductions, randomness, approximation limits, and space gives principled expectations about what software can guarantee—and when to change the problem.

Reductions and NP-Completeness

NP-completeness is the standard badge of "unlikely to have a poly-time algorithm." Earning it requires membership in **NP** plus a correct polynomial-time reduction from a known complete problem. This chapter drills the definitions, the Cook–Levin gateway, and classic gadget reductions.

1. Languages and decision problems

Encode instances as strings. Language L = YES instances. Algorithms **decide** L if they accept $x \in L$ and reject $x \notin L$.

Worked example 1

HAMCYCLE $= \{\langle G \rangle : G \text{ has a Hamiltonian cycle}\}$.

2. Classes P and NP

P: decidable in deterministic poly time $O(n^c)$.

NP: exists poly-time verifier $V(x, u)$ and poly bound p s.t.

$$x \in L \iff \exists u,\ |u| \le p(|x|),\ V(x, u) = 1.$$

Worked example 2 — certificates

- SAT: assignment
- CLIQUE: vertex set of size k
- SUBSET-SUM: subset bit vector
- COMPOSITES: nontrivial factor (historical interest; PRIMES $\in$ **P**)

Worked example 3 — coNP

UNSAT has short certificates only if **NP** = **coNP** (unknown). Universal claims differ from existential ones.

3. Karp reductions

$A \leq_p B$ via poly-time f with $x \in A \Leftrightarrow f(x) \in B$.

Lemma. If $A \leq_p B$ and $B \in \mathbf{P}$ then $A \in \mathbf{P}$.
Lemma. If $A \leq_p B$ and A NP-hard then B NP-hard.

Worked example 4 — direction

Hardness of B: prove $A \leq_p B$ for hard A. Algorithm for A: reduce to easier B if you have a solver for B.

4. NP-hard and NP-complete

- **NP-hard:** every language in **NP** reduces to it (at least as hard as all of **NP**)
- **NP-complete:** NP-hard and in **NP**

If any NP-complete language is in **P**, then **P** = **NP**.

5. Cook–Levin theorem (statement)

Theorem. SAT is NP-complete.

Idea. For any language in **NP** with verifier V, map x to a Boolean formula ϕ_x that is satisfiable iff there exists a witness u such that $V(x, u)$ accepts—by encoding the computation tableau of V with local Boolean constraints.

After Cook–Levin, hardness proofs reduce from SAT/3-SAT/etc., not from arbitrary NP machines.

6. NP-completeness proof template

1. **Membership:** $X \in \mathbf{NP}$ (certificate + verifier + poly bounds)
2. **Source:** pick known complete Y (3-SAT, VERTEX-COVER, …)
3. **Construction:** poly-time f
4. **Correctness:** $y \in Y \Leftrightarrow f(y) \in X$ (both directions!)
5. **Runtime:** argue $|f(y)|$ and construction are poly

7. Classic reductions (sketches)

3-SAT $\leq_p$ CLIQUE

For formula with clauses $C_1, \ldots, C_m$, build graph: vertices = literals occurrences in clauses; edges between compatible literals in different clauses (not negations of each other). Clique of size $m \Leftrightarrow$ satisfying assignment selecting one literal per clause consistently.

Independent set / vertex cover

$\alpha(G) + \beta(G) = n$ relationships: clique in $G \leftrightarrow$ independent set in complement; vertex cover complements independent set.

Worked example 5

Pathological check: empty formula? Unit clauses? Encode carefully so f handles edge cases.

3-SAT $\leq_p$ SUBSET-SUM (idea)

Digits encode variable choices and clause satisfaction carries—classic number gadgets.

Worked example 6 — PARTITION / KNAPSACK decision

Pseudo-poly algorithms exist; still weakly NP-complete—bit length matters.

Hamiltonian cycle $\leq_p$ TSP decision

Assign weight 1 to edges of G, weight 2 (or large) to non-edges in complete graph; threshold n forces using only original edges.

Worked example 7

Metric TSP remains NP-hard; approximation behavior differs from non-metric.

8. Optimization vs decision

NP-completeness is for decision. Optimization hardness follows: if you optimize in poly time, you decide thresholds in poly time.

Worked example 8

Max-CLIQUE optimization poly $\Rightarrow$ CLIQUE decision poly $\Rightarrow$ $\mathbf{P} = \mathbf{NP}$ if Max-CLIQUE always poly.

9. Proof hygiene examples of bugs

1. Reduction exponential in k or n
2. Only proving YES$\Rightarrow$YES
3. Gadgets that accidentally create extra solutions
4. Assuming $\mathbf{P} \neq \mathbf{NP}$ without stating when claiming "no poly algorithm"
5. Using Turing reductions while claiming Karp-completeness without care

Worked example 9 — wrong direction essay

"I reduced my scheduling problem to SAT and SAT is hard, so scheduling is hard." This shows scheduling is **no harder than SAT**, not hardness.

10. Beyond: strong NP-completeness, APX, etc.

- **Strongly NP-complete:** remains hard when numbers are small (unary)—rules out pseudo-poly algorithms unless $\mathbf{P} = \mathbf{NP}$
- **APX-hard:** hardness of approximation classes

Worked example 10

BIN PACKING strongly NP-hard; still has good approximations—hardness of exact vs approx diverge.

11. Pitfalls

1. Certificate not poly-time checkable
2. Non-deterministic "guess poly solutions" without verifier clarity
3. Graph encodings that blow size superpoly

4. Confusing NP-hard search problems with decision languages

5. Citing NP-completeness for problems over reals without encoding model

12. Checkpoint

- Define verifier-based **NP**
- Use $\leq_p$ in the correct direction
- Outline Cook–Levin's role
- Execute clique/cover relationships
- Audit a reduction for both iff sides

Exercises

Easy

1. Prove $\mathbf{P} \subseteq \mathbf{NP}$.
2. Give certificates for 3-COLORABILITY.
3. Formalize VERTEX-COVER as a language.
4. If $A \leq_p B \leq_p C$ show $A \leq_p C$.
5. Why is a poly-time algorithm for special graphs not enough for general NP-completeness collapse?

Medium

6. Detail independent set $\leftrightarrow$ clique reduction.
7. Prove: if any NP-complete problem is in P then P=NP.
8. Write verifier carefully for HAMCYCLE.
9. Identify the bug in a fictional reduction that maps SAT to "always-yes" language.
10. Explain weak vs strong NP-completeness with SUBSET-SUM vs 3-SAT.

Challenge

11. Write a full 3-SAT $\leq_p$ CLIQUE proof with both directions.
12. Sketch Cook–Levin tableau constraints (local windows).
13. Reduce 3-SAT to 3-COLOR (high-level gadgets).
14. Show decision TSP NP-complete from HAMCYCLE with weights argument.
15. Research: NP-completeness of Minesweeper / Sudoku—what is the formal language?

Summary

NP-completeness is membership plus hardness via poly-time many-one reductions from a known complete problem. Cook–Levin opens the floodgates; gadget reductions populate the catalog. Discipline about direction, certificates, and both sides of $\Leftrightarrow$ separates correct theory from folklore.

Randomized and Approximation Complexity

Randomness and approximation refine the **P** vs **NP** story: some problems admit fast randomized algorithms with tiny error; some NP-hard optimization problems admit provable approximations while others resist even weak ratios. This chapter surveys classes and classic positive/negative results.

1. Randomized complexity classes (informal)

Class	Error pattern	Intuition
RP	one-sided on YES	random poly time; YES accepted with prob $\geq 1/2$; NO never accepted
coRP	one-sided on NO	complements of RP
ZPP	zero error, expected poly time	$\mathbf{RP} \cap \mathbf{coRP}$
BPP	two-sided, error $\leq 1/3$	"efficient randomized practical"

Containment folklore: $\mathbf{P} \subseteq \mathbf{ZPP} \subseteq \mathbf{RP} \subseteq \mathbf{BPP}$. Relation of **BPP** to **NP** is subtle; widely believed $\mathbf{BPP} = \mathbf{P}$ (derandomization under hardness assumptions).

Worked example 1 — polynomial identity testing

Schwartz–Zippel yields coRP-style algorithms for identity testing—no known simple deterministic poly algorithm of same generality historically (derandomization progress exists).

Worked example 2 — primality

Miller–Rabin: historically RP-style Monte Carlo; AKS later put PRIMES in **P**.

2. Error reduction

Independent repetition + majority (BPP) or OR/AND (one-sided) drives error to $2^{-\text{poly}}$. Amplification is why "error 1/3" vs "$1/n^{100}$" are equivalent definitions up to poly time.

Worked example 3

Chernoff bounds: majority of $O(\log(1/\delta))$ trials achieves error δ.

3. Approximate counting and sampling (glimpse)

#**P** counts accepting witnesses (permanent, #SAT). Approximate counting and almost-uniform sampling connect via MCMC for some combinatorial structures (Jerrum–Valiant–Vazirani paradigm).

Worked example 4

Permanent of nonnegative matrices: exact #**P**-hard; FPRAS exists via MCMC for the permanent (deep result).

4. Approximation algorithms complexity

For optimization problem Π, algorithm has ratio ρ if it always returns solutions within ρ of OPT (definitions min/max).

Classes (informal):

- **APX:** constant-factor approximable in poly time
- **PTAS:** $(1+\varepsilon)$-approx for all $\varepsilon > 0$
- **FPTAS:** PTAS with time poly$(n, 1/\varepsilon)$

Worked example 5

Knapsack admits FPTAS; general TSP without metric assumptions does not admit any constant ratio unless $\mathbf{P} = \mathbf{NP}$.

5. Positive classics

Problem	Guarantee
Vertex cover	2-approx (matching)
Metric TSP	1.5 (Christofides); 2 via MST
Set cover	$H_n \approx \ln n$ greedy
Max-cut	0.878 SDP (Goemans–Williamson)
Knapsack	FPTAS

Worked example 6

Vertex cover 2-approx proof: maximal matching lower bound—see Algorithms part; complexity view: in APX.

Worked example 7 — gap from integrality

LP/SDP relaxations + rounding yield ratios; integrality gaps prove limits of that relaxation.

6. Hardness of approximation

Using PCPs (Probabilistically Checkable Proofs), one proves: if $\mathbf{P} \neq \mathbf{NP}$, some problems cannot be approximated better than ρ in poly time.

Worked example 8 — set cover

No $(1 - o(1)) \ln n$-approx unless **NP** has slightly superpoly algorithms (Feige et al. line of work)—greedy essentially optimal.

Worked example 9 — clique

Extremely hard to approximate; related to PCP theorem corollaries.

Worked example 10 — metric vs non-metric TSP

Non-metric: no constant approx (gap-producing reductions). Metric: constant possible.

7. PCP theorem (statement level)

PCP theorem. $\mathbf{NP} = \mathbf{PCP}(O(\log n), O(1))$—proofs checkable by reading constantly many bits with random coins log-many.

This is the engine of modern hardness of approximation. Details are graduate-level; know the existence and purpose.

8. Promise problems and gaps

Hardness reductions often produce **gap problems**: YES instances have OPT $\geq c$, NO instances OPT $\leq s$, with $c/s = \rho$. Distinguishing them in poly time would approximate within better than ρ.

Worked example 11

Label-cover / unique games (UGC) conjectures refine constant-factor hardness landscapes (e.g. for vertex cover beyond 2 under UGC).

9. Engineering reading of the map

Situation	Theory advice
Need exact small n	ILP/DP/exponential OK
Large n, need guarantee	Seek APX/PTAS literature
Even weak approx hard	Heuristics + empirics; change model
Randomization natural	Fingerprinting, hashing, MCMC

Worked example 12 — product

CDN placement: greedy set cover $\ln n$ guarantee + MIP overnight for regions—hybrid of approx class and exact methods.

10. Pitfalls

1. Monte Carlo error not amplified for SLOs
2. Calling any heuristic an "approximation algorithm" without ratio
3. Using non-metric TSP approx that assume triangle inequality
4. Pseudo-poly exact algorithms misread as poly
5. Assuming BPP algorithms always derandomize easily in practice (PRNGs usually fine; theory subtler)

11. Checkpoint

- Define BPP/RP error patterns
- Amplify two-sided error conceptually
- Place problems in FPTAS/APX vs inapprox
- State what PCP buys for hardness
- Choose algorithm class given product constraints

Exercises

Easy

1. Is randomized quicksort in ZPP-style thinking? Explain.
2. Why is error 1/3 vs 1/100 equivalent for BPP up to poly time?
3. Define approximation ratio for a maximization problem carefully.
4. Name one FPTAS and one problem without constant approx (unless P=NP).
5. What does one-sided error mean for RP?

Medium

6. Prove majority amplification reduces two-sided error (Chernoff sketch).
7. Show metric TSP 2-approx via MST + Euler tour shortcut outline.
8. Explain gap-producing reduction idea for inapproximability.
9. Contrast weak NP-completeness of KNAPSACK with existence of FPTAS.
10. Why does triangle inequality matter for TSP approximation?

Challenge

11. Goemans–Williamson: state SDP relaxation for max-cut at high level.
12. PCP theorem: explain in your words how query complexity $O(1)$ enables gap hardness.
13. Unique Games Conjecture: one implication for vertex cover approx.
14. FPRAS vs FPTAS definitions; example of counting vs optimization.
15. Design brief: pick NP-hard resource allocation; cite best known approx ratio class you can claim.

Summary

Randomized classes formalize fast algorithms with controlled error; approximation classes formalize near-optimal optimization. PCPs and gap reductions show many ratios are unimprovable if $\mathbf{P} \neq \mathbf{NP}$. The practical art is matching your problem to a positive algorithm or proving you should stop seeking one.

Space Complexity

Space complexity measures memory as a function of input size. Many natural problems that seem to need exponential time still fit in small space; others are complete for large space classes. This chapter introduces **L**, **NL**, **PSPACE**, reductions, and Savitch's theorem.

1. Model of space

Standard model: Turing machine with

- read-only input tape
- read/write work tapes (count cells used)
- optional write-only output tape (for transducers)

Space $s(n)$: max work cells on inputs of length n.

$\log n$ space is already nontrivial: indices into the input fit in binary.

Worked example 1

Scanning input to test if first symbol = last symbol uses $O(1)$ work space (plus input head positions as state)—careful accounting uses $O(\log n)$ to store positions if required by formal model variants.

2. Space classes

Class	Definition (sketch)
L	deterministic $O(\log n)$ space
NL	nondeterministic $O(\log n)$ space
PSPACE	deterministic poly(n) space
NPSPACE	nondeterministic poly space

Containments:

$$\mathbf{L} \subseteq \mathbf{NL} \subseteq \mathbf{P} \subseteq \mathbf{NP} \subseteq \mathbf{PSPACE} = \mathbf{NPSPACE} \subseteq \mathbf{EXPTIME}.$$

Equality **PSPACE** = **NPSPACE** is **Savitch's theorem**. Whether **L** = **NL** is open (like a log-space **P** vs **NP**).

Worked example 2

Any $O(s(n))$ space machine runs in time at most exponential in $s(n)$ (configuration count)—hence **PSPACE** $\subseteq$ **EXPTIME**.

3. Configuration graphs

A configuration: state, work-tape contents, head positions. For space $s(n)$, #configs $\le |\Gamma|^{O(s(n))} \cdot \text{poly}(n) = 2^{O(s(n)+\log n)}$.

Reachability in the configuration graph $\Leftrightarrow$ acceptance.

Worked example 3

Nondeterministic log-space computation $\Leftrightarrow$ s–t connectivity in digraphs of poly size with implicit edge checks—motivates NL-completeness of connectivity.

4. NL-complete problem: graph reachability

PATH / STCON: given digraph G, nodes s, t, is t reachable from s?

Theorem. STCON is NL-complete under log-space reductions.

In NL: nondeterministically walk up to n steps from s, storing only current node ($O(\log n)$ bits). **Hardness:** configure-graph reachability reduces to STCON in log space.

Worked example 4 — undirected reachability

Undirected s–t connectivity is in **L** (Reingold's theorem)—breakthrough derandomizing SL=L.

5. Savitch's theorem

Theorem. **NSPACE**$(s(n)) \subseteq$ **DSPACE**$(s(n)^2)$ for space-constructible $s(n) \ge \log n$.

Corollary. **PSPACE** = **NPSPACE**.

Proof idea. Recursive check Reach(u, v, k): exists midpoint w with Reach$(u, w, \lfloor k/2 \rfloor)$ and Reach$(w, v, \lceil k/2 \rceil)$. Depth $O(\log \#\text{configs})$; reuse space $\Rightarrow$ quadratic blowup. ■

Worked example 5

Nondeterministic poly space collapses to deterministic poly space—unlike time, where **NP** vs **P** remains open.

6. PSPACE-complete problems

QBF / TQBF: fully quantified Boolean formulas $\exists x_1 \forall x_2 \exists x_3 \ldots \phi$ truth.

Theorem. TQBF is PSPACE-complete.

Worked example 6 — games

Many two-player games (generalized geography, some chess-on-$n \times n$ variants) are PSPACE-complete—alternating moves $\approx$ quantifiers.

Worked example 7 — model checking fragments

Some linear-time logic model checking problems are PSPACE-complete—verification complexity.

7. Log-space reductions

$A \leq_L B$: log-space computable many-one reduction. Finer than poly-time reductions; needed for L/NL completeness.

Worked example 8

Composition of log-space reductions remains log-space (careful with recompute vs store).

8. Relationships and hierarchy

Space hierarchy theorem: more space yields strictly more languages (under constructibility). So $\mathbf{L} \subsetneq \mathbf{PSPACE}$ etc. in appropriate forms.

Time vs space: $\mathbf{NL} \subseteq \mathbf{P}$ because config graph poly-size BFS, but $\mathbf{P} \subseteq \mathbf{PSPACE}$ obvious; separations between intermediate classes largely open.

Worked example 9

Poly-time algorithms may use poly space; streaming algorithms aim for sublinear space—closer to L-style constraints in spirit.

9. CS practice connections

Domain	Space angle
Streaming / sketching	$o(n)$ or polylog memory
Embedded / IoT	hard RAM caps
Verifiers	certificate checking in small space
Game AI on large state spaces	PSPACE-hard exact solvers $\Rightarrow$ heuristics
Compilers	analyses in low memory for huge codebases

Worked example 10 — streaming distinct count

Exact distinct elements needs linear space hard lower bounds; approximate sketches (HyperLogLog) use tiny space—complexity lower bounds guide necessity of approximation.

Worked example 11 — reachability in huge implicit graphs

State spaces of protocols: NL-style nondeterministic search vs deterministic Savitch (impractical quadratic) vs BDD symbolic methods.

10. Complementary classes

coNL: complements of NL languages. Immerman–Szelepcsényi theorem: $\mathbf{NL} = \mathbf{coNL}$—surprising collapse via inductive counting.

Worked example 12

Non-reachability is in NL—not obvious from the walking certificate for reachability.

11. Pitfalls

1. Counting input tape as work space incorrectly
2. Claiming STCON in L without undirected/Reingold context
3. Assuming **NPSPACE** bigger than **PSPACE**
4. Poly-time reduction when log-space needed for NL-completeness
5. Exponential configuration graphs "algorithms" that still use exponential space if naively stored

12. Checkpoint

- Define work-tape space and **L/NL/PSPACE**
- Explain STCON in NL
- State Savitch and **PSPACE** = **NPSPACE**
- Name TQBF as PSPACE-complete
- Know **NL** = **coNL** existence

Exercises

Easy

1. Why does $O(\log n)$ space suffice to store an index $i \in \{1..n\}$?
2. Show $\mathbf{P} \subseteq \mathbf{PSPACE}$.
3. Give a language obviously in L (e.g., regular language decision).
4. What is a configuration of a TM?
5. Why is TQBF in PSPACE (recursive evaluation sketch)?

Medium

6. Prove that a machine using space s has at most $2^{O(s+\log n)}$ configs.
7. Detail nondeterministic log-space algorithm for STCON.
8. Write Savitch recurrence Reach(u, v, k) pseudocode; space analysis.
9. Explain why Savitch does not put **NP** in **P**.
10. Reduce a small quantified formula evaluation to a game interpretation.

Challenge

11. Outline Immerman–Szelepcsényi inductive counting idea.
12. Prove TQBF PSPACE-hard sketch (from space-bounded TM computations).
13. Discuss streaming lower bound idea for exact distinct count (communication complexity awareness).
14. Compare log-space reductions vs poly-time reductions with an example of care needed.
15. Research Reingold's undirected connectivity in L: what combinatorial object (zig-zag product) appears?

Summary

Space classes reveal a different hierarchy than time: Savitch collapses nondeterministic poly space to deterministic, STCON captures NL, and TQBF captures PSPACE. Log-space computation models tight-memory algorithms and streaming ideals, while PSPACE-completeness explains the hardness of games and rich verification questions.

Fine-Grained Complexity Breaks: Truly Subquadratic 3SUM and Subcubic APSP

For two decades, fine-grained complexity treated two running times as walls: n^2 **for 3SUM** and n^3 **for all-pairs shortest paths (APSP)** (up to $n^{o(1)}$ shavings). In October 2026 Alman and Vassilevska Williams published the first *polynomial* improvements—deterministic $O(n^{1.9992})$ for 3SUM on polynomial-size integers and $O(n^{2.9995})$ for APSP on directed graphs with polynomial integer weights—and with them refuted the 3SUM and APSP hypotheses. The algorithms are algebraic and currently impractical, but the *structure* of the breakthrough is concrete and runnable at toy scale: a sparse set of entries of a **thin** matrix product ($N \times D$ times $D \times N$ with $D \ll N$), interpreted as triangles in a **lopsided** tripartite graph.

This chapter states the hypotheses, measures the classical $O(n^2)$ 3SUM wall on no-solution instances, reduces the story to All-Edges Sparse Triangle on lopsided graphs, times the two naïve ways to compute the wanted product entries, and records what Theorem 5 of the paper actually claims. All listings use **Python 3.13.5** and **NumPy 2.5.3** from `/workspace/venv-maths`. Outputs are pasted unedited. We do **not** implement Schönhage/Coppersmith pruning; that is the paper's contribution, and the constants are enormous.

1. Mental model: hypotheses as walls, reductions as algorithms

Fine-grained complexity (FGC) studies the *exponent*. A fine-grained reduction from A to B says: an $O(b(n)^{1-\varepsilon})$ algorithm for B would yield an $O(a(n)^{1-\delta})$ algorithm for A. The **3SUM hypothesis** asserted that 3SUM on n integers of polynomial magnitude needs $n^{2-o(1)}$ time in the word RAM. The **APSP hypothesis** asserted the same for $n^{3-o(1)}$ on APSP with polynomial integer weights. Both problems reduce (through Exact Triangle) to variants of **All-Edges Sparse Triangle**: for every prescribed edge, decide whether it lies in a triangle.

The 2026 paper's punchline is that those reductions always were *algorithms*. Once a lopsided All-Edges Sparse Triangle admits a truly subquadratic algorithm, every problem that reduces to it inherits a polynomial speedup—3SUM, APSP, $(\min,+)$-product, Tree Edit Distance, Zero-Weight k-Clique, several hinted Online Matrix–Vector conjectures in the thin-hint regime, and more. CNF-SAT and Orthogonal Vectors are *not* known to reduce this way; SETH is untouched.

```
3SUM
          Exact Triangle    Lopsided All-Edges Sparse Triangle
APSP

                      thin matrix product: wanted entries of (N×D)·(D×N)
```

2. The classical 3SUM wall

3SUM: given n integers, do three of them sum to zero? The textbook algorithm sorts and runs two pointers from each left index—$\Theta(n^2)$ comparisons in the worst case. Polylogarithmic improvements existed; no polynomial improvement did, until this paper.

```
# threesum_wall.py - classical sort + two-pointers on NO-solution instances
# (all-positive arrays never contain three that sum to 0, so the loops run fully)
import time
import numpy as np


def threesum_sort(a):
    a = sorted(a)
    n = len(a)
    for i in range(n):
        lo, hi = i + 1, n - 1
        while lo < hi:
            s = a[i] + a[lo] + a[hi]
            if s == 0:
                return True
            if s < 0:
                lo += 1
            else:
                hi -= 1
    return False


rng = np.random.default_rng(0)
print("n      found   time_ms   ns_per_n2")
for n in [500, 1000, 2000, 4000]:
    a = rng.integers(1, n * n, size=n).tolist()   # all positive => no solution
    t0 = time.perf_counter()
    found = threesum_sort(a)
    dt = time.perf_counter() - t0
    print(f"{n:<7}{found!s:<8}{dt*1000:8.2f}  {dt/(n*n)*1e9:8.2f}")
```

```
$ /workspace/venv-maths/bin/python threesum_wall.py
n      found   time_ms   ns_per_n2
500    False      10.08     40.34
1000   False      42.75     42.75
2000   False     187.16     46.79
4000   False     730.06     45.63
```

Time per n^2 stays flat: the exponent really is 2. Alman–Vassilevska Williams replace 2 with 1.9992 for integer 3SUM (deterministic) and 1.998 expected for a Las Vegas real-RAM variant that only compares, adds, and subtracts reals. The saving is tiny in the exponent and huge in the theory: the

wall moved.

3. Lopsided All-Edges Sparse Triangle

Encode a tripartite graph with parts A, B of size N and a middle part M of size $D \ll N$. Let $X \in \{0,1\}^{N \times D}$ be the biadjacency A–M and $Y \in \{0,1\}^{D \times N}$ the biadjacency M–B. Then

$$(XY)[i,j] = \text{number of length-2 paths } a_i \to M \to b_j$$

—exactly the number of triangles through the pair (a_i, b_j). **All-Edges Sparse Triangle** asks for this count (or its positivity) on a prescribed set W of pairs with $|W| \le N^2/\sqrt{D}$.

Two naïve algorithms:

1. **Full product, then gather.** Compute all of XY (cost $\Theta(N^2 D)$ naïvely, or $N^{2+o(1)}$ with rectangular fast matrix multiplication when D is a small power of N), then read $|W|$ entries.
2. **One inner product per wanted pair.** Cost $\Theta(|W|\, D)$.

```
# lopsided_triangles.py - wanted entries of a thin product = triangle counts
import time
import numpy as np


def bench(n, d, seed=1):
    rng = np.random.default_rng(seed)
    X = (rng.random((n, d)) < 0.25).astype(np.float64)   # A--M
    Y = (rng.random((d, n)) < 0.25).astype(np.float64)   # M--B
    max_w = max(1, int(n * n / np.sqrt(d)))              # |W| <= N^2 / sqrt(D)
    idx = rng.choice(n * n, size=max_w, replace=False)
    W = [(int(k // n), int(k % n)) for k in idx]

    t0 = time.perf_counter()
    full = X @ Y
    ans_full = [int(full[i, j]) for i, j in W]
    t_full = time.perf_counter() - t0

    t0 = time.perf_counter()
    ans_ip = [int(X[i] @ Y[:, j]) for i, j in W]
    t_ip = time.perf_counter() - t0
    assert ans_full == ans_ip
    return dict(n=n, d=d, W=len(W), t_full_ms=t_full * 1000, t_ip_ms=t_ip * 1000,
                nonzero=sum(v > 0 for v in ans_full), mx=max(ans_full))


print(f"{'n':>4} {'d':>3} {'|W|':>7} {'full+gather_ms':>14} {'|W|_IPs_ms':>10} nonzero  max"
for n, d in [(200, 4), (400, 4), (400, 8), (800, 8)]:
```

```
    r = bench(n, d)
    print(f"{r['n']:4d} {r['d']:3d} {r['W']:7d} {r['t_full_ms']:14.2f} {r['t_ip_ms']:10.2f}
          f"{r['nonzero']:7d}  {r['mx']}")
```

```
$ /workspace/venv-maths/bin/python lopsided_triangles.py
   n   d     |W| full+gather_ms |W|_IPs_ms nonzero  max
 200   4   20000           2.07      18.61    4406  3
 400   4   80000          10.73      79.51   18994  3
 400   8   56568           7.27      56.88   22644  5
 800   8  226274         102.91     249.48   90072  6
```

On these dense-ish random instances the BLAS full product wins, because $|W|$ is still $\Theta(N^2/\sqrt{D})$ and NumPy's GEMM is heavily tuned. The FGC point is asymptotic and algebraic: when $D \le N^\varepsilon$ for $\varepsilon < 0.12$ and $|W| \le N^2/D^\kappa$, there exists $\gamma > 0$ such that the *wanted* entries cost $O(N^2/D^\gamma)$—polynomially less than writing down XY or than $|W|$ independent inner products. That is Theorem 5 / Corollary 26 of the paper.

4. What the new algorithm actually does (sketch)

The algorithm modifies Coppersmith's rectangular matrix-multiplication routine that rests on **Schönhage's ten-multiplication identity** (an outer product and an inner product computed together). Applied recursively for L levels it evaluates 2^L small products at once. Two ideas turn this into a sparse-output algorithm:

1. **Share encodings.** The linear forms that prepare each leaf depend on X alone or on Y alone. Encode each row/column band once; every tile reuses those encodings.
2. **Prune the recursion tree.** Visit a leaf only if some wanted output needs it. Most leaves of an output entry are shared with many other entries; a careful count (their Lemma 11) shows that $|W| \le N^2/\sqrt{D}$ wanted entries touch only about $N^2/D^{1/18}$ leaves when $N \ge D^{18}$.

Interpreted as a graph algorithm: All-Edges Sparse Triangle on a tripartite graph whose middle part has n^ε vertices for $\varepsilon < 0.12$ becomes truly subquadratic. Known (and newly derandomized) reductions then pull 3SUM and APSP below their textbook exponents.

The paper is explicit about discovery: Claude (Anthropic) found the core algorithm; the authors simplified, strengthened, extended, and formalized the main results in Lean 4. That does not change the mathematics. It does change how one reads the acknowledgments.

5. Consequences worth remembering

Problem	Old barrier (informal)	New (integer, deterministic)
3SUM	$n^{2-o(1)}$	$O(n^{1.9992})$

Problem	Old barrier (informal)	New (integer, deterministic)
APSP / $(\min,+)$-product	$n^{3-o(1)}$	$O(n^{2.9995})$
Exact Triangle	$n^{3-o(1)}$	$O(n^{2.9983})$
Directed unweighted APSP	Zwick $\tilde{O}(n^{2+\mu})$	first improvement not from faster rectangular MM
Tree Edit Distance (poly costs)	cubic	$O(n^{2.9995})$ via known tight reductions
Online Set Disjointness (thin D)	preprocess N^2 or query $O(D)$	polynomially better preprocess *and* query

Unaffected in spirit: SETH, Orthogonal Vectors, balanced sparse triangle in every degree regime, and the "combinatorial" versions of these hypotheses. The new algorithms are algebraic; a combinatorial truly-subcubic APSP remains open.

6. The trap

Trap: "so 3SUM is easy now." No. The exponent moved by less than 10^{-3}. Practical code should still use the $O(n^2)$ sort-and-scan (or hashing) and the geometric data structures built on it. What broke is the *hypothesis used as a conditional lower bound*. Every paper that said "3SUM-hard, therefore no $O(n^{2-\varepsilon})$ algorithm" for a problem that reduces *to* 3SUM still stands as a reduction; it no longer stands as evidence of hardness. Problems that were only known to be 3SUM-*hard* (reductions the other way)—many geometric ones—gain no algorithm from this paper.

Trap: "fast matrix multiplication already solved this." Computing the *entire* thin product still costs $\Omega(N^2)$ to write down. The novelty is computing a *sparse prescribed subset* cheaper than both "all entries" and "one IP each," by pruning inside a specific identity whose sparsity pattern cooperates.

7. Try this

1. Modify `threesum_wall.py` to plant a single solution at the end of the array (`a[-3:]=[10,-3,-7]` after making the rest positive). How early does the two-pointer algorithm return? Why is a no-solution benchmark required to see the n^2 wall?
2. In `lopsided_triangles.py`, replace the Boolean matrices with small random integers in $\{-2,-1,0,1,2\}$ and check that wanted entries of XY still match. Triangle *counting* is the matrix product; detection is the sign.

3. Implement the data-structure view at toy scale: preprocess nothing but store X and Y, answer $(XY)[i, j]$ in $O(D)$ time, and compare to materializing XY. The paper's Theorem 3 beats *both* N^2 preprocess and $\sqrt{D}$ query in the thin regime—explain which baseline each side beats.
4. Read Section 2 of arXiv:2610.06783 far enough to explain, in one paragraph, why Schönhage's identity (rather than Coppersmith–Winograd) is the one that pruning can exploit.

Checkpoint

1. State the 3SUM and APSP hypotheses and the new deterministic exponents.
2. Reduce "wanted entries of a thin product" to triangle counts in a lopsided tripartite graph.
3. Name the two naïve algorithms for those entries and the sense in which Theorem 5 beats both.
4. List three problems that inherit a polynomial speedup and two that do not.

Summary

Fine-grained complexity built a web of reductions around 3SUM and APSP to explain decades of stuck exponents. Alman–Vassilevska Williams showed that the same web is an algorithm catalog: one thin-product technique for lopsided All-Edges Sparse Triangle moves the exponents of a long list of problems by a polynomial factor. The classical $O(n^2)$ 3SUM loop still dominates practice; the hypotheses no longer do. The listings above make the wall and the lopsided product tangible; the paper supplies the pruned Schönhage/Coppersmith engine that sits between them.

Sources

- J. Alman, V. Vassilevska Williams, "Truly Subquadratic 3SUM and Truly Subcubic APSP via Triangles in Sparse Lopsided Graphs", arXiv:2610.06783 (2026): https://arxiv.org/abs/2610.06783
- V. Vassilevska Williams, "On Some Fine-Grained Questions in Algorithms and Complexity", ICM 2018 survey: https://people.csail.mit.edu/virgi/papers.html
- M. Pătraşcu, "Towards Polynomial Lower Bounds for Dynamic Problems", STOC 2010 (All-Edges Sparse Triangle hardness template)
- R. Williams, "Faster All-Pairs Shortest Paths via Circuit Complexity", STOC 2014 / SICOMP (previous best APSP shaving)